# Ginkgo

Generated from pipelines/2991456621 branch based on develop. Ginkgo version 2.0.0

Generated by Doxygen 1.8.16

**1 Main Page** 1

1.0.0.1 Modules . . . 1

**2 Installation Instructions** 3

2.0.1 Building . . . 3

2.0.2 Building Ginkgo in Windows . . . 5

2.0.3 Building Ginkgo with HIP support . . . 6

2.0.3.1 Changing the paths to search for HIP and other packages . . . 6

2.0.3.2 HIP platform detection of AMD and NVIDIA . . . 6

2.0.4 Third party libraries and packages . . . 7

2.0.5 Installing Ginkgo . . . 7

**3 Testing Instructions** 9

3.0.1 Running the unit tests . . . 9

3.0.1.1 Using make test . . . 9

3.0.1.2 Using make quick_test . . . 9

3.0.1.3 Using CTest . . . 9

**4 Running the benchmarks** 11

4.0.1 1: Ginkgo setup and best practice guidelines . . . 12

4.0.2 2: Using ssget to fetch the matrices . . . 12

4.0.3 3: Running benchmarks manually . . . 13

4.0.4 4: Benchmarking with <tt>make benchmark</tt> or <tt>run_all_benchmarks.sh</tt> . . . 14

4.0.5 5: Publishing the results on Github and analyze the results with the GPE (optional) . . . 14

4.0.6 6: Detailed performance analysis and debugging . . . 15

4.0.7 7: Available benchmark options . . . 15

**5 Contributing guidelines** 19

5.1 Table of Contents . . . 19

5.2 Most important stuff (A TL;DR) . . . 20

5.3 Project structure . . . 20

5.3.1 Extended header files . . . 20

5.3.2 Using library classes . . . 21

5.4 Git related . . . 21

5.4.1 Our git workflow . . . 21

5.4.2 Writing good commit messages . . . 21

5.4.2.1 Attributing credit . . . 21

5.4.3 Creating, Reviewing and Merging Pull Requests . . . 22

5.5 Code style . . . 22

5.5.1 Automatic code formatting . . . 22

5.5.2 Naming scheme . . . 23

5.5.2.1 Filenames . . . 23

5.5.2.2 Macros . . . 23

5.5.2.3 Variables . . . 23

5.5.2.4 Constants . . . 23
5.5.2.5 Functions . . . 23
5.5.2.6 Structures and classes . . . 23
5.5.2.7 Members . . . 24
5.5.2.8 Namespaces . . . 24
5.5.2.9 Template parameters . . . 24
5.5.3 Whitespace . . . 24
5.5.4 Include statement grouping . . . 25
5.5.4.1 Main header . . . 26
5.5.4.2 Some general comments. . . . 26
5.5.5 Other Code Formatting not handled by ClangFormat . . . 26
5.5.5.1 Control flow constructs . . . 26
5.5.5.2 Variable declarations . . . 26
5.5.6 CMake coding style . . . 26
5.5.6.1 Whitespaces . . . 26
5.5.6.2 Use of macros vs functions . . . 26
5.5.6.3 Naming style . . . 27
5.6 Helper scripts . . . 27
5.6.1 Create a new algorithm . . . 27
5.6.2 Converting CUDA code to HIP code . . . 27
5.7 Writing Tests . . . 27
5.7.1 Testing know-how . . . 27
5.7.2 Some general rules. . . . 28
5.7.3 Writing tests for kernels . . . 28
5.8 Documentation style . . . 28
5.8.1 Command prefix: <tt>@</tt> for Doxygen, <tt>\</tt> for math . . . 28
5.8.2 Developer targeted notes . . . 28
5.8.3 Whitespaces . . . 28
5.8.3.1 After named tags such as <tt>@param foo</tt> . . . 28
5.8.4 Documenting examples . . . 29
5.9 Other programming comments . . . 29
5.9.1 C++ standard stream objects . . . 29
5.9.2 Warnings . . . 29
5.9.3 Avoiding circular dependencies . . . 29

**6 Citing Ginkgo** **31**

6.0.1 The Ginkgo Software . . . 31
6.0.2 On Portability . . . 31
6.0.3 On Software Sustainability . . . 32
6.0.4 On SpMV or solvers performance . . . 32
6.0.5 On the Python bindings . . . 32

**7 Example programs** **35**

8 The adaptiveprecision-blockjacobi program 39

9 The batched-solver program 43

10 The cb-gmres program 53

11 The custom-logger program 59

12 The custom-matrix-format program 69

13 The custom-stopping-criterion program 77

14 The distributed-multigrid-preconditioned-solver program 83

15 The distributed-multigrid-reuse program 91

16 The distributed-solver program 101

17 The external-lib-interfacing program 109

18 The file-config-solver program 131

19 The ginkgo-overhead program 137

20 The ginkgo-ranges program 139

21 The heat-equation program 143

22 The ilu-preconditioned-solver program 149

23 The inverse-iteration program 153

24 The ir-ilu-preconditioned-solver program 159

25 The iterative-refinement program 165

26 The kokkos-assembly program 169

27 The minimal-cuda-solver program 175

28 The mixed-multigrid-preconditioned-solver program 177

29 The mixed-multigrid-solver program 183

30 The mixed-precision-ir program 189

31 The mixed-spmv program 195

32 The multigrid-preconditioned-solver program 203

33 The multigrid-preconditioned-solver-customized program 207

**34 The nine-pt-stencil-solver program** 213

**35 The papi-logging program** 223

**36 The par-ilu-convergence program** 229

**37 The performance-debugging program** 235

**38 The poisson-solver program** 247

**39 The preconditioned-solver program** 253

**40 The preconditioner-export program** 257

**41 The reordered-preconditioned-solver program** 265

**42 The schroedinger-splitting program** 269

**43 The simple-solver program** 275

**44 The simple-solver-logging program** 281

**45 The three-pt-stencil-solver program** 307

**46 Module Documentation** 315

46.1 Batched Linear Operators . . . 315
46.1.1 Detailed Description . . . 316
46.1.2 Batched Linear operator as a concept . . . 316
46.1.3 Macro Definition Documentation . . . 316
46.1.3.1 GKO_ENABLE_BATCH_LIN_OP_FACTORY . . . 317
46.1.4 Typedef Documentation . . . 318
46.1.4.1 EnableDefaultBatchLinOpFactory . . . 318
46.2 CUDA Executor . . . 319
46.2.1 Detailed Description . . . 319
46.3 Distributed . . . 320
46.3.1 Detailed Description . . . 320
46.4 DPC++ Executor . . . 321
46.4.1 Detailed Description . . . 321
46.5 Executors . . . 322
46.5.1 Detailed Description . . . 322
46.5.2 Executors in Ginkgo. . . . 323
46.5.3 Macro Definition Documentation . . . 323
46.5.3.1 GKO_REGISTER_HOST_OPERATION . . . 323
46.5.3.2 Example . . . 324
46.5.3.3 GKO_REGISTER_OPERATION . . . 324
46.5.3.4 Example . . . 324
46.6 Factorizations . . . 326

46.6.1 Detailed Description . . . 326
46.7 HIP Executor . . . 327
46.7.1 Detailed Description . . . 327
46.8 Jacobi Preconditioner . . . 328
46.8.1 Detailed Description . . . 328
46.9 Linear Operators . . . 329
46.9.1 Detailed Description . . . 333
46.9.2 Advantages of this approach and usage . . . 333
46.9.3 Linear operator as a concept . . . 334
46.9.4 Macro Definition Documentation . . . 334
46.9.4.1 GKO_CREATE_FACTORY_PARAMETERS . . . 335
46.9.4.2 GKO_ENABLE_BUILD_METHOD . . . 335
46.9.4.3 GKO_ENABLE_LIN_OP_FACTORY . . . 335
46.9.4.4 GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE . . . 336
46.9.4.5 GKO_FACTORY_PARAMETER . . . 337
46.9.4.6 GKO_FACTORY_PARAMETER_SCALAR . . . 338
46.9.4.7 GKO_FACTORY_PARAMETER_VECTOR . . . 338
46.9.5 Typedef Documentation . . . 338
46.9.5.1 EnableDefaultLinOpFactory . . . 339
46.9.6 Function Documentation . . . 339
46.9.6.1 initialize() [1/4] . . . 339
46.9.6.2 initialize() [2/4] . . . 340
46.9.6.3 initialize() [3/4] . . . 341
46.9.6.4 initialize() [4/4] . . . 341
46.10 Logging . . . 343
46.10.1 Detailed Description . . . 343
46.11 SpMV employing different Matrix formats . . . 344
46.11.1 Detailed Description . . . 345
46.12 Multigrid . . . 346
46.12.1 Detailed Description . . . 346
46.13 Multigrid Level . . . 347
46.13.1 Detailed Description . . . 347
46.14 OpenMP Executor . . . 348
46.14.1 Detailed Description . . . 348
46.15 Preconditioners . . . 349
46.15.1 Detailed Description . . . 349
46.16 Reference Executor . . . 350
46.16.1 Detailed Description . . . 350
46.17 Reordering Algorithms . . . 351
46.17.1 Detailed Description . . . 351
46.17.2 Macro Definition Documentation . . . 351
46.17.2.1 GKO_ENABLE_REORDERING_BASE_FACTORY . . . 351

46.18 Solvers . . . 353
46.18.1 Detailed Description . . . 354
46.19 Stopping criteria . . . 355
46.19.1 Detailed Description . . . 356
46.19.2 Enumeration Type Documentation . . . 356
46.19.2.1 mode . . . 356
46.19.3 Function Documentation . . . 356
46.19.3.1 combine() . . . 356

**47 Namespace Documentation** **359**
47.1 gko Namespace Reference . . . 359
47.1.1 Detailed Description . . . 371
47.1.2 Typedef Documentation . . . 371
47.1.2.1 highest_precision . . . 371
47.1.2.2 is_complex_or_scalar_s . . . 371
47.1.2.3 is_complex_s . . . 372
47.1.2.4 previous_precision_base . . . 372
47.1.2.5 remove_complex . . . 372
47.1.2.6 to_complex . . . 373
47.1.2.7 to_real . . . 373
47.1.3 Enumeration Type Documentation . . . 373
47.1.3.1 allocation_mode . . . 373
47.1.3.2 layout_type . . . 374
47.1.3.3 log_propagation_mode . . . 374
47.1.4 Function Documentation . . . 374
47.1.4.1 abs() . . . 375
47.1.4.2 as() [1/7] . . . 375
47.1.4.3 as() [2/7] . . . 376
47.1.4.4 as() [3/7] . . . 376
47.1.4.5 as() [4/7] . . . 377
47.1.4.6 as() [5/7] . . . 377
47.1.4.7 as() [6/7] . . . 379
47.1.4.8 as() [7/7] . . . 380
47.1.4.9 ceildiv() . . . 380
47.1.4.10 clone() [1/2] . . . 381
47.1.4.11 clone() [2/2] . . . 381
47.1.4.12 conj() . . . 382
47.1.4.13 copy_and_convert_to() [1/4] . . . 382
47.1.4.14 copy_and_convert_to() [2/4] . . . 383
47.1.4.15 copy_and_convert_to() [3/4] . . . 384
47.1.4.16 copy_and_convert_to() [4/4] . . . 384
47.1.4.17 get_significant_bit() . . . 385

47.1.4.18 get_superior_power() . . . 385
47.1.4.19 give() . . . 386
47.1.4.20 imag() . . . 387
47.1.4.21 is_complex() . . . 387
47.1.4.22 is_complex_or_scalar() . . . 387
47.1.4.23 is_finite() [1/2] . . . 388
47.1.4.24 is_finite() [2/2] . . . 388
47.1.4.25 is_nan() [1/2] . . . 389
47.1.4.26 is_nan() [2/2] . . . 389
47.1.4.27 is_nonzero() . . . 390
47.1.4.28 is_zero() . . . 390
47.1.4.29 lend() [1/2] . . . 391
47.1.4.30 lend() [2/2] . . . 391
47.1.4.31 make_array_view() . . . 392
47.1.4.32 make_const_array_view() . . . 393
47.1.4.33 make_const_dense_view() . . . 393
47.1.4.34 make_dense_view() . . . 394
47.1.4.35 make_temporary_clone() . . . 394
47.1.4.36 make_temporary_conversion() . . . 395
47.1.4.37 make_temporary_output_clone() . . . 396
47.1.4.38 max() . . . 396
47.1.4.39 min() . . . 397
47.1.4.40 mixed_precision_dispatch() . . . 397
47.1.4.41 mixed_precision_dispatch_real_complex() . . . 398
47.1.4.42 nan() [1/2] . . . 399
47.1.4.43 nan() [2/2] . . . 399
47.1.4.44 one() [1/2] . . . 400
47.1.4.45 one() [2/2] . . . 400
47.1.4.46 operator"!=() [1/2] . . . 400
47.1.4.47 operator"!=() [2/2] . . . 401
47.1.4.48 operator<<() [1/2] . . . 401
47.1.4.49 operator<<() [2/2] . . . 402
47.1.4.50 operator==() [1/2] . . . 402
47.1.4.51 operator==() [2/2] . . . 402
47.1.4.52 pi() . . . 403
47.1.4.53 precision_dispatch() . . . 403
47.1.4.54 precision_dispatch_real_complex() [1/3] . . . 404
47.1.4.55 precision_dispatch_real_complex() [2/3] . . . 405
47.1.4.56 precision_dispatch_real_complex() [3/3] . . . 405
47.1.4.57 read() . . . 406
47.1.4.58 read_binary() . . . 407
47.1.4.59 read_binary_raw() . . . 407

47.1.4.60 read_generic() . . . 408
47.1.4.61 read_generic_raw() . . . 409
47.1.4.62 read_raw() . . . 409
47.1.4.63 real() . . . 410
47.1.4.64 reduce_add() [1/2] . . . 410
47.1.4.65 reduce_add() [2/2] . . . 411
47.1.4.66 round_down() . . . 411
47.1.4.67 round_up() . . . 412
47.1.4.68 safe_divide() . . . 412
47.1.4.69 share() . . . 413
47.1.4.70 squared_norm() . . . 413
47.1.4.71 transpose() [1/2] . . . 414
47.1.4.72 transpose() [2/2] . . . 414
47.1.4.73 unit_root() . . . 415
47.1.4.74 with_matrix_type() . . . 415
47.1.4.75 write() [1/2] . . . 416
47.1.4.76 write() [2/2] . . . 417
47.1.4.77 write_binary() . . . 417
47.1.4.78 write_binary_raw() . . . 418
47.1.4.79 write_raw() . . . 418
47.1.4.80 zero() [1/2] . . . 420
47.1.4.81 zero() [2/2] . . . 420
47.2 gko::accessor Namespace Reference . . . 421
47.2.1 Detailed Description . . . 421
47.3 gko::batch::log Namespace Reference . . . 421
47.3.1 Detailed Description . . . 421
47.4 gko::experimental::distributed Namespace Reference . . . 421
47.4.1 Detailed Description . . . 423
47.4.2 Enumeration Type Documentation . . . 423
47.4.2.1 assembly_mode . . . 423
47.4.2.2 index_space . . . 423
47.4.3 Function Documentation . . . 424
47.4.3.1 assemble_rows_from_neighbors() . . . 424
47.4.3.2 build_partition_from_local_range() . . . 424
47.4.3.3 build_partition_from_local_size() . . . 425
47.4.3.4 make_temporary_conversion() [1/2] . . . 425
47.4.3.5 make_temporary_conversion() [2/2] . . . 426
47.4.3.6 precision_dispatch() . . . 427
47.4.3.7 precision_dispatch_real_complex() [1/3] . . . 428
47.4.3.8 precision_dispatch_real_complex() [2/3] . . . 428
47.4.3.9 precision_dispatch_real_complex() [3/3] . . . 429
47.5 gko::experimental::distributed::preconditioner Namespace Reference . . . 430

47.5.1 Detailed Description . . . 430
47.6 gko::experimental::mpi Namespace Reference . . . 430
47.6.1 Detailed Description . . . 431
47.6.2 Typedef Documentation . . . 431
47.6.2.1 comm_index_type . . . 431
47.6.3 Function Documentation . . . 431
47.6.3.1 get_walltime() . . . 431
47.6.3.2 map_rank_to_device_id() . . . 432
47.6.3.3 wait_all() . . . 432
47.7 gko::experimental::reorder Namespace Reference . . . 432
47.7.1 Detailed Description . . . 433
47.7.2 Enumeration Type Documentation . . . 433
47.7.2.1 mc64_strategy . . . 433
47.8 gko::factorization Namespace Reference . . . 433
47.8.1 Detailed Description . . . 434
47.8.2 Enumeration Type Documentation . . . 434
47.8.2.1 incomplete_algorithm . . . 434
47.9 gko::log Namespace Reference . . . 434
47.9.1 Detailed Description . . . 435
47.9.2 Enumeration Type Documentation . . . 435
47.9.2.1 profile_event_category . . . 435
47.10 gko::matrix Namespace Reference . . . 436
47.10.1 Detailed Description . . . 437
47.10.2 Enumeration Type Documentation . . . 437
47.10.2.1 permute_mode . . . 437
47.11 gko::multigrid Namespace Reference . . . 438
47.11.1 Detailed Description . . . 439
47.12 gko::name_demangling Namespace Reference . . . 439
47.12.1 Detailed Description . . . 439
47.12.2 Function Documentation . . . 439
47.12.2.1 get_dynamic_type() . . . 439
47.12.2.2 get_static_type() . . . 440
47.13 gko::preconditioner Namespace Reference . . . 440
47.13.1 Detailed Description . . . 441
47.13.2 Enumeration Type Documentation . . . 441
47.13.2.1 isai_type . . . 441
47.14 gko::reorder Namespace Reference . . . 441
47.14.1 Detailed Description . . . 442
47.14.2 Typedef Documentation . . . 442
47.14.2.1 EnableDefaultReorderingBaseFactory . . . 442
47.15 gko::solver Namespace Reference . . . 442
47.15.1 Detailed Description . . . 444

47.15.2 Enumeration Type Documentation . . . 445
47.15.2.1 initial_guess_mode . . . 445
47.15.2.2 trisolve_algorithm . . . 445
47.15.3 Function Documentation . . . 445
47.15.3.1 build_smoother() [1/2] . . . 445
47.15.3.2 build_smoother() [2/2] . . . 446
47.16 gko::solver::multigrid Namespace Reference . . . 446
47.16.1 Detailed Description . . . 447
47.16.2 Enumeration Type Documentation . . . 447
47.16.2.1 cycle . . . 447
47.16.2.2 mid_smooth_type . . . 447
47.17 gko::stop Namespace Reference . . . 448
47.17.1 Detailed Description . . . 449
47.17.2 Typedef Documentation . . . 449
47.17.2.1 EnableDefaultCriterionFactory . . . 449
47.17.3 Function Documentation . . . 450
47.17.3.1 absolute_implicit_residual_norm() . . . 450
47.17.3.2 absolute_residual_norm() . . . 450
47.17.3.3 initial_implicit_residual_norm() . . . 451
47.17.3.4 initial_residual_norm() . . . 452
47.17.3.5 max_iters() . . . 452
47.17.3.6 min_iters() . . . 453
47.17.3.7 relative_implicit_residual_norm() . . . 453
47.17.3.8 relative_residual_norm() . . . 454
47.17.3.9 time_limit() . . . 454
47.18 gko::syn Namespace Reference . . . 455
47.18.1 Detailed Description . . . 456
47.18.2 Typedef Documentation . . . 456
47.18.2.1 as_list . . . 456
47.18.2.2 concatenate . . . 456
47.18.3 Function Documentation . . . 456
47.18.3.1 as_array() . . . 456
47.19 gko::xstd Namespace Reference . . . 457
47.19.1 Detailed Description . . . 457

**48 Class Documentation** **459**

48.1 gko::AbsoluteComputable Class Reference . . . 459
48.1.1 Detailed Description . . . 459
48.1.2 Member Function Documentation . . . 459
48.1.2.1 compute_absolute_linop() . . . 459
48.2 gko::stop::AbsoluteResidualNorm< ValueType > Class Template Reference . . . 460
48.2.1 Detailed Description . . . 460

48.3 gko::AbstractFactory< AbstractProductType, ComponentsType > Class Template Reference . . . . 460
48.3.1 Detailed Description . . . . 461
48.3.2 Member Function Documentation . . . . 461
48.3.2.1 generate() . . . . 461
48.4 gko::AllocationError Class Reference . . . . 462
48.4.1 Detailed Description . . . . 462
48.4.2 Constructor & Destructor Documentation . . . . 462
48.4.2.1 AllocationError() . . . . 462
48.5 gko::Allocator Class Reference . . . . 463
48.5.1 Detailed Description . . . . 463
48.6 gko::experimental::reorder::Amd< IndexType > Class Template Reference . . . . 463
48.6.1 Detailed Description . . . . 463
48.6.2 Member Function Documentation . . . . 464
48.6.2.1 generate() . . . . 464
48.6.2.2 get_parameters() . . . . 464
48.7 gko::amd_device Class Reference . . . . 464
48.7.1 Detailed Description . . . . 465
48.8 gko::solver::ApplyWithInitialGuess Class Reference . . . . 465
48.8.1 Detailed Description . . . . 465
48.9 gko::are_all_integral< Args > Struct Template Reference . . . . 465
48.9.1 Detailed Description . . . . 465
48.10 gko::array< ValueType > Class Template Reference . . . . 466
48.10.1 Detailed Description . . . . 467
48.10.2 Constructor & Destructor Documentation . . . . 468
48.10.2.1 array() [1/11] . . . . 468
48.10.2.2 array() [2/11] . . . . 468
48.10.2.3 array() [3/11] . . . . 468
48.10.2.4 array() [4/11] . . . . 470
48.10.2.5 array() [5/11] . . . . 470
48.10.2.6 array() [6/11] . . . . 471
48.10.2.7 array() [7/11] . . . . 471
48.10.2.8 array() [8/11] . . . . 472
48.10.2.9 array() [9/11] . . . . 472
48.10.2.10 array() [10/11] . . . . 473
48.10.2.11 array() [11/11] . . . . 473
48.10.3 Member Function Documentation . . . . 473
48.10.3.1 as_const_view() . . . . 474
48.10.3.2 as_view() . . . . 474
48.10.3.3 clear() . . . . 474
48.10.3.4 const_view() . . . . 474
48.10.3.5 copy_to_host() . . . . 475
48.10.3.6 fill() . . . . 475

48.10.3.7 get_const_data() . . . 476
48.10.3.8 get_data() . . . 477
48.10.3.9 get_executor() . . . 477
48.10.3.10 get_num_elems() . . . 478
48.10.3.11 get_size() . . . 478
48.10.3.12 is_owning() . . . 479
48.10.3.13 operator=() [1/4] . . . 479
48.10.3.14 operator=() [2/4] . . . 480
48.10.3.15 operator=() [3/4] . . . 481
48.10.3.16 operator=() [4/4] . . . 482
48.10.3.17 resize_and_reset() . . . 483
48.10.3.18 set_executor() . . . 483
48.10.3.19 view() . . . 484
48.11 gko::device_matrix_data< ValueType, IndexType >::arrays Struct Reference . . . 484
48.11.1 Detailed Description . . . 484
48.12 gko::matrix::Hybrid< ValueType, IndexType >::automatic Class Reference . . . 485
48.12.1 Detailed Description . . . 485
48.12.2 Member Function Documentation . . . 485
48.12.2.1 compute_ell_num_stored_elements_per_row() . . . 485
48.13 gko::BadDimension Class Reference . . . 486
48.13.1 Detailed Description . . . 486
48.13.2 Constructor & Destructor Documentation . . . 486
48.13.2.1 BadDimension() . . . 486
48.14 gko::batch_dim< Dimensionality, DimensionType > Struct Template Reference . . . 487
48.14.1 Detailed Description . . . 487
48.14.2 Constructor & Destructor Documentation . . . 488
48.14.2.1 batch_dim() . . . 488
48.14.3 Member Function Documentation . . . 488
48.14.3.1 get_common_size() . . . 488
48.14.3.2 get_num_batch_items() . . . 489
48.14.4 Friends And Related Function Documentation . . . 489
48.14.4.1 operator"!= . . . 489
48.14.4.2 operator== . . . 489
48.15 gko::batch::log::BatchConvergence< ValueType > Class Template Reference . . . 490
48.15.1 Detailed Description . . . 490
48.15.2 Member Function Documentation . . . 491
48.15.2.1 create() . . . 491
48.15.2.2 get_num_iterations() . . . 491
48.15.2.3 get_residual_norm() . . . 492
48.16 gko::batch::BatchLinOpFactory Class Reference . . . 492
48.16.1 Detailed Description . . . 492
48.16.1.1 Example: using BatchCG in Ginkgo . . . 492

48.17 gko::batch::solver::BatchSolver Class Reference . . . 493
48.17.1 Detailed Description . . . 493
48.17.2 Member Function Documentation . . . 493
48.17.2.1 get_max_iterations() . . . 493
48.17.2.2 get_preconditioner() . . . 494
48.17.2.3 get_system_matrix() . . . 494
48.17.2.4 get_tolerance() . . . 494
48.17.2.5 get_tolerance_type() . . . 494
48.17.2.6 reset_max_iterations() . . . 494
48.17.2.7 reset_tolerance() . . . 495
48.17.2.8 reset_tolerance_type() . . . 495
48.18 gko::bfloat16 Class Reference . . . 496
48.18.1 Detailed Description . . . 496
48.19 gko::solver::Bicg< ValueType > Class Template Reference . . . 496
48.19.1 Detailed Description . . . 497
48.19.2 Member Function Documentation . . . 497
48.19.2.1 apply_uses_initial_guess() . . . 497
48.19.2.2 conj_transpose() . . . 498
48.19.2.3 parse() . . . 498
48.19.2.4 transpose() . . . 498
48.20 gko::batch::solver::Bicgstab< ValueType > Class Template Reference . . . 499
48.20.1 Detailed Description . . . 499
48.21 gko::solver::Bicgstab< ValueType > Class Template Reference . . . 500
48.21.1 Detailed Description . . . 500
48.21.2 Member Function Documentation . . . 500
48.21.2.1 apply_uses_initial_guess() . . . 501
48.21.2.2 conj_transpose() . . . 501
48.21.2.3 parse() . . . 501
48.21.2.4 transpose() . . . 502
48.22 gko::preconditioner::block_interleaved_storage_scheme< IndexType > Struct Template Reference 502
48.22.1 Detailed Description . . . 503
48.22.2 Member Function Documentation . . . 503
48.22.2.1 compute_storage_space() . . . 503
48.22.2.2 get_block_offset() . . . 504
48.22.2.3 get_global_block_offset() . . . 504
48.22.2.4 get_group_offset() . . . 505
48.22.2.5 get_group_size() . . . 505
48.22.2.6 get_stride() . . . 505
48.22.3 Member Data Documentation . . . 506
48.22.3.1 group_power . . . 506
48.23 gko::BlockOperator Class Reference . . . 506
48.23.1 Detailed Description . . . 507

48.23.2 Constructor & Destructor Documentation . . . 507
48.23.2.1 BlockOperator() [1/2] . . . 507
48.23.2.2 BlockOperator() [2/2] . . . 507
48.23.3 Member Function Documentation . . . 507
48.23.3.1 block_at() . . . 507
48.23.3.2 create() [1/2] . . . 508
48.23.3.3 create() [2/2] . . . 508
48.23.3.4 get_block_size() . . . 509
48.23.3.5 operator=() [1/2] . . . 509
48.23.3.6 operator=() [2/2] . . . 509
48.24 gko::BlockSizeError< IndexType > Class Template Reference . . . 509
48.24.1 Detailed Description . . . 509
48.24.2 Constructor & Destructor Documentation . . . 510
48.24.2.1 BlockSizeError() . . . 510
48.25 gko::solver::CbGmres< ValueType > Class Template Reference . . . 510
48.25.1 Detailed Description . . . 511
48.25.2 Member Function Documentation . . . 511
48.25.2.1 get_krylov_dim() . . . 511
48.25.2.2 get_storage_precision() . . . 512
48.25.2.3 parse() . . . 512
48.25.2.4 set_krylov_dim() . . . 512
48.26 gko::batch::solver::Cg< ValueType > Class Template Reference . . . 513
48.26.1 Detailed Description . . . 513
48.27 gko::solver::Cg< ValueType > Class Template Reference . . . 514
48.27.1 Detailed Description . . . 514
48.27.2 Member Function Documentation . . . 515
48.27.2.1 apply_uses_initial_guess() . . . 515
48.27.2.2 conj_transpose() . . . 515
48.27.2.3 parse() . . . 515
48.27.2.4 transpose() . . . 516
48.28 gko::solver::Cgs< ValueType > Class Template Reference . . . 516
48.28.1 Detailed Description . . . 517
48.28.2 Member Function Documentation . . . 517
48.28.2.1 apply_uses_initial_guess() . . . 517
48.28.2.2 conj_transpose() . . . 518
48.28.2.3 parse() . . . 518
48.28.2.4 transpose() . . . 519
48.29 gko::solver::Chebyshev< ValueType > Class Template Reference . . . 519
48.29.1 Detailed Description . . . 520
48.29.2 Constructor & Destructor Documentation . . . 520
48.29.2.1 Chebyshev() [1/2] . . . 520
48.29.2.2 Chebyshev() [2/2] . . . 520

48.29.3 Member Function Documentation . . . 521
48.29.3.1 apply_uses_initial_guess() . . . 521
48.29.3.2 conj_transpose() . . . 521
48.29.3.3 operator=() [1/2] . . . 521
48.29.3.4 operator=() [2/2] . . . 522
48.29.3.5 parse() . . . 522
48.29.3.6 transpose() . . . 522
48.30 gko::experimental::factorization::Cholesky< ValueType, IndexType > Class Template Reference . . . 523
48.30.1 Detailed Description . . . 523
48.30.2 Member Function Documentation . . . 523
48.30.2.1 generate() . . . 523
48.30.2.2 get_parameters() [1/2] . . . 524
48.30.2.3 get_parameters() [2/2] . . . 524
48.30.2.4 parse() . . . 524
48.31 gko::Cloneable Class Reference . . . 525
48.31.1 Detailed Description . . . 525
48.32 gko::experimental::mpi::CollectiveCommunicator Class Reference . . . 525
48.32.1 Detailed Description . . . 526
48.32.2 Member Function Documentation . . . 526
48.32.2.1 create_inverse() . . . 526
48.32.2.2 create_with_same_type() . . . 526
48.32.2.3 get_recv_size() . . . 527
48.32.2.4 get_send_size() . . . 527
48.32.2.5 i_all_to_all_v() [1/2] . . . 527
48.32.2.6 i_all_to_all_v() [2/2] . . . 528
48.32.2.7 resize() . . . 528
48.33 gko::matrix::Hybrid< ValueType, IndexType >::column_limit Class Reference . . . 529
48.33.1 Detailed Description . . . 529
48.33.2 Constructor & Destructor Documentation . . . 529
48.33.2.1 column_limit() . . . 529
48.33.3 Member Function Documentation . . . 530
48.33.3.1 compute_ell_num_stored_elements_per_row() . . . 530
48.33.3.2 get_num_columns() . . . 530
48.34 gko::Combination< ValueType > Class Template Reference . . . 531
48.34.1 Detailed Description . . . 531
48.34.2 Constructor & Destructor Documentation . . . 531
48.34.2.1 Combination() [1/2] . . . 532
48.34.2.2 Combination() [2/2] . . . 532
48.34.3 Member Function Documentation . . . 532
48.34.3.1 conj_transpose() . . . 532
48.34.3.2 get_coefficients() . . . 532
48.34.3.3 get_operators() . . . 533

48.34.3.4 operator=() [1/2] . . . 533
48.34.3.5 operator=() [2/2] . . . 533
48.34.3.6 transpose() . . . 533
48.35 gko::stop::Combined Class Reference . . . 534
48.35.1 Detailed Description . . . 534
48.36 gko::experimental::mpi::communicator Class Reference . . . 534
48.36.1 Detailed Description . . . 537
48.36.2 Constructor & Destructor Documentation . . . 538
48.36.2.1 communicator() [1/5] . . . 538
48.36.2.2 communicator() [2/5] . . . 538
48.36.2.3 communicator() [3/5] . . . 538
48.36.2.4 communicator() [4/5] . . . 539
48.36.2.5 communicator() [5/5] . . . 539
48.36.3 Member Function Documentation . . . 539
48.36.3.1 all_gather() . . . 539
48.36.3.2 all_reduce() [1/2] . . . 540
48.36.3.3 all_reduce() [2/2] . . . 541
48.36.3.4 all_to_all() [1/2] . . . 541
48.36.3.5 all_to_all() [2/2] . . . 542
48.36.3.6 all_to_all_v() [1/2] . . . 543
48.36.3.7 all_to_all_v() [2/2] . . . 543
48.36.3.8 broadcast() . . . 544
48.36.3.9 create_owning() . . . 545
48.36.3.10 gather() . . . 545
48.36.3.11 gather_v() . . . 546
48.36.3.12 get() . . . 547
48.36.3.13 i_all_gather() . . . 547
48.36.3.14 i_all_reduce() [1/2] . . . 548
48.36.3.15 i_all_reduce() [2/2] . . . 548
48.36.3.16 i_all_to_all() [1/2] . . . 549
48.36.3.17 i_all_to_all() [2/2] . . . 550
48.36.3.18 i_all_to_all_v() [1/2] . . . 551
48.36.3.19 i_all_to_all_v() [2/2] . . . 552
48.36.3.20 i_broadcast() . . . 553
48.36.3.21 i_gather() . . . 553
48.36.3.22 i_gather_v() . . . 554
48.36.3.23 i_recv() . . . 555
48.36.3.24 i_reduce() . . . 556
48.36.3.25 i_scan() . . . 557
48.36.3.26 i_scatter() . . . 557
48.36.3.27 i_scatter_v() . . . 558
48.36.3.28 i_send() . . . 559

48.36.3.29 is_congruent() . . . 560
48.36.3.30 is_identical() . . . 560
48.36.3.31 node_local_rank() . . . 561
48.36.3.32 operator"!=() . . . 561
48.36.3.33 operator=() [1/2] . . . 561
48.36.3.34 operator=() [2/2] . . . 561
48.36.3.35 operator==() . . . 562
48.36.3.36 rank() . . . 562
48.36.3.37 recv() . . . 562
48.36.3.38 reduce() . . . 563
48.36.3.39 scan() . . . 564
48.36.3.40 scatter() . . . 564
48.36.3.41 scatter_v() . . . 565
48.36.3.42 send() . . . 566
48.36.3.43 size() . . . 566
48.36.3.44 synchronize() . . . 567
48.37 gko::Composition< ValueType > Class Template Reference . . . 567
48.37.1 Detailed Description . . . 568
48.37.2 Constructor & Destructor Documentation . . . 568
48.37.2.1 Composition() [1/2] . . . 568
48.37.2.2 Composition() [2/2] . . . 568
48.37.3 Member Function Documentation . . . 569
48.37.3.1 conj_transpose() . . . 569
48.37.3.2 get_operators() . . . 569
48.37.3.3 operator=() [1/2] . . . 569
48.37.3.4 operator=() [2/2] . . . 570
48.37.3.5 transpose() . . . 570
48.38 gko::experimental::mpi::contiguous_type Class Reference . . . 570
48.38.1 Detailed Description . . . 571
48.38.2 Constructor & Destructor Documentation . . . 571
48.38.2.1 contiguous_type() [1/2] . . . 571
48.38.2.2 contiguous_type() [2/2] . . . 571
48.38.3 Member Function Documentation . . . 571
48.38.3.1 get() . . . 572
48.38.3.2 operator=() . . . 572
48.39 gko::log::Convergence< ValueType > Class Template Reference . . . 572
48.39.1 Detailed Description . . . 573
48.39.2 Member Function Documentation . . . 573
48.39.2.1 create() [1/2] . . . 573
48.39.2.2 create() [2/2] . . . 574
48.39.2.3 get_implicit_sq_resnorm() . . . 574
48.39.2.4 get_num_iterations() . . . 575

48.39.2.5 get_residual() . . . 575
48.39.2.6 get_residual_norm() . . . 575
48.39.2.7 has_converged() . . . 576
48.40 gko::ConvertibleTo< ResultType > Class Template Reference . . . 576
48.40.1 Detailed Description . . . 576
48.40.2 Member Function Documentation . . . 577
48.40.2.1 convert_to() . . . 577
48.40.2.2 move_to() . . . 577
48.41 gko::matrix::Coo< ValueType, IndexType > Class Template Reference . . . 578
48.41.1 Detailed Description . . . 579
48.41.2 Member Function Documentation . . . 580
48.41.2.1 apply2() [1/4] . . . 580
48.41.2.2 apply2() [2/4] . . . 580
48.41.2.3 apply2() [3/4] . . . 580
48.41.2.4 apply2() [4/4] . . . 581
48.41.2.5 compute_absolute() . . . 581
48.41.2.6 conj_transpose() . . . 581
48.41.2.7 create() [1/3] . . . 582
48.41.2.8 create() [2/3] . . . 582
48.41.2.9 create() [3/3] . . . 583
48.41.2.10 create_const() . . . 583
48.41.2.11 extract_diagonal() . . . 584
48.41.2.12 get_col_idxs() . . . 584
48.41.2.13 get_const_col_idxs() . . . 584
48.41.2.14 get_const_device_view() . . . 585
48.41.2.15 get_const_row_idxs() . . . 585
48.41.2.16 get_const_values() . . . 585
48.41.2.17 get_device_view() . . . 586
48.41.2.18 get_num_stored_elements() . . . 586
48.41.2.19 get_row_idxs() . . . 586
48.41.2.20 get_values() . . . 587
48.41.2.21 read() [1/3] . . . 587
48.41.2.22 read() [2/3] . . . 587
48.41.2.23 read() [3/3] . . . 587
48.41.2.24 transpose() . . . 588
48.41.2.25 write() . . . 588
48.42 gko::matrix::view::coo< ValueType, IndexType > Struct Template Reference . . . 589
48.42.1 Detailed Description . . . 589
48.43 gko::CpuAllocator Class Reference . . . 589
48.43.1 Detailed Description . . . 589
48.44 gko::CpuAllocatorBase Class Reference . . . 590
48.44.1 Detailed Description . . . 590

48.45 gko::CpuTimer Class Reference . . . 590
48.45.1 Detailed Description . . . 590
48.45.2 Member Function Documentation . . . 590
48.45.2.1 difference_async() . . . 590
48.46 gko::cpx_real_type< T > Struct Template Reference . . . 591
48.46.1 Detailed Description . . . 591
48.46.2 Member Typedef Documentation . . . 591
48.46.2.1 type . . . 591
48.47 gko::stop::Criterion Class Reference . . . 592
48.47.1 Detailed Description . . . 592
48.47.2 Member Function Documentation . . . 592
48.47.2.1 check() . . . 592
48.47.2.2 update() . . . 593
48.48 gko::log::criterion_data Struct Reference . . . 593
48.48.1 Detailed Description . . . 594
48.49 gko::stop::CriterionArgs Struct Reference . . . 594
48.49.1 Detailed Description . . . 594
48.50 gko::matrix::Csr< ValueType, IndexType > Class Template Reference . . . 594
48.50.1 Detailed Description . . . 598
48.50.2 Constructor & Destructor Documentation . . . 599
48.50.2.1 Csr() [1/2] . . . 599
48.50.2.2 Csr() [2/2] . . . 599
48.50.3 Member Function Documentation . . . 599
48.50.3.1 add_scale_reuse() . . . 599
48.50.3.2 column_permute() . . . 600
48.50.3.3 compute_absolute() . . . 600
48.50.3.4 conj_transpose() . . . 601
48.50.3.5 create() [1/6] . . . 601
48.50.3.6 create() [2/6] . . . 602
48.50.3.7 create() [3/6] . . . 602
48.50.3.8 create() [4/6] . . . 602
48.50.3.9 create() [5/6] . . . 603
48.50.3.10 create() [6/6] . . . 603
48.50.3.11 create_const() [1/2] . . . 603
48.50.3.12 create_const() [2/2] . . . 604
48.50.3.13 create_submatrix() [1/2] . . . 604
48.50.3.14 create_submatrix() [2/2] . . . 605
48.50.3.15 extract_diagonal() . . . 605
48.50.3.16 get_col_idxs() . . . 606
48.50.3.17 get_const_col_idxs() . . . 606
48.50.3.18 get_const_device_view() . . . 606
48.50.3.19 get_const_row_ptrs() . . . 607

48.50.3.20 get_const_srow() . . . 607
48.50.3.21 get_const_values() . . . 608
48.50.3.22 get_device_view() . . . 608
48.50.3.23 get_num_srow_elements() . . . 608
48.50.3.24 get_num_stored_elements() . . . 609
48.50.3.25 get_row_ptrs() . . . 609
48.50.3.26 get_srow() . . . 609
48.50.3.27 get_strategy() . . . 610
48.50.3.28 get_values() . . . 610
48.50.3.29 inv_scale() . . . 610
48.50.3.30 inverse_column_permute() . . . 610
48.50.3.31 inverse_permute() . . . 611
48.50.3.32 inverse_row_permute() . . . 611
48.50.3.33 multiply() . . . 612
48.50.3.34 multiply_add() . . . 612
48.50.3.35 multiply_add_reuse() . . . 613
48.50.3.36 multiply_reuse() . . . 613
48.50.3.37 operator=() [1/2] . . . 614
48.50.3.38 operator=() [2/2] . . . 614
48.50.3.39 permute() [1/3] . . . 614
48.50.3.40 permute() [2/3] . . . 615
48.50.3.41 permute() [3/3] . . . 615
48.50.3.42 permute_reuse() [1/2] . . . 616
48.50.3.43 permute_reuse() [2/2] . . . 616
48.50.3.44 read() [1/3] . . . 618
48.50.3.45 read() [2/3] . . . 618
48.50.3.46 read() [3/3] . . . 618
48.50.3.47 row_permute() . . . 619
48.50.3.48 scale() . . . 619
48.50.3.49 scale_add() . . . 620
48.50.3.50 scale_permute() [1/2] . . . 620
48.50.3.51 scale_permute() [2/2] . . . 621
48.50.3.52 set_strategy() . . . 621
48.50.3.53 transpose() . . . 622
48.50.3.54 transpose_reuse() . . . 622
48.50.3.55 write() [1/2] . . . 622
48.50.3.56 write() [2/2] . . . 623
48.51 gko::batch::matrix::Csr< ValueType, IndexType > Class Template Reference . . . 623
48.51.1 Detailed Description . . . 624
48.51.2 Member Function Documentation . . . 625
48.51.2.1 add_scaled_identity() . . . 625
48.51.2.2 apply() [1/4] . . . 625

48.51.2.3 apply() [2/4] . . . 626
48.51.2.4 apply() [3/4] . . . 626
48.51.2.5 apply() [4/4] . . . 626
48.51.2.6 create() [1/3] . . . 627
48.51.2.7 create() [2/3] . . . 627
48.51.2.8 create() [3/3] . . . 628
48.51.2.9 create_const() . . . 628
48.51.2.10 create_const_view_for_item() . . . 629
48.51.2.11 create_view_for_item() . . . 629
48.51.2.12 get_col_idxs() . . . 629
48.51.2.13 get_const_col_idxs() . . . 630
48.51.2.14 get_const_row_ptrs() . . . 630
48.51.2.15 get_const_values() . . . 631
48.51.2.16 get_const_values_for_item() . . . 631
48.51.2.17 get_num_elements_per_item() . . . 632
48.51.2.18 get_num_stored_elements() . . . 632
48.51.2.19 get_row_ptrs() . . . 632
48.51.2.20 get_values() . . . 633
48.51.2.21 get_values_for_item() . . . 633
48.51.2.22 scale() . . . 633
48.52 gko::matrix::view::csr< ValueType, IndexType > Struct Template Reference . . . 634
48.52.1 Detailed Description . . . 634
48.53 gko::CublasError Class Reference . . . 634
48.53.1 Detailed Description . . . 635
48.53.2 Constructor & Destructor Documentation . . . 635
48.53.2.1 CublasError() . . . 635
48.54 gko::cuda_stream Class Reference . . . 635
48.54.1 Detailed Description . . . 636
48.54.2 Constructor & Destructor Documentation . . . 636
48.54.2.1 cuda_stream() . . . 636
48.54.3 Member Function Documentation . . . 636
48.54.3.1 get() . . . 636
48.55 gko::CudaAllocator Class Reference . . . 637
48.55.1 Detailed Description . . . 637
48.56 gko::CudaAllocatorBase Class Reference . . . 637
48.56.1 Detailed Description . . . 637
48.57 gko::CudaError Class Reference . . . 637
48.57.1 Detailed Description . . . 638
48.57.2 Constructor & Destructor Documentation . . . 638
48.57.2.1 CudaError() . . . 638
48.58 gko::CudaExecutor Class Reference . . . 638
48.58.1 Detailed Description . . . 640

48.58.2 Member Function Documentation . . . 640
48.58.2.1 create() [1/2] . . . 640
48.58.2.2 create() [2/2] . . . 640
48.58.2.3 get_blas_handle() . . . 642
48.58.2.4 get_cublas_handle() . . . 642
48.58.2.5 get_cusparse_handle() . . . 642
48.58.2.6 get_description() . . . 643
48.58.2.7 get_master() [1/2] . . . 643
48.58.2.8 get_master() [2/2] . . . 643
48.58.2.9 get_sparselib_handle() . . . 643
48.58.2.10 get_stream() . . . 644
48.58.2.11 run() [1/3] . . . 644
48.58.2.12 run() [2/3] . . . 644
48.58.2.13 run() [3/3] . . . 646
48.59 gko::CudaTimer Class Reference . . . 646
48.59.1 Detailed Description . . . 647
48.59.2 Member Function Documentation . . . 647
48.59.2.1 difference_async() . . . 647
48.60 gko::ext::cuda::solver::Cudss< ValueType, IndexType > Class Template Reference . . . 648
48.60.1 Detailed Description . . . 648
48.60.2 Member Function Documentation . . . 648
48.60.2.1 get_config_map() . . . 649
48.60.2.2 refactorize() . . . 649
48.61 gko::CufftError Class Reference . . . 649
48.61.1 Detailed Description . . . 649
48.61.2 Constructor & Destructor Documentation . . . 650
48.61.2.1 CufftError() . . . 650
48.62 gko::CurandError Class Reference . . . 650
48.62.1 Detailed Description . . . 650
48.62.2 Constructor & Destructor Documentation . . . 650
48.62.2.1 CurandError() . . . 651
48.63 gko::CusparseError Class Reference . . . 651
48.63.1 Detailed Description . . . 651
48.63.2 Constructor & Destructor Documentation . . . 651
48.63.2.1 CusparseError() . . . 651
48.64 gko::default_converter< S, R > Struct Template Reference . . . 652
48.64.1 Detailed Description . . . 652
48.64.2 Member Function Documentation . . . 652
48.64.2.1 operator()() . . . 652
48.65 gko::deferred_factory_parameter< FactoryType > Class Template Reference . . . 654
48.65.1 Detailed Description . . . 654
48.65.2 Constructor & Destructor Documentation . . . 655

48.65.2.1 deferred_factory_parameter() . . . 655
48.65.3 Member Function Documentation . . . 655
48.65.3.1 on() . . . 655
48.66 gko::batch::matrix::Dense< ValueType > Class Template Reference . . . 656
48.66.1 Detailed Description . . . 657
48.66.2 Member Function Documentation . . . 657
48.66.2.1 add_scaled_identity() . . . 657
48.66.2.2 apply() [1/4] . . . 658
48.66.2.3 apply() [2/4] . . . 658
48.66.2.4 apply() [3/4] . . . 659
48.66.2.5 apply() [4/4] . . . 659
48.66.2.6 at() [1/4] . . . 659
48.66.2.7 at() [2/4] . . . 660
48.66.2.8 at() [3/4] . . . 660
48.66.2.9 at() [4/4] . . . 661
48.66.2.10 create() [1/3] . . . 661
48.66.2.11 create() [2/3] . . . 662
48.66.2.12 create() [3/3] . . . 662
48.66.2.13 create_const() . . . 663
48.66.2.14 create_const_view_for_item() . . . 663
48.66.2.15 create_view_for_item() . . . 664
48.66.2.16 get_const_values() . . . 664
48.66.2.17 get_const_values_for_item() . . . 664
48.66.2.18 get_cumulative_offset() . . . 665
48.66.2.19 get_num_elements_per_item() . . . 665
48.66.2.20 get_num_stored_elements() . . . 666
48.66.2.21 get_values() . . . 666
48.66.2.22 get_values_for_item() . . . 666
48.66.2.23 scale() . . . 667
48.66.2.24 scale_add() . . . 667
48.67 gko::matrix::view::dense< ValueType > Struct Template Reference . . . 668
48.67.1 Detailed Description . . . 668
48.68 gko::matrix::Dense< ValueType > Class Template Reference . . . 668
48.68.1 Detailed Description . . . 673
48.68.2 Constructor & Destructor Documentation . . . 674
48.68.2.1 Dense() [1/2] . . . 674
48.68.2.2 Dense() [2/2] . . . 674
48.68.3 Member Function Documentation . . . 674
48.68.3.1 add_scaled() . . . 674
48.68.3.2 at() [1/4] . . . 675
48.68.3.3 at() [2/4] . . . 675
48.68.3.4 at() [3/4] . . . 676

48.68.3.5 at() [4/4] . . . 676
48.68.3.6 column_permute() [1/4] . . . 677
48.68.3.7 column_permute() [2/4] . . . 677
48.68.3.8 column_permute() [3/4] . . . 677
48.68.3.9 column_permute() [4/4] . . . 678
48.68.3.10 compute_absolute() [1/2] . . . 678
48.68.3.11 compute_absolute() [2/2] . . . 678
48.68.3.12 compute_conj_dot() [1/2] . . . 679
48.68.3.13 compute_conj_dot() [2/2] . . . 679
48.68.3.14 compute_dot() [1/2] . . . 680
48.68.3.15 compute_dot() [2/2] . . . 680
48.68.3.16 compute_mean() [1/2] . . . 680
48.68.3.17 compute_mean() [2/2] . . . 681
48.68.3.18 compute_norm1() [1/2] . . . 681
48.68.3.19 compute_norm1() [2/2] . . . 681
48.68.3.20 compute_norm2() [1/2] . . . 682
48.68.3.21 compute_norm2() [2/2] . . . 682
48.68.3.22 compute_squared_norm2() [1/2] . . . 682
48.68.3.23 compute_squared_norm2() [2/2] . . . 683
48.68.3.24 conj_transpose() [1/2] . . . 683
48.68.3.25 conj_transpose() [2/2] . . . 683
48.68.3.26 create() [1/3] . . . 684
48.68.3.27 create() [2/3] . . . 684
48.68.3.28 create() [3/3] . . . 684
48.68.3.29 create_const() . . . 685
48.68.3.30 create_const_view_of() . . . 685
48.68.3.31 create_real_view() [1/2] . . . 686
48.68.3.32 create_real_view() [2/2] . . . 686
48.68.3.33 create_submatrix() [1/3] . . . 686
48.68.3.34 create_submatrix() [2/3] . . . 687
48.68.3.35 create_submatrix() [3/3] . . . 687
48.68.3.36 create_view_of() . . . 688
48.68.3.37 create_with_config_of() . . . 688
48.68.3.38 create_with_type_of() [1/3] . . . 688
48.68.3.39 create_with_type_of() [2/3] . . . 689
48.68.3.40 create_with_type_of() [3/3] . . . 690
48.68.3.41 extract_diagonal() [1/2] . . . 690
48.68.3.42 extract_diagonal() [2/2] . . . 691
48.68.3.43 fill() . . . 691
48.68.3.44 get_const_values() . . . 691
48.68.3.45 get_num_stored_elements() . . . 692
48.68.3.46 get_stride() . . . 692

48.68.3.47 get_values() . . . 692
48.68.3.48 inv_scale() . . . 692
48.68.3.49 inverse_column_permute() [1/4] . . . 693
48.68.3.50 inverse_column_permute() [2/4] . . . 693
48.68.3.51 inverse_column_permute() [3/4] . . . 694
48.68.3.52 inverse_column_permute() [4/4] . . . 694
48.68.3.53 inverse_permute() [1/4] . . . 694
48.68.3.54 inverse_permute() [2/4] . . . 695
48.68.3.55 inverse_permute() [3/4] . . . 695
48.68.3.56 inverse_permute() [4/4] . . . 696
48.68.3.57 inverse_row_permute() [1/4] . . . 696
48.68.3.58 inverse_row_permute() [2/4] . . . 696
48.68.3.59 inverse_row_permute() [3/4] . . . 697
48.68.3.60 inverse_row_permute() [4/4] . . . 697
48.68.3.61 make_complex() [1/2] . . . 697
48.68.3.62 make_complex() [2/2] . . . 698
48.68.3.63 operator=() [1/2] . . . 698
48.68.3.64 operator=() [2/2] . . . 698
48.68.3.65 permute() [1/12] . . . 698
48.68.3.66 permute() [2/12] . . . 699
48.68.3.67 permute() [3/12] . . . 699
48.68.3.68 permute() [4/12] . . . 700
48.68.3.69 permute() [5/12] . . . 700
48.68.3.70 permute() [6/12] . . . 700
48.68.3.71 permute() [7/12] . . . 701
48.68.3.72 permute() [8/12] . . . 701
48.68.3.73 permute() [9/12] . . . 701
48.68.3.74 permute() [10/12] . . . 702
48.68.3.75 permute() [11/12] . . . 702
48.68.3.76 permute() [12/12] . . . 702
48.68.3.77 row_gather() [1/6] . . . 702
48.68.3.78 row_gather() [2/6] . . . 703
48.68.3.79 row_gather() [3/6] . . . 703
48.68.3.80 row_gather() [4/6] . . . 703
48.68.3.81 row_gather() [5/6] . . . 704
48.68.3.82 row_gather() [6/6] . . . 704
48.68.3.83 row_permute() [1/4] . . . 704
48.68.3.84 row_permute() [2/4] . . . 705
48.68.3.85 row_permute() [3/4] . . . 705
48.68.3.86 row_permute() [4/4] . . . 706
48.68.3.87 scale() . . . 706
48.68.3.88 scale_permute() [1/8] . . . 706

48.68.3.89 scale_permute() [2/8] . . . 707
48.68.3.90 scale_permute() [3/8] . . . 707
48.68.3.91 scale_permute() [4/8] . . . 708
48.68.3.92 scale_permute() [5/8] . . . 708
48.68.3.93 scale_permute() [6/8] . . . 708
48.68.3.94 scale_permute() [7/8] . . . 708
48.68.3.95 scale_permute() [8/8] . . . 709
48.68.3.96 sub_scaled() . . . 709
48.68.3.97 transpose() [1/2] . . . 709
48.68.3.98 transpose() [2/2] . . . 709
48.69 gko::experimental::mpi::DenseCommunicator Class Reference . . . 710
48.69.1 Detailed Description . . . 711
48.69.2 Constructor & Destructor Documentation . . . 711
48.69.2.1 DenseCommunicator() [1/2] . . . 711
48.69.2.2 DenseCommunicator() [2/2] . . . 711
48.69.3 Member Function Documentation . . . 712
48.69.3.1 create_inverse() . . . 712
48.69.3.2 create_with_same_type() . . . 712
48.69.3.3 get_recv_size() . . . 712
48.69.3.4 get_send_size() . . . 713
48.69.3.5 i_all_to_all_v() [1/2] . . . 713
48.69.3.6 i_all_to_all_v() [2/2] . . . 713
48.69.3.7 resize() . . . 714
48.69.4 Friends And Related Function Documentation . . . 714
48.69.4.1 operator"!= . . . 714
48.69.4.2 operator== . . . 715
48.70 gko::device_matrix_data< ValueType, IndexType > Class Template Reference . . . 715
48.70.1 Detailed Description . . . 716
48.70.2 Constructor & Destructor Documentation . . . 718
48.70.2.1 device_matrix_data() [1/4] . . . 718
48.70.2.2 device_matrix_data() [2/4] . . . 718
48.70.2.3 device_matrix_data() [3/4] . . . 718
48.70.2.4 device_matrix_data() [4/4] . . . 719
48.70.3 Member Function Documentation . . . 719
48.70.3.1 copy_to_host() . . . 719
48.70.3.2 create_from_host() . . . 720
48.70.3.3 empty_out() . . . 720
48.70.3.4 get_col_idxs() . . . 720
48.70.3.5 get_const_col_idxs() . . . 721
48.70.3.6 get_const_row_idxs() . . . 721
48.70.3.7 get_const_values() . . . 721
48.70.3.8 get_executor() . . . 722

48.70.3.9 get_num_elems() . . . 722
48.70.3.10 get_num_stored_elements() . . . 722
48.70.3.11 get_row_idxs() . . . 723
48.70.3.12 get_size() . . . 723
48.70.3.13 get_values() . . . 723
48.70.3.14 remove_zeros() . . . 723
48.70.3.15 resize_and_reset() [1/2] . . . 723
48.70.3.16 resize_and_reset() [2/2] . . . 724
48.70.3.17 sum_duplicates() . . . 724
48.71 gko::matrix::Diagonal< ValueType > Class Template Reference . . . 724
48.71.1 Detailed Description . . . 725
48.71.2 Member Function Documentation . . . 726
48.71.2.1 compute_absolute() . . . 726
48.71.2.2 conj_transpose() . . . 726
48.71.2.3 create() [1/3] . . . 726
48.71.2.4 create() [2/3] . . . 727
48.71.2.5 create() [3/3] . . . 727
48.71.2.6 create_const() . . . 728
48.71.2.7 get_const_values() . . . 728
48.71.2.8 get_values() . . . 729
48.71.2.9 inverse_apply() . . . 729
48.71.2.10 rapply() . . . 729
48.71.2.11 transpose() . . . 730
48.72 gko::DiagonalExtractable< ValueType > Class Template Reference . . . 730
48.72.1 Detailed Description . . . 731
48.72.2 Member Function Documentation . . . 731
48.72.2.1 extract_diagonal() . . . 731
48.72.2.2 extract_diagonal_linop() . . . 731
48.73 gko::DiagonalLinOpExtractable Class Reference . . . 732
48.73.1 Detailed Description . . . 732
48.73.2 Member Function Documentation . . . 732
48.73.2.1 extract_diagonal_linop() . . . 732
48.74 gko::dim< Dimensionality, DimensionType > Struct Template Reference . . . 732
48.74.1 Detailed Description . . . 733
48.74.2 Constructor & Destructor Documentation . . . 733
48.74.2.1 dim() [1/2] . . . 733
48.74.2.2 dim() [2/2] . . . 735
48.74.3 Member Function Documentation . . . 735
48.74.3.1 operator bool() . . . 735
48.74.3.2 operator[]() [1/2] . . . 736
48.74.3.3 operator[]() [2/2] . . . 736
48.74.4 Friends And Related Function Documentation . . . 736

48.74.4.1 operator"!= . . . 736
48.74.4.2 operator∗ . . . 737
48.74.4.3 operator<< . . . 737
48.74.4.4 operator== . . . 738
48.75 gko::DimensionMismatch Class Reference . . . 738
48.75.1 Detailed Description . . . 738
48.75.2 Constructor & Destructor Documentation . . . 739
48.75.2.1 DimensionMismatch() . . . 739
48.76 gko::experimental::solver::Direct< ValueType, IndexType > Class Template Reference . . . 739
48.76.1 Detailed Description . . . 740
48.76.2 Member Function Documentation . . . 740
48.76.2.1 conj_transpose() . . . 740
48.76.2.2 parse() . . . 741
48.76.2.3 transpose() . . . 741
48.77 gko::experimental::distributed::DistributedBase Class Reference . . . 741
48.77.1 Detailed Description . . . 742
48.77.2 Member Function Documentation . . . 742
48.77.2.1 get_communicator() . . . 742
48.77.2.2 operator=() [1/2] . . . 742
48.77.2.3 operator=() [2/2] . . . 743
48.78 gko::DpcppExecutor Class Reference . . . 743
48.78.1 Detailed Description . . . 744
48.78.2 Member Function Documentation . . . 744
48.78.2.1 create() . . . 744
48.78.2.2 get_description() . . . 745
48.78.2.3 get_device_id() . . . 745
48.78.2.4 get_device_type() . . . 745
48.78.2.5 get_master() [1/2] . . . 745
48.78.2.6 get_master() [2/2] . . . 746
48.78.2.7 get_max_subgroup_size() . . . 746
48.78.2.8 get_max_workgroup_size() . . . 746
48.78.2.9 get_max_workitem_sizes() . . . 746
48.78.2.10 get_num_computing_units() . . . 747
48.78.2.11 get_num_devices() . . . 747
48.78.2.12 get_subgroup_sizes() . . . 747
48.78.2.13 run() [1/3] . . . 748
48.78.2.14 run() [2/3] . . . 749
48.78.2.15 run() [3/3] . . . 749
48.79 gko::DpcppTimer Class Reference . . . 750
48.79.1 Detailed Description . . . 750
48.79.2 Member Function Documentation . . . 751
48.79.2.1 difference_async() . . . 751

48.80 gko::batch::matrix::Ell< ValueType, IndexType > Class Template Reference . . . 751
48.80.1 Detailed Description . . . 753
48.80.2 Member Function Documentation . . . 753
48.80.2.1 add_scaled_identity() . . . 753
48.80.2.2 apply() [1/4] . . . 754
48.80.2.3 apply() [2/4] . . . 754
48.80.2.4 apply() [3/4] . . . 754
48.80.2.5 apply() [4/4] . . . 755
48.80.2.6 create() [1/3] . . . 755
48.80.2.7 create() [2/3] . . . 755
48.80.2.8 create() [3/3] . . . 756
48.80.2.9 create_const() . . . 756
48.80.2.10 create_const_view_for_item() . . . 757
48.80.2.11 create_view_for_item() . . . 757
48.80.2.12 get_col_idxs() . . . 758
48.80.2.13 get_col_idxs_for_item() . . . 758
48.80.2.14 get_const_col_idxs() . . . 759
48.80.2.15 get_const_col_idxs_for_item() . . . 759
48.80.2.16 get_const_values() . . . 760
48.80.2.17 get_const_values_for_item() . . . 760
48.80.2.18 get_num_elements_per_item() . . . 761
48.80.2.19 get_num_stored_elements() . . . 761
48.80.2.20 get_num_stored_elements_per_row() . . . 761
48.80.2.21 get_values() . . . 762
48.80.2.22 get_values_for_item() . . . 762
48.80.2.23 scale() . . . 762
48.81 gko::matrix::Ell< ValueType, IndexType > Class Template Reference . . . 763
48.81.1 Detailed Description . . . 764
48.81.2 Constructor & Destructor Documentation . . . 765
48.81.2.1 Ell() [1/2] . . . 765
48.81.2.2 Ell() [2/2] . . . 765
48.81.3 Member Function Documentation . . . 765
48.81.3.1 col_at() [1/2] . . . 765
48.81.3.2 col_at() [2/2] . . . 766
48.81.3.3 compute_absolute() . . . 766
48.81.3.4 create() [1/3] . . . 767
48.81.3.5 create() [2/3] . . . 767
48.81.3.6 create() [3/3] . . . 768
48.81.3.7 create_const() . . . 768
48.81.3.8 extract_diagonal() . . . 769
48.81.3.9 get_col_idxs() . . . 769
48.81.3.10 get_const_col_idxs() . . . 769

48.81.3.11 get_const_values() . . . 770
48.81.3.12 get_num_stored_elements() . . . 770
48.81.3.13 get_num_stored_elements_per_row() . . . 771
48.81.3.14 get_stride() . . . 771
48.81.3.15 get_values() . . . 771
48.81.3.16 operator=() [1/2] . . . 771
48.81.3.17 operator=() [2/2] . . . 772
48.81.3.18 read() [1/3] . . . 772
48.81.3.19 read() [2/3] . . . 772
48.81.3.20 read() [3/3] . . . 772
48.81.3.21 val_at() [1/2] . . . 773
48.81.3.22 val_at() [2/2] . . . 773
48.81.3.23 write() . . . 774
48.82 gko::matrix::view::ell< ValueType, IndexType > Struct Template Reference . . . 774
48.82.1 Detailed Description . . . 775
48.83 gko::enable_parameters_type< ConcreteParametersType, Factory > Class Template Reference . . . 775
48.83.1 Detailed Description . . . 775
48.83.2 Member Function Documentation . . . 776
48.83.2.1 on() . . . 776
48.84 gko::EnableAbsoluteComputation< AbsoluteLinOp > Class Template Reference . . . 776
48.84.1 Detailed Description . . . 777
48.84.2 Member Function Documentation . . . 777
48.84.2.1 compute_absolute() . . . 777
48.84.2.2 compute_absolute_linop() . . . 777
48.85 gko::solver::EnableApplyWithInitialGuess< DerivedType > Class Template Reference . . . 778
48.85.1 Detailed Description . . . 778
48.86 gko::batch::solver::EnableBatchSolver< ConcreteSolver, ValueType, PolymorphicBase > Class Template Reference . . . 778
48.86.1 Detailed Description . . . 778
48.87 gko::EnableCloneable< ConcreteType > Class Template Reference . . . 779
48.87.1 Detailed Description . . . 779
48.87.2 Member Function Documentation . . . 779
48.87.2.1 clone() [1/2] . . . 780
48.87.2.2 clone() [2/2] . . . 780
48.87.2.3 convert_to() . . . 780
48.87.2.4 move_to() . . . 781
48.88 gko::EnableCreateMethod< ConcreteType > Class Template Reference . . . 781
48.88.1 Detailed Description . . . 781
48.89 gko::EnableDefaultFactory< ConcreteFactory, ProductType, ParametersType, PolymorphicBase > Class Template Reference . . . 782
48.89.1 Detailed Description . . . 782
48.89.2 Member Function Documentation . . . 783
48.89.2.1 create() . . . 783

48.89.2.2 get_parameters() . . . 783
48.90 gko::EnableDefaultReusableLinOpFactory< ConcreteFactory, ConcreteLinOp, ParametersType, ReuseDataType, PolymorphicBase > Class Template Reference . . . 783
48.90.1 Detailed Description . . . 784
48.91 gko::solver::EnableIterativeBase< DerivedType > Class Template Reference . . . 784
48.91.1 Detailed Description . . . 785
48.91.2 Constructor & Destructor Documentation . . . 785
48.91.2.1 EnableIterativeBase() . . . 785
48.91.3 Member Function Documentation . . . 785
48.91.3.1 operator=() . . . 786
48.91.3.2 set_stop_criterion_factory() . . . 786
48.92 gko::log::EnableLogging< ConcreteLoggable, PolymorphicBase > Class Template Reference . . . 786
48.92.1 Detailed Description . . . 787
48.93 gko::multigrid::EnableMultigridLevel< ValueType > Class Template Reference . . . 787
48.93.1 Detailed Description . . . 787
48.93.2 Member Function Documentation . . . 788
48.93.2.1 get_coarse_op() . . . 788
48.93.2.2 get_fine_op() . . . 788
48.93.2.3 get_prolong_op() . . . 788
48.93.2.4 get_restrict_op() . . . 789
48.94 gko::solver::EnablePreconditionable< DerivedType > Class Template Reference . . . 789
48.94.1 Detailed Description . . . 789
48.94.2 Constructor & Destructor Documentation . . . 790
48.94.2.1 EnablePreconditionable() . . . 790
48.94.3 Member Function Documentation . . . 790
48.94.3.1 operator=() . . . 790
48.94.3.2 set_preconditioner() . . . 790
48.95 gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType > Class Template Reference . . . 791
48.95.1 Detailed Description . . . 791
48.95.2 Member Function Documentation . . . 792
48.95.2.1 get_left_nullspace() . . . 792
48.95.2.2 get_nullspace() . . . 792
48.95.2.3 get_unprojected_preconditioner() . . . 792
48.95.2.4 operator=() . . . 793
48.95.2.5 set_preconditioner() . . . 793
48.96 gko::solver::EnableSolverBase< DerivedType, MatrixType > Class Template Reference . . . 793
48.96.1 Detailed Description . . . 794
48.96.2 Constructor & Destructor Documentation . . . 794
48.96.2.1 EnableSolverBase() . . . 794
48.96.3 Member Function Documentation . . . 794
48.96.3.1 operator=() . . . 795
48.97 gko::experimental::mpi::environment Class Reference . . . 795

48.97.1 Detailed Description . . . 795
48.97.2 Constructor & Destructor Documentation . . . 795
48.97.2.1 environment() . . . 795
48.97.3 Member Function Documentation . . . 796
48.97.3.1 get_provided_thread_support() . . . 796
48.98 gko::Error Class Reference . . . 796
48.98.1 Detailed Description . . . 797
48.98.2 Constructor & Destructor Documentation . . . 797
48.98.2.1 Error() . . . 797
48.99 gko::Executor Class Reference . . . 797
48.99.1 Detailed Description . . . 799
48.99.2 Member Function Documentation . . . 800
48.99.2.1 add_logger() . . . 800
48.99.2.2 alloc() . . . 800
48.99.2.3 copy() . . . 801
48.99.2.4 copy_from() . . . 801
48.99.2.5 copy_val_to_host() . . . 802
48.99.2.6 free() . . . 803
48.99.2.7 get_description() . . . 803
48.99.2.8 get_master() [1/2] . . . 803
48.99.2.9 get_master() [2/2] . . . 804
48.99.2.10 memory_accessible() . . . 804
48.99.2.11 remove_logger() . . . 804
48.99.2.12 run() [1/3] . . . 805
48.99.2.13 run() [2/3] . . . 805
48.99.2.14 run() [3/3] . . . 806
48.99.2.15 set_log_propagation_mode() . . . 806
48.99.2.16 should_propagate_log() . . . 807
48.100 gko::log::executor_data Struct Reference . . . 807
48.100.1 Detailed Description . . . 807
48.101 gko::executor_deleter< T > Class Template Reference . . . 807
48.101.1 Detailed Description . . . 807
48.101.2 Constructor & Destructor Documentation . . . 808
48.101.2.1 executor_deleter() . . . 808
48.101.3 Member Function Documentation . . . 808
48.101.3.1 operator()() . . . 808
48.102 gko::experimental::factorization::Factorization< ValueType, IndexType > Class Template Reference 809
48.102.1 Detailed Description . . . 809
48.102.2 Member Function Documentation . . . 810
48.102.2.1 create_from_combined_lu() . . . 810
48.102.2.2 create_from_composition() . . . 810
48.102.2.3 create_from_symm_composition() . . . 811

48.102.2.4 unpack() . . . 811
48.103 gko::matrix::Fbcsr< ValueType, IndexType > Class Template Reference . . . 811
48.103.1 Detailed Description . . . 813
48.103.2 Constructor & Destructor Documentation . . . 814
48.103.2.1 Fbcsr() [1/2] . . . 814
48.103.2.2 Fbcsr() [2/2] . . . 814
48.103.3 Member Function Documentation . . . 814
48.103.3.1 compute_absolute() . . . 814
48.103.3.2 conj_transpose() . . . 815
48.103.3.3 convert_to() [1/2] . . . 815
48.103.3.4 convert_to() [2/2] . . . 815
48.103.3.5 create() [1/4] . . . 815
48.103.3.6 create() [2/4] . . . 816
48.103.3.7 create() [3/4] . . . 816
48.103.3.8 create() [4/4] . . . 817
48.103.3.9 create_const() . . . 817
48.103.3.10 extract_diagonal() . . . 818
48.103.3.11 get_block_size() . . . 818
48.103.3.12 get_col_idxs() . . . 819
48.103.3.13 get_const_col_idxs() . . . 819
48.103.3.14 get_const_row_ptrs() . . . 819
48.103.3.15 get_const_values() . . . 820
48.103.3.16 get_num_block_cols() . . . 820
48.103.3.17 get_num_block_rows() . . . 820
48.103.3.18 get_num_stored_blocks() . . . 821
48.103.3.19 get_num_stored_elements() . . . 821
48.103.3.20 get_row_ptrs() . . . 821
48.103.3.21 get_values() . . . 822
48.103.3.22 is_sorted_by_column_index() . . . 822
48.103.3.23 operator=() [1/2] . . . 822
48.103.3.24 operator=() [2/2] . . . 822
48.103.3.25 read() [1/3] . . . 822
48.103.3.26 read() [2/3] . . . 823
48.103.3.27 read() [3/3] . . . 823
48.103.3.28 transpose() . . . 824
48.103.3.29 write() . . . 824
48.104 gko::solver::Fcg< ValueType > Class Template Reference . . . 824
48.104.1 Detailed Description . . . 825
48.104.2 Member Function Documentation . . . 825
48.104.2.1 apply_uses_initial_guess() . . . 825
48.104.2.2 conj_transpose() . . . 826
48.104.2.3 parse() . . . 826

48.104.2.4 transpose() . . . 827
48.105 gko::matrix::Fft Class Reference . . . 827
48.105.1 Detailed Description . . . 828
48.105.2 Member Function Documentation . . . 828
48.105.2.1 conj_transpose() . . . 828
48.105.2.2 create() [1/2] . . . 828
48.105.2.3 create() [2/2] . . . 829
48.105.2.4 transpose() . . . 829
48.105.2.5 write() [1/4] . . . 829
48.105.2.6 write() [2/4] . . . 830
48.105.2.7 write() [3/4] . . . 830
48.105.2.8 write() [4/4] . . . 830
48.106 gko::matrix::Fft2 Class Reference . . . 831
48.106.1 Detailed Description . . . 831
48.106.2 Member Function Documentation . . . 832
48.106.2.1 conj_transpose() . . . 832
48.106.2.2 create() [1/3] . . . 832
48.106.2.3 create() [2/3] . . . 832
48.106.2.4 create() [3/3] . . . 833
48.106.2.5 transpose() . . . 833
48.106.2.6 write() [1/4] . . . 833
48.106.2.7 write() [2/4] . . . 834
48.106.2.8 write() [3/4] . . . 834
48.106.2.9 write() [4/4] . . . 834
48.107 gko::matrix::Fft3 Class Reference . . . 835
48.107.1 Detailed Description . . . 835
48.107.2 Member Function Documentation . . . 836
48.107.2.1 conj_transpose() . . . 836
48.107.2.2 create() [1/3] . . . 836
48.107.2.3 create() [2/3] . . . 836
48.107.2.4 create() [3/3] . . . 837
48.107.2.5 transpose() . . . 837
48.107.2.6 write() [1/4] . . . 837
48.107.2.7 write() [2/4] . . . 838
48.107.2.8 write() [3/4] . . . 838
48.107.2.9 write() [4/4] . . . 838
48.108 gko::multigrid::FixedCoarsening< ValueType, IndexType > Class Template Reference . . . 839
48.108.1 Detailed Description . . . 839
48.108.2 Member Function Documentation . . . 839
48.108.2.1 get_system_matrix() . . . 839
48.109 gko::preconditioner::GaussSeidel< ValueType, IndexType > Class Template Reference . . . 840
48.109.1 Detailed Description . . . 840

48.109.2 Member Function Documentation . . . 840
48.109.2.1 generate() . . . 840
48.109.2.2 get_parameters() [1/2] . . . 841
48.109.2.3 get_parameters() [2/2] . . . 841
48.110 gko::solver::Gcr< ValueType > Class Template Reference . . . 841
48.110.1 Detailed Description . . . 842
48.110.2 Member Function Documentation . . . 842
48.110.2.1 apply_uses_initial_guess() . . . 843
48.110.2.2 conj_transpose() . . . 843
48.110.2.3 get_krylov_dim() . . . 843
48.110.2.4 parse() . . . 843
48.110.2.5 set_krylov_dim() . . . 844
48.110.2.6 transpose() . . . 844
48.111 gko::solver::Gmres< ValueType > Class Template Reference . . . 844
48.111.1 Detailed Description . . . 845
48.111.2 Member Function Documentation . . . 846
48.111.2.1 apply_uses_initial_guess() . . . 846
48.111.2.2 conj_transpose() . . . 846
48.111.2.3 get_krylov_dim() . . . 846
48.111.2.4 get_restart_ratio() . . . 847
48.111.2.5 parse() . . . 847
48.111.2.6 set_krylov_dim() . . . 847
48.111.2.7 set_restart_ratio() . . . 848
48.111.2.8 transpose() . . . 848
48.112 gko::half Class Reference . . . 848
48.112.1 Detailed Description . . . 849
48.113 gko::solver::has_with_criteria< SolverType, typename > Struct Template Reference . . . 849
48.113.1 Detailed Description . . . 849
48.114 gko::hip_stream Class Reference . . . 849
48.114.1 Detailed Description . . . 850
48.114.2 Constructor & Destructor Documentation . . . 850
48.114.2.1 hip_stream() . . . 850
48.114.3 Member Function Documentation . . . 850
48.114.3.1 get() . . . 850
48.115 gko::HipAllocatorBase Class Reference . . . 851
48.115.1 Detailed Description . . . 851
48.116 gko::HipblasError Class Reference . . . 851
48.116.1 Detailed Description . . . 851
48.116.2 Constructor & Destructor Documentation . . . 851
48.116.2.1 HipblasError() . . . 851
48.117 gko::HipError Class Reference . . . 852
48.117.1 Detailed Description . . . 852

48.117.2 Constructor & Destructor Documentation . . . 852
48.117.2.1 HipError() . . . 852
48.118 gko::HipExecutor Class Reference . . . 853
48.118.1 Detailed Description . . . 854
48.118.2 Member Function Documentation . . . 854
48.118.2.1 create() . . . 854
48.118.2.2 get_blas_handle() . . . 854
48.118.2.3 get_description() . . . 855
48.118.2.4 get_hipblas_handle() . . . 855
48.118.2.5 get_hipsparse_handle() . . . 855
48.118.2.6 get_master() [1/2] . . . 855
48.118.2.7 get_master() [2/2] . . . 856
48.118.2.8 get_sparselib_handle() . . . 856
48.118.2.9 run() [1/3] . . . 856
48.118.2.10 run() [2/3] . . . 857
48.118.2.11 run() [3/3] . . . 857
48.119 gko::HipfftError Class Reference . . . 858
48.119.1 Detailed Description . . . 858
48.119.2 Constructor & Destructor Documentation . . . 858
48.119.2.1 HipfftError() . . . 858
48.120 gko::HiprandError Class Reference . . . 859
48.120.1 Detailed Description . . . 859
48.120.2 Constructor & Destructor Documentation . . . 859
48.120.2.1 HiprandError() . . . 859
48.121 gko::HipsparseError Class Reference . . . 860
48.121.1 Detailed Description . . . 860
48.121.2 Constructor & Destructor Documentation . . . 860
48.121.2.1 HipsparseError() . . . 860
48.122 gko::HipTimer Class Reference . . . 861
48.122.1 Detailed Description . . . 861
48.122.2 Member Function Documentation . . . 861
48.122.2.1 difference_async() . . . 861
48.123 gko::matrix::view::hybrid< ValueType, IndexType > Struct Template Reference . . . 862
48.123.1 Detailed Description . . . 862
48.124 gko::matrix::Hybrid< ValueType, IndexType > Class Template Reference . . . 863
48.124.1 Detailed Description . . . 865
48.124.2 Constructor & Destructor Documentation . . . 865
48.124.2.1 Hybrid() [1/2] . . . 865
48.124.2.2 Hybrid() [2/2] . . . 866
48.124.3 Member Function Documentation . . . 866
48.124.3.1 compute_absolute() . . . 866
48.124.3.2 create() [1/5] . . . 866

48.124.3.3 create() [2/5] . . . 867
48.124.3.4 create() [3/5] . . . 867
48.124.3.5 create() [4/5] . . . 868
48.124.3.6 create() [5/5] . . . 868
48.124.3.7 ell_col_at() [1/2] . . . 869
48.124.3.8 ell_col_at() [2/2] . . . 869
48.124.3.9 ell_val_at() [1/2] . . . 870
48.124.3.10 ell_val_at() [2/2] . . . 870
48.124.3.11 extract_diagonal() . . . 870
48.124.3.12 get_const_coo_col_idxs() . . . 872
48.124.3.13 get_const_coo_row_idxs() . . . 872
48.124.3.14 get_const_coo_values() . . . 873
48.124.3.15 get_const_device_view() . . . 873
48.124.3.16 get_const_ell_col_idxs() . . . 873
48.124.3.17 get_const_ell_values() . . . 874
48.124.3.18 get_coo() . . . 874
48.124.3.19 get_coo_col_idxs() . . . 874
48.124.3.20 get_coo_num_stored_elements() . . . 875
48.124.3.21 get_coo_row_idxs() . . . 875
48.124.3.22 get_coo_values() . . . 875
48.124.3.23 get_device_view() . . . 875
48.124.3.24 get_ell() . . . 876
48.124.3.25 get_ell_col_idxs() . . . 876
48.124.3.26 get_ell_num_stored_elements() . . . 876
48.124.3.27 get_ell_num_stored_elements_per_row() . . . 877
48.124.3.28 get_ell_stride() . . . 877
48.124.3.29 get_ell_values() . . . 877
48.124.3.30 get_num_stored_elements() . . . 878
48.124.3.31 get_strategy() [1/2] . . . 878
48.124.3.32 get_strategy() [2/2] . . . 878
48.124.3.33 operator=() [1/2] . . . 879
48.124.3.34 operator=() [2/2] . . . 879
48.124.3.35 read() [1/3] . . . 879
48.124.3.36 read() [2/3] . . . 879
48.124.3.37 read() [3/3] . . . 880
48.124.3.38 write() . . . 880
48.125 gko::factorization::Ic< ValueType, IndexType > Class Template Reference . . . 880
48.125.1 Detailed Description . . . 881
48.125.2 Member Function Documentation . . . 881
48.125.2.1 parse() . . . 881
48.126 gko::preconditioner::Ic< ValueType, IndexType > Class Template Reference . . . 882
48.126.1 Detailed Description . . . 883

48.126.2 Constructor & Destructor Documentation . . . 883
48.126.2.1 lc() [1/2] . . . 883
48.126.2.2 lc() [2/2] . . . 884
48.126.3 Member Function Documentation . . . 884
48.126.3.1 conj_transpose() . . . 884
48.126.3.2 get_l_solver() . . . 884
48.126.3.3 get_lh_solver() . . . 885
48.126.3.4 operator=() [1/2] . . . 885
48.126.3.5 operator=() [2/2] . . . 885
48.126.3.6 parse() . . . 885
48.126.3.7 transpose() . . . 886
48.127 gko::batch::matrix::Identity< ValueType > Class Template Reference . . . 886
48.127.1 Detailed Description . . . 887
48.127.2 Member Function Documentation . . . 887
48.127.2.1 apply() [1/4] . . . 887
48.127.2.2 apply() [2/4] . . . 888
48.127.2.3 apply() [3/4] . . . 888
48.127.2.4 apply() [4/4] . . . 888
48.127.2.5 create() . . . 889
48.128 gko::matrix::Identity< ValueType > Class Template Reference . . . 889
48.128.1 Detailed Description . . . 890
48.128.2 Member Function Documentation . . . 890
48.128.2.1 conj_transpose() . . . 890
48.128.2.2 create() [1/2] . . . 890
48.128.2.3 create() [2/2] . . . 891
48.128.2.4 transpose() . . . 891
48.129 gko::matrix::IdentityFactory< ValueType > Class Template Reference . . . 891
48.129.1 Detailed Description . . . 892
48.129.2 Member Function Documentation . . . 892
48.129.2.1 create() . . . 892
48.130 gko::solver::Idr< ValueType > Class Template Reference . . . 893
48.130.1 Detailed Description . . . 894
48.130.2 Member Function Documentation . . . 894
48.130.2.1 apply_uses_initial_guess() . . . 894
48.130.2.2 conj_transpose() . . . 895
48.130.2.3 get_complex_subspace() . . . 895
48.130.2.4 get_deterministic() . . . 895
48.130.2.5 get_kappa() . . . 895
48.130.2.6 get_subspace_dim() . . . 896
48.130.2.7 parse() . . . 896
48.130.2.8 set_complex_subpsace() . . . 896
48.130.2.9 set_complex_subspace() . . . 897

48.130.2.10 set_deterministic() . . . 897
48.130.2.11 set_kappa() . . . 897
48.130.2.12 set_subspace_dim() . . . 898
48.130.2.13 transpose() . . . 898
48.131 gko::factorization::Ilu< ValueType, IndexType > Class Template Reference . . . 898
48.131.1 Detailed Description . . . 899
48.131.2 Member Function Documentation . . . 899
48.131.2.1 parse() . . . 899
48.132 gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType > Class Template Reference . . . 900
48.132.1 Detailed Description . . . 901
48.132.2 Constructor & Destructor Documentation . . . 901
48.132.2.1 Ilu() [1/2] . . . 901
48.132.2.2 Ilu() [2/2] . . . 902
48.132.3 Member Function Documentation . . . 902
48.132.3.1 conj_transpose() . . . 902
48.132.3.2 get_l_solver() . . . 902
48.132.3.3 get_u_solver() . . . 903
48.132.3.4 operator=() [1/2] . . . 903
48.132.3.5 operator=() [2/2] . . . 903
48.132.3.6 parse() . . . 903
48.132.3.7 transpose() . . . 904
48.133 gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit Class Reference . . . 904
48.133.1 Detailed Description . . . 905
48.133.2 Member Function Documentation . . . 905
48.133.2.1 compute_ell_num_stored_elements_per_row() . . . 905
48.133.2.2 get_percentage() . . . 906
48.133.2.3 get_ratio() . . . 906
48.134 gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit Class Reference . . . 906
48.134.1 Detailed Description . . . 907
48.134.2 Constructor & Destructor Documentation . . . 907
48.134.2.1 imbalance_limit() . . . 907
48.134.3 Member Function Documentation . . . 907
48.134.3.1 compute_ell_num_stored_elements_per_row() . . . 907
48.134.3.2 get_percentage() . . . 908
48.135 gko::stop::ImplicitResidualNorm< ValueType > Class Template Reference . . . 909
48.135.1 Detailed Description . . . 909
48.136 gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType > Class Template Reference . . . 909
48.136.1 Detailed Description . . . 910
48.136.2 Constructor & Destructor Documentation . . . 911
48.136.2.1 index_map() . . . 911
48.136.3 Member Function Documentation . . . 911

48.136.3.1 get_remote_global_idxs() . . . 911
48.136.3.2 get_remote_local_idxs() . . . 912
48.136.3.3 get_remote_target_ids() . . . 912
48.136.3.4 map_to_global() . . . 912
48.136.3.5 map_to_local() . . . 913
48.137 gko::index_set< IndexType > Class Template Reference . . . 913
48.137.1 Detailed Description . . . 915
48.137.2 Constructor & Destructor Documentation . . . 915
48.137.2.1 index_set() [1/7] . . . 915
48.137.2.2 index_set() [2/7] . . . 916
48.137.2.3 index_set() [3/7] . . . 916
48.137.2.4 index_set() [4/7] . . . 917
48.137.2.5 index_set() [5/7] . . . 917
48.137.2.6 index_set() [6/7] . . . 917
48.137.2.7 index_set() [7/7] . . . 918
48.137.3 Member Function Documentation . . . 918
48.137.3.1 clear() . . . 918
48.137.3.2 contains() [1/2] . . . 918
48.137.3.3 contains() [2/2] . . . 919
48.137.3.4 get_executor() . . . 919
48.137.3.5 get_global_index() . . . 920
48.137.3.6 get_local_index() . . . 920
48.137.3.7 get_num_elems() . . . 921
48.137.3.8 get_num_subsets() . . . 921
48.137.3.9 get_size() . . . 921
48.137.3.10 get_subsets_begin() . . . 922
48.137.3.11 get_subsets_end() . . . 922
48.137.3.12 get_superset_indices() . . . 922
48.137.3.13 is_contiguous() . . . 923
48.137.3.14 map_global_to_local() . . . 923
48.137.3.15 map_local_to_global() . . . 923
48.137.3.16 operator=() [1/2] . . . 924
48.137.3.17 operator=() [2/2] . . . 924
48.137.3.18 to_global_indices() . . . 925
48.138 gko::InvalidData Class Reference . . . 925
48.138.1 Detailed Description . . . 926
48.138.2 Constructor & Destructor Documentation . . . 926
48.138.2.1 InvalidData() . . . 926
48.139 gko::InvalidStateError Class Reference . . . 926
48.139.1 Detailed Description . . . 927
48.139.2 Constructor & Destructor Documentation . . . 927
48.139.2.1 InvalidStateError() . . . 927

48.140 gko::solver::Ir< ValueType > Class Template Reference . . . 927
48.140.1 Detailed Description . . . 928
48.140.2 Constructor & Destructor Documentation . . . 929
48.140.2.1 Ir() [1/2] . . . 929
48.140.2.2 Ir() [2/2] . . . 929
48.140.3 Member Function Documentation . . . 929
48.140.3.1 apply_uses_initial_guess() . . . 929
48.140.3.2 conj_transpose() . . . 930
48.140.3.3 get_solver() . . . 930
48.140.3.4 operator=() [1/2] . . . 930
48.140.3.5 operator=() [2/2] . . . 930
48.140.3.6 parse() . . . 931
48.140.3.7 set_solver() . . . 931
48.140.3.8 transpose() . . . 931
48.141 gko::preconditioner::Isai< IsaiType, ValueType, IndexType > Class Template Reference . . . 932
48.141.1 Detailed Description . . . 933
48.141.2 Constructor & Destructor Documentation . . . 933
48.141.2.1 Isai() [1/2] . . . 934
48.141.2.2 Isai() [2/2] . . . 934
48.141.3 Member Function Documentation . . . 934
48.141.3.1 conj_transpose() . . . 934
48.141.3.2 get_approximate_inverse() . . . 934
48.141.3.3 operator=() [1/2] . . . 935
48.141.3.4 operator=() [2/2] . . . 935
48.141.3.5 parse() . . . 935
48.141.3.6 transpose() . . . 935
48.142 gko::stop::Iteration Class Reference . . . 936
48.142.1 Detailed Description . . . 936
48.143 gko::log::iteration_complete_data Struct Reference . . . 936
48.143.1 Detailed Description . . . 936
48.144 gko::solver::IterativeBase Class Reference . . . 937
48.144.1 Detailed Description . . . 937
48.144.2 Member Function Documentation . . . 937
48.144.2.1 get_stop_criterion_factory() . . . 937
48.144.2.2 set_stop_criterion_factory() . . . 937
48.145 gko::batch::preconditioner::Jacobi< ValueType, IndexType > Class Template Reference . . . 938
48.145.1 Detailed Description . . . 939
48.145.2 Member Function Documentation . . . 939
48.145.2.1 get_const_block_pointers() . . . 939
48.145.2.2 get_const_blocks() . . . 940
48.145.2.3 get_const_blocks_cumulative_offsets() . . . 940
48.145.2.4 get_const_map_block_to_row() . . . 941

48.145.2.5 get_max_block_size() . . . 941
48.145.2.6 get_num_blocks() . . . 941
48.145.2.7 get_num_stored_elements() . . . 942
48.146 gko::preconditioner::Jacobi< ValueType, IndexType > Class Template Reference . . . 942
48.146.1 Detailed Description . . . 943
48.146.2 Constructor & Destructor Documentation . . . 944
48.146.2.1 Jacobi() [1/2] . . . 944
48.146.2.2 Jacobi() [2/2] . . . 944
48.146.3 Member Function Documentation . . . 944
48.146.3.1 conj_transpose() . . . 944
48.146.3.2 convert_to() . . . 944
48.146.3.3 get_blocks() . . . 945
48.146.3.4 get_conditioning() . . . 945
48.146.3.5 get_num_blocks() . . . 946
48.146.3.6 get_num_stored_elements() . . . 946
48.146.3.7 get_storage_scheme() . . . 946
48.146.3.8 move_to() . . . 946
48.146.3.9 operator=() [1/2] . . . 947
48.146.3.10 operator=() [2/2] . . . 947
48.146.3.11 parse() . . . 947
48.146.3.12 transpose() . . . 948
48.146.3.13 write() . . . 948
48.147 gko::KernelNotFound Class Reference . . . 949
48.147.1 Detailed Description . . . 949
48.147.2 Constructor & Destructor Documentation . . . 949
48.147.2.1 KernelNotFound() . . . 949
48.148 gko::log::linop_data Struct Reference . . . 949
48.148.1 Detailed Description . . . 950
48.149 gko::log::linop_factory_data Struct Reference . . . 950
48.149.1 Detailed Description . . . 950
48.150 gko::LinOpFactory Class Reference . . . 950
48.150.1 Detailed Description . . . 951
48.150.1.1 Example: using CG in Ginkgo . . . 951
48.150.2 Member Function Documentation . . . 951
48.150.2.1 check_reuse_consistent() . . . 951
48.150.2.2 create_empty_reuse_data() . . . 952
48.150.2.3 generate_reuse() . . . 952
48.151 gko::local_span Struct Reference . . . 953
48.151.1 Detailed Description . . . 953
48.151.2 Member Function Documentation . . . 953
48.151.2.1 span() [1/2] . . . 953
48.151.2.2 span() [2/2] . . . 954

48.152 gko::log::Loggable Class Reference . . . 954
48.152.1 Detailed Description . . . 954
48.152.2 Member Function Documentation . . . 955
48.152.2.1 add_logger() . . . 955
48.152.2.2 get_loggers() . . . 955
48.152.2.3 remove_logger() . . . 955
48.153 gko::log::Record::logged_data Struct Reference . . . 956
48.153.1 Detailed Description . . . 956
48.154 gko::solver::LowerTrs< ValueType, IndexType > Class Template Reference . . . 956
48.154.1 Detailed Description . . . 957
48.154.2 Constructor & Destructor Documentation . . . 957
48.154.2.1 LowerTrs() [1/2] . . . 957
48.154.2.2 LowerTrs() [2/2] . . . 957
48.154.3 Member Function Documentation . . . 957
48.154.3.1 conj_transpose() . . . 958
48.154.3.2 operator=() [1/2] . . . 958
48.154.3.3 operator=() [2/2] . . . 958
48.154.3.4 parse() . . . 958
48.154.3.5 transpose() . . . 959
48.155 gko::experimental::factorization::Lu< ValueType, IndexType > Class Template Reference . . . 959
48.155.1 Detailed Description . . . 960
48.155.2 Member Function Documentation . . . 960
48.155.2.1 generate() . . . 960
48.155.2.2 get_parameters() [1/2] . . . 961
48.155.2.3 get_parameters() [2/2] . . . 961
48.155.2.4 parse() . . . 961
48.156 gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType > Class Template Reference . . . 962
48.156.1 Detailed Description . . . 964
48.156.2 Constructor & Destructor Documentation . . . 965
48.156.2.1 Matrix() [1/2] . . . 965
48.156.2.2 Matrix() [2/2] . . . 965
48.156.3 Member Function Documentation . . . 966
48.156.3.1 col_scale() . . . 966
48.156.3.2 create() [1/9] . . . 966
48.156.3.3 create() [2/9] . . . 966
48.156.3.4 create() [3/9] . . . 968
48.156.3.5 create() [4/9] . . . 968
48.156.3.6 create() [5/9] . . . 969
48.156.3.7 create() [6/9] . . . 969
48.156.3.8 create() [7/9] . . . 970
48.156.3.9 create() [8/9] . . . 971

48.156.3.10 create() [9/9] . . . 971
48.156.3.11 get_diag_matrix() . . . 972
48.156.3.12 get_local_matrix() . . . 972
48.156.3.13 get_non_local_matrix() . . . 973
48.156.3.14 get_off_diag_matrix() . . . 973
48.156.3.15 get_row_gatherer() . . . 973
48.156.3.16 get_row_partition() . . . 974
48.156.3.17 multiply() . . . 974
48.156.3.18 operator=() [1/2] . . . 974
48.156.3.19 operator=() [2/2] . . . 975
48.156.3.20 read_distributed() [1/4] . . . 975
48.156.3.21 read_distributed() [2/4] . . . 976
48.156.3.22 read_distributed() [3/4] . . . 977
48.156.3.23 read_distributed() [4/4] . . . 977
48.156.3.24 row_scale() . . . 977
48.156.3.25 transpose() . . . 978
48.156.3.26 write() . . . 978
48.157 gko::matrix_assembly_data< ValueType, IndexType > Class Template Reference . . . 979
48.157.1 Detailed Description . . . 979
48.157.2 Member Function Documentation . . . 979
48.157.2.1 add_value() . . . 980
48.157.2.2 contains() . . . 980
48.157.2.3 get_num_stored_elements() . . . 980
48.157.2.4 get_ordered_data() . . . 981
48.157.2.5 get_size() . . . 981
48.157.2.6 get_value() . . . 981
48.157.2.7 set_value() . . . 982
48.158 gko::matrix_data< ValueType, IndexType > Struct Template Reference . . . 982
48.158.1 Detailed Description . . . 984
48.158.2 Constructor & Destructor Documentation . . . 984
48.158.2.1 matrix_data() [1/6] . . . 984
48.158.2.2 matrix_data() [2/6] . . . 985
48.158.2.3 matrix_data() [3/6] . . . 985
48.158.2.4 matrix_data() [4/6] . . . 986
48.158.2.5 matrix_data() [5/6] . . . 986
48.158.2.6 matrix_data() [6/6] . . . 987
48.158.3 Member Function Documentation . . . 987
48.158.3.1 cond() [1/2] . . . 987
48.158.3.2 cond() [2/2] . . . 988
48.158.3.3 diag() [1/5] . . . 989
48.158.3.4 diag() [2/5] . . . 990
48.158.3.5 diag() [3/5] . . . 990

48.158.3.6 diag() [4/5] . . . 991
48.158.3.7 diag() [5/5] . . . 992
48.158.3.8 sum_duplicates() . . . 992
48.158.4 Member Data Documentation . . . 993
48.158.4.1 nonzeros . . . 993
48.159 gko::matrix_data_entry< ValueType, IndexType > Struct Template Reference . . . 993
48.159.1 Detailed Description . . . 993
48.160 gko::experimental::reorder::Mc64< ValueType, IndexType > Class Template Reference . . . 993
48.160.1 Detailed Description . . . 994
48.160.2 Member Function Documentation . . . 994
48.160.2.1 generate() . . . 995
48.160.2.2 get_parameters() . . . 995
48.161 gko::MetisError Class Reference . . . 995
48.161.1 Detailed Description . . . 996
48.161.2 Constructor & Destructor Documentation . . . 996
48.161.2.1 MetisError() . . . 996
48.162 gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit Class Reference . . . 996
48.162.1 Detailed Description . . . 997
48.162.2 Member Function Documentation . . . 997
48.162.2.1 compute_ell_num_stored_elements_per_row() . . . 997
48.162.2.2 get_percentage() . . . 997
48.163 gko::solver::Minres< ValueType > Class Template Reference . . . 998
48.163.1 Detailed Description . . . 998
48.163.2 Member Function Documentation . . . 999
48.163.2.1 conj_transpose() . . . 999
48.163.2.2 parse() . . . 999
48.163.2.3 transpose() . . . 1000
48.164 gko::MpiError Class Reference . . . 1000
48.164.1 Detailed Description . . . 1000
48.164.2 Constructor & Destructor Documentation . . . 1001
48.164.2.1 MpiError() . . . 1001
48.165 gko::solver::Multigrid Class Reference . . . 1002
48.165.1 Detailed Description . . . 1003
48.165.2 Member Function Documentation . . . 1003
48.165.2.1 apply_uses_initial_guess() . . . 1003
48.165.2.2 get_coarsest_solver() . . . 1004
48.165.2.3 get_cycle() . . . 1004
48.165.2.4 get_mg_level_list() . . . 1004
48.165.2.5 get_mid_smoother_list() . . . 1004
48.165.2.6 get_post_smoother_list() . . . 1005
48.165.2.7 get_pre_smoother_list() . . . 1005
48.165.2.8 parse() . . . 1005

48.165.2.9 set_cycle() . . . 1006
48.166 gko::multigrid::MultigridLevel Class Reference . . . 1006
48.166.1 Detailed Description . . . 1006
48.166.2 Member Function Documentation . . . 1007
48.166.2.1 get_coarse_op() . . . 1007
48.166.2.2 get_fine_op() . . . 1007
48.166.2.3 get_prolong_op() . . . 1007
48.166.2.4 get_restrict_op() . . . 1008
48.167 gko::matrix::Csr< ValueType, IndexType >::multiply_add_reuse_info Class Reference . . . 1008
48.167.1 Detailed Description . . . 1008
48.168 gko::matrix::Csr< ValueType, IndexType >::multiply_reuse_info Class Reference . . . 1009
48.168.1 Detailed Description . . . 1009
48.169 gko::batch::MultiVector< ValueType > Class Template Reference . . . 1009
48.169.1 Detailed Description . . . 1011
48.169.2 Member Function Documentation . . . 1011
48.169.2.1 add_scaled() . . . 1011
48.169.2.2 at() [1/4] . . . 1012
48.169.2.3 at() [2/4] . . . 1012
48.169.2.4 at() [3/4] . . . 1013
48.169.2.5 at() [4/4] . . . 1013
48.169.2.6 compute_conj_dot() . . . 1014
48.169.2.7 compute_dot() . . . 1014
48.169.2.8 compute_norm2() . . . 1014
48.169.2.9 create() [1/3] . . . 1015
48.169.2.10 create() [2/3] . . . 1015
48.169.2.11 create() [3/3] . . . 1016
48.169.2.12 create_const() . . . 1016
48.169.2.13 create_const_view_for_item() . . . 1016
48.169.2.14 create_view_for_item() . . . 1017
48.169.2.15 create_with_config_of() . . . 1017
48.169.2.16 fill() . . . 1018
48.169.2.17 get_common_size() . . . 1018
48.169.2.18 get_const_values() . . . 1018
48.169.2.19 get_const_values_for_item() . . . 1019
48.169.2.20 get_cumulative_offset() . . . 1019
48.169.2.21 get_num_batch_items() . . . 1020
48.169.2.22 get_num_stored_elements() . . . 1020
48.169.2.23 get_size() . . . 1020
48.169.2.24 get_values() . . . 1021
48.169.2.25 get_values_for_item() . . . 1021
48.169.2.26 scale() . . . 1021
48.170 gko::experimental::mpi::NeighborhoodCommunicator Class Reference . . . 1022

48.170.1 Detailed Description . . . 1023
48.170.2 Constructor & Destructor Documentation . . . 1023
48.170.2.1 NeighborhoodCommunicator() [1/2] . . . 1023
48.170.2.2 NeighborhoodCommunicator() [2/2] . . . 1023
48.170.3 Member Function Documentation . . . 1024
48.170.3.1 create_inverse() . . . 1024
48.170.3.2 create_with_same_type() . . . 1024
48.170.3.3 get_recv_size() . . . 1025
48.170.3.4 get_send_size() . . . 1025
48.170.3.5 i_all_to_all_v() [1/2] . . . 1025
48.170.3.6 i_all_to_all_v() [2/2] . . . 1026
48.170.3.7 resize() . . . 1026
48.170.4 Friends And Related Function Documentation . . . 1027
48.170.4.1 operator"!= . . . 1027
48.170.4.2 operator== . . . 1027
48.171 gko::log::ProfilerHook::NestedSummaryWriter Class Reference . . . 1027
48.171.1 Detailed Description . . . 1028
48.171.2 Member Function Documentation . . . 1028
48.171.2.1 write_nested() . . . 1028
48.172 gko::NotCompiled Class Reference . . . 1028
48.172.1 Detailed Description . . . 1029
48.172.2 Constructor & Destructor Documentation . . . 1029
48.172.2.1 NotCompiled() . . . 1029
48.173 gko::NotImplemented Class Reference . . . 1029
48.173.1 Detailed Description . . . 1030
48.173.2 Constructor & Destructor Documentation . . . 1030
48.173.2.1 NotImplemented() . . . 1030
48.174 gko::NotSupported Class Reference . . . 1030
48.174.1 Detailed Description . . . 1030
48.174.2 Constructor & Destructor Documentation . . . 1031
48.174.2.1 NotSupported() . . . 1031
48.175 gko::null_deleter< T > Class Template Reference . . . 1031
48.175.1 Detailed Description . . . 1031
48.175.2 Member Function Documentation . . . 1032
48.175.2.1 operator()() . . . 1032
48.176 gko::solver::Nullspace< ValueType > Class Template Reference . . . 1032
48.176.1 Detailed Description . . . 1033
48.176.2 Member Function Documentation . . . 1033
48.176.2.1 contains_constant() . . . 1033
48.176.2.2 create() . . . 1033
48.176.2.3 create_from_constant() . . . 1034
48.176.2.4 get_basis() . . . 1034

48.176.2.5 get_dimension() . . . 1035
48.176.2.6 project() . . . 1035
48.177 gko::solver::nullspace_traits< Solver > Struct Template Reference . . . 1035
48.177.1 Detailed Description . . . 1036
48.177.2 Member Data Documentation . . . 1036
48.177.2.1 is_supported . . . 1036
48.177.2.2 requires_hermitian . . . 1036
48.177.2.3 two_sided_projection . . . 1036
48.178 gko::preconditioner::NullspaceProjection< ValueType > Class Template Reference . . . 1037
48.178.1 Detailed Description . . . 1037
48.178.2 Member Function Documentation . . . 1038
48.178.2.1 conj_transpose() . . . 1038
48.178.2.2 get_left_nullspace() . . . 1038
48.178.2.3 get_nullspace() . . . 1038
48.178.2.4 get_preconditioner() . . . 1039
48.178.2.5 parse() . . . 1039
48.178.2.6 transpose() . . . 1039
48.179 gko::nvidia_device Class Reference . . . 1040
48.179.1 Detailed Description . . . 1040
48.180 gko::OmpExecutor Class Reference . . . 1040
48.180.1 Detailed Description . . . 1041
48.180.2 Member Function Documentation . . . 1041
48.180.2.1 get_description() . . . 1041
48.180.2.2 get_master() [1/2] . . . 1041
48.180.2.3 get_master() [2/2] . . . 1041
48.180.2.4 run() [1/3] . . . 1041
48.180.2.5 run() [2/3] . . . 1042
48.180.2.6 run() [3/3] . . . 1042
48.181 gko::Operation Class Reference . . . 1043
48.181.1 Detailed Description . . . 1043
48.181.2 Member Function Documentation . . . 1044
48.181.2.1 get_name() . . . 1045
48.182 gko::log::operation_data Struct Reference . . . 1045
48.182.1 Detailed Description . . . 1045
48.183 gko::OutOfBoundsError Class Reference . . . 1045
48.183.1 Detailed Description . . . 1045
48.183.2 Constructor & Destructor Documentation . . . 1045
48.183.2.1 OutOfBoundsError() . . . 1045
48.184 gko::OverflowError Class Reference . . . 1046
48.184.1 Detailed Description . . . 1046
48.184.2 Constructor & Destructor Documentation . . . 1046
48.184.2.1 OverflowError() . . . 1046

48.185 gko::log::Papi< ValueType > Class Template Reference . . . 1047
48.185.1 Detailed Description . . . 1047
48.185.2 Member Function Documentation . . . 1048
48.185.2.1 create() [1/2] . . . 1048
48.185.2.2 create() [2/2] . . . 1048
48.185.2.3 get_handle() . . . 1049
48.185.2.4 get_handle_name() . . . 1049
48.186 gko::factorization::ParIc< ValueType, IndexType > Class Template Reference . . . 1049
48.186.1 Detailed Description . . . 1050
48.186.2 Member Function Documentation . . . 1050
48.186.2.1 parse() . . . 1050
48.187 gko::factorization::ParIct< ValueType, IndexType > Class Template Reference . . . 1052
48.187.1 Detailed Description . . . 1052
48.187.2 Member Function Documentation . . . 1053
48.187.2.1 parse() . . . 1053
48.188 gko::factorization::ParIlu< ValueType, IndexType > Class Template Reference . . . 1054
48.188.1 Detailed Description . . . 1054
48.188.2 Member Function Documentation . . . 1055
48.188.2.1 parse() . . . 1055
48.189 gko::factorization::ParIlut< ValueType, IndexType > Class Template Reference . . . 1055
48.189.1 Detailed Description . . . 1056
48.189.2 Member Function Documentation . . . 1057
48.189.2.1 parse() . . . 1057
48.190 gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType > Class Template Reference . . . 1057
48.190.1 Detailed Description . . . 1058
48.190.2 Member Function Documentation . . . 1059
48.190.2.1 build_from_contiguous() . . . 1059
48.190.2.2 build_from_global_size_uniform() . . . 1060
48.190.2.3 build_from_mapping() . . . 1060
48.190.2.4 equals() . . . 1060
48.190.2.5 get_num_empty_parts() . . . 1061
48.190.2.6 get_num_parts() . . . 1061
48.190.2.7 get_num_ranges() . . . 1062
48.190.2.8 get_part_ids() . . . 1062
48.190.2.9 get_part_size() . . . 1062
48.190.2.10 get_part_sizes() . . . 1063
48.190.2.11 get_range_bounds() . . . 1063
48.190.2.12 get_range_starting_indices() . . . 1064
48.190.2.13 get_ranges_by_part() . . . 1064
48.190.2.14 get_size() . . . 1064
48.190.2.15 has_connected_parts() . . . 1065

48.190.2.16 has_ordered_parts() . . . 1065
48.191 gko::log::PerformanceHint Class Reference . . . 1065
48.191.1 Detailed Description . . . 1066
48.191.2 Member Function Documentation . . . 1066
48.191.2.1 create() . . . 1066
48.192 gko::Permutable< IndexType > Class Template Reference . . . 1066
48.192.1 Detailed Description . . . 1067
48.192.1.1 Example: Permuting a Csr matrix: . . . 1067
48.192.2 Member Function Documentation . . . 1067
48.192.2.1 column_permute() . . . 1067
48.192.2.2 inverse_column_permute() . . . 1068
48.192.2.3 inverse_permute() . . . 1068
48.192.2.4 inverse_row_permute() . . . 1069
48.192.2.5 permute() . . . 1069
48.192.2.6 row_permute() . . . 1070
48.193 gko::matrix::Permutation< IndexType > Class Template Reference . . . 1070
48.193.1 Detailed Description . . . 1071
48.193.2 Member Function Documentation . . . 1072
48.193.2.1 compose() . . . 1072
48.193.2.2 compute_inverse() . . . 1072
48.193.2.3 create() [1/2] . . . 1072
48.193.2.4 create() [2/2] . . . 1073
48.193.2.5 create_const() [1/2] . . . 1073
48.193.2.6 create_const() [2/2] . . . 1074
48.193.2.7 get_const_permutation() . . . 1074
48.193.2.8 get_permutation() . . . 1075
48.193.2.9 get_permutation_size() . . . 1075
48.193.2.10 write() . . . 1075
48.194 gko::matrix::Csr< ValueType, IndexType >::permuting_reuse_info Struct Reference . . . 1076
48.194.1 Detailed Description . . . 1076
48.194.2 Member Function Documentation . . . 1076
48.194.2.1 update_values() . . . 1076
48.195 gko::Perturbation< ValueType > Class Template Reference . . . 1077
48.195.1 Detailed Description . . . 1077
48.195.2 Member Function Documentation . . . 1078
48.195.2.1 create() [1/3] . . . 1078
48.195.2.2 create() [2/3] . . . 1078
48.195.2.3 create() [3/3] . . . 1079
48.195.2.4 get_basis() . . . 1079
48.195.2.5 get_projector() . . . 1079
48.195.2.6 get_scalar() . . . 1080
48.196 gko::multigrid::Pgm< ValueType, IndexType > Class Template Reference . . . 1080

48.196.1 Detailed Description . . . 1081
48.196.2 Member Function Documentation . . . 1081
48.196.2.1 get_agg() . . . 1081
48.196.2.2 get_const_agg() . . . 1082
48.196.2.3 get_system_matrix() . . . 1082
48.196.2.4 parse() . . . 1082
48.197 gko::solver::PipeCg< ValueType > Class Template Reference . . . 1083
48.197.1 Detailed Description . . . 1084
48.197.2 Member Function Documentation . . . 1085
48.197.2.1 apply_uses_initial_guess() . . . 1085
48.197.2.2 conj_transpose() . . . 1085
48.197.2.3 parse() . . . 1085
48.197.2.4 transpose() . . . 1086
48.198 gko::multigrid::Pmis< ValueType, IndexType > Class Template Reference . . . 1086
48.198.1 Detailed Description . . . 1087
48.198.2 Member Function Documentation . . . 1087
48.198.2.1 get_system_matrix() . . . 1087
48.198.2.2 parse() . . . 1087
48.199 gko::config::pnode Class Reference . . . 1088
48.199.1 Detailed Description . . . 1089
48.199.2 Constructor & Destructor Documentation . . . 1089
48.199.2.1 pnode() [1/7] . . . 1089
48.199.2.2 pnode() [2/7] . . . 1090
48.199.2.3 pnode() [3/7] . . . 1090
48.199.2.4 pnode() [4/7] . . . 1090
48.199.2.5 pnode() [5/7] . . . 1091
48.199.2.6 pnode() [6/7] . . . 1091
48.199.2.7 pnode() [7/7] . . . 1091
48.199.3 Member Function Documentation . . . 1091
48.199.3.1 get() [1/2] . . . 1091
48.199.3.2 get() [2/2] . . . 1092
48.199.3.3 get_array() . . . 1092
48.199.3.4 get_boolean() . . . 1093
48.199.3.5 get_integer() . . . 1093
48.199.3.6 get_map() . . . 1093
48.199.3.7 get_real() . . . 1093
48.199.3.8 get_string() . . . 1094
48.199.3.9 get_tag() . . . 1094
48.199.3.10 operator bool() . . . 1094
48.200 gko::log::polymorphic_object_data Struct Reference . . . 1094
48.200.1 Detailed Description . . . 1095
48.201 gko::PolymorphicObject Class Reference . . . 1095

48.201.1 Detailed Description . . . 1095
48.201.2 Member Function Documentation . . . 1095
48.201.2.1 get_executor() . . . 1096
48.202 gko::precision_reduction Class Reference . . . 1096
48.202.1 Detailed Description . . . 1097
48.202.2 Constructor & Destructor Documentation . . . 1097
48.202.2.1 precision_reduction() [1/2] . . . 1097
48.202.2.2 precision_reduction() [2/2] . . . 1097
48.202.3 Member Function Documentation . . . 1098
48.202.3.1 autodetect() . . . 1098
48.202.3.2 common() . . . 1098
48.202.3.3 get_nonpreserving() . . . 1099
48.202.3.4 get_preserving() . . . 1099
48.202.3.5 operator storage_type() . . . 1099
48.203 gko::Preconditionable Class Reference . . . 1100
48.203.1 Detailed Description . . . 1100
48.203.2 Member Function Documentation . . . 1100
48.203.2.1 get_preconditioner() . . . 1100
48.203.2.2 set_preconditioner() . . . 1100
48.204 gko::log::ProfilerHook Class Reference . . . 1101
48.204.1 Detailed Description . . . 1102
48.204.2 Member Function Documentation . . . 1102
48.204.2.1 create_nested_summary() . . . 1102
48.204.2.2 create_nvtx() . . . 1103
48.204.2.3 create_summary() . . . 1103
48.204.2.4 create_tau() . . . 1104
48.204.2.5 set_object_name() . . . 1104
48.204.2.6 user_range() . . . 1104
48.205 gko::log::profiling_scope_guard Class Reference . . . 1105
48.205.1 Detailed Description . . . 1105
48.205.2 Constructor & Destructor Documentation . . . 1105
48.205.2.1 profiling_scope_guard() . . . 1105
48.206 gko::ptr_param< T > Class Template Reference . . . 1106
48.206.1 Detailed Description . . . 1106
48.206.2 Member Function Documentation . . . 1106
48.206.2.1 get() . . . 1106
48.206.2.2 operator bool() . . . 1107
48.206.2.3 operator*() . . . 1107
48.206.2.4 operator->() . . . 1107
48.207 gko::syn::range< Start, End, Step > Struct Template Reference . . . 1107
48.207.1 Detailed Description . . . 1107
48.208 gko::range< Accessor > Class Template Reference . . . 1108

48.208.1 Detailed Description . . . 1109
48.208.1.1 Range operations . . . 1109
48.208.1.2 Compound operations . . . 1110
48.208.1.3 Caveats . . . 1110
48.208.1.4 Examples . . . 1110
48.208.2 Constructor & Destructor Documentation . . . 1110
48.208.2.1 range() . . . 1110
48.208.3 Member Function Documentation . . . 1111
48.208.3.1 get_accessor() . . . 1111
48.208.3.2 length() . . . 1111
48.208.3.3 operator()() . . . 1112
48.208.3.4 operator->() . . . 1112
48.208.3.5 operator=() [1/2] . . . 1113
48.208.3.6 operator=() [2/2] . . . 1113
48.209 gko::reorder::Rcm< ValueType, IndexType > Class Template Reference . . . 1114
48.209.1 Detailed Description . . . 1114
48.209.2 Member Function Documentation . . . 1114
48.209.2.1 get_inverse_permutation() . . . 1115
48.209.2.2 get_permutation() . . . 1115
48.210 gko::experimental::reorder::Rcm< IndexType > Class Template Reference . . . 1115
48.210.1 Detailed Description . . . 1116
48.210.2 Member Function Documentation . . . 1116
48.210.2.1 generate() . . . 1116
48.210.2.2 get_parameters() . . . 1117
48.211 gko::ReadableFromMatrixData< ValueType, IndexType > Class Template Reference . . . 1117
48.211.1 Detailed Description . . . 1117
48.211.2 Member Function Documentation . . . 1117
48.211.2.1 read() [1/4] . . . 1117
48.211.2.2 read() [2/4] . . . 1118
48.211.2.3 read() [3/4] . . . 1118
48.211.2.4 read() [4/4] . . . 1118
48.212 gko::log::Record Class Reference . . . 1119
48.212.1 Detailed Description . . . 1120
48.212.2 Member Function Documentation . . . 1120
48.212.2.1 create() [1/2] . . . 1120
48.212.2.2 create() [2/2] . . . 1120
48.212.2.3 get() [1/2] . . . 1121
48.212.2.4 get() [2/2] . . . 1121
48.213 gko::ReferenceExecutor Class Reference . . . 1121
48.213.1 Detailed Description . . . 1122
48.213.2 Member Function Documentation . . . 1122
48.213.2.1 get_description() . . . 1122

48.213.2.2 run() [1/4] . . . 1122
48.213.2.3 run() [2/4] . . . 1123
48.213.2.4 run() [3/4] . . . 1123
48.213.2.5 run() [4/4] . . . 1124
48.214 gko::config::registry Class Reference . . . 1124
48.214.1 Detailed Description . . . 1125
48.214.2 Constructor & Destructor Documentation . . . 1125
48.214.2.1 registry() [1/2] . . . 1125
48.214.2.2 registry() [2/2] . . . 1125
48.214.3 Member Function Documentation . . . 1126
48.214.3.1 emplace() . . . 1126
48.215 gko::stop::RelativeResidualNorm< ValueType > Class Template Reference . . . 1126
48.215.1 Detailed Description . . . 1127
48.216 gko::reorder::ReorderingBase< IndexType > Class Template Reference . . . 1127
48.216.1 Detailed Description . . . 1127
48.217 gko::reorder::ReorderingBaseArgs Struct Reference . . . 1128
48.217.1 Detailed Description . . . 1128
48.218 gko::experimental::mpi::request Class Reference . . . 1128
48.218.1 Detailed Description . . . 1128
48.218.2 Constructor & Destructor Documentation . . . 1128
48.218.2.1 request() . . . 1128
48.218.3 Member Function Documentation . . . 1129
48.218.3.1 get() . . . 1129
48.218.3.2 wait() . . . 1129
48.219 gko::stop::ResidualNorm< ValueType > Class Template Reference . . . 1129
48.219.1 Detailed Description . . . 1130
48.220 gko::stop::ResidualNormBase< ValueType > Class Template Reference . . . 1130
48.220.1 Detailed Description . . . 1131
48.221 gko::stop::ResidualNormReduction< ValueType > Class Template Reference . . . 1131
48.221.1 Detailed Description . . . 1131
48.222 gko::multigrid::Pgm< ValueType, IndexType >::reuse_data_type Class Reference . . . 1132
48.222.1 Detailed Description . . . 1132
48.223 gko::solver::Multigrid::reuse_data_type Class Reference . . . 1132
48.223.1 Detailed Description . . . 1132
48.224 gko::LinOpFactory::ReuseData Class Reference . . . 1132
48.224.1 Detailed Description . . . 1133
48.225 gko::accessor::row_major< ValueType, Dimensionality > Class Template Reference . . . 1133
48.225.1 Detailed Description . . . 1134
48.225.2 Member Function Documentation . . . 1134
48.225.2.1 copy_from() . . . 1134
48.225.2.2 length() . . . 1135
48.225.2.3 operator()() [1/2] . . . 1135

48.225.2.4 operator()() [2/2] . . . 1136
48.226 gko::matrix::RowGatherer< IndexType > Class Template Reference . . . 1136
48.226.1 Detailed Description . . . 1137
48.226.2 Member Function Documentation . . . 1137
48.226.2.1 create() [1/2] . . . 1137
48.226.2.2 create() [2/2] . . . 1138
48.226.2.3 create_const() . . . 1138
48.226.2.4 get_const_row_idxs() . . . 1139
48.226.2.5 get_row_idxs() . . . 1139
48.227 gko::experimental::distributed::RowGatherer< LocalIndexType > Class Template Reference . . . 1140
48.227.1 Detailed Description . . . 1140
48.227.2 Member Function Documentation . . . 1141
48.227.2.1 apply_async() [1/2] . . . 1141
48.227.2.2 apply_async() [2/2] . . . 1141
48.227.2.3 create() . . . 1142
48.228 gko::multigrid::Rs< ValueType, IndexType > Class Template Reference . . . 1143
48.228.1 Detailed Description . . . 1143
48.228.2 Member Function Documentation . . . 1143
48.228.2.1 get_system_matrix() . . . 1144
48.229 gko::matrix::Csr< ValueType, IndexType >::scale_add_reuse_info Class Reference . . . 1144
48.229.1 Detailed Description . . . 1144
48.230 gko::ScaledIdentityAddable Class Reference . . . 1145
48.230.1 Detailed Description . . . 1145
48.230.2 Member Function Documentation . . . 1145
48.230.2.1 add_scaled_identity() . . . 1145
48.231 gko::matrix::ScaledPermutation< ValueType, IndexType > Class Template Reference . . . 1145
48.231.1 Detailed Description . . . 1146
48.231.2 Member Function Documentation . . . 1147
48.231.2.1 compose() . . . 1147
48.231.2.2 compute_inverse() . . . 1147
48.231.2.3 create() [1/3] . . . 1147
48.231.2.4 create() [2/3] . . . 1148
48.231.2.5 create() [3/3] . . . 1148
48.231.2.6 create_const() . . . 1149
48.231.2.7 get_const_permutation() . . . 1149
48.231.2.8 get_const_scaling_factors() . . . 1150
48.231.2.9 get_permutation() . . . 1150
48.231.2.10 get_scaling_factors() . . . 1150
48.231.2.11 write() . . . 1150
48.232 gko::experimental::reorder::ScaledReordered< ValueType, IndexType > Class Template Reference . . . 1151
48.232.1 Detailed Description . . . 1151

48.233 gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType > Class Template Reference . . . 1152
48.233.1 Detailed Description . . . 1152
48.233.2 Member Function Documentation . . . 1153
48.233.2.1 apply_uses_initial_guess() . . . 1153
48.233.2.2 parse() . . . 1153
48.234 gko::scoped_device_id_guard Class Reference . . . 1154
48.234.1 Detailed Description . . . 1154
48.234.2 Constructor & Destructor Documentation . . . 1154
48.234.2.1 scoped_device_id_guard() [1/5] . . . 1154
48.234.2.2 scoped_device_id_guard() [2/5] . . . 1155
48.234.2.3 scoped_device_id_guard() [3/5] . . . 1155
48.234.2.4 scoped_device_id_guard() [4/5] . . . 1155
48.234.2.5 scoped_device_id_guard() [5/5] . . . 1156
48.235 gko::segmented_array< T > Struct Template Reference . . . 1156
48.235.1 Detailed Description . . . 1157
48.235.2 Constructor & Destructor Documentation . . . 1157
48.235.2.1 segmented_array() [1/3] . . . 1157
48.235.2.2 segmented_array() [2/3] . . . 1158
48.235.2.3 segmented_array() [3/3] . . . 1158
48.235.3 Member Function Documentation . . . 1158
48.235.3.1 create_from_offsets() [1/2] . . . 1158
48.235.3.2 create_from_offsets() [2/2] . . . 1159
48.235.3.3 create_from_sizes() [1/2] . . . 1159
48.235.3.4 create_from_sizes() [2/2] . . . 1159
48.235.3.5 get_const_flat_data() . . . 1160
48.235.3.6 get_executor() . . . 1160
48.235.3.7 get_flat_data() . . . 1160
48.235.3.8 get_offsets() . . . 1161
48.235.3.9 get_segment_count() . . . 1161
48.235.3.10 get_size() . . . 1161
48.236 gko::matrix::view::sellp< ValueType, IndexType > Struct Template Reference . . . 1161
48.236.1 Detailed Description . . . 1162
48.237 gko::matrix::Sellp< ValueType, IndexType > Class Template Reference . . . 1162
48.237.1 Detailed Description . . . 1164
48.237.2 Constructor & Destructor Documentation . . . 1164
48.237.2.1 Sellp() [1/2] . . . 1164
48.237.2.2 Sellp() [2/2] . . . 1164
48.237.3 Member Function Documentation . . . 1165
48.237.3.1 col_at() [1/2] . . . 1165
48.237.3.2 col_at() [2/2] . . . 1165
48.237.3.3 compute_absolute() . . . 1166

48.237.3.4 create() [1/2] . . . 1166
48.237.3.5 create() [2/2] . . . 1167
48.237.3.6 extract_diagonal() . . . 1167
48.237.3.7 get_col_idxs() . . . 1167
48.237.3.8 get_const_col_idxs() . . . 1168
48.237.3.9 get_const_slice_lengths() . . . 1168
48.237.3.10 get_const_slice_sets() . . . 1169
48.237.3.11 get_const_values() . . . 1169
48.237.3.12 get_num_stored_elements() . . . 1170
48.237.3.13 get_slice_lengths() . . . 1170
48.237.3.14 get_slice_sets() . . . 1170
48.237.3.15 get_slice_size() . . . 1171
48.237.3.16 get_stride_factor() . . . 1171
48.237.3.17 get_total_cols() . . . 1171
48.237.3.18 get_values() . . . 1172
48.237.3.19 operator=() [1/2] . . . 1172
48.237.3.20 operator=() [2/2] . . . 1172
48.237.3.21 read() [1/3] . . . 1172
48.237.3.22 read() [2/3] . . . 1173
48.237.3.23 read() [3/3] . . . 1173
48.237.3.24 val_at() [1/2] . . . 1173
48.237.3.25 val_at() [2/2] . . . 1174
48.237.3.26 write() . . . 1174
48.238 gko::log::SolverProgress Class Reference . . . 1175
48.238.1 Detailed Description . . . 1175
48.238.2 Member Function Documentation . . . 1175
48.238.2.1 create_scalar_csv_writer() . . . 1175
48.238.2.2 create_scalar_table_writer() . . . 1176
48.238.2.3 create_vector_storage() . . . 1176
48.239 gko::preconditioner::Sor< ValueType, IndexType > Class Template Reference . . . 1177
48.239.1 Detailed Description . . . 1177
48.239.2 Member Function Documentation . . . 1178
48.239.2.1 generate() . . . 1178
48.239.2.2 get_parameters() [1/2] . . . 1178
48.239.2.3 get_parameters() [2/2] . . . 1178
48.240 gko::span Struct Reference . . . 1179
48.240.1 Detailed Description . . . 1179
48.240.2 Constructor & Destructor Documentation . . . 1179
48.240.2.1 span() [1/2] . . . 1179
48.240.2.2 span() [2/2] . . . 1180
48.240.3 Member Function Documentation . . . 1180
48.240.3.1 is_valid() . . . 1180

48.240.3.2 length() . . . 1181
48.241 gko::matrix::SparsityCsr< ValueType, IndexType > Class Template Reference . . . 1181
48.241.1 Detailed Description . . . 1182
48.241.2 Constructor & Destructor Documentation . . . 1183
48.241.2.1 SparsityCsr() [1/2] . . . 1183
48.241.2.2 SparsityCsr() [2/2] . . . 1183
48.241.3 Member Function Documentation . . . 1183
48.241.3.1 conj_transpose() . . . 1183
48.241.3.2 create() [1/4] . . . 1184
48.241.3.3 create() [2/4] . . . 1184
48.241.3.4 create() [3/4] . . . 1185
48.241.3.5 create() [4/4] . . . 1185
48.241.3.6 create_const() . . . 1185
48.241.3.7 get_col_idxs() . . . 1186
48.241.3.8 get_const_col_idxs() . . . 1186
48.241.3.9 get_const_row_ptrs() . . . 1187
48.241.3.10 get_const_value() . . . 1187
48.241.3.11 get_num_nonzeros() . . . 1188
48.241.3.12 get_row_ptrs() . . . 1188
48.241.3.13 get_value() . . . 1188
48.241.3.14 operator=() [1/2] . . . 1189
48.241.3.15 operator=() [2/2] . . . 1189
48.241.3.16 read() [1/3] . . . 1189
48.241.3.17 read() [2/3] . . . 1189
48.241.3.18 read() [3/3] . . . 1190
48.241.3.19 to_adjacency_matrix() . . . 1190
48.241.3.20 transpose() . . . 1190
48.241.3.21 write() . . . 1191
48.242 gko::experimental::mpi::status Struct Reference . . . 1191
48.242.1 Detailed Description . . . 1191
48.242.2 Constructor & Destructor Documentation . . . 1191
48.242.2.1 status() . . . 1192
48.242.3 Member Function Documentation . . . 1192
48.242.3.1 get() . . . 1192
48.242.3.2 get_count() . . . 1192
48.243 gko::stopping_status Class Reference . . . 1193
48.243.1 Detailed Description . . . 1193
48.243.2 Member Function Documentation . . . 1193
48.243.2.1 converge() . . . 1193
48.243.2.2 get_id() . . . 1194
48.243.2.3 has_converged() . . . 1194
48.243.2.4 has_stopped() . . . 1195

48.243.2.5 is_finalized() . . . 1195
48.243.2.6 stop() . . . 1195
48.243.3 Friends And Related Function Documentation . . . 1196
48.243.3.1 operator"!= . . . 1196
48.243.3.2 operator== . . . 1196
48.244 gko::matrix::Hybrid< ValueType, IndexType >::strategy_type Class Reference . . . 1197
48.244.1 Detailed Description . . . 1197
48.244.2 Member Function Documentation . . . 1197
48.244.2.1 compute_ell_num_stored_elements_per_row() . . . 1197
48.244.2.2 compute_hybrid_config() . . . 1198
48.244.2.3 get_coo_nnz() . . . 1198
48.244.2.4 get_ell_num_stored_elements_per_row() . . . 1199
48.245 gko::log::Stream< ValueType > Class Template Reference . . . 1199
48.245.1 Detailed Description . . . 1199
48.245.2 Member Function Documentation . . . 1200
48.245.2.1 create() [1/2] . . . 1200
48.245.2.2 create() [2/2] . . . 1200
48.246 gko::StreamError Class Reference . . . 1201
48.246.1 Detailed Description . . . 1201
48.246.2 Constructor & Destructor Documentation . . . 1201
48.246.2.1 StreamError() . . . 1201
48.247 gko::log::ProfilerHook::SummaryWriter Class Reference . . . 1202
48.247.1 Detailed Description . . . 1202
48.247.2 Member Function Documentation . . . 1202
48.247.2.1 write() . . . 1202
48.248 gko::log::ProfilerHook::TableSummaryWriter Class Reference . . . 1203
48.248.1 Detailed Description . . . 1203
48.248.2 Constructor & Destructor Documentation . . . 1203
48.248.2.1 TableSummaryWriter() . . . 1203
48.248.3 Member Function Documentation . . . 1204
48.248.3.1 write() . . . 1204
48.248.3.2 write_nested() . . . 1204
48.249 gko::stop::Time Class Reference . . . 1204
48.249.1 Detailed Description . . . 1205
48.250 gko::time_point Class Reference . . . 1205
48.250.1 Detailed Description . . . 1205
48.251 gko::Timer Class Reference . . . 1205
48.251.1 Detailed Description . . . 1206
48.251.2 Member Function Documentation . . . 1206
48.251.2.1 create_for_executor() . . . 1206
48.251.2.2 create_time_point() . . . 1206
48.251.2.3 difference() . . . 1207

48.251.2.4 difference_async() . . . 1207
48.252 gko::Transposable Class Reference . . . 1208
48.252.1 Detailed Description . . . 1208
48.252.1.1 Example: Transposing a Csr matrix: . . . 1208
48.252.2 Member Function Documentation . . . 1208
48.252.2.1 conj_transpose() . . . 1208
48.252.2.2 transpose() . . . 1209
48.253 gko::config::type_descriptor Class Reference . . . 1209
48.253.1 Detailed Description . . . 1210
48.253.2 Constructor & Destructor Documentation . . . 1210
48.253.2.1 type_descriptor() . . . 1210
48.254 gko::experimental::mpi::type_impl< T > Struct Template Reference . . . 1210
48.254.1 Detailed Description . . . 1211
48.255 gko::syn::type_list< Types > Struct Template Reference . . . 1211
48.255.1 Detailed Description . . . 1211
48.256 gko::multigrid::UniformCoarsening< ValueType, IndexType > Class Template Reference . . . 1211
48.256.1 Detailed Description . . . 1212
48.256.2 Member Function Documentation . . . 1212
48.256.2.1 get_coarse_rows() . . . 1212
48.256.2.2 get_const_coarse_rows() . . . 1213
48.256.2.3 get_system_matrix() . . . 1213
48.256.2.4 parse() . . . 1213
48.257 gko::UnsupportedMatrixProperty Class Reference . . . 1214
48.257.1 Detailed Description . . . 1214
48.257.2 Constructor & Destructor Documentation . . . 1214
48.257.2.1 UnsupportedMatrixProperty() . . . 1214
48.258 gko::stop::Criterion::Updater Class Reference . . . 1215
48.258.1 Detailed Description . . . 1215
48.258.2 Member Function Documentation . . . 1215
48.258.2.1 check() . . . 1216
48.259 gko::solver::UpperTrs< ValueType, IndexType > Class Template Reference . . . 1216
48.259.1 Detailed Description . . . 1217
48.259.2 Constructor & Destructor Documentation . . . 1217
48.259.2.1 UpperTrs() [1/2] . . . 1217
48.259.2.2 UpperTrs() [2/2] . . . 1217
48.259.3 Member Function Documentation . . . 1217
48.259.3.1 conj_transpose() . . . 1218
48.259.3.2 operator=() [1/2] . . . 1218
48.259.3.3 operator=() [2/2] . . . 1218
48.259.3.4 parse() . . . 1218
48.259.3.5 transpose() . . . 1219
48.260 gko::UseComposition< ValueType > Class Template Reference . . . 1219

48.260.1 Detailed Description . . . 1219
48.260.2 Member Function Documentation . . . 1220
48.260.2.1 get_composition() . . . 1220
48.260.2.2 get_operator_at() . . . 1220
48.261 gko::syn::value_list< T, Values > Struct Template Reference . . . 1221
48.261.1 Detailed Description . . . 1221
48.262 gko::ValueMismatch Class Reference . . . 1221
48.262.1 Detailed Description . . . 1222
48.262.2 Constructor & Destructor Documentation . . . 1222
48.262.2.1 ValueMismatch() . . . 1222
48.263 gko::experimental::distributed::Vector< ValueType > Class Template Reference . . . 1222
48.263.1 Detailed Description . . . 1225
48.263.2 Member Function Documentation . . . 1225
48.263.2.1 add_scaled() . . . 1225
48.263.2.2 at_local() [1/4] . . . 1226
48.263.2.3 at_local() [2/4] . . . 1226
48.263.2.4 at_local() [3/4] . . . 1226
48.263.2.5 at_local() [4/4] . . . 1227
48.263.2.6 compute_absolute() . . . 1227
48.263.2.7 compute_conj_dot() [1/2] . . . 1227
48.263.2.8 compute_conj_dot() [2/2] . . . 1228
48.263.2.9 compute_dot() [1/2] . . . 1228
48.263.2.10 compute_dot() [2/2] . . . 1228
48.263.2.11 compute_mean() [1/2] . . . 1229
48.263.2.12 compute_mean() [2/2] . . . 1229
48.263.2.13 compute_norm1() [1/2] . . . 1229
48.263.2.14 compute_norm1() [2/2] . . . 1230
48.263.2.15 compute_norm2() [1/2] . . . 1230
48.263.2.16 compute_norm2() [2/2] . . . 1230
48.263.2.17 compute_squared_norm2() [1/2] . . . 1231
48.263.2.18 compute_squared_norm2() [2/2] . . . 1231
48.263.2.19 create() [1/4] . . . 1231
48.263.2.20 create() [2/4] . . . 1232
48.263.2.21 create() [3/4] . . . 1233
48.263.2.22 create() [4/4] . . . 1233
48.263.2.23 create_const() [1/2] . . . 1234
48.263.2.24 create_const() [2/2] . . . 1234
48.263.2.25 create_real_view() [1/2] . . . 1235
48.263.2.26 create_real_view() [2/2] . . . 1235
48.263.2.27 create_submatrix() . . . 1235
48.263.2.28 create_with_config_of() . . . 1236
48.263.2.29 create_with_type_of() [1/2] . . . 1236

48.263.2.30 create_with_type_of() [2/2] . . . 1236
48.263.2.31 fill() . . . 1237
48.263.2.32 get_const_local_values() . . . 1237
48.263.2.33 get_local_values() . . . 1238
48.263.2.34 get_local_vector() . . . 1238
48.263.2.35 inv_scale() . . . 1238
48.263.2.36 make_complex() [1/2] . . . 1238
48.263.2.37 make_complex() [2/2] . . . 1239
48.263.2.38 read_distributed() [1/2] . . . 1239
48.263.2.39 read_distributed() [2/2] . . . 1239
48.263.2.40 scale() . . . 1240
48.263.2.41 sub_scaled() . . . 1240
48.264 gko::version Struct Reference . . . 1240
48.264.1 Detailed Description . . . 1241
48.264.2 Member Data Documentation . . . 1241
48.264.2.1 tag . . . 1241
48.265 gko::version_info Class Reference . . . 1241
48.265.1 Detailed Description . . . 1242
48.265.2 Member Function Documentation . . . 1242
48.265.2.1 get() . . . 1242
48.265.3 Member Data Documentation . . . 1242
48.265.3.1 core_version . . . 1242
48.265.3.2 cuda_version . . . 1243
48.265.3.3 dpcpp_version . . . 1243
48.265.3.4 hip_version . . . 1243
48.265.3.5 omp_version . . . 1243
48.265.3.6 reference_version . . . 1243
48.266 gko::experimental::mpi::window< ValueType > Class Template Reference . . . 1243
48.266.1 Detailed Description . . . 1245
48.266.2 Constructor & Destructor Documentation . . . 1245
48.266.2.1 window() [1/3] . . . 1246
48.266.2.2 window() [2/3] . . . 1246
48.266.2.3 window() [3/3] . . . 1246
48.266.3 Member Function Documentation . . . 1247
48.266.3.1 accumulate() . . . 1247
48.266.3.2 fence() . . . 1248
48.266.3.3 fetch_and_op() . . . 1248
48.266.3.4 flush() . . . 1248
48.266.3.5 flush_local() . . . 1249
48.266.3.6 get() . . . 1249
48.266.3.7 get_accumulate() . . . 1250
48.266.3.8 get_window() . . . 1250

48.266.3.9 lock() . . . 1251
48.266.3.10 lock_all() . . . 1251
48.266.3.11 operator=() . . . 1252
48.266.3.12 put() . . . 1252
48.266.3.13 r_accumulate() . . . 1253
48.266.3.14 r_get() . . . 1253
48.266.3.15 r_get_accumulate() . . . 1254
48.266.3.16 r_put() . . . 1255
48.266.3.17 unlock() . . . 1256
48.267 gko::solver::workspace_traits< Solver > Struct Template Reference . . . 1256
48.267.1 Detailed Description . . . 1256
48.268 gko::WritableToMatrixData< ValueType, IndexType > Class Template Reference . . . 1257
48.268.1 Detailed Description . . . 1257
48.268.2 Member Function Documentation . . . 1257
48.268.2.1 write() [1/2] . . . 1257
48.268.2.2 write() [2/2] . . . 1258

**Index** **1259**

# Chapter 1

# Main Page

This is the main page for the Ginkgo library pdf documentation. The repository is hosted on `github`. Documentation on aspects such as the build system, can be found at the Installation Instructions page. The Example programs can help you get started with using Ginkgo.

#### 1.0.0.1 Modules

The Ginkgo library can be grouped into `modules` and these modules form the basic building blocks of Ginkgo. The modules can be summarized as follows:

- Executors : Where do you want your code to be executed ?
- Linear Operators : What kind of operation do you want Ginkgo to perform ?
  - Solvers : Solve a linear system for a given matrix.
  - Preconditioners : Precondition a system for a solve.
  - SpMV employing different Matrix formats : Perform a sparse matrix vector multiplication with a particular matrix format.
- Logging : Monitor your code execution.
- Stopping criteria : Manage your iteration stopping criteria.

# Chapter 2

# Installation Instructions

### 2.0.1 Building

Use the standard CMake build procedure:

```
mkdir build; cd build
cmake [OPTIONS] ..  && cmake --build .
```

For Microsoft Visual Studio, use `cmake --build .  --config <build_type>` to decide the build type. The possible options are `Debug`, `Release`, `RelWithDebInfo` and `MinSizeRel`.

Replace `[OPTIONS]` with desired cmake options for your build. Ginkgo adds the following additional switches to control what is being built:

- `-DGINKGO_DEVEL_TOOLS={ON, OFF}` sets up the build system for development (requires pre-commit, will also download the clang-format pre-commit hook), default is `OFF`. The default behavior installs a pre-commit hook, which disables git commits. If it is set to `ON`, a new pre-commit hook for formatting will be installed (enabling commits again). In both cases the hook may overwrite a user defined pre-commit hook when Ginkgo is used as a submodule.
- `-DGINKGO_MIXED_PRECISION={ON, OFF}` compiles true mixed-precision kernels instead of converting data on the fly, default is `OFF`. Enabling this flag increases the library size, but improves performance of mixed-precision kernels.
- `-DGINKGO_ENABLE_HALF={ON, OFF}` enable half precision support in Ginkgo, default is `ON`. It is `OFF` when the compiler is MSVC or MSYS2 system. If compiling is done with the CUDA backend before CUDA 12.2, we only support half precision after compute capability 5.3. CUDA 12.2+ compilers waive the compute capability limitation.
- `-DGINKGO_ENABLE_BFLOAT16={ON, OFF}` enable half precision support in Ginkgo, default is `ON`. It is `OFF` when the compiler is MSVC or MSYS2 system. If compiling is done with the CUDA backend before CUDA 12.2, we only support bfloat16 precision after compute capability 8.0. CUDA 12.2+ compilers waive the compute capability limitation.
- `-DGINKGO_BUILD_TESTS={ON, OFF}` builds Ginkgo's tests (will download googletest), default is `ON`.
- `-DGINKGO_FAST_TESTS={ON, OFF}` reduces the input sizes for a few slow tests to speed them up, default is `OFF`.
- `-DGINKGO_BUILD_BENCHMARKS={ON, OFF}` builds Ginkgo's benchmarks (will download gflags and nlohmann-json), default is `ON`.
- `-DGINKGO_BUILD_EXAMPLES={ON, OFF}` builds Ginkgo's examples, default is `ON`

- `-DGINKGO_BUILD_EXTLIB_EXAMPLE={ON, OFF}` builds the interfacing example with deal.II, default is `OFF`.
- `-DGINKGO_BUILD_REFERENCE={ON, OFF}` build reference implementations of the kernels, useful for testing, default is `ON`
- `-DGINKGO_BUILD_OMP={ON, OFF}` builds optimized OpenMP versions of the kernels, default is `ON` if the selected C++ compiler supports OpenMP, `OFF` otherwise.
- `-DGINKGO_BUILD_CUDA={ON, OFF}` builds optimized cuda versions of the kernels (requires CUDA), default is `ON` if a CUDA compiler could be detected, `OFF` otherwise.
- `-DGINKGO_BUILD_DPCPP={ON, OFF}` is deprecated. Please use `GINKGO_BUILD_SYCL` instead.
- `-DGINKGO_BUILD_SYCL={ON, OFF}` builds optimized SYCL versions of the kernels (requires `CMAKE_CXX_COMPILER` to be set to the `dpcpp` or `icpx` compiler). The default is `ON` if `CMAKE_CXX_COMPILER` is a SYCL compiler, `OFF` otherwise. Due to some differences in IEEE 754 floating point numberhandling in the Intel SYCL compilers, Ginkgo tests may fail unless compiled with `-DCMAKE_CXX_FLAGS=-ffp-model=precise`
- `-DGINKGO_BUILD_HIP={ON, OFF}` builds optimized HIP versions of the kernels (requires HIP), default is `ON` if an installation of HIP could be detected, `OFF` otherwise.
- `-DCMAKE_HIP_ARCHITECTURES="gpuarch1;gpuarch2"` the AMDGPU targets to be passed to the compiler. If empty, compiler chooses based on the available GPUs.
- `-DGINKGO_BUILD_DOC={ON, OFF}` creates an HTML version of Ginkgo's documentation from inline comments in the code. The default is `OFF`.
- `-DGINKGO_DOC_GENERATE_EXAMPLES={ON, OFF}` generates the documentation of examples in Ginkgo. The default is `ON`.
- `-DGINKGO_DOC_GENERATE_PDF={ON, OFF}` generates a PDF version of Ginkgo's documentation from inline comments in the code. The default is `OFF`.
- `-DGINKGO_DOC_GENERATE_DEV={ON, OFF}` generates the developer version of Ginkgo's documentation. The default is `OFF`.
- `-DGINKGO_WITH_CLANG_TIDY={ON, OFF}` makes Ginkgo call `clang-tidy` to find programming issues. The path can be manually controlled with the CMake variable `-DGINKGO_CLANG_TIDY_PATH=<path>`. The default is `OFF`.
- `-DGINKGO_WITH_IWYU={ON, OFF}` makes Ginkgo call `iwyu` to find include issues. The path can be manually controlled with the CMake variable `-DGINKGO_IWYU_PATH=<path>`. The default is `OFF`.
- `-DGINKGO_CHECK_CIRCULAR_DEPS={ON, OFF}` enables compile-time checks for circular dependencies between different Ginkgo libraries and self-sufficient headers. Should only be used for development purposes. The default is `OFF`.
- `-DGINKGO_VERBOSE_LEVEL=integer` sets the verbosity of Ginkgo.
  - `0` disables all output in the main libraries,
  - `1` enables a few important messages related to unexpected behavior (default).
- `GINKGO_INSTALL_RPATH` allows setting any RPATH information when installing the Ginkgo libraries. If this is `OFF`, the behavior is the same as if all other RPATH flags are set to `OFF` as well. The default is `ON`.
- `GINKGO_INSTALL_RPATH_ORIGIN` adds $ORIGIN (Linux) or @loader_path (MacOS) to the installation RPATH. The default is `ON`.
- `GINKGO_INSTALL_RPATH_DEPENDENCIES` adds the dependencies to the installation RPATH. The default is `OFF`.
- `-DCMAKE_INSTALL_PREFIX=path` sets the installation path for `make install`. The default value is usually something like `/usr/local`.

- -DCMAKE_BUILD_TYPE=type specifies which configuration will be used for this build of Ginkgo. The default is RELEASE. Supported values are CMake's standard build types such as DEBUG and RELEASE and the Ginkgo specific COVERAGE, ASAN (AddressSanitizer), LSAN (LeakSanitizer), TSAN (ThreadSanitizer) and UBSAN (undefined behavior sanitizer) types.
- -DBUILD_SHARED_LIBS={ON, OFF} builds ginkgo as shared libraries (OFF) or as dynamic libraries (ON), default is ON.
- -DGINKGO_JACOBI_FULL_OPTIMIZATIONS={ON, OFF} use all the optimizations for the CUDA Jacobi algorithm. OFF by default. Setting this option to ON may lead to very slow compile time (>20 minutes) for the jacobi_generate_kernels.cu file and high memory usage.
- -DCMAKE_CUDA_HOST_COMPILER=path instructs the build system to explicitly set CUDA's host compiler to the path given as argument. By default, CUDA uses its toolchain's host compiler. Setting this option may help if you're experiencing linking errors due to ABI incompatibilities. This option is supported since CMake 3.8 but documented starting from 3.10.
- -DGINKGO_CUDA_ARCHITECTURES=<list> where <list> is a semicolon (;) separated list of architectures. Supported values are:
  - Auto
  - Kepler, Maxwell, Pascal, Volta, Turing, Ampere
  - CODE, CODE(COMPUTE), (COMPUTE)

  Auto will automatically detect the present CUDA-enabled GPU architectures in the system. Kepler, Maxwell, Pascal, Volta and Ampere will add flags for all architectures of that particular NVIDIA GPU generation. COMPUTE and CODE are placeholders that should be replaced with compute and code numbers (e.g. for compute_70 and sm_70 COMPUTE and CODE should be replaced with 70. Default is Auto. For a more detailed explanation of this option see the ARCHITECTURES specification list section in the documentation of the CudaArchitectureSelector CMake module.

Additionally, the following CMake options have effect on the build process:

- -DCMAKE_EXPORT_PACKAGE_REGISTRY={ON,OFF} if set to ON the build directory will be stored in the current user's CMake package registry.

For example, to build everything (in debug mode), use:

```
cmake ..  -BDebug -DCMAKE_BUILD_TYPE=Debug -DGINKGO_DEVEL_TOOLS=ON \
    -DGINKGO_BUILD_TESTS=ON -DGINKGO_BUILD_REFERENCE=ON -DGINKGO_BUILD_OMP=ON \
    -DGINKGO_BUILD_CUDA=ON -DGINKGO_BUILD_HIP=ON
cmake --build Debug
```

NOTE: Ginkgo is known to work with the Unix Makefiles, Ninja, MinGW Makefiles and Visual Studio 16 2019 based generators. Other CMake generators are untested.

### 2.0.2 Building Ginkgo in Windows

Depending on the configuration settings, some manual work might be required:

- Build Ginkgo with Debug mode: Some Debug build specific issues can appear depending on the machine and environment: When you encounter the error message ld: error: export ordinal too large, add the compilation flag -O1 by adding -DCMAKE_CXX_FLAGS=-O1 to the CMake invocation.
- Build Ginkgo in *MinGW*:\ If encountering the issue cc1plus.exe: out of memory allocating 65536 bytes, please follow the workaround in reference, or trying to compile ginkgo again might work.

### 2.0.3 Building Ginkgo with HIP support

Ginkgo provides a HIP backend. This allows to compile optimized versions of the kernels for either AMD or NVIDIA GPUs. The CMake configuration step will try to auto-detect the presence of HIP either at /opt/rocm/hip or at the path specified by HIP_PATH as a CMake parameter (-DHIP_PATH=) or environment variable (export HIP_PATH=), unless -DGINKGO_BUILD_HIP=ON/OFF is set explicitly.

#### 2.0.3.1 Changing the paths to search for HIP and other packages

All HIP installation paths can be configured through the use of environment variables or CMake variables. This way of configuring the paths is currently imposed by the HIP tool suite. The variables are the following:

- CMake -DROCM_PATH= or environment export ROCM_PATH=: sets the ROCM installation path. The default value is /opt/rocm/.
- CMake -DHIP_CLANG_PATH or environment export HIP_CLANG_PATH=: sets the HIP compatible clang binary path. The default value is ${ROCM_PATH}/llvm/bin.
- CMake -DHIP_PATH= or environment export HIP_PATH=: sets the HIP installation path. The default value is ${ROCM_PATH}/hip.
- CMake -DHIPBLAS_PATH= or environment export HIPBLAS_PATH=: sets the hipBLAS installation path. The default value is ${ROCM_PATH}/hipblas.
- CMake -DHIPSPARSE_PATH= or environment export HIPSPARSE_PATH=: sets the hipSPARSE installation path. The default value is ${ROCM_PATH}/hipsparse.
- CMake -DHIPFFT_PATH= or environment export HIPFFT_PATH=: sets the hipFFT installation path. The default value is ${ROCM_PATH}/hipfft.
- CMake -DROCRAND_PATH= or environment export ROCRAND_PATH=: sets the rocRAND installation path. The default value is ${ROCM_PATH}/rocrand.
- CMake -DHIPRAND_PATH= or environment export HIPRAND_PATH=: sets the hipRAND installation path. The default value is ${ROCM_PATH}/hiprand.
- environment export CUDA_PATH=: where hipcc can find CUDA if it is not in the default /usr/local/cuda path.

#### 2.0.3.2 HIP platform detection of AMD and NVIDIA

Ginkgo relies on CMake to decide which compiler to use for HIP. To choose nvcc instead of the default ROCm clang++, set the corresponding environment variable:

```
export HIPCXX=nvcc
```

Note that this option is currently not being tested in our CI pipelines.

### 2.0.4 Third party libraries and packages

Ginkgo relies on third party packages in different cases. These third party packages can be turned off by disabling the relevant options.

- GINKGO_BUILD_TESTS=ON: Our tests are implemented with Google Test;
- GINKGO_BUILD_BENCHMARKS=ON: For argument management we use gflags and for JSON parsing we use nlohmann-json;
- GINKGO_BUILD_EXAMPLES=ON: OpenCV is required for some examples, they are disabled when OpenCV is not available.
- GINKGO_BUILD_DOC=ON: doxygen is required to build the documentation and additionally graphviz is required to build the class hierarchy graphs.
- METIS is required when using the NestedDissection reordering functionality. If METIS is not found, the functionality is disabled.
- PAPI (>= 7.1.0) is required when using the Papi logger. If PAPI is not found, the functionality is disabled.

Ginkgo attempts to use pre-installed versions of these package if they match version requirements using find_package. Otherwise, the configuration step will download the files for each of the packages GTest, gflags, and nlohmann-json and build them internally.

Note that, if the external packages were not installed to the default location, the CMake option -DCMAKE_PREFIX_PATH=<path-list> needs to be set to the semicolon (;) separated list of install paths of these external packages. For more Information, see the CMake documentation for CMAKE_PREFIX_PATH for details.

For convenience, the options GINKGO_INSTALL_RPATH[_.*] can be used to bind the installed Ginkgo shared libraries to the path of its dependencies.

### 2.0.5 Installing Ginkgo

To install Ginkgo into the specified folder, execute the following command in the build folder

```
make install
```

If the installation prefix (see CMAKE_INSTALL_PREFIX) is not writable for your user, e.g. when installing Ginkgo system-wide, it might be necessary to prefix the call with sudo.

After the installation, CMake can find ginkgo with find_package(Ginkgo). An example can be found in the test_install.

# Chapter 3

# Testing Instructions

### 3.0.1 Running the unit tests

You need to compile ginkgo with `-DGINKGO_BUILD_TESTS=ON` option to be able to run the tests.

#### 3.0.1.1 Using make test

After configuring Ginkgo, use the following command inside the build folder to run all tests:

```
make test
```

The output should contain several lines of the form:

```
    Start  1:  path/to/test
1/13 Test  #1:  path/to/test ..............................      Passed    0.01 sec
```

To run only a specific test and see more details results (e.g. if a test failed) run the following from the build folder:

```
./path/to/test
```

where `path/to/test` is the path returned by `make test`.

#### 3.0.1.2 Using make quick_test

After compiling Ginkgo, use the following command inside the build folder to run a small subset of tests that should execute quickly:

```
make quick_test
```

These tests do not use GPU features except for a few device property queries, so they may still fail if Ginkgo was compiled with GPU support, but no such GPU is available. The output is equivalent to `make test`.

#### 3.0.1.3 Using CTest

The tests can also be ran through CTest from the command line, for example when in a configured build directory:

```
ctest -T start -T build -T test -T submit
```

Will start a new test campaign (usually in `Experimental` mode), build Ginkgo with the set configuration, run the tests and submit the results to our CDash dashboard.

Another option is to use Ginkgo's CTest script which is configured to build Ginkgo with default settings, runs the tests and submits the test to our CDash dashboard automatically.

To run the script, use the following command:

```
ctest -S cmake/CTestScript.cmake
```

The default settings are for our own CI system. Feel free to configure the script before launching it through variables or by directly changing its values. A documentation can be found in the script itself.

# Chapter 4

# Running the benchmarks

In addition to the unit tests designed to verify correctness, Ginkgo also includes an extensive benchmark suite for checking its performance on all Ginkgo supported systems. The purpose of Ginkgo's benchmarking suite is to allow easy and complete reproduction of Ginkgo's performance, and to facilitate performance debugging as well. Most results published in Ginkgo papers are generated thanks to this benchmarking suite and are accessible online under the `ginkgo-data repository`. These results can also be used for performance comparison in order to ensure that you get similar performance as what is published on this repository.

To compile the benchmarks, the flag `-DGINKGO_BUILD_BENCHMARKS=ON` has to be set during the `cmake` step. In addition, the `ssget command-line utility` can be installed on the system to load matrices. The purpose of this file is to explain in detail the capacities of this benchmarking suite as well as how to properly setup everything.

There are two ways to benchmark Ginkgo. When compiling the benchmark suite, executables are generated for collecting matrix statistics, running sparse-matrix vector product, solvers (possibly distributed) benchmarks. Thus, one can run the needed executable for benchmarking **`manually`**. Another way to run (almost) **`all benchmarks`** is through the convenience script `run_all_benchmarks.sh`, but not all features are exposed through this tool!

Here is a short description of the content of this file:

1. `Ginkgo setup and best practice guidelines`
2. `Using ssget to fetch the matrices`
3. `Running benchmarks manually`
4. `Benchmarking with make benchmark or run_all_benchmarks.sh`
5. `Publishing the results on Github and analyze the results with the GPE (optional)`
6. `Detailed performance analysis and debugging`
7. `Available benchmark options`

### 4.0.1 1: Ginkgo setup and best practice guidelines

Before benchmarking Ginkgo, make sure that you follow the general guidelines in order to ensure best performance.

1. The code should be compiled in `Release` mode.
2. Make sure the machine has no competing jobs. On a Linux machine multiple commands can be used, `last` shows the currently opened sessions, `top` or `htop` allows to show the current machine load, and if considering using specific GPUs, `nvidia-smi` or `rocm-smi` can be used to check their load.
3. By default, Ginkgo's benchmarks will always do at least one warm-up run. For better accuracy, every benchmark is also averaged over 10 runs, except for the solver benchmark which are usually fairly long. These parameters can be tuned at the command line to either shorten benchmarking time or improve benchmarking accuracy.

In addition, the following specific options can be considered:

1. When specifically using the adaptive block jacobi preconditioner, enable the `GINKGO_JACOBI_FULL_O↩PTIMIZATIONS` CMake flag. Be careful that this will use much more memory and time for the compilation due to compiler performance issues with register optimizations, in particular.
2. The current benchmarking setup also allows to benchmark only the overhead by using as either (or for all) preconditioner/spmv/solver, the special `overhead` LinOp. If your purpose is to check Ginkgo's overhead, make sure to try this mode.

### 4.0.2 2: Using ssget to fetch the matrices

To benchmark `ginkgo`, matrices need to be provided as input in the `Matrix Market` format. A convenient way is to run benchmark with the SuiteSparse matrix collection. A helper tool, the ssget command-line utility can be used to facilitate downloading and extracting matrices from the suitesparse collection. When running the benchmarks with the helper script `run_all_benchmarks.sh` (or calling `make benchmark`), the `ssget` tool is required.

To install `ssget`, access the repository and copy the file `ssget` into a directory present in your `PATH` variable as per the tool's `README.md` instructions. The tool can be installed either in a global system path or a local directory such as `$HOME/.local/bin`. After installing the tool, it is important to review the `ssget` script and configure as needed the variable `ARCHIVE_LOCATION` on line 39. This is where the matrices will be stored into. As of PR#7, one can also configure the archive location using the `-a` option without modifying the script itself. You can also use this script without installing it.

The Ginkgo benchmark can be set to run on only a portion of the SuiteSparse matrix collection as we will see in the following section. Please note that the entire collection requires roughly 100GB of disk storage in its compressed format, and roughly 25GB of additional disk space for intermediate data (such us uncompressing the archive). Additionally, the benchmark runs usually take a long time (SpMV benchmarks on the complete collection take roughly 24h using the K20 GPU), and will stress the system.

Before proceeding, it can be useful in order to save time to download the matrices as preparation. This can be done by using the `ssget -f -i i` command where `i` is the ID of the matrix to be downloaded. The following loop allows to download the full SuiteSparse matrix collection:

```
for i in $(seq 0 $(ssget -n)); do
    ssget -f -i ${i}
done
```

Note that `ssget` can also be used to query properties of the matrix and filter the matrices which are downloaded. For example, the following will download only positive definite matrices with less than 500M non zero elements and 10M columns. Please refer to the ssget documentation for more information.

```
for i in $(seq 0 $(ssget -n)); do
    posdef=$(ssget -p posdef -i ${i})
    cols=$(ssget -p cols -i ${i})
    nnz=$(ssget -p nonzeros -i ${i})
    if [ "$posdef" -eq 1 -a "$cols" -lt 10000000 -a "$nnz" -lt 500000000 ]; then
        ssget -f -i ${i}
    fi
done
```

For extracting the matrices, `ssget -f -i ${i}` can be used.

### 4.0.3 3: Running benchmarks manually

As mentioned above, one way to run benchmarks is through the executable of a certain needed type of benchmarking.

When compiling Ginkgo with the flag `-DGINKGO_BUILD_BENCHMARKS=ON`, a suite of executables will be generated depending on the CMake configuration. These executables are the backbone of the benchmarking suite. Note that all of these executables describe the available options and the required input format when running them with the `--help` option. All executables have multiple variants depending on the precision, by default `double` precision is used for the type of values, but variants with `single` and `complex` (single and double) value types are also available. Here is a non exhaustive list of the available benchmarks:

- `blas/blas`: supports benchmarking many of Ginkgo's BLAS operations: dot products, axpy, copy, etc.
- `conversion/conversion`: conversion between matrix formats.
- `matrix_generator/matrix_generator`: mostly allows generating block diagonal matrices (to benchmark the block-jacobi preconditioner).
- `matrix_statistics/matrix_statistics`: computes size and other matrix statistics (such as variance, load imbalance, ...).
- `preconditioner/preconditioner`: benchmarks most Ginkgo preconditioner.
- `solver/solver`: benchmark most of Ginkgo's solvers in a non distributed setting.
- `sparse_blas/sparse_blas`: benchmarks Sparse BLAS operations, such as SpGEMM, SpGEAM, transpose.
- `spmv/spmv`: benchmarks Ginkgo's matrix formats (Sparse-Matrix Vector product).

Optionally when compiling with MPI support:

- `blas/distributed/multi_vector`: measures BLAS performance on (distributed) multi-vectors.
- `solver/distributed/solver`: distributed solver benchmarks.
- `spmv/distributed/spmv`: distributed matrix Sparse-Matrix Vector (SpMV) product benchmark.

All benchmarks require input data as in a `JSON` format. The json file has to consist of exactly one array, and within that array the test cases are defined. The exact syntax can change between executables, the `--help` option will explain the necessary `JSON` input format. For example for the `spmv` benchmark case, and many other benchmarks the following minimal input should be provided:

```
[
  {
    "filename":  "path/to/your/matrix",
    "rhs":  "path/to/your/rhs"
  },
  { ...  }
]
```

The files have to be in matrix market format.

Some benchmarks require some extra fields. For example, the **solver benchmarks** requires the field `"optimal": {"spmv": "matrix format (such as csr)"}`. This is automatically populated when running the `spmv` benchmark which finds the optimal (fastest) format among all requested formats.

The `"rhs"` field (right-hand side) is generally optional for the **solver benchmarks** if you generate the RHS with the `-rhs_generation` flag (use `--help` with benchmark executables if this is unclear).

After writing the necessary data in a JSON file, the benchmark can be called by passing in the input via stdin, i.e.

```
./solver < input.json
```

The output of our benchmarks is again JSON, and it is printed to stdout, while our status messages are printed to stderr. So, the output can be stored with

```
./solver < input.json > output.json
```

Note that in most cases, the JSON output by our benchmarks is compatible with other benchmarks, therefore it is possible to first call the `spmv` benchmark, use the resulting output JSON as input to the `solver` benchmark, and finally use the resulting solver JSON output as input to the `preconditioner` benchmark.

### 4.0.4 4: Benchmarking with <tt>make benchmark</tt> or <tt>run_all_benchmarks.sh</tt>

The benchmark suite is invoked using the `make benchmark` command in the build directory. Under the hood, this command simply calls the script `benchmark/run_all_benchmarks.sh` so it is possible to manually launch this script as well. The behavior of the suite can be modified using environment variables. Assuming the `bash` shell is used, these can either be specified via the `export` command to persist between multiple runs:

```
export VARIABLE="value"
...
make benchmark
```

or specified on the fly, on the same line as the `make benchmark` command:

```
VARIABLE="value" ...  make benchmark
```

Since `make` sets any variables passed to it as temporary environment variables, the following shorthand can also be used:

```
make benchmark VARIABLE="value" ...
```

A combination of the above approaches is also possible (e.g. it may be useful to `export` the `SYSTEM_NAME` variable, and specify the others at every benchmark run).

The benchmark suite can take a number of configuration parameters. Benchmarks can be run only for `sparse matrix vector products (spmv)`, for full solvers (with or without preconditioners), or for preconditioners only when supported. The benchmark suite also allows to target a sub-part of the SuiteSparse matrix collection. For details, see the `available benchmark options`. Here are the most important options:

- `BENCHMARK={spmv, solver, preconditioner}` - allows to select the type of benchmark to be run.
- `EXECUTOR={reference,cuda,hip,omp,dpcpp}` - select the executor and platform the benchmarks should be run on.
- `SYSTEM_NAME=<name>` - a name which will be used to designate this platform (e.g. V100, RadeonVII, ...).
- `SEGMENTS=<N>` - Split the benchmarked matrix space into `<N>` segments. If specified, `SEGMENT_ID` also has to be set.
- `SEGMENT_ID=<I>` - used in combination with the `SEGMENTS` variable. `<I>` should be an integer between 1 and `<N>`, the number of `SEGMENTS`. If specified, only the `<I>`-th segment of the benchmark suite will be run.
- `BENCHMARK_PRECISION` - defines the precision the benchmarks are run in. Supported values are↩ : "double" (default), "single", "dcomplex" and "scomplex"
- `MATRIX_LIST_FILE=/path/to/matrix_list.file` - allows to list SuiteSparse matrix id or name to benchmark. As an example, a matrix list file containing the following will ensure that benchmarks are ran for only those three matrices:

  ``` 1903 Freescale/circuit5M thermal2 ```

### 4.0.5 5: Publishing the results on Github and analyze the results with the GPE (optional)

The previous experiments generated json files for each matrices, each containing timing, iteration count, achieved precision, ... depending on the type of benchmark run. These files are available in the directory `${ginkgo↩ _build_dir}/benchmark/results/`. These files can be analyzed and processed through any tool (e.g. python). In this section, we describe how to generate the plots by using Ginkgo's `GPE` tool. First, we need to publish the experiments into a Github repository which will be then linked as source input to the GPE. For this,
  ```

we can simply fork the ginkgo-data repository. To do so, we can go to the github repository and use the forking interface: https://github.com/ginkgo-project/ginkgo-data/

Once it's done, we want to clone the repository locally, put all results online and access the GPE for plotting the results. Here are the detailed steps:

```
git clone https://github.com/<username>/ginkgo-data.git $HOME/ginkgo_benchmark/ginkgo-data
# send the benchmarked data to the ginkgo-data repository
# If needed, remove the old data so that no previous data is left.
# rm -r ${HOME}/ginkgo_benchmark/ginkgo-data/data/${SYSTEM_NAME}
rsync -rtv ${ginkgo_build_dir}/benchmark/results/ $HOME/ginkgo_benchmark/ginkgo-data/data/
cd ${HOME}/ginkgo_benchmark/ginkgo-data/data/
# The following updates the main `.json` files with the list of data.
# Ensure a python 3 installation is available.
./build-list .  > list.json
./agregate < list.json > agregate.json
./represent .  > represent.json
git config --local user.name "<Name>"
git config --local user.email "<email>"
git commit -am "Ginkgo benchmark ${BENCHMARK} of ${SYSTEM_NAME}..."
git push
```

Note that depending on what data is of interest, you may need to update the scripts `build-list` or `agregate` to change which files you want to agglomerate and summarize (depending on the system name), or which data you want to select (solver results, spmv results, ...).

For the generating the plots in the GPE, here are the steps to go through:

1. Access the GPE: https://ginkgo-project.github.io/gpe/
2. Update data root URL, from https://raw.githubusercontent.com/ginkgo-project/ginkgo-data/mas to https://raw.githubusercontent.com/<username>/ginkgo-data/<branch>/data
3. Click on the arrow to load the data, select the `Result Summary` entry above.
4. Click on `select an example` to choose a plotting script. Multiple scripts are available by default in different branches. You can use the `jsonata` and `chartjs` languages to develop your own as well.
5. The results should be available in the tab "plot" on the right side. Other tabs allow to access the result of the processed data after invoking the processing script.

### 4.0.6 6: Detailed performance analysis and debugging

Detailed performance analysis can be run by passing the environment variable `DETAILED=1` to the benchmarking script. This detailed run is available for solvers and allows to log the internal residual after every iteration as well as log the time taken by all operations. These features are also available in the `performance-debugging` example which can be used instead and modified as needed to analyze Ginkgo's performance.

These features are implemented thanks to the loggers located in the file `${ginkgo_src_dir}/benchmark/utils/loggers` Ginkgo possesses hooks at all important code location points which can be inspected thanks to the logger. In this fashion, it is easy to use these loggers also for tracking memory allocation sizes and other important library aspects.

### 4.0.7 7: Available benchmark options

There are a set amount of options available for benchmarking. Most important options can be configured through the benchmarking script itself thanks to environment variables. Otherwise, some specific options are not available through the benchmarking scripts but can be directly configured when running the benchmarking program itself. For a list of all options, run for example `${ginkgo_build_dir}/benchmark/solver/solver --help`.

The supported environment variables are described in the following list:

- BENCHMARK={spmv, solver, preconditioner} - allows to select the type of benchmark to be run. Default is spmv.
  - spmv - Runs the sparse matrix-vector product benchmarks on the SuiteSparse collection.
  - solver - Runs the solver benchmarks on the SuiteSparse collection. The matrix format is determined by running the spmv benchmarks first, and using the fastest format determined by that benchmark.
  - preconditioner - Runs the preconditioner benchmarks on artificially generated block-diagonal matrices.
- EXECUTOR={reference,cuda,hip,omp,dpcpp} - select the executor and platform the benchmarks should be run on. Default is cuda.
- SYSTEM_NAME=<name> - a name which will be used to designate this platform (e.g. V100, RadeonVII, ...) and not overwrite previous results. Default is unknown.
- SEGMENTS=<N> - Split the benchmarked matrix space into <N> segments. If specified, SEGMENT_ID also has to be set. Default is 1.
- SEGMENT_ID=<I> - used in combination with the SEGMENTS variable. <I> should be an integer between 1 and <N>, the number of SEGMENTS. If specified, only the <I>-th segment of the benchmark suite will be run. Default is 1.
- MATRIX_LIST_FILE=/path/to/matrix_list.file - allows to list SuiteSparse matrix id or name to benchmark. As an example, a matrix list file containing the following will ensure that benchmarks are run for only those three matrices: ``` 1903 Freescale/circuit5M thermal2 ``∗DEVICE_ID- the accelerator device ID to target for the benchmark. The default is0. ∗DRY_RUN={true, false}- If set totrue`, prepares the system for the benchmark runs (downloads the collections, creates the result structure, etc.) and outputs the list of commands that would normally be run, but does not run the benchmarks themselves. Default is false.
- PRECONDS={jacobi,ic,ilu,paric,parict,parilu,parilut,ic-isai,ilu-isai,paric-isai,pari the preconditioners to use for either solver or preconditioner benchmarks. Multiple options can be passed to this variable. Default is none.
- FORMATS={csr,coo,ell,hybrid,sellp,hybridxx,cusparse_xx,hipsparse_xx} the matrix formats to benchmark for the spmv phase of the benchmark. Run ${ginkgo_build_↩ dir}/benchmark/spmv/spmv --help for a full list. If needed, multiple options for hybrid with different optimization parameters are available. Depending on the libraries available at build time, vendor library formats (cuSPARSE with cusparse_ prefix or hipSPARSE with hipsparse_ prefix) can be used as well. Multiple options can be passed. The default is csr,coo,ell,hybrid,sellp.
- SOLVERS={bicgstab,bicg,cg,cgs,fcg,pipe_cg,gmres,cb_gmres_{keep,reduce1,reduce2,integ _trs,upper_trs}
  - the solvers which should be benchmarked. Multiple options can be passed. The default is bicgstab,cg,cgs,fcg,gmres,idr. Note that lower/upper_trs by default don't use a preconditioner, as they are by default exact direct solvers.
- SOLVERS_PRECISION=<precision> - the minimal residual reduction before which the solver should stop. The default is 1e-6.
- SOLVERS_MAX_ITERATIONS=<number> - the maximum number of iterations with which a solver should be run. The default is 10000.
- SOLVERS_RHS={1, random, sinus} - whether to use a vector of all ones, random values or b = A ∗ (s / |s|)$ with s(idx) = sin(idx) (for complex numbers, s(idx) = sin(2∗idx) + i ∗ sin(2∗idx+1)) as the right-hand side in solver benchmarks. Default is 1.
- SOLVERS_INITIAL_GUESS={rhs,0,random} - the initial guess generation of the solvers. rhs uses the right-hand side, 0 uses a zero vector and random generates a random vector as the initial guess.
- DETAILED={0,1} - selects whether detailed benchmarks should be run. This generally provides extra, verbose information at the cost of one or more extra benchmark runs. It can be either 0 (off) or 1 (on).

- `GPU_TIMER={true, false}` - If set to `true`, use the gpu timer, which is valid for cuda/hip executor, to measure the timing. Default is `false`.
- `SOLVERS_JACOBI_MAX_BS` - sets the maximum block size for the Jacobi preconditioner (if used, otherwise, it does nothing) in the solvers benchmark. The default is '32'.
- `SOLVERS_GMRES_RESTART` - the maximum dimension of the Krylov space to use in GMRES. The default is `100`.

# Chapter 5

# Contributing guidelines

We are glad that you are interested in contributing to Ginkgo. Please have a look at our coding guidelines before proposing a pull request.

## 5.1 Table of Contents


Most Important stuff

Project Structure

- Extended header files
- Using library classes

Git related

- Our git Workflow
- Writing good commit messages
- Creating, Reviewing and Merging Pull Requests

Code Style

- Automatic code formatting
- Naming Scheme
- Whitespace
- Include statement grouping
- Other Code Formatting not handled by ClangFormat
- CMake coding style

Helper Scripts

- Create a new algorithm
- Converting CUDA code to HIP code

Writing Tests

- Testing know-how
- Some general rules
- Writing tests for kernels

Documentation style

- Developer targeted notes
- Whitespaces
- Documenting examples

Other programming comments

- C++ standard stream objects
- Warnings
- Avoiding circular dependencies

## 5.2 Most important stuff (A TL;DR)

- GINKGO_DEVEL_TOOLS needs to be set to on to commit. This requires clang-format to be installed. See Automatic code formatting for more details. Once installed, you can run make format in your build/ folder to automatically format your modified files. As make format unstages your files post-formatting, you must stage the files again once you have verified that make format has done the appropriate formatting, before committing the files.
- See Our git workflow to get a quick overview of our workflow.
- See Creating, Reviewing and Merging Pull Requests on how to create a Pull request.

## 5.3 Project structure

Ginkgo is divided into a core module with common functionalities independent of the architecture, and several kernel modules (reference, omp, cuda, hip, dpcpp) which contain low-level computational routines for each supported architecture.

### 5.3.1 Extended header files

Some header files from the core module have to be extended to include special functionality for specific architectures. An example of this is core/base/math.hpp, which has a GPU counterpart in cuda/base/math.↩ hpp. For such files you should always include the version from the module you are working on, and this file will internally include its core counterpart.

### 5.3.2 Using library classes

You can use and call functions of existing classes inside a kernel (that are defined and not just declared in a header file), however, you are not allowed to create new instances of a polymorphic class inside a kernel (or in general inside any kernel module like cuda/hip/omp/reference) as this creates circular dependencies between the `core` and the backend library. With this in mind, our CI contains a job which checks if such a circular dependency exists. These checks can be run manually using the `-DGINKGO_CHECK_CIRCULAR_DEPS=ON` option in the CMake configuration.

For example, when creating a new matrix class `AB` by combining existing classes `A` and `B`, the `AB::apply()` function composed of invocations to `A::apply()` and `B::apply()` can only be defined in the core module, it is not possible to create instances of `A` and `B` inside the `AB` kernel files. This is to avoid the aforementioned circular dependency issue. An example for such a class is the `Hybrid` matrix format, which uses the `apply()` of the `Ell` and `Coo` matrix formats. Nevertheless, it is possible to call the kernels themselves directly within the same executor. For example, `cuda::dense::add_scaled()` can be called from any other `cuda` kernel.

## 5.4 Git related

Ginkgo uses git, the distributed version control system to track code changes and coordinate work among its developers. A general guide to git can be found in its extensive documentation.

### 5.4.1 Our git workflow

In Ginkgo, we prioritize keeping a clean history over accurate tracking of commits. `git rebase` is hence our command of choice to make sure that we have a nice and linear history, especially for pulling the latest changes from the `develop` branch. More importantly, rebasing upon develop is **required** before the commits of the PR are merged into the `develop` branch.

### 5.4.2 Writing good commit messages

With software sustainability and maintainability in mind, it is important to write commit messages that are short, clear and informative. Ideally, this would be the format to prefer:

```
Summary of the changes in a sentence, max 50 chars.
More detailed comments:
+ Changes that have been added.
- Changes that been removed.
Related PR: https://github.com/ginkgo-project/ginkgo/pull/<PR-number>
```

You can refer to this informative guide for more details.

#### 5.4.2.1 Attributing credit

Git has a nice feature where it allows you to add a co-author for your commit, if you would like to attribute credits for the changes made in the commit. This can be done by:

```
Commit message.
Co-authored-by:  Name <email@domain>
```

In the Ginkgo commit history, this is most common associated with suggested improvements from code reviews.

### 5.4.3 Creating, Reviewing and Merging Pull Requests

- The `develop` branch is the default branch to submit PR's to. From time to time, we merge the `develop` branch to the `main` branch and create tags on the `main` to create new releases of Ginkgo. Therefore, all pull requests must be merged into `develop`.
- Please have a look at the labels and make sure to add the relevant labels.
- You can mark the PR as a `WIP` if you are still working on it, `Ready for Review` when it is ready for others to review it.
- Assignees to the PR should be the ones responsible for merging that PR. Currently, it is only possible to assign members within the `ginkgo-project`.
- Each pull request requires at least two approvals before merging.
- PR's created from within the repository will automatically trigger two CI pipelines on pushing to the branch from the which the PR has been created. The Github Actions pipeline tests our framework on Mac OSX and on Windows platforms. Another comprehensive Linux based pipeline is run from a `mirror on gitlab` and contains additional checks like static analysis and test coverage.
- By default, the Github Actions pipeline only runs a reduced set of jobs on pull requests. Adding the label `1:ST:run-full-test` or `1:ST:ready-to-merge` starts the full set of jobs, as does every following push to the branch while one of these labels is present.
- Once a PR has been approved and the build has passed, one of the reviewers can mark the PR as `READY TO MERGE`. At this point the creator/assignee of the PR *needs to* verify that the branch is up to date with `develop` and rebase it on `develop` if it is not.

## 5.5 Code style

### 5.5.1 Automatic code formatting

Ginkgo uses `pre-commit` to automatically apply code formatting when committing changes to git. What formatting is applied is managed through `ClangFormat` with a custom `.clang-format` configuration file (mostly based on ClangFormat's *Google* style). **Make sure you have pre-commit set up and running properly** before committing anything that will end up in a pull request against `ginkgo-project/ginkgo` repository. In addition, you should **never** modify the `.clang-format` configuration file shipped with Ginkgo. E.g. if ClangFormat has trouble reading this file on your system, you should install a newer version of ClangFormat, and avoid commenting out parts of the configuration file.

ClangFormat is the primary tool that helps us achieve a uniform look of Ginkgo's codebase, while reducing the learning curve of potential contributors. However, ClangFormat configuration is not expressive enough to incorporate the entire coding style, so there are several additional rules that all contributed code should follow.

*Note*: To learn more about how ClangFormat will format your code, see existing files in Ginkgo, `.clang-format` configuration file shipped with Ginkgo, and ClangFormat's documentation.

### 5.5.2 Naming scheme

#### 5.5.2.1 Filenames

Filenames use `snake_case` and use the following extensions:

- C++ source files: `.cpp`
- C++ header files: `.hpp`
- CUDA source files: `.cu`
- CUDA header files: `.cuh`
- HIP source files: `.hip.cpp`
- HIP header files: `.hip.hpp`
- Common source files used by both CUDA and HIP: `.hpp.inc`
- CMake utility files: `.cmake`
- Shell scripts: `.sh`

*Note:* A C++ source/header file is considered a `CUDA` file if it contains CUDA code that is not guarded with `#if` guards that disable this code in non-CUDA compilers. I.e. if a file can be compiled by a general C++ compiler, it is not considered a CUDA file.

#### 5.5.2.2 Macros

Macros (both object-like and function-like macros) use `CAPITAL_CASE`. They have to start with `GKO_` to avoid name clashes (even if they are `#undef`-ed in the same file!).

#### 5.5.2.3 Variables

Variables use `snake_case`.

#### 5.5.2.4 Constants

Constants use `snake_case`.

#### 5.5.2.5 Functions

Functions use `snake_case`.

#### 5.5.2.6 Structures and classes

Structures and classes which do not experience polymorphic behavior (i.e. do not contain virtual methods, nor members which experience polymorphic behavior) use `snake_case`.

All other structures and classes use `CamelCase`.

#### 5.5.2.7 Members

All structure / class members use the same naming scheme as they would if they were not members:

- methods use the naming scheme for functions
- data members the naming scheme for variables or constants
- type members for classes / structures

Additionally, non-public data members end with an underscore (_).

#### 5.5.2.8 Namespaces

Namespaces use `snake_case`.

#### 5.5.2.9 Template parameters

- Type template parameters use `CamelCase`, for example `ValueType`.
- Non-type template parameters use `snake_case`, for example `subwarp_size`.

### 5.5.3 Whitespace

Spaces and tabs are handled by ClangFormat, but blank lines are only partially handled (the current configuration doesn't allow for more than 2 blank lines). Thus, contributors should be aware of the following rules for blank lines:

1. Top-level statements and statements directly within namespaces are separated with 2 blank lines. The first / last statement of a namespace is separated by two blank lines from the opening / closing brace of the namespace.
   (a) *exception*: if the first **or** the last statement in the namespace is another namespace, then no blank lines are required *example*: ```c++ namespace foo {

```
struct x {
};


}  // namespace foo


namespace bar {
namespace baz {


void f();


}  // namespace baz
}  // namespace bar
```

2. _exception_: in header files whose only purpose is to _declare_ a bunch of functions (e.g. the `*_kernel.hpp` files) these declarations can be separated by only 1 blank line (note: standard rules apply for all other statements that might be present in that file)

```
3.  _exception_: "related" statement can have 1 blank line between them.
    "Related" is not a strictly defined adjective in this sense, but is in
    general one of:

    1.  overload of a same function,
    2.  function / class template and it's specializations,
    3.  macro that modifies the meaning or adds functionality to the
        previous / following statement.

    However, simply calling function `f` from function `g` does not imply
    that `f` and `g` are "related".
```

1. Statements within structures / classes are separated with 1 blank line. There are no blank lines between the first / last statement in the structure / class.

   (a) *exception*: there is no blank line between an access modifier (`private`, `protected`, `public`) and the following statement. *example*: ```c++ class foo { public: int get_x() const noexcept { return x_; }
   int &get_x() noexcept { return x_; }
   private: int x_; }; ```

2. Function bodies cannot have multiple consecutive blank lines, and a single blank line can only appear between two logical sections of the function.

3. Unit tests should follow the `AAA` pattern, and a single blank line must appear between consecutive "A" sections. No other blank lines are allowed in unit tests.

4. Enumeration definitions should have no blank lines between consecutive enumerators.

### 5.5.4 Include statement grouping

The concrete ordering will be done by `clang-format`. Here are the rules that `clang-format` will follow. In general, all include statements should be present on the top of the file, ordered in the following groups, with *one* blank lines between each group:

1. Main header file (e.g. `core/foo/bar.hpp` included in `core/foo/bar.cpp`, or in the unit test`core/test/foo/bar.cpp`)
2. Standard library headers (e.g. `vector`)
3. Executor specific library headers (e.g. `omp.h`)
4. System third-party library headers (e.g. `papi.h`)
5. Public Ginkgo headers
6. Local headers

*Example*: A file `core/base/my_file.cpp` might have an include list like this:

```
 {c++}
#include "ginkgo/core/base/my_file.hpp"
#include <algorithm>
#include <vector>
#include <tuple>
#include <omp.h>
#include <papi.h>
#include <third_party/blas/cblas.hpp>
#include <third_party/lapack/lapack.hpp>
#include <ginkgo/core/base/executor.hpp>
#include <ginkgo/core/base/lin_op.hpp>
#include <ginkgo/core/base/types.hpp>
#include "core/base/my_file_kernels.hpp"
```

#### 5.5.4.1 Main header

This section presents the handling of the main header attributed to a file. For a given file, the main header is the header that contains the declarations of the functions, classes, etc., which are implemented in this file. In the previous example, this would be `#include "ginkgo/core/base/my_file.hpp"`. The `clang-format` tool figures out the main header. The only intervention form a contributor is to *always* include the main header using `"..."`.

Please note that this only applies to implementation files, so files ending in `.cpp` or `.cu`.

#### 5.5.4.2 Some general comments.

1. Private headers of Ginkgo should not be included within the public Ginkgo header.
2. It is a good idea to keep the headers self-sufficient, See `Google Style guide for reasoning`. When compiling with `GINKGO_CHECK_CIRCULAR_DEPS` enabled, this property is explicitly checked.
3. The recommendations of the `iwyu` (Include what you use) tool can be used to make sure that the headers are self-sufficient and that the compiled files ( `.cu`, `.cpp`, `.hip.cpp` ) include only what they use. Please be aware that this tool can be incorrect in some cases.

### 5.5.5 Other Code Formatting not handled by ClangFormat

#### 5.5.5.1 Control flow constructs

Single line statements should be avoided in all cases. Use of brackets is mandatory for all control flow constructs (e.g. `if`, `for`, `while`, ...).

#### 5.5.5.2 Variable declarations

C++ supports declaring / defining multiple variables using a single *type-specifier*. However, this is often very confusing as references and pointers exhibit strange behavior:

```
 {c++}
template <typename T> using pointer = T *;
int *        x, y;  // x is a pointer, y is not
pointer<int> x, y;  // both x and y are pointers
```

For this reason, **always** declare each variable on a separate line, with its own *type-specifier*.

### 5.5.6 CMake coding style

#### 5.5.6.1 Whitespaces

All alignment in CMake files should use four spaces.

#### 5.5.6.2 Use of macros vs functions

Macros in CMake do not have a scope. This means that any variable set in this macro will be available to the whole project. In contrast, functions in CMake have local scope and therefore all set variables are local only. In general, wrap all piece of algorithms using temporary variables in a function and use macros to propagate variables to the whole project.

#### 5.5.6.3 Naming style

All Ginkgo specific variables should be prefixed with a `GINKGO_` and all functions by `ginkgo_`.

## 5.6 Helper scripts

To facilitate easy development within Ginkgo and to encourage coders and scientists who do not want get bogged down by the details of the Ginkgo library, but rather focus on writing the algorithms and the kernels, Ginkgo provides the developers with a few helper scripts.

### 5.6.1 Create a new algorithm

A `create_new_algorithm.sh` script is available for developers to facilitate easy addition of new algorithms. The options it provides can be queried with

```
./create_new_algorithm.sh --help
```

The main objective of this script is to add files and boiler plate code for the new algorithm using a model and an instance of that model. For example, models can be any one of `factorization`, `matrix`, `preconditioner` or `solver`. For example to create a new solver named `my_solver` similar to `gmres`, you would set the `ModelType` to `solver` and set the `ModelName` to `gmres`. This would duplicate the core algorithm and kernels of the `gmres` algorithm and replace the naming to `my_solver`. Additionally, all the kernels of the new `my_solver` are marked as `GKO_NOT_IMPLEMENTED`. For easy navigation and `.txt` file is created in the folder where the script is run, which lists all the TODO's. These TODO's can also be found in the corresponding files.

### 5.6.2 Converting CUDA code to HIP code

We provide a `cuda2hip` script that converts `cuda` kernel code into `hip` kernel code. Internally, this script calls the `hipify script` provided by HIP, converting the CUDA syntax to HIP syntax. Additionally, it also automatically replaces the instances of CUDA with HIP as appropriate. Hence, this script can be called on a Ginkgo CUDA file. You can find this script in the `dev_tools/scripts/` folder.

## 5.7 Writing Tests

Ginkgo uses the `GTest framework` for the unit test framework within Ginkgo. Writing good tests are extremely important to verify the functionality of the new code and to make sure that none of the existing code has been broken.

### 5.7.1 Testing know-how

- GTest provides a `comprehensive documentation` of the functionality available within Gtest.
- Reduce code duplication with `Testing Fixtures, TEST_F`
- Write templated tests using `TYPED_TEST`.

### 5.7.2 Some general rules.

- Unit tests must follow the KISS principle.
- Unit tests must follow the AAA pattern, and a single blank line must appear between consecutive "A" sections.

### 5.7.3 Writing tests for kernels

- Reference kernels, kernels on the `ReferenceExecutor`, are meant to be single threaded reference implementations. Therefore, tests for reference kernels need to be performed with data that can be as small as possible. For example, matrices lesser than 5x5 are acceptable. This allows the reviewers to verify the results for exactness with tools such as MATLAB.
- OpenMP, CUDA, HIP and DPC++ kernels have to be tested against the reference kernels. Hence data for the tests of these kernels can be generated in the test files using helper functions or by using external files to be read through the standard input. In particular for CUDA, HIP and DPC++ the data size should be at least bigger than the architecture's warp size to ensure there is no corner case in the kernels.

## 5.8 Documentation style

Documentation uses standard Doxygen.

### 5.8.1 Command prefix: <tt>@</tt> for Doxygen, <tt>\</tt> for math

Doxygen accepts both `\cmd` and `@cmd` and treats them identically. Ginkgo uses the at-sign for all Doxygen commands (`@brief`, `@param`, `@tparam`, `@return`, `@note`, `@ingroup`, `@ref`, ...) and reserves the backslash for the LaTeX formula delimiters `\f$ ... \f$` and `\f[ ... \f]`.

Keeping the two apart means that the prefix tells you what you are looking at: `@` introduces Doxygen structure, `\` introduces LaTeX. It also keeps a formula visually consistent with the math it contains, as in `\f$\alpha \frac{a}{b}\f$`.

### 5.8.2 Developer targeted notes

Make use of `@internal` doxygen tag. This can be used for any comment which is not intended for users, but is useful to better understand a piece of code.

### 5.8.3 Whitespaces

#### 5.8.3.1 After named tags such as <tt>@param foo</tt>

The documentation tags which use an additional name should be followed by two spaces in order to better distinguish the text from the doxygen tag. It is also possible to use a line break instead.

### 5.8.4 Documenting examples

There are two main steps:

1. First, you can just copy over the doc/ folder (you can copy it from the example most relevant to you) and adapt your example names and such, then you can modify the actual documentation.

- In tooltip: A short description of the example.
- In short-intro: The name of the example.
- In results.dox: Run the example and write the output you get.
- In kind: The kind of the example. For different kinds see the documentation. Examples can be of basic, techniques, logging, stopping_criteria or preconditioners. If your example does not fit any of these categories, feel free to create one.
- In intro.dox: You write an explanation of your code with some introduction similar to what you see in an existing example most relevant to you.
- In builds-on: You write the examples it builds on.

1. You also need to modify the examples.hpp.in file. You add the name of the example in the main section and in the section that you specified in the doc/kind file in the example documentation.

## 5.9 Other programming comments

### 5.9.1 C++ standard stream objects

These are global objects and are shared inside the same translation unit. Therefore, whenever its state or formatting is changed (e.g. using std::hex or floating point formatting) inside library code, make sure to restore the state before returning the control to the user. See this stackoverflow question for examples on how to do it correctly. This is extremely important for header files.

### 5.9.2 Warnings

By default, the -DCMAKE_CXX_FLAGS should be set to -Wpedantic to emit pedantic warnings by default. The CI system additionally also has a step where it compiles for pedantic warnings to be errors.

### 5.9.3 Avoiding circular dependencies

To facilitate finding circular dependencies issues (see Using library classes for more details), a CI step no-circular-deps was created. For more details on its usage, see this pipeline, where Ginkgo did not abide to this policy and PR #278 which fixed this. Note that doing so is not enough to guarantee with 100% accuracy that no circular dependency is present. For an example of such a case, take a look at this pipeline where one of the compiler setups detected an incorrect dependency of the cuda module (due to jacobi) on the core module.

# Chapter 6

# Citing Ginkgo

The main Ginkgo paper describing Ginkgo's purpose, design and interface is available through the following reference:

```
@article{ginkgo-toms-2022,
title = {{Ginkgo:  A Modern Linear Operator Algebra Framework for High Performance Computing}},
volume = {48},
copyright = {All rights reserved},
issn = {0098-3500},
shorttitle = {Ginkgo},
url = {https://doi.org/10.1145/3480935},
doi = {10.1145/3480935},
number = {1},
urldate = {2022-02-17},
journal = {ACM Transactions on Mathematical Software},
author = {Anzt, Hartwig and Cojean, Terry and Flegar, Goran and Göbel, Fritz and Grützmacher, Thomas and
      Nayak, Pratik and Ribizel, Tobias and Tsai, Yuhsiang Mike and Quintana-Ortí, Enrique S.},
month = feb,
year = {2022},
keywords = {ginkgo, healthy software lifecycle, High performance computing, multi-core and manycore
      architectures},
pages = {2:1--2:33}
}
```

Multiple topical papers exist on Ginkgo and its algorithms. The following papers can be used to cite specific aspects of the Ginkgo project.

### 6.0.1 The Ginkgo Software

The Ginkgo software itself was reviewed and has a paper published in the Journal of Open Source Software, which can be cited with the following reference:

```
@article{GinkgoJoss2020,
  doi = {10.21105/joss.02260},
  url = {https://doi.org/10.21105/joss.02260},
  year = {2020},
  publisher = {The Open Journal},
  volume = {5},
  number = {52},
  pages = {2260},
  author = {Hartwig Anzt and Terry Cojean and Yen-Chen Chen and Goran Flegar and Fritz G\"{o}bel and Thomas
      Gr\"{u}tzmacher and Pratik Nayak and Tobias Ribizel and Yu-Hsiang Tsai},
  title = {Ginkgo:  A high performance numerical linear algebra library},
  journal = {Journal of Open Source Software}
}
```

### 6.0.2 On Portability

```
@misc{tsai2020amdportability,
    title={Preparing Ginkgo for AMD GPUs -- A Testimonial on Porting CUDA Code to HIP},
    author={Yuhsiang M. Tsai and Terry Cojean and Tobias Ribizel and Hartwig Anzt},
    year={2020},
    eprint={2006.14290},
    archivePrefix={arXiv},
    primaryClass={cs.MS}
}
```

### 6.0.3 On Software Sustainability

```
@inproceedings{anzt2019pasccb,
author = {Anzt, Hartwig and Chen, Yen-Chen and Cojean, Terry and Dongarra, Jack and Flegar, Goran and Nayak,
      Pratik and Quintana-Ort\'{\i}, Enrique S. and Tsai, Yuhsiang M. and Wang, Weichung},
title = {Towards Continuous Benchmarking:  An Automated Performance Evaluation Framework for High
      Performance Software},
year = {2019},
isbn = {9781450367707},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
url = {https://doi.org/10.1145/3324989.3325719},
doi = {10.1145/3324989.3325719},
booktitle = {Proceedings of the Platform for Advanced Scientific Computing Conference},
articleno = {9},
numpages = {11},
keywords = {interactive performance visualization, healthy software lifecycle, continuous integration,
      automated performance benchmarking},
location = {Zurich, Switzerland},
series = {PASC '19}
}
```

### 6.0.4 On SpMV or solvers performance

```
@InProceedings{tsai2020amdspmv,
author="Tsai, Yuhsiang M.
and Cojean, Terry
and Anzt, Hartwig",
editor="Sadayappan, Ponnuswamy
and Chamberlain, Bradford L.
and Juckeland, Guido
and Ltaief, Hatem",
title="Sparse Linear Algebra on AMD and&amp;nbsp;NVIDIA GPUs -- The Race Is On",
booktitle="High Performance Computing",
year="2020",
publisher="Springer International Publishing",
address="Cham",
pages="309--327",
abstract="Efficiently processing sparse matrices is a central and performance-critical part of many
      scientific simulation codes.  Recognizing the adoption of manycore accelerators in HPC, we evaluate in
      this paper the performance of the currently best sparse matrix-vector product (SpMV) implementations
      on high-end GPUs from AMD and NVIDIA. Specifically, we optimize SpMV kernels for the CSR, COO, ELL,
      and HYB format taking the hardware characteristics of the latest GPU technologies into account.  We
      compare for 2,800 test matrices the performance of our kernels against AMD's hipSPARSE library and
      NVIDIA's cuSPARSE library, and ultimately assess how the GPU technologies from AMD and NVIDIA compare
      in terms of SpMV performance.",
isbn="978-3-030-50743-5"
}
@article{anzt2020spmv,
author = {Anzt, Hartwig and Cojean, Terry and Yen-Chen, Chen and Dongarra, Jack and Flegar, Goran and Nayak,
      Pratik and Tomov, Stanimire and Tsai, Yuhsiang M. and Wang, Weichung},
title = {Load-Balancing Sparse Matrix Vector Product Kernels on GPUs},
year = {2020},
issue_date = {March 2020},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
volume = {7},
number = {1},
issn = {2329-4949},
url = {https://doi.org/10.1145/3380930},
doi = {10.1145/3380930},
journal = {ACM Trans.  Parallel Comput.},
month = mar,
articleno = {2},
numpages = {26},
keywords = {irregular matrices, GPUs, Sparse Matrix Vector Product (SpMV)}
}
@misc{tsai2020evaluating,
    title={Evaluating the Performance of NVIDIA's A100 Ampere GPU for Sparse Linear Algebra Computations},
    author={Yuhsiang Mike Tsai and Terry Cojean and Hartwig Anzt},
    year={2020},
    eprint={2008.08478},
    archivePrefix={arXiv},
    primaryClass={cs.MS}
}
```

### 6.0.5 On the Python bindings

The pyGinkgo Python bindings are described in the following reference:

```
@inproceedings{pyginkgo-icpp-2025,
author = {Tuteja, Keshvi and Olenik, Gregor and Mishchuk, Roman and Tsai, Yu-Hsiang and G\"{o}tz, Markus and
      Streit, Achim and Anzt, Hartwig and Debus, Charlotte},
title = {{pyGinkgo:  A Sparse Linear Algebra Operator Framework for Python}},
year = {2025},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
url = {https://doi.org/10.1145/3754598.3754648},
doi = {10.1145/3754598.3754648},
booktitle = {Proceedings of the 54th International Conference on Parallel Processing},
pages = {753--763},
location = {San Diego, CA, USA},
series = {ICPP '25}
}
```

# Chapter 7

# Example programs

Here you can find example programs that demonstrate the usage of Ginkgo. Some examples are built on one another and some are stand-alone and demonstrate a concept of Ginkgo, which can be used in your own code.

You can browse the available example programs

1. as **a graph** that shows how example programs build upon each other.
2. as **a list** that provides a short synopsis of each program.
3. or **grouped by topic**.

By default, all Ginkgo examples are built using CMake.

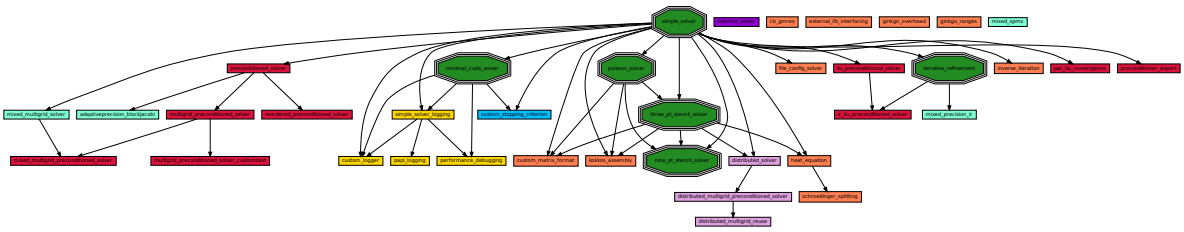
An example for building the examples and using Ginkgo as an external library without CMake can be found in the script provided for each example, which should be called with the form: `./build.sh PATH_TO_GINKGO_BUILD_DIR`

By default, Ginkgo is compiled with at least `-DGINKGO_BUILD_REFERENCE=ON`. Ginkgo also tries to detect your environment setup (presence of CUDA, ...) to enable the relevant accelerator modules. If you want to target a specific GPU, make sure that Ginkgo is compiled with the accelerator specific module enabled, such as:

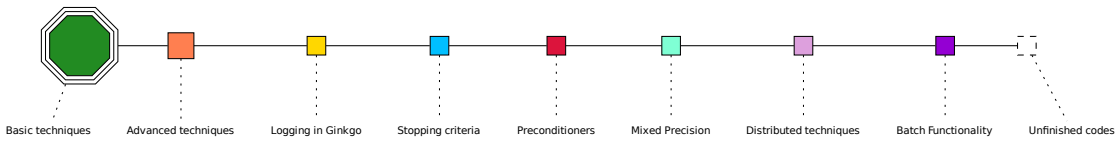
1. `-DGINKGO_BUILD_CUDA=ON` option for NVIDIA GPUs.
2. `-DGINKGO_BUILD_HIP=ON` option for AMD or NVIDIA GPUs.
3. `-DGINKGO_BUILD_SYCL=ON` option for Intel GPUs (and possibly any other platform).

### Connections between example programs

The following graph shows the connections between example programs and how they build on each other. Click on any of the boxes to go to one of the programs. If you hover your mouse pointer over a box, a brief description of the program should appear.



**Legend:**




## Example programs

| | |
|---|---|
| The simple-solver program | A minimal CG solver in Ginkgo, which reads a matrix from a file. |
| The minimal-cuda-solver program | A minimal solver on the CUDA executor than can be run on N↩ VIDIA GPU's. |
| The poisson-solver program | Solve an actual physically relevant problem, the poisson problem. The matrix is generated within Ginkgo. |
| The preconditioned-solver program | Using a Jacobi preconditioner to solve a linear system. |
| The ilu-preconditioned-solver program | Using an ILU preconditioner to solve a linear system. |
| The performance-debugging program | Using Loggers to debug the performance within Ginkgo. |
| The three-pt-stencil-solver program | Using a three point stencil to solve the poisson equation with array views. |
| The nine-pt-stencil-solver program | Using a nine point 2D stencil to solve the poisson equation with array views. |
| The external-lib-interfacing program | Using Ginkgo's solver with the external library deal.II. |
| The custom-logger program | Creating a custom logger specifically for comparing the recurrent and the real residual norms. |
| The custom-matrix-format program | Creating a matrix-free stencil solver by using Ginkgo's advanced methods to build your own custom matrix format. |
| The inverse-iteration program | Using Ginkgo to compute eigenvalues of a matrix with the inverse iteration method. |
| The simple-solver-logging program | Using the logging functionality in Ginkgo to get solver and other information to diagnose and debug your code. |
| The papi-logging program | Using the PAPI logging library in Ginkgo to get advanced information about your code and its behaviour. |
| The ginkgo-overhead program | Measuring the overhead of the Ginkgo library. |
| The custom-stopping-criterion program | Creating a custom stopping criterion for the iterative solution process. |
| The ginkgo-ranges program | Using the ranges concept to factorize a matrix with the LU factorization. |

| | |
|---|---|
| The mixed-spmv program | Shows the Ginkgo mixed precision spmv functionality. |
| The mixed-precision-ir program | Manual implementation of a Mixed Precision Iterative Refinement (MPIR) solver. |
| The adaptiveprecision-blockjacobi program | Shows how to use the adaptive precision block-Jacobi preconditioner. |
| The cb-gmres program | Using the Ginkgo CB-GMRES solver (Compressed Basis GM↩ RES). |
| The heat-equation program | Solving a 2D heat equation and showing matrix assembly, vector initialization and solver setup in a more complex setting with output visualization. |
| The iterative-refinement program | Using a low accuracy CG solver as an inner solver to an iterative refinement (IR) method which solves a linear system. |
| The ir-ilu-preconditioned-solver program | Combining iterative refinement with the adaptive precision block-Jacobi preconditioner to approximate triangular systems occurring in ILU preconditioning. |
| The par-ilu-convergence program | Convergence analysis at the examples of parallel incomplete factorization solver. |
| The preconditioner-export program | Explicit generation and storage of preconditioners for given matrices. |
| The multigrid-preconditioned-solver program | Use multigrid as preconditioner to a solver. |
| The mixed-multigrid-solver program | Use multigrid with different precision multigrid_level as a solver. |
| The distributed-solver program | Use a distributed solver to solve a 1D Laplace equation. |

**Example programs grouped by topics**

| | |
|---|---|
| Solving a simple linear system with choice of executors | The simple-solver program |
| Debug the performance of a solver or preconditioner | The performance-debugging program<br>The preconditioner-export program |
| Using the CUDA executor | The minimal-cuda-solver program |
| Using preconditioners | The preconditioned-solver program,<br>The ilu-preconditioned-solver program,<br>The ir-ilu-preconditioned-solver program,<br>The adaptiveprecision-blockjacobi program,<br>The par-ilu-convergence program,<br>The preconditioner-export program<br>The multigrid-preconditioned-solver program |
| Iterative refinement | The iterative-refinement program,<br>The mixed-precision-ir program,<br>The ir-ilu-preconditioned-solver program |
| Solving a physically relevant problem | The poisson-solver program,<br>The three-pt-stencil-solver program,<br>The nine-pt-stencil-solver program,<br>The custom-matrix-format program |

| | |
|---|---|
| Reading in a matrix and right hand side from a file | The simple-solver program,<br>The minimal-cuda-solver program,<br>The preconditioned-solver program,<br>The ilu-preconditioned-solver program,<br>The inverse-iteration program,<br>The simple-solver-logging program,<br>The papi-logging program,<br>The custom-stopping-criterion program,<br>The custom-logger program |

**Basic techniques**

| | |
|---|---|
| Using Ginkgo with external libraries | The external-lib-interfacing program |
| Customizing Ginkgo | The custom-logger program,<br>The custom-stopping-criterion program,<br>The custom-matrix-format program |
| Writing your own matrix format | The custom-matrix-format program |
| Using Ginkgo to construct more complex linear algebra routines | The inverse-iteration program |
| Logging within Ginkgo | The simple-solver-logging program,<br>The papi-logging program,<br>The performance-debugging program<br>The custom-logger program |
| Constructing your own stopping criterion | The custom-stopping-criterion program |
| Using ranges in Ginkgo | The ginkgo-ranges program |
| Mixed precision | The mixed-spmv program,<br>The mixed-precision-ir program,<br>The adaptiveprecision-blockjacobi program<br>The mixed-multigrid-solver program |
| Multigrid | The multigrid-preconditioned-solver program<br>The mixed-multigrid-solver program |
| Configure a solver from a config file | The file-config-solver program |
| Distributed | The distributed-solver program |

**Advanced techniques**

# Chapter 8

# The adaptiveprecision-blockjacobi program

The preconditioned solver example..

This example depends on preconditioned-solver.

## This example shows how to use the adaptive precision block-Jacobi preconditioner.

**In this example, we first read in a matrix from file, then generate a right-hand side and an initial guess. The preconditioned CG solver is enhanced with a block-Jacobi preconditioner that optimizes the storage format for the distinct inverted diagonal blocks to the numerical requirements. The example features the iteration count and runtime of the CG solver.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
```

```
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

Create RHS and initial guess as 1

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 1.;
}
auto x = gko::clone(exec, host_x);
auto b = gko::clone(exec, host_x);
```

Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<real_vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

copy b again

```
b->copy_from(host_x);
```

Create solver factory

```
const RealValueType reduction_factor = 1e-7;
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(10000u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
```

Add preconditioner, these 2 lines are the only difference from the simple solver example

```
.with_preconditioner(
    bj::build().with_max_block_size(16u).with_storage_optimization(
        gko::precision_reduction::autodetect()))
.on(exec);
```

Create solver

```
std::shared_ptr<const gko::log::Convergence<ValueType» logger =
    gko::log::Convergence<ValueType>::create();
solver_gen->add_logger(logger);
auto solver = solver_gen->generate(A);
```

Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

Get residual

```
auto res = gko::as<real_vec>(logger->get_residual_norm());
auto impl_res = gko::as<real_vec>(logger->get_implicit_sq_resnorm());
std::cout « "Initial residual norm sqrt(r^T r):\n";
write(std::cout, initres);
std::cout « "Final residual norm sqrt(r^T r):\n";
write(std::cout, res);
std::cout « "Implicit residual norm squared (r^2):\n";
write(std::cout, impl_res);
```

Print solver statistics

```
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
194.679
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
5.69384e-06
Implicit residual norm squared (r^2):
%%MatrixMarket matrix array real general
1 1
1.27043e-15
CG iteration count:          5
CG execution time [ms]:  0.080041
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    gko::size_type size = A->get_size()[0];
    auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    for (auto i = 0; i < size; i++) {
        host_x->at(i, 0) = 1.;
    }
    auto x = gko::clone(exec, host_x);
    auto b = gko::clone(exec, host_x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto initres = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(initres);
    b->copy_from(host_x);
    const RealValueType reduction_factor = 1e-7;
    auto solver_gen =
```

```
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(10000u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .with_preconditioner(
                bj::build().with_max_block_size(16u).with_storage_optimization(
                    gko::precision_reduction::autodetect()))
            .on(exec);
    std::shared_ptr<const gko::log::Convergence<ValueType» logger =
        gko::log::Convergence<ValueType>::create();
    solver_gen->add_logger(logger);
    auto solver = solver_gen->generate(A);
    exec->synchronize();
    std::chrono::nanoseconds time(0);
    auto tic = std::chrono::steady_clock::now();
    solver->apply(b, x);
    auto toc = std::chrono::steady_clock::now();
    time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
    auto res = gko::as<real_vec>(logger->get_residual_norm());
    auto impl_res = gko::as<real_vec>(logger->get_implicit_sq_resnorm());
    std::cout « "Initial residual norm sqrt(r^T r):\n";
    write(std::cout, initres);
    std::cout « "Final residual norm sqrt(r^T r):\n";
    write(std::cout, res);
    std::cout « "Implicit residual norm squared (r^2):\n";
    write(std::cout, impl_res);
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
}
```

# Chapter 9

# The batched-solver program

Using and interfacing with a batched solver..

## Using batched solvers

This example shows how to use Ginkgo batched solvers with data coming from an application. The "application" in this case is just a function in the example itself; nevertheless, the steps to be taken are shown.

A "batch" here means a set of small linear systems that can be solved independently, but each system is too small to use an entire computing device. A requirement is that all the systems need to have the same sparsity pattern.

## The commented program

### Include files

```
#include <fstream>
#include <iostream>
#include <map>
#include <random>
#include <string>
```

This is the main ginkgo header file.

```
#include <ginkgo/ginkgo.hpp>
```

### Type aliases for convenience

Use some shortcuts.

```
using value_type = double;
using real_type = gko::remove_complex<value_type>;
using index_type = int;
using size_type = gko::size_type;
using vec_type = gko::batch::MultiVector<value_type>;
using real_vec_type = gko::batch::MultiVector<real_type>;
using mtx_type = gko::batch::matrix::Csr<value_type, index_type>;
using bicgstab = gko::batch::solver::Bicgstab<value_type>;
```

### Convenience functions

Unbatch batch items into distinct Ginkgo types

```
namespace detail {
template <typename InputType>
auto unbatch(const InputType* batch_object)
{
    auto unbatched_mats =
        std::vector<std::unique_ptr<typename InputType::unbatch_type»{};
    for (size_type b = 0; b < batch_object->get_num_batch_items(); ++b) {
        unbatched_mats.emplace_back(
            batch_object->create_const_view_for_item(b)->clone());
    }
    return unbatched_mats;
}
}  // namespace detail
```

### 'Application' structures and functions

Structure to simulate application data related to the linear systems to be solved.

We use raw pointers below to demonstrate how to handle the situation when the application only gives us raw pointers. Ideally, one should use Ginkgo's gko::array class here. In this example, we assume that the data is in a format that can directly be given to a batch::matrix::Csr object.

```
struct ApplSysData {
```

Number of small systems in the batch.

```
size_type nsystems;
```

Number of rows in each system.

```
int nrows;
```

Number of non-zeros in each system matrix.

```
int nnz;
```

Row pointers for one matrix

```
const index_type* row_ptrs;
```

Column indices of non-zeros for one matrix

```
const index_type* col_idxs;
```

Nonzero values for all matrices in the batch, concatenated

```
const value_type* all_values;
```

RHS vectors for all systems in the batch, concatenated

```
    const value_type* all_rhs;
};
/ *
 * Generates a batch of tridiagonal systems.
 *
 * @param nrows  Number of rows in each system.
 * @param nsystems  Number of systems in the batch.
 * @param exec  The device executor to use for the solver.
 * @note  Normally, the application may not deal with Ginkgo executors, nor do
 * we need it to.  Here, we use the executor for backend-independent device
 * memory allocation.  The application, for example, might assume Hip (for AMD
 * GPUs) and use `hipMalloc` directly.
 * /
ApplSysData appl_generate_system(const int nrows, const size_type nsystems,
                                 std::shared_ptr<gko::Executor> exec);
```

Deallocate application data.

```
void appl_clean_up(ApplSysData& appl_data, std::shared_ptr<gko::Executor> exec);
int main(int argc, char* argv[])
{
```

Print ginkgo version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0]
              « " [executor] [num_systems] [num_rows] [print_residuals] "
                "[num_reps]"
              « std::endl;
    std::exit(-1);
}
```

### Where do you want to run your solver ?

The gko::Executor class is one of the cornerstones of Ginkgo. Currently, we have support for an gko::OmpExecutor, which uses OpenMP multi-threading in most of its kernels, a gko::ReferenceExecutor, a single threaded specialization of the OpenMP executor, gko::CudaExecutor, gko::HipExecutor, gko::DpcppExecutor which runs the code on a NVIDIA, AMD and Intel GPUs, respectively.

#### Note

With the help of C++, you see that you only ever need to change the executor and all the other functions/routines within Ginkgo should automatically work and run on the executor with any other changes.

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()>
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
const size_type num_systems = argc >= 3 ?  std::atoi(argv[2]) :  2;
const int num_rows = argc >= 4 ?  std::atoi(argv[3]) :  32;  // per system
```

Whether to print the residuals or not.

```
const bool print_residuals =
    argc >= 5 ?  (std::string(argv[4]) == "true") :  false;
```

The number of repetitions for the timing.

```
const int num_reps = argc >= 6 ?  std::atoi(argv[5]) :  20;
```

### Generate data

The "application" generates the batch of linear systems on the device

```
auto appl_sys = appl_generate_system(num_rows, num_systems, exec);
```

Create batch_dim object to describe the dimensions of the batch matrix.

```
auto batch_mat_size =
    gko::batch_dim<2>(num_systems, gko::dim<2>(num_rows, num_rows));
auto batch_vec_size =
    gko::batch_dim<2>(num_systems, gko::dim<2>(num_rows, 1));
```

### Use of application-allocated memory

We can either work on the existing memory allocated in the application, or we can copy it for the linear solve. Ginkgo expects the nonzero values for all the small matrices to be allocated contiguously, one matrix after the other.

```
auto vals_view = gko::array<value_type>::const_view(
    exec, num_systems * appl_sys.nnz, appl_sys.all_values);
auto rowptrs_view = gko::array<index_type>::const_view(exec, num_rows + 1,
                                                       appl_sys.row_ptrs);
auto colidxs_view = gko::array<index_type>::const_view(exec, appl_sys.nnz,
                                                       appl_sys.col_idxs);
auto A = gko::share(mtx_type::create_const(
    exec, batch_mat_size, std::move(vals_view), std::move(colidxs_view),
    std::move(rowptrs_view)));
```

### RHS and solution vectors

batch_stride object specifies the access stride within the individual matrices (vectors) in the batch. In this case, we specify a stride of 1 as the common value for all the matrices. Create RHS, again reusing application allocation

```
auto b_view = gko::array<value_type>::const_view(
    exec, num_systems * num_rows, appl_sys.all_rhs);
auto b = vec_type::create_const(exec, batch_vec_size, std::move(b_view));
```

Create initial guess as 0 and copy to device

```
auto x = vec_type::create(exec);
auto host_x = vec_type::create(exec->get_master(), batch_vec_size);
for (size_type isys = 0; isys < num_systems; isys++) {
    for (int irow = 0; irow < num_rows; irow++) {
        host_x->at(isys, irow, 0) = gko::zero<value_type>();
    }
}
x->copy_from(host_x.get());
```

### Create the batch solver factory

```
const real_type reduction_factor{1e-10};
```

Create a batched solver factory with relevant parameters.

```
auto solver =
    bicgstab::build()
        .with_max_iterations(500)
        .with_tolerance(reduction_factor)
        .with_tolerance_type(gko::batch::stop::tolerance_type::relative)
        .on(exec)
        ->generate(A);
```

### Batch logger

Create a logger to obtain the iteration counts and "implicit" residual norms for every system after the solve.

```
std::shared_ptr<const gko::batch::log::BatchConvergence<value_type»
    logger = gko::batch::log::BatchConvergence<value_type>::create();
```

### Generate and solve

add the logger to the solver

```
solver->add_logger(logger);
```

Solve the batch system

```
auto x_clone = gko::clone(x);
```

Warmup

```
for (int i = 0; i < 3; ++i) {
    x_clone->copy_from(x.get());
    solver->apply(b, x_clone);
}
double apply_time = 0.0;
for (int i = 0; i < num_reps; ++i) {
    x_clone->copy_from(x.get());
    exec->synchronize();
    std::chrono::steady_clock::time_point t1 =
        std::chrono::steady_clock::now();
    solver->apply(b, x_clone);
    exec->synchronize();
    std::chrono::steady_clock::time_point t2 =
        std::chrono::steady_clock::now();
    auto time_span =
        std::chrono::duration_cast<std::chrono::duration<double»(t2 - t1);
    apply_time += time_span.count();
}
x->copy_from(x_clone.get());
```

This is not necessary, but one might want to remove the logger before the next solve using the same solver object.

```
solver->remove_logger(logger.get());
```

### Check result

Compute norm of RHS on the device and automatically copy to host

```
auto norm_dim = gko::batch_dim<2>(num_systems, gko::dim<2>(1, 1));
auto host_b_norm = real_vec_type::create(exec->get_master(), norm_dim);
host_b_norm->fill(0.0);
b->compute_norm2(host_b_norm);
```

we need constants on the device

```
auto one = vec_type::create(exec, norm_dim);
one->fill(1.0);
auto neg_one = vec_type::create(exec, norm_dim);
neg_one->fill(-1.0);
```

allocate and compute the residual

```
auto res = vec_type::create(exec, batch_vec_size);
res->copy_from(b);
A->apply(one, x, neg_one, res);
```

allocate and compute residual norm

```
auto host_res_norm = real_vec_type::create(exec->get_master(), norm_dim);
host_res_norm->fill(0.0);
res->compute_norm2(host_res_norm);
auto host_log_resid = gko::make_temporary_clone(
    exec->get_master(), &logger->get_residual_norm());
auto host_log_iters = gko::make_temporary_clone(
    exec->get_master(), &logger->get_num_iterations());
if (print_residuals) {
    std::cout « "Residual norm sqrt(r^T r):\n";
```

"unbatch" converts a batch object into a vector of objects of the corresponding single type, eg. batch::matrix::Dense --> std::vector<Dense>.

```
    auto unb_res = detail::unbatch(host_res_norm.get());
    auto unb_bnorm = detail::unbatch(host_b_norm.get());
    for (size_type i = 0; i < num_systems; ++i) {
        std::cout « " System no.  " « i
                  « ":  residual norm = " « unb_res[i]->at(0, 0)
                  « ", implicit residual norm = "
                  « host_log_resid->get_const_data()[i]
                  « ", iterations = "
                  « host_log_iters->get_const_data()[i] « std::endl;
        const real_type relresnorm =
            unb_res[i]->at(0, 0) / unb_bnorm[i]->at(0, 0);
        if (!(relresnorm <= reduction_factor)) {
            std::cout « "System " « i « " converged only to "
                      « relresnorm « " relative residual." « std::endl;
        }
    }
}
std::cout « "Solver type:  "
          « "batch::bicgstab"
          « "\nMatrix size:  " « A->get_common_size()
          « "\nNum batch entries:  " « A->get_num_batch_items()
          « "\nEntire solve took:  " « apply_time / num_reps « " seconds."
          « std::endl;
```

Ginkgo objects are cleaned up automatically; but the "application" still needs to clean up its data in this case.

```
    appl_clean_up(appl_sys, exec);
    return 0;
}
```

Generate the matrix and the vectors. This function emulates the generation of the data from the application.

```
ApplSysData appl_generate_system(const int nrows, const size_type nsystems,
                                 std::shared_ptr<gko::Executor> exec)
{
    const int nnz = nrows * 3 - 2;
    std::default_random_engine rgen(15);
    std::normal_distribution<real_type> distb(0.5, 0.1);
    std::vector<real_type> spacings(nsystems * nrows);
    std::generate(spacings.begin(), spacings.end(),
                  [&]() { return distb(rgen); });
    std::vector<value_type> allvalues(nnz * nsystems);
    for (size_type isys = 0; isys < nsystems; isys++) {
        allvalues.at(isys * nnz) = 2.0 / spacings.at(isys * nrows);
        allvalues.at(isys * nnz + 1) = -1.0;
        for (int irow = 0; irow < nrows - 2; irow++) {
            allvalues.at(isys * nnz + 2 + irow * 3) = -1.0;
            allvalues.at(isys * nnz + 2 + irow * 3 + 1) =
                2.0 / spacings.at(isys * nrows + irow + 1);
```

```
            allvalues.at(isys * nnz + 2 + irow * 3 + 2) = -1.0;
        }
        allvalues.at(isys * nnz + 2 + (nrows - 2) * 3) = -1.0;
        allvalues.at(isys * nnz + 2 + (nrows - 2) * 3 + 1) =
            2.0 / spacings.at((isys + 1) * nrows - 1);
        assert(isys * nnz + 2 + (nrows - 2) * 3 + 2 == (isys + 1) * nnz);
    }
    std::vector<index_type> rowptrs(nrows + 1);
    rowptrs.at(0) = 0;
    rowptrs.at(1) = 2;
    for (int i = 2; i < nrows; i++) {
        rowptrs.at(i) = rowptrs.at(i - 1) + 3;
    }
    rowptrs.at(nrows) = rowptrs.at(nrows - 1) + 2;
    assert(rowptrs.at(nrows) == nnz);
    std::vector<index_type> colidxs(nnz);
    colidxs.at(0) = 0;
    colidxs.at(1) = 1;
    const int nnz_per_row = 3;
    for (int irow = 1; irow < nrows - 1; irow++) {
        colidxs.at(2 + (irow - 1) * nnz_per_row) = irow - 1;
        colidxs.at(2 + (irow - 1) * nnz_per_row + 1) = irow;
        colidxs.at(2 + (irow - 1) * nnz_per_row + 2) = irow + 1;
    }
    colidxs.at(2 + (nrows - 2) * nnz_per_row) = nrows - 2;
    colidxs.at(2 + (nrows - 2) * nnz_per_row + 1) = nrows - 1;
    assert(2 + (nrows - 2) * nnz_per_row + 1 == nnz - 1);
    std::vector<value_type> allb(nrows * nsystems);
    for (size_type isys = 0; isys < nsystems; isys++) {
        const value_type bval = distb(rgen);
        std::fill(allb.begin() + isys * nrows,
                  allb.begin() + (isys + 1) * nrows, bval);
    }
    index_type* const row_ptrs = exec->alloc<index_type>(nrows + 1);
    exec->copy_from(exec->get_master().get(), static_cast<size_type>(nrows + 1),
                    rowptrs.data(), row_ptrs);
    index_type* const col_idxs = exec->alloc<index_type>(nnz);
    exec->copy_from(exec->get_master().get(), static_cast<size_type>(nnz),
                    colidxs.data(), col_idxs);
    value_type* const all_values = exec->alloc<value_type>(nsystems * nnz);
    exec->copy_from(exec->get_master().get(), nsystems * nnz, allvalues.data(),
                    all_values);
    value_type* const all_b = exec->alloc<value_type>(nsystems * nrows);
    exec->copy_from(exec->get_master().get(), nsystems * nrows, allb.data(),
                    all_b);
    return {nsystems, nrows, nnz, row_ptrs, col_idxs, all_values, all_b};
}
void appl_clean_up(ApplSysData& appl_data, std::shared_ptr<gko::Executor> exec)
{
```

In general, the application would control non-const pointers; the const casts below would not be needed.

```
    exec->free(const_cast<index_type*>(appl_data.row_ptrs));
    exec->free(const_cast<index_type*>(appl_data.col_idxs));
    exec->free(const_cast<value_type*>(appl_data.all_values));
    exec->free(const_cast<value_type*>(appl_data.all_rhs));
}
```

# Results

The following is the expected result on the reference executor:

```
Solver type:  batch::bicgstab
Matrix size:  (32, 32)
Num batch entries:  2
Entire solve took:  2.18558e-05 seconds.
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iostream>
#include <map>
#include <random>
#include <string>
```

```
#include <ginkgo/ginkgo.hpp>
using value_type = double;
using real_type = gko::remove_complex<value_type>;
using index_type = int;
using size_type = gko::size_type;
using vec_type = gko::batch::MultiVector<value_type>;
using real_vec_type = gko::batch::MultiVector<real_type>;
using mtx_type = gko::batch::matrix::Csr<value_type, index_type>;
using bicgstab = gko::batch::solver::Bicgstab<value_type>;
namespace detail {
template <typename InputType>
auto unbatch(const InputType* batch_object)
{
    auto unbatched_mats =
        std::vector<std::unique_ptr<typename InputType::unbatch_type>>{};
    for (size_type b = 0; b < batch_object->get_num_batch_items(); ++b) {
        unbatched_mats.emplace_back(
            batch_object->create_const_view_for_item(b)->clone());
    }
    return unbatched_mats;
}
}  // namespace detail
struct ApplSysData {
    size_type nsystems;
    int nrows;
    int nnz;
    const index_type* row_ptrs;
    const index_type* col_idxs;
    const value_type* all_values;
    const value_type* all_rhs;
};
/*
* Generates a batch of tridiagonal systems.
*
* @param nrows  Number of rows in each system.
* @param nsystems  Number of systems in the batch.
* @param exec  The device executor to use for the solver.
* @note  Normally, the application may not deal with Ginkgo executors, nor do
* we need it to.  Here, we use the executor for backend-independent device
* memory allocation.  The application, for example, might assume Hip (for AMD
* GPUs) and use `hipMalloc` directly.
*/
ApplSysData appl_generate_system(const int nrows, const size_type nsystems,
                                 std::shared_ptr<gko::Executor> exec);
void appl_clean_up(ApplSysData& appl_data, std::shared_ptr<gko::Executor> exec);
int main(int argc, char* argv[])
{
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0]
                  « " [executor] [num_systems] [num_rows] [print_residuals] "
                    "[num_reps]"
                  « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    const size_type num_systems = argc >= 3 ?  std::atoi(argv[2]) :  2;
    const int num_rows = argc >= 4 ?  std::atoi(argv[3]) :  32;  // per system
    const bool print_residuals =
        argc >= 5 ?  (std::string(argv[4]) == "true") :  false;
    const int num_reps = argc >= 6 ?  std::atoi(argv[5]) :  20;
    auto appl_sys = appl_generate_system(num_rows, num_systems, exec);
    auto batch_mat_size =
        gko::batch_dim<2>(num_systems, gko::dim<2>(num_rows, num_rows));
    auto batch_vec_size =
        gko::batch_dim<2>(num_systems, gko::dim<2>(num_rows, 1));
    auto vals_view = gko::array<value_type>::const_view(
        exec, num_systems * appl_sys.nnz, appl_sys.all_values);
```

```
auto rowptrs_view = gko::array<index_type>::const_view(exec, num_rows + 1,
                                                       appl_sys.row_ptrs);
auto colidxs_view = gko::array<index_type>::const_view(exec, appl_sys.nnz,
                                                       appl_sys.col_idxs);
auto A = gko::share(mtx_type::create_const(
    exec, batch_mat_size, std::move(vals_view), std::move(colidxs_view),
    std::move(rowptrs_view)));
auto b_view = gko::array<value_type>::const_view(
    exec, num_systems * num_rows, appl_sys.all_rhs);
auto b = vec_type::create_const(exec, batch_vec_size, std::move(b_view));
auto x = vec_type::create(exec);
auto host_x = vec_type::create(exec->get_master(), batch_vec_size);
for (size_type isys = 0; isys < num_systems; isys++) {
    for (int irow = 0; irow < num_rows; irow++) {
        host_x->at(isys, irow, 0) = gko::zero<value_type>();
    }
}
x->copy_from(host_x.get());
const real_type reduction_factor{1e-10};
auto solver =
    bicgstab::build()
        .with_max_iterations(500)
        .with_tolerance(reduction_factor)
        .with_tolerance_type(gko::batch::stop::tolerance_type::relative)
        .on(exec)
        ->generate(A);
std::shared_ptr<const gko::batch::log::BatchConvergence<value_type>>
    logger = gko::batch::log::BatchConvergence<value_type>::create();
solver->add_logger(logger);
auto x_clone = gko::clone(x);
for (int i = 0; i < 3; ++i) {
    x_clone->copy_from(x.get());
    solver->apply(b, x_clone);
}
double apply_time = 0.0;
for (int i = 0; i < num_reps; ++i) {
    x_clone->copy_from(x.get());
    exec->synchronize();
    std::chrono::steady_clock::time_point t1 =
        std::chrono::steady_clock::now();
    solver->apply(b, x_clone);
    exec->synchronize();
    std::chrono::steady_clock::time_point t2 =
        std::chrono::steady_clock::now();
    auto time_span =
        std::chrono::duration_cast<std::chrono::duration<double>>(t2 - t1);
    apply_time += time_span.count();
}
x->copy_from(x_clone.get());
solver->remove_logger(logger.get());
auto norm_dim = gko::batch_dim<2>(num_systems, gko::dim<2>(1, 1));
auto host_b_norm = real_vec_type::create(exec->get_master(), norm_dim);
host_b_norm->fill(0.0);
b->compute_norm2(host_b_norm);
auto one = vec_type::create(exec, norm_dim);
one->fill(1.0);
auto neg_one = vec_type::create(exec, norm_dim);
neg_one->fill(-1.0);
auto res = vec_type::create(exec, batch_vec_size);
res->copy_from(b);
A->apply(one, x, neg_one, res);
auto host_res_norm = real_vec_type::create(exec->get_master(), norm_dim);
host_res_norm->fill(0.0);
res->compute_norm2(host_res_norm);
auto host_log_resid = gko::make_temporary_clone(
    exec->get_master(), &logger->get_residual_norm());
auto host_log_iters = gko::make_temporary_clone(
    exec->get_master(), &logger->get_num_iterations());
if (print_residuals) {
    std::cout << "Residual norm sqrt(r^T r):\n";
    auto unb_res = detail::unbatch(host_res_norm.get());
    auto unb_bnorm = detail::unbatch(host_b_norm.get());
    for (size_type i = 0; i < num_systems; ++i) {
        std::cout << " System no.  " << i
                  << ":  residual norm = " << unb_res[i]->at(0, 0)
                  << ", implicit residual norm = "
                  << host_log_resid->get_const_data()[i]
                  << ", iterations = "
                  << host_log_iters->get_const_data()[i] << std::endl;
        const real_type relresnorm =
            unb_res[i]->at(0, 0) / unb_bnorm[i]->at(0, 0);
        if (!(relresnorm <= reduction_factor)) {
            std::cout << "System " << i << " converged only to "
                      << relresnorm << " relative residual." << std::endl;
        }
    }
}
```

```
    std::cout « "Solver type:  "
              « "batch::bicgstab"
              « "\nMatrix size:  " « A->get_common_size()
              « "\nNum batch entries:  " « A->get_num_batch_items()
              « "\nEntire solve took:  " « apply_time / num_reps « " seconds."
              « std::endl;
    appl_clean_up(appl_sys, exec);
    return 0;
}
ApplSysData appl_generate_system(const int nrows, const size_type nsystems,
                                 std::shared_ptr<gko::Executor> exec)
{
    const int nnz = nrows * 3 - 2;
    std::default_random_engine rgen(15);
    std::normal_distribution<real_type> distb(0.5, 0.1);
    std::vector<real_type> spacings(nsystems * nrows);
    std::generate(spacings.begin(), spacings.end(),
                  [&]() { return distb(rgen); });
    std::vector<value_type> allvalues(nnz * nsystems);
    for (size_type isys = 0; isys < nsystems; isys++) {
        allvalues.at(isys * nnz) = 2.0 / spacings.at(isys * nrows);
        allvalues.at(isys * nnz + 1) = -1.0;
        for (int irow = 0; irow < nrows - 2; irow++) {
            allvalues.at(isys * nnz + 2 + irow * 3) = -1.0;
            allvalues.at(isys * nnz + 2 + irow * 3 + 1) =
                2.0 / spacings.at(isys * nrows + irow + 1);
            allvalues.at(isys * nnz + 2 + irow * 3 + 2) = -1.0;
        }
        allvalues.at(isys * nnz + 2 + (nrows - 2) * 3) = -1.0;
        allvalues.at(isys * nnz + 2 + (nrows - 2) * 3 + 1) =
            2.0 / spacings.at((isys + 1) * nrows - 1);
        assert(isys * nnz + 2 + (nrows - 2) * 3 + 2 == (isys + 1) * nnz);
    }
    std::vector<index_type> rowptrs(nrows + 1);
    rowptrs.at(0) = 0;
    rowptrs.at(1) = 2;
    for (int i = 2; i < nrows; i++) {
        rowptrs.at(i) = rowptrs.at(i - 1) + 3;
    }
    rowptrs.at(nrows) = rowptrs.at(nrows - 1) + 2;
    assert(rowptrs.at(nrows) == nnz);
    std::vector<index_type> colidxs(nnz);
    colidxs.at(0) = 0;
    colidxs.at(1) = 1;
    const int nnz_per_row = 3;
    for (int irow = 1; irow < nrows - 1; irow++) {
        colidxs.at(2 + (irow - 1) * nnz_per_row) = irow - 1;
        colidxs.at(2 + (irow - 1) * nnz_per_row + 1) = irow;
        colidxs.at(2 + (irow - 1) * nnz_per_row + 2) = irow + 1;
    }
    colidxs.at(2 + (nrows - 2) * nnz_per_row) = nrows - 2;
    colidxs.at(2 + (nrows - 2) * nnz_per_row + 1) = nrows - 1;
    assert(2 + (nrows - 2) * nnz_per_row + 1 == nnz - 1);
    std::vector<value_type> allb(nrows * nsystems);
    for (size_type isys = 0; isys < nsystems; isys++) {
        const value_type bval = distb(rgen);
        std::fill(allb.begin() + isys * nrows,
                  allb.begin() + (isys + 1) * nrows, bval);
    }
    index_type* const row_ptrs = exec->alloc<index_type>(nrows + 1);
    exec->copy_from(exec->get_master().get(), static_cast<size_type>(nrows + 1),
                    rowptrs.data(), row_ptrs);
    index_type* const col_idxs = exec->alloc<index_type>(nnz);
    exec->copy_from(exec->get_master().get(), static_cast<size_type>(nnz),
                    colidxs.data(), col_idxs);
    value_type* const all_values = exec->alloc<value_type>(nsystems * nnz);
    exec->copy_from(exec->get_master().get(), nsystems * nnz, allvalues.data(),
                    all_values);
    value_type* const all_b = exec->alloc<value_type>(nsystems * nrows);
    exec->copy_from(exec->get_master().get(), nsystems * nrows, allb.data(),
                    all_b);
    return {nsystems, nrows, nnz, row_ptrs, col_idxs, all_values, all_b};
}
void appl_clean_up(ApplSysData& appl_data, std::shared_ptr<gko::Executor> exec)
{
    exec->free(const_cast<index_type*>(appl_data.row_ptrs));
    exec->free(const_cast<index_type*>(appl_data.col_idxs));
    exec->free(const_cast<value_type*>(appl_data.all_values));
    exec->free(const_cast<value_type*>(appl_data.all_rhs));
}
```

# Chapter 10

# The cb-gmres program

The CB-GMRES solver example..

## Introduction

### About the example

This example showcases the usage of the Ginkgo solver CB-GMRES (Compressed Basis GMRES). A small system is solved with two un-preconditioned CB-GMRES solvers:

1. without compressing the krylov basis; it uses double precision for both the matrix and the krylov basis, and
2. with a compression of the krylov basis; it uses double precision for the matrix and all arithmetic operations, while using single precision for the storage of the krylov basis

Both solves are timed and the residual norm of each solution is computed to show that both solutions are correct.

## The commented program

This is the main ginkgo header file.

```
#include <chrono>
#include <cmath>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
```

Helper function which measures the time of `solver->apply(b, x)` in seconds To get an accurate result, the solve is repeated multiple times (while ensuring the initial guess is always the same). The result of the solve will be written to x.

```
template <typename ValueType>
double measure_solve_time_in_s(std::shared_ptr<const gko::Executor> exec,
                               gko::LinOp* solver,
                               const gko::matrix::Dense<ValueType>* b,
                               gko::matrix::Dense<ValueType>* x)
{
    constexpr int repeats{5};
    double duration{0};
```

Make a copy of x, so we can re-use the same initial guess multiple times
```
auto x_copy = clone(x);
for (int i = 0; i < repeats; ++i) {
```

No need to copy it in the first iteration
```
if (i != 0) {
    x_copy->copy_from(x);
}
```

Make sure all previous executor operations have finished before starting the time
```
exec->synchronize();
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x_copy);
```

Make sure all computations are done before stopping the time
```
    exec->synchronize();
    auto tac = std::chrono::steady_clock::now();
    duration += std::chrono::duration<double>(tac - tic).count();
}
```

Copy the solution back to x, so the caller has the result
```
    x->copy_from(x_copy);
    return duration / static_cast<double>(repeats);
}
int main(int argc, char* argv[])
{
```

Use some shortcuts. In Ginkgo, vectors are seen as a gko::matrix::Dense with one column/one row. The advantage of this concept is that using multiple vectors is a now a natural extension of adding columns/rows are necessary.
```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
```

The gko::matrix::Csr class is used here, but any other matrix class such as gko::matrix::Coo, gko::matrix::Hybrid, gko::matrix::Ell or gko::matrix::Sellp could also be used.
```
using mtx = gko::matrix::Csr<ValueType, IndexType>;
```

The gko::solver::CbGmres is used here, but any other solver class can also be used.
```
using cb_gmres = gko::solver::CbGmres<ValueType>;
```

Print the ginkgo version information.
```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor] " « std::endl;
    std::exit(-1);
}
```

Map which generates the appropriate executor
```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation
```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Note: this matrix is copied from "SOURCE_DIR/matrices" instead of from the local directory. For details, see "examples/cb-gmres/CMakeLists.txt"

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

Create a uniform right-hand side with a norm2 of 1 on the host (norm2(b) == 1), followed by copying it to the actual executor (to make sure it also works for GPUs)

```
const auto A_size = A->get_size();
auto b_host = vec::create(exec->get_master(), gko::dim<2>{A_size[0], 1});
for (gko::size_type i = 0; i < A_size[0]; ++i) {
    b_host->at(i, 0) =
        ValueType{1} / std::sqrt(static_cast<ValueType>(A_size[0]));
}
auto b_norm = gko::initialize<real_vec>({0.0}, exec);
b_host->compute_norm2(b_norm);
auto b = clone(exec, b_host);
```

As an initial guess, use the right-hand side

```
auto x_keep = clone(b);
auto x_reduce = clone(x_keep);
const RealValueType reduction_factor{1e-6};
```

Generate two solver factories: `_keep` uses the same precision for the krylov basis as the matrix, and `_reduce` uses one precision below it. If `ValueType` is double, then `_reduce` uses float as the krylov basis storage type

```
auto solver_gen_keep =
    cb_gmres::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_baseline(gko::stop::mode::rhs_norm)
                           .with_reduction_factor(reduction_factor))
        .with_krylov_dim(100u)
        .with_storage_precision(
            gko::solver::cb_gmres::storage_precision::keep)
        .on(exec);
auto solver_gen_reduce =
    cb_gmres::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_baseline(gko::stop::mode::rhs_norm)
                           .with_reduction_factor(reduction_factor))
        .with_krylov_dim(100u)
        .with_storage_precision(
            gko::solver::cb_gmres::storage_precision::reduce1)
        .on(exec);
```

Generate the actual solver from the factory and the matrix.

```
auto solver_keep = solver_gen_keep->generate(A);
auto solver_reduce = solver_gen_reduce->generate(A);
```

Solve both system and measure the time for each.

```
auto time_keep =
    measure_solve_time_in_s(exec, solver_keep.get(), b.get(), x_keep.get());
auto time_reduce = measure_solve_time_in_s(exec, solver_reduce.get(),
                                           b.get(), x_reduce.get());
```

Make sure the output is in scientific notation for easier comparison

```
std::cout « std::scientific;
```

Note: The time might not be significantly different since the matrix is quite small

```
std::cout « "Solve time without compression:  " « time_keep « " s\n"
          « "Solve time with compression:        " « time_reduce « " s\n";
```

To measure if your solution has actually converged, the error of the solution is measured. one, neg_one are objects that represent the numbers which allow for a uniform interface when computing on any device. To compute the residual, the (advanced) apply method is used.

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto res_norm_keep = gko::initialize<real_vec>({0.0}, exec);
auto res_norm_reduce = gko::initialize<real_vec>({0.0}, exec);
auto tmp = gko::clone(b);
```

tmp = Ax - tmp

```
    A->apply(one, x_keep, neg_one, tmp);
    tmp->compute_norm2(res_norm_keep);
    std::cout « "\nResidual norm without compression:\n";
    write(std::cout, res_norm_keep);
    tmp->copy_from(b);
    A->apply(one, x_reduce, neg_one, tmp);
    tmp->compute_norm2(res_norm_reduce);
    std::cout « "\nResidual norm with compression:\n";
    write(std::cout, res_norm_reduce);
}
```

# Results

The following is the expected result:

```
Solve time without compression:  1.842690e-04 s
Solve time with compression:        1.589936e-04 s
Residual norm without compression:
%%MatrixMarket matrix array real general
1 1
2.430544e-07
Residual norm with compression:
%%MatrixMarket matrix array real general
1 1
3.437257e-07
```

### Comments about programming and debugging

# The plain program

```
#include <chrono>
#include <cmath>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
template <typename ValueType>
double measure_solve_time_in_s(std::shared_ptr<const gko::Executor> exec,
                               gko::LinOp* solver,
                               const gko::matrix::Dense<ValueType>* b,
                               gko::matrix::Dense<ValueType>* x)
{
    constexpr int repeats{5};
    double duration{0};
    auto x_copy = clone(x);
    for (int i = 0; i < repeats; ++i) {
        if (i != 0) {
            x_copy->copy_from(x);
        }
        exec->synchronize();
        auto tic = std::chrono::steady_clock::now();
        solver->apply(b, x_copy);
        exec->synchronize();
        auto tac = std::chrono::steady_clock::now();
        duration += std::chrono::duration<double>(tac - tic).count();
    }
    x->copy_from(x_copy);
    return duration / static_cast<double>(repeats);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cb_gmres = gko::solver::CbGmres<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor] " « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
```

```
            }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    const auto A_size = A->get_size();
    auto b_host = vec::create(exec->get_master(), gko::dim<2>{A_size[0], 1});
    for (gko::size_type i = 0; i < A_size[0]; ++i) {
        b_host->at(i, 0) =
            ValueType{1} / std::sqrt(static_cast<ValueType>(A_size[0]));
    }
    auto b_norm = gko::initialize<real_vec>({0.0}, exec);
    b_host->compute_norm2(b_norm);
    auto b = clone(exec, b_host);
    auto x_keep = clone(b);
    auto x_reduce = clone(x_keep);
    const RealValueType reduction_factor{1e-6};
    auto solver_gen_keep =
        cb_gmres::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::rhs_norm)
                               .with_reduction_factor(reduction_factor))
            .with_krylov_dim(100u)
            .with_storage_precision(
                gko::solver::cb_gmres::storage_precision::keep)
            .on(exec);
    auto solver_gen_reduce =
        cb_gmres::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::rhs_norm)
                               .with_reduction_factor(reduction_factor))
            .with_krylov_dim(100u)
            .with_storage_precision(
                gko::solver::cb_gmres::storage_precision::reduce1)
            .on(exec);
    auto solver_keep = solver_gen_keep->generate(A);
    auto solver_reduce = solver_gen_reduce->generate(A);
    auto time_keep =
        measure_solve_time_in_s(exec, solver_keep.get(), b.get(), x_keep.get());
    auto time_reduce = measure_solve_time_in_s(exec, solver_reduce.get(),
                                               b.get(), x_reduce.get());
    std::cout << std::scientific;
    std::cout << "Solve time without compression:  " << time_keep << " s\n"
              << "Solve time with compression:        " << time_reduce << " s\n";
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res_norm_keep = gko::initialize<real_vec>({0.0}, exec);
    auto res_norm_reduce = gko::initialize<real_vec>({0.0}, exec);
    auto tmp = gko::clone(b);
    A->apply(one, x_keep, neg_one, tmp);
    tmp->compute_norm2(res_norm_keep);
    std::cout << "\nResidual norm without compression:\n";
    write(std::cout, res_norm_keep);
    tmp->copy_from(b);
    A->apply(one, x_reduce, neg_one, tmp);
    tmp->compute_norm2(res_norm_reduce);
    std::cout << "\nResidual norm with compression:\n";
    write(std::cout, res_norm_reduce);
}
```

# Chapter 11

# The custom-logger program

The simple solver with a custom logger example..

This example depends on simple-solver, simple-solver-logging, minimal-cuda-solver.

## Introduction

The custom-logger example shows how Ginkgo's API can be leveraged to implement application-specific callbacks for Ginkgo's events. This is the most basic way of extending Ginkgo and a good first step for any application developer who wants to adapt Ginkgo to his specific needs.

Ginkgo's gko::log::Logger abstraction provides hooks to the events that happen during the library execution. These hooks concern any low-level event such as memory allocations, deallocations, copies and kernel launches up to high-level events such as linear operator applications and completion of solver iterations.

In this example, a simple logger is implemented to track the solver's recurrent residual norm and compute the true residual norm. At the end of the solver execution, a comparison table is shown on-screen.

#### About the example

Each example has the following sections:

1. **Introduction:**This gives an overview of the example and mentions any interesting aspects in the example that might help the reader.
2. **The commented program:** This section is intended for you to understand the details of the example so that you can play with it and understand Ginkgo and its features better.
3. **Results:** This section shows the results of the code when run. Though the results may not be completely the same, you can expect the behaviour to be similar.
4. **The plain program:** This is the complete code without any comments to have an complete overview of the code.

# The commented program

## Include files

This is the main ginkgo header file.
```
#include <ginkgo/ginkgo.hpp>
```

Add the fstream header to read from data from files.
```
#include <fstream>
```

Add the map header for storing the executor map.
```
#include <map>
```

Add the C++ iomanip header to prettify the output.
```
#include <iomanip>
```

Add formatting flag modification capabilities.
```
#include <ios>
```

Add the C++ iostream header to output information to the console.
```
#include <iostream>
```

Add the string manipulation header to handle strings.
```
#include <string>
```

Add the vector header for storing the logger's data
```
#include <vector>
```

Utility function which returns the first element (position [0, 0]) from a given gko::matrix::Dense matrix / vector.
```
template <typename ValueType>
ValueType get_first_element(const gko::matrix::Dense<ValueType>* mtx)
{
```

Copy the matrix / vector to the host device before accessing the value in case it is stored in a GPU.
```
    return mtx->get_executor()->copy_val_to_host(mtx->get_const_values());
}
```

Utility function which computes the norm of a Ginkgo gko::matrix::Dense vector.
```
template <typename ValueType>
gko::remove_complex<ValueType> compute_norm(
    const gko::matrix::Dense<ValueType>* b)
{
```

Get the executor of the vector
```
auto exec = b->get_executor();
```

Initialize a result scalar containing the value 0.0.
```
auto b_norm =
    gko::initialize<gko::matrix::Dense<gko::remove_complex<ValueType>>>(
        {0.0}, exec);
```

Use the dense `compute_norm2` function to compute the norm.
```
b->compute_norm2(b_norm);
```

Use the other utility function to return the norm contained in `b_norm`
```
    return get_first_element(b_norm.get());
}
```

Custom logger class which intercepts the residual norm scalar and solution vector in order to print a table of real vs recurrent (internal to the solvers) residual norms.
```
template <typename ValueType>
struct ResidualLogger :  gko::log::Logger {
    using RealValueType = gko::remove_complex<ValueType>;
```

Output the logger's data in a table format
```
void write()const
{
```

Print a header for the table
```
std::cout « "Recurrent vs true vs implicit residual norm:"
          « std::endl;
std::cout « '|' « std::setw(10) « "Iteration" « '|' « std::setw(25)
          « "Recurrent Residual Norm" « '|' « std::setw(25)
          « "True Residual Norm" « '|' « std::setw(25)
          « "Implicit Residual Norm" « '|' « std::endl;
```

Print a separation line. Note that for creating `10` characters `std::setw()` should be set to `11`.
```
std::cout « '|' « std::setfill('-') « std::setw(11) « '|'
          « std::setw(26) « '|' « std::setw(26) « '|'
          « std::setw(26) « '|' « std::setfill(' ') « std::endl;
```

Print the data one by one in the form
```
std::cout « std::scientific;
for (std::size_t i = 0; i < iterations.size(); i++) {
    std::cout « '|' « std::setw(10) « iterations[i] « '|'
              « std::setw(25) « recurrent_norms[i] « '|'
              « std::setw(25) « real_norms[i] « '|' « std::setw(25)
              « implicit_norms[i] « '|' « std::endl;
}
```

std::defaultfloat could be used here but some compilers do not support it properly, e.g. the Intel compiler
```
std::cout.unsetf(std::ios_base::floatfield);
```

Print a separation line
```
    std::cout « '|' « std::setfill('-') « std::setw(11) « '|'
              « std::setw(26) « '|' « std::setw(26) « '|'
              « std::setw(26) « '|' « std::setfill(' ') « std::endl;
}
using gko_dense = gko::matrix::Dense<ValueType>;
using gko_real_dense = gko::matrix::Dense<RealValueType>;
```

Customize the logging hook which is called everytime an iteration is completed
```
void on_iteration_complete(const gko::LinOp* solver, const gko::LinOp* b,
                           const gko::LinOp* solution,
                           const gko::size_type& iteration,
                           const gko::LinOp* residual,
                           const gko::LinOp* residual_norm,
                           const gko::LinOp* implicit_sq_residual_norm,
                           const gko::array<gko::stopping_status>*,
                           bool)const override
{
```

If the solver shares a residual norm, log its value
```
if (residual_norm) {
    auto dense_norm = gko::as<gko_real_dense>(residual_norm);
```

Add the norm to the `recurrent_norms` vector
```
recurrent_norms.push_back(get_first_element(dense_norm));
```

Otherwise, use the recurrent residual vector
```
} else {
    auto dense_residual = gko::as<gko_dense>(residual);
```

Compute the residual vector's norm
```
auto norm = compute_norm(dense_residual);
```

Add the computed norm to the `recurrent_norms` vector
```
    recurrent_norms.push_back(norm);
}
```

If the solver shares the current solution vector
```
if (solution) {
```

Extract the matrix from the solver
```
auto matrix = gko::as<gko::solver::detail::SolverBaseLinOp>(solver)
                  ->get_system_matrix();
```

Store the matrix's executor
```
auto exec = matrix->get_executor();
```

Create a scalar containing the value 1.0
```
auto one = gko::initialize<gko_dense>({1.0}, exec);
```

Create a scalar containing the value -1.0
```
auto neg_one = gko::initialize<gko_dense>({-1.0}, exec);
```

Instantiate a temporary result variable
```
auto res = gko::as<gko_dense>(gko::clone(b));
```

Compute the real residual vector by calling apply on the system matrix
```
matrix->apply(one, solution, neg_one, res);
```

Compute the norm of the residual vector and add it to the `real_norms` vector
```
    real_norms.push_back(compute_norm(res.get()));
} else {
```

Add to the `real_norms` vector the value -1.0 if it could not be computed
```
    real_norms.push_back(-1.0);
}
if (implicit_sq_residual_norm) {
    auto dense_norm =
        gko::as<gko_real_dense>(implicit_sq_residual_norm);
```

Add the norm to the `implicit_norms` vector
```
    implicit_norms.push_back(std::sqrt(get_first_element(dense_norm)));
} else {
```

Add to the `implicit_norms` vector the value -1.0 if it could not be computed
```
    implicit_norms.push_back(-1.0);
}
```

Add the current iteration number to the `iterations` vector
```
    iterations.push_back(iteration);
}
```

Construct the logger
```
    ResidualLogger()
        :  gko::log::Logger(gko::log::Logger::iteration_complete_mask)
    {}
private:
```

Vector which stores all the recurrent residual norms
```
mutable std::vector<RealValueType> recurrent_norms{};
```

Vector which stores all the real residual norms
```
mutable std::vector<RealValueType> real_norms{};
```

Vector which stores all the implicit residual norms
```
mutable std::vector<RealValueType> implicit_norms{};
```

Vector which stores all the iteration numbers
```
    mutable std::vector<std::size_t> iterations{};
};
int main(int argc, char* argv[])
{
```

Use some shortcuts. In Ginkgo, vectors are seen as a gko::matrix::Dense with one column/one row. The advantage of this concept is that using multiple vectors is a now a natural extension of adding columns/rows are necessary.
```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
```

The gko::matrix::Csr class is used here, but any other matrix class such as gko::matrix::Coo, gko::matrix::Hybrid, gko::matrix::Ell or gko::matrix::Sellp could also be used.
```
using mtx = gko::matrix::Csr<ValueType, IndexType>;
```

The gko::solver::Cg is used here, but any other solver class can also be used.
```
using cg = gko::solver::Cg<ValueType>;
```

Print the ginkgo version information.
```
std::cout « gko::version_info::get() « std::endl;
```

### Where do you want to run your solver ?

The gko::Executor class is one of the cornerstones of Ginkgo. Currently, we have support for an gko::OmpExecutor, which uses OpenMP multi-threading in most of its kernels, a gko::ReferenceExecutor, a single threaded specialization of the OpenMP executor and a gko::CudaExecutor which runs the code on a NVIDIA GPU if available.

**Note**

With the help of C++, you see that you only ever need to change the executor and all the other functions/routines within Ginkgo should automatically work and run on the executor with any other changes.

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Reading your data and transfer to the proper device.

Read the matrix, right hand side and the initial solution using the read function.

**Note**

Ginkgo uses C++ smart pointers to automatically manage memory. To this end, we use our own object ownership transfer functions that under the hood call the required smart pointer functions to manage object ownership. gko::share and gko::give are the functions that you would need to use.

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
const RealValueType reduction_factor = 1e-7;
```

### Creating the solver

Generate the gko::solver factory. Ginkgo uses the concept of Factories to build solvers with certain properties. Observe the Fluent interface used here. Here a cg solver is generated with a stopping criteria of maximum iterations of 20 and a residual norm reduction of 1e-15. You also observe that the stopping criteria(gko::stop) are also generated from factories using their build methods. You need to specify the executors which each of the object needs to be built on.

```
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
```

```
                    gko::stop::ResidualNorm<ValueType>::build()
                        .with_reduction_factor(reduction_factor))
        .on(exec);
```

Instantiate a ResidualLogger logger.

```
auto logger = std::make_shared<ResidualLogger<ValueType»();
```

Add the previously created logger to the solver factory. The logger will be automatically propagated to all solvers created from this factory.

```
solver_gen->add_logger(logger);
```

Generate the solver from the matrix. The solver factory built in the previous step takes a "matrix"(a gko::LinOp to be more general) as an input. In this case we provide it with a full matrix that we previously read, but as the solver only effectively uses the apply() method within the provided "matrix" object, you can effectively create a gko::LinOp class with your own apply implementation to accomplish more tasks. We will see an example of how this can be done in the custom-matrix-format example

```
auto solver = solver_gen->generate(A);
```

Finally, solve the system. The solver, being a gko::LinOp, can be applied to a right hand side, b to obtain the solution, x.

```
solver->apply(b, x);
```

Print the solution to the command line.

```
std::cout « "Solution (x):\n";
write(std::cout, x);
```

Print the table of the residuals obtained from the logger

```
logger->write();
```

To measure if your solution has actually converged, you can measure the error of the solution. one, neg_one are objects that represent the numbers which allow for a uniform interface when computing on any device. To compute the residual, all you need to do is call the apply method, which in this case is an spmv and equivalent to the LAPACK z_spmv routine. Finally, you compute the euclidean 2-norm with the compute_norm2 function.

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Results

The following is the expected result:

```
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Recurrent vs true vs implicit residual norm:
| Iteration|  Recurrent Residual Norm|       True Residual Norm|   Implicit Residual Norm|
|----------|-------------------------|-------------------------|-------------------------|
```

```
|         0|             4.358899e+00|             4.358899e+00|             4.358899e+00|
|         1|             2.304548e+00|             2.304548e+00|             2.304548e+00|
|         2|             1.467706e+00|             1.467706e+00|             1.467706e+00|
|         3|             9.848751e-01|             9.848751e-01|             9.848751e-01|
|         4|             7.418330e-01|             7.418330e-01|             7.418330e-01|
|         5|             5.136231e-01|             5.136231e-01|             5.136231e-01|
|         6|             3.841650e-01|             3.841650e-01|             3.841650e-01|
|         7|             3.164394e-01|             3.164394e-01|             3.164394e-01|
|         8|             2.277088e-01|             2.277088e-01|             2.277088e-01|
|         9|             1.703121e-01|             1.703121e-01|             1.703121e-01|
|        10|             9.737220e-02|             9.737220e-02|             9.737220e-02|
|        11|             6.168306e-02|             6.168306e-02|             6.168306e-02|
|        12|             4.541231e-02|             4.541231e-02|             4.541231e-02|
|        13|             3.195304e-02|             3.195304e-02|             3.195304e-02|
|        14|             1.616058e-02|             1.616058e-02|             1.616058e-02|
|        15|             6.570152e-03|             6.570152e-03|             6.570152e-03|
|        16|             2.643669e-03|             2.643669e-03|             2.643669e-03|
|        17|             8.588089e-04|             8.588089e-04|             8.588089e-04|
|        18|             2.864613e-04|             2.864613e-04|             2.864613e-04|
|        19|             1.641952e-15|             2.107881e-15|             1.641952e-15|
|----------|--------------------------|--------------------------|--------------------------|
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
2.10788e-15
```

### Comments about programming and debugging

# The plain program

```
#include <ginkgo/ginkgo.hpp>
#include <fstream>
#include <map>
#include <iomanip>
#include <ios>
#include <iostream>
#include <string>
#include <vector>
template <typename ValueType>
ValueType get_first_element(const gko::matrix::Dense<ValueType>* mtx)
{
    return mtx->get_executor()->copy_val_to_host(mtx->get_const_values());
}
template <typename ValueType>
gko::remove_complex<ValueType> compute_norm(
    const gko::matrix::Dense<ValueType>* b)
{
    auto exec = b->get_executor();
    auto b_norm =
        gko::initialize<gko::matrix::Dense<gko::remove_complex<ValueType>>>(
            {0.0}, exec);
    b->compute_norm2(b_norm);
    return get_first_element(b_norm.get());
}
template <typename ValueType>
struct ResidualLogger :  gko::log::Logger {
    using RealValueType = gko::remove_complex<ValueType>;
    void write()const
{
        std::cout « "Recurrent vs true vs implicit residual norm:"
                  « std::endl;
        std::cout « '|' « std::setw(10) « "Iteration" « '|' « std::setw(25)
                  « "Recurrent Residual Norm" « '|' « std::setw(25)
                  « "True Residual Norm" « '|' « std::setw(25)
                  « "Implicit Residual Norm" « '|' « std::endl;
        std::cout « '|' « std::setfill('-') « std::setw(11) « '|'
                  « std::setw(26) « '|' « std::setw(26) « '|'
                  « std::setw(26) « '|' « std::setfill(' ') « std::endl;
        std::cout « std::scientific;
        for (std::size_t i = 0; i < iterations.size(); i++) {
            std::cout « '|' « std::setw(10) « iterations[i] « '|'
                      « std::setw(25) « recurrent_norms[i] « '|'
                      « std::setw(25) « real_norms[i] « '|' « std::setw(25)
                      « implicit_norms[i] « '|' « std::endl;
        }
        std::cout.unsetf(std::ios_base::floatfield);
        std::cout « '|' « std::setfill('-') « std::setw(11) « '|'
                  « std::setw(26) « '|' « std::setw(26) « '|'
                  « std::setw(26) « '|' « std::setfill(' ') « std::endl;
    }
    using gko_dense = gko::matrix::Dense<ValueType>;
```

```
    using gko_real_dense = gko::matrix::Dense<RealValueType>;
    void on_iteration_complete(const gko::LinOp* solver, const gko::LinOp* b,
                               const gko::LinOp* solution,
                               const gko::size_type& iteration,
                               const gko::LinOp* residual,
                               const gko::LinOp* residual_norm,
                               const gko::LinOp* implicit_sq_residual_norm,
                               const gko::array<gko::stopping_status>*,
                               bool)const override
{
        if (residual_norm) {
            auto dense_norm = gko::as<gko_real_dense>(residual_norm);
            recurrent_norms.push_back(get_first_element(dense_norm));
        } else {
            auto dense_residual = gko::as<gko_dense>(residual);
            auto norm = compute_norm(dense_residual);
            recurrent_norms.push_back(norm);
        }
        if (solution) {
            auto matrix = gko::as<gko::solver::detail::SolverBaseLinOp>(solver)
                              ->get_system_matrix();
            auto exec = matrix->get_executor();
            auto one = gko::initialize<gko_dense>({1.0}, exec);
            auto neg_one = gko::initialize<gko_dense>({-1.0}, exec);
            auto res = gko::as<gko_dense>(gko::clone(b));
            matrix->apply(one, solution, neg_one, res);
            real_norms.push_back(compute_norm(res.get()));
        } else {
            real_norms.push_back(-1.0);
        }
        if (implicit_sq_residual_norm) {
            auto dense_norm =
                gko::as<gko_real_dense>(implicit_sq_residual_norm);
            implicit_norms.push_back(std::sqrt(get_first_element(dense_norm)));
        } else {
            implicit_norms.push_back(-1.0);
        }
        iterations.push_back(iteration);
    }
    ResidualLogger()
        :  gko::log::Logger(gko::log::Logger::iteration_complete_mask)
    {}
private:
    mutable std::vector<RealValueType> recurrent_norms{};
    mutable std::vector<RealValueType> real_norms{};
    mutable std::vector<RealValueType> implicit_norms{};
    mutable std::vector<std::size_t> iterations{};
};
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    const RealValueType reduction_factor = 1e-7;
    auto solver_gen =
```

```
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .on(exec);
    auto logger = std::make_shared<ResidualLogger<ValueType»();
    solver_gen->add_logger(logger);
    auto solver = solver_gen->generate(A);
    solver->apply(b, x);
    std::cout « "Solution (x):\n";
    write(std::cout, x);
    logger->write();
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Chapter 12

# The custom-matrix-format program

The custom matrix format example..

This example depends on simple-solver, poisson-solver, three-pt-stencil-solver, .

## Introduction

This example solves a 1D Poisson equation:

$u : [0, 1] \to R$
$u" = f$
$u(0) = u0$
$u(1) = u1$

using a finite difference method on an equidistant grid with `K` discretization points (`K` can be controlled with a command line parameter). The discretization is done via the second order Taylor polynomial:

$u(x + h) = u(x) - u'(x)h + 1/2u"(x)h^2 + O(h^3)$
$u(x - h) = u(x) + u'(x)h + 1/2u"(x)h^2 + O(h^3)/ +$
$- - - - - - - - - - - - - - - - - - - - - - - - - -$
$- u(x - h) + 2u(x) + -u(x + h) = -f(x)h^2 + O(h^3)$

For an equidistant grid with K "inner" discretization points $x1, ..., xk$ and step size $h = 1/(K + 1)$, the formula produces a system of linear equations

$2u_1 - u_2 = -f_1h^2 + u0$
$- u_(k - 1) + 2u_k - u_(k + 1) = -f_kh^2, k = 2, ..., K - 1$
$- u_(K - 1) + 2u_K = -f_Kh^2 + u1$

which is then solved using Ginkgo's implementation of the CG method preconditioned with block-Jacobi. It is also possible to specify on which executor Ginkgo will solve the system via the command line. The function `f` is set to `f(x) = 6x` (making the solution `u(x) = x^3`), but that can be changed in the `main` function.

The intention of this example is to show how a custom linear operator can be created and integrated into Ginkgo to achieve performance benefits.

**About the example**

## The commented program

```
#include <iostream>
#include <map>
#include <string>
#include <omp.h>
#include <ginkgo/ginkgo.hpp>
```

A CUDA kernel implementing the stencil, which will be used if running on the CUDA executor. Unfortunately, NVCC has serious problems interpreting some parts of Ginkgo's code, so the kernel has to be compiled separately.

```
template <typename ValueType>
void stencil_kernel(std::size_t size, const ValueType* coefs,
                    const ValueType* b, ValueType* x);
```

A stencil matrix class representing the 3pt stencil linear operator. We include the gko::LinOp interface, where we need to implement the two apply_impl methods, which get called inside the default implementation of apply (after argument verification) to perform the actual application of the linear operator.

It also includes the gko::EnableCreateMethod mixin which provides a default implementation of the static create method. This method will forward all its arguments to the constructor to create the object, and return an std↩ ::unique_ptr to the created object.

```
template <typename ValueType>
class StencilMatrix :  public gko::LinOp,
                       public gko::EnableCreateMethod<StencilMatrix<ValueType>> {
public:
```

This constructor will be called by the create method. Here we initialize the coefficients of the stencil.

```
    StencilMatrix(std::shared_ptr<const gko::Executor> exec,
                  gko::size_type size = 0, ValueType left = -1.0,
                  ValueType center = 2.0, ValueType right = -1.0)
        :  gko::LinOp(exec, gko::dim<2>{size}),
          coefficients(exec, {left, center, right})
    {}
protected:
    using vec = gko::matrix::Dense<ValueType>;
    using coef_type = gko::array<ValueType>;
```

Here we implement the application of the linear operator, x = A ∗ b. apply_impl will be called by the apply method, after the arguments have been moved to the correct executor and the operators checked for conforming sizes.

For simplicity, we assume that there is always only one right hand side and the stride of consecutive elements in the vectors is 1 (both of these are always true in this example).

```
void apply_impl(const gko::LinOp* b, gko::LinOp* x)const override
{
```

we only implement the operator for dense RHS. gko::as will throw an exception if its argument is not Dense.

```
auto dense_b = gko::as<vec>(b);
auto dense_x = gko::as<vec>(x);
```

we need separate implementations depending on the executor, so we create an operation which maps the call to the correct implementation

```
struct stencil_operation :  gko::Operation {
    stencil_operation(const coef_type& coefficients, const vec* b,
                      vec* x)
        :  coefficients{coefficients}, b{b}, x{x}
    {}
```

OpenMP implementation

```
            void run(std::shared_ptr<const gko::OmpExecutor>)const override
{
                auto b_values = b->get_const_values();
                auto x_values = x->get_values();
#pragma omp parallel for
                for (std::size_t i = 0; i < x->get_size()[0]; ++i) {
                    auto coefs = coefficients.get_const_data();
                    auto result = coefs[1] * b_values[i];
                    if (i > 0) {
                        result += coefs[0] * b_values[i - 1];
                    }
                    if (i < x->get_size()[0] - 1) {
```

```
                    result += coefs[2] * b_values[i + 1];
                }
                x_values[i] = result;
            }
        }
```

CUDA implementation

```
void run(std::shared_ptr<const gko::CudaExecutor>)const override
{
    stencil_kernel(x->get_size()[0], coefficients.get_const_data(),
                   b->get_const_values(), x->get_values());
}
```

We do not provide an implementation for reference executor. If not provided, Ginkgo will use the implementation for the OpenMP executor when calling it in the reference executor.

```
        const coef_type& coefficients;
        const vec* b;
        vec* x;
    };
    this->get_executor()->run(
        stencil_operation(coefficients, dense_b, dense_x));
}
```

There is also a version of the apply function which does the operation x = alpha $*$ A $*$ b + beta $*$ x. This function is commonly used and can often be better optimized than implementing it using x = A $*$ b. However, for simplicity, we will implement it exactly like that in this example.

```
    void apply_impl(const gko::LinOp* alpha, const gko::LinOp* b,
                    const gko::LinOp* beta, gko::LinOp* x)const override
{
        auto dense_b = gko::as<vec>(b);
        auto dense_x = gko::as<vec>(x);
        auto tmp_x = dense_x->clone();
        this->apply_impl(b, tmp_x.get());
        dense_x->scale(beta);
        dense_x->add_scaled(alpha, tmp_x);
    }
private:
    coef_type coefficients;
};
```

Creates a stencil matrix in CSR format for the given number of discretization points.

```
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(gko::matrix::Csr<ValueType, IndexType>* matrix)
{
    const auto discretization_points = matrix->get_size()[0];
    auto row_ptrs = matrix->get_row_ptrs();
    auto col_idxs = matrix->get_col_idxs();
    auto values = matrix->get_values();
    IndexType pos = 0;
    const ValueType coefs[] = {-1, 2, -1};
    row_ptrs[0] = pos;
    for (int i = 0; i < discretization_points; ++i) {
        for (auto ofs :  {-1, 0, 1}) {
            if (0 <= i + ofs && i + ofs < discretization_points) {
                values[pos] = coefs[ofs + 1];
                col_idxs[pos] = i + ofs;
                ++pos;
            }
        }
        row_ptrs[i + 1] = pos;
    }
}
```

Generates the RHS vector given `f` and the boundary conditions.

```
template <typename Closure, typename ValueType>
void generate_rhs(Closure f, ValueType u0, ValueType u1,
                  gko::matrix::Dense<ValueType>* rhs)
{
    const auto discretization_points = rhs->get_size()[0];
    auto values = rhs->get_values();
    const ValueType h = 1.0 / (discretization_points + 1);
    for (int i = 0; i < discretization_points; ++i) {
        const ValueType xi = ValueType(i + 1) * h;
        values[i] = -f(xi) * h * h;
    }
    values[0] += u0;
    values[discretization_points - 1] += u1;
}
```

Prints the solution `u`.

```
template <typename ValueType>
void print_solution(ValueType u0, ValueType u1,
                    const gko::matrix::Dense<ValueType>* u)
{
    std::cout « u0 « '\n';
    for (int i = 0; i < u->get_size()[0]; ++i) {
        std::cout « u->get_const_values()[i] « '\n';
    }
    std::cout « u1 « std::endl;
}
```

Computes the 1-norm of the error given the computed `u` and the correct solution function `correct_u`.

```
template <typename Closure, typename ValueType>
double calculate_error(int discretization_points,
                       const gko::matrix::Dense<ValueType>* u,
                       Closure correct_u)
{
    const auto h = 1.0 / (discretization_points + 1);
    auto error = 0.0;
    for (int i = 0; i < discretization_points; ++i) {
        using std::abs;
        const auto xi = (i + 1) * h;
        error +=
            abs(u->get_const_values()[i] - correct_u(xi)) / abs(correct_u(xi));
    }
    return error;
}
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const unsigned int discretization_points =
    argc >= 3 ?  std::atoi(argv[2]) :  100u;
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

executor used by the application

```
const auto app_exec = exec->get_master();
```

problem:

```
auto correct_u = [](ValueType x) { return x * x * x; };
auto f = [](ValueType x) { return ValueType{6} * x; };
auto u0 = correct_u(0);
auto u1 = correct_u(1);
```

initialize vectors

```
auto rhs = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
generate_rhs(f, u0, u1, rhs.get());
auto u = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
```

```
for (int i = 0; i < u->get_size()[0]; ++i) {
    u->get_values()[i] = 0.0;
}
const RealValueType reduction_factor{1e-7};
```

Generate solver and solve the system

```
cg::build()
    .with_criteria(
        gko::stop::Iteration::build().with_max_iters(discretization_points),
        gko::stop::ResidualNorm<ValueType>::build().with_reduction_factor(
            reduction_factor))
    .on(exec)
```

notice how our custom StencilMatrix can be used in the same way as any built-in type

```
        ->generate(StencilMatrix<ValueType>::create(exec, discretization_points,
                                                    -1, 2, -1))
        ->apply(rhs, u);
    std::cout << "\nSolve complete."
              << "\nThe average relative error is "
              << calculate_error(discretization_points, u.get(), correct_u) /
                     discretization_points
              << std::endl;
}
```

# Results

### Comments about programming and debugging

# The plain program

```
#include <iostream>
#include <map>
#include <string>
#include <omp.h>
#include <ginkgo/ginkgo.hpp>
template <typename ValueType>
void stencil_kernel(std::size_t size, const ValueType* coefs,
                    const ValueType* b, ValueType* x);
template <typename ValueType>
class StencilMatrix :  public gko::LinOp,
                       public gko::EnableCreateMethod<StencilMatrix<ValueType>> {
public:
    StencilMatrix(std::shared_ptr<const gko::Executor> exec,
                  gko::size_type size = 0, ValueType left = -1.0,
                  ValueType center = 2.0, ValueType right = -1.0)
        :  gko::LinOp(exec, gko::dim<2>{size}),
          coefficients(exec, {left, center, right})
    {}
protected:
    using vec = gko::matrix::Dense<ValueType>;
    using coef_type = gko::array<ValueType>;
    void apply_impl(const gko::LinOp* b, gko::LinOp* x)const override
{
        auto dense_b = gko::as<vec>(b);
        auto dense_x = gko::as<vec>(x);
        struct stencil_operation :  gko::Operation {
            stencil_operation(const coef_type& coefficients, const vec* b,
                              vec* x)
                :  coefficients{coefficients}, b{b}, x{x}
            {}
            void run(std::shared_ptr<const gko::OmpExecutor>)const override
{
                auto b_values = b->get_const_values();
                auto x_values = x->get_values();
#pragma omp parallel for
                for (std::size_t i = 0; i < x->get_size()[0]; ++i) {
                    auto coefs = coefficients.get_const_data();
                    auto result = coefs[1] * b_values[i];
                    if (i > 0) {
                        result += coefs[0] * b_values[i - 1];
                    }
                    if (i < x->get_size()[0] - 1) {
                        result += coefs[2] * b_values[i + 1];
                    }
                    x_values[i] = result;
                }
```

```
            }
            void run(std::shared_ptr<const gko::CudaExecutor>)const override
{
                stencil_kernel(x->get_size()[0], coefficients.get_const_data(),
                               b->get_const_values(), x->get_values());
            }
            const coef_type& coefficients;
            const vec* b;
            vec* x;
        };
        this->get_executor()->run(
            stencil_operation(coefficients, dense_b, dense_x));
    }
    void apply_impl(const gko::LinOp* alpha, const gko::LinOp* b,
                    const gko::LinOp* beta, gko::LinOp* x)const override
{
        auto dense_b = gko::as<vec>(b);
        auto dense_x = gko::as<vec>(x);
        auto tmp_x = dense_x->clone();
        this->apply_impl(b, tmp_x.get());
        dense_x->scale(beta);
        dense_x->add_scaled(alpha, tmp_x);
    }
private:
    coef_type coefficients;
};
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(gko::matrix::Csr<ValueType, IndexType>* matrix)
{
    const auto discretization_points = matrix->get_size()[0];
    auto row_ptrs = matrix->get_row_ptrs();
    auto col_idxs = matrix->get_col_idxs();
    auto values = matrix->get_values();
    IndexType pos = 0;
    const ValueType coefs[] = {-1, 2, -1};
    row_ptrs[0] = pos;
    for (int i = 0; i < discretization_points; ++i) {
        for (auto ofs :  {-1, 0, 1}) {
            if (0 <= i + ofs && i + ofs < discretization_points) {
                values[pos] = coefs[ofs + 1];
                col_idxs[pos] = i + ofs;
                ++pos;
            }
        }
        row_ptrs[i + 1] = pos;
    }
}
template <typename Closure, typename ValueType>
void generate_rhs(Closure f, ValueType u0, ValueType u1,
                  gko::matrix::Dense<ValueType>* rhs)
{
    const auto discretization_points = rhs->get_size()[0];
    auto values = rhs->get_values();
    const ValueType h = 1.0 / (discretization_points + 1);
    for (int i = 0; i < discretization_points; ++i) {
        const ValueType xi = ValueType(i + 1) * h;
        values[i] = -f(xi) * h * h;
    }
    values[0] += u0;
    values[discretization_points - 1] += u1;
}
template <typename ValueType>
void print_solution(ValueType u0, ValueType u1,
                    const gko::matrix::Dense<ValueType>* u)
{
    std::cout « u0 « '\n';
    for (int i = 0; i < u->get_size()[0]; ++i) {
        std::cout « u->get_const_values()[i] « '\n';
    }
    std::cout « u1 « std::endl;
}
template <typename Closure, typename ValueType>
double calculate_error(int discretization_points,
                       const gko::matrix::Dense<ValueType>* u,
                       Closure correct_u)
{
    const auto h = 1.0 / (discretization_points + 1);
    auto error = 0.0;
    for (int i = 0; i < discretization_points; ++i) {
        using std::abs;
        const auto xi = (i + 1) * h;
        error +=
            abs(u->get_const_values()[i] - correct_u(xi)) / abs(correct_u(xi));
    }
    return error;
}
int main(int argc, char* argv[])
```

```
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    const unsigned int discretization_points =
        argc >= 3 ?  std::atoi(argv[2]) :  100u;
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    const auto app_exec = exec->get_master();
    auto correct_u = [](ValueType x) { return x * x * x; };
    auto f = [](ValueType x) { return ValueType{6} * x; };
    auto u0 = correct_u(0);
    auto u1 = correct_u(1);
    auto rhs = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
    generate_rhs(f, u0, u1, rhs.get());
    auto u = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
    for (int i = 0; i < u->get_size()[0]; ++i) {
        u->get_values()[i] = 0.0;
    }
    const RealValueType reduction_factor{1e-7};
    cg::build()
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(discretization_points),
            gko::stop::ResidualNorm<ValueType>::build().with_reduction_factor(
                reduction_factor))
        .on(exec)
        ->generate(StencilMatrix<ValueType>::create(exec, discretization_points,
                                                    -1, 2, -1))
        ->apply(rhs, u);
    std::cout « "\nSolve complete."
              « "\nThe average relative error is "
              « calculate_error(discretization_points, u.get(), correct_u) /
                    discretization_points
              « std::endl;
}
```

# Chapter 13

# The custom-stopping-criterion program

The custom stopping criterion creation example..

This example depends on simple-solver, minimal-cuda-solver.

## Introduction

### About the example

## The commented program

```
#include <atomic>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <thread>
#include <ginkgo/ginkgo.hpp>
/ **
 * The ByInteraction class is a criterion which asks for user input to stop
 * the iteration process.  Using this criterion is slightly more complex than the
 * other ones, because it is asynchronous therefore requires the use of threads.
 * /
class ByInteraction :  public gko::stop::Criterion {
    friend class gko::stop::Criterion;
    using Criterion = gko::stop::Criterion;
public:
    GKO_CREATE_FACTORY_PARAMETERS(parameters, Factory)
    {
        / **
         * Boolean set by the user to stop the iteration process
         * /
        std::atomic<bool>* GKO_FACTORY_PARAMETER_SCALAR(stop_iteration_process,
                                                        nullptr);
    };
    GKO_ENABLE_CRITERION_FACTORY(ByInteraction, parameters, Factory);
    GKO_ENABLE_BUILD_METHOD(Factory);
protected:
    bool check_impl(gko::uint8 stoppingId, bool setFinalized,
                    gko::array<gko::stopping_status>* stop_status,
                    bool* one_changed, const Criterion::Updater&)override
{
        bool result = *(parameters_.stop_iteration_process);
        if (result) {
            this->set_all_statuses(stoppingId, setFinalized, stop_status);
            *one_changed = true;
        }
        return result;
    }
    explicit ByInteraction(std::shared_ptr<const gko::Executor> exec)
        :  Criterion(std::move(exec))
    {}
```

```
    explicit ByInteraction(const Factory* factory,
                           const gko::stop::CriterionArgs& args)
        :  Criterion(factory->get_executor()),
          parameters_{factory->get_parameters()}
    {}
};
void run_solver(std::atomic<bool>* stop_iteration_process,
                std::shared_ptr<gko::Executor> exec)
{
```

### Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using bicg = gko::solver::Bicgstab<ValueType>;
```

### Read Data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
```

### Create solver factory and solve system

```
auto solver =
    bicg::build()
        .with_criteria(ByInteraction::build().with_stop_iteration_process(
            stop_iteration_process))
        .on(exec)
        ->generate(A);
solver->add_logger(gko::log::Stream<ValueType>::create(
    gko::log::Logger::iteration_complete_mask, std::cout, true));
solver->apply(b, x);
std::cout « "Solver stopped" « std::endl;
```

### Print solution

```
std::cout « "Solution (x):  \n";
write(std::cout, x);
```

### Calculate residual

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
}
int main(int argc, char* argv[])
{
```

### Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

### Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
```

### Figure out where to run the code

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

### Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
```

```
            return gko::DpcppExecutor::create(0,
                                              gko::OmpExecutor::create());
        }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Declare a user controlled boolean for the iteration process

```
std::atomic<bool> stop_iteration_process{false};
```

Create a new a thread to launch the solver

```
std::thread t(run_solver, &stop_iteration_process, exec);
```

Look for an input command "stop" in the console, which sets the boolean to true

```
    std::cout « "Type 'stop' to stop the iteration process" « std::endl;
    std::string command;
    while (std::cin » command) {
        if (command == "stop") {
            break;
        } else {
            std::cout « "Unknown command" « std::endl;
        }
    }
    std::cout « "User input command 'stop' - The solver will stop!"
              « std::endl;
    stop_iteration_process = true;
    t.join();
}
```

# Results

This is the expected output:

```
.
.
.
.
.
.
[LOG] »> iteration 22516 completed with solver LinOp[gko::solver::Bicgstab<double>,0x7fe6a4003710] with
      residual LinOp[gko::matrix::Dense<double>,0x7fe6a40050b0], solution
      LinOp[gko::matrix::Dense<double>,0x7fe6a40048e0] and residual_norm LinOp[gko::LinOp const*,0]
LinOp[gko::matrix::Dense<double>,0x7fe6a40050b0][
    5.17803e-164
    -7.6865e-165
    -2.06149e-164
    -4.84737e-165
    -3.36597e-164
    2.22353e-164
    1.47594e-165
    -1.78592e-165
    -6.17274e-166
    -3.02681e-166
    7.82009e-166
    8.57102e-165
    -1.28879e-164
    -2.62076e-165
    2.55329e-165
    -5.95988e-166
    -5.79273e-166
    -5.20172e-166
    -6.79458e-166
]
// Typing 'stop' stops the solver.
User input command 'stop' - The solver will stop
LinOp[gko::matrix::Dense<double>,0x7fe6a40048e0][
    0.252218
    0.108645
    0.0662811
    0.0630433
    0.0384088
    0.0396536
    0.0402648
    0.0338935
    0.0193098
    0.0234653
    0.0211499
```

```
    0.0196413
    0.0199151
    0.0181674
    0.0162722
    0.0150714
    0.0107016
    0.0121141
    0.0123025
]
Solver stopped
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
6.50306e-16
```

### Comments about programming and debugging

## The plain program

```
#include <atomic>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <thread>
#include <ginkgo/ginkgo.hpp>
class ByInteraction :  public gko::stop::Criterion {
    friend class gko::stop::Criterion;
    using Criterion = gko::stop::Criterion;
public:
    GKO_CREATE_FACTORY_PARAMETERS(parameters, Factory)
    {
        std::atomic<bool>* GKO_FACTORY_PARAMETER_SCALAR(stop_iteration_process,
                                                        nullptr);
    };
    GKO_ENABLE_CRITERION_FACTORY(ByInteraction, parameters, Factory);
    GKO_ENABLE_BUILD_METHOD(Factory);
protected:
    bool check_impl(gko::uint8 stoppingId, bool setFinalized,
                    gko::array<gko::stopping_status>* stop_status,
                    bool* one_changed, const Criterion::Updater&)override
{
        bool result = *(parameters_.stop_iteration_process);
        if (result) {
            this->set_all_statuses(stoppingId, setFinalized, stop_status);
            *one_changed = true;
        }
        return result;
    }
    explicit ByInteraction(std::shared_ptr<const gko::Executor> exec)
        :  Criterion(std::move(exec))
    {}
    explicit ByInteraction(const Factory* factory,
                           const gko::stop::CriterionArgs& args)
        :  Criterion(factory->get_executor()),
          parameters_{factory->get_parameters()}
    {}
};
void run_solver(std::atomic<bool>* stop_iteration_process,
```

```
                std::shared_ptr<gko::Executor> exec)
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using bicg = gko::solver::Bicgstab<ValueType>;
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    auto solver =
        bicg::build()
            .with_criteria(ByInteraction::build().with_stop_iteration_process(
                stop_iteration_process))
            .on(exec)
            ->generate(A);
    solver->add_logger(gko::log::Stream<ValueType>::create(
        gko::log::Logger::iteration_complete_mask, std::cout, true));
    solver->apply(b, x);
    std::cout << "Solver stopped" << std::endl;
    std::cout << "Solution (x):  \n";
    write(std::cout, x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout << "Residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
}
int main(int argc, char* argv[])
{
    std::cout << gko::version_info::get() << std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr << "Usage:  " << argv[0] << " [executor]" << std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()>>
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    std::atomic<bool> stop_iteration_process{false};
    std::thread t(run_solver, &stop_iteration_process, exec);
    std::cout << "Type 'stop' to stop the iteration process" << std::endl;
    std::string command;
    while (std::cin >> command) {
        if (command == "stop") {
            break;
        } else {
            std::cout << "Unknown command" << std::endl;
        }
    }
    std::cout << "User input command 'stop' - The solver will stop!"
              << std::endl;
    stop_iteration_process = true;
    t.join();
}
```

# Chapter 14

# The distributed-multigrid-preconditioned-solver program

The distributed multigrid preconditioned solver with customized components example..

This example depends on distributed-solver.

## Introduction

This distributed multigrid preconditioned solver example should help you understand customizing Ginkgo multigrid in a distributed setting. The example will solve a simple 1D Laplace equation where the system can be distributed row-wise to multiple processes. Note. Because the stencil for the discretized Laplacian is configured with equal weight, the coarsening method does not perform well on this kind of problem. To run the solver with multiple processes, use `mpirun -n NUM_PROCS ./distributed-solver [executor] [num_grid_points] [num_iterations]`.

If you are using GPU devices, please make sure that you run this example with at most as many processes as you have GPU devices available.

## The commented program

### Include files

This is the main ginkgo header file.

```
#include <ginkgo/ginkgo.hpp>
```

Add the C++ iostream header to output information to the console.

```
#include <iostream>
```

Add the STL map header for the executor selection

```
#include <map>
```

Add the string manipulation header to handle strings.

```
#include <string>
int main(int argc, char* argv[])
{
```

### Initialize the MPI environment

Since this is an MPI program, we need to initialize and finalize MPI at the begin and end respectively of our program. This can be easily done with the following helper construct that uses RAII to automate the initialization and finalization.

```
const gko::experimental::mpi::environment env(argc, argv);
```

### Type Definitions

Define the needed types. In a parallel program we need to differentiate between global and local indices, thus we have two index types.

```
using GlobalIndexType = gko::int64;
using LocalIndexType = gko::int32;
```

The underlying value type.

```
using ValueType = double;
```

As vector type we use the following, which implements a subset of gko::matrix::Dense.

```
using dist_vec = gko::experimental::distributed::Vector<ValueType>;
```

As matrix type we simply use the following type, which can read distributed data and be applied to a distributed vector.

```
using dist_mtx =
    gko::experimental::distributed::Matrix<ValueType, LocalIndexType,
                                           GlobalIndexType>;
```

We still need a localized vector type to be used as scalars in the advanced apply operations.

```
using vec = gko::matrix::Dense<ValueType>;
```

The partition type describes how the rows of the matrices are distributed.

```
using part_type =
    gko::experimental::distributed::Partition<LocalIndexType,
                                              GlobalIndexType>;
```

We can use here the same solver type as you would use in a non-distributed program. Please note that not all solvers support distributed systems at the moment.

```
using solver = gko::solver::Cg<ValueType>;
```

We use the Schwarz preconditioner to extend non-distributed preconditioners, like our Jacobi, to the distributed case. The Schwarz preconditioner wraps another preconditioner, and applies it only to the local part of a distributed matrix. This will be used as our distributed multigrid smoother.

```
using schwarz = gko::experimental::distributed::preconditioner::Schwarz<
    ValueType, LocalIndexType, GlobalIndexType>;
using bj = gko::preconditioner::Jacobi<ValueType, LocalIndexType>;
```

Multigrid and Pgm can accept the distributed matrix, so we still use the same type as the non-distributed case.

```
using mg = gko::solver::Multigrid;
using pgm = gko::multigrid::Pgm<ValueType, LocalIndexType>;
using uc = gko::multigrid::UniformCoarsening<ValueType, LocalIndexType>;
```

Create an MPI communicator get the rank of the calling process.

```
const auto comm = gko::experimental::mpi::communicator(MPI_COMM_WORLD);
const auto rank = comm.rank();
```

### User Input Handling

User input settings:

- The executor, defaults to reference.
- The number of grid points, defaults to 100.
- The number of iterations, defaults to 1000.

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    if (rank == 0) {
        std::cerr « "Usage:  " « argv[0]
                 « " [executor] [grid_side] [num_iterations]"
                 « " [mg_type=pgm|uc-agg|uc-inj] [stencil=1d|2d]"
                 « std::endl;
    }
    std::exit(-1);
}
ValueType t_init = gko::experimental::mpi::get_walltime();
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const auto grid_dim =
    static_cast<gko::size_type>(argc >= 3 ?  std::atoi(argv[2]) :  100);
const auto num_iters =
    static_cast<gko::size_type>(argc >= 4 ?  std::atoi(argv[3]) :  1000);
const std::string mg_type = argc >= 5 ?  argv[4] :  "pgm";
const std::string stencil = argc >= 6 ?  argv[5] :  "1d";
const std::map<std::string,
               std::function<std::shared_ptr<gko::Executor>(MPI_Comm)»
    executor_factory_mpi{
        {"reference",
         [](MPI_Comm) { return gko::ReferenceExecutor::create(); }},
        {"omp", [](MPI_Comm) { return gko::OmpExecutor::create(); }},
        {"cuda",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::CudaExecutor::get_num_devices());
             return gko::CudaExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"hip",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::HipExecutor::get_num_devices());
             return gko::HipExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"dpcpp", [](MPI_Comm comm) {
             int device_id = 0;
             if (gko::DpcppExecutor::get_num_devices("gpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("gpu"));
             } else if (gko::DpcppExecutor::get_num_devices("cpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("cpu"));
             } else {
                 throw std::runtime_error("No suitable DPC++ devices");
             }
             return gko::DpcppExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }}};
auto exec = executor_factory_mpi.at(executor_string)(MPI_COMM_WORLD);
```

### Creating the Distributed Matrix and Vectors

As a first step, we create a partition of the rows. The partition consists of ranges of consecutive rows which are assigned a part-id. These part-ids will be used for the distributed data structures to determine which rows will be stored locally. In this example each rank has (nearly) the same number of rows, so we can use the following specialized constructor. See gko::distributed::Partition for other modes of creating a partition.

```
const bool is_2d = (stencil == "2d");
const auto num_rows = is_2d ?  grid_dim * grid_dim :  grid_dim;
auto partition = gko::share(part_type::build_from_global_size_uniform(
    exec->get_master(), comm.size(),
    static_cast<GlobalIndexType>(num_rows)));
```

Assemble the matrix using either a 3-pt 1D Laplacian stencil or a 5-pt 2D Laplacian stencil on a grid_dim x grid_dim grid (row-major). Rows are partitioned across ranks; in 2D, rows near a rank boundary carry cross-rank non-local entries via the i ± grid_dim neighbours.

```
gko::matrix_data<ValueType, GlobalIndexType> A_data;
gko::matrix_data<ValueType, GlobalIndexType> b_data;
gko::matrix_data<ValueType, GlobalIndexType> x_data;
A_data.size = {num_rows, num_rows};
b_data.size = {num_rows, 1};
x_data.size = {num_rows, 1};
const auto range_start = partition->get_range_bounds()[rank];
const auto range_end = partition->get_range_bounds()[rank + 1];
if (is_2d) {
    const auto N = static_cast<GlobalIndexType>(grid_dim);
    for (GlobalIndexType i = range_start; i < range_end; i++) {
        const auto r = i / N;
        const auto c = i % N;
        if (r > 0) {
            A_data.nonzeros.emplace_back(i, i - N, -1);
        }
        if (c > 0) {
            A_data.nonzeros.emplace_back(i, i - 1, -1);
        }
        A_data.nonzeros.emplace_back(i, i, 4);
        if (c < N - 1) {
            A_data.nonzeros.emplace_back(i, i + 1, -1);
        }
        if (r < N - 1) {
            A_data.nonzeros.emplace_back(i, i + N, -1);
        }
        b_data.nonzeros.emplace_back(i, 0, std::sin(i * 0.01));
        x_data.nonzeros.emplace_back(i, 0, gko::zero<ValueType>());
    }
} else {
    for (GlobalIndexType i = range_start; i < range_end; i++) {
        if (i > 0) {
            A_data.nonzeros.emplace_back(i, i - 1, -1);
        }
        A_data.nonzeros.emplace_back(i, i, 2);
        if (i < static_cast<GlobalIndexType>(grid_dim) - 1) {
            A_data.nonzeros.emplace_back(i, i + 1, -1);
        }
        b_data.nonzeros.emplace_back(i, 0, std::sin(i * 0.01));
        x_data.nonzeros.emplace_back(i, 0, gko::zero<ValueType>());
    }
}
```

Take timings.

```
comm.synchronize();
ValueType t_init_end = gko::experimental::mpi::get_walltime();
```

Read the matrix data, currently this is only supported on CPU executors. This will also set up the communication pattern needed for the distributed matrix-vector multiplication.

```
auto A_host = gko::share(dist_mtx::create(exec->get_master(), comm));
auto x_host = dist_vec::create(exec->get_master(), comm);
auto b_host = dist_vec::create(exec->get_master(), comm);
A_host->read_distributed(A_data, partition);
b_host->read_distributed(b_data, partition);
x_host->read_distributed(x_data, partition);
```

After reading, the matrix and vector can be moved to the chosen executor, since the distributed matrix supports SpMV also on devices.

```
auto A = gko::share(dist_mtx::create(exec, comm));
auto x = dist_vec::create(exec, comm);
auto b = dist_vec::create(exec, comm);
A->copy_from(A_host);
b->copy_from(b_host);
x->copy_from(x_host);
```

Take timings.

```
comm.synchronize();
ValueType t_read_setup_end = gko::experimental::mpi::get_walltime();
```

### Solve the Distributed System

Generate the solver

Setup the multigrid factory with customized smoother and coarse solver Because BlockJacobi does not support distributed matrix, we need wrap it in Schwarz.

```
auto schwarz_bj_factory =
    gko::share(schwarz::build().with_local_solver(bj::build()).on(exec));
auto smoother_factory = gko::share(gko::solver::build_smoother(
    schwarz_bj_factory, 2u, static_cast<ValueType>(0.9)));
```

Cg supports distributed matrix, so we can use it as we did in non-distributed case

```
auto coarsest_factory = gko::share(
    solver::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
```

The multigrid preconditioner uses the Schwarz Jacobi as smoother and Cg as coarse solver. The mg_level can be PGM or UniformCoarsening (aggregation- or injection-style), selected by the CLI argument.

```
auto make_mg_factory = [&]() {
    auto base = mg::build()
                    .with_pre_smoother(smoother_factory)
                    .with_coarsest_solver(coarsest_factory)
                    .with_criteria(
                        gko::stop::Iteration::build().with_max_iters(1u));
    if (mg_type == "uc-agg") {
        return base
            .with_mg_level(
                uc::build().with_aggregation(true).with_coarse_skip(2))
            .on(exec);
    } else if (mg_type == "uc-inj") {
        return base
            .with_mg_level(
                uc::build().with_aggregation(false).with_coarse_skip(2))
            .on(exec);
    } else {
        return base.with_mg_level(pgm::build().with_deterministic(true))
            .on(exec);
    }
};
auto mg_factory = gko::share(make_mg_factory());
```

Setup the stopping criterion and logger

```
const gko::remove_complex<ValueType> reduction_factor{1e-8};
std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
    gko::log::Convergence<ValueType>::create();
auto Ainv = solver::build()
                .with_preconditioner(mg_factory)
                .with_criteria(
                    gko::stop::Iteration::build().with_max_iters(num_iters),
                    gko::stop::ResidualNorm<ValueType>::build()
                        .with_reduction_factor(reduction_factor))
                .on(exec)
                ->generate(A);
```

Add logger to the generated solver to log the iteration count and residual norm

```
Ainv->add_logger(logger);
```

Take timings.

```
comm.synchronize();
ValueType t_solver_generate_end = gko::experimental::mpi::get_walltime();
```

Apply the distributed solver, this is the same as in the non-distributed case.

```
Ainv->apply(b, x);
```

Take timings.

```
comm.synchronize();
ValueType t_end = gko::experimental::mpi::get_walltime();
```

Get the residual.

```
auto res_norm = gko::clone(exec->get_master(),
                           gko::as<vec>(logger->get_residual_norm()));
```

### Printing Results

Print the achieved residual norm and timings on rank 0.

```
if (comm.rank() == 0) {
```

clang-format off

```
std::cout « "\nNum rows in matrix:  " « num_rows
          « "\nNum ranks:  " « comm.size()
          « "\nFinal Res norm:  " « res_norm->at(0, 0)
          « "\nIteration count:  " « logger->get_num_iterations()
          « "\nInit time:  " « t_init_end - t_init
          « "\nRead time:  " « t_read_setup_end - t_init
          « "\nSolver generate time:  " « t_solver_generate_end - t_read_setup_end
          « "\nSolver apply time:  " « t_end - t_solver_generate_end
          « "\nTotal time:  " « t_end - t_init
          « std::endl;
```

clang-format on

```
    }
}
```

## Results

This is the expected output for `mpirun -n 4 ./distributed-multigrid-preconditioned-solver↩ :`

```
Num rows in matrix:  100
Num ranks:  4
Final Res norm:  1.61045e-08
Iteration count:  18
Init time:  0.000117699
Read time:  0.000522518
Solver generate time:  0.000430548
Solver apply time:  0.00183804
Total time:  0.00279111
```

The timings may vary depending on the machine.

## The plain program

```
#include <ginkgo/ginkgo.hpp>
#include <iostream>
#include <map>
#include <string>
int main(int argc, char* argv[])
{
    const gko::experimental::mpi::environment env(argc, argv);
    using GlobalIndexType = gko::int64;
    using LocalIndexType = gko::int32;
    using ValueType = double;
    using dist_vec = gko::experimental::distributed::Vector<ValueType>;
    using dist_mtx =
        gko::experimental::distributed::Matrix<ValueType, LocalIndexType,
                                               GlobalIndexType>;
    using vec = gko::matrix::Dense<ValueType>;
    using part_type =
        gko::experimental::distributed::Partition<LocalIndexType,
                                                  GlobalIndexType>;
    using solver = gko::solver::Cg<ValueType>;
    using schwarz = gko::experimental::distributed::preconditioner::Schwarz<
        ValueType, LocalIndexType, GlobalIndexType>;
    using bj = gko::preconditioner::Jacobi<ValueType, LocalIndexType>;
    using mg = gko::solver::Multigrid;
    using pgm = gko::multigrid::Pgm<ValueType, LocalIndexType>;
    using uc = gko::multigrid::UniformCoarsening<ValueType, LocalIndexType>;
    const auto comm = gko::experimental::mpi::communicator(MPI_COMM_WORLD);
    const auto rank = comm.rank();
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        if (rank == 0) {
            std::cerr « "Usage:  " « argv[0]
                     « " [executor] [grid_side] [num_iterations]"
                     « " [mg_type=pgm|uc-agg|uc-inj] [stencil=1d|2d]"
                     « std::endl;
```

```
    }
    std::exit(-1);
}
ValueType t_init = gko::experimental::mpi::get_walltime();
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const auto grid_dim =
    static_cast<gko::size_type>(argc >= 3 ?  std::atoi(argv[2]) :  100);
const auto num_iters =
    static_cast<gko::size_type>(argc >= 4 ?  std::atoi(argv[3]) :  1000);
const std::string mg_type = argc >= 5 ?  argv[4] :  "pgm";
const std::string stencil = argc >= 6 ?  argv[5] :  "1d";
const std::map<std::string,
               std::function<std::shared_ptr<gko::Executor>(MPI_Comm)>>
    executor_factory_mpi{
        {"reference",
         [](MPI_Comm) { return gko::ReferenceExecutor::create(); }},
        {"omp", [](MPI_Comm) { return gko::OmpExecutor::create(); }},
        {"cuda",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::CudaExecutor::get_num_devices());
             return gko::CudaExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"hip",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::HipExecutor::get_num_devices());
             return gko::HipExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"dpcpp", [](MPI_Comm comm) {
             int device_id = 0;
             if (gko::DpcppExecutor::get_num_devices("gpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("gpu"));
             } else if (gko::DpcppExecutor::get_num_devices("cpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("cpu"));
             } else {
                 throw std::runtime_error("No suitable DPC++ devices");
             }
             return gko::DpcppExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }}};
auto exec = executor_factory_mpi.at(executor_string)(MPI_COMM_WORLD);
const bool is_2d = (stencil == "2d");
const auto num_rows = is_2d ?  grid_dim * grid_dim :  grid_dim;
auto partition = gko::share(part_type::build_from_global_size_uniform(
    exec->get_master(), comm.size(),
    static_cast<GlobalIndexType>(num_rows)));
gko::matrix_data<ValueType, GlobalIndexType> A_data;
gko::matrix_data<ValueType, GlobalIndexType> b_data;
gko::matrix_data<ValueType, GlobalIndexType> x_data;
A_data.size = {num_rows, num_rows};
b_data.size = {num_rows, 1};
x_data.size = {num_rows, 1};
const auto range_start = partition->get_range_bounds()[rank];
const auto range_end = partition->get_range_bounds()[rank + 1];
if (is_2d) {
    const auto N = static_cast<GlobalIndexType>(grid_dim);
    for (GlobalIndexType i = range_start; i < range_end; i++) {
        const auto r = i / N;
        const auto c = i % N;
        if (r > 0) {
            A_data.nonzeros.emplace_back(i, i - N, -1);
        }
        if (c > 0) {
            A_data.nonzeros.emplace_back(i, i - 1, -1);
        }
        A_data.nonzeros.emplace_back(i, i, 4);
        if (c < N - 1) {
            A_data.nonzeros.emplace_back(i, i + 1, -1);
        }
        if (r < N - 1) {
            A_data.nonzeros.emplace_back(i, i + N, -1);
        }
        b_data.nonzeros.emplace_back(i, 0, std::sin(i * 0.01));
        x_data.nonzeros.emplace_back(i, 0, gko::zero<ValueType>());
    }
} else {
    for (GlobalIndexType i = range_start; i < range_end; i++) {
        if (i > 0) {
            A_data.nonzeros.emplace_back(i, i - 1, -1);
        }
        A_data.nonzeros.emplace_back(i, i, 2);
        if (i < static_cast<GlobalIndexType>(grid_dim) - 1) {
```

```
                A_data.nonzeros.emplace_back(i, i + 1, -1);
            }
            b_data.nonzeros.emplace_back(i, 0, std::sin(i * 0.01));
            x_data.nonzeros.emplace_back(i, 0, gko::zero<ValueType>());
        }
    }
    comm.synchronize();
    ValueType t_init_end = gko::experimental::mpi::get_walltime();
    auto A_host = gko::share(dist_mtx::create(exec->get_master(), comm));
    auto x_host = dist_vec::create(exec->get_master(), comm);
    auto b_host = dist_vec::create(exec->get_master(), comm);
    A_host->read_distributed(A_data, partition);
    b_host->read_distributed(b_data, partition);
    x_host->read_distributed(x_data, partition);
    auto A = gko::share(dist_mtx::create(exec, comm));
    auto x = dist_vec::create(exec, comm);
    auto b = dist_vec::create(exec, comm);
    A->copy_from(A_host);
    b->copy_from(b_host);
    x->copy_from(x_host);
    comm.synchronize();
    ValueType t_read_setup_end = gko::experimental::mpi::get_walltime();
    auto schwarz_bj_factory =
        gko::share(schwarz::build().with_local_solver(bj::build()).on(exec));
    auto smoother_factory = gko::share(gko::solver::build_smoother(
        schwarz_bj_factory, 2u, static_cast<ValueType>(0.9)));
    auto coarsest_factory = gko::share(
        solver::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
            .on(exec));
    auto make_mg_factory = [&]() {
        auto base = mg::build()
                        .with_pre_smoother(smoother_factory)
                        .with_coarsest_solver(coarsest_factory)
                        .with_criteria(
                            gko::stop::Iteration::build().with_max_iters(1u));
        if (mg_type == "uc-agg") {
            return base
                .with_mg_level(
                    uc::build().with_aggregation(true).with_coarse_skip(2))
                .on(exec);
        } else if (mg_type == "uc-inj") {
            return base
                .with_mg_level(
                    uc::build().with_aggregation(false).with_coarse_skip(2))
                .on(exec);
        } else {
            return base.with_mg_level(pgm::build().with_deterministic(true))
                .on(exec);
        }
    };
    auto mg_factory = gko::share(make_mg_factory());
    const gko::remove_complex<ValueType> reduction_factor{1e-8};
    std::shared_ptr<const gko::log::Convergence<ValueType» logger =
        gko::log::Convergence<ValueType>::create();
    auto Ainv = solver::build()
                    .with_preconditioner(mg_factory)
                    .with_criteria(
                        gko::stop::Iteration::build().with_max_iters(num_iters),
                        gko::stop::ResidualNorm<ValueType>::build()
                            .with_reduction_factor(reduction_factor))
                    .on(exec)
                    ->generate(A);
    Ainv->add_logger(logger);
    comm.synchronize();
    ValueType t_solver_generate_end = gko::experimental::mpi::get_walltime();
    Ainv->apply(b, x);
    comm.synchronize();
    ValueType t_end = gko::experimental::mpi::get_walltime();
    auto res_norm = gko::clone(exec->get_master(),
                               gko::as<vec>(logger->get_residual_norm()));
    if (comm.rank() == 0) {
        std::cout « "\nNum rows in matrix:  " « num_rows
                  « "\nNum ranks:  " « comm.size()
                  « "\nFinal Res norm:  " « res_norm->at(0, 0)
                  « "\nIteration count:  " « logger->get_num_iterations()
                  « "\nInit time:  " « t_init_end - t_init
                  « "\nRead time:  " « t_read_setup_end - t_init
                  « "\nSolver generate time:  " « t_solver_generate_end - t_read_setup_end
                  « "\nSolver apply time:  " « t_end - t_solver_generate_end
                  « "\nTotal time:  " « t_end - t_init
                  « std::endl;
    }
}
```

# Chapter 15

# The distributed-multigrid-reuse program

The distributed multigrid reuse in a Picard iteration example..

This example depends on distributed-multigrid-preconditioned-solver.

## Introduction

This example shows how an application that solves a sequence of linear systems with the same sparsity pattern can reuse the setup of a distributed multigrid preconditioner with `LinOpFactory::generate_reuse()`.

It solves the nonlinear diffusion equation -div((1 + u^2) grad u) = f on the unit square with a Picard iteration: every iteration assembles A(u_k) and solves A(u_k) u_{k+1} = f with a multigrid preconditioned Cg, until the nonlinear residual is small. All A(u_k) share their sparsity pattern, so after the hierarchy has been recorded once, generate_reuse() keeps the Pgm aggregates and transfer operators and only recomputes the coarse matrices, smoothers and the coarsest solver. The example also shows how to assemble a matrix whose entries depend on values owned by neighbouring ranks without exchanging them, by reading the matrix with `assembly_mode::communicate`.

The kept aggregates only fit the later matrices if the recorded one is representative. The first iterate u_0 = 0 gives the constant-coefficient Laplacian, which Pgm coarsens poorly, so the first iteration uses generate() and the hierarchy is recorded in the second one.

To run it with multiple processes, use `mpirun -n NUM_PROCS ./distributed-multigrid-reuse [executor] [grid_side] [max_picard_iters] [refresh_interval]`. A refresh_interval of 1 sets up the hierarchy from scratch in every iteration, which can be used to compare the setup times and iteration counts with the default reuse.

If you are using GPU devices, please make sure that you run this example with at most as many processes as you have GPU devices available.

# The commented program

### Include files

This is the main ginkgo header file.
```
#include <ginkgo/ginkgo.hpp>
```

Add the C standard library header for std::atoi and std::exit.
```
#include <cstdlib>
```

Add the C++ iomanip header to format the output.
```
#include <iomanip>
```

Add the C++ iostream header to output information to the console.
```
#include <iostream>
```

Add the STL map header for the executor selection
```
#include <map>
```

Add the string manipulation header to handle strings.
```
#include <string>
int main(int argc, char* argv[])
{
```

### Initialize the MPI environment

As in the other distributed examples, this RAII helper initializes and finalizes MPI.
```
const gko::experimental::mpi::environment env(argc, argv);
```

### Type Definitions

A distributed program needs both global and local indices.
```
using GlobalIndexType = gko::int64;
using LocalIndexType = gko::int32;
using ValueType = double;
using dist_vec = gko::experimental::distributed::Vector<ValueType>;
using dist_mtx =
    gko::experimental::distributed::Matrix<ValueType, LocalIndexType,
                                           GlobalIndexType>;
using vec = gko::matrix::Dense<ValueType>;
using part_type =
    gko::experimental::distributed::Partition<LocalIndexType,
                                              GlobalIndexType>;
using cg = gko::solver::Cg<ValueType>;
using schwarz = gko::experimental::distributed::preconditioner::Schwarz<
    ValueType, LocalIndexType, GlobalIndexType>;
using bj = gko::preconditioner::Jacobi<ValueType, LocalIndexType>;
using mg = gko::solver::Multigrid;
using pgm = gko::multigrid::Pgm<ValueType, LocalIndexType>;
const auto comm = gko::experimental::mpi::communicator(MPI_COMM_WORLD);
const auto rank = comm.rank();
```

### User Input Handling

User input settings:

- The executor, defaults to reference.
- The number of grid points per direction, defaults to 100.
- The maximum number of Picard iterations, defaults to 30.
- The number of Picard iterations after which the multigrid hierarchy is set up from scratch again, defaults to 0 (never). With 1, every iteration does a full setup.

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    if (rank == 0) {
        std::cerr « "Usage:  " « argv[0]
                 « " [executor] [grid_side] [max_picard_iters]"
                 « " [refresh_interval]" « std::endl;
    }
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const auto grid_dim =
    static_cast<gko::size_type>(argc >= 3 ?  std::atoi(argv[2]) :  100);
const auto max_picard_iters =
    static_cast<gko::size_type>(argc >= 4 ?  std::atoi(argv[3]) :  30);
const auto refresh_interval =
    static_cast<gko::size_type>(argc >= 5 ?  std::atoi(argv[4]) :  0);
const std::map<std::string,
               std::function<std::shared_ptr<gko::Executor>(MPI_Comm)»
    executor_factory_mpi{
        {"reference",
         [](MPI_Comm) { return gko::ReferenceExecutor::create(); }},
        {"omp", [](MPI_Comm) { return gko::OmpExecutor::create(); }},
        {"cuda",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::CudaExecutor::get_num_devices());
             return gko::CudaExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"hip",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::HipExecutor::get_num_devices());
             return gko::HipExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"dpcpp", [](MPI_Comm comm) {
             int device_id = 0;
             if (gko::DpcppExecutor::get_num_devices("gpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("gpu"));
             } else if (gko::DpcppExecutor::get_num_devices("cpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("cpu"));
             } else {
                 throw std::runtime_error("No suitable DPC++ devices");
             }
             return gko::DpcppExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }}};
auto exec = executor_factory_mpi.at(executor_string)(MPI_COMM_WORLD);
```

Takes a timestamp after all ranks have finished their work.

```
auto now = [&] {
    exec->synchronize();
    comm.synchronize();
    return gko::experimental::mpi::get_walltime();
};
```

### The Nonlinear Problem

We solve the nonlinear diffusion equation -div(k(u) grad u) = f with k(u) = 1 + u^2 and a constant source f on the unit square, with u = 0 on the boundary. It is discretized with the 5-point stencil on a grid_dim x grid_dim grid of interior

points, numbered row by row, and the equations are scaled by h^2. The Picard iteration freezes the coefficient at the current iterate u_k and solves A(u_k) u_{k+1} = f, until the nonlinear residual f - A(u_k) u_k is small. All matrices A(u_k) have the same sparsity pattern, only their values change.

```
const auto N = static_cast<GlobalIndexType>(grid_dim);
const auto num_rows = grid_dim * grid_dim;
const ValueType h = 1.0 / static_cast<ValueType>(grid_dim + 1);
const ValueType source = 50.0;
auto k = [](ValueType u) { return 1 + u * u; };
```

Each rank owns (nearly) the same number of consecutive rows.

```
auto partition = gko::share(part_type::build_from_global_size_uniform(
    exec->get_master(), comm.size(),
    static_cast<GlobalIndexType>(num_rows)));
const auto range_start = partition->get_range_bounds()[rank];
const auto range_end = partition->get_range_bounds()[rank + 1];
```

The edge between the neighbouring points i and j gets the coefficient (k(u_i) + k(u_j)) / 2. A rank only knows u in its own rows, so it adds the half k(u_i) / 2 of every edge of its own points i, also to the rows of neighbours owned by other ranks. Reading the data with assembly_mode::communicate sends these entries to their owners and adds up all contributions, so no values of u have to be exchanged. Edges to the boundary, where u = 0, only contribute to the diagonal.

```
auto assemble = [&](const dist_vec* u) {
    auto u_local = gko::make_temporary_clone(exec->get_master(),
                                             u->get_local_vector());
    gko::matrix_data<ValueType, GlobalIndexType> data;
    data.size = {num_rows, num_rows};
    for (auto i = range_start; i < range_end; i++) {
        const auto half_k = k(u_local->at(i - range_start, 0)) / 2;
        const auto row = i / N;
        const auto col = i % N;
        const GlobalIndexType neighbours[] = {
            row > 0 ?  i - N : -1, col > 0 ?  i - 1 :  -1,
            col < N - 1 ?  i + 1 :  -1, row < N - 1 ?  i + N : -1};
        ValueType diag{};
        for (const auto j :  neighbours) {
            if (j < 0) {
                diag += half_k + k(0) / 2;
            } else {
                diag += half_k;
                data.nonzeros.emplace_back(i, j, -half_k);
                data.nonzeros.emplace_back(j, i, -half_k);
                data.nonzeros.emplace_back(j, j, half_k);
            }
        }
        data.nonzeros.emplace_back(i, i, diag);
    }
    auto A = gko::share(dist_mtx::create(exec, comm));
    A->read_distributed(
        data, partition,
        gko::experimental::distributed::assembly_mode::communicate);
    return A;
};
gko::matrix_data<ValueType, GlobalIndexType> f_data;
f_data.size = {num_rows, 1};
for (auto i = range_start; i < range_end; i++) {
    f_data.nonzeros.emplace_back(i, 0, h * h * source);
}
auto f = dist_vec::create(exec, comm);
f->read_distributed(f_data, partition);
auto u = dist_vec::create_with_config_of(f);
u->fill(gko::zero<ValueType>());
auto r = dist_vec::create_with_config_of(f);
```

### Solver Setup

The factories are created once. The multigrid preconditioner with Pgm levels, a Schwarz-Jacobi smoother and Cg as coarsest solver is the one from the distributed-multigrid-preconditioned-solver example.

```
auto schwarz_bj_factory =
    gko::share(schwarz::build().with_local_solver(bj::build()).on(exec));
auto smoother_factory = gko::share(gko::solver::build_smoother(
    schwarz_bj_factory, 2u, static_cast<ValueType>(0.9)));
auto coarsest_factory = gko::share(
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
auto mg_factory = gko::share(
    mg::build()
```

```
        .with_mg_level(pgm::build().with_deterministic(true))
        .with_pre_smoother(smoother_factory)
        .with_coarsest_solver(coarsest_factory)
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
```

The Cg factory has no preconditioner, it is set on each solver.

```
auto cg_factory =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(1e-8))
        .on(exec);
auto logger = gko::share(gko::log::Convergence<ValueType>::create());
```

### The Picard Iteration with Multigrid Reuse

On its first call, generate_reuse() records the multigrid hierarchy, in particular the Pgm aggregates and transfer operators, in the reuse data. Later calls on a matrix with the same sparsity pattern keep them and only recompute the coarse matrices, the smoothers and the coarsest solver. Like generate(), every call returns a new preconditioner that does not depend on the reuse data. Replacing the reuse data by an empty one starts the recording again.

The kept aggregates only fit the later matrices if the recorded matrix is representative. Here, u_0 = 0 gives k = 1 everywhere, the constant-coefficient Laplacian, on which Pgm coarsens poorly. So the first iteration uses generate() and the recording starts in the second one.

```
auto reuse_data = mg_factory->create_empty_reuse_data();
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto norm = vec::create(exec, gko::dim<2>{1, 1});
f->compute_norm2(norm);
const auto f_norm = exec->copy_val_to_host(norm->get_const_values());
const ValueType tolerance = 1e-6;
if (rank == 0) {
    std::cout « "Num rows in matrix:  " « num_rows
              « "\nNum ranks:  " « comm.size() « "\n\n"
              « std::setw(5) « "iter" « std::setw(14) « "residual"
              « std::setw(10) « "setup" « std::setw(14) « "setup time"
              « std::setw(10) « "cg iters" « std::endl;
}
auto total_setup_time = 0.0;
gko::size_type total_cg_iters = 0;
gko::size_type iter = 0;
ValueType residual{};
for (;; iter++) {
    auto A = assemble(u.get());
    r->copy_from(f);
    A->apply(neg_one, u, one, r);
    r->compute_norm2(norm);
    residual = exec->copy_val_to_host(norm->get_const_values()) / f_norm;
    if (residual < tolerance || iter == max_picard_iters) {
        break;
    }
    const bool full_setup =
        iter <= 1 || (refresh_interval > 0 && iter % refresh_interval == 0);
    if (full_setup) {
        reuse_data = mg_factory->create_empty_reuse_data();
    }
    const auto t_setup = now();
    auto preconditioner =
        gko::share(iter == 0 ?  mg_factory->generate(A)
                             :  mg_factory->generate_reuse(A, *reuse_data));
    const auto setup_time = now() - t_setup;
    total_setup_time += setup_time;
    auto solver = cg_factory->generate(A);
    solver->set_preconditioner(preconditioner);
    solver->add_logger(logger);
    solver->apply(f, u);
    total_cg_iters += logger->get_num_iterations();
    if (rank == 0) {
        std::cout « std::setw(5) « iter « std::setw(14) « residual
                  « std::setw(10)
                  « (iter == 0 ?  "generate"
                               :  (full_setup ?  "record" :  "reuse"))
                  « std::setw(14) « setup_time « std::setw(10)
                  « logger->get_num_iterations() « std::endl;
    }
}
```

### Printing Results

```
    if (rank == 0) {
        std::cout « "\nPicard iterations:  " « iter
                  « "\nFinal residual:  " « residual
                  « "\nTotal setup time:  " « total_setup_time
                  « "\nTotal cg iterations:  " « total_cg_iters « std::endl;
    }
}
```

## Results

This is the expected output for `mpirun -n 4 ./distributed-multigrid-reuse`:

```
Num rows in matrix:  10000
Num ranks:  4
 iter      residual     setup    setup time  cg iters
    0             1  generate    0.00953173        27
    1       4.92756    record    0.00497701        37
    2      0.591079     reuse    0.00350611        27
    3      0.337404     reuse     0.0032835        29
    4     0.0844591     reuse    0.00333143        25
    5      0.023152     reuse    0.00330655        23
    6    0.00508051     reuse    0.00333697        21
    7   0.000999056     reuse    0.00331201        19
    8   0.000174637     reuse    0.00327568        16
    9   2.76182e-05     reuse    0.00329311        13
   10    3.9894e-06     reuse    0.00334166        11
Picard iterations:  11
Final residual:  5.31104e-07
Total setup time:  0.0444958
Total cg iterations:  248
```

The first iteration uses generate(), the second one records the hierarchy, and the later ones reuse it. With a refresh↩ _interval of 1, every iteration after the first one records a new hierarchy, which takes about 0.005 seconds instead of 0.0033, while the Picard and Cg iteration counts stay the same. The timings may vary depending on the machine.

## The plain program

```
#include <ginkgo/ginkgo.hpp>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
int main(int argc, char* argv[])
{
    const gko::experimental::mpi::environment env(argc, argv);
    using GlobalIndexType = gko::int64;
    using LocalIndexType = gko::int32;
    using ValueType = double;
    using dist_vec = gko::experimental::distributed::Vector<ValueType>;
    using dist_mtx =
        gko::experimental::distributed::Matrix<ValueType, LocalIndexType,
                                               GlobalIndexType>;
    using vec = gko::matrix::Dense<ValueType>;
    using part_type =
        gko::experimental::distributed::Partition<LocalIndexType,
                                                  GlobalIndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using schwarz = gko::experimental::distributed::preconditioner::Schwarz<
        ValueType, LocalIndexType, GlobalIndexType>;
    using bj = gko::preconditioner::Jacobi<ValueType, LocalIndexType>;
    using mg = gko::solver::Multigrid;
    using pgm = gko::multigrid::Pgm<ValueType, LocalIndexType>;
    const auto comm = gko::experimental::mpi::communicator(MPI_COMM_WORLD);
    const auto rank = comm.rank();
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        if (rank == 0) {
            std::cerr « "Usage:  " « argv[0]
                     « " [executor] [grid_side] [max_picard_iters]"
                     « " [refresh_interval]" « std::endl;
        }
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

```
const auto grid_dim =
    static_cast<gko::size_type>(argc >= 3 ?  std::atoi(argv[2]) :  100);
const auto max_picard_iters =
    static_cast<gko::size_type>(argc >= 4 ?  std::atoi(argv[3]) :  30);
const auto refresh_interval =
    static_cast<gko::size_type>(argc >= 5 ?  std::atoi(argv[4]) :  0);
const std::map<std::string,
               std::function<std::shared_ptr<gko::Executor>(MPI_Comm)>>
    executor_factory_mpi{
        {"reference",
         [](MPI_Comm) { return gko::ReferenceExecutor::create(); }},
        {"omp", [](MPI_Comm) { return gko::OmpExecutor::create(); }},
        {"cuda",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::CudaExecutor::get_num_devices());
             return gko::CudaExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"hip",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::HipExecutor::get_num_devices());
             return gko::HipExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"dpcpp", [](MPI_Comm comm) {
             int device_id = 0;
             if (gko::DpcppExecutor::get_num_devices("gpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("gpu"));
             } else if (gko::DpcppExecutor::get_num_devices("cpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("cpu"));
             } else {
                 throw std::runtime_error("No suitable DPC++ devices");
             }
             return gko::DpcppExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }}};
auto exec = executor_factory_mpi.at(executor_string)(MPI_COMM_WORLD);
auto now = [&] {
    exec->synchronize();
    comm.synchronize();
    return gko::experimental::mpi::get_walltime();
};
const auto N = static_cast<GlobalIndexType>(grid_dim);
const auto num_rows = grid_dim * grid_dim;
const ValueType h = 1.0 / static_cast<ValueType>(grid_dim + 1);
const ValueType source = 50.0;
auto k = [](ValueType u) { return 1 + u * u; };
auto partition = gko::share(part_type::build_from_global_size_uniform(
    exec->get_master(), comm.size(),
    static_cast<GlobalIndexType>(num_rows)));
const auto range_start = partition->get_range_bounds()[rank];
const auto range_end = partition->get_range_bounds()[rank + 1];
auto assemble = [&](const dist_vec* u) {
    auto u_local = gko::make_temporary_clone(exec->get_master(),
                                             u->get_local_vector());
    gko::matrix_data<ValueType, GlobalIndexType> data;
    data.size = {num_rows, num_rows};
    for (auto i = range_start; i < range_end; i++) {
        const auto half_k = k(u_local->at(i - range_start, 0)) / 2;
        const auto row = i / N;
        const auto col = i % N;
        const GlobalIndexType neighbours[] = {
            row > 0 ?  i - N : -1, col > 0 ?  i - 1 :  -1,
            col < N - 1 ?  i + 1 :  -1, row < N - 1 ?  i + N : -1};
        ValueType diag{};
        for (const auto j :  neighbours) {
            if (j < 0) {
                diag += half_k + k(0) / 2;
            } else {
                diag += half_k;
                data.nonzeros.emplace_back(i, j, -half_k);
                data.nonzeros.emplace_back(j, i, -half_k);
                data.nonzeros.emplace_back(j, j, half_k);
            }
        }
        data.nonzeros.emplace_back(i, i, diag);
    }
    auto A = gko::share(dist_mtx::create(exec, comm));
    A->read_distributed(
        data, partition,
        gko::experimental::distributed::assembly_mode::communicate);
    return A;
};
```

```
gko::matrix_data<ValueType, GlobalIndexType> f_data;
f_data.size = {num_rows, 1};
for (auto i = range_start; i < range_end; i++) {
    f_data.nonzeros.emplace_back(i, 0, h * h * source);
}
auto f = dist_vec::create(exec, comm);
f->read_distributed(f_data, partition);
auto u = dist_vec::create_with_config_of(f);
u->fill(gko::zero<ValueType>());
auto r = dist_vec::create_with_config_of(f);
auto schwarz_bj_factory =
    gko::share(schwarz::build().with_local_solver(bj::build()).on(exec));
auto smoother_factory = gko::share(gko::solver::build_smoother(
    schwarz_bj_factory, 2u, static_cast<ValueType>(0.9)));
auto coarsest_factory = gko::share(
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
auto mg_factory = gko::share(
    mg::build()
        .with_mg_level(pgm::build().with_deterministic(true))
        .with_pre_smoother(smoother_factory)
        .with_coarsest_solver(coarsest_factory)
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto cg_factory =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(1e-8))
        .on(exec);
auto logger = gko::share(gko::log::Convergence<ValueType>::create());
auto reuse_data = mg_factory->create_empty_reuse_data();
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto norm = vec::create(exec, gko::dim<2>{1, 1});
f->compute_norm2(norm);
const auto f_norm = exec->copy_val_to_host(norm->get_const_values());
const ValueType tolerance = 1e-6;
if (rank == 0) {
    std::cout « "Num rows in matrix:  " « num_rows
              « "\nNum ranks:  " « comm.size() « "\n\n"
              « std::setw(5) « "iter" « std::setw(14) « "residual"
              « std::setw(10) « "setup" « std::setw(14) « "setup time"
              « std::setw(10) « "cg iters" « std::endl;
}
auto total_setup_time = 0.0;
gko::size_type total_cg_iters = 0;
gko::size_type iter = 0;
ValueType residual{};
for (;; iter++) {
    auto A = assemble(u.get());
    r->copy_from(f);
    A->apply(neg_one, u, one, r);
    r->compute_norm2(norm);
    residual = exec->copy_val_to_host(norm->get_const_values()) / f_norm;
    if (residual < tolerance || iter == max_picard_iters) {
        break;
    }
    const bool full_setup =
        iter <= 1 || (refresh_interval > 0 && iter % refresh_interval == 0);
    if (full_setup) {
        reuse_data = mg_factory->create_empty_reuse_data();
    }
    const auto t_setup = now();
    auto preconditioner =
        gko::share(iter == 0 ?  mg_factory->generate(A)
                             :  mg_factory->generate_reuse(A, *reuse_data));
    const auto setup_time = now() - t_setup;
    total_setup_time += setup_time;
    auto solver = cg_factory->generate(A);
    solver->set_preconditioner(preconditioner);
    solver->add_logger(logger);
    solver->apply(f, u);
    total_cg_iters += logger->get_num_iterations();
    if (rank == 0) {
        std::cout « std::setw(5) « iter « std::setw(14) « residual
                  « std::setw(10)
                  « (iter == 0 ?  "generate"
                               :  (full_setup ?  "record" :  "reuse"))
                  « std::setw(14) « setup_time « std::setw(10)
                  « logger->get_num_iterations() « std::endl;
    }
}
if (rank == 0) {
    std::cout « "\nPicard iterations:  " « iter
              « "\nFinal residual:  " « residual
```

```
                  « "\nTotal setup time:  " « total_setup_time
                  « "\nTotal cg iterations:  " « total_cg_iters « std::endl;
    }
}
```

# Chapter 16

# The distributed-solver program

The distributed solver example..

This example depends on simple-solver, three-pt-stencil-solver.

## Introduction

This distributed solver example should help you understand the basics of using Ginkgo in a distributed setting. The example will solve a simple 1D Laplace equation where the system can be distributed row-wise to multiple processes. To run the solver with multiple processes, use `mpirun -n NUM_PROCS ./distributed-solver [executor] [num_grid_points] [num_iterations]`.

If you are using GPU devices, please make sure that you run this example with at most as many processes as you have GPU devices available.

## The commented program

### Include files

This is the main ginkgo header file.

```
#include <ginkgo/ginkgo.hpp>
```

Add the C++ iostream header to output information to the console.

```
#include <iostream>
```

Add the STL map header for the executor selection

```
#include <map>
```

Add the string manipulation header to handle strings.

```
#include <string>
int main(int argc, char* argv[])
{
```

### Initialize the MPI environment

Since this is an MPI program, we need to initialize and finalize MPI at the begin and end respectively of our program. This can be easily done with the following helper construct that uses RAII to automate the initialization and finalization.

```
const gko::experimental::mpi::environment env(argc, argv);
```

### Type Definitions

Define the needed types. In a parallel program we need to differentiate between global and local indices, thus we have two index types.

```
using GlobalIndexType = gko::int64;
using LocalIndexType = gko::int32;
```

The underlying value type.

```
using ValueType = double;
```

As vector type we use the following, which implements a subset of gko::matrix::Dense.

```
using dist_vec = gko::experimental::distributed::Vector<ValueType>;
```

As matrix type we simply use the following type, which can read distributed data and be applied to a distributed vector.

```
using dist_mtx =
    gko::experimental::distributed::Matrix<ValueType, LocalIndexType,
                                           GlobalIndexType>;
```

We still need a localized vector type to be used as scalars in the advanced apply operations.

```
using vec = gko::matrix::Dense<ValueType>;
```

The partition type describes how the rows of the matrices are distributed.

```
using part_type =
    gko::experimental::distributed::Partition<LocalIndexType,
                                              GlobalIndexType>;
```

We can use here the same solver type as you would use in a non-distributed program. Please note that not all solvers support distributed systems at the moment.

```
using solver = gko::solver::Cg<ValueType>;
using schwarz = gko::experimental::distributed::preconditioner::Schwarz<
    ValueType, LocalIndexType, GlobalIndexType>;
using bj = gko::preconditioner::Jacobi<ValueType, LocalIndexType>;
using pgm = gko::multigrid::Pgm<ValueType, LocalIndexType>;
```

Create an MPI communicator get the rank of the calling process.

```
const auto comm = gko::experimental::mpi::communicator(MPI_COMM_WORLD);
const auto rank = comm.rank();
```

### User Input Handling

User input settings:

- The executor, defaults to reference.
- The number of grid points, defaults to 100.
- The number of iterations, defaults to 1000.
- One-level, two-level preconditioner, and no preconditioner, defaults to one-level.

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    if (rank == 0) {
        std::cerr « "Usage:  " « argv[0]
                 « " [executor] [num_grid_points] [num_iterations] "
                    "[schwarz_prec_type] "
                 « std::endl;
    }
    std::exit(-1);
}
ValueType t_init = gko::experimental::mpi::get_walltime();
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const auto grid_dim =
    static_cast<gko::size_type>(argc >= 3 ?  std::atoi(argv[2]) :  100);
const auto num_iters =
    static_cast<gko::size_type>(argc >= 4 ?  std::atoi(argv[3]) :  1000);
std::string schw_type = argc >= 5 ?  argv[4] :  "one-level";
const std::map<std::string,
               std::function<std::shared_ptr<gko::Executor>(MPI_Comm)»
    executor_factory_mpi{
```

```
        {"reference",
         [](MPI_Comm) { return gko::ReferenceExecutor::create(); }},
        {"omp", [](MPI_Comm) { return gko::OmpExecutor::create(); }},
        {"cuda",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::CudaExecutor::get_num_devices());
             return gko::CudaExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"hip",
         [](MPI_Comm comm) {
             int device_id = gko::experimental::mpi::map_rank_to_device_id(
                 comm, gko::HipExecutor::get_num_devices());
             return gko::HipExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }},
        {"dpcpp", [](MPI_Comm comm) {
             int device_id = 0;
             if (gko::DpcppExecutor::get_num_devices("gpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("gpu"));
             } else if (gko::DpcppExecutor::get_num_devices("cpu")) {
                 device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::DpcppExecutor::get_num_devices("cpu"));
             } else {
                 throw std::runtime_error("No suitable DPC++ devices");
             }
             return gko::DpcppExecutor::create(
                 device_id, gko::ReferenceExecutor::create());
         }}};
auto exec = executor_factory_mpi.at(executor_string)(MPI_COMM_WORLD);
```

### Creating the Distributed Matrix and Vectors

As a first step, we create a partition of the rows. The partition consists of ranges of consecutive rows which are assigned a part-id. These part-ids will be used for the distributed data structures to determine which rows will be stored locally. In this example each rank has (nearly) the same number of rows, so we can use the following specialized constructor. See gko::distributed::Partition for other modes of creating a partition.

```
const auto num_rows = grid_dim;
auto partition = gko::share(part_type::build_from_global_size_uniform(
    exec->get_master(), comm.size(),
    static_cast<GlobalIndexType>(num_rows)));
```

Assemble the matrix using a 3-pt stencil and fill the right-hand-side with a sine value. The distributed matrix supports only constructing an empty matrix of zero size and filling in the values with gko::experimental::distributed::Matrix::read_distributed. Only the data that belongs to the rows by this rank will be assembled.

```
gko::matrix_data<ValueType, GlobalIndexType> A_data;
gko::matrix_data<ValueType, GlobalIndexType> b_data;
gko::matrix_data<ValueType, GlobalIndexType> x_data;
A_data.size = {num_rows, num_rows};
b_data.size = {num_rows, 1};
x_data.size = {num_rows, 1};
const auto range_start = partition->get_range_bounds()[rank];
const auto range_end = partition->get_range_bounds()[rank + 1];
for (auto i = range_start; i < range_end; i++) {
    if (i > 0) {
        A_data.nonzeros.emplace_back(i, i - 1, -1);
    }
    A_data.nonzeros.emplace_back(i, i, 2);
    if (i < grid_dim - 1) {
        A_data.nonzeros.emplace_back(i, i + 1, -1);
    }
    b_data.nonzeros.emplace_back(i, 0, std::sin(i * 0.01));
    x_data.nonzeros.emplace_back(i, 0, gko::zero<ValueType>());
}
```

Take timings.

```
comm.synchronize();
ValueType t_init_end = gko::experimental::mpi::get_walltime();
```

Read the matrix data, currently this is only supported on CPU executors. This will also set up the communication pattern needed for the distributed matrix-vector multiplication.

```
auto A_host = gko::share(dist_mtx::create(exec->get_master(), comm));
auto x_host = dist_vec::create(exec->get_master(), comm);
auto b_host = dist_vec::create(exec->get_master(), comm);
A_host->read_distributed(A_data, partition);
```

```
b_host->read_distributed(b_data, partition);
x_host->read_distributed(x_data, partition);
```

After reading, the matrix and vector can be moved to the chosen executor, since the distributed matrix supports SpMV also on devices.

```
auto A = gko::share(dist_mtx::create(exec, comm));
auto x = dist_vec::create(exec, comm);
auto b = dist_vec::create(exec, comm);
A->copy_from(A_host);
b->copy_from(b_host);
x->copy_from(x_host);
```

Take timings.

```
comm.synchronize();
ValueType t_read_setup_end = gko::experimental::mpi::get_walltime();
```

### Solve the Distributed System

Generate the solver, this is the same as in the non-distributed case. with a local block diagonal preconditioner.

Setup the local block diagonal solver factory.

```
auto local_solver = gko::share(bj::build().on(exec));
```

Setup the coarse solver. If it is more accurate, then the outer iterations will reduce, but the cost of the coarse solve increases. The coarse solver can in turn have another Schwarz preconditioner if needed.

```
auto coarse_solver = gko::share(
    solver::build()
        .with_preconditioner(
            schwarz::build().with_local_solver(local_solver).on(exec))
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(1000u).on(exec),
            gko::stop::ResidualNorm<ValueType>::build()
                .with_reduction_factor(1e-7)
                .on(exec))
        .on(exec));
auto pgm_fac = gko::share(pgm::build().on(exec));
```

Setup the stopping criterion and logger

```
const gko::remove_complex<ValueType> reduction_factor{1e-8};
std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
    gko::log::Convergence<ValueType>::create();
std::shared_ptr<gko::LinOp> Ainv{};
```

The benefit of the two-level Schwarz preconditioner is generally observable (in runtime and in number of iterations) usually for larger problem sizes and for larger number of ranks.

```
if (schw_type == "two-level") {
    Ainv =
        solver::build()
            .with_preconditioner(schwarz::build()
                                     .with_local_solver(local_solver)
                                     .with_coarse_level(pgm_fac)
                                     .with_coarse_solver(coarse_solver)
                                     .on(exec))
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(num_iters).on(
                    exec),
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(reduction_factor)
                    .on(exec))
            .on(exec)
            ->generate(A);
} else if (schw_type == "one-level") {
    Ainv =
        solver::build()
            .with_preconditioner(
                schwarz::build().with_local_solver(local_solver).on(exec))
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(num_iters).on(
                    exec),
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(reduction_factor)
                    .on(exec))
            .on(exec)
            ->generate(A);
} else {
```

```
    schw_type = "no-precond";
    Ainv =
        solver::build()
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(num_iters).on(
                    exec),
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(reduction_factor)
                    .on(exec))
            .on(exec)
            ->generate(A);
}
```

Add logger to the generated solver to log the iteration count and residual norm

```
Ainv->add_logger(logger);
```

Take timings.

```
comm.synchronize();
ValueType t_solver_generate_end = gko::experimental::mpi::get_walltime();
```

Apply the distributed solver, this is the same as in the non-distributed case.

```
Ainv->apply(b, x);
```

Take timings.

```
comm.synchronize();
ValueType t_end = gko::experimental::mpi::get_walltime();
```

Get the residual.

```
auto res_norm = gko::as<vec>(logger->get_residual_norm());
auto host_res = gko::make_temporary_clone(exec->get_master(), res_norm);
```

### Printing Results

Print the achieved residual norm and timings on rank 0.

```
if (comm.rank() == 0) {
```

clang-format off

```
std::cout << "\nNum rows in matrix:  " << num_rows
          << "\nNum ranks:  " << comm.size()
          << "\nPrecond type:  " << schw_type
          << "\nFinal Res norm:  " << *host_res->get_const_values()
          << "\nIteration count:  " << logger->get_num_iterations()
          << "\nInit time:  " << t_init_end - t_init
          << "\nRead time:  " << t_read_setup_end - t_init
          << "\nSolver generate time:  " << t_solver_generate_end - t_read_setup_end
          << "\nSolver apply time:  " << t_end - t_solver_generate_end
          << "\nTotal time:  " << t_end - t_init
          << std::endl;
```

clang-format on

```
    }
}
```

## Results

This is the expected output for `mpirun -n 4 ./distributed-solver`:

```
Num rows in matrix:  100
Num ranks:  4
Final Res norm:  5.58392e-12
Iteration count:  7
Init time:  0.0663887
Read time:  0.0729806
Solver generate time:  7.6348e-05
Solver apply time:  0.0680783
Total time:  0.141351
```

The timings may vary depending on the machine.

## The plain program

```
#include <ginkgo/ginkgo.hpp>
#include <iostream>
#include <map>
#include <string>
int main(int argc, char* argv[])
{
    const gko::experimental::mpi::environment env(argc, argv);
    using GlobalIndexType = gko::int64;
    using LocalIndexType = gko::int32;
    using ValueType = double;
    using dist_vec = gko::experimental::distributed::Vector<ValueType>;
    using dist_mtx =
        gko::experimental::distributed::Matrix<ValueType, LocalIndexType,
                                               GlobalIndexType>;
    using vec = gko::matrix::Dense<ValueType>;
    using part_type =
        gko::experimental::distributed::Partition<LocalIndexType,
                                                  GlobalIndexType>;
    using solver = gko::solver::Cg<ValueType>;
    using schwarz = gko::experimental::distributed::preconditioner::Schwarz<
        ValueType, LocalIndexType, GlobalIndexType>;
    using bj = gko::preconditioner::Jacobi<ValueType, LocalIndexType>;
    using pgm = gko::multigrid::Pgm<ValueType, LocalIndexType>;
    const auto comm = gko::experimental::mpi::communicator(MPI_COMM_WORLD);
    const auto rank = comm.rank();
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        if (rank == 0) {
            std::cerr « "Usage:  " « argv[0]
                     « " [executor] [num_grid_points] [num_iterations] "
                        "[schwarz_prec_type] "
                     « std::endl;
        }
        std::exit(-1);
    }
    ValueType t_init = gko::experimental::mpi::get_walltime();
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    const auto grid_dim =
        static_cast<gko::size_type>(argc >= 3 ?  std::atoi(argv[2]) :  100);
    const auto num_iters =
        static_cast<gko::size_type>(argc >= 4 ?  std::atoi(argv[3]) :  1000);
    std::string schw_type = argc >= 5 ?  argv[4] :  "one-level";
    const std::map<std::string,
                   std::function<std::shared_ptr<gko::Executor>(MPI_Comm)»
        executor_factory_mpi{
            {"reference",
             [](MPI_Comm) { return gko::ReferenceExecutor::create(); }},
            {"omp", [](MPI_Comm) { return gko::OmpExecutor::create(); }},
            {"cuda",
             [](MPI_Comm comm) {
                 int device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::CudaExecutor::get_num_devices());
                 return gko::CudaExecutor::create(
                     device_id, gko::ReferenceExecutor::create());
             }},
            {"hip",
             [](MPI_Comm comm) {
                 int device_id = gko::experimental::mpi::map_rank_to_device_id(
                     comm, gko::HipExecutor::get_num_devices());
                 return gko::HipExecutor::create(
                     device_id, gko::ReferenceExecutor::create());
             }},
            {"dpcpp", [](MPI_Comm comm) {
                 int device_id = 0;
                 if (gko::DpcppExecutor::get_num_devices("gpu")) {
                     device_id = gko::experimental::mpi::map_rank_to_device_id(
                         comm, gko::DpcppExecutor::get_num_devices("gpu"));
                 } else if (gko::DpcppExecutor::get_num_devices("cpu")) {
                     device_id = gko::experimental::mpi::map_rank_to_device_id(
                         comm, gko::DpcppExecutor::get_num_devices("cpu"));
                 } else {
                     throw std::runtime_error("No suitable DPC++ devices");
                 }
                 return gko::DpcppExecutor::create(
                     device_id, gko::ReferenceExecutor::create());
             }}};
    auto exec = executor_factory_mpi.at(executor_string)(MPI_COMM_WORLD);
    const auto num_rows = grid_dim;
    auto partition = gko::share(part_type::build_from_global_size_uniform(
        exec->get_master(), comm.size(),
        static_cast<GlobalIndexType>(num_rows)));
    gko::matrix_data<ValueType, GlobalIndexType> A_data;
    gko::matrix_data<ValueType, GlobalIndexType> b_data;
    gko::matrix_data<ValueType, GlobalIndexType> x_data;
    A_data.size = {num_rows, num_rows};
    b_data.size = {num_rows, 1};
```

```
    x_data.size = {num_rows, 1};
    const auto range_start = partition->get_range_bounds()[rank];
    const auto range_end = partition->get_range_bounds()[rank + 1];
    for (auto i = range_start; i < range_end; i++) {
        if (i > 0) {
            A_data.nonzeros.emplace_back(i, i - 1, -1);
        }
        A_data.nonzeros.emplace_back(i, i, 2);
        if (i < grid_dim - 1) {
            A_data.nonzeros.emplace_back(i, i + 1, -1);
        }
        b_data.nonzeros.emplace_back(i, 0, std::sin(i * 0.01));
        x_data.nonzeros.emplace_back(i, 0, gko::zero<ValueType>());
    }
    comm.synchronize();
    ValueType t_init_end = gko::experimental::mpi::get_walltime();
    auto A_host = gko::share(dist_mtx::create(exec->get_master(), comm));
    auto x_host = dist_vec::create(exec->get_master(), comm);
    auto b_host = dist_vec::create(exec->get_master(), comm);
    A_host->read_distributed(A_data, partition);
    b_host->read_distributed(b_data, partition);
    x_host->read_distributed(x_data, partition);
    auto A = gko::share(dist_mtx::create(exec, comm));
    auto x = dist_vec::create(exec, comm);
    auto b = dist_vec::create(exec, comm);
    A->copy_from(A_host);
    b->copy_from(b_host);
    x->copy_from(x_host);
    comm.synchronize();
    ValueType t_read_setup_end = gko::experimental::mpi::get_walltime();
    auto local_solver = gko::share(bj::build().on(exec));
    auto coarse_solver = gko::share(
        solver::build()
            .with_preconditioner(
                schwarz::build().with_local_solver(local_solver).on(exec))
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(1000u).on(exec),
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(1e-7)
                    .on(exec))
            .on(exec));
    auto pgm_fac = gko::share(pgm::build().on(exec));
    const gko::remove_complex<ValueType> reduction_factor{1e-8};
    std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
        gko::log::Convergence<ValueType>::create();
    std::shared_ptr<gko::LinOp> Ainv{};
    if (schw_type == "two-level") {
        Ainv =
            solver::build()
                .with_preconditioner(schwarz::build()
                                         .with_local_solver(local_solver)
                                         .with_coarse_level(pgm_fac)
                                         .with_coarse_solver(coarse_solver)
                                         .on(exec))
                .with_criteria(
                    gko::stop::Iteration::build().with_max_iters(num_iters).on(
                        exec),
                    gko::stop::ResidualNorm<ValueType>::build()
                        .with_reduction_factor(reduction_factor)
                        .on(exec))
                .on(exec)
                ->generate(A);
    } else if (schw_type == "one-level") {
        Ainv =
            solver::build()
                .with_preconditioner(
                    schwarz::build().with_local_solver(local_solver).on(exec))
                .with_criteria(
                    gko::stop::Iteration::build().with_max_iters(num_iters).on(
                        exec),
                    gko::stop::ResidualNorm<ValueType>::build()
                        .with_reduction_factor(reduction_factor)
                        .on(exec))
                .on(exec)
                ->generate(A);
    } else {
        schw_type = "no-precond";
        Ainv =
            solver::build()
                .with_criteria(
                    gko::stop::Iteration::build().with_max_iters(num_iters).on(
                        exec),
                    gko::stop::ResidualNorm<ValueType>::build()
                        .with_reduction_factor(reduction_factor)
                        .on(exec))
                .on(exec)
                ->generate(A);
```

```
    }
    Ainv->add_logger(logger);
    comm.synchronize();
    ValueType t_solver_generate_end = gko::experimental::mpi::get_walltime();
    Ainv->apply(b, x);
    comm.synchronize();
    ValueType t_end = gko::experimental::mpi::get_walltime();
    auto res_norm = gko::as<vec>(logger->get_residual_norm());
    auto host_res = gko::make_temporary_clone(exec->get_master(), res_norm);
    if (comm.rank() == 0) {
        std::cout << "\nNum rows in matrix:  " << num_rows
                  << "\nNum ranks:  " << comm.size()
                  << "\nPrecond type:  " << schw_type
                  << "\nFinal Res norm:  " << *host_res->get_const_values()
                  << "\nIteration count:  " << logger->get_num_iterations()
                  << "\nInit time:  " << t_init_end - t_init
                  << "\nRead time:  " << t_read_setup_end - t_init
                  << "\nSolver generate time:  " << t_solver_generate_end - t_read_setup_end
                  << "\nSolver apply time:  " << t_end - t_solver_generate_end
                  << "\nTotal time:  " << t_end - t_init
                  << std::endl;
    }
}
```

# Chapter 17

# The external-lib-interfacing program

The external library(deal.II) interfacing example..

## Introduction

### About the example

## The commented program

```
#include <deal.II/base/function.h>
#include <deal.II/base/logstream.h>
#include <deal.II/base/quadrature_lib.h>
#include <deal.II/dofs/dof_accessor.h>
#include <deal.II/dofs/dof_handler.h>
#include <deal.II/dofs/dof_tools.h>
#include <deal.II/fe/fe_q.h>
#include <deal.II/fe/fe_values.h>
#include <deal.II/grid/grid_generator.h>
#include <deal.II/grid/grid_out.h>
#include <deal.II/grid/grid_refinement.h>
#include <deal.II/grid/tria.h>
#include <deal.II/grid/tria_accessor.h>
#include <deal.II/grid/tria_iterator.h>
#include <deal.II/lac/constraint_matrix.h>
#include <deal.II/lac/dynamic_sparsity_pattern.h>
#include <deal.II/lac/full_matrix.h>
#include <deal.II/lac/precondition.h>
#include <deal.II/lac/solver_bicgstab.h>
#include <deal.II/lac/sparse_matrix.h>
#include <deal.II/lac/vector.h>
#include <deal.II/numerics/data_out.h>
#include <deal.II/numerics/matrix_tools.h>
#include <deal.II/numerics/vector_tools.h>
```

The following two files provide classes and information for multithreaded programs. In the first one, the classes and functions are declared which we need to do assembly in parallel (i.e. the `WorkStream` namespace). The second file has a class MultithreadInfo which can be used to query the number of processors in your system, which is often useful when deciding how many threads to start in parallel.

```
#include <deal.II/base/multithread_info.h>
#include <deal.II/base/work_stream.h>
```

The next new include file declares a base class `TensorFunction` not unlike the `Function` class, but with the difference that the return value is tensor-valued rather than scalar of vector-valued.

```
#include <deal.II/base/tensor_function.h>
#include <deal.II/numerics/error_estimator.h>
```

Ginkgo's header file

```
#include <ginkgo/ginkgo.hpp>
```

This is C++, as we want to write some output to disk:

```
#include <fstream>
#include <iostream>
```

The last step is as in previous programs:

```
namespace Step9 {
using namespace dealii;
```

### AdvectionProblem class declaration

Following we declare the main class of this program. It is very much like the main classes of previous examples, so we again only comment on the differences.

```
template <int dim>
class AdvectionProblem {
public:
    AdvectionProblem();
     AdvectionProblem();
    void run();
private:
    void setup_system();
```

The next set of functions will be used to assemble the matrix. However, unlike in the previous examples, the `assemble_system()` function will not do the work itself, but rather will delegate the actual assembly to helper functions `assemble_local_system()` and `copy_local_to_global()`. The rationale is that matrix assembly can be parallelized quite well, as the computation of the local contributions on each cell is entirely independent of other cells, and we only have to synchronize when we add the contribution of a cell to the global matrix.

The strategy for parallelization we choose here is one of the possibilities mentioned in detail in the threads module in the documentation. Specifically, we will use the WorkStream approach discussed there. Since there is so much documentation in this module, we will not repeat the rationale for the design choices here (for example, if you read through the module mentioned above, you will understand what the purpose of the `AssemblyScratchData` and `AssemblyCopyData` structures is). Rather, we will only discuss the specific implementation.

If you read the page mentioned above, you will find that in order to parallelize assembly, we need two data structures – one that corresponds to data that we need during local integration ("scratch data", i.e., things we only need as temporary storage), and one that carries information from the local integration to the function that then adds the local contributions to the corresponding elements of the global matrix. The former of these typically contains the FEValues and FEFaceValues objects, whereas the latter has the local matrix, local right hand side, and information about which degrees of freedom live on the cell for which we are assembling a local contribution. With this information, the following should be relatively self-explanatory:

```
struct AssemblyScratchData {
    AssemblyScratchData(const FiniteElement<dim>& fe);
    AssemblyScratchData(const AssemblyScratchData& scratch_data);
    FEValues<dim> fe_values;
    FEFaceValues<dim> fe_face_values;
};
struct AssemblyCopyData {
    FullMatrix<double> cell_matrix;
    Vector<double> cell_rhs;
    std::vector<types::global_dof_index> local_dof_indices;
};
void assemble_system();
void local_assemble_system(
    const typename DoFHandler<dim>::active_cell_iterator& cell,
    AssemblyScratchData& scratch, AssemblyCopyData& copy_data);
void copy_local_to_global(const AssemblyCopyData& copy_data);
```

The following functions again are as in previous examples, as are the subsequent variables.

```
    void solve();
    void refine_grid();
    void output_results(const unsigned int cycle) const;
    Triangulation<dim> triangulation;
    DoFHandler<dim> dof_handler;
    FE_Q<dim> fe;
    ConstraintMatrix hanging_node_constraints;
    SparsityPattern sparsity_pattern;
    SparseMatrix<double> system_matrix;
    Vector<double> solution;
    Vector<double> system_rhs;
};
```

### Equation data declaration

Next we declare a class that describes the advection field. This, of course, is a vector field with as many components as there are space dimensions. One could now use a class derived from the `Function` base class, as we have done for boundary values and coefficients in previous examples, but there is another possibility in the library, namely

a base class that describes tensor valued functions. In contrast to the usual `Function` objects, we provide the compiler with knowledge on the size of the objects of the return type. This enables the compiler to generate efficient code, which is not so simple for usual vector-valued functions where memory has to be allocated on the heap (thus, the `Function::vector_value` function has to be given the address of an object into which the result is to be written, in order to avoid copying and memory allocation and deallocation on the heap). In addition to the known size, it is possible not only to return vectors, but also tensors of higher rank; however, this is not very often requested by applications, to be honest...

The interface of the `TensorFunction` class is relatively close to that of the `Function` class, so there is probably no need to comment in detail the following declaration:

```
template <int dim>
class AdvectionField :  public TensorFunction<1, dim> {
public:
    AdvectionField() :  TensorFunction<1, dim>() {}
    virtual Tensor<1, dim> value(const Point<dim>& p) const;
    virtual void value_list(const std::vector<Point<dim»& points,
                            std::vector<Tensor<1, dim»& values) const;
```

In previous examples, we have used assertions that throw exceptions in several places. However, we have never seen how such exceptions are declared. This can be done as follows:

```
DeclException2(ExcDimensionMismatch, unsigned int, unsigned int,
               « "The vector has size " « arg1 « " but should have "
               « arg2 « " elements.");
```

The syntax may look a little strange, but is reasonable. The format is basically as follows: use the name of one of the macros `DeclExceptionN`, where `N` denotes the number of additional parameters which the exception object shall take. In this case, as we want to throw the exception when the sizes of two vectors differ, we need two arguments, so we use `DeclException2`. The first parameter then describes the name of the exception, while the following declare the data types of the parameters. The last argument is a sequence of output directives that will be piped into the `std::cerr` object, thus the strange format with the leading $<<$ operator and the like. Note that we can access the parameters which are passed to the exception upon construction (i.e. within the `Assert` call) by using the names `arg1` through `argN`, where `N` is the number of arguments as defined by the use of the respective macro `DeclExceptionN`.

To learn how the preprocessor expands this macro into actual code, please refer to the documentation of the exception classes in the base library. Suffice it to say that by this macro call, the respective exception class is declared, which also has error output functions already implemented.

```
};
```

The following two functions implement the interface described above. The first simply implements the function as described in the introduction, while the second uses the same trick to avoid calling a virtual function as has already been introduced in the previous example program. Note the check for the right sizes of the arguments in the second function, which should always be present in such functions; it is our experience that many if not most programming errors result from incorrectly initialized arrays, incompatible parameters to functions and the like; using assertion as in this case can eliminate many of these problems.

```
template <int dim>
Tensor<1, dim> AdvectionField<dim>::value(const Point<dim>& p)const
{
    Point<dim> value;
    value[0] = 2;
    for (unsigned int i = 1; i < dim; ++i)
        value[i] = 1 + 0.8 * std::sin(8 * numbers::PI * p[0]);
    return value;
}
template <int dim>
void AdvectionField<dim>::value_list(const std::vector<Point<dim»& points,
                                     std::vector<Tensor<1, dim»& values)const
{
    Assert(values.size() == points.size(),
           ExcDimensionMismatch(values.size(), points.size()));
    for (unsigned int i = 0; i < points.size(); ++i)
        values[i] = AdvectionField<dim>::value(points[i]);
}
```

Besides the advection field, we need two functions describing the source terms (`right hand side`) and the boundary values. First for the right hand side, which follows the same pattern as in previous examples. As described in the introduction, the source is a constant function in the vicinity of a source point, which we denote by the constant static variable `center_point`. We set the values of this center using the same template tricks as we have shown

in the step-7 example program. The rest is simple and has been shown previously, including the way to avoid virtual function calls in the `value_list` function.

```
template <int dim>
class RightHandSide :  public Function<dim> {
public:
    RightHandSide() :  Function<dim>() {}
    virtual double value(const Point<dim>& p,
                         const unsigned int component = 0) const;
    virtual void value_list(const std::vector<Point<dim>& points,
                            std::vector<double>& values,
                            const unsigned int component = 0) const;
private:
    static const Point<dim> center_point;
};
template <>
const Point<1> RightHandSide<1>::center_point = Point<1>(-0.75);
template <>
const Point<2> RightHandSide<2>::center_point = Point<2>(-0.75, -0.75);
template <>
const Point<3> RightHandSide<3>::center_point = Point<3>(-0.75, -0.75, -0.75);
```

The only new thing here is that we check for the value of the `component` parameter. As this is a scalar function, it is obvious that it only makes sense if the desired component has the index zero, so we assert that this is indeed the case. `ExcIndexRange` is a global predefined exception (probably the one most often used, we therefore made it global instead of local to some class), that takes three parameters: the index that is outside the allowed range, the first element of the valid range and the one past the last (i.e. again the half-open interval so often used in the C++ standard library):

```
template <int dim>
double RightHandSide<dim>::value(const Point<dim>& p,
                                 const unsigned int component)const
{
    (void)component;
    Assert(component == 0, ExcIndexRange(component, 0, 1));
    const double diameter = 0.1;
    return ((p - center_point).norm_square() < diameter * diameter
              ?  .1 / std::pow(diameter, dim)
              :  0);
}
template <int dim>
void RightHandSide<dim>::value_list(const std::vector<Point<dim>& points,
                                    std::vector<double>& values,
                                    const unsigned int component)const
{
    Assert(values.size() == points.size(),
           ExcDimensionMismatch(values.size(), points.size()));
    for (unsigned int i = 0; i < points.size(); ++i)
        values[i] = RightHandSide<dim>::value(points[i], component);
}
```

Finally for the boundary values, which is just another class derived from the `Function` base class:

```
template <int dim>
class BoundaryValues :  public Function<dim> {
public:
    BoundaryValues() :  Function<dim>() {}
    virtual double value(const Point<dim>& p,
                         const unsigned int component = 0) const;
    virtual void value_list(const std::vector<Point<dim>& points,
                            std::vector<double>& values,
                            const unsigned int component = 0) const;
};
template <int dim>
double BoundaryValues<dim>::value(const Point<dim>& p,
                                  const unsigned int component)const
{
    (void)component;
    Assert(component == 0, ExcIndexRange(component, 0, 1));
    const double sine_term =
        std::sin(16 * numbers::PI * std::sqrt(p.norm_square()));
    const double weight = std::exp(-5 * p.norm_square()) / std::exp(-5.);
    return sine_term * weight;
}
template <int dim>
void BoundaryValues<dim>::value_list(const std::vector<Point<dim>& points,
                                     std::vector<double>& values,
                                     const unsigned int component)const
{
    Assert(values.size() == points.size(),
           ExcDimensionMismatch(values.size(), points.size()));
    for (unsigned int i = 0; i < points.size(); ++i)
        values[i] = BoundaryValues<dim>::value(points[i], component);
}
```

### GradientEstimation class declaration

Now, finally, here comes the class that will compute the difference approximation of the gradient on each cell and weighs that with a power of the mesh size, as described in the introduction. This class is a simple version of the `DerivativeApproximation` class in the library, that uses similar techniques to obtain finite difference approximations of the gradient of a finite element field, or of higher derivatives.

The class has one public static function `estimate` that is called to compute a vector of error indicators, and a few private functions that do the actual work on all active cells. As in other parts of the library, we follow an informal convention to use vectors of floats for error indicators rather than the common vectors of doubles, as the additional accuracy is not necessary for estimated values.

In addition to these two functions, the class declares two exceptions which are raised when a cell has no neighbors in each of the space directions (in which case the matrix described in the introduction would be singular and can't be inverted), while the other one is used in the more common case of invalid parameters to a function, namely a vector of wrong size.

Two other comments: first, the class has no non-static member functions or variables, so this is not really a class, but rather serves the purpose of a `namespace` in C++. The reason that we chose a class over a namespace is that this way we can declare functions that are private. This can be done with namespaces as well, if one declares some functions in header files in the namespace and implements these and other functions in the implementation file. The functions not declared in the header file are still in the namespace but are not callable from outside. However, as we have only one file here, it is not possible to hide functions in the present case.

The second comment is that the dimension template parameter is attached to the function rather than to the class itself. This way, you don't have to specify the template parameter yourself as in most other cases, but the compiler can figure its value out itself from the dimension of the DoF handler object that one passes as first argument.

Before jumping into the fray with the implementation, let us also comment on the parallelization strategy. We have already introduced the necessary framework for using the WorkStream concept in the declaration of the main class of this program above. We will use it again here. In the current context, this means that we have to define (i) classes for scratch and copy objects, (ii) a function that does the local computation on one cell, and (iii) a function that copies the local result into a global object. Given this general framework, we will, however, deviate from it a bit. In particular, WorkStream was generally invented for cases where each local computation on a cell *adds* to a global object – for example, when assembling linear systems where we add local contributions into a global matrix and right hand side. WorkStream is designed to handle the potential conflict of multiple threads trying to do this addition at the same time, and consequently has to provide for some way to ensure that only thread gets to do this at a time. Here, however, the situation is slightly different: we compute contributions from every cell individually, but then all we need to do is put them into an element of an output vector that is unique to each cell. Consequently, there is no risk that the write operations from two cells might conflict, and the elaborate machinery of WorkStream to avoid conflicting writes is not necessary. Consequently, what we will do is this: We still need a scratch object that holds, for example, the FEValues object. However, we only create a fake, empty copy data structure. Likewise, we do need the function that computes local contributions, but since it can already put the result into its final location, we do not need a copy-local-to-global function and will instead give the WorkStream::run() function an empty function object – the equivalent to a NULL function pointer.

```
class GradientEstimation {
public:
    template <int dim>
    static void estimate(const DoFHandler<dim>& dof,
                         const Vector<double>& solution,
                         Vector<float>& error_per_cell);
    DeclException2(ExcInvalidVectorLength, int, int,
                   « "Vector has length " « arg1 « ", but should have "
                   « arg2);
    DeclException0(ExcInsufficientDirections);
private:
    template <int dim>
    struct EstimateScratchData {
        EstimateScratchData(const FiniteElement<dim>& fe,
                            const Vector<double>& solution,
                            Vector<float>& error_per_cell);
        EstimateScratchData(const EstimateScratchData& data);
        FEValues<dim> fe_midpoint_value;
        const Vector<double>& solution;
        Vector<float>& error_per_cell;
```

```
    };
    struct EstimateCopyData {};
    template <int dim>
    static void estimate_cell(
        const typename DoFHandler<dim>::active_cell_iterator& cell,
        EstimateScratchData<dim>& scratch_data,
        const EstimateCopyData& copy_data);
};
```

### AdvectionProblem class implementation

Now for the implementation of the main class. Constructor, destructor and the function `setup_system` follow the same pattern that was used previously, so we need not comment on these three function:

```
template <int dim>
AdvectionProblem<dim>::AdvectionProblem() :  dof_handler(triangulation), fe(1)
{}
template <int dim>
AdvectionProblem<dim>:: AdvectionProblem()
{
    dof_handler.clear();
}
template <int dim>
void AdvectionProblem<dim>::setup_system()
{
    dof_handler.distribute_dofs(fe);
    hanging_node_constraints.clear();
    DoFTools::make_hanging_node_constraints(dof_handler,
                                            hanging_node_constraints);
    hanging_node_constraints.close();
    DynamicSparsityPattern dsp(dof_handler.n_dofs(), dof_handler.n_dofs());
    DoFTools::make_sparsity_pattern(dof_handler, dsp, hanging_node_constraints,
                                    / *keep_constrained_dofs = * / true);
    sparsity_pattern.copy_from(dsp);
    system_matrix.reinit(sparsity_pattern);
    solution.reinit(dof_handler.n_dofs());
    system_rhs.reinit(dof_handler.n_dofs());
}
```

In the following function, the matrix and right hand side are assembled. As stated in the documentation of the main class above, it does not do this itself, but rather delegates to the function following next, utilizing the WorkStream concept discussed in threads .

If you have looked through the threads module, you will have seen that assembling in parallel does not take an incredible amount of extra code as long as you diligently describe what the scratch and copy data objects are, and if you define suitable functions for the local assembly and the copy operation from local contributions to global objects. This done, the following will do all the heavy lifting to get these operations done on multiple threads on as many cores as you have in your system:

```
template <int dim>
void AdvectionProblem<dim>::assemble_system()
{
    WorkStream::run(dof_handler.begin_active(), dof_handler.end(), *this,
                    &AdvectionProblem::local_assemble_system,
                    &AdvectionProblem::copy_local_to_global,
                    AssemblyScratchData(fe), AssemblyCopyData());
```

After the matrix has been assembled in parallel, we still have to eliminate hanging node constraints. This is something that can't be done on each of the threads separately, so we have to do it now. Note also, that unlike in previous examples, there are no boundary conditions to be applied to the system of equations. This, of course, is due to the fact that we have included them into the weak formulation of the problem.

```
    hanging_node_constraints.condense(system_matrix);
    hanging_node_constraints.condense(system_rhs);
}
```

As already mentioned above, we need to have scratch objects for the parallel computation of local contributions. These objects contain FEValues and FEFaceValues objects, and so we will need to have constructors and copy constructors that allow us to create them. In initializing them, note first that we use bilinear elements, soGauss formulae with two points in each space direction are sufficient. For the cell terms we need the values and gradients of the shape functions, the quadrature points in order to determine the source density and the advection field at a given point, and the weights of the quadrature points times the determinant of the Jacobian at these points. In contrast, for the boundary integrals, we don't need the gradients, but rather the normal vectors to the cells. This determines which update flags we will have to pass to the constructors of the members of the class:

```
template <int dim>
AdvectionProblem<dim>::AssemblyScratchData::AssemblyScratchData(
    const FiniteElement<dim>& fe)
    :  fe_values(fe, QGauss<dim>(2),
                 update_values | update_gradients | update_quadrature_points |
                     update_JxW_values),
      fe_face_values(fe, QGauss<dim - 1>(2),
                     update_values | update_quadrature_points |
                         update_JxW_values | update_normal_vectors)
{}
template <int dim>
AdvectionProblem<dim>::AssemblyScratchData::AssemblyScratchData(
    const AssemblyScratchData& scratch_data)
    :  fe_values(scratch_data.fe_values.get_fe(),
                 scratch_data.fe_values.get_quadrature(),
                 update_values | update_gradients | update_quadrature_points |
                     update_JxW_values),
      fe_face_values(scratch_data.fe_face_values.get_fe(),
                     scratch_data.fe_face_values.get_quadrature(),
                     update_values | update_quadrature_points |
                         update_JxW_values | update_normal_vectors)
{}
```

Now, this is the function that does the actual work. It is not very different from the `assemble_system` functions of previous example programs, so we will again only comment on the differences. The mathematical stuff follows closely what we have said in the introduction.

There are a number of points worth mentioning here, though. The first one is that we have moved the FEValues and FEFaceValues objects into the ScratchData object. We have done so because the alternative would have been to simply create one every time we get into this function – i.e., on every cell. It now turns out that the FEValues classes were written with the explicit goal of moving everything that remains the same from cell to cell into the construction of the object, and only do as little work as possible in FEValues::reinit() whenever we move to a new cell. What this means is that it would be very expensive to create a new object of this kind in this function as we would have to do it for every cell – exactly the thing we wanted to avoid with the FEValues class. Instead, what we do is create it only once (or a small number of times) in the scratch objects and then re-use it as often as we can.

This begs the question of whether there are other objects we create in this function whose creation is expensive compared to its use. Indeed, at the top of the function, we declare all sorts of objects. The `AdvectionField`, `RightHandSide` and `BoundaryValues` do not cost much to create, so there is no harm here. However, allocating memory in creating the `rhs_values` and similar variables below typically costs a significant amount of time, compared to just accessing the (temporary) values we store in them. Consequently, these would be candidates for moving into the `AssemblyScratchData` class. We will leave this as an exercise.

```
template <int dim>
void AdvectionProblem<dim>::local_assemble_system(
    const typename DoFHandler<dim>::active_cell_iterator& cell,
    AssemblyScratchData& scratch_data, AssemblyCopyData& copy_data)
{
```

First of all, we will need some objects that describe boundary values, right hand side function and the advection field. As we will only perform actions on these objects that do not change them, we declare them as constant, which can enable the compiler in some cases to perform additional optimizations.

```
const AdvectionField<dim> advection_field;
const RightHandSide<dim> right_hand_side;
const BoundaryValues<dim> boundary_values;
```

Then we define some abbreviations to avoid unnecessarily long lines:

```
const unsigned int dofs_per_cell = fe.dofs_per_cell;
const unsigned int n_q_points =
    scratch_data.fe_values.get_quadrature().size();
const unsigned int n_face_q_points =
    scratch_data.fe_face_values.get_quadrature().size();
```

We declare cell matrix and cell right hand side...

```
copy_data.cell_matrix.reinit(dofs_per_cell, dofs_per_cell);
copy_data.cell_rhs.reinit(dofs_per_cell);
```

... an array to hold the global indices of the degrees of freedom of the cell on which we are presently working...

```
copy_data.local_dof_indices.resize(dofs_per_cell);
```

... and array in which the values of right hand side, advection direction, and boundary values will be stored, for cell and face integrals respectively:

```
std::vector<double> rhs_values(n_q_points);
std::vector<Tensor<1, dim» advection_directions(n_q_points);
std::vector<double> face_boundary_values(n_face_q_points);
std::vector<Tensor<1, dim» face_advection_directions(n_face_q_points);
```

... then initialize the `FEValues` object...

```
scratch_data.fe_values.reinit(cell);
```

... obtain the values of right hand side and advection directions at the quadrature points...

```
advection_field.value_list(scratch_data.fe_values.get_quadrature_points(),
                           advection_directions);
right_hand_side.value_list(scratch_data.fe_values.get_quadrature_points(),
                           rhs_values);
```

... set the value of the streamline diffusion parameter as described in the introduction...

```
const double delta = 0.1 * cell->diameter();
```

... and assemble the local contributions to the system matrix and right hand side as also discussed above:

```
for (unsigned int q_point = 0; q_point < n_q_points; ++q_point)
    for (unsigned int i = 0; i < dofs_per_cell; ++i) {
        for (unsigned int j = 0; j < dofs_per_cell; ++j)
            copy_data.cell_matrix(i, j) +=
                ((advection_directions[q_point] *
                  scratch_data.fe_values.shape_grad(j, q_point) *
                  (scratch_data.fe_values.shape_value(i, q_point) +
                   delta *
                       (advection_directions[q_point] *
                        scratch_data.fe_values.shape_grad(i, q_point)))) *
                 scratch_data.fe_values.JxW(q_point));
        copy_data.cell_rhs(i) +=
            ((scratch_data.fe_values.shape_value(i, q_point) +
              delta * (advection_directions[q_point] *
                       scratch_data.fe_values.shape_grad(i, q_point))) *
             rhs_values[q_point] * scratch_data.fe_values.JxW(q_point));
    }
```

Besides the cell terms which we have built up now, the bilinear form of the present problem also contains terms on the boundary of the domain. Therefore, we have to check whether any of the faces of this cell are on the boundary of the domain, and if so assemble the contributions of this face as well. Of course, the bilinear form only contains contributions from the `inflow` part of the boundary, but to find out whether a certain part of a face of the present cell is part of the inflow boundary, we have to have information on the exact location of the quadrature points and on the direction of flow at this point; we obtain this information using the FEFaceValues object and only decide within the main loop whether a quadrature point is on the inflow boundary.

```
for (unsigned int face = 0; face < GeometryInfo<dim>::faces_per_cell;
     ++face)
    if (cell->face(face)->at_boundary()) {
```

Ok, this face of the present cell is on the boundary of the domain. Just as for the usual FEValues object which we have used in previous examples and also above, we have to reinitialize the FEFaceValues object for the present face:

```
scratch_data.fe_face_values.reinit(cell, face);
```

For the quadrature points at hand, we ask for the values of the inflow function and for the direction of flow:

```
boundary_values.value_list(
    scratch_data.fe_face_values.get_quadrature_points(),
    face_boundary_values);
advection_field.value_list(
    scratch_data.fe_face_values.get_quadrature_points(),
    face_advection_directions);
```

Now loop over all quadrature points and see whether it is on the inflow or outflow part of the boundary. This is determined by a test whether the advection direction points inwards or outwards of the domain (note that the normal vector points outwards of the cell, and since the cell is at the boundary, the normal vector points outward of the domain, so if the advection direction points into the domain, its scalar product with the normal vector must be negative):

```
for (unsigned int q_point = 0; q_point < n_face_q_points; ++q_point)
    if (scratch_data.fe_face_values.normal_vector(q_point) *
            face_advection_directions[q_point] <
        0)
```

If the is part of the inflow boundary, then compute the contributions of this face to the global matrix and right hand side, using the values obtained from the FEFaceValues object and the formulae discussed in the introduction:

```
            for (unsigned int i = 0; i < dofs_per_cell; ++i) {
                for (unsigned int j = 0; j < dofs_per_cell; ++j)
                    copy_data.cell_matrix(i, j) -=
                        (face_advection_directions[q_point] *
                         scratch_data.fe_face_values.normal_vector(
                             q_point) *
                         scratch_data.fe_face_values.shape_value(
                             i, q_point) *
                         scratch_data.fe_face_values.shape_value(
                             j, q_point) *
                         scratch_data.fe_face_values.JxW(q_point));
                copy_data.cell_rhs(i) -=
                    (face_advection_directions[q_point] *
                     scratch_data.fe_face_values.normal_vector(
                         q_point) *
                     face_boundary_values[q_point] *
                     scratch_data.fe_face_values.shape_value(i,
                                                             q_point) *
                     scratch_data.fe_face_values.JxW(q_point));
            }
}
```

Now go on by transferring the local contributions to the system of equations into the global objects. The first step was to obtain the global indices of the degrees of freedom on this cell.

```
    cell->get_dof_indices(copy_data.local_dof_indices);
}
```

The second function we needed to write was the one that copies the local contributions the previous function has computed and put into the copy data object, into the global matrix and right hand side vector objects. This is essentially what we always had as the last block of code when assembling something on every cell. The following should therefore be pretty obvious:

```
template <int dim>
void AdvectionProblem<dim>::copy_local_to_global(
    const AssemblyCopyData& copy_data)
{
    for (unsigned int i = 0; i < copy_data.local_dof_indices.size(); ++i) {
        for (unsigned int j = 0; j < copy_data.local_dof_indices.size(); ++j)
            system_matrix.add(copy_data.local_dof_indices[i],
                              copy_data.local_dof_indices[j],
                              copy_data.cell_matrix(i, j));
        system_rhs(copy_data.local_dof_indices[i]) += copy_data.cell_rhs(i);
    }
}
```

Following is the function that solves the linear system of equations. As the system is no more symmetric positive definite as in all the previous examples, we can't use the Conjugate Gradients method anymore. Rather, we use a solver that is tailored to nonsymmetric systems like the one at hand, the BiCGStab method. As preconditioner, we use the Block Jacobi method.

```
template <int dim>
void AdvectionProblem<dim>::solve()
{
```

Assert that the system be symmetric.

```
Assert(system_matrix.m() == system_matrix.n(), ExcNotQuadratic());
auto num_rows = system_matrix.m();
```

Make a copy of the rhs to use with Ginkgo.

```
std::vector<double> rhs(num_rows);
std::copy(system_rhs.begin(), system_rhs.begin() + num_rows, rhs.begin());
```

Ginkgo setup Some shortcuts: A vector is a Dense matrix with co-dimension 1. The matrix is setup in CSR. But various formats can be used. Look at Ginkgo's documentation.

```
using vec = gko::matrix::Dense<>;
using mtx = gko::matrix::Csr<>;
using bicgstab = gko::solver::Bicgstab<>;
using bj = gko::preconditioner::Jacobi<>;
using val_array = gko::array<double>;
```

Where the code is to be executed. Can be changed to `omp` or `cuda` to run on multiple threads or on gpu's

```
std::shared_ptr<gko::Executor> exec = gko::ReferenceExecutor::create();
```

Setup Ginkgo's data structures

```
auto b = vec::create(exec, gko::dim<2>(num_rows, 1),
                     val_array::view(exec, num_rows, rhs.data()), 1);
auto x = vec::create(exec, gko::dim<2>(num_rows, 1));
auto A = mtx::create(exec, gko::dim<2>(num_rows),
```

```
                    system_matrix.n_nonzero_elements());
mtx::value_type* values = A->get_values();
mtx::index_type* row_ptr = A->get_row_ptrs();
mtx::index_type* col_idx = A->get_col_idxs();
```

Convert to standard CSR format As deal.ii does not expose its system matrix pointers, we construct them individually.

```
row_ptr[0] = 0;
for (auto row = 1; row <= num_rows; ++row) {
    row_ptr[row] = row_ptr[row - 1] + system_matrix.get_row_length(row - 1);
}
std::vector<mtx::index_type> ptrs(num_rows + 1);
std::copy(A->get_row_ptrs(), A->get_row_ptrs() + num_rows + 1,
          ptrs.begin());
for (auto row = 0; row < system_matrix.m(); ++row) {
    for (auto p = system_matrix.begin(row); p != system_matrix.end(row);
         ++p) {
```

write entry into the first free one for this row

```
col_idx[ptrs[row]] = p->column();
values[ptrs[row]] = p->value();
```

then move pointer ahead

```
        ++ptrs[row];
    }
}
```

Ginkgo solve The stopping criteria is set at maximum iterations of 1000 and a reduction factor of 1e-12. For other options, refer to Ginkgo's documentation.

```
auto solver_gen =
    bicgstab::build()
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(1000),
            gko::stop::ResidualNorm<>::build().with_reduction_factor(1e-12))
        .with_preconditioner(bj::build())
        .on(exec);
auto solver = solver_gen->generate(gko::give(A));
```

Solve system

```
solver->apply(b, x);
```

Copy the solution vector back to deal.ii's data structures.

```
std::copy(x->get_values(), x->get_values() + num_rows, solution.begin());
/ ************************************************
 * deal.ii internal solver.  Here for reference.
 SolverControl           solver_control (1000, 1e-12);
 SolverBicgstab<>        bicgstab (solver_control);
 PreconditionJacobi<> preconditioner;
 preconditioner.initialize(system_matrix, 1.0);
 bicgstab.solve (system_matrix, solution, system_rhs,
                 preconditioner);
************************************************** /
```

Distribute the possible hanging nodes constraints

```
    hanging_node_constraints.distribute(solution);
}
```

The following function refines the grid according to the quantity described in the introduction. The respective computations are made in the class `GradientEstimation`. The only difference to previous examples is that we refine a little more aggressively (0.5 instead of 0.3 of the number of cells).

```
template <int dim>
void AdvectionProblem<dim>::refine_grid()
{
    Vector<float> estimated_error_per_cell(triangulation.n_active_cells());
    GradientEstimation::estimate(dof_handler, solution,
                                 estimated_error_per_cell);
    GridRefinement::refine_and_coarsen_fixed_number(
        triangulation, estimated_error_per_cell, 0.5, 0.03);
    triangulation.execute_coarsening_and_refinement();
}
```

Writing output to disk is done in the same way as in the previous examples. Indeed, the function is identical to the one in step-6.

```
template <int dim>
void AdvectionProblem<dim>::output_results(const unsigned int cycle)const
{
```

```
    {
        GridOut grid_out;
        std::ofstream output("grid-" + std::to_string(cycle) + ".eps");
        grid_out.write_eps(triangulation, output);
    }
    {
        DataOut<dim> data_out;
        data_out.attach_dof_handler(dof_handler);
        data_out.add_data_vector(solution, "solution");
        data_out.build_patches();
        std::ofstream output("solution-" + std::to_string(cycle) + ".vtk");
        data_out.write_vtk(output);
    }
}
```

... as is the main loop (setup – solve – refine)

```
template <int dim>
void AdvectionProblem<dim>::run()
{
    for (unsigned int cycle = 0; cycle < 6; ++cycle) {
        std::cout « "Cycle " « cycle « ':'  « std::endl;
        if (cycle == 0) {
            GridGenerator::hyper_cube(triangulation, -1, 1);
            triangulation.refine_global(4);
        } else {
            refine_grid();
        }
        std::cout « "   Number of active cells:              "
                  « triangulation.n_active_cells() « std::endl;
        setup_system();
        std::cout « "   Number of degrees of freedom:  " « dof_handler.n_dofs()
                  « std::endl;
        assemble_system();
        solve();
        output_results(cycle);
    }
}
```

### GradientEstimation class implementation

Now for the implementation of the `GradientEstimation` class. Let us start by defining constructors for the `EstimateScratchData` class used by the `estimate_cell()` function:

```
template <int dim>
GradientEstimation::EstimateScratchData<dim>::EstimateScratchData(
    const FiniteElement<dim>& fe, const Vector<double>& solution,
    Vector<float>& error_per_cell)
    :  fe_midpoint_value(fe, QMidpoint<dim>(),
                         update_values | update_quadrature_points),
      solution(solution),
      error_per_cell(error_per_cell)
{}
template <int dim>
GradientEstimation::EstimateScratchData<dim>::EstimateScratchData(
    const EstimateScratchData& scratch_data)
    :  fe_midpoint_value(scratch_data.fe_midpoint_value.get_fe(),
                         scratch_data.fe_midpoint_value.get_quadrature(),
                         update_values | update_quadrature_points),
      solution(scratch_data.solution),
      error_per_cell(scratch_data.error_per_cell)
{}
```

Next for the implementation of the `GradientEstimation` class. The first function does not much except for delegating work to the other function, but there is a bit of setup at the top.

Before starting with the work, we check that the vector into which the results are written has the right size. Programming mistakes in which one forgets to size arguments correctly at the calling site are quite common. Because the resulting damage from not catching such errors is often subtle (e.g., corruption of data somewhere in memory, or non-reproducible results), it is well worth the effort to check for such things.

```
template <int dim>
void GradientEstimation::estimate(const DoFHandler<dim>& dof_handler,
                                  const Vector<double>& solution,
                                  Vector<float>& error_per_cell)
{
    Assert(error_per_cell.size() ==
               dof_handler.get_triangulation().n_active_cells(),
           ExcInvalidVectorLength(
               error_per_cell.size(),
```

```
              dof_handler.get_triangulation().n_active_cells()));
    WorkStream::run(dof_handler.begin_active(), dof_handler.end(),
                    &GradientEstimation::template estimate_cell<dim>,
                    std::function<void(const EstimateCopyData&)>(),
                    EstimateScratchData<dim>(dof_handler.get_fe(), solution,
                                             error_per_cell),
                    EstimateCopyData());
}
```

Following now the function that actually computes the finite difference approximation to the gradient. The general outline of the function is to first compute the list of active neighbors of the present cell and then compute the quantities described in the introduction for each of the neighbors. The reason for this order is that it is not a one-liner to find a given neighbor with locally refined meshes. In principle, an optimized implementation would find neighbors and the quantities depending on them in one step, rather than first building a list of neighbors and in a second step their contributions but we will gladly leave this as an exercise. As discussed before, the worker function passed to WorkStream::run works on "scratch" objects that keep all temporary objects. This way, we do not need to create and initialize objects that are expensive to initialize within the function that does the work, every time it is called for a given cell. Such an argument is passed as the second argument. The third argument would be a "copy-data" object (see threads for more information) but we do not actually use any of these here. Because WorkStream::run() insists on passing three arguments, we declare this function with three arguments, but simply ignore the last one.

(This is unsatisfactory from an esthetic perspective. It can be avoided, at the cost of some other trickery. If you allow, let us here show how. First, assume that we had declared this function to only take two arguments by omitting the unused last one. Now, WorkStream::run still wants to call this function with three arguments, so we need to find a way to "forget" the third argument in the call. Simply passing WorkStream::run the pointer to the function as we do above will not do this – the compiler will complain that a function declared to have two arguments is called with three arguments. However, we can do this by passing the following as the third argument when calling WorkStream::run() above:

```
std::function<void (const typename DoFHandler<dim>::active_cell_iterator
&,
                    EstimateScratchData<dim>                  &,
                    EstimateCopyData                          &)>
  (std::bind (&GradientEstimation::template estimate_cell<dim>,
              std::placeholders::_1,
              std::placeholders::_2))
```

This creates a function object taking three arguments, but when it calls the underlying function object, it simply only uses the first and second argument – we simply "forget" to use the third argument :-) In the end, this isn't completely obvious either, and so we didn't implement it, but hey – it can be done!)

Now for the details:

```
template <int dim>
void GradientEstimation::estimate_cell(
    const typename DoFHandler<dim>::active_cell_iterator& cell,
    EstimateScratchData<dim>& scratch_data, const EstimateCopyData&)
{
```

We need space for the tensor `Y`, which is the sum of outer products of the y-vectors.

```
Tensor<2, dim> Y;
```

Then we allocate a vector to hold iterators to all active neighbors of a cell. We reserve the maximal number of active neighbors in order to avoid later reallocations. Note how this maximal number of active neighbors is computed here.

```
std::vector<typename DoFHandler<dim>::active_cell_iterator>
    active_neighbors;
active_neighbors.reserve(GeometryInfo<dim>::faces_per_cell *
                         GeometryInfo<dim>::max_children_per_face);
```

First initialize the `FEValues` object, as well as the `Y` tensor:

```
scratch_data.fe_midpoint_value.reinit(cell);
```

Then allocate the vector that will be the sum over the y-vectors times the approximate directional derivative:

```
Tensor<1, dim> projected_gradient;
```

Now before going on first compute a list of all active neighbors of the present cell. We do so by first looping over all faces and see whether the neighbor there is active, which would be the case if it is on the same level as the present cell or one level coarser (note that a neighbor can only be once coarser than the present cell, as we only allow a

maximal difference of one refinement over a face in deal.II). Alternatively, the neighbor could be on the same level and be further refined; then we have to find which of its children are next to the present cell and select these (note that if a child of a neighbor of an active cell that is next to this active cell, needs necessarily be active itself, due to the one-refinement rule cited above).

Things are slightly different in one space dimension, as there the one-refinement rule does not exist: neighboring active cells may differ in as many refinement levels as they like. In this case, the computation becomes a little more difficult, but we will explain this below.

Before starting the loop over all neighbors of the present cell, we have to clear the array storing the iterators to the active neighbors, of course.

```
active_neighbors.clear();
for (unsigned int face_no = 0; face_no < GeometryInfo<dim>::faces_per_cell;
     ++face_no)
    if (!cell->at_boundary(face_no)) {
```

First define an abbreviation for the iterator to the face and the neighbor

```
const typename DoFHandler<dim>::face_iterator face =
    cell->face(face_no);
const typename DoFHandler<dim>::cell_iterator neighbor =
    cell->neighbor(face_no);
```

Then check whether the neighbor is active. If it is, then it is on the same level or one level coarser (if we are not in 1D), and we are interested in it in any case.

```
if (neighbor->active())
    active_neighbors.push_back(neighbor);
else {
```

If the neighbor is not active, then check its children.

```
if (dim == 1) {
```

To find the child of the neighbor which bounds to the present cell, successively go to its right child if we are left of the present cell (n==0), or go to the left child if we are on the right (n==1), until we find an active cell.

```
typename DoFHandler<dim>::cell_iterator neighbor_child =
    neighbor;
while (neighbor_child->has_children())
    neighbor_child =
        neighbor_child->child(face_no == 0 ?  1 :  0);
```

As this used some non-trivial geometrical intuition, we might want to check whether we did it right, i.e. check whether the neighbor of the cell we found is indeed the cell we are presently working on. Checks like this are often useful and have frequently uncovered errors both in algorithms like the line above (where it is simple to involuntarily exchange `n==1` for `n==0` or the like) and in the library (the assumptions underlying the algorithm above could either be wrong, wrongly documented, or are violated due to an error in the library). One could in principle remove such checks after the program works for some time, but it might be a good things to leave it in anyway to check for changes in the library or in the algorithm above.

Note that if this check fails, then this is certainly an error that is irrecoverable and probably qualifies as an internal error. We therefore use a predefined exception class to throw here.

```
Assert(
    neighbor_child->neighbor(face_no == 0 ?  1 :  0) == cell,
    ExcInternalError());
```

If the check succeeded, we push the active neighbor we just found to the stack we keep:

```
    active_neighbors.push_back(neighbor_child);
} else
```

If we are not in 1d, we collect all neighbor children "behind" the subfaces of the current face

```
            for (unsigned int subface_no = 0;
                 subface_no < face->n_children(); ++subface_no)
                active_neighbors.push_back(
                    cell->neighbor_child_on_subface(face_no,
                                                    subface_no));
    }
}
```

OK, now that we have all the neighbors, lets start the computation on each of them. First we do some preliminaries: find out about the center of the present cell and the solution at this point. The latter is obtained as a vector of

function values at the quadrature points, of which there are only one, of course. Likewise, the position of the center is the position of the first (and only) quadrature point in real space.

```
const Point<dim> this_center =
    scratch_data.fe_midpoint_value.quadrature_point(0);
std::vector<double> this_midpoint_value(1);
scratch_data.fe_midpoint_value.get_function_values(scratch_data.solution,
                                                   this_midpoint_value);
```

Now loop over all active neighbors and collect the data we need. Allocate a vector just like `this_midpoint_value` which we will use to store the value of the solution in the midpoint of the neighbor cell. We allocate it here already, since that way we don't have to allocate memory repeatedly in each iteration of this inner loop (memory allocation is a rather expensive operation):

```
std::vector<double> neighbor_midpoint_value(1);
typename std::vector<typename DoFHandler<dim>::active_cell_iterator>::
    const_iterator neighbor_ptr = active_neighbors.begin();
for (; neighbor_ptr != active_neighbors.end(); ++neighbor_ptr) {
```

First define an abbreviation for the iterator to the active neighbor cell:

```
const typename DoFHandler<dim>::active_cell_iterator neighbor =
    *neighbor_ptr;
```

Then get the center of the neighbor cell and the value of the finite element function thereon. Note that for this information we have to reinitialize the `FEValues` object for the neighbor cell.

```
scratch_data.fe_midpoint_value.reinit(neighbor);
const Point<dim> neighbor_center =
    scratch_data.fe_midpoint_value.quadrature_point(0);
scratch_data.fe_midpoint_value.get_function_values(
    scratch_data.solution, neighbor_midpoint_value);
```

Compute the vector `y` connecting the centers of the two cells. Note that as opposed to the introduction, we denote by `y` the normalized difference vector, as this is the quantity used everywhere in the computations.

```
Tensor<1, dim> y = neighbor_center - this_center;
const double distance = y.norm();
y /= distance;
```

Then add up the contribution of this cell to the Y matrix...

```
for (unsigned int i = 0; i < dim; ++i)
    for (unsigned int j = 0; j < dim; ++j) Y[i][j] += y[i] * y[j];
```

... and update the sum of difference quotients:

```
    projected_gradient +=
        (neighbor_midpoint_value[0] - this_midpoint_value[0]) / distance *
        y;
}
```

If now, after collecting all the information from the neighbors, we can determine an approximation of the gradient for the present cell, then we need to have passed over vectors `y` which span the whole space, otherwise we would not have all components of the gradient. This is indicated by the invertibility of the matrix.

If the matrix should not be invertible, this means that the present cell had an insufficient number of active neighbors. In contrast to all previous cases, where we raised exceptions, this is, however, not a programming error: it is a runtime error that can happen in optimized mode even if it ran well in debug mode, so it is reasonable to try to catch this error also in optimized mode. For this case, there is the `AssertThrow` macro: it checks the condition like the `Assert` macro, but not only in debug mode; it then outputs an error message, but instead of terminating the program as in the case of the `Assert` macro, the exception is thrown using the `throw` command of C++. This way, one has the possibility to catch this error and take reasonable counter actions. One such measure would be to refine the grid globally, as the case of insufficient directions can not occur if every cell of the initial grid has been refined at least once.

```
AssertThrow(determinant(Y) != 0, ExcInsufficientDirections());
```

If, on the other hand the matrix is invertible, then invert it, multiply the other quantity with it and compute the estimated error using this quantity and the right powers of the mesh width:

```
const Tensor<2, dim> Y_inverse = invert(Y);
Tensor<1, dim> gradient = Y_inverse * projected_gradient;
```

The last part of this function is the one where we write into the element of the output vector what we have just computed. The address of this vector has been stored in the scratch data object, and all we have to do is know how to get at the correct element inside this vector – but we can ask the cell we're on the how-manyth active cell it is for this:

```
    scratch_data.error_per_cell(cell->active_cell_index()) =
        (std::pow(cell->diameter(), 1 + 1.0 * dim / 2) *
         std::sqrt(gradient.norm_square()));
}
}  // namespace Step9
```

### Main function

The `main` function is similar to the previous examples. The main difference is that we use MultithreadInfo to set the maximum number of threads (see Parallel computing with multiple processors accessing shared memory" documentation module for more explanation). The number of threads used is the minimum of the environment variable DEAL_II_NUM_THREADS and the parameter of `set_thread_limit`. If no value is given to `set_thread_limit`, the default value from the Intel Threading Building Blocks (TBB) library is used. If the call to `set_thread_limit` is omitted, the number of threads will be chosen by TBB independently of DEAL_II_NUM_THREADS.

```
int main()
{
    try {
        dealii::MultithreadInfo::set_thread_limit();
        Step9::AdvectionProblem<2> advection_problem_2d;
        advection_problem_2d.run();
    } catch (std::exception& exc) {
        std::cerr « std::endl
                  « std::endl
                  « "----------------------------------------------------"
                  « std::endl;
        std::cerr « "Exception on processing:  " « std::endl
                  « exc.what() « std::endl
                  « "Aborting!" « std::endl
                  « "----------------------------------------------------"
                  « std::endl;
        return 1;
    } catch (...)  {
        std::cerr « std::endl
                  « std::endl
                  « "----------------------------------------------------"
                  « std::endl;
        std::cerr « "Unknown exception!" « std::endl
                  « "Aborting!" « std::endl
                  « "----------------------------------------------------"
                  « std::endl;
        return 1;
    }
    return 0;
}
```

## Results

### Comments about programming and debugging

## The plain program

```
/* ---------------------------------------------------------------------
*
* Copyright (C) 2000 - 2018 by the deal.II authors
*
* This file is part of the deal.II library.
*
* The deal.II library is free software; you can use it, redistribute
* it, and/or modify it under the terms of the GNU Lesser General
* Public License as published by the Free Software Foundation; either
* version 2.1 of the License, or (at your option) any later version.
* The full text of the license can be found in the file LICENSE at
* the top level of the deal.II distribution.
*
* ---------------------------------------------------------------------

*
* Author:  Wolfgang Bangerth, University of Heidelberg, 2000
*/
/* ---------------------------------------------------------------------
*
* This file has been taken verbatim from the deal.ii (version 9.0)
* examples directory and modified.
*
* This example aims to demonstrate the ease with which Ginkgo can
* be interfaced with other libraries.  The only modification/ addition
* has been to the AdvectionProblem::solve () function.
*
```

```
*/
#include <deal.II/base/function.h>
#include <deal.II/base/logstream.h>
#include <deal.II/base/quadrature_lib.h>
#include <deal.II/dofs/dof_accessor.h>
#include <deal.II/dofs/dof_handler.h>
#include <deal.II/dofs/dof_tools.h>
#include <deal.II/fe/fe_q.h>
#include <deal.II/fe/fe_values.h>
#include <deal.II/grid/grid_generator.h>
#include <deal.II/grid/grid_out.h>
#include <deal.II/grid/grid_refinement.h>
#include <deal.II/grid/tria.h>
#include <deal.II/grid/tria_accessor.h>
#include <deal.II/grid/tria_iterator.h>
#include <deal.II/lac/constraint_matrix.h>
#include <deal.II/lac/dynamic_sparsity_pattern.h>
#include <deal.II/lac/full_matrix.h>
#include <deal.II/lac/precondition.h>
#include <deal.II/lac/solver_bicgstab.h>
#include <deal.II/lac/sparse_matrix.h>
#include <deal.II/lac/vector.h>
#include <deal.II/numerics/data_out.h>
#include <deal.II/numerics/matrix_tools.h>
#include <deal.II/numerics/vector_tools.h>
#include <deal.II/base/multithread_info.h>
#include <deal.II/base/work_stream.h>
#include <deal.II/base/tensor_function.h>
#include <deal.II/numerics/error_estimator.h>
#include <ginkgo/ginkgo.hpp>
#include <fstream>
#include <iostream>
namespace Step9 {
using namespace dealii;
template <int dim>
class AdvectionProblem {
public:
    AdvectionProblem();
     AdvectionProblem();
    void run();
private:
    void setup_system();
    struct AssemblyScratchData {
        AssemblyScratchData(const FiniteElement<dim>& fe);
        AssemblyScratchData(const AssemblyScratchData& scratch_data);
        FEValues<dim> fe_values;
        FEFaceValues<dim> fe_face_values;
    };
    struct AssemblyCopyData {
        FullMatrix<double> cell_matrix;
        Vector<double> cell_rhs;
        std::vector<types::global_dof_index> local_dof_indices;
    };
    void assemble_system();
    void local_assemble_system(
        const typename DoFHandler<dim>::active_cell_iterator& cell,
        AssemblyScratchData& scratch, AssemblyCopyData& copy_data);
    void copy_local_to_global(const AssemblyCopyData& copy_data);
    void solve();
    void refine_grid();
    void output_results(const unsigned int cycle) const;
    Triangulation<dim> triangulation;
    DoFHandler<dim> dof_handler;
    FE_Q<dim> fe;
    ConstraintMatrix hanging_node_constraints;
    SparsityPattern sparsity_pattern;
    SparseMatrix<double> system_matrix;
    Vector<double> solution;
    Vector<double> system_rhs;
};
template <int dim>
class AdvectionField :  public TensorFunction<1, dim> {
public:
    AdvectionField() :  TensorFunction<1, dim>() {}
    virtual Tensor<1, dim> value(const Point<dim>& p) const;
    virtual void value_list(const std::vector<Point<dim»& points,
                            std::vector<Tensor<1, dim»& values) const;
    DeclException2(ExcDimensionMismatch, unsigned int, unsigned int,
                   « "The vector has size " « arg1 « " but should have "
                   « arg2 « " elements.");
};
template <int dim>
Tensor<1, dim> AdvectionField<dim>::value(const Point<dim>& p)const
{
    Point<dim> value;
    value[0] = 2;
    for (unsigned int i = 1; i < dim; ++i)
```

```
        value[i] = 1 + 0.8 * std::sin(8 * numbers::PI * p[0]);
    return value;
}
template <int dim>
void AdvectionField<dim>::value_list(const std::vector<Point<dim>& points,
                                     std::vector<Tensor<1, dim>& values)const
{
    Assert(values.size() == points.size(),
           ExcDimensionMismatch(values.size(), points.size()));
    for (unsigned int i = 0; i < points.size(); ++i)
        values[i] = AdvectionField<dim>::value(points[i]);
}
template <int dim>
class RightHandSide :  public Function<dim> {
public:
    RightHandSide() :  Function<dim>() {}
    virtual double value(const Point<dim>& p,
                         const unsigned int component = 0) const;
    virtual void value_list(const std::vector<Point<dim>& points,
                            std::vector<double>& values,
                            const unsigned int component = 0) const;
private:
    static const Point<dim> center_point;
};
template <>
const Point<1> RightHandSide<1>::center_point = Point<1>(-0.75);
template <>
const Point<2> RightHandSide<2>::center_point = Point<2>(-0.75, -0.75);
template <>
const Point<3> RightHandSide<3>::center_point = Point<3>(-0.75, -0.75, -0.75);
template <int dim>
double RightHandSide<dim>::value(const Point<dim>& p,
                                 const unsigned int component)const
{
    (void)component;
    Assert(component == 0, ExcIndexRange(component, 0, 1));
    const double diameter = 0.1;
    return ((p - center_point).norm_square() < diameter * diameter
            ?  .1 / std::pow(diameter, dim)
            :  0);
}
template <int dim>
void RightHandSide<dim>::value_list(const std::vector<Point<dim>& points,
                                    std::vector<double>& values,
                                    const unsigned int component)const
{
    Assert(values.size() == points.size(),
           ExcDimensionMismatch(values.size(), points.size()));
    for (unsigned int i = 0; i < points.size(); ++i)
        values[i] = RightHandSide<dim>::value(points[i], component);
}
template <int dim>
class BoundaryValues :  public Function<dim> {
public:
    BoundaryValues() :  Function<dim>() {}
    virtual double value(const Point<dim>& p,
                         const unsigned int component = 0) const;
    virtual void value_list(const std::vector<Point<dim>& points,
                            std::vector<double>& values,
                            const unsigned int component = 0) const;
};
template <int dim>
double BoundaryValues<dim>::value(const Point<dim>& p,
                                  const unsigned int component)const
{
    (void)component;
    Assert(component == 0, ExcIndexRange(component, 0, 1));
    const double sine_term =
        std::sin(16 * numbers::PI * std::sqrt(p.norm_square()));
    const double weight = std::exp(-5 * p.norm_square()) / std::exp(-5.);
    return sine_term * weight;
}
template <int dim>
void BoundaryValues<dim>::value_list(const std::vector<Point<dim>& points,
                                     std::vector<double>& values,
                                     const unsigned int component)const
{
    Assert(values.size() == points.size(),
           ExcDimensionMismatch(values.size(), points.size()));
    for (unsigned int i = 0; i < points.size(); ++i)
        values[i] = BoundaryValues<dim>::value(points[i], component);
}
class GradientEstimation {
public:
    template <int dim>
    static void estimate(const DoFHandler<dim>& dof,
                         const Vector<double>& solution,
```

```
                        Vector<float>& error_per_cell);
    DeclException2(ExcInvalidVectorLength, int, int,
                   « "Vector has length " « arg1 « ", but should have "
                   « arg2);
    DeclException0(ExcInsufficientDirections);
private:
    template <int dim>
    struct EstimateScratchData {
        EstimateScratchData(const FiniteElement<dim>& fe,
                            const Vector<double>& solution,
                            Vector<float>& error_per_cell);
        EstimateScratchData(const EstimateScratchData& data);
        FEValues<dim> fe_midpoint_value;
        const Vector<double>& solution;
        Vector<float>& error_per_cell;
    };
    struct EstimateCopyData {};
    template <int dim>
    static void estimate_cell(
        const typename DoFHandler<dim>::active_cell_iterator& cell,
        EstimateScratchData<dim>& scratch_data,
        const EstimateCopyData& copy_data);
};
template <int dim>
AdvectionProblem<dim>::AdvectionProblem() :  dof_handler(triangulation), fe(1)
{}
template <int dim>
AdvectionProblem<dim>:: AdvectionProblem()
{
    dof_handler.clear();
}
template <int dim>
void AdvectionProblem<dim>::setup_system()
{
    dof_handler.distribute_dofs(fe);
    hanging_node_constraints.clear();
    DoFTools::make_hanging_node_constraints(dof_handler,
                                            hanging_node_constraints);
    hanging_node_constraints.close();
    DynamicSparsityPattern dsp(dof_handler.n_dofs(), dof_handler.n_dofs());
    DoFTools::make_sparsity_pattern(dof_handler, dsp, hanging_node_constraints,
                                    /*keep_constrained_dofs = */ true);
    sparsity_pattern.copy_from(dsp);
    system_matrix.reinit(sparsity_pattern);
    solution.reinit(dof_handler.n_dofs());
    system_rhs.reinit(dof_handler.n_dofs());
}
template <int dim>
void AdvectionProblem<dim>::assemble_system()
{
    WorkStream::run(dof_handler.begin_active(), dof_handler.end(), *this,
                    &AdvectionProblem::local_assemble_system,
                    &AdvectionProblem::copy_local_to_global,
                    AssemblyScratchData(fe), AssemblyCopyData());
    hanging_node_constraints.condense(system_matrix);
    hanging_node_constraints.condense(system_rhs);
}
template <int dim>
AdvectionProblem<dim>::AssemblyScratchData::AssemblyScratchData(
    const FiniteElement<dim>& fe)
    :  fe_values(fe, QGauss<dim>(2),
                 update_values | update_gradients | update_quadrature_points |
                     update_JxW_values),
      fe_face_values(fe, QGauss<dim - 1>(2),
                     update_values | update_quadrature_points |
                         update_JxW_values | update_normal_vectors)
{}
template <int dim>
AdvectionProblem<dim>::AssemblyScratchData::AssemblyScratchData(
    const AssemblyScratchData& scratch_data)
    :  fe_values(scratch_data.fe_values.get_fe(),
                 scratch_data.fe_values.get_quadrature(),
                 update_values | update_gradients | update_quadrature_points |
                     update_JxW_values),
      fe_face_values(scratch_data.fe_face_values.get_fe(),
                     scratch_data.fe_face_values.get_quadrature(),
                     update_values | update_quadrature_points |
                         update_JxW_values | update_normal_vectors)
{}
template <int dim>
void AdvectionProblem<dim>::local_assemble_system(
    const typename DoFHandler<dim>::active_cell_iterator& cell,
    AssemblyScratchData& scratch_data, AssemblyCopyData& copy_data)
{
    const AdvectionField<dim> advection_field;
    const RightHandSide<dim> right_hand_side;
    const BoundaryValues<dim> boundary_values;
```

```
    const unsigned int dofs_per_cell = fe.dofs_per_cell;
    const unsigned int n_q_points =
        scratch_data.fe_values.get_quadrature().size();
    const unsigned int n_face_q_points =
        scratch_data.fe_face_values.get_quadrature().size();
    copy_data.cell_matrix.reinit(dofs_per_cell, dofs_per_cell);
    copy_data.cell_rhs.reinit(dofs_per_cell);
    copy_data.local_dof_indices.resize(dofs_per_cell);
    std::vector<double> rhs_values(n_q_points);
    std::vector<Tensor<1, dim» advection_directions(n_q_points);
    std::vector<double> face_boundary_values(n_face_q_points);
    std::vector<Tensor<1, dim» face_advection_directions(n_face_q_points);
    scratch_data.fe_values.reinit(cell);
    advection_field.value_list(scratch_data.fe_values.get_quadrature_points(),
                               advection_directions);
    right_hand_side.value_list(scratch_data.fe_values.get_quadrature_points(),
                               rhs_values);
    const double delta = 0.1 * cell->diameter();
    for (unsigned int q_point = 0; q_point < n_q_points; ++q_point)
        for (unsigned int i = 0; i < dofs_per_cell; ++i) {
            for (unsigned int j = 0; j < dofs_per_cell; ++j)
                copy_data.cell_matrix(i, j) +=
                    ((advection_directions[q_point] *
                      scratch_data.fe_values.shape_grad(j, q_point) *
                      (scratch_data.fe_values.shape_value(i, q_point) +
                       delta *
                           (advection_directions[q_point] *
                            scratch_data.fe_values.shape_grad(i, q_point)))) *
                     scratch_data.fe_values.JxW(q_point));
            copy_data.cell_rhs(i) +=
                ((scratch_data.fe_values.shape_value(i, q_point) +
                  delta * (advection_directions[q_point] *
                           scratch_data.fe_values.shape_grad(i, q_point))) *
                 rhs_values[q_point] * scratch_data.fe_values.JxW(q_point));
        }
    for (unsigned int face = 0; face < GeometryInfo<dim>::faces_per_cell;
         ++face)
        if (cell->face(face)->at_boundary()) {
            scratch_data.fe_face_values.reinit(cell, face);
            boundary_values.value_list(
                scratch_data.fe_face_values.get_quadrature_points(),
                face_boundary_values);
            advection_field.value_list(
                scratch_data.fe_face_values.get_quadrature_points(),
                face_advection_directions);
            for (unsigned int q_point = 0; q_point < n_face_q_points; ++q_point)
                if (scratch_data.fe_face_values.normal_vector(q_point) *
                        face_advection_directions[q_point] <
                    0)
                    for (unsigned int i = 0; i < dofs_per_cell; ++i) {
                        for (unsigned int j = 0; j < dofs_per_cell; ++j)
                            copy_data.cell_matrix(i, j) -=
                                (face_advection_directions[q_point] *
                                 scratch_data.fe_face_values.normal_vector(
                                     q_point) *
                                 scratch_data.fe_face_values.shape_value(
                                     i, q_point) *
                                 scratch_data.fe_face_values.shape_value(
                                     j, q_point) *
                                 scratch_data.fe_face_values.JxW(q_point));
                        copy_data.cell_rhs(i) -=
                            (face_advection_directions[q_point] *
                             scratch_data.fe_face_values.normal_vector(
                                 q_point) *
                             face_boundary_values[q_point] *
                             scratch_data.fe_face_values.shape_value(i,
                                                                     q_point) *
                             scratch_data.fe_face_values.JxW(q_point));
                    }
        }
    cell->get_dof_indices(copy_data.local_dof_indices);
}
template <int dim>
void AdvectionProblem<dim>::copy_local_to_global(
    const AssemblyCopyData& copy_data)
{
    for (unsigned int i = 0; i < copy_data.local_dof_indices.size(); ++i) {
        for (unsigned int j = 0; j < copy_data.local_dof_indices.size(); ++j)
            system_matrix.add(copy_data.local_dof_indices[i],
                              copy_data.local_dof_indices[j],
                              copy_data.cell_matrix(i, j));
        system_rhs(copy_data.local_dof_indices[i]) += copy_data.cell_rhs(i);
    }
}
template <int dim>
void AdvectionProblem<dim>::solve()
{
```

```
    Assert(system_matrix.m() == system_matrix.n(), ExcNotQuadratic());
    auto num_rows = system_matrix.m();
    std::vector<double> rhs(num_rows);
    std::copy(system_rhs.begin(), system_rhs.begin() + num_rows, rhs.begin());
    using vec = gko::matrix::Dense<>;
    using mtx = gko::matrix::Csr<>;
    using bicgstab = gko::solver::Bicgstab<>;
    using bj = gko::preconditioner::Jacobi<>;
    using val_array = gko::array<double>;
    std::shared_ptr<gko::Executor> exec = gko::ReferenceExecutor::create();
    auto b = vec::create(exec, gko::dim<2>(num_rows, 1),
                         val_array::view(exec, num_rows, rhs.data()), 1);
    auto x = vec::create(exec, gko::dim<2>(num_rows, 1));
    auto A = mtx::create(exec, gko::dim<2>(num_rows),
                         system_matrix.n_nonzero_elements());
    mtx::value_type* values = A->get_values();
    mtx::index_type* row_ptr = A->get_row_ptrs();
    mtx::index_type* col_idx = A->get_col_idxs();
    row_ptr[0] = 0;
    for (auto row = 1; row <= num_rows; ++row) {
        row_ptr[row] = row_ptr[row - 1] + system_matrix.get_row_length(row - 1);
    }
    std::vector<mtx::index_type> ptrs(num_rows + 1);
    std::copy(A->get_row_ptrs(), A->get_row_ptrs() + num_rows + 1,
              ptrs.begin());
    for (auto row = 0; row < system_matrix.m(); ++row) {
        for (auto p = system_matrix.begin(row); p != system_matrix.end(row);
             ++p) {
            col_idx[ptrs[row]] = p->column();
            values[ptrs[row]] = p->value();
            ++ptrs[row];
        }
    }
    auto solver_gen =
        bicgstab::build()
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(1000),
                gko::stop::ResidualNorm<>::build().with_reduction_factor(1e-12))
            .with_preconditioner(bj::build())
            .on(exec);
    auto solver = solver_gen->generate(gko::give(A));
    solver->apply(b, x);
    std::copy(x->get_values(), x->get_values() + num_rows, solution.begin());
/*************************************************
* deal.ii internal solver.  Here for reference.
SolverControl           solver_control (1000, 1e-12);
SolverBicgstab<>        bicgstab (solver_control);

PreconditionJacobi<> preconditioner;
preconditioner.initialize(system_matrix, 1.0);

bicgstab.solve (system_matrix, solution, system_rhs,
preconditioner);
*************************************************/
    hanging_node_constraints.distribute(solution);
}
template <int dim>
void AdvectionProblem<dim>::refine_grid()
{
    Vector<float> estimated_error_per_cell(triangulation.n_active_cells());
    GradientEstimation::estimate(dof_handler, solution,
                                 estimated_error_per_cell);
    GridRefinement::refine_and_coarsen_fixed_number(
        triangulation, estimated_error_per_cell, 0.5, 0.03);
    triangulation.execute_coarsening_and_refinement();
}
template <int dim>
void AdvectionProblem<dim>::output_results(const unsigned int cycle)const
{
    {
        GridOut grid_out;
        std::ofstream output("grid-" + std::to_string(cycle) + ".eps");
        grid_out.write_eps(triangulation, output);
    }
    {
        DataOut<dim> data_out;
        data_out.attach_dof_handler(dof_handler);
        data_out.add_data_vector(solution, "solution");
        data_out.build_patches();
        std::ofstream output("solution-" + std::to_string(cycle) + ".vtk");
        data_out.write_vtk(output);
    }
}
template <int dim>
void AdvectionProblem<dim>::run()
{
    for (unsigned int cycle = 0; cycle < 6; ++cycle) {
```

```
        std::cout « "Cycle " « cycle « ':'  « std::endl;
        if (cycle == 0) {
            GridGenerator::hyper_cube(triangulation, -1, 1);
            triangulation.refine_global(4);
        } else {
            refine_grid();
        }
        std::cout « "   Number of active cells:              "
                  « triangulation.n_active_cells() « std::endl;
        setup_system();
        std::cout « "   Number of degrees of freedom:  " « dof_handler.n_dofs()
                  « std::endl;
        assemble_system();
        solve();
        output_results(cycle);
    }
}
template <int dim>
GradientEstimation::EstimateScratchData<dim>::EstimateScratchData(
    const FiniteElement<dim>& fe, const Vector<double>& solution,
    Vector<float>& error_per_cell)
    :  fe_midpoint_value(fe, QMidpoint<dim>(),
                         update_values | update_quadrature_points),
      solution(solution),
      error_per_cell(error_per_cell)
{}
template <int dim>
GradientEstimation::EstimateScratchData<dim>::EstimateScratchData(
    const EstimateScratchData& scratch_data)
    :  fe_midpoint_value(scratch_data.fe_midpoint_value.get_fe(),
                         scratch_data.fe_midpoint_value.get_quadrature(),
                         update_values | update_quadrature_points),
      solution(scratch_data.solution),
      error_per_cell(scratch_data.error_per_cell)
{}
template <int dim>
void GradientEstimation::estimate(const DoFHandler<dim>& dof_handler,
                                  const Vector<double>& solution,
                                  Vector<float>& error_per_cell)
{
    Assert(error_per_cell.size() ==
               dof_handler.get_triangulation().n_active_cells(),
           ExcInvalidVectorLength(
               error_per_cell.size(),
               dof_handler.get_triangulation().n_active_cells()));
    WorkStream::run(dof_handler.begin_active(), dof_handler.end(),
                    &GradientEstimation::template estimate_cell<dim>,
                    std::function<void(const EstimateCopyData&)>(),
                    EstimateScratchData<dim>(dof_handler.get_fe(), solution,
                                             error_per_cell),
                    EstimateCopyData());
}
template <int dim>
void GradientEstimation::estimate_cell(
    const typename DoFHandler<dim>::active_cell_iterator& cell,
    EstimateScratchData<dim>& scratch_data, const EstimateCopyData&)
{
    Tensor<2, dim> Y;
    std::vector<typename DoFHandler<dim>::active_cell_iterator>
        active_neighbors;
    active_neighbors.reserve(GeometryInfo<dim>::faces_per_cell *
                             GeometryInfo<dim>::max_children_per_face);
    scratch_data.fe_midpoint_value.reinit(cell);
    Tensor<1, dim> projected_gradient;
    active_neighbors.clear();
    for (unsigned int face_no = 0; face_no < GeometryInfo<dim>::faces_per_cell;
         ++face_no)
        if (!cell->at_boundary(face_no)) {
            const typename DoFHandler<dim>::face_iterator face =
                cell->face(face_no);
            const typename DoFHandler<dim>::cell_iterator neighbor =
                cell->neighbor(face_no);
            if (neighbor->active())
                active_neighbors.push_back(neighbor);
            else {
                if (dim == 1) {
                    typename DoFHandler<dim>::cell_iterator neighbor_child =
                        neighbor;
                    while (neighbor_child->has_children())
                        neighbor_child =
                            neighbor_child->child(face_no == 0 ?  1 :  0);
                    Assert(
                        neighbor_child->neighbor(face_no == 0 ?  1 :  0) == cell,
                        ExcInternalError());
                    active_neighbors.push_back(neighbor_child);
                } else
                    for (unsigned int subface_no = 0;
```

```
                        subface_no < face->n_children(); ++subface_no)
                      active_neighbors.push_back(
                          cell->neighbor_child_on_subface(face_no,
                                                          subface_no));
            }
        }
    const Point<dim> this_center =
        scratch_data.fe_midpoint_value.quadrature_point(0);
    std::vector<double> this_midpoint_value(1);
    scratch_data.fe_midpoint_value.get_function_values(scratch_data.solution,
                                                       this_midpoint_value);
    std::vector<double> neighbor_midpoint_value(1);
    typename std::vector<typename DoFHandler<dim>::active_cell_iterator>::
        const_iterator neighbor_ptr = active_neighbors.begin();
    for (; neighbor_ptr != active_neighbors.end(); ++neighbor_ptr) {
        const typename DoFHandler<dim>::active_cell_iterator neighbor =
            *neighbor_ptr;
        scratch_data.fe_midpoint_value.reinit(neighbor);
        const Point<dim> neighbor_center =
            scratch_data.fe_midpoint_value.quadrature_point(0);
        scratch_data.fe_midpoint_value.get_function_values(
            scratch_data.solution, neighbor_midpoint_value);
        Tensor<1, dim> y = neighbor_center - this_center;
        const double distance = y.norm();
        y /= distance;
        for (unsigned int i = 0; i < dim; ++i)
            for (unsigned int j = 0; j < dim; ++j) Y[i][j] += y[i] * y[j];
        projected_gradient +=
            (neighbor_midpoint_value[0] - this_midpoint_value[0]) / distance *
            y;
    }
    AssertThrow(determinant(Y) != 0, ExcInsufficientDirections());
    const Tensor<2, dim> Y_inverse = invert(Y);
    Tensor<1, dim> gradient = Y_inverse * projected_gradient;
    scratch_data.error_per_cell(cell->active_cell_index()) =
        (std::pow(cell->diameter(), 1 + 1.0 * dim / 2) *
         std::sqrt(gradient.norm_square()));
}
}  // namespace Step9
int main()
{
    try {
        dealii::MultithreadInfo::set_thread_limit();
        Step9::AdvectionProblem<2> advection_problem_2d;
        advection_problem_2d.run();
    } catch (std::exception& exc) {
        std::cerr << std::endl
                  << std::endl
                  << "----------------------------------------------------"
                  << std::endl;
        std::cerr << "Exception on processing:  " << std::endl
                  << exc.what() << std::endl
                  << "Aborting!" << std::endl
                  << "----------------------------------------------------"
                  << std::endl;
        return 1;
    } catch (...)  {
        std::cerr << std::endl
                  << std::endl
                  << "----------------------------------------------------"
                  << std::endl;
        std::cerr << "Unknown exception!" << std::endl
                  << "Aborting!" << std::endl
                  << "----------------------------------------------------"
                  << std::endl;
        return 1;
    }
    return 0;
}
```

# Chapter 18

# The file-config-solver program

The file config solver example..

This example depends on simple-solver.

## This example shows how to use file to configure solver.

**In this example, we first read in a matrix from a file. Then we read the config file to configure the solver. The example application reports the generation and run time of the solver.**

## The commented program

```
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
```

the header in extensions is not shipped with ginkgo.hpp

```
#include <ginkgo/extensions/config/json_config.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Print usage

```
std::cout « argv[0] « " executor configfile matrix" « std::endl;
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const auto configfile = argc >= 3 ?  argv[2] :  "config/cg.json";
const std::string matrix_path = argc >= 4 ?  argv[3] :  "data/A.mtx";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
```

```
                                            gko::OmpExecutor::create());
        }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(
                 0, gko::ReferenceExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream(matrix_path), exec));
```

Create RHS as 1 and initial guess as 0

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 0.;
    host_b->at(i, 0) = 1.;
}
auto x = vec::create(exec);
auto b = vec::create(exec);
x->copy_from(host_x);
b->copy_from(host_b);
```

Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

Copy b again

```
b->copy_from(host_b);
```

Read the json config file to configure the ginkgo solver. The following files, which are mapped to corresponding examples, are available cg.json: simple-solver blockjacobi-cg.json: preconditioned-solver ir.json: iterative-refinement parilu.json: ilu-preconditioned-solver (by using factorization parameter directly) Ilu only supports the value_type variant for parse. pgm-multigrid-cg.json: multigrid-preconditioned-solver (set min_coarse_rows additionally due to this small example matrix) mixed-pgm-multigrid-cg.json: mixed-multigrid-preconditioned-solver (assuming there are always more than one level)

```
auto config = gko::ext::config::parse_json_file(configfile);
```

Create the registry, which allows passing the existing data into config This example does not use existing data.

```
auto reg = gko::config::registry();
```

Create the default type descriptor, which gives the default common type (value/index) for solver generation. If the solver does not specify value type, the solver will use these types.

```
auto td = gko::config::make_type_descriptor<ValueType, IndexType>();
```

generate the linopfactory on the given executors

```
auto solver_gen = gko::config::parse(config, reg, td).on(exec);
```

Create solver

```
const auto gen_tic = std::chrono::steady_clock::now();
auto solver = solver_gen->generate(A);
exec->synchronize();
const auto gen_toc = std::chrono::steady_clock::now();
const auto gen_time = std::chrono::duration_cast<std::chrono::milliseconds>(
    gen_toc - gen_tic);
```

Add logger

```
std::shared_ptr<const gko::log::Convergence<ValueType» logger =
    gko::log::Convergence<ValueType>::create();
solver->add_logger(logger);
```

Solve system

```
exec->synchronize();
const auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
exec->synchronize();
const auto toc = std::chrono::steady_clock::now();
const auto time =
    std::chrono::duration_cast<std::chrono::milliseconds>(toc - tic);
```

Print out the solver config

```
std::cout « "Config file:  " « configfile « std::endl;
std::ifstream f(configfile);
std::cout « f.rdbuf() « std::endl;
```

Calculate residual

```
auto res = gko::as<vec>(logger->get_residual_norm());
std::cout « "Initial residual norm sqrt(r^T r):  \n";
write(std::cout, initres);
std::cout « "Final residual norm sqrt(r^T r):  \n";
write(std::cout, res);
```

Print solver statistics

```
    std::cout « "Solver iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "Solver generation time [ms]:  "
              « static_cast<double>(gen_time.count()) « std::endl;
    std::cout « "Solver execution time [ms]:  "
              « static_cast<double>(time.count()) « std::endl;
    std::cout « "Solver execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Results

This is the expected output:

```
Config file:  config/cg.json
{
    "type":  "solver::Cg",
    "criteria":  [
        {
            "type":  "Iteration",
            "max_iters":  20
        },
        {
            "type":  "ResidualNorm",
            "reduction_factor":  1e-7
        }
    ]
}
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
25.9808
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
58.79
Solver iteration count:          20
Solver generation time [ms]:  0.065244
Solver execution time [ms]:  0.793764
Solver execution time per iteration[ms]:  0.0396882
```

### Comments about programming and debugging

# The plain program

```
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
```

```
#include <string>
#include <ginkgo/ginkgo.hpp>
#include <ginkgo/extensions/config/json_config.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    std::cout << gko::version_info::get() << std::endl;
    std::cout << argv[0] << " executor configfile matrix" << std::endl;
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    const auto configfile = argc >= 3 ?  argv[2] :  "config/cg.json";
    const std::string matrix_path = argc >= 4 ?  argv[3] :  "data/A.mtx";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()>
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(
                     0, gko::ReferenceExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream(matrix_path), exec));
    gko::size_type size = A->get_size()[0];
    auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    for (auto i = 0; i < size; i++) {
        host_x->at(i, 0) = 0.;
        host_b->at(i, 0) = 1.;
    }
    auto x = vec::create(exec);
    auto b = vec::create(exec);
    x->copy_from(host_x);
    b->copy_from(host_b);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto initres = gko::initialize<vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(initres);
    b->copy_from(host_b);
    auto config = gko::ext::config::parse_json_file(configfile);
    auto reg = gko::config::registry();
    auto td = gko::config::make_type_descriptor<ValueType, IndexType>();
    auto solver_gen = gko::config::parse(config, reg, td).on(exec);
    const auto gen_tic = std::chrono::steady_clock::now();
    auto solver = solver_gen->generate(A);
    exec->synchronize();
    const auto gen_toc = std::chrono::steady_clock::now();
    const auto gen_time = std::chrono::duration_cast<std::chrono::milliseconds>(
        gen_toc - gen_tic);
    std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
        gko::log::Convergence<ValueType>::create();
    solver->add_logger(logger);
    exec->synchronize();
    const auto tic = std::chrono::steady_clock::now();
    solver->apply(b, x);
    exec->synchronize();
    const auto toc = std::chrono::steady_clock::now();
    const auto time =
        std::chrono::duration_cast<std::chrono::milliseconds>(toc - tic);
    std::cout << "Config file:  " << configfile << std::endl;
    std::ifstream f(configfile);
    std::cout << f.rdbuf() << std::endl;
    auto res = gko::as<vec>(logger->get_residual_norm());
    std::cout << "Initial residual norm sqrt(r^T r):  \n";
    write(std::cout, initres);
    std::cout << "Final residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
    std::cout << "Solver iteration count:          " << logger->get_num_iterations()
              << std::endl;
    std::cout << "Solver generation time [ms]:  "
              << static_cast<double>(gen_time.count()) << std::endl;
    std::cout << "Solver execution time [ms]:  "
              << static_cast<double>(time.count()) << std::endl;
    std::cout << "Solver execution time per iteration[ms]:  "
              << static_cast<double>(time.count()) /
```

```
                logger->get_num_iterations()
            « std::endl;
}
```

# Chapter 19

# The ginkgo-overhead program

The ginkgo overhead measurement example..

## Introduction

### About the example

## The commented program

```
#include <chrono>
#include <cmath>
#include <iostream>
#include <ginkgo/ginkgo.hpp>
[[noreturn]] void print_usage_and_exit(const char* name)
{
    std::cerr « "Usage:  " « name « " [NUM_ITERS]" « std::endl;
    std::exit(-1);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    long unsigned num_iters = 1000000;
    if (argc > 2) {
        print_usage_and_exit(argv[0]);
    }
    if (argc == 2) {
        num_iters = std::atol(argv[1]);
        if (num_iters == 0) {
            print_usage_and_exit(argv[0]);
        }
    }
    std::cout « gko::version_info::get() « std::endl;
    auto exec = gko::ReferenceExecutor::create();
    auto cg_factory =
        cg::build()
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(num_iters))
            .on(exec);
    auto A = gko::initialize<mtx>({1.0}, exec);
    auto b = gko::initialize<vec>({std::nan("")}, exec);
    auto x = gko::initialize<vec>({0.0}, exec);
    auto tic = std::chrono::steady_clock::now();
    auto solver = cg_factory->generate(gko::give(A));
    solver->apply(x, b);
    exec->synchronize();
    auto tac = std::chrono::steady_clock::now();
    auto time = std::chrono::duration_cast<std::chrono::nanoseconds>(tac - tic);
    std::cout « "Running " « num_iters
              « " iterations of the CG solver took a total of "
```

```
              « static_cast<double>(time.count()) /
                     static_cast<double>(std::nano::den)
              « " seconds." « std::endl
              « "\tAverage library overhead:          "
              « static_cast<double>(time.count()) /
                     static_cast<double>(num_iters)
              « " [nanoseconds / iteration]" « std::endl;
}
```

# Results

This is the expected output:

```
Running 1000000 iterations of the CG solver took a total of 1.60337 seconds.
    Average library overhead:          1603.37 [nanoseconds / iteration]
```

### Comments about programming and debugging

# The plain program

```
#include <chrono>
#include <cmath>
#include <iostream>
#include <ginkgo/ginkgo.hpp>
[[noreturn]] void print_usage_and_exit(const char* name)
{
    std::cerr « "Usage:  " « name « " [NUM_ITERS]" « std::endl;
    std::exit(-1);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    long unsigned num_iters = 1000000;
    if (argc > 2) {
        print_usage_and_exit(argv[0]);
    }
    if (argc == 2) {
        num_iters = std::atol(argv[1]);
        if (num_iters == 0) {
            print_usage_and_exit(argv[0]);
        }
    }
    std::cout « gko::version_info::get() « std::endl;
    auto exec = gko::ReferenceExecutor::create();
    auto cg_factory =
        cg::build()
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(num_iters))
            .on(exec);
    auto A = gko::initialize<mtx>({1.0}, exec);
    auto b = gko::initialize<vec>({std::nan("")}, exec);
    auto x = gko::initialize<vec>({0.0}, exec);
    auto tic = std::chrono::steady_clock::now();
    auto solver = cg_factory->generate(gko::give(A));
    solver->apply(x, b);
    exec->synchronize();
    auto tac = std::chrono::steady_clock::now();
    auto time = std::chrono::duration_cast<std::chrono::nanoseconds>(tac - tic);
    std::cout « "Running " « num_iters
              « " iterations of the CG solver took a total of "
              « static_cast<double>(time.count()) /
                     static_cast<double>(std::nano::den)
              « " seconds." « std::endl
              « "\tAverage library overhead:          "
              « static_cast<double>(time.count()) /
                     static_cast<double>(num_iters)
              « " [nanoseconds / iteration]" « std::endl;
}
```

# Chapter 20

# The ginkgo-ranges program

The ranges and accessor example..

## Introduction

### About the example

## The commented program

```
#include <iomanip>
#include <iostream>
#include <ginkgo/ginkgo.hpp>
```

LU factorization implementation using Ginkgo ranges For simplicity, we only consider square matrices, and no pivoting.

```
template <typename Accessor>
void factorize(const gko::range<Accessor>& A)
```

note: const means that the range (i.e. the data handler) is constant, not that the underlying data is constant!

```
{
    using gko::span;
    assert(A.length(0) == A.length(1));
    for (gko::size_type i = 0; i < A.length(0) - 1; ++i) {
        const auto trail = span{i + 1, A.length(0)};
```

note: neither of the lines below need additional memory to store intermediate arrays, all computation is done at the point of assignment

```
A(trail, i) = A(trail, i) / A(i, i);
```

caveat: operator $*$ is element-wise multiplication, mmul is matrix multiplication

```
        A(trail, trail) = A(trail, trail) - mmul(A(trail, i), A(i, trail));
    }
}
```

a utility function for printing the factorization on screen

```
template <typename Accessor>
void print_lu(const gko::range<Accessor>& A)
{
    std::cout « std::setprecision(2) « std::fixed;
    std::cout « "L = [";
    for (int i = 0; i < A.length(0); ++i) {
        std::cout « "\n  ";
        for (int j = 0; j < A.length(1); ++j) {
            std::cout « (i > j ?  A(i, j) :  (i == j) * 1.)  « " ";
        }
    }
    std::cout « "\n]\n\nU = [";
```

```
    for (int i = 0; i < A.length(0); ++i) {
        std::cout « "\n  ";
        for (int j = 0; j < A.length(1); ++j) {
            std::cout « (i <= j ?  A(i, j) :  0.)  « " ";
        }
    }
    std::cout « "\n]" « std::endl;
}
int main(int argc, char* argv[])
{
    using ValueType = double;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Create some test data, add some padding just to demonstrate how to use it with ranges. clang-format off

```
ValueType data[] = {
    2.,  4.,  5., -1.0,
    4., 11., 12., -1.0,
    6., 24., 24., -1.0
};
```

clang-format on

Create a 3-by-3 range, with a 2D row-major accessor using data as the underlying storage. Set the stride (a.k.a. "LDA") to 4.

```
auto A =
    gko::range<gko::accessor::row_major<ValueType, 2>>(data, 3u, 3u, 4u);
```

use the LU factorization routine defined above to factorize the matrix

```
factorize(A);
```

print the factorization on screen

```
    print_lu(A);
}
```

# Results

This is the expected output:

```
L = [
  1.00 0.00 0.00
  2.00 1.00 0.00
  3.00 4.00 1.00
]
U = [
  2.00 4.00 5.00
  0.00 3.00 2.00
  0.00 0.00 1.00
]
```

### Comments about programming and debugging

# The plain program

```
#include <iomanip>
#include <iostream>
#include <ginkgo/ginkgo.hpp>
template <typename Accessor>
void factorize(const gko::range<Accessor>& A)
{
    using gko::span;
    assert(A.length(0) == A.length(1));
    for (gko::size_type i = 0; i < A.length(0) - 1; ++i) {
        const auto trail = span{i + 1, A.length(0)};
        A(trail, i) = A(trail, i) / A(i, i);
        A(trail, trail) = A(trail, trail) - mmul(A(trail, i), A(i, trail));
    }
}
template <typename Accessor>
```

```
void print_lu(const gko::range<Accessor>& A)
{
    std::cout « std::setprecision(2) « std::fixed;
    std::cout « "L = [";
    for (int i = 0; i < A.length(0); ++i) {
        std::cout « "\n  ";
        for (int j = 0; j < A.length(1); ++j) {
            std::cout « (i > j ?  A(i, j) :  (i == j) * 1.)  « " ";
        }
    }
    std::cout « "\n]\n\nU = [";
    for (int i = 0; i < A.length(0); ++i) {
        std::cout « "\n  ";
        for (int j = 0; j < A.length(1); ++j) {
            std::cout « (i <= j ?  A(i, j) :  0.)  « " ";
        }
    }
    std::cout « "\n]" « std::endl;
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    std::cout « gko::version_info::get() « std::endl;
    ValueType data[] = {
        2.,  4.,  5., -1.0,
        4., 11., 12., -1.0,
        6., 24., 24., -1.0
    };
    auto A =
        gko::range<gko::accessor::row_major<ValueType, 2>>(data, 3u, 3u, 4u);
    factorize(A);
    print_lu(A);
}
```

# Chapter 21

# The heat-equation program

The heat equation example..

This example depends on simple-solver, three-pt-stencil-solver.

## Introduction

This example solves a 2D heat conduction equation

$u : [0, d]^2 \to R$
$\partial_t u = \delta u + f$

with Dirichlet boundary conditions and given initial condition and constant-in-time source function f.

The partial differential equation (PDE) is solved with a finite difference spatial discretization on an equidistant grid: For $\mathrm{n}$ grid points, and grid distance $h = 1/n$ we write

$u_{i,j}{}' = \alpha \frac{u_{i-1,j}+u_{i+1,j}+u_{i,j-1}+u_{i,j+1}-4u_{i,j}}{h^2} + f_{i,j}$

We then build an implicit Euler integrator by discretizing with time step $\tau$

$\frac{u_{i,j}^{k+1}-u_{i,j}^{k}}{\tau} = \alpha \frac{u_{i-1,j}^{k+1}+u_{i+1,j}^{k+1}-u_{i,j-1}^{k+1}-u_{i,j+1}^{k+1}+4u_{i,j}^{k+1}}{h^2} + f_{i,j}$

and solve the resulting linear system for $u_{\cdot}^{k+1}$ using Ginkgo's CG solver preconditioned with an incomplete Cholesky factorization for each time step, occasionally writing the resulting grid values into a video file using OpenCV and a custom color mapping.

The intention of this example is to provide a mini-app showing matrix assembly, vector initialization, solver setup and the use of Ginkgo in a more complex setting.

### About the example

## The commented program

```
/ ****************************<DESCRIPTION>**********************************
This example solves a 2D heat conduction equation
    u :  [0, d]^2 \rightarrow R\\
    \partial_t u = \delta u + f
with Dirichlet boundary conditions and given initial condition and
constant-in-time source function f.
The partial differential equation (PDE) is solved with a finite difference
spatial discretization on an equidistant grid:  For `n` grid points,
and grid distance \fh = 1/n\f we write
    u_{i,j}' = \alpha (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1}
- 4 u_{i,j}) / h^2
+ f_{i,j}

We then build an implicit Euler integrator by discretizing with time step
\f\tau\f

(u_{i,j}^{k+1} - u_{i,j}^k) / \tau =
\alpha (u_{i-1,j}^{k+1} - u_{i+1,j}^{k+1}
+ u_{i,j-1}^{k+1} - u_{i,j+1}^{k+1} - 4 u_{i,j}^{k+1}) / h^2
+ f_{i,j}

and solve the resulting linear system for \f u_{\cdot}^{k+1}\f using
Ginkgo's CG solver preconditioned with an incomplete Cholesky factorization
for each time step, occasionally writing the resulting grid values into a
video file using OpenCV and a custom color mapping.
The intention of this example is to provide a mini-app showing matrix assembly,
vector initialization, solver setup and the use of Ginkgo in a more complex
setting.
****************************<DESCRIPTION>********************************** /
#include <chrono>
#include <fstream>
#include <iostream>
#include <opencv2/core.hpp>
#include <opencv2/videoio.hpp>
#include <ginkgo/ginkgo.hpp>
```

This function implements a simple Ginkgo-themed clamped color mapping for values in the range [0,5].

```
void set_val(unsigned char* data, double value)
{
```

RGB values for the 6 colors used for values 0, 1, ..., 5 We will interpolate linearly between these values.

```
double col_r[] = {255, 221, 129, 201, 249, 255};
double col_g[] = {255, 220, 130, 161, 158, 204};
double col_b[] = {255, 220, 133, 93, 24, 8};
value = std::max(0.0, value);
auto i = std::max(0, std::min(4, int(value)));
auto d = std::max(0.0, std::min(1.0, value - i));
```

OpenCV uses BGR instead of RGB by default, revert indices

```
    data[2] = static_cast<unsigned char>(col_r[i + 1] * d + col_r[i] * (1 - d));
    data[1] = static_cast<unsigned char>(col_g[i + 1] * d + col_g[i] * (1 - d));
    data[0] = static_cast<unsigned char>(col_b[i + 1] * d + col_b[i] * (1 - d));
}
```

Initialize video output with given dimension and FPS (frames per seconds)

```
std::pair<cv::VideoWriter, cv::Mat> build_output(int n, double fps)
{
    cv::Size videosize{n, n};
    auto output =
        std::make_pair(cv::VideoWriter{}, cv::Mat{videosize, CV_8UC3});
    auto fourcc = cv::VideoWriter::fourcc('a', 'v', 'c', '1');
    output.first.open("heat.mp4", fourcc, fps, videosize);
    return output;
}
```

Write the current frame to video output using the above color mapping

```
void output_timestep(std::pair<cv::VideoWriter, cv::Mat>& output, int n,
                     const double* data)
{
    for (int i = 0; i < n; i++) {
        auto row = output.second.ptr(i);
        for (int j = 0; j < n; j++) {
            set_val(&row[3 * j], data[i * n + j]);
        }
    }
    output.first.write(output.second);
```

```
}
int main(int argc, char* argv[])
{
    using mtx = gko::matrix::Csr<>;
    using vec = gko::matrix::Dense<>;
```

Problem parameters: simulation length

```
auto t0 = 5.0;
```

diffusion factor

```
auto diffusion = 0.0005;
```

scaling factor for heat source

```
auto source_scale = 2.5;
```

Simulation parameters: inner grid points per discretization direction

```
auto n = 256;
```

number of simulation steps per second

```
auto steps_per_sec = 500;
```

number of video frames per second

```
auto fps = 25;
```

number of grid points

```
auto n2 = n * n;
```

grid point distance (ignoring boundary points)

```
auto h = 1.0 / (n + 1);
auto h2 = h * h;
```

time step size for the simulation

```
auto tau = 1.0 / steps_per_sec;
```

create a CUDA executor with an associated OpenMP host executor

```
auto exec = gko::CudaExecutor::create(0, gko::OmpExecutor::create());
```

load heat source and initial state vectors

```
std::ifstream initial_stream("data/gko_logo_2d.mtx");
std::ifstream source_stream("data/gko_text_2d.mtx");
auto source = gko::read<vec>(source_stream, exec);
auto in_vector = gko::read<vec>(initial_stream, exec);
```

create output vector with initial guess for

```
auto out_vector = in_vector->clone();
```

create scalar for source update

```
auto tau_source_scalar = gko::initialize<vec>({source_scale * tau}, exec);
```

create stencil matrix as shared_ptr for solver

```
auto stencil_matrix = gko::share(mtx::create(exec));
```

assemble matrix

```
gko::matrix_data<> mtx_data{gko::dim<2>(n2, n2)};
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        auto c = i * n + j;
        auto c_val = diffusion * tau * 4.0 / h2 + 1.0;
        auto off_val = -diffusion * tau / h2;
```

for each grid point: insert 5 stencil points with Dirichlet boundary conditions, i.e. with zero boundary value

```
        if (i > 0) {
            mtx_data.nonzeros.emplace_back(c, c - n, off_val);
        }
        if (j > 0) {
            mtx_data.nonzeros.emplace_back(c, c - 1, off_val);
        }
        mtx_data.nonzeros.emplace_back(c, c, c_val);
        if (j < n - 1) {
            mtx_data.nonzeros.emplace_back(c, c + 1, off_val);
        }
        if (i < n - 1) {
```

```
            mtx_data.nonzeros.emplace_back(c, c + n, off_val);
        }
    }
}
stencil_matrix->read(mtx_data);
```

prepare video output

```
auto output = build_output(n, fps);
```

build CG solver on stencil with incomplete Cholesky preconditioner stopping at 1e-10 relative accuracy

```
auto solver =
    gko::solver::Cg<>::build()
        .with_preconditioner(gko::preconditioner::Ic<>::build())
        .with_criteria(gko::stop::ResidualNorm<>::build()
                           .with_baseline(gko::stop::mode::rhs_norm)
                           .with_reduction_factor(1e-10))
        .on(exec)
        ->generate(stencil_matrix);
```

time stamp of the last output frame (initialized to a sentinel value)

```
double last_t = -t0;
```

execute implicit Euler method: for each timestep, solve stencil system

```
for (double t = 0; t < t0; t += tau) {
```

if enough time has passed, output the next video frame

```
if (t - last_t > 1.0 / fps) {
    last_t = t;
    std::cout « t « std::endl;
    output_timestep(
        output, n,
        gko::make_temporary_clone(exec->get_master(), in_vector)
            ->get_const_values());
}
```

add heat source contribution

```
in_vector->add_scaled(tau_source_scalar, source);
```

execute Euler step

```
solver->apply(in_vector, out_vector);
```

swap input and output

```
        std::swap(in_vector, out_vector);
    }
}
```

# Results

The program will generate a video file named heat.mp4 and output the timestamp of each generated frame.

### Comments about programming and debugging

# The plain program

```
/*****************************<DESCRIPTION>**********************************
This example solves a 2D heat conduction equation

u :  [0, d]^2 \rightarrow R\\
\partial_t u = \delta u + f

with Dirichlet boundary conditions and given initial condition and
constant-in-time source function f.

The partial differential equation (PDE) is solved with a finite difference
spatial discretization on an equidistant grid:  For `n` grid points,
and grid distance \f$h = 1/n\f$ we write

u_{i,j}' = \alpha (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1}
```

```
- 4 u_{i,j}) / h^2
+ f_{i,j}

We then build an implicit Euler integrator by discretizing with time step
\f$\tau\f$

(u_{i,j}^{k+1} - u_{i,j}^k) / \tau =
\alpha (u_{i-1,j}^{k+1} - u_{i+1,j}^{k+1}
+ u_{i,j-1}^{k+1} - u_{i,j+1}^{k+1} - 4 u_{i,j}^{k+1}) / h^2
+ f_{i,j}

and solve the resulting linear system for \f$ u_{\cdot}^{k+1}\f$ using
Ginkgo's CG solver preconditioned with an incomplete Cholesky factorization
for each time step, occasionally writing the resulting grid values into a
video file using OpenCV and a custom color mapping.

The intention of this example is to provide a mini-app showing matrix assembly,
vector initialization, solver setup and the use of Ginkgo in a more complex
setting.
*****************************<DESCRIPTION>*********************************/
#include <chrono>
#include <fstream>
#include <iostream>
#include <opencv2/core.hpp>
#include <opencv2/videoio.hpp>
#include <ginkgo/ginkgo.hpp>
void set_val(unsigned char* data, double value)
{
    double col_r[] = {255, 221, 129, 201, 249, 255};
    double col_g[] = {255, 220, 130, 161, 158, 204};
    double col_b[] = {255, 220, 133, 93, 24, 8};
    value = std::max(0.0, value);
    auto i = std::max(0, std::min(4, int(value)));
    auto d = std::max(0.0, std::min(1.0, value - i));
    data[2] = static_cast<unsigned char>(col_r[i + 1] * d + col_r[i] * (1 - d));
    data[1] = static_cast<unsigned char>(col_g[i + 1] * d + col_g[i] * (1 - d));
    data[0] = static_cast<unsigned char>(col_b[i + 1] * d + col_b[i] * (1 - d));
}
std::pair<cv::VideoWriter, cv::Mat> build_output(int n, double fps)
{
    cv::Size videosize{n, n};
    auto output =
        std::make_pair(cv::VideoWriter{}, cv::Mat{videosize, CV_8UC3});
    auto fourcc = cv::VideoWriter::fourcc('a', 'v', 'c', '1');
    output.first.open("heat.mp4", fourcc, fps, videosize);
    return output;
}
void output_timestep(std::pair<cv::VideoWriter, cv::Mat>& output, int n,
                     const double* data)
{
    for (int i = 0; i < n; i++) {
        auto row = output.second.ptr(i);
        for (int j = 0; j < n; j++) {
            set_val(&row[3 * j], data[i * n + j]);
        }
    }
    output.first.write(output.second);
}
int main(int argc, char* argv[])
{
    using mtx = gko::matrix::Csr<>;
    using vec = gko::matrix::Dense<>;
    auto t0 = 5.0;
    auto diffusion = 0.0005;
    auto source_scale = 2.5;
    auto n = 256;
    auto steps_per_sec = 500;
    auto fps = 25;
    auto n2 = n * n;
    auto h = 1.0 / (n + 1);
    auto h2 = h * h;
    auto tau = 1.0 / steps_per_sec;
    auto exec = gko::CudaExecutor::create(0, gko::OmpExecutor::create());
    std::ifstream initial_stream("data/gko_logo_2d.mtx");
    std::ifstream source_stream("data/gko_text_2d.mtx");
    auto source = gko::read<vec>(source_stream, exec);
    auto in_vector = gko::read<vec>(initial_stream, exec);
    auto out_vector = in_vector->clone();
    auto tau_source_scalar = gko::initialize<vec>({source_scale * tau}, exec);
    auto stencil_matrix = gko::share(mtx::create(exec));
    gko::matrix_data<> mtx_data{gko::dim<2>(n2, n2)};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            auto c = i * n + j;
            auto c_val = diffusion * tau * 4.0 / h2 + 1.0;
            auto off_val = -diffusion * tau / h2;
            if (i > 0) {
```

```
                mtx_data.nonzeros.emplace_back(c, c - n, off_val);
            }
            if (j > 0) {
                mtx_data.nonzeros.emplace_back(c, c - 1, off_val);
            }
            mtx_data.nonzeros.emplace_back(c, c, c_val);
            if (j < n - 1) {
                mtx_data.nonzeros.emplace_back(c, c + 1, off_val);
            }
            if (i < n - 1) {
                mtx_data.nonzeros.emplace_back(c, c + n, off_val);
            }
        }
    }
    stencil_matrix->read(mtx_data);
    auto output = build_output(n, fps);
    auto solver =
        gko::solver::Cg<>::build()
            .with_preconditioner(gko::preconditioner::Ic<>::build())
            .with_criteria(gko::stop::ResidualNorm<>::build()
                               .with_baseline(gko::stop::mode::rhs_norm)
                               .with_reduction_factor(1e-10))
            .on(exec)
            ->generate(stencil_matrix);
    double last_t = -t0;
    for (double t = 0; t < t0; t += tau) {
        if (t - last_t > 1.0 / fps) {
            last_t = t;
            std::cout « t « std::endl;
            output_timestep(
                output, n,
                gko::make_temporary_clone(exec->get_master(), in_vector)
                    ->get_const_values());
        }
        in_vector->add_scaled(tau_source_scalar, source);
        solver->apply(in_vector, out_vector);
        std::swap(in_vector, out_vector);
    }
}
```

# Chapter 22

# The ilu-preconditioned-solver program

The ILU-preconditioned solver example..

This example depends on simple-solver.

## Introduction

### About the example

This example shows how to use incomplete factors generated via the ParILU algorithm to generate an incomplete factorization (ILU) preconditioner, how to specify the sparse triangular solves in the ILU preconditioner application, and how to generate an ILU-preconditioned solver and apply it to a specific problem.

## The commented program

```
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using gmres = gko::solver::Gmres<ValueType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
```

```
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = gko::share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
```

Generate incomplete factors using ParILU

```
auto par_ilu_fact =
    gko::factorization::ParIlu<ValueType, IndexType>::build().on(exec);
```

Generate concrete factorization for input matrix

```
auto par_ilu = gko::share(par_ilu_fact->generate(A));
```

Generate an ILU preconditioner factory by setting lower and upper triangular solver - in this case the exact triangular solves

```
auto ilu_pre_factory =
    gko::preconditioner::Ilu<ValueType, false, IndexType>::build().on(exec);
```

Use incomplete factors to generate ILU preconditioner

```
auto ilu_preconditioner = gko::share(ilu_pre_factory->generate(par_ilu));
```

Use preconditioner inside GMRES solver factory Generating a solver factory tied to a specific preconditioner makes sense if there are several very similar systems to solve, and the same solver+preconditioner combination is expected to be effective.

```
const RealValueType reduction_factor{1e-7};
auto ilu_gmres_factory =
    gmres::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
        .with_generated_preconditioner(ilu_preconditioner)
        .on(exec);
```

Generate preconditioned solver for a specific target system

```
auto ilu_gmres = ilu_gmres_factory->generate(A);
```

Solve system

```
ilu_gmres->apply(b, x);
```

Print solution

```
std::cout « "Solution (x):\n";
write(std::cout, x);
```

Calculate residual

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Results

This is the expected output:

```
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
1.46249e-08
```

### Comments about programming and debugging

# The plain program

```
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using gmres = gko::solver::Gmres<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = gko::share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    auto par_ilu_fact =
        gko::factorization::ParIlu<ValueType, IndexType>::build().on(exec);
```

```
    auto par_ilu = gko::share(par_ilu_fact->generate(A));
    auto ilu_pre_factory =
        gko::preconditioner::Ilu<ValueType, false, IndexType>::build().on(exec);
    auto ilu_preconditioner = gko::share(ilu_pre_factory->generate(par_ilu));
    const RealValueType reduction_factor{1e-7};
    auto ilu_gmres_factory =
        gmres::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1000u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .with_generated_preconditioner(ilu_preconditioner)
            .on(exec);
    auto ilu_gmres = ilu_gmres_factory->generate(A);
    ilu_gmres->apply(b, x);
    std::cout << "Solution (x):\n";
    write(std::cout, x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout << "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Chapter 23

# The inverse-iteration program

The inverse iteration example..

This example depends on simple-solver, .

## Introduction

This example shows how components available in Ginkgo can be used to implement higher-level numerical methods. The method used here will be the shifted inverse iteration method for eigenvalue computation which find the eigenvalue and eigenvector of A closest to z, for some scalar z. The method requires repeatedly solving the shifted linear system (A - zI)x = b, as well as performing matrix-vector products with the matrix `A`. Here is the complete pseudocode of the method:

```
x_0 = initial guess
for i = 0 ..  max_iterations:
    solve (A - zI) y_i = x_i for y_i+1
    x_(i+1) = y_i / || y_i ||       # compute next eigenvector approximation
    g_(i+1) = x_(i+1)^* A x_(i+1)  # approximate eigenvalue (Rayleigh quotient)
    if ||A x_(i+1) - g_(i+1)x_(i+1)|| < tol * g_(i+1):    # check convergence
break
```

#### About the example

## The commented program

```
#include <cmath>
#include <complex>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using precision = std::complex<double>;
using real_precision = gko::remove_complex<precision>;
using vec = gko::matrix::Dense<precision>;
using real_vec = gko::matrix::Dense<real_precision>;
using mtx = gko::matrix::Csr<precision>;
using solver_type = gko::solver::Bicgstab<precision>;
using std::abs;
using std::sqrt;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
std::cout « std::scientific « std::setprecision(8) « std::showpos;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
auto this_exec = exec->get_master();
```

linear system solver parameters

```
auto system_max_iterations = 100u;
auto system_residual_goal = real_precision{1e-16};
```

eigensolver parameters

```
auto max_iterations = 20u;
auto residual_goal = real_precision{1e-8};
auto z = precision{20.0, 2.0};
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

Generate shifted matrix A - zI

- we avoid duplicating memory by not storing both A and A - zI, but compute A - zI on the fly by using Ginkgo's utilities for creating linear combinations of operators

```
auto one = share(gko::initialize<vec>({precision{1.0}}, exec));
auto neg_one = share(gko::initialize<vec>({-precision{1.0}}, exec));
auto neg_z = gko::initialize<vec>({-z}, exec);
auto system_matrix = share(gko::Combination<precision>::create(
    one, A, gko::initialize<vec>({-z}, exec),
    gko::matrix::Identity<precision>::create(exec, A->get_size()[0])));
```

Generate solver operator (A - zI)^-1

```
auto solver =
    solver_type::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(
                           system_max_iterations),
                       gko::stop::ResidualNorm<precision>::build()
                           .with_reduction_factor(system_residual_goal))
        .on(exec)
        ->generate(system_matrix);
```

inverse iterations

start with guess [1, 1, ..., 1]

```
auto x = [&] {
    auto work = vec::create(this_exec, gko::dim<2>{A->get_size()[0], 1});
    const auto n = work->get_size()[0];
    for (int i = 0; i < n; ++i) {
        work->get_values()[i] = precision{1.0} / sqrt(n);
    }
```

```
    return clone(exec, work);
}();
auto y = clone(x);
auto tmp = clone(x);
auto norm = gko::initialize<real_vec>({1.0}, exec);
auto inv_norm = clone(this_exec, one);
auto g = clone(one);
for (auto i = 0u; i < max_iterations; ++i) {
    std::cout << "{ ";
```

(A - zI)y = x

```
solver->apply(x, y);
system_matrix->apply(one, y, neg_one, x);
x->compute_norm2(norm);
std::cout << "\"system_residual\":  "
          << clone(this_exec, norm)->get_values()[0] << ", ";
x->copy_from(y);
```

x = y / || y ||

```
x->compute_norm2(norm);
inv_norm->get_values()[0] =
    real_precision{1.0} / clone(this_exec, norm)->get_values()[0];
x->scale(clone(exec, inv_norm));
```

g = x^* A x

```
A->apply(x, tmp);
x->compute_conj_dot(tmp, g);
auto g_val = clone(this_exec, g)->get_values()[0];
std::cout << "\"eigenvalue\":  " << g_val << ", ";
```

||Ax - gx|| < tol * g

```
        auto v = gko::initialize<vec>({-g_val}, exec);
        tmp->add_scaled(v, x);
        tmp->compute_norm2(norm);
        auto res_val = clone(exec->get_master(), norm)->get_values()[0];
        std::cout << "\"residual\":  " << res_val / g_val << " }," << std::endl;
        if (abs(res_val) < residual_goal * abs(g_val)) {
            break;
        }
    }
}
```

# Results

This is the expected output:

```
{ "system_residual":  +1.61736920e-14, "eigenvalue":  (+2.03741410e+01,-1.17744356e-16), "residual":
      (+2.92231055e-01,+1.68883476e-18) },
{ "system_residual":  +4.98014795e-15, "eigenvalue":  (+1.94878474e+01,+1.25948378e-15), "residual":
      (+7.94370276e-02,-5.13395071e-18) },
{ "system_residual":  +3.39296916e-15, "eigenvalue":  (+1.93282121e+01,-1.19329332e-15), "residual":
      (+4.11149623e-02,+2.53837290e-18) },
{ "system_residual":  +3.35953656e-15, "eigenvalue":  (+1.92638912e+01,+3.28657016e-16), "residual":
      (+2.34717040e-02,-4.00445585e-19) },
{ "system_residual":  +2.91474009e-15, "eigenvalue":  (+1.92409166e+01,+3.65597737e-16), "residual":
      (+1.34709547e-02,-2.55962367e-19) },
{ "system_residual":  +3.09863953e-15, "eigenvalue":  (+1.92331106e+01,-1.07919176e-15), "residual":
      (+7.72060707e-03,+4.33212063e-19) },
{ "system_residual":  +2.31198069e-15, "eigenvalue":  (+1.92305014e+01,-2.89755360e-16), "residual":
      (+4.42106625e-03,+6.66143651e-20) },
{ "system_residual":  +3.02771202e-15, "eigenvalue":  (+1.92296339e+01,+8.04259901e-16), "residual":
      (+2.53081312e-03,-1.05848687e-19) },
{ "system_residual":  +2.02954523e-15, "eigenvalue":  (+1.92293461e+01,+7.81834016e-16), "residual":
      (+1.44862114e-03,-5.88985854e-20) },
{ "system_residual":  +2.31762332e-15, "eigenvalue":  (+1.92292506e+01,-1.11718775e-16), "residual":
      (+8.29183451e-04,+4.81741912e-21) },
{ "system_residual":  +8.12541038e-15, "eigenvalue":  (+1.92292190e+01,-6.55606254e-16), "residual":
      (+4.74636702e-04,+1.61823936e-20) },
{ "system_residual":  +2.77259926e-15, "eigenvalue":  (+1.92292085e+01,+4.30588140e-16), "residual":
      (+2.71701077e-04,-6.08403935e-21) },
{ "system_residual":  +8.87888675e-14, "eigenvalue":  (+1.92292051e+01,+9.67936313e-18), "residual":
      (+1.55539937e-04,-7.82937998e-23) },
{ "system_residual":  +2.85077117e-15, "eigenvalue":  (+1.92292039e+01,-4.52923128e-16), "residual":
      (+8.90457139e-05,+2.09737561e-21) },
{ "system_residual":  +6.46865302e-14, "eigenvalue":  (+1.92292035e+01,+1.58710681e-17), "residual":
      (+5.09805252e-05,-4.20774259e-23) },
{ "system_residual":  +4.18913713e-15, "eigenvalue":  (+1.92292034e+01,+1.06839590e-15), "residual":
      (+2.91887365e-05,-1.62175862e-21) },
```

```
{ "system_residual":  +1.06421578e-11, "eigenvalue":  (+1.92292034e+01,+3.26089685e-17), "residual":
      (+1.67126561e-05,-2.83413965e-23) },
{ "system_residual":  +2.97434420e-13, "eigenvalue":  (+1.92292034e+01,-7.85427712e-16), "residual":
      (+9.56961199e-06,+3.90876227e-22) },
{ "system_residual":  +1.63230281e-11, "eigenvalue":  (+1.92292033e+01,+3.69307000e-16), "residual":
      (+5.47975753e-06,-1.05241636e-22) },
{ "system_residual":  +6.14939758e-14, "eigenvalue":  (+1.92292033e+01,+1.36057865e-15), "residual":
      (+3.13794996e-06,-2.22028320e-22) },
```

### Comments about programming and debugging

## The plain program

```
#include <cmath>
#include <complex>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using precision = std::complex<double>;
    using real_precision = gko::remove_complex<precision>;
    using vec = gko::matrix::Dense<precision>;
    using real_vec = gko::matrix::Dense<real_precision>;
    using mtx = gko::matrix::Csr<precision>;
    using solver_type = gko::solver::Bicgstab<precision>;
    using std::abs;
    using std::sqrt;
    std::cout « gko::version_info::get() « std::endl;
    std::cout « std::scientific « std::setprecision(8) « std::showpos;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto this_exec = exec->get_master();
    auto system_max_iterations = 100u;
    auto system_residual_goal = real_precision{1e-16};
    auto max_iterations = 20u;
    auto residual_goal = real_precision{1e-8};
    auto z = precision{20.0, 2.0};
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto one = share(gko::initialize<vec>({precision{1.0}}, exec));
    auto neg_one = share(gko::initialize<vec>({-precision{1.0}}, exec));
    auto neg_z = gko::initialize<vec>({-z}, exec);
    auto system_matrix = share(gko::Combination<precision>::create(
        one, A, gko::initialize<vec>({-z}, exec),
        gko::matrix::Identity<precision>::create(exec, A->get_size()[0])));
    auto solver =
        solver_type::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(
                               system_max_iterations),
                           gko::stop::ResidualNorm<precision>::build()
                               .with_reduction_factor(system_residual_goal))
            .on(exec)
            ->generate(system_matrix);
    auto x = [&] {
        auto work = vec::create(this_exec, gko::dim<2>{A->get_size()[0], 1});
        const auto n = work->get_size()[0];
```

```
        for (int i = 0; i < n; ++i) {
            work->get_values()[i] = precision{1.0} / sqrt(n);
        }
        return clone(exec, work);
    }();
    auto y = clone(x);
    auto tmp = clone(x);
    auto norm = gko::initialize<real_vec>({1.0}, exec);
    auto inv_norm = clone(this_exec, one);
    auto g = clone(one);
    for (auto i = 0u; i < max_iterations; ++i) {
        std::cout << "{ ";
        solver->apply(x, y);
        system_matrix->apply(one, y, neg_one, x);
        x->compute_norm2(norm);
        std::cout << "\"system_residual\":  "
                  << clone(this_exec, norm)->get_values()[0] << ", ";
        x->copy_from(y);
        x->compute_norm2(norm);
        inv_norm->get_values()[0] =
            real_precision{1.0} / clone(this_exec, norm)->get_values()[0];
        x->scale(clone(exec, inv_norm));
        A->apply(x, tmp);
        x->compute_conj_dot(tmp, g);
        auto g_val = clone(this_exec, g)->get_values()[0];
        std::cout << "\"eigenvalue\":  " << g_val << ", ";
        auto v = gko::initialize<vec>({-g_val}, exec);
        tmp->add_scaled(v, x);
        tmp->compute_norm2(norm);
        auto res_val = clone(exec->get_master(), norm)->get_values()[0];
        std::cout << "\"residual\":  " << res_val / g_val << " }," << std::endl;
        if (abs(res_val) < residual_goal * abs(g_val)) {
            break;
        }
    }
}
```

# Chapter 24

# The ir-ilu-preconditioned-solver program

The IR-ILU preconditioned solver example..

This example depends on ilu-preconditioned-solver, iterative-refinement.

## Introduction

### About the example

This example shows how to combine iterative refinement with the adaptive precision block-Jacobi preconditioner in order to approximately solve the triangular systems occurring in ILU preconditioning. Using an adaptive precision block-Jacobi preconditioner matrix as inner solver for the iterative refinement method is equivalent to doing adaptive precision block-Jacobi relaxation in the triangular solves. This example roughly approximates the triangular solves with five adaptive precision block-Jacobi sweeps with a maximum block size of 16.

This example is motivated by "Multiprecision block-Jacobi for Iterative Triangular Solves" (Göbel, Anzt, Cojean, Flegar, Quintana-Ortí, Euro-Par 2020). The theory and a detailed analysis can be found there.

## The commented program

```
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using gmres = gko::solver::Gmres<ValueType>;
using ir = gko::solver::Ir<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const unsigned int sweeps = argc == 3 ?  std::atoi(argv[2]) :  5u;
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = gko::share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

Create RHS and initial guess as 1

```
gko::size_type num_rows = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(num_rows, 1));
for (gko::size_type i = 0; i < num_rows; i++) {
    host_x->at(i, 0) = 1.;
}
auto x = gko::clone(exec, host_x);
auto b = gko::clone(exec, host_x);
auto clone_x = gko::clone(exec, x);
```

Generate incomplete factors using ParILU

```
auto par_ilu_fact =
    gko::factorization::ParIlu<ValueType, IndexType>::build().on(exec);
```

Generate concrete factorization for input matrix

```
auto par_ilu = gko::share(par_ilu_fact->generate(A));
```

Generate an iterative refinement factory to be used as a triangular solver in the preconditioner application. The generated method is equivalent to doing five block-Jacobi sweeps with a maximum block size of 16.

```
auto bj_factory = gko::share(
    bj::build()
        .with_max_block_size(16u)
        .with_storage_optimization(gko::precision_reduction::autodetect())
        .on(exec));
auto trisolve_factory = gko::share(
    ir::build()
        .with_solver(bj_factory)
        .with_criteria(gko::stop::Iteration::build().with_max_iters(sweeps))
        .on(exec));
```

Generate an ILU preconditioner factory by setting lower and upper triangular solver - in this case the previously defined iterative refinement method.

```
auto ilu_pre_factory = gko::preconditioner::Ilu<ValueType>::build()
                           .with_l_solver(trisolve_factory)
                           .with_u_solver(trisolve_factory)
                           .on(exec);
```

Use incomplete factors to generate ILU preconditioner

```
auto ilu_preconditioner = gko::share(ilu_pre_factory->generate(par_ilu));
```

Create stopping criteria for Gmres

```
const RealValueType reduction_factor{1e-12};
auto iter_stop = gko::share(
    gko::stop::Iteration::build().with_max_iters(1000u).on(exec));
auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor)
                               .on(exec));
```

Use preconditioner inside GMRES solver factory Generating a solver factory tied to a specific preconditioner makes sense if there are several very similar systems to solve, and the same solver+preconditioner combination is expected to be effective.

```
auto ilu_gmres_factory =
    gmres::build()
        .with_criteria(iter_stop, tol_stop)
        .with_generated_preconditioner(ilu_preconditioner)
        .on(exec);
```

Generate preconditioned solver for a specific target system

```
auto ilu_gmres = ilu_gmres_factory->generate(A);
```

Add logger

```
std::shared_ptr<const gko::log::Convergence<ValueType» logger =
    gko::log::Convergence<ValueType>::create();
ilu_gmres->add_logger(logger);
```

Warmup run

```
ilu_gmres->apply(b, x);
```

Solve system 100 times and take the average time.

```
std::chrono::nanoseconds time(0);
for (int i = 0; i < 100; i++) {
    x->copy_from(clone_x);
    auto tic = std::chrono::high_resolution_clock::now();
    ilu_gmres->apply(b, x);
    auto toc = std::chrono::high_resolution_clock::now();
    time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
}
std::cout « "Using " « sweeps « " block-Jacobi sweeps.\n";
```

Print solution

```
std::cout « "Solution (x):\n";
write(std::cout, x);
```

Get residual

```
    auto res = gko::as<vec>(logger->get_residual_norm());
    std::cout « "GMRES iteration count:          " « logger->get_num_iterations()
              « "\n";
    std::cout « "GMRES execution time [ms]:  "
              « static_cast<double>(time.count()) / 100000000.0 « "\n";
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Results

This is the expected output:

```
Using 5 block-Jacobi sweeps.
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
GMRES iteration count:          8
GMRES execution time [ms]:  0.377673
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
1.65303e-12
```

### Comments about programming and debugging

## The plain program

```
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using gmres = gko::solver::Gmres<ValueType>;
    using ir = gko::solver::Ir<ValueType>;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    const unsigned int sweeps = argc == 3 ?  std::atoi(argv[2]) :  5u;
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = gko::share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    gko::size_type num_rows = A->get_size()[0];
    auto host_x = vec::create(exec->get_master(), gko::dim<2>(num_rows, 1));
    for (gko::size_type i = 0; i < num_rows; i++) {
        host_x->at(i, 0) = 1.;
    }
    auto x = gko::clone(exec, host_x);
    auto b = gko::clone(exec, host_x);
    auto clone_x = gko::clone(exec, x);
    auto par_ilu_fact =
        gko::factorization::ParIlu<ValueType, IndexType>::build().on(exec);
    auto par_ilu = gko::share(par_ilu_fact->generate(A));
    auto bj_factory = gko::share(
        bj::build()
            .with_max_block_size(16u)
            .with_storage_optimization(gko::precision_reduction::autodetect())
            .on(exec));
    auto trisolve_factory = gko::share(
        ir::build()
            .with_solver(bj_factory)
            .with_criteria(gko::stop::Iteration::build().with_max_iters(sweeps))
            .on(exec));
    auto ilu_pre_factory = gko::preconditioner::Ilu<ValueType>::build()
                               .with_l_solver(trisolve_factory)
                               .with_u_solver(trisolve_factory)
                               .on(exec);
    auto ilu_preconditioner = gko::share(ilu_pre_factory->generate(par_ilu));
    const RealValueType reduction_factor{1e-12};
    auto iter_stop = gko::share(
        gko::stop::Iteration::build().with_max_iters(1000u).on(exec));
    auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                                   .with_reduction_factor(reduction_factor)
                                   .on(exec));
    auto ilu_gmres_factory =
        gmres::build()
            .with_criteria(iter_stop, tol_stop)
            .with_generated_preconditioner(ilu_preconditioner)
            .on(exec);
```

```
    auto ilu_gmres = ilu_gmres_factory->generate(A);
    std::shared_ptr<const gko::log::Convergence<ValueType» logger =
        gko::log::Convergence<ValueType>::create();
    ilu_gmres->add_logger(logger);
    ilu_gmres->apply(b, x);
    std::chrono::nanoseconds time(0);
    for (int i = 0; i < 100; i++) {
        x->copy_from(clone_x);
        auto tic = std::chrono::high_resolution_clock::now();
        ilu_gmres->apply(b, x);
        auto toc = std::chrono::high_resolution_clock::now();
        time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
    }
    std::cout « "Using " « sweeps « " block-Jacobi sweeps.\n";
    std::cout « "Solution (x):\n";
    write(std::cout, x);
    auto res = gko::as<vec>(logger->get_residual_norm());
    std::cout « "GMRES iteration count:          " « logger->get_num_iterations()
              « "\n";
    std::cout « "GMRES execution time [ms]:  "
              « static_cast<double>(time.count()) / 100000000.0 « "\n";
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Chapter 25

# The iterative-refinement program

The iterative refinement solver example..

This example depends on simple-solver.

## This example shows how to use the iterative refinement solver.

**In this example, we first read in a matrix from file, then generate a right-hand side and an initial guess. An inaccurate CG solver is used as the inner solver to an iterative refinement (IR) method which solves a linear system. The example features the iteration count and runtime of the IR solver.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using ir = gko::solver::Ir<ValueType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
```

```
        }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

### executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

### Create RHS and initial guess as 1

```
gko::size_type size = A->get_size()[0];
auto host_x = gko::matrix::Dense<ValueType>::create(exec->get_master(),
                                                    gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 1.;
}
auto x = gko::clone(exec, host_x);
auto b = gko::clone(exec, host_x);
```

### Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<real_vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

### copy b again

```
b->copy_from(host_x);
```

### Create solver factory

```
gko::size_type max_iters = 10000u;
RealValueType outer_reduction_factor{1e-12};
RealValueType inner_reduction_factor{1e-2};
auto solver_gen =
    ir::build()
        .with_solver(cg::build().with_criteria(
            gko::stop::ResidualNorm<ValueType>::build()
                .with_reduction_factor(inner_reduction_factor)))
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(max_iters),
            gko::stop::ResidualNorm<ValueType>::build()
                .with_reduction_factor(outer_reduction_factor))
        .on(exec);
```

### Create solver

```
auto solver = solver_gen->generate(A);
std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
    gko::log::Convergence<ValueType>::create();
solver->add_logger(logger);
```

### Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

### Get residual

```
auto res = gko::as<real_vec>(logger->get_residual_norm());
std::cout << "Initial residual norm sqrt(r^T r):\n";
write(std::cout, initres);
std::cout << "Final residual norm sqrt(r^T r):\n";
write(std::cout, res);
```

### Print solver statistics

```
    std::cout << "IR iteration count:          " << logger->get_num_iterations()
              << std::endl;
    std::cout << "IR execution time [ms]:  "
              << static_cast<double>(time.count()) / 1000000.0 << std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
194.679
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
4.23821e-11
IR iteration count:          24
IR execution time [ms]:  0.794962
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using ir = gko::solver::Ir<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    gko::size_type size = A->get_size()[0];
    auto host_x = gko::matrix::Dense<ValueType>::create(exec->get_master(),
                                                        gko::dim<2>(size, 1));
    for (auto i = 0; i < size; i++) {
        host_x->at(i, 0) = 1.;
    }
    auto x = gko::clone(exec, host_x);
    auto b = gko::clone(exec, host_x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto initres = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(initres);
    b->copy_from(host_x);
    gko::size_type max_iters = 10000u;
    RealValueType outer_reduction_factor{1e-12};
    RealValueType inner_reduction_factor{1e-2};
    auto solver_gen =
        ir::build()
```

```
            .with_solver(cg::build().with_criteria(
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(inner_reduction_factor)))
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(max_iters),
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(outer_reduction_factor))
            .on(exec);
    auto solver = solver_gen->generate(A);
    std::shared_ptr<const gko::log::Convergence<ValueType» logger =
        gko::log::Convergence<ValueType>::create();
    solver->add_logger(logger);
    exec->synchronize();
    std::chrono::nanoseconds time(0);
    auto tic = std::chrono::steady_clock::now();
    solver->apply(b, x);
    auto toc = std::chrono::steady_clock::now();
    time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
    auto res = gko::as<real_vec>(logger->get_residual_norm());
    std::cout « "Initial residual norm sqrt(r^T r):\n";
    write(std::cout, initres);
    std::cout « "Final residual norm sqrt(r^T r):\n";
    write(std::cout, res);
    std::cout « "IR iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "IR execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
}
```

# Chapter 26

# The kokkos-assembly program

The Kokkos assembly example..

This example depends on simple-solver, poisson-solver, three-pt-stencil-solver, .

## Introduction

This example solves a 1D Poisson equation:

$$u : [0, 1] \to Ru" = fu(0) = u0u(1) = u1$$

using a finite difference method on an equidistant grid with `K` discretization points (`K` can be controlled with a command line parameter).

The resulting CSR matrix is assembled using Kokkos kernels. This example show how to use Ginkgo data with Kokkos kernels.

#### Notes

If this example is built as part of Ginkgo, it is advised to configure Ginkgo with `-DGINKGO_WITH_CCACHE=OFF` to prevent incompabilities with Kokkos' compiler wrapper for `nvcc`.

## The commented program

```
#include <iostream>
#include <string>
#include <Kokkos_Core.hpp>
#include <ginkgo/ginkgo.hpp>
#include <ginkgo/extensions/kokkos.hpp>
namespace gko::ext::kokkos::detail {
/ **
 * Specialization of type mapper for gko::device_matrix_data.
 *
 * @tparam ValueType  The value type of the matrix elements
 * @tparam IndexType   The index type of the matrix elements
 * @tparam MemorySpace  The Kokkos memory space to use.
 * /
template <typename ValueType, typename IndexType, typename MemorySpace>
struct mapper<device_matrix_data<ValueType, IndexType>, MemorySpace> {
    using index_mapper = mapper<array<IndexType>, MemorySpace>;
```

```
    using value_mapper = mapper<array<ValueType>, MemorySpace>;
    / **
     * This struct defines the layout of the device_matrix_data type in terms
     * of arrays.
     *
     * @tparam ValueType_c  The value type of the matrix elements, might have
     *                      other cv qualifiers than ValueType
     * @tparam IndexType_c  The index type of the matrix elements, might have
     *                      other cv qualifiers than IndexType
     * /
    template <typename ValueType_c, typename IndexType_c>
    struct type {
        using index_array = typename index_mapper::template type<IndexType_c>;
        using value_array = typename value_mapper::template type<ValueType_c>;
        / **
         * Constructor based on size and raw pointers
         *
         * @param size  The number of stored elements
         * @param row_idxs  Pointer to the row indices
         * @param col_idxs  Pointer to the column indices
         * @param values  Pointer to the values
         *
         * @return  An object which has each gko::array of the
         *          device_matrix_data mapped to a Kokkos view
         * /
        static type map(size_type size, IndexType_c* row_idxs,
                        IndexType_c* col_idxs, ValueType_c* values)
        {
            return {index_mapper::map(row_idxs, size),
                    index_mapper::map(col_idxs, size),
                    value_mapper::map(values, size)};
        }
        index_array row_idxs;
        index_array col_idxs;
        value_array values;
    };
    static type<ValueType, IndexType> map(
        device_matrix_data<ValueType, IndexType>& md)
    {
        assert_compatibility<MemorySpace>(md);
        return type<ValueType, IndexType>::map(
            md.get_num_stored_elements(), md.get_row_idxs(), md.get_col_idxs(),
            md.get_values());
    }
    static type<const ValueType, const IndexType> map(
        const device_matrix_data<ValueType, IndexType>& md)
    {
        assert_compatibility<MemorySpace>(md);
        return type<const ValueType, const IndexType>::map(
            md.get_num_stored_elements(), md.get_const_row_idxs(),
            md.get_const_col_idxs(), md.get_const_values());
    }
};
}  // namespace gko::ext::kokkos::detail
```

Creates a stencil matrix in CSR format for the given number of discretization points.

```
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(gko::matrix::Csr<ValueType, IndexType>* matrix)
{
    auto exec = matrix->get_executor();
    const auto discretization_points = matrix->get_size()[0];
```

Over-allocate storage for the matrix elements. Each row has 3 entries, except for the first and last one. To handle each row uniformly, we allocate space for 3x the number of rows.

```
gko::device_matrix_data<ValueType, IndexType> md(exec, matrix->get_size(),
                                                 discretization_points * 3);
```

Create Kokkos views on Ginkgo data.

```
auto k_md = gko::ext::kokkos::map_data(md);
```

Create the matrix entries. This also creates zero entries for the first and second row to handle all rows uniformly.

```
Kokkos::parallel_for(
    "generate_stencil_matrix", md.get_num_stored_elements(),
    KOKKOS_LAMBDA(int i) {
        const ValueType coefs[] = {-1, 2, -1};
        auto ofs = static_cast<IndexType>((i % 3) - 1);
        auto row = static_cast<IndexType>(i / 3);
        auto col = row + ofs;
```

To prevent branching, a mask is used to set the entry to zero, if the column is out-of-bounds

```
    auto mask =
        static_cast<IndexType>(0 <= col && col < discretization_points);
```

```
    k_md.row_idxs[i] = mask * row;
    k_md.col_idxs[i] = mask * col;
    k_md.values[i] = mask * coefs[ofs + 1];
});
```

Add up duplicate (zero) entries.

```
md.sum_duplicates();
```

Build Csr matrix.

```
    matrix->read(std::move(md));
}
```

Generates the RHS vector given `f` and the boundary conditions.

```
template <typename Closure, typename ValueType>
void generate_rhs(Closure&& f, ValueType u0, ValueType u1,
                  gko::matrix::Dense<ValueType>* rhs)
{
    const auto discretization_points = rhs->get_size()[0];
    auto k_rhs = gko::ext::kokkos::map_data(rhs);
    Kokkos::parallel_for(
        "generate_rhs", discretization_points, KOKKOS_LAMBDA(int i) {
            const ValueType h = 1.0 / (discretization_points + 1);
            const ValueType xi = ValueType(i + 1) * h;
            k_rhs(i, 0) = -f(xi) * h * h;
            if (i == 0) {
                k_rhs(i, 0) += u0;
            }
            if (i == discretization_points - 1) {
                k_rhs(i, 0) += u1;
            }
        });
}
```

Computes the 1-norm of the error given the computed `u` and the correct solution function `correct_u`.

```
template <typename Closure, typename ValueType>
double calculate_error(int discretization_points,
                       const gko::matrix::Dense<ValueType>* u,
                       Closure&& correct_u)
{
    auto k_u = gko::ext::kokkos::map_data(u);
    auto error = 0.0;
    Kokkos::parallel_reduce(
        "calculate_error", discretization_points,
        KOKKOS_LAMBDA(int i, double& lsum) {
            const auto h = 1.0 / (discretization_points + 1);
            const auto xi = (i + 1) * h;
            lsum += Kokkos::abs((k_u(i, 0) - correct_u(xi)) /
                                Kokkos::abs(correct_u(xi)));
        },
        error);
    return error;
}
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType>;
```

Print help message. For details on the kokkos-options see https://kokkos.github.io/kokkos-core-wiki/↩ProgrammingGuide/Initialization.html#initialization-by-command-line-arguments

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0]
              « " [discretization_points] [kokkos-options]" « std::endl;
    Kokkos::ScopeGuard kokkos(argc, argv);  // print Kokkos help
    std::exit(1);
}
Kokkos::ScopeGuard kokkos(argc, argv);
const auto discretization_points =
    static_cast<gko::size_type>(argc >= 2 ?  std::atoi(argv[1]) :  100u);
```

chooses the executor that corresponds to the Kokkos DefaultExecutionSpace

```
auto exec = gko::ext::kokkos::create_default_executor();
```

problem:

```
auto correct_u = [] KOKKOS_FUNCTION(ValueType x) { return x * x * x; };
auto f = [] KOKKOS_FUNCTION(ValueType x) { return ValueType{6} * x; };
auto u0 = correct_u(0);
auto u1 = correct_u(1);
```

initialize vectors

```
auto rhs = vec::create(exec, gko::dim<2>(discretization_points, 1));
generate_rhs(f, u0, u1, rhs.get());
auto u = vec::create(exec, gko::dim<2>(discretization_points, 1));
u->fill(0.0);
```

initialize the stencil matrix

```
auto A = share(mtx::create(
    exec, gko::dim<2>{discretization_points, discretization_points}));
generate_stencil_matrix(A.get());
const RealValueType reduction_factor{1e-7};
```

Generate solver and solve the system

```
    cg::build()
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(discretization_points),
            gko::stop::ResidualNorm<ValueType>::build().with_reduction_factor(
                reduction_factor))
        .with_preconditioner(bj::build())
        .on(exec)
        ->generate(A)
        ->apply(rhs, u);
    std::cout « "\nSolve complete."
              « "\nThe average relative error is "
              « calculate_error(discretization_points, u.get(), correct_u) /
                    discretization_points
              « std::endl;
}
```

# Results

Example output:

```
> ./kokkos-assembly
Solve complete.
The average relative error is 1.05488e-11
```

The actual error depends on the used hardware.

# The plain program

```
#include <iostream>
#include <string>
#include <Kokkos_Core.hpp>
#include <ginkgo/ginkgo.hpp>
#include <ginkgo/extensions/kokkos.hpp>
namespace gko::ext::kokkos::detail {
template <typename ValueType, typename IndexType, typename MemorySpace>
struct mapper<device_matrix_data<ValueType, IndexType>, MemorySpace> {
    using index_mapper = mapper<array<IndexType>, MemorySpace>;
    using value_mapper = mapper<array<ValueType>, MemorySpace>;
    template <typename ValueType_c, typename IndexType_c>
    struct type {
        using index_array = typename index_mapper::template type<IndexType_c>;
        using value_array = typename value_mapper::template type<ValueType_c>;
        static type map(size_type size, IndexType_c* row_idxs,
                        IndexType_c* col_idxs, ValueType_c* values)
        {
            return {index_mapper::map(row_idxs, size),
                    index_mapper::map(col_idxs, size),
                    value_mapper::map(values, size)};
        }
        index_array row_idxs;
        index_array col_idxs;
        value_array values;
    };
    static type<ValueType, IndexType> map(
        device_matrix_data<ValueType, IndexType>& md)
    {
```

```
        assert_compatibility<MemorySpace>(md);
        return type<ValueType, IndexType>::map(
            md.get_num_stored_elements(), md.get_row_idxs(), md.get_col_idxs(),
            md.get_values());
    }
    static type<const ValueType, const IndexType> map(
        const device_matrix_data<ValueType, IndexType>& md)
    {
        assert_compatibility<MemorySpace>(md);
        return type<const ValueType, const IndexType>::map(
            md.get_num_stored_elements(), md.get_const_row_idxs(),
            md.get_const_col_idxs(), md.get_const_values());
    }
};
}  // namespace gko::ext::kokkos::detail
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(gko::matrix::Csr<ValueType, IndexType>* matrix)
{
    auto exec = matrix->get_executor();
    const auto discretization_points = matrix->get_size()[0];
    gko::device_matrix_data<ValueType, IndexType> md(exec, matrix->get_size(),
                                                     discretization_points * 3);
    auto k_md = gko::ext::kokkos::map_data(md);
    Kokkos::parallel_for(
        "generate_stencil_matrix", md.get_num_stored_elements(),
        KOKKOS_LAMBDA(int i) {
            const ValueType coefs[] = {-1, 2, -1};
            auto ofs = static_cast<IndexType>((i % 3) - 1);
            auto row = static_cast<IndexType>(i / 3);
            auto col = row + ofs;
            auto mask =
                static_cast<IndexType>(0 <= col && col < discretization_points);
            k_md.row_idxs[i] = mask * row;
            k_md.col_idxs[i] = mask * col;
            k_md.values[i] = mask * coefs[ofs + 1];
        });
    md.sum_duplicates();
    matrix->read(std::move(md));
}
template <typename Closure, typename ValueType>
void generate_rhs(Closure&& f, ValueType u0, ValueType u1,
                  gko::matrix::Dense<ValueType>* rhs)
{
    const auto discretization_points = rhs->get_size()[0];
    auto k_rhs = gko::ext::kokkos::map_data(rhs);
    Kokkos::parallel_for(
        "generate_rhs", discretization_points, KOKKOS_LAMBDA(int i) {
            const ValueType h = 1.0 / (discretization_points + 1);
            const ValueType xi = ValueType(i + 1) * h;
            k_rhs(i, 0) = -f(xi) * h * h;
            if (i == 0) {
                k_rhs(i, 0) += u0;
            }
            if (i == discretization_points - 1) {
                k_rhs(i, 0) += u1;
            }
        });
}
template <typename Closure, typename ValueType>
double calculate_error(int discretization_points,
                       const gko::matrix::Dense<ValueType>* u,
                       Closure&& correct_u)
{
    auto k_u = gko::ext::kokkos::map_data(u);
    auto error = 0.0;
    Kokkos::parallel_reduce(
        "calculate_error", discretization_points,
        KOKKOS_LAMBDA(int i, double& lsum) {
            const auto h = 1.0 / (discretization_points + 1);
            const auto xi = (i + 1) * h;
            lsum += Kokkos::abs((k_u(i, 0) - correct_u(xi)) /
                                Kokkos::abs(correct_u(xi)));
        },
        error);
    return error;
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using bj = gko::preconditioner::Jacobi<ValueType>;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0]
```

```
                  « " [discretization_points] [kokkos-options]" « std::endl;
        Kokkos::ScopeGuard kokkos(argc, argv);  // print Kokkos help
        std::exit(1);
    }
    Kokkos::ScopeGuard kokkos(argc, argv);
    const auto discretization_points =
        static_cast<gko::size_type>(argc >= 2 ?  std::atoi(argv[1]) :  100u);
    auto exec = gko::ext::kokkos::create_default_executor();
    auto correct_u = [] KOKKOS_FUNCTION(ValueType x) { return x * x * x; };
    auto f = [] KOKKOS_FUNCTION(ValueType x) { return ValueType{6} * x; };
    auto u0 = correct_u(0);
    auto u1 = correct_u(1);
    auto rhs = vec::create(exec, gko::dim<2>(discretization_points, 1));
    generate_rhs(f, u0, u1, rhs.get());
    auto u = vec::create(exec, gko::dim<2>(discretization_points, 1));
    u->fill(0.0);
    auto A = share(mtx::create(
        exec, gko::dim<2>{discretization_points, discretization_points}));
    generate_stencil_matrix(A.get());
    const RealValueType reduction_factor{1e-7};
    cg::build()
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(discretization_points),
            gko::stop::ResidualNorm<ValueType>::build().with_reduction_factor(
                reduction_factor))
        .with_preconditioner(bj::build())
        .on(exec)
        ->generate(A)
        ->apply(rhs, u);
    std::cout « "\nSolve complete."
              « "\nThe average relative error is "
              « calculate_error(discretization_points, u.get(), correct_u) /
                    discretization_points
              « std::endl;
}
```

# Chapter 27

# The minimal-cuda-solver program

The minimal CUDA solver example..

This example depends on simple-solver.

## Introduction

This is a minimal example that solves a system with Ginkgo. The matrix, right hand side and initial guess are read from standard input, and the result is written to standard output. The system matrix is stored in CSR format, and the system solved using the CG method, preconditioned with the block-Jacobi preconditioner. All computations are done on the GPU.

The easiest way to use the example data from the data/ folder is to concatenate the matrix, the right hand side and the initial solution (in that exact order), and pipe the result to the minimal_solver_cuda executable:

```
cat data/A.mtx data/b.mtx data/x0.mtx | ./minimal-cuda-solver
```

#### About the example

## The commented program

```
#include <iostream>
#include <ginkgo/ginkgo.hpp>
int main()
{
```

Instantiate a CUDA executor

```
auto gpu = gko::CudaExecutor::create(0, gko::OmpExecutor::create());
```

Read data

```
auto A = gko::read<gko::matrix::Csr<>>(std::cin, gpu);
auto b = gko::read<gko::matrix::Dense<>>(std::cin, gpu);
auto x = gko::read<gko::matrix::Dense<>>(std::cin, gpu);
```

Create the solver

```
auto solver =
    gko::solver::Cg<>::build()
        .with_preconditioner(gko::preconditioner::Jacobi<>::build())
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(20u),
            gko::stop::ResidualNorm<>::build().with_reduction_factor(1e-15))
        .on(gpu);
```

Solve system

```
solver->generate(give(A))->apply(b, x);
```

Write result

```
    write(std::cout, x);
}
```

## Results

The following is the expected result when using the data contained in the folder `data` as input:

```
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
```

### Comments about programming and debugging

## The plain program

```
#include <iostream>
#include <ginkgo/ginkgo.hpp>
int main()
{
    auto gpu = gko::CudaExecutor::create(0, gko::OmpExecutor::create());
    auto A = gko::read<gko::matrix::Csr<>>(std::cin, gpu);
    auto b = gko::read<gko::matrix::Dense<>>(std::cin, gpu);
    auto x = gko::read<gko::matrix::Dense<>>(std::cin, gpu);
    auto solver =
        gko::solver::Cg<>::build()
            .with_preconditioner(gko::preconditioner::Jacobi<>::build())
            .with_criteria(
                gko::stop::Iteration::build().with_max_iters(20u),
                gko::stop::ResidualNorm<>::build().with_reduction_factor(1e-15))
            .on(gpu);
    solver->generate(give(A))->apply(b, x);
    write(std::cout, x);
}
```

# Chapter 28

# The mixed-multigrid-preconditioned-solver program

The preconditioned solver example..

This example depends on multigrid-preconditioned-solver, mixed-multigrid-solver.

## This example shows how to use the mixed-precision multigrid preconditioner.

**In this example, we first read in a matrix from a file. The preconditioned CG solver is enhanced with a mixed-precision multigrid preconditioner. The example features the generating time and runtime of the CG solver.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using MixedType = float;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using ir = gko::solver::Ir<ValueType>;
using mg = gko::solver::Multigrid;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
using ir_f = gko::solver::Ir<MixedType>;
using bj_f = gko::preconditioner::Jacobi<MixedType, IndexType>;
using pgm_f = gko::multigrid::Pgm<MixedType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
```

```
        [] {
            return gko::CudaExecutor::create(0,
                                             gko::OmpExecutor::create());
        }},
        {"hip",
        [] {
            return gko::HipExecutor::create(0, gko::OmpExecutor::create());
        }},
        {"dpcpp",
        [] {
            return gko::DpcppExecutor::create(
                0, gko::ReferenceExecutor::create());
        }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

### executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
const int mixed_int = argc >= 3 ?  std::atoi(argv[2]) :  1;
const bool use_mixed = mixed_int != 0;  // nonzero uses mixed
std::cout « "Using mixed precision?  " « use_mixed « std::endl;
```

### Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

### Create RHS as 1 and initial guess as 0

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 0.;
    host_b->at(i, 0) = 1.;
}
auto x = vec::create(exec);
auto b = vec::create(exec);
x->copy_from(host_x);
b->copy_from(host_b);
```

### Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

### copy b again

```
b->copy_from(host_b);
```

### Prepare the stopping criteria

```
const gko::remove_complex<ValueType> tolerance = 1e-8;
auto iter_stop =
    gko::share(gko::stop::Iteration::build().with_max_iters(100u).on(exec));
auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::absolute)
                               .with_reduction_factor(tolerance)
                               .on(exec));
std::shared_ptr<const gko::log::Convergence<ValueType» logger =
    gko::log::Convergence<ValueType>::create();
iter_stop->add_logger(logger);
tol_stop->add_logger(logger);
```

### Create smoother factory (ir with bj)

```
auto inner_solver_gen =
    gko::share(bj::build().with_max_block_size(1u).on(exec));
auto inner_solver_gen_f =
    gko::share(bj_f::build().with_max_block_size(1u).on(exec));
auto smoother_gen = gko::share(
    ir::build()
        .with_solver(inner_solver_gen)
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto smoother_gen_f = gko::share(
    ir_f::build()
        .with_solver(inner_solver_gen_f)
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
```

### Create MultigridLevel factory

```
auto mg_level_gen =
```

```
    gko::share(pgm::build().with_deterministic(true).on(exec));
auto mg_level_gen_f =
    gko::share(pgm_f::build().with_deterministic(true).on(exec));
```

### Create CoarsestSolver factory

```
auto coarsest_gen = gko::share(
    ir::build()
        .with_solver(inner_solver_gen)
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
auto coarsest_gen_f = gko::share(
    ir_f::build()
        .with_solver(inner_solver_gen_f)
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
```

### Create multigrid factory

```
std::shared_ptr<gko::LinOpFactory> multigrid_gen;
if (use_mixed) {
    multigrid_gen =
        mg::build()
            .with_max_levels(10u)
            .with_min_coarse_rows(2u)
            .with_pre_smoother(smoother_gen, smoother_gen_f)
            .with_post_uses_pre(true)
            .with_mg_level(mg_level_gen, mg_level_gen_f)
            .with_level_selector([](const gko::size_type level,
                                    const gko::LinOp*) -> gko::size_type {
```

The first (index 0) level will use the first mg_level_gen, smoother_gen which are the factories with ValueType. The rest of levels (>= 1) will use the second (index 1) mg_level_gen2 and smoother_gen2 which use the MixedType. The rest of levels will use different type than the normal multigrid.

```
                return level >= 1 ?  1 :  0;
            })
            .with_coarsest_solver(coarsest_gen_f)
            .with_default_initial_guess(
                gko::solver::initial_guess_mode::zero)
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
            .on(exec);
} else {
    multigrid_gen =
        mg::build()
            .with_max_levels(10u)
            .with_min_coarse_rows(2u)
            .with_pre_smoother(smoother_gen)
            .with_post_uses_pre(true)
            .with_mg_level(mg_level_gen)
            .with_coarsest_solver(coarsest_gen)
            .with_default_initial_guess(
                gko::solver::initial_guess_mode::zero)
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
            .on(exec);
}
```

### Create solver factory

```
auto solver_gen = cg::build()
                      .with_criteria(iter_stop, tol_stop)
                      .with_preconditioner(multigrid_gen)
                      .on(exec);
```

### Create solver

```
std::chrono::nanoseconds gen_time(0);
auto gen_tic = std::chrono::steady_clock::now();
auto solver = solver_gen->generate(A);
exec->synchronize();
auto gen_toc = std::chrono::steady_clock::now();
gen_time +=
    std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
```

### Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
exec->synchronize();
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

Calculate residual

```
auto res = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(res);
std::cout « "Initial residual norm sqrt(r^T r):  \n";
write(std::cout, initres);
std::cout « "Final residual norm sqrt(r^T r):  \n";
write(std::cout, res);
```

Print solver statistics

```
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
4.3589
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
1.69858e-09
CG iteration count:          39
CG generation time [ms]:  2.04293
CG execution time [ms]:  22.3874
CG execution time per iteration[ms]:  0.574036
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using MixedType = float;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using ir = gko::solver::Ir<ValueType>;
    using mg = gko::solver::Multigrid;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
    using ir_f = gko::solver::Ir<MixedType>;
    using bj_f = gko::preconditioner::Jacobi<MixedType, IndexType>;
    using pgm_f = gko::multigrid::Pgm<MixedType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
```

```
          [] {
              return gko::HipExecutor::create(0, gko::OmpExecutor::create());
          }},
         {"dpcpp",
          [] {
              return gko::DpcppExecutor::create(
                  0, gko::ReferenceExecutor::create());
          }},
         {"reference", [] { return gko::ReferenceExecutor::create(); }}};
const auto exec = exec_map.at(executor_string)();  // throws if not valid
const int mixed_int = argc >= 3 ?  std::atoi(argv[2]) :  1;
const bool use_mixed = mixed_int != 0;  // nonzero uses mixed
std::cout « "Using mixed precision?  " « use_mixed « std::endl;
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 0.;
    host_b->at(i, 0) = 1.;
}
auto x = vec::create(exec);
auto b = vec::create(exec);
x->copy_from(host_x);
b->copy_from(host_b);
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
b->copy_from(host_b);
const gko::remove_complex<ValueType> tolerance = 1e-8;
auto iter_stop =
    gko::share(gko::stop::Iteration::build().with_max_iters(100u).on(exec));
auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::absolute)
                               .with_reduction_factor(tolerance)
                               .on(exec));
std::shared_ptr<const gko::log::Convergence<ValueType» logger =
    gko::log::Convergence<ValueType>::create();
iter_stop->add_logger(logger);
tol_stop->add_logger(logger);
auto inner_solver_gen =
    gko::share(bj::build().with_max_block_size(1u).on(exec));
auto inner_solver_gen_f =
    gko::share(bj_f::build().with_max_block_size(1u).on(exec));
auto smoother_gen = gko::share(
    ir::build()
        .with_solver(inner_solver_gen)
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto smoother_gen_f = gko::share(
    ir_f::build()
        .with_solver(inner_solver_gen_f)
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto mg_level_gen =
    gko::share(pgm::build().with_deterministic(true).on(exec));
auto mg_level_gen_f =
    gko::share(pgm_f::build().with_deterministic(true).on(exec));
auto coarsest_gen = gko::share(
    ir::build()
        .with_solver(inner_solver_gen)
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
auto coarsest_gen_f = gko::share(
    ir_f::build()
        .with_solver(inner_solver_gen_f)
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
std::shared_ptr<gko::LinOpFactory> multigrid_gen;
if (use_mixed) {
    multigrid_gen =
        mg::build()
            .with_max_levels(10u)
            .with_min_coarse_rows(2u)
            .with_pre_smoother(smoother_gen, smoother_gen_f)
            .with_post_uses_pre(true)
            .with_mg_level(mg_level_gen, mg_level_gen_f)
            .with_level_selector([](const gko::size_type level,
                                    const gko::LinOp*) -> gko::size_type {
                return level >= 1 ?  1 :  0;
            })
```

```
                .with_coarsest_solver(coarsest_gen_f)
                .with_default_initial_guess(
                    gko::solver::initial_guess_mode::zero)
                .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
                .on(exec);
    } else {
        multigrid_gen =
            mg::build()
                .with_max_levels(10u)
                .with_min_coarse_rows(2u)
                .with_pre_smoother(smoother_gen)
                .with_post_uses_pre(true)
                .with_mg_level(mg_level_gen)
                .with_coarsest_solver(coarsest_gen)
                .with_default_initial_guess(
                    gko::solver::initial_guess_mode::zero)
                .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
                .on(exec);
    }
    auto solver_gen = cg::build()
                          .with_criteria(iter_stop, tol_stop)
                          .with_preconditioner(multigrid_gen)
                          .on(exec);
    std::chrono::nanoseconds gen_time(0);
    auto gen_tic = std::chrono::steady_clock::now();
    auto solver = solver_gen->generate(A);
    exec->synchronize();
    auto gen_toc = std::chrono::steady_clock::now();
    gen_time +=
        std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
    exec->synchronize();
    std::chrono::nanoseconds time(0);
    auto tic = std::chrono::steady_clock::now();
    solver->apply(b, x);
    exec->synchronize();
    auto toc = std::chrono::steady_clock::now();
    time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
    auto res = gko::initialize<vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Initial residual norm sqrt(r^T r):  \n";
    write(std::cout, initres);
    std::cout « "Final residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Chapter 29

# The mixed-multigrid-solver program

The mixed multigrid solver example..

This example depends on simple-solver.

## This example shows how to use the mixed-precision multigrid solver.

**In this example, we first read in a matrix from a file, then generate a right-hand side and an initial guess. The multigrid solver can mix different precision of MultigridLevel. The example features the generating time and runtime of the multigrid solver.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using MixedType = float;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using ir = gko::solver::Ir<ValueType>;
using ir2 = gko::solver::Ir<MixedType>;
using mg = gko::solver::Multigrid;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
using bj2 = gko::preconditioner::Jacobi<MixedType, IndexType>;
using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
using pgm2 = gko::multigrid::Pgm<MixedType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
```

```
        }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(
                 0, gko::ReferenceExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

### executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
const int mixed_int = argc >= 3 ?  std::atoi(argv[2]) :  1;
const bool use_mixed = mixed_int != 0;  // nonzero uses mixed
std::cout « "Using mixed precision?  " « use_mixed « std::endl;
```

### Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

### Create RHS as 1 and initial guess as 0

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 0.;
    host_b->at(i, 0) = 1.;
}
auto x = vec::create(exec);
auto b = vec::create(exec);
x->copy_from(host_x);
b->copy_from(host_b);
```

### Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

### copy b again

```
b->copy_from(host_b);
```

### Prepare the stopping criteria

```
const gko::remove_complex<ValueType> tolerance = 1e-12;
auto iter_stop =
    gko::share(gko::stop::Iteration::build().with_max_iters(100u).on(exec));
auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::absolute)
                               .with_reduction_factor(tolerance)
                               .on(exec));
```

### Create smoother factory (ir with bj)

```
auto smoother_gen = gko::share(
    ir::build()
        .with_solver(bj::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto smoother_gen2 = gko::share(
    ir2::build()
        .with_solver(bj2::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
```

### Create RestrictProlong factory

```
auto mg_level_gen =
    gko::share(pgm::build().with_deterministic(true).on(exec));
auto mg_level_gen2 =
    gko::share(pgm2::build().with_deterministic(true).on(exec));
```

### Create CoarsesSolver factory

```
auto coarsest_solver_gen = gko::share(
    ir::build()
        .with_solver(bj::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
```

```
        .on(exec));
auto coarsest_solver_gen2 = gko::share(
    ir2::build()
        .with_solver(bj2::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
```

### Create multigrid factory

```
std::shared_ptr<gko::LinOpFactory> multigrid_gen;
if (use_mixed) {
    multigrid_gen =
        mg::build()
            .with_max_levels(10u)
            .with_min_coarse_rows(2u)
            .with_pre_smoother(smoother_gen, smoother_gen2)
            .with_post_uses_pre(true)
            .with_mg_level(mg_level_gen, mg_level_gen2)
            .with_level_selector([](const gko::size_type level,
                                    const gko::LinOp*) -> gko::size_type {
```

The first (index 0) level will use the first mg_level_gen, smoother_gen which are the factories with ValueType. The rest of levels (>= 1) will use the second (index 1) mg_level_gen2 and smoother_gen2 which use the MixedType. The rest of levels will use different type than the normal multigrid.

```
                return level >= 1 ?  1 :  0;
            })
            .with_coarsest_solver(coarsest_solver_gen2)
            .with_criteria(iter_stop, tol_stop)
            .on(exec);
} else {
    multigrid_gen = mg::build()
                        .with_max_levels(10u)
                        .with_min_coarse_rows(2u)
                        .with_pre_smoother(smoother_gen)
                        .with_post_uses_pre(true)
                        .with_mg_level(mg_level_gen)
                        .with_coarsest_solver(coarsest_solver_gen)
                        .with_criteria(iter_stop, tol_stop)
                        .on(exec);
}
std::chrono::nanoseconds gen_time(0);
auto gen_tic = std::chrono::steady_clock::now();
auto solver = multigrid_gen->generate(A);
exec->synchronize();
auto gen_toc = std::chrono::steady_clock::now();
gen_time +=
    std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
```

### Add logger

```
std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
    gko::log::Convergence<ValueType>::create();
solver->add_logger(logger);
```

### Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
exec->synchronize();
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

### Calculate residual explicitly, because the residual is not available inside of the multigrid solver

```
auto res = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(res);
std::cout << "Initial residual norm sqrt(r^T r):  \n";
write(std::cout, initres);
std::cout << "Final residual norm sqrt(r^T r):  \n";
write(std::cout, res);
```

### Print solver statistics

```
    std::cout << "Multigrid iteration count:          "
              << logger->get_num_iterations() << std::endl;
    std::cout << "Multigrid generation time [ms]:  "
              << static_cast<double>(gen_time.count()) / 1000000.0 << std::endl;
    std::cout << "Multigrid execution time [ms]:  "
              << static_cast<double>(time.count()) / 1000000.0 << std::endl;
    std::cout << "Multigrid execution time per iteration[ms]:  "
              << static_cast<double>(time.count()) / 1000000.0 /
                     logger->get_num_iterations()
              << std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
4.3589
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
6.31088e-14
Multigrid iteration count:          9
Multigrid generation time [ms]:  3.35361
Multigrid execution time [ms]:  10.048
Multigrid execution time per iteration[ms]:  1.11644
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using MixedType = float;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using ir = gko::solver::Ir<ValueType>;
    using ir2 = gko::solver::Ir<MixedType>;
    using mg = gko::solver::Multigrid;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    using bj2 = gko::preconditioner::Jacobi<MixedType, IndexType>;
    using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
    using pgm2 = gko::multigrid::Pgm<MixedType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(
                     0, gko::ReferenceExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    const int mixed_int = argc >= 3 ?  std::atoi(argv[2]) :  1;
    const bool use_mixed = mixed_int != 0;  // nonzero uses mixed
    std::cout « "Using mixed precision?  " « use_mixed « std::endl;
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    gko::size_type size = A->get_size()[0];
    auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    for (auto i = 0; i < size; i++) {
        host_x->at(i, 0) = 0.;
        host_b->at(i, 0) = 1.;
    }
    auto x = vec::create(exec);
    auto b = vec::create(exec);
    x->copy_from(host_x);
    b->copy_from(host_b);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto initres = gko::initialize<vec>({0.0}, exec);
```

```
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
b->copy_from(host_b);
const gko::remove_complex<ValueType> tolerance = 1e-12;
auto iter_stop =
    gko::share(gko::stop::Iteration::build().with_max_iters(100u).on(exec));
auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::absolute)
                               .with_reduction_factor(tolerance)
                               .on(exec));
auto smoother_gen = gko::share(
    ir::build()
        .with_solver(bj::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto smoother_gen2 = gko::share(
    ir2::build()
        .with_solver(bj2::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec));
auto mg_level_gen =
    gko::share(pgm::build().with_deterministic(true).on(exec));
auto mg_level_gen2 =
    gko::share(pgm2::build().with_deterministic(true).on(exec));
auto coarsest_solver_gen = gko::share(
    ir::build()
        .with_solver(bj::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<ValueType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
auto coarsest_solver_gen2 = gko::share(
    ir2::build()
        .with_solver(bj2::build().with_max_block_size(1u))
        .with_relaxation_factor(static_cast<MixedType>(0.9))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(4u))
        .on(exec));
std::shared_ptr<gko::LinOpFactory> multigrid_gen;
if (use_mixed) {
    multigrid_gen =
        mg::build()
            .with_max_levels(10u)
            .with_min_coarse_rows(2u)
            .with_pre_smoother(smoother_gen, smoother_gen2)
            .with_post_uses_pre(true)
            .with_mg_level(mg_level_gen, mg_level_gen2)
            .with_level_selector([](const gko::size_type level,
                                    const gko::LinOp*) -> gko::size_type {
                return level >= 1 ?  1 :  0;
            })
            .with_coarsest_solver(coarsest_solver_gen2)
            .with_criteria(iter_stop, tol_stop)
            .on(exec);
} else {
    multigrid_gen = mg::build()
                        .with_max_levels(10u)
                        .with_min_coarse_rows(2u)
                        .with_pre_smoother(smoother_gen)
                        .with_post_uses_pre(true)
                        .with_mg_level(mg_level_gen)
                        .with_coarsest_solver(coarsest_solver_gen)
                        .with_criteria(iter_stop, tol_stop)
                        .on(exec);
}
std::chrono::nanoseconds gen_time(0);
auto gen_tic = std::chrono::steady_clock::now();
auto solver = multigrid_gen->generate(A);
exec->synchronize();
auto gen_toc = std::chrono::steady_clock::now();
gen_time +=
    std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
std::shared_ptr<const gko::log::Convergence<ValueType» logger =
    gko::log::Convergence<ValueType>::create();
solver->add_logger(logger);
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
exec->synchronize();
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
auto res = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(res);
std::cout « "Initial residual norm sqrt(r^T r):  \n";
write(std::cout, initres);
```

```
    std::cout « "Final residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
    std::cout « "Multigrid iteration count:          "
              « logger->get_num_iterations() « std::endl;
    std::cout « "Multigrid generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "Multigrid execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "Multigrid execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Chapter 30

# The mixed-precision-ir program

The Mixed Precision Iterative Refinement (MPIR) solver example..

This example depends on iterative-refinement.

## This example manually implements a Mixed Precision Iterative Refinement (MPIR) solver.

**In this example, we first read in a matrix from file, then generate a right-hand side and an initial guess. An inaccurate CG solver in single precision is used as the inner solver to an iterative refinement (IR) in double precision method which solves a linear system.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using SolverType = float;
using RealSolverType = gko::remove_complex<SolverType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using solver_vec = gko::matrix::Dense<SolverType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using solver_mtx = gko::matrix::Csr<SolverType, IndexType>;
using cg = gko::solver::Cg<SolverType>;
gko::size_type max_outer_iters = 100u;
gko::size_type max_inner_iters = 100u;
RealValueType outer_reduction_factor{1e-12};
RealSolverType inner_reduction_factor{1e-2};
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
```

```
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(
                 0, gko::ReferenceExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

### executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

### Create RHS and initial guess as 1

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 1.;
}
auto x = gko::clone(exec, host_x);
auto b = gko::clone(exec, host_x);
```

### Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres_vec = gko::initialize<real_vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres_vec);
```

### Build lower-precision system matrix and residual

```
auto solver_A = solver_mtx::create(exec);
auto inner_residual = solver_vec::create(exec);
auto outer_residual = vec::create(exec);
A->convert_to(solver_A);
b->convert_to(outer_residual);
```

### restore b

```
b->copy_from(host_x);
```

### Create inner solver

```
auto inner_solver =
    cg::build()
        .with_criteria(
            gko::stop::ResidualNorm<SolverType>::build()
                .with_reduction_factor(inner_reduction_factor),
            gko::stop::Iteration::build().with_max_iters(max_inner_iters))
        .on(exec)
        ->generate(give(solver_A));
```

### Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto res_vec = gko::initialize<real_vec>({0.0}, exec);
auto initres = exec->copy_val_to_host(initres_vec->get_const_values());
auto inner_solution = solver_vec::create(exec);
auto outer_delta = vec::create(exec);
auto tic = std::chrono::steady_clock::now();
int iter = -1;
while (true) {
    ++iter;
```

### convert residual to inner precision

```
outer_residual->convert_to(inner_residual);
outer_residual->compute_norm2(res_vec);
auto res = exec->copy_val_to_host(res_vec->get_const_values());
```

break if we exceed the number of iterations or have converged

```
if (iter > max_outer_iters || res / initres < outer_reduction_factor) {
    break;
}
```

Use the inner solver to solve A ∗ inner_solution = inner_residual with residual as initial guess.

```
inner_solution->copy_from(inner_residual);
inner_solver->apply(inner_residual, inner_solution);
```

convert inner solution to outer precision

```
inner_solution->convert_to(outer_delta);
```

x = x + inner_solution

```
x->add_scaled(one, outer_delta);
```

residual = b - A ∗ x

```
    outer_residual->copy_from(b);
    A->apply(neg_one, x, one, outer_residual);
}
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

Calculate residual

```
A->apply(one, x, neg_one, b);
b->compute_norm2(res_vec);
std::cout « "Initial residual norm sqrt(r^T r):\n";
write(std::cout, initres_vec);
std::cout « "Final residual norm sqrt(r^T r):\n";
write(std::cout, res_vec);
```

Print solver statistics

```
    std::cout « "MPIR iteration count:          " « iter « std::endl;
    std::cout « "MPIR execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
194.679
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
1.22728e-10
MPIR iteration count:          25
MPIR execution time [ms]:  0.846559
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using SolverType = float;
    using RealSolverType = gko::remove_complex<SolverType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using solver_vec = gko::matrix::Dense<SolverType>;
```

```
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using solver_mtx = gko::matrix::Csr<SolverType, IndexType>;
using cg = gko::solver::Cg<SolverType>;
gko::size_type max_outer_iters = 100u;
gko::size_type max_inner_iters = 100u;
RealValueType outer_reduction_factor{1e-12};
RealSolverType inner_reduction_factor{1e-2};
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(
                 0, gko::ReferenceExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
const auto exec = exec_map.at(executor_string)();  // throws if not valid
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 1.;
}
auto x = gko::clone(exec, host_x);
auto b = gko::clone(exec, host_x);
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres_vec = gko::initialize<real_vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres_vec);
auto solver_A = solver_mtx::create(exec);
auto inner_residual = solver_vec::create(exec);
auto outer_residual = vec::create(exec);
A->convert_to(solver_A);
b->convert_to(outer_residual);
b->copy_from(host_x);
auto inner_solver =
    cg::build()
        .with_criteria(
            gko::stop::ResidualNorm<SolverType>::build()
                .with_reduction_factor(inner_reduction_factor),
            gko::stop::Iteration::build().with_max_iters(max_inner_iters))
        .on(exec)
        ->generate(give(solver_A));
exec->synchronize();
std::chrono::nanoseconds time(0);
auto res_vec = gko::initialize<real_vec>({0.0}, exec);
auto initres = exec->copy_val_to_host(initres_vec->get_const_values());
auto inner_solution = solver_vec::create(exec);
auto outer_delta = vec::create(exec);
auto tic = std::chrono::steady_clock::now();
int iter = -1;
while (true) {
    ++iter;
    outer_residual->convert_to(inner_residual);
    outer_residual->compute_norm2(res_vec);
    auto res = exec->copy_val_to_host(res_vec->get_const_values());
    if (iter > max_outer_iters || res / initres < outer_reduction_factor) {
        break;
    }
    inner_solution->copy_from(inner_residual);
    inner_solver->apply(inner_residual, inner_solution);
    inner_solution->convert_to(outer_delta);
    x->add_scaled(one, outer_delta);
    outer_residual->copy_from(b);
    A->apply(neg_one, x, one, outer_residual);
}
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
A->apply(one, x, neg_one, b);
b->compute_norm2(res_vec);
std::cout « "Initial residual norm sqrt(r^T r):\n";
```

```
    write(std::cout, initres_vec);
    std::cout « "Final residual norm sqrt(r^T r):\n";
    write(std::cout, res_vec);
    std::cout « "MPIR iteration count:          " « iter « std::endl;
    std::cout « "MPIR execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
}
```

# Chapter 31

# The mixed-spmv program

The mixed spmv example..

## Introduction

This mixed spmv example should give the usage of Ginkgo mixed precision. This example is meant for you to understand how Ginkgo works with different precision of data. We encourage you to play with the code, change the parameters and see what is best suited for your purposes.

#### About the example

Each example has the following sections:

1. **Introduction:**This gives an overview of the example and mentions any interesting aspects in the example that might help the reader.
2. **The commented program:** This section is intended for you to understand the details of the example so that you can play with it and understand Ginkgo and its features better.
3. **Results:** This section shows the results of the code when run. Though the results may not be completely the same, you can expect the behaviour to be similar.
4. **The plain program:** This is the complete code without any comments to have an complete overview of the code.

## The commented program

#### Include files

This is the main ginkgo header file.
```
#include <ginkgo/ginkgo.hpp>
```

Add the fstream header to read from data from files.
```
#include <fstream>
```

Add the C++ iostream header to output information to the console.

```
#include <iostream>
```

Add the STL map header for the executor selection.

```
#include <map>
```

Add the string manipulation header to handle strings.

```
#include <string>
```

Add the timing header for timing.

```
#include <chrono>
```

Add the random header to generate random vectors.

```
#include <random>
namespace {
/ **
 * Generate a random value.
 *
 * @tparam ValueType  valuetype of the value
 * @tparam ValueDistribution  type of value distribution
 * @tparam Engine  type of random engine
 *
 * @param value_dist  distribution of array values
 * @param engine  a random engine
 *
 * @return ValueType
 * /
template <typename ValueType, typename ValueDistribution, typename Engine>
typename std::enable_if<!gko::is_complex_s<ValueType>::value, ValueType>::type
get_rand_value(ValueDistribution&& value_dist, Engine&& gen)
{
    return value_dist(gen);
}
/ **
 * Specialization for complex types.
 *
 * @copydoc get_rand_value
 * /
template <typename ValueType, typename ValueDistribution, typename Engine>
typename std::enable_if<gko::is_complex_s<ValueType>::value, ValueType>::type
get_rand_value(ValueDistribution&& value_dist, Engine&& gen)
{
    return ValueType(value_dist(gen), value_dist(gen));
}
/ **
 * timing the apply operation A->apply(b, x).  It will runs 2 warmup and get
 * average time among 10 times.
 *
 * @return seconds
 * /
template <typename ValueTypeIn, typename ValueTypeOut>
double timing(std::shared_ptr<const gko::Executor> exec,
              std::shared_ptr<const gko::LinOp> A,
              std::shared_ptr<gko::matrix::Dense<ValueTypeIn>> b,
              std::shared_ptr<gko::matrix::Dense<ValueTypeOut>> x)
{
    int warmup = 2;
    int rep = 10;
    for (int i = 0; i < warmup; i++) {
        A->apply(b, x);
    }
    double total_sec = 0;
    for (int i = 0; i < rep; i++) {
```

always clone the x in each apply

```
auto xx = x->clone();
```

synchronize to make sure data is already on device

```
exec->synchronize();
auto start = std::chrono::steady_clock::now();
A->apply(b, xx);
```

synchronize to make sure the operation is done

```
exec->synchronize();
auto stop = std::chrono::steady_clock::now();
```

get the duration in seconds

```
std::chrono::duration<double> duration_time = stop - start;
total_sec += duration_time.count();
if (i + 1 == rep) {
```

copy the result back to x

```
            x->copy_from(xx);
        }
    }
    return total_sec / rep;
}
}  // namespace
int main(int argc, char* argv[])
{
```

Use some shortcuts. In Ginkgo, vectors are seen as a gko::matrix::Dense with one column/one row. The advantage of this concept is that using multiple vectors is a now a natural extension of adding columns/rows are necessary.

```
using HighPrecision = double;
using RealValueType = gko::remove_complex<HighPrecision>;
using LowPrecision = float;
using IndexType = int;
using hp_vec = gko::matrix::Dense<HighPrecision>;
using lp_vec = gko::matrix::Dense<LowPrecision>;
using real_vec = gko::matrix::Dense<RealValueType>;
```

The gko::matrix::Ell class is used here, but any other matrix class such as gko::matrix::Coo, gko::matrix::Hybrid, gko::matrix::Csr or gko::matrix::Sellp could also be used. Note. the behavior will depends GINKGO_MIXED_PR↩ ECISION flags and the actual implementation from different matrices.

```
using hp_mtx = gko::matrix::Ell<HighPrecision, IndexType>;
using lp_mtx = gko::matrix::Ell<LowPrecision, IndexType>;
```

Print the ginkgo version information.

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor] " « std::endl;
    std::exit(-1);
}
```

### Where do you want to run your operation?

The gko::Executor class is one of the cornerstones of Ginkgo. Currently, we have support for an gko::OmpExecutor, which uses OpenMP multi-threading in most of its kernels, a gko::ReferenceExecutor, a single threaded specialization of the OpenMP executor and a gko::CudaExecutor which runs the code on a NVIDIA GPU if available.

#### Note

With the help of C++, you see that you only ever need to change the executor and all the other functions/ routines within Ginkgo should automatically work and run on the executor with any other changes.

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Preparing your data and transfer to the proper device.

Read the matrix using the read function and set the right hand side randomly.

**Note**

Ginkgo uses C++ smart pointers to automatically manage memory. To this end, we use our own object ownership transfer functions that under the hood call the required smart pointer functions to manage object ownership. gko::share and gko::give are the functions that you would need to use.

read the matrix into HighPrecision and LowPrecision.

```
auto hp_A = share(gko::read<hp_mtx>(std::ifstream("data/A.mtx"), exec));
auto lp_A = share(gko::read<lp_mtx>(std::ifstream("data/A.mtx"), exec));
```

Set the shortcut for each dimension

```
auto A_dim = hp_A->get_size();
auto b_dim = gko::dim<2>{A_dim[1], 1};
auto x_dim = gko::dim<2>{A_dim[0], b_dim[1]};
auto host_b = hp_vec::create(exec->get_master(), b_dim);
```

fill the b vector with some random data

```
std::default_random_engine rand_engine(32);
auto dist = std::uniform_real_distribution<RealValueType>(0.0, 1.0);
for (int i = 0; i < host_b->get_size()[0]; i++) {
    host_b->at(i, 0) = get_rand_value<HighPrecision>(dist, rand_engine);
}
```

copy the data from host to device

```
auto hp_b = share(gko::clone(exec, host_b));
auto lp_b = share(lp_vec::create(exec));
lp_b->copy_from(hp_b);
```

create several result x vector in different precision

```
auto hp_x = share(hp_vec::create(exec, x_dim));
auto lp_x = share(lp_vec::create(exec, x_dim));
auto hplp_x = share(hp_x->clone());
auto lplp_x = share(hp_x->clone());
auto lphp_x = share(hp_x->clone());
```

### Measure the time of apply

We measure the time among different combination of apply operation.

Hp $*$ Hp -> Hp

```
auto hp_sec = timing(exec, hp_A, hp_b, hp_x);
```

Lp $*$ Lp -> Lp

```
auto lp_sec = timing(exec, lp_A, lp_b, lp_x);
```

Hp $*$ Lp -> Hp

```
auto hplp_sec = timing(exec, hp_A, lp_b, hplp_x);
```

Lp $*$ Lp -> Hp

```
auto lplp_sec = timing(exec, lp_A, lp_b, lplp_x);
```

Lp $*$ Hp -> Hp

```
auto lphp_sec = timing(exec, lp_A, hp_b, lphp_x);
```

To measure error of result. neg_one is an object that represent the number -1.0 which allows for a uniform interface when computing on any device. To compute the residual, all you need to do is call the add_scaled method, which in this case is an axpy and equivalent to the LAPACK axpy routine. Finally, you compute the euclidean 2-norm with the compute_norm2 function.

```
    auto neg_one = gko::initialize<hp_vec>({-1.0}, exec);
    auto hp_x_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
```

```
    auto hplp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lplp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lphp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lp_diff = hp_x->clone();
    auto hplp_diff = hp_x->clone();
    auto lplp_diff = hp_x->clone();
    auto lphp_diff = hp_x->clone();
    hp_x->compute_norm2(hp_x_norm);
    lp_diff->add_scaled(neg_one, lp_x);
    lp_diff->compute_norm2(lp_diff_norm);
    hplp_diff->add_scaled(neg_one, hplp_x);
    hplp_diff->compute_norm2(hplp_diff_norm);
    lplp_diff->add_scaled(neg_one, lplp_x);
    lplp_diff->compute_norm2(lplp_diff_norm);
    lphp_diff->add_scaled(neg_one, lphp_x);
    lphp_diff->compute_norm2(lphp_diff_norm);
    exec->synchronize();
    std::cout.precision(10);
    std::cout « std::scientific;
    std::cout « "High Precision time(s):  " « hp_sec « std::endl;
    std::cout « "High Precision result norm:  " « hp_x_norm->at(0)
              « std::endl;
    std::cout « "Low Precision time(s):  " « lp_sec « std::endl;
    std::cout « "Low Precision relative error:  "
              « lp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
    std::cout « "Hp * Lp -> Hp time(s):  " « hplp_sec « std::endl;
    std::cout « "Hp * Lp -> Hp relative error:  "
              « hplp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
    std::cout « "Lp * Lp -> Hp time(s):  " « lplp_sec « std::endl;
    std::cout « "Lp * Lp -> Hp relative error:  "
              « lplp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
    std::cout « "Lp * Hp -> Hp time(s):  " « lphp_sec « std::endl;
    std::cout « "Lp * Hp -> Hp relative error:  "
              « lphp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
}
```

# Results

The following is the expected result (omp):

```
High Precision time(s):  2.0568800000e-05
High Precision result norm:  1.7725534898e+05
Low Precision time(s):  2.0955600000e-05
Low Precision relative error:  9.1052887738e-08
Hp * Lp -> Hp time(s):  2.1186100000e-05
Hp * Lp -> Hp relative error:  3.7799774251e-08
Lp * Lp -> Hp time(s):  2.0312300000e-05
Lp * Lp -> Hp relative error:  5.7910008031e-08
Lp * Hp -> Hp time(s):  2.0312300000e-05
Lp * Hp -> Hp relative error:  3.7173133506e-08
```

### Comments about programming and debugging

# The plain program

```
#include <ginkgo/ginkgo.hpp>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <chrono>
#include <random>
namespace {
template <typename ValueType, typename ValueDistribution, typename Engine>
typename std::enable_if<!gko::is_complex_s<ValueType>::value, ValueType>::type
get_rand_value(ValueDistribution&& value_dist, Engine&& gen)
{
    return value_dist(gen);
}
template <typename ValueType, typename ValueDistribution, typename Engine>
typename std::enable_if<gko::is_complex_s<ValueType>::value, ValueType>::type
get_rand_value(ValueDistribution&& value_dist, Engine&& gen)
{
    return ValueType(value_dist(gen), value_dist(gen));
}
template <typename ValueTypeIn, typename ValueTypeOut>
```

```
double timing(std::shared_ptr<const gko::Executor> exec,
              std::shared_ptr<const gko::LinOp> A,
              std::shared_ptr<gko::matrix::Dense<ValueTypeIn>> b,
              std::shared_ptr<gko::matrix::Dense<ValueTypeOut>> x)
{
    int warmup = 2;
    int rep = 10;
    for (int i = 0; i < warmup; i++) {
        A->apply(b, x);
    }
    double total_sec = 0;
    for (int i = 0; i < rep; i++) {
        auto xx = x->clone();
        exec->synchronize();
        auto start = std::chrono::steady_clock::now();
        A->apply(b, xx);
        exec->synchronize();
        auto stop = std::chrono::steady_clock::now();
        std::chrono::duration<double> duration_time = stop - start;
        total_sec += duration_time.count();
        if (i + 1 == rep) {
            x->copy_from(xx);
        }
    }
    return total_sec / rep;
}
}  // namespace
int main(int argc, char* argv[])
{
    using HighPrecision = double;
    using RealValueType = gko::remove_complex<HighPrecision>;
    using LowPrecision = float;
    using IndexType = int;
    using hp_vec = gko::matrix::Dense<HighPrecision>;
    using lp_vec = gko::matrix::Dense<LowPrecision>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using hp_mtx = gko::matrix::Ell<HighPrecision, IndexType>;
    using lp_mtx = gko::matrix::Ell<LowPrecision, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor] " « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto hp_A = share(gko::read<hp_mtx>(std::ifstream("data/A.mtx"), exec));
    auto lp_A = share(gko::read<lp_mtx>(std::ifstream("data/A.mtx"), exec));
    auto A_dim = hp_A->get_size();
    auto b_dim = gko::dim<2>{A_dim[1], 1};
    auto x_dim = gko::dim<2>{A_dim[0], b_dim[1]};
    auto host_b = hp_vec::create(exec->get_master(), b_dim);
    std::default_random_engine rand_engine(32);
    auto dist = std::uniform_real_distribution<RealValueType>(0.0, 1.0);
    for (int i = 0; i < host_b->get_size()[0]; i++) {
        host_b->at(i, 0) = get_rand_value<HighPrecision>(dist, rand_engine);
    }
    auto hp_b = share(gko::clone(exec, host_b));
    auto lp_b = share(lp_vec::create(exec));
    lp_b->copy_from(hp_b);
    auto hp_x = share(hp_vec::create(exec, x_dim));
    auto lp_x = share(lp_vec::create(exec, x_dim));
    auto hplp_x = share(hp_x->clone());
    auto lplp_x = share(hp_x->clone());
    auto lphp_x = share(hp_x->clone());
    auto hp_sec = timing(exec, hp_A, hp_b, hp_x);
    auto lp_sec = timing(exec, lp_A, lp_b, lp_x);
    auto hplp_sec = timing(exec, hp_A, lp_b, hplp_x);
    auto lplp_sec = timing(exec, lp_A, lp_b, lplp_x);
    auto lphp_sec = timing(exec, lp_A, hp_b, lphp_x);
```

```
    auto neg_one = gko::initialize<hp_vec>({-1.0}, exec);
    auto hp_x_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto hplp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lplp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lphp_diff_norm = gko::initialize<real_vec>({0.0}, exec->get_master());
    auto lp_diff = hp_x->clone();
    auto hplp_diff = hp_x->clone();
    auto lplp_diff = hp_x->clone();
    auto lphp_diff = hp_x->clone();
    hp_x->compute_norm2(hp_x_norm);
    lp_diff->add_scaled(neg_one, lp_x);
    lp_diff->compute_norm2(lp_diff_norm);
    hplp_diff->add_scaled(neg_one, hplp_x);
    hplp_diff->compute_norm2(hplp_diff_norm);
    lplp_diff->add_scaled(neg_one, lplp_x);
    lplp_diff->compute_norm2(lplp_diff_norm);
    lphp_diff->add_scaled(neg_one, lphp_x);
    lphp_diff->compute_norm2(lphp_diff_norm);
    exec->synchronize();
    std::cout.precision(10);
    std::cout « std::scientific;
    std::cout « "High Precision time(s):  " « hp_sec « std::endl;
    std::cout « "High Precision result norm:  " « hp_x_norm->at(0)
              « std::endl;
    std::cout « "Low Precision time(s):  " « lp_sec « std::endl;
    std::cout « "Low Precision relative error:  "
              « lp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
    std::cout « "Hp * Lp -> Hp time(s):  " « hplp_sec « std::endl;
    std::cout « "Hp * Lp -> Hp relative error:  "
              « hplp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
    std::cout « "Lp * Lp -> Hp time(s):  " « lplp_sec « std::endl;
    std::cout « "Lp * Lp -> Hp relative error:  "
              « lplp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
    std::cout « "Lp * Hp -> Hp time(s):  " « lphp_sec « std::endl;
    std::cout « "Lp * Hp -> Hp relative error:  "
              « lphp_diff_norm->at(0) / hp_x_norm->at(0) « "\n";
}
```

# Chapter 32

# The multigrid-preconditioned-solver program

The preconditioned solver example..

This example depends on preconditioned-solver.

## This example shows how to use the multigrid preconditioner.

**In this example, we first read in a matrix from a file. The preconditioned CG solver is enhanced with a multigrid preconditioner. The example features the generating time and runtime of the CG solver.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using mg = gko::solver::Multigrid;
using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
```

```
            return gko::DpcppExecutor::create(
                0, gko::ReferenceExecutor::create());
        }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

### executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

### Create RHS as 1 and initial guess as 0

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 0.;
    host_b->at(i, 0) = 1.;
}
auto x = vec::create(exec);
auto b = vec::create(exec);
x->copy_from(host_x);
b->copy_from(host_b);
```

### Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

### copy b again

```
b->copy_from(host_b);
```

### Create multigrid factory

```
std::shared_ptr<gko::LinOpFactory> multigrid_gen;
multigrid_gen =
    mg::build()
        .with_mg_level(pgm::build().with_deterministic(true))
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec);
const gko::remove_complex<ValueType> tolerance = 1e-8;
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(100u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_baseline(gko::stop::mode::absolute)
                           .with_reduction_factor(tolerance))
        .with_preconditioner(multigrid_gen)
        .on(exec);
```

### Create solver

```
std::chrono::nanoseconds gen_time(0);
auto gen_tic = std::chrono::steady_clock::now();
auto solver = solver_gen->generate(A);
exec->synchronize();
auto gen_toc = std::chrono::steady_clock::now();
gen_time +=
    std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
```

### Add logger

```
std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
    gko::log::Convergence<ValueType>::create();
solver->add_logger(logger);
```

### Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
exec->synchronize();
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

### Calculate residual

```
auto res = gko::as<vec>(logger->get_residual_norm());
std::cout << "Initial residual norm sqrt(r^T r):  \n";
write(std::cout, initres);
```

```
std::cout « "Final residual norm sqrt(r^T r):  \n";
write(std::cout, res);
```

Print solver statistics

```
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
4.3589
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
1.69858e-09
CG iteration count:          39
CG generation time [ms]:  2.04293
CG execution time [ms]:  22.3874
CG execution time per iteration[ms]:  0.574036
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using mg = gko::solver::Multigrid;
    using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(
                     0, gko::ReferenceExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    gko::size_type size = A->get_size()[0];
```

```
    auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    for (auto i = 0; i < size; i++) {
        host_x->at(i, 0) = 0.;
        host_b->at(i, 0) = 1.;
    }
    auto x = vec::create(exec);
    auto b = vec::create(exec);
    x->copy_from(host_x);
    b->copy_from(host_b);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto initres = gko::initialize<vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(initres);
    b->copy_from(host_b);
    std::shared_ptr<gko::LinOpFactory> multigrid_gen;
    multigrid_gen =
        mg::build()
            .with_mg_level(pgm::build().with_deterministic(true))
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
            .on(exec);
    const gko::remove_complex<ValueType> tolerance = 1e-8;
    auto solver_gen =
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(100u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::absolute)
                               .with_reduction_factor(tolerance))
            .with_preconditioner(multigrid_gen)
            .on(exec);
    std::chrono::nanoseconds gen_time(0);
    auto gen_tic = std::chrono::steady_clock::now();
    auto solver = solver_gen->generate(A);
    exec->synchronize();
    auto gen_toc = std::chrono::steady_clock::now();
    gen_time +=
        std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
    std::shared_ptr<const gko::log::Convergence<ValueType» logger =
        gko::log::Convergence<ValueType>::create();
    solver->add_logger(logger);
    exec->synchronize();
    std::chrono::nanoseconds time(0);
    auto tic = std::chrono::steady_clock::now();
    solver->apply(b, x);
    exec->synchronize();
    auto toc = std::chrono::steady_clock::now();
    time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
    auto res = gko::as<vec>(logger->get_residual_norm());
    std::cout « "Initial residual norm sqrt(r^T r):  \n";
    write(std::cout, initres);
    std::cout « "Final residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Chapter 33

# The multigrid-preconditioned-solver-customized program

The customized multigrid preconditioned solver example..

This example depends on multigrid-preconditioned-solver.

## This example shows how to customize the multigrid preconditioner.

**In this example, we first read in a matrix from a file. The preconditioned CG solver is enhanced with a multigrid preconditioner. Several non-default options are used to create this preconditioner. The example features the generating time and runtime of the CG solver.**

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using mg = gko::solver::Multigrid;
using ic = gko::preconditioner::Ic<ValueType>;
using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
```

```
        }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(
                 0, gko::ReferenceExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

Create RHS as 1 and initial guess as 0

```
gko::size_type size = A->get_size()[0];
auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
for (auto i = 0; i < size; i++) {
    host_x->at(i, 0) = 0.;
    host_b->at(i, 0) = 1.;
}
auto x = vec::create(exec);
auto b = vec::create(exec);
x->copy_from(host_x);
b->copy_from(host_b);
```

Calculate initial residual by overwriting b

```
auto one = gko::initialize<vec>({1.0}, exec);
auto neg_one = gko::initialize<vec>({-1.0}, exec);
auto initres = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(initres);
```

copy b again

```
b->copy_from(host_b);
```

Prepare the stopping criteria

```
const gko::remove_complex<ValueType> tolerance = 1e-8;
auto iter_stop =
    gko::share(gko::stop::Iteration::build().with_max_iters(100u).on(exec));
auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                               .with_baseline(gko::stop::mode::absolute)
                               .with_reduction_factor(tolerance)
                               .on(exec));
auto exact_tol_stop =
    gko::share(gko::stop::ResidualNorm<ValueType>::build()
                   .with_baseline(gko::stop::mode::rhs_norm)
                   .with_reduction_factor(1e-14)
                   .on(exec));
std::shared_ptr<const gko::log::Convergence<ValueType>> logger =
    gko::log::Convergence<ValueType>::create();
iter_stop->add_logger(logger);
tol_stop->add_logger(logger);
```

Now we customize some settings of the multigrid preconditioner. First we choose a smoother. Since the input matrix is spd, we use iterative refinement with two iterations and an Ic solver.

```
auto ic_gen = gko::share(
    ic::build()
        .with_factorization(gko::factorization::Ic<ValueType, int>::build())
        .on(exec));
auto smoother_gen = gko::share(
    gko::solver::build_smoother(ic_gen, 2u, static_cast<ValueType>(0.9)));
```

Use Pgm as the MultigridLevel factory.

```
auto mg_level_gen =
    gko::share(pgm::build().with_deterministic(true).on(exec));
```

Next we select a CG solver for the coarsest level. Again, since the input matrix is known to be spd, and the Pgm restriction preserves this characteristic, we can safely choose the CG. We reuse the Ic factory here to generate an Ic preconditioner. It is important to solve until machine precision here to get a good convergence rate.

```
auto coarsest_gen = gko::share(cg::build()
                                   .with_preconditioner(ic_gen)
```

```
                                    .with_criteria(iter_stop, exact_tol_stop)
                                    .on(exec));
```

Here we put the customized options together and create the multigrid factory.

```
std::shared_ptr<gko::LinOpFactory> multigrid_gen;
multigrid_gen =
    mg::build()
        .with_max_levels(10u)
        .with_min_coarse_rows(32u)
        .with_pre_smoother(smoother_gen)
        .with_post_uses_pre(true)
        .with_mg_level(mg_level_gen)
        .with_coarsest_solver(coarsest_gen)
        .with_default_initial_guess(gko::solver::initial_guess_mode::zero)
        .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
        .on(exec);
```

Create solver factory

```
auto solver_gen = cg::build()
                      .with_criteria(iter_stop, tol_stop)
                      .with_preconditioner(multigrid_gen)
                      .on(exec);
```

Create solver

```
std::chrono::nanoseconds gen_time(0);
auto gen_tic = std::chrono::steady_clock::now();
auto solver = solver_gen->generate(A);
exec->synchronize();
auto gen_toc = std::chrono::steady_clock::now();
gen_time +=
    std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
```

Solve system

```
exec->synchronize();
std::chrono::nanoseconds time(0);
auto tic = std::chrono::steady_clock::now();
solver->apply(b, x);
exec->synchronize();
auto toc = std::chrono::steady_clock::now();
time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
```

Calculate residual

```
auto res = gko::initialize<vec>({0.0}, exec);
A->apply(one, x, neg_one, b);
b->compute_norm2(res);
std::cout « "Initial residual norm sqrt(r^T r):  \n";
write(std::cout, initres);
std::cout « "Final residual norm sqrt(r^T r):  \n";
write(std::cout, res);
```

Print solver statistics

```
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Results

This is the expected output:

```
Initial residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
25.9808
Final residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
5.81328e-09
CG iteration count:          12
CG generation time [ms]:  1.41642
CG execution time [ms]:  6.59244
CG execution time per iteration[ms]:  0.54937
```

### Comments about programming and debugging

## The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using mg = gko::solver::Multigrid;
    using ic = gko::preconditioner::Ic<ValueType>;
    using pgm = gko::multigrid::Pgm<ValueType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(
                     0, gko::ReferenceExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    gko::size_type size = A->get_size()[0];
    auto host_x = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    auto host_b = vec::create(exec->get_master(), gko::dim<2>(size, 1));
    for (auto i = 0; i < size; i++) {
        host_x->at(i, 0) = 0.;
        host_b->at(i, 0) = 1.;
    }
    auto x = vec::create(exec);
    auto b = vec::create(exec);
    x->copy_from(host_x);
    b->copy_from(host_b);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto initres = gko::initialize<vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(initres);
    b->copy_from(host_b);
    const gko::remove_complex<ValueType> tolerance = 1e-8;
    auto iter_stop =
        gko::share(gko::stop::Iteration::build().with_max_iters(100u).on(exec));
    auto tol_stop = gko::share(gko::stop::ResidualNorm<ValueType>::build()
                                   .with_baseline(gko::stop::mode::absolute)
                                   .with_reduction_factor(tolerance)
                                   .on(exec));
    auto exact_tol_stop =
        gko::share(gko::stop::ResidualNorm<ValueType>::build()
                       .with_baseline(gko::stop::mode::rhs_norm)
                       .with_reduction_factor(1e-14)
                       .on(exec));
    std::shared_ptr<const gko::log::Convergence<ValueType» logger =
        gko::log::Convergence<ValueType>::create();
    iter_stop->add_logger(logger);
    tol_stop->add_logger(logger);
    auto ic_gen = gko::share(
        ic::build()
            .with_factorization(gko::factorization::Ic<ValueType, int>::build())
            .on(exec));
    auto smoother_gen = gko::share(
        gko::solver::build_smoother(ic_gen, 2u, static_cast<ValueType>(0.9)));
    auto mg_level_gen =
        gko::share(pgm::build().with_deterministic(true).on(exec));
    auto coarsest_gen = gko::share(cg::build()
```

```
                                    .with_preconditioner(ic_gen)
                                    .with_criteria(iter_stop, exact_tol_stop)
                                    .on(exec));
    std::shared_ptr<gko::LinOpFactory> multigrid_gen;
    multigrid_gen =
        mg::build()
            .with_max_levels(10u)
            .with_min_coarse_rows(32u)
            .with_pre_smoother(smoother_gen)
            .with_post_uses_pre(true)
            .with_mg_level(mg_level_gen)
            .with_coarsest_solver(coarsest_gen)
            .with_default_initial_guess(gko::solver::initial_guess_mode::zero)
            .with_criteria(gko::stop::Iteration::build().with_max_iters(1u))
            .on(exec);
    auto solver_gen = cg::build()
                          .with_criteria(iter_stop, tol_stop)
                          .with_preconditioner(multigrid_gen)
                          .on(exec);
    std::chrono::nanoseconds gen_time(0);
    auto gen_tic = std::chrono::steady_clock::now();
    auto solver = solver_gen->generate(A);
    exec->synchronize();
    auto gen_toc = std::chrono::steady_clock::now();
    gen_time +=
        std::chrono::duration_cast<std::chrono::nanoseconds>(gen_toc - gen_tic);
    exec->synchronize();
    std::chrono::nanoseconds time(0);
    auto tic = std::chrono::steady_clock::now();
    solver->apply(b, x);
    exec->synchronize();
    auto toc = std::chrono::steady_clock::now();
    time += std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic);
    auto res = gko::initialize<vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Initial residual norm sqrt(r^T r):  \n";
    write(std::cout, initres);
    std::cout « "Final residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
    std::cout « "CG iteration count:          " « logger->get_num_iterations()
              « std::endl;
    std::cout « "CG generation time [ms]:  "
              « static_cast<double>(gen_time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time [ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 « std::endl;
    std::cout « "CG execution time per iteration[ms]:  "
              « static_cast<double>(time.count()) / 1000000.0 /
                    logger->get_num_iterations()
              « std::endl;
}
```

# Chapter 34

# The nine-pt-stencil-solver program

The 9-point stencil example..

This example depends on simple-solver, three-pt-stencil-solver, poisson-solver, .

## Introduction

This example solves a 2D Poisson equation:

[ \Omega = (0,1)^2 \ \Omega_b = [0,1]^2 \text{ (with boundary)} \ \partial\Omega = \Omega_b \backslash \Omega \ u : \Omega_b -> R \ u'' = f \in \Omega \ u = u_D \in \partial\Omega \ ]

using a finite difference method on an equidistant grid with `K` discretization points (`K` can be controlled with a command line parameter). The discretization may be done by any order Taylor polynomial. For an equidistant grid with K "inner" discretization points ((x1,y1), \ldots, (xk,y1),(x1,y2), \ldots, (xk,yk,z1)) step size (h = 1 / (K + 1)) and a stencil (\in \mathbb{R}^{3 \times 3}), the formula produces a system of linear equations

(\sum_{a,b=-1}^1 stencil(a,b) * u_{(i+a,j+b} = -f_k h^2), on any inner node with a neighborhood of inner nodes

On any node, where neighbor is on the border, the neighbor is replaced with a (-stencil(a,b) * u_{i+a,j+b}) and added to the right hand side vector. For example a node with a neighborhood of only edge nodes may look like this

[ \sum_{a,b=-1}^(1,0) stencil(a,b) * u_{(i+a,j+b} = -f_k h^2 - \sum_{a=-1}^1 stencil(a,1) * u_{(i+a,j+1} ]

which is then solved using Ginkgo's implementation of the CG method preconditioned with block-Jacobi. It is also possible to specify on which executor Ginkgo will solve the system via the command line. The function `f` is set to (f(x,y) = 6x + 6y) (making the solution (u(x,y) = x^3

- y^3)), but that can be changed in the `main` function. Also the stencil values for the core, the faces, the edge and the corners can be changed when passing additional parameters.

The intention of this is to show how generation of stencil values and the right hand side vector changes when increasing the dimension.

### About the example

## The commented program

```
/ ****************************<DESCRIPTION>**********************************
This example solves a 2D Poisson equation:
    \Omega = (0,1)^2
    \Omega_b = [0,1]^2   (with boundary)
    \partial\Omega = \Omega_b \backslash \Omega
    u :  \Omega_b -> R
    u" = f in \Omega
    u = u_D on \partial\Omega
using a finite difference method on an equidistant grid with `K` discretization
points (`K` can be controlled with a command line parameter).  The discretization
may be done by any order Taylor polynomial.
For an equidistant grid with K "inner" discretization points (x1,y1), ...,
(xk,y1),(x1,y2), ..., (xk,yk) step size h = 1 / (K + 1) and a stencil \in
\R^{3 x 3}, the formula produces a system of linear equations
\sum_{a,b=-1}^1 stencil(a,b) * u_{(i+a,j+b} = -f_k h^2,  on any inner node with
a neighborhood of inner nodes
On any node, where neighbor is on the border, the neighbor is replaced with a
'-stencil(a,b) * u_{i+a,j+b}' and added to the right hand side vector.  For
example a node with a neighborhood of only edge nodes may look like this
\sum_{a,b=-1}^(1,0) stencil(a,b) * u_{(i+a,j+b} = -f_k h^2 - \sum_{a=-1}^1
stencil(a,1) * u_{(i+a,j+1}
which is then solved using Ginkgo's implementation of the CG method
preconditioned with block-Jacobi.  It is also possible to specify on which
executor Ginkgo will solve the system via the command line.
The function `f` is set to `f(x,y) = 6x + 6y` (making the solution `u(x,y) = x^3
+ y^3`), but that can be changed in the `main` function.  Also the stencil values
for the core, the faces, the edge and the corners can be changed when passing
additional parameters.

The intention of this is to show how generation of stencil values and the right
hand side vector changes when increasing the dimension.
****************************<DESCRIPTION>********************************** /

#include <array>
#include <chrono>
#include <iostream>
#include <map>
#include <string>
#include <vector>

#include <ginkgo/ginkgo.hpp>
```

Stencil values. Ordering can be seen in the main function Can also be changed by passing additional parameter when executing

```
constexpr double default_alpha = 10.0 / 3.0;
constexpr double default_beta = -2.0 / 3.0;
constexpr double default_gamma = -1.0 / 6.0;
/ * Possible alternative default values are
 * default_alpha = 8.0;
 * default_beta = -1.0;
 * default_gamma = -1.0;
 * /
```

Creates a stencil matrix in CSR format for the given number of discretization points.

```
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(IndexType dp, IndexType* row_ptrs,
                             IndexType* col_idxs, ValueType* values,
                             ValueType* coefs)
{
    IndexType pos = 0;
    row_ptrs[0] = pos;
    for (IndexType k = 0; k < dp; ++k) {
        for (IndexType i = 0; i < dp; ++i) {
            const size_t index = i + k * dp;
            for (IndexType j = -1; j <= 1; ++j) {
                for (IndexType l = -1; l <= 1; ++l) {
                    const IndexType offset = l + 1 + 3 * (j + 1);
                    if ((k + j) >= 0 && (k + j) < dp && (i + l) >= 0 &&
                        (i + l) < dp) {
                        values[pos] = coefs[offset];
                        col_idxs[pos] = index + l + dp * j;
                        ++pos;
                    }
                }
            }
            row_ptrs[index + 1] = pos;
        }
    }
```

```
}
```

Generates the RHS vector given `f` and the boundary conditions.

```
template <typename Closure, typename ClosureT, typename ValueType,
          typename IndexType>
void generate_rhs(IndexType dp, Closure f, ClosureT u, ValueType* rhs,
                  ValueType* coefs)
{
    const ValueType h = 1.0 / (dp + 1.0);
    for (IndexType i = 0; i < dp; ++i) {
        const auto yi = ValueType(i + 1) * h;
        for (IndexType j = 0; j < dp; ++j) {
            const auto xi = ValueType(j + 1) * h;
            const auto index = i * dp + j;
            rhs[index] = -f(xi, yi) * h * h;
        }
    }
```

Iterating over the edges to add boundary values and adding the overlapping 3x1 to the rhs

```
for (size_t i = 0; i < dp; ++i) {
    const auto xi = ValueType(i + 1) * h;
    const auto index_top = i;
    const auto index_bot = i + dp * (dp - 1);
    rhs[index_top] -= u(xi - h, 0.0) * coefs[0];
    rhs[index_top] -= u(xi, 0.0) * coefs[1];
    rhs[index_top] -= u(xi + h, 0.0) * coefs[2];
    rhs[index_bot] -= u(xi - h, 1.0) * coefs[6];
    rhs[index_bot] -= u(xi, 1.0) * coefs[7];
    rhs[index_bot] -= u(xi + h, 1.0) * coefs[8];
}
for (size_t i = 0; i < dp; ++i) {
    const auto yi = ValueType(i + 1) * h;
    const auto index_left = i * dp;
    const auto index_right = i * dp + (dp - 1);
    rhs[index_left] -= u(0.0, yi - h) * coefs[0];
    rhs[index_left] -= u(0.0, yi) * coefs[3];
    rhs[index_left] -= u(0.0, yi + h) * coefs[6];
    rhs[index_right] -= u(1.0, yi - h) * coefs[2];
    rhs[index_right] -= u(1.0, yi) * coefs[5];
    rhs[index_right] -= u(1.0, yi + h) * coefs[8];
}
```

remove the double corner values

```
    rhs[0] += u(0.0, 0.0) * coefs[0];
    rhs[(dp - 1)] += u(1.0, 0.0) * coefs[2];
    rhs[(dp - 1) * dp] += u(0.0, 1.0) * coefs[6];
    rhs[dp * dp - 1] += u(1.0, 1.0) * coefs[8];
}
```

Prints the solution `u`.

```
template <typename ValueType, typename IndexType>
void print_solution(IndexType dp, const ValueType* u)
{
    for (IndexType i = 0; i < dp; ++i) {
        for (IndexType j = 0; j < dp; ++j) {
            std::cout « u[i * dp + j] « ' ';
        }
        std::cout « '\n';
    }
    std::cout « std::endl;
}
```

Computes the 1-norm of the error given the computed `u` and the correct solution function `correct_u`.

```
template <typename Closure, typename ValueType, typename IndexType>
gko::remove_complex<ValueType> calculate_error(IndexType dp, const ValueType* u,
                                               Closure correct_u)
{
    const ValueType h = 1.0 / (dp + 1);
    gko::remove_complex<ValueType> error = 0.0;
    for (IndexType j = 0; j < dp; ++j) {
        const auto xi = ValueType(j + 1) * h;
        for (IndexType i = 0; i < dp; ++i) {
            using std::abs;
            const auto yi = ValueType(i + 1) * h;
            error +=
                abs(u[i * dp + j] - correct_u(xi, yi)) / abs(correct_u(xi, yi));
        }
    }
    return error;
}
template <typename ValueType, typename IndexType>
void solve_system(const std::string& executor_string,
```

```
                unsigned int discretization_points, IndexType* row_ptrs,
                IndexType* col_idxs, ValueType* values, ValueType* rhs,
                ValueType* u, gko::remove_complex<ValueType> reduction_factor)
{
```

Some shortcuts

```
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
using val_array = gko::array<ValueType>;
using idx_array = gko::array<IndexType>;
const auto& dp = discretization_points;
const gko::size_type dp_2 = dp * dp;
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()>
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

executor where the application initialized the data

```
const auto app_exec = exec->get_master();
```

Tell Ginkgo to use the data in our application

Matrix: we have to set the executor of the matrix to the one where we want SpMVs to run (in this case `exec`). When creating array views, we have to specify the executor where the data is (in this case `app_exec`).

If the two do not match, Ginkgo will automatically create a copy of the data on `exec` (however, it will not copy the data back once it is done

- here this is not important since we are not modifying the matrix).

```
auto matrix = mtx::create(
    exec, gko::dim<2>(dp_2),
    val_array::view(app_exec, (3 * dp - 2) * (3 * dp - 2), values),
    idx_array::view(app_exec, (3 * dp - 2) * (3 * dp - 2), col_idxs),
    idx_array::view(app_exec, dp_2 + 1, row_ptrs));
```

RHS: similar to matrix

```
auto b = vec::create(exec, gko::dim<2>(dp_2, 1),
                     val_array::view(app_exec, dp_2, rhs), 1);
```

Solution: we have to be careful here - if the executors are different, once we compute the solution the array will not be automatically copied back to the original memory locations. Fortunately, whenever `apply` is called on a linear operator (e.g. matrix, solver) the arguments automatically get copied to the executor where the operator is, and copied back once the operation is completed. Thus, in this case, we can just define the solution on `app_exec`, and it will be automatically transferred to/from `exec` if needed.

```
auto x = vec::create(app_exec, gko::dim<2>(dp_2, 1),
                     val_array::view(app_exec, dp_2, u), 1);
```

Generate solver

```
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(dp_2),
```

```
                        gko::stop::ResidualNorm<ValueType>::build()
                            .with_reduction_factor(reduction_factor))
        .with_preconditioner(bj::build())
        .on(exec);
auto solver = solver_gen->generate(gko::give(matrix));
```

### Solve system

```
    solver->apply(b, x);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
```

### Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && std::string(argv[1]) == "--help") {
    std::cerr
        « "Usage:  " « argv[0]
        « " [executor] [DISCRETIZATION_POINTS] [alpha] [beta] [gamma]"
        « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const IndexType discretization_points =
    argc >= 3 ?  std::atoi(argv[2]) :  100;
const ValueType alpha_c = argc >= 4 ?  std::atof(argv[3]) :  default_alpha;
const ValueType beta_c = argc >= 5 ?  std::atof(argv[4]) :  default_beta;
const ValueType gamma_c = argc >= 6 ?  std::atof(argv[5]) :  default_gamma;
```

### clang-format off

```
std::array<ValueType, 9> coefs{
    gamma_c, beta_c, gamma_c,
        beta_c, alpha_c, beta_c,
    gamma_c, beta_c, gamma_c};
```

### clang-format on

```
const auto dp = discretization_points;
const size_t dp_2 = dp * dp;
```

### problem:

```
auto correct_u = [](ValueType x, ValueType y) {
    return x * x * x + y * y * y;
};
auto f = [](ValueType x, ValueType y) {
    return ValueType(6) * x + ValueType(6) * y;
};
```

### matrix

```
std::vector<IndexType> row_ptrs(dp_2 + 1);
std::vector<IndexType> col_idxs((3 * dp - 2) * (3 * dp - 2));
std::vector<ValueType> values((3 * dp - 2) * (3 * dp - 2));
```

### right hand side

```
std::vector<ValueType> rhs(dp_2);
```

### solution

```
std::vector<ValueType> u(dp_2, 0.0);
generate_stencil_matrix(dp, row_ptrs.data(), col_idxs.data(), values.data(),
                        coefs.data());
```

### looking for solution u = x^3: f = 6x, u(0) = 0, u(1) = 1

```
generate_rhs(dp, f, correct_u, rhs.data(), coefs.data());
const gko::remove_complex<ValueType> reduction_factor = 1e-7;
auto start_time = std::chrono::steady_clock::now();
solve_system(executor_string, dp, row_ptrs.data(), col_idxs.data(),
             values.data(), rhs.data(), u.data(), reduction_factor);
auto stop_time = std::chrono::steady_clock::now();
auto runtime_duration =
    static_cast<double>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(stop_time -
                                                             start_time)
            .count()) *
    1e-6;
```

### Uncomment to print the solution print_solution(dp, u.data());

```
    std::cout « "The average relative error is "
              « calculate_error(dp, u.data(), correct_u) /
                    static_cast<gko::remove_complex<ValueType>>(dp_2)
              « std::endl;
    std::cout « "The runtime is " « std::to_string(runtime_duration) « " ms"
              « std::endl;
}
```

# Results

The expected output should be

```
The average relative error is 6.35715e-06
The runtime is 167.320520 ms
```

### Comments about programming and debugging

# The plain program

```
/***************************<DESCRIPTION>**********************************
This example solves a 2D Poisson equation:

\Omega = (0,1)^2
\Omega_b = [0,1]^2   (with boundary)
\partial\Omega = \Omega_b \backslash \Omega
u :  \Omega_b -> R
u" = f in \Omega
u = u_D on \partial\Omega

using a finite difference method on an equidistant grid with `K` discretization
points (`K` can be controlled with a command line parameter).  The discretization
may be done by any order Taylor polynomial.
For an equidistant grid with K "inner" discretization points (x1,y1), ...,
(xk,y1),(x1,y2), ..., (xk,yk) step size h = 1 / (K + 1) and a stencil \in
\R^{3 x 3}, the formula produces a system of linear equations

\sum_{a,b=-1}^1 stencil(a,b) * u_{(i+a,j+b} = -f_k h^2,  on any inner node with
a neighborhood of inner nodes

On any node, where neighbor is on the border, the neighbor is replaced with a
'-stencil(a,b) * u_{i+a,j+b}' and added to the right hand side vector.  For
example a node with a neighborhood of only edge nodes may look like this

\sum_{a,b=-1}^(1,0) stencil(a,b) * u_{(i+a,j+b} = -f_k h^2 - \sum_{a=-1}^1
stencil(a,1) * u_{(i+a,j+1}

which is then solved using Ginkgo's implementation of the CG method
preconditioned with block-Jacobi.  It is also possible to specify on which
executor Ginkgo will solve the system via the command line.
The function `f` is set to `f(x,y) = 6x + 6y` (making the solution `u(x,y) = x^3
+ y^3`), but that can be changed in the `main` function.  Also the stencil values
for the core, the faces, the edge and the corners can be changed when passing
additional parameters.

The intention of this is to show how generation of stencil values and the right
hand side vector changes when increasing the dimension.
***************************<DESCRIPTION>**********************************/
#include <array>
#include <chrono>
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <ginkgo/ginkgo.hpp>
constexpr double default_alpha = 10.0 / 3.0;
constexpr double default_beta = -2.0 / 3.0;
constexpr double default_gamma = -1.0 / 6.0;
/* Possible alternative default values are
* default_alpha = 8.0;
* default_beta = -1.0;
* default_gamma = -1.0;
*/
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(IndexType dp, IndexType* row_ptrs,
                             IndexType* col_idxs, ValueType* values,
                             ValueType* coefs)
{
    IndexType pos = 0;
    row_ptrs[0] = pos;
    for (IndexType k = 0; k < dp; ++k) {
        for (IndexType i = 0; i < dp; ++i) {
            const size_t index = i + k * dp;
            for (IndexType j = -1; j <= 1; ++j) {
                for (IndexType l = -1; l <= 1; ++l) {
                    const IndexType offset = l + 1 + 3 * (j + 1);
                    if ((k + j) >= 0 && (k + j) < dp && (i + l) >= 0 &&
                        (i + l) < dp) {
                        values[pos] = coefs[offset];
```

```
                        col_idxs[pos] = index + 1 + dp * j;
                        ++pos;
                    }
                }
            }
            row_ptrs[index + 1] = pos;
        }
    }
}
template <typename Closure, typename ClosureT, typename ValueType,
          typename IndexType>
void generate_rhs(IndexType dp, Closure f, ClosureT u, ValueType* rhs,
                  ValueType* coefs)
{
    const ValueType h = 1.0 / (dp + 1.0);
    for (IndexType i = 0; i < dp; ++i) {
        const auto yi = ValueType(i + 1) * h;
        for (IndexType j = 0; j < dp; ++j) {
            const auto xi = ValueType(j + 1) * h;
            const auto index = i * dp + j;
            rhs[index] = -f(xi, yi) * h * h;
        }
    }
    for (size_t i = 0; i < dp; ++i) {
        const auto xi = ValueType(i + 1) * h;
        const auto index_top = i;
        const auto index_bot = i + dp * (dp - 1);
        rhs[index_top] -= u(xi - h, 0.0) * coefs[0];
        rhs[index_top] -= u(xi, 0.0) * coefs[1];
        rhs[index_top] -= u(xi + h, 0.0) * coefs[2];
        rhs[index_bot] -= u(xi - h, 1.0) * coefs[6];
        rhs[index_bot] -= u(xi, 1.0) * coefs[7];
        rhs[index_bot] -= u(xi + h, 1.0) * coefs[8];
    }
    for (size_t i = 0; i < dp; ++i) {
        const auto yi = ValueType(i + 1) * h;
        const auto index_left = i * dp;
        const auto index_right = i * dp + (dp - 1);
        rhs[index_left] -= u(0.0, yi - h) * coefs[0];
        rhs[index_left] -= u(0.0, yi) * coefs[3];
        rhs[index_left] -= u(0.0, yi + h) * coefs[6];
        rhs[index_right] -= u(1.0, yi - h) * coefs[2];
        rhs[index_right] -= u(1.0, yi) * coefs[5];
        rhs[index_right] -= u(1.0, yi + h) * coefs[8];
    }
    rhs[0] += u(0.0, 0.0) * coefs[0];
    rhs[(dp - 1)] += u(1.0, 0.0) * coefs[2];
    rhs[(dp - 1) * dp] += u(0.0, 1.0) * coefs[6];
    rhs[dp * dp - 1] += u(1.0, 1.0) * coefs[8];
}
template <typename ValueType, typename IndexType>
void print_solution(IndexType dp, const ValueType* u)
{
    for (IndexType i = 0; i < dp; ++i) {
        for (IndexType j = 0; j < dp; ++j) {
            std::cout << u[i * dp + j] << ' ';
        }
        std::cout << '\n';
    }
    std::cout << std::endl;
}
template <typename Closure, typename ValueType, typename IndexType>
gko::remove_complex<ValueType> calculate_error(IndexType dp, const ValueType* u,
                                               Closure correct_u)
{
    const ValueType h = 1.0 / (dp + 1);
    gko::remove_complex<ValueType> error = 0.0;
    for (IndexType j = 0; j < dp; ++j) {
        const auto xi = ValueType(j + 1) * h;
        for (IndexType i = 0; i < dp; ++i) {
            using std::abs;
            const auto yi = ValueType(i + 1) * h;
            error +=
                abs(u[i * dp + j] - correct_u(xi, yi)) / abs(correct_u(xi, yi));
        }
    }
    return error;
}
template <typename ValueType, typename IndexType>
void solve_system(const std::string& executor_string,
                  unsigned int discretization_points, IndexType* row_ptrs,
                  IndexType* col_idxs, ValueType* values, ValueType* rhs,
                  ValueType* u, gko::remove_complex<ValueType> reduction_factor)
{
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
```

```
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    using val_array = gko::array<ValueType>;
    using idx_array = gko::array<IndexType>;
    const auto& dp = discretization_points;
    const gko::size_type dp_2 = dp * dp;
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    const auto app_exec = exec->get_master();
    auto matrix = mtx::create(
        exec, gko::dim<2>(dp_2),
        val_array::view(app_exec, (3 * dp - 2) * (3 * dp - 2), values),
        idx_array::view(app_exec, (3 * dp - 2) * (3 * dp - 2), col_idxs),
        idx_array::view(app_exec, dp_2 + 1, row_ptrs));
    auto b = vec::create(exec, gko::dim<2>(dp_2, 1),
                         val_array::view(app_exec, dp_2, rhs), 1);
    auto x = vec::create(app_exec, gko::dim<2>(dp_2, 1),
                         val_array::view(app_exec, dp_2, u), 1);
    auto solver_gen =
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(dp_2),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .with_preconditioner(bj::build())
            .on(exec);
    auto solver = solver_gen->generate(gko::give(matrix));
    solver->apply(b, x);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && std::string(argv[1]) == "--help") {
        std::cerr
            « "Usage:  " « argv[0]
            « " [executor] [DISCRETIZATION_POINTS] [alpha] [beta] [gamma]"
            « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    const IndexType discretization_points =
        argc >= 3 ?  std::atoi(argv[2]) :  100;
    const ValueType alpha_c = argc >= 4 ?  std::atof(argv[3]) :  default_alpha;
    const ValueType beta_c = argc >= 5 ?  std::atof(argv[4]) :  default_beta;
    const ValueType gamma_c = argc >= 6 ?  std::atof(argv[5]) :  default_gamma;
    std::array<ValueType, 9> coefs{
        gamma_c, beta_c, gamma_c,
        beta_c, alpha_c, beta_c,
        gamma_c, beta_c, gamma_c};
    const auto dp = discretization_points;
    const size_t dp_2 = dp * dp;
    auto correct_u = [](ValueType x, ValueType y) {
        return x * x * x + y * y * y;
    };
    auto f = [](ValueType x, ValueType y) {
        return ValueType(6) * x + ValueType(6) * y;
    };
    std::vector<IndexType> row_ptrs(dp_2 + 1);
    std::vector<IndexType> col_idxs((3 * dp - 2) * (3 * dp - 2));
    std::vector<ValueType> values((3 * dp - 2) * (3 * dp - 2));
    std::vector<ValueType> rhs(dp_2);
    std::vector<ValueType> u(dp_2, 0.0);
    generate_stencil_matrix(dp, row_ptrs.data(), col_idxs.data(), values.data(),
                            coefs.data());
    generate_rhs(dp, f, correct_u, rhs.data(), coefs.data());
    const gko::remove_complex<ValueType> reduction_factor = 1e-7;
    auto start_time = std::chrono::steady_clock::now();
    solve_system(executor_string, dp, row_ptrs.data(), col_idxs.data(),
                 values.data(), rhs.data(), u.data(), reduction_factor);
    auto stop_time = std::chrono::steady_clock::now();
```

```
    auto runtime_duration =
        static_cast<double>(
            std::chrono::duration_cast<std::chrono::nanoseconds>(stop_time -
                                                                 start_time)
                .count()) *
        1e-6;
    std::cout « "The average relative error is "
              « calculate_error(dp, u.data(), correct_u) /
                    static_cast<gko::remove_complex<ValueType>>(dp_2)
              « std::endl;
    std::cout « "The runtime is " « std::to_string(runtime_duration) « " ms"
              « std::endl;
}
```

# Chapter 35

# The papi-logging program

The papi logging example..

This example depends on simple-solver-logging.

## Introduction

### About the example

## The commented program

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <thread>
#include <papi.h>
#include <ginkgo/ginkgo.hpp>
namespace {
void papi_add_event(const std::string& event_name, int& eventset)
{
    int code;
    int ret_val = PAPI_event_name_to_code(event_name.c_str(), &code);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_name_to_code()" « std::endl;
        std::exit(-1);
    }
    ret_val = PAPI_add_event(eventset, code);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_name_to_code()" « std::endl;
        std::exit(-1);
    }
}
template <typename T>
std::string to_string(T* ptr)
{
    std::ostringstream os;
    os « reinterpret_cast<gko::uintptr>(ptr);
    return os.str();
}
}  // namespace
int init_papi_counters(std::string solver_name, std::string A_name)
{
```

Initialize PAPI, add events and start it up

```
    int eventset = PAPI_NULL;
    int ret_val = PAPI_library_init(PAPI_VER_CURRENT);
    if (ret_val != PAPI_VER_CURRENT) {
        std::cerr « "Error at PAPI_library_init()" « std::endl;
        std::exit(-1);
    }
```

```
    ret_val = PAPI_create_eventset(&eventset);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_create_eventset()" « std::endl;
        std::exit(-1);
    }
    std::string simple_apply_string("sde:::ginkgo0::linop_apply_completed::");
    std::string advanced_apply_string(
        "sde:::ginkgo0::linop_advanced_apply_completed::");
    papi_add_event(simple_apply_string + solver_name, eventset);
    papi_add_event(simple_apply_string + A_name, eventset);
    papi_add_event(advanced_apply_string + A_name, eventset);
    ret_val = PAPI_start(eventset);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_start()" « std::endl;
        std::exit(-1);
    }
    return eventset;
}
void print_papi_counters(int eventset)
{
```

### Stop PAPI and read the linop_apply_completed event for all of them

```
long long int values[3];
int ret_val = PAPI_stop(eventset, values);
if (PAPI_OK != ret_val) {
    std::cerr « "Error at PAPI_stop()" « std::endl;
    std::exit(-1);
}
PAPI_shutdown();
```

### Print all values returned from PAPI

```
    std::cout « "PAPI SDE counters:" « std::endl;
    std::cout « "solver did " « values[0] « " applies." « std::endl;
    std::cout « "A did " « values[1] « " simple applies." « std::endl;
    std::cout « "A did " « values[2] « " advanced applies." « std::endl;
}
int main(int argc, char* argv[])
{
```

### Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
```

### Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
```

### Figure out where to run the code

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

### executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
```

```
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
```

Generate solver

```
const RealValueType reduction_factor{1e-7};
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
        .on(exec);
auto solver = solver_gen->generate(A);
```

In this example, we split as much as possible the Ginkgo solver/logger and the PAPI interface. Note that the PAPI ginkgo namespaces are of the form sde:::ginkgo<x> where <x> starts from 0 and is incremented with every new PAPI logger.

```
int eventset =
    init_papi_counters(to_string(solver.get()), to_string(A.get()));
```

Create a PAPI logger and add it to relevant LinOps

```
auto logger = gko::log::Papi<ValueType>::create(
    gko::log::Logger::linop_apply_completed_mask |
    gko::log::Logger::linop_advanced_apply_completed_mask);
solver->add_logger(logger);
A->add_logger(logger);
```

Solve system

```
solver->apply(b, x);
```

Stop PAPI event gathering and print the counters

```
print_papi_counters(eventset);
```

Print solution

```
std::cout « "Solution (x):  \n";
write(std::cout, x);
```

Calculate residual

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
}
```

# Results

The following is the expected result:

```
PAPI SDE counters:
solver did 1 applies.
A did 20 simple applies.
A did 1 advanced applies.
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
8.87107e-16
```

### Comments about programming and debugging

## The plain program

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <thread>
#include <papi.h>
#include <ginkgo/ginkgo.hpp>
namespace {
void papi_add_event(const std::string& event_name, int& eventset)
{
    int code;
    int ret_val = PAPI_event_name_to_code(event_name.c_str(), &code);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_name_to_code()" « std::endl;
        std::exit(-1);
    }
    ret_val = PAPI_add_event(eventset, code);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_name_to_code()" « std::endl;
        std::exit(-1);
    }
}
template <typename T>
std::string to_string(T* ptr)
{
    std::ostringstream os;
    os « reinterpret_cast<gko::uintptr>(ptr);
    return os.str();
}
}  // namespace
int init_papi_counters(std::string solver_name, std::string A_name)
{
    int eventset = PAPI_NULL;
    int ret_val = PAPI_library_init(PAPI_VER_CURRENT);
    if (ret_val != PAPI_VER_CURRENT) {
        std::cerr « "Error at PAPI_library_init()" « std::endl;
        std::exit(-1);
    }
    ret_val = PAPI_create_eventset(&eventset);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_create_eventset()" « std::endl;
        std::exit(-1);
    }
    std::string simple_apply_string("sde:::ginkgo0::linop_apply_completed::");
    std::string advanced_apply_string(
        "sde:::ginkgo0::linop_advanced_apply_completed::");
    papi_add_event(simple_apply_string + solver_name, eventset);
    papi_add_event(simple_apply_string + A_name, eventset);
    papi_add_event(advanced_apply_string + A_name, eventset);
    ret_val = PAPI_start(eventset);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_start()" « std::endl;
        std::exit(-1);
    }
    return eventset;
}
void print_papi_counters(int eventset)
{
    long long int values[3];
    int ret_val = PAPI_stop(eventset, values);
    if (PAPI_OK != ret_val) {
        std::cerr « "Error at PAPI_stop()" « std::endl;
        std::exit(-1);
    }
    PAPI_shutdown();
    std::cout « "PAPI SDE counters:" « std::endl;
    std::cout « "solver did " « values[0] « " applies." « std::endl;
    std::cout « "A did " « values[1] « " simple applies." « std::endl;
    std::cout « "A did " « values[2] « " advanced applies." « std::endl;
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
```

```
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    const RealValueType reduction_factor{1e-7};
    auto solver_gen =
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .on(exec);
    auto solver = solver_gen->generate(A);
    int eventset =
        init_papi_counters(to_string(solver.get()), to_string(A.get()));
    auto logger = gko::log::Papi<ValueType>::create(
        gko::log::Logger::linop_apply_completed_mask |
        gko::log::Logger::linop_advanced_apply_completed_mask);
    solver->add_logger(logger);
    A->add_logger(logger);
    solver->apply(b, x);
    print_papi_counters(eventset);
    std::cout « "Solution (x):  \n";
    write(std::cout, x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):  \n";
    write(std::cout, res);
}
```

# Chapter 36

# The par-ilu-convergence program

The ParILU convergence example..

This example depends on simple-solver.

## Introduction

### About the example

This example can be used to inspect the convergence behavior of parallel incomplete factorizations.

## The commented program

```
#include <fstream>
#include <functional>
#include <iostream>
#include <map>
#include <memory>
#include <string>
#include <ginkgo/ginkgo.hpp>
const std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    executors{
        {"reference", [] { return gko::ReferenceExecutor::create(); }},
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp", [] {
             return gko::DpcppExecutor::create(0, gko::OmpExecutor::create());
         }}};
template <typename Function>
auto try_generate(Function fun) -> decltype(fun())
{
    decltype(fun()) result;
    try {
        result = fun();
    } catch (const gko::Error& err) {
        std::cerr « "Error:  " « err.what() « '\n';
        std::exit(-1);
    }
    return result;
}
template <typename ValueType, typename IndexType>
double compute_ilu_residual_norm(
```

```
    const gko::matrix::Csr<ValueType, IndexType>* residual,
    const gko::matrix::Csr<ValueType, IndexType>* mtx)
{
    gko::matrix_data<ValueType, IndexType> residual_data;
    gko::matrix_data<ValueType, IndexType> mtx_data;
    residual->write(residual_data);
    mtx->write(mtx_data);
    residual_data.sort_row_major();
    mtx_data.sort_row_major();
    auto it = mtx_data.nonzeros.begin();
    double residual_norm{};
    for (auto entry :  residual_data.nonzeros) {
        auto ref_row = it->row;
        auto ref_col = it->column;
        if (entry.row == ref_row && entry.column == ref_col) {
            residual_norm += gko::squared_norm(entry.value);
            ++it;
        }
    }
    return std::sqrt(residual_norm);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
```

### print usage message

```
if (argc < 2 || executors.find(argv[1]) == executors.end()) {
    std::cerr « "Usage:  executable"
             « " <reference|omp|cuda|hip|dpcpp> [<matrix-file>] "
               "[<parilu|parilut|paric|paric] [<max-iterations>] "
               "[<num-repetitions>] [<fill-in-limit>]\n";
    return -1;
}
```

### generate executor based on first argument

```
auto exec = try_generate([&] { return executors.at(argv[1])(); });
```

### set matrix and preconditioner name with default values

```
std::string matrix = argc < 3 ?  "data/A.mtx" :  argv[2];
std::string precond = argc < 4 ?  "parilu" :  argv[3];
int max_iterations = argc < 5 ?  10 :  std::stoi(argv[4]);
int num_repetitions = argc < 6 ?  10 :  std::stoi(argv[5]);
double limit = argc < 7 ?  2 :  std::stod(argv[6]);
```

### load matrix file into Csr format

```
    auto mtx = gko::share(try_generate([&] {
        std::ifstream mtx_stream{matrix};
        if (!mtx_stream) {
            throw GKO_STREAM_ERROR("Unable to open matrix file");
        }
        std::cerr « "Reading " « matrix « std::endl;
        return gko::read<gko::matrix::Csr<ValueType, IndexType»(mtx_stream,
                                                                exec);
    }));
    auto factory_generator =
        [&](gko::size_type iteration) -> std::shared_ptr<gko::LinOpFactory> {
        if (precond == "parilu") {
            return gko::factorization::ParIlu<ValueType, IndexType>::build()
                .with_iterations(iteration)
                .on(exec);
        } else if (precond == "paric") {
            return gko::factorization::ParIc<ValueType, IndexType>::build()
                .with_iterations(iteration)
                .on(exec);
        } else if (precond == "parilut") {
            return gko::factorization::ParIlut<ValueType, IndexType>::build()
                .with_fill_in_limit(limit)
                .with_iterations(iteration)
                .on(exec);
        } else if (precond == "parict") {
            return gko::factorization::ParIct<ValueType, IndexType>::build()
                .with_fill_in_limit(limit)
                .with_iterations(iteration)
                .on(exec);
        } else {
            GKO_NOT_IMPLEMENTED;
        }
    };
    auto one = gko::initialize<gko::matrix::Dense<ValueType»({1.0}, exec);
    auto minus_one =
        gko::initialize<gko::matrix::Dense<ValueType»({-1.0}, exec);
    for (int it = 1; it <= max_iterations; ++it) {
```

```
        auto factory = factory_generator(it);
        std::cout « it « ';';
        std::vector<long> times;
        std::vector<double> residuals;
        for (int rep = 0; rep < num_repetitions; ++rep) {
            auto tic = std::chrono::high_resolution_clock::now();
            auto result =
                gko::as<gko::Composition<ValueType»(factory->generate(mtx));
            exec->synchronize();
            auto toc = std::chrono::high_resolution_clock::now();
            auto residual = gko::clone(exec, mtx);
            result->get_operators()[0]->apply(one, result->get_operators()[1],
                                              minus_one, residual);
            times.push_back(
                std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic)
                    .count());
            residuals.push_back(
                compute_ilu_residual_norm(residual.get(), mtx.get()));
        }
        for (auto el :  times) {
            std::cout « el « ';';
        }
        for (auto el :  residuals) {
            std::cout « el « ';';
        }
        std::cout « '\n';
    }
}
```

# Results

This is the expected output:

```
Usage:  executable <reference|omp|cuda|hip|dpcpp> [<matrix-file>] [<parilu|parilut|paric|parict]
      [<max-iterations>] [<num-repetitions>] [fill-in-limit]
```

When specifying an executor:

```
Reading data/A.mtx
1;71800;10300;8800;8200;8000;7700;7500;7500;7500;7400;1.0331e-14;1.0331e-14;1.0331e-14;1.0331e-14;1.0331e-14;1.0331e-14;1.033
2;15500;9100;13500;9000;8600;8800;8700;8600;8600;8500;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16
3;16500;10200;10100;10100;9900;10000;9800;9800;9900;9900;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-
4;17500;11500;11200;15600;11300;11200;11400;11200;11200;11100;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15
5;18800;12800;12700;12600;12500;12400;12400;12400;12400;14100;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15
6;19200;13400;23100;15400;13200;13000;13000;13000;13100;13000;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15
7;20500;14500;14400;14200;14200;14300;14200;14100;14300;14200;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15
8;21600;15700;86200;16300;15700;15600;15500;15400;15500;15600;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15
9;22700;17000;16700;16600;16700;16800;20400;17400;17500;17400;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15
10;25500;19000;18800;18700;18700;18800;18600;18700;18600;18700;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.15407e-16;4.1
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <functional>
#include <iostream>
#include <map>
#include <memory>
#include <string>
#include <ginkgo/ginkgo.hpp>
const std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    executors{
        {"reference", [] { return gko::ReferenceExecutor::create(); }},
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp", [] {
             return gko::DpcppExecutor::create(0, gko::OmpExecutor::create());
         }}};
```

```
template <typename Function>
auto try_generate(Function fun) -> decltype(fun())
{
    decltype(fun()) result;
    try {
        result = fun();
    } catch (const gko::Error& err) {
        std::cerr « "Error:  " « err.what() « '\n';
        std::exit(-1);
    }
    return result;
}
template <typename ValueType, typename IndexType>
double compute_ilu_residual_norm(
    const gko::matrix::Csr<ValueType, IndexType>* residual,
    const gko::matrix::Csr<ValueType, IndexType>* mtx)
{
    gko::matrix_data<ValueType, IndexType> residual_data;
    gko::matrix_data<ValueType, IndexType> mtx_data;
    residual->write(residual_data);
    mtx->write(mtx_data);
    residual_data.sort_row_major();
    mtx_data.sort_row_major();
    auto it = mtx_data.nonzeros.begin();
    double residual_norm{};
    for (auto entry :  residual_data.nonzeros) {
        auto ref_row = it->row;
        auto ref_col = it->column;
        if (entry.row == ref_row && entry.column == ref_col) {
            residual_norm += gko::squared_norm(entry.value);
            ++it;
        }
    }
    return std::sqrt(residual_norm);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    if (argc < 2 || executors.find(argv[1]) == executors.end()) {
        std::cerr « "Usage:  executable"
                  « " <reference|omp|cuda|hip|dpcpp> [<matrix-file>] "
                    "[<parilu|parilut|paric|parict] [<max-iterations>] "
                    "[<num-repetitions>] [<fill-in-limit>]\n";
        return -1;
    }
    auto exec = try_generate([&] { return executors.at(argv[1])(); });
    std::string matrix = argc < 3 ?  "data/A.mtx" :  argv[2];
    std::string precond = argc < 4 ?  "parilu" :  argv[3];
    int max_iterations = argc < 5 ?  10 :  std::stoi(argv[4]);
    int num_repetitions = argc < 6 ?  10 :  std::stoi(argv[5]);
    double limit = argc < 7 ?  2 :  std::stod(argv[6]);
    auto mtx = gko::share(try_generate([&] {
        std::ifstream mtx_stream{matrix};
        if (!mtx_stream) {
            throw GKO_STREAM_ERROR("Unable to open matrix file");
        }
        std::cerr « "Reading " « matrix « std::endl;
        return gko::read<gko::matrix::Csr<ValueType, IndexType»(mtx_stream,
                                                                exec);
    }));
    auto factory_generator =
        [&](gko::size_type iteration) -> std::shared_ptr<gko::LinOpFactory> {
        if (precond == "parilu") {
            return gko::factorization::ParIlu<ValueType, IndexType>::build()
                .with_iterations(iteration)
                .on(exec);
        } else if (precond == "paric") {
            return gko::factorization::ParIc<ValueType, IndexType>::build()
                .with_iterations(iteration)
                .on(exec);
        } else if (precond == "parilut") {
            return gko::factorization::ParIlut<ValueType, IndexType>::build()
                .with_fill_in_limit(limit)
                .with_iterations(iteration)
                .on(exec);
        } else if (precond == "parict") {
            return gko::factorization::ParIct<ValueType, IndexType>::build()
                .with_fill_in_limit(limit)
                .with_iterations(iteration)
                .on(exec);
        } else {
            GKO_NOT_IMPLEMENTED;
        }
    };
    auto one = gko::initialize<gko::matrix::Dense<ValueType»({1.0}, exec);
    auto minus_one =
```

```
        gko::initialize<gko::matrix::Dense<ValueType»({-1.0}, exec);
    for (int it = 1; it <= max_iterations; ++it) {
        auto factory = factory_generator(it);
        std::cout « it « ';';
        std::vector<long> times;
        std::vector<double> residuals;
        for (int rep = 0; rep < num_repetitions; ++rep) {
            auto tic = std::chrono::high_resolution_clock::now();
            auto result =
                gko::as<gko::Composition<ValueType»(factory->generate(mtx));
            exec->synchronize();
            auto toc = std::chrono::high_resolution_clock::now();
            auto residual = gko::clone(exec, mtx);
            result->get_operators()[0]->apply(one, result->get_operators()[1],
                                              minus_one, residual);
            times.push_back(
                std::chrono::duration_cast<std::chrono::nanoseconds>(toc - tic)
                    .count());
            residuals.push_back(
                compute_ilu_residual_norm(residual.get(), mtx.get()));
        }
        for (auto el :  times) {
            std::cout « el « ';';
        }
        for (auto el :  residuals) {
            std::cout « el « ';';
        }
        std::cout « '\n';
    }
}
```

# Chapter 37

# The performance-debugging program

The simple solver with performance debugging example..

This example depends on simple-solver-logging, minimal-cuda-solver.

## Introduction

### About the example

This example runs a solver on a test problem and shows how to use loggers to debug performance and convergence rate.

## The commented program

```
#include <algorithm>
#include <array>
#include <chrono>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <ostream>
#include <sstream>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
#include <ginkgo/ginkgo.hpp>
template <typename ValueType>
using vec = gko::matrix::Dense<ValueType>;
template <typename ValueType>
using real_vec = gko::matrix::Dense<gko::remove_complex<ValueType>>;
namespace utils {
```

creates a zero vector

```
template <typename ValueType>
std::unique_ptr<vec<ValueType>> create_vector(
    std::shared_ptr<const gko::Executor> exec, gko::size_type size,
    ValueType value)
{
    auto res = vec<ValueType>::create(exec);
    res->read(gko::matrix_data<ValueType>(gko::dim<2>{size, 1}, value));
    return res;
}
```

utilities for computing norms and residuals

```
template <typename ValueType>
ValueType get_first_element(const vec<ValueType>* norm)
{
    return norm->get_executor()->copy_val_to_host(norm->get_const_values());
}
template <typename ValueType>
gko::remove_complex<ValueType> compute_norm(const vec<ValueType>* b)
{
    auto exec = b->get_executor();
    auto b_norm = gko::initialize<real_vec<ValueType»({0.0}, exec);
    b->compute_norm2(b_norm);
    return get_first_element(b_norm.get());
}
template <typename ValueType>
gko::remove_complex<ValueType> compute_residual_norm(
    const gko::LinOp* system_matrix, const vec<ValueType>* b,
    const vec<ValueType>* x)
{
    auto exec = system_matrix->get_executor();
    auto one = gko::initialize<vec<ValueType»({1.0}, exec);
    auto neg_one = gko::initialize<vec<ValueType»({-1.0}, exec);
    auto res = gko::clone(b);
    system_matrix->apply(one, x, neg_one, res);
    return compute_norm(res.get());
}
}  // namespace utils
namespace loggers {
```

A logger that accumulates the time of all operations. For each operation type (allocations, free, copy, internal operations i.e. kernels), the timing is taken before and after. This can create significant overhead since to ensure proper timings, calls to `synchronize` are required.

```
struct OperationLogger :  gko::log::Logger {
    void on_allocation_started(const gko::Executor* exec,
                               const gko::size_type&)const override
{
        this->start_operation(exec, "allocate");
    }
    void on_allocation_completed(const gko::Executor* exec,
                                 const gko::size_type&,
                                 const gko::uintptr&)const override
{
        this->end_operation(exec, "allocate");
    }
    void on_free_started(const gko::Executor* exec,
                         const gko::uintptr&)const override
{
        this->start_operation(exec, "free");
    }
    void on_free_completed(const gko::Executor* exec,
                           const gko::uintptr&)const override
{
        this->end_operation(exec, "free");
    }
    void on_copy_started(const gko::Executor* from, const gko::Executor* to,
                         const gko::uintptr&, const gko::uintptr&,
                         const gko::size_type&)const override
{
        from->synchronize();
        this->start_operation(to, "copy");
    }
    void on_copy_completed(const gko::Executor* from, const gko::Executor* to,
                           const gko::uintptr&, const gko::uintptr&,
                           const gko::size_type&)const override
{
        from->synchronize();
        this->end_operation(to, "copy");
    }
    void on_operation_launched(const gko::Executor* exec,
                               const gko::Operation* op)const override
{
        this->start_operation(exec, op->get_name());
    }
    void on_operation_completed(const gko::Executor* exec,
                                const gko::Operation* op)const override
{
        this->end_operation(exec, op->get_name());
    }
    void write_data(std::ostream& ostream)
    {
        for (const auto& entry :  total) {
            ostream « "\t" « entry.first.c_str() « ":  "
                    « std::chrono::duration_cast<std::chrono::nanoseconds>(
                           entry.second)
                           .count()
                    « std::endl;
```

```
        }
    }
private:
```

Helper which synchronizes and starts the time before every operation.

```
void start_operation(const gko::Executor* exec,
                     const std::string& name)const
{
    nested.emplace_back(0);
    exec->synchronize();
    start[name] = std::chrono::steady_clock::now();
}
```

Helper to compute the end time and store the operation's time at its end. Also time nested operations.

```
void end_operation(const gko::Executor* exec, const std::string& name)const
{
    exec->synchronize();
    const auto end = std::chrono::steady_clock::now();
    const auto diff = end - start[name];
```

make sure timings for nested operations are not counted twice

```
    total[name] += diff - nested.back();
    nested.pop_back();
    if (nested.size() > 0) {
        nested.back() += diff;
    }
}
mutable std::map<std::string, std::chrono::steady_clock::time_point> start;
mutable std::map<std::string, std::chrono::steady_clock::duration> total;
```

the position i of this vector holds the total time spend on child operations on nesting level i

```
    mutable std::vector<std::chrono::steady_clock::duration> nested;
};
```

This logger tracks the persistently allocated data

```
struct StorageLogger :  gko::log::Logger {
```

Store amount of bytes allocated on every allocation

```
void on_allocation_completed(const gko::Executor*,
                             const gko::size_type& num_bytes,
                             const gko::uintptr& location)const override
{
    storage[location] = num_bytes;
}
```

Reset the amount of bytes on every free

```
void on_free_completed(const gko::Executor*,
                       const gko::uintptr& location)const override
{
    storage[location] = 0;
}
```

Write the data after summing the total from all allocations

```
    void write_data(std::ostream& ostream)
    {
        gko::size_type total{};
        for (const auto& e :  storage) {
            total += e.second;
        }
        ostream << "Storage:  " << total << std::endl;
    }
private:
    mutable std::unordered_map<gko::uintptr, gko::size_type> storage;
};
```

Logs true and recurrent residuals of the solver

```
template <typename ValueType>
struct ResidualLogger :  gko::log::Logger {
```

Depending on the available information, store the norm or compute it from the residual. If the true residual norm could not be computed, store the value $-1.0$.

```
    void on_iteration_complete(const gko::LinOp*, const gko::size_type&,
                               const gko::LinOp* residual,
                               const gko::LinOp* solution,
                               const gko::LinOp* residual_norm)const override
{
        if (residual_norm) {
```

```
            rec_res_norms.push_back(utils::get_first_element(
                gko::as<real_vec<ValueType>>(residual_norm)));
        } else {
            rec_res_norms.push_back(
                utils::compute_norm(gko::as<vec<ValueType>>(residual)));
        }
        if (solution) {
            true_res_norms.push_back(utils::compute_residual_norm(
                matrix, b, gko::as<vec<ValueType>>(solution)));
        } else {
            true_res_norms.push_back(-1.0);
        }
    }
    ResidualLogger(const gko::LinOp* matrix, const vec<ValueType>* b)
        :  gko::log::Logger(gko::log::Logger::iteration_complete_mask),
          matrix{matrix},
          b{b}
    {}
    void write_data(std::ostream& ostream)
    {
        ostream << "Recurrent Residual Norms:  " << std::endl;
        ostream << "[" << std::endl;
        for (const auto& entry :  rec_res_norms) {
            ostream << "\t" << entry << std::endl;
        }
        ostream << "];" << std::endl;
        ostream << "True Residual Norms:  " << std::endl;
        ostream << "[" << std::endl;
        for (const auto& entry :  true_res_norms) {
            ostream << "\t" << entry << std::endl;
        }
        ostream << "];" << std::endl;
    }
private:
    const gko::LinOp* matrix;
    const vec<ValueType>* b;
    mutable std::vector<gko::remove_complex<ValueType>> rec_res_norms;
    mutable std::vector<gko::remove_complex<ValueType>> true_res_norms;
};
}  // namespace loggers
namespace {
```

### Print usage help

```
void print_usage(const char* filename)
{
    std::cerr << "Usage:  " << filename << " [executor] [matrix file]"
              << std::endl;
    std::cerr << "matrix file should be a file in matrix market format.  "
                 "The file data/A.mtx is provided as an example."
              << std::endl;
    std::exit(-1);
}
template <typename ValueType>
void print_vector(const gko::matrix::Dense<ValueType>* vec)
{
    auto elements_to_print = std::min(gko::size_type(10), vec->get_size()[0]);
    std::cout << "[" << std::endl;
    for (int i = 0; i < elements_to_print; ++i) {
        std::cout << "\t" << vec->at(i) << std::endl;
    }
    std::cout << "];" << std::endl;
}
}  // namespace
int main(int argc, char* argv[])
{
```

### Parametrize the benchmark here Pick a value type

```
using ValueType = double;
using IndexType = int;
```

### Pick a matrix format

```
using mtx = gko::matrix::Csr<ValueType, IndexType>;
```

### Pick a solver

```
using solver = gko::solver::Cg<ValueType>;
```

### Pick a preconditioner type

```
using preconditioner = gko::matrix::IdentityFactory<ValueType>;
```

### Pick a residual norm reduction value

```
const gko::remove_complex<ValueType> reduction_factor = 1e-12;
```

Pick an output file name
```
const auto of_name = "log.txt";
```

Print version information
```
std::cout « gko::version_info::get() « std::endl;
```

Figure out where to run the code
```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    print_usage(argv[0]);
}
```

Figure out where to run the code
```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation
```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read the input matrix file directory
```
std::string input_mtx = "data/A.mtx";
if (argc == 3) {
    input_mtx = std::string(argv[2]);
}
```

Read data: A is read from disk Create a StorageLogger to track the size of A
```
auto storage_logger = std::make_shared<loggers::StorageLogger>();
```

Add the logger to the executor
```
exec->add_logger(storage_logger);
```

Read the matrix A from file
```
auto A = gko::share(gko::read<mtx>(std::ifstream(input_mtx), exec));
```

Remove the storage logger
```
exec->remove_logger(storage_logger);
```

Pick a maximum iteration count
```
const auto max_iters = A->get_size()[0];
```

Generate b and x vectors
```
auto b = utils::create_vector<ValueType>(exec, A->get_size()[0], 1.0);
auto x = utils::create_vector<ValueType>(exec, A->get_size()[0], 0.0);
```

Declare the solver factory. The preconditioner's arguments should be adapted if needed.
```
auto solver_factory =
    solver::build()
        .with_criteria(
            gko::stop::ResidualNorm<ValueType>::build()
                .with_reduction_factor(reduction_factor),
            gko::stop::Iteration::build().with_max_iters(max_iters))
        .with_preconditioner(preconditioner::create(exec))
        .on(exec);
```

Declare the output file for all our loggers
```
std::ofstream output_file(of_name);
```

Do a warmup run

```
{
```

Clone x to not overwrite the original one

```
auto x_clone = gko::clone(x);
```

Generate and call apply on a solver

```
    solver_factory->generate(A)->apply(b, x_clone);
    exec->synchronize();
}
```

Do a timed run

```
{
```

Clone x to not overwrite the original one

```
auto x_clone = gko::clone(x);
```

Synchronize ensures no operation are ongoing

```
exec->synchronize();
```

Time before generate

```
auto g_tic = std::chrono::steady_clock::now();
```

Generate a solver

```
auto generated_solver = solver_factory->generate(A);
exec->synchronize();
```

Time after generate

```
auto g_tac = std::chrono::steady_clock::now();
```

Compute the generation time

```
auto generate_time =
    std::chrono::duration_cast<std::chrono::nanoseconds>(g_tac - g_tic);
```

Write the generate time to the output file

```
output_file « "Generate time (ns):  " « generate_time.count()
            « std::endl;
```

Similarly time the apply

```
exec->synchronize();
auto a_tic = std::chrono::steady_clock::now();
generated_solver->apply(b, x_clone);
exec->synchronize();
auto a_tac = std::chrono::steady_clock::now();
auto apply_time =
    std::chrono::duration_cast<std::chrono::nanoseconds>(a_tac - a_tic);
output_file « "Apply time (ns):  " « apply_time.count() « std::endl;
```

Compute the residual norm

```
    auto residual =
        utils::compute_residual_norm(A.get(), b.get(), x_clone.get());
    output_file « "Residual_norm:  " « residual « std::endl;
}
```

Log the internal operations using the OperationLogger without timing

```
{
```

Create an OperationLogger to analyze the generate step

```
auto gen_logger = std::make_shared<loggers::OperationLogger>();
```

Add the generate logger to the executor

```
exec->add_logger(gen_logger);
```

Generate a solver

```
auto generated_solver = solver_factory->generate(A);
```

Remove the generate logger from the executor

```
exec->remove_logger(gen_logger);
```

Write the data to the output file

```
output_file « "Generate operations times (ns):" « std::endl;
```

```
gen_logger->write_data(output_file);
```

Create an OperationLogger to analyze the apply step

```
auto apply_logger = std::make_shared<loggers::OperationLogger>();
exec->add_logger(apply_logger);
```

Create a ResidualLogger to log the recurrent residual

```
auto res_logger = std::make_shared<loggers::ResidualLogger<ValueType»(
    A.get(), b.get());
generated_solver->add_logger(res_logger);
```

Solve the system

```
generated_solver->apply(b, x);
exec->remove_logger(apply_logger);
```

Write the data to the output file

```
    output_file « "Apply operations times (ns):" « std::endl;
    apply_logger->write_data(output_file);
    res_logger->write_data(output_file);
}
```

Print solution

```
std::cout « "Solution, first ten entries:  \n";
print_vector(x.get());
```

Print output file location

```
    std::cout « "The performance and residual data can be found in " « of_name
              « std::endl;
}
```

# Results

This is the expected standard output:

```
Solution, first ten entries:
[
    0.252218
    0.108645
    0.0662811
    0.0630433
    0.0384088
    0.0396536
    0.0402648
    0.0338935
    0.0193098
    0.0234653
];
The performance and residual data can be found in log.txt
```

Here is a sample output in the file log.txt:

```
Generate time (ns):  861
Apply time (ns):  108144
Residual_norm:  2.10788e-15
Generate operations times (ns):
Apply operations times (ns):
    allocate:  14991
    cg::initialize#8:  872
    cg::step_1#5:  7683
    cg::step_2#7:  7756
    copy:  7751
    csr::advanced_spmv#5:  21819
    csr::spmv#3:  20429
    dense::compute_dot#3:  18043
    dense::compute_norm2#2:  16726
    free:  8857
    residual_norm::residual_norm#9:  3614
Recurrent Residual Norms:
[
    4.3589
    2.30455
    1.46771
    0.984875
    0.741833
    0.513623
    0.384165
    0.316439
```

```
    0.227709
    0.170312
    0.0973722
    0.0616831
    0.0454123
    0.031953
    0.0161606
    0.00657015
    0.00264367
    0.000858809
    0.000286461
    1.64195e-15
];
True Residual Norms:
[
    4.3589
    2.30455
    1.46771
    0.984875
    0.741833
    0.513623
    0.384165
    0.316439
    0.227709
    0.170312
    0.0973722
    0.0616831
    0.0454123
    0.031953
    0.0161606
    0.00657015
    0.00264367
    0.000858809
    0.000286461
    2.10788e-15
];
```

### Comments about programming and debugging

# The plain program

```
#include <algorithm>
#include <array>
#include <chrono>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <ostream>
#include <sstream>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
#include <ginkgo/ginkgo.hpp>
template <typename ValueType>
using vec = gko::matrix::Dense<ValueType>;
template <typename ValueType>
using real_vec = gko::matrix::Dense<gko::remove_complex<ValueType>>;
namespace utils {
template <typename ValueType>
std::unique_ptr<vec<ValueType>> create_vector(
    std::shared_ptr<const gko::Executor> exec, gko::size_type size,
    ValueType value)
{
    auto res = vec<ValueType>::create(exec);
    res->read(gko::matrix_data<ValueType>(gko::dim<2>{size, 1}, value));
    return res;
}
template <typename ValueType>
ValueType get_first_element(const vec<ValueType>* norm)
{
    return norm->get_executor()->copy_val_to_host(norm->get_const_values());
}
template <typename ValueType>
gko::remove_complex<ValueType> compute_norm(const vec<ValueType>* b)
{
    auto exec = b->get_executor();
    auto b_norm = gko::initialize<real_vec<ValueType>>({0.0}, exec);
    b->compute_norm2(b_norm);
```

```
    return get_first_element(b_norm.get());
}
template <typename ValueType>
gko::remove_complex<ValueType> compute_residual_norm(
    const gko::LinOp* system_matrix, const vec<ValueType>* b,
    const vec<ValueType>* x)
{
    auto exec = system_matrix->get_executor();
    auto one = gko::initialize<vec<ValueType>>({1.0}, exec);
    auto neg_one = gko::initialize<vec<ValueType>>({-1.0}, exec);
    auto res = gko::clone(b);
    system_matrix->apply(one, x, neg_one, res);
    return compute_norm(res.get());
}
}  // namespace utils
namespace loggers {
struct OperationLogger :  gko::log::Logger {
    void on_allocation_started(const gko::Executor* exec,
                               const gko::size_type&)const override
{
        this->start_operation(exec, "allocate");
    }
    void on_allocation_completed(const gko::Executor* exec,
                                 const gko::size_type&,
                                 const gko::uintptr&)const override
{
        this->end_operation(exec, "allocate");
    }
    void on_free_started(const gko::Executor* exec,
                         const gko::uintptr&)const override
{
        this->start_operation(exec, "free");
    }
    void on_free_completed(const gko::Executor* exec,
                           const gko::uintptr&)const override
{
        this->end_operation(exec, "free");
    }
    void on_copy_started(const gko::Executor* from, const gko::Executor* to,
                         const gko::uintptr&, const gko::uintptr&,
                         const gko::size_type&)const override
{
        from->synchronize();
        this->start_operation(to, "copy");
    }
    void on_copy_completed(const gko::Executor* from, const gko::Executor* to,
                           const gko::uintptr&, const gko::uintptr&,
                           const gko::size_type&)const override
{
        from->synchronize();
        this->end_operation(to, "copy");
    }
    void on_operation_launched(const gko::Executor* exec,
                               const gko::Operation* op)const override
{
        this->start_operation(exec, op->get_name());
    }
    void on_operation_completed(const gko::Executor* exec,
                                const gko::Operation* op)const override
{
        this->end_operation(exec, op->get_name());
    }
    void write_data(std::ostream& ostream)
    {
        for (const auto& entry :  total) {
            ostream « "\t" « entry.first.c_str() « ":  "
                    « std::chrono::duration_cast<std::chrono::nanoseconds>(
                           entry.second)
                           .count()
                    « std::endl;
        }
    }
private:
    void start_operation(const gko::Executor* exec,
                         const std::string& name)const
{
        nested.emplace_back(0);
        exec->synchronize();
        start[name] = std::chrono::steady_clock::now();
    }
    void end_operation(const gko::Executor* exec, const std::string& name)const
{
        exec->synchronize();
        const auto end = std::chrono::steady_clock::now();
        const auto diff = end - start[name];
        total[name] += diff - nested.back();
        nested.pop_back();
```

```
        if (nested.size() > 0) {
            nested.back() += diff;
        }
    }
    mutable std::map<std::string, std::chrono::steady_clock::time_point> start;
    mutable std::map<std::string, std::chrono::steady_clock::duration> total;
    mutable std::vector<std::chrono::steady_clock::duration> nested;
};
struct StorageLogger :  gko::log::Logger {
    void on_allocation_completed(const gko::Executor*,
                                 const gko::size_type& num_bytes,
                                 const gko::uintptr& location)const override
{
        storage[location] = num_bytes;
    }
    void on_free_completed(const gko::Executor*,
                           const gko::uintptr& location)const override
{
        storage[location] = 0;
    }
    void write_data(std::ostream& ostream)
    {
        gko::size_type total{};
        for (const auto& e :  storage) {
            total += e.second;
        }
        ostream « "Storage:  " « total « std::endl;
    }
private:
    mutable std::unordered_map<gko::uintptr, gko::size_type> storage;
};
template <typename ValueType>
struct ResidualLogger :  gko::log::Logger {
    void on_iteration_complete(const gko::LinOp*, const gko::size_type&,
                               const gko::LinOp* residual,
                               const gko::LinOp* solution,
                               const gko::LinOp* residual_norm)const override
{
        if (residual_norm) {
            rec_res_norms.push_back(utils::get_first_element(
                gko::as<real_vec<ValueType»(residual_norm)));
        } else {
            rec_res_norms.push_back(
                utils::compute_norm(gko::as<vec<ValueType»(residual)));
        }
        if (solution) {
            true_res_norms.push_back(utils::compute_residual_norm(
                matrix, b, gko::as<vec<ValueType»(solution)));
        } else {
            true_res_norms.push_back(-1.0);
        }
    }
    ResidualLogger(const gko::LinOp* matrix, const vec<ValueType>* b)
        :  gko::log::Logger(gko::log::Logger::iteration_complete_mask),
          matrix{matrix},
          b{b}
    {}
    void write_data(std::ostream& ostream)
    {
        ostream « "Recurrent Residual Norms:  " « std::endl;
        ostream « "[" « std::endl;
        for (const auto& entry :  rec_res_norms) {
            ostream « "\t" « entry « std::endl;
        }
        ostream « "];" « std::endl;
        ostream « "True Residual Norms:  " « std::endl;
        ostream « "[" « std::endl;
        for (const auto& entry :  true_res_norms) {
            ostream « "\t" « entry « std::endl;
        }
        ostream « "];" « std::endl;
    }
private:
    const gko::LinOp* matrix;
    const vec<ValueType>* b;
    mutable std::vector<gko::remove_complex<ValueType» rec_res_norms;
    mutable std::vector<gko::remove_complex<ValueType» true_res_norms;
};
}  // namespace loggers
namespace {
void print_usage(const char* filename)
{
    std::cerr « "Usage:  " « filename « " [executor] [matrix file]"
              « std::endl;
    std::cerr « "matrix file should be a file in matrix market format.  "
                "The file data/A.mtx is provided as an example."
              « std::endl;
```

```
    std::exit(-1);
}
template <typename ValueType>
void print_vector(const gko::matrix::Dense<ValueType>* vec)
{
    auto elements_to_print = std::min(gko::size_type(10), vec->get_size()[0]);
    std::cout << "[" << std::endl;
    for (int i = 0; i < elements_to_print; ++i) {
        std::cout << "\t" << vec->at(i) << std::endl;
    }
    std::cout << "];" << std::endl;
}
}  // namespace
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using solver = gko::solver::Cg<ValueType>;
    using preconditioner = gko::matrix::IdentityFactory<ValueType>;
    const gko::remove_complex<ValueType> reduction_factor = 1e-12;
    const auto of_name = "log.txt";
    std::cout << gko::version_info::get() << std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        print_usage(argv[0]);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()>
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    std::string input_mtx = "data/A.mtx";
    if (argc == 3) {
        input_mtx = std::string(argv[2]);
    }
    auto storage_logger = std::make_shared<loggers::StorageLogger>();
    exec->add_logger(storage_logger);
    auto A = gko::share(gko::read<mtx>(std::ifstream(input_mtx), exec));
    exec->remove_logger(storage_logger);
    const auto max_iters = A->get_size()[0];
    auto b = utils::create_vector<ValueType>(exec, A->get_size()[0], 1.0);
    auto x = utils::create_vector<ValueType>(exec, A->get_size()[0], 0.0);
    auto solver_factory =
        solver::build()
            .with_criteria(
                gko::stop::ResidualNorm<ValueType>::build()
                    .with_reduction_factor(reduction_factor),
                gko::stop::Iteration::build().with_max_iters(max_iters))
            .with_preconditioner(preconditioner::create(exec))
            .on(exec);
    std::ofstream output_file(of_name);
    {
        auto x_clone = gko::clone(x);
        solver_factory->generate(A)->apply(b, x_clone);
        exec->synchronize();
    }
    {
        auto x_clone = gko::clone(x);
        exec->synchronize();
        auto g_tic = std::chrono::steady_clock::now();
        auto generated_solver = solver_factory->generate(A);
        exec->synchronize();
        auto g_tac = std::chrono::steady_clock::now();
        auto generate_time =
            std::chrono::duration_cast<std::chrono::nanoseconds>(g_tac - g_tic);
        output_file << "Generate time (ns):  " << generate_time.count()
                    << std::endl;
        exec->synchronize();
        auto a_tic = std::chrono::steady_clock::now();
        generated_solver->apply(b, x_clone);
        exec->synchronize();
        auto a_tac = std::chrono::steady_clock::now();
```

```
        auto apply_time =
            std::chrono::duration_cast<std::chrono::nanoseconds>(a_tac - a_tic);
        output_file « "Apply time (ns):  " « apply_time.count() « std::endl;
        auto residual =
            utils::compute_residual_norm(A.get(), b.get(), x_clone.get());
        output_file « "Residual_norm:  " « residual « std::endl;
    }
    {
        auto gen_logger = std::make_shared<loggers::OperationLogger>();
        exec->add_logger(gen_logger);
        auto generated_solver = solver_factory->generate(A);
        exec->remove_logger(gen_logger);
        output_file « "Generate operations times (ns):" « std::endl;
        gen_logger->write_data(output_file);
        auto apply_logger = std::make_shared<loggers::OperationLogger>();
        exec->add_logger(apply_logger);
        auto res_logger = std::make_shared<loggers::ResidualLogger<ValueType»(
            A.get(), b.get());
        generated_solver->add_logger(res_logger);
        generated_solver->apply(b, x);
        exec->remove_logger(apply_logger);
        output_file « "Apply operations times (ns):" « std::endl;
        apply_logger->write_data(output_file);
        res_logger->write_data(output_file);
    }
    std::cout « "Solution, first ten entries:  \n";
    print_vector(x.get());
    std::cout « "The performance and residual data can be found in " « of_name
              « std::endl;
}
```

# Chapter 38

# The poisson-solver program

The poisson solver example..

This example depends on simple-solver.

## Introduction

This example solves a 1D Poisson equation:

$u : [0, 1] \to R$
$u" = f$
$u(0) = u0$
$u(1) = u1$

using a finite difference method on an equidistant grid with `K` discretization points (`K` can be controlled with a command line parameter). The discretization is done via the second order Taylor polynomial:

$u(x + h) = u(x) - u'(x)h + 1/2u"(x)h^2 + O(h^3)$
$u(x - h) = u(x) + u'(x)h + 1/2u"(x)h^2 + O(h^3)/ +$
$- - - - - - - - - - - - - - - - - - - - - - - - - -$
$- u(x - h) + 2u(x) + -u(x + h) = -f(x)h^2 + O(h^3)$

For an equidistant grid with K "inner" discretization points $x1, ..., xk$ and step size $h = 1/(K + 1)$, the formula produces a system of linear equations

$2u_1 - u_2 = -f_1h^2 + u0$
$- u_(k - 1) + 2u_k - u_(k + 1) = -f_kh^2, k = 2, ..., K - 1$
$- u_(K - 1) + 2u_K = -f_Kh^2 + u1$

which is then solved using Ginkgo's implementation of the CG method preconditioned with block-Jacobi. It is also possible to specify on which executor Ginkgo will solve the system via the command line. The function `f` is set to `f(x) = 6x` (making the solution `u(x) = x^3`), but that can be changed in the `main` function.

The intention of the example is to show how Ginkgo can be used to build an application solving a real-world problem, which includes a solution of a large, sparse linear system as a component.

### About the example

## The commented program

```
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <ginkgo/ginkgo.hpp>
```

Creates a stencil matrix in CSR format for the given number of discretization points.

```
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(gko::matrix::Csr<ValueType, IndexType>* matrix)
{
    const auto discretization_points = matrix->get_size()[0];
    auto row_ptrs = matrix->get_row_ptrs();
    auto col_idxs = matrix->get_col_idxs();
    auto values = matrix->get_values();
    int pos = 0;
    const ValueType coefs[] = {-1, 2, -1};
    row_ptrs[0] = pos;
    for (int i = 0; i < discretization_points; ++i) {
        for (auto ofs :  {-1, 0, 1}) {
            if (0 <= i + ofs && i + ofs < discretization_points) {
                values[pos] = coefs[ofs + 1];
                col_idxs[pos] = i + ofs;
                ++pos;
            }
        }
        row_ptrs[i + 1] = pos;
    }
}
```

Generates the RHS vector given `f` and the boundary conditions.

```
template <typename Closure, typename ValueType>
void generate_rhs(Closure f, ValueType u0, ValueType u1,
                  gko::matrix::Dense<ValueType>* rhs)
{
    const auto discretization_points = rhs->get_size()[0];
    auto values = rhs->get_values();
    const ValueType h = 1.0 / static_cast<ValueType>(discretization_points + 1);
    for (gko::size_type i = 0; i < discretization_points; ++i) {
        const auto xi = static_cast<ValueType>(i + 1) * h;
        values[i] = -f(xi) * h * h;
    }
    values[0] += u0;
    values[discretization_points - 1] += u1;
}
```

Prints the solution `u`.

```
template <typename Closure, typename ValueType>
void print_solution(ValueType u0, ValueType u1,
                    const gko::matrix::Dense<ValueType>* u)
{
    std::cout « u0 « '\n';
    for (int i = 0; i < u->get_size()[0]; ++i) {
        std::cout « u->get_const_values()[i] « '\n';
    }
    std::cout « u1 « std::endl;
}
```

Computes the 1-norm of the error given the computed `u` and the correct solution function `correct_u`.

```
template <typename Closure, typename ValueType>
gko::remove_complex<ValueType> calculate_error(
    int discretization_points, const gko::matrix::Dense<ValueType>* u,
    Closure correct_u)
{
    const ValueType h = 1.0 / static_cast<ValueType>(discretization_points + 1);
    gko::remove_complex<ValueType> error = 0.0;
    for (int i = 0; i < discretization_points; ++i) {
        using std::abs;
        const auto xi = static_cast<ValueType>(i + 1) * h;
        error +=
            abs(u->get_const_values()[i] - correct_u(xi)) / abs(correct_u(xi));
    }
    return error;
}
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0]
              « " [executor] [DISCRETIZATION_POINTS]" « std::endl;
    std::exit(-1);
}
```

Get number of discretization points

```
const unsigned int discretization_points =
    argc >= 3 ?  std::atoi(argv[2]) :  100;
```

Get the executor string

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

executor used by the application

```
const auto app_exec = exec->get_master();
```

Set up the problem: define the exact solution, the right hand side and the Dirichlet boundary condition.

```
auto correct_u = [](ValueType x) { return x * x * x; };
auto f = [](ValueType x) { return ValueType(6) * x; };
auto u0 = correct_u(0);
auto u1 = correct_u(1);
```

initialize matrix and vectors

```
auto matrix = mtx::create(app_exec, gko::dim<2>(discretization_points),
                          3 * discretization_points - 2);
generate_stencil_matrix(matrix.get());
auto rhs = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
generate_rhs(f, u0, u1, rhs.get());
auto u = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
for (int i = 0; i < u->get_size()[0]; ++i) {
    u->get_values()[i] = 0.0;
}
const gko::remove_complex<ValueType> reduction_factor = 1e-7;
```

Generate solver and solve the system

```
cg::build()
    .with_criteria(
        gko::stop::Iteration::build().with_max_iters(discretization_points),
        gko::stop::ResidualNorm<ValueType>::build().with_reduction_factor(
            reduction_factor))
    .with_preconditioner(bj::build())
    .on(exec)
    ->generate(clone(exec, matrix))  // copy the matrix to the executor
    ->apply(rhs, u);
```

Uncomment to print the solution print_solution<ValueType>(u0, u1, u.get());

```
    std::cout « "Solve complete.\nThe average relative error is "
              « calculate_error(discretization_points, u.get(), correct_u) /
                    static_cast<gko::remove_complex<ValueType>>(
                        discretization_points)
              « std::endl;
}
```

# Results

This is the expected output:

```
Solve complete.
The average relative error is 2.52236e-11
```

### Comments about programming and debugging

# The plain program

```
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <ginkgo/ginkgo.hpp>
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(gko::matrix::Csr<ValueType, IndexType>* matrix)
{
    const auto discretization_points = matrix->get_size()[0];
    auto row_ptrs = matrix->get_row_ptrs();
    auto col_idxs = matrix->get_col_idxs();
    auto values = matrix->get_values();
    int pos = 0;
    const ValueType coefs[] = {-1, 2, -1};
    row_ptrs[0] = pos;
    for (int i = 0; i < discretization_points; ++i) {
        for (auto ofs :  {-1, 0, 1}) {
            if (0 <= i + ofs && i + ofs < discretization_points) {
                values[pos] = coefs[ofs + 1];
                col_idxs[pos] = i + ofs;
                ++pos;
            }
        }
        row_ptrs[i + 1] = pos;
    }
}
template <typename Closure, typename ValueType>
void generate_rhs(Closure f, ValueType u0, ValueType u1,
                  gko::matrix::Dense<ValueType>* rhs)
{
    const auto discretization_points = rhs->get_size()[0];
    auto values = rhs->get_values();
    const ValueType h = 1.0 / static_cast<ValueType>(discretization_points + 1);
    for (gko::size_type i = 0; i < discretization_points; ++i) {
        const auto xi = static_cast<ValueType>(i + 1) * h;
        values[i] = -f(xi) * h * h;
    }
    values[0] += u0;
    values[discretization_points - 1] += u1;
}
template <typename Closure, typename ValueType>
void print_solution(ValueType u0, ValueType u1,
                    const gko::matrix::Dense<ValueType>* u)
{
    std::cout « u0 « '\n';
    for (int i = 0; i < u->get_size()[0]; ++i) {
        std::cout « u->get_const_values()[i] « '\n';
    }
    std::cout « u1 « std::endl;
}
template <typename Closure, typename ValueType>
gko::remove_complex<ValueType> calculate_error(
    int discretization_points, const gko::matrix::Dense<ValueType>* u,
    Closure correct_u)
{
    const ValueType h = 1.0 / static_cast<ValueType>(discretization_points + 1);
    gko::remove_complex<ValueType> error = 0.0;
    for (int i = 0; i < discretization_points; ++i) {
        using std::abs;
        const auto xi = static_cast<ValueType>(i + 1) * h;
        error +=
            abs(u->get_const_values()[i] - correct_u(xi)) / abs(correct_u(xi));
    }
    return error;
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
```

```
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0]
                  « " [executor] [DISCRETIZATION_POINTS]" « std::endl;
        std::exit(-1);
    }
    const unsigned int discretization_points =
        argc >= 3 ?  std::atoi(argv[2]) :  100;
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    const auto app_exec = exec->get_master();
    auto correct_u = [](ValueType x) { return x * x * x; };
    auto f = [](ValueType x) { return ValueType(6) * x; };
    auto u0 = correct_u(0);
    auto u1 = correct_u(1);
    auto matrix = mtx::create(app_exec, gko::dim<2>(discretization_points),
                              3 * discretization_points - 2);
    generate_stencil_matrix(matrix.get());
    auto rhs = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
    generate_rhs(f, u0, u1, rhs.get());
    auto u = vec::create(app_exec, gko::dim<2>(discretization_points, 1));
    for (int i = 0; i < u->get_size()[0]; ++i) {
        u->get_values()[i] = 0.0;
    }
    const gko::remove_complex<ValueType> reduction_factor = 1e-7;
    cg::build()
        .with_criteria(
            gko::stop::Iteration::build().with_max_iters(discretization_points),
            gko::stop::ResidualNorm<ValueType>::build().with_reduction_factor(
                reduction_factor))
        .with_preconditioner(bj::build())
        .on(exec)
        ->generate(clone(exec, matrix))  // copy the matrix to the executor
        ->apply(rhs, u);
    std::cout « "Solve complete.\nThe average relative error is "
              « calculate_error(discretization_points, u.get(), correct_u) /
                    static_cast<gko::remove_complex<ValueType>>(
                        discretization_points)
              « std::endl;
}
```

# Chapter 39

# The preconditioned-solver program

The preconditioned solver example..

This example depends on simple-solver.

## Introduction

### About the example

## The commented program

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
```

Figure out where to run the code

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
```

```
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
const RealValueType reduction_factor{1e-7};
```

Create solver factory

```
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
```

Add preconditioner, these 2 lines are the only difference from the simple solver example

```
.with_preconditioner(bj::build().with_max_block_size(8u))
.on(exec);
```

Create solver

```
auto solver = solver_gen->generate(A);
```

Solve system

```
solver->apply(b, x);
```

Print solution

```
std::cout << "Solution (x):\n";
write(std::cout, x);
```

Calculate residual

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout << "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Results

This is the expected output:

```
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
4.82005e-08
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    const RealValueType reduction_factor{1e-7};
    auto solver_gen =
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .with_preconditioner(bj::build().with_max_block_size(8u))
            .on(exec);
    auto solver = solver_gen->generate(A);
    solver->apply(b, x);
    std::cout « "Solution (x):\n";
    write(std::cout, x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Chapter 40

# The preconditioner-export program

The preconditioner export example..

This example depends on simple-solver.

## Introduction

### About the example

This example shows how to explicitly generate and store preconditioners for a given matrix. It can also be used to inspect and debug the preconditioner generation.

## The commented program

```
#include <fstream>
#include <functional>
#include <iostream>
#include <map>
#include <memory>
#include <string>
#include <ginkgo/ginkgo.hpp>
const std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    executors{{"reference", [] { return gko::ReferenceExecutor::create(); }},
              {"omp", [] { return gko::OmpExecutor::create(); }},
              {"cuda",
               [] {
                   return gko::CudaExecutor::create(
                       0, gko::ReferenceExecutor::create());
               }},
              {"hip",
               [] {
                   return gko::HipExecutor::create(
                       0, gko::ReferenceExecutor::create());
               }},
              {"dpcpp", [] {
                   return gko::DpcppExecutor::create(
                       0, gko::ReferenceExecutor::create());
               }}};
void output(gko::ptr_param<const gko::WritableToMatrixData<double, int>> mtx,
            std::string name)
{
    std::ofstream stream{name};
    std::cerr « "Writing " « name « std::endl;
    gko::write(stream, mtx, gko::layout_type::coordinate);
}
template <typename Function>
auto try_generate(Function fun) -> decltype(fun())
{
```

```
    decltype(fun()) result;
    try {
        result = fun();
    } catch (const gko::Error& err) {
        std::cerr « "Error:  " « err.what() « '\n';
        std::exit(-1);
    }
    return result;
}
int main(int argc, char* argv[])
{
```

print usage message

```
if (argc < 2 || executors.find(argv[1]) == executors.end()) {
    std::cerr « "Usage:  executable"
             « " <reference|omp|cuda|hip|dpcpp> [<matrix-file>] "
                "[<jacobi|ilu|parilu|parilut|ilu-isai|parilu-isai|parilut-"
                "isai] [<preconditioner args>]\n";
    std::cerr « "Jacobi parameters:  [<max-block-size>] [<accuracy>] "
                "[<storage-optimization:auto|0|1|2>]\n";
    std::cerr « "ParILU parameters:  [<iteration-count>]\n";
    std::cerr
        « "ParILUT parameters:  [<iteration-count>] [<fill-in-limit>]\n";
    std::cerr « "ILU-ISAI parameters:  [<sparsity-power>]\n";
    std::cerr « "ParILU-ISAI parameters:  [<iteration-count>] "
                "[<sparsity-power>]\n";
    std::cerr « "ParILUT-ISAI parameters:  [<iteration-count>] "
                "[<fill-in-limit>] [<sparsity-power>]\n";
    return -1;
}
```

generate executor based on first argument

```
auto exec = try_generate([&] { return executors.at(argv[1])(); });
```

set matrix and preconditioner name with default values

```
std::string matrix = argc < 3 ?  "data/A.mtx" :  argv[2];
std::string precond = argc < 4 ?  "jacobi" :  argv[3];
```

load matrix file into Csr format

```
auto mtx = gko::share(try_generate([&] {
    std::ifstream mtx_stream{matrix};
    if (!mtx_stream) {
        throw GKO_STREAM_ERROR("Unable to open matrix file");
    }
    std::cerr « "Reading " « matrix « std::endl;
    return gko::read<gko::matrix::Csr<»(mtx_stream, exec);
}));
```

concatenate remaining arguments for filename

```
std::string output_suffix;
for (auto i = 4; i < argc; ++i) {
    output_suffix = output_suffix + "-" + argv[i];
}
using LowerIsai = gko::preconditioner::LowerIsai<>;
using UpperIsai = gko::preconditioner::UpperIsai<>;
```

handle different preconditioners

```
if (precond == "jacobi") {
```

jacobi: max_block_size, accuracy, storage_optimization

```
    auto factory_parameter = gko::preconditioner::Jacobi<>::build();
    if (argc >= 5) {
        factory_parameter.with_max_block_size(
            static_cast<gko::uint32>(std::stoi(argv[4])));
    }
    if (argc >= 6) {
        factory_parameter.with_accuracy(std::stod(argv[5]));
    }
    if (argc >= 7) {
        factory_parameter.with_storage_optimization(
            std::string{argv[6]} == "auto"
                ?  gko::precision_reduction::autodetect()
                :  gko::precision_reduction(0, std::stoi(argv[6])));
    }
    auto factory = factory_parameter.on(exec);
    auto jacobi = try_generate([&] { return factory->generate(mtx); });
    output(jacobi, matrix + ".jacobi" + output_suffix);
} else if (precond == "ilu") {
```

ilu: no parameters

```
    auto ilu = gko::as<gko::Composition<>>(try_generate([&] {
        return gko::factorization::Ilu<>::build().on(exec)->generate(mtx);
    }));
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[0]),
           matrix + ".ilu-l");
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[1]),
           matrix + ".ilu-u");
} else if (precond == "parilu") {
```

### parilu: iterations

```
    auto factory_parameter = gko::factorization::ParIlu<>::build();
    if (argc >= 5) {
        factory_parameter.with_iterations(
            static_cast<gko::size_type>(std::stoi(argv[4])));
    }
    auto factory = factory_parameter.on(exec);
    auto ilu = gko::as<gko::Composition<>>(
        try_generate([&] { return factory->generate(mtx); }));
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[0]),
           matrix + ".parilu" + output_suffix + "-l");
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[1]),
           matrix + ".parilu" + output_suffix + "-u");
} else if (precond == "parilut") {
```

### parilut: iterations, fill-in limit

```
    auto factory_parameter = gko::factorization::ParIlut<>::build();
    if (argc >= 5) {
        factory_parameter.with_iterations(
            static_cast<gko::size_type>(std::stoi(argv[4])));
    }
    if (argc >= 6) {
        factory_parameter.with_fill_in_limit(std::stod(argv[5]));
    }
    auto factory = factory_parameter.on(exec);
    auto ilut = gko::as<gko::Composition<>>(
        try_generate([&] { return factory->generate(mtx); }));
    output(gko::as<gko::matrix::Csr<>>(ilut->get_operators()[0]),
           matrix + ".parilut" + output_suffix + "-l");
    output(gko::as<gko::matrix::Csr<>>(ilut->get_operators()[1]),
           matrix + ".parilut" + output_suffix + "-u");
} else if (precond == "ilu-isai") {
```

### ilu-isai: sparsity power

```
    auto fact_factory =
        gko::share(gko::factorization::Ilu<>::build().on(exec));
    int sparsity_power = 1;
    if (argc >= 5) {
        sparsity_power = std::stoi(argv[4]);
    }
    auto factory =
        gko::preconditioner::Ilu<>::build()
            .with_factorization(fact_factory)
            .with_l_solver(
                LowerIsai::build().with_sparsity_power(sparsity_power))
            .with_u_solver(
                UpperIsai::build().with_sparsity_power(sparsity_power))
            .on(exec);
    auto ilu_isai = try_generate([&] { return factory->generate(mtx); });
    output(gko::as<LowerIsai>(ilu_isai->get_l_solver())
               ->get_approximate_inverse(),
           matrix + ".ilu-isai" + output_suffix + "-l");
    output(gko::as<UpperIsai>(ilu_isai->get_u_solver())
               ->get_approximate_inverse(),
           matrix + ".ilu-isai" + output_suffix + "-u");
} else if (precond == "parilu-isai") {
```

### parilu-isai: iterations, sparsity power

```
    auto fact_parameter = gko::factorization::ParIlu<>::build();
    int sparsity_power = 1;
    if (argc >= 5) {
        fact_parameter.with_iterations(
            static_cast<gko::size_type>(std::stoi(argv[4])));
    }
    if (argc >= 6) {
        sparsity_power = std::stoi(argv[5]);
    }
    auto fact_factory = gko::share(fact_parameter.on(exec));
    auto factory =
        gko::preconditioner::Ilu<>::build()
            .with_factorization(fact_factory)
            .with_l_solver(
                LowerIsai::build().with_sparsity_power(sparsity_power))
            .with_u_solver(
```

```
                UpperIsai::build().with_sparsity_power(sparsity_power))
            .on(exec);
    auto ilu_isai = try_generate([&] { return factory->generate(mtx); });
    output(gko::as<LowerIsai>(ilu_isai->get_l_solver())
               ->get_approximate_inverse(),
           matrix + ".parilu-isai" + output_suffix + "-l");
    output(gko::as<UpperIsai>(ilu_isai->get_u_solver())
               ->get_approximate_inverse(),
           matrix + ".parilu-isai" + output_suffix + "-u");
} else if (precond == "parilut-isai") {
```

parilut-isai: iterations, fill-in limit, sparsity power

```
        auto fact_parameter = gko::factorization::ParIlut<>::build();
        int sparsity_power = 1;
        if (argc >= 5) {
            fact_parameter.with_iterations(
                static_cast<gko::size_type>(std::stoi(argv[4])));
        }
        if (argc >= 6) {
            fact_parameter.with_fill_in_limit(std::stod(argv[5]));
        }
        if (argc >= 7) {
            sparsity_power = std::stoi(argv[6]);
        }
        auto fact_factory = gko::share(fact_parameter.on(exec));
        auto factory =
            gko::preconditioner::Ilu<>::build()
                .with_factorization(fact_factory)
                .with_l_solver(
                    LowerIsai::build().with_sparsity_power(sparsity_power))
                .with_u_solver(
                    UpperIsai::build().with_sparsity_power(sparsity_power))
                .on(exec);
        auto ilu_isai = try_generate([&] { return factory->generate(mtx); });
        output(gko::as<LowerIsai>(ilu_isai->get_l_solver())
                   ->get_approximate_inverse(),
               matrix + ".parilut-isai" + output_suffix + "-l");
        output(gko::as<UpperIsai>(ilu_isai->get_u_solver())
                   ->get_approximate_inverse(),
               matrix + ".parilut-isai" + output_suffix + "-u");
    }
}
```

# Results

This is the expected output:

```
Usage:  ./preconditioner-export <reference|omp|cuda|hip|dpcpp> [<matrix-file>]
      [<jacobi|ilu|parilu|parilut|ilu-isai|parilu-isai|parilut-isai] [<preconditioner args>]
Jacobi parameters:  [<max-block-size>] [<accuracy>] [<storage-optimization:auto|0|1|2>]
ParILU parameters:  [<iteration-count>]
ParILUT parameters:  [<iteration-count>] [<fill-in-limit>]
ILU-ISAI parameters:  [<sparsity-power>]
ParILU-ISAI parameters:  [<iteration-count>] [<sparsity-power>]
ParILUT-ISAI parameters:  [<iteration-count>] [<fill-in-limit>] [<sparsity-power>]
```

When specifying an executor:

```
Reading data/A.mtx
Writing data/A.mtx.jacobi
```

### Comments about programming and debugging

# The plain program

```
#include <fstream>
#include <functional>
#include <iostream>
#include <map>
#include <memory>
#include <string>
#include <ginkgo/ginkgo.hpp>
const std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    executors{{"reference", [] { return gko::ReferenceExecutor::create(); }},
              {"omp", [] { return gko::OmpExecutor::create(); }},
              {"cuda",
```

```
                [] {
                    return gko::CudaExecutor::create(
                        0, gko::ReferenceExecutor::create());
                }},
               {"hip",
                [] {
                    return gko::HipExecutor::create(
                        0, gko::ReferenceExecutor::create());
                }},
               {"dpcpp", [] {
                    return gko::DpcppExecutor::create(
                        0, gko::ReferenceExecutor::create());
                }}};
void output(gko::ptr_param<const gko::WritableToMatrixData<double, int>> mtx,
            std::string name)
{
    std::ofstream stream{name};
    std::cerr « "Writing " « name « std::endl;
    gko::write(stream, mtx, gko::layout_type::coordinate);
}
template <typename Function>
auto try_generate(Function fun) -> decltype(fun())
{
    decltype(fun()) result;
    try {
        result = fun();
    } catch (const gko::Error& err) {
        std::cerr « "Error:  " « err.what() « '\n';
        std::exit(-1);
    }
    return result;
}
int main(int argc, char* argv[])
{
    if (argc < 2 || executors.find(argv[1]) == executors.end()) {
        std::cerr « "Usage:  executable"
                 « " <reference|omp|cuda|hip|dpcpp> [<matrix-file>] "
                   "[<jacobi|ilu|parilu|parilut|ilu-isai|parilu-isai|parilut-"
                   "isai] [<preconditioner args>]\n";
        std::cerr « "Jacobi parameters:  [<max-block-size>] [<accuracy>] "
                   "[<storage-optimization:auto|0|1|2>]\n";
        std::cerr « "ParILU parameters:  [<iteration-count>]\n";
        std::cerr
            « "ParILUT parameters:  [<iteration-count>] [<fill-in-limit>]\n";
        std::cerr « "ILU-ISAI parameters:  [<sparsity-power>]\n";
        std::cerr « "ParILU-ISAI parameters:  [<iteration-count>] "
                   "[<sparsity-power>]\n";
        std::cerr « "ParILUT-ISAI parameters:  [<iteration-count>] "
                   "[<fill-in-limit>] [<sparsity-power>]\n";
        return -1;
    }
    auto exec = try_generate([&] { return executors.at(argv[1])(); });
    std::string matrix = argc < 3 ?  "data/A.mtx" :  argv[2];
    std::string precond = argc < 4 ?  "jacobi" :  argv[3];
    auto mtx = gko::share(try_generate([&] {
        std::ifstream mtx_stream{matrix};
        if (!mtx_stream) {
            throw GKO_STREAM_ERROR("Unable to open matrix file");
        }
        std::cerr « "Reading " « matrix « std::endl;
        return gko::read<gko::matrix::Csr<»(mtx_stream, exec);
    }));
    std::string output_suffix;
    for (auto i = 4; i < argc; ++i) {
        output_suffix = output_suffix + "-" + argv[i];
    }
    using LowerIsai = gko::preconditioner::LowerIsai<>;
    using UpperIsai = gko::preconditioner::UpperIsai<>;
    if (precond == "jacobi") {
        auto factory_parameter = gko::preconditioner::Jacobi<>::build();
        if (argc >= 5) {
            factory_parameter.with_max_block_size(
                static_cast<gko::uint32>(std::stoi(argv[4])));
        }
        if (argc >= 6) {
            factory_parameter.with_accuracy(std::stod(argv[5]));
        }
        if (argc >= 7) {
            factory_parameter.with_storage_optimization(
                std::string{argv[6]} == "auto"
                    ?  gko::precision_reduction::autodetect()
                    :  gko::precision_reduction(0, std::stoi(argv[6])));
        }
        auto factory = factory_parameter.on(exec);
        auto jacobi = try_generate([&] { return factory->generate(mtx); });
        output(jacobi, matrix + ".jacobi" + output_suffix);
    } else if (precond == "ilu") {
```

```
    auto ilu = gko::as<gko::Composition<>>(try_generate([&] {
        return gko::factorization::Ilu<>::build().on(exec)->generate(mtx);
    }));
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[0]),
           matrix + ".ilu-l");
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[1]),
           matrix + ".ilu-u");
} else if (precond == "parilu") {
    auto factory_parameter = gko::factorization::ParIlu<>::build();
    if (argc >= 5) {
        factory_parameter.with_iterations(
            static_cast<gko::size_type>(std::stoi(argv[4])));
    }
    auto factory = factory_parameter.on(exec);
    auto ilu = gko::as<gko::Composition<>>(
        try_generate([&] { return factory->generate(mtx); }));
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[0]),
           matrix + ".parilu" + output_suffix + "-l");
    output(gko::as<gko::matrix::Csr<>>(ilu->get_operators()[1]),
           matrix + ".parilu" + output_suffix + "-u");
} else if (precond == "parilut") {
    auto factory_parameter = gko::factorization::ParIlut<>::build();
    if (argc >= 5) {
        factory_parameter.with_iterations(
            static_cast<gko::size_type>(std::stoi(argv[4])));
    }
    if (argc >= 6) {
        factory_parameter.with_fill_in_limit(std::stod(argv[5]));
    }
    auto factory = factory_parameter.on(exec);
    auto ilut = gko::as<gko::Composition<>>(
        try_generate([&] { return factory->generate(mtx); }));
    output(gko::as<gko::matrix::Csr<>>(ilut->get_operators()[0]),
           matrix + ".parilut" + output_suffix + "-l");
    output(gko::as<gko::matrix::Csr<>>(ilut->get_operators()[1]),
           matrix + ".parilut" + output_suffix + "-u");
} else if (precond == "ilu-isai") {
    auto fact_factory =
        gko::share(gko::factorization::Ilu<>::build().on(exec));
    int sparsity_power = 1;
    if (argc >= 5) {
        sparsity_power = std::stoi(argv[4]);
    }
    auto factory =
        gko::preconditioner::Ilu<>::build()
            .with_factorization(fact_factory)
            .with_l_solver(
                LowerIsai::build().with_sparsity_power(sparsity_power))
            .with_u_solver(
                UpperIsai::build().with_sparsity_power(sparsity_power))
            .on(exec);
    auto ilu_isai = try_generate([&] { return factory->generate(mtx); });
    output(gko::as<LowerIsai>(ilu_isai->get_l_solver())
               ->get_approximate_inverse(),
           matrix + ".ilu-isai" + output_suffix + "-l");
    output(gko::as<UpperIsai>(ilu_isai->get_u_solver())
               ->get_approximate_inverse(),
           matrix + ".ilu-isai" + output_suffix + "-u");
} else if (precond == "parilu-isai") {
    auto fact_parameter = gko::factorization::ParIlu<>::build();
    int sparsity_power = 1;
    if (argc >= 5) {
        fact_parameter.with_iterations(
            static_cast<gko::size_type>(std::stoi(argv[4])));
    }
    if (argc >= 6) {
        sparsity_power = std::stoi(argv[5]);
    }
    auto fact_factory = gko::share(fact_parameter.on(exec));
    auto factory =
        gko::preconditioner::Ilu<>::build()
            .with_factorization(fact_factory)
            .with_l_solver(
                LowerIsai::build().with_sparsity_power(sparsity_power))
            .with_u_solver(
                UpperIsai::build().with_sparsity_power(sparsity_power))
            .on(exec);
    auto ilu_isai = try_generate([&] { return factory->generate(mtx); });
    output(gko::as<LowerIsai>(ilu_isai->get_l_solver())
               ->get_approximate_inverse(),
           matrix + ".parilu-isai" + output_suffix + "-l");
    output(gko::as<UpperIsai>(ilu_isai->get_u_solver())
               ->get_approximate_inverse(),
           matrix + ".parilu-isai" + output_suffix + "-u");
} else if (precond == "parilut-isai") {
    auto fact_parameter = gko::factorization::ParIlut<>::build();
    int sparsity_power = 1;
```

```
        if (argc >= 5) {
            fact_parameter.with_iterations(
                static_cast<gko::size_type>(std::stoi(argv[4])));
        }
        if (argc >= 6) {
            fact_parameter.with_fill_in_limit(std::stod(argv[5]));
        }
        if (argc >= 7) {
            sparsity_power = std::stoi(argv[6]);
        }
        auto fact_factory = gko::share(fact_parameter.on(exec));
        auto factory =
            gko::preconditioner::Ilu<>::build()
                .with_factorization(fact_factory)
                .with_l_solver(
                    LowerIsai::build().with_sparsity_power(sparsity_power))
                .with_u_solver(
                    UpperIsai::build().with_sparsity_power(sparsity_power))
                .on(exec);
        auto ilu_isai = try_generate([&] { return factory->generate(mtx); });
        output(gko::as<LowerIsai>(ilu_isai->get_l_solver())
                   ->get_approximate_inverse(),
               matrix + ".parilut-isai" + output_suffix + "-l");
        output(gko::as<UpperIsai>(ilu_isai->get_u_solver())
                   ->get_approximate_inverse(),
               matrix + ".parilut-isai" + output_suffix + "-u");
    }
}
```

# Chapter 41

# The reordered-preconditioned-solver program

The reordered preconditioned solver example..

This example depends on preconditioned-solver.

## Introduction

### About the example

This uses an RCM reordering on an input matrix to construct a ParILU preconditioned solver and solve a linear system.

## The commented program

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using ilu = gko::preconditioner::Ilu<ValueType, false, IndexType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
```

Figure out where to run the code

```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
```

Figure out where to run the code

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
```

```
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
auto reordering =
    gko::experimental::reorder::Rcm<IndexType>::build().on(exec)->generate(
        A);
```

Permute matrix and vectors

```
auto A_reordered = share(A->permute(reordering));
auto b_reordered = b->permute(reordering, gko::matrix::permute_mode::rows);
```

this reordering is not necessary, but it maps the initial guess to the unreordered case

```
auto x_reordered = x->permute(reordering, gko::matrix::permute_mode::rows);
const RealValueType reduction_factor{1e-7};
```

Create solver factory

```
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
        .with_preconditioner(ilu::build())
        .on(exec);
```

Create solver

```
auto solver = solver_gen->generate(A_reordered);
```

Solve system

```
solver->apply(b_reordered, x_reordered);
```

Revert permutation to get the unpermuted solution

```
x_reordered->permute(reordering, x,
                     gko::matrix::permute_mode::inverse_rows);
```

Print solution

```
std::cout << "Solution (x):\n";
write(std::cout, x);
```

Calculate residual

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout << "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

## Results

This is the expected output:

```
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
4.82005e-08
```

## The plain program

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using ilu = gko::preconditioner::Ilu<ValueType, false, IndexType>;
    std::cout << gko::version_info::get() << std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr << "Usage:  " << argv[0] << " [executor]" << std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()>
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    auto reordering =
        gko::experimental::reorder::Rcm<IndexType>::build().on(exec)->generate(
            A);
    auto A_reordered = share(A->permute(reordering));
    auto b_reordered = b->permute(reordering, gko::matrix::permute_mode::rows);
    auto x_reordered = x->permute(reordering, gko::matrix::permute_mode::rows);
```

```
    const RealValueType reduction_factor{1e-7};
    auto solver_gen =
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .with_preconditioner(ilu::build())
            .on(exec);
    auto solver = solver_gen->generate(A_reordered);
    solver->apply(b_reordered, x_reordered);
    x_reordered->permute(reordering, x,
                         gko::matrix::permute_mode::inverse_rows);
    std::cout « "Solution (x):\n";
    write(std::cout, x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Chapter 42

# The schroedinger-splitting program

The Schroedinger equation example..

This example depends on heat-equation.

## Introduction

This example shows how to use the FFT and iFFT implementations in Ginkgo to solve the non-linear Schrödinger equation with a splitting method.

The non-linear Schrödinger equation (NLS) is given by

$i\partial_t \theta = -\delta\theta + |\theta|^2\theta$

Here $\theta$ is the wave function of a single particle in two dimensions. Its magnitude $|\theta|^2$ describes the probability distribution of the particle's position.

This equation can be split in to its linear (1) and non-linear (2) part

$$\begin{aligned} (1) \quad & i\partial_t \theta = -\delta\theta \\ (2) \quad & i\partial_t \theta = |\theta|^2\theta \end{aligned}$$

For both of these equations, we can compute exact solutions, assuming periodic boundary conditions and using the Fourier series expansion for (1) and using the fact that $|\theta|^2$ is constant in (2):

$$\begin{aligned} (\hat{1}) \quad & \partial_t \hat{\theta}_k = -i|k|^2\theta \\ (2') \quad & \partial_t |\theta|^2 = i|\theta|^2(\theta - \theta) = 0 \end{aligned}$$

The exact solutions are then given by

$$\begin{aligned} (\hat{1}) \quad & \hat{\theta}(t) = e^{-i|k|^2 t}\hat{\theta}(0) \\ (2') \quad & \theta(t) = e^{-i|\theta|^2 t}\theta(0) \end{aligned}$$

These partial solutions can be used to approximate a solution to the full NLS by alternating between small time steps for (1) and (2).

For nicer visual results, we add another constant potential term V(x) \theta to the non-linear part, which turns it into the Gross–Pitaevskii equation.

### About the example

## The commented program

```
/ ****************************<DESCRIPTION>**********************************
This example shows how to use the FFT and iFFT implementations in Ginkgo
to solve the non-linear Schrödinger equation with a splitting method.
The non-linear Schrödinger equation (NLS) is given by
\f
    i \partial_t \theta = -\delta \theta + |\theta|^2 \theta
\f
Here \f\theta\f is the wave function of a single particle in two dimensions.
Its magnitude \f|\theta|^2\f describes the probability distribution of the
particle's position.

This equation can be split in to its linear (1) and non-linear (2) part

\f{align*}{
(1) \quad i \partial_t \theta &= -\delta \theta\\
(2) \quad i \partial_t \theta &= |\theta|^2 \theta
\f}

For both of these equations, we can compute exact solutions, assuming periodic
boundary conditions and using the Fourier series expansion for (1) and using the
fact that \f| \theta |^2\f is constant in (2):

\f{align*}{
(\hat 1) \quad \quad \partial_t \hat\theta_k &= -i |k|^2 \theta \\
(2') \quad \partial_t |\theta|^2 &= i |\theta|^2 (\theta - \theta) = 0
\f}
The exact solutions are then given by
\f{align*}{
    (\hat 1) \quad \hat\theta(t) &= e^{-i |k|^2 t} \hat\theta(0)\\
    (2') \quad \theta(t) &= e^{-i |\theta|^2 t} \theta(0)
\f}

These partial solutions can be used to approximate a solution to the full NLS
by alternating between small time steps for (1) and (2).

For nicer visual results, we add another constant potential term V(x) \theta
to the non-linear part, which turns it into the Gross-Pitaevskii equation.

****************************<DESCRIPTION>********************************** /

#include <algorithm>
#include <chrono>
#include <fstream>
#include <iostream>
#include <utility>

#include <opencv2/core.hpp>
#include <opencv2/videoio.hpp>

#include <ginkgo/ginkgo.hpp>
```

This function implements a simple Ginkgo-themed clamped color mapping for values in the range [0,5].

```
void set_val(unsigned char* data, double value)
{
```

RGB values for the 6 colors used for values 0, 1, ..., 5 We will interpolate linearly between these values.

```
double col_r[] = {255, 221, 129, 201, 249, 255};
double col_g[] = {255, 220, 130, 161, 158, 204};
double col_b[] = {255, 220, 133, 93, 24, 8};
value = std::max(0.0, value);
auto i = std::max(0, std::min(4, int(value)));
auto d = std::max(0.0, std::min(1.0, value - i));
```

OpenCV uses BGR instead of RGB by default, revert indices

```
    data[2] = static_cast<unsigned char>(col_r[i + 1] * d + col_r[i] * (1 - d));
    data[1] = static_cast<unsigned char>(col_g[i + 1] * d + col_g[i] * (1 - d));
    data[0] = static_cast<unsigned char>(col_b[i + 1] * d + col_b[i] * (1 - d));
}
```

Initialize video output with given dimension and FPS (frames per seconds)

```
std::pair<cv::VideoWriter, cv::Mat> build_output(int n, double fps)
{
    cv::Size videosize{n, n};
    auto output =
        std::make_pair(cv::VideoWriter{}, cv::Mat{videosize, CV_8UC3});
    auto fourcc = cv::VideoWriter::fourcc('a', 'v', 'c', '1');
```

```
    output.first.open("nls.mp4", fourcc, fps, videosize);
    return output;
}
```

Write the current frame to video output using the above color mapping

```
void output_timestep(std::pair<cv::VideoWriter, cv::Mat>& output, int n,
                     const std::complex<double>* data)
{
    for (int i = 0; i < n; i++) {
        auto row = output.second.ptr(i);
        for (int j = 0; j < n; j++) {
            set_val(&row[3 * j], abs(data[i * n + j]));
        }
    }
    output.first.write(output.second);
}
int main(int argc, char* argv[])
{
    using vec = gko::matrix::Dense<std::complex<double>>;
    using real_vec = gko::matrix::Dense<double>;
    using fft2 = gko::matrix::Fft2;
```

Problem parameters: simulation length

```
const auto t0 = 15.0;
```

scaling factor for non-linearity

```
const auto nonlinear_scale = 1.0;
```

scaling factor for potential

```
const auto potential_scale = 3.0;
```

Simulation parameters: time scaling factor

```
const auto time_scale = 0.25;
```

number of grid points in each dimension

```
const auto n = 256;
```

number of simulation steps per second

```
const auto steps_per_sec = 1000;
```

number of video frames per second

```
const auto fps = 25;
```

number of grid points

```
const auto n2 = n * n;
```

phase difference between neighboring grid points

```
const auto h = 2.0 * gko::pi<double>() / n;
const auto h2 = h * h;
```

time step size for the simulation

```
const auto tau = 1.0 / steps_per_sec;
const auto idx = [&](int i, int j) { return i * n + j; };
```

create an OpenMP executor

```
auto exec = gko::OmpExecutor::create();
```

load initial state vector

```
std::ifstream initial_stream("data/gko_logo_2d.mtx");
std::ifstream potential_stream("data/gko_text_2d.mtx");
auto amplitude = gko::read<vec>(initial_stream, exec);
auto potential = gko::read<real_vec>(potential_stream, exec);
```

create vector for frequency space representation

```
auto frequency = vec::create(exec, amplitude->get_size());
```

create Fourier matrix

```
auto fft = fft2::create(exec, n, n);
auto ifft = fft->conj_transpose();
```

prepare video output

```
auto output = build_output(n, fps);
```

time stamp of the last output frame (sentinel value)

```
double last_t = -t0;
```

execute splitting method: time step in linear part, then non-linear part

```
for (double t = 0; t < t0; t += tau) {
```

if enough time has passed, output the next frame

```
if (t - last_t > 1.0 / fps) {
    last_t = t;
    std::cout « t « std::endl;
    output_timestep(output, n, amplitude->get_const_values());
}
```

time step in linear part

```
fft->apply(amplitude, frequency);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        frequency->at(idx(i, j)) *=
            std::polar(1.0, -h2 * (i * i + j * j) * tau * time_scale);
```

scale by FFT∗iFFT normalization factor

```
        frequency->at(idx(i, j)) *= 1.0 / n2;
    }
}
ifft->apply(frequency, amplitude);
```

time step in non-linear part

```
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                amplitude->at(idx(i, j)) *= std::polar(
                    1.0, -(nonlinear_scale *
                                gko::squared_norm(amplitude->at(idx(i, j))) +
                           potential_scale * potential->at(idx(i, j))) *
                             tau * time_scale);
            }
        }
    }
}
```

# Results

The program will generate a video file named nls.mp4 and output the timestamp of each generated frame.

### Comments about programming and debugging

# The plain program

```
/*****************************<DESCRIPTION>**********************************
This example shows how to use the FFT and iFFT implementations in Ginkgo
to solve the non-linear Schrödinger equation with a splitting method.

The non-linear Schrödinger equation (NLS) is given by

\f$
i \partial_t \theta = -\delta \theta + |\theta|^2 \theta
\f$

Here \f$\theta\f$ is the wave function of a single particle in two dimensions.
Its magnitude \f$|\theta|^2\f$ describes the probability distribution of the
particle's position.

This equation can be split in to its linear (1) and non-linear (2) part

\f{align*}{
(1) \quad i \partial_t \theta &= -\delta \theta\\
(2) \quad i \partial_t \theta &= |\theta|^2 \theta
\f}

For both of these equations, we can compute exact solutions, assuming periodic
boundary conditions and using the Fourier series expansion for (1) and using the
fact that \f$| \theta |^2\f$ is constant in (2):
```

```
\f{align*}{
(\hat 1) \quad \quad \partial_t \hat\theta_k &= -i |k|^2 \theta \\
(2') \quad \partial_t |\theta|^2 &= i |\theta|^2 (\theta - \theta) = 0
\f}

The exact solutions are then given by

\f{align*}{
(\hat 1) \quad \hat\theta(t) &= e^{-i |k|^2 t} \hat\theta(0)\\
(2') \quad \theta(t) &= e^{-i |\theta|^2 t} \theta(0)
\f}

These partial solutions can be used to approximate a solution to the full NLS
by alternating between small time steps for (1) and (2).

For nicer visual results, we add another constant potential term V(x) \theta
to the non-linear part, which turns it into the Gross-Pitaevskii equation.

*****************************<DESCRIPTION>*********************************/
#include <algorithm>
#include <chrono>
#include <fstream>
#include <iostream>
#include <utility>
#include <opencv2/core.hpp>
#include <opencv2/videoio.hpp>
#include <ginkgo/ginkgo.hpp>
void set_val(unsigned char* data, double value)
{
    double col_r[] = {255, 221, 129, 201, 249, 255};
    double col_g[] = {255, 220, 130, 161, 158, 204};
    double col_b[] = {255, 220, 133, 93, 24, 8};
    value = std::max(0.0, value);
    auto i = std::max(0, std::min(4, int(value)));
    auto d = std::max(0.0, std::min(1.0, value - i));
    data[2] = static_cast<unsigned char>(col_r[i + 1] * d + col_r[i] * (1 - d));
    data[1] = static_cast<unsigned char>(col_g[i + 1] * d + col_g[i] * (1 - d));
    data[0] = static_cast<unsigned char>(col_b[i + 1] * d + col_b[i] * (1 - d));
}
std::pair<cv::VideoWriter, cv::Mat> build_output(int n, double fps)
{
    cv::Size videosize{n, n};
    auto output =
        std::make_pair(cv::VideoWriter{}, cv::Mat{videosize, CV_8UC3});
    auto fourcc = cv::VideoWriter::fourcc('a', 'v', 'c', '1');
    output.first.open("nls.mp4", fourcc, fps, videosize);
    return output;
}
void output_timestep(std::pair<cv::VideoWriter, cv::Mat>& output, int n,
                     const std::complex<double>* data)
{
    for (int i = 0; i < n; i++) {
        auto row = output.second.ptr(i);
        for (int j = 0; j < n; j++) {
            set_val(&row[3 * j], abs(data[i * n + j]));
        }
    }
    output.first.write(output.second);
}
int main(int argc, char* argv[])
{
    using vec = gko::matrix::Dense<std::complex<double>>;
    using real_vec = gko::matrix::Dense<double>;
    using fft2 = gko::matrix::Fft2;
    const auto t0 = 15.0;
    const auto nonlinear_scale = 1.0;
    const auto potential_scale = 3.0;
    const auto time_scale = 0.25;
    const auto n = 256;
    const auto steps_per_sec = 1000;
    const auto fps = 25;
    const auto n2 = n * n;
    const auto h = 2.0 * gko::pi<double>() / n;
    const auto h2 = h * h;
    const auto tau = 1.0 / steps_per_sec;
    const auto idx = [&](int i, int j) { return i * n + j; };
    auto exec = gko::OmpExecutor::create();
    std::ifstream initial_stream("data/gko_logo_2d.mtx");
    std::ifstream potential_stream("data/gko_text_2d.mtx");
    auto amplitude = gko::read<vec>(initial_stream, exec);
    auto potential = gko::read<real_vec>(potential_stream, exec);
    auto frequency = vec::create(exec, amplitude->get_size());
    auto fft = fft2::create(exec, n, n);
    auto ifft = fft->conj_transpose();
    auto output = build_output(n, fps);
    double last_t = -t0;
```

```
    for (double t = 0; t < t0; t += tau) {
        if (t - last_t > 1.0 / fps) {
            last_t = t;
            std::cout « t « std::endl;
            output_timestep(output, n, amplitude->get_const_values());
        }
        fft->apply(amplitude, frequency);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                frequency->at(idx(i, j)) *=
                    std::polar(1.0, -h2 * (i * i + j * j) * tau * time_scale);
                frequency->at(idx(i, j)) *= 1.0 / n2;
            }
        }
        ifft->apply(frequency, amplitude);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                amplitude->at(idx(i, j)) *= std::polar(
                    1.0, -(nonlinear_scale *
                                gko::squared_norm(amplitude->at(idx(i, j))) +
                           potential_scale * potential->at(idx(i, j))) *
                             tau * time_scale);
            }
        }
    }
}
```

# Chapter 43

# The simple-solver program

The simple solver example..

## Introduction

This simple solver example should help you get started with Ginkgo. This example is meant for you to understand how Ginkgo works and how you can solve a simple linear system with Ginkgo. We encourage you to play with the code, change the parameters and see what is best suited for your purposes.

#### About the example

Each example has the following sections:

1. **Introduction:**This gives an overview of the example and mentions any interesting aspects in the example that might help the reader.
2. **The commented program:** This section is intended for you to understand the details of the example so that you can play with it and understand Ginkgo and its features better.
3. **Results:** This section shows the results of the code when run. Though the results may not be completely the same, you can expect the behaviour to be similar.
4. **The plain program:** This is the complete code without any comments to have an complete overview of the code.

## The commented program

#### Include files

This is the main ginkgo header file.

```
#include <ginkgo/ginkgo.hpp>
```

Add the fstream header to read from data from files.

```
#include <fstream>
```

Add the C++ iostream header to output information to the console.
```
#include <iostream>
```

Add the STL map header for the executor selection
```
#include <map>
```

Add the string manipulation header to handle strings.
```
#include <string>
int main(int argc, char* argv[])
{
```

Use some shortcuts. In Ginkgo, vectors are seen as a gko::matrix::Dense with one column/one row. The advantage of this concept is that using multiple vectors is a now a natural extension of adding columns/rows are necessary.
```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using real_vec = gko::matrix::Dense<RealValueType>;
```

The gko::matrix::Csr class is used here, but any other matrix class such as gko::matrix::Coo, gko::matrix::Hybrid, gko::matrix::Ell or gko::matrix::Sellp could also be used.
```
using mtx = gko::matrix::Csr<ValueType, IndexType>;
```

The gko::solver::Cg is used here, but any other solver class can also be used.
```
using cg = gko::solver::Cg<ValueType>;
```

Print the ginkgo version information.
```
std::cout « gko::version_info::get() « std::endl;
```

Print help on how to execute this example.
```
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor] " « std::endl;
    std::exit(-1);
}
```

### Where do you want to run your solver ?

The gko::Executor class is one of the cornerstones of Ginkgo. Currently, we have support for an gko::OmpExecutor, which uses OpenMP multi-threading in most of its kernels, a gko::ReferenceExecutor, a single threaded specialization of the OpenMP executor and a gko::CudaExecutor which runs the code on a NVIDIA GPU if available.

#### Note

With the help of C++, you see that you only ever need to change the executor and all the other functions/routines within Ginkgo should automatically work and run on the executor with any other changes.

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation
```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

### Reading your data and transfer to the proper device.

Read the matrix, right hand side and the initial solution using the read function.

**Note**

Ginkgo uses C++ smart pointers to automatically manage memory. To this end, we use our own object ownership transfer functions that under the hood call the required smart pointer functions to manage object ownership. gko::share and gko::give are the functions that you would need to use.

```
auto A = gko::share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
```

### Creating the solver

Generate the gko::solver factory. Ginkgo uses the concept of Factories to build solvers with certain properties. Observe the Fluent interface used here. Here a cg solver is generated with a stopping criteria of maximum iterations of 20 and a residual norm reduction of 1e-7. You also observe that the stopping criteria(gko::stop) are also generated from factories using their build methods. You need to specify the executors which each of the object needs to be built on.

```
const RealValueType reduction_factor{1e-7};
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
        .on(exec);
```

Generate the solver from the matrix. The solver factory built in the previous step takes a "matrix"(a gko::LinOp to be more general) as an input. In this case we provide it with a full matrix that we previously read, but as the solver only effectively uses the apply() method within the provided "matrix" object, you can effectively create a gko::LinOp class with your own apply implementation to accomplish more tasks. We will see an example of how this can be done in the custom-matrix-format example

```
auto solver = solver_gen->generate(A);
```

Finally, solve the system. The solver, being a gko::LinOp, can be applied to a right hand side, b to obtain the solution, x.

```
solver->apply(b, x);
```

Print the solution to the command line.

```
std::cout << "Solution (x):\n";
write(std::cout, x);
```

To measure if your solution has actually converged, you can measure the error of the solution. one, neg_one are objects that represent the numbers which allow for a uniform interface when computing on any device. To compute the residual, all you need to do is call the apply method, which in this case is an spmv and equivalent to the LAPACK z_spmv routine. Finally, you compute the euclidean 2-norm with the compute_norm2 function.

```
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout << "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

## Results

The following is the expected result:

```
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
2.10788e-15
```

### Comments about programming and debugging

## The plain program

```
#include <ginkgo/ginkgo.hpp>
#include <fstream>
#include <iostream>
#include <map>
#include <string>
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using real_vec = gko::matrix::Dense<RealValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor] " « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = gko::share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    const RealValueType reduction_factor{1e-7};
    auto solver_gen =
        cg::build()
```

```
            .with_criteria(gko::stop::Iteration::build().with_max_iters(20u),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .on(exec);
    auto solver = solver_gen->generate(A);
    solver->apply(b, x);
    std::cout << "Solution (x):\n";
    write(std::cout, x);
    auto one = gko::initialize<vec>({1.0}, exec);
    auto neg_one = gko::initialize<vec>({-1.0}, exec);
    auto res = gko::initialize<real_vec>({0.0}, exec);
    A->apply(one, x, neg_one, b);
    b->compute_norm2(res);
    std::cout << "Residual norm sqrt(r^T r):\n";
    write(std::cout, res);
}
```

# Chapter 44

# The simple-solver-logging program

The simple solver with logging example..

This example depends on simple-solver, minimal-cuda-solver.

## Introduction

### About the example

## The commented program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
namespace {
template <typename ValueType>
void print_vector(const std::string& name,
                  const gko::matrix::Dense<ValueType>* vec)
{
    std::cout « name « " = [" « std::endl;
    for (int i = 0; i < vec->get_size()[0]; ++i) {
        std::cout « "    " « vec->at(i, 0) « std::endl;
    }
    std::cout « "];" « std::endl;
}
}  // namespace
int main(int argc, char* argv[])
{
```

Some shortcuts

```
using ValueType = double;
using RealValueType = gko::remove_complex<ValueType>;
using IndexType = int;
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && (std::string(argv[1]) == "--help")) {
    std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
    std::exit(-1);
}
```

Figure out where to run the code

```
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
```

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

Read data

```
auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
```

Let's declare a logger which prints to std::cout instead of printing to a file. We log all events except for all linop factory and polymorphic object events. Events masks are group of events which are provided for convenience.

```
std::shared_ptr<gko::log::Stream<ValueType» stream_logger =
    gko::log::Stream<ValueType>::create(
        gko::log::Logger::all_events_mask ^
            gko::log::Logger::linop_factory_events_mask ^
            gko::log::Logger::polymorphic_object_events_mask,
        std::cout);
```

Add stream_logger to the executor

```
exec->add_logger(stream_logger);
```

Add stream_logger only to the ResidualNorm criterion Factory Note that the logger will get automatically propagated to every criterion generated from this factory.

```
const RealValueType reduction_factor{1e-7};
using ResidualCriterionFactory =
    gko::stop::ResidualNorm<ValueType>::Factory;
std::shared_ptr<ResidualCriterionFactory> residual_criterion =
    ResidualCriterionFactory::create()
        .with_reduction_factor(reduction_factor)
        .on(exec);
residual_criterion->add_logger(stream_logger);
```

Generate solver

```
auto solver_gen =
    cg::build()
        .with_criteria(residual_criterion,
                       gko::stop::Iteration::build().with_max_iters(20u))
        .on(exec);
auto solver = solver_gen->generate(A);
```

First we add facilities to only print to a file. It's possible to select events, using masks, e.g. only iterations mask: gko::log::Logger::iteration_complete_mask. See the documentation of Logger class for more information.

```
std::ofstream filestream("my_file.txt");
solver->add_logger(gko::log::Stream<ValueType>::create(
    gko::log::Logger::all_events_mask, filestream));
solver->add_logger(stream_logger);
```

This adds a simple logger that only reports convergence state at the end of the solver. Specifically it captures the last residual norm, the final number of iterations, and the converged or not converged status.

```
std::shared_ptr<gko::log::Convergence<ValueType» convergence_logger =
    gko::log::Convergence<ValueType>::create();
solver->add_logger(convergence_logger);
```

Add another logger which puts all the data in an object, we can later retrieve this object in our code. Here we only have want Executor and criterion check completed events.

```
std::shared_ptr<gko::log::Record> record_logger =
    gko::log::Record::create(gko::log::Logger::executor_events_mask |
                             gko::log::Logger::iteration_complete_mask);
```

```
exec->add_logger(record_logger);
solver->add_logger(record_logger);
```

Solve system

```
solver->apply(b, x);
```

Print the residual of the last criterion check event (where convergence happened)

```
auto residual =
    record_logger->get().iteration_completed.back()->residual.get();
auto residual_d = gko::as<vec>(residual);
print_vector("Residual", residual_d);
```

Print solution

```
    std::cout « "Solution (x):\n";
    write(std::cout, x);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, gko::as<vec>(convergence_logger->get_residual_norm()));
    std::cout « "Number of iterations "
              « convergence_logger->get_num_iterations() « std::endl;
    std::cout « "Convergence status " « std::boolalpha
              « convergence_logger->has_converged() « std::endl;
}
```

# Results

This is the expected output:

```
[LOG] »» apply started on A LinOp[gko::solver::Cg<double>,0x2142d60] with b
      LinOp[gko::matrix::Dense<double>,0x2142140] and x LinOp[gko::matrix::Dense<double>,0x2143450]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2142280] with
      Bytes[8]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143410] with
      Bytes[8]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21480a0] with
      Bytes[152]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21482f0] with
      Bytes[152]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21484d0] with
      Bytes[152]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21486b0] with
      Bytes[152]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148010] with
      Bytes[8]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148a60] with
      Bytes[8]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21482b0] with
      Bytes[8]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148a40] with
      Bytes[8]
[LOG] »» allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[1]
[LOG] »» allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2147c90] with
      Bytes[1]
[LOG] »» Operation[gko::solver::cg::initialize_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »» Operation[gko::solver::cg::initialize_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »» Operation[gko::matrix::csr::advanced_spmv_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Csr<double, int> const*, gko::matrix::Dense<double> const*, gko::matrix::Dense<double>
      const*, gko::matrix::Dense<double>*>,0x7ffd93d14aa0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »» Operation[gko::matrix::csr::advanced_spmv_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Csr<double, int> const*, gko::matrix::Dense<double> const*, gko::matrix::Dense<double>
```

```
      const*, gko::matrix::Dense<double>*>,0x7ffd93d14aa0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[2]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148ee0] with
      Bytes[2]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148e50] with
      Bytes[8]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2147ce0] with
      Bytes[8]
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14a20] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14a20] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> iteration 0 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] >> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 0 with ID
      1 and finalized set to 1
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 0 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149550] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149550] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149550] with
      Bytes[152]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149730] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149730] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149730] with
      Bytes[152]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
```

```
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> iteration 1 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] >> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 1 with ID
      1 and finalized set to 1
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 1 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149980] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149980] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149980] with
      Bytes[152]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b80] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149b80] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149b80] with
      Bytes[152]
[LOG] >> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149730]
[LOG] >> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149730]
[LOG] >> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149550]
[LOG] >> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149550]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
```

```
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 2 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 2 with ID
      1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 2 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149290] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149290] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149290] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149690] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149690] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149690] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b80]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b80]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149980]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149980]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
```

```
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 3 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 3 with ID
      1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 3 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149890] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149890] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149ae0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149ae0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149ae0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149690]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149690]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149290]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149290]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
```

```
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 4 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 4 with ID
      1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 4 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149200] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149200] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149200] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149310] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149310] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149310] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149ae0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149ae0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
```

```
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 5 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 5 with ID
      1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 5 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149890] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149890] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149cc0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149cc0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149cc0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149310]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149310]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149200]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149200]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 6 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
```

```
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] >> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 6 with ID
      1 and finalized set to 1
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 6 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149450] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149450] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149450] with
      Bytes[152]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21494f0] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21494f0] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21494f0] with
      Bytes[152]
[LOG] >> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149cc0]
[LOG] >> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149cc0]
[LOG] >> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890]
[LOG] >> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> iteration 7 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] >> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 7 with ID
      1 and finalized set to 1
```

```
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 7 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149730] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149730] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149730] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21497d0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21497d0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21497d0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21494f0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21494f0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149450]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149450]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 8 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 8 with ID
      1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
```

```
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 8 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149200] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149200] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149200] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21492a0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21492a0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21492a0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21497d0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21497d0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149730]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149730]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 9 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 9 with ID
      1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
```

```
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 9 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149620] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149620] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149620] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21496c0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21496c0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21496c0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21492a0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21492a0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149200]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149200]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 10 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 10 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
```

```
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 10 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149450] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149450] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149450] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149760] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149760] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149760] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21496c0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21496c0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149620]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149620]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 11 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 11 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 11 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149860] with
```

```
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149860] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149860] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149900] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149900] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149900] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149760]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149760]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149450]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149450]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 12 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 12 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 12 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21499a0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21499a0] with
      Bytes[152]
```

```
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21499a0] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21493d0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21493d0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21493d0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149900]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149900]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149860]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149860]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 13 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 13 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 13 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149490] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149490] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149490] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
```

```
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149580] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149580] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149580] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21493d0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21493d0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21499a0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21499a0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 14 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 14 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 14 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b50] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149b50] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149b50] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21499c0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21499c0] with
```

```
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x21499c0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149580]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149580]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149490]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149490]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 15 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 15 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 15 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149a70] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149a70] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149a70] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149340] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149340] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149340] with
      Bytes[152]
```

```
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21499c0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21499c0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b50]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b50]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 16 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 16 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 16 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149970] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149970] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149970] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b10] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149b10] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149b10] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149340]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149340]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149a70]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149a70]
```

```
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> iteration 17 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] >> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 17 with
      ID 1 and finalized set to 1
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] >> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 17 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149780] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149780] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149780] with
      Bytes[152]
[LOG] >> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] >> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890] with
      Bytes[152]
[LOG] >> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149890] with
      Bytes[152]
[LOG] >> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149890] with
      Bytes[152]
[LOG] >> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b10]
[LOG] >> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149b10]
[LOG] >> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149970]
[LOG] >> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149970]
[LOG] >> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
```

```
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 18 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 18 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 18 with
      ID 1 and finalized set to 1.  It changed one RHS 0, stopped the iteration process 0
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149620] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149620] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149620] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149cf0] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149cf0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149cf0] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149780]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149780]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_1_operation<gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
```

```
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::spmv_operation<gko::matrix::Csr<double, int> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14b80] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::solver::cg::step_2_operation<gko::matrix::Dense<double>*&,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::matrix::Dense<double>*, gko::matrix::Dense<double>*,
      gko::array<gko::stopping_status>*>,0x7ffd93d14ef0] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x21482f0] with
      Bytes[152]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_dot_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double> const*, gko::matrix::Dense<double>*>,0x7ffd93d14c50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> iteration 19 completed with solver LinOp[gko::solver::Cg<double>,0x2142d60] with residual
      LinOp[gko::matrix::Dense<double>,0x2147b30], solution LinOp[gko::matrix::Dense<double>,0x2143450] and
      residual_norm LinOp[gko::LinOp const*,0]
[LOG] »> check started for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 19 with
      ID 1 and finalized set to 1
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14ad0] completed on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::stop::residual_norm::residual_norm_operation<gko::matrix::Dense<double> const*&,
      gko::matrix::Dense<double>*, double&, unsigned char&, bool&, gko::array<gko::stopping_status>*&,
      gko::array<bool>*, bool*, bool*&>,0x7ffd93d14b90] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> check completed for stop::Criterion[gko::stop::ResidualNorm<double>,0x2148db0] at iteration 19 with
      ID 1 and finalized set to 1.  It changed one RHS 1, stopped the iteration process 1
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149890] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149890] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x21480a0] to Location[0x2149890] with
      Bytes[152]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[152]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149340] with
      Bytes[152]
[LOG] »> copy started from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149340] with
      Bytes[152]
[LOG] »> copy completed from Executor[gko::ReferenceExecutor,0x21400d0] to
      Executor[gko::ReferenceExecutor,0x21400d0] from Location[0x2143e90] to Location[0x2149340] with
      Bytes[152]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149cf0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149cf0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149620]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149620]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148ee0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148ee0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2147ce0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2147ce0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148e50]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148e50]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2147c90]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2147c90]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21482b0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21482b0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148a40]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148a40]
```

```
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148a60]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148a60]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148010]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2148010]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21486b0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21486b0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21484d0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21484d0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21482f0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21482f0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21480a0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x21480a0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143410]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143410]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2142280]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2142280]
[LOG] »> apply completed on A LinOp[gko::solver::Cg<double>,0x2142d60] with b
      LinOp[gko::matrix::Dense<double>,0x2142140] and x LinOp[gko::matrix::Dense<double>,0x2143450]
Last memory copied was of size 98 FROM executor 0x21400d0 pointer 2143e90 TO executor 0x21400d0 pointer
      2149340
Residual = [
    8.1654e-19
    -1.51449e-17
    2.23854e-17
    -1.0842e-19
    6.09864e-20
    -1.92446e-18
    1.97867e-18
    -4.58075e-18
    -1.55854e-18
    -2.64274e-17
    4.20128e-17
    -8.71427e-18
    -2.62919e-18
    -5.49947e-17
    5.51893e-17
    -1.57022e-16
    -4.2034e-17
    -8.71951e-16
    1.37837e-15
];
Solution (x):
%%MatrixMarket matrix array real general
19 1
0.252218
0.108645
0.0662811
0.0630433
0.0384088
0.0396536
0.0402648
0.0338935
0.0193098
0.0234653
0.0211499
0.0196413
0.0199151
0.0181674
0.0162722
0.0150714
0.0107016
0.0121141
0.0123025
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149bb0] with
      Bytes[8]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149870] with
      Bytes[8]
[LOG] »> allocation started on Executor[gko::ReferenceExecutor,0x21400d0] with Bytes[8]
[LOG] »> allocation completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149500] with
      Bytes[8]
[LOG] »> Operation[gko::matrix::csr::advanced_spmv_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Csr<double, int> const*, gko::matrix::Dense<double> const*, gko::matrix::Dense<double>
      const*, gko::matrix::Dense<double>*>,0x7ffd93d14e50] started on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::csr::advanced_spmv_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Csr<double, int> const*, gko::matrix::Dense<double> const*, gko::matrix::Dense<double>
      const*, gko::matrix::Dense<double>*>,0x7ffd93d14e50] completed on
      Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14f70] started on Executor[gko::ReferenceExecutor,0x21400d0]
[LOG] »> Operation[gko::matrix::dense::compute_norm2_operation<gko::matrix::Dense<double> const*,
      gko::matrix::Dense<double>*>,0x7ffd93d14f70] completed on Executor[gko::ReferenceExecutor,0x21400d0]
Residual norm sqrt(r^T r):
%%MatrixMarket matrix array real general
1 1
```

```
2.10788e-15
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149500]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149500]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149870]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149870]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149bb0]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2149bb0]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143e90]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143e90]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143590]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143590]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2142b10]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2142b10]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143c30]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143c30]
[LOG] »> free started on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143790]
[LOG] »> free completed on Executor[gko::ReferenceExecutor,0x21400d0] at Location[0x2143790]
```

### Comments about programming and debugging

## The plain program

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <ginkgo/ginkgo.hpp>
namespace {
template <typename ValueType>
void print_vector(const std::string& name,
                  const gko::matrix::Dense<ValueType>* vec)
{
    std::cout « name « " = [" « std::endl;
    for (int i = 0; i < vec->get_size()[0]; ++i) {
        std::cout « "    " « vec->at(i, 0) « std::endl;
    }
    std::cout « "];" « std::endl;
}
}  // namespace
int main(int argc, char* argv[])
{
    using ValueType = double;
    using RealValueType = gko::remove_complex<ValueType>;
    using IndexType = int;
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && (std::string(argv[1]) == "--help")) {
        std::cerr « "Usage:  " « argv[0] « " [executor]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    auto A = share(gko::read<mtx>(std::ifstream("data/A.mtx"), exec));
    auto b = gko::read<vec>(std::ifstream("data/b.mtx"), exec);
    auto x = gko::read<vec>(std::ifstream("data/x0.mtx"), exec);
    std::shared_ptr<gko::log::Stream<ValueType» stream_logger =
        gko::log::Stream<ValueType>::create(
            gko::log::Logger::all_events_mask ^
                gko::log::Logger::linop_factory_events_mask ^
                gko::log::Logger::polymorphic_object_events_mask,
```

```
            std::cout);
    exec->add_logger(stream_logger);
    const RealValueType reduction_factor{1e-7};
    using ResidualCriterionFactory =
        gko::stop::ResidualNorm<ValueType>::Factory;
    std::shared_ptr<ResidualCriterionFactory> residual_criterion =
        ResidualCriterionFactory::create()
            .with_reduction_factor(reduction_factor)
            .on(exec);
    residual_criterion->add_logger(stream_logger);
    auto solver_gen =
        cg::build()
            .with_criteria(residual_criterion,
                           gko::stop::Iteration::build().with_max_iters(20u))
            .on(exec);
    auto solver = solver_gen->generate(A);
    std::ofstream filestream("my_file.txt");
    solver->add_logger(gko::log::Stream<ValueType>::create(
        gko::log::Logger::all_events_mask, filestream));
    solver->add_logger(stream_logger);
    std::shared_ptr<gko::log::Convergence<ValueType» convergence_logger =
        gko::log::Convergence<ValueType>::create();
    solver->add_logger(convergence_logger);
    std::shared_ptr<gko::log::Record> record_logger =
        gko::log::Record::create(gko::log::Logger::executor_events_mask |
                                 gko::log::Logger::iteration_complete_mask);
    exec->add_logger(record_logger);
    solver->add_logger(record_logger);
    solver->apply(b, x);
    auto residual =
        record_logger->get().iteration_completed.back()->residual.get();
    auto residual_d = gko::as<vec>(residual);
    print_vector("Residual", residual_d);
    std::cout « "Solution (x):\n";
    write(std::cout, x);
    std::cout « "Residual norm sqrt(r^T r):\n";
    write(std::cout, gko::as<vec>(convergence_logger->get_residual_norm()));
    std::cout « "Number of iterations "
              « convergence_logger->get_num_iterations() « std::endl;
    std::cout « "Convergence status " « std::boolalpha
              « convergence_logger->has_converged() « std::endl;
}
```

# Chapter 45

# The three-pt-stencil-solver program

The 3-point stencil example..

This example depends on simple-solver, poisson-solver.

## Introduction

This example solves a 1D Poisson equation:

$u : [0, 1] \to R$
$u" = f$
$u(0) = u0$
$u(1) = u1$

using a finite difference method on an equidistant grid with `K` discretization points (`K` can be controlled with a command line parameter). The discretization is done via the second order Taylor polynomial:

$u(x + h) = u(x) - u'(x)h + 1/2u"(x)h^2 + O(h^3)$
$u(x - h) = u(x) + u'(x)h + 1/2u"(x)h^2 + O(h^3)/ +$
$- - - - - - - - - - - - - - - - - - - - - - - - - - - - - -$
$- u(x - h) + 2u(x) + -u(x + h) = -f(x)h^2 + O(h^3)$

For an equidistant grid with K "inner" discretization points $x1, ..., xk$ and step size $h = 1/(K + 1)$, the formula produces a system of linear equations

$2u_1 - u_2 = -f_1h^2 + u0$
$- u_(k - 1) + 2u_k - u_(k + 1) = -f_kh^2, k = 2, ..., K - 1$
$- u_(K - 1) + 2u_K = -f_Kh^2 + u1$

which is then solved using Ginkgo's implementation of the CG method preconditioned with block-Jacobi. It is also possible to specify on which executor Ginkgo will solve the system via the command line. The function `f` is set to `f(x) = 6x` (making the solution `u(x) = x^3`), but that can be changed in the `main` function.

The intention of the example is to show how Ginkgo can be integrated into existing software - the `generate_stencil_matrix`, `generate_rhs`, `print_solution`, `compute_error` and `main` function do not reference Ginkgo at all (i.e. they could have been there before the application developer decided to use Ginkgo, and the only part where Ginkgo is introduced is inside the `solve_system` function.

### About the example

## The commented program

```
/ ****************************<DESCRIPTION>**********************************
This example solves a 1D Poisson equation:
    u :  [0, 1] -> R
    u'' = f
    u(0) = u0
    u(1) = u1
using a finite difference method on an equidistant grid with `K` discretization
points (`K` can be controlled with a command line parameter).  The discretization
is done via the second order Taylor polynomial:
u(x + h) = u(x) - u'(x)h + 1/2 u''(x)h^2 + O(h^3)
u(x - h) = u(x) + u'(x)h + 1/2 u''(x)h^2 + O(h^3)  / +
-----------------------------------------------
-u(x - h) + 2u(x) + -u(x + h) = -f(x)h^2 + O(h^3)
For an equidistant grid with K "inner" discretization points x1, ..., xk, and
step size h = 1 / (K + 1), the formula produces a system of linear equations
           2u_1 - u_2     = -f_1 h^2 + u0
-u_(k-1) + 2u_k - u_(k+1) = -f_k h^2,       k = 2, ..., K - 1
-u_(K-1) + 2u_K           = -f_K h^2 + u1
which is then solved using Ginkgo's implementation of the CG method
preconditioned with block-Jacobi.  It is also possible to specify on which
executor Ginkgo will solve the system via the command line.
The function `f` is set to `f(x) = 6x` (making the solution `u(x) = x^3`), but
that can be changed in the `main` function.

The intention of the example is to show how Ginkgo can be integrated into
existing software - the `generate_stencil_matrix`, `generate_rhs`,
`print_solution`, `compute_error` and `main` function do not reference Ginkgo at
all (i.e.  they could have been there before the application developer decided to
use Ginkgo, and the only part where Ginkgo is introduced is inside the
`solve_system` function.
****************************<DESCRIPTION>********************************** /

#include <iostream>
#include <map>
#include <string>
#include <vector>

#include <ginkgo/ginkgo.hpp>
```

Creates a stencil matrix in CSR format for the given number of discretization points.

```
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(IndexType discretization_points,
                             IndexType* row_ptrs, IndexType* col_idxs,
                             ValueType* values)
{
    IndexType pos = 0;
    const ValueType coefs[] = {-1, 2, -1};
    row_ptrs[0] = pos;
    for (IndexType i = 0; i < discretization_points; ++i) {
        for (auto ofs :  {-1, 0, 1}) {
            if (0 <= i + ofs && i + ofs < discretization_points) {
                values[pos] = coefs[ofs + 1];
                col_idxs[pos] = i + ofs;
                ++pos;
            }
        }
        row_ptrs[i + 1] = pos;
    }
}
```

Generates the RHS vector given `f` and the boundary conditions.

```
template <typename Closure, typename ValueType, typename IndexType>
void generate_rhs(IndexType discretization_points, Closure f, ValueType u0,
                  ValueType u1, ValueType* rhs)
{
    const ValueType h = 1.0 / (discretization_points + 1);
    for (IndexType i = 0; i < discretization_points; ++i) {
        const ValueType xi = ValueType(i + 1) * h;
        rhs[i] = -f(xi) * h * h;
    }
    rhs[0] += u0;
    rhs[discretization_points - 1] += u1;
}
```

Prints the solution `u`.

```
template <typename ValueType, typename IndexType>
void print_solution(IndexType discretization_points, ValueType u0, ValueType u1,
```

```
                    const ValueType* u)
{
    std::cout « u0 « '\n';
    for (IndexType i = 0; i < discretization_points; ++i) {
        std::cout « u[i] « '\n';
    }
    std::cout « u1 « std::endl;
}
```

Computes the 1-norm of the error given the computed `u` and the correct solution function `correct_u`.

```
template <typename Closure, typename ValueType, typename IndexType>
gko::remove_complex<ValueType> calculate_error(IndexType discretization_points,
                                               const ValueType* u,
                                               Closure correct_u)
{
    const ValueType h = 1.0 / (discretization_points + 1);
    gko::remove_complex<ValueType> error = 0.0;
    for (IndexType i = 0; i < discretization_points; ++i) {
        using std::abs;
        const ValueType xi = ValueType(i + 1) * h;
        error += abs(u[i] - correct_u(xi)) / abs(correct_u(xi));
    }
    return error;
}
template <typename ValueType, typename IndexType>
void solve_system(const std::string& executor_string,
                  IndexType discretization_points, IndexType* row_ptrs,
                  IndexType* col_idxs, ValueType* values, ValueType* rhs,
                  ValueType* u, gko::remove_complex<ValueType> reduction_factor)
{
```

Some shortcuts

```
using vec = gko::matrix::Dense<ValueType>;
using mtx = gko::matrix::Csr<ValueType, IndexType>;
using cg = gko::solver::Cg<ValueType>;
using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
using val_array = gko::array<ValueType>;
using idx_array = gko::array<IndexType>;
const auto& dp = discretization_points;
```

Figure out where to run the code

```
std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
    exec_map{
        {"omp", [] { return gko::OmpExecutor::create(); }},
        {"cuda",
         [] {
             return gko::CudaExecutor::create(0,
                                              gko::OmpExecutor::create());
         }},
        {"hip",
         [] {
             return gko::HipExecutor::create(0, gko::OmpExecutor::create());
         }},
        {"dpcpp",
         [] {
             return gko::DpcppExecutor::create(0,
                                               gko::OmpExecutor::create());
         }},
        {"reference", [] { return gko::ReferenceExecutor::create(); }}};
```

executor where Ginkgo will perform the computation

```
const auto exec = exec_map.at(executor_string)();  // throws if not valid
```

executor where the application initialized the data

```
const auto app_exec = exec->get_master();
```

Tell Ginkgo to use the data in our application

Matrix: we have to set the executor of the matrix to the one where we want SpMVs to run (in this case `exec`). When creating array views, we have to specify the executor where the data is (in this case `app_exec`).

If the two do not match, Ginkgo will automatically create a copy of the data on `exec` (however, it will not copy the data back once it is done

- here this is not important since we are not modifying the matrix).

```
auto matrix = mtx::create(exec, gko::dim<2>(dp),
                          val_array::view(app_exec, 3 * dp - 2, values),
                          idx_array::view(app_exec, 3 * dp - 2, col_idxs),
                          idx_array::view(app_exec, dp + 1, row_ptrs));
```

RHS: similar to matrix

```
auto b = vec::create(exec, gko::dim<2>(dp, 1),
                     val_array::view(app_exec, dp, rhs), 1);
```

Solution: we have to be careful here - if the executors are different, once we compute the solution the array will not be automatically copied back to the original memory locations. Fortunately, whenever `apply` is called on a linear operator (e.g. matrix, solver) the arguments automatically get copied to the executor where the operator is, and copied back once the operation is completed. Thus, in this case, we can just define the solution on `app_exec`, and it will be automatically transferred to/from `exec` if needed.

```
auto x = vec::create(app_exec, gko::dim<2>(dp, 1),
                     val_array::view(app_exec, dp, u), 1);
```

Generate solver

```
auto solver_gen =
    cg::build()
        .with_criteria(gko::stop::Iteration::build().with_max_iters(
                           gko::size_type(dp)),
                       gko::stop::ResidualNorm<ValueType>::build()
                           .with_reduction_factor(reduction_factor))
        .with_preconditioner(bj::build())
        .on(exec);
auto solver = solver_gen->generate(gko::give(matrix));
```

Solve system

```
    solver->apply(b, x);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
```

Print version information

```
std::cout « gko::version_info::get() « std::endl;
if (argc == 2 && std::string(argv[1]) == "--help") {
    std::cerr « "Usage:  " « argv[0]
              « " [executor] [DISCRETIZATION_POINTS]" « std::endl;
    std::exit(-1);
}
const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
const IndexType discretization_points =
    argc >= 3 ?  std::atoi(argv[2]) :  100;
```

problem:

```
auto correct_u = [](ValueType x) { return x * x * x; };
auto f = [](ValueType x) { return ValueType(6) * x; };
auto u0 = correct_u(0);
auto u1 = correct_u(1);
```

matrix

```
std::vector<IndexType> row_ptrs(discretization_points + 1);
std::vector<IndexType> col_idxs(3 * discretization_points - 2);
std::vector<ValueType> values(3 * discretization_points - 2);
```

right hand side

```
std::vector<ValueType> rhs(discretization_points);
```

solution

```
std::vector<ValueType> u(discretization_points, 0.0);
const gko::remove_complex<ValueType> reduction_factor = 1e-7;
generate_stencil_matrix(discretization_points, row_ptrs.data(),
                        col_idxs.data(), values.data());
```

looking for solution u = x^3: f = 6x, u(0) = 0, u(1) = 1

```
generate_rhs(discretization_points, f, u0, u1, rhs.data());
solve_system(executor_string, discretization_points, row_ptrs.data(),
             col_idxs.data(), values.data(), rhs.data(), u.data(),
             reduction_factor);
```

Uncomment to print the solution print_solution<ValueType, IndexType>(discretization_points, 0, 1, u.data());

```
    std::cout « "The average relative error is "
              « calculate_error(discretization_points, u.data(), correct_u) /
                    discretization_points
              « std::endl;
}
```

# Results

This is the expected output:

```
The average relative error is 2.52236e-11
```

### Comments about programming and debugging

# The plain program

```
/*****************************<DESCRIPTION>**********************************
This example solves a 1D Poisson equation:

u :  [0, 1] -> R
u'' = f
u(0) = u0
u(1) = u1

using a finite difference method on an equidistant grid with `K` discretization
points (`K` can be controlled with a command line parameter).  The discretization
is done via the second order Taylor polynomial:

u(x + h) = u(x) + u'(x)h + 1/2 u''(x)h^2 + O(h^3)
u(x - h) = u(x) + u'(x)h + 1/2 u''(x)h^2 + O(h^3)   / +
-------------------------------------------------
-u(x - h) + 2u(x) + -u(x + h) = -f(x)h^2 + O(h^3)

For an equidistant grid with K "inner" discretization points x1, ..., xk, and
step size h = 1 / (K + 1), the formula produces a system of linear equations

2u_1 - u_2     = -f_1 h^2 + u0
-u_(k-1) + 2u_k - u_(k+1) = -f_k h^2,       k = 2, ..., K - 1
-u_(K-1) + 2u_K           = -f_K h^2 + u1


which is then solved using Ginkgo's implementation of the CG method
preconditioned with block-Jacobi.  It is also possible to specify on which
executor Ginkgo will solve the system via the command line.
The function `f` is set to `f(x) = 6x` (making the solution `u(x) = x^3`), but
that can be changed in the `main` function.

The intention of the example is to show how Ginkgo can be integrated into
existing software - the `generate_stencil_matrix`, `generate_rhs`,
`print_solution`, `compute_error` and `main` function do not reference Ginkgo at
all (i.e.  they could have been there before the application developer decided to
use Ginkgo, and the only part where Ginkgo is introduced is inside the
`solve_system` function.
*****************************<DESCRIPTION>**********************************/
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <ginkgo/ginkgo.hpp>
template <typename ValueType, typename IndexType>
void generate_stencil_matrix(IndexType discretization_points,
                             IndexType* row_ptrs, IndexType* col_idxs,
                             ValueType* values)
{
    IndexType pos = 0;
    const ValueType coefs[] = {-1, 2, -1};
    row_ptrs[0] = pos;
    for (IndexType i = 0; i < discretization_points; ++i) {
        for (auto ofs :  {-1, 0, 1}) {
            if (0 <= i + ofs && i + ofs < discretization_points) {
                values[pos] = coefs[ofs + 1];
                col_idxs[pos] = i + ofs;
                ++pos;
            }
        }
        row_ptrs[i + 1] = pos;
    }
}
template <typename Closure, typename ValueType, typename IndexType>
void generate_rhs(IndexType discretization_points, Closure f, ValueType u0,
                  ValueType u1, ValueType* rhs)
{
    const ValueType h = 1.0 / (discretization_points + 1);
    for (IndexType i = 0; i < discretization_points; ++i) {
        const ValueType xi = ValueType(i + 1) * h;
        rhs[i] = -f(xi) * h * h;
```

```
    }
    rhs[0] += u0;
    rhs[discretization_points - 1] += u1;
}
template <typename ValueType, typename IndexType>
void print_solution(IndexType discretization_points, ValueType u0, ValueType u1,
                    const ValueType* u)
{
    std::cout « u0 « '\n';
    for (IndexType i = 0; i < discretization_points; ++i) {
        std::cout « u[i] « '\n';
    }
    std::cout « u1 « std::endl;
}
template <typename Closure, typename ValueType, typename IndexType>
gko::remove_complex<ValueType> calculate_error(IndexType discretization_points,
                                               const ValueType* u,
                                               Closure correct_u)
{
    const ValueType h = 1.0 / (discretization_points + 1);
    gko::remove_complex<ValueType> error = 0.0;
    for (IndexType i = 0; i < discretization_points; ++i) {
        using std::abs;
        const ValueType xi = ValueType(i + 1) * h;
        error += abs(u[i] - correct_u(xi)) / abs(correct_u(xi));
    }
    return error;
}
template <typename ValueType, typename IndexType>
void solve_system(const std::string& executor_string,
                  IndexType discretization_points, IndexType* row_ptrs,
                  IndexType* col_idxs, ValueType* values, ValueType* rhs,
                  ValueType* u, gko::remove_complex<ValueType> reduction_factor)
{
    using vec = gko::matrix::Dense<ValueType>;
    using mtx = gko::matrix::Csr<ValueType, IndexType>;
    using cg = gko::solver::Cg<ValueType>;
    using bj = gko::preconditioner::Jacobi<ValueType, IndexType>;
    using val_array = gko::array<ValueType>;
    using idx_array = gko::array<IndexType>;
    const auto& dp = discretization_points;
    std::map<std::string, std::function<std::shared_ptr<gko::Executor>()»
        exec_map{
            {"omp", [] { return gko::OmpExecutor::create(); }},
            {"cuda",
             [] {
                 return gko::CudaExecutor::create(0,
                                                  gko::OmpExecutor::create());
             }},
            {"hip",
             [] {
                 return gko::HipExecutor::create(0, gko::OmpExecutor::create());
             }},
            {"dpcpp",
             [] {
                 return gko::DpcppExecutor::create(0,
                                                   gko::OmpExecutor::create());
             }},
            {"reference", [] { return gko::ReferenceExecutor::create(); }}};
    const auto exec = exec_map.at(executor_string)();  // throws if not valid
    const auto app_exec = exec->get_master();
    auto matrix = mtx::create(exec, gko::dim<2>(dp),
                              val_array::view(app_exec, 3 * dp - 2, values),
                              idx_array::view(app_exec, 3 * dp - 2, col_idxs),
                              idx_array::view(app_exec, dp + 1, row_ptrs));
    auto b = vec::create(exec, gko::dim<2>(dp, 1),
                         val_array::view(app_exec, dp, rhs), 1);
    auto x = vec::create(app_exec, gko::dim<2>(dp, 1),
                         val_array::view(app_exec, dp, u), 1);
    auto solver_gen =
        cg::build()
            .with_criteria(gko::stop::Iteration::build().with_max_iters(
                               gko::size_type(dp)),
                           gko::stop::ResidualNorm<ValueType>::build()
                               .with_reduction_factor(reduction_factor))
            .with_preconditioner(bj::build())
            .on(exec);
    auto solver = solver_gen->generate(gko::give(matrix));
    solver->apply(b, x);
}
int main(int argc, char* argv[])
{
    using ValueType = double;
    using IndexType = int;
    std::cout « gko::version_info::get() « std::endl;
    if (argc == 2 && std::string(argv[1]) == "--help") {
        std::cerr « "Usage:  " « argv[0]
```

```
                  « " [executor] [DISCRETIZATION_POINTS]" « std::endl;
        std::exit(-1);
    }
    const auto executor_string = argc >= 2 ?  argv[1] :  "reference";
    const IndexType discretization_points =
        argc >= 3 ?  std::atoi(argv[2]) :  100;
    auto correct_u = [](ValueType x) { return x * x * x; };
    auto f = [](ValueType x) { return ValueType(6) * x; };
    auto u0 = correct_u(0);
    auto u1 = correct_u(1);
    std::vector<IndexType> row_ptrs(discretization_points + 1);
    std::vector<IndexType> col_idxs(3 * discretization_points - 2);
    std::vector<ValueType> values(3 * discretization_points - 2);
    std::vector<ValueType> rhs(discretization_points);
    std::vector<ValueType> u(discretization_points, 0.0);
    const gko::remove_complex<ValueType> reduction_factor = 1e-7;
    generate_stencil_matrix(discretization_points, row_ptrs.data(),
                            col_idxs.data(), values.data());
    generate_rhs(discretization_points, f, u0, u1, rhs.data());
    solve_system(executor_string, discretization_points, row_ptrs.data(),
                 col_idxs.data(), values.data(), rhs.data(), u.data(),
                 reduction_factor);
    std::cout « "The average relative error is "
              « calculate_error(discretization_points, u.data(), correct_u) /
                    discretization_points
              « std::endl;
}
```

# Chapter 46

# Module Documentation

## 46.1 Batched Linear Operators

A module dedicated to the implementation and usage of the batched linear operators in Ginkgo.

### Classes

- class gko::batch::BatchLinOpFactory

  *A BatchLinOpFactory represents a higher order mapping which transforms one batch linear operator into another.*
- class gko::batch::MultiVector< ValueType >

  *MultiVector stores multiple vectors in a batched fashion and is useful for batched operations.*
- class gko::batch::matrix::Csr< ValueType, IndexType >

  *Csr is a general sparse matrix format that stores the column indices for each nonzero entry and a cumulative sum of the number of nonzeros in each row.*
- class gko::batch::matrix::Dense< ValueType >

  *Dense is a batch matrix format which explicitly stores all values of the matrix in each of the batches.*
- class gko::batch::matrix::Ell< ValueType, IndexType >

  *Ell is a sparse matrix format that stores the same number of nonzeros in each row, enabling coalesced accesses.*
- class gko::batch::matrix::Identity< ValueType >

  *The batch Identity matrix, which represents a batch of Identity matrices.*
- class gko::batch::preconditioner::Jacobi< ValueType, IndexType >

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks (stored in a dense row major fashion) of the source operator.*
- class gko::batch::solver::Bicgstab< ValueType >

  *BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.*
- class gko::batch::solver::Cg< ValueType >

  *Cg or the Conjugate Gradient is a Krylov subspace solver.*

### Macros

- #define GKO_ENABLE_BATCH_LIN_OP_FACTORY(_batch_lin_op, _parameters_name, _factory_name)

  *This macro will generate a default implementation of a BatchLinOpFactory for the BatchLinOp subclass it is defined in.*

## Typedefs

- template<typename ConcreteFactory , typename ConcreteBatchLinOp , typename ParametersType , typename PolymorphicBase = BatchLinOpFactory>
  using gko::batch::EnableDefaultBatchLinOpFactory = EnableDefaultFactory< ConcreteFactory, ConcreteBatchLinOp, ParametersType, PolymorphicBase >

  *This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of BatchLinOpFactory.*

### 46.1.1 Detailed Description

A module dedicated to the implementation and usage of the batched linear operators in Ginkgo.

A batched linear operator applies the same operation to a large number of small, independent systems at once. Every item of a batch shares the same dimensions and sparsity pattern, and no data is exchanged between items, which is what allows the whole batch to be solved in a single kernel.

### 46.1.2 Batched Linear operator as a concept

A batch linear operator (BatchLinOp) forms the base class for all batched linear algebra objects. In general, it follows the same structure as the LinOp class, but has some crucial differences which make it not strictly representable through or with the LinOp class.

A batched operator is defined as a set of independent linear operators which have no communication/information exchange between them. Therefore, any collective operations between the batches is not possible and not implemented. This allows for each batch to be computed and operated on in an embarrassingly parallel fashion.

A key difference between the LinOp and the BatchLinOp class is that the apply between BatchLinOps is no longer supported. The user can apply a BatchLinOp to a batch::MultiVector but not to any general BatchLinOp.

Therefore, the BatchLinOp serves only as a base class providing necessary core functionality from Polymorphic object and store the dimensions of the batched object.

**Note**

Apply to batch::MultiVector objects are handled by the concrete LinOp and may be moved to the base BatchLinOp class in the future.

Batched Linear Operators

### 46.1.3 Macro Definition Documentation

#### 46.1.3.1 GKO_ENABLE_BATCH_LIN_OP_FACTORY

```
#define GKO_ENABLE_BATCH_LIN_OP_FACTORY(
              _batch_lin_op,
              _parameters_name,
              _factory_name )
```

This macro will generate a default implementation of a BatchLinOpFactory for the BatchLinOp subclass it is defined in.

It is required to first call the macro GKO_CREATE_FACTORY_PARAMETERS() before this one in order to instantiate the parameters type first.

The list of parameters for the factory should be defined in a code block after the macro definition, and should contain a list of GKO_FACTORY_PARAMETER_∗ declarations. The class should provide a constructor with signature _↩ batch_lin_op(const _factory_name ∗, std::shared_ptr<const BatchLinOp>) which the factory will use a callback to construct the object.

A minimal example of a batch linear operator is the following:

```c++ struct MyBatchLinOp : public BatchLinOp { GKO_ENABLE_BATCH_LIN_OP_FACTORY(MyBatchLinOp, my_parameters, Fact { // a factory parameter named "my_value", of type int and default // value of 5 int GKO_FACTORY_PARAMETER_SCALAR(my_value, // a factory parameter named my_pair of type std::pair<int,int> // and default value {5, 5} std::pair<int, int> GKO_FACTORY_PARAMETER_VECTOR(my_pair, 5, 5); }; // constructor needed by BatchLinOp explicit MyBatchLinOp(std::shared_ptr<const Executor> exec) { : BatchLinOp(exec) {} // constructor needed by the factory explicit MyBatchLinOp(const Factory ∗factory, std::shared_ptr<const BatchLinOp> matrix) : BatchLin↩ Op(factory->get_executor()), matrix->get_size()), // store factory's parameters locally my_parameters_{factory->get_parameters()} { int value = my_parameters_.my_value; // do something with value }
MyBatchLinOp can then be created as follows:
```c++
auto exec = gko::ReferenceExecutor::create();
// create a factory with default `my_value` parameter
auto fact = MyBatchLinOp::build().on(exec);
// create a operator using the factory:
auto my_op = fact->generate(gko::batch::matrix::Identity::create(exec, 2));
std::cout « my_op->get_my_parameters().my_value;  // prints 5
// create a factory with custom `my_value` parameter
auto fact = MyLinOp::build().with_my_value(0).on(exec);
// create a operator using the factory:
auto my_op = fact->generate(gko::batch::matrix::Identity::create(exec, 2));
std::cout « my_op->get_my_parameters().my_value;  // prints 0
```

**Note**

It is possible to combine both the #GKO_CREATE_FACTORY_PARAMETER_∗() macros with this one in a unique macro for class **templates** (not with regular classes). Splitting this into two distinct macros allows to use them in all contexts. See https://stackoverflow.com/q/50202718/9385966 for more details.

**Parameters**

| | |
|---|---|
| *_batch_lin_op* | concrete operator for which the factory is to be created [CRTP parameter] |
| *_parameters_name* | name of the parameters member in the class (its type is <_parameters_name>_type, the protected member's name is <_parameters_name>_, and the public getter's name is get_<_parameters_name>()) |
| *_factory_name* | name of the generated factory type |
```

## 46.1.4 Typedef Documentation

### 46.1.4.1 EnableDefaultBatchLinOpFactory

```
template<typename ConcreteFactory , typename ConcreteBatchLinOp , typename ParametersType ,
typename PolymorphicBase = BatchLinOpFactory>
using gko::batch::EnableDefaultBatchLinOpFactory = typedef EnableDefaultFactory<Concrete↩
Factory, ConcreteBatchLinOp, ParametersType, PolymorphicBase>
```

This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of BatchLinOpFactory.

**Template Parameters**

| | |
|---|---|
| *ConcreteFactory* | the concrete factory which is being implemented [CRTP parameter] |
| *ConcreteBatchLinOp* | the concrete BatchLinOp type which this factory produces, needs to have a constructor which takes a const ConcreteFactory $*$, and an std::shared_ptr<const BatchLinOp> as parameters. |
| *ParametersType* | a subclass of enable_parameters_type template which defines all of the parameters of the factory |
| *PolymorphicBase* | parent of ConcreteFactory in the polymorphic hierarchy, has to be a subclass of BatchLinOpFactory |

## 46.2 CUDA Executor

A module dedicated to the implementation and usage of the CUDA executor in Ginkgo.

### Classes

- class gko::CudaExecutor

  *This is the Executor subclass which represents the CUDA device.*

### 46.2.1 Detailed Description

A module dedicated to the implementation and usage of the CUDA executor in Ginkgo.

## 46.3 Distributed

A module dedicated to the implementation and usage of the distributed linear algebra objects in Ginkgo.

### Namespaces

- gko::experimental::mpi

  *The mpi namespace, contains wrapper for many MPI functions.*
- gko::experimental::distributed

  *The distributed namespace.*

### Classes

- class gko::experimental::distributed::DistributedBase

  *A base class for distributed objects.*
- class gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >

  *Represents a partition of a range of indices [0, size) into a disjoint set of parts.*
- class gko::experimental::distributed::Vector< ValueType >

  *Vector is a format which explicitly stores (multiple) distributed column vectors in a dense storage format.*

### 46.3.1 Detailed Description

A module dedicated to the implementation and usage of the distributed linear algebra objects in Ginkgo.

These types partition their rows over the ranks of an MPI communicator, so that each rank stores and operates on only its own part of the data.

## 46.4 DPC++ Executor

A module dedicated to the implementation and usage of the DPC++ executor in Ginkgo.

### Classes

- class gko::DpcppExecutor

  *This is the Executor subclass which represents a DPC++ enhanced device.*

### 46.4.1 Detailed Description

A module dedicated to the implementation and usage of the DPC++ executor in Ginkgo.

## 46.5 Executors

A module dedicated to the implementation and usage of the executors in Ginkgo.

### Modules

- CUDA Executor

  *A module dedicated to the implementation and usage of the CUDA executor in Ginkgo.*
- DPC++ Executor

  *A module dedicated to the implementation and usage of the DPC++ executor in Ginkgo.*
- HIP Executor

  *A module dedicated to the implementation and usage of the HIP executor in Ginkgo.*
- OpenMP Executor

  *A module dedicated to the implementation and usage of the OpenMP executor in Ginkgo.*
- Reference Executor

  *A module dedicated to the implementation and usage of the Reference executor in Ginkgo.*

### Classes

- class gko::Operation

  *Operations can be used to define functionalities whose implementations differ among devices.*
- class gko::Executor

  *The first step in using the Ginkgo library consists of creating an executor.*
- class gko::executor_deleter< T >

  *This is a deleter that uses an executor's `free` method to deallocate the data.*
- class gko::OmpExecutor

  *This is the Executor subclass which represents the OpenMP device (typically CPU).*
- class gko::ReferenceExecutor

  *This is a specialization of the OmpExecutor, which runs the reference implementations of the kernels used for debugging purposes.*
- class gko::CudaExecutor

  *This is the Executor subclass which represents the CUDA device.*
- class gko::HipExecutor

  *This is the Executor subclass which represents the HIP enhanced device.*
- class gko::DpcppExecutor

  *This is the Executor subclass which represents a DPC++ enhanced device.*

### Macros

- #define GKO_REGISTER_OPERATION(_name, _kernel)

  *Binds a set of device-specific kernels to an Operation.*
- #define GKO_REGISTER_HOST_OPERATION(_name, _kernel)

  *Binds a host-side kernel (independent of executor type) to an Operation.*

### 46.5.1 Detailed Description

A module dedicated to the implementation and usage of the executors in Ginkgo.

Below, we provide a brief introduction to executors in Ginkgo, how they have been implemented, how to best make use of them and how to add new executors.

### 46.5.2 Executors in Ginkgo.

The first step in using the Ginkgo library consists of creating an executor. Executors are used to specify the location for the data of linear algebra objects, and to determine where the operations will be executed. Ginkgo currently supports three different executor types:

- OpenMP Executor specifies that the data should be stored and the associated operations executed on an OpenMP-supporting device (e.g. host CPU);
- CUDA Executor specifies that the data should be stored and the operations executed on the NVIDIA GPU accelerator;
- HIP Executor uses the HIP library to compile code for either NVIDIA or AMD GPU accelerator;
- DPC++ Executor uses the DPC++ compiler for any DPC++ supported hardware (e.g. Intel CPUs, GPU, FPGAs, ...);
- Reference Executor executes a non-optimized reference implementation, which can be used to debug the library.

### 46.5.3 Macro Definition Documentation

#### 46.5.3.1 GKO_REGISTER_HOST_OPERATION

```
#define GKO_REGISTER_HOST_OPERATION(
              _name,
              _kernel )
```

**Value:**

```
template <typename...  Args>                                                \
    auto make_##_name(Args&&...  args)                                        \
    {                                                                         \
        return ::gko::detail::make_register_operation(                        \
            #_kernel,                                                         \
            [&args...](auto) { _kernel(std::forward<Args>(args)...); });      \
    }                                                                         \
    static_assert(true,                                                       \
                  "This assert is used to counter the false positive extra " \
                  "semi-colon warnings")
```

Binds a host-side kernel (independent of executor type) to an Operation.

It also defines a helper function which creates the associated operation. Any input arguments passed to the helper function are forwarded to the kernel when the operation is executed. The kernel name is searched for in the namespace where this macro is called. Host operations are used to make computations that are not part of the device kernels visible to profiling loggers and benchmarks.

**Parameters**

| | |
|---|---|
| *_name* | operation name |
| *_kernel* | kernel which will be bound to the operation |

#### 46.5.3.2 Example

```
 {c++}
void host_kernel(int) {
    // do some expensive computations
}
// Bind the kernels to the operation
GKO_REGISTER_HOST_OPERATION(my_op, host_kernel);
int main() {
    // create executor
    auto ref = ReferenceExecutor::create();
    // create the operation
    auto op = make_my_op(5); // x = 5
    ref->run(op);  // run host kernel
}
```

#### 46.5.3.3 GKO_REGISTER_OPERATION

```
#define GKO_REGISTER_OPERATION(
                _name,
                _kernel )
```

Binds a set of device-specific kernels to an Operation.

It also defines a helper function which creates the associated operation. Any input arguments passed to the helper function are forwarded to the kernel when the operation is executed.

The kernels used to bind the operation are searched in `kernels::DEV_TYPE` namespace, where `DEV_TYPE` is replaced by `omp`, `cuda`, `hip`, `dpcpp` and `reference`.

**Parameters**

| | |
|---|---|
| *_name* | operation name |
| *_kernel* | kernel which will be bound to the operation |

#### 46.5.3.4 Example

```
 {c++}
// define the omp, cuda, hip and reference kernels which will be bound to the
// operation
namespace kernels {
namespace omp {
void my_kernel(int x) {
     // omp code
}
}
namespace cuda {
void my_kernel(int x) {
     // cuda code
}
}
namespace hip {
void my_kernel(int x) {
     // hip code
}
}
namespace dpcpp {
void my_kernel(int x) {
     // dpcpp code
}
}
namespace reference {
void my_kernel(int x) {
    // reference code
}
}
```

```
// Bind the kernels to the operation
GKO_REGISTER_OPERATION(my_op, my_kernel);
int main() {
    // create executors
    auto omp = OmpExecutor::create();
    auto cuda = CudaExecutor::create(0, omp);
    auto hip = HipExecutor::create(0, omp);
    auto dpcpp = DpcppExecutor::create(0, omp);
    auto ref = ReferenceExecutor::create();
    // create the operation
    auto op = make_my_op(5); // x = 5
    omp->run(op);  // run omp kernel
    cuda->run(op);  // run cuda kernel
    hip->run(op);  // run hip kernel
    dpcpp->run(op);  // run DPC++ kernel
    ref->run(op);  // run reference kernel
}
```

## 46.6 Factorizations

A module dedicated to the implementation and usage of the Factorizations in Ginkgo.

### Namespaces

- gko::factorization

  *The Factorization namespace.*

### Classes

- class gko::factorization::Ic< ValueType, IndexType >

  *Represents an incomplete Cholesky factorization (IC(0)) of a sparse matrix.*
- class gko::factorization::Ilu< ValueType, IndexType >

  *Represents an incomplete LU factorization – ILU(0) – of a sparse matrix.*
- class gko::factorization::ParIc< ValueType, IndexType >

  *ParIC is an incomplete Cholesky factorization which is computed in parallel.*
- class gko::factorization::ParIct< ValueType, IndexType >

  *ParICT is an incomplete threshold-based Cholesky factorization which is computed in parallel.*
- class gko::factorization::ParIlu< ValueType, IndexType >

  *ParILU is an incomplete LU factorization which is computed in parallel.*
- class gko::factorization::ParIlut< ValueType, IndexType >

  *ParILUT is an incomplete threshold-based LU factorization which is computed in parallel.*

### 46.6.1 Detailed Description

A module dedicated to the implementation and usage of the Factorizations in Ginkgo.

## 46.7 HIP Executor

A module dedicated to the implementation and usage of the HIP executor in Ginkgo.

### Classes

- class gko::HipExecutor

  *This is the Executor subclass which represents the HIP enhanced device.*

### 46.7.1 Detailed Description

A module dedicated to the implementation and usage of the HIP executor in Ginkgo.

## 46.8 Jacobi Preconditioner

A module dedicated to the implementation and usage of the Jacobi Preconditioner in Ginkgo.

### Classes

- class gko::batch::preconditioner::Jacobi< ValueType, IndexType >

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks (stored in a dense row major fashion) of the source operator.*
- struct gko::preconditioner::block_interleaved_storage_scheme< IndexType >

  *Defines the parameters of the interleaved block storage scheme used by block-Jacobi blocks.*
- class gko::preconditioner::Jacobi< ValueType, IndexType >

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks of the source operator.*

### 46.8.1 Detailed Description

A module dedicated to the implementation and usage of the Jacobi Preconditioner in Ginkgo.

# 46.9 Linear Operators

A module dedicated to the implementation and usage of the Linear operators in Ginkgo.

## Modules

- Distributed

  *A module dedicated to the implementation and usage of the distributed linear algebra objects in Ginkgo.*
- Factorizations

  *A module dedicated to the implementation and usage of the Factorizations in Ginkgo.*
- SpMV employing different Matrix formats

  *A module dedicated to the implementation and usage of the various Matrix Formats in Ginkgo.*
- Preconditioners

  *A module dedicated to the implementation and usage of the Preconditioners in Ginkgo.*
- Reordering Algorithms

  *A module dedicated to the implementation and usage of the reordering algorithms in Ginkgo.*
- Solvers

  *A module dedicated to the implementation and usage of the Solvers in Ginkgo.*

## Classes

- class gko::Combination< ValueType >

  *The Combination class can be used to construct a linear combination of multiple linear operators `c1 * op1 + c2 * op2 + ...`*
- class gko::Composition< ValueType >

  *The Composition class can be used to compose linear operators `op1, op2, ..., opn` and obtain the operator `op1 * op2 * ...`*
- class gko::LinOpFactory

  *A LinOpFactory represents a higher order mapping which transforms one linear operator into another.*
- class gko::ReadableFromMatrixData< ValueType, IndexType >

  *A LinOp implementing this interface can read its data from a matrix_data structure.*
- class gko::WritableToMatrixData< ValueType, IndexType >

  *A LinOp implementing this interface can write its data to a matrix_data structure.*
- class gko::Preconditionable

  *A LinOp implementing this interface can be preconditioned.*
- class gko::DiagonalLinOpExtractable

  *The diagonal of a LinOp can be extracted.*
- class gko::DiagonalExtractable< ValueType >

  *The diagonal of a LinOp implementing this interface can be extracted.*
- class gko::EnableAbsoluteComputation< AbsoluteLinOp >

  *The EnableAbsoluteComputation mixin provides the default implementations of `compute_absolute_linop` and the absolute interface.*
- class gko::EnableDefaultReusableLinOpFactory< ConcreteFactory, ConcreteLinOp, ParametersType, ReuseDataType, Polymo

  *EnableDefaultLinOpFactory with LinOpFactory::generate_reuse() support, usually used through GKO_ENABLE_LIN_OP_FACTORY_WI*
- class gko::Perturbation< ValueType >

  *The Perturbation class can be used to construct a LinOp to represent the operation `(identity + scalar * basis * projector)`.*
- class gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType >

  *A Schwarz preconditioner is a simple domain decomposition preconditioner that generalizes the Block Jacobi preconditioner, incorporating options for different local subdomain solvers and overlaps between the subdomains.*

- class gko::experimental::distributed::Vector< ValueType >

  *Vector is a format which explicitly stores (multiple) distributed column vectors in a dense storage format.*
- class gko::factorization::Ic< ValueType, IndexType >

  *Represents an incomplete Cholesky factorization (IC(0)) of a sparse matrix.*
- class gko::factorization::Ilu< ValueType, IndexType >

  *Represents an incomplete LU factorization – ILU(0) – of a sparse matrix.*
- class gko::factorization::ParIc< ValueType, IndexType >

  *ParIC is an incomplete Cholesky factorization which is computed in parallel.*
- class gko::factorization::ParIct< ValueType, IndexType >

  *ParICT is an incomplete threshold-based Cholesky factorization which is computed in parallel.*
- class gko::factorization::ParIlu< ValueType, IndexType >

  *ParILU is an incomplete LU factorization which is computed in parallel.*
- class gko::factorization::ParIlut< ValueType, IndexType >

  *ParILUT is an incomplete threshold-based LU factorization which is computed in parallel.*
- class gko::matrix::Coo< ValueType, IndexType >

  *COO stores a matrix in the coordinate matrix format.*
- class gko::matrix::Csr< ValueType, IndexType >

  *CSR is a matrix format which stores only the nonzero coefficients by compressing each row of the matrix (compressed sparse row format).*
- class gko::matrix::Dense< ValueType >

  *Dense is a matrix format which explicitly stores all values of the matrix.*
- class gko::matrix::Diagonal< ValueType >

  *This class is a utility which efficiently implements the diagonal matrix (a linear operator which scales a vector row wise).*
- class gko::matrix::Ell< ValueType, IndexType >

  *ELL is a matrix format where stride with explicit zeros is used such that all rows have the same number of stored elements.*
- class gko::matrix::Fbcsr< ValueType, IndexType >

  *Fixed-block compressed sparse row storage matrix format.*
- class gko::matrix::Fft

  *This LinOp implements a 1D Fourier matrix using the FFT algorithm.*
- class gko::matrix::Fft2

  *This LinOp implements a 2D Fourier matrix using the FFT algorithm.*
- class gko::matrix::Fft3

  *This LinOp implements a 3D Fourier matrix using the FFT algorithm.*
- class gko::matrix::Hybrid< ValueType, IndexType >

  *HYBRID is a matrix format which splits the matrix into ELLPACK and COO format.*
- class gko::matrix::Identity< ValueType >

  *This class is a utility which efficiently implements the identity matrix (a linear operator which maps each vector to itself).*
- class gko::matrix::IdentityFactory< ValueType >

  *This factory is a utility which can be used to generate Identity operators.*
- class gko::matrix::Permutation< IndexType >

  *Permutation is a matrix format that represents a permutation matrix, i.e.*
- class gko::matrix::RowGatherer< IndexType >

  *RowGatherer is a matrix "format" which stores the gather indices arrays which can be used to gather rows to another matrix.*
- class gko::matrix::ScaledPermutation< ValueType, IndexType >

  *ScaledPermutation is a matrix combining a permutation with scaling factors.*
- class gko::matrix::Sellp< ValueType, IndexType >

  *SELL-P is a matrix format similar to ELL format.*
- class gko::matrix::SparsityCsr< ValueType, IndexType >

*SparsityCsr is a matrix format which stores only the sparsity pattern of a sparse matrix by compressing each row of the matrix (compressed sparse row format).*

- class gko::multigrid::FixedCoarsening< ValueType, IndexType >

  *FixedCoarsening is a very simple coarse grid generation algorithm.*
- class gko::multigrid::Pgm< ValueType, IndexType >

  *Parallel graph match (Pgm) is the aggregate method introduced in the paper M.*
- class gko::multigrid::Pmis< ValueType, IndexType >

  *Parallel modified independent set (Pmis) is the classical coarsening method introduced in the paper H.*
- class gko::multigrid::Rs< ValueType, IndexType >

  *Rs implements the Ruge–Stueben (classical) Algebraic Multigrid (AMG) coarsening strategy for M-matrices.*
- class gko::multigrid::UniformCoarsening< ValueType, IndexType >

  *UniformCoarsening is a simple coarse grid generation algorithm.*
- class gko::preconditioner::Ic< ValueType, IndexType >

  *The Incomplete Cholesky (IC) preconditioner solves the equation $LL^H x = b$ for a given lower triangular matrix $L$ and the right hand side $b$ (can contain multiple right hand sides).*
- class gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >

  *The Incomplete LU (ILU) preconditioner solves the equation $LUx = b$ for a given lower triangular matrix $L$, an upper triangular matrix $U$ and the right hand side $b$ (can contain multiple right hand sides).*
- class gko::preconditioner::Isai< IsaiType, ValueType, IndexType >

  *The Incomplete Sparse Approximate Inverse (ISAI) Preconditioner generates an approximate inverse matrix for a given square matrix $A$, lower triangular matrix $L$, upper triangular matrix $U$ or symmetric positive definite (s.p.d) matrix $B$.*
- class gko::preconditioner::Jacobi< ValueType, IndexType >

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks of the source operator.*
- class gko::preconditioner::NullspaceProjection< ValueType >

  *NullspaceProjection wraps a preconditioner $M$ with projections onto the complement of a nullspace, $z = PMQr$.*
- class gko::solver::Bicg< ValueType >

  *BICG or the Biconjugate gradient method is a Krylov subspace solver.*
- class gko::solver::Bicgstab< ValueType >

  *BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.*
- class gko::solver::CbGmres< ValueType >

  *CB-GMRES or the compressed basis generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.*
- class gko::solver::Cg< ValueType >

  *CG or the conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class gko::solver::Cgs< ValueType >

  *CGS or the conjugate gradient square method is an iterative type Krylov subspace method which is suitable for general systems.*
- class gko::solver::Chebyshev< ValueType >

  *Chebyshev iteration is an iterative method for solving nonsymmetric problems based on some knowledge of the spectrum of the (preconditioned) system matrix.*
- class gko::solver::Fcg< ValueType >

  *FCG or the flexible conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class gko::solver::Gcr< ValueType >

  *GCR or the generalized conjugate residual method is an iterative type Krylov subspace method similar to GMRES which is suitable for nonsymmetric linear systems.*
- class gko::solver::Gmres< ValueType >

  *GMRES or the generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.*
- class gko::solver::Idr< ValueType >

*IDR(s) is an efficient method for solving large nonsymmetric systems of linear equations.*

- class gko::solver::Ir< ValueType >

  *Iterative refinement (IR) is an iterative method that uses another coarse method to approximate the error of the current solution via the current residual.*

- class gko::solver::Minres< ValueType >

  *Minres is an iterative type Krylov subspace method, which is suitable for indefinite and full-rank symmetric/hermitian operators.*

- class gko::solver::Multigrid

  *Multigrid solves a linear system by combining cheap iterative sweeps (smoothers) on a hierarchy of progressively coarser representations of the operator with corrections transferred between adjacent levels.*

- class gko::solver::Nullspace< ValueType >

  *Nullspace represents the (right or left) nullspace of an operator and is the orthogonal projector $P = I - VV^H$ onto its complement, where $V$ is an orthonormal basis of the nullspace.*

- class gko::solver::PipeCg< ValueType >

  *PIPE_CG or the pipelined conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*

- class gko::solver::EnableSolverBase< DerivedType, MatrixType >

  *A LinOp deriving from this CRTP class stores a system matrix.*

- class gko::solver::IterativeBase

  *A LinOp implementing this interface stores a stopping criterion factory.*

- class gko::solver::EnableIterativeBase< DerivedType >

  *A LinOp deriving from this CRTP class stores a stopping criterion factory and allows applying with a guess.*

- class gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >

  *A LinOp implementing this interface stores a system matrix and stopping criterion factory.*

- class gko::solver::LowerTrs< ValueType, IndexType >

  *LowerTrs is the triangular solver which solves the system L x = b, when L is a lower triangular matrix.*

- class gko::solver::UpperTrs< ValueType, IndexType >

  *UpperTrs is the triangular solver which solves the system U x = b, when U is an upper triangular matrix.*

## Macros

- #define GKO_CREATE_FACTORY_PARAMETERS(_parameters_name, _factory_name)

  *This Macro will generate a new type containing the parameters for the factory `_factory_name`.*

- #define GKO_ENABLE_BUILD_METHOD(_factory_name)

  *Defines a build method for the factory, simplifying its construction by removing the repetitive typing of factory's name.*

- #define GKO_FACTORY_PARAMETER(_name, ...)

  *Creates a factory parameter in the factory parameters structure.*

- #define GKO_FACTORY_PARAMETER_SCALAR(_name, _default) GKO_FACTORY_PARAMETER(_name, _default)

  *Creates a scalar factory parameter in the factory parameters structure.*

- #define GKO_FACTORY_PARAMETER_VECTOR(_name, ...) GKO_FACTORY_PARAMETER(_name, __VA_ARGS__)

  *Creates a vector factory parameter in the factory parameters structure.*

- #define GKO_ENABLE_LIN_OP_FACTORY(_lin_op, _parameters_name, _factory_name)

  *This macro will generate a default implementation of a LinOpFactory for the LinOp subclass it is defined in.*

- #define GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE(_lin_op, _parameters_name, _factory_name, _reuse_data_type)

  *Like GKO_ENABLE_LIN_OP_FACTORY, but the factory also supports LinOpFactory::generate_reuse(), with `_reuse_data_type` as its ReuseData.*

## Typedefs

- template<typename ConcreteFactory , typename ConcreteLinOp , typename ParametersType , typename PolymorphicBase = LinOpFactory>
  using gko::EnableDefaultLinOpFactory = EnableDefaultFactory< ConcreteFactory, ConcreteLinOp, ParametersType, PolymorphicBase >

  *This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of LinOpFactory.*

## Functions

- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > gko::initialize (size_type stride, std::initializer_list< typename Matrix::value_type > vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a column-vector.*
- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > gko::initialize (std::initializer_list< typename Matrix::value_type > vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a column-vector.*
- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > gko::initialize (size_type stride, std::initializer_list< std::initializer_list< typename Matrix::value_type >> vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a matrix.*
- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > gko::initialize (std::initializer_list< std::initializer_list< typename Matrix::value_type >> vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a matrix.*

### 46.9.1 Detailed Description

A module dedicated to the implementation and usage of the Linear operators in Ginkgo.

Below we elaborate on one of the most important concepts of Ginkgo, the linear operator. The linear operator (LinOp) is a base class for all linear algebra objects in Ginkgo. The main benefit of having a single base class for the entire collection of linear algebra objects (as opposed to having separate hierarchies for matrices, solvers and preconditioners) is the generality it provides.

### 46.9.2 Advantages of this approach and usage

A common interface often allows for writing more generic code. If a user's routine requires only operations provided by the LinOp interface, the same code can be used for any kind of linear operators, independent of whether these are matrices, solvers or preconditioners. This feature is also extensively used in Ginkgo itself. For example, a preconditioner used inside a Krylov solver is a LinOp. This allows the user to supply a wide variety of preconditioners: either the ones which were designed to be used in this scenario (like ILU or block-Jacobi), a user-supplied matrix which is known to be a good preconditioner for the specific problem, or even another solver (e.g., if constructing a flexible GMRES solver).

For example, a matrix free implementation would require the user to provide an apply implementation and instead of passing the generated matrix to the solver, they would have to provide their apply implementation for all the executors needed and no other code needs to be changed. See The custom-matrix-format program example for more details.

### 46.9.3 Linear operator as a concept

The linear operator (LinOp) is a base class for all linear algebra objects in Ginkgo. The main benefit of having a single base class for the entire collection of linear algebra objects (as opposed to having separate hierarchies for matrices, solvers and preconditioners) is the generality it provides.

First, since all subclasses provide a common interface, the library users are exposed to a smaller set of routines. For example, a matrix-vector product, a preconditioner application, or even a system solve are just different terms given to the operation of applying a certain linear operator to a vector. As such, Ginkgo uses the same routine name, LinOp::apply() for each of these operations, where the actual operation performed depends on the type of linear operator involved in the operation.

Second, a common interface often allows for writing more generic code. If a user's routine requires only operations provided by the LinOp interface, the same code can be used for any kind of linear operators, independent of whether these are matrices, solvers or preconditioners. This feature is also extensively used in Ginkgo itself. For example, a preconditioner used inside a Krylov solver is a LinOp. This allows the user to supply a wide variety of preconditioners: either the ones which were designed to be used in this scenario (like ILU or block-Jacobi), a user-supplied matrix which is known to be a good preconditioner for the specific problem, or even another solver (e.g., if constructing a flexible GMRES solver).

A key observation for providing a unified interface for matrices, solvers, and preconditioners is that the most common operation performed on all of them can be expressed as an application of a linear operator to a vector:

- the sparse matrix-vector product with a matrix $A$ is a linear operator application $y = Ax$;
- the application of a preconditioner is a linear operator application $y = M^{-1}x$, where $M$ is an approximation of the original system matrix $A$ (thus a preconditioner represents an "approximate inverse" operator $M^{-1}$).
- the system solve $Ax = b$ can be viewed as linear operator application $x = A^{-1}b$ (it goes without saying that the implementation of linear system solves does not follow this conceptual idea), so a linear system solver can be viewed as a representation of the operator $A^{-1}$.

Finally, direct manipulation of LinOp objects is rarely required in simple scenarios. As an illustrative example, one could construct a fixed-point iteration routine $x_{k+1} = Lx_k + b$ as follows:

```
std::unique_ptr<matrix::Dense<» calculate_fixed_point(
        int iters, const LinOp *L, const matrix::Dense<> *x0
        const matrix::Dense<> *b)
{
    auto x = gko::clone(x0);
    auto tmp = gko::clone(x0);
    auto one = Dense<>::create(L->get_executor(), {1.0,});
    for (int i = 0; i < iters; ++i) {
        L->apply(tmp, x);
        x->add_scaled(one, b);
        tmp->copy_from(x);
    }
    return x;
}
```

Here, if $L$ is a matrix, LinOp::apply() refers to the matrix vector product, and `L->apply(a, b)` computes $b = L \cdot a$. `x->add_scaled(one, b)` is the `axpy` vector update $x := x + b$.

The interesting part of this example is the apply() routine at line 5 of the function body. Since this routine is part of the LinOp base class, the fixed-point iteration routine can calculate a fixed point not only for matrices, but for any type of linear operator.

Linear Operators

### 46.9.4 Macro Definition Documentation

#### 46.9.4.1 GKO_CREATE_FACTORY_PARAMETERS

```
#define GKO_CREATE_FACTORY_PARAMETERS(
              _parameters_name,
              _factory_name )
```

**Value:**

```
public:
      \
    class _factory_name;                                                \
    struct _parameters_name##_type                                      \
        :  public ::gko::enable_parameters_type<_parameters_name##_type, \
                                                _factory_name>
```

This Macro will generate a new type containing the parameters for the factory `_factory_name`.

For more details, see GKO_ENABLE_LIN_OP_FACTORY(). It is required to use this macro **before** calling the macro GKO_ENABLE_LIN_OP_FACTORY(). It is also required to use the same names for all parameters between both macros.

**Parameters**

| | |
|---|---|
| *_parameters_name* | name of the parameters member in the class |
| *_factory_name* | name of the generated factory type |

#### 46.9.4.2 GKO_ENABLE_BUILD_METHOD

```
#define GKO_ENABLE_BUILD_METHOD(
              _factory_name )
```

**Value:**

```
static auto build()->decltype(_factory_name::create())                  \
    {                                                                       \
        return _factory_name::create();                                     \
    }                                                                       \
    static_assert(true,                                                     \
                  "This assert is used to counter the false positive extra " \
                  "semi-colon warnings")
```

Defines a build method for the factory, simplifying its construction by removing the repetitive typing of factory's name.

**Parameters**

| | |
|---|---|
| *_factory_name* | the factory for which to define the method |

#### 46.9.4.3 GKO_ENABLE_LIN_OP_FACTORY

```
#define GKO_ENABLE_LIN_OP_FACTORY(
              _lin_op,
              _parameters_name,
              _factory_name )
```

This macro will generate a default implementation of a LinOpFactory for the LinOp subclass it is defined in.

It is required to first call the macro GKO_CREATE_FACTORY_PARAMETERS() before this one in order to instantiate the parameters type first.

The list of parameters for the factory should be defined in a code block after the macro definition, and should contain a list of GKO_FACTORY_PARAMETER_∗ declarations. The class should provide a constructor with signature ↩ _lin_op(const _factory_name ∗, std::shared_ptr<const LinOp>) which the factory will use a callback to construct the object.

A minimal example of a linear operator is the following:

```c++ struct MyLinOp : public LinOp { GKO_ENABLE_LIN_OP_FACTORY(MyLinOp, my_parameters, Factory) { // a factory parameter named "my_value", of type int and default // value of 5 int GKO_FACTORY_PARAMETER_SCALAR(my_value, 5); // a factory parameter named my_pair of type std::pair<int, int> // and default value {5, 5} std::pair<int, int> GKO_FACTORY_PARAMETER_VECTOR(my_pair, 5, 5); }; // constructor needed by LinOp explicit My↩ LinOp(std::shared_ptr<const Executor> exec) { : LinOp(exec) {} // constructor needed by the factory explicit MyLinOp(const Factory ∗factory, std::shared_ptr<const LinOp> matrix) : LinOp(factory->get_executor()), matrix->get_size()), // store factory's parameters locally my_parameters_{factory->get_parameters()}, { int value = my_parameters_.my_value; // do something with value }
MyLinOp can then be created as follows:
```c++
auto exec = gko::ReferenceExecutor::create();
// create a factory with default `my_value` parameter
auto fact = MyLinOp::build().on(exec);
// create a operator using the factory:
auto my_op = fact->generate(gko::matrix::Identity::create(exec, 2));
std::cout « my_op->get_my_parameters().my_value;  // prints 5
// create a factory with custom `my_value` parameter
auto fact = MyLinOp::build().with_my_value(0).on(exec);
// create a operator using the factory:
auto my_op = fact->generate(gko::matrix::Identity::create(exec, 2));
std::cout « my_op->get_my_parameters().my_value;  // prints 0
```

**Note**

It is possible to combine both the #GKO_CREATE_FACTORY_PARAMETER_∗() macros with this one in a unique macro for class **templates** (not with regular classes). Splitting this into two distinct macros allows to use them in all contexts. See https://stackoverflow.com/q/50202718/9385966 for more details.

**Parameters**

| | |
|---|---|
| *_lin_op* | concrete operator for which the factory is to be created [CRTP parameter] |
| *_parameters_name* | name of the parameters member in the class (its type is <_parameters_name>_type, the protected member's name is <_parameters_name>_, and the public getter's name is get_<_parameters_name>()) |
| *_factory_name* | name of the generated factory type |

#### 46.9.4.4 GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE

```
#define GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE(
              _lin_op,
              _parameters_name,
```
```

```
                _factory_name,
                _reuse_data_type )
```

Like GKO_ENABLE_LIN_OP_FACTORY, but the factory also supports LinOpFactory::generate_reuse(), with _↩ reuse_data_type as its ReuseData.

The LinOp additionally provides what EnableDefaultReusableLinOpFactory requires, which may be private. The reuse data type can be defined after the macro, but has to be declared before it:

```
 {c++}
class MyLinOp :  public LinOp {
public:
    GKO_CREATE_FACTORY_PARAMETERS(parameters, Factory) { ...  };
    class reuse_data_type;
    GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE(MyLinOp, parameters, Factory,
                                         reuse_data_type);
    GKO_ENABLE_BUILD_METHOD(Factory);
    class reuse_data_type :  public LinOpFactory::ReuseData { ...  };
private:
    // used by generate() and generate_reuse()
    MyLinOp(const Factory*, std::shared_ptr<const LinOp>);
    MyLinOp(const Factory*, std::shared_ptr<const LinOp>, reuse_data_type&);
    static void check_reuse_consistent(const Factory*, const LinOp*,
                                       const reuse_data_type&);
};
```

**Parameters**

| | |
|---|---|
| *_lin_op* | concrete operator for which the factory is to be created |
| *_parameters_name* | name of the parameters member in the class |
| *_factory_name* | name of the generated factory type |
| *_reuse_data_type* | the ReuseData type of the factory |

**Note**

This interface is experimental.

#### 46.9.4.5 GKO_FACTORY_PARAMETER

```
#define GKO_FACTORY_PARAMETER(
                _name,
                ...  )
```

**Value:**

```
_name{__VA_ARGS__};                                                          \
                                                                               \
    template <typename...  Args>                                               \
    auto with_##_name(Args&&...  _value)                                       \
        ->std::decay_t<decltype(*(this->self()))>&                             \
    {                                                                          \
        using type = decltype(this->_name);                                    \
        this->_name = type{std::forward<Args>(_value)...};                     \
        return *(this->self());                                                \
    }                                                                          \
    static_assert(true,                                                        \
                  "This assert is used to counter the false positive extra " \
                  "semi-colon warnings")
```

Creates a factory parameter in the factory parameters structure.

**Parameters**

| | |
|---|---|
| *_name* | name of the parameter |
| *VA_ARGS* | default value of the parameter |

**See also**

GKO_ENABLE_LIN_OP_FACTORY for more details, and usage example

#### 46.9.4.6 GKO_FACTORY_PARAMETER_SCALAR

```
#define GKO_FACTORY_PARAMETER_SCALAR(
            _name,
            _default ) GKO_FACTORY_PARAMETER(_name, _default)
```

Creates a scalar factory parameter in the factory parameters structure.

Scalar in this context means that the constructor for this type only takes a single parameter.

**Parameters**

| | |
|---|---|
| *_name* | name of the parameter |
| *_default* | default value of the parameter |

**See also**

GKO_ENABLE_LIN_OP_FACTORY for more details, and usage example

#### 46.9.4.7 GKO_FACTORY_PARAMETER_VECTOR

```
#define GKO_FACTORY_PARAMETER_VECTOR(
            _name,
            ...  ) GKO_FACTORY_PARAMETER(_name, __VA_ARGS__)
```

Creates a vector factory parameter in the factory parameters structure.

Vector in this context means that the constructor for this type takes multiple parameters.

**Parameters**

| | |
|---|---|
| *_name* | name of the parameter |
| *_default* | default value of the parameter |

**See also**

GKO_ENABLE_LIN_OP_FACTORY for more details, and usage example

### 46.9.5 Typedef Documentation

#### 46.9.5.1 EnableDefaultLinOpFactory

```
template<typename ConcreteFactory , typename ConcreteLinOp , typename ParametersType , typename
PolymorphicBase = LinOpFactory>
using gko::EnableDefaultLinOpFactory = typedef EnableDefaultFactory<ConcreteFactory, Concrete↩
LinOp, ParametersType, PolymorphicBase>
```

This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of LinOpFactory.

**Template Parameters**

| | |
|---|---|
| *ConcreteFactory* | the concrete factory which is being implemented [CRTP parameter] |
| *ConcreteLinOp* | the concrete LinOp type which this factory produces, needs to have a constructor which takes a const ConcreteFactory $*$, and an std::shared_ptr<const LinOp> as parameters. |
| *ParametersType* | a subclass of enable_parameters_type template which defines all of the parameters of the factory |
| *PolymorphicBase* | parent of ConcreteFactory in the polymorphic hierarchy, has to be a subclass of LinOpFactory |

## 46.9.6 Function Documentation

#### 46.9.6.1 initialize() [1/4]

```
template<typename Matrix , typename...  TArgs>
std::unique_ptr<Matrix> gko::initialize (
            size_type stride,
            std::initializer_list< std::initializer_list< typename Matrix::value_type >>
vals,
            std::shared_ptr< const Executor > exec,
            TArgs &&...  create_args )
```

Creates and initializes a matrix.

This function first creates a temporary Dense matrix, fills it with passed in values, and then converts the matrix to the requested type.

**Template Parameters**

| | |
|---|---|
| *Matrix* | matrix type to initialize (Dense has to implement the ConvertibleTo<Matrix> interface) |
| *TArgs* | argument types for Matrix::create method (not including the implied Executor as the first argument) |

**Parameters**

| | |
|---|---|
| *stride* | row stride for the temporary Dense matrix |
| *vals* | values used to initialize the matrix |
| *exec* | Executor associated to the matrix |

**Parameters**

| | |
|---|---|
| *create_args* | additional arguments passed to Matrix::create, not including the Executor, which is passed as the first argument |

```
1685 {
1686     using dense = matrix::Dense<typename Matrix::value_type>;
1687     size_type num_rows = vals.size();
1688     size_type num_cols = num_rows > 0 ?  begin(vals)->size() :  1;
1689     auto tmp =
1690         dense::create(exec->get_master(), dim<2>{num_rows, num_cols}, stride);
1691     size_type ridx = 0;
1692     for (const auto& row :  vals) {
1693         size_type cidx = 0;
1694         for (const auto& elem :  row) {
1695             tmp->at(ridx, cidx) = elem;
1696             ++cidx;
1697         }
1698         ++ridx;
1699     }
1700     auto mtx = Matrix::create(exec, std::forward<TArgs>(create_args)...);
1701     tmp->move_to(mtx);
1702     return mtx;
1703 }
```

References gko::matrix::Dense< ValueType >::at().

#### 46.9.6.2 initialize() [2/4]

```
template<typename Matrix , typename...  TArgs>
std::unique_ptr<Matrix> gko::initialize (
              size_type stride,
              std::initializer_list< typename Matrix::value_type > vals,
              std::shared_ptr< const Executor > exec,
              TArgs &&...  create_args )
```

Creates and initializes a column-vector.

This function first creates a temporary Dense matrix, fills it with passed in values, and then converts the matrix to the requested type.

**Template Parameters**

| | |
|---|---|
| *Matrix* | matrix type to initialize (Dense has to implement the ConvertibleTo<Matrix> interface) |
| *TArgs* | argument types for Matrix::create method (not including the implied Executor as the first argument) |

**Parameters**

| | |
|---|---|
| *stride* | row stride for the temporary Dense matrix |
| *vals* | values used to initialize the vector |
| *exec* | Executor associated to the vector |
| *create_args* | additional arguments passed to Matrix::create, not including the Executor, which is passed as the first argument |

```
1615 {
1616     using dense = matrix::Dense<typename Matrix::value_type>;
1617     size_type num_rows = vals.size();
1618     auto tmp = dense::create(exec->get_master(), dim<2>{num_rows, 1}, stride);
1619     size_type idx = 0;
1620     for (const auto& elem :  vals) {
```

```
1621         tmp->at(idx) = elem;
1622         ++idx;
1623     }
1624     auto mtx = Matrix::create(exec, std::forward<TArgs>(create_args)...);
1625     tmp->move_to(mtx);
1626     return mtx;
1627 }
```

References gko::matrix::Dense< ValueType >::at().

#### 46.9.6.3 initialize() [3/4]

```
template<typename Matrix , typename...  TArgs>
std::unique_ptr<Matrix> gko::initialize (
             std::initializer_list< std::initializer_list< typename Matrix::value_type >>
vals,
             std::shared_ptr< const Executor > exec,
             TArgs &&...  create_args )
```

Creates and initializes a matrix.

This function first creates a temporary Dense matrix, fills it with passed in values, and then converts the matrix to the requested type. The stride of the intermediate Dense matrix is set to the number of columns of the initializer list.

**Template Parameters**

| | |
|---|---|
| *Matrix* | matrix type to initialize (Dense has to implement the ConvertibleTo<Matrix> interface) |
| *TArgs* | argument types for Matrix::create method (not including the implied Executor as the first argument) |

**Parameters**

| | |
|---|---|
| *vals* | values used to initialize the matrix |
| *exec* | Executor associated to the matrix |
| *create_args* | additional arguments passed to Matrix::create, not including the Executor, which is passed as the first argument |

```
1732 {
1733     return initialize<Matrix>(vals.size() > 0 ?  begin(vals)->size() :  0, vals,
1734                               std::move(exec),
1735                               std::forward<TArgs>(create_args)...);
1736 }
```

#### 46.9.6.4 initialize() [4/4]

```
template<typename Matrix , typename...  TArgs>
std::unique_ptr<Matrix> gko::initialize (
             std::initializer_list< typename Matrix::value_type > vals,
             std::shared_ptr< const Executor > exec,
             TArgs &&...  create_args )
```

Creates and initializes a column-vector.

This function first creates a temporary Dense matrix, fills it with passed in values, and then converts the matrix to the requested type. The stride of the intermediate Dense matrix is set to 1.

### Template Parameters

| | |
|---|---|
| *Matrix* | matrix type to initialize (Dense has to implement the ConvertibleTo<Matrix> interface) |
| *TArgs* | argument types for Matrix::create method (not including the implied Executor as the first argument) |

### Parameters

| | |
|---|---|
| *vals* | values used to initialize the vector |
| *exec* | Executor associated to the vector |
| *create_args* | additional arguments passed to Matrix::create, not including the Executor, which is passed as the first argument |

```
1653 {
1654     return initialize<Matrix>(1, vals, std::move(exec),
1655                               std::forward<TArgs>(create_args)...);
1656 }
```

## 46.10 Logging

A module dedicated to the implementation and usage of the Logging in Ginkgo.

### Namespaces

- gko::batch::log

  *The logger namespace .*
- gko::log

  *The logger namespace .*

### Classes

- class gko::batch::log::BatchConvergence< ValueType >

  *Logs the final residuals and iteration counts for a batch solver.*
- class gko::log::Convergence< ValueType >

  *Convergence is a Logger which logs data strictly from the* `criterion_check_completed` *event.*
- class gko::log::Papi< ValueType >

  *Papi is a Logger which logs every event to the PAPI software.*
- class gko::log::PerformanceHint

  *PerformanceHint is a Logger which analyzes the performance of the application and outputs hints for unnecessary copies and allocations.*
- class gko::log::Stream< ValueType >

  *Stream is a Logger which logs every event to a stream.*

### 46.10.1 Detailed Description

A module dedicated to the implementation and usage of the Logging in Ginkgo.

The Logger class represents a simple Logger object. It comprises all masks and events internally. Every new logging event addition should be done here. The Logger class also provides a default implementation for most events which do nothing, therefore it is not an obligation to change all classes which derive from Logger, although it is good practice. The logger class is built using event masks to control which events should be logged, and which should not.

## 46.11 SpMV employing different Matrix formats

A module dedicated to the implementation and usage of the various Matrix Formats in Ginkgo.

### Classes

- class gko::experimental::distributed::Vector< ValueType >

  *Vector is a format which explicitly stores (multiple) distributed column vectors in a dense storage format.*
- class gko::batch::matrix::Csr< ValueType, IndexType >

  *Csr is a general sparse matrix format that stores the column indices for each nonzero entry and a cumulative sum of the number of nonzeros in each row.*
- class gko::batch::matrix::Dense< ValueType >

  *Dense is a batch matrix format which explicitly stores all values of the matrix in each of the batches.*
- class gko::batch::matrix::Ell< ValueType, IndexType >

  *Ell is a sparse matrix format that stores the same number of nonzeros in each row, enabling coalesced accesses.*
- class gko::batch::matrix::Identity< ValueType >

  *The batch Identity matrix, which represents a batch of Identity matrices.*
- class gko::matrix::Coo< ValueType, IndexType >

  *COO stores a matrix in the coordinate matrix format.*
- class gko::matrix::Csr< ValueType, IndexType >

  *CSR is a matrix format which stores only the nonzero coefficients by compressing each row of the matrix (compressed sparse row format).*
- class gko::matrix::Dense< ValueType >

  *Dense is a matrix format which explicitly stores all values of the matrix.*
- class gko::matrix::Diagonal< ValueType >

  *This class is a utility which efficiently implements the diagonal matrix (a linear operator which scales a vector row wise).*
- class gko::matrix::Ell< ValueType, IndexType >

  *ELL is a matrix format where stride with explicit zeros is used such that all rows have the same number of stored elements.*
- class gko::matrix::Fbcsr< ValueType, IndexType >

  *Fixed-block compressed sparse row storage matrix format.*
- class gko::matrix::Fft

  *This LinOp implements a 1D Fourier matrix using the FFT algorithm.*
- class gko::matrix::Fft2

  *This LinOp implements a 2D Fourier matrix using the FFT algorithm.*
- class gko::matrix::Fft3

  *This LinOp implements a 3D Fourier matrix using the FFT algorithm.*
- class gko::matrix::Hybrid< ValueType, IndexType >

  *HYBRID is a matrix format which splits the matrix into ELLPACK and COO format.*
- class gko::matrix::Identity< ValueType >

  *This class is a utility which efficiently implements the identity matrix (a linear operator which maps each vector to itself).*
- class gko::matrix::IdentityFactory< ValueType >

  *This factory is a utility which can be used to generate Identity operators.*
- class gko::matrix::Permutation< IndexType >

  *Permutation is a matrix format that represents a permutation matrix, i.e.*
- class gko::matrix::ScaledPermutation< ValueType, IndexType >

  *ScaledPermutation is a matrix combining a permutation with scaling factors.*
- class gko::matrix::Sellp< ValueType, IndexType >

  *SELL-P is a matrix format similar to ELL format.*
- class gko::matrix::SparsityCsr< ValueType, IndexType >

  *SparsityCsr is a matrix format which stores only the sparsity pattern of a sparse matrix by compressing each row of the matrix (compressed sparse row format).*

### 46.11.1 Detailed Description

A module dedicated to the implementation and usage of the various Matrix Formats in Ginkgo.

## 46.12 Multigrid

A module dedicated to the implementation and usage of the Multigrid solver and its components in Ginkgo.

### Modules

- Multigrid Level

  *A module dedicated to the implementation and usage of the individual levels of a Multigrid hierarchy in Ginkgo.*

### Classes

- class gko::multigrid::FixedCoarsening< ValueType, IndexType >

  *FixedCoarsening is a very simple coarse grid generation algorithm.*
- class gko::multigrid::MultigridLevel

  *This class represents two levels in a multigrid hierarchy.*
- class gko::multigrid::EnableMultigridLevel< ValueType >

  *The EnableMultigridLevel gives the default implementation of MultigridLevel with composition and provides* `set_multigrid_level` *function.*
- class gko::multigrid::Pgm< ValueType, IndexType >

  *Parallel graph match (Pgm) is the aggregate method introduced in the paper M.*
- class gko::multigrid::Pmis< ValueType, IndexType >

  *Parallel modified independent set (Pmis) is the classical coarsening method introduced in the paper H.*
- class gko::multigrid::Rs< ValueType, IndexType >

  *Rs implements the Ruge–Stueben (classical) Algebraic Multigrid (AMG) coarsening strategy for M-matrices.*
- class gko::multigrid::UniformCoarsening< ValueType, IndexType >

  *UniformCoarsening is a simple coarse grid generation algorithm.*
- class gko::solver::Multigrid

  *Multigrid solves a linear system by combining cheap iterative sweeps (smoothers) on a hierarchy of progressively coarser representations of the operator with corrections transferred between adjacent levels.*

### 46.12.1 Detailed Description

A module dedicated to the implementation and usage of the Multigrid solver and its components in Ginkgo.

## 46.13 Multigrid Level

A module dedicated to the implementation and usage of the individual levels of a Multigrid hierarchy in Ginkgo.

### Classes

- class gko::multigrid::FixedCoarsening< ValueType, IndexType >

  *FixedCoarsening is a very simple coarse grid generation algorithm.*
- class gko::multigrid::Pgm< ValueType, IndexType >

  *Parallel graph match (Pgm) is the aggregate method introduced in the paper M.*
- class gko::multigrid::Pmis< ValueType, IndexType >

  *Parallel modified independent set (Pmis) is the classical coarsening method introduced in the paper H.*
- class gko::multigrid::Rs< ValueType, IndexType >

  *Rs implements the Ruge–Stueben (classical) Algebraic Multigrid (AMG) coarsening strategy for M-matrices.*
- class gko::multigrid::UniformCoarsening< ValueType, IndexType >

  *UniformCoarsening is a simple coarse grid generation algorithm.*

### 46.13.1 Detailed Description

A module dedicated to the implementation and usage of the individual levels of a Multigrid hierarchy in Ginkgo.

A multigrid level provides the restriction and prolongation operators that transfer between a fine and a coarse representation of the problem.

## 46.14 OpenMP Executor

A module dedicated to the implementation and usage of the OpenMP executor in Ginkgo.

### Classes

- class gko::OmpExecutor

  *This is the Executor subclass which represents the OpenMP device (typically CPU).*

### 46.14.1 Detailed Description

A module dedicated to the implementation and usage of the OpenMP executor in Ginkgo.

## 46.15 Preconditioners

A module dedicated to the implementation and usage of the Preconditioners in Ginkgo.

### Modules

- Jacobi Preconditioner

  *A module dedicated to the implementation and usage of the Jacobi Preconditioner in Ginkgo.*

### Namespaces

- gko::experimental::distributed::preconditioner

  *The Preconditioner namespace.*
- gko::preconditioner

  *The Preconditioner namespace.*

### Classes

- class gko::Preconditionable

  *A LinOp implementing this interface can be preconditioned.*
- class gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType >

  *A Schwarz preconditioner is a simple domain decomposition preconditioner that generalizes the Block Jacobi preconditioner, incorporating options for different local subdomain solvers and overlaps between the subdomains.*
- class gko::batch::preconditioner::Jacobi< ValueType, IndexType >

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks (stored in a dense row major fashion) of the source operator.*
- class gko::preconditioner::GaussSeidel< ValueType, IndexType >

  *This class generates the Gauss-Seidel preconditioner.*
- class gko::preconditioner::Ic< ValueType, IndexType >

  *The Incomplete Cholesky (IC) preconditioner solves the equation $LL^H x = b$ for a given lower triangular matrix $L$ and the right hand side $b$ (can contain multiple right hand sides).*
- class gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >

  *The Incomplete LU (ILU) preconditioner solves the equation $LUx = b$ for a given lower triangular matrix $L$, an upper triangular matrix $U$ and the right hand side $b$ (can contain multiple right hand sides).*
- class gko::preconditioner::Isai< IsaiType, ValueType, IndexType >

  *The Incomplete Sparse Approximate Inverse (ISAI) Preconditioner generates an approximate inverse matrix for a given square matrix $A$, lower triangular matrix $L$, upper triangular matrix $U$ or symmetric positive definite (s.p.d) matrix $B$.*
- class gko::preconditioner::Jacobi< ValueType, IndexType >

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks of the source operator.*
- class gko::preconditioner::NullspaceProjection< ValueType >

  *NullspaceProjection wraps a preconditioner $M$ with projections onto the complement of a nullspace, $z = PMQr$.*
- class gko::preconditioner::Sor< ValueType, IndexType >

  *This class generates the (S)SOR preconditioner.*

### 46.15.1 Detailed Description

A module dedicated to the implementation and usage of the Preconditioners in Ginkgo.

## 46.16 Reference Executor

A module dedicated to the implementation and usage of the Reference executor in Ginkgo.

### Classes

- class gko::ReferenceExecutor

  *This is a specialization of the OmpExecutor, which runs the reference implementations of the kernels used for debugging purposes.*

### 46.16.1 Detailed Description

A module dedicated to the implementation and usage of the Reference executor in Ginkgo.

# 46.17 Reordering Algorithms

A module dedicated to the implementation and usage of the reordering algorithms in Ginkgo.

## Namespaces

- gko::experimental::reorder

  *The Reorder namespace.*
- gko::reorder

  *The Reorder namespace.*

## Classes

- class gko::reorder::Rcm< ValueType, IndexType >

  *Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.*
- class gko::experimental::reorder::Rcm< IndexType >

  *Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.*

## Macros

- #define GKO_ENABLE_REORDERING_BASE_FACTORY(_reordering_base, _parameters_name, _↩ factory_name)

  *This macro will generate a default implementation of a ReorderingBaseFactory for the ReorderingBase subclass it is defined in.*

## 46.17.1 Detailed Description

A module dedicated to the implementation and usage of the reordering algorithms in Ginkgo.

A reordering permutes the rows and columns of a matrix, for example to reduce fill-in in a factorization or to move large entries onto the diagonal.

## 46.17.2 Macro Definition Documentation

### 46.17.2.1 GKO_ENABLE_REORDERING_BASE_FACTORY

```
#define GKO_ENABLE_REORDERING_BASE_FACTORY(
            _reordering_base,
            _parameters_name,
            _factory_name )
```

This macro will generate a default implementation of a ReorderingBaseFactory for the ReorderingBase subclass it is defined in.

This macro is very similar to the macro #ENABLE_LIN_OP_FACTORY(). A more detailed description of the use of these type of macros can be found there.

**Parameters**

| | |
|---|---|
| *_reordering_base* | concrete operator for which the factory is to be created [CRTP parameter] |
| *_parameters_name* | name of the parameters member in the class (its type is `<_parameters_name>_type`, the protected member's name is `<_parameters_name>_`, and the public getter's name is `get_<_parameters_name>()`) |
| *_factory_name* | name of the generated factory type |

# 46.18 Solvers

A module dedicated to the implementation and usage of the Solvers in Ginkgo.

## Modules

- Multigrid

  *A module dedicated to the implementation and usage of the Multigrid solver and its components in Ginkgo.*

## Namespaces

- gko::solver

  *The ginkgo Solve namespace.*
- gko::solver::multigrid

  *The solver multigrid namespace.*

## Classes

- class gko::batch::solver::Bicgstab< ValueType >

  *BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.*
- class gko::batch::solver::Cg< ValueType >

  *Cg or the Conjugate Gradient is a Krylov subspace solver.*
- class gko::batch::solver::BatchSolver

  *The BatchSolver is a base class for all batched solvers and provides the common getters and setter for these batched solver classes.*
- class gko::solver::Bicg< ValueType >

  *BICG or the Biconjugate gradient method is a Krylov subspace solver.*
- class gko::solver::Bicgstab< ValueType >

  *BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.*
- class gko::solver::CbGmres< ValueType >

  *CB-GMRES or the compressed basis generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.*
- class gko::solver::Cg< ValueType >

  *CG or the conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class gko::solver::Cgs< ValueType >

  *CGS or the conjugate gradient square method is an iterative type Krylov subspace method which is suitable for general systems.*
- class gko::solver::Chebyshev< ValueType >

  *Chebyshev iteration is an iterative method for solving nonsymmetric problems based on some knowledge of the spectrum of the (preconditioned) system matrix.*
- class gko::solver::Fcg< ValueType >

  *FCG or the flexible conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class gko::solver::Gcr< ValueType >

  *GCR or the generalized conjugate residual method is an iterative type Krylov subspace method similar to GMRES which is suitable for nonsymmetric linear systems.*
- class gko::solver::Gmres< ValueType >

  *GMRES or the generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.*

- class gko::solver::Idr< ValueType >

  *IDR(s) is an efficient method for solving large nonsymmetric systems of linear equations.*
- class gko::solver::Ir< ValueType >

  *Iterative refinement (IR) is an iterative method that uses another coarse method to approximate the error of the current solution via the current residual.*
- class gko::solver::Minres< ValueType >

  *Minres is an iterative type Krylov subspace method, which is suitable for indefinite and full-rank symmetric/hermitian operators.*
- class gko::solver::Multigrid

  *Multigrid solves a linear system by combining cheap iterative sweeps (smoothers) on a hierarchy of progressively coarser representations of the operator with corrections transferred between adjacent levels.*
- class gko::solver::Nullspace< ValueType >

  *Nullspace represents the (right or left) nullspace of an operator and is the orthogonal projector $P = I - VV^H$ onto its complement, where $V$ is an orthonormal basis of the nullspace.*
- class gko::solver::PipeCg< ValueType >

  *PIPE_CG or the pipelined conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class gko::solver::EnableSolverBase< DerivedType, MatrixType >

  *A LinOp deriving from this CRTP class stores a system matrix.*
- class gko::solver::IterativeBase

  *A LinOp implementing this interface stores a stopping criterion factory.*
- class gko::solver::EnableIterativeBase< DerivedType >

  *A LinOp deriving from this CRTP class stores a stopping criterion factory and allows applying with a guess.*
- class gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >

  *A LinOp implementing this interface stores a system matrix and stopping criterion factory.*
- class gko::solver::LowerTrs< ValueType, IndexType >

  *LowerTrs is the triangular solver which solves the system L x = b, when L is a lower triangular matrix.*
- class gko::solver::UpperTrs< ValueType, IndexType >

  *UpperTrs is the triangular solver which solves the system U x = b, when U is an upper triangular matrix.*

### 46.18.1 Detailed Description

A module dedicated to the implementation and usage of the Solvers in Ginkgo.

# 46.19 Stopping criteria

A module dedicated to the implementation and usage of the Stopping Criteria in Ginkgo.

## Namespaces

- gko::stop

  *The Stopping criterion namespace.*

## Classes

- class gko::stop::Combined

  *The Combined class is used to combine multiple criterions together through an OR operation.*
- class gko::stop::Iteration

  *The Iteration class is a stopping criterion which stops the iteration process after a preset number of iterations.*
- class gko::stop::ResidualNormBase< ValueType >

  *The ResidualNormBase class provides a framework for stopping criteria related to the residual norm.*
- class gko::stop::ResidualNorm< ValueType >

  *Stopping criterion based on the explicit residual norm* $\|r_k\| = \|b - Ax_k\|$.
- class gko::stop::ImplicitResidualNorm< ValueType >

  *Stopping criterion based on a solver-maintained squared residual-norm estimate* $\rho_k^2 \approx \|r_k\|^2$.
- class gko::stop::ResidualNormReduction< ValueType >

  *The ResidualNormReduction class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the initial residual, i.e.*
- class gko::stop::RelativeResidualNorm< ValueType >

  *The RelativeResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the right-hand side, i.e.*
- class gko::stop::AbsoluteResidualNorm< ValueType >

  *The AbsoluteResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold, i.e.*
- class gko::stopping_status

  *This class is used to keep track of the stopping status of one vector.*
- class gko::stop::Time

  *The Time class is a stopping criterion which stops the iteration process after a certain amount of time has passed.*

## Enumerations

- enum gko::stop::mode

  *The mode for the residual norm criterion.*

## Functions

- template<typename FactoryContainer >
  std::shared_ptr< const CriterionFactory > gko::stop::combine (FactoryContainer &&factories)

  *Combines multiple criterion factories into a single combined criterion factory.*

### 46.19.1 Detailed Description

A module dedicated to the implementation and usage of the Stopping Criteria in Ginkgo.

### 46.19.2 Enumeration Type Documentation

#### 46.19.2.1 mode

```
enum gko::stop::mode  [strong]
```

The mode for the residual norm criterion.

- absolute: Check for tolerance against residual norm. $||r|| \leq \tau$
- initial_resnorm: Check for tolerance relative to the initial residual norm. $||r|| \leq \tau \times ||r_0||$
- rhs_norm: Check for tolerance relative to the rhs norm. $||r|| \leq \tau \times ||b||$

```
37 { absolute, initial_resnorm, rhs_norm };
```

### 46.19.3 Function Documentation

#### 46.19.3.1 combine()

```
template<typename FactoryContainer >
std::shared_ptr<const CriterionFactory> gko::stop::combine (
            FactoryContainer && factories )
```

Combines multiple criterion factories into a single combined criterion factory.

This function treats a singleton container as a special case and avoids creating an additional object and just returns the input factory.

**Template Parameters**

| | |
|---|---|
| *FactoryContainer* | a random access container type |

**Parameters**

| | |
|---|---|
| *factories* | a list of factories to combined |

### Returns

a combined criterion factory if the input contains multiple factories or the input factory if the input contains only one factory

```
106 {
107     switch (factories.size()) {
108     case 0:
109         GKO_NOT_SUPPORTED(nullptr);
110     case 1:
111         if (factories[0] == nullptr) {
112             GKO_NOT_SUPPORTED(nullptr);
113         }
114         return factories[0];
115     default:
116         if (factories[0] == nullptr) {
117             // first factory must be valid to capture executor
118             GKO_NOT_SUPPORTED(nullptr);
119         } else {
120             auto exec = factories[0]->get_executor();
121             return Combined::build()
122                 .with_criteria(std::forward<FactoryContainer>(factories))
123                 .on(exec);
124         }
125     }
126 }
```

# Chapter 47

# Namespace Documentation

## 47.1 gko Namespace Reference

The Ginkgo namespace.

### Namespaces

- accessor

  *The accessor namespace.*
- factorization

  *The Factorization namespace.*
- log

  *The logger namespace .*
- matrix

  *The matrix namespace.*
- multigrid

  *The multigrid components namespace.*
- name_demangling

  *The name demangling namespace.*
- preconditioner

  *The Preconditioner namespace.*
- reorder

  *The Reorder namespace.*
- solver

  *The ginkgo Solve namespace.*
- stop

  *The Stopping criterion namespace.*
- syn

  *The Synthesizer namespace.*
- xstd

  *The namespace for functionalities after C++14 standard.*

## Classes

- class AbsoluteComputable

  *The AbsoluteComputable is an interface that allows to get the component wise absolute of a LinOp.*
- class AbstractFactory

  *The AbstractFactory is a generic interface template that enables easy implementation of the abstract factory design pattern.*
- class AllocationError

  *AllocationError is thrown if a memory allocation fails.*
- class Allocator

  *Provides generic allocation and deallocation functionality to be used by an Executor.*
- class amd_device

  *amd_device handles the number of executor on Amd devices and have the corresponding recursive_mutex.*
- struct are_all_integral

  *Evaluates if all template arguments Args fulfill std::is_integral.*
- class array

  *An array is a container which encapsulates fixed-sized arrays, stored on the Executor tied to the array.*
- class BadDimension

  *BadDimension is thrown if an operation is being applied to a LinOp with bad dimensions.*
- struct batch_dim

  *A type representing the dimensions of a multidimensional batch object.*
- class bfloat16

  *A class providing basic support for bfloat16 precision floating point types.*
- class BlockOperator

  *A BlockOperator represents a linear operator that is partitioned into multiple blocks.*
- class BlockSizeError

  *Error that denotes issues between block sizes and matrix dimensions.*
- class Cloneable

  *Interface for objects that can be cloned.*
- class Combination

  *The Combination class can be used to construct a linear combination of multiple linear operators* `c1 * op1 + c2 * op2 + ...`
- class Composition

  *The Composition class can be used to compose linear operators* `op1, op2, ..., opn` *and obtain the operator* `op1 * op2 * ...`
- class ConvertibleTo

  *ConvertibleTo interface is used to mark that the implementer can be converted to the object of ResultType.*
- class CpuAllocator

  *Allocator using new/delete.*
- class CpuAllocatorBase

  *Implement this interface to provide an allocator for OmpExecutor or ReferenceExecutor.*
- class CpuTimer

  *A timer using std::chrono::steady_clock for timing.*
- struct cpx_real_type

  *Access the underlying real type of a complex number.*
- class CublasError

  *CublasError is thrown when a cuBLAS routine throws a non-zero error code.*
- class cuda_stream

  *An RAII wrapper for a custom CUDA stream.*
- class CudaAllocator

  *Allocator using cudaMalloc.*
- class CudaAllocatorBase

*Implement this interface to provide an allocator for CudaExecutor.*

- class CudaError

  *CudaError is thrown when a CUDA routine throws a non-zero error code.*
- class CudaExecutor

  *This is the Executor subclass which represents the CUDA device.*
- class CudaTimer

  *A timer using events for timing on a CudaExecutor.*
- class CufftError

  *CufftError is thrown when a cuFFT routine throws a non-zero error code.*
- class CurandError

  *CurandError is thrown when a cuRAND routine throws a non-zero error code.*
- class CusparseError

  *CusparseError is thrown when a cuSPARSE routine throws a non-zero error code.*
- struct default_converter

  *Used to convert objects of type `S` to objects of type `R` using static_cast.*
- class deferred_factory_parameter

  *Represents a factory parameter of factory type that can either initialized by a pre-existing factory or by passing in a factory_parameters object whose `.on(exec)` will be called to instantiate a factory.*
- class device_matrix_data

  *This type is a device-side equivalent to matrix_data.*
- class DiagonalExtractable

  *The diagonal of a LinOp implementing this interface can be extracted.*
- class DiagonalLinOpExtractable

  *The diagonal of a LinOp can be extracted.*
- struct dim

  *A type representing the dimensions of a multidimensional object.*
- class DimensionMismatch

  *DimensionMismatch is thrown if an operation is being applied to LinOps of incompatible size.*
- class DpcppExecutor

  *This is the Executor subclass which represents a DPC++ enhanced device.*
- class DpcppTimer

  *A timer using kernels for timing on a DpcppExecutor in profiling mode.*
- class enable_parameters_type

  *The enable_parameters_type mixin is used to create a base implementation of the factory parameters structure.*
- class EnableAbsoluteComputation

  *The EnableAbsoluteComputation mixin provides the default implementations of `compute_absolute_linop` and the absolute interface.*
- class EnableCloneable

  *This mixin is used to enable a default Cloneable::clone() implementation and similar for objects that have implemented copy and move constructors.*
- class EnableCreateMethod

  *This mixin implements a static `create()` method on `ConcreteType` that dynamically allocates the memory, uses the passed-in arguments to construct the object, and returns an std::unique_ptr to such an object.*
- class EnableDefaultFactory

  *This mixin provides a default implementation of a concrete factory.*
- class EnableDefaultReusableLinOpFactory

  *EnableDefaultLinOpFactory with LinOpFactory::generate_reuse() support, usually used through GKO_ENABLE_LIN_OP_FACTORY_WI*
- class Error

  *The Error class is used to report exceptional behaviour in library functions.*
- class Executor

  *The first step in using the Ginkgo library consists of creating an executor.*

- class executor_deleter

  *This is a deleter that uses an executor's `free` method to deallocate the data.*
- class half

  *A class providing basic support for half precision floating point types.*
- class hip_stream

  *An RAII wrapper for a custom HIP stream.*
- class HipAllocatorBase

  *Implement this interface to provide an allocator for HipExecutor.*
- class HipblasError

  *HipblasError is thrown when a hipBLAS routine throws a non-zero error code.*
- class HipError

  *HipError is thrown when a HIP routine throws a non-zero error code.*
- class HipExecutor

  *This is the Executor subclass which represents the HIP enhanced device.*
- class HipfftError

  *HipfftError is thrown when a hipFFT routine throws a non-zero error code.*
- class HiprandError

  *HiprandError is thrown when a hipRAND routine throws a non-zero error code.*
- class HipsparseError

  *HipsparseError is thrown when a hipSPARSE routine throws a non-zero error code.*
- class HipTimer

  *A timer using events for timing on a HipExecutor.*
- class index_set

  *An index set class represents an ordered set of intervals.*
- class InvalidData

  *InvalidData is thrown in case an object in Ginkgo contains invalid data, either because the data was user-provided, or because of a bug.*
- class InvalidStateError

  *Exception thrown if an object is in an invalid state.*
- class KernelNotFound

  *KernelNotFound is thrown if Ginkgo cannot find a kernel which satisfies the criteria imposed by the input arguments.*
- class LinOpFactory

  *A LinOpFactory represents a higher order mapping which transforms one linear operator into another.*
- struct local_span

  *A span that is used exclusively for local numbering.*
- class matrix_assembly_data

  *This structure is used as an intermediate type to assemble a sparse matrix.*
- struct matrix_data

  *This structure is used as an intermediate data type to store a sparse matrix.*
- struct matrix_data_entry

  *Type used to store nonzeros.*
- class MetisError

  *MetisError is thrown when METIS routine throws an error code.*
- class MpiError

  *MpiError is thrown when a MPI routine throws a non-zero error code.*
- class NotCompiled

  *NotCompiled is thrown when attempting to call an operation which is a part of a module that was not compiled on the system.*
- class NotImplemented

  *NotImplemented is thrown in case an operation has not yet been implemented (but will be implemented in the future).*
- class NotSupported

*NotSupported is thrown in case it is not possible to perform the requested operation on the given object type.*

- class null_deleter

  *This is a deleter that does not delete the object.*

- class nvidia_device

  *nvidia_device handles the number of executor on Nvidia devices and have the corresponding recursive_mutex.*

- class OmpExecutor

  *This is the Executor subclass which represents the OpenMP device (typically CPU).*

- class Operation

  *Operations can be used to define functionalities whose implementations differ among devices.*

- class OutOfBoundsError

  *OutOfBoundsError is thrown if a memory access is detected to be out-of-bounds.*

- class OverflowError

  *OverflowError is thrown when an index calculation for storage requirements overflows.*

- class Permutable

  *Linear operators which support permutation should implement the Permutable interface.*

- class Perturbation

  *The Perturbation class can be used to construct a LinOp to represent the operation `(identity + scalar * basis * projector)`.*

- class PolymorphicObject

  *A PolymorphicObject is the abstract base for all "heavy" objects in Ginkgo that behave polymorphically.*

- class precision_reduction

  *This class is used to encode storage precisions of low precision algorithms.*

- class Preconditionable

  *A LinOp implementing this interface can be preconditioned.*

- class ptr_param

  *This class is used for function parameters in the place of raw pointers.*

- class range

  *A range is a multidimensional view of the memory.*

- class ReadableFromMatrixData

  *A LinOp implementing this interface can read its data from a matrix_data structure.*

- class ReferenceExecutor

  *This is a specialization of the OmpExecutor, which runs the reference implementations of the kernels used for debugging purposes.*

- class ScaledIdentityAddable

  *Adds the operation M <- a I + b M for matrix M, identity operator I and scalars a and b, where M is the calling object.*

- class scoped_device_id_guard

  *This move-only class uses RAII to set the device id within a scoped block, if necessary.*

- struct segmented_array

  *A minimal interface for a segmented array.*

- struct span

  *A span is a lightweight structure used to create sub-ranges from other ranges.*

- class stopping_status

  *This class is used to keep track of the stopping status of one vector.*

- class StreamError

  *StreamError is thrown if accessing a stream failed.*

- class time_point

  *An opaque wrapper for a time point generated by a timer.*

- class Timer

  *Represents a generic timer that can be used to record time points and measure time differences on host or device streams.*

- class Transposable

*Linear operators which support transposition should implement the Transposable interface.*

- class UnsupportedMatrixProperty

  *Exception throws if a matrix does not have a property required by a numerical method.*

- class UseComposition

  *The UseComposition class can be used to store the composition information in LinOp.*

- class ValueMismatch

  *ValueMismatch is thrown if two values are not equal.*

- struct version

  *This structure is used to represent versions of various Ginkgo modules.*

- class version_info

  *Ginkgo uses version numbers to label new features and to communicate backward compatibility guarantees:*

- class WritableToMatrixData

  *A LinOp implementing this interface can write its data to a matrix_data structure.*

## Typedefs

- template<typename ConcreteFactory , typename ConcreteLinOp , typename ParametersType , typename PolymorphicBase = LinOpFactory>
  using EnableDefaultLinOpFactory = EnableDefaultFactory< ConcreteFactory, ConcreteLinOp, ParameterType, PolymorphicBase >

  *This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of LinOpFactory.*

- template<typename T >
  using is_complex_s = detail::is_complex_impl< T >

  *Allows to check if T is a complex value during compile time by accessing the `value` attribute of this struct.*

- template<typename T >
  using is_complex_or_scalar_s = detail::is_complex_or_scalar_impl< T >

  *Allows to check if T is a complex or scalar value during compile time by accessing the `value` attribute of this struct.*

- template<typename T >
  using remove_complex = typename detail::remove_complex_s< T >::type

  *Obtain the type which removed the complex of complex/scalar type or the template parameter of class by accessing the `type` attribute of this struct.*

- template<typename T >
  using to_complex = typename detail::to_complex_s< T >::type

  *Obtain the type which adds the complex of complex/scalar type or the template parameter of class by accessing the `type` attribute of this struct.*

- template<typename T >
  using to_real = remove_complex< T >

  *to_real is alias of remove_complex*

- template<typename T >
  using next_precision_base = typename detail::next_precision_base_impl< T >::type

  *Obtains the next type in the singly-linked precision list.*

- template<typename T >
  using previous_precision_base = next_precision_base< T >

  *Obtains the previous type in the singly-linked precision list.*

- template<typename T , int step = 1>
  using next_precision = typename detail::find_precision_impl< T, step >::type

  *Obtains the next `move` type of T in the singly-linked precision corresponding bfloat16/half.*

- template<typename T , int step = 1>
  using previous_precision = typename detail::find_precision_impl< T, -step >::type

  *Obtains the previous `move` type of T in the singly-linked precision corresponding bfloat16/half.*

- template<typename T >
  using reduce_precision = typename detail::reduce_precision_impl< T >::type

  *Obtains the next type in the hierarchy with lower precision than T.*
- template<typename T >
  using increase_precision = typename detail::increase_precision_impl< T >::type

  *Obtains the next type in the hierarchy with higher precision than T.*
- template<typename... Ts>
  using highest_precision = typename detail::highest_precision_variadic< Ts... >::type

  *Obtains the smallest arithmetic type that is able to store elements of all template parameter types exactly.*
- template<typename T , size_type Limit = sizeof(uint16) * byte_size>
  using truncate_type = std::conditional_t< detail::type_size_impl< T >::value >=2 *Limit, typename detail↩
  ::truncate_type_impl< T >::type, T >

  *Truncates the type by half (by dropping bits), but ensures that it is at least `Limit` bits wide.*
- using size_type = std::size_t

  *Integral type used for allocation quantities.*
- using int8 = std::int8_t

  *8-bit signed integral type.*
- using int16 = std::int16_t

  *16-bit signed integral type.*
- using int32 = std::int32_t

  *32-bit signed integral type.*
- using int64 = std::int64_t

  *64-bit signed integral type.*
- using uint8 = std::uint8_t

  *8-bit unsigned integral type.*
- using uint16 = std::uint16_t

  *16-bit unsigned integral type.*
- using uint32 = std::uint32_t

  *32-bit unsigned integral type.*
- using uint64 = std::uint64_t

  *64-bit unsigned integral type.*
- using uintptr = std::uintptr_t

  *Unsigned integer type capable of holding a pointer to void.*
- using float16 = half

  *16 bit floating point type.*
- using float32 = float

  *Single precision floating point type.*
- using float64 = double

  *Double precision floating point type.*
- using full_precision = double

  *The most precise floating-point type.*
- using default_precision = double

  *Precision used if no precision is explicitly specified.*

## Enumerations

- enum log_propagation_mode { log_propagation_mode::never, log_propagation_mode::automatic }

  *How Logger events are propagated to their Executor.*
- enum allocation_mode

  *Specify the mode of allocation for CUDA/HIP GPUs.*
- enum layout_type { layout_type::array, layout_type::coordinate }

  *Specifies the layout type when writing data in matrix market format.*

## Functions

- template<typename ValueType >
  ValueType reduce_add (const array< ValueType > &input_arr, const ValueType init_val=0)

  *Reduce (sum) the values in the array.*
- template<typename ValueType >
  void reduce_add (const array< ValueType > &input_arr, array< ValueType > &result)

  *Reduce (sum) the values in the array.*
- template<typename ValueType >
  array< ValueType > make_array_view (std::shared_ptr< const Executor > exec, size_type size, ValueType *data)

  *Helper function to create an array view deducing the value type.*
- template<typename ValueType >
  detail::const_array_view< ValueType > make_const_array_view (std::shared_ptr< const Executor > exec, size_type size, const ValueType *data)

  *Helper function to create a const array view deducing the value type.*
- template<typename DimensionType >
  batch_dim< 2, DimensionType > transpose (const batch_dim< 2, DimensionType > &input)

  *Returns a batch_dim object with its dimensions swapped for batched operators.*
- template<typename DimensionType >
  constexpr dim< 2, DimensionType > transpose (const dim< 2, DimensionType > &dimensions) noexcept

  *Returns a dim<2> object with its dimensions swapped.*
- template<typename T >
  constexpr bool is_complex ()

  *Checks if T is a complex type.*
- template<typename T >
  constexpr bool is_complex_or_scalar ()

  *Checks if T is a complex/scalar type.*
- template<typename T >
  constexpr reduce_precision< T > round_down (T val)

  *Reduces the precision of the input parameter.*
- template<typename T >
  constexpr increase_precision< T > round_up (T val)

  *Increases the precision of the input parameter.*
- constexpr int64 ceildiv (int64 num, int64 den)

  *Performs integer division with rounding up.*
- template<typename T >
  constexpr T zero ()

  *Returns the additive identity for T.*
- template<typename T >
  constexpr T zero (const T &)

  *Returns the additive identity for T.*
- template<typename T >
  constexpr T one ()

  *Returns the multiplicative identity for T.*
- template<typename T >
  constexpr T one (const T &)

  *Returns the multiplicative identity for T.*
- template<typename T >
  constexpr bool is_zero (T value)

  *Returns true if and only if the given value is zero.*
- template<typename T >
  constexpr bool is_nonzero (T value)

*Returns true if and only if the given value is not zero.*

- template<typename T >
  constexpr T max (const T &x, const T &y)

  *Returns the larger of the arguments.*
- template<typename T >
  constexpr T min (const T &x, const T &y)

  *Returns the smaller of the arguments.*
- template<typename T >
  constexpr auto real (const T &x)

  *Returns the real part of the object.*
- template<typename T >
  constexpr auto imag (const T &x)

  *Returns the imaginary part of the object.*
- template<typename T >
  constexpr auto conj (const T &x)

  *Returns the conjugate of an object.*
- template<typename T >
  constexpr auto squared_norm (const T &x) -> decltype(real(conj(x) ∗x))

  *Returns the squared norm of the object.*
- template<typename T >
  constexpr std::enable_if_t<!is_complex_s< T >::value, T > abs (const T &x)

  *Returns the absolute value of the object.*
- template<typename T >
  constexpr T pi ()

  *Returns the value of pi.*
- template<typename T >
  constexpr std::complex< remove_complex< T > > unit_root (int64 n, int64 k=1)

  *Returns the value of exp(2 ∗ pi ∗ i ∗ k / n), i.e.*
- template<typename T >
  constexpr uint32 get_significant_bit (const T &n, uint32 hint=0u) noexcept

  *Returns the position of the most significant bit of the number.*
- template<typename T >
  constexpr T get_superior_power (const T &base, const T &limit, const T &hint=T{1}) noexcept

  *Returns the smallest power of `base` not smaller than `limit`.*
- template<typename T >
  std::enable_if_t<!is_complex_s< T >::value, bool > is_finite (const T &value)

  *Checks if a floating point number is finite, meaning it is neither +/- infinity nor NaN.*
- template<typename T >
  std::enable_if_t< is_complex_s< T >::value, bool > is_finite (const T &value)

  *Checks if all components of a complex value are finite, meaning they are neither +/- infinity nor NaN.*
- template<typename T >
  T safe_divide (T a, T b)

  *Computes the quotient of the given parameters, guarding against division by zero.*
- template<typename T >
  std::enable_if_t<!is_complex_s< T >::value, bool > is_nan (const T &value)

  *Checks if a floating point number is NaN.*
- template<typename T >
  std::enable_if_t< is_complex_s< T >::value, bool > is_nan (const T &value)

  *Checks if any component of a complex value is NaN.*
- template<typename T >
  constexpr std::enable_if_t<!is_complex_s< T >::value, T > nan ()

  *Returns a quiet NaN of the given type.*

- template<typename T >
  constexpr std::enable_if_t< is_complex_s< T >::value, T > nan ()

  *Returns a complex with both components quiet NaN.*
- template<typename ValueType = default_precision, typename IndexType = int32>
  matrix_data< ValueType, IndexType > read_raw (std::istream &is)

  *Reads a matrix stored in matrix market format from an input stream.*
- template<typename ValueType = default_precision, typename IndexType = int32>
  matrix_data< ValueType, IndexType > read_binary_raw (std::istream &is)

  *Reads a matrix stored in Ginkgo's binary matrix format from an input stream.*
- template<typename ValueType = default_precision, typename IndexType = int32>
  matrix_data< ValueType, IndexType > read_generic_raw (std::istream &is)

  *Reads a matrix stored in either binary or matrix market format from an input stream.*
- template<typename ValueType , typename IndexType >
  void write_raw (std::ostream &os, const matrix_data< ValueType, IndexType > &data, layout_type layout=layout_type::coordinate)

  *Writes a matrix_data structure to a stream in matrix market format.*
- template<typename ValueType , typename IndexType >
  void write_binary_raw (std::ostream &os, const matrix_data< ValueType, IndexType > &data)

  *Writes a matrix_data structure to a stream in binary format.*
- template<typename MatrixType , typename StreamType , typename... MatrixArgs>
  std::unique_ptr< MatrixType > read (StreamType &&is, MatrixArgs &&... args)

  *Reads a matrix stored in matrix market format from an input stream.*
- template<typename MatrixType , typename StreamType , typename... MatrixArgs>
  std::unique_ptr< MatrixType > read_binary (StreamType &&is, MatrixArgs &&... args)

  *Reads a matrix stored in binary format from an input stream.*
- template<typename MatrixType , typename StreamType , typename... MatrixArgs>
  std::unique_ptr< MatrixType > read_generic (StreamType &&is, MatrixArgs &&... args)

  *Reads a matrix stored either in binary or matrix market format from an input stream.*
- template<typename MatrixPtrType , typename StreamType , std::enable_if_t<!std::is_same_v< std::remove_cv_t< detail::pointee< MatrixPtrType >>, LinOp >> * = nullptr>
  void write (StreamType &&os, MatrixPtrType &&matrix, layout_type layout=detail::mtx_io_traits< std::remove_cv_t< detail::pointee< MatrixPtrType >>>::default_layout)

  *Writes a matrix into an output stream in matrix market format.*
- void write (std::ostream &os, ptr_param< const LinOp > matrix)

  *Writes a LinOp into an output stream in matrix market format.*
- template<typename MatrixPtrType , typename StreamType >
  void write_binary (StreamType &&os, MatrixPtrType &&matrix)

  *Writes a matrix into an output stream in binary format.*
- template<typename R , typename T >
  std::unique_ptr< R, std::function< void(R *)> > copy_and_convert_to (std::shared_ptr< const Executor > exec, T *obj)

  *Converts the object to R and places it on Executor exec.*
- template<typename R , typename T >
  std::unique_ptr< const R, std::function< void(const R *)> > copy_and_convert_to (std::shared_ptr< const Executor > exec, const T *obj)

  *Converts the object to R and places it on Executor exec.*
- template<typename R , typename T >
  std::shared_ptr< R > copy_and_convert_to (std::shared_ptr< const Executor > exec, std::shared_ptr< T > obj)

  *Converts the object to R and places it on Executor exec.*
- template<typename R , typename T >
  std::shared_ptr< const R > copy_and_convert_to (std::shared_ptr< const Executor > exec, std::shared_ptr< const T > obj)

- template<typename ValueType , typename Ptr >
  detail::temporary_conversion< std::conditional_t< std::is_const< detail::pointee< Ptr > >::value, const matrix::Dense< ValueType >, matrix::Dense< ValueType > > > make_temporary_conversion (Ptr &&matrix)

  *Convert the given LinOp from matrix::Dense<...> to matrix::Dense<ValueType>.*
- template<typename ValueType , typename Function , typename... Args>
  void precision_dispatch (Function fn, Args *... linops)

  *Calls the given function with each given argument LinOp temporarily converted into matrix::Dense<ValueType> as parameters.*
- template<typename ValueType , typename Function >
  void precision_dispatch_real_complex (Function fn, const LinOp *in, LinOp *out)

  *Calls the given function with the given LinOps temporarily converted to matrix::Dense<ValueType>* as parameters.*
- template<typename ValueType , typename Function >
  void precision_dispatch_real_complex (Function fn, const LinOp *alpha, const LinOp *in, LinOp *out)

  *Calls the given function with the given LinOps temporarily converted to matrix::Dense<ValueType>* as parameters.*
- template<typename ValueType , typename Function >
  void precision_dispatch_real_complex (Function fn, const LinOp *alpha, const LinOp *in, const LinOp *beta, LinOp *out)

  *Calls the given function with the given LinOps temporarily converted to matrix::Dense<ValueType>* as parameters.*
- template<typename ValueType , typename Function >
  void mixed_precision_dispatch (Function fn, const LinOp *in, LinOp *out)

  *Calls the given function with each given argument LinOp converted into matrix::Dense<ValueType> as parameters.*
- template<typename ValueType , typename Function , std::enable_if_t< is_complex< ValueType >() > * = nullptr>
  void mixed_precision_dispatch_real_complex (Function fn, const LinOp *in, LinOp *out)

  *Calls the given function with the given LinOps cast to their dynamic type matrix::Dense<ValueType>* as parameters.*
- template<typename Ptr >
  detail::temporary_clone< detail::pointee< Ptr > > make_temporary_clone (std::shared_ptr< const Executor > exec, Ptr &&ptr)

  *Creates a temporary_clone.*
- template<typename Ptr >
  detail::temporary_clone< detail::pointee< Ptr > > make_temporary_output_clone (std::shared_ptr< const Executor > exec, Ptr &&ptr)

  *Creates a uninitialized temporary_clone that will be copied back to the input afterwards.*
- constexpr bool operator== (precision_reduction x, precision_reduction y) noexcept

  *Checks if two precision_reduction encodings are equal.*
- constexpr bool operator!= (precision_reduction x, precision_reduction y) noexcept

  *Checks if two precision_reduction encodings are different.*
- template<typename IndexType >
  constexpr IndexType invalid_index ()

  *Value for an invalid signed index type.*
- template<typename OwningPointer >
  detail::shared_type< OwningPointer > share (OwningPointer &&p)

  *Marks the object pointed to by $p$ as shared.*
- template<typename OwningPointer >
  std::remove_reference< OwningPointer >::type && give (OwningPointer &&p)

  *Marks that the object pointed to by $p$ can be given to the callee.*
- template<typename Pointer >
  std::enable_if< detail::have_ownership_s< Pointer >::value, detail::pointee< Pointer > * >::type lend (const Pointer &p)

  *Returns a non-owning (plain) pointer to the object pointed to by $p$.*
- template<typename Pointer >
  std::enable_if<!detail::have_ownership_s< Pointer >::value, detail::pointee< Pointer > * >::type lend (const Pointer &p)

  *Returns a non-owning (plain) pointer to the object pointed to by $p$.*

- template<typename T , typename U >
  std::decay_t< T > * as (U *obj)

  *Performs polymorphic type conversion.*
- template<typename T , typename U >
  const std::decay_t< T > * as (const U *obj)

  *Performs polymorphic type conversion.*
- template<typename T , typename U >
  std::decay_t< T > * as (ptr_param< U > obj)

  *Performs polymorphic type conversion on a ptr_param.*
- template<typename T , typename U >
  const std::decay_t< T > * as (ptr_param< const U > obj)

  *Performs polymorphic type conversion.*
- template<typename T , typename U >
  std::unique_ptr< std::decay_t< T > > as (std::unique_ptr< U > &&obj)

  *Performs polymorphic type conversion of a unique_ptr.*
- template<typename T , typename U >
  std::shared_ptr< std::decay_t< T > > as (std::shared_ptr< U > obj)

  *Performs polymorphic type conversion of a shared_ptr.*
- template<typename T , typename U >
  std::shared_ptr< const std::decay_t< T > > as (std::shared_ptr< const U > obj)

  *Performs polymorphic type conversion of a shared_ptr.*
- template<typename Pointer , std::enable_if_t< detail::is_cloneable< detail::pointee< Pointer >>()> * = nullptr>
  detail::cloned_type< Pointer > clone (const Pointer &p)

  *Creates a unique clone of the object pointed to by* `p`.
- template<typename Pointer , std::enable_if_t< detail::has_clone_to< detail::pointee< Pointer >>()> * = nullptr>
  detail::cloned_type< Pointer > clone (std::shared_ptr< const Executor > exec, const Pointer &p)

  *Creates a unique clone of the object pointed to by* `p` *on Executor* `exec`.
- std::ostream & operator<< (std::ostream &os, const version &ver)

  *Prints version information to a stream.*
- std::ostream & operator<< (std::ostream &os, const version_info &ver_info)

  *Prints library version information in human-readable format to a stream.*
- template<template< typename, typename > class MatrixType, typename... Args>
  auto with_matrix_type (Args &&... create_args)

  *This function returns a type that delays a call to MatrixType::create.*
- template<typename VecPtr >
  std::unique_ptr< matrix::Dense< typename detail::pointee< VecPtr >::value_type > > make_dense_view (VecPtr &&vector)

  *Creates a view of a given Dense vector.*
- template<typename VecPtr >
  std::unique_ptr< const matrix::Dense< typename detail::pointee< VecPtr >::value_type > > make_const_dense_view (VecPtr &&vector)

  *Creates a view of a given Dense vector.*
- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > initialize (size_type stride, std::initializer_list< typename Matrix::value_type > vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a column-vector.*
- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > initialize (std::initializer_list< typename Matrix::value_type > vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a column-vector.*
- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > initialize (size_type stride, std::initializer_list< std::initializer_list< typename Matrix::value_type >> vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

*Creates and initializes a matrix.*

- template<typename Matrix , typename... TArgs>
  std::unique_ptr< Matrix > initialize (std::initializer_list< std::initializer_list< typename Matrix::value_type >> vals, std::shared_ptr< const Executor > exec, TArgs &&... create_args)

  *Creates and initializes a matrix.*
- bool operator== (const stopping_status &x, const stopping_status &y) noexcept

  *Checks if two stopping statuses are equivalent.*
- bool operator!= (const stopping_status &x, const stopping_status &y) noexcept

  *Checks if two stopping statuses are different.*

## Variables

- constexpr size_type byte_size = CHAR_BIT

  *Number of bits in a byte.*

### 47.1.1 Detailed Description

The Ginkgo namespace.

### 47.1.2 Typedef Documentation

#### 47.1.2.1 highest_precision

```
template<typename... Ts>
using gko::highest_precision = typedef typename detail::highest_precision_variadic<Ts...>↩
::type
```

Obtains the smallest arithmetic type that is able to store elements of all template parameter types exactly.

All template type parameters need to be either real or complex types, mixing them is not possible.

Formally, it computes a right-fold over the type list, with the highest precision of a pair of real arithmetic types T1, T2 computed as `decltype(T1{} + T2{})`, or `std::complex<highest_precision<remove_↩complex<T1>, remove_complex<T2>>>` for complex types.

#### 47.1.2.2 is_complex_or_scalar_s

```
template<typename T >
using gko::is_complex_or_scalar_s = typedef detail::is_complex_or_scalar_impl<T>
```

Allows to check if T is a complex or scalar value during compile time by accessing the `value` attribute of this struct.

If `value` is `true`, T is a complex/scalar type, if it is `false`, T is not a complex/scalar type.

**Template Parameters**

| | |
|---|---|
| *T* | type to check |

#### 47.1.2.3 is_complex_s

```
template<typename T >
using gko::is_complex_s = typedef detail::is_complex_impl<T>
```

Allows to check if T is a complex value during compile time by accessing the `value` attribute of this struct.

If `value` is `true`, T is a complex type, if it is `false`, T is not a complex type.

**Template Parameters**

| | |
|---|---|
| *T* | type to check |

#### 47.1.2.4 previous_precision_base

```
template<typename T >
using gko::previous_precision_base = typedef next_precision_base<T>
```

Obtains the previous type in the singly-linked precision list.

**Note**

Currently our lists contains only two elements, so this is the same as next_precision.

#### 47.1.2.5 remove_complex

```
template<typename T >
using gko::remove_complex = typedef typename detail::remove_complex_s<T>::type
```

Obtain the type which removed the complex of complex/scalar type or the template parameter of class by accessing the `type` attribute of this struct.

**Template Parameters**

| | |
|---|---|
| *T* | type to remove complex |

**Note**

remove_complex<class> can not be used in friend class declaration.

#### 47.1.2.6 to_complex

```
template<typename T >
using gko::to_complex = typedef typename detail::to_complex_s<T>::type
```

Obtain the type which adds the complex of complex/scalar type or the template parameter of class by accessing the `type` attribute of this struct.

**Template Parameters**

| | |
|---|---|
| *T* | type to complex_type |

**Note**

to_complex<class> can not be used in friend class declaration. the followings are the error message from different combination. friend to_complex<Csr>; error: can not recognize it is class correctly. friend class to_complex<Csr>; error: using alias template specialization friend class to_complex_s<Csr<ValueType,↩ IndexType>>::type; error: can not recognize it is class correctly.

#### 47.1.2.7 to_real

```
template<typename T >
using gko::to_real = typedef remove_complex<T>
```

to_real is alias of remove_complex

**Template Parameters**

| | |
|---|---|
| *T* | type to real |

### 47.1.3 Enumeration Type Documentation

#### 47.1.3.1 allocation_mode

```
enum gko::allocation_mode  [strong]
```

Specify the mode of allocation for CUDA/HIP GPUs.

`device` allocates memory on the device and Unified Memory model is not used.

`unified_global` allocates memory on the device, but is accessible by the host through the Unified memory model.

`unified_host` allocates memory on the host and it is not available on devices which do not have concurrent accesses switched on, but this access can be explicitly switched on, when necessary.

```
61 { device, unified_global, unified_host };
```

#### 47.1.3.2 layout_type

```
enum gko::layout_type  [strong]
```

Specifies the layout type when writing data in matrix market format.

**Enumerator**

| | |
|---|---|
| array | The matrix should be written as dense matrix in column-major order. |
| coordinate | The matrix should be written as a sparse matrix in coordinate format. |

```
93                      {
97     array,
101     coordinate
102 };
```

#### 47.1.3.3 log_propagation_mode

```
enum gko::log_propagation_mode  [strong]
```

How Logger events are propagated to their Executor.

**Enumerator**

| | |
|---|---|
| never | Events only get reported at loggers attached to the triggering object. (Except for allocation/free, copy and Operations, since they happen at the Executor). |
| automatic | Events get reported to loggers attached to the triggering object and propagating loggers (Logger::needs_propagation() return true) attached to its Executor. |

```
33                              {
39     never,
45     automatic
46 };
```

### 47.1.4 Function Documentation

#### 47.1.4.1 abs()

```
template<typename T >
constexpr std::enable_if_t<!is_complex_s<T>::value, T> gko::abs (
            const T & x )  [inline], [constexpr]
```

Returns the absolute value of the object.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the object |

**Parameters**

| | |
|---|---|
| *x* | the object |

**Returns**

x >= zero<T>() ? x : -x;

```
963 {
964     return x >= zero<T>() ?  x :  -x;
965 }
```

Referenced by is_finite().

#### 47.1.4.2 as() [1/7]

```
template<typename T , typename U >
const std::decay_t<T>* gko::as (
            const U * obj )  [inline]
```

Performs polymorphic type conversion.

This is the constant version of the function.

**Template Parameters**

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

**Parameters**

| | |
|---|---|
| *obj* | the object which should be converted |

**Returns**

If successful, returns a pointer to the subtype, otherwise throws NotSupported.

```
284 {
```

```
285     if (auto p = dynamic_cast<const std::decay_t<T>*>(obj)) {
286         return p;
287     } else {
288         throw NotSupported(__FILE__, __LINE__,
289                            std::string{"gko::as<"} +
290                                name_demangling::get_type_name(typeid(T)) + ">",
291                            name_demangling::get_type_name(typeid(*obj)));
292     }
293 }
```

#### 47.1.4.3 as() [2/7]

```
template<typename T , typename U >
const std::decay_t<T>* gko::as (
            ptr_param< const U > obj )  [inline]
```

Performs polymorphic type conversion.

This is the constant version of the function.

**Template Parameters**

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

**Parameters**

| | |
|---|---|
| *obj* | the object which should be converted |

**Returns**

If successful, returns a pointer to the subtype, otherwise throws NotSupported.

```
328 {
329     return as<T>(obj.get());
330 }
```

References gko::ptr_param< T >::get().

#### 47.1.4.4 as() [3/7]

```
template<typename T , typename U >
std::decay_t<T>* gko::as (
            ptr_param< U > obj )  [inline]
```

Performs polymorphic type conversion on a ptr_param.

**Template Parameters**

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

#### Parameters

| | |
|---|---|
| *obj* | the object which should be converted |

#### Returns

If successful, returns a pointer to the subtype, otherwise throws NotSupported.

```
309 {
310     return as<T>(obj.get());
311 }
```

References gko::ptr_param< T >::get().

### 47.1.4.5 as() [4/7]

```
template<typename T , typename U >
std::shared_ptr<const std::decay_t<T> > gko::as (
            std::shared_ptr< const U > obj )  [inline]
```

Performs polymorphic type conversion of a shared_ptr.

This is the constant version of the function.

#### Template Parameters

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

#### Parameters

| | |
|---|---|
| *obj* | the shared_ptr to the object which should be converted. |

#### Returns

If successful, returns a shared_ptr to the subtype, otherwise throws NotSupported. This pointer shares ownership with the input pointer.

```
397 {
398     auto ptr = std::dynamic_pointer_cast<const std::decay_t<T>>(obj);
399     if (ptr) {
400         return ptr;
401     } else {
402         throw NotSupported(__FILE__, __LINE__, __func__,
403                            name_demangling::get_type_name(typeid(*obj)));
404     }
405 }
```

### 47.1.4.6 as() [5/7]

```
template<typename T , typename U >
std::shared_ptr<std::decay_t<T> > gko::as (
            std::shared_ptr< U > obj )  [inline]
```

Performs polymorphic type conversion of a shared_ptr.

#### Template Parameters

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

#### Parameters

| | |
|---|---|
| *obj* | the shared_ptr to the object which should be converted. |

#### Returns

If successful, returns a shared_ptr to the subtype, otherwise throws NotSupported. This pointer shares ownership with the input pointer.

```
371 {
372     auto ptr = std::dynamic_pointer_cast<std::decay_t<T»(obj);
373     if (ptr) {
374         return ptr;
375     } else {
376         throw NotSupported(__FILE__, __LINE__, __func__,
377                            name_demangling::get_type_name(typeid(*obj)));
378     }
379 }
```

### 47.1.4.7 as() [6/7]

```
template<typename T , typename U >
std::unique_ptr<std::decay_t<T> > gko::as (
             std::unique_ptr< U > && obj )  [inline]
```

Performs polymorphic type conversion of a unique_ptr.

#### Template Parameters

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

#### Parameters

| | |
|---|---|
| *obj* | the unique_ptr to the object which should be converted. If successful, it will be reset to a nullptr. |

#### Returns

If successful, returns a unique_ptr to the subtype, otherwise throws NotSupported.

```
347 {
348     if (auto p = dynamic_cast<std::decay_t<T>*>(obj.get())) {
349         obj.release();
350         return std::unique_ptr<std::decay_t<T»{p};
351     } else {
352         throw NotSupported(__FILE__, __LINE__, __func__,
353                            name_demangling::get_type_name(typeid(*obj)));
354     }
355 }
```

#### 47.1.4.8 as() [7/7]

```
template<typename T , typename U >
std::decay_t<T>* gko::as (
            U * obj )  [inline]
```

Performs polymorphic type conversion.

**Template Parameters**

| | |
|---|---|
| *T* | requested result type |
| *U* | static type of the passed object |

**Parameters**

| | |
|---|---|
| *obj* | the object which should be converted |

**Returns**

If successful, returns a pointer to the subtype, otherwise throws NotSupported.

```
258 {
259     if (auto p = dynamic_cast<std::decay_t<T>*>(obj)) {
260         return p;
261     } else {
262         throw NotSupported(__FILE__, __LINE__,
263                            std::string{"gko::as<"} +
264                                name_demangling::get_type_name(typeid(T)) + ">",
265                            name_demangling::get_type_name(typeid(*obj)));
266     }
267 }
```

Referenced by gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::get_approximate_inverse().

#### 47.1.4.9 ceildiv()

```
constexpr int64 gko::ceildiv (
              int64 num,
              int64 den )  [inline], [constexpr]
```

Performs integer division with rounding up.

**Parameters**

| | |
|---|---|
| *num* | numerator |
| *den* | denominator |

**Returns**

returns the ceiled quotient.

```
614 {
615     return (num + den - 1) / den;
616 }
```

Referenced by gko::preconditioner::block_interleaved_storage_scheme< index_type >::compute_storage_↩ space().

#### 47.1.4.10 clone() [1/2]

```
template<typename Pointer , std::enable_if_t< detail::is_cloneable< detail::pointee< Pointer
>>()> * = nullptr>
detail::cloned_type< Pointer > gko::clone (
            const Pointer & p )  [inline]
```

Creates a unique clone of the object pointed to by p.

when the class does not have clone, try Cloneable

The pointee (i.e. *p) needs to have a clone method that returns a std::unique_ptr in order for this method to work.

**Template Parameters**

| | |
|---|---|
| *Pointer* | type of pointer to the object (plain or smart pointer) |

**Parameters**

| | |
|---|---|
| *p* | a pointer to the object |

**Note**

The difference between this function and directly calling Cloneable::clone() is that this one preserves the static type of the object.

```
426 {
427     return detail::cloned_type<Pointer>(
428         static_cast<typename std::remove_cv<detail::pointee<Pointer>>::type*>(
429             p->clone().release()));
430 }
```

Referenced by gko::solver::EnablePreconditionable< Chebyshev< ValueType > >::set_preconditioner(), and gko::solver::EnableIterativeBase< Chebyshev< ValueType > >::set_stop_criterion_factory().

#### 47.1.4.11 clone() [2/2]

```
template<typename Pointer , std::enable_if_t< detail::has_clone_to< detail::pointee< Pointer
>>()> * = nullptr>
detail::cloned_type< Pointer > gko::clone (
            std::shared_ptr< const Executor > exec,
            const Pointer & p )  [inline]
```

Creates a unique clone of the object pointed to by p on Executor exec.

when the class does not have clone, try Cloneable

The pointee (i.e. *p) needs to have a clone method that takes an executor and returns a std::unique_ptr in order for this method to work.

**Template Parameters**

| | |
|---|---|
| *Pointer* | type of pointer to the object (plain or smart pointer) |

**Parameters**

| | |
|---|---|
| *exec* | the executor where the cloned object should be stored |
| *p* | a pointer to the object |

**Note**

The difference between this function and directly calling LinOp::clone() is that this one preserves the static type of the object.

```
467 {
468     return detail::cloned_type<Pointer>(
469         static_cast<typename std::remove_cv<detail::pointee<Pointer>>::type*>(
470             p->clone(std::move(exec)).release()));
471 }
```

#### 47.1.4.12 conj()

```
template<typename T >
constexpr auto gko::conj (
             const T & x )  [inline], [constexpr]
```

Returns the conjugate of an object.

**Parameters**

| | |
|---|---|
| *x* | the number to conjugate |

**Returns**

conjugate of the object (by default, the object itself)

```
930 {
931     return detail::conj_impl(detail::to_arithmetic_type(x));
932 }
```

Referenced by squared_norm().

#### 47.1.4.13 copy_and_convert_to() [1/4]

```
template<typename R , typename T >
std::unique_ptr<const R, std::function<void(const R*)> > gko::copy_and_convert_to (
             std::shared_ptr< const Executor > exec,
             const T * obj )
```

Converts the object to R and places it on Executor exec.

If the object is already of the requested type and on the requested executor, the copy and conversion is avoided and a reference to the original object is returned instead.

**Template Parameters**

| | |
|---|---|
| *R* | the type to which the object should be converted |
| *T* | the type of the input object |

**Parameters**

| | |
|---|---|
| *exec* | the executor where the result should be placed |
| *obj* | the object that should be converted |

**Returns**

a unique pointer (with dynamically bound deleter) to the converted object

**Note**

This is a version of the function which adds the const qualifier to the result if the input had the same qualifier.

```
255 {
256     return detail::copy_and_convert_to_impl<const R>(std::move(exec), obj);
257 }
```

#### 47.1.4.14 copy_and_convert_to() [2/4]

```
template<typename R , typename T >
std::shared_ptr<const R> gko::copy_and_convert_to (
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< const T > obj )
```

This is the version that takes in the std::shared_ptr and returns a std::shared_ptr

If the object is already of the requested type and on the requested executor, the copy and conversion is avoided and a reference to the original object is returned instead.

**Template Parameters**

| | |
|---|---|
| *R* | the type to which the object should be converted |
| *T* | the type of the input object |

**Parameters**

| | |
|---|---|
| *exec* | the executor where the result should be placed |
| *obj* | the object that should be converted |

**Returns**

a shared pointer to the converted object

**Note**

This is a version of the function which adds the const qualifier to the result if the input had the same qualifier.

```
295 {
296     return detail::copy_and_convert_to_impl<const R>(std::move(exec), obj);
297 }
```

#### 47.1.4.15 copy_and_convert_to() [3/4]

```
template<typename R , typename T >
std::shared_ptr<R> gko::copy_and_convert_to (
             std::shared_ptr< const Executor > exec,
             std::shared_ptr< T > obj )
```

Converts the object to R and places it on Executor exec.

This is the version that takes in the std::shared_ptr and returns a std::shared_ptr

If the object is already of the requested type and on the requested executor, the copy and conversion is avoided and a reference to the original object is returned instead.

**Template Parameters**

| | |
|---|---|
| *R* | the type to which the object should be converted |
| *T* | the type of the input object |

**Parameters**

| | |
|---|---|
| *exec* | the executor where the result should be placed |
| *obj* | the object that should be converted |

**Returns**

a shared pointer to the converted object

```
280 {
281     return detail::copy_and_convert_to_impl<R>(std::move(exec), obj);
282 }
```

#### 47.1.4.16 copy_and_convert_to() [4/4]

```
template<typename R , typename T >
std::unique_ptr<R, std::function<void(R*)> > gko::copy_and_convert_to (
             std::shared_ptr< const Executor > exec,
             T * obj )
```

Converts the object to R and places it on Executor exec.

If the object is already of the requested type and on the requested executor, the copy and conversion is avoided and a reference to the original object is returned instead.

**Template Parameters**

| | |
|---|---|
| *R* | the type to which the object should be converted |
| *T* | the type of the input object |

**Parameters**

| | |
|---|---|
| *exec* | the executor where the result should be placed |
| *obj* | the object that should be converted |

**Returns**

a unique pointer (with dynamically bound deleter) to the converted object

```
241 {
242     return detail::copy_and_convert_to_impl<R>(std::move(exec), obj);
243 }
```

#### 47.1.4.17 get_significant_bit()

```
template<typename T >
constexpr uint32 gko::get_significant_bit (
             const T & n,
             uint32 hint = 0u )  [constexpr], [noexcept]
```

Returns the position of the most significant bit of the number.

This is the same as the rounded down base-2 logarithm of the number.

**Template Parameters**

| | |
|---|---|
| *T* | a numeric type supporting bit shift and comparison |

**Parameters**

| | |
|---|---|
| *n* | a number |
| *hint* | a lower bound for the position o the significant bit |

**Returns**

maximum of `hint` and the significant bit position of `n`

```
1057 {
1058     return (T{1} << (hint + 1)) > n ?  hint :  get_significant_bit(n, hint + 1u);
1059 }
```

#### 47.1.4.18 get_superior_power()

```
template<typename T >
constexpr T gko::get_superior_power (
```

```
            const T & base,
            const T & limit,
            const T & hint = T{1} )  [constexpr], [noexcept]
```

Returns the smallest power of `base` not smaller than `limit`.

#### Template Parameters

| | |
|---|---|
| *T* | a numeric type supporting multiplication and comparison |

#### Parameters

| | |
|---|---|
| *base* | the base of the power to be returned |
| *limit* | the lower limit on the size of the power returned |
| *hint* | a lower bound on the result, has to be a power of base |

#### Returns

the smallest power of `base` not smaller than `limit`

```
1075                                                {1}) noexcept
1076 {
1077     return hint >= limit ?  hint :  get_superior_power(base, limit, hint * base);
1078 }
```

### 47.1.4.19 give()

```
template<typename OwningPointer >
std::remove_reference<OwningPointer>::type&& gko::give (
            OwningPointer && p )  [inline]
```

Marks that the object pointed to by `p` can be given to the callee.

Effectively calls `std::move(p)`.

#### Template Parameters

| | |
|---|---|
| *OwningPointer* | type of pointer with ownership to the object (has to be a smart pointer) |

#### Parameters

| | |
|---|---|
| *p* | a pointer to the object |

#### Note

The original pointer `p` becomes invalid after this call.

```
199 {
200     static_assert(detail::have_ownership<OwningPointer>(),
201                   "OwningPointer does not have ownership of the object");
202     return std::move(p);
203 }
```

#### 47.1.4.20 imag()

```
template<typename T >
constexpr auto gko::imag (
            const T & x )  [inline], [constexpr]
```

Returns the imaginary part of the object.

**Template Parameters**

| | |
|---|---|
| *T* | type of the object |

**Parameters**

| | |
|---|---|
| *x* | the object |

**Returns**

imaginary part of the object (by default, zero<T>())

```
916 {
917     return detail::imag_impl(detail::to_arithmetic_type(x));
918 }
```

#### 47.1.4.21 is_complex()

```
template<typename T >
constexpr bool gko::is_complex ( )  [inline], [constexpr]
```

Checks if T is a complex type.

**Template Parameters**

| | |
|---|---|
| *T* | type to check |

**Returns**

`true` if T is a complex type, `false` otherwise

```
225 {
226     return detail::is_complex_impl<T>::value;
227 }
```

#### 47.1.4.22 is_complex_or_scalar()

```
template<typename T >
constexpr bool gko::is_complex_or_scalar ( )  [inline], [constexpr]
```

Checks if T is a complex/scalar type.

**Template Parameters**

| | |
|---|---|
| *T* | type to check |

**Returns**

`true` if T is a complex/scalar type, `false` otherwise

```
249 {
250     return detail::is_complex_or_scalar_impl<T>::value;
251 }
```

#### 47.1.4.23 is_finite() [1/2]

```
template<typename T >
std::enable_if_t<!is_complex_s<T>::value, bool> gko::is_finite (
            const T & value )  [inline]
```

Checks if a floating point number is finite, meaning it is neither +/- infinity nor NaN.

**Template Parameters**

| | |
|---|---|
| *T* | type of the value to check |

**Parameters**

| | |
|---|---|
| *value* | value to check |

**Returns**

`true` if the value is finite, meaning it are neither +/- infinity nor NaN.

```
1095 {
1096     constexpr T infinity{detail::infinity_impl<T>::value};
1097     return abs(value) < infinity;
1098 }
```

References abs().

Referenced by is_finite().

#### 47.1.4.24 is_finite() [2/2]

```
template<typename T >
std::enable_if_t<is_complex_s<T>::value, bool> gko::is_finite (
            const T & value )  [inline]
```

Checks if all components of a complex value are finite, meaning they are neither +/- infinity nor NaN.

**Template Parameters**

| | |
|---|---|
| *T* | complex type of the value to check |

**Parameters**

| | |
|---|---|
| *value* | complex value to check |

**Returns**

`true` if both components of the given value are finite, meaning they are neither +/- infinity nor NaN.

```
1115 {
1116     return is_finite(value.real()) && is_finite(value.imag());
1117 }
```

References is_finite().

#### 47.1.4.25 is_nan() [1/2]

```
template<typename T >
std::enable_if_t<!is_complex_s<T>::value, bool> gko::is_nan (
            const T & value )  [inline]
```

Checks if a floating point number is NaN.

**Template Parameters**

| | |
|---|---|
| *T* | type of the value to check |

**Parameters**

| | |
|---|---|
| *value* | value to check |

**Returns**

`true` if the value is NaN.

```
1153 {
1154     using std::isnan;
1155     return isnan(value);
1156 }
```

Referenced by is_nan().

#### 47.1.4.26 is_nan() [2/2]

```
template<typename T >
std::enable_if_t<is_complex_s<T>::value, bool> gko::is_nan (
            const T & value )  [inline]
```

Checks if any component of a complex value is NaN.

**Template Parameters**

| | |
|---|---|
| *T* | complex type of the value to check |

**Parameters**

| | |
|---|---|
| *value* | complex value to check |

**Returns**

`true` if any component of the given value is NaN.

```
1174 {
1175     return is_nan(value.real()) || is_nan(value.imag());
1176 }
```

References is_nan().

#### 47.1.4.27 is_nonzero()

```
template<typename T >
constexpr bool gko::is_nonzero (
            T value )  [inline], [constexpr]
```

Returns true if and only if the given value is not zero.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the value |

**Parameters**

| | |
|---|---|
| *value* | the given value |

**Returns**

true iff the given value is not zero, i.e. `value != zero<T>()`

```
714 {
715     return value != zero<T>();
716 }
```

Referenced by gko::matrix_data< ValueType, IndexType >::diag().

#### 47.1.4.28 is_zero()

```
template<typename T >
constexpr bool gko::is_zero (
            T value )  [inline], [constexpr]
```

Returns true if and only if the given value is zero.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the value |

**Parameters**

| | |
|---|---|
| *value* | the given value |

**Returns**

true iff the given value is zero, i.e. `value == zero<T>()`

```
699 {
700     return value == zero<T>();
701 }
```

Referenced by gko::matrix_data< ValueType, IndexType >::remove_zeros().

#### 47.1.4.29 lend() [1/2]

```
template<typename Pointer >
std::enable_if<detail::have_ownership_s<Pointer>::value, detail::pointee<Pointer>*>::type
gko::lend (
             const Pointer & p )  [inline]
```

Returns a non-owning (plain) pointer to the object pointed to by `p`.

**Template Parameters**

| | |
|---|---|
| *Pointer* | type of pointer to the object (plain or smart pointer) |

**Parameters**

| | |
|---|---|
| *p* | a pointer to the object |

**Note**

This is the overload for owning (smart) pointers, that behaves the same as calling .get() on the smart pointer.

```
221 {
222     return p.get();
223 }
```

#### 47.1.4.30 lend() [2/2]

```
template<typename Pointer >
std::enable_if<!detail::have_ownership_s<Pointer>::value, detail::pointee<Pointer>*>::type
gko::lend (
             const Pointer & p )  [inline]
```

Returns a non-owning (plain) pointer to the object pointed to by `p`.

**Template Parameters**

| | |
|---|---|
| *Pointer* | type of pointer to the object (plain or smart pointer) |

**Parameters**

| | |
|---|---|
| *p* | a pointer to the object |

**Note**

This is the overload for non-owning (plain) pointers, that just returns `p`.

```
240 {
241     return p;
242 }
```

#### 47.1.4.31 make_array_view()

```
template<typename ValueType >
array<ValueType> gko::make_array_view (
            std::shared_ptr< const Executor > exec,
            size_type size,
            ValueType * data )
```

Helper function to create an array view deducing the value type.

**Parameters**

| | |
|---|---|
| *exec* | the executor on which the array resides |
| *size* | the number of elements for the array |
| *data* | the pointer to the array we create a view on. |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type of the array elements |

**Returns**

`array<ValueType>::view(exec, size, data)`

```
802 {
803     return array<ValueType>::view(exec, size, data);
804 }
```

References make_array_view().

Referenced by gko::array< index_type >::copy_to_host(), and make_array_view().

#### 47.1.4.32 make_const_array_view()

```
template<typename ValueType >
detail::const_array_view<ValueType> gko::make_const_array_view (
            std::shared_ptr< const Executor > exec,
            size_type size,
            const ValueType * data )
```

Helper function to create a const array view deducing the value type.

**Parameters**

| | |
|---|---|
| *exec* | the executor on which the array resides |
| *size* | the number of elements for the array |
| *data* | the pointer to the array we create a view on. |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type of the array elements |

**Returns**

`array<ValueType>::const_view(exec, size, data)`

```
821 {
822     return array<ValueType>::const_view(exec, size, data);
823 }
```

References make_const_array_view().

Referenced by make_const_array_view().

#### 47.1.4.33 make_const_dense_view()

```
template<typename VecPtr >
std::unique_ptr< const matrix::Dense<typename detail::pointee<VecPtr>::value_type> > gko↩
::make_const_dense_view (
            VecPtr && vector )
```

Creates a view of a given Dense vector.

**Template Parameters**

| | |
|---|---|
| *VecPtr* | a (smart or raw) pointer to the vector. |

**Parameters**

| | |
|---|---|
| *vector* | the vector on which to create the view |

```
1585 {
1586     using value_type = typename detail::pointee<VecPtr>::value_type;
```

```
1587     return matrix::Dense<value_type>::create_const_view_of(vector);
1588 }
```

References gko::matrix::Dense< ValueType >::create_const_view_of().

#### 47.1.4.34 make_dense_view()

```
template<typename VecPtr >
std::unique_ptr<matrix::Dense<typename detail::pointee<VecPtr>::value_type> > gko::make_↩
dense_view (
             VecPtr && vector )
```

Creates a view of a given Dense vector.

**Template Parameters**

| | |
|---|---|
| *VecPtr* | a (smart or raw) pointer to the vector. |

**Parameters**

| | |
|---|---|
| *vector* | the vector on which to create the view |

```
1568 {
1569     using value_type = typename detail::pointee<VecPtr>::value_type;
1570     return matrix::Dense<value_type>::create_view_of(vector);
1571 }
```

References gko::matrix::Dense< ValueType >::create_view_of().

#### 47.1.4.35 make_temporary_clone()

```
template<typename Ptr >
detail::temporary_clone<detail::pointee<Ptr> > gko::make_temporary_clone (
             std::shared_ptr< const Executor > exec,
             Ptr && ptr )
```

Creates a temporary_clone.

This is a helper function which avoids the need to explicitly specify the type of the object, as would be the case if using the constructor of temporary_clone.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the clone will be created |
| *ptr* | a pointer to the object of which the clone will be created |

**Template Parameters**

| | |
|---|---|
| *Ptr* | the (raw or smart) pointer type to be temporarily cloned |

```
210 {
211     using T = detail::pointee<Ptr>;
212     return detail::temporary_clone<T>(std::move(exec), std::forward<Ptr>(ptr));
213 }
```

Referenced by gko::ScaledIdentityAddable::add_scaled_identity(), gko::matrix::Csr< ValueType, IndexType >::inv_scale(), and gko::matrix::Csr< ValueType, IndexType >::scale().

#### 47.1.4.36 make_temporary_conversion()

```
template<typename ValueType , typename Ptr >
detail::temporary_conversion<std::conditional_t< std::is_const<detail::pointee<Ptr> >
::value, const matrix::Dense<ValueType>, matrix::Dense<ValueType> > > gko::make_temporary
_conversion (
             Ptr && matrix )
```

Convert the given LinOp from matrix::Dense<...> to matrix::Dense<ValueType>.

The conversion tries to convert the input LinOp to all Dense types with value type recursively reachable by next_precision_base<...> starting from the ValueType template parameter. This means that all real-to-real and complex-to-complex conversions for default precisions are being considered. If the input matrix is non-const, the contents of the modified converted object will be converted back to the input matrix when the returned object is destroyed. This may lead to a loss of precision!

**Parameters**

| | |
|---|---|
| *matrix* | the input matrix which is supposed to be converted. It is wrapped unchanged if it is already of type matrix::Dense<ValueType>, otherwise it will be converted to this type if possible. |

**Returns**

a detail::temporary_conversion pointing to the (potentially converted) object.

**Exceptions**

| | |
|---|---|
| *NotSupported* | if the input matrix cannot be converted to matrix::Dense<ValueType> |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type into whose associated matrix::Dense type to convert the input LinOp. |

```
48 {
49     using Pointee = detail::pointee<Ptr>;
50     using Dense = matrix::Dense<ValueType>;
51     using NextDense = matrix::Dense<next_precision<ValueType»;
52     using Next2Dense = matrix::Dense<next_precision<ValueType, 2»;
53     using Next3Dense = matrix::Dense<next_precision<ValueType, 3»;
54     using MaybeConstDense =
55         std::conditional_t<std::is_const<Pointee>::value, const Dense, Dense>;
56     auto result =
57         detail::temporary_conversion<MaybeConstDense>::template create<
58             NextDense, Next2Dense, Next3Dense>(matrix);
59     if (!result) {
60         GKO_NOT_SUPPORTED(matrix);
61     }
62     return result;
63 }
```

#### 47.1.4.37 make_temporary_output_clone()

```
template<typename Ptr >
detail::temporary_clone<detail::pointee<Ptr> > gko::make_temporary_output_clone (
            std::shared_ptr< const Executor > exec,
            Ptr && ptr )
```

Creates a uninitialized temporary_clone that will be copied back to the input afterwards.

It can be used for output parameters to avoid an unnecessary copy in make_temporary_clone.

This is a helper function which avoids the need to explicitly specify the type of the object, as would be the case if using the constructor of temporary_clone.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the uninitialized clone will be created |
| *ptr* | a pointer to the object of which the clone will be created |

**Template Parameters**

| | |
|---|---|
| *Ptr* | the (raw or smart) pointer type to be temporarily cloned |

```
233 {
234     using T = detail::pointee<Ptr>;
235     static_assert(
236         !std::is_const<T>::value,
237         "make_temporary_output_clone should only be used on non-const objects");
238     return detail::temporary_clone<T>(std::move(exec), std::forward<Ptr>(ptr),
239                                       false);
240 }
```

#### 47.1.4.38 max()

```
template<typename T >
constexpr T gko::max (
            const T & x,
            const T & y )  [inline], [constexpr]
```

Returns the larger of the arguments.

**Template Parameters**

| | |
|---|---|
| *T* | type of the arguments |

**Parameters**

| | |
|---|---|
| *x* | first argument |
| *y* | second argument |

**Returns**

x >= y ? x : y

```
732 {
733     return x >= y ?  x :  y;
734 }
```

#### 47.1.4.39 min()

```
template<typename T >
constexpr T gko::min (
            const T & x,
            const T & y )  [inline], [constexpr]
```

Returns the smaller of the arguments.

**Template Parameters**

| | |
|---|---|
| *T* | type of the arguments |

**Parameters**

| | |
|---|---|
| *x* | first argument |
| *y* | second argument |

**Returns**

x <= y ? x : y

```
750 {
751     return x <= y ?  x :  y;
752 }
```

#### 47.1.4.40 mixed_precision_dispatch()

```
template<typename ValueType , typename Function >
void gko::mixed_precision_dispatch (
            Function fn,
            const LinOp * in,
            LinOp * out )
```

Calls the given function with each given argument LinOp converted into matrix::Dense<ValueType> as parameters.

If GINKGO_MIXED_PRECISION is defined, this means that the function will be called with its dynamic type as a static type, so the (templated/generic) function will be instantiated with all pairs of Dense<ValueType> and Dense<next_precision_base<ValueType>> parameter types, and the appropriate overload will be called based on the dynamic type of the parameter.

If GINKGO_MIXED_PRECISION is not defined, it will behave exactly like precision_dispatch.

**Parameters**

| | |
|---|---|
| *fn* | the given function. It will be called with one const and one non-const matrix::Dense<...> parameter based on the dynamic type of the inputs (GINKGO_MIXED_PRECISION) or of type matrix::Dense<ValueType> (no GINKGO_MIXED_PRECISION). |
| *in* | The first parameter to be cast (GINKGO_MIXED_PRECISION) or converted (no GINKGO_MIXED_PRECISION) and used to call `fn`. |
| *out* | The second parameter to be cast (GINKGO_MIXED_PRECISION) or converted (no GINKGO_MIXED_PRECISION) and used to call `fn`. |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type to use for the parameters of `fn` (no GINKGO_MIXED_PRECISION). With GINKGO_MIXED_PRECISION enabled, it only matters whether this type is complex or real. |
| *Function* | the function pointer, lambda or other functor type to call with the converted arguments. |

```
229 {
230 #ifdef GINKGO_MIXED_PRECISION
231     using fst_type = matrix::Dense<ValueType>;
232     using snd_type = matrix::Dense<next_precision<ValueType»;
233     using trd_type = matrix::Dense<next_precision<ValueType, 2»;
234     using fth_type = matrix::Dense<next_precision<ValueType, 3»;
235     auto dispatch_out_vector = [&](auto dense_in) {
236         if (auto dense_out = dynamic_cast<fst_type*>(out)) {
237             fn(dense_in, dense_out);
238         } else if (auto dense_out = dynamic_cast<snd_type*>(out)) {
239             fn(dense_in, dense_out);
240         } else if (auto dense_out = dynamic_cast<trd_type*>(out)) {
241             fn(dense_in, dense_out);
242         } else if (auto dense_out = dynamic_cast<fth_type*>(out)) {
243             fn(dense_in, dense_out);
244         } else {
245             GKO_NOT_SUPPORTED(out);
246         }
247     };
248     if (auto dense_in = dynamic_cast<const fst_type*>(in)) {
249         dispatch_out_vector(dense_in);
250     } else if (auto dense_in = dynamic_cast<const snd_type*>(in)) {
251         dispatch_out_vector(dense_in);
252     } else if (auto dense_in = dynamic_cast<const trd_type*>(in)) {
253         dispatch_out_vector(dense_in);
254     } else if (auto dense_in = dynamic_cast<const fth_type*>(in)) {
255         dispatch_out_vector(dense_in);
256     } else {
257         GKO_NOT_SUPPORTED(in);
258     }
259 #else
260     precision_dispatch<ValueType>(fn, in, out);
261 #endif
262 }
```

#### 47.1.4.41 mixed_precision_dispatch_real_complex()

```
template<typename ValueType , typename Function , std::enable_if_t< is_complex< ValueType
>()> * = nullptr>
void gko::mixed_precision_dispatch_real_complex (
            Function fn,
            const LinOp * in,
            LinOp * out )
```

Calls the given function with the given LinOps cast to their dynamic type matrix::Dense<ValueType>* as parameters.

If ValueType is real and both `in` and `out` are complex, uses matrix::Dense::get_real_view() to convert them into real matrices after precision conversion.

**See also**

mixed_precision_dispatch()

```
278 {
279 #ifdef GINKGO_MIXED_PRECISION
280     mixed_precision_dispatch<ValueType>(fn, in, out);
281 #else
282     precision_dispatch<ValueType>(fn, in, out);
283 #endif
284 }
```

#### 47.1.4.42 nan() [1/2]

```
template<typename T >
constexpr std::enable_if_t<!is_complex_s<T>::value, T> gko::nan ( )  [inline], [constexpr]
```

Returns a quiet NaN of the given type.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the object |

**Returns**

NaN.

```
1188 {
1189     return std::numeric_limits<T>::quiet_NaN();
1190 }
```

Referenced by nan().

#### 47.1.4.43 nan() [2/2]

```
template<typename T >
constexpr std::enable_if_t<is_complex_s<T>::value, T> gko::nan ( )  [inline], [constexpr]
```

Returns a complex with both components quiet NaN.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the object |

**Returns**

complex{NaN, NaN}.

```
1202 {
1203     return T{nan<remove_complex<T»(), nan<remove_complex<T»()};
1204 }
```

References nan().

#### 47.1.4.44 one() [1/2]

```
template<typename T >
constexpr T gko::one ( )  [inline], [constexpr]
```

Returns the multiplicative identity for T.

**Returns**

the multiplicative identity for T

```
654 {
655     return T(1);
656 }
```

Referenced by unit_root().

#### 47.1.4.45 one() [2/2]

```
template<typename T >
constexpr T gko::one (
             const T &  )  [inline], [constexpr]
```

Returns the multiplicative identity for T.

**Returns**

the multiplicative identity for T

**Note**

This version takes an unused reference argument to avoid complicated calls like `one<decltype(x)>()`. Instead, it allows `one(x)`.

```
684 {
685     return one<T>();
686 }
```

#### 47.1.4.46 operator"!=() [1/2]

```
bool gko::operator!= (
             const stopping_status & x,
             const stopping_status & y )  [inline], [noexcept]
```

Checks if two stopping statuses are different.

**Parameters**

| | |
|---|---|
| *x* | a stopping status |
| *y* | a stopping status |

**Returns**

true if and only if `!(x == y)`

```
151 {
152     return x.data_ != y.data_;
153 }
```

#### 47.1.4.47 operator"!=() [2/2]

```
constexpr bool gko::operator!= (
            precision_reduction x,
            precision_reduction y )  [constexpr], [noexcept]
```

Checks if two precision_reduction encodings are different.

**Parameters**

| | |
|---|---|
| *x* | an encoding |
| *y* | an encoding |

**Returns**

true if and only if `x` and `y` are different encodings.

```
390 {
391     using st = precision_reduction::storage_type;
392     return static_cast<st>(x) != static_cast<st>(y);
393 }
```

#### 47.1.4.48 operator<<() [1/2]

```
std::ostream& gko::operator<< (
            std::ostream & os,
            const version & ver )  [inline]
```

Prints version information to a stream.

**Parameters**

| | |
|---|---|
| *os* | output stream |
| *ver* | version structure |

**Returns**

os

```
101 {
102     os « ver.major « "." « ver.minor « "." « ver.patch;
103     if (ver.tag) {
104         os « " (" « ver.tag « ")";
105     }
106     return os;
```

```
107 }
```

References gko::version::major, gko::version::minor, gko::version::patch, and gko::version::tag.

#### 47.1.4.49 operator<<() [2/2]

```
std::ostream& gko::operator<< (
            std::ostream & os,
            const version_info & ver_info )
```

Prints library version information in human-readable format to a stream.

**Parameters**

| | |
|---|---|
| *os* | output stream |
| *ver_info* | version information |

**Returns**

os

#### 47.1.4.50 operator==() [1/2]

```
bool gko::operator== (
            const stopping_status & x,
            const stopping_status & y )  [inline], [noexcept]
```

Checks if two stopping statuses are equivalent.

**Parameters**

| | |
|---|---|
| *x* | a stopping status |
| *y* | a stopping status |

**Returns**

true if and only if both x and y have the same mask and converged and finalized state

```
136 {
137     return x.data_ == y.data_;
138 }
```

#### 47.1.4.51 operator==() [2/2]

```
constexpr bool gko::operator== (
            precision_reduction x,
            precision_reduction y )  [constexpr], [noexcept]
```

Checks if two precision_reduction encodings are equal.

**Parameters**

| | |
|---|---|
| *x* | an encoding |
| *y* | an encoding |

**Returns**

true if and only if `x` and `y` are the same encodings

```
374 {
375     using st = precision_reduction::storage_type;
376     return static_cast<st>(x) == static_cast<st>(y);
377 }
```

#### 47.1.4.52 pi()

```
template<typename T >
constexpr T gko::pi ( )  [inline], [constexpr]
```

Returns the value of pi.

**Template Parameters**

| | |
|---|---|
| *T* | the value type to return |

```
1021 {
1022     return static_cast<T>(3.14159265358979323846264338);
1023 }
```

#### 47.1.4.53 precision_dispatch()

```
template<typename ValueType , typename Function , typename...  Args>
void gko::precision_dispatch (
             Function fn,
             Args *...  linops )
```

Calls the given function with each given argument LinOp temporarily converted into matrix::Dense<ValueType> as parameters.

**Parameters**

| | |
|---|---|
| *fn* | the given function. It will be passed one (potentially const) matrix::Dense<ValueType>* parameter per parameter in the parameter pack `linops`. |
| *linops* | the given arguments to be converted and passed on to fn. |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type to use for the parameters of `fn`. |

**Template Parameters**

| | |
|---|---|
| *Function* | the function pointer, lambda or other functor type to call with the converted arguments. |
| *Args* | the argument type list. |

```
82 {
83     fn(make_temporary_conversion<ValueType>(linops).get()...);
84 }
```

#### 47.1.4.54 precision_dispatch_real_complex() [1/3]

```
template<typename ValueType , typename Function >
void gko::precision_dispatch_real_complex (
             Function fn,
             const LinOp * alpha,
             const LinOp * in,
             const LinOp * beta,
             LinOp * out )
```

Calls the given function with the given LinOps temporarily converted to matrix::Dense<ValueType>* as parameters.

If ValueType is real and both `in` and `out` are complex, uses matrix::Dense::get_real_view() to convert them into real matrices after precision conversion.

**See also**

precision_dispatch()

```
171 {
172     // do we need to convert complex Dense to real Dense?
173     // all real dense vectors are intra-convertible, thus by casting to
174     // ConvertibleTo<matrix::Dense<>>, we can check whether a LinOp is a real
175     // dense matrix:
176     auto complex_to_real =
177         !(is_complex<ValueType>() ||
178           dynamic_cast<const ConvertibleTo<matrix::Dense<>>*>(in));
179     if (complex_to_real) {
180         auto dense_in = make_temporary_conversion<to_complex<ValueType>>(in);
181         auto dense_out = make_temporary_conversion<to_complex<ValueType>>(out);
182         auto dense_alpha = make_temporary_conversion<ValueType>(alpha);
183         auto dense_beta = make_temporary_conversion<ValueType>(beta);
184         using Dense = matrix::Dense<ValueType>;
185         // These dynamic_casts are only needed to make the code compile
186         // If ValueType is complex, this branch will never be taken
187         // If ValueType is real, the cast is a no-op
188         fn(dense_alpha.get(),
189            dynamic_cast<const Dense*>(dense_in->create_real_view().get()),
190            dense_beta.get(),
191            dynamic_cast<Dense*>(dense_out->create_real_view().get()));
192     } else {
193         precision_dispatch<ValueType>(fn, alpha, in, beta, out);
194     }
195 }
```

#### 47.1.4.55 precision_dispatch_real_complex() [2/3]

```
template<typename ValueType , typename Function >
void gko::precision_dispatch_real_complex (
            Function fn,
            const LinOp * alpha,
            const LinOp * in,
            LinOp * out )
```

Calls the given function with the given LinOps temporarily converted to matrix::Dense<ValueType>* as parameters.

If ValueType is real and both `in` and `out` are complex, uses matrix::Dense::create_real_view() to convert them into real matrices after precision conversion.

**See also**

precision_dispatch()

```
133 {
134     // do we need to convert complex Dense to real Dense?
135     // all real dense vectors are intra-convertible, thus by casting to
136     // ConvertibleTo<matrix::Dense<>>, we can check whether a LinOp is a real
137     // dense matrix:
138     auto complex_to_real =
139         !(is_complex<ValueType>() ||
140           dynamic_cast<const ConvertibleTo<matrix::Dense<>>*>(in));
141     if (complex_to_real) {
142         auto dense_in = make_temporary_conversion<to_complex<ValueType>>(in);
143         auto dense_out = make_temporary_conversion<to_complex<ValueType>>(out);
144         auto dense_alpha = make_temporary_conversion<ValueType>(alpha);
145         using Dense = matrix::Dense<ValueType>;
146         // These dynamic_casts are only needed to make the code compile
147         // If ValueType is complex, this branch will never be taken
148         // If ValueType is real, the cast is a no-op
149         fn(dense_alpha.get(),
150            dynamic_cast<const Dense*>(dense_in->create_real_view().get()),
151            dynamic_cast<Dense*>(dense_out->create_real_view().get()));
152     } else {
153         precision_dispatch<ValueType>(fn, alpha, in, out);
154     }
155 }
```

#### 47.1.4.56 precision_dispatch_real_complex() [3/3]

```
template<typename ValueType , typename Function >
void gko::precision_dispatch_real_complex (
            Function fn,
            const LinOp * in,
            LinOp * out )
```

Calls the given function with the given LinOps temporarily converted to matrix::Dense<ValueType>* as parameters.

If ValueType is real and both input vectors are complex, uses matrix::Dense::create_real_view() to convert them into real matrices after precision conversion.

**See also**

precision_dispatch()

```
98 {
99     // do we need to convert complex Dense to real Dense?
100     // all real dense vectors are intra-convertible, thus by casting to
101     // ConvertibleTo<matrix::Dense<>>, we can check whether a LinOp is a real
102     // dense matrix:
103     auto complex_to_real =
104         !(is_complex<ValueType>() ||
105           dynamic_cast<const ConvertibleTo<matrix::Dense<>>*>(in));
106     if (complex_to_real) {
107         auto dense_in = make_temporary_conversion<to_complex<ValueType>>(in);
108         auto dense_out = make_temporary_conversion<to_complex<ValueType>>(out);
109         using Dense = matrix::Dense<ValueType>;
110         // These dynamic_casts are only needed to make the code compile
111         // If ValueType is complex, this branch will never be taken
112         // If ValueType is real, the cast is a no-op
113         fn(dynamic_cast<const Dense*>(dense_in->create_real_view().get()),
114            dynamic_cast<Dense*>(dense_out->create_real_view().get()));
115     } else {
116         precision_dispatch<ValueType>(fn, in, out);
117     }
118 }
```

### 47.1.4.57 read()

```
template<typename MatrixType , typename StreamType , typename...  MatrixArgs>
std::unique_ptr<MatrixType> gko::read (
             StreamType && is,
             MatrixArgs &&...  args )  [inline]
```

Reads a matrix stored in matrix market format from an input stream.

**Template Parameters**

| | |
|---|---|
| *MatrixType* | a ReadableFromMatrixData LinOp type used to store the matrix once it's been read from disk. |
| *StreamType* | type of stream used to write the data to |
| *MatrixArgs* | additional argument types passed to MatrixType constructor |

**Parameters**

| | |
|---|---|
| *is* | input stream from which to read the data |
| *args* | additional arguments passed to MatrixType constructor |

**Returns**

A MatrixType LinOp filled with data from filename

```
161 {
162     auto mtx = MatrixType::create(std::forward<MatrixArgs>(args)...);
163     mtx->read(read_raw<typename MatrixType::value_type,
164                        typename MatrixType::index_type>(is));
165     return mtx;
166 }
```

References read_raw().

Referenced by gko::ReadableFromMatrixData< ValueType, int32 >::read().

#### 47.1.4.58 read_binary()

```
template<typename MatrixType , typename StreamType , typename...  MatrixArgs>
std::unique_ptr<MatrixType> gko::read_binary (
            StreamType && is,
            MatrixArgs &&...  args )  [inline]
```

Reads a matrix stored in binary format from an input stream.

**Template Parameters**

| | |
|---|---|
| *MatrixType* | a ReadableFromMatrixData LinOp type used to store the matrix once it's been read from disk. |
| *StreamType* | type of stream used to write the data to |
| *MatrixArgs* | additional argument types passed to MatrixType constructor |

**Parameters**

| | |
|---|---|
| *is* | input stream from which to read the data |
| *args* | additional arguments passed to MatrixType constructor |

**Returns**

A MatrixType LinOp filled with data from filename

```
186 {
187     auto mtx = MatrixType::create(std::forward<MatrixArgs>(args)...);
188     mtx->read(read_binary_raw<typename MatrixType::value_type,
189                               typename MatrixType::index_type>(is));
190     return mtx;
191 }
```

References read_binary_raw().

#### 47.1.4.59 read_binary_raw()

```
template<typename ValueType = default_precision, typename IndexType = int32>
matrix_data<ValueType, IndexType> gko::read_binary_raw (
            std::istream & is )
```

Reads a matrix stored in Ginkgo's binary matrix format from an input stream.

Note that this format depends on the processor's endianness, so files from a big endian processor can't be read from a little endian processor and vice-versa.

The binary format has the following structure (in system endianness):

1. A 32 byte header consisting of 4 uint64_t values: magic = GINKGO__: The highest two bytes stand for value and index type. value type: S (float), D (double), C (complex<float>), Z(complex<double>) index type: I (int32), L (int64) num_rows: Number of rows num_cols: Number of columns num_entries: Number of (row, column, value) tuples to follow
2. Following are num_entries blocks of size sizeof(IndexType) ∗ 2 + sizeof(ValueType). Each consists of a row index stored as IndexType, followed by a column index stored as IndexType and a value stored as ValueType.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values |
| *IndexType* | type of matrix indexes |

**Parameters**

| | |
|---|---|
| *is* | input stream from which to read the data |

**Returns**

A matrix_data structure containing the matrix. The nonzero elements are sorted in lexicographic order of their (row, column) indexes.

**Note**

This is an advanced routine that will return the raw matrix data structure. Consider using gko::read_binary instead.

Referenced by read_binary().

#### 47.1.4.60 read_generic()

```
template<typename MatrixType , typename StreamType , typename...  MatrixArgs>
std::unique_ptr<MatrixType> gko::read_generic (
            StreamType && is,
            MatrixArgs &&...  args )  [inline]
```

Reads a matrix stored either in binary or matrix market format from an input stream.

**Template Parameters**

| | |
|---|---|
| *MatrixType* | a ReadableFromMatrixData LinOp type used to store the matrix once it's been read from disk. |
| *StreamType* | type of stream used to write the data to |
| *MatrixArgs* | additional argument types passed to MatrixType constructor |

**Parameters**

| | |
|---|---|
| *is* | input stream from which to read the data |
| *args* | additional arguments passed to MatrixType constructor |

**Returns**

A MatrixType LinOp filled with data from filename

```
212 {
213     auto mtx = MatrixType::create(std::forward<MatrixArgs>(args)...);
```

```
214     mtx->read(read_generic_raw<typename MatrixType::value_type,
215                                typename MatrixType::index_type>(is));
216     return mtx;
217 }
```

References read_generic_raw().

#### 47.1.4.61 read_generic_raw()

```
template<typename ValueType = default_precision, typename IndexType = int32>
matrix_data<ValueType, IndexType> gko::read_generic_raw (
             std::istream & is )
```

Reads a matrix stored in either binary or matrix market format from an input stream.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values |
| *IndexType* | type of matrix indexes |

**Parameters**

| | |
|---|---|
| *is* | input stream from which to read the data |

**Returns**

A matrix_data structure containing the matrix. The nonzero elements are sorted in lexicographic order of their (row, column) indexes.

**Note**

This is an advanced routine that will return the raw matrix data structure. Consider using gko::read_generic instead.

Referenced by read_generic().

#### 47.1.4.62 read_raw()

```
template<typename ValueType = default_precision, typename IndexType = int32>
matrix_data<ValueType, IndexType> gko::read_raw (
             std::istream & is )
```

Reads a matrix stored in matrix market format from an input stream.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values |
| *IndexType* | type of matrix indexes |

**Parameters**

| | |
|---|---|
| *is* | input stream from which to read the data |

**Returns**

A matrix_data structure containing the matrix. The nonzero elements are sorted in lexicographic order of their (row, column) indexes.

**Note**

This is an advanced routine that will return the raw matrix data structure. Consider using gko::read instead.

Referenced by read().

#### 47.1.4.63 real()

```
template<typename T >
constexpr auto gko::real (
            const T & x )  [inline], [constexpr]
```

Returns the real part of the object.

**Template Parameters**

| | |
|---|---|
| *T* | type of the object |

**Parameters**

| | |
|---|---|
| *x* | the object |

**Returns**

real part of the object (by default, the object itself)

```
900 {
901     return detail::real_impl(detail::to_arithmetic_type(x));
902 }
```

Referenced by squared_norm().

#### 47.1.4.64 reduce_add() [1/2]

```
template<typename ValueType >
void gko::reduce_add (
            const array< ValueType > & input_arr,
            array< ValueType > & result )
```

Reduce (sum) the values in the array.

**Template Parameters**

| | |
|---|---|
| *The* | type of the input data |

**Parameters**

| | | |
|---|---|---|
| in | *input_arr* | the input array to be reduced |
| in,out | *result* | the reduced value. The result is written into the first entry and the value in the first entry is used as the initial value for the reduce. |

#### 47.1.4.65 reduce_add() [2/2]

```
template<typename ValueType >
ValueType gko::reduce_add (
            const array< ValueType > & input_arr,
            const ValueType init_val = 0 )
```

Reduce (sum) the values in the array.

**Template Parameters**

| | |
|---|---|
| *The* | type of the input data |

**Parameters**

| | | |
|---|---|---|
| in | *input_arr* | the input array to be reduced |
| in | *init_val* | the initial value |

**Returns**

the reduced value

#### 47.1.4.66 round_down()

```
template<typename T >
constexpr reduce_precision<T> gko::round_down (
            T val )  [inline], [constexpr]
```

Reduces the precision of the input parameter.

**Template Parameters**

| | |
|---|---|
| *T* | the original precision |

**Parameters**

| | |
|---|---|
| *val* | the value to round down |

**Returns**

the rounded down value

```
516 {
517     return static_cast<reduce_precision<T>>(val);
518 }
```

#### 47.1.4.67 round_up()

```
template<typename T >
constexpr increase_precision<T> gko::round_up (
            T val )  [inline], [constexpr]
```

Increases the precision of the input parameter.

**Template Parameters**

| | |
|---|---|
| *T* | the original precision |

**Parameters**

| | |
|---|---|
| *val* | the value to round up |

**Returns**

the rounded up value

```
532 {
533     return static_cast<increase_precision<T>>(val);
534 }
```

#### 47.1.4.68 safe_divide()

```
template<typename T >
T gko::safe_divide (
            T a,
            T b )  [inline]
```

Computes the quotient of the given parameters, guarding against division by zero.

**Template Parameters**

| | |
|---|---|
| *T* | value type of the parameters |

**Parameters**

| | |
|---|---|
| *a* | the dividend |
| *b* | the divisor |

**Returns**

the value of `a / b` if b is non-zero, zero otherwise.

```
1133 {
1134     return b == zero<T>() ?  zero<T>() :  a / b;
1135 }
```

#### 47.1.4.69 share()

```
template<typename OwningPointer >
detail::shared_type<OwningPointer> gko::share (
            OwningPointer && p )  [inline]
```

Marks the object pointed to by `p` as shared.

Effectively converts a pointer with ownership to std::shared_ptr.

**Template Parameters**

| | |
|---|---|
| *OwningPointer* | type of pointer with ownership to the object (has to be a smart pointer) |

**Parameters**

| | |
|---|---|
| *p* | a pointer to the object. It must be a temporary or explicitly marked movable (rvalue reference). |

**Note**

The original pointer `p` becomes invalid after this call.

```
175 {
176     static_assert(detail::have_ownership<OwningPointer>(),
177                   "OwningPointer does not have ownership of the object");
178     static_assert(std::is_rvalue_reference<decltype(p)>::value,
179                   "p must be an rvalue for this function to work");
180     return detail::shared_type<OwningPointer>(std::move(p));
181 }
```

#### 47.1.4.70 squared_norm()

```
template<typename T >
constexpr auto gko::squared_norm (
            const T & x ) -> decltype(real(conj(x) * x))   [inline], [constexpr]
```

Returns the squared norm of the object.

**Template Parameters**

| | |
|---|---|
| *T* | type of the object. |

**Returns**

The squared norm of the object.

```
945 {
946     return real(conj(x) * x);
947 }
```

References conj(), and real().

#### 47.1.4.71 transpose() [1/2]

```
template<typename DimensionType >
batch_dim<2, DimensionType> gko::transpose (
            const batch_dim< 2, DimensionType > & input )  [inline]
```

Returns a batch_dim object with its dimensions swapped for batched operators.

**Template Parameters**

| | |
|---|---|
| *DimensionType* | datatype used to represent each dimension |

**Parameters**

| | |
|---|---|
| *input* | original object |

**Returns**

a batch_dim object with dimensions swapped

```
120 {
121     return batch_dim<2, DimensionType>(input.get_num_batch_items(),
122                                        transpose(input.get_common_size()));
123 }
```

References gko::batch_dim< Dimensionality, DimensionType >::get_common_size(), and gko::batch_dim< Dimensionality, DimensionType >::get_num_batch_items().

#### 47.1.4.72 transpose() [2/2]

```
template<typename DimensionType >
constexpr dim<2, DimensionType> gko::transpose (
            const dim< 2, DimensionType > & dimensions )  [inline], [constexpr], [noexcept]
```

Returns a dim<2> object with its dimensions swapped.

#### Template Parameters

| | |
|---|---|
| *DimensionType* | datatype used to represent each dimension |

#### Parameters

| | |
|---|---|
| *dimensions* | original object |

#### Returns

a dim<2> object with its dimensions swapped

```
252 {
253     return {dimensions[1], dimensions[0]};
254 }
```

### 47.1.4.73 unit_root()

```
template<typename T >
constexpr std::complex<remove_complex<T> > gko::unit_root (
             int64 n,
             int64 k = 1 )  [inline], [constexpr]
```

Returns the value of exp(2 ∗ pi ∗ i ∗ k / n), i.e.

an nth root of unity.

#### Parameters

| | |
|---|---|
| *n* | the denominator of the argument |
| *k* | the numerator of the argument. Defaults to 1. |

#### Template Parameters

| | |
|---|---|
| *T* | the corresponding real value type. |

```
1037 {
1038     return std::polar(one<remove_complex<T»(),
1039                       remove_complex<T>{2} * pi<remove_complex<T»() * k / n);
1040 }
```

References one().

### 47.1.4.74 with_matrix_type()

```
template<template< typename, typename > class MatrixType, typename...  Args>
auto gko::with_matrix_type (
             Args &&...  create_args )
```

This function returns a type that delays a call to MatrixType::create.

It can be used to set the used value and index type, as well as the executor at a later stage.

For example, the following code creates first a temporary object, which is then used later to construct an operator of the previously defined base type:

```
auto type = gko::with_matrix_type<gko::matrix::Csr>();
...
std::unique_ptr<LinOp> concrete_op
if(flag1){
  concrete_op = type.template create<double, int>(exec);
} else {
  concrete_op = type.template create<float, int>(exec);
}
```

**Note**

This is mainly a helper function to specify the local matrix type for a gko::experimental::distributed::Matrix more easily.

**Template Parameters**

| | |
|---|---|
| *MatrixType* | A template type that accepts two types, the first one will be set to the value type, the second one to the index type. |
| *Args* | Types of the arguments passed to MatrixType::create. |

**Parameters**

| | |
|---|---|
| *create_args* | arguments that will be forwarded to MatrixType::create |

**Returns**

A type with a function `create<value_type, index_type>(executor)`.

```
134 {
135     return detail::MatrixTypeBuilderFromValueAndIndex<MatrixType, Args...>{
136         std::forward_as_tuple(create_args...)};
137 }
```

#### 47.1.4.75 write() [1/2]

```
void gko::write (
            std::ostream & os,
            ptr_param< const LinOp > matrix )
```

Writes a LinOp into an output stream in matrix market format.

The matrix market format use coordinate format if the sparsity is less than 25%. Otherwise, use array format.

**Parameters**

| | |
|---|---|
| *os* | output stream where the data is to be written |
| *matrix* | the matrix to write |

#### 47.1.4.76 write() [2/2]

```
template<typename MatrixPtrType , typename StreamType , std::enable_if_t<!std::is_same_v<
std::remove_cv_t< detail::pointee< MatrixPtrType >>, LinOp >> * = nullptr>
void gko::write (
            StreamType && os,
            MatrixPtrType && matrix,
            layout_type layout = detail::mtx_io_traits< std::remove_cv_t<detail::pointee<MatrixPtrType>>>←
::default_layout )  [inline]
```

Writes a matrix into an output stream in matrix market format.

**Template Parameters**

| | |
|---|---|
| *MatrixPtrType* | a (smart or raw) pointer to a WritableToMatrixData object providing data to be written. |
| *StreamType* | type of stream used to write the data to |

**Parameters**

| | |
|---|---|
| *os* | output stream where the data is to be written |
| *matrix* | the matrix to write |
| *layout* | the layout used in the output |

```
303 {
304     using MatrixType = detail::pointee<MatrixPtrType>;
305     matrix_data<typename MatrixType::value_type,
306                 typename MatrixType::index_type>
307         data{};
308     matrix->write(data);
309     write_raw(os, data, layout);
310 }
```

References write_raw().

Referenced by gko::WritableToMatrixData< ValueType, int32 >::write().

#### 47.1.4.77 write_binary()

```
template<typename MatrixPtrType , typename StreamType >
void gko::write_binary (
            StreamType && os,
            MatrixPtrType && matrix )  [inline]
```

Writes a matrix into an output stream in binary format.

Note that this format depends on the processor's endianness, so files from a big endian processor can't be read from a little endian processor and vice-versa.

**Template Parameters**

| | |
|---|---|
| *MatrixPtrType* | a (smart or raw) pointer to a WritableToMatrixData object providing data to be written. |
| *StreamType* | type of stream used to write the data to |

**Parameters**

| | |
|---|---|
| *os* | output stream where the data is to be written |
| *matrix* | the matrix to write |

```
338 {
339     using MatrixType = detail::pointee<MatrixPtrType>;
340     matrix_data<typename MatrixType::value_type,
341                 typename MatrixType::index_type>
342         data{};
343     matrix->write(data);
344     write_binary_raw(os, data);
345 }
```

References write_binary_raw().

#### 47.1.4.78 write_binary_raw()

```
template<typename ValueType , typename IndexType >
void gko::write_binary_raw (
            std::ostream & os,
            const matrix_data< ValueType, IndexType > & data )
```

Writes a matrix_data structure to a stream in binary format.

Note that this format depends on the processor's endianness, so files from a big endian processor can't be read from a little endian processor and vice-versa.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values |
| *IndexType* | type of matrix indexes |

**Parameters**

| | |
|---|---|
| *os* | output stream where the data is to be written |
| *data* | the matrix data to write |

**Note**

This is an advanced routine that writes the raw matrix data structure. If you are trying to write an existing matrix, consider using gko::write_binary instead.

Referenced by write_binary().

#### 47.1.4.79 write_raw()

```
template<typename ValueType , typename IndexType >
void gko::write_raw (
```

```
std::ostream & os,
const matrix_data< ValueType, IndexType > & data,
layout_type layout = layout_type::coordinate )
```

Writes a matrix_data structure to a stream in matrix market format.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values |
| *IndexType* | type of matrix indexes |

**Parameters**

| | |
|---|---|
| *os* | output stream where the data is to be written |
| *data* | the matrix data to write |
| *layout* | the layout used in the output |

**Note**

This is an advanced routine that writes the raw matrix data structure. If you are trying to write an existing matrix, consider using gko::write instead.

Referenced by write().

#### 47.1.4.80 zero() [1/2]

```
template<typename T >
constexpr T gko::zero ( )  [inline], [constexpr]
```

Returns the additive identity for T.

**Returns**

additive identity for T

```
626 {
627     return T{};
628 }
```

Referenced by gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::strategy_type().

#### 47.1.4.81 zero() [2/2]

```
template<typename T >
constexpr T gko::zero (
             const T &  )  [inline], [constexpr]
```

Returns the additive identity for T.

**Returns**

additive identity for T

**Note**

This version takes an unused reference argument to avoid complicated calls like `zero<decltype(x)>()`. Instead, it allows `zero(x)`.

```
642 {
643     return zero<T>();
644 }
```

## 47.2 gko::accessor Namespace Reference

The accessor namespace.

### Classes

- class row_major

  *A row_major accessor is a bridge between a range and the row-major memory layout.*

### 47.2.1 Detailed Description

The accessor namespace.

## 47.3 gko::batch::log Namespace Reference

The logger namespace .

### Classes

- class BatchConvergence

  *Logs the final residuals and iteration counts for a batch solver.*

### 47.3.1 Detailed Description

The logger namespace .

Logging

## 47.4 gko::experimental::distributed Namespace Reference

The distributed namespace.

### Namespaces

- preconditioner

  *The Preconditioner namespace.*

## Classes

- class DistributedBase

  *A base class for distributed objects.*
- class index_map

  *This class defines mappings between global and local indices.*
- class Matrix

  *The Matrix class defines a (MPI-)distributed matrix.*
- class Partition

  *Represents a partition of a range of indices [0, size) into a disjoint set of parts.*
- class RowGatherer

  *The distributed::RowGatherer gathers the rows of distributed::Vector that are located on other processes.*
- class Vector

  *Vector is a format which explicitly stores (multiple) distributed column vectors in a dense storage format.*

## Enumerations

- enum index_space { index_space::local, index_space::non_local, index_space::combined }

  *Index space classification for the locally stored indices.*
- enum assembly_mode

  *assembly_mode defines how the read_distributed function of the distributed matrix treats non-local indices in the (device_)matrix_data:*

## Functions

- template<typename ValueType >
  gko::detail::temporary_conversion< Vector< ValueType > > make_temporary_conversion (LinOp *matrix)

  *Convert the given LinOp from experimental::distributed::Vector<...> to experimental::distributed::Vector<ValueType>.*
- template<typename ValueType >
  gko::detail::temporary_conversion< const Vector< ValueType > > make_temporary_conversion (const LinOp *matrix)

  *Convert the given LinOp from experimental::distributed::Vector<...> to experimental::distributed::Vector<ValueType>.*
- template<typename ValueType , typename Function , typename... Args>
  void precision_dispatch (Function fn, Args *... linops)

  *Calls the given function with each given argument LinOp temporarily converted into experimental::distributed::Vector<ValueType> as parameters.*
- template<typename ValueType , typename Function >
  void precision_dispatch_real_complex (Function fn, const LinOp *in, LinOp *out)

  *Calls the given function with the given LinOps temporarily converted to experimental::distributed::Vector<ValueType>* as parameters.*
- template<typename ValueType , typename Function >
  void precision_dispatch_real_complex (Function fn, const LinOp *alpha, const LinOp *in, LinOp *out)

  *Calls the given function with the given LinOps temporarily converted to experimental::distributed::Vector<ValueType>* as parameters.*
- template<typename ValueType , typename Function >
  void precision_dispatch_real_complex (Function fn, const LinOp *alpha, const LinOp *in, const LinOp *beta, LinOp *out)

  *Calls the given function with the given LinOps temporarily converted to experimental::distributed::Vector<ValueType>* as parameters.*

- template<typename ValueType , typename LocalIndexType , typename GlobalIndexType >
  device_matrix_data< ValueType, GlobalIndexType > assemble_rows_from_neighbors (mpi::communicator comm, const device_matrix_data< ValueType, GlobalIndexType > &input, ptr_param< const Partition< LocalIndexType, GlobalIndexType >> partition)

  *Assembles device_matrix_data entries owned by this MPI rank from other ranks and communicates entries located on this MPI rank owned by other ranks to their respective owners.*
- template<typename LocalIndexType , typename GlobalIndexType >
  std::unique_ptr< Partition< LocalIndexType, GlobalIndexType > > build_partition_from_local_range (std::shared_ptr< const Executor > exec, mpi::communicator comm, span local_range)

  *Builds a partition from a local range.*
- template<typename LocalIndexType , typename GlobalIndexType >
  std::unique_ptr< Partition< LocalIndexType, GlobalIndexType > > build_partition_from_local_size (std::shared_ptr< const Executor > exec, mpi::communicator comm, size_type local_size)

  *Builds a partition from a local size.*

## 47.4.1 Detailed Description

The distributed namespace.

## 47.4.2 Enumeration Type Documentation

### 47.4.2.1 assembly_mode

```
enum gko::experimental::distributed::assembly_mode  [strong]
```

assembly_mode defines how the read_distributed function of the distributed matrix treats non-local indices in the (device_)matrix_data:

- `communicate` communicates the overlap between ranks and adds up all local contributions. Indices smaller than 0 or larger than the global size of the matrix are ignored.
- `local_only` does not communicate any overlap but ignores all non-local indices.

```
153 { communicate, local_only };
```

### 47.4.2.2 index_space

```
enum gko::experimental::distributed::index_space  [strong]
```

Index space classification for the locally stored indices.

The definitions of the enum values is clarified in index_map.

**Enumerator**

| | |
|---|---|
| local | indices that are locally owned |
| non_local | indices that are owned by other processes |
| combined | both local and non_local indices |

```
23                      {
24     local,
25     non_local,
26     combined
27 };
```

### 47.4.3 Function Documentation

#### 47.4.3.1 assemble_rows_from_neighbors()

```
template<typename ValueType , typename LocalIndexType , typename GlobalIndexType >
device_matrix_data<ValueType, GlobalIndexType> gko::experimental::distributed::assemble_rows↩
_from_neighbors (
             mpi::communicator comm,
             const device_matrix_data< ValueType, GlobalIndexType > & input,
             ptr_param< const Partition< LocalIndexType, GlobalIndexType >> partition )
```

Assembles device_matrix_data entries owned by this MPI rank from other ranks and communicates entries located on this MPI rank owned by other ranks to their respective owners.

This can be useful e.g. in a finite element code where each rank assembles a local contribution to a global system matrix and the global matrix has to be assembled by summing up the local contributions on rank boundaries. The partition used is only relevant for row ownership.

**Parameters**

| | |
|---|---|
| *comm* | the communicator used to assemble the global matrix. |
| *input* | the device_matrix_data structure. |
| *partition* | the partition used to determine row owndership. |

**Returns**

the globally assembled device_matrix_data structure for this MPI rank.

#### 47.4.3.2 build_partition_from_local_range()

```
template<typename LocalIndexType , typename GlobalIndexType >
std::unique_ptr<Partition<LocalIndexType, GlobalIndexType> > gko::experimental::distributed↩
::build_partition_from_local_range (
             std::shared_ptr< const Executor > exec,
             mpi::communicator comm,
             span local_range )
```

Builds a partition from a local range.

**Parameters**

| | |
|---|---|
| *exec* | the Executor on which the partition should be built. |
| *comm* | the communicator used to determine the global partition. |
| *local_range* | the start and end indices of the local range. |

**Warning**

This throws, if the resulting partition would contain gaps. That means that for a partition of size `n` every local range `r_i = [s_i, e_i)` either `s_i != 0` and another local range `r_j = [s_j, e_j = s_i)` exists, or `e_i != n` and another local range `r_j = [s_j = e_i, e_j)` exists.

**Returns**

a Partition where each range has the individual local_start and local_ends.

#### 47.4.3.3 build_partition_from_local_size()

```
template<typename LocalIndexType , typename GlobalIndexType >
std::unique_ptr<Partition<LocalIndexType, GlobalIndexType> > gko::experimental::distributed↩
::build_partition_from_local_size (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            size_type local_size )
```

Builds a partition from a local size.

**Parameters**

| | |
|---|---|
| *exec* | the Executor on which the partition should be built. |
| *comm* | the communicator used to determine the global partition. |
| *local_size* | the number of the locally owned indices |

**Returns**

a Partition where each range has the specified local size. More specifically, if this is called on process i with local_size `s_i`, then the range `i` has size `s_i`, and range `r_i = [start, start + s_i)`, where `start = sum_j^(i-1) s_j`.

#### 47.4.3.4 make_temporary_conversion() [1/2]

```
template<typename ValueType >
gko::detail::temporary_conversion<const Vector<ValueType> > gko::experimental::distributed↩
::make_temporary_conversion (
            const LinOp * matrix )
```

Convert the given LinOp from experimental::distributed::Vector<...> to experimental::distributed::Vector<ValueType>.

The conversion tries to convert the input LinOp to all Dense types with value type recursively reachable by next_precision_base<...> starting from the ValueType template parameter. This means that all real-to-real and complex-to-complex conversions for default precisions are being considered. If the input matrix is non-const, the contents of the modified converted object will be converted back to the input matrix when the returned object is destroyed. This may lead to a loss of precision!

**Parameters**

| | |
|---|---|
| *matrix* | the input matrix which is supposed to be converted. It is wrapped unchanged if it is already of type experimental::distributed::Vector<ValueType>, otherwise it will be converted to this type if possible. |

**Returns**

a detail::temporary_conversion pointing to the (potentially converted) object.

**Exceptions**

| | |
|---|---|
| *NotSupported* | if the input matrix cannot be converted to experimental::distributed::Vector<ValueType> |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type into whose associated Vector type to convert the input LinOp. |

```
365 {
366     auto result = gko::detail::temporary_conversion<const Vector<ValueType>>::
367         template create<Vector<next_precision<ValueType>,
368                         Vector<next_precision<ValueType, 2>,
369                         Vector<next_precision<ValueType, 3>>(matrix);
370     if (!result) {
371         GKO_NOT_SUPPORTED(matrix);
372     }
373     return result;
374 }
```

### 47.4.3.5 make_temporary_conversion() [2/2]

```
template<typename ValueType >
gko::detail::temporary_conversion<Vector<ValueType> > gko::experimental::distributed::make_temporary_conversion (
             LinOp * matrix )
```

Convert the given LinOp from experimental::distributed::Vector<...> to experimental::distributed::Vector<ValueType>.

The conversion tries to convert the input LinOp to all Dense types with value type recursively reachable by next_precision_base<...> starting from the ValueType template parameter. This means that all real-to-real and complex-to-complex conversions for default precisions are being considered. If the input matrix is non-const, the contents of the modified converted object will be converted back to the input matrix when the returned object is destroyed. This may lead to a loss of precision!

**Parameters**

| | |
|---|---|
| *matrix* | the input matrix which is supposed to be converted. It is wrapped unchanged if it is already of type experimental::distributed::Vector<ValueType>, otherwise it will be converted to this type if possible. |

**Returns**

a detail::temporary_conversion pointing to the (potentially converted) object.

**Exceptions**

| | |
|---|---|
| *NotSupported* | if the input matrix cannot be converted to experimental::distributed::Vector<ValueType> |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type into whose associated Vector type to convert the input LinOp. |

```
346 {
347     auto result =
348         gko::detail::temporary_conversion<Vector<ValueType>::template create<
349             Vector<next_precision<ValueType>,
350             Vector<next_precision<ValueType, 2>,
351             Vector<next_precision<ValueType, 3>>(matrix);
352     if (!result) {
353         GKO_NOT_SUPPORTED(matrix);
354     }
355     return result;
356 }
```

### 47.4.3.6 precision_dispatch()

```
template<typename ValueType , typename Function , typename...  Args>
void gko::experimental::distributed::precision_dispatch (
             Function fn,
             Args *...  linops )
```

Calls the given function with each given argument LinOp temporarily converted into experimental::distributed::↩ Vector<ValueType> as parameters.

**Parameters**

| | |
|---|---|
| *fn* | the given function. It will be passed one (potentially const) experimental::distributed::Vector<ValueType>* parameter per parameter in the parameter pack `linops`. |
| *linops* | the given arguments to be converted and passed on to fn. |

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type to use for the parameters of `fn`. |
| *Function* | the function pointer, lambda or other functor type to call with the converted arguments. |
| *Args* | the argument type list. |

```
393 {
394     fn(distributed::make_temporary_conversion<ValueType>(linops).get()...);
395 }
```

#### 47.4.3.7 precision_dispatch_real_complex() [1/3]

```
template<typename ValueType , typename Function >
void gko::experimental::distributed::precision_dispatch_real_complex (
             Function fn,
             const LinOp * alpha,
             const LinOp * in,
             const LinOp * beta,
             LinOp * out )
```

Calls the given function with the given LinOps temporarily converted to experimental::distributed::Vector<Value↩ Type>* as parameters.

If ValueType is real and both input vectors are complex, uses experimental::distributed::Vector::get_real_view() to convert them into real matrices after precision conversion.

**See also**

precision_dispatch()

```
525 {
526     auto complex_to_real = !(
527         is_complex<ValueType>() ||
528         dynamic_cast<const ConvertibleTo<experimental::distributed::Vector<>>*>(
529             in));
530     if (complex_to_real) {
531         auto dense_in =
532             distributed::make_temporary_conversion<to_complex<ValueType»(in);
533         auto dense_out =
534             distributed::make_temporary_conversion<to_complex<ValueType»(out);
535         auto dense_alpha = gko::make_temporary_conversion<ValueType>(alpha);
536         auto dense_beta = gko::make_temporary_conversion<ValueType>(beta);
537         using Vector = experimental::distributed::Vector<ValueType>;
538         // These dynamic_casts are only needed to make the code compile
539         // If ValueType is complex, this branch will never be taken
540         // If ValueType is real, the cast is a no-op
541         fn(dense_alpha.get(),
542            dynamic_cast<const Vector*>(dense_in->create_real_view().get()),
543            dense_beta.get(),
544            dynamic_cast<Vector*>(dense_out->create_real_view().get()));
545     } else {
546         fn(gko::make_temporary_conversion<ValueType>(alpha).get(),
547            distributed::make_temporary_conversion<ValueType>(in).get(),
548            gko::make_temporary_conversion<ValueType>(beta).get(),
549            distributed::make_temporary_conversion<ValueType>(out).get());
550     }
551 }
```

#### 47.4.3.8 precision_dispatch_real_complex() [2/3]

```
template<typename ValueType , typename Function >
void gko::experimental::distributed::precision_dispatch_real_complex (
             Function fn,
             const LinOp * alpha,
             const LinOp * in,
             LinOp * out )
```

Calls the given function with the given LinOps temporarily converted to experimental::distributed::Vector<Value↩ Type>* as parameters.

If ValueType is real and both input vectors are complex, uses experimental::distributed::Vector::get_real_view() to convert them into real matrices after precision conversion.

**See also**

precision_dispatch()

```
492 {
493     auto complex_to_real = !(
494         is_complex<ValueType>() ||
495         dynamic_cast<const ConvertibleTo<experimental::distributed::Vector<>>*>(
496             in));
497     if (complex_to_real) {
498         auto dense_in =
499             distributed::make_temporary_conversion<to_complex<ValueType»(in);
500         auto dense_out =
501             distributed::make_temporary_conversion<to_complex<ValueType»(out);
502         auto dense_alpha = gko::make_temporary_conversion<ValueType>(alpha);
503         using Vector = experimental::distributed::Vector<ValueType>;
504         // These dynamic_casts are only needed to make the code compile
505         // If ValueType is complex, this branch will never be taken
506         // If ValueType is real, the cast is a no-op
507         fn(dense_alpha.get(),
508            dynamic_cast<const Vector*>(dense_in->create_real_view().get()),
509            dynamic_cast<Vector*>(dense_out->create_real_view().get()));
510     } else {
511         fn(gko::make_temporary_conversion<ValueType>(alpha).get(),
512            distributed::make_temporary_conversion<ValueType>(in).get(),
513            distributed::make_temporary_conversion<ValueType>(out).get());
514     }
515 }
```

#### 47.4.3.9 precision_dispatch_real_complex() [3/3]

```
template<typename ValueType , typename Function >
void gko::experimental::distributed::precision_dispatch_real_complex (
             Function fn,
             const LinOp * in,
             LinOp * out )
```

Calls the given function with the given LinOps temporarily converted to experimental::distributed::Vector<Value↩ Type>* as parameters.

If ValueType is real and both input vectors are complex, uses experimental::distributed::Vector::get_real_view() to convert them into real matrices after precision conversion.

**See also**

precision_dispatch()

```
443 {
444     auto complex_to_real = !(
445         is_complex<ValueType>() ||
446         dynamic_cast<const ConvertibleTo<experimental::distributed::Vector<>>*>(
447             in));
448     if (complex_to_real) {
449         auto dense_in =
450             distributed::make_temporary_conversion<to_complex<ValueType»(in);
451         auto dense_out =
452             distributed::make_temporary_conversion<to_complex<ValueType»(out);
453         using Vector = experimental::distributed::Vector<ValueType>;
454         // These dynamic_casts are only needed to make the code compile
455         // If ValueType is complex, this branch will never be taken
456         // If ValueType is real, the cast is a no-op
457         fn(dynamic_cast<const Vector*>(dense_in->create_real_view().get()),
458            dynamic_cast<Vector*>(dense_out->create_real_view().get()));
459     } else {
460         distributed::precision_dispatch<ValueType>(fn, in, out);
461     }
462 }
```

## 47.5 gko::experimental::distributed::preconditioner Namespace Reference

The Preconditioner namespace.

### Classes

- class Schwarz

  *A Schwarz preconditioner is a simple domain decomposition preconditioner that generalizes the Block Jacobi preconditioner, incorporating options for different local subdomain solvers and overlaps between the subdomains.*

### 47.5.1 Detailed Description

The Preconditioner namespace.

## 47.6 gko::experimental::mpi Namespace Reference

The mpi namespace, contains wrapper for many MPI functions.

### Classes

- class CollectiveCommunicator

  *Interface for a collective communicator.*
- class communicator

  *A thin wrapper of MPI_Comm that supports most MPI calls.*
- class contiguous_type

  *A move-only wrapper for a contiguous MPI_Datatype.*
- class DenseCommunicator

  *A CollectiveCommunicator that uses a dense communication.*
- class environment

  *Class that sets up and finalizes the MPI environment.*
- class NeighborhoodCommunicator

  *A CollectiveCommunicator that uses a neighborhood topology.*
- class request

  *The request class is a light, move-only wrapper around the MPI_Request handle.*
- struct status

  *The status struct is a light wrapper around the MPI_Status struct.*
- struct type_impl

  *A struct that is used to determine the MPI_Datatype of a specified type.*
- class window

  *This class wraps the MPI_Window class with RAII functionality.*

### Typedefs

- using comm_index_type = int

  *Index type for enumerating processes in a distributed application.*

## Enumerations

- enum thread_type

  *This enum specifies the threading type to be used when creating an MPI environment.*

## Functions

- constexpr bool is_gpu_aware ()

  *Return if GPU aware functionality is available.*
- int map_rank_to_device_id (MPI_Comm comm, int num_devices)

  *Maps each MPI rank to a single device id in a round robin manner.*
- std::vector< status > wait_all (std::vector< request > &req)

  *Allows a rank to wait on multiple request handles.*
- bool requires_host_buffer (const std::shared_ptr< const Executor > &exec, const communicator &comm)

  *Checks if the combination of Executor and communicator requires passing MPI buffers from the host memory.*
- double get_walltime ()

  *Get the rank in the communicator of the calling process.*

### 47.6.1 Detailed Description

The mpi namespace, contains wrapper for many MPI functions.

### 47.6.2 Typedef Documentation

#### 47.6.2.1 comm_index_type

```
using gko::experimental::mpi::comm_index_type = typedef int
```

Index type for enumerating processes in a distributed application.

Conforms to the MPI C interface of e.g. MPI rank or size

### 47.6.3 Function Documentation

#### 47.6.3.1 get_walltime()

```
double gko::experimental::mpi::get_walltime ( )  [inline]
```

Get the rank in the communicator of the calling process.

**Parameters**

| | |
|---|---|
| *comm* | the communicator |

```
1591 { return MPI_Wtime(); }
```

#### 47.6.3.2 map_rank_to_device_id()

```
int gko::experimental::mpi::map_rank_to_device_id (
            MPI_Comm comm,
            int num_devices )
```

Maps each MPI rank to a single device id in a round robin manner.

**Parameters**

| | |
|---|---|
| *comm* | used to determine the node-local rank, if no suitable environment variable is available. |
| *num_devices* | the number of devices per node. |

**Returns**

device id that this rank should use.

#### 47.6.3.3 wait_all()

```
std::vector<status> gko::experimental::mpi::wait_all (
            std::vector< request > & req )  [inline]
```

Allows a rank to wait on multiple request handles.

**Parameters**

| | |
|---|---|
| *req* | The vector of request handles to be waited on. |

**Returns**

status The vector of status objects that can be queried.

```
395 {
396     std::vector<status> stat;
397     for (std::size_t i = 0; i < req.size(); ++i) {
398         stat.emplace_back(req[i].wait());
399     }
400     return stat;
401 }
```

## 47.7 gko::experimental::reorder Namespace Reference

The Reorder namespace.

## Classes

- class Amd

  *Computes an Approximate Minimum Degree (AMD) reordering of an input matrix.*
- class Mc64

  *MC64 is an algorithm for permuting large entries to the diagonal of a sparse matrix.*
- class Rcm

  *Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.*
- class ScaledReordered

  *Provides an interface to wrap reorderings like Rcm and diagonal scaling like equilibration around a LinOp like e.g.*

## Enumerations

- enum mc64_strategy

  *Strategy defining the goal of the MC64 reordering.*

### 47.7.1 Detailed Description

The Reorder namespace.

### 47.7.2 Enumeration Type Documentation

#### 47.7.2.1 mc64_strategy

```
enum gko::experimental::reorder::mc64_strategy  [strong]
```

Strategy defining the goal of the MC64 reordering.

max_diagonal_product aims at maximizing the product of absolute diagonal entries. max_diag_sum aims at maximizing the sum of absolute values for the diagonal entries.

```
44 { max_diagonal_product, max_diagonal_sum };
```

## 47.8 gko::factorization Namespace Reference

The Factorization namespace.

## Classes

- class Ic

  *Represents an incomplete Cholesky factorization (IC(0)) of a sparse matrix.*
- class Ilu

  *Represents an incomplete LU factorization – ILU(0) – of a sparse matrix.*
- class ParIc

  *ParIC is an incomplete Cholesky factorization which is computed in parallel.*
- class ParIct

  *ParICT is an incomplete threshold-based Cholesky factorization which is computed in parallel.*
- class ParIlu

  *ParILU is an incomplete LU factorization which is computed in parallel.*
- class ParIlut

  *ParILUT is an incomplete threshold-based LU factorization which is computed in parallel.*

## Enumerations

- enum incomplete_algorithm

  *An enum class for algorithm selection in the incomplete factorization.*

### 47.8.1 Detailed Description

The Factorization namespace.

### 47.8.2 Enumeration Type Documentation

#### 47.8.2.1 incomplete_algorithm

`enum gko::factorization::incomplete_algorithm [strong]`

An enum class for algorithm selection in the incomplete factorization.

`sparselib` is only available for CUDA, HIP, and reference. `syncfree` is Ginkgo's implementation by using the Lu/Cholesky factorization components with given sparsity.

```
19 { sparselib, syncfree };
```

## 47.9 gko::log Namespace Reference

The logger namespace .

## Classes

- class Convergence

  *Convergence is a Logger which logs data strictly from the `criterion_check_completed` event.*
- struct criterion_data

  *Struct representing Criterion related data.*
- class EnableLogging

  *EnableLogging is a mixin which should be inherited by any class which wants to enable logging.*
- struct executor_data

  *Struct representing Executor related data.*
- struct iteration_complete_data

  *Struct representing iteration complete related data.*
- struct linop_data

  *Struct representing LinOp related data.*
- struct linop_factory_data

  *Struct representing LinOp factory related data.*
- class Loggable

  *Loggable class is an interface which should be implemented by classes wanting to support logging.*
- struct operation_data

*Struct representing Operator related data.*

- class Papi

  *Papi is a Logger which logs every event to the PAPI software.*
- class PerformanceHint

  *PerformanceHint is a Logger which analyzes the performance of the application and outputs hints for unnecessary copies and allocations.*
- struct polymorphic_object_data

  *Struct representing PolymorphicObject related data.*
- class ProfilerHook

  *This Logger can be used to annotate the execution of Ginkgo functionality with profiler-specific ranges.*
- class profiling_scope_guard

  *Scope guard that annotates its scope with the provided profiler hooks.*
- class Record

  *Record is a Logger which logs every event to an object.*
- class SolverProgress

  *This Logger outputs the value of all scalar values (and potentially vectors) stored internally by the solver after each iteration.*
- class Stream

  *Stream is a Logger which logs every event to a stream.*

## Enumerations

- enum profile_event_category { profile_event_category::memory, profile_event_category::operation, profile_event_category::object, profile_event_category::linop, profile_event_category::factory, profile_event_category::solver, profile_event_category::criterion, profile_event_category::user, profile_event_category::internal }

  *Categorization of logger events.*

### 47.9.1 Detailed Description

The logger namespace .

The Logging namespace.

Logging

### 47.9.2 Enumeration Type Documentation

#### 47.9.2.1 profile_event_category

```
enum gko::log::profile_event_category  [strong]
```

Categorization of logger events.

**Enumerator**

| | |
|---|---|
| memory | Memory allocation. |
| operation | Kernel execution and data movement. |
| object | PolymorphicObject events. |
| linop | LinOp events. |
| factory | LinOpFactory events. |
| solver | Solver events. |
| criterion | Stopping criterion events. |
| user | User-defined events. |
| internal | For development use. |

```
22                                      {
24     memory,
26     operation,
28     object,
30     linop,
32     factory,
34     solver,
36     criterion,
38     user,
40     internal,
41 };
```

## 47.10 gko::matrix Namespace Reference

The matrix namespace.

### Classes

- class Coo

  *COO stores a matrix in the coordinate matrix format.*
- class Csr

  *CSR is a matrix format which stores only the nonzero coefficients by compressing each row of the matrix (compressed sparse row format).*
- class Dense

  *Dense is a matrix format which explicitly stores all values of the matrix.*
- class Diagonal

  *This class is a utility which efficiently implements the diagonal matrix (a linear operator which scales a vector row wise).*
- class Ell

  *ELL is a matrix format where stride with explicit zeros is used such that all rows have the same number of stored elements.*
- class Fbcsr

  *Fixed-block compressed sparse row storage matrix format.*
- class Fft

  *This LinOp implements a 1D Fourier matrix using the FFT algorithm.*
- class Fft2

  *This LinOp implements a 2D Fourier matrix using the FFT algorithm.*
- class Fft3

  *This LinOp implements a 3D Fourier matrix using the FFT algorithm.*
- class Hybrid

  *HYBRID is a matrix format which splits the matrix into ELLPACK and COO format.*

- class Identity

  *This class is a utility which efficiently implements the identity matrix (a linear operator which maps each vector to itself).*
- class IdentityFactory

  *This factory is a utility which can be used to generate Identity operators.*
- class Permutation

  *Permutation is a matrix format that represents a permutation matrix, i.e.*
- class RowGatherer

  *RowGatherer is a matrix "format" which stores the gather indices arrays which can be used to gather rows to another matrix.*
- class ScaledPermutation

  *ScaledPermutation is a matrix combining a permutation with scaling factors.*
- class Sellp

  *SELL-P is a matrix format similar to ELL format.*
- class SparsityCsr

  *SparsityCsr is a matrix format which stores only the sparsity pattern of a sparse matrix by compressing each row of the matrix (compressed sparse row format).*

## Enumerations

- enum permute_mode : unsigned {
  permute_mode::none = 0b000u, permute_mode::rows = 0b001u, permute_mode::columns = 0b010u, permute_mode::symmetric = 0b011u,
  permute_mode::inverse = 0b100u, permute_mode::inverse_rows = 0b101u, permute_mode::inverse_columns = 0b110u, permute_mode::inverse_symmetric = 0b111u }

  *Specifies how a permutation will be applied to a matrix.*

## Functions

- permute_mode operator| (permute_mode a, permute_mode b)

  *Combines two permutation modes.*
- permute_mode operator& (permute_mode a, permute_mode b)

  *Computes the intersection of two permutation modes.*
- permute_mode operator^ (permute_mode a, permute_mode b)

  *Computes the symmetric difference of two permutation modes.*
- std::ostream & operator<< (std::ostream &stream, permute_mode mode)

  *Prints a permutation mode.*

### 47.10.1 Detailed Description

The matrix namespace.

### 47.10.2 Enumeration Type Documentation

#### 47.10.2.1 permute_mode

```
enum gko::matrix::permute_mode : unsigned [strong]
```

Specifies how a permutation will be applied to a matrix.

For the effect of the different permutation modes, see the following table.

| mode | entry mapping | matrix representation |
|---|---|---|
| none | $A'(i,j) = A(i,j)$ | $A' = A$ |
| rows | $A'(i,j) = A(p[i],j)$ | $A' = PA$ |
| columns | $A'(i,j) = A(i,p[j])$ | $A' = AP^T$ |
| inverse_rows | $A'(p[i],j) = A(i,j)$ | $A' = P^{-1}A$ |
| inverse_columns | $A'(i,p[j]) = A(i,j)$ | $A' = AP^{-T}$ |
| symmetric | $A'(i,j) = A(p[i],p[j])$ | $A' = PAP^T$ |
| inverse_symmetric | $A'(p[i],p[j]) = A(i,j)$ | $A' = P^{-1}AP^{-T}$ |

**Enumerator**

| | |
|---|---|
| none | Neither rows nor columns will be permuted. |
| rows | The rows will be permuted. |
| columns | The columns will be permuted. |
| symmetric | The rows and columns will be permuted. This is equivalent to `permute_mode::rows \| permute_mode::columns`. |
| inverse | The permutation will be inverted before being applied. |
| inverse_rows | The rows will be permuted using the inverse permutation. This is equivalent to `permute_mode::rows \| permute_mode::inverse`. |
| inverse_columns | The columns will be permuted using the inverse permutation. This is equivalent to `permute_mode::columns \| permute_mode::inverse`. |
| inverse_symmetric | The rows and columns will be permuted using the inverse permutation. This is equivalent to `permute_mode::symmetric \| permute_mode::inverse`. |

```
42                      :  unsigned {
44     none = 0b000u,
46     rows = 0b001u,
48     columns = 0b010u,
53     symmetric = 0b011u,
55     inverse = 0b100u,
60     inverse_rows = 0b101u,
65     inverse_columns = 0b110u,
70     inverse_symmetric = 0b111u
71 };
```

## 47.11 gko::multigrid Namespace Reference

The multigrid components namespace.

### Classes

- class EnableMultigridLevel

  *The EnableMultigridLevel gives the default implementation of MultigridLevel with composition and provides `set_multigrid_level` function.*
- class FixedCoarsening

  *FixedCoarsening is a very simple coarse grid generation algorithm.*
- class MultigridLevel

  *This class represents two levels in a multigrid hierarchy.*
- class Pgm

  *Parallel graph match (Pgm) is the aggregate method introduced in the paper M.*
- class Pmis

*Parallel modified independent set (Pmis) is the classical coarsening method introduced in the paper H.*

- class Rs

  *Rs implements the Ruge–Stueben (classical) Algebraic Multigrid (AMG) coarsening strategy for M-matrices.*

- class UniformCoarsening

  *UniformCoarsening is a simple coarse grid generation algorithm.*

### 47.11.1 Detailed Description

The multigrid components namespace.

## 47.12 gko::name_demangling Namespace Reference

The name demangling namespace.

### Functions

- template<typename T >
  std::string get_static_type (const T &)

  *This function uses name demangling facilities to get the name of the static type (T) of the object passed in arguments.*

- template<typename T >
  std::string get_dynamic_type (const T &t)

  *This function uses name demangling facilities to get the name of the dynamic type of the object passed in arguments.*

### 47.12.1 Detailed Description

The name demangling namespace.

### 47.12.2 Function Documentation

#### 47.12.2.1 get_dynamic_type()

```
template<typename T >
std::string gko::name_demangling::get_dynamic_type (
            const T & t )
```

This function uses name demangling facilities to get the name of the dynamic type of the object passed in arguments.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the object to demangle |

**Parameters**

| | |
|---|---|
| *t* | the object we get the dynamic type of |

```
71 {
72     return get_type_name(typeid(t));
73 }
```

#### 47.12.2.2 get_static_type()

```
template<typename T >
std::string gko::name_demangling::get_static_type (
             const T &  )
```

This function uses name demangling facilities to get the name of the static type (`T`) of the object passed in arguments.

**Template Parameters**

| | |
|---|---|
| *T* | the type of the object to demangle |

**Parameters**

| | |
|---|---|
| *unused* | |

```
56 {
57     return get_type_name(typeid(T));
58 }
```

## 47.13 gko::preconditioner Namespace Reference

The Preconditioner namespace.

### Classes

- struct block_interleaved_storage_scheme

  *Defines the parameters of the interleaved block storage scheme used by block-Jacobi blocks.*
- class GaussSeidel

  *This class generates the Gauss-Seidel preconditioner.*
- class Ic

  *The Incomplete Cholesky (IC) preconditioner solves the equation $LL^Hx = b$ for a given lower triangular matrix $L$ and the right hand side $b$ (can contain multiple right hand sides).*
- class Ilu

  *The Incomplete LU (ILU) preconditioner solves the equation $LUx = b$ for a given lower triangular matrix $L$, an upper triangular matrix $U$ and the right hand side $b$ (can contain multiple right hand sides).*
- class Isai

  *The Incomplete Sparse Approximate Inverse (ISAI) Preconditioner generates an approximate inverse matrix for a given square matrix $A$, lower triangular matrix $L$, upper triangular matrix $U$ or symmetric positive definite (s.p.d) matrix $B$.*

- class Jacobi

  *A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks of the source operator.*
- class NullspaceProjection

  *NullspaceProjection wraps a preconditioner $M$ with projections onto the complement of a nullspace, $z = PMQr$.*
- class Sor

  *This class generates the (S)SOR preconditioner.*

### Enumerations

- enum isai_type

  *This enum lists the types of the ISAI preconditioner.*

### 47.13.1 Detailed Description

The Preconditioner namespace.

### 47.13.2 Enumeration Type Documentation

#### 47.13.2.1 isai_type

```
enum gko::preconditioner::isai_type  [strong]
```

This enum lists the types of the ISAI preconditioner.

ISAI can either be generated for a general square matrix, a lower triangular matrix, an upper triangular matrix or an s.p.d matrix.

```
36 { lower, upper, general, spd };
```

## 47.14 gko::reorder Namespace Reference

The Reorder namespace.

### Classes

- class Rcm

  *Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.*
- class ReorderingBase

  *The ReorderingBase class is a base class for all the reordering algorithms.*
- struct ReorderingBaseArgs

  *This struct is used to pass parameters to the EnableDefaultReorderingBaseFactory::generate() method.*

### Typedefs

- template<typename IndexType = int32>
  using ReorderingBaseFactory = AbstractFactory< ReorderingBase< IndexType >, ReorderingBaseArgs >

  *Declares an Abstract Factory specialized for ReorderingBases.*
- template<typename ConcreteFactory , typename ConcreteReorderingBase , typename ParametersType , typename IndexType = int32, typename PolymorphicBase = ReorderingBaseFactory<IndexType>>
  using EnableDefaultReorderingBaseFactory = EnableDefaultFactory< ConcreteFactory, ConcreteReorderingBase, ParametersType, PolymorphicBase >

  *This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of ReorderingBaseFactory.*

## 47.14.1 Detailed Description

The Reorder namespace.

The reordering namespace.

## 47.14.2 Typedef Documentation

### 47.14.2.1 EnableDefaultReorderingBaseFactory

```
template<typename ConcreteFactory , typename ConcreteReorderingBase , typename ParametersType
, typename IndexType = int32, typename PolymorphicBase = ReorderingBaseFactory<IndexType>>
using gko::reorder::EnableDefaultReorderingBaseFactory = typedef EnableDefaultFactory<Concrete↩
Factory, ConcreteReorderingBase, ParametersType, PolymorphicBase>
```

This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of ReorderingBaseFactory.

**Template Parameters**

| | |
|---|---|
| *ConcreteFactory* | the concrete factory which is being implemented [CRTP parameter] |
| *ConcreteReorderingBase* | the concrete ReorderingBase type which this factory produces, needs to have a constructor which takes a const ConcreteFactory *, and a const ReorderingBaseArgs * as parameters. |
| *ParametersType* | a subclass of enable_parameters_type template which defines all of the parameters of the factory |
| *PolymorphicBase* | parent of ConcreteFactory in the polymorphic hierarchy, has to be a subclass of ReorderingBaseFactory |

# 47.15 gko::solver Namespace Reference

The ginkgo Solve namespace.

## Namespaces

- multigrid

  *The solver multigrid namespace.*

## Classes

- class ApplyWithInitialGuess

  *ApplyWithInitialGuess provides a way to give the input guess for apply function.*
- class Bicg

  *BICG or the Biconjugate gradient method is a Krylov subspace solver.*
- class Bicgstab

  *BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.*
- class CbGmres

  *CB-GMRES or the compressed basis generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.*
- class Cg

  *CG or the conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class Cgs

  *CGS or the conjugate gradient square method is an iterative type Krylov subspace method which is suitable for general systems.*
- class Chebyshev

  *Chebyshev iteration is an iterative method for solving nonsymmetric problems based on some knowledge of the spectrum of the (preconditioned) system matrix.*
- class EnableApplyWithInitialGuess

  *EnableApplyWithInitialGuess providing default operation for ApplyWithInitialGuess with correct validation and log.*
- class EnableIterativeBase

  *A LinOp deriving from this CRTP class stores a stopping criterion factory and allows applying with a guess.*
- class EnablePreconditionable

  *Mixin providing default operation for Preconditionable with correct value semantics.*
- class EnablePreconditionedIterativeSolver

  *A LinOp implementing this interface stores a system matrix and stopping criterion factory.*
- class EnableSolverBase

  *A LinOp deriving from this CRTP class stores a system matrix.*
- class Fcg

  *FCG or the flexible conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*
- class Gcr

  *GCR or the generalized conjugate residual method is an iterative type Krylov subspace method similar to GMRES which is suitable for nonsymmetric linear systems.*
- class Gmres

  *GMRES or the generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.*
- struct has_with_criteria

  *Helper structure to test if the Factory of SolverType has a function `with_criteria`.*
- class Idr

  *IDR(s) is an efficient method for solving large nonsymmetric systems of linear equations.*
- class Ir

  *Iterative refinement (IR) is an iterative method that uses another coarse method to approximate the error of the current solution via the current residual.*
- class IterativeBase

*A LinOp implementing this interface stores a stopping criterion factory.*

- class LowerTrs

  *LowerTrs is the triangular solver which solves the system L x = b, when L is a lower triangular matrix.*

- class Minres

  *Minres is an iterative type Krylov subspace method, which is suitable for indefinite and full-rank symmetric/hermitian operators.*

- class Multigrid

  *Multigrid solves a linear system by combining cheap iterative sweeps (smoothers) on a hierarchy of progressively coarser representations of the operator with corrections transferred between adjacent levels.*

- class Nullspace

  *Nullspace represents the (right or left) nullspace of an operator and is the orthogonal projector $P = I - VV^H$ onto its complement, where $V$ is an orthonormal basis of the nullspace.*

- struct nullspace_traits

  *Traits describing how an iterative solver handles the nullspaces given by the `nullspace` and `left_nullspace` factory parameters.*

- class PipeCg

  *PIPE_CG or the pipelined conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.*

- class UpperTrs

  *UpperTrs is the triangular solver which solves the system U x = b, when U is an upper triangular matrix.*

- struct workspace_traits

  *Traits class providing information on the type and location of workspace vectors inside a solver.*

## Enumerations

- enum initial_guess_mode { initial_guess_mode::zero, initial_guess_mode::rhs, initial_guess_mode::provided }

  *Give a initial guess mode about the input of the apply method.*

- enum trisolve_algorithm

  *A helper for algorithm selection in the triangular solvers.*

## Functions

- template<typename ValueType >
  auto build_smoother (std::shared_ptr< const LinOpFactory > factory, size_type iteration=1, ValueType relaxation_factor=0.9)

  *build_smoother gives a shortcut to build a smoother by IR(Richardson) with limited stop criterion(iterations and relacation_factor).*

- template<typename ValueType >
  auto build_smoother (std::shared_ptr< const LinOp > solver, size_type iteration=1, ValueType relaxation_↩ factor=0.9)

  *build_smoother gives a shortcut to build a smoother by IR(Richardson) with limited stop criterion(iterations and relacation_factor).*

### 47.15.1 Detailed Description

The ginkgo Solve namespace.

The ginkgo Solver namespace.

## 47.15.2 Enumeration Type Documentation

### 47.15.2.1 initial_guess_mode

```
enum gko::solver::initial_guess_mode  [strong]
```

Give a initial guess mode about the input of the apply method.

**Enumerator**

| | |
|---|---|
| zero | the input is zero |
| rhs | the input is right hand side |
| provided | the input is provided |

```
39                                  {
43     zero,
47     rhs,
51     provided
52 };
```

### 47.15.2.2 trisolve_algorithm

```
enum gko::solver::trisolve_algorithm  [strong]
```

A helper for algorithm selection in the triangular solvers.

It currently only matters for the Cuda executor as there, we have a choice between the Ginkgo syncfree and cuS↩ PARSE implementations.

```
40 { sparselib, syncfree };
```

## 47.15.3 Function Documentation

### 47.15.3.1 build_smoother() [1/2]

```
template<typename ValueType >
auto gko::solver::build_smoother (
             std::shared_ptr< const LinOp > solver,
             size_type iteration = 1,
             ValueType relaxation_factor = 0.9 )
```

build_smoother gives a shortcut to build a smoother by IR(Richardson) with limited stop criterion(iterations and relacation_factor).

**Parameters**

| | |
|---|---|
| *solver* | the shared pointer of solver |
| *iteration* | the maximum number of iteration, which default is 1 |
| *relaxation_factor* | the relaxation factor for Richardson |

**Returns**

the pointer of Ir(Richardson)

**Note**

this is the overload function for LinOp.

```
354 {
355     auto exec = solver->get_executor();
356     return Ir<ValueType>::build()
357         .with_generated_solver(solver)
358         .with_relaxation_factor(relaxation_factor)
359         .with_criteria(gko::stop::Iteration::build().with_max_iters(iteration))
360         .on(exec);
361 }
```

#### 47.15.3.2 build_smoother() [2/2]

```
template<typename ValueType >
auto gko::solver::build_smoother (
              std::shared_ptr< const LinOpFactory > factory,
              size_type iteration = 1,
              ValueType relaxation_factor = 0.9 )
```

build_smoother gives a shortcut to build a smoother by IR(Richardson) with limited stop criterion(iterations and relacation_factor).

**Parameters**

| | |
|---|---|
| *factory* | the shared pointer of factory |
| *iteration* | the maximum number of iteration, which default is 1 |
| *relaxation_factor* | the relaxation factor for Richardson |

**Returns**

the pointer of Ir(Richardson)

```
330 {
331     auto exec = factory->get_executor();
332     return Ir<ValueType>::build()
333         .with_solver(factory)
334         .with_relaxation_factor(relaxation_factor)
335         .with_criteria(gko::stop::Iteration::build().with_max_iters(iteration))
336         .on(exec);
337 }
```

## 47.16 gko::solver::multigrid Namespace Reference

The solver multigrid namespace.

## Enumerations

- enum cycle

  *cycle defines which kind of multigrid cycle can be used.*
- enum mid_smooth_type

  *mid_smooth_type gives the options to handle the middle smoother behavior between the two cycles in the same level.*

### 47.16.1 Detailed Description

The solver multigrid namespace.

### 47.16.2 Enumeration Type Documentation

#### 47.16.2.1 cycle

```
enum gko::solver::multigrid::cycle  [strong]
```

cycle defines which kind of multigrid cycle can be used.

It contains V, W, and F cycle.

- V, W cycles use the algorithm from Briggs–Henson–McCormick (*A Multigrid Tutorial*, 2nd ed., SIAM 2000).
- F cycle uses the algorithm from Trottenberg–Oosterlee–Schüller (*Multigrid*, 1st ed., Academic Press 2001); F-cycle first does the recursive call then a V-cycle call, sitting between V and W in cost.

See the Multigrid class for the full reference list with DOIs.

```
56 { v, f, w };
```

#### 47.16.2.2 mid_smooth_type

```
enum gko::solver::multigrid::mid_smooth_type  [strong]
```

mid_smooth_type gives the options to handle the middle smoother behavior between the two cycles in the same level.

It only affects the behavior when there's no operation between the post smoother of previous cycle and the pre smoother of next cycle. Thus, it only affects W cycle and F cycle.

- both: gives the same behavior as the original algorithm, which use posts smoother from previous cycle and pre smoother from next cycle.
- post_smoother: only uses the post smoother of previous cycle in the mid smoother
- pre_smoother: only uses the pre smoother of next cycle in the mid smoother
- standalone: uses the defined smoother in the mid smoother

```
72 { both, post_smoother, pre_smoother, standalone };
```

## 47.17 gko::stop Namespace Reference

The Stopping criterion namespace.

### Classes

- class AbsoluteResidualNorm

  *The AbsoluteResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold, i.e.*
- class Combined

  *The Combined class is used to combine multiple criterions together through an OR operation.*
- class Criterion

  *The Criterion class is a base class for all stopping criteria.*
- struct CriterionArgs

  *This struct is used to pass parameters to the EnableDefaultCriterionFactoryCriterionFactory::generate() method.*
- class ImplicitResidualNorm

  *Stopping criterion based on a solver-maintained squared residual-norm estimate* $\rho_k^2 \approx \|r_k\|^2$.
- class Iteration

  *The Iteration class is a stopping criterion which stops the iteration process after a preset number of iterations.*
- class RelativeResidualNorm

  *The RelativeResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the right-hand side, i.e.*
- class ResidualNorm

  *Stopping criterion based on the explicit residual norm* $\|r_k\| = \|b - Ax_k\|$.
- class ResidualNormBase

  *The ResidualNormBase class provides a framework for stopping criteria related to the residual norm.*
- class ResidualNormReduction

  *The ResidualNormReduction class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the initial residual, i.e.*
- class Time

  *The Time class is a stopping criterion which stops the iteration process after a certain amount of time has passed.*

### Typedefs

- using CriterionFactory = AbstractFactory< Criterion, CriterionArgs >

  *Declares an Abstract Factory specialized for Criterions.*
- template<typename ConcreteFactory , typename ConcreteCriterion , typename ParametersType , typename PolymorphicBase = CriterionFactory>
  using EnableDefaultCriterionFactory = EnableDefaultFactory< ConcreteFactory, ConcreteCriterion, ParametersType, PolymorphicBase >

  *This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of CriterionFactory.*

### Enumerations

- enum mode

  *The mode for the residual norm criterion.*

## Functions

- template<typename FactoryContainer >
  std::shared_ptr< const CriterionFactory > combine (FactoryContainer &&factories)

  *Combines multiple criterion factories into a single combined criterion factory.*
- deferred_factory_parameter< const Iteration::Factory > max_iters (size_type count)

  *Creates the precursor to an Iteration stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< const CriterionFactory > min_iters (size_type count, deferred_factory_parameter< const CriterionFactory > criterion)

  *Creates the precursor to an MinimumIteration stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< CriterionFactory > absolute_residual_norm (double tolerance)

  *Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< CriterionFactory > relative_residual_norm (double tolerance)

  *Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< CriterionFactory > initial_residual_norm (double tolerance)

  *Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< CriterionFactory > absolute_implicit_residual_norm (double tolerance)

  *Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< CriterionFactory > relative_implicit_residual_norm (double tolerance)

  *Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< CriterionFactory > initial_implicit_residual_norm (double tolerance)

  *Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*
- deferred_factory_parameter< Time::Factory > time_limit (std::chrono::nanoseconds duration)

  *Creates the precursor to a Time stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.*

### 47.17.1 Detailed Description

The Stopping criterion namespace.

Stopping criteria

### 47.17.2 Typedef Documentation

#### 47.17.2.1 EnableDefaultCriterionFactory

```
template<typename ConcreteFactory , typename ConcreteCriterion , typename ParametersType ,
typename PolymorphicBase = CriterionFactory>
using gko::stop::EnableDefaultCriterionFactory = typedef EnableDefaultFactory<ConcreteFactory,
ConcreteCriterion, ParametersType, PolymorphicBase>
```

This is an alias for the EnableDefaultFactory mixin, which correctly sets the template parameters to enable a subclass of CriterionFactory.

**Template Parameters**

| | |
|---|---|
| *ConcreteFactory* | the concrete factory which is being implemented [CRTP parameter] |
| *ConcreteCriterion* | the concrete Criterion type which this factory produces, needs to have a constructor which takes a const ConcreteFactory $*$, and a const CriterionArgs $*$ as parameters. |
| *ParametersType* | a subclass of enable_parameters_type template which defines all of the parameters of the factory |
| *PolymorphicBase* | parent of ConcreteFactory in the polymorphic hierarchy, has to be a subclass of CriterionFactory |

## 47.17.3 Function Documentation

### 47.17.3.1 absolute_implicit_residual_norm()

```
deferred_factory_parameter<CriterionFactory> gko::stop::absolute_implicit_residual_norm (
            double tolerance )
```

Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the residual norm has decreased below the specified value or by the specified amount.

Full usage example: Stop after 100 iterations or when the absolute residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(
                       gko::stop::max_iters(100),
                       gko::stop::absolute_residual_norm(1e-10))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *tolerance* | the value the residual norm needs to be below. With residual $r$, initial guess $x_0$, right-hand side $b$ and matrix $A$, `absolute` means the exact value of the norm $\|\|r\|\|$, `relative` means the norm relative to the right-hand side $\|\|r\|\|/\|\|b\|\|$, `initial` means the norm relative to the initial residual $\|\|r\|\|/\|\|b - Ax_0\|\|$. An implicit stopping criterion is only available with some solvers, and refers to either the energy norm $\|\|r\|\|_A$ in short-recurrence solvers like Cg or the euclidian norm $\|\|r\|\|$ in solvers like GMRES. Implicit residual norms are cheaper to compute, but may be less precise due to accumulating rounding errors. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

### 47.17.3.2 absolute_residual_norm()

```
deferred_factory_parameter<CriterionFactory> gko::stop::absolute_residual_norm (
            double tolerance )
```

Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the residual norm has decreased below the specified value or by the specified amount.

Full usage example: Stop after 100 iterations or when the absolute residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                    .with_criteria(
                        gko::stop::max_iters(100),
                        gko::stop::absolute_residual_norm(1e-10))
                    .on(exec);
```

**Parameters**

| | |
|---|---|
| *tolerance* | the value the residual norm needs to be below. With residual $r$, initial guess $x_0$, right-hand side $b$ and matrix $A$, `absolute` means the exact value of the norm $\|\|r\|\|$, `relative` means the norm relative to the right-hand side $\|\|r\|\|/\|\|b\|\|$, `initial` means the norm relative to the initial residual $\|\|r\|\|/\|\|b - Ax_0\|\|$. An implicit stopping criterion is only available with some solvers, and refers to either the energy norm $\|\|r\|\|_A$ in short-recurrence solvers like Cg or the euclidian norm $\|\|r\|\|$ in solvers like GMRES. Implicit residual norms are cheaper to compute, but may be less precise due to accumulating rounding errors. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.3 initial_implicit_residual_norm()

```
deferred_factory_parameter<CriterionFactory> gko::stop::initial_implicit_residual_norm (
            double tolerance )
```

Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the residual norm has decreased below the specified value or by the specified amount.

Full usage example: Stop after 100 iterations or when the absolute residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                    .with_criteria(
                        gko::stop::max_iters(100),
                        gko::stop::absolute_residual_norm(1e-10))
                    .on(exec);
```

**Parameters**

| | |
|---|---|
| *tolerance* | the value the residual norm needs to be below. With residual $r$, initial guess $x_0$, right-hand side $b$ and matrix $A$, `absolute` means the exact value of the norm $\|\|r\|\|$, `relative` means the norm relative to the right-hand side $\|\|r\|\|/\|\|b\|\|$, `initial` means the norm relative to the initial residual $\|\|r\|\|/\|\|b - Ax_0\|\|$. An implicit stopping criterion is only available with some solvers, and refers to either the energy norm $\|\|r\|\|_A$ in short-recurrence solvers like Cg or the euclidian norm $\|\|r\|\|$ in solvers like GMRES. Implicit residual norms are cheaper to compute, but may be less precise due to accumulating rounding errors. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.4 initial_residual_norm()

```
deferred_factory_parameter<CriterionFactory> gko::stop::initial_residual_norm (
            double tolerance )
```

Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the residual norm has decreased below the specified value or by the specified amount.

Full usage example: Stop after 100 iterations or when the absolute residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(
                       gko::stop::max_iters(100),
                       gko::stop::absolute_residual_norm(1e-10))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *tolerance* | the value the residual norm needs to be below. With residual $r$, initial guess $x_0$, right-hand side $b$ and matrix $A$, `absolute` means the exact value of the norm $\|\|r\|\|$, `relative` means the norm relative to the right-hand side $\|\|r\|\|/\|\|b\|\|$, `initial` means the norm relative to the initial residual $\|\|r\|\|/\|\|b - Ax_0\|\|$. An implicit stopping criterion is only available with some solvers, and refers to either the energy norm $\|\|r\|\|_A$ in short-recurrence solvers like Cg or the euclidian norm $\|\|r\|\|$ in solvers like GMRES. Implicit residual norms are cheaper to compute, but may be less precise due to accumulating rounding errors. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.5 max_iters()

```
deferred_factory_parameter<const Iteration::Factory> gko::stop::max_iters (
            size_type count )
```

Creates the precursor to an Iteration stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after `count` iterations of the solver have finished.

Full usage example: Stop after 100 iterations or when the relative residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(
                       gko::stop::max_iters(100),
                       gko::stop::relative_residual_norm(1e-10))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *count* | the number of iterations after which to stop |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.6 min_iters()

```
deferred_factory_parameter<const CriterionFactory> gko::stop::min_iters (
            size_type count,
            deferred_factory_parameter< const CriterionFactory > criterion )
```

Creates the precursor to an MinimumIteration stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion wraps another stopping criterion inside, which only starts getting checked after the first `count` iterations finished.

Full usage example: Stop when the relative residual norm is below $10^{-10}$, but with at least 100 iterations.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(gko::stop::min_iters(100,
                       gko::stop::relative_residual_norm(1e-10)))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *count* | the number of iterations after which to start checking the inner criterion |
| *criterion* | the inner criterion, which will not be checked until `count` iterations finished, afterwards the min_iters stopping criterion behaves like the inner criterion. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.7 relative_implicit_residual_norm()

```
deferred_factory_parameter<CriterionFactory> gko::stop::relative_implicit_residual_norm (
            double tolerance )
```

Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the residual norm has decreased below the specified value or by the specified amount.

Full usage example: Stop after 100 iterations or when the absolute residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(
                       gko::stop::max_iters(100),
                       gko::stop::absolute_residual_norm(1e-10))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *tolerance* | the value the residual norm needs to be below. With residual $r$, initial guess $x_0$, right-hand side $b$ and matrix $A$, `absolute` means the exact value of the norm $\|\|r\|\|$, `relative` means the norm relative to the right-hand side $\|\|r\|\|/\|\|b\|\|$, `initial` means the norm relative to the initial residual $\|\|r\|\|/\|\|b - Ax_0\|\|$. An implicit stopping criterion is only available with some solvers, and refers to either the energy norm $\|\|r\|\|_A$ in short-recurrence solvers like Cg or the euclidian norm $\|\|r\|\|$ in solvers like GMRES. Implicit residual norms are cheaper to compute, but may be less precise due to accumulating rounding errors. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.8 relative_residual_norm()

```
deferred_factory_parameter<CriterionFactory> gko::stop::relative_residual_norm (
            double tolerance )
```

Creates the precursor to a ResidualNorm stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the residual norm has decreased below the specified value or by the specified amount.

Full usage example: Stop after 100 iterations or when the absolute residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(
                       gko::stop::max_iters(100),
                       gko::stop::absolute_residual_norm(1e-10))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *tolerance* | the value the residual norm needs to be below. With residual $r$, initial guess $x_0$, right-hand side $b$ and matrix $A$, `absolute` means the exact value of the norm $\|\|r\|\|$, `relative` means the norm relative to the right-hand side $\|\|r\|\|/\|\|b\|\|$, `initial` means the norm relative to the initial residual $\|\|r\|\|/\|\|b - Ax_0\|\|$. An implicit stopping criterion is only available with some solvers, and refers to either the energy norm $\|\|r\|\|_A$ in short-recurrence solvers like Cg or the euclidian norm $\|\|r\|\|$ in solvers like GMRES. Implicit residual norms are cheaper to compute, but may be less precise due to accumulating rounding errors. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

#### 47.17.3.9 time_limit()

```
deferred_factory_parameter<Time::Factory> gko::stop::time_limit (
            std::chrono::nanoseconds duration )
```

Creates the precursor to a Time stopping criterion factory, to be used in conjunction with `.with_criteria(...)` function calls when building a solver factory.

This stopping criterion will stop the iteration after the specified amount of time since the start of the solver run has elapsed.

Full usage example: Stop after 1 second or when the relative residual norm is below $10^{-10}$, whichever happens first.

```
auto factory = gko::solver::Cg<double>::build()
                   .with_criteria(
                       gko::stop::time_limit(std::chrono::seconds(1)),
                       gko::stop::relative_residual_norm(1e-10))
                   .on(exec);
```

**Parameters**

| | |
|---|---|
| *duration* | the time limit after which to stop the iteration. Thanks to std::chrono's converting constructors, you can specify any time units, and they will be converted to nanoseconds automatically. |

**Returns**

a deferred_factory_parameter that can be passed to the `with_criteria` function when building a solver.

## 47.18 gko::syn Namespace Reference

The Synthesizer namespace.

### Classes

- struct range

  *range records start, end, step in template*
- struct type_list

  *type_list records several types in template*
- struct value_list

  *value_list records several values with the same type in template.*

### Typedefs

- template<typename List1 , typename List2 >
  using concatenate = typename detail::concatenate_impl< List1, List2 >::type

  *concatenate combines two value_list into one value_list.*
- template<typename T >
  using as_list = typename detail::as_list_impl< T >::type

  *as_list<T> gives the alias type of as_list_impl<T>::type.*

### Functions

- template<typename T , T... Value>
  constexpr std::array< T, sizeof...(Value)> as_array (value_list< T, Value... > vl)

  *as_array<T> returns the array from value_list.*

### 47.18.1 Detailed Description

The Synthesizer namespace.

### 47.18.2 Typedef Documentation

#### 47.18.2.1 as_list

```
template<typename T >
using gko::syn::as_list = typedef typename detail::as_list_impl<T>::type
```

as_list<T> gives the alias type of as_list_impl<T>::type.

It gives a list (itself) if input is already a list, or generates list type from range input.

**Template Parameters**

| | |
|---|---|
| *T* | list or range |

#### 47.18.2.2 concatenate

```
template<typename List1 , typename List2 >
using gko::syn::concatenate = typedef typename detail::concatenate_impl<List1, List2>::type
```

concatenate combines two value_list into one value_list.

**Template Parameters**

| | |
|---|---|
| *List1* | the first list |
| *List2* | the second list |

### 47.18.3 Function Documentation

#### 47.18.3.1 as_array()

```
template<typename T , T...  Value>
constexpr std::array<T, sizeof...(Value)> gko::syn::as_array (
            value_list< T, Value...  > vl )  [constexpr]
```

as_array<T> returns the array from value_list.

It will be helpful if using for in runtime on the array.

**Template Parameters**

| | |
|---|---|
| *T* | the type of value_list |
| *Value* | the values of value_list |

**Parameters**

| | |
|---|---|
| *vl* | the input value_list |

**Returns**

std::array the std::array contains the values of value_list

```
175 {
176     return std::array<T, sizeof...(Value)>{Value...};
177 }
```

References gko::array.

## 47.19 gko::xstd Namespace Reference

The namespace for functionalities after C++14 standard.

### Typedefs

- template<typename... Ts>
  using void_t = typename detail::make_void< Ts... >::type

  *Use the custom implementation, since the std::void_t used in is_matrix_type_builder seems to trigger a compiler bug in GCC 7.5.*

### 47.19.1 Detailed Description

The namespace for functionalities after C++14 standard.

# Chapter 48

# Class Documentation

## 48.1 gko::AbsoluteComputable Class Reference

The AbsoluteComputable is an interface that allows to get the component wise absolute of a LinOp.

`#include <ginkgo/core/base/lin_op.hpp>`

### Public Member Functions

- virtual std::unique_ptr< LinOp > compute_absolute_linop () const =0

  *Gets the absolute LinOp.*
- virtual void compute_absolute_inplace ()=0

  *Compute absolute inplace on each element.*

### 48.1.1 Detailed Description

The AbsoluteComputable is an interface that allows to get the component wise absolute of a LinOp.

Use EnableAbsoluteComputation<AbsoluteLinOp> to implement this interface.

### 48.1.2 Member Function Documentation

#### 48.1.2.1 compute_absolute_linop()

```
virtual std::unique_ptr<LinOp> gko::AbsoluteComputable::compute_absolute_linop ( ) const
[pure virtual]
```

Gets the absolute LinOp.

**Returns**

a pointer to the new absolute LinOp

Implemented in gko::EnableAbsoluteComputation< AbsoluteLinOp >, gko::EnableAbsoluteComputation< remove_complex< Hybrid
gko::EnableAbsoluteComputation< remove_complex< Sellp< ValueType, IndexType > > >, gko::EnableAbsoluteComputation< re
gko::EnableAbsoluteComputation< remove_complex< Coo< ValueType, IndexType > > >, gko::EnableAbsoluteComputation< rem
gko::EnableAbsoluteComputation< remove_complex< Vector< ValueType > > >, gko::EnableAbsoluteComputation< remove_com
gko::EnableAbsoluteComputation< remove_complex< Csr< ValueType, IndexType > > >, and gko::EnableAbsoluteComputation<

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.2 gko::stop::AbsoluteResidualNorm< ValueType > Class Template Reference

The AbsoluteResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold, i.e.

`#include <ginkgo/core/stop/residual_norm.hpp>`

### Additional Inherited Members

### 48.2.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::stop::AbsoluteResidualNorm< ValueType >**

The AbsoluteResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold, i.e.

when norm(residual) < threshold. For better performance, the checks are run thanks to kernels on the executor where the algorithm is executed.

**Note**

To use this stopping criterion there are some dependencies. The constructor depends on `b` in order to get the number of right-hand sides. If this is not correctly provided, an exception ::gko::NotSupported() is thrown.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/residual_norm.hpp

## 48.3 gko::AbstractFactory< AbstractProductType, ComponentsType > Class Template Reference

The AbstractFactory is a generic interface template that enables easy implementation of the abstract factory design pattern.

`#include <ginkgo/core/base/abstract_factory.hpp>`

### Public Member Functions

- template<typename... Args>
  std::unique_ptr< abstract_product_type > generate (Args &&... args) const

  *Creates a new product from the given components.*

## 48.3.1 Detailed Description

**template<typename AbstractProductType, typename ComponentsType>**
**class gko::AbstractFactory< AbstractProductType, ComponentsType >**

The AbstractFactory is a generic interface template that enables easy implementation of the abstract factory design pattern.

The interface provides the AbstractFactory::generate() method that can produce products of type `AbstractProductType` using an object of `ComponentsType` (which can be constructed on the fly from parameters to its constructors). The generate() method is not declared as virtual, as this allows subclasses to hide the method with a variant that preserves the compile-time type of the objects. Instead, implementers should override the generate_impl() method, which is declared virtual.

Implementers of concrete factories should consider using the EnableDefaultFactory mixin to obtain default implementations of utility methods of PolymorphicObject and AbstractFactory.

**Template Parameters**

| | |
|---|---|
| *AbstractProductType* | the type of products the factory produces |
| *ComponentsType* | the type of components the factory needs to produce the product |

## 48.3.2 Member Function Documentation

### 48.3.2.1 generate()

```
template<typename AbstractProductType, typename ComponentsType>
template<typename...  Args>
std::unique_ptr<abstract_product_type> gko::AbstractFactory< AbstractProductType, Components↩
Type >::generate (
            Args &&...  args ) const  [inline]
```

Creates a new product from the given components.

The method will create an ComponentsType object from the arguments of this method, and pass it to the generate_impl() function which will create a new AbstractProductType.

**Template Parameters**

| | |
|---|---|
| *Args* | types of arguments passed to the constructor of ComponentsType |

**Parameters**

| | |
|---|---|
| *args* | arguments passed to the constructor of ComponentsType |

**Returns**

an instance of AbstractProductType

```
98      {
99          auto product =
100             this->generate_impl(components_type{std::forward<Args>(args)...});
101         for (auto logger :  this->loggers_) {
102             product->add_logger(logger);
103         }
104         return product;
105     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/abstract_factory.hpp

## 48.4 gko::AllocationError Class Reference

AllocationError is thrown if a memory allocation fails.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- AllocationError (const std::string &file, int line, const std::string &device, size_type bytes)

  *Initializes an allocation error.*

### 48.4.1 Detailed Description

AllocationError is thrown if a memory allocation fails.

### 48.4.2 Constructor & Destructor Documentation

#### 48.4.2.1 AllocationError()

```
gko::AllocationError::AllocationError (
            const std::string & file,
            int line,
            const std::string & device,
            size_type bytes )  [inline]
```

Initializes an allocation error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *device* | The device on which the error occurred |
| *bytes* | The size of the memory block whose allocation failed. |

```
539         :  Error(file, line,
540                  device + ":  failed to allocate memory block of " +
541                      std::to_string(bytes) + "B")
542     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.5 gko::Allocator Class Reference

Provides generic allocation and deallocation functionality to be used by an Executor.

`#include <ginkgo/core/base/memory.hpp>`

### 48.5.1 Detailed Description

Provides generic allocation and deallocation functionality to be used by an Executor.

The documentation for this class was generated from the following file:

- ginkgo/core/base/memory.hpp

## 48.6 gko::experimental::reorder::Amd< IndexType > Class Template Reference

Computes an Approximate Minimum Degree (AMD) reordering of an input matrix.

`#include <ginkgo/core/reorder/amd.hpp>`

### Public Member Functions

- const parameters_type & get_parameters ()

  *Returns the parameters used to construct the factory.*
- std::unique_ptr< permutation_type > generate (std::shared_ptr< const LinOp > system_matrix) const

### Static Public Member Functions

- static parameters_type build ()

  *Creates a new parameter_type to set up the factory.*

### 48.6.1 Detailed Description

**template<typename IndexType = int32>**
**class gko::experimental::reorder::Amd< IndexType >**

Computes an Approximate Minimum Degree (AMD) reordering of an input matrix.

The implementation reuses the AMD routine from the `SuiteSparse` suite (Tim Davis et al.) — Ginkgo wraps it in a `LinOpFactory` that produces a `gko::matrix::Permutation`. The system matrix must therefore be a structurally-symmetric CSR matrix.

**References**

- Amestoy, P. R., Davis, T. A., Duff, I. S. *Algorithm 837: AMD, an Approximate Minimum Degree Ordering Algorithm.* ACM Transactions on Mathematical Software, 30 (3), 381–388, 2004. `https://doi.org/10.1145/1024074.1024081`

**Template Parameters**

| | |
|---|---|
| *IndexType* | the type used to store sparsity pattern indices of the system matrix |

### 48.6.2 Member Function Documentation

#### 48.6.2.1 generate()

```
template<typename IndexType = int32>
std::unique_ptr<permutation_type> gko::experimental::reorder::Amd< IndexType >::generate (
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix to compute the reordering for. |

**Note**

This function overrides the default LinOpFactory::generate to return a Permutation instead of a generic LinOp, which would need to be cast to Permutation again to access its indices. It is only necessary because smart pointers aren't covariant.

#### 48.6.2.2 get_parameters()

```
template<typename IndexType = int32>
const parameters_type& gko::experimental::reorder::Amd< IndexType >::get_parameters ( )  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
76 { return parameters_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/reorder/amd.hpp

## 48.7 gko::amd_device Class Reference

amd_device handles the number of executor on Amd devices and have the corresponding recursive_mutex.

```
#include <ginkgo/core/base/device.hpp>
```

### 48.7.1 Detailed Description

amd_device handles the number of executor on Amd devices and have the corresponding recursive_mutex.

The documentation for this class was generated from the following file:

- ginkgo/core/base/device.hpp

## 48.8 gko::solver::ApplyWithInitialGuess Class Reference

ApplyWithInitialGuess provides a way to give the input guess for apply function.

`#include <ginkgo/core/solver/solver_base.hpp>`

### 48.8.1 Detailed Description

ApplyWithInitialGuess provides a way to give the input guess for apply function.

All functionalities have the protected access specifier. It should only be used internally.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

## 48.9 gko::are_all_integral< Args > Struct Template Reference

Evaluates if all template arguments Args fulfill std::is_integral.

`#include <ginkgo/core/base/types.hpp>`

### 48.9.1 Detailed Description

**template<typename... Args>**
**struct gko::are_all_integral< Args >**

Evaluates if all template arguments Args fulfill std::is_integral.

If that is the case, this class inherits from `std::true_type`, otherwise, it inherits from `std::false_type`. If no values are passed in, `std::true_type` is inherited from.

**Template Parameters**

| | |
|---|---|
| *Args...* | Arguments to test for std::is_integral |

The documentation for this struct was generated from the following file:

- ginkgo/core/base/types.hpp

## 48.10 gko::array< ValueType > Class Template Reference

An array is a container which encapsulates fixed-sized arrays, stored on the Executor tied to the array.

`#include <ginkgo/core/base/array.hpp>`

### Public Types

- using value_type = ValueType
  *The type of elements stored in the array.*
- using default_deleter = executor_deleter< value_type[ ]>
  *The default deleter type used by array.*
- using view_deleter = null_deleter< value_type[ ]>
  *The deleter type used for views.*

### Public Member Functions

- array () noexcept
  *Creates an empty array not tied to any executor.*
- array (std::shared_ptr< const Executor > exec) noexcept
  *Creates an empty array tied to the specified Executor.*
- array (std::shared_ptr< const Executor > exec, size_type size)
  *Creates an array on the specified Executor.*
- template<typename DeleterType >
  array (std::shared_ptr< const Executor > exec, size_type size, value_type *data, DeleterType deleter)
  *Creates an array from existing memory.*
- array (std::shared_ptr< const Executor > exec, size_type size, value_type *data)
  *Creates an array from existing memory.*
- template<typename RandomAccessIterator >
  array (std::shared_ptr< const Executor > exec, RandomAccessIterator begin, RandomAccessIterator end)
  *Creates an array on the specified Executor and initializes it with values.*
- template<typename T >
  array (std::shared_ptr< const Executor > exec, std::initializer_list< T > init_list)
  *Creates an array on the specified Executor and initializes it with values.*
- array (std::shared_ptr< const Executor > exec, const array &other)
  *Creates a copy of another array on a different executor.*
- array (const array &other)
  *Creates a copy of another array.*
- array (std::shared_ptr< const Executor > exec, array &&other)
  *Moves another array to a different executor.*
- array (array &&other)
  *Moves another array.*
- array< ValueType > as_view ()
  *Returns a non-owning view of the memory owned by this array.*

- detail::const_array_view< ValueType > as_const_view () const

  *Returns a non-owning constant view of the memory owned by this array.*
- array & operator= (const array &other)

  *Copies data from another array or view.*
- array & operator= (array &&other)

  *Moves data from another array or view.*
- template<typename OtherValueType >
  std::enable_if_t<!std::is_same< ValueType, OtherValueType >::value, array > & operator= (const array< OtherValueType > &other)

  *Copies and converts data from another array with another data type.*
- array & operator= (const detail::const_array_view< ValueType > &other)

  *Copies data from a const_array_view.*
- void clear () noexcept

  *Deallocates all data used by the array.*
- void resize_and_reset (size_type size)

  *Resizes the array so it is able to hold the specified number of elements.*
- std::vector< value_type > copy_to_host () const

  *Copies the data into an std::vector.*
- void fill (const value_type value)

  *Fill the array with the given value.*
- size_type get_size () const noexcept

  *Returns the number of elements in the array.*
- size_type get_num_elems () const noexcept

  *Returns the number of elements in the array.*
- value_type * get_data () noexcept

  *Returns a pointer to the block of memory used to store the elements of the array.*
- const value_type * get_const_data () const noexcept

  *Returns a constant pointer to the block of memory used to store the elements of the array.*
- std::shared_ptr< const Executor > get_executor () const noexcept

  *Returns the Executor associated with the array.*
- void set_executor (std::shared_ptr< const Executor > exec)

  *Changes the Executor of the array, moving the allocated data to the new Executor.*
- bool is_owning ()

  *Tells whether this array owns its data or not.*

## Static Public Member Functions

- static array view (std::shared_ptr< const Executor > exec, size_type size, value_type *data)

  *Creates an array from existing memory.*
- static detail::const_array_view< ValueType > const_view (std::shared_ptr< const Executor > exec, size_type size, const value_type *data)

  *Creates a constant (immutable) array from existing memory.*

### 48.10.1 Detailed Description

**template<typename ValueType>**
**class gko::array< ValueType >**

An array is a container which encapsulates fixed-sized arrays, stored on the Executor tied to the array.

The array stores and transfers its data as **raw** memory, which means that the constructors of its elements are not called when constructing, copying or moving the array. Thus, the array class is most suitable for storing POD types.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type of elements stored in the array |

## 48.10.2 Constructor & Destructor Documentation

### 48.10.2.1 array() [1/11]

```
template<typename ValueType>
gko::array< ValueType >::array ( )  [inline], [noexcept]
```

Creates an empty array not tied to any executor.

An array without an assigned executor can only be empty. Attempts to change its size (e.g. via the resize_and_reset method) will result in an exception. If such an array is used as the right hand side of an assignment or move assignment expression, the data of the target array will be cleared, but its executor will not be modified.

The executor can later be set by using the set_executor method. If an array with no assigned executor is assigned or moved to, it will inherit the executor of the source array.

```
196         :  size_(0), data_(nullptr, default_deleter{nullptr}), exec_(nullptr)
197     {}
```

### 48.10.2.2 array() [2/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
            std::shared_ptr< const Executor > exec )  [inline], [explicit], [noexcept]
```

Creates an empty array tied to the specified Executor.

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the array data is allocated |

```
205         :  size_(0),
206           data_(nullptr, default_deleter{exec}),
207           exec_(std::move(exec))
208     {}
```

### 48.10.2.3 array() [3/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
            std::shared_ptr< const Executor > exec,
            size_type size )  [inline]
```

Creates an array on the specified Executor.

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the array data will be allocated |
| *size* | the amount of memory (expressed as the number of `value_type` elements) allocated on the Executor |

```
218         :  size_(size),
219           data_(nullptr, default_deleter{exec}),
220           exec_(std::move(exec))
221     {
222         if (size > 0) {
223             data_.reset(exec_->alloc<value_type>(size));
224         }
225     }
```

### 48.10.2.4 array() [4/11]

```
template<typename ValueType>
template<typename DeleterType >
gko::array< ValueType >::array (
              std::shared_ptr< const Executor > exec,
              size_type size,
              value_type * data,
              DeleterType deleter )  [inline]
```

Creates an array from existing memory.

The memory will be managed by the array, and deallocated using the specified deleter (e.g. use std::default_delete for data allocated with new).

**Template Parameters**

| | |
|---|---|
| *DeleterType* | type of the deleter |

**Parameters**

| | |
|---|---|
| *exec* | executor where `data` is located |
| *size* | number of elements in `data` |
| *data* | chunk of memory used to create the array |
| *deleter* | the deleter used to free the memory |

**See also**

array::view() to create an array that does not deallocate memory

array(std::shared_ptr<cont Executor>, size_type, value_type*) to deallocate the memory using Executor::free() method

```
248         :  size_{size}, data_(data, deleter), exec_{exec}
249     {}
```

### 48.10.2.5 array() [5/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
```

```
            std::shared_ptr< const Executor > exec,
            size_type size,
            value_type * data )  [inline]
```

Creates an array from existing memory.

The memory will be managed by the array, and deallocated using the Executor::free method.

**Parameters**

| | |
|---|---|
| *exec* | executor where `data` is located |
| *size* | number of elements in `data` |
| *data* | chunk of memory used to create the array |

```
263         :  array(exec, size, data, default_deleter{exec})
264     {}
```

### 48.10.2.6 array() [6/11]

```
template<typename ValueType>
template<typename RandomAccessIterator >
gko::array< ValueType >::array (
            std::shared_ptr< const Executor > exec,
            RandomAccessIterator begin,
            RandomAccessIterator end )  [inline]
```

Creates an array on the specified Executor and initializes it with values.

**Template Parameters**

| | |
|---|---|
| *RandomAccessIterator* | type of the iterators |

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the array data will be allocated |
| *begin* | start of range of values |
| *end* | end of range of values |

```
279         :  array(exec)
280     {
281         array tmp(exec->get_master(), std::distance(begin, end));
282         std::copy(begin, end, tmp.data_.get());
283         *this = std::move(tmp);
284     }
```

### 48.10.2.7 array() [7/11]

```
template<typename ValueType>
template<typename T >
gko::array< ValueType >::array (
```

```
            std::shared_ptr< const Executor > exec,
            std::initializer_list< T > init_list )  [inline]
```

Creates an array on the specified Executor and initializes it with values.

**Template Parameters**

| | |
|---|---|
| *T* | type of values used to initialize the array (T has to be implicitly convertible to value_type) |

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the array data will be allocated |
| *init_list* | list of values used to initialize the array |

```
299         :  array(exec, begin(init_list), end(init_list))
300     {}
```

### 48.10.2.8 array() [8/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
            std::shared_ptr< const Executor > exec,
            const array< ValueType > & other )  [inline]
```

Creates a copy of another array on a different executor.

This does not invoke the constructors of the elements, instead they are copied as POD types.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the new array will be created |
| *other* | the array to copy from |

```
312         :  array(exec)
313     {
314         *this = other;
315     }
```

### 48.10.2.9 array() [9/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
            const array< ValueType > & other )  [inline]
```

Creates a copy of another array.

This does not invoke the constructors of the elements, instead they are copied as POD types.

**Parameters**

| | |
|---|---|
| *other* | the array to copy from |

```
325 :  array(other.get_executor(), other) {}
```

#### 48.10.2.10 array() [10/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
            std::shared_ptr< const Executor > exec,
            array< ValueType > && other )  [inline]
```

Moves another array to a different executor.

This does not invoke the constructors of the elements, instead they are copied as POD types.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the new array will be moved |
| *other* | the array to move |

```
336                                                     :  array(exec)
337     {
338         *this = std::move(other);
339     }
```

#### 48.10.2.11 array() [11/11]

```
template<typename ValueType>
gko::array< ValueType >::array (
            array< ValueType > && other )  [inline]
```

Moves another array.

This does not invoke the constructors of the elements, instead they are copied as POD types.

**Parameters**

| | |
|---|---|
| *other* | the array to move |

```
349 :  array(other.get_executor(), std::move(other)) {}
```

### 48.10.3 Member Function Documentation

#### 48.10.3.1 as_const_view()

```
template<typename ValueType>
detail::const_array_view<ValueType> gko::array< ValueType >::as_const_view ( ) const  [inline]
```

Returns a non-owning constant view of the memory owned by this array.

It can only be used until this array gets deleted, cleared or resized.

```
404     {
405         return const_view(this->get_executor(), this->get_size(),
406                           this->get_const_data());
407     }
```

#### 48.10.3.2 as_view()

```
template<typename ValueType>
array<ValueType> gko::array< ValueType >::as_view ( )  [inline]
```

Returns a non-owning view of the memory owned by this array.

It can only be used until this array gets deleted, cleared or resized.

```
395     {
396         return view(this->get_executor(), this->get_size(), this->get_data());
397     }
```

#### 48.10.3.3 clear()

```
template<typename ValueType>
void gko::array< ValueType >::clear ( )  [inline], [noexcept]
```

Deallocates all data used by the array.

The array is left in a valid, but empty state, so the same array can be used to allocate new memory. Calls to array::get_data() will return a `nullptr`.

```
604     {
605         size_ = 0;
606         data_.reset(nullptr);
607     }
```

Referenced by gko::index_set< IndexType >::clear(), and gko::array< index_type >::operator=().

#### 48.10.3.4 const_view()

```
template<typename ValueType>
static detail::const_array_view<ValueType> gko::array< ValueType >::const_view (
             std::shared_ptr< const Executor > exec,
             size_type size,
             const value_type * data )  [inline], [static]
```

Creates a constant (immutable) array from existing memory.

The array does not take ownership of the memory, and will not deallocate it once it goes out of scope. This array type cannot use the function `resize_and_reset` since it does not own the data it should resize.

**Parameters**

| | |
|---|---|
| *exec* | executor where `data` is located |
| *size* | number of elements in `data` |
| *data* | chunk of memory used to create the array |

**Returns**

an array constructed from `data`

```
386     {
387         return {exec, size, data};
388     }
```

Referenced by gko::array< index_type >::as_const_view().

### 48.10.3.5 copy_to_host()

```
template<typename ValueType>
std::vector<value_type> gko::array< ValueType >::copy_to_host ( ) const  [inline]
```

Copies the data into an std::vector.

**Returns**

an std::vector containing this array's data.

```
649     {
650         std::vector<value_type> result(this->get_size());
651         auto view = make_array_view(this->get_executor()->get_master(),
652                                     this->get_size(), result.data());
653         view = *this;
654         return result;
655     }
```

### 48.10.3.6 fill()

```
template<typename ValueType>
void gko::array< ValueType >::fill (
            const value_type value )
```

Fill the array with the given value.

**Parameters**

| | |
|---|---|
| *value* | the value to be filled |

#### 48.10.3.7 get_const_data()

```
template<typename ValueType>
const value_type* gko::array< ValueType >::get_const_data ( ) const  [inline], [noexcept]
```

Returns a constant pointer to the block of memory used to store the elements of the array.

**Returns**

a constant pointer to the block of memory used to store the elements of the array

```
695 { return data_.get(); }
```

Referenced by gko::array< index_type >::as_const_view(), gko::batch::matrix::Dense< ValueType >::at(), gko::batch::MultiVector< ValueType >::at(), gko::matrix::Dense< value_type >::at(), gko::preconditioner::Jacobi< ValueType, IndexType >::get_blocks(), gko::preconditioner::Jacobi< ValueType, IndexType >::get_conditioning(), gko::multigrid::Pgm< ValueType, IndexType >::get_const_agg(), gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_block_pointers(), gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_blocks(), gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_blocks_cumulative_offsets(), gko::multigrid::UniformCoarsening< ValueType, IndexType >::get_const_coarse_rows(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_const_col_idxs(), gko::batch::matrix::Csr< ValueType, IndexType >::get_const_col_idxs(), gko::batch::matrix::Ell< ValueType, IndexType >::get_const_col_idxs(), gko::matrix::Sellp< ValueType, IndexType >::get_const_col_idxs(), gko::matrix::Ell< ValueType, IndexType >::get_const_col_idxs(), gko::matrix::Coo< ValueType, IndexType >::get_const_col_idxs(), gko::device_matrix_data< ValueType, IndexType >::get_const_col_idxs(), gko::matrix::Fbcsr< ValueType, IndexType >::get_const_col_idxs(), gko::matrix::Csr< ValueType, IndexType >::get_const_col_idxs(), gko::batch::matrix::Ell< ValueType, IndexType >::get_const_col_idxs_for_item(), gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_map_block_to_row(), gko::matrix::ScaledPermutation< ValueType, IndexType >::get_const_permutation(), gko::matrix::Permutation< IndexType >::get_const_permutation(), gko::matrix::RowGatherer< IndexType >::get_const_row_idxs(), gko::device_matrix_data< ValueType, IndexType >::get_const_row_idxs(), gko::matrix::Coo< ValueType, IndexType >::get_const_row_idxs(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_const_row_ptrs(), gko::batch::matrix::Csr< ValueType, IndexType >::get_const_row_ptrs(), gko::matrix::Fbcsr< ValueType, IndexType >::get_const_row_ptrs(), gko::matrix::Csr< ValueType, IndexType >::get_const_row_ptrs(), gko::matrix::ScaledPermutation< ValueType, IndexType >::get_const_scaling_factors(), gko::matrix::Sellp< ValueType, IndexType >::get_const_slice_lengths(), gko::matrix::Sellp< ValueType, IndexType >::get_const_slice_sets(), gko::matrix::Csr< ValueType, IndexType >::get_const_srow(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_const_value(), gko::batch::matrix::Csr< ValueType, IndexType >::get_const_values(), gko::matrix::Diagonal< ValueType >::get_const_values(), gko::batch::matrix::Ell< ValueType, IndexType >::get_const_values(), gko::batch::matrix::Dense< ValueType >::get_const_values(), gko::matrix::Sellp< ValueType, IndexType >::get_const_values(), gko::matrix::Ell< ValueType, IndexType >::get_const_values(), gko::matrix::Coo< ValueType, IndexType >::get_const_values(), gko::batch::MultiVector< ValueType >::get_const_values(), gko::device_matrix_data< ValueType, IndexType >::get_const_values(), gko::matrix::Fbcsr< ValueType, IndexType >::get_const_values(), gko::matrix::Csr< ValueType, IndexType >::get_const_values(), gko::matrix::Dense< value_type >::get_const_values(), gko::batch::MultiVector< ValueType >::get_const_values_for_item(), gko::batch::matrix::Dense< ValueType >::get_const_values_for_item(), gko::batch::matrix::Csr< ValueType, IndexType >::get_const_values_for_item(), gko::batch::matrix::Ell< ValueType, IndexType >::get_const_values_for_item(), gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_part_ids(), gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_part_sizes(), gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_range_bounds(), gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_range_starting_indices(), gko::index_set< IndexType >::get_subsets_begin(), gko::index_set< IndexType >::get_subsets_end(), gko::index_set< IndexType >::get_superset_indices(), gko::matrix::Ell< ValueType, IndexType >::val_at(), and gko::matrix::Sellp< ValueType, IndexType >::val_at().

#### 48.10.3.8 get_data()

```
template<typename ValueType>
value_type* gko::array< ValueType >::get_data ( )  [inline], [noexcept]
```

Returns a pointer to the block of memory used to store the elements of the array.

**Returns**

a pointer to the block of memory used to store the elements of the array

```
686 { return data_.get(); }
```

Referenced by gko::array< index_type >::as_view(), gko::batch::matrix::Dense< ValueType >::at(), gko::batch::MultiVector< ValueType >::at(), gko::matrix::Dense< value_type >::at(), gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::compute_ell_num_stored_elements_per_row(), gko::multigrid::Pgm< ValueType, IndexType >::get_agg(), gko::multigrid::UniformCoarsening< ValueType, IndexType >::get_coarse_rows(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_col_idxs(), gko::batch::matrix::Csr< ValueType, IndexType >::get_col_idxs(), gko::batch::matrix::Ell< ValueType, IndexType >::get_col_idxs(), gko::matrix::Sellp< ValueType, IndexType >::get_col_idxs(), gko::matrix::Ell< ValueType, IndexType >::get_col_idxs(), gko::matrix::Coo< ValueType, IndexType >::get_col_idxs(), gko::device_matrix_data< ValueType, IndexType >::get_col_idxs(), gko::matrix::Fbcsr< ValueType, IndexType >::get_col_idxs(), gko::matrix::Csr< ValueType, IndexType >::get_col_idxs(), gko::batch::matrix::Ell< ValueType, IndexType >::get_col_idxs_for_item(), gko::matrix::ScaledPermutation< ValueType, IndexType >::get_permutation(), gko::matrix::Permutation< IndexType >::get_permutation(), gko::matrix::RowGatherer< IndexType >::get_row_idxs(), gko::device_matrix_data< ValueType, IndexType >::get_row_idxs(), gko::matrix::Coo< ValueType, IndexType >::get_row_idxs(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_row_ptrs(), gko::batch::matrix::Csr< ValueType, IndexType >::get_row_ptrs(), gko::matrix::Fbcsr< ValueType, IndexType >::get_row_ptrs(), gko::matrix::Csr< ValueType, IndexType >::get_row_ptrs(), gko::matrix::ScaledPermutation< ValueType, IndexType >::get_scaling_factors(), gko::matrix::Sellp< ValueType, IndexType >::get_slice_lengths(), gko::matrix::Sellp< ValueType, IndexType >::get_slice_sets(), gko::matrix::Csr< ValueType, IndexType >::get_srow(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_value(), gko::batch::matrix::Csr< ValueType, IndexType >::get_values(), gko::batch::matrix::Ell< ValueType, IndexType >::get_values(), gko::matrix::Diagonal< ValueType >::get_values(), gko::batch::matrix::Dense< ValueType >::get_values(), gko::matrix::Sellp< ValueType, IndexType >::get_values(), gko::matrix::Ell< ValueType, IndexType >::get_values(), gko::matrix::Coo< ValueType, IndexType >::get_values(), gko::batch::MultiVector< ValueType >::get_values(), gko::device_matrix_data< ValueType, IndexType >::get_values(), gko::matrix::Fbcsr< ValueType, IndexType >::get_values(), gko::matrix::Csr< ValueType, IndexType >::get_values(), gko::matrix::Dense< value_type >::get_values(), gko::batch::MultiVector< ValueType >::get_values_for_item(), gko::batch::matrix::Csr< ValueType, IndexType >::get_values_for_item(), gko::batch::matrix::Dense< ValueType >::get_values_for_item(), gko::batch::matrix::Ell< ValueType, IndexType >::get_values_for_item(), gko::array< index_type >::operator=(), gko::matrix::Ell< ValueType, IndexType >::val_at(), and gko::matrix::Sellp< ValueType, IndexType >::val_at().

#### 48.10.3.9 get_executor()

```
template<typename ValueType>
std::shared_ptr<const Executor> gko::array< ValueType >::get_executor ( ) const  [inline],
[noexcept]
```

Returns the Executor associated with the array.

**Returns**

the Executor associated with the array

```
703     {
704         return exec_;
705     }
```

Referenced by gko::array< index_type >::as_const_view(), gko::array< index_type >::as_view(), gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::compute_hybrid_config(), and gko::device_matrix_data< ValueType, IndexType >::get_executor().

#### 48.10.3.10 get_num_elems()

```
template<typename ValueType>
size_type gko::array< ValueType >::get_num_elems ( ) const  [inline], [noexcept]
```

Returns the number of elements in the array.

**Returns**

the number of elements in the array

```
677 { return get_size(); }
```

#### 48.10.3.11 get_size()

```
template<typename ValueType>
size_type gko::array< ValueType >::get_size ( ) const  [inline], [noexcept]
```

Returns the number of elements in the array.

**Returns**

the number of elements in the array

```
669 { return size_; }
```

Referenced by gko::array< index_type >::as_const_view(), gko::array< index_type >::as_view(), gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::compute_ell_num_stored_elements_per_row(), gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::compute_ell_num_stored_elements_per_row(), gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::compute_hybrid_config(), gko::batch::matrix::Csr< ValueType, IndexType >::get_const_values_for_item(), gko::matrix::SparsityCsr< ValueType, IndexType >::get_num_nonzeros(), gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_num_ranges(), gko::matrix::Csr< ValueType, IndexType >::get_num_srow_elements(), gko::matrix::Fbcsr< ValueType, IndexType >::get_num_stored_blocks(), gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_num_stored_elements(), gko::device_matrix_data< ValueType, IndexType >::get_num_stored_elements(), gko::batch::matrix::Ell< ValueType, IndexType >::get_num_stored_elements(), gko::batch::matrix::Csr< ValueType, IndexType >::get_num_stored_elements(), gko::matrix::Ell< ValueType, IndexType >::get_num_stored_elements(), gko::matrix::Coo< ValueType, IndexType >::get_num_stored_elements(), gko::batch::MultiVector< ValueType >::get_num_stored_elements(), gko::batch::matrix::Dense< ValueType >::get_num_stored_elements(), gko::matrix::Sellp< ValueType, IndexType >::get_num_stored_elements(), gko::preconditioner::Jacobi< ValueType, IndexType >::get_num_stored_elements(), gko::matrix::Fbcsr< ValueType, IndexType >::get_num_stored_elements(), gko::matrix::Dense< value_type >::get_num_stored_elements(), gko::matrix::Csr< ValueType, IndexType >::get_num_stored_elements(), gko::index_set< IndexType >::get_num_subsets(), gko::matrix::Sellp< ValueType, IndexType >::get_total_cols(), gko::batch::matrix::Csr< ValueType, IndexType >::get_values_for_item(), gko::index_set< IndexType >::index_set(), and gko::array< index_type >::operator=().

#### 48.10.3.12 is_owning()

```
template<typename ValueType>
bool gko::array< ValueType >::is_owning ( )  [inline]
```

Tells whether this array owns its data or not.

Views do not own their data and this has multiple implications. They cannot be resized since the data is not owned by the array which stores a view. It is also unclear whether custom deleter types are owning types as they could be a user-created view-type, therefore only proper array which use the `default_deleter` are considered owning types.

**Returns**

whether this array can be resized or not.

```
737     {
738         return data_.get_deleter().target_type() == typeid(default_deleter);
739     }
```

Referenced by gko::array< index_type >::operator=().

#### 48.10.3.13 operator=() [1/4]

```
template<typename ValueType>
array& gko::array< ValueType >::operator= (
             array< ValueType > && other )  [inline]
```

Moves data from another array or view.

Only the pointer and deleter type change, a copy only happens when targeting another executor's data. This means that in the following situation:

```
gko::array<int> a; // an existing array or view
gko::array<int> b; // an existing array or view
b = std::move(a);
```

Depending on whether `a` and `b` are array or view, this happens:

- `a` and `b` are views, `b` becomes the only valid view of `a`;
- `a` and `b` are arrays, `b` becomes the only valid array of `a`;
- `a` is a view and `b` is an array, `b` frees its data and becomes the only valid view of `a` ();
- `a` is an array and `b` is a view, `b` becomes the only valid array of `a`.

In all the previous cases, `a` becomes empty (e.g., a `nullptr`).

This does not invoke the constructors of the elements, instead they are copied as POD types.

The executor of this is preserved. In case this does not have an assigned executor, it will inherit the executor of other.

**Parameters**

| | |
|---|---|
| *other* | the array to move data from |

**Returns**

this

```
479     {
480         if (&other == this) {
481             return *this;
482         }
483         if (exec_ == nullptr) {
484             exec_ = other.get_executor();
485             data_ = data_manager{nullptr, default_deleter{exec_}};
486         }
487         if (other.get_executor() == nullptr) {
488             this->clear();
489             return *this;
490         }
491         if (exec_ == other.get_executor()) {
492             // same device, only move the pointer
493             data_ = std::exchange(
494                 other.data_, data_manager{nullptr, default_deleter{exec_}});
495             size_ = std::exchange(other.size_, 0);
496         } else {
497             // different device, copy the data
498             *this = other;
499             other.clear();
500         }
501         return *this;
502     }
```

### 48.10.3.14 operator=() [2/4]

```
template<typename ValueType>
array& gko::array< ValueType >::operator= (
            const array< ValueType > & other )  [inline]
```

Copies data from another array or view.

In the case of an array target, the array is resized to match the source's size. In the case of a view target, if the dimensions are not compatible a gko::OutOfBoundsError is thrown.

This does not invoke the constructors of the elements, instead they are copied as POD types.

The executor of this is preserved. In case this does not have an assigned executor, it will inherit the executor of other.

**Parameters**

| | |
|---|---|
| *other* | the array to copy from |

**Returns**

this

```
426     {
427         if (&other == this) {
428             return *this;
429         }
430         if (exec_ == nullptr) {
```

```
431             exec_ = other.get_executor();
432             data_ = data_manager{nullptr, other.data_.get_deleter()};
433         }
434         if (other.get_executor() == nullptr) {
435             this->clear();
436             return *this;
437         }
438
439         if (this->is_owning()) {
440             this->resize_and_reset(other.get_size());
441         } else {
442             GKO_ENSURE_COMPATIBLE_BOUNDS(other.get_size(), this->get_size());
443         }
444         exec_->copy_from(other.get_executor(), other.get_size(),
445                          other.get_const_data(), this->get_data());
446         return *this;
447     }
```

#### 48.10.3.15 operator=() [3/4]

```
template<typename ValueType>
template<typename OtherValueType >
std::enable_if_t<!std::is_same<ValueType, OtherValueType>::value, array>& gko::array< Value↩
Type >::operator= (
              const array< OtherValueType > & other )  [inline]
```

Copies and converts data from another array with another data type.

In the case of an array target, the array is resized to match the source's size. In the case of a view target, if the dimensions are not compatible a gko::OutOfBoundsError is thrown.

This does not invoke the constructors of the elements, instead they are copied as POD types.

The executor of this is preserved. In case this does not have an assigned executor, it will inherit the executor of other.

**Parameters**

| | |
|---|---|
| *other* | the array to copy from |

**Template Parameters**

| | |
|---|---|
| *OtherValueType* | the value type of `other` |

**Returns**

this

```
524     {
525         if (this->exec_ == nullptr) {
526             this->exec_ = other.get_executor();
527             this->data_ = data_manager{nullptr, default_deleter{this->exec_}};
528         }
529         if (other.get_executor() == nullptr) {
530             this->clear();
531             return *this;
532         }
533
534         if (this->is_owning()) {
535             this->resize_and_reset(other.get_size());
536         } else {
537             GKO_ENSURE_COMPATIBLE_BOUNDS(other.get_size(), this->get_size());
```

```
538         }
539         array<OtherValueType> tmp{this->exec_};
540         const OtherValueType* source = other.get_const_data();
541         // if we are on different executors:  copy, then convert
542         if (this->exec_ != other.get_executor()) {
543             tmp = other;
544             source = tmp.get_const_data();
545         }
546         detail::convert_data(this->exec_, other.get_size(), source,
547                              this->get_data());
548         return *this;
549     }
```

#### 48.10.3.16 operator=() [4/4]

```
template<typename ValueType>
array& gko::array< ValueType >::operator= (
            const detail::const_array_view< ValueType > & other )  [inline]
```

Copies data from a const_array_view.

In the case of an array target, the array is resized to match the source's size. In the case of a view target, if the dimensions are not compatible a gko::OutOfBoundsError is thrown.

This does not invoke the constructors of the elements, instead they are copied as POD types.

The executor of this is preserved. In case this does not have an assigned executor, it will inherit the executor of other.

**Parameters**

| | |
|---|---|
| *other* | the const_array_view to copy from |

**Returns**

this

```
569     {
570         if (this->exec_ == nullptr) {
571             this->exec_ = other.get_executor();
572             this->data_ = data_manager{nullptr, default_deleter{this->exec_}};
573         }
574         if (other.get_executor() == nullptr) {
575             this->clear();
576             return *this;
577         }
578
579         if (this->is_owning()) {
580             this->resize_and_reset(other.get_size());
581         } else {
582             GKO_ENSURE_COMPATIBLE_BOUNDS(other.get_size(), this->get_size());
583         }
584         array tmp{this->exec_};
585         const ValueType* source = other.get_const_data();
586         // if we are on different executors:  copy
587         if (this->exec_ != other.get_executor()) {
588             tmp = other.copy_to_array();
589             source = tmp.get_const_data();
590         }
591         exec_->copy_from(other.get_executor(), other.get_size(), source,
592                          this->get_data());
593         return *this;
594     }
```

#### 48.10.3.17 resize_and_reset()

```
template<typename ValueType>
void gko::array< ValueType >::resize_and_reset (
            size_type size )  [inline]
```

Resizes the array so it is able to hold the specified number of elements.

For a view and other non-owning array types, this throws an exception since these types cannot be resized.

All data stored in the array will be lost.

If the array is not assigned an executor, an exception will be thrown.

**Parameters**

| | |
|---|---|
| *size* | the amount of memory (expressed as the number of `value_type` elements) allocated on the Executor |

```
622     {
623         if (size == size_) {
624             return;
625         }
626         if (exec_ == nullptr) {
627             throw gko::NotSupported(__FILE__, __LINE__, __func__,
628                                     "gko::Executor (nullptr)");
629         }
630         if (!this->is_owning()) {
631             throw gko::NotSupported(__FILE__, __LINE__, __func__,
632                                     "Non owning gko::array cannot be resized.");
633         }
634
635         if (size > 0 && this->is_owning()) {
636             data_.reset(exec_->alloc<value_type>(size));
637             size_ = size;
638         } else {
639             this->clear();
640         }
641     }
```

Referenced by gko::array< index_type >::operator=().

#### 48.10.3.18 set_executor()

```
template<typename ValueType>
void gko::array< ValueType >::set_executor (
            std::shared_ptr< const Executor > exec )  [inline]
```

Changes the Executor of the array, moving the allocated data to the new Executor.

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the data will be moved to |

```
714     {
715         if (exec == exec_) {
716             // moving to the same executor, no-op
717             return;
718         }
719         array tmp(std::move(exec));
720         tmp = *this;
721         exec_ = std::move(tmp.exec_);
722         data_ = std::move(tmp.data_);
723     }
```

#### 48.10.3.19 view()

```
template<typename ValueType>
static array gko::array< ValueType >::view (
            std::shared_ptr< const Executor > exec,
            size_type size,
            value_type * data )  [inline], [static]
```

Creates an array from existing memory.

The array does not take ownership of the memory, and will not deallocate it once it goes out of scope. This array type cannot use the function `resize_and_reset` since it does not own the data it should resize.

**Parameters**

| | |
|---|---|
| *exec* | executor where `data` is located |
| *size* | number of elements in `data` |
| *data* | chunk of memory used to create the array |

**Returns**

an array constructed from `data`

```
366     {
367         return array{exec, size, data, view_deleter{}};
368     }
```

Referenced by gko::array< index_type >::as_view().

The documentation for this class was generated from the following file:

- ginkgo/core/base/array.hpp

## 48.11 gko::device_matrix_data< ValueType, IndexType >::arrays Struct Reference

Stores the internal arrays of a device_matrix_data object.

`#include <ginkgo/core/base/device_matrix_data.hpp>`

### 48.11.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**struct gko::device_matrix_data< ValueType, IndexType >::arrays**

Stores the internal arrays of a device_matrix_data object.

The documentation for this struct was generated from the following file:

- ginkgo/core/base/device_matrix_data.hpp

# 48.12 gko::matrix::Hybrid< ValueType, IndexType >::automatic Class Reference

automatic is a strategy_type which decides the number of stored elements per row of the ell part automatically.

`#include <ginkgo/core/matrix/hybrid.hpp>`

## Public Member Functions

- automatic ()
  *Creates an automatic strategy.*
- size_type compute_ell_num_stored_elements_per_row (array< size_type > *row_nnz) const override
  *Computes the number of stored elements per row of the ell part.*

## 48.12.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Hybrid< ValueType, IndexType >::automatic**

automatic is a strategy_type which decides the number of stored elements per row of the ell part automatically.

## 48.12.2 Member Function Documentation

### 48.12.2.1 compute_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::automatic::compute_ell_num_stored_↩
elements_per_row (
            array< size_type > * row_nnz ) const  [inline], [override], [virtual]
```

Computes the number of stored elements per row of the ell part.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |

**Returns**

the number of stored elements per row of the ell part

Implements gko::matrix::Hybrid< ValueType, IndexType >::strategy_type.

```
360         {
361             return strategy_.compute_ell_num_stored_elements_per_row(row_nnz);
362         }
```

References gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::compute_ell_num_stored↩
_elements_per_row().

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/hybrid.hpp

## 48.13 gko::BadDimension Class Reference

BadDimension is thrown if an operation is being applied to a LinOp with bad dimensions.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- BadDimension (const std::string &file, int line, const std::string &func, const std::string &op_name, size_type op_num_rows, size_type op_num_cols, const std::string &clarification)

  *Initializes a bad dimension error.*

### 48.13.1 Detailed Description

BadDimension is thrown if an operation is being applied to a LinOp with bad dimensions.

### 48.13.2 Constructor & Destructor Documentation

#### 48.13.2.1 BadDimension()

```
gko::BadDimension::BadDimension (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & op_name,
            size_type op_num_rows,
            size_type op_num_cols,
            const std::string & clarification )  [inline]
```

Initializes a bad dimension error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The function name where the error occurred |
| *op_name* | The name of the operator |
| *op_num_rows* | The row dimension of the operator |
| *op_num_cols* | The column dimension of the operator |
| *clarification* | An additional message further describing the error |

```
467         :  Error(file, line,
468                 func + ":  Object " + op_name + " has dimensions [" +
469                     std::to_string(op_num_rows) + " x " +
470                     std::to_string(op_num_cols) + "]:  " + clarification)
471     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.14 gko::batch_dim< Dimensionality, DimensionType > Struct Template Reference

A type representing the dimensions of a multidimensional batch object.

`#include <ginkgo/core/base/batch_dim.hpp>`

### Public Member Functions

- size_type get_num_batch_items () const

  *Get the number of batch items stored.*
- dim< dimensionality, dimension_type > get_common_size () const

  *Get the common size of the batch items.*
- batch_dim ()

  *The default constructor.*
- batch_dim (const size_type num_batch_items, const dim< dimensionality, dimension_type > &common_↩ size)

  *Creates a batch_dim object which stores a uniform size for all batch entries.*

### Friends

- bool operator== (const batch_dim &x, const batch_dim &y)

  *Checks if two batch_dim objects are equal.*
- bool operator!= (const batch_dim< Dimensionality, DimensionType > &x, const batch_dim< Dimensionality, DimensionType > &y)

  *Checks if two batch_dim objects are different.*

### 48.14.1 Detailed Description

**template<size_type Dimensionality = 2, typename DimensionType = size_type>**
**struct gko::batch_dim< Dimensionality, DimensionType >**

A type representing the dimensions of a multidimensional batch object.

**Template Parameters**

| | |
|---|---|
| *Dimensionality* | number of dimensions of the object |
| *DimensionType* | datatype used to represent each dimension |

## 48.14.2 Constructor & Destructor Documentation

### 48.14.2.1 batch_dim()

```
template<size_type Dimensionality = 2, typename DimensionType = size_type>
gko::batch_dim< Dimensionality, DimensionType >::batch_dim (
            const size_type num_batch_items,
            const dim< dimensionality, dimension_type > & common_size )  [inline], [explicit]
```

Creates a batch_dim object which stores a uniform size for all batch entries.

**Parameters**

| | |
|---|---|
| *num_batch_items* | the number of batch items to be stored |
| *common_size* | the common size of all the batch items stored |

**Note**

Use this constructor when uniform batches need to be stored.

```
99          :  num_batch_items_(num_batch_items), common_size_(common_size)
100     {}
```

## 48.14.3 Member Function Documentation

### 48.14.3.1 get_common_size()

```
template<size_type Dimensionality = 2, typename DimensionType = size_type>
dim<dimensionality, dimension_type> gko::batch_dim< Dimensionality, DimensionType >::get_↩
common_size ( ) const  [inline]
```

Get the common size of the batch items.

**Returns**

common_size

```
43     {
44         return common_size_;
45     }
```

Referenced by gko::batch::MultiVector< ValueType >::get_common_size(), and gko::transpose().

#### 48.14.3.2 get_num_batch_items()

```
template<size_type Dimensionality = 2, typename DimensionType = size_type>
size_type gko::batch_dim< Dimensionality, DimensionType >::get_num_batch_items ( ) const
[inline]
```

Get the number of batch items stored.

**Returns**

num_batch_items

```
35 { return num_batch_items_; }
```

Referenced by gko::batch::MultiVector< ValueType >::get_num_batch_items(), and gko::transpose().

### 48.14.4 Friends And Related Function Documentation

#### 48.14.4.1 operator"!=

```
template<size_type Dimensionality = 2, typename DimensionType = size_type>
bool operator!= (
            const batch_dim< Dimensionality, DimensionType > & x,
            const batch_dim< Dimensionality, DimensionType > & y )  [friend]
```

Checks if two batch_dim objects are different.

**Template Parameters**

| | |
|---|---|
| *Dimensionality* | number of dimensions of the dim objects |
| *DimensionType* | datatype used to represent each dimension |

**Parameters**

| | |
|---|---|
| *x* | first object |
| *y* | second object |

**Returns**

!(x == y)

```
75     {
76         return !(x == y);
77     }
```

#### 48.14.4.2 operator==

```
template<size_type Dimensionality = 2, typename DimensionType = size_type>
bool operator== (
```

```
const batch_dim< Dimensionality, DimensionType > & x,
const batch_dim< Dimensionality, DimensionType > & y )  [friend]
```

Checks if two batch_dim objects are equal.

**Parameters**

| | |
|---|---|
| *x* | first object |
| *y* | second object |

**Returns**

true if and only if all dimensions of both objects are equal.

```
56     {
57         return x.num_batch_items_ == y.num_batch_items_ &&
58                x.common_size_ == y.common_size_;
59     }
```

The documentation for this struct was generated from the following file:

- ginkgo/core/base/batch_dim.hpp

## 48.15 gko::batch::log::BatchConvergence< ValueType > Class Template Reference

Logs the final residuals and iteration counts for a batch solver.

`#include <ginkgo/core/log/batch_logger.hpp>`

### Public Member Functions

- const array< index_type > & get_num_iterations () const noexcept
- const array< real_type > & get_residual_norm () const noexcept

### Static Public Member Functions

- static std::unique_ptr< BatchConvergence > create (const mask_type &enabled_events=gko::log::Logger↩ ::batch_solver_completed_mask)

  *Creates a convergence logger.*

### 48.15.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::batch::log::BatchConvergence< ValueType >**

Logs the final residuals and iteration counts for a batch solver.

The purpose of this logger is to give simple access to standard data generated by the solver once it has converged.

**Note**

The final logged residuals are the implicit residuals that have been computed within the solver process. Depending on the solver algorithm, this may be significantly different from the true residual (||b - Ax||).

## 48.15.2 Member Function Documentation

### 48.15.2.1 create()

```
template<typename ValueType = default_precision>
static std::unique_ptr<BatchConvergence> gko::batch::log::BatchConvergence< ValueType >↩
::create (
            const mask_type & enabled_events = gko::log::Logger::batch_solver_completed_mask
)  [inline], [static]
```

Creates a convergence logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object. TODO: See if the objects can be pre-allocated beforehand instead of being copied in the on_<> event

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *enabled_events* | the events enabled for this logger. By default all events. |

**Returns**

an std::unique_ptr to the the constructed object

```
97     {
98         return std::unique_ptr<BatchConvergence>(
99             new BatchConvergence(enabled_events));
100     }
```

### 48.15.2.2 get_num_iterations()

```
template<typename ValueType = default_precision>
const array<index_type>& gko::batch::log::BatchConvergence< ValueType >::get_num_iterations (
) const  [inline], [noexcept]
```

**Returns**

The number of iterations for entire batch

```
106     {
107         return iteration_count_;
108     }
```

#### 48.15.2.3 get_residual_norm()

```
template<typename ValueType = default_precision>
const array<real_type>& gko::batch::log::BatchConvergence< ValueType >::get_residual_norm ( )
const  [inline], [noexcept]
```

**Returns**

The residual norms for the entire batch.

```
114     {
115         return residual_norm_;
116     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/log/batch_logger.hpp

## 48.16 gko::batch::BatchLinOpFactory Class Reference

A BatchLinOpFactory represents a higher order mapping which transforms one batch linear operator into another.

`#include <ginkgo/core/base/batch_lin_op.hpp>`

### Additional Inherited Members

### 48.16.1 Detailed Description

A BatchLinOpFactory represents a higher order mapping which transforms one batch linear operator into another.

In a similar fashion to LinOps, BatchLinOps are also "generated" from the BatchLinOpFactory. A function of this class is to provide a generate method, which internally cals the generate_impl(), which the concrete BatchLinOps have to implement.

#### 48.16.1.1 Example: using BatchCG in Ginkgo

```
 {c++}
// Suppose A is a batch matrix, batch_b, a batch rhs vector, and batch_x, an
// initial guess
// Create a BatchCG which runs for at most 1000 iterations, and stops after
// reducing the residual norm by 6 orders of magnitude
auto batch_cg_factory = solver::BatchCg<>::build()
    .with_max_iters(1000)
    .with_rel_residual_goal(1e-6)
    .on(cuda);
// create a batch linear operator which represents the solver
auto batch_cg = batch_cg_factory->generate(A);
// solve the system
batch_cg->apply(batch_b, batch_x);
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/batch_lin_op.hpp

# 48.17 gko::batch::solver::BatchSolver Class Reference

The BatchSolver is a base class for all batched solvers and provides the common getters and setter for these batched solver classes.

`#include <ginkgo/core/solver/batch_solver_base.hpp>`

## Public Member Functions

- std::shared_ptr< const BatchLinOp > get_system_matrix () const

  *Returns the system operator (matrix) of the linear system.*
- std::shared_ptr< const BatchLinOp > get_preconditioner () const

  *Returns the generated preconditioner.*
- double get_tolerance () const

  *Get the residual tolerance used by the solver.*
- void reset_tolerance (double res_tol)

  *Update the residual tolerance to be used by the solver.*
- int get_max_iterations () const

  *Get the maximum number of iterations set on the solver.*
- void reset_max_iterations (int max_iterations)

  *Set the maximum number of iterations for the solver to use, independent of the factory that created it.*
- ::gko::batch::stop::tolerance_type get_tolerance_type () const

  *Get the tolerance type.*
- void reset_tolerance_type (::gko::batch::stop::tolerance_type tol_type)

  *Set the type of tolerance check to use inside the solver.*

## 48.17.1 Detailed Description

The BatchSolver is a base class for all batched solvers and provides the common getters and setter for these batched solver classes.

## 48.17.2 Member Function Documentation

### 48.17.2.1 get_max_iterations()

`int gko::batch::solver::BatchSolver::get_max_iterations ( ) const [inline]`

Get the maximum number of iterations set on the solver.

**Returns**

Maximum number of iterations.

```
77 { return this->max_iterations_; }
```

#### 48.17.2.2 get_preconditioner()

```
std::shared_ptr<const BatchLinOp> gko::batch::solver::BatchSolver::get_preconditioner ( )
const  [inline]
```

Returns the generated preconditioner.

**Returns**

the generated preconditioner.

```
47     {
48         return this->preconditioner_;
49     }
```

#### 48.17.2.3 get_system_matrix()

```
std::shared_ptr<const BatchLinOp> gko::batch::solver::BatchSolver::get_system_matrix ( ) const
[inline]
```

Returns the system operator (matrix) of the linear system.

**Returns**

the system operator (matrix)

```
37     {
38         return this->system_matrix_;
39     }
```

#### 48.17.2.4 get_tolerance()

```
double gko::batch::solver::BatchSolver::get_tolerance ( ) const  [inline]
```

Get the residual tolerance used by the solver.

**Returns**

The residual tolerance.

```
56 { return this->residual_tol_; }
```

#### 48.17.2.5 get_tolerance_type()

```
::gko::batch::stop::tolerance_type gko::batch::solver::BatchSolver::get_tolerance_type ( )
const  [inline]
```

Get the tolerance type.

**Returns**

The tolerance type.

```
99     {
100         return this->tol_type_;
101     }
```

#### 48.17.2.6 reset_max_iterations()

```
void gko::batch::solver::BatchSolver::reset_max_iterations (
            int max_iterations )  [inline]
```

Set the maximum number of iterations for the solver to use, independent of the factory that created it.

**Parameters**

| | |
|---|---|
| *max_iterations* | The maximum number of iterations for the solver. |

```
86     {
87         if (max_iterations < 0) {
88             GKO_INVALID_STATE("Max iterations cannot be negative!");
89         }
90         this->max_iterations_ = max_iterations;
91     }
```

#### 48.17.2.7 reset_tolerance()

```
void gko::batch::solver::BatchSolver::reset_tolerance (
             double res_tol )  [inline]
```

Update the residual tolerance to be used by the solver.

**Parameters**

| | |
|---|---|
| *res_tol* | The residual tolerance to be used for subsequent invocations of the solver. |

```
65     {
66         if (res_tol < 0) {
67             GKO_INVALID_STATE("Tolerance cannot be negative!");
68         }
69         this->residual_tol_ = res_tol;
70     }
```

#### 48.17.2.8 reset_tolerance_type()

```
void gko::batch::solver::BatchSolver::reset_tolerance_type (
             ::gko::batch::stop::tolerance_type tol_type )  [inline]
```

Set the type of tolerance check to use inside the solver.

**Parameters**

| | |
|---|---|
| *tol_type* | The tolerance type. |

```
109     {
110         if (tol_type == ::gko::batch::stop::tolerance_type::absolute ||
111             tol_type == ::gko::batch::stop::tolerance_type::relative) {
112             this->tol_type_ = tol_type;
113         } else {
114             GKO_INVALID_STATE("Invalid tolerance type specified!");
115         }
116     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/solver/batch_solver_base.hpp

## 48.18 gko::bfloat16 Class Reference

A class providing basic support for bfloat16 precision floating point types.

`#include <ginkgo/core/base/bfloat16.hpp>`

### 48.18.1 Detailed Description

A class providing basic support for bfloat16 precision floating point types.

For now the only features are reduced storage compared to single precision and conversions from and to single precision floating point type.

The documentation for this class was generated from the following file:

- ginkgo/core/base/bfloat16.hpp

## 48.19 gko::solver::Bicg< ValueType > Class Template Reference

BICG or the Biconjugate gradient method is a Krylov subspace solver.

`#include <ginkgo/core/solver/bicg.hpp>`

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.19.1 Detailed Description

template<typename ValueType = default_precision>
class gko::solver::Bicg< ValueType >

BICG or the Biconjugate gradient method is a Krylov subspace solver.

Being a generic solver, it is capable of solving general matrices, including non-s.p.d matrices. Though, the memory and the computational requirement of the BiCG solver are higher than of its s.p.d solver counterpart, it has the capability to solve generic systems.

BiCG is based on the bi-Lanczos tridiagonalization method and in exact arithmetic should terminate in at most $N$ iterations ( $2N$ matrix-vector products — one per iteration with $A$ and $A^H$ each). It couples two Krylov sequences and maintains residuals $r_k$, shadow residuals $\tilde{r}_k$, and search directions $p_k, \tilde{p}_k$ that satisfy the biorthogonality conditions $\tilde{r}_i^H r_j = 0$ for $i \neq j$. Each iteration performs the coupled update

$$\alpha_k = \frac{\tilde{r}_k^H r_k}{\tilde{p}_k^H A p_k}, \qquad r_{k+1} = r_k - \alpha_k A p_k, \qquad \tilde{r}_{k+1} = \tilde{r}_k - \alpha_k A^H \tilde{p}_k.$$

It forms the basis of cheaper variants such as BiCGSTAB and CGS, which avoid the explicit $A^H$ apply.

### References

- Fletcher, R. *Conjugate gradient methods for indefinite systems.* Numerical Analysis (Dundee 1975), Lecture Notes in Mathematics 506, Springer, 1976. https://doi.org/10.1007/BFb0080116

### Template Parameters

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.19.2 Member Function Documentation

### 48.19.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Bicg< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

### Returns

true as iterative solvers use the data in x as an initial guess.

```
86 { return true; }
```

#### 48.19.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Bicg< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.19.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Bicg< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.19.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Bicg< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/bicg.hpp

# 48.20 gko::batch::solver::Bicgstab< ValueType > Class Template Reference

BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.

`#include <ginkgo/core/solver/batch_bicgstab.hpp>`

## Additional Inherited Members

## 48.20.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::batch::solver::Bicgstab< ValueType >**

BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.

Being a generic solver, it is capable of solving general matrices, including non-s.p.d matrices.

This solver solves a batch of linear systems using the Bicgstab algorithm. Each linear system in the batch can converge independently.

Unless otherwise specified via the `preconditioner` factory parameter, this implementation does not use any preconditioner by default. The type of tolerance (absolute or relative) and the maximum number of iterations to be used in the stopping criterion can be set via the factory parameters.

**Note**

The tolerance check is against the internal residual computed within the solver process. This implicit (internal) residual, can diverge from the true residual (||b - Ax||). A posterori checks (by computing the true residual, ||b - Ax||) are recommended to ensure that the solution has converged to the desired tolerance.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

The documentation for this class was generated from the following file:

- ginkgo/core/solver/batch_bicgstab.hpp

# 48.21 gko::solver::Bicgstab< ValueType > Class Template Reference

BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.

`#include <ginkgo/core/solver/bicgstab.hpp>`

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.21.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Bicgstab< ValueType >**

BiCGSTAB or the Bi-Conjugate Gradient-Stabilized is a Krylov subspace solver.

Being a generic solver, it is capable of solving general matrices, including non-s.p.d matrices. Though, the memory and the computational requirement of the BiCGSTAB solver are higher than of its s.p.d solver counterpart, it has the capability to solve generic systems. It was developed by stabilizing the BiCG method: both operator applications of an iteration use $A$ itself, so no $A^H$ apply (nor a transposed preconditioner) is needed, at the cost of one additional inner product per iteration.

Each iteration interleaves a BiCG-style step with a one-dimensional GMRES minimization: an intermediate residual $s = r_k - \alpha_k A p_k$ is constructed, then the stabilizing coefficient is chosen to minimize $\|r_{k+1}\|_2$ along $As$:

$$\omega_k = \frac{(As)^H s}{(As)^H (As)}, \qquad x_{k+1} = x_k + \alpha_k p_k + \omega_k s, \qquad r_{k+1} = s - \omega_k A s.$$

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of the elements of the system matrix. |

## 48.21.2 Member Function Documentation

#### 48.21.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Bicgstab< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
83 { return true; }
```

#### 48.21.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Bicgstab< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.21.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Bicgstab< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.21.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Bicgstab< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/bicgstab.hpp

## 48.22 gko::preconditioner::block_interleaved_storage_scheme< IndexType > Struct Template Reference

Defines the parameters of the interleaved block storage scheme used by block-Jacobi blocks.

```
#include <ginkgo/core/preconditioner/jacobi.hpp>
```

### Public Member Functions

- IndexType get_group_size () const noexcept

  *Returns the number of elements in the group.*
- size_type compute_storage_space (size_type num_blocks) const noexcept

  *Computes the storage space required for the requested number of blocks.*
- IndexType get_group_offset (IndexType block_id) const noexcept

  *Returns the offset of the group belonging to the block with the given ID.*
- IndexType get_block_offset (IndexType block_id) const noexcept

  *Returns the offset of the block with the given ID within its group.*
- IndexType get_global_block_offset (IndexType block_id) const noexcept

  *Returns the offset of the block with the given ID.*
- IndexType get_stride () const noexcept

  *Returns the stride between columns of the block.*

## Public Attributes

- IndexType block_offset

  *The offset between consecutive blocks within the group.*
- IndexType group_offset

  *The offset between two block groups.*
- uint32 group_power

  *Then base 2 power of the group.*

### 48.22.1 Detailed Description

**template<typename IndexType>**
**struct gko::preconditioner::block_interleaved_storage_scheme< IndexType >**

Defines the parameters of the interleaved block storage scheme used by block-Jacobi blocks.

**Template Parameters**

| | |
|---|---|
| *IndexType* | type used for storing indices of the matrix |

### 48.22.2 Member Function Documentation

#### 48.22.2.1 compute_storage_space()

```
template<typename IndexType>
size_type gko::preconditioner::block_interleaved_storage_scheme< IndexType >::compute_storage↩
_space (
            size_type num_blocks ) const  [inline], [noexcept]
```

Computes the storage space required for the requested number of blocks.

**Parameters**

| | |
|---|---|
| *num_blocks* | the total number of blocks that needs to be stored |

**Returns**

the total memory (as the number of elements) that need to be allocated for the scheme

**Note**

To simplify using the method in situations where the number of blocks is not known, for a special input `size↩_type{} - 1` the method returns `0` to avoid overallocation of memory.

```
88      {
89          return (num_blocks + 1 == size_type{0})
90                     ?  size_type{0}
91                     :  ceildiv(num_blocks, this->get_group_size()) * group_offset;
92      }
```

### 48.22.2.2 get_block_offset()

```
template<typename IndexType>
IndexType gko::preconditioner::block_interleaved_storage_scheme< IndexType >::get_block_offset
(
              IndexType block_id ) const  [inline], [noexcept]
```

Returns the offset of the block with the given ID within its group.

**Parameters**

| | |
|---|---|
| *block_id* | the ID of the block |

**Returns**

the offset of the block with ID `block_id` within its group

```
114     {
115         return block_offset * (block_id & (this->get_group_size() - 1));
116     }
```

Referenced by gko::preconditioner::block_interleaved_storage_scheme< index_type >::get_global_block_offset().

### 48.22.2.3 get_global_block_offset()

```
template<typename IndexType>
IndexType gko::preconditioner::block_interleaved_storage_scheme< IndexType >::get_global_↩
block_offset (
              IndexType block_id ) const  [inline], [noexcept]
```

Returns the offset of the block with the given ID.

**Parameters**

| | |
|---|---|
| *block_id* | the ID of the block |

**Returns**

the offset of the block with ID `block_id`

```
127     {
128         return this->get_group_offset(block_id) +
129                this->get_block_offset(block_id);
130     }
```

#### 48.22.2.4 get_group_offset()

```
template<typename IndexType>
IndexType gko::preconditioner::block_interleaved_storage_scheme< IndexType >::get_group_offset
(
            IndexType block_id ) const  [inline], [noexcept]
```

Returns the offset of the group belonging to the block with the given ID.

**Parameters**

| | |
|---|---|
| *block_id* | the ID of the block |

**Returns**

the offset of the group belonging to block with ID `block_id`

```
102     {
103         return group_offset * (block_id » group_power);
104     }
```

Referenced by gko::preconditioner::block_interleaved_storage_scheme< index_type >::get_global_block_offset().

#### 48.22.2.5 get_group_size()

```
template<typename IndexType>
IndexType gko::preconditioner::block_interleaved_storage_scheme< IndexType >::get_group_size (
) const  [inline], [noexcept]
```

Returns the number of elements in the group.

**Returns**

the number of elements in the group

```
70     {
71         return one<IndexType>() « group_power;
72     }
```

Referenced by gko::preconditioner::block_interleaved_storage_scheme< index_type >::compute_storage_space(), and gko::preconditioner::block_interleaved_storage_scheme< index_type >::get_block_offset().

#### 48.22.2.6 get_stride()

```
template<typename IndexType>
IndexType gko::preconditioner::block_interleaved_storage_scheme< IndexType >::get_stride ( )
const  [inline], [noexcept]
```

Returns the stride between columns of the block.

**Returns**

stride between columns of the block

```
138     {
139         return block_offset « group_power;
140     }
```

### 48.22.3 Member Data Documentation

#### 48.22.3.1 group_power

```
template<typename IndexType>
uint32 gko::preconditioner::block_interleaved_storage_scheme< IndexType >::group_power
```

Then base 2 power of the group.

I.e. the group contains `1 << group_power` elements.

Referenced by gko::preconditioner::block_interleaved_storage_scheme< index_type >::get_group_offset(), gko↩ ::preconditioner::block_interleaved_storage_scheme< index_type >::get_group_size(), and gko::preconditioner↩ ::block_interleaved_storage_scheme< index_type >::get_stride().

The documentation for this struct was generated from the following file:

- ginkgo/core/preconditioner/jacobi.hpp

## 48.23 gko::BlockOperator Class Reference

A BlockOperator represents a linear operator that is partitioned into multiple blocks.

```
#include <ginkgo/core/base/block_operator.hpp>
```

### Public Member Functions

- dim< 2 > get_block_size () const
  *Get the block dimension of this, i.e.*
- const LinOp * block_at (size_type i, size_type j) const
  *Const access to a specific block.*
- BlockOperator (const BlockOperator &other)
  *Copy constructs a BlockOperator.*
- BlockOperator (BlockOperator &&other) noexcept
  *Move constructs a BlockOperator.*
- BlockOperator & operator= (const BlockOperator &other)
  *Copy assigns a BlockOperator.*
- BlockOperator & operator= (BlockOperator &&other)
  *Move assigns a BlockOperator.*

### Static Public Member Functions

- static std::unique_ptr< BlockOperator > create (std::shared_ptr< const Executor > exec)
  *Create empty BlockOperator.*
- static std::unique_ptr< BlockOperator > create (std::shared_ptr< const Executor > exec, std::vector< std↩ ::vector< std::shared_ptr< const LinOp >>> blocks)
  *Create BlockOperator from the given blocks.*

### 48.23.1 Detailed Description

A BlockOperator represents a linear operator that is partitioned into multiple blocks.

For example, a BlockOperator can be used to define the operator:

```
| A B |
| C D |
```

where `A, B, C, D` itself are matrices of compatible size. This can be created with:

```
 {c++}
std::shared_ptr<LinOp> A = ...;
std::shared_ptr<LinOp> B = ...;
std::shared_ptr<LinOp> C = ...;
std::shared_ptr<LinOp> D = ...;
auto bop = BlockOperator::create(exec, {{A, B}, {C, D}});
```

The requirements on the individual blocks passed to the create method are:

- In each block-row, all blocks have the same number of rows
- In each block-column, all blocks have the same number of columns
- Each block-row must have the same number of blocks It is possible to set blocks to zero, by passing in a nullptr. But every block-row and block-column must contain at least one non-nullptr block.

The constructor will store all passed in blocks on the same executor as the BlockOperator, which will requires copying any block that is associated with a different executor.

### 48.23.2 Constructor & Destructor Documentation

#### 48.23.2.1 BlockOperator() [1/2]

```
gko::BlockOperator::BlockOperator (
            const BlockOperator & other )
```

Copy constructs a BlockOperator.

The executor of other is used for this. The blocks of other are deep-copied into this, using clone.

#### 48.23.2.2 BlockOperator() [2/2]

```
gko::BlockOperator::BlockOperator (
            BlockOperator && other )  [noexcept]
```

Move constructs a BlockOperator.

The executor of other is used for this. All remaining data of other is moved into this. After this operation, other will be empty.

### 48.23.3 Member Function Documentation

#### 48.23.3.1 block_at()

```
const LinOp* gko::BlockOperator::block_at (
            size_type i,
            size_type j ) const  [inline]
```

Const access to a specific block.

**Parameters**

| | |
|---|---|
| *i* | block row. |
| *j* | block column. |

**Returns**

the block stored at (i, j).

```
98      {
99          GKO_ENSURE_IN_DIMENSION_BOUNDS(i, j, block_size_);
100          return blocks_[i * block_size_[1] + j].get();
101      }
```

#### 48.23.3.2 create() [1/2]

```
static std::unique_ptr<BlockOperator> gko::BlockOperator::create (
              std::shared_ptr< const Executor > exec )  [static]
```

Create empty BlockOperator.

**Parameters**

| | |
|---|---|
| *exec* | the executor of this. |

**Returns**

empty BlockOperator.

#### 48.23.3.3 create() [2/2]

```
static std::unique_ptr<BlockOperator> gko::BlockOperator::create (
              std::shared_ptr< const Executor > exec,
              std::vector< std::vector< std::shared_ptr< const LinOp >>> blocks )  [static]
```

Create BlockOperator from the given blocks.

**Parameters**

| | |
|---|---|
| *exec* | the executor of this. |
| *blocks* | the blocks of this operator. The blocks will be used in a row-major form. |

**Returns**

BlockOperator with the given blocks.

#### 48.23.3.4 get_block_size()

```
dim<2> gko::BlockOperator::get_block_size ( ) const  [inline]
```

Get the block dimension of this, i.e.

the number of blocks per row and column.

**Returns**

The block dimension of this.

```
87 { return block_size_; }
```

#### 48.23.3.5 operator=() [1/2]

```
BlockOperator& gko::BlockOperator::operator= (
            BlockOperator && other )
```

Move assigns a BlockOperator.

The executor of this is not modified. All data of other (except its executor) is moved into this. If the executor of this and other differ, the blocks will be copied to the executor of this. After this operation, other will be empty.

#### 48.23.3.6 operator=() [2/2]

```
BlockOperator& gko::BlockOperator::operator= (
            const BlockOperator & other )
```

Copy assigns a BlockOperator.

The executor of this is not modified. The blocks of other are deep-copied into this, using clone.

The documentation for this class was generated from the following file:

- ginkgo/core/base/block_operator.hpp

## 48.24 gko::BlockSizeError< IndexType > Class Template Reference

Error that denotes issues between block sizes and matrix dimensions.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- BlockSizeError (const std::string &file, const int line, const int block_size, const IndexType size)

### 48.24.1 Detailed Description

**template<typename IndexType>**
**class gko::BlockSizeError< IndexType >**

Error that denotes issues between block sizes and matrix dimensions.

**Template Parameters**

| | |
|---|---|
| *IndexType* | Type of index used by the linear algebra object that is incompatible with the required block size. |

### 48.24.2 Constructor & Destructor Documentation

#### 48.24.2.1 BlockSizeError()

```
template<typename IndexType >
gko::BlockSizeError< IndexType >::BlockSizeError (
            const std::string & file,
            const int line,
            const int block_size,
            const IndexType size )  [inline]
```

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *block_size* | Size of small dense blocks in a matrix |
| *size* | The size that is not exactly divided by the block size |

```
492         :  Error(file, line,
493                  "block size = " + std::to_string(block_size) +
494                      ", size = " + std::to_string(size))
495     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.25 gko::solver::CbGmres< ValueType > Class Template Reference

CB-GMRES or the compressed basis generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.

```
#include <ginkgo/core/solver/cb_gmres.hpp>
```

### Public Member Functions

- size_type get_krylov_dim () const

  *Returns the Krylov dimension.*
- void set_krylov_dim (size_type other)

  *Sets the Krylov dimension.*
- cb_gmres::storage_precision get_storage_precision () const

  *Returns the storage precision used internally.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.25.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::CbGmres< ValueType >**

CB-GMRES or the compressed basis generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of CB-GMRES are merged into 2 separate steps. Classical Gram-Schmidt with reorthogonalization is used.

The Krylov basis can be stored in reduced precision (compressed) to reduce memory accesses, while all computations (including Krylov basis operations) are performed in the same arithmetic precision ValueType. By default, the Krylov basis are stored in one precision lower than ValueType.

Mathematically CB-GMRES is the same algorithm as standard GMRES — the Arnoldi process produces an orthonormal basis $V_m$ and an upper Hessenberg $\bar{H}_m$ with $AV_m = V_{m+1}\bar{H}_m$, and the iterate is chosen as

$$x_m = x_0 + V_m y_m, \qquad y_m = \arg\min_y \|\beta e_1 - \bar{H}_m y\|_2.$$

The "compressed basis" variant stores the columns of $V_m$ in a precision lower than ValueType: each basis vector is cast down to the storage type when written and cast back up when read. The Hessenberg matrix, the Givens rotations, and the working residual all stay in full ValueType precision, so the final approximation has the same accuracy as standard GMRES while the basis storage and the bandwidth needed to apply it are reduced. See the `storage_precision` enum for the supported compression options.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the arithmetic precision and the precision of matrix elements |

## 48.25.2 Member Function Documentation

### 48.25.2.1 get_krylov_dim()

```
template<typename ValueType = default_precision>
size_type gko::solver::CbGmres< ValueType >::get_krylov_dim ( ) const  [inline]
```

Returns the Krylov dimension.

**Returns**

the Krylov dimension

```
128 { return parameters_.krylov_dim; }
```

#### 48.25.2.2 get_storage_precision()

```
template<typename ValueType = default_precision>
cb_gmres::storage_precision gko::solver::CbGmres< ValueType >::get_storage_precision ( ) const
[inline]
```

Returns the storage precision used internally.

**Returns**

the storage precision used internally

```
143     {
144         return parameters_.storage_precision;
145     }
```

#### 48.25.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::CbGmres< ValueType >::parse (
             const config::pnode & config,
             const config::registry & context,
             const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.25.2.4 set_krylov_dim()

```
template<typename ValueType = default_precision>
void gko::solver::CbGmres< ValueType >::set_krylov_dim (
             size_type other )  [inline]
```

Sets the Krylov dimension.

**Parameters**

| | |
|---|---|
| *other* | the new Krylov dimension |

```
135 { parameters_.krylov_dim = other; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/solver/cb_gmres.hpp

# 48.26 gko::batch::solver::Cg< ValueType > Class Template Reference

Cg or the Conjugate Gradient is a Krylov subspace solver.

```
#include <ginkgo/core/solver/batch_cg.hpp>
```

## Additional Inherited Members

## 48.26.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::batch::solver::Cg< ValueType >**

Cg or the Conjugate Gradient is a Krylov subspace solver.

It is a short recurrence solver that is generally used to solve linear systems with SPD matrices.

This solver solves a batch of linear systems using the Cg algorithm. Each linear system in the batch can converge independently.

Unless otherwise specified via the `preconditioner` factory parameter, this implementation does not use any preconditioner by default. The type of tolerance (absolute or relative) and the maximum number of iterations to be used in the stopping criterion can be set via the factory parameters.

**Note**

The tolerance check is against the internal residual computed within the solver process. This implicit (internal) residual can diverge from the true residual (||b - Ax||). A posterori checks (by computing the true residual, ||b - Ax||) are recommended to ensure that the solution has converged to the desired tolerance.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

The documentation for this class was generated from the following file:

- ginkgo/core/solver/batch_cg.hpp

# 48.27 gko::solver::Cg< ValueType > Class Template Reference

CG or the conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.

`#include <ginkgo/core/solver/cg.hpp>`

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.27.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Cg< ValueType >**

CG or the conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.

Though this method performs very well for symmetric positive definite matrices, it is in general not suitable for general matrices.

CG constructs a sequence of search directions $p_0, p_1, \ldots$ that are mutually $A$-conjugate ( $\langle p_i, Ap_j \rangle = 0$ for $i \neq j$). The iterate chosen along these directions minimizes the error in the energy norm over the affine Krylov subspace,

$$x_k = \arg \min_{x \in x_0 + \mathcal{K}_k(A, r_0)} \|x - x^*\|_A, \qquad \|e\|_A = \sqrt{\langle e, Ae \rangle},$$

which yields a short recurrence: at each step the new search direction is built from the preconditioned residual $z_k = Mr_k$ and the previous direction via the Fletcher-Reeves coefficient

$$\beta_k = \frac{\langle r_k, z_k \rangle}{\langle r_{k-1}, z_{k-1} \rangle}, \qquad p_k = z_k + \beta_k p_{k-1}.$$

This formula assumes that $M$ stays fixed across iterations; for a varying preconditioner, use the Polak-Ribière variant in FCG.

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of CG are merged into 2 separate steps.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.27.2 Member Function Documentation

### 48.27.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Cg< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
88 { return true; }
```

### 48.27.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Cg< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.27.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Cg< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.27.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Cg< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/cg.hpp

## 48.28 gko::solver::Cgs< ValueType > Class Template Reference

CGS or the conjugate gradient square method is an iterative type Krylov subspace method which is suitable for general systems.

```
#include <ginkgo/core/solver/cgs.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.28.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Cgs< ValueType >**

CGS or the conjugate gradient square method is an iterative type Krylov subspace method which is suitable for general systems.

CGS rests on the identity that BiCG produces a residual of the form $r_k^{\mathrm{BiCG}} = \psi_k(A)r_0$, where $\psi_k$ is the residual polynomial. Squaring this polynomial yields the CGS residual

$$r_k^{\mathrm{CGS}} = \psi_k(A)^2 r_0,$$

so the iteration avoids the explicit $A^H$ apply that BiCG requires, and it contracts the residual twice as fast per step when BiCG converges.

Starting from $r_0 = b - Ax_0$, a shadow residual $\tilde{r}_0$ (Ginkgo uses $\tilde{r}_0 = r_0$), $q_{-1} = p_{-1} = 0$ and $\rho_{-1} = 1$, one iteration with the preconditioner $M \approx A^{-1}$ reads

$$\begin{aligned} \rho_k &= \langle \tilde{r}_0, r_k \rangle, & \beta_k &= \rho_k / \rho_{k-1}, \\ u_k &= r_k + \beta_k q_{k-1}, & p_k &= u_k + \beta_k (q_{k-1} + \beta_k p_{k-1}), \\ \hat{v}_k &= AMp_k, & \alpha_k &= \rho_k / \langle \tilde{r}_0, \hat{v}_k \rangle, \\ q_k &= u_k - \alpha_k \hat{v}_k, & \hat{u}_k &= M(u_k + q_k), \\ x_{k+1} &= x_k + \alpha_k \hat{u}_k, & r_{k+1} &= r_k - \alpha_k A \hat{u}_k. \end{aligned}$$

The pair $u_k, q_k$ takes the place of the single BiCG search direction, which is why the update of $x$ uses their sum. The trade-off for dropping $A^H$ is that the squared polynomial also squares the error: the residuals lose monotonicity and can oscillate strongly on poorly conditioned systems. When this matters in practice, BiCGSTAB replaces the second application of $\psi_k$ by a locally minimizing polynomial for the same number of operator applies and one extra inner product.

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of CGS are merged into 3 separate steps.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.28.2 Member Function Documentation

### 48.28.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Cgs< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
102 { return true; }
```

#### 48.28.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Cgs< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.28.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Cgs< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.28.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Cgs< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/cgs.hpp

## 48.29 gko::solver::Chebyshev< ValueType > Class Template Reference

Chebyshev iteration is an iterative method for solving nonsymmetric problems based on some knowledge of the spectrum of the (preconditioned) system matrix.

```
#include <ginkgo/core/solver/chebyshev.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*
- Chebyshev & operator= (const Chebyshev &)

  *Copy-assigns a Chebyshev solver.*
- Chebyshev & operator= (Chebyshev &&)

  *Move-assigns a Chebyshev solver.*
- Chebyshev (const Chebyshev &)

  *Copy-constructs an Chebyshev solver.*
- Chebyshev (Chebyshev &&)

  *Move-constructs an Chebyshev solver.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.29.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Chebyshev< ValueType >**

Chebyshev iteration is an iterative method for solving nonsymmetric problems based on some knowledge of the spectrum of the (preconditioned) system matrix.

It avoids the computation of inner products, which may be a performance bottleneck for distributed system. Chebyshev iteration is developed based on Chebyshev polynomials of the first kind.

Given a preconditioner $M \approx A^{-1}$, the iterate is updated with the three-term recurrence

$$x_{k+1} = x_k + \alpha_k M(b - Ax_k) + \beta_k(x_k - x_{k-1}),$$

where the scalars $\alpha_k, \beta_k$ are derived from an estimate of the spectrum of $MA$. In pseudo-code:

```
solution = initial_guess
while not converged:
    residual = b - A solution
    error = preconditioner(A) * residual
    solution = solution + alpha_i * error + beta_i * (solution_i -
solution_{i-1})
```

#### References

- Barrett, R., Berry, M., Chan, T. F., et al. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods.* 2nd ed. SIAM, 1994. https://doi.org/10.1137/1.9781611971538

#### Template Parameters

| | |
|---|---|
| *ValueType* | precision of matrix elements |

### 48.29.2 Constructor & Destructor Documentation

#### 48.29.2.1 Chebyshev() [1/2]

```
template<typename ValueType = default_precision>
gko::solver::Chebyshev< ValueType >::Chebyshev (
            const Chebyshev< ValueType > &  )
```

Copy-constructs an Chebyshev solver.

Inherits the executor, shallow-copies inner solver, stopping criterion and system matrix.

#### 48.29.2.2 Chebyshev() [2/2]

```
template<typename ValueType = default_precision>
gko::solver::Chebyshev< ValueType >::Chebyshev (
            Chebyshev< ValueType > &&  )
```

Move-constructs an Chebyshev solver.

Preserves the executor, moves inner solver, stopping criterion and system matrix. The moved-from object is empty (0x0 and nullptr inner solver, stopping criterion and system matrix)

## 48.29.3 Member Function Documentation

### 48.29.3.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Chebyshev< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
96     {
97         return this->get_default_initial_guess() ==
98                initial_guess_mode::provided;
99     }
```

References gko::solver::provided.

### 48.29.3.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Chebyshev< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.29.3.3 operator=() [1/2]

```
template<typename ValueType = default_precision>
Chebyshev& gko::solver::Chebyshev< ValueType >::operator= (
             Chebyshev< ValueType > &&  )
```

Move-assigns a Chebyshev solver.

Preserves the executor, moves inner solver, stopping criterion and system matrix. If the executors mismatch, clones inner solver, stopping criterion and system matrix onto this executor. The moved-from object is empty (0x0 and nullptr inner solver, stopping criterion and system matrix)

#### 48.29.3.4 operator=() [2/2]

```
template<typename ValueType = default_precision>
Chebyshev& gko::solver::Chebyshev< ValueType >::operator= (
            const Chebyshev< ValueType > &  )
```

Copy-assigns a Chebyshev solver.

Preserves the executor, shallow-copies inner solver, stopping criterion and system matrix. If the executors mismatch, clones inner solver, stopping criterion and system matrix onto this executor.

#### 48.29.3.5 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Chebyshev< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.29.3.6 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Chebyshev< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/chebyshev.hpp

# 48.30 gko::experimental::factorization::Cholesky< ValueType, IndexType > Class Template Reference

Computes a Cholesky factorization of a symmetric, positive-definite sparse matrix.

`#include <ginkgo/core/factorization/cholesky.hpp>`

## Public Member Functions

- const parameters_type & get_parameters ()

  *Returns the parameters used to construct the factory.*
- const parameters_type & get_parameters () const

  *Returns the parameters used to construct the factory.*
- std::unique_ptr< factorization_type > generate (std::shared_ptr< const LinOp > system_matrix) const

## Static Public Member Functions

- static parameters_type build ()

  *Creates a new parameter_type to set up the factory.*
- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## 48.30.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::experimental::factorization::Cholesky< ValueType, IndexType >**

Computes a Cholesky factorization of a symmetric, positive-definite sparse matrix.

This LinOpFactory returns a Factorization storing the L and L^H factors for the provided system matrix in matrix::Csr format. If no symbolic factorization is provided, it will be computed first. It expects all fill-in entries to be present in the symbolic factorization. If the symbolic factorization is missing some entries, please refer to Ic.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type used to store values of the system matrix |
| *IndexType* | the type used to store sparsity pattern indices of the system matrix |

## 48.30.2 Member Function Documentation

### 48.30.2.1 generate()

```
template<typename ValueType , typename IndexType >
std::unique_ptr<factorization_type> gko::experimental::factorization::Cholesky< ValueType,
```

```
IndexType >::generate (
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix used to generate the factors. |

**Note**

This function overrides the default LinOpFactory::generate to return a Factorization instead of a generic LinOp, which would need to be cast to Factorization again to access its factors. It is only necessary because smart pointers aren't covariant.

### 48.30.2.2 get_parameters() [1/2]

```
template<typename ValueType , typename IndexType >
const parameters_type& gko::experimental::factorization::Cholesky< ValueType, IndexType >↩
::get_parameters ( )  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
77 { return parameters_; }
```

### 48.30.2.3 get_parameters() [2/2]

```
template<typename ValueType , typename IndexType >
const parameters_type& gko::experimental::factorization::Cholesky< ValueType, IndexType >↩
::get_parameters ( ) const  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
82 { return parameters_; }
```

### 48.30.2.4 parse()

```
template<typename ValueType , typename IndexType >
static parameters_type gko::experimental::factorization::Cholesky< ValueType, IndexType >↩
::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/cholesky.hpp

## 48.31 gko::Cloneable Class Reference

Interface for objects that can be cloned.

`#include <ginkgo/core/base/polymorphic_object.hpp>`

### 48.31.1 Detailed Description

Interface for objects that can be cloned.

**Note**

The derived types must also derive from PolymorphicObject.

The documentation for this class was generated from the following file:

- ginkgo/core/base/polymorphic_object.hpp

## 48.32 gko::experimental::mpi::CollectiveCommunicator Class Reference

Interface for a collective communicator.

`#include <ginkgo/core/distributed/collective_communicator.hpp>`

### Public Types

- using index_map_ptr = std::variant< const distributed::index_map< int32, int32 > *, const distributed::index_map< int32, int64 > *, const distributed::index_map< int64, int64 > * >

  *All allowed index_map types (as const *)*

## Public Member Functions

- template<typename SendType , typename RecvType >
  request i_all_to_all_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, RecvType *recv_buffer) const

  *Non-blocking all-to-all communication.*
- request i_all_to_all_v (std::shared_ptr< const Executor > exec, const void *send_buffer, MPI_Datatype send_type, void *recv_buffer, MPI_Datatype recv_type) const
- virtual std::unique_ptr< CollectiveCommunicator > create_with_same_type (communicator base, index_map_ptr imap) const =0

  *Creates a new CollectiveCommunicator with the same dynamic type.*
- virtual std::unique_ptr< CollectiveCommunicator > create_inverse () const =0

  *Creates a CollectiveCommunicator with the inverse communication pattern than this object.*
- virtual std::pair< std::unique_ptr< CollectiveCommunicator >, std::vector< comm_index_type > > resize (std::shared_ptr< const Executor > exec, const std::vector< comm_index_type > &send_factors) const =0

  *Create new communicator that sends and receives a different number of elements to/from each rank in its current pattern.*
- virtual comm_index_type get_recv_size () const =0

  *Get the number of elements received by this process within this communication pattern.*
- virtual comm_index_type get_send_size () const =0

  *Get the number of elements sent by this process within this communication pattern.*

### 48.32.1 Detailed Description

Interface for a collective communicator.

A collective communicator only provides routines for collective communications. At the moment this is restricted to the variable all-to-all.

### 48.32.2 Member Function Documentation

#### 48.32.2.1 create_inverse()

```
virtual std::unique_ptr<CollectiveCommunicator> gko::experimental::mpi::CollectiveCommunicator↩
::create_inverse ( ) const  [pure virtual]
```

Creates a CollectiveCommunicator with the inverse communication pattern than this object.

**Returns**

a CollectiveCommunicator with the inverse communication pattern.

Implemented in gko::experimental::mpi::DenseCommunicator, and gko::experimental::mpi::NeighborhoodCommunicator.

#### 48.32.2.2 create_with_same_type()

```
virtual std::unique_ptr<CollectiveCommunicator> gko::experimental::mpi::CollectiveCommunicator↩
::create_with_same_type (
            communicator base,
            index_map_ptr imap ) const  [pure virtual]
```

Creates a new CollectiveCommunicator with the same dynamic type.

**Parameters**

| | |
|---|---|
| *base* | The base communicator |
| *imap* | The index_map that defines the communication pattern |

**Returns**

a CollectiveCommunicator with the same dynamic type

Implemented in gko::experimental::mpi::NeighborhoodCommunicator, and gko::experimental::mpi::DenseCommunicator.

#### 48.32.2.3 get_recv_size()

```
virtual comm_index_type gko::experimental::mpi::CollectiveCommunicator::get_recv_size ( )
const  [pure virtual]
```

Get the number of elements received by this process within this communication pattern.

**Returns**

number of received elements.

Implemented in gko::experimental::mpi::NeighborhoodCommunicator, and gko::experimental::mpi::DenseCommunicator.

#### 48.32.2.4 get_send_size()

```
virtual comm_index_type gko::experimental::mpi::CollectiveCommunicator::get_send_size ( )
const  [pure virtual]
```

Get the number of elements sent by this process within this communication pattern.

**Returns**

number of sent elements.

Implemented in gko::experimental::mpi::NeighborhoodCommunicator, and gko::experimental::mpi::DenseCommunicator.

#### 48.32.2.5 i_all_to_all_v() [1/2]

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::CollectiveCommunicator::i_all_to_all_v (
            std::shared_ptr< const Executor > exec,
            const SendType * send_buffer,
            RecvType * recv_buffer ) const
```

Non-blocking all-to-all communication.

The send_buffer must have allocated at least get_send_size number of elements, and the recv_buffer must have allocated at least get_recv_size number of elements.

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the elements to send |
| *RecvType* | the type of the elements to receive |

**Parameters**

| | |
|---|---|
| *exec* | the executor for the communication |
| *send_buffer* | the send buffer |
| *recv_buffer* | the receive buffer |

**Returns**

a request handle

```
160 {
161     return this->i_all_to_all_v(std::move(exec), send_buffer,
162                                 type_impl<SendType>::get_type(), recv_buffer,
163                                 type_impl<RecvType>::get_type());
164 }
```

#### 48.32.2.6 i_all_to_all_v() [2/2]

```
request gko::experimental::mpi::CollectiveCommunicator::i_all_to_all_v (
            std::shared_ptr< const Executor > exec,
            const void * send_buffer,
            MPI_Datatype send_type,
            void * recv_buffer,
            MPI_Datatype recv_type ) const
```

#### 48.32.2.7 resize()

```
virtual std::pair<std::unique_ptr<CollectiveCommunicator>, std::vector<comm_index_type> >
gko::experimental::mpi::CollectiveCommunicator::resize (
            std::shared_ptr< const Executor > exec,
            const std::vector< comm_index_type > & send_factors ) const  [pure virtual]
```

Create new communicator that sends and receives a different number of elements to/from each rank in its current pattern.

The number of elements to send is deduces from the send_factors. If this communicator is used to send 2 elements to rank 0, and 3 elements to rank 1, then send_factors needs to have size 5, which is equal to get_send_size(). An send_factors = { 4, 2, 1, 3, 2} means that instead of the first element to send to rank 0, four elements are sent, instead of the second element to send to rank 0, two elements are sent, and so on. A visual representation of the transformation is:

```
[ 1 1 | 1 1 1 ] -> [ 4 2 | 1 3 2] -> [ 1 1 1 1 1 1 | 1 1 1 1 1 1 ]
```

The communication pattern will not change. Even if send_factors is zero for all elements of one rank, the rank will stay in the new pattern.

**Note**

This is a collective call.

**Parameters**

| | |
|---|---|
| *exec* | the executor to use for the new communicator |
| *send_factors* | the new number of elements to send. |

**Returns**

The new communicator and the recv_factors implied by the send_factors

Implemented in gko::experimental::mpi::NeighborhoodCommunicator, and gko::experimental::mpi::DenseCommunicator.

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/collective_communicator.hpp

# 48.33 gko::matrix::Hybrid< ValueType, IndexType >::column_limit Class Reference

column_limit is a strategy_type which decides the number of stored elements per row of the ell part by specifying the number of columns.

`#include <ginkgo/core/matrix/hybrid.hpp>`

## Public Member Functions

- column_limit (size_type num_column=0)
  *Creates a column_limit strategy.*
- size_type compute_ell_num_stored_elements_per_row (array< size_type > *row_nnz) const override
  *Computes the number of stored elements per row of the ell part.*
- auto get_num_columns () const
  *Get the number of columns limit.*

## 48.33.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Hybrid< ValueType, IndexType >::column_limit**

column_limit is a strategy_type which decides the number of stored elements per row of the ell part by specifying the number of columns.

## 48.33.2 Constructor & Destructor Documentation

### 48.33.2.1 column_limit()

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Hybrid< ValueType, IndexType >::column_limit::column_limit (
            size_type num_column = 0 )  [inline], [explicit]
```

Creates a column_limit strategy.

**Parameters**

| | |
|---|---|
| *num_column* | the specified number of columns of the ell part |

```
200             :  num_columns_(num_column)
201         {}
```

### 48.33.3 Member Function Documentation

#### 48.33.3.1 compute_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::column_limit::compute_ell_num_stored_↩
elements_per_row (
            array< size_type > * row_nnz ) const  [inline], [override], [virtual]
```

Computes the number of stored elements per row of the ell part.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |

**Returns**

the number of stored elements per row of the ell part

Implements gko::matrix::Hybrid< ValueType, IndexType >::strategy_type.

```
205         {
206             return num_columns_;
207         }
```

#### 48.33.3.2 get_num_columns()

```
template<typename ValueType = default_precision, typename IndexType = int32>
auto gko::matrix::Hybrid< ValueType, IndexType >::column_limit::get_num_columns ( ) const
[inline]
```

Get the number of columns limit.

**Returns**

the number of columns limit

```
214 { return num_columns_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/hybrid.hpp

# 48.34 gko::Combination< ValueType > Class Template Reference

The Combination class can be used to construct a linear combination of multiple linear operators `c1 * op1 + c2 * op2 + ...`

`#include <ginkgo/core/base/combination.hpp>`

## Public Member Functions

- const std::vector< std::shared_ptr< const LinOp > > & get_coefficients () const noexcept

  *Returns a list of coefficients of the combination.*
- const std::vector< std::shared_ptr< const LinOp > > & get_operators () const noexcept

  *Returns a list of operators of the combination.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- Combination & operator= (const Combination &)

  *Copy-assigns a Combination.*
- Combination & operator= (Combination &&)

  *Move-assigns a Combination.*
- Combination (const Combination &)

  *Copy-constructs a Combination.*
- Combination (Combination &&)

  *Move-constructs a Combination.*

### 48.34.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::Combination< ValueType >**

The Combination class can be used to construct a linear combination of multiple linear operators `c1 * op1 + c2 * op2 + ...`

- ck * opk.

Combination ensures that all LinOps passed to its constructor use the same executor, and if not, copies the operators to the executor of the first operator.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of input and result vectors |

### 48.34.2 Constructor & Destructor Documentation

#### 48.34.2.1 Combination() [1/2]

```
template<typename ValueType = default_precision>
gko::Combination< ValueType >::Combination (
            const Combination< ValueType > &  )
```

Copy-constructs a Combination.

This inherits the executor of the input Combination and all of its operators with shared ownership.

#### 48.34.2.2 Combination() [2/2]

```
template<typename ValueType = default_precision>
gko::Combination< ValueType >::Combination (
            Combination< ValueType > &&  )
```

Move-constructs a Combination.

This inherits the executor of the input Combination and all of its operators. The moved-from object is empty (0x0 LinOp without operators) afterwards.

### 48.34.3 Member Function Documentation

#### 48.34.3.1 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::Combination< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.34.3.2 get_coefficients()

```
template<typename ValueType = default_precision>
const std::vector<std::shared_ptr<const LinOp> >& gko::Combination< ValueType >::get_↩
coefficients ( ) const  [inline], [noexcept]
```

Returns a list of coefficients of the combination.

**Returns**

a list of coefficients

```
48     {
49         return coefficients_;
50     }
```

#### 48.34.3.3 get_operators()

```
template<typename ValueType = default_precision>
const std::vector<std::shared_ptr<const LinOp> >& gko::Combination< ValueType >::get_↩
operators ( ) const  [inline], [noexcept]
```

Returns a list of operators of the combination.

**Returns**

a list of operators

```
59     {
60         return operators_;
61     }
```

#### 48.34.3.4 operator=() [1/2]

```
template<typename ValueType = default_precision>
Combination& gko::Combination< ValueType >::operator= (
            Combination< ValueType > &&  )
```

Move-assigns a Combination.

The executor is not modified, and the wrapped LinOps are only being cloned if they are on a different executor, otherwise they share ownership. The moved-from object is empty (0x0 LinOp without operators) afterwards.

#### 48.34.3.5 operator=() [2/2]

```
template<typename ValueType = default_precision>
Combination& gko::Combination< ValueType >::operator= (
            const Combination< ValueType > &  )
```

Copy-assigns a Combination.

The executor is not modified, and the wrapped LinOps are only being cloned if they are on a different executor.

#### 48.34.3.6 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::Combination< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/base/combination.hpp

## 48.35 gko::stop::Combined Class Reference

The Combined class is used to combine multiple criterions together through an OR operation.

`#include <ginkgo/core/stop/combined.hpp>`

### Additional Inherited Members

### 48.35.1 Detailed Description

The Combined class is used to combine multiple criterions together through an OR operation.

The typical use case is to stop the iteration process if any of the criteria is fulfilled, e.g. a number of iterations, the relative residual norm has reached a threshold, etc.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/combined.hpp

## 48.36 gko::experimental::mpi::communicator Class Reference

A thin wrapper of MPI_Comm that supports most MPI calls.

`#include <ginkgo/core/base/mpi.hpp>`

### Public Member Functions

- communicator (const MPI_Comm &comm, bool force_host_buffer=false)

  *Non-owning constructor for an existing communicator of type MPI_Comm.*
- communicator (const MPI_Comm &comm, int color, int key)

  *Create a communicator object from an existing MPI_Comm object using color and key.*
- communicator (const communicator &comm, int color, int key)

  *Create a communicator object from an existing MPI_Comm object using color and key.*
- communicator (const communicator &other)=default

  *Create a copy of a communicator.*
- communicator (communicator &&other)

  *Move constructor.*
- communicator & operator= (const communicator &other)=default
- communicator & operator= (communicator &&other)
- const MPI_Comm & get () const

  *Return the underlying MPI_Comm object.*
- int size () const

  *Return the size of the communicator (number of ranks).*
- int rank () const

  *Return the rank of the calling process in the communicator.*
- int node_local_rank () const

  *Return the node local rank of the calling process in the communicator.*

- bool operator== (const communicator &rhs) const

  *Compare two communicator objects for equality.*
- bool operator!= (const communicator &rhs) const

  *Compare two communicator objects for non-equality.*
- bool is_identical (const communicator &rhs) const

  *Checks if the rhs communicator is identical to this communicator.*
- bool is_congruent (const communicator &rhs) const

  *Checks if the rhs communicator is congruent to this communicator.*
- void synchronize () const

  *This function is used to synchronize the ranks in the communicator.*
- template<typename SendType >
  void send (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, const int destination_rank, const int send_tag) const

  *Send (Blocking) data from calling process to destination rank.*
- template<typename SendType >
  request i_send (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, const int destination_rank, const int send_tag) const

  *Send (Non-blocking, Immediate return) data from calling process to destination rank.*
- template<typename RecvType >
  status recv (std::shared_ptr< const Executor > exec, RecvType *recv_buffer, const int recv_count, const int source_rank, const int recv_tag) const

  *Receive data from source rank.*
- template<typename RecvType >
  request i_recv (std::shared_ptr< const Executor > exec, RecvType *recv_buffer, const int recv_count, const int source_rank, const int recv_tag) const

  *Receive (Non-blocking, Immediate return) data from source rank.*
- template<typename BroadcastType >
  void broadcast (std::shared_ptr< const Executor > exec, BroadcastType *buffer, int count, int root_rank) const

  *Broadcast data from calling process to all ranks in the communicator.*
- template<typename BroadcastType >
  request i_broadcast (std::shared_ptr< const Executor > exec, BroadcastType *buffer, int count, int root_rank) const

  *(Non-blocking) Broadcast data from calling process to all ranks in the communicator*
- template<typename ReduceType >
  void reduce (std::shared_ptr< const Executor > exec, const ReduceType *send_buffer, ReduceType *recv_buffer, int count, MPI_Op operation, int root_rank) const

  *Reduce data into root from all calling processes on the same communicator.*
- template<typename ReduceType >
  request i_reduce (std::shared_ptr< const Executor > exec, const ReduceType *send_buffer, ReduceType *recv_buffer, int count, MPI_Op operation, int root_rank) const

  *(Non-blocking) Reduce data into root from all calling processes on the same communicator.*
- template<typename ReduceType >
  void all_reduce (std::shared_ptr< const Executor > exec, ReduceType *recv_buffer, int count, MPI_Op operation) const

  *(In-place) Reduce data from all calling processes from all calling processes on same communicator.*
- template<typename ReduceType >
  request i_all_reduce (std::shared_ptr< const Executor > exec, ReduceType *recv_buffer, int count, MPI_Op operation) const

  *(In-place, non-blocking) Reduce data from all calling processes from all calling processes on same communicator.*
- template<typename ReduceType >
  void all_reduce (std::shared_ptr< const Executor > exec, const ReduceType *send_buffer, ReduceType *recv_buffer, int count, MPI_Op operation) const

*Reduce data from all calling processes from all calling processes on same communicator.*

- template<typename ReduceType >
  request i_all_reduce (std::shared_ptr< const Executor > exec, const ReduceType *send_buffer, ReduceType *recv_buffer, int count, MPI_Op operation) const

  *Reduce data from all calling processes from all calling processes on same communicator.*

- template<typename SendType , typename RecvType >
  void gather (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count, int root_rank) const

  *Gather data onto the root rank from all ranks in the communicator.*

- template<typename SendType , typename RecvType >
  request i_gather (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count, int root_rank) const

  *(Non-blocking) Gather data onto the root rank from all ranks in the communicator.*

- template<typename SendType , typename RecvType >
  void gather_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int *recv_counts, const int *displacements, int root_rank) const

  *Gather data onto the root rank from all ranks in the communicator with offsets.*

- template<typename SendType , typename RecvType >
  request i_gather_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int *recv_counts, const int *displacements, int root_rank) const

  *(Non-blocking) Gather data onto the root rank from all ranks in the communicator with offsets.*

- template<typename SendType , typename RecvType >
  void all_gather (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count) const

  *Gather data onto all ranks from all ranks in the communicator.*

- template<typename SendType , typename RecvType >
  request i_all_gather (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count) const

  *(Non-blocking) Gather data onto all ranks from all ranks in the communicator.*

- template<typename SendType , typename RecvType >
  void scatter (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count, int root_rank) const

  *Scatter data from root rank to all ranks in the communicator.*

- template<typename SendType , typename RecvType >
  request i_scatter (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count, int root_rank) const

  *(Non-blocking) Scatter data from root rank to all ranks in the communicator.*

- template<typename SendType , typename RecvType >
  void scatter_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int *send_counts, const int *displacements, RecvType *recv_buffer, const int recv_count, int root_rank) const

  *Scatter data from root rank to all ranks in the communicator with offsets.*

- template<typename SendType , typename RecvType >
  request i_scatter_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int *send_counts, const int *displacements, RecvType *recv_buffer, const int recv_count, int root_rank) const

  *(Non-blocking) Scatter data from root rank to all ranks in the communicator with offsets.*

- template<typename RecvType >
  void all_to_all (std::shared_ptr< const Executor > exec, RecvType *recv_buffer, const int recv_count) const

  *(In-place) Communicate data from all ranks to all other ranks in place (MPI_Alltoall).*

- template<typename RecvType >
  request i_all_to_all (std::shared_ptr< const Executor > exec, RecvType *recv_buffer, const int recv_count) const

  *(In-place, Non-blocking) Communicate data from all ranks to all other ranks in place (MPI_Ialltoall).*

- template<typename SendType , typename RecvType >
  void all_to_all (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send_count, RecvType *recv_buffer, const int recv_count) const

*Communicate data from all ranks to all other ranks (MPI_Alltoall).*

- template<typename SendType , typename RecvType >
  request i_all_to_all (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int send↩
  _count, RecvType *recv_buffer, const int recv_count) const

  *(Non-blocking) Communicate data from all ranks to all other ranks (MPI_Ialltoall).*
- template<typename SendType , typename RecvType >
  void all_to_all_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int *send↩
  _counts, const int *send_offsets, RecvType *recv_buffer, const int *recv_counts, const int *recv_offsets) const

  *Communicate data from all ranks to all other ranks with offsets (MPI_Alltoallv).*
- void all_to_all_v (std::shared_ptr< const Executor > exec, const void *send_buffer, const int *send_↩
  counts, const int *send_offsets, MPI_Datatype send_type, void *recv_buffer, const int *recv_counts, const int *recv_offsets, MPI_Datatype recv_type) const

  *Communicate data from all ranks to all other ranks with offsets (MPI_Alltoallv).*
- request i_all_to_all_v (std::shared_ptr< const Executor > exec, const void *send_buffer, const int *send↩
  _counts, const int *send_offsets, MPI_Datatype send_type, void *recv_buffer, const int *recv_counts, const int *recv_offsets, MPI_Datatype recv_type) const

  *Communicate data from all ranks to all other ranks with offsets (MPI_Ialltoallv).*
- template<typename SendType , typename RecvType >
  request i_all_to_all_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, const int *send_counts, const int *send_offsets, RecvType *recv_buffer, const int *recv_counts, const int *recv_↩
  offsets) const

  *Communicate data from all ranks to all other ranks with offsets (MPI_Ialltoallv).*
- template<typename ScanType >
  void scan (std::shared_ptr< const Executor > exec, const ScanType *send_buffer, ScanType *recv_buffer, int count, MPI_Op operation) const

  *Does a scan operation with the given operator.*
- template<typename ScanType >
  request i_scan (std::shared_ptr< const Executor > exec, const ScanType *send_buffer, ScanType *recv_↩
  buffer, int count, MPI_Op operation) const

  *Does a scan operation with the given operator.*

## Static Public Member Functions

- static communicator create_owning (const MPI_Comm &comm, bool force_host_buffer=false)

  *Creates a new communicator and takes ownership of the MPI_Comm.*

### 48.36.1 Detailed Description

A thin wrapper of MPI_Comm that supports most MPI calls.

A wrapper class that takes in the given MPI communicator. If a bare MPI_Comm is provided, the wrapper takes no ownership of the MPI_Comm. Thus the MPI_Comm must remain valid throughout the lifetime of the communicator. If the communicator was created through splitting, the wrapper takes ownership of the MPI_Comm. In this case, as the class or object goes out of scope, the underlying MPI_Comm is freed.

**Note**

All MPI calls that work on a buffer take in an Executor as an additional argument. This argument specifies the memory space the buffer lives in.

## 48.36.2 Constructor & Destructor Documentation

### 48.36.2.1 communicator() [1/5]

```
gko::experimental::mpi::communicator::communicator (
            const MPI_Comm & comm,
            bool force_host_buffer = false )  [inline]
```

Non-owning constructor for an existing communicator of type MPI_Comm.

The MPI_Comm object will not be deleted after the communicator object has been freed and an explicit MPI_↩ Comm_free needs to be called on the original MPI_Comm object.

**Parameters**

| | |
|---|---|
| *comm* | The input MPI_Comm object. |
| *force_host_buffer* | If set to true, always communicates through host memory |

```
431         :  comm_(), force_host_buffer_(force_host_buffer)
432     {
433         this->comm_.reset(new MPI_Comm(comm));
434     }
```

### 48.36.2.2 communicator() [2/5]

```
gko::experimental::mpi::communicator::communicator (
            const MPI_Comm & comm,
            int color,
            int key )  [inline]
```

Create a communicator object from an existing MPI_Comm object using color and key.

**Parameters**

| | |
|---|---|
| *comm* | The input MPI_Comm object. |
| *color* | The color to split the original comm object |
| *key* | The key to split the comm object |

```
445         :  force_host_buffer_(false)
446     {
447         MPI_Comm comm_out;
448         GKO_ASSERT_NO_MPI_ERRORS(MPI_Comm_split(comm, color, key, &comm_out));
449         this->comm_.reset(new MPI_Comm(comm_out), detail::comm_deleter{});
450     }
```

### 48.36.2.3 communicator() [3/5]

```
gko::experimental::mpi::communicator::communicator (
            const communicator & comm,
```

```
            int color,
            int key )  [inline]
```

Create a communicator object from an existing MPI_Comm object using color and key.

**Parameters**

| | |
|---|---|
| *comm* | The input communicator object. |
| *color* | The color to split the original comm object |
| *key* | The key to split the comm object |

```
461         :  force_host_buffer_(comm.force_host_buffer())
462     {
463         MPI_Comm comm_out;
464         GKO_ASSERT_NO_MPI_ERRORS(
465             MPI_Comm_split(comm.get(), color, key, &comm_out));
466         this->comm_.reset(new MPI_Comm(comm_out), detail::comm_deleter{});
467     }
```

References get().

#### 48.36.2.4 communicator() [4/5]

```
gko::experimental::mpi::communicator::communicator (
            const communicator & other )  [default]
```

Create a copy of a communicator.

Potential ownership of the underlying MPI_Comm will be shared.

#### 48.36.2.5 communicator() [5/5]

```
gko::experimental::mpi::communicator::communicator (
            communicator && other )  [inline]
```

Move constructor.

The other communicator will relinquish any potential ownership and use MPI_COMM_NULL as underlying MPI_↩ Comm after the move operation.

```
499 { *this = std::move(other); }
```

### 48.36.3 Member Function Documentation

#### 48.36.3.1 all_gather()

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::all_gather (
            std::shared_ptr< const Executor > exec,
            const SendType * send_buffer,
            const int send_count,
            RecvType * recv_buffer,
            const int recv_count ) const  [inline]
```

Gather data onto all ranks from all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
1081     {
1082         auto guard = exec->get_scoped_device_id_guard();
1083         GKO_ASSERT_NO_MPI_ERRORS(MPI_Allgather(
1084             send_buffer, send_count, type_impl<SendType>::get_type(),
1085             recv_buffer, recv_count, type_impl<RecvType>::get_type(),
1086             this->get()));
1087     }
```

References get().

### 48.36.3.2 all_reduce() [1/2]

```
template<typename ReduceType >
void gko::experimental::mpi::communicator::all_reduce (
              std::shared_ptr< const Executor > exec,
              const ReduceType * send_buffer,
              ReduceType * recv_buffer,
              int count,
              MPI_Op operation ) const  [inline]
```

Reduce data from all calling processes from all calling processes on same communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the data to reduce |
| *recv_buffer* | the reduced result |
| *count* | the number of elements to reduce |
| *operation* | the reduce operation. See @MPI_Op |

**Template Parameters**

| | |
|---|---|
| *ReduceType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

```
896     {
897         auto guard = exec->get_scoped_device_id_guard();
898         GKO_ASSERT_NO_MPI_ERRORS(MPI_Allreduce(
899             send_buffer, recv_buffer, count, type_impl<ReduceType>::get_type(),
```

```
900             operation, this->get()));
901     }
```

References get().

#### 48.36.3.3 all_reduce() [2/2]

```
template<typename ReduceType >
void gko::experimental::mpi::communicator::all_reduce (
             std::shared_ptr< const Executor > exec,
             ReduceType * recv_buffer,
             int count,
             MPI_Op operation ) const  [inline]
```

(In-place) Reduce data from all calling processes from all calling processes on same communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *recv_buffer* | the data to reduce and the reduced result |
| *count* | the number of elements to reduce |
| *operation* | the MPI_Op type reduce operation. |

**Template Parameters**

| | |
|---|---|
| *ReduceType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

```
843     {
844         auto guard = exec->get_scoped_device_id_guard();
845         GKO_ASSERT_NO_MPI_ERRORS(MPI_Allreduce(
846             MPI_IN_PLACE, recv_buffer, count, type_impl<ReduceType>::get_type(),
847             operation, this->get()));
848     }
```

References get().

#### 48.36.3.4 all_to_all() [1/2]

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::all_to_all (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             RecvType * recv_buffer,
             const int recv_count ) const  [inline]
```

Communicate data from all ranks to all other ranks (MPI_Alltoall).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to send |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to receive |
| *recv_count* | the number of elements to receive |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
1329     {
1330         auto guard = exec->get_scoped_device_id_guard();
1331         GKO_ASSERT_NO_MPI_ERRORS(MPI_Alltoall(
1332             send_buffer, send_count, type_impl<SendType>::get_type(),
1333             recv_buffer, recv_count, type_impl<RecvType>::get_type(),
1334             this->get()));
1335     }
```

References get().

#### 48.36.3.5 all_to_all() [2/2]

```
template<typename RecvType >
void gko::experimental::mpi::communicator::all_to_all (
              std::shared_ptr< const Executor > exec,
              RecvType * recv_buffer,
              const int recv_count ) const  [inline]
```

(In-place) Communicate data from all ranks to all other ranks in place (MPI_Alltoall).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *recv_buffer* | the buffer to send and the buffer receive |
| *recv_count* | the number of elements to receive |
| *comm* | the communicator |

**Template Parameters**

| | |
|---|---|
| *RecvType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Note**

This overload uses MPI_IN_PLACE and the source and destination buffers are the same.

```
1270     {
1271         auto guard = exec->get_scoped_device_id_guard();
1272         GKO_ASSERT_NO_MPI_ERRORS(MPI_Alltoall(
1273             MPI_IN_PLACE, recv_count, type_impl<RecvType>::get_type(),
1274             recv_buffer, recv_count, type_impl<RecvType>::get_type(),
1275             this->get()));
1276     }
```

References get().

#### 48.36.3.6 all_to_all_v() [1/2]

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::all_to_all_v (
              std::shared_ptr< const Executor > exec,
              const SendType * send_buffer,
              const int * send_counts,
              const int * send_offsets,
              RecvType * recv_buffer,
              const int * recv_counts,
              const int * recv_offsets ) const  [inline]
```

Communicate data from all ranks to all other ranks with offsets (MPI_Alltoallv).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to send |
| *send_counts* | the number of elements to send |
| *send_offsets* | the offsets for the send buffer |
| *recv_buffer* | the buffer to gather into |
| *recv_counts* | the number of elements to receive |
| *recv_offsets* | the offsets for the recv buffer |
| *comm* | the communicator |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
1393     {
1394         this->all_to_all_v(std::move(exec), send_buffer, send_counts,
1395                            send_offsets, type_impl<SendType>::get_type(),
1396                            recv_buffer, recv_counts, recv_offsets,
1397                            type_impl<RecvType>::get_type());
1398     }
```

#### 48.36.3.7 all_to_all_v() [2/2]

```
void gko::experimental::mpi::communicator::all_to_all_v (
              std::shared_ptr< const Executor > exec,
```

```
            const void * send_buffer,
            const int * send_counts,
            const int * send_offsets,
            MPI_Datatype send_type,
            void * recv_buffer,
            const int * recv_counts,
            const int * recv_offsets,
            MPI_Datatype recv_type ) const  [inline]
```

Communicate data from all ranks to all other ranks with offsets (MPI_Alltoallv).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to send |
| *send_counts* | the number of elements to send |
| *send_offsets* | the offsets for the send buffer |
| *send_type* | the MPI_Datatype for the send buffer |
| *recv_buffer* | the buffer to gather into |
| *recv_counts* | the number of elements to receive |
| *recv_offsets* | the offsets for the recv buffer |
| *recv_type* | the MPI_Datatype for the recv buffer |
| *comm* | the communicator |

```
1420     {
1421         auto guard = exec->get_scoped_device_id_guard();
1422         GKO_ASSERT_NO_MPI_ERRORS(MPI_Alltoallv(
1423             send_buffer, send_counts, send_offsets, send_type, recv_buffer,
1424             recv_counts, recv_offsets, recv_type, this->get()));
1425     }
```

References get().

#### 48.36.3.8 broadcast()

```
template<typename BroadcastType >
void gko::experimental::mpi::communicator::broadcast (
            std::shared_ptr< const Executor > exec,
            BroadcastType * buffer,
            int count,
            int root_rank ) const  [inline]
```

Broadcast data from calling process to all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *buffer* | the buffer to broadcast |
| *count* | the number of elements to broadcast |
| *root_rank* | the rank to broadcast from |

**Template Parameters**

| | |
|---|---|
| *BroadcastType* | the type of the data to broadcast. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

```
737     {
738         auto guard = exec->get_scoped_device_id_guard();
739         GKO_ASSERT_NO_MPI_ERRORS(MPI_Bcast(buffer, count,
740                                            type_impl<BroadcastType>::get_type(),
741                                            root_rank, this->get()));
742     }
```

References get().

### 48.36.3.9 create_owning()

```
static communicator gko::experimental::mpi::communicator::create_owning (
              const MPI_Comm & comm,
              bool force_host_buffer = false )  [inline], [static]
```

Creates a new communicator and takes ownership of the MPI_Comm.

The ownership is shared with all mpi::communicator objects that are a copy from the newly created communicator. The underlying MPI_Comm will be freed when the last communicator with ownership is destroyed.

**See also**

communicator(const MPI_Comm&, bool)

```
480     {
481         communicator comm_out(MPI_COMM_NULL, force_host_buffer);
482         comm_out.comm_.reset(new MPI_Comm(comm), detail::comm_deleter{});
483         return comm_out;
484     }
```

### 48.36.3.10 gather()

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::gather (
              std::shared_ptr< const Executor > exec,
              const SendType * send_buffer,
              const int send_count,
              RecvType * recv_buffer,
              const int recv_count,
              int root_rank ) const  [inline]
```

Gather data onto the root rank from all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |
| *root_rank* | the rank to gather into |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
953     {
954         auto guard = exec->get_scoped_device_id_guard();
955         GKO_ASSERT_NO_MPI_ERRORS(
956             MPI_Gather(send_buffer, send_count, type_impl<SendType>::get_type(),
957                        recv_buffer, recv_count, type_impl<RecvType>::get_type(),
958                        root_rank, this->get()));
959     }
```

References get().

### 48.36.3.11 gather_v()

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::gather_v (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             RecvType * recv_buffer,
             const int * recv_counts,
             const int * displacements,
             int root_rank ) const  [inline]
```

Gather data onto the root rank from all ranks in the communicator with offsets.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_counts* | the number of elements to receive |
| *displacements* | the offsets for the buffer |
| *root_rank* | the rank to gather into |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
1018     {
1019         auto guard = exec->get_scoped_device_id_guard();
1020         GKO_ASSERT_NO_MPI_ERRORS(MPI_Gatherv(
1021             send_buffer, send_count, type_impl<SendType>::get_type(),
1022             recv_buffer, recv_counts, displacements,
1023             type_impl<RecvType>::get_type(), root_rank, this->get()));
1024     }
```

References get().

#### 48.36.3.12 get()

```
const MPI_Comm& gko::experimental::mpi::communicator::get ( ) const  [inline]
```

Return the underlying MPI_Comm object.

**Returns**

the MPI_Comm object

```
524 { return *(this->comm_.get()); }
```

Referenced by all_gather(), all_reduce(), all_to_all(), all_to_all_v(), broadcast(), communicator(), gather(), gather_v(), i_all_gather(), i_all_reduce(), i_all_to_all(), i_all_to_all_v(), i_broadcast(), i_gather(), i_gather_v(), i_recv(), i_reduce(), i_scan(), i_scatter(), i_scatter_v(), i_send(), is_congruent(), is_identical(), recv(), reduce(), scan(), scatter(), scatter_v(), send(), synchronize(), and gko::experimental::mpi::window< ValueType >::window().

#### 48.36.3.13 i_all_gather()

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_all_gather (
            std::shared_ptr< const Executor > exec,
            const SendType * send_buffer,
            const int send_count,
            RecvType * recv_buffer,
            const int recv_count ) const  [inline]
```

(Non-blocking) Gather data onto all ranks from all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
1111     {
1112         auto guard = exec->get_scoped_device_id_guard();
1113         request req;
1114         GKO_ASSERT_NO_MPI_ERRORS(MPI_Iallgather(
1115             send_buffer, send_count, type_impl<SendType>::get_type(),
```

```
1116             recv_buffer, recv_count, type_impl<RecvType>::get_type(),
1117             this->get(), req.get()));
1118         return req;
1119     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.14 i_all_reduce() [1/2]

```
template<typename ReduceType >
request gko::experimental::mpi::communicator::i_all_reduce (
             std::shared_ptr< const Executor > exec,
             const ReduceType * send_buffer,
             ReduceType * recv_buffer,
             int count,
             MPI_Op operation ) const  [inline]
```

Reduce data from all calling processes from all calling processes on same communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the data to reduce |
| *recv_buffer* | the reduced result |
| *count* | the number of elements to reduce |
| *operation* | the reduce operation. See @MPI_Op |

**Template Parameters**

| | |
|---|---|
| *ReduceType* | the type of the data to reduce. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the call

```
923     {
924         auto guard = exec->get_scoped_device_id_guard();
925         request req;
926         GKO_ASSERT_NO_MPI_ERRORS(MPI_Iallreduce(
927             send_buffer, recv_buffer, count, type_impl<ReduceType>::get_type(),
928             operation, this->get(), req.get()));
929         return req;
930     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.15 i_all_reduce() [2/2]

```
template<typename ReduceType >
request gko::experimental::mpi::communicator::i_all_reduce (
```

```
            std::shared_ptr< const Executor > exec,
            ReduceType * recv_buffer,
            int count,
            MPI_Op operation ) const  [inline]
```

(In-place, non-blocking) Reduce data from all calling processes from all calling processes on same communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *recv_buffer* | the data to reduce and the reduced result |
| *count* | the number of elements to reduce |
| *operation* | the reduce operation. See @MPI_Op |

**Template Parameters**

| | |
|---|---|
| *ReduceType* | the type of the data to reduce. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the call

```
869     {
870         auto guard = exec->get_scoped_device_id_guard();
871         request req;
872         GKO_ASSERT_NO_MPI_ERRORS(MPI_Iallreduce(
873             MPI_IN_PLACE, recv_buffer, count, type_impl<ReduceType>::get_type(),
874             operation, this->get(), req.get()));
875         return req;
876     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.16 i_all_to_all() [1/2]

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_all_to_all (
            std::shared_ptr< const Executor > exec,
            const SendType * send_buffer,
            const int send_count,
            RecvType * recv_buffer,
            const int recv_count ) const  [inline]
```

(Non-blocking) Communicate data from all ranks to all other ranks (MPI_Ialltoall).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to send |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to receive |
| *recv_count* | the number of elements to receive |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
1359     {
1360         auto guard = exec->get_scoped_device_id_guard();
1361         request req;
1362         GKO_ASSERT_NO_MPI_ERRORS(MPI_Ialltoall(
1363             send_buffer, send_count, type_impl<SendType>::get_type(),
1364             recv_buffer, recv_count, type_impl<RecvType>::get_type(),
1365             this->get(), req.get()));
1366         return req;
1367     }
```

References gko::experimental::mpi::request::get(), and get().

### 48.36.3.17 i_all_to_all() [2/2]

```
template<typename RecvType >
request gko::experimental::mpi::communicator::i_all_to_all (
             std::shared_ptr< const Executor > exec,
             RecvType * recv_buffer,
             const int recv_count ) const  [inline]
```

(In-place, Non-blocking) Communicate data from all ranks to all other ranks in place (MPI_Ialltoall).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *recv_buffer* | the buffer to send and the buffer receive |
| *recv_count* | the number of elements to receive |
| *comm* | the communicator |

**Template Parameters**

| | |
|---|---|
| *RecvType* | the type of the data to receive. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the call

**Note**

This overload uses MPI_IN_PLACE and the source and destination buffers are the same.

```
1299     {
1300         auto guard = exec->get_scoped_device_id_guard();
1301         request req;
1302         GKO_ASSERT_NO_MPI_ERRORS(MPI_Ialltoall(
1303             MPI_IN_PLACE, recv_count, type_impl<RecvType>::get_type(),
1304             recv_buffer, recv_count, type_impl<RecvType>::get_type(),
1305             this->get(), req.get()));
1306         return req;
1307     }
```

References gko::experimental::mpi::request::get(), and get().

### 48.36.3.18 i_all_to_all_v() [1/2]

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_all_to_all_v (
              std::shared_ptr< const Executor > exec,
              const SendType * send_buffer,
              const int * send_counts,
              const int * send_offsets,
              RecvType * recv_buffer,
              const int * recv_counts,
              const int * recv_offsets ) const  [inline]
```

Communicate data from all ranks to all other ranks with offsets (MPI_Ialltoallv).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to send |
| *send_counts* | the number of elements to send |
| *send_offsets* | the offsets for the send buffer |
| *recv_buffer* | the buffer to gather into |
| *recv_counts* | the number of elements to receive |
| *recv_offsets* | the offsets for the recv buffer |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
1487     {
1488         return this->i_all_to_all_v(
1489             std::move(exec), send_buffer, send_counts, send_offsets,
1490             type_impl<SendType>::get_type(), recv_buffer, recv_counts,
```

```
1491             recv_offsets, type_impl<RecvType>::get_type());
1492     }
```

References i_all_to_all_v().

#### 48.36.3.19 i_all_to_all_v() [2/2]

```
request gko::experimental::mpi::communicator::i_all_to_all_v (
             std::shared_ptr< const Executor > exec,
             const void * send_buffer,
             const int * send_counts,
             const int * send_offsets,
             MPI_Datatype send_type,
             void * recv_buffer,
             const int * recv_counts,
             const int * recv_offsets,
             MPI_Datatype recv_type ) const  [inline]
```

Communicate data from all ranks to all other ranks with offsets (MPI_Ialltoallv).

See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to send |
| *send_counts* | the number of elements to send |
| *send_offsets* | the offsets for the send buffer |
| *send_type* | the MPI_Datatype for the send buffer |
| *recv_buffer* | the buffer to gather into |
| *recv_counts* | the number of elements to receive |
| *recv_offsets* | the offsets for the recv buffer |
| *recv_type* | the MPI_Datatype for the recv buffer |

**Returns**

the request handle for the call

**Note**

This overload allows specifying the MPI_Datatype for both the send and received data.

```
1452     {
1453         auto guard = exec->get_scoped_device_id_guard();
1454         request req;
1455         GKO_ASSERT_NO_MPI_ERRORS(MPI_Ialltoallv(
1456             send_buffer, send_counts, send_offsets, send_type, recv_buffer,
1457             recv_counts, recv_offsets, recv_type, this->get(), req.get()));
1458         return req;
1459     }
```

References gko::experimental::mpi::request::get(), and get().

Referenced by i_all_to_all_v().

#### 48.36.3.20 i_broadcast()

```
template<typename BroadcastType >
request gko::experimental::mpi::communicator::i_broadcast (
             std::shared_ptr< const Executor > exec,
             BroadcastType * buffer,
             int count,
             int root_rank ) const  [inline]
```

(Non-blocking) Broadcast data from calling process to all ranks in the communicator

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *buffer* | the buffer to broadcast |
| *count* | the number of elements to broadcast |
| *root_rank* | the rank to broadcast from |

**Template Parameters**

| | |
|---|---|
| *BroadcastType* | the type of the data to broadcast. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the call

```
762     {
763         auto guard = exec->get_scoped_device_id_guard();
764         request req;
765         GKO_ASSERT_NO_MPI_ERRORS(
766             MPI_Ibcast(buffer, count, type_impl<BroadcastType>::get_type(),
767                        root_rank, this->get(), req.get()));
768         return req;
769     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.21 i_gather()

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_gather (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             RecvType * recv_buffer,
             const int recv_count,
             int root_rank ) const  [inline]
```

(Non-blocking) Gather data onto the root rank from all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |

**Parameters**

| | |
|---|---|
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |
| *root_rank* | the rank to gather into |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
985     {
986         auto guard = exec->get_scoped_device_id_guard();
987         request req;
988         GKO_ASSERT_NO_MPI_ERRORS(MPI_Igather(
989             send_buffer, send_count, type_impl<SendType>::get_type(),
990             recv_buffer, recv_count, type_impl<RecvType>::get_type(), root_rank,
991             this->get(), req.get()));
992         return req;
993     }
```

References gko::experimental::mpi::request::get(), and get().

### 48.36.3.22 i_gather_v()

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_gather_v (
              std::shared_ptr< const Executor > exec,
              const SendType * send_buffer,
              const int send_count,
              RecvType * recv_buffer,
              const int * recv_counts,
              const int * displacements,
              int root_rank ) const  [inline]
```

(Non-blocking) Gather data onto the root rank from all ranks in the communicator with offsets.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_counts* | the number of elements to receive |
| *displacements* | the offsets for the buffer |
| *root_rank* | the rank to gather into |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
1051     {
1052         auto guard = exec->get_scoped_device_id_guard();
1053         request req;
1054         GKO_ASSERT_NO_MPI_ERRORS(MPI_Igatherv(
1055             send_buffer, send_count, type_impl<SendType>::get_type(),
1056             recv_buffer, recv_counts, displacements,
1057             type_impl<RecvType>::get_type(), root_rank, this->get(),
1058             req.get()));
1059         return req;
1060     }
```

References gko::experimental::mpi::request::get(), and get().

### 48.36.3.23 i_recv()

```
template<typename RecvType >
request gko::experimental::mpi::communicator::i_recv (
             std::shared_ptr< const Executor > exec,
             RecvType * recv_buffer,
             const int recv_count,
             const int source_rank,
             const int recv_tag ) const  [inline]
```

Receive (Non-blocking, Immediate return) data from source rank.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *recv_buffer* | the buffer to send |
| *recv_count* | the number of elements to receive |
| *source_rank* | the rank to receive the data from |
| *recv_tag* | the tag for the recv call |

**Template Parameters**

| | |
|---|---|
| *RecvType* | the type of the data to receive. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the recv call

```
713     {
714         auto guard = exec->get_scoped_device_id_guard();
```

```
715         request req;
716         GKO_ASSERT_NO_MPI_ERRORS(
717             MPI_Irecv(recv_buffer, recv_count, type_impl<RecvType>::get_type(),
718                       source_rank, recv_tag, this->get(), req.get()));
719         return req;
720     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.24 i_reduce()

```
template<typename ReduceType >
request gko::experimental::mpi::communicator::i_reduce (
             std::shared_ptr< const Executor > exec,
             const ReduceType * send_buffer,
             ReduceType * recv_buffer,
             int count,
             MPI_Op operation,
             int root_rank ) const  [inline]
```

(Non-blocking) Reduce data into root from all calling processes on the same communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *send_buffer* | the buffer to reduce |
| *recv_buffer* | the reduced result |
| *count* | the number of elements to reduce |
| *operation* | the MPI_Op type reduce operation. |
| *root_rank* | the rank to reduce into |

**Template Parameters**

| | |
|---|---|
| *ReduceType* | the type of the data to reduce. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the call

```
818     {
819         auto guard = exec->get_scoped_device_id_guard();
820         request req;
821         GKO_ASSERT_NO_MPI_ERRORS(MPI_Ireduce(
822             send_buffer, recv_buffer, count, type_impl<ReduceType>::get_type(),
823             operation, root_rank, this->get(), req.get()));
824         return req;
825     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.25 i_scan()

```
template<typename ScanType >
request gko::experimental::mpi::communicator::i_scan (
             std::shared_ptr< const Executor > exec,
             const ScanType * send_buffer,
             ScanType * recv_buffer,
             int count,
             MPI_Op operation ) const  [inline]
```

Does a scan operation with the given operator.

(MPI_Iscan). See MPI documentation for more details.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to scan from |
| *recv_buffer* | the result buffer |
| *count* | the number of elements to scan |
| *operation* | the operation type to be used for the scan. See @MPI_Op |

**Template Parameters**

| | |
|---|---|
| *ScanType* | the type of the data to scan. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the call

```
1538     {
1539         auto guard = exec->get_scoped_device_id_guard();
1540         request req;
1541         GKO_ASSERT_NO_MPI_ERRORS(MPI_Iscan(send_buffer, recv_buffer, count,
1542                                            type_impl<ScanType>::get_type(),
1543                                            operation, this->get(), req.get()));
1544         return req;
1545     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.26 i_scatter()

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_scatter (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             RecvType * recv_buffer,
             const int recv_count,
             int root_rank ) const  [inline]
```

(Non-blocking) Scatter data from root rank to all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |
| *root_rank* | the rank to scatter from |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
1174     {
1175         auto guard = exec->get_scoped_device_id_guard();
1176         request req;
1177         GKO_ASSERT_NO_MPI_ERRORS(MPI_Iscatter(
1178             send_buffer, send_count, type_impl<SendType>::get_type(),
1179             recv_buffer, recv_count, type_impl<RecvType>::get_type(), root_rank,
1180             this->get(), req.get()));
1181         return req;
1182     }
```

References gko::experimental::mpi::request::get(), and get().

### 48.36.3.27 i_scatter_v()

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::communicator::i_scatter_v (
              std::shared_ptr< const Executor > exec,
              const SendType * send_buffer,
              const int * send_counts,
              const int * displacements,
              RecvType * recv_buffer,
              const int recv_count,
              int root_rank ) const  [inline]
```

(Non-blocking) Scatter data from root rank to all ranks in the communicator with offsets.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_counts* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |
| *displacements* | the offsets for the buffer |
| *root_rank* | the rank to scatter from |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

**Returns**

the request handle for the call

```
1240     {
1241         auto guard = exec->get_scoped_device_id_guard();
1242         request req;
1243         GKO_ASSERT_NO_MPI_ERRORS(
1244             MPI_Iscatterv(send_buffer, send_counts, displacements,
1245                           type_impl<SendType>::get_type(), recv_buffer,
1246                           recv_count, type_impl<RecvType>::get_type(),
1247                           root_rank, this->get(), req.get()));
1248         return req;
1249     }
```

References gko::experimental::mpi::request::get(), and get().

### 48.36.3.28 i_send()

```
template<typename SendType >
request gko::experimental::mpi::communicator::i_send (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             const int destination_rank,
             const int send_tag ) const  [inline]
```

Send (Non-blocking, Immediate return) data from calling process to destination rank.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *send_buffer* | the buffer to send |
| *send_count* | the number of elements to send |
| *destination_rank* | the rank to send the data to |
| *send_tag* | the tag for the send call |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the request handle for the send call

```
657     {
658         auto guard = exec->get_scoped_device_id_guard();
```

```
659         request req;
660         GKO_ASSERT_NO_MPI_ERRORS(
661             MPI_Isend(send_buffer, send_count, type_impl<SendType>::get_type(),
662                       destination_rank, send_tag, this->get(), req.get()));
663         return req;
664     }
```

References gko::experimental::mpi::request::get(), and get().

#### 48.36.3.29 is_congruent()

```
bool gko::experimental::mpi::communicator::is_congruent (
            const communicator & rhs ) const  [inline]
```

Checks if the rhs communicator is congruent to this communicator.

Congruent communicators are defined as having identical rank members and rank ordering.

**Note**

If MPI_COMM_NULL is the underlying MPI_COMM of this, then the communicator is congruent to rhs, iff MPI_COMM_NULL is also the underlying MPI_COMM of rhs.

**Returns**

true if rhs is congruent to this

```
595     {
596         if (get() == MPI_COMM_NULL || rhs.get() == MPI_COMM_NULL) {
597             return get() == rhs.get();
598         }
599         int flag;
600         GKO_ASSERT_NO_MPI_ERRORS(MPI_Comm_compare(get(), rhs.get(), &flag));
601         return flag == MPI_CONGRUENT;
602     }
```

References get().

#### 48.36.3.30 is_identical()

```
bool gko::experimental::mpi::communicator::is_identical (
            const communicator & rhs ) const  [inline]
```

Checks if the rhs communicator is identical to this communicator.

**Note**

If MPI_COMM_NULL is the underlying MPI_COMM of this, then the communicator is equal to rhs, iff MPI_↩ COMM_NULL is also the underlying MPI_COMM of rhs.

**Returns**

true if rhs is identical to this

```
573     {
574         if (get() == MPI_COMM_NULL || rhs.get() == MPI_COMM_NULL) {
575             return get() == rhs.get();
576         }
577         int flag;
578         GKO_ASSERT_NO_MPI_ERRORS(MPI_Comm_compare(get(), rhs.get(), &flag));
579         return flag == MPI_IDENT;
580     }
```

References get().

Referenced by operator==().

#### 48.36.3.31 node_local_rank()

```
int gko::experimental::mpi::communicator::node_local_rank ( ) const  [inline]
```

Return the node local rank of the calling process in the communicator.

**Returns**

the node local rank

```
547 { return get_node_local_rank(); };
```

#### 48.36.3.32 operator"!=()

```
bool gko::experimental::mpi::communicator::operator!= (
            const communicator & rhs ) const  [inline]
```

Compare two communicator objects for non-equality.

**Returns**

if the two comm objects are not equal

```
561 { return !(*this == rhs); }
```

#### 48.36.3.33 operator=() [1/2]

```
communicator& gko::experimental::mpi::communicator::operator= (
            communicator && other )  [inline]
```

**See also**

communicator(communicator&&)

```
510     {
511         if (this != &other) {
512             comm_ = std::exchange(other.comm_,
513                                   std::make_shared<MPI_Comm>(MPI_COMM_NULL));
514             force_host_buffer_ = other.force_host_buffer_;
515         }
516         return *this;
517     }
```

#### 48.36.3.34 operator=() [2/2]

```
communicator& gko::experimental::mpi::communicator::operator= (
            const communicator & other )  [default]
```

**See also**

communicator(const communicator&)

#### 48.36.3.35 operator==()

```
bool gko::experimental::mpi::communicator::operator== (
            const communicator & rhs ) const  [inline]
```

Compare two communicator objects for equality.

**Returns**

if the two comm objects are equal

```
554 { return is_identical(rhs); }
```

References is_identical().

#### 48.36.3.36 rank()

```
int gko::experimental::mpi::communicator::rank ( ) const  [inline]
```

Return the rank of the calling process in the communicator.

**Returns**

the rank

```
540 { return get_my_rank(); };
```

#### 48.36.3.37 recv()

```
template<typename RecvType >
status gko::experimental::mpi::communicator::recv (
            std::shared_ptr< const Executor > exec,
            RecvType * recv_buffer,
            const int recv_count,
            const int source_rank,
            const int recv_tag ) const  [inline]
```

Receive data from source rank.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *recv_buffer* | the buffer to receive |
| *recv_count* | the number of elements to receive |
| *source_rank* | the rank to receive the data from |
| *recv_tag* | the tag for the recv call |

**Template Parameters**

| | |
|---|---|
| *RecvType* | the type of the data to receive. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

**Returns**

the status of completion of this call

```
685     {
686         auto guard = exec->get_scoped_device_id_guard();
687         status st;
688         GKO_ASSERT_NO_MPI_ERRORS(
689             MPI_Recv(recv_buffer, recv_count, type_impl<RecvType>::get_type(),
690                      source_rank, recv_tag, this->get(), st.get()));
691         return st;
692     }
```

References gko::experimental::mpi::status::get(), and get().

### 48.36.3.38 reduce()

```
template<typename ReduceType >
void gko::experimental::mpi::communicator::reduce (
             std::shared_ptr< const Executor > exec,
             const ReduceType * send_buffer,
             ReduceType * recv_buffer,
             int count,
             MPI_Op operation,
             int root_rank ) const  [inline]
```

Reduce data into root from all calling processes on the same communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *send_buffer* | the buffer to reduce |
| *recv_buffer* | the reduced result |
| *count* | the number of elements to reduce |
| *operation* | the MPI_Op type reduce operation. |
| *root_rank* | the rank to reduce into |

**Template Parameters**

| | |
|---|---|
| *ReduceType* | the type of the data to reduce. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

```
790     {
791         auto guard = exec->get_scoped_device_id_guard();
792         GKO_ASSERT_NO_MPI_ERRORS(MPI_Reduce(send_buffer, recv_buffer, count,
793                                             type_impl<ReduceType>::get_type(),
794                                             operation, root_rank, this->get()));
795     }
```

References get().

### 48.36.3.39 scan()

```
template<typename ScanType >
void gko::experimental::mpi::communicator::scan (
             std::shared_ptr< const Executor > exec,
             const ScanType * send_buffer,
             ScanType * recv_buffer,
             int count,
             MPI_Op operation ) const  [inline]
```

Does a scan operation with the given operator.

(MPI_Scan). See MPI documentation for more details.

#### Parameters

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to scan from |
| *recv_buffer* | the result buffer |
| *count* | the number of elements to scan |
| *operation* | the operation type to be used for the scan. See @MPI_Op |

#### Template Parameters

| | |
|---|---|
| *ScanType* | the type of the data to scan. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

```
1511     {
1512         auto guard = exec->get_scoped_device_id_guard();
1513         GKO_ASSERT_NO_MPI_ERRORS(MPI_Scan(send_buffer, recv_buffer, count,
1514                                           type_impl<ScanType>::get_type(),
1515                                           operation, this->get()));
1516     }
```

References get().

### 48.36.3.40 scatter()

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::scatter (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             RecvType * recv_buffer,
             const int recv_count,
             int root_rank ) const  [inline]
```

Scatter data from root rank to all ranks in the communicator.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_count* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |
| *root_rank* | the rank to scatter from |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
1142     {
1143         auto guard = exec->get_scoped_device_id_guard();
1144         GKO_ASSERT_NO_MPI_ERRORS(MPI_Scatter(
1145             send_buffer, send_count, type_impl<SendType>::get_type(),
1146             recv_buffer, recv_count, type_impl<RecvType>::get_type(), root_rank,
1147             this->get()));
1148     }
```

References get().

#### 48.36.3.41 scatter_v()

```
template<typename SendType , typename RecvType >
void gko::experimental::mpi::communicator::scatter_v (
              std::shared_ptr< const Executor > exec,
              const SendType * send_buffer,
              const int * send_counts,
              const int * displacements,
              RecvType * recv_buffer,
              const int recv_count,
              int root_rank ) const  [inline]
```

Scatter data from root rank to all ranks in the communicator with offsets.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *send_buffer* | the buffer to gather from |
| *send_counts* | the number of elements to send |
| *recv_buffer* | the buffer to gather into |
| *recv_count* | the number of elements to receive |
| *displacements* | the offsets for the buffer |
| *root_rank* | the rank to scatter from |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |
| *RecvType* | the type of the data to receive. The same restrictions as for SendType apply. |

```
1207     {
1208         auto guard = exec->get_scoped_device_id_guard();
1209         GKO_ASSERT_NO_MPI_ERRORS(MPI_Scatterv(
1210             send_buffer, send_counts, displacements,
1211             type_impl<SendType>::get_type(), recv_buffer, recv_count,
1212             type_impl<RecvType>::get_type(), root_rank, this->get()));
1213     }
```

References get().

#### 48.36.3.42 send()

```
template<typename SendType >
void gko::experimental::mpi::communicator::send (
             std::shared_ptr< const Executor > exec,
             const SendType * send_buffer,
             const int send_count,
             const int destination_rank,
             const int send_tag ) const  [inline]
```

Send (Blocking) data from calling process to destination rank.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *send_buffer* | the buffer to send |
| *send_count* | the number of elements to send |
| *destination_rank* | the rank to send the data to |
| *send_tag* | the tag for the send call |

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the data to send. Has to be a type which has a specialization of type_impl that defines its MPI_Datatype. |

```
630     {
631         auto guard = exec->get_scoped_device_id_guard();
632         GKO_ASSERT_NO_MPI_ERRORS(
633             MPI_Send(send_buffer, send_count, type_impl<SendType>::get_type(),
634                      destination_rank, send_tag, this->get()));
635     }
```

References get().

#### 48.36.3.43 size()

```
int gko::experimental::mpi::communicator::size ( ) const  [inline]
```

Return the size of the communicator (number of ranks).

**Returns**

the size

```
533 { return get_num_ranks(); }
```

#### 48.36.3.44 synchronize()

```
void gko::experimental::mpi::communicator::synchronize ( ) const  [inline]
```

This function is used to synchronize the ranks in the communicator.

Calls MPI_Barrier

```
609     {
610         GKO_ASSERT_NO_MPI_ERRORS(MPI_Barrier(this->get()));
611     }
```

References get().

The documentation for this class was generated from the following file:

- ginkgo/core/base/mpi.hpp

## 48.37 gko::Composition< ValueType > Class Template Reference

The Composition class can be used to compose linear operators `op1, op2, ..., opn` and obtain the operator `op1 * op2 * ...`

`#include <ginkgo/core/base/composition.hpp>`

### Public Member Functions

- const std::vector< std::shared_ptr< const LinOp > > & get_operators () const noexcept

  *Returns a list of operators of the composition.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- Composition & operator= (const Composition &)

  *Copy-assigns a Composition.*
- Composition & operator= (Composition &&)

  *Move-assigns a Composition.*
- Composition (const Composition &)

  *Copy-constructs a Composition.*
- Composition (Composition &&)

  *Move-constructs a Composition.*

### 48.37.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::Composition< ValueType >**

The Composition class can be used to compose linear operators `op1, op2, ..., opn` and obtain the operator `op1 * op2 * ...`

- opn.

All LinOps of the Composition must operate on Dense inputs. For an operator `op_k` that require an initial guess for their `apply`, Composition provides either

- the output of the previous `op_{k+1}->apply` if `op_k` has square dimension
- zero if `op_k` is rectangular as an initial guess.

Composition ensures that all LinOps passed to its constructor use the same executor, and if not, copies the operators to the executor of the first operator.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of input and result vectors |

### 48.37.2 Constructor & Destructor Documentation

#### 48.37.2.1 Composition() [1/2]

```
template<typename ValueType = default_precision>
gko::Composition< ValueType >::Composition (
            const Composition< ValueType > &  )
```

Copy-constructs a Composition.

This inherits the executor of the input Composition and all of its operators with shared ownership.

#### 48.37.2.2 Composition() [2/2]

```
template<typename ValueType = default_precision>
gko::Composition< ValueType >::Composition (
            Composition< ValueType > &&  )
```

Move-constructs a Composition.

This inherits the executor of the input Composition and all of its operators. The moved-from object is empty (0x0 LinOp without operators) afterwards.

## 48.37.3 Member Function Documentation

### 48.37.3.1 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::Composition< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.37.3.2 get_operators()

```
template<typename ValueType = default_precision>
const std::vector<std::shared_ptr<const LinOp> >& gko::Composition< ValueType >::get_↩
operators ( ) const  [inline], [noexcept]
```

Returns a list of operators of the composition.

**Returns**

a list of operators

```
56     {
57         return operators_;
58     }
```

### 48.37.3.3 operator=() [1/2]

```
template<typename ValueType = default_precision>
Composition& gko::Composition< ValueType >::operator= (
            Composition< ValueType > &&  )
```

Move-assigns a Composition.

The executor is not modified, and the wrapped LinOps are only being cloned if they are on a different executor, otherwise they share ownership. The moved-from object is empty (0x0 LinOp without operators) afterwards.

#### 48.37.3.4 operator=() [2/2]

```
template<typename ValueType = default_precision>
Composition& gko::Composition< ValueType >::operator= (
            const Composition< ValueType > &  )
```

Copy-assigns a Composition.

The executor is not modified, and the wrapped LinOps are only being cloned if they are on a different executor.

#### 48.37.3.5 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::Composition< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/base/composition.hpp

## 48.38 gko::experimental::mpi::contiguous_type Class Reference

A move-only wrapper for a contiguous MPI_Datatype.

```
#include <ginkgo/core/base/mpi.hpp>
```

### Public Member Functions

- contiguous_type (int count, MPI_Datatype old_type)
  *Constructs a wrapper for a contiguous MPI_Datatype.*
- contiguous_type ()
  *Constructs empty wrapper with MPI_DATATYPE_NULL.*
- contiguous_type (const contiguous_type &)=delete
  *Disallow copying of wrapper type.*
- contiguous_type & operator= (const contiguous_type &)=delete
  *Disallow copying of wrapper type.*
- contiguous_type (contiguous_type &&other) noexcept
  *Move constructor, leaves other with MPI_DATATYPE_NULL.*
- contiguous_type & operator= (contiguous_type &&other) noexcept
  *Move assignment, leaves other with MPI_DATATYPE_NULL.*
- ~contiguous_type ()
  *Destructs object by freeing wrapped MPI_Datatype.*
- MPI_Datatype get () const
  *Access the underlying MPI_Datatype.*

## 48.38.1 Detailed Description

A move-only wrapper for a contiguous MPI_Datatype.

The underlying MPI_Datatype is automatically created and committed when an object of this type is constructed, and freed when it is destructed.

## 48.38.2 Constructor & Destructor Documentation

### 48.38.2.1 contiguous_type() [1/2]

```
gko::experimental::mpi::contiguous_type::contiguous_type (
            int count,
            MPI_Datatype old_type )  [inline]
```

Constructs a wrapper for a contiguous MPI_Datatype.

**Parameters**

| | |
|---|---|
| *count* | the number of old_type elements the new datatype contains. |
| *old_type* | the MPI_Datatype that is contained. |

```
120                                                 :  type_(MPI_DATATYPE_NULL)
121     {
122         GKO_ASSERT_NO_MPI_ERRORS(MPI_Type_contiguous(count, old_type, &type_));
123         GKO_ASSERT_NO_MPI_ERRORS(MPI_Type_commit(&type_));
124     }
```

### 48.38.2.2 contiguous_type() [2/2]

```
gko::experimental::mpi::contiguous_type::contiguous_type (
            contiguous_type && other )  [inline], [noexcept]
```

Move constructor, leaves other with MPI_DATATYPE_NULL.

**Parameters**

| | |
|---|---|
| *other* | to be moved from object. |

```
146                                                 :  type_(MPI_DATATYPE_NULL)
147     {
148         *this = std::move(other);
149     }
```

## 48.38.3 Member Function Documentation

#### 48.38.3.1 get()

```
MPI_Datatype gko::experimental::mpi::contiguous_type::get ( ) const  [inline]
```

Access the underlying MPI_Datatype.

**Returns**

the underlying MPI_Datatype.

```
181 { return type_; }
```

#### 48.38.3.2 operator=()

```
contiguous_type& gko::experimental::mpi::contiguous_type::operator= (
            contiguous_type && other )  [inline], [noexcept]
```

Move assignment, leaves other with MPI_DATATYPE_NULL.

**Parameters**

| | |
|---|---|
| *other* | to be moved from object. |

**Returns**

this object.

```
159     {
160         if (this != &other) {
161             this->type_ = std::exchange(other.type_, MPI_DATATYPE_NULL);
162         }
163         return *this;
164     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/mpi.hpp

## 48.39 gko::log::Convergence< ValueType > Class Template Reference

Convergence is a Logger which logs data strictly from the `criterion_check_completed` event.

`#include <ginkgo/core/log/convergence.hpp>`

### Public Member Functions

- bool has_converged () const noexcept

  *Returns true if the solver has converged.*
- void reset_convergence_status ()

  *Resets the convergence status to false.*

- const size_type & get_num_iterations () const noexcept

  *Returns the number of iterations.*
- const LinOp * get_residual () const noexcept

  *Returns the residual.*
- const LinOp * get_residual_norm () const noexcept

  *Returns the residual norm.*
- const LinOp * get_implicit_sq_resnorm () const noexcept

  *Returns the implicit squared residual norm.*

## Static Public Member Functions

- static std::unique_ptr< Convergence > create (std::shared_ptr< const Executor >, const mask_type &enabled_events=Logger::criterion_events_mask|Logger::iteration_complete_mask)

  *Creates a convergence logger.*
- static std::unique_ptr< Convergence > create (const mask_type &enabled_events=Logger::criterion_↩ events_mask|Logger::iteration_complete_mask)

  *Creates a convergence logger.*

### 48.39.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::log::Convergence< ValueType >**

Convergence is a Logger which logs data strictly from the `criterion_check_completed` event.

The purpose of this logger is to give a simple access to standard data generated by the solver once it has stopped with minimal overhead.

This logger also computes the residual norm from the residual when the residual norm was not available. This can add some slight overhead.

### 48.39.2 Member Function Documentation

#### 48.39.2.1 create() [1/2]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Convergence> gko::log::Convergence< ValueType >::create (
            const mask_type & enabled_events = Logger::criterion_events_mask | Logger::iteration↩
_complete_mask )  [inline], [static]
```

Creates a convergence logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *enabled_events* | the events enabled for this logger. By default all events. |

**Returns**

an std::unique_ptr to the the constructed object

```
95      {
96          return std::unique_ptr<Convergence>(new Convergence(enabled_events));
97      }
```

#### 48.39.2.2 create() [2/2]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Convergence> gko::log::Convergence< ValueType >::create (
            std::shared_ptr< const Executor > ,
            const mask_type & enabled_events = Logger::criterion_events_mask | Logger::iteration↩
_complete_mask )  [inline], [static]
```

Creates a convergence logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *enabled_events* | the events enabled for this logger. By default all events. |

**Returns**

an std::unique_ptr to the the constructed object

```
79      {
80          return std::unique_ptr<Convergence>(new Convergence(enabled_events));
81      }
```

#### 48.39.2.3 get_implicit_sq_resnorm()

```
template<typename ValueType = default_precision>
const LinOp* gko::log::Convergence< ValueType >::get_implicit_sq_resnorm ( ) const  [inline],
[noexcept]
```

Returns the implicit squared residual norm.

**Returns**

the implicit squared residual norm

```
144     {
145         return implicit_sq_resnorm_.get();
146     }
```

### 48.39.2.4 get_num_iterations()

```
template<typename ValueType = default_precision>
const size_type& gko::log::Convergence< ValueType >::get_num_iterations ( ) const  [inline],
[noexcept]
```

Returns the number of iterations.

#### Returns

the number of iterations

```
117     {
118         return num_iterations_;
119     }
```

### 48.39.2.5 get_residual()

```
template<typename ValueType = default_precision>
const LinOp* gko::log::Convergence< ValueType >::get_residual ( ) const  [inline], [noexcept]
```

Returns the residual.

#### Returns

the residual

```
126 { return residual_.get(); }
```

### 48.39.2.6 get_residual_norm()

```
template<typename ValueType = default_precision>
const LinOp* gko::log::Convergence< ValueType >::get_residual_norm ( ) const  [inline], [noexcept]
```

Returns the residual norm.

#### Returns

the residual norm

```
134     {
135         return residual_norm_.get();
136     }
```

#### 48.39.2.7 has_converged()

```
template<typename ValueType = default_precision>
bool gko::log::Convergence< ValueType >::has_converged ( ) const  [inline], [noexcept]
```

Returns true if the solver has converged.

**Returns**

the bool flag for convergence status

```
104 { return convergence_status_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/log/convergence.hpp

## 48.40 gko::ConvertibleTo< ResultType > Class Template Reference

ConvertibleTo interface is used to mark that the implementer can be converted to the object of ResultType.

`#include <ginkgo/core/base/polymorphic_object.hpp>`

### Public Member Functions

- virtual void convert_to (result_type *result) const =0

  *Converts the implementer to an object of type result_type.*
- virtual void move_to (result_type *result)=0

  *Converts the implementer to an object of type result_type by moving data from this object.*

### 48.40.1 Detailed Description

**template<typename ResultType>**
**class gko::ConvertibleTo< ResultType >**

ConvertibleTo interface is used to mark that the implementer can be converted to the object of ResultType.

This interface is used to enable conversions between polymorphic objects. To mark that an object of type `U` can be converted to an object of type `V`, `U` should implement ConvertibleTo<V>. Then, the implementation of `Cloneable::copy_from` automatically generated by `EnableCloneable` mixin will use RTTI to figure out that `U` implements the interface and convert it using the convert_to / move_to methods of the interface.

As an example, the following function:

```
 {c++}
void my_function(const U *u, V *v) {
    v->copy_from(u);
}
```

will convert object `u` to object `v` by checking that `u` can be dynamically casted to `ConvertibleTo\<V\>`, and calling `ConvertibleTo\<V\>::convert_to(V*)` to do the actual conversion.

In case `u` is passed as a unique_ptr, call to `convert_to` will be replaced by a call to `move_to` and trigger move semantics.

**Template Parameters**

| | |
|---|---|
| *ResultType* | the type to which the implementer can be converted to, has to be a subclass of PolymorphicObject |

## 48.40.2 Member Function Documentation

### 48.40.2.1 convert_to()

```
template<typename ResultType>
virtual void gko::ConvertibleTo< ResultType >::convert_to (
            result_type * result ) const  [pure virtual]
```

Converts the implementer to an object of type result_type.

**Parameters**

| | |
|---|---|
| *result* | the object used to store the result of the conversion |

Implemented in gko::EnableCloneable< ConcreteType >, gko::EnableCloneable< SparsityCsr< ValueType, IndexType > >, gko::EnableCloneable< Vector< ValueType > >, gko::EnableCloneable< Diagonal< ValueType > >, gko::EnableCloneable< Dens gko::EnableCloneable< Matrix< ValueType, LocalIndexType, GlobalIndexType > >, gko::EnableCloneable< Hybrid< ValueType, In gko::EnableCloneable< RowGatherer< LocalIndexType > >, gko::EnableCloneable< Identity< ValueType > >, gko::EnableCloneable< NullspaceProjection< ValueType > >, gko::EnableCloneable< Nullspace< ValueType > >, gko::EnableCloneable< Fft3 >, gko::EnableCloneable< Partition< LocalIndexType, GlobalIndexType > >, gko::EnableCloneable< RowGatherer< IndexType > >, gko::EnableCloneable< Fft2 >, gko::EnableCloneable< Fbcsr< ValueType gko::EnableCloneable< MultiVector< ValueType > >, gko::EnableCloneable< GaussSeidel< ValueType, IndexType > >, gko::EnableCloneable< Csr< ValueType, IndexType > >, gko::EnableCloneable< Coo< ValueType, IndexType > >, gko::EnableCloneable< Fft >, gko::EnableCloneable< BlockOperator >, gko::EnableCloneable< Ell< ValueType, IndexType > >, gko::EnableCloneable< Permutation< IndexType > >, gko::EnableCloneable< ScaledPermutation< ValueType, IndexType > >, gko::EnableCloneable< Jacobi< ValueType, IndexType > >, gko::EnableCloneable< Sellp< ValueType, IndexType > >, gko::EnableCloneable< Perturbation< ValueType > >, and gko::preconditioner::Jacobi< ValueType, IndexType >.

### 48.40.2.2 move_to()

```
template<typename ResultType>
virtual void gko::ConvertibleTo< ResultType >::move_to (
            result_type * result )  [pure virtual]
```

Converts the implementer to an object of type result_type by moving data from this object.

This method is used when the implementer is a temporary object, and move semantics can be used.

**Parameters**

| | |
|---|---|
| *result* | the object used to emplace the result of the conversion |

**Note**

ConvertibleTo::move_to can be implemented by simply calling ConvertibleTo::convert_to. However, this operation can often be optimized by exploiting the fact that implementer's data can be moved to the result.

Implemented in gko::EnableCloneable< ConcreteType >, gko::EnableCloneable< SparsityCsr< ValueType, IndexType > >, gko::EnableCloneable< Vector< ValueType > >, gko::EnableCloneable< Diagonal< ValueType > >, gko::EnableCloneable< Dens gko::EnableCloneable< Matrix< ValueType, LocalIndexType, GlobalIndexType > >, gko::EnableCloneable< Hybrid< ValueType, In gko::EnableCloneable< RowGatherer< LocalIndexType > >, gko::EnableCloneable< Identity< ValueType > >, gko::EnableCloneable< NullspaceProjection< ValueType > >, gko::EnableCloneable< Nullspace< ValueType > >, gko::EnableCloneable< Fft3 >, gko::EnableCloneable< Partition< LocalIndexType, GlobalIndexType > >, gko::EnableCloneable< RowGatherer< IndexType > >, gko::EnableCloneable< Fft2 >, gko::EnableCloneable< Fbcsr< ValueType gko::EnableCloneable< MultiVector< ValueType > >, gko::EnableCloneable< GaussSeidel< ValueType, IndexType > >, gko::EnableCloneable< Csr< ValueType, IndexType > >, gko::EnableCloneable< Coo< ValueType, IndexType > >, gko::EnableCloneable< Fft >, gko::EnableCloneable< BlockOperator >, gko::EnableCloneable< Ell< ValueType, IndexType > >, gko::EnableCloneable< Permutation< IndexType > >, gko::EnableCloneable< ScaledPermutation< ValueType, IndexType > >, gko::EnableCloneable< Jacobi< ValueType, IndexType > >, gko::EnableCloneable< Sellp< ValueType, IndexType > >, gko::EnableCloneable< Perturbation< ValueType > >, and gko::preconditioner::Jacobi< ValueType, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/base/polymorphic_object.hpp

## 48.41 gko::matrix::Coo< ValueType, IndexType > Class Template Reference

COO stores a matrix in the coordinate matrix format.

`#include <ginkgo/core/matrix/coo.hpp>`

### Public Member Functions

- void read (const mat_data &data) override

  *Reads a matrix from a matrix_data structure.*
- void read (const device_mat_data &data) override

  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override

  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override

  *Writes a matrix to a matrix_data structure.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- value_type * get_values () noexcept

*Returns the values of the matrix.*

- const value_type * get_const_values () const noexcept

  *Returns the values of the matrix.*
- index_type * get_col_idxs () noexcept

  *Returns the column indexes of the matrix.*
- const index_type * get_const_col_idxs () const noexcept

  *Returns the column indexes of the matrix.*
- index_type * get_row_idxs () noexcept

  *Returns the row indexes of the matrix.*
- const index_type * get_const_row_idxs () const noexcept
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*
- device_view get_device_view ()

  *Returns a non-owning device view of this matrix.*
- const_device_view get_const_device_view () const

  *Returns a non-owning const device view of this matrix.*
- void apply2 (ptr_param< const LinOp > b, ptr_param< LinOp > x)

  *Applies Coo matrix axpy to a vector (or a sequence of vectors).*
- void apply2 (ptr_param< const LinOp > b, ptr_param< LinOp > x) const
- void apply2 (ptr_param< const LinOp > alpha, ptr_param< const LinOp > b, ptr_param< LinOp > x)

  *Performs the operation x = alpha * Coo * b + x.*
- void apply2 (ptr_param< const LinOp > alpha, ptr_param< const LinOp > b, ptr_param< LinOp > x) const
- void validate_data () const override

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*

## Static Public Member Functions

- static std::unique_ptr< Coo > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size=dim< 2 >{}, size_type num_nonzeros={})

  *Creates an uninitialized COO matrix of the specified size.*
- static std::unique_ptr< Coo > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< value_type > values, array< index_type > col_idxs, array< index_type > row_idxs)

  *Creates a COO matrix from already allocated (and initialized) row index, column index and value arrays.*
- template<typename InputValueType , typename InputColumnIndexType , typename InputRowIndexType >
  static std::unique_ptr< Coo > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, std::initializer_list< InputValueType > values, std::initializer_list< InputColumnIndexType > col_idxs, std::initializer_list< InputRowIndexType > row_idxs)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Coo > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< ValueType > &&values, gko::detail::const_array_view< IndexType > &&col_idxs, gko::detail::const_array_view< IndexType > &&row_idxs)

  *Creates a constant (immutable) Coo matrix from a set of constant arrays.*

### 48.41.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Coo< ValueType, IndexType >**

COO stores a matrix in the coordinate matrix format.

The nonzero elements are stored in an array row-wise (but not necessarily sorted by column index within a row). Two extra arrays contain the row and column indexes of each nonzero element of the matrix.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.41.2 Member Function Documentation

### 48.41.2.1 apply2() [1/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::apply2 (
            ptr_param< const LinOp > alpha,
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x )
```

Performs the operation x = alpha ∗ Coo ∗ b + x.

**Parameters**

| | |
|---|---|
| *alpha* | scaling of the result of Coo ∗ b |
| *b* | vector(s) on which the operator is applied |
| *x* | output vector(s) |

**Returns**

this

### 48.41.2.2 apply2() [2/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::apply2 (
            ptr_param< const LinOp > alpha,
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x ) const
```

### 48.41.2.3 apply2() [3/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::apply2 (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x )
```

Applies Coo matrix axpy to a vector (or a sequence of vectors).

Performs the operation x = Coo ∗ b + x

**Parameters**

| | |
|---|---|
| *b* | the input vector(s) on which the operator is applied |
| *x* | the output vector(s) where the result is stored |

**Returns**

this

#### 48.41.2.4 apply2() [4/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::apply2 (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x ) const
```

#### 48.41.2.5 compute_absolute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<absolute_type> gko::matrix::Coo< ValueType, IndexType >::compute_absolute ( )
const  [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Coo< ValueType, IndexType > > >.

#### 48.41.2.6 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Coo< ValueType, IndexType >::conj_transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.41.2.7 create() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Coo> gko::matrix::Coo< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< value_type > values,
            array< index_type > col_idxs,
            array< index_type > row_idxs )  [static]
```

Creates a COO matrix from already allocated (and initialized) row index, column index and value arrays.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *values* | array of matrix values |
| *col_idxs* | array of column indexes |
| *row_idxs* | array of row pointers |

**Note**

If one of `row_idxs`, `col_idxs` or `values` is not an rvalue, not an array of IndexType, IndexType and ValueType, respectively, or is on the wrong executor, an internal copy of that array will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of these arrays on the correct executor.

#### 48.41.2.8 create() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename InputValueType , typename InputColumnIndexType , typename InputRowIndexType
>
static std::unique_ptr<Coo> gko::matrix::Coo< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            std::initializer_list< InputValueType > values,
            std::initializer_list< InputColumnIndexType > col_idxs,
            std::initializer_list< InputRowIndexType > row_idxs )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const dim<2>&, array<value_type>, array<index_type>, array<index_type>)

```
326     {
327         return create(exec, size, array<value_type>{exec, std::move(values)},
328                       array<index_type>{exec, std::move(col_idxs)},
329                       array<index_type>{exec, std::move(row_idxs)});
330     }
```

References gko::matrix::Coo< ValueType, IndexType >::create().

#### 48.41.2.9 create() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Coo> gko::matrix::Coo< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = dim< 2 >{},
            size_type num_nonzeros = {} )  [static]
```

Creates an uninitialized COO matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_nonzeros* | number of nonzeros |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::matrix::Coo< ValueType, IndexType >::create().

#### 48.41.2.10 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Coo> gko::matrix::Coo< ValueType, IndexType >::create_const (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            gko::detail::const_array_view< ValueType > && values,
            gko::detail::const_array_view< IndexType > && col_idxs,
            gko::detail::const_array_view< IndexType > && row_idxs )  [static]
```

Creates a constant (immutable) Coo matrix from a set of constant arrays.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *values* | the value array of the matrix |
| *col_idxs* | the column index array of the matrix |
| *row_idxs* | the row index array of the matrix |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of these arrays on the correct executor.

#### 48.41.2.11 extract_diagonal()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Coo< ValueType, IndexType >::extract_↩
diagonal ( ) const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

#### 48.41.2.12 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Coo< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

```
178 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.41.2.13 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Coo< ValueType, IndexType >::get_const_col_idxs ( ) const  [inline],
[noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
188     {
189         return col_idxs_.get_const_data();
190     }
```

References gko::array< ValueType >::get_const_data().

#### 48.41.2.14 get_const_device_view()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const_device_view gko::matrix::Coo< ValueType, IndexType >::get_const_device_view ( ) const
```

Returns a non-owning const device view of this matrix.

**Returns**

a const device view of this matrix.

#### 48.41.2.15 get_const_row_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Coo< ValueType, IndexType >::get_const_row_idxs ( ) const  [inline],
[noexcept]
```

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
207     {
208         return row_idxs_.get_const_data();
209     }
```

References gko::array< ValueType >::get_const_data().

#### 48.41.2.16 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Coo< ValueType, IndexType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
169     {
170         return values_.get_const_data();
171     }
```

References gko::array< ValueType >::get_const_data().

#### 48.41.2.17 get_device_view()

```
template<typename ValueType = default_precision, typename IndexType = int32>
device_view gko::matrix::Coo< ValueType, IndexType >::get_device_view ( )
```

Returns a non-owning device view of this matrix.

**Returns**

a device view of this matrix.

#### 48.41.2.18 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Coo< ValueType, IndexType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
217     {
218         return values_.get_size();
219     }
```

References gko::array< ValueType >::get_size().

#### 48.41.2.19 get_row_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Coo< ValueType, IndexType >::get_row_idxs ( )  [inline], [noexcept]
```

Returns the row indexes of the matrix.

**Returns**

the row indexes of the matrix.

```
197 { return row_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.41.2.20 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Coo< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

```
159 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.41.2.21 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.41.2.22 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.41.2.23 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
```

```
void gko::matrix::Coo< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.41.2.24 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Coo< ValueType, IndexType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.41.2.25 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Coo< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/coo.hpp

# 48.42 gko::matrix::view::coo< ValueType, IndexType > Struct Template Reference

Non-owning view of a matrix::Coo to be used inside device kernels.

`#include <ginkgo/core/matrix/device_views.hpp>`

## Public Member Functions

- constexpr coo (dim< 2 > size, size_type num_stored_elements, ValueType *values, IndexType *row_idxs, IndexType *col_idxs)

  *Constructs a coo view from size, nnz, values, row and column indices.*
- constexpr coo< const ValueType, const IndexType > as_const () const

  *Returns a const view of the same data.*

## 48.42.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**struct gko::matrix::view::coo< ValueType, IndexType >**

Non-owning view of a matrix::Coo to be used inside device kernels.

This type is used to provide a simple and stable ABI for passing data between libraries.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store matrix entries. |
| *IndexType* | the index type used to store row and column indices. |

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/device_views.hpp

# 48.43 gko::CpuAllocator Class Reference

Allocator using new/delete.

`#include <ginkgo/core/base/memory.hpp>`

## 48.43.1 Detailed Description

Allocator using new/delete.

The documentation for this class was generated from the following file:

- ginkgo/core/base/memory.hpp

## 48.44 gko::CpuAllocatorBase Class Reference

Implement this interface to provide an allocator for OmpExecutor or ReferenceExecutor.

`#include <ginkgo/core/base/memory.hpp>`

### 48.44.1 Detailed Description

Implement this interface to provide an allocator for OmpExecutor or ReferenceExecutor.

The documentation for this class was generated from the following file:

- ginkgo/core/base/memory.hpp

## 48.45 gko::CpuTimer Class Reference

A timer using std::chrono::steady_clock for timing.

`#include <ginkgo/core/base/timer.hpp>`

### Public Member Functions

- void record (time_point &time) override

  *Records a time point at the current time.*
- void wait (time_point &time) override

  *Waits until all kernels in-process when recording the time point are finished.*
- std::chrono::nanoseconds difference_async (const time_point &start, const time_point &stop) override

  *Computes the difference between the two time points in nanoseconds.*

### Additional Inherited Members

### 48.45.1 Detailed Description

A timer using std::chrono::steady_clock for timing.

### 48.45.2 Member Function Documentation

#### 48.45.2.1 difference_async()

```
std::chrono::nanoseconds gko::CpuTimer::difference_async (
            const time_point & start,
            const time_point & stop )  [override], [virtual]
```

Computes the difference between the two time points in nanoseconds.

This asynchronous version does not synchronize itself, so the time points need to have been synchronized with, i.e. `timer->wait(stop)` needs to have been called. The version is intended for more advanced users who want to measure the overhead of timing functionality separately.

**Parameters**

| | |
|---|---|
| *start* | the first time point (earlier) |
| *stop* | the second time point (later) |

**Returns**

the difference between the time points in nanoseconds.

Implements gko::Timer.

The documentation for this class was generated from the following file:

- ginkgo/core/base/timer.hpp

## 48.46 gko::cpx_real_type< T > Struct Template Reference

Access the underlying real type of a complex number.

`#include <ginkgo/core/base/math.hpp>`

### Public Types

- using type = T
  *The type.*

### 48.46.1 Detailed Description

**template<typename T>**
**struct gko::cpx_real_type< T >**

Access the underlying real type of a complex number.

**Template Parameters**

| | |
|---|---|
| *T* | the type being checked. |

### 48.46.2 Member Typedef Documentation

#### 48.46.2.1 type

```
template<typename T >
using gko::cpx_real_type< T >::type = T
```

The type.

When the type is not complex, return the type itself.

The documentation for this struct was generated from the following file:

- ginkgo/core/base/math.hpp

## 48.47 gko::stop::Criterion Class Reference

The Criterion class is a base class for all stopping criteria.

`#include <ginkgo/core/stop/criterion.hpp>`

### Classes

- class Updater

  *The Updater class serves for convenient argument passing to the Criterion's check function.*

### Public Member Functions

- Updater update ()

  *Returns the updater object.*
- bool check (uint8 stopping_id, bool set_finalized, array< stopping_status > *stop_status, bool *one_changed, const Updater &updater)

  *This checks whether convergence was reached for a certain criterion.*

### 48.47.1 Detailed Description

The Criterion class is a base class for all stopping criteria.

It contains a factory to instantiate criteria. It is up to each specific stopping criterion to decide what to do with the data that is passed to it.

Note that depending on the criterion, convergence may not have happened after stopping.

### 48.47.2 Member Function Documentation

#### 48.47.2.1 check()

```
bool gko::stop::Criterion::check (
            uint8 stopping_id,
            bool set_finalized,
            array< stopping_status > * stop_status,
            bool * one_changed,
            const Updater & updater )  [inline]
```

This checks whether convergence was reached for a certain criterion.

The actual implantation of the criterion goes here.

**Parameters**

| | |
|---|---|
| *stopping_id* | id of the stopping criterion |
| *set_finalized* | Controls if the current version should count as finalized or not |
| *stop_status* | status of the stopping criterion |
| *one_changed* | indicates if the status of a vector has changed |
| *updater* | the Updater object containing all the information |

**Returns**

whether convergence was completely reached

```
144     {
145         this->template log<log::Logger::criterion_check_started>(
146             this, updater.num_iterations_, updater.residual_,
147             updater.residual_norm_, updater.solution_, stopping_id,
148             set_finalized);
149         auto all_converged = this->check_impl(
150             stopping_id, set_finalized, stop_status, one_changed, updater);
151         this->template log<log::Logger::criterion_check_completed>(
152             this, updater.num_iterations_, updater.residual_,
153             updater.residual_norm_, updater.implicit_sq_residual_norm_,
154             updater.solution_, stopping_id, set_finalized, stop_status,
155             *one_changed, all_converged);
156         return all_converged;
157     }
```

Referenced by gko::stop::Criterion::Updater::check().

#### 48.47.2.2 update()

```
Updater gko::stop::Criterion::update ( )  [inline]
```

Returns the updater object.

**Returns**

the updater object

```
126 { return {this}; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/stop/criterion.hpp

## 48.48 gko::log::criterion_data Struct Reference

Struct representing Criterion related data.

```
#include <ginkgo/core/log/record.hpp>
```

### 48.48.1 Detailed Description

Struct representing Criterion related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.49 gko::stop::CriterionArgs Struct Reference

This struct is used to pass parameters to the EnableDefaultCriterionFactoryCriterionFactory::generate() method.

`#include <ginkgo/core/stop/criterion.hpp>`

### 48.49.1 Detailed Description

This struct is used to pass parameters to the EnableDefaultCriterionFactoryCriterionFactory::generate() method.

It is the ComponentsType of CriterionFactory.

**Note**

Dependly on the use case, some of these parameters can be `nullptr` as only some stopping criterion require them to be set. An example is the `ResidualNormReduction` which really requires the `initial_residual` to be set.

The documentation for this struct was generated from the following file:

- ginkgo/core/stop/criterion.hpp

## 48.50 gko::matrix::Csr< ValueType, IndexType > Class Template Reference

CSR is a matrix format which stores only the nonzero coefficients by compressing each row of the matrix (compressed sparse row format).

`#include <ginkgo/core/matrix/csr.hpp>`

### Classes

- class multiply_add_reuse_info

  *Class describing the internal lookup structures created by multiply_add_reuse to recompute a sparse matrix-matrix product with updated values.*
- class multiply_reuse_info

  *Class describing the internal lookup structures created by multiply_reuse(const Csr*) to recompute a sparse matrix-matrix product with updated values.*
- struct permuting_reuse_info

  *A struct describing a transformation of the matrix that reorders the values of the matrix into the transformed matrix.*
- class scale_add_reuse_info

  *Class describing the internal lookup structures created by scale_add_reuse to recompute a sparse matrix-matrix sum with updated values.*

## Public Member Functions

- void read (const mat_data &data) override

  *Reads a matrix from a matrix_data structure.*
- void read (const device_mat_data &data) override

  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override

  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override

  *Writes a matrix to a matrix_data structure.*
- void write (device_mat_data &data) const override

  *Writes a matrix to a device_matrix_data structure.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- device_view get_device_view ()

  *Returns a non-owning device view of this matrix.*
- const_device_view get_const_device_view () const

  *Returns a non-owning const device view of this matrix.*
- std::unique_ptr< Csr > multiply (ptr_param< const Csr > other) const

  *Computes the sparse matrix product `this * other` on the executor of this matrix.*
- std::pair< std::unique_ptr< Csr >, multiply_reuse_info > multiply_reuse (ptr_param< const Csr > other) const

  *Computes the sparse matrix product `this * other` on the executor of this matrix, and necessary data for value updates:*
- std::unique_ptr< Csr > multiply_add (ptr_param< const Dense< value_type >> scale_mult, ptr_param< const Csr > mtx_mult, ptr_param< const Dense< value_type >> scale_add, ptr_param< const Csr > mtx_add) const

  *Computes the sparse matrix product `scale_mult * this * mtx_mult + scale_add * mtx_add` on the executor of this matrix.*
- std::pair< std::unique_ptr< Csr >, multiply_add_reuse_info > multiply_add_reuse (ptr_param< const Dense< value_type >> scale_mult, ptr_param< const Csr > mtx_mult, ptr_param< const Dense< value_type >> scale_add, ptr_param< const Csr > mtx_add) const

  *Computes the sparse matrix product `scale_mult * this * mtx_mult + scale_add * mtx_add` on the executor of this matrix, and necessary data for value updates:*
- std::unique_ptr< Csr > scale_add (ptr_param< const Dense< value_type >> scale_this, ptr_param< const Dense< value_type >> scale_other, ptr_param< const Csr > mtx_other) const

  *Computes the sparse matrix sum `scale_this * this + scale_other * mtx_add` on the executor of this matrix.*
- std::pair< std::unique_ptr< Csr >, scale_add_reuse_info > add_scale_reuse (ptr_param< const Dense< value_type >> scale_this, ptr_param< const Dense< value_type >> scale_other, ptr_param< const Csr > mtx_other) const

  *Computes the sparse matrix sum `scale_this * this + scale_other * mtx_add` on the executor of this matrix, and necessary data for value updates:*
- std::pair< std::unique_ptr< Csr >, permuting_reuse_info > transpose_reuse () const

  *Computes the necessary data to update a transposed matrix from its original matrix.*
- std::unique_ptr< Csr > permute (ptr_param< const Permutation< index_type >> permutation, permute_mode mode=permute_mode::symmetric) const

  *Creates a permuted copy $A'$ of this matrix $A$ with the given permutation $P$.*
- std::unique_ptr< Csr > permute (ptr_param< const Permutation< index_type >> row_permutation, ptr_param< const Permutation< index_type >> column_permutation, bool invert=false) const

  *Creates a non-symmetrically permuted copy $A'$ of this matrix $A$ with the given row and column permutations $P$ and $Q$.*

- std::pair< std::unique_ptr< Csr >, permuting_reuse_info > permute_reuse (ptr_param< const Permutation< index_type >> permutation, permute_mode mode=permute_mode::symmetric) const

  *Computes the operations necessary to propagate changed values from a matrix A to a permuted matrix.*
- std::pair< std::unique_ptr< Csr >, permuting_reuse_info > permute_reuse (ptr_param< const Permutation< index_type >> row_permutation, ptr_param< const Permutation< index_type >> column↩ _permutation, bool invert=false) const

  *Computes the operations necessary to propagate changed values from a matrix A to a permuted matrix.*
- std::unique_ptr< Csr > scale_permute (ptr_param< const ScaledPermutation< value_type, index_type >> permutation, permute_mode=permute_mode::symmetric) const

  *Creates a scaled and permuted copy of this matrix.*
- std::unique_ptr< Csr > scale_permute (ptr_param< const ScaledPermutation< value_type, index_type >> row_permutation, ptr_param< const ScaledPermutation< value_type, index_type >> column_permutation, bool invert=false) const

  *Creates a scaled and permuted copy of this matrix.*
- std::unique_ptr< LinOp > permute (const array< IndexType > *permutation_indices) const override

  *Returns a LinOp representing the symmetric row and column permutation of the Permutable object.*
- std::unique_ptr< LinOp > inverse_permute (const array< IndexType > *inverse_permutation_indices) const override

  *Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.*
- std::unique_ptr< LinOp > row_permute (const array< IndexType > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the Permutable object.*
- std::unique_ptr< LinOp > column_permute (const array< IndexType > *permutation_indices) const override

  *Returns a LinOp representing the column permutation of the Permutable object.*
- std::unique_ptr< LinOp > inverse_row_permute (const array< IndexType > *inverse_permutation_indices) const override

  *Returns a LinOp representing the row permutation of the inverse permuted object.*
- std::unique_ptr< LinOp > inverse_column_permute (const array< IndexType > *inverse_permutation_↩ indices) const override

  *Returns a LinOp representing the row permutation of the inverse permuted object.*
- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- void sort_by_column_index ()

  *Sorts all (value, col_idx) pairs in each row by column index.*
- value_type * get_values () noexcept

  *Returns the values of the matrix.*
- const value_type * get_const_values () const noexcept

  *Returns the values of the matrix.*
- std::unique_ptr< Dense< ValueType > > create_value_view ()

  *Creates a Dense view of the value array of this matrix as a column vector of dimensions nnz x 1.*
- std::unique_ptr< const Dense< ValueType > > create_const_value_view () const

  *Creates a const Dense view of the value array of this matrix as a column vector of dimensions nnz x 1.*
- index_type * get_col_idxs () noexcept

  *Returns the column indexes of the matrix.*
- const index_type * get_const_col_idxs () const noexcept

  *Returns the column indexes of the matrix.*
- index_type * get_row_ptrs () noexcept

  *Returns the row pointers of the matrix.*
- const index_type * get_const_row_ptrs () const noexcept

*Returns the row pointers of the matrix.*

- index_type * get_srow () noexcept

  *Returns the starting rows.*

- const index_type * get_const_srow () const noexcept

  *Returns the starting rows.*

- size_type get_num_srow_elements () const noexcept

  *Returns the number of the srow stored elements (involved warps)*

- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*

- csr::spmv_strategy get_strategy () const noexcept

  *Returns the strategy.*

- void set_strategy (csr::spmv_strategy strategy)

  *Set the strategy.*

- void scale (ptr_param< const LinOp > alpha)

  *Scales the matrix with a scalar.*

- void inv_scale (ptr_param< const LinOp > alpha)

  *Scales the matrix with the inverse of a scalar.*

- void validate_data () const override

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*

- std::unique_ptr< Csr< ValueType, IndexType > > create_submatrix (const index_set< IndexType > &row_index_set, const index_set< IndexType > &column_index_set) const

  *Creates a submatrix from this Csr matrix given row and column index_set objects.*

- std::unique_ptr< Csr< ValueType, IndexType > > create_submatrix (const span &row_span, const span &column_span) const

  *Creates a submatrix from this Csr matrix given row and column spans.*

- Csr & operator= (const Csr &)

  *Copy-assigns a Csr matrix.*

- Csr & operator= (Csr &&)

  *Move-assigns a Csr matrix.*

- Csr (const Csr &)

  *Copy-constructs a Csr matrix.*

- Csr (Csr &&)

  *Move-constructs a Csr matrix.*

## Static Public Member Functions

- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, csr::spmv_strategy strategy)

  *Creates an uninitialized CSR matrix of the specified size.*

- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size={}, size_type num_nonzeros={}, csr::spmv_strategy strategy=csr::spmv_strategy::automatic)

  *Creates an uninitialized CSR matrix of the specified size.*

- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< value_type > values, array< index_type > col_idxs, array< index_type > row_ptrs, csr::spmv_strategy strategy=csr::spmv_strategy::automatic)

  *Creates a CSR matrix from already allocated (and initialized) row pointer, column index and value arrays.*

- template<typename InputValueType , typename InputColumnIndexType , typename InputRowPtrType >
  static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, std::initializer_list< InputValueType > values, std::initializer_list< InputColumnIndexType > col_idxs, std::initializer_list< InputRowPtrType > row_ptrs)

  *create(std::shared_ptr<const Executor>,*

- static std::unique_ptr< const Csr > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< ValueType > &&values, gko::detail::const_array_view< IndexType > &&col_idxs, gko::detail::const_array_view< IndexType > &&row_ptrs, csr::spmv_strategy strategy=csr↩ ::spmv_strategy::automatic)

  *Creates a constant (immutable) Csr matrix from a set of constant arrays.*
- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, std::shared_ptr< strategy↩ _type > strategy)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< value_type > values, array< index_type > col_idxs, array< index_type > row_ptrs, std::shared↩ _ptr< strategy_type > strategy)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Csr > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< ValueType > &&values, gko::detail::const_array_view< IndexType > &&col_idxs, gko::detail::const_array_view< IndexType > &&row_ptrs, std::shared_ptr< strategy_type > strategy)

  *Csr> create_const(std::shared_ptr<const*

## 48.50.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Csr< ValueType, IndexType >**

CSR is a matrix format which stores only the nonzero coefficients by compressing each row of the matrix (compressed sparse row format).

The nonzero elements are stored in a 1D array row-wise, and accompanied with a row pointer array which stores the starting index of each row. An additional column index array is used to identify the column of each nonzero element.

The Csr LinOp supports three families of `apply` operations, dispatched on the type of the right operand:

- Against a `Dense` operand `b`, `apply` computes a sparse matrix-vector (or matrix-multivector) product:

$$x = Ab, \qquad x = \alpha Ab + \beta x.$$

- Against another `Csr` operand `B`, `apply` computes a sparse-sparse matrix product (SpGEMM):

$$C = AB, \qquad C = \alpha AB + \beta C.$$

- Against an `Identity` operand, `apply` reduces to a sparse-sparse matrix addition (SpGEAM):

$$B = \alpha A + \beta B.$$

In code:

```
matrix::Csr *A, *B, *C;      // matrices
matrix::Dense *b, *x;        // vectors tall-and-skinny matrices
matrix::Dense *alpha, *beta; // scalars of dimension 1x1
matrix::Identity *I;         // identity matrix
// Applying to Dense matrices computes an SpMV/SpMM product
A->apply(b, x)               // x = A*b
A->apply(alpha, b, beta, x) // x = alpha*A*b + beta*x
// Applying to Csr matrices computes a SpGEMM product of two sparse matrices
A->apply(B, C)               // C = A*B
A->apply(alpha, B, beta, C) // C = alpha*A*B + beta*C
// Applying to an Identity matrix computes a SpGEAM sparse matrix addition
A->apply(alpha, I, beta, B) // B = alpha*A + beta*B
```

Both the SpGEMM and SpGEAM operation require the input matrices to be sorted by column index, otherwise the algorithms will produce incorrect results.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.50.2 Constructor & Destructor Documentation

### 48.50.2.1 Csr() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Csr< ValueType, IndexType >::Csr (
            const Csr< ValueType, IndexType > &  )
```

Copy-constructs a Csr matrix.

Inherits executor, strategy and data.

### 48.50.2.2 Csr() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Csr< ValueType, IndexType >::Csr (
            Csr< ValueType, IndexType > &&  )
```

Move-constructs a Csr matrix.

Inherits executor and strategy, moves the data and leaves the moved-from object in an empty state (0x0 LinOp with unchanged executor and strategy, no nonzeros and valid row pointers).

## 48.50.3 Member Function Documentation

### 48.50.3.1 add_scale_reuse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::pair<std::unique_ptr<Csr>, scale_add_reuse_info> gko::matrix::Csr< ValueType, IndexType
>::add_scale_reuse (
            ptr_param< const Dense< value_type >> scale_this,
            ptr_param< const Dense< value_type >> scale_other,
            ptr_param< const Csr< ValueType, IndexType > > mtx_other ) const
```

Computes the sparse matrix sum `scale_this * this + scale_other * mtx_add` on the executor of this matrix, and necessary data for value updates:

```
auto [result, reuse] = mtx->add_scale_reuse(alpha, beta, mtx2);
change_values(alpha, mtx, beta, mtx2);
reuse->update_values(alpha, mtx, beta, mtx2, result);
```

This matrix needs to be sorted by column index, otherwise the result will be incorrect.

**Parameters**

| | |
|---|---|
| *scale_this* | the scalar by which this matrix will be scaled. |
| *scale_other* | the scalar by which this matrix will be scaled. |
| *mtx_other* | the matrix which will be added to this, scaled by scale_other. It needs to be sorted by column index, otherwise the result will be incorrect. |

**Returns**

std::pair containing the result of the computation, stored on the same executor as this matrix, and a scale_add_reuse_info object allowing value updates to the output matrix.

#### 48.50.3.2 column_permute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::column_permute (
            const array< IndexType > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the column permutation of the Permutable object.

In the resulting LinOp, the column `i` contains the input column `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new column permuted object

Implements gko::Permutable< IndexType >.

#### 48.50.3.3 compute_absolute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<absolute_type> gko::matrix::Csr< ValueType, IndexType >::compute_absolute ( )
const  [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Csr< ValueType, IndexType > > >.

#### 48.50.3.4 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::conj_transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.50.3.5 create() [1/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< value_type > values,
            array< index_type > col_idxs,
            array< index_type > row_ptrs,
            csr::spmv_strategy strategy = csr::spmv_strategy::automatic )  [static]
```

Creates a CSR matrix from already allocated (and initialized) row pointer, column index and value arrays.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *values* | array of matrix values |
| *col_idxs* | array of column indexes |
| *row_ptrs* | array of row pointers |
| *strategy* | the strategy the matrix uses for SpMV operations, default is automatic. |

**Note**

If one of `row_ptrs`, `col_idxs` or `values` is not an rvalue, not an array of IndexType, IndexType and ValueType, respectively, or is on the wrong executor, an internal copy of that array will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

#### 48.50.3.6 create() [2/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< value_type > values,
            array< index_type > col_idxs,
            array< index_type > row_ptrs,
            std::shared_ptr< strategy_type > strategy )  [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const dim<2>&, array<value_type>, array<index_type>, array<index_type>, csr::spmv_strategy)

#### 48.50.3.7 create() [3/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename InputValueType , typename InputColumnIndexType , typename InputRowPtrType >
static std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            std::initializer_list< InputValueType > values,
            std::initializer_list< InputColumnIndexType > col_idxs,
            std::initializer_list< InputRowPtrType > row_ptrs )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const dim<2>&, array<value_type>, array<index_type>, array<index_type>)

```
1035     {
1036         return create(exec, size, array<value_type>{exec, std::move(values)},
1037                       array<index_type>{exec, std::move(col_idxs)},
1038                       array<index_type>{exec, std::move(row_ptrs)});
1039     }
```

References gko::matrix::Csr< ValueType, IndexType >::create().

#### 48.50.3.8 create() [4/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = {},
            size_type num_nonzeros = {},
            csr::spmv_strategy strategy = csr::spmv_strategy::automatic )  [static]
```

Creates an uninitialized CSR matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_nonzeros* | number of nonzeros |
| *strategy* | the strategy the matrix uses for SpMV operations, default is automatic. |

**Returns**

A smart pointer to the newly created matrix.

#### 48.50.3.9 create() [5/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            csr::spmv_strategy strategy )  [static]
```

Creates an uninitialized CSR matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *strategy* | the strategy of CSR |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::matrix::Csr< ValueType, IndexType >::create().

#### 48.50.3.10 create() [6/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< strategy_type > strategy )  [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, csr::spmv_strategy)

#### 48.50.3.11 create_const() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Csr> gko::matrix::Csr< ValueType, IndexType >::create_const (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            gko::detail::const_array_view< ValueType > && values,
            gko::detail::const_array_view< IndexType > && col_idxs,
            gko::detail::const_array_view< IndexType > && row_ptrs,
            csr::spmv_strategy strategy = csr::spmv_strategy::automatic )  [static]
```

Creates a constant (immutable) Csr matrix from a set of constant arrays.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *values* | the value array of the matrix |
| *col_idxs* | the column index array of the matrix |
| *row_ptrs* | the row pointer array of the matrix |
| *strategy* | the strategy the matrix uses for SpMV operations, default is automatic. |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of these arrays on the correct executor.

A smart pointer to the newly created matrix.

#### 48.50.3.12 create_const() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Csr> gko::matrix::Csr< ValueType, IndexType >::create_const (
             std::shared_ptr< const Executor > exec,
             const dim< 2 > & size,
             gko::detail::const_array_view< ValueType > && values,
             gko::detail::const_array_view< IndexType > && col_idxs,
             gko::detail::const_array_view< IndexType > && row_ptrs,
             std::shared_ptr< strategy_type > strategy )  [static]
```

Csr> create_const(std::shared_ptr<const

Csr> create_const(std::shared_ptr<const executor>="">, const dim<2>&, gko::detail::const_array_view<↩ ValueType>&&, gko::detail::const_array_view<IndexType>&&, gko::detail::const_array_view<IndexType>&&, csr::spmv_strategy)

#### 48.50.3.13 create_submatrix() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr<ValueType, IndexType> > gko::matrix::Csr< ValueType, IndexType >::create↩
_submatrix (
             const index_set< IndexType > & row_index_set,
             const index_set< IndexType > & column_index_set ) const
```

Creates a submatrix from this Csr matrix given row and column index_set objects.

**Parameters**

| | |
|---|---|
| *row_index_set* | the row index set containing the set of rows to be in the submatrix. |
| *column_index_set* | the col index set containing the set of columns to be in the submatrix. |

**Returns**

A new CSR matrix with the elements that belong to the row and columns of this matrix as specified by the index sets.

**Note**

This is not a view but creates a new, separate CSR matrix.

#### 48.50.3.14 create_submatrix() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr<ValueType, IndexType> > gko::matrix::Csr< ValueType, IndexType >::create↩
_submatrix (
            const span & row_span,
            const span & column_span ) const
```

Creates a submatrix from this Csr matrix given row and column spans.

**Parameters**

| | |
|---|---|
| *row_span* | the row span containing the contiguous set of rows to be in the submatrix. |
| *column_span* | the column span containing the contiguous set of columns to be in the submatrix. |

**Returns**

A new CSR matrix with the elements that belong to the row and columns of this matrix as specified by the index sets.

**Note**

This is not a view but creates a new, separate CSR matrix.

#### 48.50.3.15 extract_diagonal()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Csr< ValueType, IndexType >::extract_↩
diagonal ( ) const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

#### 48.50.3.16 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Csr< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

```
850 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.50.3.17 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Csr< ValueType, IndexType >::get_const_col_idxs ( ) const  [inline],
[noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
860     {
861         return col_idxs_.get_const_data();
862     }
```

References gko::array< ValueType >::get_const_data().

#### 48.50.3.18 get_const_device_view()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const_device_view gko::matrix::Csr< ValueType, IndexType >::get_const_device_view ( ) const
```

Returns a non-owning const device view of this matrix.

**Returns**

a const device view of this matrix.

#### 48.50.3.19 get_const_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Csr< ValueType, IndexType >::get_const_row_ptrs ( ) const  [inline],
[noexcept]
```

Returns the row pointers of the matrix.

**Returns**

the row pointers of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
879     {
880         return row_ptrs_.get_const_data();
881     }
```

References gko::array< ValueType >::get_const_data().

#### 48.50.3.20 get_const_srow()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Csr< ValueType, IndexType >::get_const_srow ( ) const  [inline],
[noexcept]
```

Returns the starting rows.

**Returns**

the starting rows.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
898     {
899         return srow_.get_const_data();
900     }
```

References gko::array< ValueType >::get_const_data().

#### 48.50.3.21 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Csr< ValueType, IndexType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
829     {
830         return values_.get_const_data();
831     }
```

References gko::array< ValueType >::get_const_data().

#### 48.50.3.22 get_device_view()

```
template<typename ValueType = default_precision, typename IndexType = int32>
device_view gko::matrix::Csr< ValueType, IndexType >::get_device_view ( )
```

Returns a non-owning device view of this matrix.

**Returns**

a device view of this matrix.

#### 48.50.3.23 get_num_srow_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Csr< ValueType, IndexType >::get_num_srow_elements ( ) const  [inline],
[noexcept]
```

Returns the number of the srow stored elements (involved warps)

**Returns**

the number of the srow stored elements (involved warps)

```
908     {
909         return srow_.get_size();
910     }
```

References gko::array< ValueType >::get_size().

#### 48.50.3.24 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Csr< ValueType, IndexType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
918     {
919         return values_.get_size();
920     }
```

References gko::array< ValueType >::get_size().

#### 48.50.3.25 get_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Csr< ValueType, IndexType >::get_row_ptrs ( )  [inline], [noexcept]
```

Returns the row pointers of the matrix.

**Returns**

the row pointers of the matrix.

```
869 { return row_ptrs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.50.3.26 get_srow()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Csr< ValueType, IndexType >::get_srow ( )  [inline], [noexcept]
```

Returns the starting rows.

**Returns**

the starting rows.

```
888 { return srow_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.50.3.27 get_strategy()

```
template<typename ValueType = default_precision, typename IndexType = int32>
csr::spmv_strategy gko::matrix::Csr< ValueType, IndexType >::get_strategy ( ) const  [noexcept]
```

Returns the strategy.

**Returns**

the strategy

#### 48.50.3.28 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Csr< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

```
819 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.50.3.29 inv_scale()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::inv_scale (
            ptr_param< const LinOp > alpha )  [inline]
```

Scales the matrix with the inverse of a scalar.

**Parameters**

| | |
|---|---|
| *alpha* | The entire matrix is scaled by 1 / alpha. alpha has to be a 1x1 Dense matrix. |

```
960     {
961         auto exec = this->get_executor();
962         GKO_ASSERT_EQUAL_DIMENSIONS(alpha, dim<2>(1, 1));
963         this->inv_scale_impl(make_temporary_clone(exec, alpha).get());
964     }
```

References gko::PolymorphicObject::get_executor(), and gko::make_temporary_clone().

#### 48.50.3.30 inverse_column_permute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
```

```
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::inverse_column_permute (
            const array< IndexType > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the column `perm[i]` contains the input column `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implements gko::Permutable< IndexType >.

#### 48.50.3.31 inverse_permute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::inverse_permute (
            const array< IndexType > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(perm[i],perm[j])` contains the input value `(i,j)`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented from gko::Permutable< IndexType >.

#### 48.50.3.32 inverse_row_permute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::inverse_row_permute (
            const array< IndexType > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the row `perm[i]` contains the input row `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}A$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implements gko::Permutable< IndexType >.

#### 48.50.3.33 multiply()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::multiply (
            ptr_param< const Csr< ValueType, IndexType > > other ) const
```

Computes the sparse matrix product `this * other` on the executor of this matrix.

**Parameters**

| | |
|---|---|
| *other* | the matrix with which the product will be computed. It needs to be sorted by column indices when using OmpExecutor or DpcppExecutor for `this`. |

**Returns**

the product of the two matrices, stored on the same executor as this matrix.

#### 48.50.3.34 multiply_add()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::multiply_add (
            ptr_param< const Dense< value_type >> scale_mult,
            ptr_param< const Csr< ValueType, IndexType > > mtx_mult,
            ptr_param< const Dense< value_type >> scale_add,
            ptr_param< const Csr< ValueType, IndexType > > mtx_add ) const
```

Computes the sparse matrix product `scale_mult * this * mtx_mult + scale_add * mtx_add` on the executor of this matrix.

**Parameters**

| | |
|---|---|
| *scale_mult* | the scalar by which the matrix product will be scaled. |
| *mtx_mult* | the matrix with which the product will be computed. It needs to be sorted by column indices when using OmpExecutor or DpcppExecutor for `this`. |
| *scale_add* | the scalar by which the matrix mtx_add will be scaled. |
| *mtx_add* | the matrix which will be added to the product, scaled by scale_add. |

**Returns**

the result of the computation, stored on the same executor as this matrix.

### 48.50.3.35 multiply_add_reuse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::pair<std::unique_ptr<Csr>, multiply_add_reuse_info> gko::matrix::Csr< ValueType, Index↩
Type >::multiply_add_reuse (
            ptr_param< const Dense< value_type >> scale_mult,
            ptr_param< const Csr< ValueType, IndexType > > mtx_mult,
            ptr_param< const Dense< value_type >> scale_add,
            ptr_param< const Csr< ValueType, IndexType > > mtx_add ) const
```

Computes the sparse matrix product `scale_mult * this * mtx_mult + scale_add * mtx_add` on the executor of this matrix, and necessary data for value updates:

```
auto [result, reuse] = mtx->multiply_add_reuse(sm, mm, sa, ma);
change_values(mtx, sm, mm, sa, ma);
reuse->update_values(mtx, sm, mm, sa, ma, result);
```

**Parameters**

| | |
|---|---|
| *scale_mult* | the scalar by which the matrix product will be scaled. |
| *mtx_mult* | the matrix with which the product will be computed. It needs to be sorted by column indices when using OmpExecutor or DpcppExecutor for `this`. |
| *scale_add* | the scalar by which the matrix mtx_add will be scaled. |
| *mtx_add* | the matrix which will be added to the product, scaled by scale_add. |

**Returns**

std::pair containing the result of the computation, stored on the same executor as this matrix, and a multiply_add_reuse_info object allowing value updates to the output matrix.

### 48.50.3.36 multiply_reuse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::pair<std::unique_ptr<Csr>, multiply_reuse_info> gko::matrix::Csr< ValueType, IndexType
```

```
>::multiply_reuse (
            ptr_param< const Csr< ValueType, IndexType > > other ) const
```

Computes the sparse matrix product `this * other` on the executor of this matrix, and necessary data for value updates:

```
auto [C, reuse] = A->multiply_reuse(B);
change_values(A, B);
reuse->update_values(A, B, C);
```

**Parameters**

| | |
|---|---|
| *other* | the matrix with which the product will be computed. It needs to be sorted by column indices when using OmpExecutor or DpcppExecutor for `this`. |

**Returns**

std::pair containing the product of the two matrices, stored on the same executor as this matrix, and a multiply_reuse_info object allowing value updates to the output matrix.

#### 48.50.3.37 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Csr& gko::matrix::Csr< ValueType, IndexType >::operator= (
            const Csr< ValueType, IndexType > &  )
```

Copy-assigns a Csr matrix.

Preserves executor, copies everything else.

#### 48.50.3.38 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Csr& gko::matrix::Csr< ValueType, IndexType >::operator= (
            Csr< ValueType, IndexType > &&  )
```

Move-assigns a Csr matrix.

Preserves executor, moves the data and leaves the moved-from object in an empty state (0x0 LinOp with unchanged executor and strategy, no nonzeros and valid row pointers).

#### 48.50.3.39 permute() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::permute (
            const array< IndexType > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the symmetric row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(i,j)` contains the input value `(perm[i],perm[j])`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PAP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented from gko::Permutable< IndexType >.

#### 48.50.3.40 permute() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::permute (
            ptr_param< const Permutation< index_type >> permutation,
            permute_mode mode = permute_mode::symmetric ) const
```

Creates a permuted copy $A$' of this matrix $A$ with the given permutation $P$.

By default, this computes a symmetric permutation (permute_mode::symmetric). For the effect of the different permutation modes, see permute_mode.

**Parameters**

| | |
|---|---|
| *permutation* | The input permutation. |
| *mode* | The permutation mode, see permute_mode. |

**Returns**

The permuted matrix.

#### 48.50.3.41 permute() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::permute (
            ptr_param< const Permutation< index_type >> row_permutation,
            ptr_param< const Permutation< index_type >> column_permutation,
            bool invert = false ) const
```

Creates a non-symmetrically permuted copy $A$' of this matrix $A$ with the given row and column permutations $P$ and $Q$.

The operation will compute $A'(i,j) = A(p[i], q[j])$, or $A' = PAQ^T$ if `invert` is `false`, and $A'(p[i], q[j]) = A(i,j)$, or $A' = P^{-1}AQ^{-T}$ if `invert` is `true`.

**Parameters**

| | |
|---|---|
| *row_permutation* | The permutation $P$ to apply to the rows |
| *column_permutation* | The permutation $Q$ to apply to the columns |
| *invert* | If set to `false`, uses the input permutations, otherwise uses their inverses $P^{-1}, Q^{-1}$ |

**Returns**

The permuted matrix.

### 48.50.3.42 permute_reuse() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::pair<std::unique_ptr<Csr>, permuting_reuse_info> gko::matrix::Csr< ValueType, IndexType
>::permute_reuse (
            ptr_param< const Permutation< index_type >> permutation,
            permute_mode mode = permute_mode::symmetric ) const
```

Computes the operations necessary to propagate changed values from a matrix A to a permuted matrix.

The semantics of this function match those of permute(ptr_param<const Permutation<index_type>>, permute↩ _mode). Updating values works as follows:

```
auto [permuted, reuse] = matrix->permute_reuse(permutation, mode);
change_values(matrix);
reuse->update_values(matrix, permuted);
```

**Parameters**

| | |
|---|---|
| *permutation* | The input permutation. |
| *mode* | The permutation mode, see permute_mode. |

**Returns**

an std::pair consisting of the permuted matrix and the reuse info that can be used to update values in the permuted matrix.

### 48.50.3.43 permute_reuse() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::pair<std::unique_ptr<Csr>, permuting_reuse_info> gko::matrix::Csr< ValueType, IndexType
>::permute_reuse (
            ptr_param< const Permutation< index_type >> row_permutation,
            ptr_param< const Permutation< index_type >> column_permutation,
            bool invert = false ) const
```

Computes the operations necessary to propagate changed values from a matrix A to a permuted matrix.

The semantics of this function match those of permute(ptr_param<const Permutation<index_type>>, ptr_param<const Permutation<index_type>>, bool). Updating values works as follows:

```
auto [permuted, reuse] = matrix->permute_reuse(row_perm, col_perm, inv);
change_values(matrix);
reuse->update_values(matrix, permuted);
```

**Parameters**

| | |
|---|---|
| *row_permutation* | The permutation $P$ to apply to the rows |
| *column_permutation* | The permutation $Q$ to apply to the columns |
| *invert* | If set to `false`, uses the input permutations, otherwise uses their inverses $P^{-1}, Q^{-1}$ |

**Returns**

an std::pair consisting of the permuted matrix and the reuse info that can be used to update values in the permuted matrix.

#### 48.50.3.44 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.50.3.45 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.50.3.46 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.50.3.47 row_permute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::row_permute (
            const array< IndexType > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the Permutable object.

In the resulting LinOp, the row `i` contains the input row `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PA$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Implements gko::Permutable< IndexType >.

#### 48.50.3.48 scale()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::scale (
            ptr_param< const LinOp > alpha )  [inline]
```

Scales the matrix with a scalar.

**Parameters**

| | |
|---|---|
| *alpha* | The entire matrix is scaled by alpha. alpha has to be a 1x1 Dense matrix. |

```
947     {
948         auto exec = this->get_executor();
949         GKO_ASSERT_EQUAL_DIMENSIONS(alpha, dim<2>(1, 1));
```

```
950         this->scale_impl(make_temporary_clone(exec, alpha).get());
951     }
```

References gko::PolymorphicObject::get_executor(), and gko::make_temporary_clone().

#### 48.50.3.49 scale_add()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::scale_add (
             ptr_param< const Dense< value_type >> scale_this,
             ptr_param< const Dense< value_type >> scale_other,
             ptr_param< const Csr< ValueType, IndexType > > mtx_other ) const
```

Computes the sparse matrix sum `scale_this * this + scale_other * mtx_add` on the executor of this matrix.

This matrix needs to be sorted by column index, otherwise the result will be incorrect.

**Parameters**

| | |
|---|---|
| *scale_this* | the scalar by which this matrix will be scaled. |
| *scale_other* | the scalar by which this matrix will be scaled. |
| *mtx_other* | the matrix which will be added to this, scaled by scale_other. It needs to be sorted by column index, otherwise the result will be incorrect. |

**Returns**

the result of the computation, stored on the same executor as this matrix.

#### 48.50.3.50 scale_permute() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::scale_permute (
             ptr_param< const ScaledPermutation< value_type, index_type >> permutation,
             permute_mode  = permute_mode::symmetric ) const
```

Creates a scaled and permuted copy of this matrix.

For an explanation of the permutation modes, see permute(ptr_param<const Permutation<index_type>>, permute_mode)

**Parameters**

| | |
|---|---|
| *permutation* | The scaled permutation. |
| *mode* | The permutation mode. |

**Returns**

The permuted matrix.

#### 48.50.3.51 scale_permute() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Csr> gko::matrix::Csr< ValueType, IndexType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, index_type >> row_permutation,
            ptr_param< const ScaledPermutation< value_type, index_type >> column_permutation,
            bool invert = false ) const
```

Creates a scaled and permuted copy of this matrix.

For an explanation of the parameters, see permute(ptr_param<const Permutation<index_type>>, ptr_param<const Permutation<index_type>>, permute_mode)

**Parameters**

| | |
|---|---|
| *row_permutation* | The scaled row permutation. |
| *column_permutation* | The scaled column permutation. |
| *invert* | If set to `false`, uses the input permutations, otherwise uses their inverses $P^{-1}, Q^{-1}$ |

**Returns**

The permuted matrix.

#### 48.50.3.52 set_strategy()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::set_strategy (
            csr::spmv_strategy strategy )  [inline]
```

Set the strategy.

**Parameters**

| | |
|---|---|
| *strategy* | the csr strategy |

```
935     {
936         strategy_ = strategy;
937         this->make_srow();
938     }
```

#### 48.50.3.53 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Csr< ValueType, IndexType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.50.3.54 transpose_reuse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::pair<std::unique_ptr<Csr>, permuting_reuse_info> gko::matrix::Csr< ValueType, IndexType
>::transpose_reuse ( ) const
```

Computes the necessary data to update a transposed matrix from its original matrix.

```
auto [transposed, reuse] = matrix->transpose_reuse();
change_values(matrix);
reuse->update_values(matrix, transposed);
```

**Returns**

an std::pair consisting of the transposed matrix and a reuse info struct that can be used to update values in the transposed matrix.

#### 48.50.3.55 write() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::write (
            device_mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a device_matrix_data structure.

The entries are written on the executor of the output data. The default implementation goes through the host; formats that can produce the entries on the executor should override this.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure |

Reimplemented from gko::WritableToMatrixData< ValueType, IndexType >.

#### 48.50.3.56 write() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following files:

- ginkgo/core/distributed/matrix.hpp
- ginkgo/core/matrix/csr.hpp

## 48.51 gko::batch::matrix::Csr< ValueType, IndexType > Class Template Reference

Csr is a general sparse matrix format that stores the column indices for each nonzero entry and a cumulative sum of the number of nonzeros in each row.

```
#include <ginkgo/core/matrix/batch_csr.hpp>
```

### Public Member Functions

- std::unique_ptr< unbatch_type > create_view_for_item (size_type item_id)

  *Creates a mutable view (of matrix::Csr type) of one item of the batch::matrix::Csr<value_type> object.*
- std::unique_ptr< const unbatch_type > create_const_view_for_item (size_type item_id) const

  *Creates a mutable view (of matrix::Csr type) of one item of the batch::matrix::Csr<value_type> object.*
- value_type * get_values () noexcept

  *Returns a pointer to the array of values of the matrix.*
- const value_type * get_const_values () const noexcept

  *Returns a pointer to the array of values of the matrix.*
- index_type * get_col_idxs () noexcept

  *Returns a pointer to the array of column indices of the matrix.*
- const index_type * get_const_col_idxs () const noexcept

  *Returns a pointer to the array of column indices of the matrix.*
- index_type * get_row_ptrs () noexcept

  *Returns a pointer to the array of row pointers of the matrix.*
- const index_type * get_const_row_ptrs () const noexcept

  *Returns a pointer to the array of row pointers of the matrix.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.*
- size_type get_num_elements_per_item () const noexcept

*Returns the number of stored elements in each batch item.*

- value_type * get_values_for_item (size_type batch_id) noexcept

  *Returns a pointer to the array of values of the matrix for a specific batch item.*
- const value_type * get_const_values_for_item (size_type batch_id) const noexcept

  *Returns a pointer to the array of values of the matrix for a specific batch item.*
- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector.*
- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector with a linear combination of the given input vector.*
- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x) const
- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x) const
- void scale (const array< value_type > &row_scale, const array< value_type > &col_scale)

  *Performs in-place row and column scaling for this matrix.*
- void add_scaled_identity (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> beta)

  *Performs the operation this = alpha*I + beta*this.*

## Static Public Member Functions

- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size=batch_dim< 2 >{}, size_type num_nonzeros_per_item={})

  *Creates an uninitialized Csr matrix of the specified size.*
- static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, array< value_type > values, array< index_type > col_idxs, array< index_type > row_ptrs)

  *Creates a Csr matrix from an already allocated (and initialized) array.*
- template<typename InputValueType , typename ColIndexType , typename RowPtrType >
  static std::unique_ptr< Csr > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, std::initializer_list< InputValueType > values, std::initializer_list< ColIndexType > col_idxs, std::initializer_list< RowPtrType > row_ptrs)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Csr > create_const (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &sizes, gko::detail::const_array_view< value_type > &&values, gko::detail::const_array_view< index_type > &&col_idxs, gko::detail::const_array_view< index_type > &&row_ptrs)

  *Creates a constant (immutable) batch csr matrix from a constant array.*

### 48.51.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::batch::matrix::Csr< ValueType, IndexType >**

Csr is a general sparse matrix format that stores the column indices for each nonzero entry and a cumulative sum of the number of nonzeros in each row.

It is one of the most popular sparse matrix formats due to its versatility and ability to store a wide range of sparsity patterns in an efficient fashion.

**Note**

It is assumed that the sparsity pattern of all the items in the batch is the same and therefore only a single copy of the sparsity pattern is stored.

Currently only IndexType of int32 is supported.

**Template Parameters**

| | |
|---|---|
| *ValueType* | value precision of matrix elements |
| *IndexType* | index precision of matrix elements |

## 48.51.2 Member Function Documentation

### 48.51.2.1 add_scaled_identity()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Csr< ValueType, IndexType >::add_scaled_identity (
            ptr_param< const MultiVector< value_type >> alpha,
            ptr_param< const MultiVector< value_type >> beta )
```

Performs the operation this = alpha∗I + beta∗this.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar for identity |
| *beta* | the scalar to multiply this matrix |

**Note**

Performs the operation in-place for this batch matrix

This operation fails in case this matrix does not have all its diagonal entries.

### 48.51.2.2 apply() [1/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Csr< ValueType, IndexType >::apply (
            ptr_param< const MultiVector< value_type >> alpha,
            ptr_param< const MultiVector< value_type >> b,
            ptr_param< const MultiVector< value_type >> beta,
            ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector with a linear combination of the given input vector.

Represents the matrix vector multiplication, x = alpha ∗ A

- b + beta ∗ x, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar to scale the matrix-vector product with |
| *b* | the multi-vector to be applied to |
| *beta* | the scalar to scale the x vector with |
| *x* | the output multi-vector |

#### 48.51.2.3 apply() [2/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Csr< ValueType, IndexType >::apply (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< const MultiVector< value_type >> beta,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.51.2.4 apply() [3/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Csr< ValueType, IndexType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector.

Represents the matrix vector multiplication, x = A $*$ b, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *b* | the multi-vector to be applied to |
| *x* | the output multi-vector |

#### 48.51.2.5 apply() [4/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Csr< ValueType, IndexType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.51.2.6 create() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::batch::matrix::Csr< ValueType, IndexType >::create (
             std::shared_ptr< const Executor > exec,
             const batch_dim< 2 > & size,
             array< value_type > values,
             array< index_type > col_idxs,
             array< index_type > row_ptrs )  [static]
```

Creates a Csr matrix from an already allocated (and initialized) array.

The column indices array needs to be the same for all batch items.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *values* | array of matrix values |
| *col_idxs* | the col_idxs array of a single batch item of the matrix. |
| *row_ptrs* | the row_ptrs array of a single batch item of the matrix. |

**Note**

If `values` is not an rvalue, not an array of ValueType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

#### 48.51.2.7 create() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename InputValueType , typename ColIndexType , typename RowPtrType >
static std::unique_ptr<Csr> gko::batch::matrix::Csr< ValueType, IndexType >::create (
             std::shared_ptr< const Executor > exec,
             const batch_dim< 2 > & size,
             std::initializer_list< InputValueType > values,
             std::initializer_list< ColIndexType > col_idxs,
             std::initializer_list< RowPtrType > row_ptrs )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const batch_dim<2>&, array<value_type>, array<index_type>, array<index_type>)

```
290     {
291         return create(exec, size, array<value_type>{exec, std::move(values)},
292                       array<index_type>{exec, std::move(col_idxs)},
293                       array<index_type>{exec, std::move(row_ptrs)});
294     }
```

References gko::batch::matrix::Csr< ValueType, IndexType >::create().

#### 48.51.2.8 create() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Csr> gko::batch::matrix::Csr< ValueType, IndexType >::create (
              std::shared_ptr< const Executor > exec,
              const batch_dim< 2 > & size = batch_dim< 2 >{},
              size_type num_nonzeros_per_item = {} )  [static]
```

Creates an uninitialized Csr matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_nonzeros_per_item* | number of nonzeros in each item of the batch matrix |

Referenced by gko::batch::matrix::Csr< ValueType, IndexType >::create().

#### 48.51.2.9 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Csr> gko::batch::matrix::Csr< ValueType, IndexType >::create_↩
const (
              std::shared_ptr< const Executor > exec,
              const batch_dim< 2 > & sizes,
              gko::detail::const_array_view< value_type > && values,
              gko::detail::const_array_view< index_type > && col_idxs,
              gko::detail::const_array_view< index_type > && row_ptrs )  [static]
```

Creates a constant (immutable) batch csr matrix from a constant array.

Only a single sparsity pattern (column indices and row pointers) is stored and hence the user needs to ensure that each batch item has the same sparsity pattern.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *sizes* | the dimensions of the matrix |
| *values* | the value array of the matrix |
| *col_idxs* | the col_idxs array of a single batch item of the matrix. |
| *row_ptrs* | the row_ptrs array of a single batch item of the matrix. |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

A smart pointer to the newly created matrix.

#### 48.51.2.10 create_const_view_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<const unbatch_type> gko::batch::matrix::Csr< ValueType, IndexType >::create↩
_const_view_for_item (
            size_type item_id ) const
```

Creates a mutable view (of matrix::Csr type) of one item of the batch::matrix::Csr<value_type> object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a batch::matrix::Csr object with the data from the batch item at the given index.

#### 48.51.2.11 create_view_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<unbatch_type> gko::batch::matrix::Csr< ValueType, IndexType >::create_view_↩
for_item (
            size_type item_id )
```

Creates a mutable view (of matrix::Csr type) of one item of the batch::matrix::Csr<value_type> object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a batch::matrix::Csr object with the data from the batch item at the given index.

#### 48.51.2.12 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

Returns a pointer to the array of column indices of the matrix.

**Returns**

the pointer to the array of column indices

```
144 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.51.2.13 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_const_col_idxs ( )
const  [inline], [noexcept]
```

Returns a pointer to the array of column indices of the matrix.

**Returns**

the pointer to the array of column indices

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
154     {
155         return col_idxs_.get_const_data();
156     }
```

References gko::array< ValueType >::get_const_data().

#### 48.51.2.14 get_const_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_const_row_ptrs ( )
const  [inline], [noexcept]
```

Returns a pointer to the array of row pointers of the matrix.

**Returns**

the pointer to the array of row pointers

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
173     {
174         return row_ptrs_.get_const_data();
175     }
```

References gko::array< ValueType >::get_const_data().

### 48.51.2.15 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_const_values ( ) const
[inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

#### Returns

the pointer to the array of values

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
135     {
136         return values_.get_const_data();
137     }
```

References gko::array< ValueType >::get_const_data().

### 48.51.2.16 get_const_values_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_const_values_for_item (
              size_type batch_id ) const  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix for a specific batch item.

#### Parameters

| | |
|---|---|
| *batch_id* | the id of the batch item. |

#### Returns

the pointer to the array of values

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
225     {
226         GKO_ASSERT(batch_id < this->get_num_batch_items());
227         GKO_ASSERT(values_.get_size() >=
228                    batch_id * this->get_num_elements_per_item());
229         return values_.get_const_data() +
230                batch_id * this->get_num_elements_per_item();
231     }
```

References gko::array< ValueType >::get_const_data(), gko::batch::matrix::Csr< ValueType, IndexType >::get_num_elements_per_item(), and gko::array< ValueType >::get_size().

#### 48.51.2.17 get_num_elements_per_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::batch::matrix::Csr< ValueType, IndexType >::get_num_elements_per_item ( ) const
[inline], [noexcept]
```

Returns the number of stored elements in each batch item.

**Returns**

the number of stored elements per batch item.

```
195     {
196         return this->get_num_stored_elements() / this->get_num_batch_items();
197     }
```

References gko::batch::matrix::Csr< ValueType, IndexType >::get_num_stored_elements().

Referenced by gko::batch::matrix::Csr< ValueType, IndexType >::get_const_values_for_item(), and gko::batch↩ ::matrix::Csr< ValueType, IndexType >::get_values_for_item().

#### 48.51.2.18 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::batch::matrix::Csr< ValueType, IndexType >::get_num_stored_elements ( ) const
[inline], [noexcept]
```

Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.

**Returns**

the number of elements explicitly stored in the vector, cumulative across all the batch items

```
185     {
186         return values_.get_size();
187     }
```

References gko::array< ValueType >::get_size().

Referenced by gko::batch::matrix::Csr< ValueType, IndexType >::get_num_elements_per_item().

#### 48.51.2.19 get_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_row_ptrs ( )  [inline], [noexcept]
```

Returns a pointer to the array of row pointers of the matrix.

**Returns**

the pointer to the array of row pointers

```
163 { return row_ptrs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.51.2.20 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

```
125 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.51.2.21 get_values_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::batch::matrix::Csr< ValueType, IndexType >::get_values_for_item (
            size_type batch_id )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix for a specific batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the id of the batch item. |

**Returns**

the pointer to the array of values

```
208     {
209         GKO_ASSERT(batch_id < this->get_num_batch_items());
210         GKO_ASSERT(values_.get_size() >=
211                    batch_id * this->get_num_elements_per_item());
212         return values_.get_data() +
213                batch_id * this->get_num_elements_per_item();
214     }
```

References gko::array< ValueType >::get_data(), gko::batch::matrix::Csr< ValueType, IndexType >::get_num_elements_per_item(), and gko::array< ValueType >::get_size().

#### 48.51.2.22 scale()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Csr< ValueType, IndexType >::scale (
            const array< value_type > & row_scale,
            const array< value_type > & col_scale )
```

Performs in-place row and column scaling for this matrix.

**Parameters**

| | |
|---|---|
| *row_scale* | the row scalars |
| *col_scale* | the column scalars |

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/batch_csr.hpp

## 48.52 gko::matrix::view::csr< ValueType, IndexType > Struct Template Reference

Non-owning view of a matrix::Csr to be used inside device kernels.

`#include <ginkgo/core/matrix/device_views.hpp>`

### Public Member Functions

- constexpr csr (dim< 2 > size, size_type num_stored_elements, ValueType *values, IndexType *row_ptrs, IndexType *col_idxs)

  *Constructs a coo view from size, nnz, values, row pointers, and column indices.*
- constexpr csr< const ValueType, const IndexType > as_const () const

  *Returns a const view of the same data.*

### 48.52.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**struct gko::matrix::view::csr< ValueType, IndexType >**

Non-owning view of a matrix::Csr to be used inside device kernels.

This type is used to provide a simple and stable ABI for passing data between libraries.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store matrix entries. |
| *IndexType* | the index type used to store row and column indices. |

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/device_views.hpp

## 48.53 gko::CublasError Class Reference

CublasError is thrown when a cuBLAS routine throws a non-zero error code.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- CublasError (const std::string &file, int line, const std::string &func, int64 error_code)

  *Initializes a cuBLAS error.*

## 48.53.1 Detailed Description

CublasError is thrown when a cuBLAS routine throws a non-zero error code.

## 48.53.2 Constructor & Destructor Documentation

### 48.53.2.1 CublasError()

```
gko::CublasError::CublasError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a cuBLAS error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the cuBLAS routine that failed |
| *error_code* | The resulting cuBLAS error code |

```
199         :  Error(file, line, func + ":  " + get_error(error_code))
200     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

# 48.54 gko::cuda_stream Class Reference

An RAII wrapper for a custom CUDA stream.

```
#include <ginkgo/core/base/stream.hpp>
```

## Public Member Functions

- cuda_stream ()

  *Creates an empty stream wrapper, representing the default stream.*
- cuda_stream (int device_id, bool non_blocking=false)

  *Creates a new custom CUDA stream on the given device.*
- ~cuda_stream ()

  *Destroys the custom CUDA stream, if it isn't empty.*
- cuda_stream (cuda_stream &&) noexcept

  *Move-constructs from an existing stream, which will be emptied.*
- cuda_stream & operator= (cuda_stream &&)=delete

  *Move-assigns from an existing stream, which will be emptied.*
- CUstream_st * get () const

  *Returns the native CUDA stream handle.*

### 48.54.1 Detailed Description

An RAII wrapper for a custom CUDA stream.

The stream will be created on construction and destroyed when the lifetime of the wrapper ends.

### 48.54.2 Constructor & Destructor Documentation

#### 48.54.2.1 cuda_stream()

```
gko::cuda_stream::cuda_stream (
            int device_id,
            bool non_blocking = false )
```

Creates a new custom CUDA stream on the given device.

**Parameters**

| | |
|---|---|
| *device_id* | the device ID to create the stream on. |
| *non_blocking* | create the stream with cudaStreamNonBlocking, so that it does not implicitly order against the legacy default stream. |

### 48.54.3 Member Function Documentation

#### 48.54.3.1 get()

```
CUstream_st* gko::cuda_stream::get ( ) const
```

Returns the native CUDA stream handle.

In an empty cuda_stream, this will return nullptr.

The documentation for this class was generated from the following file:

- ginkgo/core/base/stream.hpp

## 48.55 gko::CudaAllocator Class Reference

Allocator using cudaMalloc.

`#include <ginkgo/core/base/memory.hpp>`

### 48.55.1 Detailed Description

Allocator using cudaMalloc.

The documentation for this class was generated from the following file:

- ginkgo/core/base/memory.hpp

## 48.56 gko::CudaAllocatorBase Class Reference

Implement this interface to provide an allocator for CudaExecutor.

`#include <ginkgo/core/base/memory.hpp>`

### 48.56.1 Detailed Description

Implement this interface to provide an allocator for CudaExecutor.

The documentation for this class was generated from the following file:

- ginkgo/core/base/memory.hpp

## 48.57 gko::CudaError Class Reference

CudaError is thrown when a CUDA routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- CudaError (const std::string &file, int line, const std::string &func, int64 error_code)
  *Initializes a CUDA error.*

## 48.57.1 Detailed Description

CudaError is thrown when a CUDA routine throws a non-zero error code.

## 48.57.2 Constructor & Destructor Documentation

### 48.57.2.1 CudaError()

```
gko::CudaError::CudaError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a CUDA error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the CUDA routine that failed |
| *error_code* | The resulting CUDA error code |

```
176         :  Error(file, line, func + ":  " + get_error(error_code))
177     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

# 48.58 gko::CudaExecutor Class Reference

This is the Executor subclass which represents the CUDA device.

```
#include <ginkgo/core/base/executor.hpp>
```

## Public Member Functions

- std::shared_ptr< Executor > get_master () noexcept override

  *Returns the master OmpExecutor of this Executor.*
- std::shared_ptr< const Executor > get_master () const noexcept override

  *Returns the master OmpExecutor of this Executor.*
- void synchronize () const override

  *Synchronize the operations launched on the executor with its master.*
- std::string get_description () const override
- int get_device_id () const noexcept

  *Get the CUDA device id of the device associated to this executor.*
- int get_num_warps_per_sm () const noexcept

  *Get the number of warps per SM of this executor.*
- int get_num_multiprocessor () const noexcept

  *Get the number of multiprocessor of this executor.*
- int get_num_warps () const noexcept

  *Get the number of warps of this executor.*
- int get_warp_size () const noexcept

  *Get the warp size of this executor.*
- int get_major_version () const noexcept

  *Get the major version of compute capability.*
- int get_minor_version () const noexcept

  *Get the minor version of compute capability.*
- int get_compute_capability () const noexcept

  *Get the compute capability.*
- cublasContext * get_cublas_handle () const

  *Get the cublas handle for this executor.*
- cublasContext * get_blas_handle () const

  *Get the cublas handle for this executor.*
- cusparseContext * get_cusparse_handle () const

  *Get the cusparse handle for this executor.*
- cusparseContext * get_sparselib_handle () const

  *Get the cusparse handle for this executor.*
- CUstream_st * get_stream () const

  *Returns the CUDA stream used by this executor.*
- virtual void run (const Operation &op) const=0

  *Runs the specified Operation using this Executor.*
- template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*
- template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
  Dpcpp >
  void run (std::string name, const ClosureReference &op_ref, const ClosureOmp &op_omp, const Closure↩
  Cuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

## Static Public Member Functions

- static std::shared_ptr< CudaExecutor > create (int device_id, std::shared_ptr< Executor > master, bool device_reset, allocation_mode alloc_mode=default_cuda_alloc_mode, CUstream_st *stream=nullptr)

  *Creates a new CudaExecutor.*
- static std::shared_ptr< CudaExecutor > create (int device_id, std::shared_ptr< Executor > master, std::shared_ptr< CudaAllocatorBase > alloc=std::make_shared< CudaAllocator >(), CUstream_st *stream=nullptr)

  *Creates a new CudaExecutor with a custom allocator and device stream.*
- static int get_num_devices ()

  *Get the number of devices present on the system.*

### 48.58.1 Detailed Description

This is the Executor subclass which represents the CUDA device.

### 48.58.2 Member Function Documentation

#### 48.58.2.1 create() [1/2]

```
static std::shared_ptr<CudaExecutor> gko::CudaExecutor::create (
            int device_id,
            std::shared_ptr< Executor > master,
            bool device_reset,
            allocation_mode alloc_mode = default_cuda_alloc_mode,
            CUstream_st * stream = nullptr )  [static]
```

Creates a new CudaExecutor.

**Parameters**

| | |
|---|---|
| *device_id* | the CUDA device id of this device |
| *master* | an executor on the host that is used to invoke the device kernels |
| *device_reset* | this option no longer has any effect. |
| *alloc_mode* | the allocation mode that the executor should operate on. See @allocation_mode for more details |
| *stream* | the stream to execute operations on. |

#### 48.58.2.2 create() [2/2]

```
static std::shared_ptr<CudaExecutor> gko::CudaExecutor::create (
            int device_id,
            std::shared_ptr< Executor > master,
            std::shared_ptr< CudaAllocatorBase > alloc = std::make_shared< CudaAllocator >(),
            CUstream_st * stream = nullptr )  [static]
```

Creates a new CudaExecutor with a custom allocator and device stream.

**Parameters**

| | |
|---|---|
| *device↩ _id* | the CUDA device id of this device |
| *master* | an executor on the host that is used to invoke the device kernels. |
| *alloc* | the allocator to use for device memory allocations. |
| *stream* | the stream to execute operations on. |

### 48.58.2.3 get_blas_handle()

```
cublasContext* gko::CudaExecutor::get_blas_handle ( ) const  [inline]
```

Get the cublas handle for this executor.

**Returns**

the cublas handle (cublasContext*) for this executor

```
1661 { return cublas_handle_.get(); }
```

### 48.58.2.4 get_cublas_handle()

```
cublasContext* gko::CudaExecutor::get_cublas_handle ( ) const  [inline]
```

Get the cublas handle for this executor.

**Returns**

the cublas handle (cublasContext*) for this executor

```
1656 { return get_blas_handle(); }
```

### 48.58.2.5 get_cusparse_handle()

```
cusparseContext* gko::CudaExecutor::get_cusparse_handle ( ) const  [inline]
```

Get the cusparse handle for this executor.

**Returns**

the cusparse handle (cusparseContext*) for this executor

```
1670     {
1671         return get_sparselib_handle();
1672     }
```

#### 48.58.2.6 get_description()

```
std::string gko::CudaExecutor::get_description ( ) const  [override], [virtual]
```

**Returns**

a textual representation of the executor and its device.

Implements gko::Executor.

#### 48.58.2.7 get_master() [1/2]

```
std::shared_ptr<const Executor> gko::CudaExecutor::get_master ( ) const  [override], [virtual],
[noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

#### 48.58.2.8 get_master() [2/2]

```
std::shared_ptr<Executor> gko::CudaExecutor::get_master ( )  [override], [virtual], [noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

#### 48.58.2.9 get_sparselib_handle()

```
cusparseContext* gko::CudaExecutor::get_sparselib_handle ( ) const  [inline]
```

Get the cusparse handle for this executor.

**Returns**

the cusparse handle (cusparseContext*) for this executor

```
1678     {
1679         return cusparse_handle_.get();
1680     }
```

#### 48.58.2.10 get_stream()

```
CUstream_st* gko::CudaExecutor::get_stream ( ) const  [inline]
```

Returns the CUDA stream used by this executor.

Can be nullptr for the default stream.

**Returns**

the stream used to execute kernels and memory operations.

```
1688 { return stream_; }
```

#### 48.58.2.11 run() [1/3]

```
template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
Dpcpp >
void gko::Executor::run (
            typename ClosureOmp ,
            typename ClosureCuda ,
            typename ClosureHip ,
            typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *op_omp* | functor to run in case of a OmpExecutor or ReferenceExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
660     {
661         LambdaOperation<ClosureOmp, ClosureOmp, ClosureCuda, ClosureHip,
662                         ClosureDpcpp>
663             op(op_omp, op_cuda, op_hip, op_dpcpp);
664         this->run(op);
665     }
```

#### 48.58.2.12 run() [2/3]

```
virtual void gko::Executor::run
```

Runs the specified Operation using this Executor.

**Parameters**

| | |
|---|---|
| *op* | the operation to run |

#### 48.58.2.13 run() [3/3]

```
template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename
ClosureHip , typename ClosureDpcpp >
void gko::Executor::run (
            typename ClosureReference ,
            typename ClosureOmp ,
            typename ClosureCuda ,
            typename ClosureHip ,
            typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureReference* | type of op_ref |
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *name* | the name of the operation |
| *op_ref* | functor to run in case of a ReferenceExecutor |
| *op_omp* | functor to run in case of a OmpExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
688     {
689         LambdaOperation<ClosureReference, ClosureOmp, ClosureCuda, ClosureHip,
690                         ClosureDpcpp>
691             op(std::move(name), op_ref, op_omp, op_cuda, op_hip, op_dpcpp);
692         this->run(op);
693     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

## 48.59 gko::CudaTimer Class Reference

A timer using events for timing on a CudaExecutor.

`#include <ginkgo/core/base/timer.hpp>`

## Public Member Functions

- void record (time_point &time) override

  *Records a time point at the current time.*
- void wait (time_point &time) override

  *Waits until all kernels in-process when recording the time point are finished.*
- std::chrono::nanoseconds difference_async (const time_point &start, const time_point &stop) override

  *Computes the difference between the two time points in nanoseconds.*

## Additional Inherited Members

### 48.59.1 Detailed Description

A timer using events for timing on a CudaExecutor.

**Note**

When using a CudaExecutor with a custom stream, make sure that the stream's lifetime is longer than the lifetime of this timer.

### 48.59.2 Member Function Documentation

#### 48.59.2.1 difference_async()

```
std::chrono::nanoseconds gko::CudaTimer::difference_async (
            const time_point & start,
            const time_point & stop )  [override], [virtual]
```

Computes the difference between the two time points in nanoseconds.

This asynchronous version does not synchronize itself, so the time points need to have been synchronized with, i.e. `timer->wait(stop)` needs to have been called. The version is intended for more advanced users who want to measure the overhead of timing functionality separately.

**Parameters**

| | |
|---|---|
| *start* | the first time point (earlier) |
| *stop* | the second time point (later) |

**Returns**

the difference between the time points in nanoseconds.

Implements gko::Timer.

The documentation for this class was generated from the following file:

- ginkgo/core/base/timer.hpp

# 48.60 gko::ext::cuda::solver::Cudss< ValueType, IndexType > Class Template Reference

A direct solver using NVIDIA's cuDSS library.

`#include <ginkgo/extensions/cuda/solver/cudss.hpp>`

## Public Member Functions

- Cudss (const Cudss &)
  *Creates a copy of the solver (shares factorization state).*
- Cudss (Cudss &&) noexcept
  *Moves from the given solver, leaving it empty.*
- void refactorize (std::shared_ptr< const LinOp > new_matrix)
  *Re-run the numeric factorization with updated matrix values.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())
  *Parse parameters from a configuration property tree.*
- static config::configuration_map get_config_map ()
  *Returns a configuration_map for registering this type with a config::registry.*

### 48.60.1 Detailed Description

**template<typename ValueType, typename IndexType = int32>**
**class gko::ext::cuda::solver::Cudss< ValueType, IndexType >**

A direct solver using NVIDIA's cuDSS library.

This solver is only supported on the CudaExecutor. It wraps the cuDSS sparse direct solver, performing analysis, factorization, and solve phases. The factorization is computed during construction (generate) and reused across apply calls.

The solver is opaque — factorization data is stored internally in cuDSS-native format and cannot be extracted.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type of the system matrix and vectors |
| *IndexType* | the index type of the system matrix |

### 48.60.2 Member Function Documentation

#### 48.60.2.1 get_config_map()

```
template<typename ValueType , typename IndexType = int32>
static config::configuration_map gko::ext::cuda::solver::Cudss< ValueType, IndexType >::get_↩
config_map ( )  [static]
```

Returns a configuration_map for registering this type with a config::registry.

Users can pass this to the registry constructor to enable JSON/YAML configuration of Cudss.

#### 48.60.2.2 refactorize()

```
template<typename ValueType , typename IndexType = int32>
void gko::ext::cuda::solver::Cudss< ValueType, IndexType >::refactorize (
            std::shared_ptr< const LinOp > new_matrix )
```

Re-run the numeric factorization with updated matrix values.

The new matrix must have the same sparsity pattern (dimensions and number of non-zeros) as the matrix used in generate(). Only the numeric factorization phase is re-executed; the symbolic analysis from the initial generate() is reused.

**Parameters**

| | |
|---|---|
| *new_matrix* | the updated system matrix (same sparsity pattern) |

The documentation for this class was generated from the following file:

- ginkgo/extensions/cuda/solver/cudss.hpp

## 48.61 gko::CufftError Class Reference

CufftError is thrown when a cuFFT routine throws a non-zero error code.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- CufftError (const std::string &file, int line, const std::string &func, int64 error_code)

  *Initializes a cuFFT error.*

### 48.61.1 Detailed Description

CufftError is thrown when a cuFFT routine throws a non-zero error code.

### 48.61.2 Constructor & Destructor Documentation

#### 48.61.2.1 CufftError()

```
gko::CufftError::CufftError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a cuFFT error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the cuFFT routine that failed |
| *error_code* | The resulting cuFFT error code |

```
268         :  Error(file, line, func + ":  " + get_error(error_code))
269     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.62 gko::CurandError Class Reference

CurandError is thrown when a cuRAND routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- CurandError (const std::string &file, int line, const std::string &func, int64 error_code)

  *Initializes a cuRAND error.*

### 48.62.1 Detailed Description

CurandError is thrown when a cuRAND routine throws a non-zero error code.

### 48.62.2 Constructor & Destructor Documentation

#### 48.62.2.1 CurandError()

```
gko::CurandError::CurandError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a cuRAND error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the cuRAND routine that failed |
| *error_code* | The resulting cuRAND error code |

```
222         :  Error(file, line, func + ":  " + get_error(error_code))
223     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.63 gko::CusparseError Class Reference

CusparseError is thrown when a cuSPARSE routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- CusparseError (const std::string &file, int line, const std::string &func, int64 error_code)
  *Initializes a cuSPARSE error.*

### 48.63.1 Detailed Description

CusparseError is thrown when a cuSPARSE routine throws a non-zero error code.

### 48.63.2 Constructor & Destructor Documentation

#### 48.63.2.1 CusparseError()

```
gko::CusparseError::CusparseError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a cuSPARSE error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the cuSPARSE routine that failed |
| *error_code* | The resulting cuSPARSE error code |

```
245         :  Error(file, line, func + ":  " + get_error(error_code))
246     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.64 gko::default_converter< S, R > Struct Template Reference

Used to convert objects of type `S` to objects of type `R` using static_cast.

`#include <ginkgo/core/base/math.hpp>`

### Public Member Functions

- R operator() (S val)

  *Converts the object to result type.*

### 48.64.1 Detailed Description

**template<typename S, typename R>**
**struct gko::default_converter< S, R >**

Used to convert objects of type `S` to objects of type `R` using static_cast.

**Template Parameters**

| | |
|---|---|
| *S* | source type |
| *R* | result type |

### 48.64.2 Member Function Documentation

#### 48.64.2.1 operator()()

```
template<typename S , typename R >
R gko::default_converter< S, R >::operator() (
            S val )  [inline]
```

Converts the object to result type.

**Parameters**

| | |
|---|---|
| *val* | the object to convert |

**Returns**

the converted object

```
598 { return static_cast<R>(val); }
```

The documentation for this struct was generated from the following file:

- ginkgo/core/base/math.hpp

## 48.65 gko::deferred_factory_parameter< FactoryType > Class Template Reference

Represents a factory parameter of factory type that can either initialized by a pre-existing factory or by passing in a factory_parameters object whose `.on(exec)` will be called to instantiate a factory.

`#include <ginkgo/core/base/abstract_factory.hpp>`

### Public Member Functions

- deferred_factory_parameter ()=default
  *Creates an empty deferred factory parameter.*
- deferred_factory_parameter (std::nullptr_t)
  *Creates a deferred factory parameter returning a nullptr.*
- template<typename ConcreteFactoryType , std::enable_if_t< detail::is_pointer_convertible< ConcreteFactoryType, FactoryType >::value > * = nullptr>
  deferred_factory_parameter (std::shared_ptr< ConcreteFactoryType > factory)
  *Creates a deferred factory parameter from a preexisting factory with shared ownership.*
- template<typename ConcreteFactoryType , typename Deleter , std::enable_if_t< detail::is_pointer_convertible< ConcreteFactoryType, FactoryType >::value > * = nullptr>
  deferred_factory_parameter (std::unique_ptr< ConcreteFactoryType, Deleter > factory)
  *Creates a deferred factory parameter by taking ownership of a preexisting factory with unique ownership.*
- template<typename ParametersType , typename U = decltype(std::declval<ParametersType>().on( std::shared_ptr<const Executor>{})), std::enable_if_t< detail::is_pointer_convertible< typename U::element_type, FactoryType >::value > * = nullptr>
  deferred_factory_parameter (ParametersType parameters)
  *Creates a deferred factory parameter object from a factory_parameters-like object.*
- std::shared_ptr< FactoryType > on (std::shared_ptr< const Executor > exec) const
  *Instantiates the deferred parameter into an actual factory.*
- bool is_empty () const
  *Returns true iff the parameter is empty.*

### 48.65.1 Detailed Description

**template<typename FactoryType>**
**class gko::deferred_factory_parameter< FactoryType >**

Represents a factory parameter of factory type that can either initialized by a pre-existing factory or by passing in a factory_parameters object whose `.on(exec)` will be called to instantiate a factory.

**Template Parameters**

| | |
|---|---|
| *FactoryType* | the type of factory that can be instantiated from this object. |

## 48.65.2 Constructor & Destructor Documentation

### 48.65.2.1 deferred_factory_parameter()

```
template<typename FactoryType>
template<typename ParametersType , typename U = decltype(std::declval<ParametersType>().↩
on( std::shared_ptr<const Executor>{})), std::enable_if_t< detail::is_pointer_convertible<
typename U::element_type, FactoryType >::value > * = nullptr>
gko::deferred_factory_parameter< FactoryType >::deferred_factory_parameter (
            ParametersType parameters )  [inline]
```

Creates a deferred factory parameter object from a factory_parameters-like object.

To instantiate the actual factory, the parameter's `.on(exec)` function will be called.

```
384     {
385         generator_ = [parameters](std::shared_ptr<const Executor> exec)
386             -> std::shared_ptr<FactoryType> { return parameters.on(exec); };
387     }
```

## 48.65.3 Member Function Documentation

### 48.65.3.1 on()

```
template<typename FactoryType>
std::shared_ptr<FactoryType> gko::deferred_factory_parameter< FactoryType >::on (
            std::shared_ptr< const Executor > exec ) const  [inline]
```

Instantiates the deferred parameter into an actual factory.

This will throw if the deferred factory parameter is empty.

```
394     {
395         if (this->is_empty()) {
396             GKO_NOT_SUPPORTED(*this);
397         }
398         return generator_(exec);
399     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/abstract_factory.hpp

# 48.66 gko::batch::matrix::Dense< ValueType > Class Template Reference

Dense is a batch matrix format which explicitly stores all values of the matrix in each of the batches.

`#include <ginkgo/core/matrix/batch_dense.hpp>`

## Public Member Functions

- std::unique_ptr< unbatch_type > create_view_for_item (size_type item_id)

  *Creates a mutable view (of gko::matrix::Dense type) of one item of the batch::matrix::Dense<value_type> object.*
- std::unique_ptr< const unbatch_type > create_const_view_for_item (size_type item_id) const

  *Creates a mutable view (of gko::matrix::Dense type) of one item of the batch::matrix::Dense<value_type> object.*
- size_type get_cumulative_offset (size_type batch_id) const

  *Get the cumulative storage size offset.*
- value_type * get_values () noexcept

  *Returns a pointer to the array of values of the multi-vector.*
- const value_type * get_const_values () const noexcept

  *Returns a pointer to the array of values of the multi-vector.*
- value_type & at (size_type batch_id, size_type row, size_type col)

  *Returns a single element for a particular batch item.*
- value_type at (size_type batch_id, size_type row, size_type col) const

  *Returns a single element for a particular batch item.*
- ValueType & at (size_type batch_id, size_type idx) noexcept

  *Returns a single element for a particular batch item.*
- ValueType at (size_type batch_id, size_type idx) const noexcept

  *Returns a single element for a particular batch item.*
- value_type * get_values_for_item (size_type batch_id) noexcept

  *Returns a pointer to the array of values of the matrix for a specific batch item.*
- const value_type * get_const_values_for_item (size_type batch_id) const noexcept

  *Returns a pointer to the array of values of the matrix for a specific batch item.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.*
- size_type get_num_elements_per_item () const noexcept

  *Returns the number of stored elements in each batch item.*
- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector.*
- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector with a linear combination of the given input vector.*
- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x) const
- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x) const
- void scale (const array< value_type > &row_scale, const array< value_type > &col_scale)

  *Performs in-place row and column scaling for this matrix.*
- void scale_add (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const batch::matrix::Dense< value_type >> b)

  *Performs the operation this = alpha∗this + b.*
- void add_scaled_identity (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> beta)

  *Performs the operation this = alpha∗I + beta∗this.*

## Static Public Member Functions

- static std::unique_ptr< Dense > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size=batch_dim< 2 >{})

  *Creates an uninitialized Dense matrix of the specified size.*
- static std::unique_ptr< Dense > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, array< value_type > values)

  *Creates a Dense matrix from an already allocated (and initialized) array.*
- template<typename InputValueType >
  static std::unique_ptr< Dense > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, std::initializer_list< InputValueType > values)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Dense > create_const (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &sizes, gko::detail::const_array_view< ValueType > &&values)

  *Creates a constant (immutable) batch dense matrix from a constant array.*

### 48.66.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::batch::matrix::Dense< ValueType >**

Dense is a batch matrix format which explicitly stores all values of the matrix in each of the batches.

The values in each of the batches are stored in row-major format (values belonging to the same row appear consecutive in the memory and the values of each batch item are also stored consecutively in memory).

**Note**

Though the storage layout is the same as the multi-vector object, the class semantics and the operations it aims to provide are different. Hence it is recommended to create multi-vector objects if the user means to view the data as a set of vectors.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

### 48.66.2 Member Function Documentation

#### 48.66.2.1 add_scaled_identity()

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::add_scaled_identity (
            ptr_param< const MultiVector< value_type >> alpha,
            ptr_param< const MultiVector< value_type >> beta )
```

Performs the operation this = alpha∗I + beta∗this.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar for identity |
| *beta* | the scalar to multiply this matrix |

**Note**

Performs the operation in-place for this batch matrix

#### 48.66.2.2 apply() [1/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< const MultiVector< value_type >> beta,
             ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector with a linear combination of the given input vector.

Represents the matrix vector multiplication, x = alpha $*$ A

- b + beta $*$ x, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar to scale the matrix-vector product with |
| *b* | the multi-vector to be applied to |
| *beta* | the scalar to scale the x vector with |
| *x* | the output multi-vector |

#### 48.66.2.3 apply() [2/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< const MultiVector< value_type >> beta,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.66.2.4 apply() [3/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector.

Represents the matrix vector multiplication, x = A ∗ b, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *b* | the multi-vector to be applied to |
| *x* | the output multi-vector |

#### 48.66.2.5 apply() [4/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.66.2.6 at() [1/4]

```
template<typename ValueType = default_precision>
ValueType gko::batch::matrix::Dense< ValueType >::at (
             size_type batch_id,
             size_type idx ) const  [inline], [noexcept]
```

Returns a single element for a particular batch item.

Useful for iterating across all elements of the matrix. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the batch item index to be queried |
| *idx* | a linear index of the requested element |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU Dense object from the OMP may result in incorrect behaviour)

```
197     {
```

```
198         GKO_ASSERT(batch_id < this->get_num_batch_items());
199         return values_.get_const_data()[linearize_index(batch_id, idx)];
200     }
```

References gko::array< ValueType >::get_const_data().

#### 48.66.2.7 at() [2/4]

```
template<typename ValueType = default_precision>
ValueType& gko::batch::matrix::Dense< ValueType >::at (
              size_type batch_id,
              size_type idx )  [inline], [noexcept]
```

Returns a single element for a particular batch item.

Useful for iterating across all elements of the matrix. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the batch item index to be queried |
| *idx* | a linear index of the requested element |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU Dense object from the OMP may result in incorrect behaviour)

```
188     {
189         GKO_ASSERT(batch_id < this->get_num_batch_items());
190         return values_.get_data()[linearize_index(batch_id, idx)];
191     }
```

References gko::array< ValueType >::get_data().

#### 48.66.2.8 at() [3/4]

```
template<typename ValueType = default_precision>
value_type& gko::batch::matrix::Dense< ValueType >::at (
              size_type batch_id,
              size_type row,
              size_type col )  [inline]
```

Returns a single element for a particular batch item.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the batch item index to be queried |
| *row* | the row of the requested element |
| *col* | the column of the requested element |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU Dense object from the OMP may result in incorrect behaviour)

```
159     {
160         GKO_ASSERT(batch_id < this->get_num_batch_items());
161         return values_.get_data()[linearize_index(batch_id, row, col)];
162     }
```

References gko::array< ValueType >::get_data().

### 48.66.2.9 at() [4/4]

```
template<typename ValueType = default_precision>
value_type gko::batch::matrix::Dense< ValueType >::at (
              size_type batch_id,
              size_type row,
              size_type col ) const  [inline]
```

Returns a single element for a particular batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the batch item index to be queried |
| *row* | the row of the requested element |
| *col* | the column of the requested element |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU Dense object from the OMP may result in incorrect behaviour)

```
168     {
169         GKO_ASSERT(batch_id < this->get_num_batch_items());
170         return values_.get_const_data()[linearize_index(batch_id, row, col)];
171     }
```

References gko::array< ValueType >::get_const_data().

### 48.66.2.10 create() [1/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::batch::matrix::Dense< ValueType >::create (
              std::shared_ptr< const Executor > exec,
              const batch_dim< 2 > & size,
              array< value_type > values )  [static]
```

Creates a Dense matrix from an already allocated (and initialized) array.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | sizes of the batch matrices in a batch_dim object |
| *values* | array of matrix values |

**Note**

If `values` is not an rvalue, not an array of ValueType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

### 48.66.2.11 create() [2/3]

```
template<typename ValueType = default_precision>
template<typename InputValueType >
static std::unique_ptr<Dense> gko::batch::matrix::Dense< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size,
            std::initializer_list< InputValueType > values )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const batch_dim<2>&, array<value_type>)

```
293     {
294         return create(exec, size, array<value_type>{exec, std::move(values)});
295     }
```

References gko::batch::matrix::Dense< ValueType >::create().

### 48.66.2.12 create() [3/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::batch::matrix::Dense< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size = batch_dim< 2 >{} )  [static]
```

Creates an uninitialized Dense matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::batch::matrix::Dense< ValueType >::create().

### 48.66.2.13 create_const()

```
template<typename ValueType = default_precision>
static std::unique_ptr<const Dense> gko::batch::matrix::Dense< ValueType >::create_const (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & sizes,
            gko::detail::const_array_view< ValueType > && values )  [static]
```

Creates a constant (immutable) batch dense matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *sizes* | the dimensions of the matrix |
| *values* | the value array of the matrix |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

A smart pointer to the newly created matrix.

### 48.66.2.14 create_const_view_for_item()

```
template<typename ValueType = default_precision>
std::unique_ptr<const unbatch_type> gko::batch::matrix::Dense< ValueType >::create_const_↩
view_for_item (
            size_type item_id ) const
```

Creates a mutable view (of gko::matrix::Dense type) of one item of the batch::matrix::Dense<value_type> object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a gko::matrix::Dense object with the data from the batch item at the given index.

### 48.66.2.15 create_view_for_item()

```
template<typename ValueType = default_precision>
std::unique_ptr<unbatch_type> gko::batch::matrix::Dense< ValueType >::create_view_for_item (
            size_type item_id )
```

Creates a mutable view (of gko::matrix::Dense type) of one item of the batch::matrix::Dense<value_type> object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a gko::matrix::Dense object with the data from the batch item at the given index.

### 48.66.2.16 get_const_values()

```
template<typename ValueType = default_precision>
const value_type* gko::batch::matrix::Dense< ValueType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns a pointer to the array of values of the multi-vector.

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
143     {
144         return values_.get_const_data();
145     }
```

References gko::array< ValueType >::get_const_data().

### 48.66.2.17 get_const_values_for_item()

```
template<typename ValueType = default_precision>
const value_type* gko::batch::matrix::Dense< ValueType >::get_const_values_for_item (
            size_type batch_id ) const  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix for a specific batch item.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the id of the batch item. |

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
225     {
226         GKO_ASSERT(batch_id < this->get_num_batch_items());
227         return values_.get_const_data() + this->get_cumulative_offset(batch_id);
228     }
```

References gko::array< ValueType >::get_const_data(), and gko::batch::matrix::Dense< ValueType >::get_↩ cumulative_offset().

### 48.66.2.18 get_cumulative_offset()

```
template<typename ValueType = default_precision>
size_type gko::batch::matrix::Dense< ValueType >::get_cumulative_offset (
            size_type batch_id ) const  [inline]
```

Get the cumulative storage size offset.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the batch id |

**Returns**

the cumulative offset

```
122     {
123         GKO_ASSERT(batch_id < this->get_num_batch_items());
124         return batch_id * this->get_common_size()[0] *
125                this->get_common_size()[1];
126     }
```

Referenced by gko::batch::matrix::Dense< ValueType >::get_const_values_for_item(), and gko::batch::matrix::↩ Dense< ValueType >::get_values_for_item().

### 48.66.2.19 get_num_elements_per_item()

```
template<typename ValueType = default_precision>
size_type gko::batch::matrix::Dense< ValueType >::get_num_elements_per_item ( ) const  [inline],
[noexcept]
```

Returns the number of stored elements in each batch item.

**Returns**

the number of stored elements per batch item.

```
248     {
249         return this->get_num_stored_elements() / this->get_num_batch_items();
250     }
```

References gko::batch::matrix::Dense< ValueType >::get_num_stored_elements().

#### 48.66.2.20 get_num_stored_elements()

```
template<typename ValueType = default_precision>
size_type gko::batch::matrix::Dense< ValueType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.

**Returns**

the number of elements explicitly stored in the vector, cumulative across all the batch items

```
238     {
239         return values_.get_size();
240     }
```

References gko::array< ValueType >::get_size().

Referenced by gko::batch::matrix::Dense< ValueType >::get_num_elements_per_item().

#### 48.66.2.21 get_values()

```
template<typename ValueType = default_precision>
value_type* gko::batch::matrix::Dense< ValueType >::get_values ( )  [inline], [noexcept]
```

Returns a pointer to the array of values of the multi-vector.

**Returns**

the pointer to the array of values

```
133 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.66.2.22 get_values_for_item()

```
template<typename ValueType = default_precision>
value_type* gko::batch::matrix::Dense< ValueType >::get_values_for_item (
              size_type batch_id )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix for a specific batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the id of the batch item. |

**Returns**

the pointer to the array of values

```
211     {
212         GKO_ASSERT(batch_id < this->get_num_batch_items());
213         return values_.get_data() + this->get_cumulative_offset(batch_id);
214     }
```

References gko::batch::matrix::Dense< ValueType >::get_cumulative_offset(), and gko::array< ValueType >::get_data().

### 48.66.2.23 scale()

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::scale (
             const array< value_type > & row_scale,
             const array< value_type > & col_scale )
```

Performs in-place row and column scaling for this matrix.

**Parameters**

| | |
|---|---|
| *row_scale* | the row scalars |
| *col_scale* | the column scalars |

### 48.66.2.24 scale_add()

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Dense< ValueType >::scale_add (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const batch::matrix::Dense< value_type >> b )
```

Performs the operation this = alpha∗this + b.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar to multiply this matrix |
| *b* | the matrix to add |

**Note**

Performs the operation in-place for this batch matrix

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/batch_dense.hpp

## 48.67 gko::matrix::view::dense< ValueType > Struct Template Reference

Non-owning view of a matrix::Dense to be used inside device kernels.

`#include <ginkgo/core/matrix/device_views.hpp>`

### Public Member Functions

- constexpr dense (dim< 2 > size, size_type stride, ValueType *values)

  *Constructs a dense view from size, stride and values.*
- constexpr dense< const ValueType > as_const () const

  *Returns a const view of the same values.*
- constexpr ValueType & operator() (size_type row, size_type col) const

  *Subscript operator accessing the given row and column.*

### 48.67.1 Detailed Description

**template<typename ValueType>**
**struct gko::matrix::view::dense< ValueType >**

Non-owning view of a matrix::Dense to be used inside device kernels.

This type is used to provide a simple and stable ABI for passing data between libraries. The data is stored in row-major order.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store matrix entries. |

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/device_views.hpp

## 48.68 gko::matrix::Dense< ValueType > Class Template Reference

Dense is a matrix format which explicitly stores all values of the matrix.

`#include <ginkgo/core/matrix/dense.hpp>`

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- void transpose (ptr_param< Dense > output) const

  *Writes the transposed matrix into the given output matrix.*
- void conj_transpose (ptr_param< Dense > output) const

  *Writes the conjugate-transposed matrix into the given output matrix.*
- void fill (const ValueType value)

  *Fill the dense matrix with a given value.*
- std::unique_ptr< Dense > permute (ptr_param< const Permutation< int32 >> permutation, permute_mode mode=permute_mode::symmetric) const

  *Creates a permuted copy $A'$ of this matrix $A$ with the given permutation $P$.*
- std::unique_ptr< Dense > permute (ptr_param< const Permutation< int64 >> permutation, permute_mode mode=permute_mode::symmetric) const
- void permute (ptr_param< const Permutation< int32 >> permutation, ptr_param< Dense > output, permute_mode mode) const

  *Overload of permute(ptr_param<const Permutation<int32>>, permute_mode) that writes the permuted copy into an existing Dense matrix.*
- void permute (ptr_param< const Permutation< int64 >> permutation, ptr_param< Dense > output, permute_mode mode) const
- std::unique_ptr< Dense > permute (ptr_param< const Permutation< int32 >> row_permutation, ptr_param< const Permutation< int32 >> column_permutation, bool invert=false) const

  *Creates a non-symmetrically permuted copy $A'$ of this matrix $A$ with the given row and column permutations $P$ and $Q$.*
- std::unique_ptr< Dense > permute (ptr_param< const Permutation< int64 >> row_permutation, ptr_param< const Permutation< int64 >> column_permutation, bool invert=false) const
- void permute (ptr_param< const Permutation< int32 >> row_permutation, ptr_param< const Permutation< int32 >> column_permutation, ptr_param< Dense > output, bool invert=false) const

  *Overload of permute(ptr_param<const Permutation<int32>>, ptr_param<const Permutation<int32>>, permute_mode) that writes the permuted copy into an existing Dense matrix.*
- void permute (ptr_param< const Permutation< int64 >> row_permutation, ptr_param< const Permutation< int64 >> column_permutation, ptr_param< Dense > output, bool invert=false) const
- std::unique_ptr< Dense > scale_permute (ptr_param< const ScaledPermutation< value_type, int32 >> permutation, permute_mode mode=permute_mode::symmetric) const

  *Creates a scaled and permuted copy of this matrix.*
- std::unique_ptr< Dense > scale_permute (ptr_param< const ScaledPermutation< value_type, int64 >> permutation, permute_mode mode=permute_mode::symmetric) const
- void scale_permute (ptr_param< const ScaledPermutation< value_type, int32 >> permutation, ptr_param< Dense > output, permute_mode mode) const

  *Overload of scale_permute(ptr_param<const ScaledPermutation<value_type, int32>>, permute_mode) that writes the permuted copy into an existing Dense matrix.*
- void scale_permute (ptr_param< const ScaledPermutation< value_type, int64 >> permutation, ptr_param< Dense > output, permute_mode mode) const
- std::unique_ptr< Dense > scale_permute (ptr_param< const ScaledPermutation< value_type, int32 >> row_permutation, ptr_param< const ScaledPermutation< value_type, int32 >> column_permutation, bool invert=false) const

  *Creates a scaled and permuted copy of this matrix.*
- std::unique_ptr< Dense > scale_permute (ptr_param< const ScaledPermutation< value_type, int64 >> row_permutation, ptr_param< const ScaledPermutation< value_type, int64 >> column_permutation, bool invert=false) const

- void scale_permute (ptr_param< const ScaledPermutation< value_type, int32 >> row_permutation, ptr_param< const ScaledPermutation< value_type, int32 >> column_permutation, ptr_param< Dense > output, bool invert=false) const

  *Overload of scale_permute(ptr_param<const ScaledPermutation<value_type, int32>>, ptr_param<const ScaledPermutation<value_type, int32>>, bool) that writes the permuted copy into an existing Dense matrix.*
- void scale_permute (ptr_param< const ScaledPermutation< value_type, int64 >> row_permutation, ptr_param< const ScaledPermutation< value_type, int64 >> column_permutation, ptr_param< Dense > output, bool invert=false) const
- std::unique_ptr< LinOp > permute (const array< int32 > *permutation_indices) const override

  *Returns a LinOp representing the symmetric row and column permutation of the Permutable object.*
- std::unique_ptr< LinOp > permute (const array< int64 > *permutation_indices) const override

  *Returns a LinOp representing the symmetric row and column permutation of the Permutable object.*
- void permute (const array< int32 > *permutation_indices, ptr_param< Dense > output) const

  *Writes the symmetrically permuted matrix into the given output matrix.*
- void permute (const array< int64 > *permutation_indices, ptr_param< Dense > output) const
- std::unique_ptr< LinOp > inverse_permute (const array< int32 > *permutation_indices) const override

  *Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.*
- std::unique_ptr< LinOp > inverse_permute (const array< int64 > *permutation_indices) const override

  *Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.*
- void inverse_permute (const array< int32 > *permutation_indices, ptr_param< Dense > output) const

  *Writes the inverse symmetrically permuted matrix into the given output matrix.*
- void inverse_permute (const array< int64 > *permutation_indices, ptr_param< Dense > output) const
- std::unique_ptr< LinOp > row_permute (const array< int32 > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the Permutable object.*
- std::unique_ptr< LinOp > row_permute (const array< int64 > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the Permutable object.*
- void row_permute (const array< int32 > *permutation_indices, ptr_param< Dense > output) const

  *Writes the row-permuted matrix into the given output matrix.*
- void row_permute (const array< int64 > *permutation_indices, ptr_param< Dense > output) const
- std::unique_ptr< Dense > row_gather (const array< int32 > *gather_indices) const

  *Create a Dense matrix consisting of the given rows from this matrix.*
- std::unique_ptr< Dense > row_gather (const array< int64 > *gather_indices) const

  *Create a Dense matrix consisting of the given rows from this matrix.*
- void row_gather (const array< int32 > *gather_indices, ptr_param< LinOp > row_collection) const

  *Copies the given rows from this matrix into `row_collection`*
- void row_gather (const array< int64 > *gather_indices, ptr_param< LinOp > row_collection) const
- void row_gather (ptr_param< const LinOp > alpha, const array< int32 > *gather_indices, ptr_param< const LinOp > beta, ptr_param< LinOp > row_collection) const

  *Copies the given rows from this matrix into `row_collection` with scaling.*
- void row_gather (ptr_param< const LinOp > alpha, const array< int64 > *gather_indices, ptr_param< const LinOp > beta, ptr_param< LinOp > row_collection) const
- std::unique_ptr< LinOp > column_permute (const array< int32 > *permutation_indices) const override

  *Returns a LinOp representing the column permutation of the Permutable object.*
- std::unique_ptr< LinOp > column_permute (const array< int64 > *permutation_indices) const override

  *Returns a LinOp representing the column permutation of the Permutable object.*
- void column_permute (const array< int32 > *permutation_indices, ptr_param< Dense > output) const

  *Writes the column-permuted matrix into the given output matrix.*
- void column_permute (const array< int64 > *permutation_indices, ptr_param< Dense > output) const
- std::unique_ptr< LinOp > inverse_row_permute (const array< int32 > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the inverse permuted object.*
- std::unique_ptr< LinOp > inverse_row_permute (const array< int64 > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the inverse permuted object.*

- void inverse_row_permute (const array< int32 > *permutation_indices, ptr_param< Dense > output) const

  *Writes the inverse row-permuted matrix into the given output matrix.*
- void inverse_row_permute (const array< int64 > *permutation_indices, ptr_param< Dense > output) const
- std::unique_ptr< LinOp > inverse_column_permute (const array< int32 > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the inverse permuted object.*
- std::unique_ptr< LinOp > inverse_column_permute (const array< int64 > *permutation_indices) const override

  *Returns a LinOp representing the row permutation of the inverse permuted object.*
- void inverse_column_permute (const array< int32 > *permutation_indices, ptr_param< Dense > output) const

  *Writes the inverse column-permuted matrix into the given output matrix.*
- void inverse_column_permute (const array< int64 > *permutation_indices, ptr_param< Dense > output) const
- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- void extract_diagonal (ptr_param< Diagonal< ValueType >> output) const

  *Writes the diagonal of this matrix into an existing diagonal matrix.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute (ptr_param< absolute_type > output) const

  *Writes the absolute values of this matrix into an existing matrix.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- std::unique_ptr< complex_type > make_complex () const

  *Creates a complex copy of the original matrix.*
- void make_complex (ptr_param< complex_type > result) const

  *Writes a complex copy of the original matrix to a given complex matrix.*
- std::unique_ptr< real_type > get_real () const

  *Creates a new real matrix and extracts the real part of the original matrix into that.*
- void get_real (ptr_param< real_type > result) const

  *Extracts the real part of the original matrix into a given real matrix.*
- std::unique_ptr< real_type > get_imag () const

  *Creates a new real matrix and extracts the imaginary part of the original matrix into that.*
- void get_imag (ptr_param< real_type > result) const

  *Extracts the imaginary part of the original matrix into a given real matrix.*
- value_type * get_values () noexcept

  *Returns a pointer to the array of values of the matrix.*
- const value_type * get_const_values () const noexcept

  *Returns a pointer to the array of values of the matrix.*
- size_type get_stride () const noexcept

  *Returns the stride of the matrix.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*
- value_type & at (size_type row, size_type col) noexcept

  *Returns a single element of the matrix.*
- value_type at (size_type row, size_type col) const noexcept

  *Returns a single element of the matrix.*
- ValueType & at (size_type idx) noexcept

  *Returns a single element of the matrix.*
- ValueType at (size_type idx) const noexcept

*Returns a single element of the matrix.*

- void scale (ptr_param< const LinOp > alpha)

  *Scales the matrix with a scalar (aka: BLAS scal).*

- void inv_scale (ptr_param< const LinOp > alpha)

  *Scales the matrix with the inverse of a scalar.*

- void validate_data () const override

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*

- void add_scaled (ptr_param< const LinOp > alpha, ptr_param< const LinOp > b)

  *Adds `b` scaled by `alpha` to the matrix (aka: BLAS axpy).*

- void sub_scaled (ptr_param< const LinOp > alpha, ptr_param< const LinOp > b)

  *Subtracts `b` scaled by `alpha` from the matrix (aka: BLAS axpy).*

- void compute_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result) const

  *Computes the column-wise dot product of this matrix and `b`.*

- void compute_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise dot product of this matrix and `b`.*

- void compute_conj_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result) const

  *Computes the column-wise dot product of `conj(this matrix)` and `b`.*

- void compute_conj_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise dot product of `conj(this matrix)` and `b`.*

- void compute_norm2 (ptr_param< LinOp > result) const

  *Computes the column-wise Euclidean (L^2) norm of this matrix.*

- void compute_norm2 (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise Euclidean (L^2) norm of this matrix.*

- void compute_norm1 (ptr_param< LinOp > result) const

  *Computes the column-wise (L^1) norm of this matrix.*

- void compute_norm1 (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise (L^1) norm of this matrix.*

- void compute_squared_norm2 (ptr_param< LinOp > result) const

  *Computes the square of the column-wise Euclidean (L^2) norm of this matrix.*

- void compute_squared_norm2 (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the square of the column-wise Euclidean (L^2) norm of this matrix.*

- void compute_mean (ptr_param< LinOp > result) const

  *Computes the column-wise arithmetic mean of this matrix.*

- void compute_mean (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise arithmetic mean of this matrix.*

- std::unique_ptr< Dense > create_submatrix (const span &rows, const span &columns, const size_type stride)

  *Create a submatrix from the original matrix.*

- std::unique_ptr< Dense > create_submatrix (const span &rows, const span &columns)

  *Create a submatrix from the original matrix.*

- std::unique_ptr< Dense > create_submatrix (const local_span &rows, const local_span &columns, dim< 2 > size)

  *Create a submatrix from the original matrix.*

- std::unique_ptr< real_type > create_real_view ()

  *Create a real view of the (potentially) complex original matrix.*

- std::unique_ptr< const real_type > create_real_view () const

  *Create a real view of the (potentially) complex original matrix.*

- Dense & operator= (const Dense &)

  *Copy-assigns a Dense matrix.*

- Dense & operator= (Dense &&)

  *Move-assigns a Dense matrix.*
- Dense (const Dense &)

  *Copy-constructs a Dense matrix.*
- Dense (Dense &&)

  *Move-constructs a Dense matrix.*

## Static Public Member Functions

- static std::unique_ptr< Dense > create_with_config_of (ptr_param< const Dense > other)

  *Creates a Dense matrix with the same size and stride as another Dense matrix.*
- static std::unique_ptr< Dense > create_with_type_of (ptr_param< const Dense > other, std::shared_ptr< const Executor > exec, const dim< 2 > &size=dim< 2 >{})

  *Creates a Dense matrix with the same type as another Dense matrix but on a different executor and with a different size.*
- static std::unique_ptr< Dense > create_with_type_of (ptr_param< const Dense > other, std::shared_ptr< const Executor > exec, const dim< 2 > &size, size_type stride)
- static std::unique_ptr< Dense > create_with_type_of (ptr_param< const Dense > other, std::shared_ptr< const Executor > exec, const dim< 2 > &size, const dim< 2 > &local_size, size_type stride)
- static std::unique_ptr< Dense > create_view_of (ptr_param< Dense > other)

  *Creates a Dense matrix, where the underlying array is a view of another Dense matrix' array.*
- static std::unique_ptr< const Dense > create_const_view_of (ptr_param< const Dense > other)

  *Creates a immutable Dense matrix, where the underlying array is a view of another Dense matrix' array.*
- static std::unique_ptr< Dense > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size={}, size_type stride=0)

  *Creates an uninitialized Dense matrix of the specified size.*
- static std::unique_ptr< Dense > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< value_type > values, size_type stride)

  *Creates a Dense matrix from an already allocated (and initialized) array.*
- template<typename InputValueType >
  static std::unique_ptr< Dense > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, std::initializer_list< InputValueType > values, size_type stride)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Dense > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< ValueType > &&values, size_type stride)

  *Creates a constant (immutable) Dense matrix from a constant array.*

### 48.68.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::matrix::Dense< ValueType >**

Dense is a matrix format which explicitly stores all values of the matrix.

The values are stored in row-major format (values belonging to the same row appear consecutive in the memory). Optionally, rows can be padded for better memory access.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

**Note**

While this format is not very useful for storing sparse matrices, it is often suitable to store vectors, and sets of vectors.

## 48.68.2 Constructor & Destructor Documentation

### 48.68.2.1 Dense() [1/2]

```
template<typename ValueType = default_precision>
gko::matrix::Dense< ValueType >::Dense (
            const Dense< ValueType > &  )
```

Copy-constructs a Dense matrix.

Inherits executor and dimensions, but copies data without padding.

### 48.68.2.2 Dense() [2/2]

```
template<typename ValueType = default_precision>
gko::matrix::Dense< ValueType >::Dense (
            Dense< ValueType > &&  )
```

Move-constructs a Dense matrix.

Inherits executor, dimensions and data with padding. The moved-from object is empty (0x0 with empty Array).

## 48.68.3 Member Function Documentation

### 48.68.3.1 add_scaled()

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::add_scaled (
            ptr_param< const LinOp > alpha,
            ptr_param< const LinOp > b )
```

Adds `b` scaled by `alpha` to the matrix (aka: BLAS axpy).

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, the entire matrix is scaled by alpha. If it is a Dense row vector of values, then i-th column of the matrix is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of columns of the matrix). |
| *b* | a matrix of the same dimension as this |

### 48.68.3.2 at() [1/4]

```
template<typename ValueType = default_precision>
ValueType gko::matrix::Dense< ValueType >::at (
            size_type idx ) const  [inline], [noexcept]
```

Returns a single element of the matrix.

Useful for iterating across all elements of the matrix. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *idx* | a linear index of the requested element (ignoring the stride) |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
955     {
956         return values_.get_const_data()[linearize_index(idx)];
957     }
```

### 48.68.3.3 at() [2/4]

```
template<typename ValueType = default_precision>
ValueType& gko::matrix::Dense< ValueType >::at (
            size_type idx )  [inline], [noexcept]
```

Returns a single element of the matrix.

Useful for iterating across all elements of the matrix. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *idx* | a linear index of the requested element (ignoring the stride) |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
947     {
948         return values_.get_data()[linearize_index(idx)];
949     }
```

#### 48.68.3.4 at() [3/4]

```
template<typename ValueType = default_precision>
value_type gko::matrix::Dense< ValueType >::at (
              size_type row,
              size_type col ) const  [inline], [noexcept]
```

Returns a single element of the matrix.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *col* | the column of the requested element |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
928     {
929         return values_.get_const_data()[linearize_index(row, col)];
930     }
```

#### 48.68.3.5 at() [4/4]

```
template<typename ValueType = default_precision>
value_type& gko::matrix::Dense< ValueType >::at (
              size_type row,
              size_type col )  [inline], [noexcept]
```

Returns a single element of the matrix.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *col* | the column of the requested element |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
920     {
921         return values_.get_data()[linearize_index(row, col)];
922     }
```

Referenced by gko::initialize().

#### 48.68.3.6 column_permute() [1/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::column_permute (
            const array< int32 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the column permutation of the Permutable object.

In the resulting LinOp, the column `i` contains the input column `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new column permuted object

Implements gko::Permutable< int32 >.

#### 48.68.3.7 column_permute() [2/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::column_permute (
            const array< int32 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

Writes the column-permuted matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *permutation_indices* | The array containing permutation indices. It must have `this->get_size()[1]` elements. |
| *output* | The output matrix. It must have the dimensions `this->get_size()` |

**See also**

Dense::column_permute(const array<int32>*)

#### 48.68.3.8 column_permute() [3/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::column_permute (
            const array< int64 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the column permutation of the Permutable object.

In the resulting LinOp, the column `i` contains the input column `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new column permuted object

Implements gko::Permutable< int64 >.

#### 48.68.3.9 column_permute() [4/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::column_permute (
            const array< int64 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

#### 48.68.3.10 compute_absolute() [1/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<absolute_type> gko::matrix::Dense< ValueType >::compute_absolute ( ) const
[override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Dense< ValueType > > >.

#### 48.68.3.11 compute_absolute() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_absolute (
            ptr_param< absolute_type > output ) const
```

Writes the absolute values of this matrix into an existing matrix.

**Parameters**

| | |
|---|---|
| *output* | The output matrix. Its size must match the size of this matrix. |

**See also**

Dense::compute_absolute()

### 48.68.3.12 compute_conj_dot() [1/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_conj_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result ) const
```

Computes the column-wise dot product of `conj(this matrix)` and `b`.

**Parameters**

| | |
|---|---|
| *b* | a Dense matrix of same dimension as this |
| *result* | a Dense row vector, used to store the dot product (the number of column in the vector must match the number of columns of this) |

### 48.68.3.13 compute_conj_dot() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_conj_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise dot product of `conj(this matrix)` and `b`.

**Parameters**

| | |
|---|---|
| *b* | a Dense matrix of same dimension as this |
| *result* | a Dense row vector, used to store the dot product (the number of column in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.68.3.14 compute_dot() [1/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result ) const
```

Computes the column-wise dot product of this matrix and b.

**Parameters**

| | |
|---|---|
| *b* | a Dense matrix of same dimension as this |
| *result* | a Dense row vector, used to store the dot product (the number of column in the vector must match the number of columns of this) |

#### 48.68.3.15 compute_dot() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise dot product of this matrix and b.

**Parameters**

| | |
|---|---|
| *b* | a Dense matrix of same dimension as this |
| *result* | a Dense row vector, used to store the dot product (the number of column in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.68.3.16 compute_mean() [1/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_mean (
            ptr_param< LinOp > result ) const
```

Computes the column-wise arithmetic mean of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the mean (the number of columns in the vector must match the number of columns of this) |

#### 48.68.3.17 compute_mean() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_mean (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise arithmetic mean of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the mean (the number of columns in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.68.3.18 compute_norm1() [1/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_norm1 (
            ptr_param< LinOp > result ) const
```

Computes the column-wise (L^1) norm of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |

#### 48.68.3.19 compute_norm1() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_norm1 (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise (L^1) norm of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.68.3.20 compute_norm2() [1/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_norm2 (
            ptr_param< LinOp > result ) const
```

Computes the column-wise Euclidean (L^2) norm of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |

#### 48.68.3.21 compute_norm2() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_norm2 (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise Euclidean (L^2) norm of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.68.3.22 compute_squared_norm2() [1/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_squared_norm2 (
            ptr_param< LinOp > result ) const
```

Computes the square of the column-wise Euclidean (L^2) norm of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |

#### 48.68.3.23 compute_squared_norm2() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::compute_squared_norm2 (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the square of the column-wise Euclidean (L^2) norm of this matrix.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.68.3.24 conj_transpose() [1/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.68.3.25 conj_transpose() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::conj_transpose (
            ptr_param< Dense< ValueType > > output ) const
```

Writes the conjugate-transposed matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *output* | The output matrix. It must have the dimensions `gko::transpose(this->get_size())` |

#### 48.68.3.26 create() [1/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< value_type > values,
            size_type stride )  [static]
```

Creates a Dense matrix from an already allocated (and initialized) array.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *values* | array of matrix values |
| *stride* | stride of the rows (i.e. offset between the first elements of two consecutive rows, expressed as the number of matrix elements) |

**Note**

If `values` is not an rvalue, not an array of ValueType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

#### 48.68.3.27 create() [2/3]

```
template<typename ValueType = default_precision>
template<typename InputValueType >
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            std::initializer_list< InputValueType > values,
            size_type stride )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const dim<2>&, array<value_type>, size_type)

```
1251     {
1252         return create(exec, size, array<value_type>{exec, std::move(values)},
1253                       stride);
1254     }
```

#### 48.68.3.28 create() [3/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = {},
            size_type stride = 0 )  [static]
```

Creates an uninitialized Dense matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *stride* | stride of the rows (i.e. offset between the first elements of two consecutive rows, expressed as the number of matrix elements). If it is set to 0, size[1] will be used instead. |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::matrix::Dense< value_type >::create().

### 48.68.3.29 create_const()

```
template<typename ValueType = default_precision>
static std::unique_ptr<const Dense> gko::matrix::Dense< ValueType >::create_const (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            gko::detail::const_array_view< ValueType > && values,
            size_type stride )  [static]
```

Creates a constant (immutable) Dense matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *values* | the value array of the matrix |
| *stride* | the row-stride of the matrix |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

### 48.68.3.30 create_const_view_of()

```
template<typename ValueType = default_precision>
static std::unique_ptr<const Dense> gko::matrix::Dense< ValueType >::create_const_view_of (
            ptr_param< const Dense< ValueType > > other )  [inline], [static]
```

Creates a immutable Dense matrix, where the underlying array is a view of another Dense matrix' array.

**Parameters**

| | |
|---|---|
| *other* | The other matrix on which to create the view |

**Returns**

A immutable Dense matrix that is a view of other

```
285     {
286         return other->create_const_view_of_impl();
287     }
```

Referenced by gko::make_const_dense_view().

#### 48.68.3.31 create_real_view() [1/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<real_type> gko::matrix::Dense< ValueType >::create_real_view ( )
```

Create a real view of the (potentially) complex original matrix.

If the original matrix is real, nothing changes. If the original matrix is complex, the result is created by viewing the complex matrix with as real with a reinterpret_cast with twice the number of columns and double the stride.

#### 48.68.3.32 create_real_view() [2/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<const real_type> gko::matrix::Dense< ValueType >::create_real_view ( ) const
```

Create a real view of the (potentially) complex original matrix.

If the original matrix is real, nothing changes. If the original matrix is complex, the result is created by viewing the complex matrix with as real with a reinterpret_cast with twice the number of columns and double the stride.

#### 48.68.3.33 create_submatrix() [1/3]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_submatrix (
            const local_span & rows,
            const local_span & columns,
            dim< 2 > size )  [inline]
```

Create a submatrix from the original matrix.

**Parameters**

| | |
|---|---|
| *rows* | row span |
| *columns* | column span |
| *size* | size of the submatrix (only used for consistency with distributed::Vector) |

```
1183     {
1184         dim<2> deduced_size{rows.length(), columns.length()};
1185         GKO_ASSERT_EQUAL_DIMENSIONS(deduced_size, size);
1186         return create_submatrix(rows, columns, this->get_stride());
1187     }
```

#### 48.68.3.34 create_submatrix() [2/3]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_submatrix (
              const span & rows,
              const span & columns )  [inline]
```

Create a submatrix from the original matrix.

**Parameters**

| | |
|---|---|
| *rows* | row span |
| *columns* | column span |

```
1167     {
1168         return create_submatrix(rows, columns, this->get_stride());
1169     }
```

#### 48.68.3.35 create_submatrix() [3/3]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_submatrix (
              const span & rows,
              const span & columns,
              const size_type stride )  [inline]
```

Create a submatrix from the original matrix.

Warning: defining stride for this create_submatrix method might cause wrong memory access. Better use the create_submatrix(rows, columns) method instead.

**Parameters**

| | |
|---|---|
| *rows* | row span |
| *columns* | column span |
| *stride* | stride of the new submatrix. |

```
1155     {
1156         return this->create_submatrix_impl(rows, columns, stride);
1157     }
```

Referenced by gko::matrix::Dense< value_type >::create_submatrix().

#### 48.68.3.36 create_view_of()

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_view_of (
            ptr_param< Dense< ValueType > > other )  [inline], [static]
```

Creates a Dense matrix, where the underlying array is a view of another Dense matrix' array.

**Parameters**

| | |
|---|---|
| *other* | The other matrix on which to create the view |

**Returns**

A Dense matrix that is a view of other

```
272     {
273         return other->create_view_of_impl();
274     }
```

Referenced by gko::make_dense_view().

#### 48.68.3.37 create_with_config_of()

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_with_config_of (
            ptr_param< const Dense< ValueType > > other )  [inline], [static]
```

Creates a Dense matrix with the same size and stride as another Dense matrix.

**Parameters**

| | |
|---|---|
| *other* | The other matrix whose configuration needs to copied. |

```
202     {
203         // De-referencing `other` before calling the functions (instead of
204         // using operator `->`) is currently required to be compatible with
205         // CUDA 10.1.
206         // Otherwise, it results in a compile error.
207         return (*other).create_with_same_config();
208     }
```

#### 48.68.3.38 create_with_type_of() [1/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_with_type_of (
            ptr_param< const Dense< ValueType > > other,
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            const dim< 2 > & local_size,
            size_type stride )  [inline], [static]
```

**Parameters**

| | |
|---|---|
| *other* | The other matrix whose type we target. |
| *exec* | The executor of the new matrix. |
| *size* | The size of the new matrix. |
| *stride* | The stride of the new matrix. |

**Returns**

a Dense matrix with the type of other.

**Parameters**

| | |
|---|---|
| *local_size* | Unused |
| *stride* | The stride of the new matrix. |

**Note**

This is an overload to stay consistent with gko::experimental::distributed::Vector

```
258     {
259         // See create_with_config_of()
260         return (*other).create_with_type_of_impl(exec, size, stride);
261     }
```

### 48.68.3.39 create_with_type_of() [2/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_with_type_of (
            ptr_param< const Dense< ValueType > > other,
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            size_type stride )  [inline], [static]
```

**Parameters**

| | |
|---|---|
| *other* | The other matrix whose type we target. |
| *exec* | The executor of the new matrix. |
| *size* | The size of the new matrix. |
| *stride* | The stride of the new matrix. |

**Returns**

a Dense matrix with the type of other.

**Parameters**

| | |
|---|---|
| *stride* | The stride of the new matrix. |

**Note**

This is an overload which allows full parameter specification.

```
240     {
241         // See create_with_config_of()
242         return (*other).create_with_type_of_impl(exec, size, stride);
243     }
```

#### 48.68.3.40 create_with_type_of() [3/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::create_with_type_of (
            ptr_param< const Dense< ValueType > > other,
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = dim<2>{} )  [inline], [static]
```

Creates a Dense matrix with the same type as another Dense matrix but on a different executor and with a different size.

**Parameters**

| | |
|---|---|
| *other* | The other matrix whose type we target. |
| *exec* | The executor of the new matrix. |
| *size* | The size of the new matrix. |
| *stride* | The stride of the new matrix. |

**Returns**

a Dense matrix with the type of other.

```
223                                     {})
224     {
225         // See create_with_config_of()
226         return (*other).create_with_type_of_impl(exec, size, size[1]);
227     }
```

#### 48.68.3.41 extract_diagonal() [1/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Dense< ValueType >::extract_diagonal ( )
const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

#### 48.68.3.42 extract_diagonal() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::extract_diagonal (
            ptr_param< Diagonal< ValueType >> output ) const
```

Writes the diagonal of this matrix into an existing diagonal matrix.

**Parameters**

| | |
|---|---|
| *output* | The output matrix. Its size must match the size of this matrix's diagonal. |

**See also**

Dense::extract_diagonal()

#### 48.68.3.43 fill()

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::fill (
            const ValueType value )
```

Fill the dense matrix with a given value.

**Parameters**

| | |
|---|---|
| *value* | the value to be filled |

#### 48.68.3.44 get_const_values()

```
template<typename ValueType = default_precision>
const value_type* gko::matrix::Dense< ValueType >::get_const_values ( ) const  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
884     {
885         return values_.get_const_data();
886     }
```

#### 48.68.3.45 get_num_stored_elements()

```
template<typename ValueType = default_precision>
size_type gko::matrix::Dense< ValueType >::get_num_stored_elements ( ) const  [inline], [noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
901     {
902         return values_.get_size();
903     }
```

#### 48.68.3.46 get_stride()

```
template<typename ValueType = default_precision>
size_type gko::matrix::Dense< ValueType >::get_stride ( ) const  [inline], [noexcept]
```

Returns the stride of the matrix.

**Returns**

the stride of the matrix.

```
893 { return stride_; }
```

Referenced by gko::matrix::Dense< value_type >::create_submatrix().

#### 48.68.3.47 get_values()

```
template<typename ValueType = default_precision>
value_type* gko::matrix::Dense< ValueType >::get_values ( )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

```
874 { return values_.get_data(); }
```

#### 48.68.3.48 inv_scale()

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inv_scale (
            ptr_param< const LinOp > alpha )
```

Scales the matrix with the inverse of a scalar.

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, the entire matrix is scaled by 1 / alpha. If it is a Dense row vector of values, then i-th column of the matrix is scaled with the inverse of the i-th element of alpha (the number of columns of alpha has to match the number of columns of the matrix). |

#### 48.68.3.49 inverse_column_permute() [1/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::inverse_column_permute (
            const array< int32 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the column `perm[i]` contains the input column `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implements gko::Permutable< int32 >.

#### 48.68.3.50 inverse_column_permute() [2/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inverse_column_permute (
            const array< int32 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

Writes the inverse column-permuted matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *permutation_indices* | The array containing permutation indices. It must have `this->get_size()[1]` elements. |
| *output* | The output matrix. It must have the dimensions `this->get_size()` |

**See also**

Dense::inverse_column_permute(const array<int32>*)

#### 48.68.3.51 inverse_column_permute() [3/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::inverse_column_permute (
            const array< int64 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the column `perm[i]` contains the input column `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implements gko::Permutable< int64 >.

#### 48.68.3.52 inverse_column_permute() [4/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inverse_column_permute (
            const array< int64 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

#### 48.68.3.53 inverse_permute() [1/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::inverse_permute (
            const array< int32 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(perm[i],perm[j])` contains the input value `(i,j)`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented from gko::Permutable< int32 >.

#### 48.68.3.54 inverse_permute() [2/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inverse_permute (
            const array< int32 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

Writes the inverse symmetrically permuted matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *permutation_indices* | The array containing permutation indices. It must have `this->get_size()[0]` elements. |
| *output* | The output matrix. It must have the dimensions `this->get_size()` |

**See also**

Dense::inverse_permute(const array<int32>*)

#### 48.68.3.55 inverse_permute() [3/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::inverse_permute (
            const array< int64 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(perm[i],perm[j])` contains the input value `(i,j)`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented from gko::Permutable< int64 >.

#### 48.68.3.56 inverse_permute() [4/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inverse_permute (
            const array< int64 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

#### 48.68.3.57 inverse_row_permute() [1/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::inverse_row_permute (
            const array< int32 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the row `perm[i]` contains the input row `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}A$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implements gko::Permutable< int32 >.

#### 48.68.3.58 inverse_row_permute() [2/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inverse_row_permute (
            const array< int32 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

Writes the inverse row-permuted matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *permutation_indices* | The array containing permutation indices. It must have `this->get_size()[0]` elements. |
| *output* | The output matrix. It must have the dimensions `this->get_size()` |

**See also**

Dense::inverse_row_permute(const array<int32>*)

#### 48.68.3.59 inverse_row_permute() [3/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::inverse_row_permute (
            const array< int64 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the row `perm[i]` contains the input row `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}A$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implements gko::Permutable< int64 >.

#### 48.68.3.60 inverse_row_permute() [4/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::inverse_row_permute (
            const array< int64 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

#### 48.68.3.61 make_complex() [1/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<complex_type> gko::matrix::Dense< ValueType >::make_complex ( ) const
```

Creates a complex copy of the original matrix.

If the original matrix was real, the imaginary part of the result will be zero.

#### 48.68.3.62 make_complex() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::make_complex (
            ptr_param< complex_type > result ) const
```

Writes a complex copy of the original matrix to a given complex matrix.

If the original matrix was real, the imaginary part of the result will be zero.

#### 48.68.3.63 operator=() [1/2]

```
template<typename ValueType = default_precision>
Dense& gko::matrix::Dense< ValueType >::operator= (
            const Dense< ValueType > &  )
```

Copy-assigns a Dense matrix.

Preserves the executor, reallocates the matrix with minimal stride if the dimensions don't match, then copies the data over, ignoring padding.

#### 48.68.3.64 operator=() [2/2]

```
template<typename ValueType = default_precision>
Dense& gko::matrix::Dense< ValueType >::operator= (
            Dense< ValueType > &&  )
```

Move-assigns a Dense matrix.

Preserves the executor, moves the data over preserving size and stride. Leaves the moved-from object in an empty state (0x0 with empty Array).

#### 48.68.3.65 permute() [1/12]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::permute (
            const array< int32 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the symmetric row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(i,j)` contains the input value `(perm[i],perm[j])`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PAP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented from gko::Permutable< int32 >.

#### 48.68.3.66 permute() [2/12]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::permute (
            const array< int32 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

Writes the symmetrically permuted matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *permutation_indices* | The array containing permutation indices. It must have `this->get_size()[0]` elements. |
| *output* | The output matrix. It must have the dimensions `this->get_size()` |

**See also**

Dense::permute(const array<int32>*)

#### 48.68.3.67 permute() [3/12]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::permute (
            const array< int64 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the symmetric row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(i,j)` contains the input value `(perm[i],perm[j])`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PAP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented from gko::Permutable< int64 >.

#### 48.68.3.68 permute() [4/12]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::permute (
            const array< int64 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

#### 48.68.3.69 permute() [5/12]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int32 >> permutation,
            permute_mode mode = permute_mode::symmetric ) const
```

Creates a permuted copy $A$' of this matrix $A$ with the given permutation $P$.

By default, this computes a symmetric permutation (permute_mode::symmetric). For the effect of the different permutation modes, see permute_mode.

**Parameters**

| | |
|---|---|
| *permutation* | The input permutation. |
| *mode* | The permutation mode, see permute_mode. |

**Returns**

The permuted matrix.

#### 48.68.3.70 permute() [6/12]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int32 >> permutation,
            ptr_param< Dense< ValueType > > output,
            permute_mode mode ) const
```

Overload of permute(ptr_param<const Permutation<int32>>, permute_mode) that writes the permuted copy into an existing Dense matrix.

**Parameters**

| | |
|---|---|
| *permutation* | the input permutation. |
| *output* | the output matrix. |
| *mode* | the permutation mode, see permute_mode. |

#### 48.68.3.71 permute() [7/12]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int32 >> row_permutation,
            ptr_param< const Permutation< int32 >> column_permutation,
            bool invert = false ) const
```

Creates a non-symmetrically permuted copy $A$' of this matrix $A$ with the given row and column permutations $P$ and $Q$.

The operation will compute $A'(i,j) = A(p[i], q[j])$, or $A' = PAQ^T$ if `invert` is `false`, and $A'(p[i], q[j]) = A(i,j)$, or $A' = P^{-1}AQ^{-T}$ if `invert` is `true`.

**Parameters**

| | |
|---|---|
| *row_permutation* | The permutation $P$ to apply to the rows |
| *column_permutation* | The permutation $Q$ to apply to the columns |
| *invert* | If set to `false`, uses the input permutations, otherwise uses their inverses $P^{-1}, Q^{-1}$ |

**Returns**

The permuted matrix.

#### 48.68.3.72 permute() [8/12]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int32 >> row_permutation,
            ptr_param< const Permutation< int32 >> column_permutation,
            ptr_param< Dense< ValueType > > output,
            bool invert = false ) const
```

Overload of permute(ptr_param<const Permutation<int32>>, ptr_param<const Permutation<int32>>, permute_mode) that writes the permuted copy into an existing Dense matrix.

**Parameters**

| | |
|---|---|
| *row_permutation* | the row permutation. |
| *column_permutation* | the column permutation. |
| *output* | the output matrix. |
| *invert* | if true, the inverse permutations are used. |

#### 48.68.3.73 permute() [9/12]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::permute (
```

```
            ptr_param< const Permutation< int64 >> permutation,
            permute_mode mode = permute_mode::symmetric ) const
```

#### 48.68.3.74 permute() [10/12]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int64 >> permutation,
            ptr_param< Dense< ValueType > > output,
            permute_mode mode ) const
```

#### 48.68.3.75 permute() [11/12]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int64 >> row_permutation,
            ptr_param< const Permutation< int64 >> column_permutation,
            bool invert = false ) const
```

#### 48.68.3.76 permute() [12/12]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::permute (
            ptr_param< const Permutation< int64 >> row_permutation,
            ptr_param< const Permutation< int64 >> column_permutation,
            ptr_param< Dense< ValueType > > output,
            bool invert = false ) const
```

#### 48.68.3.77 row_gather() [1/6]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::row_gather (
            const array< int32 > * gather_indices ) const
```

Create a Dense matrix consisting of the given rows from this matrix.

**Parameters**

| | |
|---|---|
| *gather_indices* | pointer to an array containing row indices from this matrix. It may contain duplicates. |

**Returns**

Dense matrix on the same executor with the same number of columns and `gather_indices->get_size()` rows containing the gathered rows from this matrix: `output(i,j) = input(gather_indices(i), j)`

### 48.68.3.78 row_gather() [2/6]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::row_gather (
            const array< int32 > * gather_indices,
            ptr_param< LinOp > row_collection ) const
```

Copies the given rows from this matrix into `row_collection`

**Parameters**

| | |
|---|---|
| *gather_indices* | pointer to an array containing row indices from this matrix. It may contain duplicates. |
| *row_collection* | pointer to a LinOp that will store the gathered rows: `row_collection(i,j) = input(gather_indices(i), j)` It must have the same number of columns as this matrix and `gather_indices->get_size()` rows. |

### 48.68.3.79 row_gather() [3/6]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::row_gather (
            const array< int64 > * gather_indices ) const
```

Create a Dense matrix consisting of the given rows from this matrix.

**Parameters**

| | |
|---|---|
| *gather_indices* | pointer to an array containing row indices from this matrix. It may contain duplicates. |

**Returns**

Dense matrix on the same executor with the same number of columns and `gather_indices->get_size()` rows containing the gathered rows from this matrix: `output(i,j) = input(gather_indices(i), j)`

### 48.68.3.80 row_gather() [4/6]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::row_gather (
```

```
            const array< int64 > * gather_indices,
            ptr_param< LinOp > row_collection ) const
```

#### 48.68.3.81 row_gather() [5/6]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::row_gather (
            ptr_param< const LinOp > alpha,
            const array< int32 > * gather_indices,
            ptr_param< const LinOp > beta,
            ptr_param< LinOp > row_collection ) const
```

Copies the given rows from this matrix into `row_collection` with scaling.

**Parameters**

| | |
|---|---|
| *alpha* | scaling the result of row gathering |
| *gather_indices* | pointer to an array containing row indices from this matrix. It may contain duplicates. |
| *beta* | scaling the input row_collection |
| *row_collection* | pointer to a LinOp that will store the gathered rows: `row_collection(i,j) = input(gather_indices(i), j)` It must have the same number of columns as this matrix and `gather_indices->get_size()` rows. |

#### 48.68.3.82 row_gather() [6/6]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::row_gather (
            ptr_param< const LinOp > alpha,
            const array< int64 > * gather_indices,
            ptr_param< const LinOp > beta,
            ptr_param< LinOp > row_collection ) const
```

#### 48.68.3.83 row_permute() [1/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::row_permute (
            const array< int32 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the Permutable object.

In the resulting LinOp, the row `i` contains the input row `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PA$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Implements gko::Permutable< int32 >.

#### 48.68.3.84 row_permute() [2/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::row_permute (
            const array< int32 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

Writes the row-permuted matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *permutation_indices* | The array containing permutation indices. It must have `this->get_size()[0]` elements. |
| *output* | The output matrix. It must have the dimensions `this->get_size()` |

**See also**

Dense::row_permute(const array<int32>*)

#### 48.68.3.85 row_permute() [3/4]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::row_permute (
            const array< int64 > * permutation_indices ) const  [override], [virtual]
```

Returns a LinOp representing the row permutation of the Permutable object.

In the resulting LinOp, the row `i` contains the input row `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PA$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Implements gko::Permutable< int64 >.

#### 48.68.3.86 row_permute() [4/4]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::row_permute (
            const array< int64 > * permutation_indices,
            ptr_param< Dense< ValueType > > output ) const
```

#### 48.68.3.87 scale()

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::scale (
            ptr_param< const LinOp > alpha )
```

Scales the matrix with a scalar (aka: BLAS scal).

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, the entire matrix is scaled by alpha. If it is a Dense row vector of values, then i-th column of the matrix is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of columns of the matrix). |

#### 48.68.3.88 scale_permute() [1/8]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int32 >> permutation,
            permute_mode mode = permute_mode::symmetric ) const
```

Creates a scaled and permuted copy of this matrix.

For an explanation of the permutation modes, see permute(ptr_param<const Permutation<index_type>>, permute_mode)

**Parameters**

| | |
|---|---|
| *permutation* | The scaled permutation. |
| *mode* | The permutation mode. |

**Returns**

The permuted matrix.

### 48.68.3.89 scale_permute() [2/8]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int32 >> permutation,
            ptr_param< Dense< ValueType > > output,
            permute_mode mode ) const
```

Overload of scale_permute(ptr_param<const ScaledPermutation<value_type, int32>>, permute_mode) that writes the permuted copy into an existing Dense matrix.

**Parameters**

| | |
|---|---|
| *permutation* | the input scaled permutation. |
| *output* | the output matrix. |
| *mode* | the permutation mode, see permute_mode. |

### 48.68.3.90 scale_permute() [3/8]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int32 >> row_permutation,
            ptr_param< const ScaledPermutation< value_type, int32 >> column_permutation,
            bool invert = false ) const
```

Creates a scaled and permuted copy of this matrix.

For an explanation of the parameters, see permute(ptr_param<const Permutation<index_type>>, ptr_param<const Permutation<index_type>>, permute_mode)

**Parameters**

| | |
|---|---|
| *row_permutation* | The scaled row permutation. |
| *column_permutation* | The scaled column permutation. |
| *invert* | If set to `false`, uses the input permutations, otherwise uses their inverses $P^{-1}, Q^{-1}$ |

**Returns**

The permuted matrix.

#### 48.68.3.91 scale_permute() [4/8]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int32 >> row_permutation,
            ptr_param< const ScaledPermutation< value_type, int32 >> column_permutation,
            ptr_param< Dense< ValueType > > output,
            bool invert = false ) const
```

Overload of scale_permute(ptr_param<const ScaledPermutation<value_type, int32>>, ptr_param<const ScaledPermutation<value_type, int32>>, bool) that writes the permuted copy into an existing Dense matrix.

**Parameters**

| | |
|---|---|
| *row_permutation* | the row scaled permutation. |
| *column_permutation* | the column scaled permutation. |
| *output* | the output matrix. |
| *invert* | if true, the inverse permutations are used. |

#### 48.68.3.92 scale_permute() [5/8]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int64 >> permutation,
            permute_mode mode = permute_mode::symmetric ) const
```

#### 48.68.3.93 scale_permute() [6/8]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int64 >> permutation,
            ptr_param< Dense< ValueType > > output,
            permute_mode mode ) const
```

#### 48.68.3.94 scale_permute() [7/8]

```
template<typename ValueType = default_precision>
std::unique_ptr<Dense> gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int64 >> row_permutation,
            ptr_param< const ScaledPermutation< value_type, int64 >> column_permutation,
            bool invert = false ) const
```

#### 48.68.3.95 scale_permute() [8/8]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::scale_permute (
            ptr_param< const ScaledPermutation< value_type, int64 >> row_permutation,
            ptr_param< const ScaledPermutation< value_type, int64 >> column_permutation,
            ptr_param< Dense< ValueType > > output,
            bool invert = false ) const
```

#### 48.68.3.96 sub_scaled()

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::sub_scaled (
            ptr_param< const LinOp > alpha,
            ptr_param< const LinOp > b )
```

Subtracts `b` scaled by `alpha` from the matrix (aka: BLAS axpy).

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, b is scaled by alpha. If it is a Dense row vector of values, then i-th column of b is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of columns of the matrix). |
| *b* | a matrix of the same dimension as this |

#### 48.68.3.97 transpose() [1/2]

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Dense< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.68.3.98 transpose() [2/2]

```
template<typename ValueType = default_precision>
void gko::matrix::Dense< ValueType >::transpose (
            ptr_param< Dense< ValueType > > output ) const
```

Writes the transposed matrix into the given output matrix.

**Parameters**

| | |
|---|---|
| *output* | The output matrix. It must have the dimensions `gko::transpose(this->get_size())` |

The documentation for this class was generated from the following files:

- ginkgo/core/base/dense_cache.hpp
- ginkgo/core/matrix/dense.hpp

## 48.69 gko::experimental::mpi::DenseCommunicator Class Reference

A CollectiveCommunicator that uses a dense communication.

`#include <ginkgo/core/distributed/dense_communicator.hpp>`

### Public Member Functions

- DenseCommunicator (communicator base)

  *Default constructor with empty communication pattern.*
- template<typename LocalIndexType , typename GlobalIndexType >
  DenseCommunicator (communicator base, const distributed::index_map< LocalIndexType, GlobalIndexType > &imap)

  *Create a DenseCommunicator from an index map.*
- std::unique_ptr< CollectiveCommunicator > create_with_same_type (communicator base, index_map_ptr imap) const override

  *Creates a new CollectiveCommunicator with the same dynamic type.*
- std::pair< std::unique_ptr< CollectiveCommunicator >, std::vector< comm_index_type > > resize (std::shared_ptr< const Executor > exec, const std::vector< comm_index_type > &send_factors) const override

  *Create new communicator that sends and receives a different number of elements to/from each rank in its current pattern.*
- std::unique_ptr< CollectiveCommunicator > create_inverse () const override

  *Creates the inverse DenseCommunicator by switching sources and destinations.*
- comm_index_type get_recv_size () const override

  *Get the number of elements received by this process within this communication pattern.*
- comm_index_type get_send_size () const override

  *Get the number of elements sent by this process within this communication pattern.*
- template<typename SendType , typename RecvType >
  request i_all_to_all_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, RecvType *recv_buffer) const

  *Non-blocking all-to-all communication.*
- request i_all_to_all_v (std::shared_ptr< const Executor > exec, const void *send_buffer, MPI_Datatype send_type, void *recv_buffer, MPI_Datatype recv_type) const

### Friends

- bool operator== (const DenseCommunicator &a, const DenseCommunicator &b)

  *Compares two communicators for equality.*
- bool operator!= (const DenseCommunicator &a, const DenseCommunicator &b)

  *Compares two communicators for inequality.*

## Additional Inherited Members

### 48.69.1 Detailed Description

A CollectiveCommunicator that uses a dense communication.

The dense communicator uses the MPI_Alltoall function for its communication.

### 48.69.2 Constructor & Destructor Documentation

#### 48.69.2.1 DenseCommunicator() [1/2]

```
gko::experimental::mpi::DenseCommunicator::DenseCommunicator (
            communicator base )  [explicit]
```

Default constructor with empty communication pattern.

**Parameters**

| | |
|---|---|
| *base* | the base communicator |

#### 48.69.2.2 DenseCommunicator() [2/2]

```
template<typename LocalIndexType , typename GlobalIndexType >
gko::experimental::mpi::DenseCommunicator::DenseCommunicator (
            communicator base,
            const distributed::index_map< LocalIndexType, GlobalIndexType > & imap )
```

Create a DenseCommunicator from an index map.

The receiving neighbors are defined by the remote indices and their owning ranks of the index map. The send neighbors are deduced from that through collective communication.

**Template Parameters**

| | |
|---|---|
| *LocalIndexType* | the local index type of the map |
| *GlobalIndexType* | the global index type of the map |

**Parameters**

| | |
|---|---|
| *base* | the base communicator |
| *imap* | the index map that defines the communication pattern |

## 48.69.3 Member Function Documentation

### 48.69.3.1 create_inverse()

```
std::unique_ptr<CollectiveCommunicator> gko::experimental::mpi::DenseCommunicator::create_↩
inverse ( ) const  [override], [virtual]
```

Creates the inverse DenseCommunicator by switching sources and destinations.

**Returns**

CollectiveCommunicator with the inverse communication pattern

Implements gko::experimental::mpi::CollectiveCommunicator.

### 48.69.3.2 create_with_same_type()

```
std::unique_ptr<CollectiveCommunicator> gko::experimental::mpi::DenseCommunicator::create_↩
with_same_type (
            communicator base,
            index_map_ptr imap ) const  [override], [virtual]
```

Creates a new CollectiveCommunicator with the same dynamic type.

**Parameters**

| | |
|---|---|
| *base* | The base communicator |
| *imap* | The index_map that defines the communication pattern |

**Returns**

a CollectiveCommunicator with the same dynamic type

Implements gko::experimental::mpi::CollectiveCommunicator.

### 48.69.3.3 get_recv_size()

```
comm_index_type gko::experimental::mpi::DenseCommunicator::get_recv_size ( ) const  [override],
[virtual]
```

Get the number of elements received by this process within this communication pattern.

**Returns**

number of received elements.

Implements gko::experimental::mpi::CollectiveCommunicator.

#### 48.69.3.4 get_send_size()

```
comm_index_type gko::experimental::mpi::DenseCommunicator::get_send_size ( ) const  [override],
[virtual]
```

Get the number of elements sent by this process within this communication pattern.

**Returns**

number of sent elements.

Implements gko::experimental::mpi::CollectiveCommunicator.

#### 48.69.3.5 i_all_to_all_v() [1/2]

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::CollectiveCommunicator::i_all_to_all_v (
            typename SendType ,
            typename RecvType  )
```

Non-blocking all-to-all communication.

The send_buffer must have allocated at least get_send_size number of elements, and the recv_buffer must have allocated at least get_recv_size number of elements.

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the elements to send |
| *RecvType* | the type of the elements to receive |

**Parameters**

| | |
|---|---|
| *exec* | the executor for the communication |
| *send_buffer* | the send buffer |
| *recv_buffer* | the receive buffer |

**Returns**

a request handle

```
160 {
161     return this->i_all_to_all_v(std::move(exec), send_buffer,
162                                 type_impl<SendType>::get_type(), recv_buffer,
163                                 type_impl<RecvType>::get_type());
164 }
```

#### 48.69.3.6 i_all_to_all_v() [2/2]

```
request gko::experimental::mpi::CollectiveCommunicator::i_all_to_all_v
```

#### 48.69.3.7 resize()

```
std::pair<std::unique_ptr<CollectiveCommunicator>, std::vector<comm_index_type> > gko↩
::experimental::mpi::DenseCommunicator::resize (
            std::shared_ptr< const Executor > exec,
            const std::vector< comm_index_type > & send_factors ) const  [override], [virtual]
```

Create new communicator that sends and receives a different number of elements to/from each rank in its current pattern.

The number of elements to send is deduces from the send_factors. If this communicator is used to send 2 elements to rank 0, and 3 elements to rank 1, then send_factors needs to have size 5, which is equal to get_send_size(). An send_factors = { 4, 2, 1, 3, 2} means that instead of the first element to send to rank 0, four elements are sent, instead of the second element to send to rank 0, two elements are sent, and so on. A visual representation of the transformation is:
`[ 1 1 | 1 1 1 ] -> [ 4 2 | 1 3 2] -> [ 1 1 1 1 1 1 | 1 1 1 1 1 1 ]`

The communication pattern will not change. Even if send_factors is zero for all elements of one rank, the rank will stay in the new pattern.

**Note**

This is a collective call.

**Parameters**

| | |
|---|---|
| *exec* | the executor to use for the new communicator |
| *send_factors* | the new number of elements to send. |

**Returns**

The new communicator and the recv_factors implied by the send_factors

Implements gko::experimental::mpi::CollectiveCommunicator.

### 48.69.4 Friends And Related Function Documentation

#### 48.69.4.1 operator"!=

```
bool operator!= (
            const DenseCommunicator & a,
            const DenseCommunicator & b )  [friend]
```

Compares two communicators for inequality.

**See also**

operator==

#### 48.69.4.2 operator==

```
bool operator== (
            const DenseCommunicator & a,
            const DenseCommunicator & b )  [friend]
```

Compares two communicators for equality.

Equality is defined as having identical or congruent communicators and their communication pattern is equal. No communication is done, i.e. there is no reduction over the local equality check results.

**Returns**

true if both communicators are equal.

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/dense_communicator.hpp

## 48.70 gko::device_matrix_data< ValueType, IndexType > Class Template Reference

This type is a device-side equivalent to matrix_data.

```
#include <ginkgo/core/base/device_matrix_data.hpp>
```

### Classes

- struct arrays

  *Stores the internal arrays of a device_matrix_data object.*

### Public Member Functions

- device_matrix_data (std::shared_ptr< const Executor > exec, dim< 2 > size={}, size_type num_entries=0)

  *Initializes a new device_matrix_data object.*
- device_matrix_data (std::shared_ptr< const Executor > exec, const device_matrix_data &data)

  *Initializes a device_matrix_data object by copying an existing object on another executor.*
- device_matrix_data (std::shared_ptr< const Executor > exec, dim< 2 > size, array< index_type > row_idxs, array< index_type > col_idxs, array< value_type > values)

  *Initializes a new device_matrix_data object from existing data.*
- template<typename InputValueType , typename RowIndexType , typename ColIndexType >
  device_matrix_data (std::shared_ptr< const Executor > exec, dim< 2 > size, std::initializer_list< RowIndexType > row_idxs, std::initializer_list< ColIndexType > col_idxs, std::initializer_list< InputValueType > values)
- host_type copy_to_host () const

  *Copies the device_matrix_data entries to the host to return a regular matrix_data object with the same dimensions and entries.*
- void fill_zero ()

  *Fills the matrix entries with zeros.*

- void sort_row_major ()

  *Sorts the matrix entries in row-major order This means that they will be sorted by row index first, and then by column index inside each row.*
- void remove_zeros ()

  *Removes all zero entries from the storage.*
- void sum_duplicates ()

  *Sums up all duplicate entries pointing to the same non-zero location.*
- std::shared_ptr< const Executor > get_executor () const

  *Returns the executor used to store the device_matrix_data entries.*
- dim< 2 > get_size () const

  *Returns the dimensions of the matrix.*
- size_type get_num_elems () const

  *Returns the number of stored elements of the matrix.*
- size_type get_num_stored_elements () const

  *Returns the number of stored elements of the matrix.*
- index_type * get_row_idxs ()

  *Returns a pointer to the row index array.*
- const index_type * get_const_row_idxs () const

  *Returns a pointer to the constant row index array.*
- index_type * get_col_idxs ()

  *Returns a pointer to the column index array.*
- const index_type * get_const_col_idxs () const

  *Returns a pointer to the constant column index array.*
- value_type * get_values ()

  *Returns a pointer to the value array.*
- const value_type * get_const_values () const

  *Returns a pointer to the constant value array.*
- void resize_and_reset (size_type new_num_entries)

  *Resizes the internal storage to the given number of stored matrix entries.*
- void resize_and_reset (dim< 2 > new_size, size_type new_num_entries)

  *Resizes the matrix and internal storage to the given dimensions.*
- arrays empty_out ()

  *Moves out the internal arrays of the device_matrix_data object and resets it to an empty 0x0 matrix.*

## Static Public Member Functions

- static device_matrix_data create_from_host (std::shared_ptr< const Executor > exec, const host_type &data)

  *Creates a device_matrix_data object from the given host data on the given executor.*

### 48.70.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::device_matrix_data< ValueType, IndexType >**

This type is a device-side equivalent to matrix_data.

It stores the data necessary to initialize any matrix format in Ginkgo in individual value, column and row index arrays together with associated matrix dimensions. matrix_data uses array-of-Structs storage (AoS), while device_matrix_data uses Struct-of-Arrays (SoA).

**Note**

To be used with a Ginkgo matrix type, the entry array must be sorted in row-major order, i.e. by row index, then by column index within rows. This can be achieved by calling the sort_row_major function.

The data must not contain any duplicate (row, column) pairs.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type used to store matrix values |
| *IndexType* | the type used to store matrix row and column indices |

## 48.70.2 Constructor & Destructor Documentation

### 48.70.2.1 device_matrix_data() [1/4]

```
template<typename ValueType, typename IndexType>
gko::device_matrix_data< ValueType, IndexType >::device_matrix_data (
            std::shared_ptr< const Executor > exec,
            dim< 2 > size = {},
            size_type num_entries = 0 )  [explicit]
```

Initializes a new device_matrix_data object.

It uses the given executor to allocate storage for the given number of entries and matrix dimensions.

**Parameters**

| | |
|---|---|
| *exec* | the executor to be used to store the matrix entries |
| *size* | the matrix dimensions |
| *num_entries* | the number of entries to be stored |

### 48.70.2.2 device_matrix_data() [2/4]

```
template<typename ValueType, typename IndexType>
gko::device_matrix_data< ValueType, IndexType >::device_matrix_data (
            std::shared_ptr< const Executor > exec,
            const device_matrix_data< ValueType, IndexType > & data )
```

Initializes a device_matrix_data object by copying an existing object on another executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor to be used to store the matrix entries |
| *data* | the device_matrix data object to copy, potentially stored on another executor. |

### 48.70.2.3 device_matrix_data() [3/4]

```
template<typename ValueType, typename IndexType>
```

```
gko::device_matrix_data< ValueType, IndexType >::device_matrix_data (
            std::shared_ptr< const Executor > exec,
            dim< 2 > size,
            array< index_type > row_idxs,
            array< index_type > col_idxs,
            array< value_type > values )
```

Initializes a new device_matrix_data object from existing data.

**Parameters**

| | |
|---|---|
| *exec* | the executor to be used to store the matrix entries |
| *size* | the matrix dimensions |
| *values* | the array containing the matrix values |
| *col_idxs* | the array containing the matrix column indices |
| *row_idxs* | the array containing the matrix row indices |

#### 48.70.2.4 device_matrix_data() [4/4]

```
template<typename ValueType, typename IndexType>
template<typename InputValueType , typename RowIndexType , typename ColIndexType >
gko::device_matrix_data< ValueType, IndexType >::device_matrix_data (
            std::shared_ptr< const Executor > exec,
            dim< 2 > size,
            std::initializer_list< RowIndexType > row_idxs,
            std::initializer_list< ColIndexType > col_idxs,
            std::initializer_list< InputValueType > values )  [inline]
```

**Parameters**

| | |
|---|---|
| *exec* | the executor to be used to store the matrix entries |
| *size* | the matrix dimensions |
| *values* | the array containing the matrix values |
| *col_idxs* | the array containing the matrix column indices |
| *row_idxs* | the array containing the matrix row indices |

```
94        :  device_matrix_data{exec, size,
95                              array<index_type>{exec, std::move(row_idxs)},
96                              array<index_type>{exec, std::move(col_idxs)},
97                              array<value_type>{exec, std::move(values)}}
98    {}
```

### 48.70.3 Member Function Documentation

#### 48.70.3.1 copy_to_host()

```
template<typename ValueType, typename IndexType>
host_type gko::device_matrix_data< ValueType, IndexType >::copy_to_host ( ) const
```

Copies the device_matrix_data entries to the host to return a regular matrix_data object with the same dimensions and entries.

**Returns**

a matrix_data object with the same dimensions and entries.

Referenced by gko::ReadableFromMatrixData< ValueType, int32 >::read().

#### 48.70.3.2 create_from_host()

```
template<typename ValueType, typename IndexType>
static device_matrix_data gko::device_matrix_data< ValueType, IndexType >::create_from_host (
            std::shared_ptr< const Executor > exec,
            const host_type & data )  [static]
```

Creates a device_matrix_data object from the given host data on the given executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the device_matrix_data on. |
| *data* | the data to be wrapped or copied into a device_matrix_data. |

**Returns**

a device_matrix_data object with the same size and entries as `data` copied to the device executor.

Referenced by gko::WritableToMatrixData< ValueType, int32 >::write().

#### 48.70.3.3 empty_out()

```
template<typename ValueType, typename IndexType>
arrays gko::device_matrix_data< ValueType, IndexType >::empty_out ( )
```

Moves out the internal arrays of the device_matrix_data object and resets it to an empty 0x0 matrix.

**Returns**

a struct containing the internal arrays.

#### 48.70.3.4 get_col_idxs()

```
template<typename ValueType, typename IndexType>
index_type* gko::device_matrix_data< ValueType, IndexType >::get_col_idxs ( )  [inline]
```

Returns a pointer to the column index array.

**Returns**

a pointer to the column index array

```
200 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.70.3.5 get_const_col_idxs()

```
template<typename ValueType, typename IndexType>
const index_type* gko::device_matrix_data< ValueType, IndexType >::get_const_col_idxs ( )
const  [inline]
```

Returns a pointer to the constant column index array.

**Returns**

a pointer to the constant column index array

```
208     {
209         return col_idxs_.get_const_data();
210     }
```

References gko::array< ValueType >::get_const_data().

#### 48.70.3.6 get_const_row_idxs()

```
template<typename ValueType, typename IndexType>
const index_type* gko::device_matrix_data< ValueType, IndexType >::get_const_row_idxs ( )
const  [inline]
```

Returns a pointer to the constant row index array.

**Returns**

a pointer to the constant row index array

```
191     {
192         return row_idxs_.get_const_data();
193     }
```

References gko::array< ValueType >::get_const_data().

#### 48.70.3.7 get_const_values()

```
template<typename ValueType, typename IndexType>
const value_type* gko::device_matrix_data< ValueType, IndexType >::get_const_values ( ) const
[inline]
```

Returns a pointer to the constant value array.

**Returns**

a pointer to the constant value array

```
225     {
226         return values_.get_const_data();
227     }
```

References gko::array< ValueType >::get_const_data().

#### 48.70.3.8 get_executor()

```
template<typename ValueType, typename IndexType>
std::shared_ptr<const Executor> gko::device_matrix_data< ValueType, IndexType >::get_executor
( ) const  [inline]
```

Returns the executor used to store the device_matrix_data entries.

**Returns**

the executor used to store the device_matrix_data entries.

```
152     {
153         return values_.get_executor();
154     }
```

References gko::array< ValueType >::get_executor().

Referenced by gko::WritableToMatrixData< ValueType, int32 >::write().

#### 48.70.3.9 get_num_elems()

```
template<typename ValueType, typename IndexType>
size_type gko::device_matrix_data< ValueType, IndexType >::get_num_elems ( ) const  [inline]
```

Returns the number of stored elements of the matrix.

**Returns**

the number of stored elements of the matrix.

```
169 { return get_num_stored_elements(); }
```

References gko::device_matrix_data< ValueType, IndexType >::get_num_stored_elements().

#### 48.70.3.10 get_num_stored_elements()

```
template<typename ValueType, typename IndexType>
size_type gko::device_matrix_data< ValueType, IndexType >::get_num_stored_elements ( ) const
[inline]
```

Returns the number of stored elements of the matrix.

**Returns**

the number of stored elements of the matrix.

```
176 { return values_.get_size(); }
```

References gko::array< ValueType >::get_size().

Referenced by gko::device_matrix_data< ValueType, IndexType >::get_num_elems().

#### 48.70.3.11 get_row_idxs()

```
template<typename ValueType, typename IndexType>
index_type* gko::device_matrix_data< ValueType, IndexType >::get_row_idxs ( )  [inline]
```

Returns a pointer to the row index array.

**Returns**

a pointer to the row index array

```
183 { return row_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.70.3.12 get_size()

```
template<typename ValueType, typename IndexType>
dim<2> gko::device_matrix_data< ValueType, IndexType >::get_size ( ) const  [inline]
```

Returns the dimensions of the matrix.

**Returns**

the dimensions of the matrix.

```
161 { return size_; }
```

#### 48.70.3.13 get_values()

```
template<typename ValueType, typename IndexType>
value_type* gko::device_matrix_data< ValueType, IndexType >::get_values ( )  [inline]
```

Returns a pointer to the value array.

**Returns**

a pointer to the value array

```
217 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.70.3.14 remove_zeros()

```
template<typename ValueType, typename IndexType>
void gko::device_matrix_data< ValueType, IndexType >::remove_zeros ( )
```

Removes all zero entries from the storage.

This does not modify the storage if there are no zero entries, and keeps the relative order of nonzero entries otherwise.

#### 48.70.3.15 resize_and_reset() [1/2]

```
template<typename ValueType, typename IndexType>
void gko::device_matrix_data< ValueType, IndexType >::resize_and_reset (
            dim< 2 > new_size,
            size_type new_num_entries )
```

Resizes the matrix and internal storage to the given dimensions.

The resulting storage should be assumed uninitialized.

**Parameters**

| | |
|---|---|
| *new_size* | the new matrix dimensions. |
| *new_num_entries* | the new number of stored matrix entries. |

#### 48.70.3.16 resize_and_reset() [2/2]

```
template<typename ValueType, typename IndexType>
void gko::device_matrix_data< ValueType, IndexType >::resize_and_reset (
            size_type new_num_entries )
```

Resizes the internal storage to the given number of stored matrix entries.

The resulting storage should be assumed uninitialized.

**Parameters**

| | |
|---|---|
| *new_num_entries* | the new number of stored matrix entries. |

#### 48.70.3.17 sum_duplicates()

```
template<typename ValueType, typename IndexType>
void gko::device_matrix_data< ValueType, IndexType >::sum_duplicates ( )
```

Sums up all duplicate entries pointing to the same non-zero location.

The output will be sorted in row-major order, and it will only reallocate if duplicates exist.

The documentation for this class was generated from the following file:

- ginkgo/core/base/device_matrix_data.hpp

## 48.71 gko::matrix::Diagonal< ValueType > Class Template Reference

This class is a utility which efficiently implements the diagonal matrix (a linear operator which scales a vector row wise).

```
#include <ginkgo/core/matrix/diagonal.hpp>
```

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- value_type * get_values () noexcept

  *Returns a pointer to the array of values of the matrix.*
- const value_type * get_const_values () const noexcept

  *Returns a pointer to the array of values of the matrix.*
- void rapply (ptr_param< const LinOp > b, ptr_param< LinOp > x) const

  *Applies the diagonal matrix from the right side to a matrix b, which means scales the columns of b with the according diagonal entries.*
- void inverse_apply (ptr_param< const LinOp > b, ptr_param< LinOp > x) const

  *Applies the inverse of the diagonal matrix to a matrix b, which means scales the columns of b with the inverse of the according diagonal entries.*
- void validate_data () const override

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*

## Static Public Member Functions

- static std::unique_ptr< Diagonal > create (std::shared_ptr< const Executor > exec, size_type size=0)

  *Creates an Diagonal matrix of the specified size.*
- static std::unique_ptr< Diagonal > create (std::shared_ptr< const Executor > exec, const size_type size, array< value_type > values)

  *Creates a Diagonal matrix from an already allocated (and initialized) array.*
- template<typename InputValueType >
  static std::unique_ptr< Diagonal > create (std::shared_ptr< const Executor > exec, const size_type size, std::initializer_list< InputValueType > values)

  *create(std::shared_ptr<const*
- static std::unique_ptr< const Diagonal > create_const (std::shared_ptr< const Executor > exec, size_type size, gko::detail::const_array_view< ValueType > &&values)

  *Creates a constant (immutable) Diagonal matrix from a constant array.*

### 48.71.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::matrix::Diagonal< ValueType >**

This class is a utility which efficiently implements the diagonal matrix (a linear operator which scales a vector row wise).

Objects of the Diagonal class always represent a square matrix, and require one array to store their values.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes of a CSR matrix the diagonal is applied or converted to. |

## 48.71.2 Member Function Documentation

### 48.71.2.1 compute_absolute()

```
template<typename ValueType = default_precision>
std::unique_ptr<absolute_type> gko::matrix::Diagonal< ValueType >::compute_absolute ( ) const
[override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Diagonal< ValueType > > >.

### 48.71.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Diagonal< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.71.2.3 create() [1/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Diagonal> gko::matrix::Diagonal< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const size_type size,
            array< value_type > values )  [static]
```

Creates a Diagonal matrix from an already allocated (and initialized) array.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *values* | array of matrix values |

**Note**

If `values` is not an rvalue, not an array of ValueType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

### 48.71.2.4 create() [2/3]

```
template<typename ValueType = default_precision>
template<typename InputValueType >
static std::unique_ptr<Diagonal> gko::matrix::Diagonal< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const size_type size,
            std::initializer_list< InputValueType > values )  [inline], [static]
```

create(std::shared_ptr<const

create(std::shared_ptr<const executor>="">, const size_type, array<value_type>)

```
239     {
240         return create(exec, size, array<value_type>{exec, std::move(values)});
241     }
```

References gko::matrix::Diagonal< ValueType >::create().

### 48.71.2.5 create() [3/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Diagonal> gko::matrix::Diagonal< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            size_type size = 0 )  [static]
```

Creates an Diagonal matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::matrix::Diagonal< ValueType >::create().

#### 48.71.2.6 create_const()

```
template<typename ValueType = default_precision>
static std::unique_ptr<const Diagonal> gko::matrix::Diagonal< ValueType >::create_const (
            std::shared_ptr< const Executor > exec,
            size_type size,
            gko::detail::const_array_view< ValueType > && values )  [static]
```

Creates a constant (immutable) Diagonal matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the size of the square matrix |
| *values* | the value array of the matrix |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

#### 48.71.2.7 get_const_values()

```
template<typename ValueType = default_precision>
const value_type* gko::matrix::Diagonal< ValueType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
140     {
141         return values_.get_const_data();
142     }
```

References gko::array< ValueType >::get_const_data().

#### 48.71.2.8 get_values()

```
template<typename ValueType = default_precision>
value_type* gko::matrix::Diagonal< ValueType >::get_values ( )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

```
130 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.71.2.9 inverse_apply()

```
template<typename ValueType = default_precision>
void gko::matrix::Diagonal< ValueType >::inverse_apply (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x ) const  [inline]
```

Applies the inverse of the diagonal matrix to a matrix b, which means scales the columns of b with the inverse of the according diagonal entries.

**Parameters**

| | |
|---|---|
| *b* | the input vector(s) on which the inverse of the diagonal matrix is applied |
| *x* | the output vector(s) where the result is stored |

```
170     {
171         GKO_ASSERT_CONFORMANT(this, b);
172         GKO_ASSERT_EQUAL_ROWS(b, x);
173         GKO_ASSERT_EQUAL_ROWS(this, x);
174
175         this->inverse_apply_impl(b.get(), x.get());
176     }
```

References gko::ptr_param< T >::get().

#### 48.71.2.10 rapply()

```
template<typename ValueType = default_precision>
void gko::matrix::Diagonal< ValueType >::rapply (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x ) const  [inline]
```

Applies the diagonal matrix from the right side to a matrix b, which means scales the columns of b with the according diagonal entries.

**Parameters**

| | |
|---|---|
| *b* | the input vector(s) on which the diagonal matrix is applied |
| *x* | the output vector(s) where the result is stored |

```
152     {
153         GKO_ASSERT_REVERSE_CONFORMANT(this, b);
154         GKO_ASSERT_EQUAL_ROWS(b, x);
155         GKO_ASSERT_EQUAL_COLS(this, x);
156
157         this->rapply_impl(b.get(), x.get());
158     }
```

References gko::ptr_param< T >::get().

#### 48.71.2.11 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Diagonal< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following files:

- ginkgo/core/base/lin_op.hpp
- ginkgo/core/matrix/diagonal.hpp

## 48.72 gko::DiagonalExtractable< ValueType > Class Template Reference

The diagonal of a LinOp implementing this interface can be extracted.

```
#include <ginkgo/core/base/lin_op.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > extract_diagonal_linop () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- virtual std::unique_ptr< matrix::Diagonal< ValueType > > extract_diagonal () const =0

  *Extracts the diagonal entries of the matrix into a vector.*

## 48.72.1 Detailed Description

template<typename ValueType>
class gko::DiagonalExtractable< ValueType >

The diagonal of a LinOp implementing this interface can be extracted.

extract_diagonal extracts the elements whose col and row index are the same and stores the result in a min(nrows, ncols) x 1 dense matrix.

## 48.72.2 Member Function Documentation

### 48.72.2.1 extract_diagonal()

```
template<typename ValueType >
virtual std::unique_ptr<matrix::Diagonal<ValueType> > gko::DiagonalExtractable< ValueType
>::extract_diagonal ( ) const  [pure virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Hybrid< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::matrix::Coo< ValueType, IndexTyp gko::matrix::Ell< ValueType, IndexType >, and gko::matrix::Sellp< ValueType, IndexType >.

### 48.72.2.2 extract_diagonal_linop()

```
template<typename ValueType >
std::unique_ptr<LinOp> gko::DiagonalExtractable< ValueType >::extract_diagonal_linop ( )
const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Returns**

linop the linop of diagonal format

Implements gko::DiagonalLinOpExtractable.

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.73 gko::DiagonalLinOpExtractable Class Reference

The diagonal of a LinOp can be extracted.

`#include <ginkgo/core/base/lin_op.hpp>`

### Public Member Functions

- virtual std::unique_ptr< LinOp > extract_diagonal_linop () const =0

  *Extracts the diagonal entries of the matrix into a vector.*

### 48.73.1 Detailed Description

The diagonal of a LinOp can be extracted.

It will be implemented by DiagonalExtractable<ValueType>, so the class does not need to implement it. extract↩ _diagonal_linop returns a linop which extracts the elements whose col and row index are the same and stores the result in a min(nrows, ncols) x 1 dense matrix.

### 48.73.2 Member Function Documentation

#### 48.73.2.1 extract_diagonal_linop()

```
virtual std::unique_ptr<LinOp> gko::DiagonalLinOpExtractable::extract_diagonal_linop ( ) const
[pure virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Returns**

linop the linop of diagonal format

Implemented in gko::DiagonalExtractable< ValueType >.

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.74 gko::dim< Dimensionality, DimensionType > Struct Template Reference

A type representing the dimensions of a multidimensional object.

`#include <ginkgo/core/base/dim.hpp>`

## Public Member Functions

- constexpr dim ()

  *Creates a dimension object with all dimensions set to zero.*
- constexpr dim (const dimension_type &size)

  *Creates a dimension object with all dimensions set to the same value.*
- template<typename... Rest, std::enable_if_t< sizeof...(Rest)==Dimensionality - 1 > * = nullptr>
  constexpr dim (const dimension_type &first, const Rest &... rest)

  *Creates a dimension object with the specified dimensions.*
- constexpr const dimension_type & operator[ ] (const size_type &dimension) const noexcept

  *Returns the requested dimension.*
- dimension_type & operator[ ] (const size_type &dimension) noexcept
- constexpr operator bool () const

  *Checks if all dimensions evaluate to true.*

## Friends

- constexpr friend bool operator== (const dim &x, const dim &y)

  *Checks if two dim objects are equal.*
- constexpr friend bool operator!= (const dim &x, const dim &y)

  *Checks if two dim objects are not equal.*
- constexpr friend dim operator* (const dim &x, const dim &y)

  *Multiplies two dim objects.*
- std::ostream & operator<< (std::ostream &os, const dim &x)

  *A stream operator overload for dim.*

### 48.74.1 Detailed Description

**template<size_type Dimensionality, typename DimensionType = size_type>**
**struct gko::dim< Dimensionality, DimensionType >**

A type representing the dimensions of a multidimensional object.

**Template Parameters**

| | |
|---|---|
| *Dimensionality* | number of dimensions of the object |
| *DimensionType* | datatype used to represent each dimension |

### 48.74.2 Constructor & Destructor Documentation

#### 48.74.2.1 dim() [1/2]

```
template<size_type Dimensionality, typename DimensionType = size_type>
constexpr gko::dim< Dimensionality, DimensionType >::dim (
            const dimension_type & size )  [inline], [explicit], [constexpr]
```

Creates a dimension object with all dimensions set to the same value.

**Parameters**

| | |
|---|---|
| *size* | the size of each dimension |

```
42         :  first_{size}, rest_{size}
43     {}
```

#### 48.74.2.2 dim() [2/2]

```
template<size_type Dimensionality, typename DimensionType = size_type>
template<typename...  Rest, std::enable_if_t< sizeof...(Rest)==Dimensionality - 1 > * = nullptr>
constexpr gko::dim< Dimensionality, DimensionType >::dim (
             const dimension_type & first,
             const Rest &...  rest )  [inline], [constexpr]
```

Creates a dimension object with the specified dimensions.

If the number of dimensions given is less than the dimensionality of the object, the remaining dimensions are set to the same value as the last value given.

For example, in the context of matrices `dim<2>{2, 3}` creates the dimensions for a 2-by-3 matrix.

**Parameters**

| | |
|---|---|
| *first* | first dimension |
| *rest* | other dimensions |

```
62         :  first_{first}, rest_{static_cast<dimension_type>(rest)...}
63     {}
```

### 48.74.3 Member Function Documentation

#### 48.74.3.1 operator bool()

```
template<size_type Dimensionality, typename DimensionType = size_type>
constexpr gko::dim< Dimensionality, DimensionType >::operator bool ( ) const  [inline], [explicit],
[constexpr]
```

Checks if all dimensions evaluate to true.

For standard arithmetic types, this is equivalent to all dimensions being different than zero.

**Returns**

true if and only if all dimensions evaluate to true

**Note**

This operator is explicit to avoid implicit dim-to-int casts. It will still be used in contextual conversions (if, &&, ||, !)

```
105     {
106         return static_cast<bool>(first_) && static_cast<bool>(rest_);
107     }
```

#### 48.74.3.2 operator[]() [1/2]

```
template<size_type Dimensionality, typename DimensionType = size_type>
constexpr const dimension_type& gko::dim< Dimensionality, DimensionType >::operator[] (
            const size_type & dimension ) const  [inline], [constexpr], [noexcept]
```

Returns the requested dimension.

For example, if d is a dim<2> object representing matrix dimensions, d[0] returns the number of rows, and d[1] returns the number of columns.

**Parameters**

| | |
|---|---|
| *dimension* | the requested dimension |

**Returns**

the dimension-th dimension

```
78     {
79         return GKO_ASSERT(dimension < dimensionality),
80                dimension == 0 ?  first_ :  rest_[dimension - 1];
81     }
```

#### 48.74.3.3 operator[]() [2/2]

```
template<size_type Dimensionality, typename DimensionType = size_type>
dimension_type& gko::dim< Dimensionality, DimensionType >::operator[] (
            const size_type & dimension )  [inline], [noexcept]
88     {
89         return GKO_ASSERT(dimension < dimensionality),
90                dimension == 0 ?  first_ :  rest_[dimension - 1];
91     }
```

### 48.74.4 Friends And Related Function Documentation

#### 48.74.4.1 operator"!=

```
template<size_type Dimensionality, typename DimensionType = size_type>
constexpr friend bool operator!= (
            const dim< Dimensionality, DimensionType > & x,
            const dim< Dimensionality, DimensionType > & y )  [friend]
```

Checks if two dim objects are not equal.

**Parameters**

| | |
|---|---|
| *x* | first object |
| *y* | second object |

**Returns**

false if and only if all dimensions of both objects are equal.

```
131     {
132         return !(x == y);
133     }
```

### 48.74.4.2 operator*

```
template<size_type Dimensionality, typename DimensionType = size_type>
constexpr friend dim operator* (
            const dim< Dimensionality, DimensionType > & x,
            const dim< Dimensionality, DimensionType > & y )  [friend]
```

Multiplies two dim objects.

**Parameters**

| | |
|---|---|
| *x* | first object |
| *y* | second object |

**Returns**

a dim object representing the size of the tensor product `x * y`

```
144     {
145         return dim(x.first_ * y.first_, x.rest_ * y.rest_);
146     }
```

### 48.74.4.3 operator<<

```
template<size_type Dimensionality, typename DimensionType = size_type>
std::ostream& operator<< (
            std::ostream & os,
            const dim< Dimensionality, DimensionType > & x )  [friend]
```

A stream operator overload for dim.

**Parameters**

| | |
|---|---|
| *os* | stream object |
| *x* | dim object |

**Returns**

a stream object appended with the dim output

```
157     {
158         os « "(";
159         x.print_to(os);
160         os « ")";
```

```
161         return os;
162     }
```

#### 48.74.4.4 operator==

```
template<size_type Dimensionality, typename DimensionType = size_type>
constexpr friend bool operator== (
              const dim< Dimensionality, DimensionType > & x,
              const dim< Dimensionality, DimensionType > & y )  [friend]
```

Checks if two dim objects are equal.

**Parameters**

| | |
|---|---|
| *x* | first object |
| *y* | second object |

**Returns**

true if and only if all dimensions of both objects are equal.

```
118     {
119         return x.first_ == y.first_ && x.rest_ == y.rest_;
120     }
```

The documentation for this struct was generated from the following file:

- ginkgo/core/base/dim.hpp

## 48.75 gko::DimensionMismatch Class Reference

DimensionMismatch is thrown if an operation is being applied to LinOps of incompatible size.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- DimensionMismatch (const std::string &file, int line, const std::string &func, const std::string &first_name, size_type first_rows, size_type first_cols, const std::string &second_name, size_type second_rows, size_type second_cols, const std::string &clarification)

  *Initializes a dimension mismatch error.*

### 48.75.1 Detailed Description

DimensionMismatch is thrown if an operation is being applied to LinOps of incompatible size.

### 48.75.2 Constructor & Destructor Documentation

#### 48.75.2.1 DimensionMismatch()

```
gko::DimensionMismatch::DimensionMismatch (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & first_name,
            size_type first_rows,
            size_type first_cols,
            const std::string & second_name,
            size_type second_rows,
            size_type second_cols,
            const std::string & clarification )  [inline]
```

Initializes a dimension mismatch error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The function name where the error occurred |
| *first_name* | The name of the first operator |
| *first_rows* | The output dimension of the first operator |
| *first_cols* | The input dimension of the first operator |
| *second_name* | The name of the second operator |
| *second_rows* | The output dimension of the second operator |
| *second_cols* | The input dimension of the second operator |
| *clarification* | An additional message describing the error further |

```
437         :  Error(file, line,
438                  func + ":  attempting to combine operators " + first_name +
439                      " [" + std::to_string(first_rows) + " x " +
440                      std::to_string(first_cols) + "] and " + second_name + " [" +
441                      std::to_string(second_rows) + " x " +
442                      std::to_string(second_cols) + "]:  " + clarification)
443     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.76 gko::experimental::solver::Direct< ValueType, IndexType > Class Template Reference

A direct solver based on a factorization into lower and upper triangular factors (with an optional diagonal scaling).

```
#include <ginkgo/core/solver/direct.hpp>
```

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- Direct (const Direct &)

  *Creates a copy of the solver.*
- Direct (Direct &&)

  *Moves from the given solver, leaving it empty.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

### 48.76.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::experimental::solver::Direct< ValueType, IndexType >**

A direct solver based on a factorization into lower and upper triangular factors (with an optional diagonal scaling).

The solver is built from the Factorization returned by the provided LinOpFactory.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type used to store values of the system matrix |
| *IndexType* | the type used to store sparsity pattern indices of the system matrix |

### 48.76.2 Member Function Documentation

#### 48.76.2.1 conj_transpose()

```
template<typename ValueType , typename IndexType >
std::unique_ptr<LinOp> gko::experimental::solver::Direct< ValueType, IndexType >::conj_↩
transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.76.2.2 parse()

```
template<typename ValueType , typename IndexType >
static parameters_type gko::experimental::solver::Direct< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

#### 48.76.2.3 transpose()

```
template<typename ValueType , typename IndexType >
std::unique_ptr<LinOp> gko::experimental::solver::Direct< ValueType, IndexType >::transpose (
) const [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/direct.hpp

## 48.77 gko::experimental::distributed::DistributedBase Class Reference

A base class for distributed objects.

```
#include <ginkgo/core/distributed/base.hpp>
```

## Public Member Functions

- DistributedBase & operator= (const DistributedBase &)

  *Copy assignment that doesn't change the used mpi::communicator.*
- DistributedBase & operator= (DistributedBase &&) noexcept

  *Move assignment that doesn't change the used mpi::communicator.*
- mpi::communicator get_communicator () const

  *Access the used mpi::communicator.*

### 48.77.1 Detailed Description

A base class for distributed objects.

This class stores and gives access to the used mpi::communicator object.

**Note**

The communicator is not changed on assignment.

### 48.77.2 Member Function Documentation

#### 48.77.2.1 get_communicator()

```
mpi::communicator gko::experimental::distributed::DistributedBase::get_communicator ( ) const
[inline]
```

Access the used mpi::communicator.

**Returns**

used mpi::communicator

```
56 { return comm_; }
```

#### 48.77.2.2 operator=() [1/2]

```
DistributedBase& gko::experimental::distributed::DistributedBase::operator= (
            const DistributedBase &  )  [inline]
```

Copy assignment that doesn't change the used mpi::communicator.

**Returns**

unmodified *this

```
44 { return *this; }
```

#### 48.77.2.3 operator=() [2/2]

```
DistributedBase& gko::experimental::distributed::DistributedBase::operator= (
            DistributedBase &&  )  [inline], [noexcept]
```

Move assignment that doesn't change the used mpi::communicator.

**Returns**

unmodified *this

```
50 { return *this; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/base.hpp

## 48.78 gko::DpcppExecutor Class Reference

This is the Executor subclass which represents a DPC++ enhanced device.

```
#include <ginkgo/core/base/executor.hpp>
```

### Public Member Functions

- std::shared_ptr< Executor > get_master () noexcept override
  *Returns the master OmpExecutor of this Executor.*
- std::shared_ptr< const Executor > get_master () const noexcept override
  *Returns the master OmpExecutor of this Executor.*
- void synchronize () const override
  *Synchronize the operations launched on the executor with its master.*
- std::string get_description () const override
- int get_device_id () const noexcept
  *Get the DPCPP device id of the device associated to this executor.*
- const std::vector< int > & get_subgroup_sizes () const noexcept
  *Get the available subgroup sizes for this device.*
- int get_num_computing_units () const noexcept
  *Get the number of Computing Units of this executor.*
- int get_num_subgroups () const noexcept
  *Get the number of subgroups of this executor.*
- const std::vector< int > & get_max_workitem_sizes () const noexcept
  *Get the maximum work item sizes.*
- int get_max_workgroup_size () const noexcept
  *Get the maximum workgroup size.*
- int get_max_subgroup_size () const noexcept
  *Get the maximum subgroup size.*
- std::string get_device_type () const noexcept
  *Get a string representing the device type.*
- virtual void run (const Operation &op) const=0
  *Runs the specified Operation using this Executor.*

- template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*
- template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (std::string name, const ClosureReference &op_ref, const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

## Static Public Member Functions

- static std::shared_ptr< DpcppExecutor > create (int device_id, std::shared_ptr< Executor > master, std::string device_type="all", dpcpp_queue_property property=dpcpp_queue_property::in_order)

  *Creates a new DpcppExecutor.*
- static int get_num_devices (std::string device_type)

  *Get the number of devices present on the system.*

### 48.78.1 Detailed Description

This is the Executor subclass which represents a DPC++ enhanced device.

### 48.78.2 Member Function Documentation

#### 48.78.2.1 create()

```
static std::shared_ptr<DpcppExecutor> gko::DpcppExecutor::create (
            int device_id,
            std::shared_ptr< Executor > master,
            std::string device_type = "all",
            dpcpp_queue_property property = dpcpp_queue_property::in_order )  [static]
```

Creates a new DpcppExecutor.

**Parameters**

| | |
|---|---|
| *device_id* | the DPCPP device id of this device |
| *master* | an executor on the host that is used to invoke the device kernels |
| *device_type* | a string representing the type of device to consider (accelerator, cpu, gpu or all). |
| *property* | the properties of the DPC++ queue, which may be combined with the binary or operator. See dpcpp_queue_property for more details |

#### 48.78.2.2 get_description()

```
std::string gko::DpcppExecutor::get_description ( ) const  [override], [virtual]
```

**Returns**

a textual representation of the executor and its device.

Implements gko::Executor.

#### 48.78.2.3 get_device_id()

```
int gko::DpcppExecutor::get_device_id ( ) const  [inline], [noexcept]
```

Get the DPCPP device id of the device associated to this executor.

**Returns**

the DPCPP device id of the device associated to this executor

```
1992     {
1993         return this->get_exec_info().device_id;
1994     }
```

#### 48.78.2.4 get_device_type()

```
std::string gko::DpcppExecutor::get_device_type ( ) const  [inline], [noexcept]
```

Get a string representing the device type.

**Returns**

a string representing the device type

```
2072     {
2073         return this->get_exec_info().device_type;
2074     }
```

#### 48.78.2.5 get_master() [1/2]

```
std::shared_ptr<const Executor> gko::DpcppExecutor::get_master ( ) const  [override], [virtual],
[noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

#### 48.78.2.6 get_master() [2/2]

```
std::shared_ptr<Executor> gko::DpcppExecutor::get_master ( )  [override], [virtual], [noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

#### 48.78.2.7 get_max_subgroup_size()

```
int gko::DpcppExecutor::get_max_subgroup_size ( ) const  [inline], [noexcept]
```

Get the maximum subgroup size.

**Returns**

the maximum subgroup size

```
2062     {
2063         return this->get_exec_info().max_subgroup_size;
2064     }
```

#### 48.78.2.8 get_max_workgroup_size()

```
int gko::DpcppExecutor::get_max_workgroup_size ( ) const  [inline], [noexcept]
```

Get the maximum workgroup size.

**Returns**

the maximum workgroup size

```
2052     {
2053         return this->get_exec_info().max_workgroup_size;
2054     }
```

#### 48.78.2.9 get_max_workitem_sizes()

```
const std::vector<int>& gko::DpcppExecutor::get_max_workitem_sizes ( ) const  [inline], [noexcept]
```

Get the maximum work item sizes.

**Returns**

the maximum work item sizes

```
2042     {
2043         return this->get_exec_info().max_workitem_sizes;
2044     }
```

#### 48.78.2.10 get_num_computing_units()

```
int gko::DpcppExecutor::get_num_computing_units ( ) const  [inline], [noexcept]
```

Get the number of Computing Units of this executor.

**Returns**

the number of Computing Units of this executor

```
2023     {
2024         return this->get_exec_info().num_computing_units;
2025     }
```

#### 48.78.2.11 get_num_devices()

```
static int gko::DpcppExecutor::get_num_devices (
             std::string device_type )  [static]
```

Get the number of devices present on the system.

**Parameters**

| | |
|---|---|
| *device_type* | a string representing the device type |

**Returns**

the number of devices present on the system

#### 48.78.2.12 get_subgroup_sizes()

```
const std::vector<int>& gko::DpcppExecutor::get_subgroup_sizes ( ) const  [inline], [noexcept]
```

Get the available subgroup sizes for this device.

**Returns**

the available subgroup sizes for this device

```
2013     {
2014         return this->get_exec_info().subgroup_sizes;
2015     }
```

#### 48.78.2.13 run() [1/3]

```
template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
Dpcpp >
void gko::Executor::run (
            typename ClosureOmp ,
            typename ClosureCuda ,
            typename ClosureHip ,
            typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *op_omp* | functor to run in case of a OmpExecutor or ReferenceExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
660     {
661         LambdaOperation<ClosureOmp, ClosureOmp, ClosureCuda, ClosureHip,
662                         ClosureDpcpp>
663             op(op_omp, op_cuda, op_hip, op_dpcpp);
664         this->run(op);
665     }
```

### 48.78.2.14 run() [2/3]

```
virtual void gko::Executor::run
```

Runs the specified Operation using this Executor.

**Parameters**

| | |
|---|---|
| *op* | the operation to run |

### 48.78.2.15 run() [3/3]

```
template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename
ClosureHip , typename ClosureDpcpp >
void gko::Executor::run (
             typename ClosureReference ,
             typename ClosureOmp ,
             typename ClosureCuda ,
             typename ClosureHip ,
             typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureReference* | type of op_ref |
| *ClosureOmp* | type of op_omp |

**Template Parameters**

| | |
|---|---|
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *name* | the name of the operation |
| *op_ref* | functor to run in case of a ReferenceExecutor |
| *op_omp* | functor to run in case of a OmpExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
688     {
689         LambdaOperation<ClosureReference, ClosureOmp, ClosureCuda, ClosureHip,
690                         ClosureDpcpp>
691             op(std::move(name), op_ref, op_omp, op_cuda, op_hip, op_dpcpp);
692         this->run(op);
693     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

## 48.79 gko::DpcppTimer Class Reference

A timer using kernels for timing on a DpcppExecutor in profiling mode.

`#include <ginkgo/core/base/timer.hpp>`

### Public Member Functions

- void record (time_point &time) override

  *Records a time point at the current time.*
- void wait (time_point &time) override

  *Waits until all kernels in-process when recording the time point are finished.*
- std::chrono::nanoseconds difference_async (const time_point &start, const time_point &stop) override

  *Computes the difference between the two time points in nanoseconds.*

### Additional Inherited Members

### 48.79.1 Detailed Description

A timer using kernels for timing on a DpcppExecutor in profiling mode.

**Note**

With the Level Zero backend, the SYCL runtime truncates the event timestamps to the valid bits of the GPU's kernel timestamps, so they wrap around periodically, about every 223 seconds on an Intel Arc A770. A difference between two time points that spans a wrap-around is too small by one period, which usually makes it negative.

### 48.79.2 Member Function Documentation

#### 48.79.2.1 difference_async()

```
std::chrono::nanoseconds gko::DpcppTimer::difference_async (
            const time_point & start,
            const time_point & stop )  [override], [virtual]
```

Computes the difference between the two time points in nanoseconds.

This asynchronous version does not synchronize itself, so the time points need to have been synchronized with, i.e. `timer->wait(stop)` needs to have been called. The version is intended for more advanced users who want to measure the overhead of timing functionality separately.

**Parameters**

| | |
|---|---|
| *start* | the first time point (earlier) |
| *stop* | the second time point (later) |

**Returns**

the difference between the time points in nanoseconds.

Implements gko::Timer.

The documentation for this class was generated from the following file:

- ginkgo/core/base/timer.hpp

## 48.80 gko::batch::matrix::Ell< ValueType, IndexType > Class Template Reference

Ell is a sparse matrix format that stores the same number of nonzeros in each row, enabling coalesced accesses.

```
#include <ginkgo/core/matrix/batch_ell.hpp>
```

### Public Member Functions

- std::unique_ptr< unbatch_type > create_view_for_item (size_type item_id)

  *Creates a mutable view (of matrix::Ell type) of one item of the batch::matrix::Ell<value_type> object.*
- std::unique_ptr< const unbatch_type > create_const_view_for_item (size_type item_id) const

  *Creates a mutable view (of matrix::Ell type) of one item of the batch::matrix::Ell<value_type> object.*
- value_type * get_values () noexcept

  *Returns a pointer to the array of values of the matrix.*
- const value_type * get_const_values () const noexcept

*Returns a pointer to the array of values of the matrix.*

- index_type * get_col_idxs () noexcept

  *Returns a pointer to the array of column indices of the matrix.*

- const index_type * get_const_col_idxs () const noexcept

  *Returns a pointer to the array of column indices of the matrix.*

- index_type get_num_stored_elements_per_row () const noexcept

  *Returns the number of elements per row explicitly stored.*

- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.*

- size_type get_num_elements_per_item () const noexcept

  *Returns the number of stored elements in each batch item.*

- index_type * get_col_idxs_for_item (size_type batch_id) noexcept

  *Returns a pointer to the array of col_idxs of the matrix.*

- const index_type * get_const_col_idxs_for_item (size_type batch_id) const noexcept

  *Returns a pointer to the array of col_idxs of the matrix.*

- value_type * get_values_for_item (size_type batch_id) noexcept

  *Returns a pointer to the array of values of the matrix for a specific batch item.*

- const value_type * get_const_values_for_item (size_type batch_id) const noexcept

  *Returns a pointer to the array of values of the matrix for a specific batch item.*

- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector.*

- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector with a linear combination of the given input vector.*

- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x) const

- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x) const

- void scale (const array< value_type > &row_scale, const array< value_type > &col_scale)

  *Performs in-place row and column scaling for this matrix.*

- void add_scaled_identity (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> beta)

  *Performs the operation this = alpha*I + beta*this.*

## Static Public Member Functions

- static std::unique_ptr< Ell > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size=batch_dim< 2 >{}, const IndexType num_elems_per_row=0)

  *Creates an uninitialized Ell matrix of the specified size.*

- static std::unique_ptr< Ell > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, const IndexType num_elems_per_row, array< value_type > values, array< index_type > col_idxs)

  *Creates a Ell matrix from an already allocated (and initialized) array.*

- template<typename InputValueType , typename ColIndexType >
  static std::unique_ptr< Ell > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, const IndexType num_elems_per_row, std::initializer_list< InputValueType > values, std::initializer_list< ColIndexType > col_idxs)

  *create(std::shared_ptr<const Executor>,*

- static std::unique_ptr< const Ell > create_const (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &sizes, const index_type num_elems_per_row, gko::detail::const_array_view< value_type > &&values, gko::detail::const_array_view< index_type > &&col_idxs)

  *Creates a constant (immutable) batch ell matrix from a constant array.*

## 48.80.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::batch::matrix::Ell< ValueType, IndexType >

Ell is a sparse matrix format that stores the same number of nonzeros in each row, enabling coalesced accesses.

It is suitable for sparsity patterns that have a similar number of nonzeros in every row. The values are stored in a column-major fashion similar to the monolithic gko::matrix::Ell class.

Similar to the monolithic gko::matrix::Ell class, invalid_index<IndexType> is used as the column index for padded zero entries.

**Note**

It is also assumed that the sparsity pattern of all the items in the batch is the same and therefore only a single copy of the sparsity pattern is stored.

Currently only IndexType of int32 is supported.

**Template Parameters**

| | |
|---|---|
| *ValueType* | value precision of matrix elements |
| *IndexType* | index precision of matrix elements |

## 48.80.2 Member Function Documentation

### 48.80.2.1 add_scaled_identity()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Ell< ValueType, IndexType >::add_scaled_identity (
            ptr_param< const MultiVector< value_type >> alpha,
            ptr_param< const MultiVector< value_type >> beta )
```

Performs the operation this = alpha∗I + beta∗this.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar for identity |
| *beta* | the scalar to multiply this matrix |

**Note**

Performs the operation in-place for this batch matrix.

This operation fails in case this matrix does not have all its diagonal entries.

#### 48.80.2.2 apply() [1/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Ell< ValueType, IndexType >::apply (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< const MultiVector< value_type >> beta,
             ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector with a linear combination of the given input vector.

Represents the matrix vector multiplication, x = alpha ∗ A

- b + beta ∗ x, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar to scale the matrix-vector product with |
| *b* | the multi-vector to be applied to |
| *beta* | the scalar to scale the x vector with |
| *x* | the output multi-vector |

#### 48.80.2.3 apply() [2/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Ell< ValueType, IndexType >::apply (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< const MultiVector< value_type >> beta,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.80.2.4 apply() [3/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Ell< ValueType, IndexType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector.

Represents the matrix vector multiplication, x = A ∗ b, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *b* | the multi-vector to be applied to |
| *x* | the output multi-vector |

#### 48.80.2.5 apply() [4/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Ell< ValueType, IndexType >::apply (
            ptr_param< const MultiVector< value_type >> b,
            ptr_param< MultiVector< value_type >> x ) const
```

#### 48.80.2.6 create() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Ell> gko::batch::matrix::Ell< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size,
            const IndexType num_elems_per_row,
            array< value_type > values,
            array< index_type > col_idxs )  [static]
```

Creates a Ell matrix from an already allocated (and initialized) array.

The column indices array needs to be the same for all batch items.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_elems_per_row* | the number of elements to be stored in each row |
| *values* | array of matrix values |
| *col_idxs* | the col_idxs array of a single batch item of the matrix. |

**Note**

If `values` is not an rvalue, not an array of ValueType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

#### 48.80.2.7 create() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename InputValueType , typename ColIndexType >
static std::unique_ptr<Ell> gko::batch::matrix::Ell< ValueType, IndexType >::create (
```

```
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size,
            const IndexType num_elems_per_row,
            std::initializer_list< InputValueType > values,
            std::initializer_list< ColIndexType > col_idxs )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const batch_dim<2>&, const IndexType, array<value_type>, array<index_type>)

```
304     {
305         return create(exec, size, num_elems_per_row,
306                       array<value_type>{exec, std::move(values)},
307                       array<index_type>{exec, std::move(col_idxs)});
308     }
```

References gko::batch::matrix::Ell< ValueType, IndexType >::create().

#### 48.80.2.8 create() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Ell> gko::batch::matrix::Ell< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size = batch_dim< 2 >{},
            const IndexType num_elems_per_row = 0 )  [static]
```

Creates an uninitialized Ell matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_elems_per_row* | the number of elements to be stored in each row |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::batch::matrix::Ell< ValueType, IndexType >::create().

#### 48.80.2.9 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Ell> gko::batch::matrix::Ell< ValueType, IndexType >::create_↩
const (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & sizes,
            const index_type num_elems_per_row,
            gko::detail::const_array_view< value_type > && values,
            gko::detail::const_array_view< index_type > && col_idxs )  [static]
```

Creates a constant (immutable) batch ell matrix from a constant array.

The column indices array needs to be the same for all batch items.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *sizes* | the dimensions of the matrix |
| *num_elems_per_row* | the number of elements to be stored in each row |
| *values* | the value array of the matrix |
| *col_idxs* | the col_idxs array of a single batch item of the matrix. |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

A smart pointer to the newly created matrix.

### 48.80.2.10 create_const_view_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<const unbatch_type> gko::batch::matrix::Ell< ValueType, IndexType >::create↩
_const_view_for_item (
            size_type item_id ) const
```

Creates a mutable view (of matrix::Ell type) of one item of the batch::matrix::Ell<value_type> object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a batch::matrix::Ell object with the data from the batch item at the given index.

### 48.80.2.11 create_view_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<unbatch_type> gko::batch::matrix::Ell< ValueType, IndexType >::create_view_↩
for_item (
            size_type item_id )
```

Creates a mutable view (of matrix::Ell type) of one item of the batch::matrix::Ell<value_type> object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a batch::matrix::Ell object with the data from the batch item at the given index.

### 48.80.2.12 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

Returns a pointer to the array of column indices of the matrix.

**Returns**

the pointer to the array of column indices

```
149 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

### 48.80.2.13 get_col_idxs_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_col_idxs_for_item (
            size_type batch_id )  [inline], [noexcept]
```

Returns a pointer to the array of col_idxs of the matrix.

This is shared across all batch items.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the id of the batch item. |

**Returns**

the pointer to the array of col_idxs

```
205     {
206         GKO_ASSERT(batch_id < this->get_num_batch_items());
207         return col_idxs_.get_data();
208     }
```

References gko::array< ValueType >::get_data().

#### 48.80.2.14 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_const_col_idxs ( )
const  [inline], [noexcept]
```

Returns a pointer to the array of column indices of the matrix.

**Returns**

the pointer to the array of column indices

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
159     {
160         return col_idxs_.get_const_data();
161     }
```

References gko::array< ValueType >::get_const_data().

#### 48.80.2.15 get_const_col_idxs_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_const_col_idxs_for_item
(
              size_type batch_id ) const  [inline], [noexcept]
```

Returns a pointer to the array of col_idxs of the matrix.

This is shared across all batch items.

**Parameters**

| | |
|---|---|
| *batch_id* | the id of the batch item. |

**Returns**

the pointer to the array of col_idxs

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
219     {
220         GKO_ASSERT(batch_id < this->get_num_batch_items());
221         return col_idxs_.get_const_data();
222     }
```

References gko::array< ValueType >::get_const_data().

#### 48.80.2.16 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_const_values ( ) const
[inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
140     {
141         return values_.get_const_data();
142     }
```

References gko::array< ValueType >::get_const_data().

#### 48.80.2.17 get_const_values_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_const_values_for_item (
              size_type batch_id ) const  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix for a specific batch item.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the id of the batch item. |

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
248     {
249         GKO_ASSERT(batch_id < this->get_num_batch_items());
250         return values_.get_const_data() +
251                batch_id * this->get_num_elements_per_item();
252     }
```

References gko::array< ValueType >::get_const_data(), and gko::batch::matrix::Ell< ValueType, IndexType >↩ ::get_num_elements_per_item().

#### 48.80.2.18 get_num_elements_per_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::batch::matrix::Ell< ValueType, IndexType >::get_num_elements_per_item ( ) const
[inline], [noexcept]
```

Returns the number of stored elements in each batch item.

**Returns**

the number of stored elements per batch item.

```
192     {
193         return this->get_num_stored_elements() / this->get_num_batch_items();
194     }
```

References gko::batch::matrix::Ell< ValueType, IndexType >::get_num_stored_elements().

Referenced by gko::batch::matrix::Ell< ValueType, IndexType >::get_const_values_for_item(), and gko::batch::matrix::Ell< ValueType, IndexType >::get_values_for_item().

#### 48.80.2.19 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::batch::matrix::Ell< ValueType, IndexType >::get_num_stored_elements ( ) const
[inline], [noexcept]
```

Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.

**Returns**

the number of elements explicitly stored in the vector, cumulative across all the batch items

```
182     {
183         return values_.get_size();
184     }
```

References gko::array< ValueType >::get_size().

Referenced by gko::batch::matrix::Ell< ValueType, IndexType >::get_num_elements_per_item().

#### 48.80.2.20 get_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type gko::batch::matrix::Ell< ValueType, IndexType >::get_num_stored_elements_per_row (
) const  [inline], [noexcept]
```

Returns the number of elements per row explicitly stored.

**Returns**

the number of elements stored in each row of the ELL matrix. Same for each batch item

```
170     {
171         return num_elems_per_row_;
172     }
```

#### 48.80.2.21 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix.

**Returns**

the pointer to the array of values

```
130 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.80.2.22 get_values_for_item()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::batch::matrix::Ell< ValueType, IndexType >::get_values_for_item (
            size_type batch_id )  [inline], [noexcept]
```

Returns a pointer to the array of values of the matrix for a specific batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the id of the batch item. |

**Returns**

the pointer to the array of values

```
233     {
234         GKO_ASSERT(batch_id < this->get_num_batch_items());
235         return values_.get_data() +
236                batch_id * this->get_num_elements_per_item();
237     }
```

References gko::array< ValueType >::get_data(), and gko::batch::matrix::Ell< ValueType, IndexType >::get_num_elements_per_item().

#### 48.80.2.23 scale()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::batch::matrix::Ell< ValueType, IndexType >::scale (
            const array< value_type > & row_scale,
            const array< value_type > & col_scale )
```

Performs in-place row and column scaling for this matrix.

**Parameters**

| | |
|---|---|
| *row_scale* | the row scalars |
| *col_scale* | the column scalars |

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/batch_ell.hpp

## 48.81 gko::matrix::Ell< ValueType, IndexType > Class Template Reference

ELL is a matrix format where stride with explicit zeros is used such that all rows have the same number of stored elements.

`#include <ginkgo/core/matrix/ell.hpp>`

### Public Member Functions

- void read (const mat_data &data) override
  *Reads a matrix from a matrix_data structure.*
- void read (const device_mat_data &data) override
  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override
  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override
  *Writes a matrix to a matrix_data structure.*
- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override
  *Extracts the diagonal entries of the matrix into a vector.*
- std::unique_ptr< absolute_type > compute_absolute () const override
  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override
  *Compute absolute inplace on each element.*
- value_type * get_values () noexcept
  *Returns the values of the matrix.*
- const value_type * get_const_values () const noexcept
  *Returns the values of the matrix.*
- index_type * get_col_idxs () noexcept
  *Returns the column indexes of the matrix.*
- const index_type * get_const_col_idxs () const noexcept
  *Returns the column indexes of the matrix.*
- size_type get_num_stored_elements_per_row () const noexcept
  *Returns the number of stored elements per row.*
- size_type get_stride () const noexcept
  *Returns the stride of the matrix.*
- size_type get_num_stored_elements () const noexcept
  *Returns the number of elements explicitly stored in the matrix.*
- value_type & val_at (size_type row, size_type idx) noexcept

*Returns the `idx`-th non-zero element of the `row`-th row .*

- value_type val_at (size_type row, size_type idx) const noexcept

  *Returns the `idx`-th non-zero element of the `row`-th row .*
- index_type & col_at (size_type row, size_type idx) noexcept

  *Returns the `idx`-th column index of the `row`-th row .*
- index_type col_at (size_type row, size_type idx) const noexcept

  *Returns the `idx`-th column index of the `row`-th row .*
- device_view get_device_view ()

  *get the non-owning device view*
- const_device_view get_const_device_view () const

  *get the const non-owning device view*
- void validate_data () const override

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*
- Ell & operator= (const Ell &)

  *Copy-assigns an Ell matrix.*
- Ell & operator= (Ell &&)

  *Move-assigns an Ell matrix.*
- Ell (const Ell &)

  *Copy-constructs an Ell matrix.*
- Ell (Ell &&)

  *Move-constructs an Ell matrix.*

## Static Public Member Functions

- static std::unique_ptr< Ell > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size={}, size_type num_stored_elements_per_row=0, size_type stride=0)

  *Creates an uninitialized Ell matrix of the specified size.*
- static std::unique_ptr< Ell > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< value_type > values, array< index_type > col_idxs, size_type num_stored_elements_per_row, size_type stride)

  *Creates an ELL matrix from already allocated (and initialized) column index and value arrays.*
- template<typename InputValueType , typename InputColumnIndexType >
  static std::unique_ptr< Ell > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, std::initializer_list< InputValueType > values, std::initializer_list< InputColumnIndexType > col_idxs, size_type num_stored_elements_per_row, size_type stride)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Ell > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< ValueType > &&values, gko::detail::const_array_view< IndexType > &&col_idxs, size_type num_stored_elements_per_row, size_type stride)

  *Creates a constant (immutable) Ell matrix from a set of constant arrays.*

### 48.81.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Ell< ValueType, IndexType >**

ELL is a matrix format where stride with explicit zeros is used such that all rows have the same number of stored elements.

The number of elements stored in each row is the largest number of nonzero elements in any of the rows (obtainable through get_num_stored_elements_per_row() method). This removes the need of a row pointer like in the CSR format, and allows for SIMD processing of the distinct rows. For efficient processing, the nonzero elements and the corresponding column indices are stored in column-major fashion. The columns are padded to the length by user-defined stride parameter whose default value is the number of rows of the matrix.

This implementation uses the column index value invalid_index<IndexType>() to mark padding entries that are not part of the sparsity pattern.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.81.2 Constructor & Destructor Documentation

### 48.81.2.1 Ell() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Ell< ValueType, IndexType >::Ell (
            const Ell< ValueType, IndexType > &  )
```

Copy-constructs an Ell matrix.

Inherits executor and dimensions, but copies data without padding.

### 48.81.2.2 Ell() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Ell< ValueType, IndexType >::Ell (
            Ell< ValueType, IndexType > &&  )
```

Move-constructs an Ell matrix.

Inherits executor, dimensions and data with padding. The moved-from object is empty (0x0 with empty Array).

## 48.81.3 Member Function Documentation

### 48.81.3.1 col_at() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type gko::matrix::Ell< ValueType, IndexType >::col_at (
            size_type row,
            size_type idx ) const  [inline], [noexcept]
```

Returns the `idx`-th column index of the `row`-th row .

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
257     {
258         return this->get_const_col_idxs()[this->linearize_index(row, idx)];
259     }
```

References gko::matrix::Ell< ValueType, IndexType >::get_const_col_idxs().

#### 48.81.3.2 col_at() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type& gko::matrix::Ell< ValueType, IndexType >::col_at (
              size_type row,
              size_type idx )  [inline], [noexcept]
```

Returns the `idx`-th column index of the `row`-th row .

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
249     {
250         return this->get_col_idxs()[this->linearize_index(row, idx)];
251     }
```

References gko::matrix::Ell< ValueType, IndexType >::get_col_idxs().

#### 48.81.3.3 compute_absolute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<absolute_type> gko::matrix::Ell< ValueType, IndexType >::compute_absolute ( )
const  [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Ell< ValueType, IndexType > > >.

#### 48.81.3.4 create() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Ell> gko::matrix::Ell< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< value_type > values,
            array< index_type > col_idxs,
            size_type num_stored_elements_per_row,
            size_type stride )  [static]
```

Creates an ELL matrix from already allocated (and initialized) column index and value arrays.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *values* | array of matrix values |
| *col_idxs* | array of column indexes |
| *num_stored_elements_per_row* | the number of stored elements per row |
| *stride* | stride of the rows |

**Note**

If one of `col_idxs` or `values` is not an rvalue, not an array of IndexType and ValueType, respectively, or is on the wrong executor, an internal copy of that array will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

#### 48.81.3.5 create() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename InputValueType , typename InputColumnIndexType >
static std::unique_ptr<Ell> gko::matrix::Ell< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            std::initializer_list< InputValueType > values,
            std::initializer_list< InputColumnIndexType > col_idxs,
            size_type num_stored_elements_per_row,
            size_type stride )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const dim<2>&, array<value_type>, array<index_type>, size_type, size_type)

```
324     {
325         return create(exec, size, array<value_type>{exec, std::move(values)},
326                       array<index_type>{exec, std::move(col_idxs)},
327                       num_stored_elements_per_row, stride);
328     }
```

References gko::matrix::Ell< ValueType, IndexType >::create().

#### 48.81.3.6 create() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Ell> gko::matrix::Ell< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = {},
            size_type num_stored_elements_per_row = 0,
            size_type stride = 0 )  [static]
```

Creates an uninitialized Ell matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_stored_elements_per_row* | the number of stored elements per row |
| *stride* | stride of the columns. If it is set to 0, size[0] will be used instead. |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::matrix::Ell< ValueType, IndexType >::create().

#### 48.81.3.7 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Ell> gko::matrix::Ell< ValueType, IndexType >::create_const (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            gko::detail::const_array_view< ValueType > && values,
            gko::detail::const_array_view< IndexType > && col_idxs,
            size_type num_stored_elements_per_row,
            size_type stride )  [static]
```

Creates a constant (immutable) Ell matrix from a set of constant arrays.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *values* | the value array of the matrix |
| *col_idxs* | the column index array of the matrix |
| *num_stored_elements_per_row* | the number of stored nonzeros per row |
| *stride* | the column-stride of the value and column index array |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of the arrays on the correct executor.

### 48.81.3.8 extract_diagonal()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Ell< ValueType, IndexType >::extract_↩
diagonal ( ) const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

### 48.81.3.9 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Ell< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

```
174 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

Referenced by gko::matrix::Ell< ValueType, IndexType >::col_at().

### 48.81.3.10 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Ell< ValueType, IndexType >::get_const_col_idxs ( ) const  [inline],
[noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
184     {
185         return col_idxs_.get_const_data();
186     }
```

References gko::array< ValueType >::get_const_data().

Referenced by gko::matrix::Ell< ValueType, IndexType >::col_at().

#### 48.81.3.11 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Ell< ValueType, IndexType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
165     {
166         return values_.get_const_data();
167     }
```

References gko::array< ValueType >::get_const_data().

#### 48.81.3.12 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Ell< ValueType, IndexType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
211     {
212         return values_.get_size();
213     }
```

References gko::array< ValueType >::get_size().

#### 48.81.3.13 get_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Ell< ValueType, IndexType >::get_num_stored_elements_per_row ( ) const
[inline], [noexcept]
```

Returns the number of stored elements per row.

**Returns**

the number of stored elements per row.

```
194     {
195         return num_stored_elements_per_row_;
196     }
```

#### 48.81.3.14 get_stride()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Ell< ValueType, IndexType >::get_stride ( ) const  [inline], [noexcept]
```

Returns the stride of the matrix.

**Returns**

the stride of the matrix.

```
203 { return stride_; }
```

#### 48.81.3.15 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Ell< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

```
155 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.81.3.16 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Ell& gko::matrix::Ell< ValueType, IndexType >::operator= (
            const Ell< ValueType, IndexType > &  )
```

Copy-assigns an Ell matrix.

Preserves the executor, reallocates the matrix with minimal stride if the dimensions don't match, then copies the data over, ignoring padding.

#### 48.81.3.17 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Ell& gko::matrix::Ell< ValueType, IndexType >::operator= (
            Ell< ValueType, IndexType > &&  )
```

Move-assigns an Ell matrix.

Preserves the executor, moves the data over preserving size and stride. Leaves the moved-from object in an empty state (0x0 with empty Array).

#### 48.81.3.18 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Ell< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.81.3.19 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Ell< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.81.3.20 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Ell< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.81.3.21 val_at() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type gko::matrix::Ell< ValueType, IndexType >::val_at (
              size_type row,
              size_type idx ) const  [inline], [noexcept]
```

Returns the idx-th non-zero element of the row-th row .

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
234     {
235         return values_.get_const_data()[this->linearize_index(row, idx)];
236     }
```

References gko::array< ValueType >::get_const_data().

#### 48.81.3.22 val_at() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type& gko::matrix::Ell< ValueType, IndexType >::val_at (
              size_type row,
              size_type idx )  [inline], [noexcept]
```

Returns the idx-th non-zero element of the row-th row .

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
226     {
227         return values_.get_data()[this->linearize_index(row, idx)];
228     }
```

References gko::array< ValueType >::get_data().

#### 48.81.3.23 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Ell< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following files:

- ginkgo/core/matrix/csr.hpp
- ginkgo/core/matrix/ell.hpp

## 48.82 gko::matrix::view::ell< ValueType, IndexType > Struct Template Reference

Non-owning view of a matrix::Ell to be used inside device kernels.

```
#include <ginkgo/core/matrix/device_views.hpp>
```

### Public Member Functions

- constexpr ell (dim< 2 > size, size_type num_stored_elements_per_row, size_type stride, ValueType *values, IndexType *col_idxs)

  *Constructs a ell view.*
- constexpr ell< const ValueType, const IndexType > as_const () const

  *Returns a const view of the same values.*
- constexpr ValueType & val_at (size_type row, size_type idx) const

  *accessing the value of the idx-th element within the given row*
- constexpr IndexType & col_at (size_type row, size_type idx) const

  *accessing the column index of the idx-th element within the given row*

### 48.82.1 Detailed Description

template<typename ValueType, typename IndexType>
struct gko::matrix::view::ell< ValueType, IndexType >

Non-owning view of a matrix::Ell to be used inside device kernels.

This type is used to provide a simple and stable ABI for passing data between libraries.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store matrix values. |
| *IndexType* | the index type used to store matrix columns. |

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/device_views.hpp

## 48.83 gko::enable_parameters_type< ConcreteParametersType, Factory > Class Template Reference

The enable_parameters_type mixin is used to create a base implementation of the factory parameters structure.

`#include <ginkgo/core/base/abstract_factory.hpp>`

### Public Member Functions

- template<typename... Args>
  ConcreteParametersType & with_loggers (Args &&... _value)

  *Provides the loggers to be added to the factory and its generated objects in a fluent interface.*
- std::unique_ptr< Factory > on (std::shared_ptr< const Executor > exec) const

  *Creates a new factory on the specified executor.*

### 48.83.1 Detailed Description

template<typename ConcreteParametersType, typename Factory>
class gko::enable_parameters_type< ConcreteParametersType, Factory >

The enable_parameters_type mixin is used to create a base implementation of the factory parameters structure.

It provides only the on() method which can be used to instantiate the factory give the parameters stored in the structure.

**Template Parameters**

| | |
|---|---|
| *ConcreteParametersType* | the concrete parameters type which is being implemented [CRTP parameter] |
| *Factory* | the concrete factory for which these parameters are being used |

### 48.83.2 Member Function Documentation

#### 48.83.2.1 on()

```
template<typename ConcreteParametersType, typename Factory>
std::unique_ptr<Factory> gko::enable_parameters_type< ConcreteParametersType, Factory >::on (
            std::shared_ptr< const Executor > exec ) const  [inline]
```

Creates a new factory on the specified executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the factory will be created |

**Returns**

a new factory instance

```
258     {
259         ConcreteParametersType copy = *self();
260         for (const auto& item :  deferred_factories) {
261             item.second(exec, copy);
262         }
263         auto factory = std::unique_ptr<Factory>(new Factory(exec, copy));
264         for (auto& logger :  loggers) {
265             factory->add_logger(logger);
266         }
267         return factory;
268     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/abstract_factory.hpp

## 48.84 gko::EnableAbsoluteComputation< AbsoluteLinOp > Class Template Reference

The EnableAbsoluteComputation mixin provides the default implementations of `compute_absolute_linop` and the absolute interface.

```
#include <ginkgo/core/base/lin_op.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > compute_absolute_linop () const override

  *Gets the absolute LinOp.*
- virtual std::unique_ptr< absolute_type > compute_absolute () const =0

  *Gets the AbsoluteLinOp.*

## 48.84.1 Detailed Description

**template<typename AbsoluteLinOp>**
**class gko::EnableAbsoluteComputation< AbsoluteLinOp >**

The EnableAbsoluteComputation mixin provides the default implementations of `compute_absolute_linop` and the absolute interface.

`compute_absolute` gets a new AbsoluteLinOp. `compute_absolute_inplace` applies absolute inplace, so it still keeps the value_type of the class.

**Template Parameters**

| | |
|---|---|
| *AbsoluteLinOp* | the absolute LinOp which is being returned [CRTP parameter] |

## 48.84.2 Member Function Documentation

### 48.84.2.1 compute_absolute()

```
template<typename AbsoluteLinOp>
virtual std::unique_ptr<absolute_type> gko::EnableAbsoluteComputation< AbsoluteLinOp >↩
::compute_absolute ( ) const  [pure virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Hybrid< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::experimental::distributed::Vector< gko::matrix::Coo< ValueType, IndexType >, gko::matrix::Ell< ValueType, IndexType >, gko::matrix::Sellp< ValueType, IndexType >, and gko::matrix::Diagonal< ValueType >.

Referenced by gko::EnableAbsoluteComputation< remove_complex< Ell< ValueType, IndexType > > >↩
::compute_absolute_linop().

### 48.84.2.2 compute_absolute_linop()

```
template<typename AbsoluteLinOp>
std::unique_ptr<LinOp> gko::EnableAbsoluteComputation< AbsoluteLinOp >::compute_absolute_↩
linop ( ) const  [inline], [override], [virtual]
```

Gets the absolute LinOp.

**Returns**

a pointer to the new absolute LinOp

Implements gko::AbsoluteComputable.

```
878     {
879         return this->compute_absolute();
880     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.85 gko::solver::EnableApplyWithInitialGuess< DerivedType > Class Template Reference

EnableApplyWithInitialGuess providing default operation for ApplyWithInitialGuess with correct validation and log.

`#include <ginkgo/core/solver/solver_base.hpp>`

### 48.85.1 Detailed Description

**template<typename DerivedType>**
**class gko::solver::EnableApplyWithInitialGuess< DerivedType >**

EnableApplyWithInitialGuess providing default operation for ApplyWithInitialGuess with correct validation and log.

It ensures that vectors of apply_with_initial_guess will always have the same executor as the object this mixin is used in, creating a clone on the correct executor if necessary.

**Template Parameters**

| | |
|---|---|
| *DerivedType* | The type that this Mixin is used in. It must provide get_size() and get_executor() functions that return correctly initialized values and the logger functionality. |

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

## 48.86 gko::batch::solver::EnableBatchSolver< ConcreteSolver, ValueType, PolymorphicBase > Class Template Reference

This mixin provides apply and common iterative solver functionality to all the batched solvers.

`#include <ginkgo/core/solver/batch_solver_base.hpp>`

### Additional Inherited Members

### 48.86.1 Detailed Description

**template<typename ConcreteSolver, typename ValueType, typename PolymorphicBase = BatchLinOp>**
**class gko::batch::solver::EnableBatchSolver< ConcreteSolver, ValueType, PolymorphicBase >**

This mixin provides apply and common iterative solver functionality to all the batched solvers.

**Template Parameters**

| | |
|---|---|
| *ConcreteSolver* | The concrete solver class. |
| *ValueType* | The value type of the multivectors. |
| *PolymorphicBase* | The base class; must be a subclass of BatchLinOp. |

The documentation for this class was generated from the following file:

- ginkgo/core/solver/batch_solver_base.hpp

# 48.87 gko::EnableCloneable< ConcreteType > Class Template Reference

This mixin is used to enable a default Cloneable::clone() implementation and similar for objects that have implemented copy and move constructors.

`#include <ginkgo/core/base/polymorphic_object.hpp>`

## Public Member Functions

- std::unique_ptr< ConcreteType > clone (std::shared_ptr< const Executor > exec) const

  *Creates a clone of the object.*
- std::unique_ptr< ConcreteType > clone () const

  *Creates a clone of the object.*
- void convert_to (result_type *result) const override

  *Converts the implementer to an object of type result_type.*
- void move_to (result_type *result) override

  *Converts the implementer to an object of type result_type by moving data from this object.*

## 48.87.1 Detailed Description

**template<typename ConcreteType>**
**class gko::EnableCloneable< ConcreteType >**

This mixin is used to enable a default Cloneable::clone() implementation and similar for objects that have implemented copy and move constructors.

**Note**

The ConcreteType must have public constructors, or this class as a friend class.

**Template Parameters**

| | |
|---|---|
| *ConcreteType* | the concrete type from which the copy_from is being enabled [CRTP parameter] |

## 48.87.2 Member Function Documentation

#### 48.87.2.1 clone() [1/2]

```
template<typename ConcreteType>
std::unique_ptr<ConcreteType> gko::EnableCloneable< ConcreteType >::clone ( ) const  [inline]
```

Creates a clone of the object.

This is a shorthand for clone(std::shared_ptr<const Executor>) that uses the executor of this object to construct the new object.

**Returns**

A clone of the LinOp.

```
407     {
408         return as<ConcreteType>(this->clone_impl());
409     }
```

#### 48.87.2.2 clone() [2/2]

```
template<typename ConcreteType>
std::unique_ptr<ConcreteType> gko::EnableCloneable< ConcreteType >::clone (
             std::shared_ptr< const Executor > exec ) const  [inline]
```

Creates a clone of the object.

This is the polymorphic equivalent of the *executor copy constructor* `decltype(*this)(exec, this)`.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the clone will be created |

**Returns**

A clone of the LinOp.

```
394     {
395         return as<ConcreteType>(this->clone_impl(std::move(exec)));
396     }
```

#### 48.87.2.3 convert_to()

```
template<typename ConcreteType>
void gko::EnableCloneable< ConcreteType >::convert_to (
             result_type * result ) const  [inline], [override], [virtual]
```

Converts the implementer to an object of type result_type.

**Parameters**

| | |
|---|---|
| *result* | the object used to store the result of the conversion |

Implements gko::ConvertibleTo< ConcreteType >.
411 { *result = *self(); }

#### 48.87.2.4 move_to()

```
template<typename ConcreteType>
void gko::EnableCloneable< ConcreteType >::move_to (
            result_type * result )  [inline], [override], [virtual]
```

Converts the implementer to an object of type result_type by moving data from this object.

This method is used when the implementer is a temporary object, and move semantics can be used.

**Parameters**

| | |
|---|---|
| *result* | the object used to emplace the result of the conversion |

**Note**

ConvertibleTo::move_to can be implemented by simply calling ConvertibleTo::convert_to. However, this operation can often be optimized by exploiting the fact that implementer's data can be moved to the result.

Implements gko::ConvertibleTo< ConcreteType >.
413 { *result = std::move(*self()); }

The documentation for this class was generated from the following file:

- ginkgo/core/base/polymorphic_object.hpp

## 48.88 gko::EnableCreateMethod< ConcreteType > Class Template Reference

This mixin implements a static `create()` method on `ConcreteType` that dynamically allocates the memory, uses the passed-in arguments to construct the object, and returns an std::unique_ptr to such an object.

`#include <ginkgo/core/base/polymorphic_object.hpp>`

### 48.88.1 Detailed Description

**template<typename ConcreteType>**
**class gko::EnableCreateMethod< ConcreteType >**

This mixin implements a static `create()` method on `ConcreteType` that dynamically allocates the memory, uses the passed-in arguments to construct the object, and returns an std::unique_ptr to such an object.

**Template Parameters**

| | |
|---|---|
| *ConcreteObject* | the concrete type for which `create()` is being implemented [CRTP parameter] |

The documentation for this class was generated from the following file:

- ginkgo/core/base/polymorphic_object.hpp

## 48.89 gko::EnableDefaultFactory< ConcreteFactory, ProductType, ParametersType, PolymorphicBase > Class Template Reference

This mixin provides a default implementation of a concrete factory.

`#include <ginkgo/core/base/abstract_factory.hpp>`

### Public Member Functions

- const parameters_type & get_parameters () const noexcept
  *Returns the parameters of the factory.*

### Static Public Member Functions

- static parameters_type create ()
  *Creates a new ParametersType object which can be used to instantiate a new ConcreteFactory.*

### 48.89.1 Detailed Description

**template<typename ConcreteFactory, typename ProductType, typename ParametersType, typename PolymorphicBase>**
**class gko::EnableDefaultFactory< ConcreteFactory, ProductType, ParametersType, PolymorphicBase >**

This mixin provides a default implementation of a concrete factory.

It implements all the methods of AbstractFactory and PolymorphicObject. Its implementation of the generate_impl() method delegates the creation of the product by calling the `ProductType::ProductType(const ConcreteFactory *, const components_type &)` constructor. The factory also supports parameters by using the `ParametersType` structure, which is defined by the user.

For a simple example, see IntFactory in `core/test/base/abstract_factory.cpp`.

**Template Parameters**

| | |
|---|---|
| *ConcreteFactory* | the concrete factory which is being implemented [CRTP parameter] |
| *ProductType* | the concrete type of products which this factory produces, has to be a subclass of PolymorphicBase::abstract_product_type |
| *ParametersType* | a type representing the parameters of the factory, has to inherit from the enable_parameters_type mixin |
| *PolymorphicBase* | parent of ConcreteFactory in the polymorphic hierarchy, has to be a subclass of AbstractFactory |

## 48.89.2 Member Function Documentation

### 48.89.2.1 create()

```
template<typename ConcreteFactory , typename ProductType , typename ParametersType , typename
PolymorphicBase >
static parameters_type gko::EnableDefaultFactory< ConcreteFactory, ProductType, Parameters↩
Type, PolymorphicBase >::create ( )  [inline], [static]
```

Creates a new ParametersType object which can be used to instantiate a new ConcreteFactory.

This method does not construct the factory directly, but returns a new parameters_type object, which can be used to set the parameters of the factory. Once the parameters have been set, the parameters_type::on() method can be used to obtain an instance of the factory with those parameters.

#### Returns

a default parameters_type object

```
194 { return {}; }
```

### 48.89.2.2 get_parameters()

```
template<typename ConcreteFactory , typename ProductType , typename ParametersType , typename
PolymorphicBase >
const parameters_type& gko::EnableDefaultFactory< ConcreteFactory, ProductType, Parameters↩
Type, PolymorphicBase >::get_parameters ( ) const  [inline], [noexcept]
```

Returns the parameters of the factory.

#### Returns

the parameters of the factory

```
178     {
179         return parameters_;
180     };
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/abstract_factory.hpp

# 48.90 gko::EnableDefaultReusableLinOpFactory< ConcreteFactory, ConcreteLinOp, ParametersType, ReuseDataType, PolymorphicBase > Class Template Reference

EnableDefaultLinOpFactory with LinOpFactory::generate_reuse() support, usually used through GKO_ENABLE_LIN_OP_FACTORY_

```
#include <ginkgo/core/base/lin_op.hpp>
```

### Public Member Functions

- std::unique_ptr< ConcreteLinOp > generate_reuse (std::shared_ptr< const LinOp > input, LinOpFactory::ReuseData &reuse_data) const

  *LinOpFactory::generate_reuse(), returning the concrete LinOp type.*

### Additional Inherited Members

## 48.90.1 Detailed Description

**template<typename ConcreteFactory, typename ConcreteLinOp, typename ParametersType, typename ReuseDataType, typename PolymorphicBase = LinOpFactory>**
**class gko::EnableDefaultReusableLinOpFactory< ConcreteFactory, ConcreteLinOp, ParametersType, ReuseDataType, PolymorphicBase >**

EnableDefaultLinOpFactory with LinOpFactory::generate_reuse() support, usually used through GKO_ENABLE_LIN_OP_FACTORY_

ReuseDataType has to be default-constructible and derived from LinOpFactory::ReuseData, and ConcreteLinOp has to provide

- a constructor `(const ConcreteFactory*, std::shared_ptr<const LinOp>, ReuseDataType&)` keeping the guarantees of LinOpFactory::generate_reuse_impl();
- `static void check_reuse_consistent(const ConcreteFactory*, const LinOp*, const ReuseDataType&)`, accepting any input while the data is empty.

ReuseDataType only has to be complete where the factory's member functions are used. The other template parameters are the same as for EnableDefaultLinOpFactory.

**Note**

This interface is experimental.

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

# 48.91 gko::solver::EnableIterativeBase< DerivedType > Class Template Reference

A LinOp deriving from this CRTP class stores a stopping criterion factory and allows applying with a guess.

`#include <ginkgo/core/solver/solver_base.hpp>`

## Public Member Functions

- EnableIterativeBase & operator= (const EnableIterativeBase &other)

  *Creates a shallow copy of the provided stopping criterion, clones it onto this executor if executors don't match.*
- EnableIterativeBase & operator= (EnableIterativeBase &&other)

  *Moves the provided stopping criterion, clones it onto this executor if executors don't match.*
- EnableIterativeBase (const EnableIterativeBase &other)

  *Creates a shallow copy of the provided stopping criterion.*
- EnableIterativeBase (EnableIterativeBase &&other)

  *Moves the provided stopping criterion.*
- void set_stop_criterion_factory (std::shared_ptr< const stop::CriterionFactory > new_stop_factory) override

  *Sets the stopping criterion of the solver.*

### 48.91.1 Detailed Description

**template<typename DerivedType>**
**class gko::solver::EnableIterativeBase< DerivedType >**

A LinOp deriving from this CRTP class stores a stopping criterion factory and allows applying with a guess.

**Template Parameters**

| | |
|---|---|
| *DerivedType* | the CRTP type that derives from this |

### 48.91.2 Constructor & Destructor Documentation

#### 48.91.2.1 EnableIterativeBase()

```
template<typename DerivedType>
gko::solver::EnableIterativeBase< DerivedType >::EnableIterativeBase (
            EnableIterativeBase< DerivedType > && other )  [inline]
```

Moves the provided stopping criterion.

The moved-from object has a nullptr stopping criterion.

```
775     {
776         *this = std::move(other);
777     }
```

### 48.91.3 Member Function Documentation

#### 48.91.3.1 operator=()

```
template<typename DerivedType>
EnableIterativeBase& gko::solver::EnableIterativeBase< DerivedType >::operator= (
            EnableIterativeBase< DerivedType > && other )  [inline]
```

Moves the provided stopping criterion, clones it onto this executor if executors don't match.

The moved-from object has a nullptr stopping criterion.

```
749     {
750         if (&other != this) {
751             set_stop_criterion_factory(other.get_stop_criterion_factory());
752             other.set_stop_criterion_factory(nullptr);
753         }
754         return *this;
755     }
```

#### 48.91.3.2 set_stop_criterion_factory()

```
template<typename DerivedType>
void gko::solver::EnableIterativeBase< DerivedType >::set_stop_criterion_factory (
            std::shared_ptr< const stop::CriterionFactory > new_stop_factory )  [inline],
[override], [virtual]
```

Sets the stopping criterion of the solver.

**Parameters**

| | |
|---|---|
| *new_stop_factory* | the new stopping criterion factory |

Reimplemented from gko::solver::IterativeBase.

```
781     {
782         auto exec = self()->get_executor();
783         if (new_stop_factory && new_stop_factory->get_executor() != exec) {
784             // no stopping criterion support clone currently though.
785             new_stop_factory = gko::clone(exec, new_stop_factory);
786         }
787         IterativeBase::set_stop_criterion_factory(new_stop_factory);
788     }
```

Referenced by gko::solver::EnableIterativeBase< Chebyshev< ValueType > >::operator=().

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

## 48.92 gko::log::EnableLogging< ConcreteLoggable, PolymorphicBase > Class Template Reference

EnableLogging is a mixin which should be inherited by any class which wants to enable logging.

```
#include <ginkgo/core/log/logger.hpp>
```

### 48.92.1 Detailed Description

template<typename ConcreteLoggable, typename PolymorphicBase = Loggable>
class gko::log::EnableLogging< ConcreteLoggable, PolymorphicBase >

EnableLogging is a mixin which should be inherited by any class which wants to enable logging.

All the received events are passed to the loggers this class contains.

**Template Parameters**

| | |
|---|---|
| *ConcreteLoggable* | the object being logged [CRTP parameter] |
| *PolymorphicBase* | the polymorphic base of this class. By default it is Loggable. Change it if you want to use a new superclass of `Loggable` as polymorphic base of this class. |

The documentation for this class was generated from the following file:

- ginkgo/core/log/logger.hpp

## 48.93 gko::multigrid::EnableMultigridLevel< ValueType > Class Template Reference

The EnableMultigridLevel gives the default implementation of MultigridLevel with composition and provides `set_multigrid_level` function.

`#include <ginkgo/core/multigrid/multigrid_level.hpp>`

### Public Member Functions

- std::shared_ptr< const LinOp > get_fine_op () const override
  *Returns the operator on fine level.*
- std::shared_ptr< const LinOp > get_restrict_op () const override
  *Returns the restrict operator.*
- std::shared_ptr< const LinOp > get_coarse_op () const override
  *Returns the operator on coarse level.*
- std::shared_ptr< const LinOp > get_prolong_op () const override
  *Returns the prolong operator.*

### 48.93.1 Detailed Description

template<typename ValueType>
class gko::multigrid::EnableMultigridLevel< ValueType >

The EnableMultigridLevel gives the default implementation of MultigridLevel with composition and provides `set_multigrid_level` function.

A class inherit from EnableMultigridLevel should use the this->get_compositions()->apply(...) as its own apply, which represents op(b) = prolong(coarse(restrict(b))).

## 48.93.2 Member Function Documentation

### 48.93.2.1 get_coarse_op()

```
template<typename ValueType >
std::shared_ptr<const LinOp> gko::multigrid::EnableMultigridLevel< ValueType >::get_coarse_op
( ) const  [inline], [override], [virtual]
```

Returns the operator on coarse level.

**Returns**

the operator on coarse level.

Implements gko::multigrid::MultigridLevel.

```
98      {
99          return this->get_operator_at(1);
100     }
```

References gko::UseComposition< ValueType >::get_operator_at().

### 48.93.2.2 get_fine_op()

```
template<typename ValueType >
std::shared_ptr<const LinOp> gko::multigrid::EnableMultigridLevel< ValueType >::get_fine_op (
) const  [inline], [override], [virtual]
```

Returns the operator on fine level.

**Returns**

the operator on fine level.

Implements gko::multigrid::MultigridLevel.

```
88      {
89          return fine_op_;
90      }
```

### 48.93.2.3 get_prolong_op()

```
template<typename ValueType >
std::shared_ptr<const LinOp> gko::multigrid::EnableMultigridLevel< ValueType >::get_prolong↩
_op ( ) const  [inline], [override], [virtual]
```

Returns the prolong operator.

**Returns**

the prolong operator.

Implements gko::multigrid::MultigridLevel.

```
103     {
104         return this->get_operator_at(0);
105     }
```

References gko::UseComposition< ValueType >::get_operator_at().

#### 48.93.2.4 get_restrict_op()

```
template<typename ValueType >
std::shared_ptr<const LinOp> gko::multigrid::EnableMultigridLevel< ValueType >::get_restrict↩
_op ( ) const  [inline], [override], [virtual]
```

Returns the restrict operator.

**Returns**

the restrict operator.

Implements gko::multigrid::MultigridLevel.

```
93     {
94         return this->get_operator_at(2);
95     }
```

References gko::UseComposition< ValueType >::get_operator_at().

The documentation for this class was generated from the following file:

- ginkgo/core/multigrid/multigrid_level.hpp

## 48.94 gko::solver::EnablePreconditionable< DerivedType > Class Template Reference

Mixin providing default operation for Preconditionable with correct value semantics.

```
#include <ginkgo/core/solver/solver_base.hpp>
```

### Public Member Functions

- void set_preconditioner (std::shared_ptr< const LinOp > new_precond) override
  *Sets the preconditioner operator used by the Preconditionable.*
- EnablePreconditionable & operator= (const EnablePreconditionable &other)
  *Creates a shallow copy of the provided preconditioner, clones it onto this executor if executors don't match.*
- EnablePreconditionable & operator= (EnablePreconditionable &&other)
  *Moves the provided preconditioner, clones it onto this executor if executors don't match.*
- EnablePreconditionable (const EnablePreconditionable &other)
  *Creates a shallow copy of the provided preconditioner.*
- EnablePreconditionable (EnablePreconditionable &&other)
  *Moves the provided preconditioner.*

### 48.94.1 Detailed Description

**template<typename DerivedType>**
**class gko::solver::EnablePreconditionable< DerivedType >**

Mixin providing default operation for Preconditionable with correct value semantics.

It ensures that the preconditioner stored in this class will always have the same executor as the object this mixin is used in, creating a clone on the correct executor if necessary.

**Template Parameters**

| | |
|---|---|
| *DerivedType* | The type that this Mixin is used in. It must provide get_size() and get_executor() functions that return correctly initialized values when the EnablePreconditionable constructor is called, i.e. the constructor must be provided by a base class added before EnablePreconditionable, since the member initialization order also applying to multiple inheritance. |

## 48.94.2 Constructor & Destructor Documentation

### 48.94.2.1 EnablePreconditionable()

```
template<typename DerivedType>
gko::solver::EnablePreconditionable< DerivedType >::EnablePreconditionable (
            EnablePreconditionable< DerivedType > && other )  [inline]
```

Moves the provided preconditioner.

The moved-from object has a nullptr preconditioner.

```
346     {
347         *this = std::move(other);
348     }
```

## 48.94.3 Member Function Documentation

### 48.94.3.1 operator=()

```
template<typename DerivedType>
EnablePreconditionable& gko::solver::EnablePreconditionable< DerivedType >::operator= (
            EnablePreconditionable< DerivedType > && other )  [inline]
```

Moves the provided preconditioner, clones it onto this executor if executors don't match.

The moved-from object has a nullptr preconditioner.

```
318     {
319         if (&other != this) {
320             set_preconditioner(other.get_preconditioner());
321             other.set_preconditioner(nullptr);
322         }
323         return *this;
324     }
```

### 48.94.3.2 set_preconditioner()

```
template<typename DerivedType>
void gko::solver::EnablePreconditionable< DerivedType >::set_preconditioner (
            std::shared_ptr< const LinOp > new_precond )  [inline], [override], [virtual]
```

Sets the preconditioner operator used by the Preconditionable.

**Parameters**

| | |
|---|---|
| *new_precond* | the new preconditioner operator used by the Preconditionable |

Reimplemented from gko::Preconditionable.

Reimplemented in gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, gko::solver::EnablePreconditionedI
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Idr< ValueType > >, gko::solver::EnablePreconditionedIterativeSolve
gko::solver::EnablePreconditionedIterativeSolver< ValueType, CbGmres< ValueType > >, gko::solver::EnablePreconditionedIterativ
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Bicg< ValueType > >, gko::solver::EnablePreconditionedIterativeSolv
gko::solver::EnablePreconditionedIterativeSolver< ValueType, PipeCg< ValueType > >, gko::solver::EnablePreconditionedIterativeS
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Bicgstab< ValueType > >, gko::solver::EnablePreconditionedIterative
and gko::solver::EnablePreconditionedIterativeSolver< ValueType, Cg< ValueType > >.

```
288     {
289         auto exec = self()->get_executor();
290         if (new_precond) {
291             GKO_ASSERT_EQUAL_DIMENSIONS(self(), new_precond);
292             GKO_ASSERT_IS_SQUARE_MATRIX(new_precond);
293             if (new_precond->get_executor() != exec) {
294                 new_precond = gko::clone(exec, new_precond);
295             }
296         }
297         Preconditionable::set_preconditioner(new_precond);
298     }
```

Referenced by gko::solver::EnablePreconditionable< Chebyshev< ValueType > >::operator=(), and gko::solver↩
::EnablePreconditionedIterativeSolver< ValueType, Cg< ValueType > >::set_preconditioner().

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

# 48.95 gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType > Class Template Reference

A LinOp implementing this interface stores a system matrix and stopping criterion factory.

`#include <ginkgo/core/solver/solver_base.hpp>`

## Public Member Functions

- EnablePreconditionedIterativeSolver & operator= (const EnablePreconditionedIterativeSolver &other)

  *Copies the solver state, cloning the nullspaces onto this executor if the executors don't match.*
- EnablePreconditionedIterativeSolver & operator= (EnablePreconditionedIterativeSolver &&other)

  *Moves the solver state, cloning the nullspaces onto this executor if the executors don't match.*
- std::shared_ptr< const Nullspace< ValueType > > get_nullspace () const
- std::shared_ptr< const Nullspace< ValueType > > get_left_nullspace () const
- void set_preconditioner (std::shared_ptr< const LinOp > new_precond) override

  *Sets the preconditioner.*
- std::shared_ptr< const LinOp > get_unprojected_preconditioner () const

## 48.95.1 Detailed Description

**template<typename ValueType, typename DerivedType>**
**class gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >**

A LinOp implementing this interface stores a system matrix and stopping criterion factory.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type that iterative solver uses for its vectors |
| *DerivedType* | the CRTP type that derives from this |

## 48.95.2 Member Function Documentation

### 48.95.2.1 get_left_nullspace()

```
template<typename ValueType, typename DerivedType>
std::shared_ptr<const Nullspace<ValueType> > gko::solver::EnablePreconditionedIterativeSolver<
ValueType, DerivedType >::get_left_nullspace ( ) const  [inline]
```

**Returns**

the left nullspace $N(A^H)$ (the complement of $\text{range}(A)$), or nullptr if none was set.

```
950     {
951         return left_nullspace_;
952     }
```

### 48.95.2.2 get_nullspace()

```
template<typename ValueType, typename DerivedType>
std::shared_ptr<const Nullspace<ValueType> > gko::solver::EnablePreconditionedIterativeSolver<
ValueType, DerivedType >::get_nullspace ( ) const  [inline]
```

**Returns**

the right nullspace $N(A)$, or nullptr if none was set.

```
941     {
942         return nullspace_;
943     }
```

### 48.95.2.3 get_unprojected_preconditioner()

```
template<typename ValueType, typename DerivedType>
std::shared_ptr<const LinOp> gko::solver::EnablePreconditionedIterativeSolver< ValueType,
DerivedType >::get_unprojected_preconditioner ( ) const  [inline]
```

**Returns**

the preconditioner without its nullspace projection: if a nullspace is set, the preconditioner wrapped by get_preconditioner(), which is a preconditioner::NullspaceProjection, otherwise get_preconditioner() itself.

```
978     {
979         auto precond = this->get_preconditioner();
980         if (nullspace_) {
981             if (auto projection = std::dynamic_pointer_cast<
982                     const ::gko::preconditioner::NullspaceProjection<
983                         ValueType»(precond)) {
984                 return projection->get_preconditioner();
985             }
986         }
987         return precond;
988     }
```

#### 48.95.2.4 operator=()

```
template<typename ValueType, typename DerivedType>
EnablePreconditionedIterativeSolver& gko::solver::EnablePreconditionedIterativeSolver< Value↩
Type, DerivedType >::operator= (
            EnablePreconditionedIterativeSolver< ValueType, DerivedType > && other )  [inline]
```

Moves the solver state, cloning the nullspaces onto this executor if the executors don't match.

The moved-from object has no nullspaces.

```
925     {
926         if (&other != this) {
927             nullspace_ = on_executor(std::exchange(other.nullspace_, nullptr));
928             left_nullspace_ =
929                 on_executor(std::exchange(other.left_nullspace_, nullptr));
930             EnableSolverBase<DerivedType>::operator=(std::move(other));
931             EnableIterativeBase<DerivedType>::operator=(std::move(other));
932             EnablePreconditionable<DerivedType>::operator=(std::move(other));
933         }
934         return *this;
935     }
```

#### 48.95.2.5 set_preconditioner()

```
template<typename ValueType, typename DerivedType>
void gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >::set_preconditioner
(
            std::shared_ptr< const LinOp > new_precond )  [inline], [override], [virtual]
```

Sets the preconditioner.

If a nullspace is set, the preconditioner is wrapped in a preconditioner::NullspaceProjection, unless it already is one that removes the nullspace (and, for solvers projecting on both sides, the left nullspace).

Reimplemented from gko::solver::EnablePreconditionable< DerivedType >.

```
961     {
962         if (new_precond && nullspace_ &&
963             !removes_nullspaces(new_precond.get())) {
964             new_precond = project_preconditioner(std::move(new_precond));
965         }
966         EnablePreconditionable<DerivedType>::set_preconditioner(
967             std::move(new_precond));
968     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

## 48.96 gko::solver::EnableSolverBase< DerivedType, MatrixType > Class Template Reference

A LinOp deriving from this CRTP class stores a system matrix.

```
#include <ginkgo/core/solver/solver_base.hpp>
```

## Public Member Functions

- EnableSolverBase & operator= (const EnableSolverBase &other)

  *Creates a shallow copy of the provided system matrix, clones it onto this executor if executors don't match.*
- EnableSolverBase & operator= (EnableSolverBase &&other)

  *Moves the provided system matrix, clones it onto this executor if executors don't match.*
- EnableSolverBase (const EnableSolverBase &other)

  *Creates a shallow copy of the provided system matrix.*
- EnableSolverBase (EnableSolverBase &&other)

  *Moves the provided system matrix.*
- std::vector< int > get_workspace_scalars () const override

  *Returns the IDs of all scalars (workspace vectors with system dimension-independent size, usually 1 x num_rhs).*
- std::vector< int > get_workspace_vectors () const override

  *Returns the IDs of all vectors (workspace vectors with system dimension-dependent size, usually system_matrix_size x num_rhs).*

### 48.96.1 Detailed Description

**template<typename DerivedType, typename MatrixType = LinOp>**
**class gko::solver::EnableSolverBase< DerivedType, MatrixType >**

A LinOp deriving from this CRTP class stores a system matrix.

**Template Parameters**

| | |
|---|---|
| *DerivedType* | the CRTP type that derives from this |
| *MatrixType* | the concrete matrix type to be stored as system_matrix |

### 48.96.2 Constructor & Destructor Documentation

#### 48.96.2.1 EnableSolverBase()

```
template<typename DerivedType, typename MatrixType = LinOp>
gko::solver::EnableSolverBase< DerivedType, MatrixType >::EnableSolverBase (
            EnableSolverBase< DerivedType, MatrixType > && other )  [inline]
```

Moves the provided system matrix.

The moved-from object has a nullptr system matrix.

```
614         :  SolverBase<MatrixType>{other.self()->get_executor()}
615     {
616         *this = std::move(other);
617     }
```

### 48.96.3 Member Function Documentation

#### 48.96.3.1 operator=()

```
template<typename DerivedType, typename MatrixType = LinOp>
EnableSolverBase& gko::solver::EnableSolverBase< DerivedType, MatrixType >::operator= (
            EnableSolverBase< DerivedType, MatrixType > && other )  [inline]
```

Moves the provided system matrix, clones it onto this executor if executors don't match.

The moved-from object has a nullptr system matrix.

```
584     {
585         if (&other != this) {
586             set_system_matrix(other.get_system_matrix());
587             other.set_system_matrix(nullptr);
588         }
589         return *this;
590     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

## 48.97 gko::experimental::mpi::environment Class Reference

Class that sets up and finalizes the MPI environment.

`#include <ginkgo/core/base/mpi.hpp>`

### Public Member Functions

- int get_provided_thread_support () const
  *Return the provided thread support.*
- environment (int &argc, char **&argv, const thread_type thread_t=thread_type::serialized)
  *Call MPI_Init_thread and initialize the MPI environment.*
- ~environment ()
  *Call MPI_Finalize at the end of the scope of this class.*

### 48.97.1 Detailed Description

Class that sets up and finalizes the MPI environment.

This class is a simple RAII wrapper to MPI_Init and MPI_Finalize.

MPI_Init must have been called before calling any MPI functions.

**Note**

If MPI_Init has already been called, then this class should not be used.

### 48.97.2 Constructor & Destructor Documentation

#### 48.97.2.1 environment()

```
gko::experimental::mpi::environment::environment (
            int & argc,
            char **& argv,
            const thread_type thread_t = thread_type::serialized )  [inline]
```

Call MPI_Init_thread and initialize the MPI environment.

**Parameters**

| | |
|---|---|
| *argc* | the number of arguments to the main function. |
| *argv* | the arguments provided to the main function. |
| *thread_t* | the type of threading for initialization. See @thread_type |

```
242     {
243         this->required_thread_support_ = static_cast<int>(thread_t);
244         GKO_ASSERT_NO_MPI_ERRORS(
245             MPI_Init_thread(&argc, &argv, this->required_thread_support_,
246                             &(this->provided_thread_support_)));
247     }
```

### 48.97.3 Member Function Documentation

#### 48.97.3.1 get_provided_thread_support()

```
int gko::experimental::mpi::environment::get_provided_thread_support ( ) const  [inline]
```

Return the provided thread support.

**Returns**

the provided thread support

```
230 { return provided_thread_support_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/mpi.hpp

## 48.98 gko::Error Class Reference

The Error class is used to report exceptional behaviour in library functions.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- Error (const std::string &file, int line, const std::string &what)

  *Initializes an error.*

### 48.98.1 Detailed Description

The Error class is used to report exceptional behaviour in library functions.

Ginkgo uses C++ exception mechanism to this end, and the Error class represents a base class for all types of errors. The exact list of errors which could occur during the execution of a certain library routine is provided in the documentation of that routine, along with a short description of the situation when that error can occur. During runtime, these errors can be detected by using standard C++ try-catch blocks, and a human-readable error description can be obtained by calling the Error::what() method.

As an example, trying to compute a matrix-vector product with arguments of incompatible size will result in a DimensionMismatch error, which is demonstrated in the following program.

```
#include <ginkgo.h>
#include <iostream>
using namespace gko;
int main()
{
    auto omp = create<OmpExecutor>();
    auto A = randn_fill<matrix::Csr<float»(5, 5, 0f, 1f, omp);
    auto x = fill<matrix::Dense<float»(6, 1, 1f, omp);
    try {
        auto y = apply(A, x);
    } catch(Error e) {
        // an error occurred, write the message to screen and exit
        std::cout « e.what() « std::endl;
        return -1;
    }
    return 0;
}
```

### 48.98.2 Constructor & Destructor Documentation

#### 48.98.2.1 Error()

```
gko::Error::Error (
            const std::string & file,
            int line,
            const std::string & what )  [inline]
```

Initializes an error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *what* | The error message |

```
68         :  std::runtime_error(file + ":" + std::to_string(line) + ":  " + what)
69     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.99 gko::Executor Class Reference

The first step in using the Ginkgo library consists of creating an executor.

`#include <ginkgo/core/base/executor.hpp>`

## Public Member Functions

- virtual void run (const Operation &op) const =0

  *Runs the specified Operation using this Executor.*

- template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

- template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩ Dpcpp >
  void run (std::string name, const ClosureReference &op_ref, const ClosureOmp &op_omp, const Closure↩ Cuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

- template<typename T >
  T * alloc (size_type num_elems) const

  *Allocates memory in this Executor.*

- void free (void *ptr) const noexcept

  *Frees memory previously allocated with Executor::alloc().*

- template<typename T >
  void copy_from (ptr_param< const Executor > src_exec, size_type num_elems, const T *src_ptr, T *dest↩ _ptr) const

  *Copies data from another Executor.*

- template<typename T >
  void copy (size_type num_elems, const T *src_ptr, T *dest_ptr) const

  *Copies data within this Executor.*

- template<typename T >
  T copy_val_to_host (const T *ptr) const

  *Retrieves a single element at the given location from executor memory.*

- virtual std::shared_ptr< Executor > get_master () noexcept=0

  *Returns the master OmpExecutor of this Executor.*

- virtual std::shared_ptr< const Executor > get_master () const noexcept=0

  *Returns the master OmpExecutor of this Executor.*

- virtual void synchronize () const =0

  *Synchronize the operations launched on the executor with its master.*

- void add_logger (std::shared_ptr< const log::Logger > logger) override
- void remove_logger (const log::Logger *logger) override
- void set_log_propagation_mode (log_propagation_mode mode)

  *Sets the logger event propagation mode for the executor.*

- bool should_propagate_log () const

  *Returns true iff events occurring at an object created on this executor should be logged at propagating loggers attached to this executor, and there is at least one such propagating logger.*

- bool memory_accessible (const std::shared_ptr< const Executor > &other) const

  *Verifies whether the executors share the same memory.*

- virtual std::string get_description () const =0

### 48.99.1 Detailed Description

The first step in using the Ginkgo library consists of creating an executor.

Executors are used to specify the location for the data of linear algebra objects, and to determine where the operations will be executed. Ginkgo currently supports five different executor types:

- OmpExecutor specifies that the data should be stored and the associated operations executed on an OpenMP-supporting device (e.g. host CPU);
- CudaExecutor specifies that the data should be stored and the operations executed on the NVIDIA GPU accelerator;
- HipExecutor specifies that the data should be stored and the operations executed on either an NVIDIA or AMD GPU accelerator;
- DpcppExecutor specifies that the data should be stored and the operations executed on an hardware supporting DPC++;
- ReferenceExecutor executes a non-optimized reference implementation, which can be used to debug the library.

The following code snippet demonstrates the simplest possible use of the Ginkgo library:

```
auto omp = gko::create<gko::OmpExecutor>();
auto A = gko::read_from_mtx<gko::matrix::Csr<float»("A.mtx", omp);
```

First, we create a OMP executor, which will be used in the next line to specify where we want the data for the matrix A to be stored. The second line will read a matrix from the matrix market file 'A.mtx', and store the data on the CPU in CSR format (gko::matrix::Csr is a Ginkgo matrix class which stores its data in CSR format). At this point, matrix A is bound to the CPU, and any routines called on it will be performed on the CPU. This approach is usually desired in sparse linear algebra, as the cost of individual operations is several orders of magnitude lower than the cost of copying the matrix to the GPU.

If matrix A is going to be reused multiple times, it could be beneficial to copy it over to the accelerator, and perform the operations there, as demonstrated by the next code snippet:

```
auto cuda = gko::create<gko::CudaExecutor>(0, omp);
auto dA = gko::copy_to<gko::matrix::Csr<float»(A.get(), cuda);
```

The first line of the snippet creates a new CUDA executor. Since there may be multiple NVIDIA GPUs present on the system, the first parameter instructs the library to use the first device (i.e. the one with device ID zero, as in cudaSetDevice() routine from the CUDA runtime API). In addition, since GPUs are not stand-alone processors, it is required to pass a "master" OmpExecutor which will be used to schedule the requested CUDA kernels on the accelerator.

The second command creates a copy of the matrix A on the GPU. Notice the use of the get() method. As Ginkgo aims to provide automatic memory management of its objects, the result of calling gko::read_from_mtx() is a smart pointer (std::unique_ptr) to the created object. On the other hand, as the library will not hold a reference to A once the copy is completed, the input parameter for gko::copy_to() is a plain pointer. Thus, the get() method is used to convert from a std::unique_ptr to a plain pointer, as expected by gko::copy_to().

As a side note, the gko::copy_to routine is far more powerful than just copying data between different devices. It can also be used to convert data between different formats. For example, if the above code used gko::matrix::Ell as the template parameter, dA would be stored on the GPU, in ELLPACK format.

Finally, if all the processing of the matrix is supposed to be done on the GPU, and a CPU copy of the matrix is not required, we could have read the matrix to the GPU directly:

```
auto omp = gko::create<gko::OmpExecutor>();
auto cuda = gko::create<gko::CudaExecutor>(0, omp);
auto dA = gko::read_from_mtx<gko::matrix::Csr<float»("A.mtx", cuda);
```

Notice that even though reading the matrix directly from a file to the accelerator is not supported, the library is designed to abstract away the intermediate step of reading the matrix to the CPU memory. This is a general design approach taken by the library: in case an operation is not supported by the device, the data will be copied to the CPU, the operation performed there, and finally the results copied back to the device. This approach makes using the library more concise, as explicit copies are not required by the user. Nevertheless, this feature should be taken into account when considering performance implications of using such operations.

## 48.99.2 Member Function Documentation

### 48.99.2.1 add_logger()

```
void gko::Executor::add_logger (
            std::shared_ptr< const log::Logger > logger )  [inline], [override], [virtual]
```

**Note**

This specialization keeps track of whether any propagating loggers were attached to the executor.

**See also**

Logger::needs_propagation()

Implements gko::log::Loggable.

```
841     {
842         this->propagating_logger_refcount_.fetch_add(
843             logger->needs_propagation() ?  1 :  0);
844         this->EnableLogging<Executor>::add_logger(logger);
845     }
```

### 48.99.2.2 alloc()

```
template<typename T >
T* gko::Executor::alloc (
            size_type num_elems ) const  [inline]
```

Allocates memory in this Executor.

**Template Parameters**

| | |
|---|---|
| *T* | datatype to allocate |

**Parameters**

| | |
|---|---|
| *num_elems* | number of elements of type T to allocate |

**Exceptions**

| | |
|---|---|
| *AllocationError* | if the allocation failed |

**Returns**

pointer to allocated memory

```
708     {
709         this->template log<log::Logger::allocation_started>(
710             this, num_elems * sizeof(T));
711         T* allocated = static_cast<T*>(this->raw_alloc(num_elems * sizeof(T)));
712         this->template log<log::Logger::allocation_completed>(
713             this, num_elems * sizeof(T), reinterpret_cast<uintptr>(allocated));
714         return allocated;
715     }
```

### 48.99.2.3 copy()

```
template<typename T >
void gko::Executor::copy (
             size_type num_elems,
             const T * src_ptr,
             T * dest_ptr ) const  [inline]
```

Copies data within this Executor.

#### Template Parameters

| | |
|---|---|
| *T* | datatype to copy |

#### Parameters

| | |
|---|---|
| *num_elems* | number of elements of type T to copy |
| *src_ptr* | pointer to a block of memory containing the data to be copied |
| *dest_ptr* | pointer to an allocated block of memory where the data will be copied to |

```
797     {
798         this->copy_from(this, num_elems, src_ptr, dest_ptr);
799     }
```

References copy_from().

### 48.99.2.4 copy_from()

```
template<typename T >
void gko::Executor::copy_from (
             ptr_param< const Executor > src_exec,
             size_type num_elems,
             const T * src_ptr,
             T * dest_ptr ) const  [inline]
```

Copies data from another Executor.

#### Template Parameters

| | |
|---|---|
| *T* | datatype to copy |

**Parameters**

| | |
|---|---|
| *src_exec* | Executor from which the memory will be copied |
| *num_elems* | number of elements of type T to copy |
| *src_ptr* | pointer to a block of memory containing the data to be copied |
| *dest_ptr* | pointer to an allocated block of memory where the data will be copied to |

```
748     {
749         const auto src_loc = reinterpret_cast<uintptr>(src_ptr);
750         const auto dest_loc = reinterpret_cast<uintptr>(dest_ptr);
751         this->template log<log::Logger::copy_started>(
752             src_exec.get(), this, src_loc, dest_loc, num_elems * sizeof(T));
753         if (this != src_exec.get()) {
754             src_exec->template log<log::Logger::copy_started>(
755                 src_exec.get(), this, src_loc, dest_loc, num_elems * sizeof(T));
756         }
757         try {
758             this->raw_copy_from(src_exec.get(), num_elems * sizeof(T), src_ptr,
759                                 dest_ptr);
760         } catch (NotSupported&) {
761 #if (GKO_VERBOSE_LEVEL >= 1) && !defined(NDEBUG)
762             // Unoptimized copy.  Try to go through the masters.
763             // output to log when verbose >= 1 and debug build
764             std::cerr « "Not direct copy.  Try to copy data from the masters."
765                      « std::endl;
766 #endif
767             auto src_master = src_exec->get_master().get();
768             if (num_elems > 0 && src_master != src_exec.get()) {
769                 auto* master_ptr = src_exec->get_master()->alloc<T>(num_elems);
770                 src_master->copy_from<T>(src_exec, num_elems, src_ptr,
771                                          master_ptr);
772                 this->copy_from<T>(src_master, num_elems, master_ptr, dest_ptr);
773                 src_master->free(master_ptr);
774             }
775         }
776         this->template log<log::Logger::copy_completed>(
777             src_exec.get(), this, src_loc, dest_loc, num_elems * sizeof(T));
778         if (this != src_exec.get()) {
779             src_exec->template log<log::Logger::copy_completed>(
780                 src_exec.get(), this, src_loc, dest_loc, num_elems * sizeof(T));
781         }
782     }
```

References gko::ptr_param< T >::get().

Referenced by copy().

### 48.99.2.5 copy_val_to_host()

```
template<typename T >
T gko::Executor::copy_val_to_host (
            const T * ptr ) const  [inline]
```

Retrieves a single element at the given location from executor memory.

**Template Parameters**

| | |
|---|---|
| *T* | datatype to copy |

**Parameters**

| | |
|---|---|
| *ptr* | the pointer to the element to be copied |

**Returns**

the value stored at ptr

```
812     {
813         T out{};
814         this->get_master()->copy_from(this, 1, ptr, &out);
815         return out;
816     }
```

References get_master().

### 48.99.2.6 free()

```
void gko::Executor::free (
            void * ptr ) const  [inline], [noexcept]
```

Frees memory previously allocated with Executor::alloc().

If `ptr` is a `nullptr`, the function has no effect.

**Parameters**

| | |
|---|---|
| *ptr* | pointer to the allocated memory block |

```
725     {
726         this->template log<log::Logger::free_started>(
727             this, reinterpret_cast<uintptr>(ptr));
728         this->raw_free(ptr);
729         this->template log<log::Logger::free_completed>(
730             this, reinterpret_cast<uintptr>(ptr));
731     }
```

### 48.99.2.7 get_description()

```
virtual std::string gko::Executor::get_description ( ) const  [pure virtual]
```

**Returns**

a textual representation of the executor and its device.

Implemented in gko::DpcppExecutor, gko::HipExecutor, gko::CudaExecutor, gko::ReferenceExecutor, and gko::OmpExecutor.

### 48.99.2.8 get_master() [1/2]

```
virtual std::shared_ptr<const Executor> gko::Executor::get_master ( ) const  [pure virtual],
[noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implemented in gko::DpcppExecutor, gko::HipExecutor, gko::CudaExecutor, and gko::OmpExecutor.

#### 48.99.2.9 get_master() [2/2]

```
virtual std::shared_ptr<Executor> gko::Executor::get_master ( )  [pure virtual], [noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implemented in gko::DpcppExecutor, gko::HipExecutor, gko::CudaExecutor, and gko::OmpExecutor.

Referenced by copy_val_to_host().

#### 48.99.2.10 memory_accessible()

```
bool gko::Executor::memory_accessible (
            const std::shared_ptr< const Executor > & other ) const  [inline]
```

Verifies whether the executors share the same memory.

**Parameters**

| | |
|---|---|
| *other* | the other Executor to compare against |

**Returns**

whether the executors this and other share the same memory.

```
895     {
896         return this->verify_memory_from(other.get());
897     }
```

#### 48.99.2.11 remove_logger()

```
void gko::Executor::remove_logger (
            const log::Logger * logger )  [inline], [override], [virtual]
```

**Note**

This specialization keeps track of whether any propagating loggers were attached to the executor.

**See also**

Logger::needs_propagation()

Implements gko::log::Loggable.

```
854     {
855         this->propagating_logger_refcount_.fetch_sub(
856             logger->needs_propagation() ?  1 :  0);
857         this->EnableLogging<Executor>::remove_logger(logger);
858     }
```

### 48.99.2.12 run() [1/3]

```
template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
Dpcpp >
void gko::Executor::run (
            const ClosureOmp & op_omp,
            const ClosureCuda & op_cuda,
            const ClosureHip & op_hip,
            const ClosureDpcpp & op_dpcpp ) const  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

#### Template Parameters

| | |
|---|---|
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

#### Parameters

| | |
|---|---|
| *op_omp* | functor to run in case of a OmpExecutor or ReferenceExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
660     {
661         LambdaOperation<ClosureOmp, ClosureOmp, ClosureCuda, ClosureHip,
662                         ClosureDpcpp>
663             op(op_omp, op_cuda, op_hip, op_dpcpp);
664         this->run(op);
665     }
```

References run().

### 48.99.2.13 run() [2/3]

```
virtual void gko::Executor::run (
            const Operation & op ) const  [pure virtual]
```

Runs the specified Operation using this Executor.

#### Parameters

| | |
|---|---|
| *op* | the operation to run |

Implemented in gko::ReferenceExecutor.

Referenced by run().

#### 48.99.2.14 run() [3/3]

```
template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename
ClosureHip , typename ClosureDpcpp >
void gko::Executor::run (
            std::string name,
            const ClosureReference & op_ref,
            const ClosureOmp & op_omp,
            const ClosureCuda & op_cuda,
            const ClosureHip & op_hip,
            const ClosureDpcpp & op_dpcpp ) const  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureReference* | type of op_ref |
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *name* | the name of the operation |
| *op_ref* | functor to run in case of a ReferenceExecutor |
| *op_omp* | functor to run in case of a OmpExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
688     {
689         LambdaOperation<ClosureReference, ClosureOmp, ClosureCuda, ClosureHip,
690                         ClosureDpcpp>
691             op(std::move(name), op_ref, op_omp, op_cuda, op_hip, op_dpcpp);
692         this->run(op);
693     }
```

References run().

#### 48.99.2.15 set_log_propagation_mode()

```
void gko::Executor::set_log_propagation_mode (
            log_propagation_mode mode )  [inline]
```

Sets the logger event propagation mode for the executor.

This controls whether events that happen at objects created on this executor will also be logged at propagating loggers attached to the executor.

**See also**

Logger::needs_propagation()

```
870     {
871         log_propagation_mode_ = mode;
872     }
```

#### 48.99.2.16 should_propagate_log()

```
bool gko::Executor::should_propagate_log ( ) const  [inline]
```

Returns true iff events occurring at an object created on this executor should be logged at propagating loggers attached to this executor, and there is at least one such propagating logger.

**See also**

Logger::needs_propagation()

Executor::set_log_propagation_mode(log_propagation_mode)

```
882     {
883         return this->propagating_logger_refcount_.load() > 0 &&
884                log_propagation_mode_ == log_propagation_mode::automatic;
885     }
```

References gko::automatic.

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

## 48.100 gko::log::executor_data Struct Reference

Struct representing Executor related data.

`#include <ginkgo/core/log/record.hpp>`

### 48.100.1 Detailed Description

Struct representing Executor related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.101 gko::executor_deleter< T > Class Template Reference

This is a deleter that uses an executor's `free` method to deallocate the data.

`#include <ginkgo/core/base/executor.hpp>`

### Public Member Functions

- executor_deleter (std::shared_ptr< const Executor > exec)

  *Creates a new deleter.*
- void operator() (pointer ptr) const

  *Deletes the object.*

### 48.101.1 Detailed Description

**template<typename T>**
**class gko::executor_deleter< T >**

This is a deleter that uses an executor's `free` method to deallocate the data.

**Template Parameters**

| | |
|---|---|
| *T* | the type of object being deleted |

## 48.101.2 Constructor & Destructor Documentation

### 48.101.2.1 executor_deleter()

```
template<typename T >
gko::executor_deleter< T >::executor_deleter (
            std::shared_ptr< const Executor > exec )  [inline], [explicit]
```

Creates a new deleter.

**Parameters**

| | |
|---|---|
| *exec* | the executor used to free the data |

```
1211         :  exec_{exec}
1212     {}
```

## 48.101.3 Member Function Documentation

### 48.101.3.1 operator()()

```
template<typename T >
void gko::executor_deleter< T >::operator() (
            pointer ptr ) const  [inline]
```

Deletes the object.

**Parameters**

| | |
|---|---|
| *ptr* | pointer to the object being deleted |

```
1220     {
1221         if (exec_) {
1222             exec_->free(ptr);
1223         }
1224     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

# 48.102 gko::experimental::factorization::Factorization< ValueType, IndexType > Class Template Reference

Represents a generic factorization consisting of two triangular factors (upper and lower) and an optional diagonal scaling matrix.

`#include <ginkgo/core/factorization/factorization.hpp>`

## Public Member Functions

- std::unique_ptr< Factorization > unpack () const

  *Transforms the factorization from a compact representation suitable only for triangular solves to a composition representation that can also be used to access individual factors and multiply with the factorization.*
- storage_type get_storage_type () const

  *Returns the storage type used by this factorization.*
- std::shared_ptr< const matrix_type > get_lower_factor () const

  *Returns the lower triangular factor of the factorization, if available, nullptr otherwise.*
- std::shared_ptr< const diag_type > get_diagonal () const

  *Returns the diagonal scaling matrix of the factorization, if available, nullptr otherwise.*
- std::shared_ptr< const matrix_type > get_upper_factor () const

  *Returns the upper triangular factor of the factorization, if available, nullptr otherwise.*
- std::shared_ptr< const matrix_type > get_combined () const

  *Returns the matrix storing a compact representation of the factorization, if available, nullptr otherwise.*
- Factorization (const Factorization &)

  *Creates a deep copy of the factorization.*
- Factorization (Factorization &&)

  *Moves from the given factorization, leaving it empty.*

## Static Public Member Functions

- static std::unique_ptr< Factorization > create_from_composition (std::unique_ptr< composition_type > composition)

  *Creates a Factorization from an existing composition.*
- static std::unique_ptr< Factorization > create_from_symm_composition (std::unique_ptr< composition_type > composition)

  *Creates a Factorization from an existing symmetric composition.*
- static std::unique_ptr< Factorization > create_from_combined_lu (std::unique_ptr< matrix_type > matrix)

  *Creates a Factorization from an existing combined representation of an LU factorization.*

### 48.102.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::experimental::factorization::Factorization< ValueType, IndexType >**

Represents a generic factorization consisting of two triangular factors (upper and lower) and an optional diagonal scaling matrix.

This class is used to represent a wide range of different factorizations to be passed on to direct solvers and other similar operations. The storage_type represents how the individual factors are stored internally: They may be stored as separate matrices or in a single matrix, and be symmetric or unsymmetric, with the diagonal belonging to both factory, a single factor or being a separate scaling factor (Cholesky vs. LDL^H vs. LU vs. LDU).

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store the factorization entries |
| *IndexType* | the index type used to represent the sparsity pattern |

## 48.102.2 Member Function Documentation

### 48.102.2.1 create_from_combined_lu()

```
template<typename ValueType , typename IndexType >
static std::unique_ptr<Factorization> gko::experimental::factorization::Factorization< Value↩
Type, IndexType >::create_from_combined_lu (
            std::unique_ptr< matrix_type > matrix )  [static]
```

Creates a Factorization from an existing combined representation of an LU factorization.

**Parameters**

| | |
|---|---|
| *matrix* | the composition consisting of 2 or 3 elements. We expect the first entry to be a lower triangular matrix, and the last entry to be the transpose of the first entry. If the composition has 3 elements, we expect the middle entry to be a diagonal matrix. |

**Returns**

a symmetric Factorization storing the elements from the Composition.

### 48.102.2.2 create_from_composition()

```
template<typename ValueType , typename IndexType >
static std::unique_ptr<Factorization> gko::experimental::factorization::Factorization< Value↩
Type, IndexType >::create_from_composition (
            std::unique_ptr< composition_type > composition )  [static]
```

Creates a Factorization from an existing composition.

**Parameters**

| | |
|---|---|
| *composition* | the composition consisting of 2 or 3 elements. We expect the first entry to be a lower triangular matrix, and the last entry to be an upper triangular matrix. If the composition has 3 elements, we expect the middle entry to be a diagonal matrix. |

**Returns**

a Factorization storing the elements from the Composition.

#### 48.102.2.3 create_from_symm_composition()

```
template<typename ValueType , typename IndexType >
static std::unique_ptr<Factorization> gko::experimental::factorization::Factorization< Value↩
Type, IndexType >::create_from_symm_composition (
            std::unique_ptr< composition_type > composition )  [static]
```

Creates a Factorization from an existing symmetric composition.

**Parameters**

| | |
|---|---|
| *composition* | the composition consisting of 2 or 3 elements. We expect the first entry to be a lower triangular matrix, and the last entry to be the transpose of the first entry. If the composition has 3 elements, we expect the middle entry to be a diagonal matrix. |

**Returns**

a symmetric Factorization storing the elements from the Composition.

#### 48.102.2.4 unpack()

```
template<typename ValueType , typename IndexType >
std::unique_ptr<Factorization> gko::experimental::factorization::Factorization< ValueType,
IndexType >::unpack ( ) const
```

Transforms the factorization from a compact representation suitable only for triangular solves to a composition representation that can also be used to access individual factors and multiply with the factorization.

**Returns**

a new Factorization object containing this factorization represented as storage_type::composition.

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/factorization.hpp

## 48.103 gko::matrix::Fbcsr< ValueType, IndexType > Class Template Reference

Fixed-block compressed sparse row storage matrix format.

```
#include <ginkgo/core/matrix/fbcsr.hpp>
```

## Public Member Functions

- void convert_to (Csr< ValueType, IndexType > *result) const override

  *Converts the matrix to CSR format.*
- void convert_to (SparsityCsr< ValueType, IndexType > *result) const override

  *Get the block sparsity pattern in CSR-like format.*
- void read (const mat_data &data) override

  *Reads a matrix_data into Fbcsr format.*
- void read (const device_mat_data &data) override

  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override

  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override

  *Writes a matrix to a matrix_data structure.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- void sort_by_column_index ()

  *Sorts the values blocks and block-column indices in each row by column index.*
- bool is_sorted_by_column_index () const

  *Tests if all row entry pairs (value, col_idx) are sorted by column index.*
- value_type * get_values () noexcept
- const value_type * get_const_values () const noexcept
- index_type * get_col_idxs () noexcept
- const index_type * get_const_col_idxs () const noexcept
- index_type * get_row_ptrs () noexcept
- const index_type * get_const_row_ptrs () const noexcept
- size_type get_num_stored_elements () const noexcept
- size_type get_num_stored_blocks () const noexcept
- int get_block_size () const noexcept
- index_type get_num_block_rows () const noexcept
- index_type get_num_block_cols () const noexcept
- Fbcsr & operator= (const Fbcsr &)

  *Copy-assigns an Fbcsr matrix.*
- Fbcsr & operator= (Fbcsr &&)

  *Move-assigns an Fbcsr matrix.*
- Fbcsr (const Fbcsr &)

  *Copy-constructs an Ell matrix.*
- Fbcsr (Fbcsr &&)

  *Move-constructs an Fbcsr matrix.*

## Static Public Member Functions

- static std::unique_ptr< Fbcsr > create (std::shared_ptr< const Executor > exec, int block_size=1)

  *Creates an uninitialized FBCSR matrix with the given block size.*
- static std::unique_ptr< Fbcsr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, size_type num_nonzeros, int block_size)

  *Creates an uninitialized FBCSR matrix of the specified size.*
- static std::unique_ptr< Fbcsr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, int block_size, array< value_type > values, array< index_type > col_idxs, array< index_type > row_ptrs)

  *Creates a FBCSR matrix from already allocated (and initialized) row pointer, column index and value arrays.*
- template<typename InputValueType , typename InputColumnIndexType , typename InputRowPtrType >
  static std::unique_ptr< Fbcsr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, int block_size, std::initializer_list< InputValueType > values, std::initializer_list< InputColumnIndexType > col_idxs, std::initializer_list< InputRowPtrType > row_ptrs)

  *create(std::shared_ptr<const Executor>,*
- static std::unique_ptr< const Fbcsr > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, int blocksize, gko::detail::const_array_view< ValueType > &&values, gko::detail::const_array_↩ view< IndexType > &&col_idxs, gko::detail::const_array_view< IndexType > &&row_ptrs)

  *Creates a constant (immutable) Fbcsr matrix from a constant array.*

### 48.103.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Fbcsr< ValueType, IndexType >**

Fixed-block compressed sparse row storage matrix format.

FBCSR is a matrix format meant for matrices having a natural block structure made up of small, dense, disjoint blocks. It is similar to CSR

**See also**

Csr. However, unlike Csr, each non-zero location stores a small dense block of entries having a constant size. This reduces the number of integers that need to be stored in order to refer to a given non-zero entry, and enables efficient implementation of certain block methods.

The block size is expected to be known in advance and passed to the constructor.

**Note**

The total number of rows and the number of columns are expected to be divisible by the block size.

The nonzero elements are stored in a 1D array row-wise, and accompanied with a row pointer array which stores the starting index of each block-row. An additional block-column index array is used to identify the block-column of each nonzero block.

The Fbcsr LinOp supports different operations:

```
matrix::Fbcsr *A, *B, *C;      // matrices
matrix::Dense *b, *x;        // vectors tall-and-skinny matrices
matrix::Dense *alpha, *beta; // scalars of dimension 1x1
// Applying to Dense matrices computes an SpMV/SpMM product
A->apply(b, x)              // x = A*b
A->apply(alpha, b, beta, x) // x = alpha*A*b + beta*x
```

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.103.2 Constructor & Destructor Documentation

### 48.103.2.1 Fbcsr() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Fbcsr< ValueType, IndexType >::Fbcsr (
            const Fbcsr< ValueType, IndexType > &  )
```

Copy-constructs an Ell matrix.

Inherits executor and data.

### 48.103.2.2 Fbcsr() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Fbcsr< ValueType, IndexType >::Fbcsr (
            Fbcsr< ValueType, IndexType > &&  )
```

Move-constructs an Fbcsr matrix.

Inherits executor and data. The moved-from object is empty (0x0 with no nonzeros, but valid row pointers).

## 48.103.3 Member Function Documentation

### 48.103.3.1 compute_absolute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<absolute_type> gko::matrix::Fbcsr< ValueType, IndexType >::compute_absolute (
) const  [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Fbcsr< ValueType, IndexType > > >.

#### 48.103.3.2 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Fbcsr< ValueType, IndexType >::conj_transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.103.3.3 convert_to() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Fbcsr< ValueType, IndexType >::convert_to (
            Csr< ValueType, IndexType > * result ) const  [override]
```

Converts the matrix to CSR format.

**Note**

Any explicit zeros in the original matrix are retained in the converted result.

#### 48.103.3.4 convert_to() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Fbcsr< ValueType, IndexType >::convert_to (
            SparsityCsr< ValueType, IndexType > * result ) const  [override]
```

Get the block sparsity pattern in CSR-like format.

**Note**

The actual non-zero values are never copied; the result always has a value array of size 1 with the value 1.

#### 48.103.3.5 create() [1/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Fbcsr> gko::matrix::Fbcsr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            int block_size,
            array< value_type > values,
            array< index_type > col_idxs,
            array< index_type > row_ptrs )  [static]
```

Creates a FBCSR matrix from already allocated (and initialized) row pointer, column index and value arrays.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *block_size* | Size of the small square dense nonzero blocks |
| *values* | the value array of the matrix, stored in column-major order for each block |
| *col_idxs* | the block column index array of the matrix |
| *row_ptrs* | the block row pointer array of the matrix |

**Note**

If one of `row_ptrs`, `col_idxs` or `values` is not an rvalue, not an array of IndexType, IndexType and ValueType, respectively, or is on the wrong executor, an internal copy of that array will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

### 48.103.3.6 create() [2/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename InputValueType , typename InputColumnIndexType , typename InputRowPtrType >
static std::unique_ptr<Fbcsr> gko::matrix::Fbcsr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            int block_size,
            std::initializer_list< InputValueType > values,
            std::initializer_list< InputColumnIndexType > col_idxs,
            std::initializer_list< InputRowPtrType > row_ptrs )  [inline], [static]
```

create(std::shared_ptr<const Executor>,

create(std::shared_ptr<const Executor>, const dim<2>&, int, array<value_type>, array<index_type>, array<index_type>)

```
412     {
413         return create(exec, size, block_size,
414                       array<value_type>{exec, std::move(values)},
415                       array<index_type>{exec, std::move(col_idxs)},
416                       array<index_type>{exec, std::move(row_ptrs)});
417     }
```

References gko::matrix::Fbcsr< ValueType, IndexType >::create().

### 48.103.3.7 create() [3/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Fbcsr> gko::matrix::Fbcsr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            size_type num_nonzeros,
            int block_size )  [static]
```

Creates an uninitialized FBCSR matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_nonzeros* | number of stored nonzeros. It needs to be a multiple of block_size * block_size. |
| *block_size* | size of the small dense square blocks |

**Returns**

A smart pointer to the newly created matrix.

### 48.103.3.8 create() [4/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Fbcsr> gko::matrix::Fbcsr< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            int block_size = 1 )  [static]
```

Creates an uninitialized FBCSR matrix with the given block size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *block_size* | The desired size of the dense square nonzero blocks; defaults to 1. |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::matrix::Fbcsr< ValueType, IndexType >::create().

### 48.103.3.9 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const Fbcsr> gko::matrix::Fbcsr< ValueType, IndexType >::create_const
(
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            int blocksize,
            gko::detail::const_array_view< ValueType > && values,
            gko::detail::const_array_view< IndexType > && col_idxs,
            gko::detail::const_array_view< IndexType > && row_ptrs )  [static]
```

Creates a constant (immutable) Fbcsr matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *blocksize* | the block size of the matrix |
| *values* | the value array of the matrix, stored in column-major order for each block |
| *col_idxs* | the block column index array of the matrix |
| *row_ptrs* | the block row pointer array of the matrix |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of the arrays on the correct executor.

### 48.103.3.10 extract_diagonal()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Fbcsr< ValueType, IndexType >::extract_↩
diagonal ( ) const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

### 48.103.3.11 get_block_size()

```
template<typename ValueType = default_precision, typename IndexType = int32>
int gko::matrix::Fbcsr< ValueType, IndexType >::get_block_size ( ) const  [inline], [noexcept]
```

**Returns**

The fixed block size for this matrix

```
325 { return bs_; }
```

#### 48.103.3.12 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Fbcsr< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

**Returns**

The column indexes of the matrix.

```
275 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.103.3.13 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Fbcsr< ValueType, IndexType >::get_const_col_idxs ( ) const
[inline], [noexcept]
```

**Returns**

The column indexes of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
285     {
286         return col_idxs_.get_const_data();
287     }
```

References gko::array< ValueType >::get_const_data().

#### 48.103.3.14 get_const_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Fbcsr< ValueType, IndexType >::get_const_row_ptrs ( ) const
[inline], [noexcept]
```

**Returns**

The row pointers of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
302     {
303         return row_ptrs_.get_const_data();
304     }
```

References gko::array< ValueType >::get_const_data().

#### 48.103.3.15 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Fbcsr< ValueType, IndexType >::get_const_values ( ) const  [inline],
[noexcept]
```

**Returns**

The values of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
268     {
269         return values_.get_const_data();
270     }
```

References gko::array< ValueType >::get_const_data().

#### 48.103.3.16 get_num_block_cols()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type gko::matrix::Fbcsr< ValueType, IndexType >::get_num_block_cols ( ) const  [inline],
[noexcept]
```

**Returns**

The number of block-columns in the matrix

```
339     {
340         return this->get_size()[1] / bs_;
341     }
```

#### 48.103.3.17 get_num_block_rows()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type gko::matrix::Fbcsr< ValueType, IndexType >::get_num_block_rows ( ) const  [inline],
[noexcept]
```

**Returns**

The number of block-rows in the matrix

```
331     {
332         return this->get_size()[0] / bs_;
333     }
```

### 48.103.3.18 get_num_stored_blocks()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Fbcsr< ValueType, IndexType >::get_num_stored_blocks ( ) const  [inline],
[noexcept]
```

**Returns**

The number of non-zero blocks explicitly stored in the matrix

```
318     {
319         return col_idxs_.get_size();
320     }
```

References gko::array< ValueType >::get_size().

### 48.103.3.19 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Fbcsr< ValueType, IndexType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

**Returns**

The number of elements explicitly stored in the matrix

```
310     {
311         return values_.get_size();
312     }
```

References gko::array< ValueType >::get_size().

### 48.103.3.20 get_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Fbcsr< ValueType, IndexType >::get_row_ptrs ( )  [inline], [noexcept]
```

**Returns**

The row pointers of the matrix.

```
292 { return row_ptrs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.103.3.21 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Fbcsr< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

**Returns**

The values of the matrix.

```
258 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.103.3.22 is_sorted_by_column_index()

```
template<typename ValueType = default_precision, typename IndexType = int32>
bool gko::matrix::Fbcsr< ValueType, IndexType >::is_sorted_by_column_index ( ) const
```

Tests if all row entry pairs (value, col_idx) are sorted by column index.

**Returns**

True if all row entry pairs (value, col_idx) are sorted by column index

#### 48.103.3.23 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Fbcsr& gko::matrix::Fbcsr< ValueType, IndexType >::operator= (
            const Fbcsr< ValueType, IndexType > &  )
```

Copy-assigns an Fbcsr matrix.

Preserves the executor, copies data and block size from the input.

#### 48.103.3.24 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Fbcsr& gko::matrix::Fbcsr< ValueType, IndexType >::operator= (
            Fbcsr< ValueType, IndexType > &&  )
```

Move-assigns an Fbcsr matrix.

Preserves the executor, moves the data over preserving size and stride. Leaves the moved-from object in an empty state (0x0 with no nonzeros, but valid row pointers).

#### 48.103.3.25 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Fbcsr< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.103.3.26 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Fbcsr< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix_data into Fbcsr format.

Requires the block size to be set beforehand

**See also**

set_block_size.

**Warning**

Unlike Csr::read, here explicit non-zeros are NOT dropped.

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.103.3.27 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Fbcsr< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.103.3.28 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::Fbcsr< ValueType, IndexType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.103.3.29 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Fbcsr< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following files:

- ginkgo/core/matrix/csr.hpp
- ginkgo/core/matrix/fbcsr.hpp

## 48.104 gko::solver::Fcg< ValueType > Class Template Reference

FCG or the flexible conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.

```
#include <ginkgo/core/solver/fcg.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.104.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Fcg< ValueType >**

FCG or the flexible conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.

Though this method performs very well for symmetric positive definite matrices, it is in general not suitable for general matrices.

In contrast to the standard CG, which uses the Fletcher-Reeves formula

$$\beta_k = \frac{\langle r_k, z_k \rangle}{\langle r_{k-1}, z_{k-1} \rangle},$$

the flexible CG uses the Polak-Ribière formula

$$\beta_k = \frac{\langle r_k - r_{k-1}, z_k \rangle}{\langle r_{k-1}, z_{k-1} \rangle}$$

for the next search direction. In CG the denominator $\langle r_{k-1}, z_{k-1} \rangle$ is exactly the numerator that the previous iteration already computed, so a single dot product per iteration is enough. FCG still needs $\langle r_k, z_k \rangle$ as the denominator of the next iteration, and has to compute $\langle r_k - r_{k-1}, z_k \rangle$ on top of it — one extra dot product, and therefore one extra global reduction, per iteration.

In exchange, $\beta_k$ no longer relies on the search directions staying $A$-conjugate, so the preconditioner $M$ (and therefore $z = Mr$) may change between iterations — useful when the preconditioner is itself an inner iterative solve, a randomized smoother, or otherwise not a fixed linear operator.

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of FCG are merged into 2 separate steps.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.104.2 Member Function Documentation

### 48.104.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Fcg< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
90 { return true; }
```

### 48.104.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Fcg< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.104.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Fcg< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.104.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Fcg< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/fcg.hpp

## 48.105 gko::matrix::Fft Class Reference

This LinOp implements a 1D Fourier matrix using the FFT algorithm.

```
#include <ginkgo/core/matrix/fft.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- void write (matrix_data< std::complex< float >, int32 > &data) const override

  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< float >, int64 > &data) const override

  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< double >, int32 > &data) const override

  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< double >, int64 > &data) const override

  *Writes a matrix to a matrix_data structure.*

### Static Public Member Functions

- static std::unique_ptr< Fft > create (std::shared_ptr< const Executor > exec)

  *Creates an empty Fourier matrix.*
- static std::unique_ptr< Fft > create (std::shared_ptr< const Executor > exec, size_type size=0, bool inverse=false)

  *Creates an Fourier matrix with the given dimensions.*

### 48.105.1 Detailed Description

This LinOp implements a 1D Fourier matrix using the FFT algorithm.

It implements forward and inverse DFT.

For a power-of-two size n with corresponding root of unity $\omega = e^{-2\pi i/n}$ for forward DFT and $\omega = e^{2\pi i/n}$ for inverse DFT it computes

$$x_k = \sum_{j=0}^{n-1} \omega^{jk} b_j$$

without normalization factors.

The Reference and OpenMP implementations support only power-of-two input sizes, as they use the Radix-2 algorithm by J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965, https://doi.org/10.2307/2003354. The CUDA and HIP implementations use cuSPARSE/hipSPARSE with full support for non-power-of-two input sizes and special optimizations for products of small prime powers.

### 48.105.2 Member Function Documentation

#### 48.105.2.1 conj_transpose()

```
std::unique_ptr<LinOp> gko::matrix::Fft::conj_transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.105.2.2 create() [1/2]

```
static std::unique_ptr<Fft> gko::matrix::Fft::create (
            std::shared_ptr< const Executor > exec )  [static]
```

Creates an empty Fourier matrix.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |

**Returns**

A smart pointer to the newly created matrix.

#### 48.105.2.3 create() [2/2]

```
static std::unique_ptr<Fft> gko::matrix::Fft::create (
            std::shared_ptr< const Executor > exec,
            size_type size = 0,
            bool inverse = false )  [static]
```

Creates an Fourier matrix with the given dimensions.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | size of the matrix |
| *inverse* | true to compute an inverse DFT instead of a normal DFT |

**Returns**

A smart pointer to the newly created matrix.

#### 48.105.2.4 transpose()

```
std::unique_ptr<LinOp> gko::matrix::Fft::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.105.2.5 write() [1/4]

```
void gko::matrix::Fft::write (
            matrix_data< std::complex< double >, int32 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< double >, int32 >.

#### 48.105.2.6 write() [2/4]

```
void gko::matrix::Fft::write (
            matrix_data< std::complex< double >, int64 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< double >, int64 >.

#### 48.105.2.7 write() [3/4]

```
void gko::matrix::Fft::write (
            matrix_data< std::complex< float >, int32 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< float >, int32 >.

#### 48.105.2.8 write() [4/4]

```
void gko::matrix::Fft::write (
            matrix_data< std::complex< float >, int64 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< float >, int64 >.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/fft.hpp

## 48.106 gko::matrix::Fft2 Class Reference

This LinOp implements a 2D Fourier matrix using the FFT algorithm.

`#include <ginkgo/core/matrix/fft.hpp>`

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override
  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override
  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- void write (matrix_data< std::complex< float >, int32 > &data) const override
  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< float >, int64 > &data) const override
  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< double >, int32 > &data) const override
  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< double >, int64 > &data) const override
  *Writes a matrix to a matrix_data structure.*

### Static Public Member Functions

- static std::unique_ptr< Fft2 > create (std::shared_ptr< const Executor > exec)
  *Creates an empty Fourier matrix.*
- static std::unique_ptr< Fft2 > create (std::shared_ptr< const Executor > exec, size_type size)
  *Creates an Fourier matrix with the given dimensions.*
- static std::unique_ptr< Fft2 > create (std::shared_ptr< const Executor > exec, size_type size1, size_type size2, bool inverse=false)
  *Creates an Fourier matrix with the given dimensions.*

### 48.106.1 Detailed Description

This LinOp implements a 2D Fourier matrix using the FFT algorithm.

For indexing purposes, the first dimension is the major axis.

It implements complex-to-complex forward and inverse FFT.

For a power-of-two sizes $n_1, n_2$ with corresponding root of unity $\omega = e^{-2\pi i/(n_1 n_2)}$ for forward DFT and $\omega = e^{2\pi i/(n_1 n_2)}$ for inverse DFT it computes

$$x_{k_1 n_2 + k_2} = \sum_{i_1=0}^{n_1-1} \sum_{i_2=0}^{n_2-1} \omega^{i_1 k_1 + i_2 k_2} b_{i_1 n_2 + i_2}$$

without normalization factors.

The Reference and OpenMP implementations support only power-of-two input sizes, as they use the Radix-2 algorithm by J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965, https://doi.org/10.2307/2003354. The CUDA and HIP implementations use cuSPARSE/hipSPARSE with full support for non-power-of-two input sizes and special optimizations for products of small prime powers.

## 48.106.2 Member Function Documentation

### 48.106.2.1 conj_transpose()

```
std::unique_ptr<LinOp> gko::matrix::Fft2::conj_transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

#### Returns

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.106.2.2 create() [1/3]

```
static std::unique_ptr<Fft2> gko::matrix::Fft2::create (
            std::shared_ptr< const Executor > exec )  [static]
```

Creates an empty Fourier matrix.

#### Parameters

| | |
|---|---|
| *exec* | Executor associated to the matrix |

#### Returns

A smart pointer to the newly created matrix.

### 48.106.2.3 create() [2/3]

```
static std::unique_ptr<Fft2> gko::matrix::Fft2::create (
            std::shared_ptr< const Executor > exec,
            size_type size )  [static]
```

Creates an Fourier matrix with the given dimensions.

#### Parameters

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | size of both FFT dimensions |

**Returns**

A smart pointer to the newly created matrix.

#### 48.106.2.4 create() [3/3]

```
static std::unique_ptr<Fft2> gko::matrix::Fft2::create (
            std::shared_ptr< const Executor > exec,
            size_type size1,
            size_type size2,
            bool inverse = false )  [static]
```

Creates an Fourier matrix with the given dimensions.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size1* | size of the first FFT dimension |
| *size2* | size of the second FFT dimension |
| *inverse* | true to compute an inverse DFT instead of a normal DFT |

**Returns**

A smart pointer to the newly created matrix.

#### 48.106.2.5 transpose()

```
std::unique_ptr<LinOp> gko::matrix::Fft2::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.106.2.6 write() [1/4]

```
void gko::matrix::Fft2::write (
            matrix_data< std::complex< double >, int32 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< double >, int32 >.

#### 48.106.2.7 write() [2/4]

```
void gko::matrix::Fft2::write (
            matrix_data< std::complex< double >, int64 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< double >, int64 >.

#### 48.106.2.8 write() [3/4]

```
void gko::matrix::Fft2::write (
            matrix_data< std::complex< float >, int32 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< float >, int32 >.

#### 48.106.2.9 write() [4/4]

```
void gko::matrix::Fft2::write (
            matrix_data< std::complex< float >, int64 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< float >, int64 >.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/fft.hpp

## 48.107 gko::matrix::Fft3 Class Reference

This LinOp implements a 3D Fourier matrix using the FFT algorithm.

`#include <ginkgo/core/matrix/fft.hpp>`

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override
  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override
  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- void write (matrix_data< std::complex< float >, int32 > &data) const override
  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< float >, int64 > &data) const override
  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< double >, int32 > &data) const override
  *Writes a matrix to a matrix_data structure.*
- void write (matrix_data< std::complex< double >, int64 > &data) const override
  *Writes a matrix to a matrix_data structure.*

### Static Public Member Functions

- static std::unique_ptr< Fft3 > create (std::shared_ptr< const Executor > exec)
  *Creates an empty Fourier matrix.*
- static std::unique_ptr< Fft3 > create (std::shared_ptr< const Executor > exec, size_type size)
  *Creates an Fourier matrix with the given dimensions.*
- static std::unique_ptr< Fft3 > create (std::shared_ptr< const Executor > exec, size_type size1, size_type size2, size_type size3, bool inverse=false)
  *Creates an Fourier matrix with the given dimensions.*

### 48.107.1 Detailed Description

This LinOp implements a 3D Fourier matrix using the FFT algorithm.

For indexing purposes, the first dimension is the major axis.

It implements complex-to-complex forward and inverse FFT.

For a power-of-two sizes $n_1, n_2, n_3$ with corresponding root of unity $\omega = e^{-2\pi i/(n_1 n_2 n_3)}$ for forward DFT and $\omega = e^{2\pi i/(n_1 n_2 n_3)}$ for inverse DFT it computes

$$x_{k_1 n_2 n_3 + k_2 n_3 + k_3} = \sum_{i_1=0}^{n_1-1} \sum_{i_2=0}^{n_2-1} \sum_{i_3=0}^{n_3-1} \omega^{i_1 k_1 + i_2 k_2 + i_3 k_3} b_{i_1 n_2 n_3 + i_2 n_3 + i_3}$$

without normalization factors.

The Reference and OpenMP implementations support only power-of-two input sizes, as they use the Radix-2 algorithm by J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965, https://doi.org/10.2307/2003354. The CUDA and HIP implementations use cuSPARSE/hipSPARSE with full support for non-power-of-two input sizes and special optimizations for products of small prime powers.

## 48.107.2 Member Function Documentation

### 48.107.2.1 conj_transpose()

```
std::unique_ptr<LinOp> gko::matrix::Fft3::conj_transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

#### Returns

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.107.2.2 create() [1/3]

```
static std::unique_ptr<Fft3> gko::matrix::Fft3::create (
            std::shared_ptr< const Executor > exec )  [static]
```

Creates an empty Fourier matrix.

#### Parameters

| | |
|---|---|
| *exec* | Executor associated to the matrix |

#### Returns

A smart pointer to the newly created matrix.

### 48.107.2.3 create() [2/3]

```
static std::unique_ptr<Fft3> gko::matrix::Fft3::create (
            std::shared_ptr< const Executor > exec,
            size_type size )  [static]
```

Creates an Fourier matrix with the given dimensions.

#### Parameters

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | size of all FFT dimensions |

**Returns**

A smart pointer to the newly created matrix.

#### 48.107.2.4 create() [3/3]

```
static std::unique_ptr<Fft3> gko::matrix::Fft3::create (
            std::shared_ptr< const Executor > exec,
            size_type size1,
            size_type size2,
            size_type size3,
            bool inverse = false )  [static]
```

Creates an Fourier matrix with the given dimensions.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size1* | size of the first FFT dimension |
| *size2* | size of the second FFT dimension |
| *size3* | size of the third FFT dimension |
| *inverse* | true to compute an inverse DFT instead of a normal DFT |

**Returns**

A smart pointer to the newly created matrix.

#### 48.107.2.5 transpose()

```
std::unique_ptr<LinOp> gko::matrix::Fft3::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.107.2.6 write() [1/4]

```
void gko::matrix::Fft3::write (
            matrix_data< std::complex< double >, int32 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< double >, int32 >.

#### 48.107.2.7 write() [2/4]

```
void gko::matrix::Fft3::write (
            matrix_data< std::complex< double >, int64 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< double >, int64 >.

#### 48.107.2.8 write() [3/4]

```
void gko::matrix::Fft3::write (
            matrix_data< std::complex< float >, int32 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< float >, int32 >.

#### 48.107.2.9 write() [4/4]

```
void gko::matrix::Fft3::write (
            matrix_data< std::complex< float >, int64 > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< std::complex< float >, int64 >.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/fft.hpp

# 48.108 gko::multigrid::FixedCoarsening< ValueType, IndexType > Class Template Reference

FixedCoarsening is a very simple coarse grid generation algorithm.

`#include <ginkgo/core/multigrid/fixed_coarsening.hpp>`

## Public Member Functions

- std::shared_ptr< const LinOp > get_system_matrix () const

  *Returns the system operator (matrix) of the linear system.*

## 48.108.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::multigrid::FixedCoarsening< ValueType, IndexType >**

FixedCoarsening is a very simple coarse grid generation algorithm.

It selects the coarse matrix from the fine matrix by with user-specified indices.

The user needs to specify the indices (with global numbering) of the fine matrix, they wish to be in the coarse matrix. The restriction and prolongation matrices will map to and from the coarse space without any interpolation or weighting.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.108.2 Member Function Documentation

### 48.108.2.1 get_system_matrix()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::multigrid::FixedCoarsening< ValueType, IndexType >::get_↩
system_matrix ( ) const  [inline]
```

Returns the system operator (matrix) of the linear system.

**Returns**

the system operator (matrix)

```
55     {
56         return system_matrix_;
57     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/multigrid/fixed_coarsening.hpp

## 48.109 gko::preconditioner::GaussSeidel< ValueType, IndexType > Class Template Reference

This class generates the Gauss-Seidel preconditioner.

`#include <ginkgo/core/preconditioner/gauss_seidel.hpp>`

### Public Member Functions

- const parameters_type & get_parameters ()
  *Returns the parameters used to construct the factory.*
- const parameters_type & get_parameters () const
  *Returns the parameters used to construct the factory.*
- std::unique_ptr< composition_type > generate (std::shared_ptr< const LinOp > system_matrix) const

### Static Public Member Functions

- static parameters_type build ()
  *Creates a new parameter_type to set up the factory.*

### 48.109.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::preconditioner::GaussSeidel< ValueType, IndexType >**

This class generates the Gauss-Seidel preconditioner.

This is the special case of the relaxation factor $\omega = 1$ of the (S)SOR preconditioner.

**See also**

Sor

### 48.109.2 Member Function Documentation

#### 48.109.2.1 generate()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<composition_type> gko::preconditioner::GaussSeidel< ValueType, IndexType >↩
::generate (
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix used to generate the preconditioner. |

**Note**

This function overrides the default LinOpFactory::generate to return a Factorization instead of a generic LinOp, which would need to be cast to Factorization again to access its factors. It is only necessary because smart pointers aren't covariant.

#### 48.109.2.2 get_parameters() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
const parameters_type& gko::preconditioner::GaussSeidel< ValueType, IndexType >::get_parameters
( )  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
69 { return parameters_; }
```

#### 48.109.2.3 get_parameters() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
const parameters_type& gko::preconditioner::GaussSeidel< ValueType, IndexType >::get_parameters
( ) const  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
74 { return parameters_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/gauss_seidel.hpp

## 48.110 gko::solver::Gcr< ValueType > Class Template Reference

GCR or the generalized conjugate residual method is an iterative type Krylov subspace method similar to GMRES which is suitable for nonsymmetric linear systems.

```
#include <ginkgo/core/solver/gcr.hpp>
```

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*
- size_type get_krylov_dim () const

  *Gets the Krylov dimension of the solver.*
- void set_krylov_dim (size_type other)

  *Sets the Krylov dimension.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.110.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Gcr< ValueType >**

GCR or the generalized conjugate residual method is an iterative type Krylov subspace method similar to GMRES which is suitable for nonsymmetric linear systems.

GCR maintains a sequence of search directions $p_0, p_1, \ldots$ chosen so that the vectors $Ap_k$ are mutually orthogonal, $\langle Ap_i, Ap_j \rangle = 0$ for $i \neq j$. At each iteration the residual is updated by orthogonal projection along $Ap_k$,

$$\alpha_k = \frac{\langle r_k, Ap_k \rangle}{\langle Ap_k, Ap_k \rangle}, \qquad r_{k+1} = r_k - \alpha_k Ap_k,$$

so the iterate $x_k = x_0 + \sum_{j<k} \alpha_j p_j$ minimizes $\|r\|_2$ over $x_0 + \mathcal{K}_k(A, r_0)$ — the same minimization property as GMRES, but with a long recurrence over the search directions instead of an Arnoldi basis and Hessenberg solve. The memory cost grows linearly with the iteration count, so GCR is typically run with a restart parameter (`krylov_dim`).

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of GCR are merged into one step. Modified Gram-Schmidt is used.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

### 48.110.2 Member Function Documentation

#### 48.110.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Gcr< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
86 { return true; }
```

#### 48.110.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Gcr< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.110.2.3 get_krylov_dim()

```
template<typename ValueType = default_precision>
size_type gko::solver::Gcr< ValueType >::get_krylov_dim ( ) const  [inline]
```

Gets the Krylov dimension of the solver.

**Returns**

the Krylov dimension

```
93 { return parameters_.krylov_dim; }
```

#### 48.110.2.4 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Gcr< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.110.2.5 set_krylov_dim()

```
template<typename ValueType = default_precision>
void gko::solver::Gcr< ValueType >::set_krylov_dim (
            size_type other )  [inline]
```

Sets the Krylov dimension.

**Parameters**

| | |
|---|---|
| *other* | the new Krylov dimension |

```
100 { parameters_.krylov_dim = other; }
```

#### 48.110.2.6 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Gcr< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/gcr.hpp

## 48.111 gko::solver::Gmres< ValueType > Class Template Reference

GMRES or the generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.

```
#include <ginkgo/core/solver/gmres.hpp>
```

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*
- size_type get_krylov_dim () const

  *Gets the Krylov dimension of the solver.*
- void set_krylov_dim (size_type other)

  *Sets the Krylov dimension.*
- remove_complex< ValueType > get_restart_ratio () const

  *Returns the restart ratio.*
- void set_restart_ratio (remove_complex< ValueType > other)

  *Sets the restart ratio.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.111.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Gmres< ValueType >**

GMRES or the generalized minimal residual method is an iterative type Krylov subspace method which is suitable for nonsymmetric linear systems.

The Arnoldi process builds an orthonormal basis $V_m$ of the Krylov subspace

$$\mathcal{K}_m(A, r_0) = \text{span}\{r_0, Ar_0, A^2 r_0, \ldots, A^{m-1} r_0\},$$

and produces an upper Hessenberg matrix $\bar{H}_m$ satisfying the Arnoldi relation $AV_m = V_{m+1}\bar{H}_m$. GMRES then chooses the approximate solution $x_m = x_0 + V_m y_m$ with $y_m$ minimizing the residual norm over the subspace,

$$y_m = \arg\min_{y \in \mathbb{R}^m} \|\beta e_1 - \bar{H}_m y\|_2, \qquad \beta = \|r_0\|_2.$$

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of GMRES are merged into 2 separate steps. Modified Gram-Schmidt is used.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.111.2 Member Function Documentation

### 48.111.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Gmres< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

#### Returns

true as iterative solvers use the data in x as an initial guess.

```
108 { return true; }
```

### 48.111.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Gmres< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

#### Returns

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.111.2.3 get_krylov_dim()

```
template<typename ValueType = default_precision>
size_type gko::solver::Gmres< ValueType >::get_krylov_dim ( ) const  [inline]
```

Gets the Krylov dimension of the solver.

#### Returns

the Krylov dimension

```
115 { return parameters_.krylov_dim; }
```

### 48.111.2.4 get_restart_ratio()

```
template<typename ValueType = default_precision>
remove_complex<ValueType> gko::solver::Gmres< ValueType >::get_restart_ratio ( ) const  [inline]
```

Returns the restart ratio.

#### Returns

the restart ratio

```
130     {
131         return parameters_.restart_ratio;
132     }
```

### 48.111.2.5 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Gmres< ValueType >::parse (
             const config::pnode & config,
             const config::registry & context,
             const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

#### Parameters

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

#### Returns

parameters

### 48.111.2.6 set_krylov_dim()

```
template<typename ValueType = default_precision>
void gko::solver::Gmres< ValueType >::set_krylov_dim (
             size_type other )  [inline]
```

Sets the Krylov dimension.

**Parameters**

| | |
|---|---|
| *other* | the new Krylov dimension |

```
122 { parameters_.krylov_dim = other; }
```

#### 48.111.2.7 set_restart_ratio()

```
template<typename ValueType = default_precision>
void gko::solver::Gmres< ValueType >::set_restart_ratio (
            remove_complex< ValueType > other )  [inline]
```

Sets the restart ratio.

**Parameters**

| | |
|---|---|
| *other* | the new restart ratio |

```
140     {
141         parameters_.restart_ratio = other;
142     }
```

#### 48.111.2.8 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Gmres< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/gmres.hpp

## 48.112 gko::half Class Reference

A class providing basic support for half precision floating point types.

```
#include <ginkgo/core/base/half.hpp>
```

### 48.112.1 Detailed Description

A class providing basic support for half precision floating point types.

For now the only features are reduced storage compared to single precision and conversions from and to single precision floating point type.

The documentation for this class was generated from the following file:

- ginkgo/core/base/half.hpp

## 48.113 gko::solver::has_with_criteria< SolverType, typename > Struct Template Reference

Helper structure to test if the Factory of SolverType has a function `with_criteria`.

`#include <ginkgo/core/solver/solver_traits.hpp>`

### 48.113.1 Detailed Description

**template<typename SolverType, typename = void>**
**struct gko::solver::has_with_criteria< SolverType, typename >**

Helper structure to test if the Factory of SolverType has a function `with_criteria`.

Contains a constexpr boolean `value`, which is true if the Factory class of SolverType has a `with_criteria`, and false otherwise.

**Template Parameters**

| | |
|---|---|
| *SolverType* | Solver to test if its factory has a with_criteria function. |

The documentation for this struct was generated from the following file:

- ginkgo/core/solver/solver_traits.hpp

## 48.114 gko::hip_stream Class Reference

An RAII wrapper for a custom HIP stream.

`#include <ginkgo/core/base/stream.hpp>`

## Public Member Functions

- hip_stream ()

  *Creates an empty stream wrapper, representing the default stream.*
- hip_stream (int device_id, bool non_blocking=false)

  *Creates a new custom HIP stream on the given device.*
- ~hip_stream ()

  *Destroys the custom HIP stream, if it isn't empty.*
- hip_stream (hip_stream &&) noexcept

  *Move-constructs from an existing stream, which will be emptied.*
- hip_stream & operator= (hip_stream &&)=delete

  *Move-assigns from an existing stream, which will be emptied.*
- CUstream_st * get () const

  *Returns the native HIP stream handle.*

### 48.114.1 Detailed Description

An RAII wrapper for a custom HIP stream.

The stream will be created on construction and destroyed when the lifetime of the wrapper ends.

### 48.114.2 Constructor & Destructor Documentation

#### 48.114.2.1 hip_stream()

```
gko::hip_stream::hip_stream (
            int device_id,
            bool non_blocking = false )
```

Creates a new custom HIP stream on the given device.

**Parameters**

| | |
|---|---|
| *device_id* | the device ID to create the stream on. |
| *non_blocking* | create the stream with hipStreamNonBlocking, so that it does not implicitly order against the legacy default stream. |

### 48.114.3 Member Function Documentation

#### 48.114.3.1 get()

```
CUstream_st* gko::hip_stream::get ( ) const
```

Returns the native HIP stream handle.

In an empty hip_stream, this will return nullptr.

The documentation for this class was generated from the following file:

- ginkgo/core/base/stream.hpp

## 48.115 gko::HipAllocatorBase Class Reference

Implement this interface to provide an allocator for HipExecutor.

`#include <ginkgo/core/base/memory.hpp>`

### 48.115.1 Detailed Description

Implement this interface to provide an allocator for HipExecutor.

The documentation for this class was generated from the following file:

- ginkgo/core/base/memory.hpp

## 48.116 gko::HipblasError Class Reference

HipblasError is thrown when a hipBLAS routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- HipblasError (const std::string &file, int line, const std::string &func, int64 error_code)
  *Initializes a hipBLAS error.*

### 48.116.1 Detailed Description

HipblasError is thrown when a hipBLAS routine throws a non-zero error code.

### 48.116.2 Constructor & Destructor Documentation

#### 48.116.2.1 HipblasError()

```
gko::HipblasError::HipblasError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a hipBLAS error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the hipBLAS routine that failed |
| *error_code* | The resulting hipBLAS error code |

```
314         :  Error(file, line, func + ":  " + get_error(error_code))
315     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.117 gko::HipError Class Reference

HipError is thrown when a HIP routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- HipError (const std::string &file, int line, const std::string &func, int64 error_code)
  *Initializes a HIP error.*

### 48.117.1 Detailed Description

HipError is thrown when a HIP routine throws a non-zero error code.

### 48.117.2 Constructor & Destructor Documentation

#### 48.117.2.1 HipError()

```
gko::HipError::HipError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a HIP error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the HIP routine that failed |
| *error_code* | The resulting HIP error code |

```
291         :  Error(file, line, func + ":  " + get_error(error_code))
292     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.118 gko::HipExecutor Class Reference

This is the Executor subclass which represents the HIP enhanced device.

`#include <ginkgo/core/base/executor.hpp>`

### Public Member Functions

- std::shared_ptr< Executor > get_master () noexcept override

  *Returns the master OmpExecutor of this Executor.*
- std::shared_ptr< const Executor > get_master () const noexcept override

  *Returns the master OmpExecutor of this Executor.*
- void synchronize () const override

  *Synchronize the operations launched on the executor with its master.*
- std::string get_description () const override
- int get_device_id () const noexcept

  *Get the HIP device id of the device associated to this executor.*
- int get_num_warps_per_sm () const noexcept

  *Get the number of warps per SM of this executor.*
- int get_num_multiprocessor () const noexcept

  *Get the number of multiprocessor of this executor.*
- int get_major_version () const noexcept

  *Get the major version of compute capability.*
- int get_minor_version () const noexcept

  *Get the minor version of compute capability.*
- int get_num_warps () const noexcept

  *Get the number of warps of this executor.*
- int get_warp_size () const noexcept

  *Get the warp size of this executor.*
- hipblasContext * get_hipblas_handle () const

  *Get the hipblas handle for this executor.*
- hipblasContext * get_blas_handle () const

  *Get the hipblas handle for this executor.*
- hipsparseContext * get_hipsparse_handle () const

  *Get the hipsparse handle for this executor.*
- hipsparseContext * get_sparselib_handle () const

  *Get the hipsparse handle for this executor.*
- virtual void run (const Operation &op) const=0

  *Runs the specified Operation using this Executor.*
- template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*
- template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩ Dpcpp >
  void run (std::string name, const ClosureReference &op_ref, const ClosureOmp &op_omp, const Closure↩ Cuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

## Static Public Member Functions

- static std::shared_ptr< HipExecutor > create (int device_id, std::shared_ptr< Executor > master, bool device_reset, allocation_mode alloc_mode=default_hip_alloc_mode, CUstream_st *stream=nullptr)

  *Creates a new HipExecutor.*
- static int get_num_devices ()

  *Get the number of devices present on the system.*

### 48.118.1 Detailed Description

This is the Executor subclass which represents the HIP enhanced device.

### 48.118.2 Member Function Documentation

#### 48.118.2.1 create()

```
static std::shared_ptr<HipExecutor> gko::HipExecutor::create (
            int device_id,
            std::shared_ptr< Executor > master,
            bool device_reset,
            allocation_mode alloc_mode = default_hip_alloc_mode,
            CUstream_st * stream = nullptr )  [static]
```

Creates a new HipExecutor.

**Parameters**

| | |
|---|---|
| *device_id* | the HIP device id of this device |
| *master* | an executor on the host that is used to invoke the device kernels |
| *device_reset* | whether to reset the device after the object exits the scope. |
| *alloc_mode* | the allocation mode that the executor should operate on. See @allocation_mode for more details |
| *stream* | the stream to execute operations on. |

#### 48.118.2.2 get_blas_handle()

```
hipblasContext* gko::HipExecutor::get_blas_handle ( ) const  [inline]
```

Get the hipblas handle for this executor.

**Returns**

the hipblas handle (hipblasContext*) for this executor

```
1868 { return hipblas_handle_.get(); }
```

Referenced by get_hipblas_handle().

#### 48.118.2.3 get_description()

```
std::string gko::HipExecutor::get_description ( ) const  [override], [virtual]
```

**Returns**

a textual representation of the executor and its device.

Implements gko::Executor.

#### 48.118.2.4 get_hipblas_handle()

```
hipblasContext* gko::HipExecutor::get_hipblas_handle ( ) const  [inline]
```

Get the hipblas handle for this executor.

**Returns**

the hipblas handle (hipblasContext*) for this executor

```
1863 { return get_blas_handle(); }
```

References get_blas_handle().

#### 48.118.2.5 get_hipsparse_handle()

```
hipsparseContext* gko::HipExecutor::get_hipsparse_handle ( ) const  [inline]
```

Get the hipsparse handle for this executor.

**Returns**

the hipsparse handle (hipsparseContext*) for this executor

```
1877     {
1878         return get_sparselib_handle();
1879     }
```

References get_sparselib_handle().

#### 48.118.2.6 get_master() [1/2]

```
std::shared_ptr<const Executor> gko::HipExecutor::get_master ( ) const  [override], [virtual],
[noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

#### 48.118.2.7 get_master() [2/2]

```
std::shared_ptr<Executor> gko::HipExecutor::get_master ( )  [override], [virtual], [noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

#### 48.118.2.8 get_sparselib_handle()

```
hipsparseContext* gko::HipExecutor::get_sparselib_handle ( ) const  [inline]
```

Get the hipsparse handle for this executor.

**Returns**

the hipsparse handle (hipsparseContext*) for this executor

```
1885     {
1886         return hipsparse_handle_.get();
1887     }
```

Referenced by get_hipsparse_handle().

#### 48.118.2.9 run() [1/3]

```
template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
Dpcpp >
void gko::Executor::run (
             typename ClosureOmp ,
             typename ClosureCuda ,
             typename ClosureHip ,
             typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *op_omp* | functor to run in case of a OmpExecutor or ReferenceExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
660     {
661         LambdaOperation<ClosureOmp, ClosureOmp, ClosureCuda, ClosureHip,
662                         ClosureDpcpp>
663             op(op_omp, op_cuda, op_hip, op_dpcpp);
664         this->run(op);
665     }
```

### 48.118.2.10 run() [2/3]

```
virtual void gko::Executor::run
```

Runs the specified Operation using this Executor.

**Parameters**

| | |
|---|---|
| *op* | the operation to run |

### 48.118.2.11 run() [3/3]

```
template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename
ClosureHip , typename ClosureDpcpp >
void gko::Executor::run (
             typename ClosureReference ,
             typename ClosureOmp ,
             typename ClosureCuda ,
             typename ClosureHip ,
             typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureReference* | type of op_ref |
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *name* | the name of the operation |

**Parameters**

| | |
|---|---|
| *op_ref* | functor to run in case of a ReferenceExecutor |
| *op_omp* | functor to run in case of a OmpExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
688     {
689         LambdaOperation<ClosureReference, ClosureOmp, ClosureCuda, ClosureHip,
690                         ClosureDpcpp>
691             op(std::move(name), op_ref, op_omp, op_cuda, op_hip, op_dpcpp);
692         this->run(op);
693     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

## 48.119 gko::HipfftError Class Reference

HipfftError is thrown when a hipFFT routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- HipfftError (const std::string &file, int line, const std::string &func, int64 error_code)
  *Initializes a hipFFT error.*

### 48.119.1 Detailed Description

HipfftError is thrown when a hipFFT routine throws a non-zero error code.

### 48.119.2 Constructor & Destructor Documentation

#### 48.119.2.1 HipfftError()

```
gko::HipfftError::HipfftError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a hipFFT error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the hipFFT routine that failed |
| *error_code* | The resulting hipFFT error code |

```
384         :  Error(file, line, func + ":  " + get_error(error_code))
385     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.120 gko::HiprandError Class Reference

HiprandError is thrown when a hipRAND routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- HiprandError (const std::string &file, int line, const std::string &func, int64 error_code)

  *Initializes a hipRAND error.*

### 48.120.1 Detailed Description

HiprandError is thrown when a hipRAND routine throws a non-zero error code.

### 48.120.2 Constructor & Destructor Documentation

#### 48.120.2.1 HiprandError()

```
gko::HiprandError::HiprandError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a hipRAND error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the hipRAND routine that failed |
| *error_code* | The resulting hipRAND error code |

```
337         :  Error(file, line, func + ":  " + get_error(error_code))
338     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.121 gko::HipsparseError Class Reference

HipsparseError is thrown when a hipSPARSE routine throws a non-zero error code.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- HipsparseError (const std::string &file, int line, const std::string &func, int64 error_code)
  *Initializes a hipSPARSE error.*

### 48.121.1 Detailed Description

HipsparseError is thrown when a hipSPARSE routine throws a non-zero error code.

### 48.121.2 Constructor & Destructor Documentation

#### 48.121.2.1 HipsparseError()

```
gko::HipsparseError::HipsparseError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a hipSPARSE error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the hipSPARSE routine that failed |
| *error_code* | The resulting hipSPARSE error code |

```
361         :  Error(file, line, func + ":  " + get_error(error_code))
362     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.122 gko::HipTimer Class Reference

A timer using events for timing on a HipExecutor.

`#include <ginkgo/core/base/timer.hpp>`

### Public Member Functions

- void record (time_point &time) override

  *Records a time point at the current time.*
- void wait (time_point &time) override

  *Waits until all kernels in-process when recording the time point are finished.*
- std::chrono::nanoseconds difference_async (const time_point &start, const time_point &stop) override

  *Computes the difference between the two time points in nanoseconds.*

### Additional Inherited Members

### 48.122.1 Detailed Description

A timer using events for timing on a HipExecutor.

**Note**

When using a HipExecutor with a custom stream, make sure that the stream's lifetime is longer than the lifetime of this timer.

### 48.122.2 Member Function Documentation

#### 48.122.2.1 difference_async()

```
std::chrono::nanoseconds gko::HipTimer::difference_async (
            const time_point & start,
            const time_point & stop )  [override], [virtual]
```

Computes the difference between the two time points in nanoseconds.

This asynchronous version does not synchronize itself, so the time points need to have been synchronized with, i.e. `timer->wait(stop)` needs to have been called. The version is intended for more advanced users who want to measure the overhead of timing functionality separately.

**Parameters**

| | |
|---|---|
| *start* | the first time point (earlier) |
| *stop* | the second time point (later) |

**Returns**

the difference between the time points in nanoseconds.

Implements gko::Timer.

The documentation for this class was generated from the following file:

- ginkgo/core/base/timer.hpp

## 48.123 gko::matrix::view::hybrid< ValueType, IndexType > Struct Template Reference

Non-owning view of a matrix::Hybrid to be used inside device kernels.

`#include <ginkgo/core/matrix/device_views.hpp>`

### Public Member Functions

- constexpr hybrid (ell< ValueType, IndexType > ell_, coo< ValueType, IndexType > coo_)
  *Constructs a hybrid view.*
- constexpr hybrid< const ValueType, const IndexType > as_const () const
  *Returns a const view of the same values.*

### 48.123.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**struct gko::matrix::view::hybrid< ValueType, IndexType >**

Non-owning view of a matrix::Hybrid to be used inside device kernels.

This type is used to provide a simple and stable ABI for passing data between libraries.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store matrix values. |
| *IndexType* | the index type used to store matrix columns. |

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/device_views.hpp

# 48.124 gko::matrix::Hybrid< ValueType, IndexType > Class Template Reference

HYBRID is a matrix format which splits the matrix into ELLPACK and COO format.

`#include <ginkgo/core/matrix/hybrid.hpp>`

## Classes

- class automatic

  *automatic is a strategy_type which decides the number of stored elements per row of the ell part automatically.*
- class column_limit

  *column_limit is a strategy_type which decides the number of stored elements per row of the ell part by specifying the number of columns.*
- class imbalance_bounded_limit

  *imbalance_bounded_limit is a strategy_type which decides the number of stored elements per row of the ell part.*
- class imbalance_limit

  *imbalance_limit is a strategy_type which decides the number of stored elements per row of the ell part according to the percent.*
- class minimal_storage_limit

  *minimal_storage_limit is a strategy_type which decides the number of stored elements per row of the ell part.*
- class strategy_type

  *strategy_type is to decide how to set the hybrid config.*

## Public Member Functions

- void read (const mat_data &data) override

  *Reads a matrix from a matrix_data structure.*
- void read (const device_mat_data &data) override

  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override

  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override

  *Writes a matrix to a matrix_data structure.*
- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- value_type * get_ell_values () noexcept

  *Returns the values of the ell part.*
- const value_type * get_const_ell_values () const noexcept

  *Returns the values of the ell part.*
- index_type * get_ell_col_idxs () noexcept

  *Returns the column indexes of the ell part.*
- const index_type * get_const_ell_col_idxs () const noexcept

  *Returns the column indexes of the ell part.*
- size_type get_ell_num_stored_elements_per_row () const noexcept

*Returns the number of stored elements per row of ell part.*

- size_type get_ell_stride () const noexcept

  *Returns the stride of the ell part.*

- size_type get_ell_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the ell part.*

- value_type & ell_val_at (size_type row, size_type idx) noexcept

  *Returns the `idx`-th non-zero element of the `row`-th row in the ell part.*

- value_type ell_val_at (size_type row, size_type idx) const noexcept

  *Returns the `idx`-th non-zero element of the `row`-th row in the ell part.*

- index_type & ell_col_at (size_type row, size_type idx) noexcept

  *Returns the `idx`-th column index of the `row`-th row in the ell part.*

- index_type ell_col_at (size_type row, size_type idx) const noexcept

  *Returns the `idx`-th column index of the `row`-th row in the ell part.*

- const ell_type * get_ell () const noexcept

  *Returns the matrix of the ell part.*

- value_type * get_coo_values () noexcept

  *Returns the values of the coo part.*

- const value_type * get_const_coo_values () const noexcept

  *Returns the values of the coo part.*

- index_type * get_coo_col_idxs () noexcept

  *Returns the column indexes of the coo part.*

- const index_type * get_const_coo_col_idxs () const noexcept

  *Returns the column indexes of the coo part.*

- index_type * get_coo_row_idxs () noexcept

  *Returns the row indexes of the coo part.*

- const index_type * get_const_coo_row_idxs () const noexcept

  *Returns the row indexes of the coo part.*

- size_type get_coo_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the coo part.*

- const coo_type * get_coo () const noexcept

  *Returns the matrix of the coo part.*

- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*

- std::shared_ptr< strategy_type > get_strategy () const noexcept

  *Returns the strategy.*

- template<typename HybType >
  std::shared_ptr< typename HybType::strategy_type > get_strategy () const

  *Returns the current strategy allowed in given hybrid format.*

- device_view get_device_view ()

  *Returns a non-owning device view of this matrix.*

- const_device_view get_const_device_view () const

  *Returns a non-owning const device view of this matrix.*

- Hybrid & operator= (const Hybrid &)

  *Copy-assigns a Hybrid matrix.*

- Hybrid & operator= (Hybrid &&)

  *Move-assigns a Hybrid matrix.*

- Hybrid (const Hybrid &)

  *Copy-assigns a Hybrid matrix.*

- Hybrid (Hybrid &&)

  *Move-assigns a Hybrid matrix.*

## Static Public Member Functions

- static std::unique_ptr< Hybrid > create (std::shared_ptr< const Executor > exec, std::shared_ptr< strategy_type > strategy=std::make_shared< automatic >())

  *Creates an uninitialized Hybrid matrix of specified method.*
- static std::unique_ptr< Hybrid > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, std::shared_ptr< strategy_type > strategy=std::make_shared< automatic >())

  *Creates an uninitialized Hybrid matrix of the specified size and method.*
- static std::unique_ptr< Hybrid > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, size_type num_stored_elements_per_row, std::shared_ptr< strategy_type > strategy=std::make_shared< automatic >())

  *Creates an uninitialized Hybrid matrix of the specified size and method.*
- static std::unique_ptr< Hybrid > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, size_type num_stored_elements_per_row, size_type stride, std::shared_ptr< strategy_type > strategy)

  *Creates an uninitialized Hybrid matrix of the specified size and method.*
- static std::unique_ptr< Hybrid > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, size_type num_stored_elements_per_row, size_type stride, size_type num_nonzeros={}, std::shared_ptr< strategy_type > strategy=std::make_shared< automatic >())

  *Creates an uninitialized Hybrid matrix of the specified size and method.*

### 48.124.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Hybrid< ValueType, IndexType >**

HYBRID is a matrix format which splits the matrix into ELLPACK and COO format.

Achieve the excellent performance with a proper partition of ELLPACK and COO.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

### 48.124.2 Constructor & Destructor Documentation

#### 48.124.2.1 Hybrid() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Hybrid< ValueType, IndexType >::Hybrid (
            const Hybrid< ValueType, IndexType > &  )
```

Copy-assigns a Hybrid matrix.

Inherits the executor, copies the Ell and Coo matrices.

#### 48.124.2.2 Hybrid() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Hybrid< ValueType, IndexType >::Hybrid (
            Hybrid< ValueType, IndexType > &&  )
```

Move-assigns a Hybrid matrix.

Inherits the executor, moves the Ell and Coo matrices. The moved-from matrix is empty (0x0 with empty Ell/Coo matrices).

### 48.124.3 Member Function Documentation

#### 48.124.3.1 compute_absolute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<absolute_type> gko::matrix::Hybrid< ValueType, IndexType >::compute_absolute
( ) const  [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Hybrid< ValueType, IndexType > > >.

#### 48.124.3.2 create() [1/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Hybrid> gko::matrix::Hybrid< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            size_type num_stored_elements_per_row,
            size_type stride,
            size_type num_nonzeros = {},
            std::shared_ptr< strategy_type > strategy = std::make_shared< automatic >() )
[static]
```

Creates an uninitialized Hybrid matrix of the specified size and method.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_stored_elements_per_row* | the number of stored elements per row |
| *stride* | stride of the rows |
| *num_nonzeros* | number of nonzeros |
| *strategy* | strategy of deciding the Hybrid config |

**Returns**

A smart pointer to the newly created matrix.

### 48.124.3.3 create() [2/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Hybrid> gko::matrix::Hybrid< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            size_type num_stored_elements_per_row,
            size_type stride,
            std::shared_ptr< strategy_type > strategy )  [static]
```

Creates an uninitialized Hybrid matrix of the specified size and method.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_stored_elements_per_row* | the number of stored elements per row |
| *stride* | stride of the rows |
| *strategy* | strategy of deciding the Hybrid config |

**Returns**

A smart pointer to the newly created matrix.

### 48.124.3.4 create() [3/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Hybrid> gko::matrix::Hybrid< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            size_type num_stored_elements_per_row,
            std::shared_ptr< strategy_type > strategy = std::make_shared< automatic >() )
[static]
```

Creates an uninitialized Hybrid matrix of the specified size and method.

(ell_stride is set to the number of rows of the matrix.)

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_stored_elements_per_row* | the number of stored elements per row |
| *strategy* | strategy of deciding the Hybrid config |

**Returns**

A smart pointer to the newly created matrix.

#### 48.124.3.5 create() [4/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Hybrid> gko::matrix::Hybrid< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            std::shared_ptr< strategy_type > strategy = std::make_shared< automatic >() )
[static]
```

Creates an uninitialized Hybrid matrix of the specified size and method.

(ell_num_stored_elements_per_row is set to the number of cols of the matrix. ell_stride is set to the number of rows of the matrix.)

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *strategy* | strategy of deciding the Hybrid config |

**Returns**

A smart pointer to the newly created matrix.

#### 48.124.3.6 create() [5/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Hybrid> gko::matrix::Hybrid< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< strategy_type > strategy = std::make_shared< automatic >() )
[static]
```

Creates an uninitialized Hybrid matrix of specified method.

(ell_num_stored_elements_per_row is set to the number of cols of the matrix. ell_stride is set to the number of rows of the matrix.)

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *strategy* | strategy of deciding the Hybrid config |

**Returns**

A smart pointer to the newly created matrix.

### 48.124.3.7 ell_col_at() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type gko::matrix::Hybrid< ValueType, IndexType >::ell_col_at (
            size_type row,
            size_type idx ) const  [inline], [noexcept]
```

Returns the `idx`-th column index of the `row`-th row in the ell part.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
535     {
536         return ell_->col_at(row, idx);
537     }
```

### 48.124.3.8 ell_col_at() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type& gko::matrix::Hybrid< ValueType, IndexType >::ell_col_at (
            size_type row,
            size_type idx )  [inline], [noexcept]
```

Returns the `idx`-th column index of the `row`-th row in the ell part.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
527     {
528         return ell_->col_at(row, idx);
529     }
```

#### 48.124.3.9 ell_val_at() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type gko::matrix::Hybrid< ValueType, IndexType >::ell_val_at (
              size_type row,
              size_type idx ) const  [inline], [noexcept]
```

Returns the `idx`-th non-zero element of the `row`-th row in the ell part.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
512     {
513         return ell_->val_at(row, idx);
514     }
```

#### 48.124.3.10 ell_val_at() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type& gko::matrix::Hybrid< ValueType, IndexType >::ell_val_at (
              size_type row,
              size_type idx )  [inline], [noexcept]
```

Returns the `idx`-th non-zero element of the `row`-th row in the ell part.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element |
| *idx* | the idx-th stored element of the row |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

```
504     {
505         return ell_->val_at(row, idx);
506     }
```

#### 48.124.3.11 extract_diagonal()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Hybrid< ValueType, IndexType >::extract↩
_diagonal ( ) const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

### 48.124.3.12 get_const_coo_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Hybrid< ValueType, IndexType >::get_const_coo_col_idxs ( )
const  [inline], [noexcept]
```

Returns the column indexes of the coo part.

**Returns**

the column indexes of the coo part.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
580     {
581         return coo_->get_const_col_idxs();
582     }
```

### 48.124.3.13 get_const_coo_row_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Hybrid< ValueType, IndexType >::get_const_coo_row_idxs ( )
const  [inline], [noexcept]
```

Returns the row indexes of the coo part.

**Returns**

the row indexes of the coo part.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
599     {
600         return coo_->get_const_row_idxs();
601     }
```

### 48.124.3.14 get_const_coo_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Hybrid< ValueType, IndexType >::get_const_coo_values ( ) const
[inline], [noexcept]
```

Returns the values of the coo part.

#### Returns

the values of the coo part.

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
561     {
562         return coo_->get_const_values();
563     }
```

### 48.124.3.15 get_const_device_view()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const_device_view gko::matrix::Hybrid< ValueType, IndexType >::get_const_device_view ( ) const
```

Returns a non-owning const device view of this matrix.

#### Returns

a const device view of this matrix.

### 48.124.3.16 get_const_ell_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Hybrid< ValueType, IndexType >::get_const_ell_col_idxs ( )
const  [inline], [noexcept]
```

Returns the column indexes of the ell part.

#### Returns

the column indexes of the ell part

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
461     {
462         return ell_->get_const_col_idxs();
463     }
```

### 48.124.3.17 get_const_ell_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Hybrid< ValueType, IndexType >::get_const_ell_values ( ) const
[inline], [noexcept]
```

Returns the values of the ell part.

#### Returns

the values of the ell part

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
442     {
443         return ell_->get_const_values();
444     }
```

### 48.124.3.18 get_coo()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const coo_type* gko::matrix::Hybrid< ValueType, IndexType >::get_coo ( ) const  [inline],
[noexcept]
```

Returns the matrix of the coo part.

#### Returns

the matrix of the coo part

```
618 { return coo_.get(); }
```

### 48.124.3.19 get_coo_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Hybrid< ValueType, IndexType >::get_coo_col_idxs ( )  [inline], [noexcept]
```

Returns the column indexes of the coo part.

#### Returns

the column indexes of the coo part.

```
570 { return coo_->get_col_idxs(); }
```

### 48.124.3.20 get_coo_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::get_coo_num_stored_elements ( ) const
[inline], [noexcept]
```

Returns the number of elements explicitly stored in the coo part.

#### Returns

the number of elements explicitly stored in the coo part

```
609     {
610         return coo_->get_num_stored_elements();
611     }
```

### 48.124.3.21 get_coo_row_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Hybrid< ValueType, IndexType >::get_coo_row_idxs ( )  [inline], [noexcept]
```

Returns the row indexes of the coo part.

#### Returns

the row indexes of the coo part.

```
589 { return coo_->get_row_idxs(); }
```

### 48.124.3.22 get_coo_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Hybrid< ValueType, IndexType >::get_coo_values ( )  [inline], [noexcept]
```

Returns the values of the coo part.

#### Returns

the values of the coo part.

```
551 { return coo_->get_values(); }
```

### 48.124.3.23 get_device_view()

```
template<typename ValueType = default_precision, typename IndexType = int32>
device_view gko::matrix::Hybrid< ValueType, IndexType >::get_device_view ( )
```

Returns a non-owning device view of this matrix.

#### Returns

a device view of this matrix.

#### 48.124.3.24 get_ell()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const ell_type* gko::matrix::Hybrid< ValueType, IndexType >::get_ell ( ) const  [inline],
[noexcept]
```

Returns the matrix of the ell part.

**Returns**

the matrix of the ell part

```
544 { return ell_.get(); }
```

#### 48.124.3.25 get_ell_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Hybrid< ValueType, IndexType >::get_ell_col_idxs ( )  [inline], [noexcept]
```

Returns the column indexes of the ell part.

**Returns**

the column indexes of the ell part

```
451 { return ell_->get_col_idxs(); }
```

#### 48.124.3.26 get_ell_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::get_ell_num_stored_elements ( ) const
[inline], [noexcept]
```

Returns the number of elements explicitly stored in the ell part.

**Returns**

the number of elements explicitly stored in the ell part

```
488     {
489         return ell_->get_num_stored_elements();
490     }
```

### 48.124.3.27 get_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::get_ell_num_stored_elements_per_row ( )
const  [inline], [noexcept]
```

Returns the number of stored elements per row of ell part.

#### Returns

the number of stored elements per row of ell part

```
471     {
472         return ell_->get_num_stored_elements_per_row();
473     }
```

### 48.124.3.28 get_ell_stride()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::get_ell_stride ( ) const  [inline],
[noexcept]
```

Returns the stride of the ell part.

#### Returns

the stride of the ell part

```
480 { return ell_->get_stride(); }
```

### 48.124.3.29 get_ell_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Hybrid< ValueType, IndexType >::get_ell_values ( )  [inline], [noexcept]
```

Returns the values of the ell part.

#### Returns

the values of the ell part

```
432 { return ell_->get_values(); }
```

#### 48.124.3.30 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::get_num_stored_elements ( ) const
[inline], [noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
626     {
627         return coo_->get_num_stored_elements() +
628                ell_->get_num_stored_elements();
629     }
```

#### 48.124.3.31 get_strategy() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename HybType >
std::shared_ptr<typename HybType::strategy_type> gko::matrix::Hybrid< ValueType, IndexType
>::get_strategy ( ) const
```

Returns the current strategy allowed in given hybrid format.

**Template Parameters**

| | |
|---|---|
| *HybType* | hybrid type |

**Returns**

the strategy

#### 48.124.3.32 get_strategy() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr< typename HybType::strategy_type > gko::matrix::Hybrid< ValueType, IndexType
>::get_strategy ( ) const  [inline], [noexcept]
```

Returns the strategy.

**Returns**

the strategy

```
637     {
638         return strategy_;
639     }
```

#### 48.124.3.33 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Hybrid& gko::matrix::Hybrid< ValueType, IndexType >::operator= (
            const Hybrid< ValueType, IndexType > &  )
```

Copy-assigns a Hybrid matrix.

Preserves the executor, copy-assigns the Ell and Coo matrices.

#### 48.124.3.34 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Hybrid& gko::matrix::Hybrid< ValueType, IndexType >::operator= (
            Hybrid< ValueType, IndexType > &&  )
```

Move-assigns a Hybrid matrix.

Preserves the executor, move-assigns the Ell and Coo matrices. The moved-from matrix is empty (0x0 with empty Ell/Coo matrices).

#### 48.124.3.35 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Hybrid< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.124.3.36 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Hybrid< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.124.3.37 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Hybrid< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.124.3.38 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Hybrid< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following files:

- ginkgo/core/matrix/coo.hpp
- ginkgo/core/matrix/hybrid.hpp

## 48.125 gko::factorization::Ic< ValueType, IndexType > Class Template Reference

Represents an incomplete Cholesky factorization (IC(0)) of a sparse matrix.

```
#include <ginkgo/core/factorization/ic.hpp>
```

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## Additional Inherited Members

## 48.125.1 Detailed Description

**template<typename ValueType = gko::default_precision, typename IndexType = gko::int32>**
**class gko::factorization::Ic< ValueType, IndexType >**

Represents an incomplete Cholesky factorization (IC(0)) of a sparse matrix.

More specifically, it consists of a lower triangular factor $L$ and its conjugate transpose $L^H$ with sparsity pattern $\mathcal{S}(L + L^H) = \mathcal{S}(A)$ fulfilling $LL^H = A$ at every non-zero location of $A$.

**Template Parameters**

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

## 48.125.2 Member Function Documentation

### 48.125.2.1 parse()

```
template<typename ValueType = gko::default_precision, typename IndexType = gko::int32>
static parameters_type gko::factorization::Ic< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/ic.hpp

## 48.126 gko::preconditioner::Ic< ValueType, IndexType > Class Template Reference

The Incomplete Cholesky (IC) preconditioner solves the equation $LL^H x = b$ for a given lower triangular matrix $L$ and the right hand side $b$ (can contain multiple right hand sides).

`#include <ginkgo/core/preconditioner/ic.hpp>`

### Public Member Functions

- std::shared_ptr< const LinOp > get_l_solver () const

  *Returns the solver which is used for the provided $L$ matrix.*
- std::shared_ptr< const LinOp > get_lh_solver () const

  *Returns the solver which is used for the $L^H$ matrix.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- Ic & operator= (const Ic &other)

  *Copy-assigns an IC preconditioner.*
- Ic & operator= (Ic &&other)

  *Move-assigns an IC preconditioner.*
- Ic (const Ic &other)

  *Copy-constructs an IC preconditioner.*
- Ic (Ic &&other)

  *Move-constructs an IC preconditioner.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< value_type, index_type >())

  *Create the parameters from the property_tree.*

## 48.126.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::preconditioner::Ic< ValueType, IndexType >

The Incomplete Cholesky (IC) preconditioner solves the equation $LL^H x = b$ for a given lower triangular matrix $L$ and the right hand side $b$ (can contain multiple right hand sides).

It allows setting the solver for $L$, defaulting to solver::LowerTrs, which is a direct triangular solver. The solver for $L^H$ is the conjugate-transposed solver for $L$, ensuring that the preconditioner is symmetric and positive-definite. For this $L$ solver, a factory can be provided (using `with_l_solver`) to have more control over their behavior. In particular, it is possible to use an iterative method for solving the triangular systems.

An object of this class can be created with a matrix or a gko::Composition containing two matrices. If created with a matrix, it is factorized before creating the solver. If a gko::Composition (containing two matrices) is used, the first operand will be taken as the $L$ matrix, the second will be considered the $L^H$ matrix, which helps to avoid the otherwise necessary transposition of $L$ inside the solver. ParIc can be directly used, since it orders the factors in the correct way.

**Note**

When providing a gko::Composition, the first matrix must be the lower matrix ( $L$), and the second matrix must be its conjugate-transpose ( $L^H$). If they are swapped, solving might crash or return the wrong result.

Do not use symmetric solvers (like CG) for the $L$ solver since both matrices ( $L$ and $L^H$) are, by design, not symmetric.

This class is not thread safe (even a const object is not) because it uses an internal cache to accelerate multiple (sequential) applies. Using it in parallel can lead to segmentation faults, wrong results and other unwanted behavior.

The default template during parse is <ValueType, IndexType> not <LowerTrs, IndexType>. Only the variants with ValueType are supported in parse.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used for the $L$ matrix. |
| *IndexType* | type of the indices when ParIc is used to generate the $L$ and $L^H$ factors. Irrelevant otherwise. |

## 48.126.2 Constructor & Destructor Documentation

### 48.126.2.1 Ic() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::preconditioner::Ic< ValueType, IndexType >::Ic (
            const Ic< ValueType, IndexType > & other )
```

Copy-constructs an IC preconditioner.

Inherits the executor, shallow-copies the solvers and parameters.

#### 48.126.2.2 Ic() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::preconditioner::Ic< ValueType, IndexType >::Ic (
            Ic< ValueType, IndexType > && other )
```

Move-constructs an IC preconditioner.

Inherits the executor, moves the solvers and parameters. The moved-from object is empty (0x0 with nullptr solvers and default parameters)

### 48.126.3 Member Function Documentation

#### 48.126.3.1 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::preconditioner::Ic< ValueType, IndexType >::conj_transpose ( )
const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.126.3.2 get_l_solver()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::preconditioner::Ic< ValueType, IndexType >::get_l_solver (
) const  [inline]
```

Returns the solver which is used for the provided $L$ matrix.

**Returns**

the solver which is used for the provided $L$ matrix

```
172 { return l_solver_; }
```

#### 48.126.3.3 get_lh_solver()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::preconditioner::Ic< ValueType, IndexType >::get_lh_solver (
) const  [inline]
```

Returns the solver which is used for the $L^H$ matrix.

**Returns**

the solver which is used for the $L^H$ matrix

```
179 { return lh_solver_; }
```

#### 48.126.3.4 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Ic& gko::preconditioner::Ic< ValueType, IndexType >::operator= (
            const Ic< ValueType, IndexType > & other )
```

Copy-assigns an IC preconditioner.

Preserves the executor, shallow-copies the solvers and parameters. Creates a clone of the solvers if they are on the wrong executor.

#### 48.126.3.5 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Ic& gko::preconditioner::Ic< ValueType, IndexType >::operator= (
            Ic< ValueType, IndexType > && other )
```

Move-assigns an IC preconditioner.

Preserves the executor, moves the solvers and parameters. Creates a clone of the solvers if they are on the wrong executor. The moved-from object is empty (0x0 with nullptr solvers and default parameters)

#### 48.126.3.6 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::preconditioner::Ic< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< value↩
_type, index_type >() )  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. When l_solver_type uses LinOp not concrete type, it will use the default_precision in ginkgo. |

**Returns**

parameters

**Note**

only support the following when using <ValueType, IndexType> not <LSolverType, IndexType> variants

#### 48.126.3.7 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::preconditioner::Ic< ValueType, IndexType >::transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/ic.hpp

## 48.127 gko::batch::matrix::Identity< ValueType > Class Template Reference

The batch Identity matrix, which represents a batch of Identity matrices.

```
#include <ginkgo/core/matrix/batch_identity.hpp>
```

## Public Member Functions

- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector.*
- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x)

  *Apply the matrix to a multi-vector with a linear combination of the given input vector.*
- void apply (ptr_param< const MultiVector< value_type >> b, ptr_param< MultiVector< value_type >> x) const
- void apply (ptr_param< const MultiVector< value_type >> alpha, ptr_param< const MultiVector< value_type >> b, ptr_param< const MultiVector< value_type >> beta, ptr_param< MultiVector< value_type >> x) const

## Static Public Member Functions

- static std::unique_ptr< Identity > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size=batch_dim< 2 >{})

  *Creates an Identity matrix of the specified size.*

### 48.127.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::batch::matrix::Identity< ValueType >**

The batch Identity matrix, which represents a batch of Identity matrices.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

### 48.127.2 Member Function Documentation

#### 48.127.2.1 apply() [1/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Identity< ValueType >::apply (
            ptr_param< const MultiVector< value_type >> alpha,
            ptr_param< const MultiVector< value_type >> b,
            ptr_param< const MultiVector< value_type >> beta,
            ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector with a linear combination of the given input vector.

Represents the matrix vector multiplication, x = alpha ∗ I

- b + beta ∗ x, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *alpha* | the scalar to scale the matrix-vector product with |
| *b* | the multi-vector to be applied to |
| *beta* | the scalar to scale the x vector with |
| *x* | the output multi-vector |

#### 48.127.2.2 apply() [2/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Identity< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> alpha,
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< const MultiVector< value_type >> beta,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.127.2.3 apply() [3/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Identity< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x )
```

Apply the matrix to a multi-vector.

Represents the matrix vector multiplication, x = I ∗ b, where x and b are both multi-vectors.

**Parameters**

| | |
|---|---|
| *b* | the multi-vector to be applied to |
| *x* | the output multi-vector |

#### 48.127.2.4 apply() [4/4]

```
template<typename ValueType = default_precision>
void gko::batch::matrix::Identity< ValueType >::apply (
             ptr_param< const MultiVector< value_type >> b,
             ptr_param< MultiVector< value_type >> x ) const
```

#### 48.127.2.5 create()

```
template<typename ValueType = default_precision>
static std::unique_ptr<Identity> gko::batch::matrix::Identity< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size = batch_dim< 2 >{} )  [static]
```

Creates an Identity matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the batch matrices in a batch_dim object |

**Returns**

A smart pointer to the newly created matrix.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/batch_identity.hpp

## 48.128 gko::matrix::Identity< ValueType > Class Template Reference

This class is a utility which efficiently implements the identity matrix (a linear operator which maps each vector to itself).

```
#include <ginkgo/core/matrix/identity.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*

### Static Public Member Functions

- static std::unique_ptr< Identity > create (std::shared_ptr< const Executor > exec, dim< 2 > size)

  *Creates an Identity matrix of the specified size.*
- static std::unique_ptr< Identity > create (std::shared_ptr< const Executor > exec, size_type size=0)

  *Creates an Identity matrix of the specified size.*

### 48.128.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::matrix::Identity< ValueType >**

This class is a utility which efficiently implements the identity matrix (a linear operator which maps each vector to itself).

Thus, objects of the Identity class always represent a square matrix, and don't require any storage for their values. The apply method is implemented as a simple copy (or a linear combination).

**Note**

This class is useful when composing it with other operators. For example, it can be used instead of a preconditioner in Krylov solvers, if one wants to run a "plain" solver, without using a preconditioner.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

### 48.128.2 Member Function Documentation

#### 48.128.2.1 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Identity< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.128.2.2 create() [1/2]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Identity> gko::matrix::Identity< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            dim< 2 > size )  [static]
```

Creates an Identity matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the Identity operator will be stored |
| *size* | size of the matrix (must be square) |

#### 48.128.2.3 create() [2/2]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Identity> gko::matrix::Identity< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            size_type size = 0 )  [static]
```

Creates an Identity matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the Identity operator will be stored |
| *size* | size of the matrix |

#### 48.128.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::matrix::Identity< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/identity.hpp

## 48.129 gko::matrix::IdentityFactory< ValueType > Class Template Reference

This factory is a utility which can be used to generate Identity operators.

```
#include <ginkgo/core/matrix/identity.hpp>
```

## Static Public Member Functions

- static std::unique_ptr< IdentityFactory > create (std::shared_ptr< const Executor > exec)

  *Creates a new Identity factory.*

## Additional Inherited Members

### 48.129.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::matrix::IdentityFactory< ValueType >**

This factory is a utility which can be used to generate Identity operators.

The factory will generate the Identity matrix with the same dimension as the passed in operator. It will throw an exception if the operator is not square.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

### 48.129.2 Member Function Documentation

#### 48.129.2.1 create()

```
template<typename ValueType = default_precision>
static std::unique_ptr<IdentityFactory> gko::matrix::IdentityFactory< ValueType >::create (
            std::shared_ptr< const Executor > exec )  [inline], [static]
```

Creates a new Identity factory.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the Identity operator will be stored |

**Returns**

a unique pointer to the newly created factory

```
106     {
107         return std::unique_ptr<IdentityFactory>(
108             new IdentityFactory(std::move(exec)));
109     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/identity.hpp

# 48.130 gko::solver::Idr< ValueType > Class Template Reference

IDR(s) is an efficient method for solving large nonsymmetric systems of linear equations.

`#include <ginkgo/core/solver/idr.hpp>`

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*
- size_type get_subspace_dim () const

  *Gets the subspace dimension of the solver.*
- void set_subspace_dim (const size_type other)

  *Sets the subspace dimension of the solver.*
- remove_complex< ValueType > get_kappa () const

  *Gets the kappa parameter of the solver.*
- void set_kappa (const remove_complex< ValueType > other)

  *Sets the kappa parameter of the solver.*
- bool get_deterministic () const

  *Gets the deterministic parameter of the solver.*
- void set_deterministic (const bool other)

  *Sets the deterministic parameter of the solver.*
- bool get_complex_subspace () const

  *Gets the complex_subspace parameter of the solver.*
- void set_complex_subpsace (const bool other)

  *Sets the complex_subspace parameter of the solver.*
- void set_complex_subspace (const bool other)

  *Sets the complex_subspace parameter of the solver.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.130.1 Detailed Description

template<typename ValueType = default_precision>
class gko::solver::Idr< ValueType >

IDR(s) is an efficient method for solving large nonsymmetric systems of linear equations.

The implementation follows the elegant variant that exploits the biorthogonality of the shadow vectors.

The method is based on the induced dimension reduction (IDR) theorem. Let $P = [p_1, \ldots, p_s]$ be the shadow matrix — the $s$ random, orthonormalized vectors that Ginkgo stores in a dense matrix — and let

$$\mathcal{S} = \{v \in \mathbb{C}^N : P^H v = 0\}$$

be its left null space, that is, the orthogonal complement of the space spanned by the shadow vectors. Starting from the full Krylov space $\mathcal{G}_0 = \mathcal{K}_N(A, r_0)$, the theorem states that the subspaces

$$\mathcal{G}_j = (I - \omega_j A)(\mathcal{G}_{j-1} \cap \mathcal{S}), \qquad \omega_j \neq 0,$$

are nested, $\mathcal{G}_j \subset \mathcal{G}_{j-1}$, and that $\mathcal{G}_j = \{0\}$ for some $j \leq N$. IDR(s) forces the residuals into these shrinking spaces; in exact arithmetic it therefore reaches the true solution after at most $N + N/s$ matrix-vector products.

#### References

- Van Gijzen, M. B., Sonneveld, P. *Algorithm 913: An Elegant IDR(s) Variant that Efficiently Exploits Biorthogonality Properties.* ACM Transactions on Mathematical Software, 38 (1), Article 5, 2011. https://doi.org/10.1145/2049662.2049667 (the implemented variant; the IDR theorem is Theorem 2.1 in Section 2.1)
- Sonneveld, P., Van Gijzen, M. B. *IDR(s): A Family of Simple and Fast Algorithms for Solving Large Nonsymmetric Systems of Linear Equations.* SIAM Journal on Scientific Computing, 31 (2), 1035–1062, 2008. https://doi.org/10.1137/070685804 (proof of the IDR theorem and of the $N + N/s$ bound)

#### Template Parameters

| | |
|---|---|
| *ValueType* | precision of the elements of the system matrix. |

### 48.130.2 Member Function Documentation

#### 48.130.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Idr< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

#### Returns

true as iterative solvers use the data in x as an initial guess.

```
102 { return true; }
```

#### 48.130.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Idr< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.130.2.3 get_complex_subspace()

```
template<typename ValueType = default_precision>
bool gko::solver::Idr< ValueType >::get_complex_subspace ( ) const  [inline]
```

Gets the complex_subspace parameter of the solver.

**Returns**

the complex_subspace parameter

```
158 { return parameters_.complex_subspace; }
```

#### 48.130.2.4 get_deterministic()

```
template<typename ValueType = default_precision>
bool gko::solver::Idr< ValueType >::get_deterministic ( ) const  [inline]
```

Gets the deterministic parameter of the solver.

**Returns**

the deterministic parameter

```
141 { return parameters_.deterministic; }
```

#### 48.130.2.5 get_kappa()

```
template<typename ValueType = default_precision>
remove_complex<ValueType> gko::solver::Idr< ValueType >::get_kappa ( ) const  [inline]
```

Gets the kappa parameter of the solver.

**Returns**

the kappa parameter

```
124 { return parameters_.kappa; }
```

#### 48.130.2.6 get_subspace_dim()

```
template<typename ValueType = default_precision>
size_type gko::solver::Idr< ValueType >::get_subspace_dim ( ) const  [inline]
```

Gets the subspace dimension of the solver.

**Returns**

the subspace Dimension

```
108 { return parameters_.subspace_dim; }
```

#### 48.130.2.7 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Idr< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.130.2.8 set_complex_subpsace()

```
template<typename ValueType = default_precision>
void gko::solver::Idr< ValueType >::set_complex_subpsace (
            const bool other )  [inline]
```

Sets the complex_subspace parameter of the solver.

**Parameters**

| | |
|---|---|
| *other* | the new complex_subspace parameter |

```
168     {
169         this->set_complex_subspace(other);
170     }
```

References gko::solver::Idr< ValueType >::set_complex_subspace().

#### 48.130.2.9 set_complex_subspace()

```
template<typename ValueType = default_precision>
void gko::solver::Idr< ValueType >::set_complex_subspace (
             const bool other )  [inline]
```

Sets the complex_subspace parameter of the solver.

**Parameters**

| | |
|---|---|
| *other* | the new complex_subspace parameter |

```
178     {
179         parameters_.complex_subspace = other;
180     }
```

Referenced by gko::solver::Idr< ValueType >::set_complex_subpsace().

#### 48.130.2.10 set_deterministic()

```
template<typename ValueType = default_precision>
void gko::solver::Idr< ValueType >::set_deterministic (
             const bool other )  [inline]
```

Sets the deterministic parameter of the solver.

**Parameters**

| | |
|---|---|
| *other* | the new deterministic parameter |

```
149     {
150         parameters_.deterministic = other;
151     }
```

#### 48.130.2.11 set_kappa()

```
template<typename ValueType = default_precision>
void gko::solver::Idr< ValueType >::set_kappa (
             const remove_complex< ValueType > other )  [inline]
```

Sets the kappa parameter of the solver.

**Parameters**

| | |
|---|---|
| *other* | the new kappa parameter |

```
132     {
133         parameters_.kappa = other;
134     }
```

#### 48.130.2.12 set_subspace_dim()

```
template<typename ValueType = default_precision>
void gko::solver::Idr< ValueType >::set_subspace_dim (
             const size_type other )  [inline]
```

Sets the subspace dimension of the solver.

**Parameters**

| | |
|---|---|
| *other* | the new subspace Dimension |

```
115     {
116         parameters_.subspace_dim = other;
117     }
```

#### 48.130.2.13 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Idr< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/idr.hpp

## 48.131 gko::factorization::Ilu< ValueType, IndexType > Class Template Reference

Represents an incomplete LU factorization – ILU(0) – of a sparse matrix.

```
#include <ginkgo/core/factorization/ilu.hpp>
```

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## Additional Inherited Members

### 48.131.1 Detailed Description

**template<typename ValueType = gko::default_precision, typename IndexType = gko::int32>**
**class gko::factorization::Ilu< ValueType, IndexType >**

Represents an incomplete LU factorization – ILU(0) – of a sparse matrix.

More specifically, it consists of a lower unitriangular factor $L$ and an upper triangular factor $U$ with sparsity pattern $\mathcal{S}(L+U) = \mathcal{S}(A)$ fulfilling $LU = A$ at every non-zero location of $A$.

**Template Parameters**

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

### 48.131.2 Member Function Documentation

#### 48.131.2.1 parse()

```
template<typename ValueType = gko::default_precision, typename IndexType = gko::int32>
static parameters_type gko::factorization::Ilu< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/ilu.hpp

## 48.132 gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType > Class Template Reference

The Incomplete LU (ILU) preconditioner solves the equation $LUx = b$ for a given lower triangular matrix $L$, an upper triangular matrix $U$ and the right hand side $b$ (can contain multiple right hand sides).

`#include <ginkgo/core/preconditioner/ilu.hpp>`

### Public Member Functions

- std::shared_ptr< const LinOp > get_l_solver () const

  *Returns the solver which is used for the provided $L$ matrix.*
- std::shared_ptr< const LinOp > get_u_solver () const

  *Returns the solver which is used for the provided $U$ matrix.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- Ilu & operator= (const Ilu &other)

  *Copy-assigns an ILU preconditioner.*
- Ilu & operator= (Ilu &&other)

  *Move-assigns an ILU preconditioner.*
- Ilu (const Ilu &other)

  *Copy-constructs an ILU preconditioner.*
- Ilu (Ilu &&other)

  *Move-constructs an ILU preconditioner.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< value_type, index_type >())

  *Create the parameters from the property_tree.*

## 48.132.1 Detailed Description

template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType = int32>
class gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >

The Incomplete LU (ILU) preconditioner solves the equation $LUx = b$ for a given lower triangular matrix $L$, an upper triangular matrix $U$ and the right hand side $b$ (can contain multiple right hand sides).

It allows to set both the solver for $L$ and the solver for $U$ independently, while providing the defaults solver::LowerTrs and solver::UpperTrs, which are direct triangular solvers. For these solvers, a factory can be provided (with `with_l_solver` and `with_u_solver`) to have more control over their behavior. In particular, it is possible to use an iterative method for solving the triangular systems.

An object of this class can be created with a matrix or a gko::Composition containing two matrices. If created with a matrix, it is factorized before creating the solver. If a gko::Composition (containing two matrices) is used, the first operand will be taken as the $L$ matrix, the second will be considered the $U$ matrix. ParIlu can be directly used, since it orders the factors in the correct way.

**Note**

When providing a gko::Composition, the first matrix must be the lower matrix ( $L$), and the second matrix must be the upper matrix ( $U$). If they are swapped, solving might crash or return the wrong result.

Do not use symmetric solvers (like CG) for $L$ or $U$ solvers since both matrices ( $L$ and $U$) are, by design, not symmetric.

This class is not thread safe (even a const object is not) because it uses an internal cache to accelerate multiple (sequential) applies. Using it in parallel can lead to segmentation faults, wrong results and other unwanted behavior.

The default template during parse is <ValueType, IndexType> not <LowerTrs, IndexType>. Only the variants with ValueType are supported in parse.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used for the $L$ and $U$ matrices. |
| *ReverseApply* | default behavior (ReverseApply = false) is first to solve with $L$ ( $Ly = b$) and then with $U$ ( $Ux = y$). When set to true, it will solve first with $U$, and then with $L$. |
| *IndexType* | Type of the indices when ParIlu is used to generate both $L$ and $U$ factors. Irrelevant otherwise. |

## 48.132.2 Constructor & Destructor Documentation

### 48.132.2.1 Ilu() [1/2]

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::Ilu (
            const Ilu< ValueType, ReverseApply, IndexType > & other )
```

Copy-constructs an ILU preconditioner.

Inherits the executor, shallow-copies the solvers and parameters.

#### 48.132.2.2 Ilu() [2/2]

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::Ilu (
            Ilu< ValueType, ReverseApply, IndexType > && other )
```

Move-constructs an ILU preconditioner.

Inherits the executor, moves the solvers and parameters. The moved-from object is empty (0x0 with nullptr solvers and default parameters)

### 48.132.3 Member Function Documentation

#### 48.132.3.1 conj_transpose()

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
std::unique_ptr<LinOp> gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::conj↩
_transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.132.3.2 get_l_solver()

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
std::shared_ptr<const LinOp> gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >↩
::get_l_solver ( ) const  [inline]
```

Returns the solver which is used for the provided $L$ matrix.

**Returns**

the solver which is used for the provided $L$ matrix

```
190 { return l_solver_; }
```

#### 48.132.3.3 get_u_solver()

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
std::shared_ptr<const LinOp> gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >↩
::get_u_solver ( ) const  [inline]
```

Returns the solver which is used for the provided $U$ matrix.

**Returns**

the solver which is used for the provided $U$ matrix

```
197 { return u_solver_; }
```

#### 48.132.3.4 operator=() [1/2]

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
Ilu& gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::operator= (
            const Ilu< ValueType, ReverseApply, IndexType > & other )
```

Copy-assigns an ILU preconditioner.

Preserves the executor, shallow-copies the solvers and parameters. Creates a clone of the solvers if they are on the wrong executor.

#### 48.132.3.5 operator=() [2/2]

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
Ilu& gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::operator= (
            Ilu< ValueType, ReverseApply, IndexType > && other )
```

Move-assigns an ILU preconditioner.

Preserves the executor, moves the solvers and parameters. Creates a clone of the solvers if they are on the wrong executor. The moved-from object is empty (0x0 with nullptr solvers and default parameters)

#### 48.132.3.6 parse()

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
static parameters_type gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< value↩
_type, index_type >() )  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

**Note**

only support the following when using <ValueType, ValueType, ReverseApply, IndexType> not <LSolverType, USolverType, ReverseApply, IndexType> variants

#### 48.132.3.7 transpose()

```
template<typename ValueType = default_precision, bool ReverseApply = false, typename IndexType
= int32>
std::unique_ptr<LinOp> gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >::transpose
( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/ilu.hpp

## 48.133 gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit Class Reference

imbalance_bounded_limit is a strategy_type which decides the number of stored elements per row of the ell part.

```
#include <ginkgo/core/matrix/hybrid.hpp>
```

## Public Member Functions

- imbalance_bounded_limit (double percent=0.8, double ratio=0.0001)

  *Creates a imbalance_bounded_limit strategy.*
- size_type compute_ell_num_stored_elements_per_row (array< size_type > *row_nnz) const override

  *Computes the number of stored elements per row of the ell part.*
- auto get_percentage () const

  *Get the percent setting.*
- auto get_ratio () const

  *Get the ratio setting.*

### 48.133.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit**

imbalance_bounded_limit is a strategy_type which decides the number of stored elements per row of the ell part.

It uses the imbalance_limit and adds the upper bound of the number of ell's cols by the number of rows.

### 48.133.2 Member Function Documentation

#### 48.133.2.1 compute_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::compute_ell_↩
num_stored_elements_per_row (
            array< size_type > * row_nnz ) const  [inline], [override], [virtual]
```

Computes the number of stored elements per row of the ell part.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |

**Returns**

the number of stored elements per row of the ell part

Implements gko::matrix::Hybrid< ValueType, IndexType >::strategy_type.

```
285         {
286             auto num_rows = row_nnz->get_size();
287             auto ell_cols =
288                 strategy_.compute_ell_num_stored_elements_per_row(row_nnz);
289             return std::min(ell_cols,
290                             static_cast<size_type>(num_rows * ratio_));
291         }
```

References gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::compute_ell_num_stored_elements↩
_per_row(), and gko::array< ValueType >::get_size().

Referenced by gko::matrix::Hybrid< ValueType, IndexType >::automatic::compute_ell_num_stored_elements_↩ per_row().

#### 48.133.2.2 get_percentage()

```
template<typename ValueType = default_precision, typename IndexType = int32>
auto gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::get_percentage ( )
const  [inline]
```

Get the percent setting.

**Returns**

percent

```
298 { return strategy_.get_percentage(); }
```

References gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::get_percentage().

#### 48.133.2.3 get_ratio()

```
template<typename ValueType = default_precision, typename IndexType = int32>
auto gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::get_ratio ( ) const
[inline]
```

Get the ratio setting.

**Returns**

ratio

```
305 { return ratio_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/hybrid.hpp

## 48.134 gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit Class Reference

imbalance_limit is a strategy_type which decides the number of stored elements per row of the ell part according to the percent.

```
#include <ginkgo/core/matrix/hybrid.hpp>
```

## Public Member Functions

- imbalance_limit (double percent=0.8)

  *Creates a imbalance_limit strategy.*
- size_type compute_ell_num_stored_elements_per_row (array< size_type > *row_nnz) const override

  *Computes the number of stored elements per row of the ell part.*
- auto get_percentage () const

  *Get the percent setting.*

### 48.134.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit**

imbalance_limit is a strategy_type which decides the number of stored elements per row of the ell part according to the percent.

It sorts the number of nonzeros of each row and takes the value at the position `floor(percent * num_row)` as the number of stored elements per row of the ell part. Thus, at least `percent` rows of all are in the ell part.

### 48.134.2 Constructor & Destructor Documentation

#### 48.134.2.1 imbalance_limit()

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::imbalance_limit (
            double percent = 0.8 )  [inline], [explicit]
```

Creates a imbalance_limit strategy.

**Parameters**

| | |
|---|---|
| *percent* | the row_nnz[floor(num_rows*percent)] is the number of stored elements per row of the ell part |

```
235                                                 :  percent_(percent)
236         {
237             percent_ = std::min(percent_, 1.0);
238             percent_ = std::max(percent_, 0.0);
239         }
```

### 48.134.3 Member Function Documentation

#### 48.134.3.1 compute_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::compute_ell_num_↩
```

```
stored_elements_per_row (
            array< size_type > * row_nnz ) const  [inline], [override], [virtual]
```

Computes the number of stored elements per row of the ell part.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |

**Returns**

the number of stored elements per row of the ell part

Implements gko::matrix::Hybrid< ValueType, IndexType >::strategy_type.

```
243         {
244             auto row_nnz_val = row_nnz->get_data();
245             auto num_rows = row_nnz->get_size();
246             if (num_rows == 0) {
247                 return 0;
248             }
249             std::sort(row_nnz_val, row_nnz_val + num_rows);
250             if (percent_ < 1) {
251                 auto percent_pos = static_cast<size_type>(num_rows * percent_);
252                 return row_nnz_val[percent_pos];
253             } else {
254                 return row_nnz_val[num_rows - 1];
255             }
256         }
```

References gko::array< ValueType >::get_data(), and gko::array< ValueType >::get_size().

Referenced by gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::compute_ell_num_↩stored_elements_per_row(), and gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit::compute↩_ell_num_stored_elements_per_row().

### 48.134.3.2 get_percentage()

```
template<typename ValueType = default_precision, typename IndexType = int32>
auto gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::get_percentage ( ) const
[inline]
```

Get the percent setting.

**Returns**

percent

```
263 { return percent_; }
```

Referenced by gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit::get_percentage(), and gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit::get_percentage().

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/hybrid.hpp

# 48.135 gko::stop::ImplicitResidualNorm< ValueType > Class Template Reference

Stopping criterion based on a solver-maintained squared residual-norm estimate $\rho_k^2 \approx \|r_k\|^2$.

`#include <ginkgo/core/stop/residual_norm.hpp>`

## Additional Inherited Members

### 48.135.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::stop::ImplicitResidualNorm< ValueType >**

Stopping criterion based on a solver-maintained squared residual-norm estimate $\rho_k^2 \approx \|r_k\|^2$.

Several Krylov methods (CG, BiCGSTAB, ...) already compute this quantity per iteration as a by-product of their inner-product recurrences, so the criterion can check convergence without computing a norm of its own — saving the global reduction the explicit ResidualNorm form needs when the solver only passes $r_k$. The iteration halts once $\rho_k \leq \tau \cdot \beta$, with the same `reduction_factor` and `baseline` factory parameters as ResidualNorm:

- `mode::rhs_norm` (default) — $\rho_k \leq \tau \|b\|$.
- `mode::initial_resnorm` — $\rho_k \leq \tau \|r_0\|$.
- `mode::absolute` — $\rho_k \leq \tau$.

Because $\rho_k^2$ is updated by short recurrences rather than recomputed from $b - Ax_k$ each step, it can drift from the true residual norm on long runs in finite precision; pair this criterion with an Iteration cap when that matters.

**Note**

The solver must pass the squared estimate through `Updater::implicit_sq_residual_norm` on every check — there is no fallback path. If it is missing, ::gko::NotSupported() is thrown at check time.

Baseline prerequisites at construction time mirror ResidualNorm:

- `mode::rhs_norm` requires $b$.
- `mode::initial_resnorm` requires either $r_0$ explicitly or the triple $(A, b, x_0)$ to derive it.
- `mode::absolute` requires $b$ for sizing the per-RHS baseline. If the required arguments are missing, ::gko::NotSupported() is thrown.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/residual_norm.hpp

# 48.136 gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType > Class Template Reference

This class defines mappings between global and local indices.

`#include <ginkgo/core/distributed/index_map.hpp>`

## Public Member Functions

- array< LocalIndexType > map_to_local (const array< GlobalIndexType > &global_ids, index_space index_space_v) const

  *Maps global indices to local indices.*
- array< GlobalIndexType > map_to_global (const array< LocalIndexType > &local_idxs, index_space index_space_v) const

  *Maps local indices to global indices.*
- size_type get_global_size () const

  *get size of the global index space*
- size_type get_local_size () const

  *get size of index_space::local*
- size_type get_non_local_size () const

  *get size of index_space::non_local*
- index_map (std::shared_ptr< const Executor > exec, std::shared_ptr< const partition_type > partition, comm_index_type rank, const array< GlobalIndexType > &recv_connections)

  *Creates a new index map.*
- index_map (std::shared_ptr< const Executor > exec)

  *Creates an empty index map.*
- const segmented_array< GlobalIndexType > & get_remote_global_idxs () const

  *get the index set $R_k$ for this rank.*
- const segmented_array< LocalIndexType > & get_remote_local_idxs () const

  *get the index set $R_k$, but mapped to their respective local index space.*
- const array< comm_index_type > & get_remote_target_ids () const

  *get the rank ids which contain indices accessed by this rank.*
- std::shared_ptr< const partition_type > get_partition () const

  *get the partition associated with this index map.*
- std::shared_ptr< const Executor > get_executor () const

  *get the associated executor.*

### 48.136.1 Detailed Description

**template<typename LocalIndexType, typename GlobalIndexType = int64>**
**class gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >**

This class defines mappings between global and local indices.

This class splits up the global index space and differentiates between locally owned and non-locally owned index spaces. Given an index space $I = [0, \dots, N)$ that is partitioned into $P$ disjoint subsets $I_k, k = 1, \dots, P$, this class defines for each subset an extended global index set $\hat{I}_k \supset I_K$. The extended index set contains the global indices owned by part $k$, as well as remote indices $R_k = \hat{I}_k \setminus I_k$, which are also accessed by part $k$, but owned by parts $l \neq k$. At the core, this class provides mappings from the global index space $I$ into different local index spaces. The combined local index space (index_space::combined) is then defined as $[0, \dots, |\hat{I}_k|)$. Additionally, the combined index space can be separated into locally owned (index_space::local) and non-locally owned (index_space::non_local). The locally owned indices are defined as $[0, \dots, |I_k|)$, and the non-locally owned as $[0, \dots, |R_k|)$. With these index sets, the following mappings are defined:

- $c_k : \hat{I}_k \mapsto [0, \dots, |\hat{I}_k|)$ which maps global indices into the combined/full local index space (denoted as index_space::combined),
- $l_k : I_k \mapsto [0, \dots, |I_k|)$ which maps global indices into the locally owned index space (denoted as index_space::local),

- $r_k : R_k \mapsto [0, \ldots, |R_k|)$ which maps global indices into the non-locally owned index space (denoted as index_space::non_local).

The required map can be selected by passing the appropriate type of an index_space.

The index map for $I_k$ has no knowledge about any other index maps for $I_l, l \neq k$. In particular, any global index passed to the `map_to_local` map that is not part of the specified index space, will be mapped to an invalid_index.

**Template Parameters**

| | |
|---|---|
| *LocalIndexType* | type for local indices |
| *GlobalIndexType* | type for global indices |

## 48.136.2 Constructor & Destructor Documentation

### 48.136.2.1 index_map()

```
template<typename LocalIndexType, typename GlobalIndexType = int64>
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >::index_map (
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< const partition_type > partition,
            comm_index_type rank,
            const array< GlobalIndexType > & recv_connections )
```

Creates a new index map.

The passed in recv_connections may contain duplicates, which will be filtered out.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *partition* | the partition of the global index set |
| *rank* | the id of the global index space subset |
| *recv_connections* | the global indices that are not owned by this rank, but accessed by it. Indices that are owned by this rank are ignored. |

## 48.136.3 Member Function Documentation

### 48.136.3.1 get_remote_global_idxs()

```
template<typename LocalIndexType, typename GlobalIndexType = int64>
const segmented_array<GlobalIndexType>& gko::experimental::distributed::index_map< Local↩
IndexType, GlobalIndexType >::get_remote_global_idxs ( ) const
```

get the index set $R_k$ for this rank.

The indices are ordered by their owning rank and global index.

#### 48.136.3.2 get_remote_local_idxs()

```
template<typename LocalIndexType, typename GlobalIndexType = int64>
const segmented_array<LocalIndexType>& gko::experimental::distributed::index_map< Local↩
IndexType, GlobalIndexType >::get_remote_local_idxs ( ) const
```

get the index set $R_k$, but mapped to their respective local index space.

The indices are grouped by their owning rank and sorted according to their global index within each group.

The set $R_k = \hat{I}_k \setminus I_k$ can also be written as the union of the intersection of $\hat{I}_k$ with other disjoint sets $I_l, l \neq k$, i.e. $R_k = \bigcup_{j \neq k} \hat{I}_k \cap I_j = \bigcup_{j \neq k} R_{k,j}$. The set $R_{k,j}$ can then be mapped by $l_j$ to get the local indices wrt. part $j$. The indices here are mapped by $l_j$.

#### 48.136.3.3 get_remote_target_ids()

```
template<typename LocalIndexType, typename GlobalIndexType = int64>
const array<comm_index_type>& gko::experimental::distributed::index_map< LocalIndexType,
GlobalIndexType >::get_remote_target_ids ( ) const
```

get the rank ids which contain indices accessed by this rank.

The order matches the order of the sets in get_remote_global_idxs and get_remote_local_idxs.

#### 48.136.3.4 map_to_global()

```
template<typename LocalIndexType, typename GlobalIndexType = int64>
array<GlobalIndexType> gko::experimental::distributed::index_map< LocalIndexType, Global↩
IndexType >::map_to_global (
            const array< LocalIndexType > & local_idxs,
            index_space index_space_v ) const
```

Maps local indices to global indices.

**Parameters**

| | |
|---|---|
| *local_idxs* | the local indices to map |
| *index_space↩ _v* | the index space in which the passed-in local indices are defined |

**Returns**

the mapped global indices. Any local index that is not in the specified index space is mapped to invalid_index

#### 48.136.3.5 map_to_local()

```
template<typename LocalIndexType, typename GlobalIndexType = int64>
array<LocalIndexType> gko::experimental::distributed::index_map< LocalIndexType, Global↩
IndexType >::map_to_local (
            const array< GlobalIndexType > & global_ids,
            index_space index_space_v ) const
```

Maps global indices to local indices.

**Parameters**

| | |
|---|---|
| *global_ids* | the global indices to map |
| *index_space↩ _v* | the index space in which the returned local indices are defined |

**Returns**

the mapped local indices. Any global index that is not in the specified index space is mapped to invalid_index.

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/index_map.hpp

## 48.137 gko::index_set< IndexType > Class Template Reference

An index set class represents an ordered set of intervals.

```
#include <ginkgo/core/base/index_set.hpp>
```

### Public Types

- using index_type = IndexType

  *The type of elements stored in the index set.*

### Public Member Functions

- index_set (std::shared_ptr< const Executor > exec) noexcept

  *Creates an empty index_set tied to the specified Executor.*
- index_set (std::shared_ptr< const gko::Executor > exec, std::initializer_list< IndexType > init_list, const bool is_sorted=false)

  *Creates an index set on the specified executor from the initializer list.*
- index_set (std::shared_ptr< const gko::Executor > exec, const index_type size, const gko::array< index_type > &indices, const bool is_sorted=false)

  *Creates an index set on the specified executor and the given size.*
- index_set (std::shared_ptr< const Executor > exec, const index_set &other)

  *Creates a copy of the input index_set on a different executor.*
- index_set (const index_set &other)

*Creates a copy of the input index_set.*

- index_set (std::shared_ptr< const Executor > exec, index_set &&other)

  *Moves the input index_set to a different executor.*

- index_set (index_set &&other)

  *Moves the input index_set.*

- index_set & operator= (const index_set &other)

  *Copies data from another index_set.*

- index_set & operator= (index_set &&other)

  *Moves data from another index_set.*

- void clear () noexcept

  *Deallocates all data used by the index_set.*

- std::shared_ptr< const Executor > get_executor () const

  *Returns the executor of the index_set.*

- index_type get_size () const

  *Returns the size of the index set space.*

- bool is_contiguous () const

  *Returns if the index set is contiguous.*

- index_type get_num_elems () const

  *Return the actual number of indices stored in the index set.*

- index_type get_global_index (index_type local_index) const

  *Return the global index given a local index.*

- index_type get_local_index (index_type global_index) const

  *Return the local index given a global index.*

- array< index_type > map_local_to_global (const array< index_type > &local_indices, const bool is_↩ sorted=false) const

  *This is an array version of the scalar function above.*

- array< index_type > map_global_to_local (const array< index_type > &global_indices, const bool is_↩ sorted=false) const

  *This is an array version of the scalar function above.*

- array< index_type > to_global_indices () const

  *This function allows the user obtain a decompressed global_indices array from the indices stored in the index set.*

- array< bool > contains (const array< index_type > &global_indices, const bool is_sorted=false) const

  *Checks if the individual global indeices exist in the index set.*

- bool contains (const index_type global_index) const

  *Checks if the global index exists in the index set.*

- index_type get_num_subsets () const

  *Returns the number of subsets stored in the index set.*

- const index_type * get_subsets_begin () const

  *Returns a pointer to the beginning indices of the subsets.*

- const index_type * get_subsets_end () const

  *Returns a pointer to the end indices of the subsets.*

- const index_type * get_superset_indices () const

  *Returns a pointer to the cumulative indices of the superset of the subsets.*

## 48.137.1 Detailed Description

template<typename IndexType = int32>
class gko::index_set< IndexType >

An index set class represents an ordered set of intervals.

The index set contains subsets which store the starting and end points of a range, [a,b), storing the first index and one past the last index. As the index set only stores the end-points of ranges, it can be quite efficient in terms of storage.

This class is particularly useful in storing continuous ranges. For example, consider the index set (1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 18, 19, 20, 21, 42). Instead of storing the entire array of indices, one can store intervals ([1,9), [10,13), [18,22), [42,43)), thereby only using half the storage.

We store three arrays, one (subsets_begin) with the starting indices of the subsets in the index set, another (subsets_end) storing one index beyond the end indices of the subsets and the last (superset_cumulative_indices) storing the cumulative number of indices in the subsequent subsets with an initial zero which speeds up the querying. Additionally, the arrays containing the range boundaries (subsets_begin, subsets_end) are stored in a sorted fashion.

Therefore the storage would look as follows

index_set = (1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 18, 19, 20, 21, 42) subsets_begin = {1, 10, 18, 42} subsets_end = {9, 13, 22, 43} superset_cumulative_indices = {0, 8, 11, 15, 16}

**Template Parameters**

| | |
|---|---|
| *index_type* | type of the indices being stored in the index set. |

## 48.137.2 Constructor & Destructor Documentation

### 48.137.2.1 index_set() [1/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
            std::shared_ptr< const Executor > exec )  [inline], [explicit], [noexcept]
```

Creates an empty index_set tied to the specified Executor.

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the index_set data is allocated |

```
68         :  exec_(std::move(exec)),
69           index_space_size_{0},
70           num_stored_indices_{0},
71           subsets_begin_{array<index_type>(exec_)},
72           subsets_end_{array<index_type>(exec_)},
```

```
73           superset_cumulative_indices_{array<index_type>(exec_)}
74     {}
```

#### 48.137.2.2 index_set() [2/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
              std::shared_ptr< const gko::Executor > exec,
              std::initializer_list< IndexType > init_list,
              const bool is_sorted = false )  [inline], [explicit]
```

Creates an index set on the specified executor from the initializer list.

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the index set data will be allocated |
| *init_list* | the indices that the index set should hold in an initializer_list. |
| *is_sorted* | a parameter that specifies if the indices array is sorted or not. `true` if sorted. |

```
88         :  exec_(std::move(exec)),
89           index_space_size_(init_list.size() > 0
90                               ?  *(std::max_element(std::begin(init_list),
91                                                     std::end(init_list))) +
92                                     1
93                               :  0),
94           num_stored_indices_{static_cast<IndexType>(init_list.size())}
95     {
96         GKO_ASSERT(index_space_size_ > 0);
97         this->populate_subsets(
98             array<IndexType>(this->get_executor(), init_list), is_sorted);
99     }
```

#### 48.137.2.3 index_set() [3/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
              std::shared_ptr< const gko::Executor > exec,
              const index_type size,
              const gko::array< index_type > & indices,
              const bool is_sorted = false )  [inline], [explicit]
```

Creates an index set on the specified executor and the given size.

**Parameters**

| | |
|---|---|
| *exec* | the Executor where the index set data will be allocated |
| *size* | the maximum index the index set it allowed to hold. This is the size of the index space. |
| *indices* | the indices that the index set should hold. |
| *is_sorted* | a parameter that specifies if the indices array is sorted or not. `true` if sorted. |

```
115         :  exec_(std::move(exec)), index_space_size_(size)
116     {
117         GKO_ASSERT(index_space_size_ >= indices.get_size());
118         this->populate_subsets(indices, is_sorted);
119     }
```

References gko::array< ValueType >::get_size().

### 48.137.2.4 index_set() [4/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
            std::shared_ptr< const Executor > exec,
            const index_set< IndexType > & other )  [inline]
```

Creates a copy of the input index_set on a different executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the new index_set will be created |
| *other* | the index_set to copy from |

```
128         :  index_set(exec)
129     {
130         *this = other;
131     }
```

### 48.137.2.5 index_set() [5/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
            const index_set< IndexType > & other )  [inline]
```

Creates a copy of the input index_set.

**Parameters**

| | |
|---|---|
| *other* | the index_set to copy from |

```
138                                     :  index_set(other.get_executor(), other)
139     {}
```

### 48.137.2.6 index_set() [6/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
            std::shared_ptr< const Executor > exec,
            index_set< IndexType > && other )  [inline]
```

Moves the input index_set to a different executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor where the new index_set will be moved to |
| *other* | the index_set to move from |

```
148         :  index_set(exec)
149     {
150         *this = std::move(other);
151     }
```

#### 48.137.2.7 index_set() [7/7]

```
template<typename IndexType = int32>
gko::index_set< IndexType >::index_set (
             index_set< IndexType > && other )  [inline]
```

Moves the input index_set.

**Parameters**

| | |
|---|---|
| *other* | the index_set to move from |

```
159         :  index_set(other.get_executor(), std::move(other))
160     {}
```

### 48.137.3 Member Function Documentation

#### 48.137.3.1 clear()

```
template<typename IndexType = int32>
void gko::index_set< IndexType >::clear ( )  [inline], [noexcept]
```

Deallocates all data used by the index_set.

The index_set is left in a valid, but empty state, so the same index_set can be used to allocate new memory. Calls to index_set::get_subsets_begin() will return a `nullptr`.

```
219     {
220         this->index_space_size_ = 0;
221         this->num_stored_indices_ = 0;
222         this->subsets_begin_.clear();
223         this->subsets_end_.clear();
224         this->superset_cumulative_indices_.clear();
225     }
```

References gko::array< ValueType >::clear().

#### 48.137.3.2 contains() [1/2]

```
template<typename IndexType = int32>
array<bool> gko::index_set< IndexType >::contains (
             const array< index_type > & global_indices,
             const bool is_sorted = false ) const
```

Checks if the individual global indeices exist in the index set.

**Parameters**

| | |
|---|---|
| *global_indices* | the indices to check. |
| *is_sorted* | a parameter that specifies if the query array is sorted or not. `true` if sorted. |

**Returns**

the array that contains element wise whether the corresponding global index in the index set or not.

### 48.137.3.3 contains() [2/2]

```
template<typename IndexType = int32>
bool gko::index_set< IndexType >::contains (
            const index_type global_index ) const
```

Checks if the global index exists in the index set.

**Parameters**

| | |
|---|---|
| *global_index* | the index to check. |

**Returns**

whether the element exists in the index set.

**Warning**

This single entry query can have significant kernel launch overheads and should be avoided if possible.

### 48.137.3.4 get_executor()

```
template<typename IndexType = int32>
std::shared_ptr<const Executor> gko::index_set< IndexType >::get_executor ( ) const  [inline]
```

Returns the executor of the index_set.

**Returns**

the executor.

```
232 { return this->exec_; }
```

#### 48.137.3.5 get_global_index()

```
template<typename IndexType = int32>
index_type gko::index_set< IndexType >::get_global_index (
            index_type local_index ) const
```

Return the global index given a local index.

Consider the set idx_set = (0, 1, 2, 4, 6, 7, 8, 9). This function returns the element at the global index k stored in the index set. For example, `idx_set.get_global_index(0) == 0` `idx_set.get_global_index(3) == 4` and `idx_set.get_global_index(7) == 9`

**Note**

This function returns a scalar value and needs a scalar value. For repeated queries, it is more efficient to use the array functions that take and return arrays which allow for more throughput.

**Parameters**

| | |
|---|---|
| *local_index* | the local index. |

**Returns**

the global index from the index set.

**Warning**

This single entry query can have significant kernel launch overheads and should be avoided if possible.

#### 48.137.3.6 get_local_index()

```
template<typename IndexType = int32>
index_type gko::index_set< IndexType >::get_local_index (
            index_type global_index ) const
```

Return the local index given a global index.

Consider the set idx_set = (0, 1, 2, 4, 6, 7, 8, 9). This function returns the local index in the index set of the provided index set. For example, `idx_set.get_local_index(0) == 0` `idx_set.get_local↩ _index(4) == 3` and `idx_set.get_local_index(6) == 4`.

**Note**

This function returns a scalar value and needs a scalar value. For repeated queries, it is more efficient to use the array functions that take and return arrays which allow for more throughput.

**Parameters**

| | |
|---|---|
| *global_index* | the global index. |

**Returns**

the local index of the element in the index set.

**Warning**

This single entry query can have significant kernel launch overheads and should be avoided if possible.

### 48.137.3.7 get_num_elems()

```
template<typename IndexType = int32>
index_type gko::index_set< IndexType >::get_num_elems ( ) const  [inline]
```

Return the actual number of indices stored in the index set.

**Returns**

number of indices stored in the index set

```
253 { return this->num_stored_indices_; };
```

### 48.137.3.8 get_num_subsets()

```
template<typename IndexType = int32>
index_type gko::index_set< IndexType >::get_num_subsets ( ) const  [inline]
```

Returns the number of subsets stored in the index set.

**Returns**

the number of stored subsets.

```
371     {
372         return this->subsets_begin_.get_size();
373     }
```

References gko::array< ValueType >::get_size().

Referenced by gko::index_set< IndexType >::is_contiguous().

### 48.137.3.9 get_size()

```
template<typename IndexType = int32>
index_type gko::index_set< IndexType >::get_size ( ) const  [inline]
```

Returns the size of the index set space.

**Returns**

the size of the index set space.

```
239 { return this->index_space_size_; }
```

#### 48.137.3.10 get_subsets_begin()

```
template<typename IndexType = int32>
const index_type* gko::index_set< IndexType >::get_subsets_begin ( ) const  [inline]
```

Returns a pointer to the beginning indices of the subsets.

**Returns**

a pointer to the beginning indices of the subsets.

```
381     {
382         return this->subsets_begin_.get_const_data();
383     }
```

References gko::array< ValueType >::get_const_data().

#### 48.137.3.11 get_subsets_end()

```
template<typename IndexType = int32>
const index_type* gko::index_set< IndexType >::get_subsets_end ( ) const  [inline]
```

Returns a pointer to the end indices of the subsets.

**Returns**

a pointer to the end indices of the subsets.

```
391     {
392         return this->subsets_end_.get_const_data();
393     }
```

References gko::array< ValueType >::get_const_data().

#### 48.137.3.12 get_superset_indices()

```
template<typename IndexType = int32>
const index_type* gko::index_set< IndexType >::get_superset_indices ( ) const  [inline]
```

Returns a pointer to the cumulative indices of the superset of the subsets.

**Returns**

a pointer to the cumulative indices of the superset of the subsets.

```
403     {
404         return this->superset_cumulative_indices_.get_const_data();
405     }
```

References gko::array< ValueType >::get_const_data().

#### 48.137.3.13 is_contiguous()

```
template<typename IndexType = int32>
bool gko::index_set< IndexType >::is_contiguous ( ) const  [inline]
```

Returns if the index set is contiguous.

**Returns**

if the index set is contiguous.

```
246 { return (this->get_num_subsets() <= 1); }
```

References gko::index_set< IndexType >::get_num_subsets().

#### 48.137.3.14 map_global_to_local()

```
template<typename IndexType = int32>
array<index_type> gko::index_set< IndexType >::map_global_to_local (
            const array< index_type > & global_indices,
            const bool is_sorted = false ) const
```

This is an array version of the scalar function above.

**Parameters**

| | |
|---|---|
| *global_indices* | the global index array. |
| *is_sorted* | a parameter that specifies if the query array is sorted or not. `true` if sorted. |

**Returns**

the local index array from the index set.

**Note**

Whenever possible, passing a sorted array is preferred as the queries can be significantly faster.

#### 48.137.3.15 map_local_to_global()

```
template<typename IndexType = int32>
array<index_type> gko::index_set< IndexType >::map_local_to_global (
            const array< index_type > & local_indices,
            const bool is_sorted = false ) const
```

This is an array version of the scalar function above.

**Parameters**

| | |
|---|---|
| *local_indices* | the local index array. |
| *is_sorted* | a parameter that specifies if the query array is sorted or not. `true` if sorted . |

**Returns**

the global index array from the index set.

**Note**

Whenever possible, passing a sorted array is preferred as the queries can be significantly faster.

Passing local indices from [0, size) is equivalent to using the @to_global_indices function.

### 48.137.3.16 operator=() [1/2]

```
template<typename IndexType = int32>
index_set& gko::index_set< IndexType >::operator= (
            const index_set< IndexType > & other )  [inline]
```

Copies data from another index_set.

The executor of this is preserved. In case this does not have an assigned executor, it will inherit the executor of other.

**Parameters**

| | |
|---|---|
| *other* | the index_set to copy from |

**Returns**

this

```
173     {
174         if (&other == this) {
175             return *this;
176         }
177         this->index_space_size_ = other.index_space_size_;
178         this->num_stored_indices_ = other.num_stored_indices_;
179         this->subsets_begin_ = other.subsets_begin_;
180         this->subsets_end_ = other.subsets_end_;
181         this->superset_cumulative_indices_ = other.superset_cumulative_indices_;
182
183         return *this;
184     }
```

### 48.137.3.17 operator=() [2/2]

```
template<typename IndexType = int32>
index_set& gko::index_set< IndexType >::operator= (
            index_set< IndexType > && other )  [inline]
```

Moves data from another index_set.

The executor of this is preserved. In case this does not have an assigned executor, it will inherit the executor of other.

**Parameters**

| | |
|---|---|
| *other* | the index_set to move from |

**Returns**

this

```
197     {
198         if (&other == this) {
199             return *this;
200         }
201         this->index_space_size_ = std::exchange(other.index_space_size_, 0);
202         this->num_stored_indices_ = std::exchange(other.num_stored_indices_, 0);
203         this->subsets_begin_ = std::move(other.subsets_begin_);
204         this->subsets_end_ = std::move(other.subsets_end_);
205         this->superset_cumulative_indices_ =
206             std::move(other.superset_cumulative_indices_);
207
208         return *this;
209     }
```

#### 48.137.3.18 to_global_indices()

```
template<typename IndexType = int32>
array<index_type> gko::index_set< IndexType >::to_global_indices ( ) const
```

This function allows the user obtain a decompressed global_indices array from the indices stored in the index set.

**Returns**

the decompressed set of indices.

The documentation for this class was generated from the following file:

- ginkgo/core/base/index_set.hpp

## 48.138 gko::InvalidData Class Reference

InvalidData is thrown in case an object in Ginkgo contains invalid data, either because the data was user-provided, or because of a bug.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- InvalidData (const std::string &file, int line, const std::type_info &type, const std::string &invariant)

  *Initializes an InvalidData error.*

### 48.138.1 Detailed Description

InvalidData is thrown in case an object in Ginkgo contains invalid data, either because the data was user-provided, or because of a bug.

Invalid data is described by the invariant it violated.

### 48.138.2 Constructor & Destructor Documentation

#### 48.138.2.1 InvalidData()

```
gko::InvalidData::InvalidData (
            const std::string & file,
            int line,
            const std::type_info & type,
            const std::string & invariant )  [inline]
```

Initializes an InvalidData error.

**Parameters**

| | |
|---|---|
| *file* | The name of the source file containing the check |
| *line* | The source code line number where the check failed |
| *type* | The object type on which the requested operation failed. |
| *invariant* | The invariant that was violated by the invalid data. |

```
685         :  Error(file, line,
686                  "Invariant violated in " +
687                      name_demangling::get_type_name(type) + ":  " + invariant)
688     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.139 gko::InvalidStateError Class Reference

Exception thrown if an object is in an invalid state.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- InvalidStateError (const std::string &file, int line, const std::string &func, const std::string &clarification)

  *Initializes an invalid state error.*

### 48.139.1 Detailed Description

Exception thrown if an object is in an invalid state.

### 48.139.2 Constructor & Destructor Documentation

#### 48.139.2.1 InvalidStateError()

```
gko::InvalidStateError::InvalidStateError (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & clarification )  [inline]
```

Initializes an invalid state error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The function name where the error occurred |
| *clarification* | A message describing the invalid state |

```
662         :  Error(file, line,
663                  func + ":  Invalid state encountered :  " + clarification)
664     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.140 gko::solver::Ir< ValueType > Class Template Reference

Iterative refinement (IR) is an iterative method that uses another coarse method to approximate the error of the current solution via the current residual.

```
#include <ginkgo/core/solver/ir.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

- std::shared_ptr< const LinOp > get_solver () const

  *Returns the solver operator used as the inner solver.*
- void set_solver (std::shared_ptr< const LinOp > new_solver)

  *Sets the solver operator used as the inner solver.*
- Ir & operator= (const Ir &)

  *Copy-assigns an IR solver.*
- Ir & operator= (Ir &&)

  *Move-assigns an IR solver.*
- Ir (const Ir &)

  *Copy-constructs an IR solver.*
- Ir (Ir &&)

  *Move-constructs an IR solver.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.140.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Ir< ValueType >**

Iterative refinement (IR) is an iterative method that uses another coarse method to approximate the error of the current solution via the current residual.

Moreover, it can be also considered as preconditioned Richardson iteration with relaxation factor = 1.

Let $x_k$ be the approximation of the solution of $Ax = b$ after $k$ iterations and let $x$ denote the exact solution. The residual and the error of $x_k$ are

$$r_k = b - Ax_k, \qquad e_k = x - x_k,$$

and they are linked by the residual equation $Ae_k = Ax - Ax_k = b - Ax_k = r_k$. Knowing $e_k$ exactly would give the exact solution in a single update $x = x_k + e_k$. Instead of solving $Ae_k = r_k$ exactly, IR approximates $e_k$ by $\tilde{e}_k$ using a cheap inner `solver`, applies that correction with a relaxation factor $\alpha$,

$$x_{k+1} = x_k + \alpha\tilde{e}_k,$$

and repeats the process with the updated iterate. Written with the names used below, $x_k$ is `solution`, $r_k$ is `residual` and $\tilde{e}_k$ is `error`:

```
solution = initial_guess
while not converged:
    residual = b - A solution
    error = solver(A, residual)
    solution = solution + relaxation_factor * error
```

With `relaxation_factor` equal to 1 (default), the solver is Iterative Refinement, with `relaxation_factor` equal to a value other than `1`, the solver is a Richardson iteration, with possibility for additional preconditioning.

Assume $\alpha = 1$ and that `solver` has accuracy $c$, i.e. $\|e_k - \tilde{e}_k\| \leq c\|e_k\|$. Then iterative refinement converges with a convergence rate of $c$: from $e_k - \tilde{e}_k = (x - x_k) - \tilde{e}_k = x - x_{k+1}$ it follows that $\|x - x_{k+1}\| \leq c\|x - x_k\|$.

Unless otherwise specified via the `solver` factory parameter, this implementation uses the identity operator (i.e. the solver that approximates the solution of a system $Ax = b$ by setting $x := b$) as the default inner solver. It leaves the residual unchanged, $\tilde{e}_k = r_k$, so the iteration reduces to the Richardson iteration $x_{k+1} = x_k + \alpha r_k$, which converges for every initial guess if and only if $|1 - \alpha\lambda| < 1$ holds for every eigenvalue $\lambda$ of $A$.

**References**

- Saad, Y. *Iterative Methods for Sparse Linear Systems.* 2nd ed. SIAM, 2003. https://doi.org/10.1137/1.9780898718003

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.140.2 Constructor & Destructor Documentation

### 48.140.2.1 Ir() [1/2]

```
template<typename ValueType = default_precision>
gko::solver::Ir< ValueType >::Ir (
            const Ir< ValueType > &  )
```

Copy-constructs an IR solver.

Inherits the executor, shallow-copies inner solver, stopping criterion and system matrix.

### 48.140.2.2 Ir() [2/2]

```
template<typename ValueType = default_precision>
gko::solver::Ir< ValueType >::Ir (
            Ir< ValueType > &&  )
```

Move-constructs an IR solver.

Preserves the executor, moves inner solver, stopping criterion and system matrix. The moved-from object is empty (0x0 and nullptr inner solver, stopping criterion and system matrix)

## 48.140.3 Member Function Documentation

### 48.140.3.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::Ir< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
114     {
115         return this->get_default_initial_guess() ==
116                initial_guess_mode::provided;
117     }
```

References gko::solver::provided.

#### 48.140.3.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Ir< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.140.3.3 get_solver()

```
template<typename ValueType = default_precision>
std::shared_ptr<const LinOp> gko::solver::Ir< ValueType >::get_solver ( ) const  [inline]
```

Returns the solver operator used as the inner solver.

**Returns**

the solver operator used as the inner solver

```
124 { return solver_; }
```

#### 48.140.3.4 operator=() [1/2]

```
template<typename ValueType = default_precision>
Ir& gko::solver::Ir< ValueType >::operator= (
            const Ir< ValueType > &  )
```

Copy-assigns an IR solver.

Preserves the executor, shallow-copies inner solver, stopping criterion and system matrix. If the executors mismatch, clones inner solver, stopping criterion and system matrix onto this executor.

#### 48.140.3.5 operator=() [2/2]

```
template<typename ValueType = default_precision>
Ir& gko::solver::Ir< ValueType >::operator= (
            Ir< ValueType > &&  )
```

Move-assigns an IR solver.

Preserves the executor, moves inner solver, stopping criterion and system matrix. If the executors mismatch, clones inner solver, stopping criterion and system matrix onto this executor. The moved-from object is empty (0x0 and nullptr inner solver, stopping criterion and system matrix)

#### 48.140.3.6 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Ir< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.140.3.7 set_solver()

```
template<typename ValueType = default_precision>
void gko::solver::Ir< ValueType >::set_solver (
            std::shared_ptr< const LinOp > new_solver )
```

Sets the solver operator used as the inner solver.

**Parameters**

| | |
|---|---|
| *new_solver* | the new inner solver |

#### 48.140.3.8 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Ir< ValueType >::transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/ir.hpp

## 48.141 gko::preconditioner::Isai< IsaiType, ValueType, IndexType > Class Template Reference

The Incomplete Sparse Approximate Inverse (ISAI) Preconditioner generates an approximate inverse matrix for a given square matrix $A$, lower triangular matrix $L$, upper triangular matrix $U$ or symmetric positive definite (s.p.d) matrix $B$.

`#include <ginkgo/core/preconditioner/isai.hpp>`

### Public Member Functions

- std::shared_ptr< const typename std::conditional< IsaiType==isai_type::spd, Comp, Csr >::type > get_approximate_inverse () const

  *Returns the approximate inverse of the given matrix (either a CSR matrix for IsaiType general, upper or lower or a composition of two CSR matrices for IsaiType spd).*
- Isai & operator= (const Isai &other)

  *Copy-assigns an ISAI preconditioner.*
- Isai & operator= (Isai &&other)

  *Move-assigns an ISAI preconditioner.*
- Isai (const Isai &other)

  *Copy-constructs an ISAI preconditioner.*
- Isai (Isai &&other)

  *Move-constructs an ISAI preconditioner.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

### 48.141.1 Detailed Description

template<isai_type IsaiType, typename ValueType, typename IndexType>
class gko::preconditioner::Isai< IsaiType, ValueType, IndexType >

The Incomplete Sparse Approximate Inverse (ISAI) Preconditioner generates an approximate inverse matrix for a given square matrix $A$, lower triangular matrix $L$, upper triangular matrix $U$ or symmetric positive definite (s.p.d) matrix $B$.

Let $M_A \approx A^{-1}$ denote the approximate inverse of a general matrix $A$, and similarly $M_L \approx L^{-1}$, $M_U \approx U^{-1}$ for lower and upper triangular matrices, and $M_C \approx C^{-1}$ for the Cholesky factor of an s.p.d matrix $B = C^T C$ (the last case is commonly called a Factorized Sparse Approximate Inverse, FSPAI). For a given vector $x$ (which may carry multiple right-hand sides), applying the preconditioner computes

$$\begin{aligned} \texttt{isai_type::general}: &\quad M_A x, \\ \texttt{isai_type::lower}: &\quad M_L x, \\ \texttt{isai_type::upper}: &\quad M_U x, \\ \texttt{isai_type::spd}: &\quad M_C^T M_C x. \end{aligned}$$

The sparsity pattern used for the approximate inverse of $A$, $L$ and $U$ is the same as the sparsity pattern of the respective matrix. For $B$, the sparsity pattern of $M_C$ matches the sparsity pattern of the lower triangular half of $B$.

Note that, except for the s.p.d case, in general $\mathrm{ISAI}(A)^T \neq \mathrm{ISAI}(A^T)$.

#### Note

GPU implementations can only handle the vector unit width `width` (warp size for CUDA) as number of elements per row in the sparse matrix. If there are more than `width` elements per row, the remaining elements will be ignored.

#### References

- Anzt, H., Huckle, T. K., Bräckle, J., Dongarra, J. *Incomplete Sparse Approximate Inverses for Parallel Preconditioning.* Parallel Computing, 71, 1–22, 2018. https://doi.org/10.1016/j.parco.2017.10.003

#### Template Parameters

| | |
|---|---|
| *IsaiType* | determines if the ISAI is generated for a general square matrix, a lower triangular matrix, an upper triangular matrix or an s.p.d matrix |
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

### 48.141.2 Constructor & Destructor Documentation

#### 48.141.2.1 Isai() [1/2]

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::Isai (
            const Isai< IsaiType, ValueType, IndexType > & other )
```

Copy-constructs an ISAI preconditioner.

Inherits the executor, shallow-copies the matrix and parameters.

#### 48.141.2.2 Isai() [2/2]

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::Isai (
            Isai< IsaiType, ValueType, IndexType > && other )
```

Move-constructs an ISAI preconditioner.

Inherits the executor, moves the matrix and parameters. The moved-from object is empty (0x0 with nullptr matrix and default parameters)

### 48.141.3 Member Function Documentation

#### 48.141.3.1 conj_transpose()

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
std::unique_ptr<LinOp> gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::conj_↩
transpose ( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.141.3.2 get_approximate_inverse()

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
std::shared_ptr<const typename std::conditional<IsaiType == isai_type::spd, Comp, Csr>::type>
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::get_approximate_inverse ( ) const
[inline]
```

Returns the approximate inverse of the given matrix (either a CSR matrix for IsaiType general, upper or lower or a composition of two CSR matrices for IsaiType spd).

**Returns**

the generated approximate inverse

```
120     {
121         return as<typename std::conditional<IsaiType == isai_type::spd, Comp,
122                                             Csr>::type>(approximate_inverse_);
123     }
```

References gko::as().

#### 48.141.3.3 operator=() [1/2]

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
Isai& gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::operator= (
            const Isai< IsaiType, ValueType, IndexType > & other )
```

Copy-assigns an ISAI preconditioner.

Preserves the executor, shallow-copies the matrix and parameters. Creates a clone of the matrix if it is on the wrong executor.

#### 48.141.3.4 operator=() [2/2]

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
Isai& gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::operator= (
            Isai< IsaiType, ValueType, IndexType > && other )
```

Move-assigns an ISAI preconditioner.

Preserves the executor, moves the matrix and parameters. Creates a clone of the matrix if it is on the wrong executor. The moved-from object is empty (0x0 with nullptr matrix and default parameters)

#### 48.141.3.5 parse()

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
static parameters_type gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

#### 48.141.3.6 transpose()

```
template<isai_type IsaiType, typename ValueType , typename IndexType >
```

`std::unique_ptr<LinOp> gko::preconditioner::Isai< IsaiType, ValueType, IndexType >::transpose ( ) const [override], [virtual]`

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/isai.hpp

## 48.142 gko::stop::Iteration Class Reference

The Iteration class is a stopping criterion which stops the iteration process after a preset number of iterations.

`#include <ginkgo/core/stop/iteration.hpp>`

### Additional Inherited Members

### 48.142.1 Detailed Description

The Iteration class is a stopping criterion which stops the iteration process after a preset number of iterations.

**Note**

to use this stopping criterion, it is required to update the iteration count for the ::check() method.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/iteration.hpp

## 48.143 gko::log::iteration_complete_data Struct Reference

Struct representing iteration complete related data.

`#include <ginkgo/core/log/record.hpp>`

### 48.143.1 Detailed Description

Struct representing iteration complete related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

# 48.144 gko::solver::IterativeBase Class Reference

A LinOp implementing this interface stores a stopping criterion factory.

`#include <ginkgo/core/solver/solver_base.hpp>`

## Public Member Functions

- std::shared_ptr< const stop::CriterionFactory > get_stop_criterion_factory () const
  *Gets the stopping criterion factory of the solver.*
- virtual void set_stop_criterion_factory (std::shared_ptr< const stop::CriterionFactory > new_stop_factory)
  *Sets the stopping criterion of the solver.*

## 48.144.1 Detailed Description

A LinOp implementing this interface stores a stopping criterion factory.

## 48.144.2 Member Function Documentation

### 48.144.2.1 get_stop_criterion_factory()

```
std::shared_ptr<const stop::CriterionFactory> gko::solver::IterativeBase::get_stop_criterion↩
_factory ( ) const  [inline]
```

Gets the stopping criterion factory of the solver.

**Returns**

the stopping criterion factory

```
699     {
700         return stop_factory_;
701     }
```

Referenced by gko::solver::EnableIterativeBase< Chebyshev< ValueType > >::operator=().

### 48.144.2.2 set_stop_criterion_factory()

```
virtual void gko::solver::IterativeBase::set_stop_criterion_factory (
            std::shared_ptr< const stop::CriterionFactory > new_stop_factory )  [inline],
[virtual]
```

Sets the stopping criterion of the solver.

**Parameters**

| | |
|---|---|
| *new_stop_factory* | the new stopping criterion factory |

Reimplemented in gko::solver::EnableIterativeBase< DerivedType >, gko::solver::EnableIterativeBase< PipeCg< ValueType > >, gko::solver::EnableIterativeBase< Bicgstab< ValueType > >, gko::solver::EnableIterativeBase< Multigrid >, gko::solver::EnableIterativeBase< Gmres< ValueType > >, gko::solver::EnableIterativeBase< CbGmres< ValueType > >, gko::solver::EnableIterativeBase< Ir< ValueType > >, gko::solver::EnableIterativeBase< Fcg< ValueType > >, gko::solver::EnableIterativeBase< Minres< ValueType > >, gko::solver::EnableIterativeBase< Idr< ValueType > >, gko::solver::EnableIterativeBase< Cgs< ValueType > >, gko::solver::EnableIterativeBase< Gcr< ValueType > >, gko::solver::EnableIterativeBase< Cg< ValueType > >, gko::solver::EnableIterativeBase< Bicg< ValueType > >, and gko::solver::EnableIterativeBase< Chebyshev< ValueType > >.

```
710     {
711         stop_factory_ = new_stop_factory;
712     }
```

Referenced by gko::solver::EnableIterativeBase< Chebyshev< ValueType > >::set_stop_criterion_factory().

The documentation for this class was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

## 48.145 gko::batch::preconditioner::Jacobi< ValueType, IndexType > Class Template Reference

A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks (stored in a dense row major fashion) of the source operator.

```
#include <ginkgo/core/preconditioner/batch_jacobi.hpp>
```

### Public Member Functions

- const index_type * get_const_block_pointers () const noexcept

  *Returns the block pointers.*
- const index_type * get_const_map_block_to_row () const noexcept

  *Returns the mapping between the blocks and the row id.*
- const index_type * get_const_blocks_cumulative_offsets () const noexcept

  *Returns the cumulative blocks storage array.*
- uint32 get_max_block_size () const noexcept

  *Returns the max block size.*
- size_type get_num_blocks () const noexcept

  *Returns the number of blocks in an individual batch entry.*
- const value_type * get_const_blocks () const noexcept

  *Returns the pointer to the memory used for storing the block data.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the dense blocks.*

## 48.145.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::batch::preconditioner::Jacobi< ValueType, IndexType >

A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks (stored in a dense row major fashion) of the source operator.

With the batched preconditioners, it is required that all items in the batch have the same sparsity pattern. The detection of the blocks and the block pointers require that the sparsity pattern of all the items be the same. Other cases is undefined behaviour. The input batch matrix must be in batch::Csr matrix format or must be convertible to batch::Csr matrix format. The block detection algorithm and the conversion to dense blocks kernels require this assumption.

**Note**

In a fashion similar to the non-batched Jacobi preconditioner, the maximum possible size of the diagonal blocks is equal to the maximum warp size on the device (32 for NVIDIA GPUs, 64 for AMD GPUs).

**Template Parameters**

| | |
|---|---|
| *ValueType* | value precision of matrix elements |
| *IndexType* | index precision of matrix elements |

## 48.145.2 Member Function Documentation

### 48.145.2.1 get_const_block_pointers()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_↩
block_pointers ( ) const  [inline], [noexcept]
```

Returns the block pointers.

**Note**

Returns nullptr in case of a scalar jacobi preconditioner (max_block_size = 1).

**Returns**

the block pointers

```
66     {
67         return block_pointers_.get_const_data();
68     }
```

References gko::array< ValueType >::get_const_data().

#### 48.145.2.2 get_const_blocks()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_blocks
( ) const  [inline], [noexcept]
```

Returns the pointer to the memory used for storing the block data.

Element $(i, j)$ of the block that belongs to batch entry `batch_id` and has local id `block_id` within that entry is stored at the address

```
get_const_blocks()
    + detail::get_global_block_offset(batch_id, num_blocks, block_id,
                                      cumulative_blocks_storage)
    + i * detail::get_stride(block_id, block_pointers)
    + j
```

**Note**

Returns nullptr in case of a scalar jacobi preconditioner (max_block_size = 1). The blocks array is empty in case of scalar jacobi preconditioner as the preconditioner is generated inside the batched solver kernel.

**Returns**

the pointer to the memory used for storing the block data

```
133     {
134         return blocks_.get_const_data();
135     }
```

References gko::array< ValueType >::get_const_data().

#### 48.145.2.3 get_const_blocks_cumulative_offsets()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_↩
blocks_cumulative_offsets ( ) const  [inline], [noexcept]
```

Returns the cumulative blocks storage array.

**Note**

Returns nullptr in case of a scalar jacobi preconditioner (max_block_size = 1).

```
88      {
89          return blocks_cumulative_offsets_.get_const_data();
90      }
```

References gko::array< ValueType >::get_const_data().

### 48.145.2.4 get_const_map_block_to_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_const_map_↩
block_to_row ( ) const  [inline], [noexcept]
```

Returns the mapping between the blocks and the row id.

#### Note

Returns nullptr in case of a scalar jacobi preconditioner (max_block_size = 1).

```
77     {
78         return map_block_to_row_.get_const_data();
79     }
```

References gko::array< ValueType >::get_const_data().

### 48.145.2.5 get_max_block_size()

```
template<typename ValueType = default_precision, typename IndexType = int32>
uint32 gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_max_block_size ( )
const  [inline], [noexcept]
```

Returns the max block size.

#### Returns

the max block size

```
98     {
99         return parameters_.max_block_size;
100    }
```

### 48.145.2.6 get_num_blocks()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_num_blocks ( ) const
[inline], [noexcept]
```

Returns the number of blocks in an individual batch entry.

#### Returns

the number of blocks in an individual batch entry.

```
107 { return num_blocks_; }
```

#### 48.145.2.7 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::batch::preconditioner::Jacobi< ValueType, IndexType >::get_num_stored_elements
( ) const  [inline], [noexcept]
```

Returns the number of elements explicitly stored in the dense blocks.

**Note**

Returns 0 in case of scalar jacobi preconditioner as the preconditioner is generated inside the batched solver kernels, hence, blocks array storage is not required.

**Returns**

the number of elements explicitly stored in the dense blocks.

```
147     {
148         return blocks_.get_size();
149     }
```

References gko::array< ValueType >::get_size().

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/batch_jacobi.hpp

## 48.146 gko::preconditioner::Jacobi< ValueType, IndexType > Class Template Reference

A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks of the source operator.

```
#include <ginkgo/core/preconditioner/jacobi.hpp>
```

### Public Member Functions

- size_type get_num_blocks () const noexcept

  *Returns the number of blocks of the operator.*
- const block_interleaved_storage_scheme< index_type > & get_storage_scheme () const noexcept

  *Returns the storage scheme used for storing Jacobi blocks.*
- const value_type * get_blocks () const noexcept

  *Returns the pointer to the memory used for storing the block data.*
- const remove_complex< value_type > * get_conditioning () const noexcept

  *Returns an array of 1-norm condition numbers of the blocks.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*
- void convert_to (matrix::Dense< value_type > *result) const override

  *Converts the implementer to an object of type result_type.*
- void move_to (matrix::Dense< value_type > *result) override

  *Converts the implementer to an object of type result_type by moving data from this object.*

- void write (mat_data &data) const override

  *Writes a matrix to a matrix_data structure.*
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- Jacobi & operator= (const Jacobi &other)

  *Copy-assigns a Jacobi preconditioner.*
- Jacobi & operator= (Jacobi &&other)

  *Move-assigns a Jacobi preconditioner.*
- Jacobi (const Jacobi &other)

  *Copy-constructs a Jacobi preconditioner.*
- Jacobi (Jacobi &&other)

  *Move-assigns a Jacobi preconditioner.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## 48.146.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::preconditioner::Jacobi< ValueType, IndexType >**

A block-Jacobi preconditioner is a block-diagonal linear operator, obtained by inverting the diagonal blocks of the source operator.

The Jacobi class implements the inversion of the diagonal blocks using Gauss-Jordan elimination with column pivoting, and stores the inverse explicitly in a customized format.

If the diagonal blocks of the matrix are not explicitly set by the user, the implementation will try to automatically detect the blocks by first finding the natural blocks of the matrix, and then applying the supervariable agglomeration procedure on them. However, if problem-specific knowledge regarding the block diagonal structure is available, it is usually beneficial to explicitly pass the starting rows of the diagonal blocks, as the block detection is merely a heuristic and cannot perfectly detect the diagonal block structure. The current implementation supports blocks of up to 32 rows / columns.

The implementation also includes an improved, adaptive version of the block-Jacobi preconditioner, which can store some of the blocks in lower precision and thus improve the performance of preconditioner application by reducing the amount of memory transfers. This variant can be enabled by setting the Jacobi::Factory's `storage_optimization` parameter. Refer to the documentation of the parameter for more details.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | integral type used to store pointers to the start of each block |

**Note**

The current implementation supports blocks of up to 32 rows / columns.

When using the adaptive variant, there may be a trade-off in terms of slightly longer preconditioner generation due to extra work required to detect the optimal precision of the blocks.

When the max_block_size is set to 1, specialized kernels are used, both for generation (inverting the diagonals) and application (diagonal scaling) to reduce the overhead involved in the usual (adaptive) block case.

## 48.146.2 Constructor & Destructor Documentation

### 48.146.2.1 Jacobi() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::preconditioner::Jacobi< ValueType, IndexType >::Jacobi (
            const Jacobi< ValueType, IndexType > & other )
```

Copy-constructs a Jacobi preconditioner.

Inherits executor, copies all data and parameters.

### 48.146.2.2 Jacobi() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::preconditioner::Jacobi< ValueType, IndexType >::Jacobi (
            Jacobi< ValueType, IndexType > && other )
```

Move-assigns a Jacobi preconditioner.

Inherits executor, moves all data and parameters. The moved-from object will be empty (0x0 and default parameters).

## 48.146.3 Member Function Documentation

### 48.146.3.1 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::preconditioner::Jacobi< ValueType, IndexType >::conj_transpose (
) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.146.3.2 convert_to()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::preconditioner::Jacobi< ValueType, IndexType >::convert_to (
            matrix::Dense< value_type > * result ) const  [override], [virtual]
```

Converts the implementer to an object of type result_type.

**Parameters**

| | |
|---|---|
| *result* | the object used to store the result of the conversion |

Implements gko::ConvertibleTo< matrix::Dense< ValueType > >.

### 48.146.3.3 get_blocks()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::preconditioner::Jacobi< ValueType, IndexType >::get_blocks ( ) const
[inline], [noexcept]
```

Returns the pointer to the memory used for storing the block data.

Element (`i`, `j`) of block `b` is stored in position `(get_block_pointers()[b] + i) * stride + j` of the array.

**Returns**

the pointer to the memory used for storing the block data

```
239     {
240         return blocks_.get_const_data();
241     }
```

References gko::array< ValueType >::get_const_data().

### 48.146.3.4 get_conditioning()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const remove_complex<value_type>* gko::preconditioner::Jacobi< ValueType, IndexType >::get_↩
conditioning ( ) const  [inline], [noexcept]
```

Returns an array of 1-norm condition numbers of the blocks.

**Returns**

an array of 1-norm condition numbers of the blocks

**Note**

This value is valid only if adaptive precision variant is used, and implementations of the standard non-adaptive variant are allowed to omit the calculation of condition numbers.

```
253     {
254         return conditioning_.get_const_data();
255     }
```

References gko::array< ValueType >::get_const_data().

#### 48.146.3.5 get_num_blocks()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::preconditioner::Jacobi< ValueType, IndexType >::get_num_blocks ( ) const  [inline],
[noexcept]
```

Returns the number of blocks of the operator.

**Returns**

the number of blocks of the operator

```
211 { return num_blocks_; }
```

#### 48.146.3.6 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::preconditioner::Jacobi< ValueType, IndexType >::get_num_stored_elements ( )
const  [inline], [noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
263     {
264         return blocks_.get_size();
265     }
```

References gko::array< ValueType >::get_size().

#### 48.146.3.7 get_storage_scheme()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const block_interleaved_storage_scheme<index_type>& gko::preconditioner::Jacobi< ValueType,
IndexType >::get_storage_scheme ( ) const  [inline], [noexcept]
```

Returns the storage scheme used for storing Jacobi blocks.

**Returns**

the storage scheme used for storing Jacobi blocks

```
223     {
224         return storage_scheme_;
225     }
```

#### 48.146.3.8 move_to()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::preconditioner::Jacobi< ValueType, IndexType >::move_to (
            matrix::Dense< value_type > * result )  [override], [virtual]
```

Converts the implementer to an object of type result_type by moving data from this object.

This method is used when the implementer is a temporary object, and move semantics can be used.

**Parameters**

| | |
|---|---|
| *result* | the object used to emplace the result of the conversion |

**Note**

ConvertibleTo::move_to can be implemented by simply calling ConvertibleTo::convert_to. However, this operation can often be optimized by exploiting the fact that implementer's data can be moved to the result.

Implements gko::ConvertibleTo< matrix::Dense< ValueType > >.

#### 48.146.3.9 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Jacobi& gko::preconditioner::Jacobi< ValueType, IndexType >::operator= (
            const Jacobi< ValueType, IndexType > & other )
```

Copy-assigns a Jacobi preconditioner.

Preserves executor, copies all data and parameters.

#### 48.146.3.10 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Jacobi& gko::preconditioner::Jacobi< ValueType, IndexType >::operator= (
            Jacobi< ValueType, IndexType > && other )
```

Move-assigns a Jacobi preconditioner.

Preserves executor, moves all data and parameters. The moved-from object will be empty (0x0 and default parameters).

#### 48.146.3.11 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::preconditioner::Jacobi< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

**Note**

Jacobi does not support block_pointers and storage_optimization array.

### 48.146.3.12 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::preconditioner::Jacobi< ValueType, IndexType >::transpose ( )
const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

### 48.146.3.13 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::preconditioner::Jacobi< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/jacobi.hpp

# 48.147 gko::KernelNotFound Class Reference

KernelNotFound is thrown if Ginkgo cannot find a kernel which satisfies the criteria imposed by the input arguments.

`#include <ginkgo/core/base/exception.hpp>`

## Public Member Functions

- KernelNotFound (const std::string &file, int line, const std::string &func)

  *Initializes a KernelNotFound error.*

## 48.147.1 Detailed Description

KernelNotFound is thrown if Ginkgo cannot find a kernel which satisfies the criteria imposed by the input arguments.

## 48.147.2 Constructor & Destructor Documentation

### 48.147.2.1 KernelNotFound()

```
gko::KernelNotFound::KernelNotFound (
            const std::string & file,
            int line,
            const std::string & func )  [inline]
```

Initializes a KernelNotFound error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the function where the error occurred |

```
622         :  Error(file, line, func + ":  unable to find an eligible kernel")
623     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

# 48.148 gko::log::linop_data Struct Reference

Struct representing LinOp related data.

`#include <ginkgo/core/log/record.hpp>`

### 48.148.1 Detailed Description

Struct representing LinOp related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.149 gko::log::linop_factory_data Struct Reference

Struct representing LinOp factory related data.

`#include <ginkgo/core/log/record.hpp>`

### 48.149.1 Detailed Description

Struct representing LinOp factory related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.150 gko::LinOpFactory Class Reference

A LinOpFactory represents a higher order mapping which transforms one linear operator into another.

`#include <ginkgo/core/base/lin_op.hpp>`

### Classes

- class ReuseData

  *Opaque base for the data a factory records in generate_reuse() to speed up later generations.*

### Public Member Functions

- virtual std::unique_ptr< ReuseData > create_empty_reuse_data () const

  *Creates an empty ReuseData.*
- std::unique_ptr< LinOp > generate_reuse (std::shared_ptr< const LinOp > input, ReuseData &reuse_data) const

  *Generates a LinOp like generate(), reusing what earlier calls recorded in `reuse_data`.*
- virtual void check_reuse_consistent (const LinOp *input, const ReuseData &reuse_data) const

  *Throws if `input` can't be used with `reuse_data`.*

### 48.150.1 Detailed Description

A LinOpFactory represents a higher order mapping which transforms one linear operator into another.

In Ginkgo, every linear solver is viewed as a mapping. For example, given an s.p.d linear system $Ax = b$, the solution $x = A^{-1}b$ can be computed using the CG method. This algorithm can be represented in terms of linear operators and mappings between them as follows:

- A Cg::Factory is a higher order mapping which, given an input operator $A$, returns a new linear operator $A^{-1}$ stored in "CG format"
- Storing the operator $A^{-1}$ in "CG format" means that the data structure used to store the operator is just a simple pointer to the original matrix $A$. The application $x = A^{-1}b$ of such an operator can then be implemented by solving the linear system $Ax = b$ using the CG method. This is achieved in code by having a special class for each of those "formats" (e.g. the "Cg" class defines such a format for the CG solver).

Another example of a LinOpFactory is a preconditioner. A preconditioner for a linear operator $A$ is a linear operator $M^{-1}$, which approximates $A^{-1}$. In addition, it is stored in a way such that both the data of $M^{-1}$ is cheap to compute from $A$, and the operation $x = M^{-1}b$ can be computed quickly. These operators are useful to accelerate the convergence of Krylov solvers. Thus, a preconditioner also fits into the LinOpFactory framework:

- The factory maps a linear operator $A$ into a preconditioner $M^{-1}$ which is stored in suitable format (e.g. as a product of two factors in case of ILU preconditioners).
- The resulting linear operator implements the application operation $x = M^{-1}b$ depending on the format the preconditioner is stored in (e.g. as two triangular solves in case of ILU)

#### 48.150.1.1 Example: using CG in Ginkgo

```
 {c++}
// Suppose A is a matrix, b a rhs vector, and x an initial guess
// Create a CG which runs for at most 1000 iterations, and stops after
// reducing the residual norm by 6 orders of magnitude
auto cg_factory = solver::Cg<>::build()
    .with_max_iters(1000)
    .with_rel_residual_goal(1e-6)
    .on(cuda);
// create a linear operator which represents the solver
auto cg = cg_factory->generate(A);
// solve the system
cg->apply(b, x);
```

### 48.150.2 Member Function Documentation

#### 48.150.2.1 check_reuse_consistent()

```
virtual void gko::LinOpFactory::check_reuse_consistent (
             const LinOp * input,
             const ReuseData & reuse_data ) const  [virtual]
```

Throws if `input` can't be used with `reuse_data`.

generate_reuse() calls this first, also while `reuse_data` is still empty, so overrides have to accept any input then. Checks that need a converted input may run during the generation instead. The default implementation, used by factories without reuse support, accepts any input, since they generate from scratch.

**Exceptions**

| | |
|---|---|
| *DimensionMismatch* | if a dimension of `input` differs from the input `reuse_data` was initialized with |
| *InvalidStateError* | if `input` does not fit `reuse_data` otherwise |

#### 48.150.2.2 create_empty_reuse_data()

```
virtual std::unique_ptr<ReuseData> gko::LinOpFactory::create_empty_reuse_data ( ) const  [virtual]
```

Creates an empty ReuseData.

The default implementation returns a plain ReuseData.

#### 48.150.2.3 generate_reuse()

```
std::unique_ptr<LinOp> gko::LinOpFactory::generate_reuse (
            std::shared_ptr< const LinOp > input,
            ReuseData & reuse_data ) const
```

Generates a LinOp like generate(), reusing what earlier calls recorded in `reuse_data`.

A ReuseData is empty when it comes from create_empty_reuse_data() and initialized after the first successful generate_reuse(). With an empty `reuse_data`, this is a full generation that also fills it. Once initialized, the input must have the same size and sparsity pattern. What is reused is up to the factory and may depend on values, so the result can differ from generate(). Factories without reuse support just call generate().

As with generate(), the result is independent of `reuse_data` and of earlier results: it may outlive `reuse_data`, and later calls leave it unchanged. One ReuseData serves a whole sequence of generations:

```
 {c++}
auto reuse_data = factory->create_empty_reuse_data();
auto solver = factory->generate_reuse(matrix, *reuse_data);
// change the values of matrix, keeping its sparsity pattern
solver = factory->generate_reuse(matrix, *reuse_data);
```

If this throws, an empty `reuse_data` stays empty and an initialized one stays usable.

**Parameters**

| | |
|---|---|
| *input* | the input, copied to this factory's executor if needed |
| *reuse_data* | created by this factory, or by one with the same type, parameters and executor |

**Returns**

the generated LinOp, with this factory's loggers

**Exceptions**

| | |
|---|---|
| *NotSupported* | if `reuse_data` has the wrong type for this factory |
| *DimensionMismatch* | if a dimension of `input`, like its size or its number of stored elements, differs from the input `reuse_data` was initialized with |
| *InvalidStateError* | if `input` does not fit `reuse_data` otherwise |

**Note**

This interface is experimental.

Referenced by gko::EnableDefaultReusableLinOpFactory< Factory, Multigrid, parameters_type, reuse_data_type >::generate_reuse().

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.151 gko::local_span Struct Reference

A span that is used exclusively for local numbering.

`#include <ginkgo/core/base/range.hpp>`

### Public Member Functions

- constexpr span (size_type point) noexcept

  *Creates a span representing a point `point`.*
- constexpr span (size_type begin, size_type end) noexcept

  *Creates a span.*

### Additional Inherited Members

### 48.151.1 Detailed Description

A span that is used exclusively for local numbering.

### 48.151.2 Member Function Documentation

#### 48.151.2.1 span() [1/2]

`constexpr gko::span::span  [inline], [constexpr], [noexcept]`

Creates a span.

**Parameters**

| | |
|---|---|
| *begin* | the beginning of the span |
| *end* | the end of the span |

```
64         :  begin{begin}, end{end}
65     {}
```

#### 48.151.2.2 span() [2/2]

```
constexpr gko::span::span  [inline], [constexpr], [noexcept]
```

Creates a span representing a point `point`.

The `begin` of this span is set to `point`, and the `end` to `point + 1`.

**Parameters**

| | |
|---|---|
| *point* | the point which the span represents |

```
54         :  span{point, point + 1}
55     {}
```

The documentation for this struct was generated from the following file:

- ginkgo/core/base/range.hpp

## 48.152 gko::log::Loggable Class Reference

Loggable class is an interface which should be implemented by classes wanting to support logging.

```
#include <ginkgo/core/log/logger.hpp>
```

### Public Member Functions

- virtual void add_logger (std::shared_ptr< const Logger > logger)=0

  *Adds a new logger to the list of subscribed loggers.*
- virtual void remove_logger (const Logger *logger)=0

  *Removes a logger from the list of subscribed loggers.*
- virtual const std::vector< std::shared_ptr< const Logger > > & get_loggers () const =0

  *Returns the vector containing all loggers registered at this object.*
- virtual void clear_loggers ()=0

  *Remove all loggers registered at this object.*

### 48.152.1 Detailed Description

Loggable class is an interface which should be implemented by classes wanting to support logging.

For most cases, one can rely on the EnableLogging mixin which provides a default implementation of this interface.

## 48.152.2 Member Function Documentation

### 48.152.2.1 add_logger()

```
virtual void gko::log::Loggable::add_logger (
            std::shared_ptr< const Logger > logger )  [pure virtual]
```

Adds a new logger to the list of subscribed loggers.

**Parameters**

| | |
|---|---|
| *logger* | the logger to add |

Implemented in gko::Executor.

### 48.152.2.2 get_loggers()

```
virtual const std::vector<std::shared_ptr<const Logger> >& gko::log::Loggable::get_loggers (
) const  [pure virtual]
```

Returns the vector containing all loggers registered at this object.

**Returns**

the vector containing all registered loggers.

### 48.152.2.3 remove_logger()

```
virtual void gko::log::Loggable::remove_logger (
            const Logger * logger )  [pure virtual]
```

Removes a logger from the list of subscribed loggers.

**Parameters**

| | |
|---|---|
| *logger* | the logger to remove |

**Note**

The comparison is done using the logger's object unique identity. Thus, two loggers constructed in the same way are not considered equal.

Implemented in gko::Executor.

The documentation for this class was generated from the following file:

- ginkgo/core/log/logger.hpp

## 48.153 gko::log::Record::logged_data Struct Reference

Struct storing the actually logged data.

`#include <ginkgo/core/log/record.hpp>`

### 48.153.1 Detailed Description

Struct storing the actually logged data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.154 gko::solver::LowerTrs< ValueType, IndexType > Class Template Reference

LowerTrs is the triangular solver which solves the system L x = b, when L is a lower triangular matrix.

`#include <ginkgo/core/solver/triangular.hpp>`

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- LowerTrs (const LowerTrs &)

  *Copy-assigns a triangular solver.*
- LowerTrs (LowerTrs &&)

  *Move-assigns a triangular solver.*
- LowerTrs & operator= (const LowerTrs &)

  *Copy-constructs a triangular solver.*
- LowerTrs & operator= (LowerTrs &&)

  *Move-constructs a triangular solver.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## 48.154.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::solver::LowerTrs< ValueType, IndexType >

LowerTrs is the triangular solver which solves the system L x = b, when L is a lower triangular matrix.

It works best when passing in a matrix in CSR format. If the matrix is not in CSR, then the generate step converts it into a CSR matrix. The generation fails if the matrix is not convertible to CSR.

**Note**

As the constructor uses the copy and convert functionality, it is not possible to create a empty solver or a solver with a matrix in any other format other than CSR, if none of the executor modules are being compiled with.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indices |

## 48.154.2 Constructor & Destructor Documentation

### 48.154.2.1 LowerTrs() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::solver::LowerTrs< ValueType, IndexType >::LowerTrs (
            const LowerTrs< ValueType, IndexType > &  )
```

Copy-assigns a triangular solver.

Preserves the executor, shallow-copies the system matrix. If the executors mismatch, clones system matrix onto this executor. Solver analysis information will be regenerated.

### 48.154.2.2 LowerTrs() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::solver::LowerTrs< ValueType, IndexType >::LowerTrs (
            LowerTrs< ValueType, IndexType > &&  )
```

Move-assigns a triangular solver.

Preserves the executor, moves the system matrix. If the executors mismatch, clones system matrix onto this executor and regenerates solver analysis information. Moved-from object is empty (0x0 and nullptr system matrix)

## 48.154.3 Member Function Documentation

#### 48.154.3.1 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::solver::LowerTrs< ValueType, IndexType >::conj_transpose ( )
const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.154.3.2 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
LowerTrs& gko::solver::LowerTrs< ValueType, IndexType >::operator= (
            const LowerTrs< ValueType, IndexType > &  )
```

Copy-constructs a triangular solver.

Preserves the executor, shallow-copies the system matrix. Solver analysis information will be regenerated.

#### 48.154.3.3 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
LowerTrs& gko::solver::LowerTrs< ValueType, IndexType >::operator= (
            LowerTrs< ValueType, IndexType > &&  )
```

Move-constructs a triangular solver.

Preserves the executor, moves the system matrix and solver analysis information. Moved-from object is empty (0x0 and nullptr system matrix)

#### 48.154.3.4 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::solver::LowerTrs< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

#### 48.154.3.5 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::solver::LowerTrs< ValueType, IndexType >::transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/triangular.hpp

## 48.155 gko::experimental::factorization::Lu< ValueType, IndexType > Class Template Reference

Computes an LU factorization of a sparse matrix.

```
#include <ginkgo/core/factorization/lu.hpp>
```

### Public Member Functions

- const parameters_type & get_parameters ()

  *Returns the parameters used to construct the factory.*
- const parameters_type & get_parameters () const

  *Returns the parameters used to construct the factory.*
- std::unique_ptr< factorization_type > generate (std::shared_ptr< const LinOp > system_matrix) const

## Static Public Member Functions

- static parameters_type build ()

  *Creates a new parameter_type to set up the factory.*
- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

### 48.155.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::experimental::factorization::Lu< ValueType, IndexType >**

Computes an LU factorization of a sparse matrix.

This LinOpFactory returns a Factorization storing the L and U factors for the provided system matrix in matrix::Csr format. If no symbolic factorization is provided, it will be computed first. It expects all fill-in entries to be present in the symbolic factorization. If the symbolic factorization is missing some entries, please refer to Ilu.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type used to store values of the system matrix |
| *IndexType* | the type used to store sparsity pattern indices of the system matrix |

### 48.155.2 Member Function Documentation

#### 48.155.2.1 generate()

```
template<typename ValueType , typename IndexType >
std::unique_ptr<factorization_type> gko::experimental::factorization::Lu< ValueType, Index↩
Type >::generate (
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix used to generate the factors. |

**Note**

This function overrides the default LinOpFactory::generate to return a Factorization instead of a generic LinOp, which would need to be cast to Factorization again to access its factors. It is only necessary because smart pointers aren't covariant.

### 48.155.2.2 get_parameters() [1/2]

```
template<typename ValueType , typename IndexType >
const parameters_type& gko::experimental::factorization::Lu< ValueType, IndexType >::get_↩
parameters ( )  [inline]
```

Returns the parameters used to construct the factory.

#### Returns

the parameters used to construct the factory.

```
108 { return parameters_; }
```

### 48.155.2.3 get_parameters() [2/2]

```
template<typename ValueType , typename IndexType >
const parameters_type& gko::experimental::factorization::Lu< ValueType, IndexType >::get_↩
parameters ( ) const  [inline]
```

Returns the parameters used to construct the factory.

#### Returns

the parameters used to construct the factory.

```
113 { return parameters_; }
```

### 48.155.2.4 parse()

```
template<typename ValueType , typename IndexType >
static parameters_type gko::experimental::factorization::Lu< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

#### Parameters

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/lu.hpp

## 48.156 gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType > Class Template Reference

The Matrix class defines a (MPI-)distributed matrix.

`#include <ginkgo/core/distributed/matrix.hpp>`

### Public Member Functions

- void read_distributed (const device_matrix_data< value_type, global_index_type > &data, std::shared_ptr< const Partition< local_index_type, global_index_type >> partition, assembly_mode assembly_type=assembly_mode::local_only)

  *Reads a square matrix from the device_matrix_data structure and a global partition.*
- void read_distributed (const matrix_data< value_type, global_index_type > &data, std::shared_ptr< const Partition< local_index_type, global_index_type >> partition, assembly_mode assembly_type=assembly_mode::local_only)

  *Reads a square matrix from the matrix_data structure and a global partition.*
- void read_distributed (const device_matrix_data< value_type, global_index_type > &data, std::shared_ptr< const Partition< local_index_type, global_index_type >> row_partition, std::shared_ptr< const Partition< local_index_type, global_index_type >> col_partition, assembly_mode assembly_type=assembly_mode::local_only)

  *Reads a matrix from the device_matrix_data structure, a global row partition, and a global column partition.*
- void read_distributed (const matrix_data< value_type, global_index_type > &data, std::shared_ptr< const Partition< local_index_type, global_index_type >> row_partition, std::shared_ptr< const Partition< local_index_type, global_index_type >> col_partition, assembly_mode assembly_type=assembly_mode::local_only)

  *Reads a matrix from the matrix_data structure, a global row partition, and a global column partition.*
- void write (matrix_data< value_type, global_index_type > &data) const override

  *Writes a matrix to a matrix_data structure.*
- std::shared_ptr< const LinOp > get_diag_matrix () const

  *Get read access to the stored diagonal matrix block.*
- std::shared_ptr< const LinOp > get_off_diag_matrix () const

  *Get read access to the stored off-diagonal matrix block.*
- std::shared_ptr< const RowGatherer< LocalIndexType > > get_row_gatherer () const

  *Get read access to the row gatherer used to fetch the non-local rows during an apply.*
- std::shared_ptr< const LinOp > get_local_matrix () const
- std::shared_ptr< const LinOp > get_non_local_matrix () const
- std::shared_ptr< const Partition< local_index_type, global_index_type > > get_row_partition () const

  *Get the row partition of this matrix.*
- void multiply (ptr_param< const Matrix > b, ptr_param< Matrix > c) const

  *Computes the distributed sparse matrix-matrix product C = A ∗ B, where A is this matrix.*
- void transpose (ptr_param< Matrix > result) const

*Computes the transpose of this matrix into result.*

- Matrix (const Matrix &other)

  *Copy constructs a Matrix.*

- Matrix (Matrix &&other) noexcept

  *Move constructs a Matrix.*

- Matrix & operator= (const Matrix &other)

  *Copy assigns a Matrix.*

- Matrix & operator= (Matrix &&other)

  *Move assigns a Matrix.*

- void col_scale (ptr_param< const global_vector_type > scaling_factors)

  *Scales the columns of the matrix by the respective entries of the vector.*

- void row_scale (ptr_param< const global_vector_type > scaling_factors)

  *Scales the rows of the matrix by the respective entries of the vector.*

## Static Public Member Functions

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm)

  *Creates an empty distributed matrix.*

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, std::shared_ptr< const RowGatherer< LocalIndexType >> row_gatherer_template)

  *Creates an empty distributed matrix with a specified implementation of the row gather operation.*

- template<typename MatrixType , typename = std::enable_if_t<gko::detail::is_matrix_type_builder< MatrixType, ValueType, LocalIndexType>::value>>
  static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, MatrixType matrix_template)

  *Creates an empty distributed matrix with specified type for local matrices.*

- template<typename DiagMatrixType , typename OffDiagMatrixType , typename = std::enable_if_t< gko::detail::is_matrix_type_builder<DiagMatrixType, ValueType, LocalIndexType>::value && gko::detail::is_matrix_type_builder< OffDiagMatrixType, ValueType, LocalIndexType>::value>>
  static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, DiagMatrixType diag_matrix_template, OffDiagMatrixType off_diag_matrix_template)

  *Creates an empty distributed matrix with specified types for the diagonal matrix and the off-diagonal matrix.*

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, ptr_param< const LinOp > matrix_template)

  *Creates an empty distributed matrix with specified type for local matrices.*

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, ptr_param< const LinOp > diag_matrix_template, ptr_param< const LinOp > off_diag_matrix_template)

  *Creates an empty distributed matrix with specified types for the diagonal matrix and the off-diagonal matrix.*

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, dim< 2 > size, std::shared_ptr< LinOp > diag_linop)

  *Creates a diag-only distributed matrix with existent LinOp.*

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, dim< 2 > size, std::shared_ptr< LinOp > diag_linop, std::shared_ptr< LinOp > off_diag_linop, std::vector< comm_index_type > recv_sizes, std::vector< comm_index_type > recv_offsets, array< local_index_type > recv_gather_idxs)

  *Creates distributed matrix with existent diagonal and off-diagonal LinOp and the corresponding mapping to collect the off-diagonal data from the other ranks.*

- static std::unique_ptr< Matrix > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, index_map< local_index_type, global_index_type > imap, std::shared_ptr< LinOp > diag_linop, std::shared_ptr< LinOp > off_diag_linop)

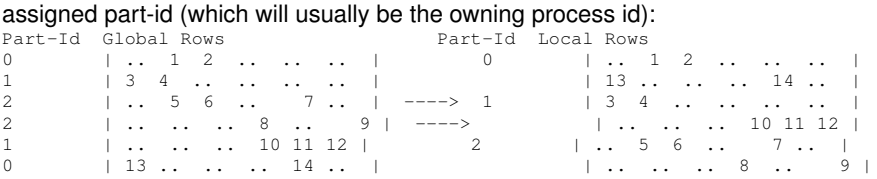
  *Creates distributed matrix with existent diagonal and off-diagonal LinOp and the corresponding mapping to collect the off-diagonal data from the other ranks.*

### 48.156.1 Detailed Description

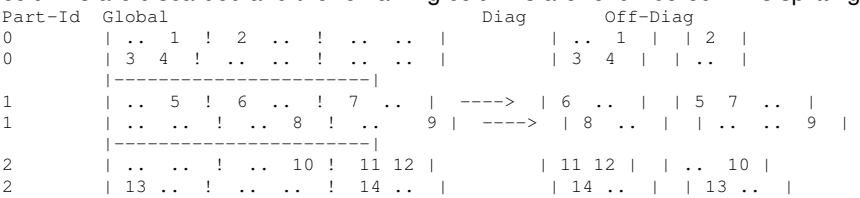
**template<typename ValueType = default_precision, typename LocalIndexType = int32, typename GlobalIndexType = int64>**
**class gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >**

The Matrix class defines a (MPI-)distributed matrix.

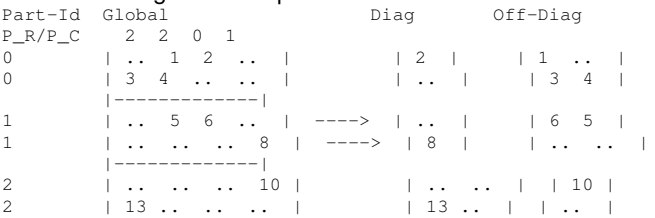
The matrix is stored in a row-wise distributed format. Each process owns a specific set of rows, where the assignment of rows is defined by a row Partition. The following depicts the distribution of global rows according to their assigned part-id (which will usually be the owning process id):




The local rows are further split into two matrices on each process. One matrix, called `diag`, contains only entries from columns that are also owned by the process, while the other one, called `off_diag`, contains entries from columns that are not owned by the process. The off-diagonal matrix is stored in a compressed format, where empty columns are discarded and the remaining columns are renumbered. This splitting is depicted in the following:




This uses the same ownership of the columns as for the rows. Additionally, the ownership of the columns may be explicitly defined with an second column partition. If that is not provided, the same row partition will be used for the columns. Using a column partition also allows to create non-square matrices, like the one below:




Here `P_R` denotes the row partition and `P_C` denotes the column partition.

The Matrix should be filled using the read_distributed method, e.g.

```
auto part = Partition<...>::build_from_mapping(...);
auto mat = Matrix<...>::create(exec, comm);
mat->read_distributed(matrix_data, part);
```

or if different partitions for the rows and columns are used:

```
auto row_part = Partition<...>::build_from_mapping(...);
auto col_part = Partition<...>::build_from_mapping(...);
auto mat = Matrix<...>::create(exec, comm);
mat->read_distributed(matrix_data, row_part, col_part);
```

This will set the dimensions of the global and local matrices automatically by deducing the sizes from the partitions.

By default the Matrix type uses Csr for both stored matrices. It is possible to explicitly change the datatype for the stored matrices, with the constraint that the new type should implement the LinOp and ReadableFromMatrixData interface. The type can be set by:

```
auto mat = Matrix<ValueType, LocalIndexType[, ...]>::create(
  exec, comm,
  Ell<ValueType, LocalIndexType>::create(exec).get(),
  Coo<ValueType, LocalIndexType>::create(exec).get());
```

Alternatively, the helper function with_matrix_type can be used:

```
auto mat = Matrix<ValueType, LocalIndexType>::create(
  exec, comm,
  with_matrix_type<Ell>(),
  with_matrix_type<Coo>());
```

**See also**

with_matrix_type

The Matrix LinOp supports the following operations:

```
experimental::distributed::Matrix *A;       // distributed matrix
experimental::distributed::Vector *b, *x;   // distributed multi-vectors
matrix::Dense *alpha, *beta;  // scalars of dimension 1x1
// Applying to distributed multi-vectors computes an SpMV/SpMM product
A->apply(b, x)                // x = A*b
A->apply(alpha, b, beta, x) // x = alpha*A*b + beta*x
```

**Template Parameters**

| | |
|---|---|
| *ValueType* | The underlying value type. |
| *LocalIndexType* | The index type used by the local matrices. |
| *GlobalIndexType* | The type for global indices. |

## 48.156.2 Constructor & Destructor Documentation

### 48.156.2.1 Matrix() [1/2]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::Matrix (
            const Matrix< ValueType, LocalIndexType, GlobalIndexType > & other )
```

Copy constructs a Matrix.

**Parameters**

| | |
|---|---|
| *other* | Matrix to copy from. |

### 48.156.2.2 Matrix() [2/2]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::Matrix (
            Matrix< ValueType, LocalIndexType, GlobalIndexType > && other )  [noexcept]
```

Move constructs a Matrix.

**Parameters**

| | |
|---|---|
| *other* | Matrix to move from. |

## 48.156.3 Member Function Documentation

### 48.156.3.1 col_scale()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::col_scale (
            ptr_param< const global_vector_type > scaling_factors )
```

Scales the columns of the matrix by the respective entries of the vector.

The vector's row partition has to be the same as the matrix's column partition. The scaling is done in-place.

**Parameters**

| | |
|---|---|
| *scaling_factors* | The vector containing the scaling factors. |

### 48.156.3.2 create() [1/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm )  [static]
```

Creates an empty distributed matrix.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. The default is the MPI_COMM_WORLD. |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::create().

### 48.156.3.3 create() [2/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
```

```
template<typename DiagMatrixType , typename OffDiagMatrixType , typename = std::enable_if_t<
gko::detail::is_matrix_type_builder<DiagMatrixType, ValueType, LocalIndexType>::value && gko↩
::detail::is_matrix_type_builder< OffDiagMatrixType, ValueType, LocalIndexType>::value>>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            DiagMatrixType diag_matrix_template,
            OffDiagMatrixType off_diag_matrix_template )  [inline], [static]
```

Creates an empty distributed matrix with specified types for the diagonal matrix and the off-diagonal matrix.

#### Note

This is a convenience wrapper around the `create` overload that takes already-constructed `LinOp` templates for the diagonal and off-diagonal blocks; the two template arguments here are materialized internally.

#### Template Parameters

| | |
|---|---|
| *DiagMatrixType* | A type that has a `create<ValueType, IndexType>(exec)` function to create a smart pointer of a type derived from LinOp and ReadableFromMatrixData. |

#### See also

with_matrix_type

#### Template Parameters

| | |
|---|---|
| *OffDiagMatrixType* | A (possible different) type with the same constraints as DiagMatrixType. |

#### Parameters

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *diag_matrix_template* | the diagonal matrix will be constructed with the same type as `create` returns. It should be the return value of make_matrix_template. |
| *off_diag_matrix_template* | the off-diagonal matrix will be constructed with the same type as `create` returns. It should be the return value of make_matrix_template. |

#### Returns

A smart pointer to the newly created matrix.

```
672     {
673         return create(
674             exec, comm,
675             diag_matrix_template.template create<ValueType, LocalIndexType>(
676                 exec),
677             off_diag_matrix_template.template create<ValueType, LocalIndexType>(
678                 exec));
679     }
```

References gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::create().

#### 48.156.3.4 create() [3/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            dim< 2 > size,
            std::shared_ptr< LinOp > diag_linop )  [static]
```

Creates a diag-only distributed matrix with existent LinOp.

**Note**

It use the input to build up the distributed matrix

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *size* | the global size |
| *diag_linop* | the diagonal block linop |

**Returns**

A smart pointer to the newly created matrix.

#### 48.156.3.5 create() [4/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static std::  unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            dim< 2 > size,
            std::shared_ptr< LinOp > diag_linop,
            std::shared_ptr< LinOp > off_diag_linop,
            std::vector< comm_index_type > recv_sizes,
            std::vector< comm_index_type > recv_offsets,
            array< local_index_type > recv_gather_idxs )  [static]
```

Creates distributed matrix with existent diagonal and off-diagonal LinOp and the corresponding mapping to collect the off-diagonal data from the other ranks.

**Note**

It use the input to build up the distributed matrix

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *size* | the global size |
| *diag_linop* | the diagonal block linop |
| *off_diag_linop* | the off-diagonal block linop |
| *recv_sizes* | the size of off-diagonal receiver |
| *recv_offsets* | the offset of off-diagonal receiver |
| *recv_gather_idxs* | the gathering index of off-diagonal receiver |

**Returns**

A smart pointer to the newly created matrix.

### 48.156.3.6 create() [5/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
              std::shared_ptr< const Executor > exec,
              mpi::communicator comm,
              index_map< local_index_type, global_index_type > imap,
              std::shared_ptr< LinOp > diag_linop,
              std::shared_ptr< LinOp > off_diag_linop )  [static]
```

Creates distributed matrix with existent diagonal and off-diagonal LinOp and the corresponding mapping to collect the off-diagonal data from the other ranks.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *imap* | The index map to define the communication pattern |
| *diag_linop* | the diagonal block linop |
| *off_diag_linop* | the off-diagonal block linop |

**Returns**

A smart pointer to the newly created matrix.

### 48.156.3.7 create() [6/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
```

```
template<typename MatrixType , typename = std::enable_if_t<gko::detail::is_matrix_type_↩
builder< MatrixType, ValueType, LocalIndexType>::value>>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            MatrixType matrix_template )  [inline], [static]
```

Creates an empty distributed matrix with specified type for local matrices.

#### Note

This is a convenience wrapper around the `create` overload that takes an already-constructed `LinOp` template; the `matrix_template` argument here is materialized internally.

#### Template Parameters

| | |
|---|---|
| *MatrixType* | A type that has a `create<ValueType, IndexType>(exec)` function to create a smart pointer of a type derived from LinOp and ReadableFromMatrixData. |

#### See also

with_matrix_type

#### Parameters

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *matrix_template* | the local matrices will be constructed with the same type as `create` returns. It should be the return value of make_matrix_template. |

#### Returns

A smart pointer to the newly created matrix.

```
627     {
628         return create(
629             exec, comm,
630             matrix_template.template create<ValueType, LocalIndexType>(exec));
631     }
```

References gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::create().

### 48.156.3.8 create() [7/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            ptr_param< const LinOp > diag_matrix_template,
            ptr_param< const LinOp > off_diag_matrix_template )  [static]
```

Creates an empty distributed matrix with specified types for the diagonal matrix and the off-diagonal matrix.

**Note**

It internally clones the passed in diag_matrix_template and off_diag_matrix_template. Therefore, those LinOps should be empty.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *diag_matrix_template* | the diagonal matrix will be constructed with the same runtime type. |
| *off_diag_matrix_template* | the off-diagonal matrix will be constructed with the same runtime type. |

**Returns**

A smart pointer to the newly created matrix.

### 48.156.3.9 create() [8/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            ptr_param< const LinOp > matrix_template )  [static]
```

Creates an empty distributed matrix with specified type for local matrices.

**Note**

It internally clones the passed in matrix_template. Therefore, the LinOp should be empty.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *comm* | Communicator associated with this matrix. |
| *matrix_template* | the local matrices will be constructed with the same runtime type. |

**Returns**

A smart pointer to the newly created matrix.

### 48.156.3.10 create() [9/9]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
```

```
static std::unique_ptr<Matrix> gko::experimental::distributed::Matrix< ValueType, Local↩
IndexType, GlobalIndexType >::create (
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< const RowGatherer< LocalIndexType >> row_gatherer_template )
[static]
```

Creates an empty distributed matrix with a specified implementation of the row gather operation.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this matrix. |
| *row_gatherer_template* | A template for the used row gather operation. This is only used to create a new row gatherer during the read_distributed. |

**Returns**

A smart pointer to the newly created matrix.

#### 48.156.3.11 get_diag_matrix()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
std::shared_ptr<const LinOp> gko::experimental::distributed::Matrix< ValueType, LocalIndex↩
Type, GlobalIndexType >::get_diag_matrix ( ) const  [inline]
```

Get read access to the stored diagonal matrix block.

The diagonal block contains entries from columns that are owned by this process.

**Returns**

Shared pointer to the stored diagonal matrix block

```
446 { return diag_mtx_; }
```

Referenced by gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::get_local↩
_matrix().

#### 48.156.3.12 get_local_matrix()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
std::shared_ptr<const LinOp> gko::experimental::distributed::Matrix< ValueType, LocalIndex↩
Type, GlobalIndexType >::get_local_matrix ( ) const  [inline]
482     {
483         return get_diag_matrix();
484     }
```

References gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::get_diag_↩
matrix().

### 48.156.3.13 get_non_local_matrix()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
std::shared_ptr<const LinOp> gko::experimental::distributed::Matrix< ValueType, LocalIndex↩
Type, GlobalIndexType >::get_non_local_matrix ( ) const  [inline]
491     {
492         return get_off_diag_matrix();
493     }
```

References gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::get_off_↩ diag_matrix().

### 48.156.3.14 get_off_diag_matrix()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
std::shared_ptr<const LinOp> gko::experimental::distributed::Matrix< ValueType, LocalIndex↩
Type, GlobalIndexType >::get_off_diag_matrix ( ) const  [inline]
```

Get read access to the stored off-diagonal matrix block.

The off-diagonal block contains entries from columns that are not owned by this process.

#### Returns

Shared pointer to the stored off-diagonal matrix block

```
457     {
458         return off_diag_mtx_;
459     }
```

Referenced by gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >::get_non↩ _local_matrix().

### 48.156.3.15 get_row_gatherer()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
std::shared_ptr<const RowGatherer<LocalIndexType> > gko::experimental::distributed::Matrix<
ValueType, LocalIndexType, GlobalIndexType >::get_row_gatherer ( ) const  [inline]
```

Get read access to the row gatherer used to fetch the non-local rows during an apply.

Its collective communicator determines how this matrix exchanges data, and is carried over to every matrix created from it – in particular to the one read_distributed rebuilds – so this is how to check which exchange a matrix actually uses.

#### Returns

Shared pointer to the stored row gatherer

```
473     {
474         return row_gatherer_;
475     }
```

#### 48.156.3.16 get_row_partition()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
std::shared_ptr<const Partition<local_index_type, global_index_type> > gko::experimental::distributed::Matri
ValueType, LocalIndexType, GlobalIndexType >::get_row_partition ( ) const
```

Get the row partition of this matrix.

It is set by read_distributed, and by the index_map constructor to the partition of the index_map. May be null otherwise.

#### 48.156.3.17 multiply()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::multiply (
            ptr_param< const Matrix< ValueType, LocalIndexType, GlobalIndexType > > b,
            ptr_param< Matrix< ValueType, LocalIndexType, GlobalIndexType > > c ) const
```

Computes the distributed sparse matrix-matrix product C = A ∗ B, where A is this matrix.

The output c is filled in place; any prior contents are overwritten.

Matmul compatibility requires this matrix's column partition to equal b's row partition. The result c has this matrix's row partition and b's column partition, and shares this matrix's communicator.

**Note**

This is only supported when the local and non-local blocks of both operands are of type Csr.

**Parameters**

| | |
|---|---|
| *b* | the right-hand operand of the product. |
| *c* | a pre-created distributed Matrix (may be empty) to fill. |

#### 48.156.3.18 operator=() [1/2]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
Matrix& gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::operator= (
            const Matrix< ValueType, LocalIndexType, GlobalIndexType > & other )
```

Copy assigns a Matrix.

**Parameters**

| | |
|---|---|
| *other* | Matrix to copy from, has to have a communicator of the same size as this. |

**Returns**

this.

### 48.156.3.19 operator=() [2/2]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
Matrix& gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::operator= (
            Matrix< ValueType, LocalIndexType, GlobalIndexType > && other )
```

Move assigns a Matrix.

**Parameters**

| | |
|---|---|
| *other* | Matrix to move from, has to have a communicator of the same size as this. |

**Returns**

this.

### 48.156.3.20 read_distributed() [1/4]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::read_distributed (
            const device_matrix_data< value_type, global_index_type > & data,
            std::shared_ptr< const Partition< local_index_type, global_index_type >> partition,
            assembly_mode assembly_type = assembly_mode::local_only )
```

Reads a square matrix from the device_matrix_data structure and a global partition.

The global size of the final matrix is inferred from the size of the partition. Both the number of rows and columns of the device_matrix_data are ignored.

**Note**

The matrix data can contain entries for rows other than those owned by the process. Entries for those rows are discarded.

**Parameters**

| | |
|---|---|
| *data* | The device_matrix_data structure. |
| *partition* | The global row and column partition. |
| *assembly_type* | The mode of assembly. |

**Returns**

the index_map induced by the partitions and the matrix structure

### 48.156.3.21 read_distributed() [2/4]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::read_distributed (
            const device_matrix_data< value_type, global_index_type > & data,
            std::shared_ptr< const Partition< local_index_type, global_index_type >> row_↩
partition,
            std::shared_ptr< const Partition< local_index_type, global_index_type >> col_↩
partition,
            assembly_mode assembly_type = assembly_mode::local_only )
```

Reads a matrix from the device_matrix_data structure, a global row partition, and a global column partition.

The global size of the final matrix is inferred from the size of the row partition and the size of the column partition. Both the number of rows and columns of the device_matrix_data are ignored.

**Note**

The matrix data can contain entries for rows other than those owned by the process. Entries for those rows are discarded.

**Parameters**

| | |
|---|---|
| *data* | The device_matrix_data structure. |
| *row_partition* | The global row partition. |
| *col_partition* | The global col partition. |
| *assembly_type* | The mode of assembly. |

**Returns**

the index_map induced by the partitions and the matrix structure

### 48.156.3.22 read_distributed() [3/4]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::read_distributed (
            const matrix_data< value_type, global_index_type > & data,
            std::shared_ptr< const Partition< local_index_type, global_index_type >> partition,
            assembly_mode assembly_type = assembly_mode::local_only )
```

Reads a square matrix from the matrix_data structure and a global partition.

**See also**

read_distributed

**Note**

For efficiency it is advised to use the device_matrix_data overload.

### 48.156.3.23 read_distributed() [4/4]

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::read_distributed (
            const matrix_data< value_type, global_index_type > & data,
            std::shared_ptr< const Partition< local_index_type, global_index_type >> row_↩
partition,
            std::shared_ptr< const Partition< local_index_type, global_index_type >> col_↩
partition,
            assembly_mode assembly_type = assembly_mode::local_only )
```

Reads a matrix from the matrix_data structure, a global row partition, and a global column partition.

**See also**

read_distributed

**Note**

For efficiency it is advised to use the device_matrix_data overload.

### 48.156.3.24 row_scale()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::row_scale (
            ptr_param< const global_vector_type > scaling_factors )
```

Scales the rows of the matrix by the respective entries of the vector.

The vector and the matrix have to have the same row partition. The scaling is done in-place.

**Parameters**

| | |
|---|---|
| *scaling_factors* | The vector containing the scaling factors. |

#### 48.156.3.25 transpose()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::transpose (
            ptr_param< Matrix< ValueType, LocalIndexType, GlobalIndexType > > result ) const
```

Computes the transpose of this matrix into result.

The output is filled in place; any prior contents are overwritten.

The result takes this matrix's column partition as its row partition and this matrix's row partition as its column partition, so it is a valid operand of multiply. Entries whose transposed row is owned by another rank are communicated to that rank, so this is a collective call.

**Note**

This requires that this matrix was filled by read_distributed, which is what sets both partitions, and that result uses the same executor and communicator as this matrix.

**Parameters**

| | |
|---|---|
| *result* | a pre-created distributed Matrix (may be empty) to fill. |

#### 48.156.3.26 write()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
void gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >↩
::write (
            matrix_data< value_type, global_index_type > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, GlobalIndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/matrix.hpp

# 48.157 gko::matrix_assembly_data< ValueType, IndexType > Class Template Reference

This structure is used as an intermediate type to assemble a sparse matrix.

`#include <ginkgo/core/base/matrix_assembly_data.hpp>`

## Public Member Functions

- void add_value (index_type row, index_type col, value_type val)

  *Sets the matrix value at (row, col).*
- void set_value (index_type row, index_type col, value_type val)

  *Sets the matrix value at (row, col).*
- value_type get_value (index_type row, index_type col)

  *Gets the matrix value at (row, col).*
- bool contains (index_type row, index_type col)

  *Returns true iff the matrix contains an entry at (row, col).*
- dim< 2 > get_size () const noexcept
- size_type get_num_stored_elements () const noexcept
- matrix_data< ValueType, IndexType > get_ordered_data () const

## 48.157.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix_assembly_data< ValueType, IndexType >**

This structure is used as an intermediate type to assemble a sparse matrix.

The matrix is stored as a set of nonzero elements, where each element is a triplet of the form (row_index, column↩ _index, value).

New values can be added by using the matrix_assembly_data::add_value or matrix_assembly_data::set_value

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values stored in the structure |
| *IndexType* | type of matrix indexes stored in the structure |

## 48.157.2 Member Function Documentation

#### 48.157.2.1 add_value()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix_assembly_data< ValueType, IndexType >::add_value (
            index_type row,
            index_type col,
            value_type val )  [inline]
```

Sets the matrix value at (row, col).

If there is an existing value, it will be set to the sum of the existing and new value, otherwise the value will be inserted.

**Parameters**

| | |
|---|---|
| *row* | the row where the value should be added |
| *col* | the column where the value should be added |
| *val* | the value to be added to (row, col) |

```
81     {
82         auto ind = std::make_pair(row, col);
83         nonzeros_[ind] += val;
84     }
```

#### 48.157.2.2 contains()

```
template<typename ValueType = default_precision, typename IndexType = int32>
bool gko::matrix_assembly_data< ValueType, IndexType >::contains (
            index_type row,
            index_type col )  [inline]
```

Returns true iff the matrix contains an entry at (row, col).

**Parameters**

| | |
|---|---|
| *row* | the row index |
| *col* | the column index |

**Returns**

true if the value at (row, col) exists, false otherwise

```
125     {
126         return nonzeros_.find(std::make_pair(row, col)) != nonzeros_.end();
127     }
```

#### 48.157.2.3 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix_assembly_data< ValueType, IndexType >::get_num_stored_elements ( ) const
[inline], [noexcept]
```

**Returns**

the number of non-zeros in the (partially) assembled matrix

```
134     {
135         return nonzeros_.size();
136     }
```

### 48.157.2.4 get_ordered_data()

```
template<typename ValueType = default_precision, typename IndexType = int32>
matrix_data<ValueType, IndexType> gko::matrix_assembly_data< ValueType, IndexType >::get_↩
ordered_data ( ) const  [inline]
```

**Returns**

a matrix_data instance containing the assembled non-zeros in row-major order to be used by all matrix formats.

```
143     {
144         using output_type = matrix_data<ValueType, IndexType>;
145         using nonzero_type = typename output_type::nonzero_type;
146         using entry_type =
147             std::pair<std::pair<index_type, index_type>, value_type>;
148         output_type data{size_};
149         data.nonzeros.reserve(nonzeros_.size());
150         std::transform(nonzeros_.begin(), nonzeros_.end(),
151                        std::back_inserter(data.nonzeros), [](entry_type entry) {
152                            return nonzero_type{entry.first.first,
153                                                entry.first.second,
154                                                entry.second};
155                        });
156         data.sort_row_major();
157         return data;
158     }
```

Referenced by gko::ReadableFromMatrixData< ValueType, int32 >::read().

### 48.157.2.5 get_size()

```
template<typename ValueType = default_precision, typename IndexType = int32>
dim<2> gko::matrix_assembly_data< ValueType, IndexType >::get_size ( ) const  [inline], [noexcept]
```

**Returns**

the dimensions of the matrix being assembled

```
130 { return size_; }
```

### 48.157.2.6 get_value()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type gko::matrix_assembly_data< ValueType, IndexType >::get_value (
              index_type row,
              index_type col )  [inline]
```

Gets the matrix value at (row, col).

**Parameters**

| | |
|---|---|
| *row* | the row index |
| *col* | the column index |

**Returns**

the value at (row, col) or 0 if it doesn't exist.

```
108     {
109         const auto it = nonzeros_.find(std::make_pair(row, col));
110         if (it == nonzeros_.end()) {
111             return zero<value_type>();
112         } else {
113             return it->second;
114         }
115     }
```

#### 48.157.2.7 set_value()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix_assembly_data< ValueType, IndexType >::set_value (
              index_type row,
              index_type col,
              value_type val )  [inline]
```

Sets the matrix value at (row, col).

If there is an existing value, it will be overwritten by the new value.

**Parameters**

| | |
|---|---|
| *row* | the row index |
| *col* | the column index |
| *val* | the value to be written to (row, col) |

```
95     {
96         auto ind = std::make_pair(row, col);
97         nonzeros_[ind] = val;
98     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/matrix_assembly_data.hpp

## 48.158 gko::matrix_data< ValueType, IndexType > Struct Template Reference

This structure is used as an intermediate data type to store a sparse matrix.

```
#include <ginkgo/core/base/matrix_data.hpp>
```

## Public Member Functions

- matrix_data (dim< 2 > size_=dim< 2 >{}, ValueType value=zero< ValueType >())

  *Initializes a matrix filled with the specified value.*

- template<typename RandomDistribution , typename RandomEngine >
  matrix_data (dim< 2 > size_, RandomDistribution &&dist, RandomEngine &&engine)

  *Initializes a matrix with random values from the specified distribution.*

- matrix_data (std::initializer_list< std::initializer_list< ValueType >> values)

  *List-initializes the structure from a matrix of values.*

- matrix_data (dim< 2 > size_, std::initializer_list< detail::input_triple< ValueType, IndexType >> nonzeros_)

  *Initializes the structure from a list of nonzeros.*

- matrix_data (dim< 2 > size_, const matrix_data &block)

  *Initializes a matrix out of a matrix block via duplication.*

- template<typename Accessor >
  matrix_data (const range< Accessor > &data)

  *Initializes a matrix from a range.*

- void sort_row_major ()

  *Sorts the nonzero vector so the values follow row-major order.*

- void ensure_row_major_order ()

  *Sorts the nonzero vector so the values follow row-major order.*

- void remove_zeros ()

  *Remove entries with value zero from the matrix data.*

- void sum_duplicates ()

  *Sum up all values that refer to the same matrix entry.*

## Static Public Member Functions

- static matrix_data diag (dim< 2 > size_, ValueType value)

  *Initializes a diagonal matrix.*

- static matrix_data diag (dim< 2 > size_, std::initializer_list< ValueType > nonzeros_)

  *Initializes a diagonal matrix using a list of diagonal elements.*

- static matrix_data diag (dim< 2 > size_, const matrix_data &block)

  *Initializes a block-diagonal matrix.*

- template<typename ForwardIterator >
  static matrix_data diag (ForwardIterator begin, ForwardIterator end)

  *Initializes a block-diagonal matrix from a list of diagonal blocks.*

- static matrix_data diag (std::initializer_list< matrix_data > blocks)

  *Initializes a block-diagonal matrix from a list of diagonal blocks.*

- template<typename RandomDistribution , typename RandomEngine >
  static matrix_data cond (size_type size, remove_complex< ValueType > condition_number, RandomDistribution &&dist, RandomEngine &&engine, size_type num_reflectors)

  *Initializes a random dense matrix with a specific condition number.*

- template<typename RandomDistribution , typename RandomEngine >
  static matrix_data cond (size_type size, remove_complex< ValueType > condition_number, RandomDistribution &&dist, RandomEngine &&engine)

  *Initializes a random dense matrix with a specific condition number.*

## Public Attributes

- dim< 2 > size

  *Size of the matrix.*
- std::vector< nonzero_type > nonzeros

  *A vector of tuples storing the non-zeros of the matrix.*

### 48.158.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**struct gko::matrix_data< ValueType, IndexType >**

This structure is used as an intermediate data type to store a sparse matrix.

The matrix is stored as a sequence of nonzero elements, where each element is a triple of the form (row_index, column_index, value).

**Note**

All Ginkgo functions returning such a structure will return the nonzeros sorted in row-major order.

All Ginkgo functions that take this structure as input expect that the nonzeros are sorted in row-major order and that the index pair (row_index, column_index) of each nonzero is unique.

This structure is not optimized for usual access patterns and it can only exist on the CPU. Thus, it should only be used for utility functions which do not have to be optimized for performance.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of matrix values stored in the structure |
| *IndexType* | type of matrix indexes stored in the structure |

### 48.158.2 Constructor & Destructor Documentation

#### 48.158.2.1 matrix_data() [1/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix_data< ValueType, IndexType >::matrix_data (
            dim< 2 > size_ = dim<2>{},
            ValueType value = zero<ValueType>() )  [inline]
```

Initializes a matrix filled with the specified value.

**Parameters**

| | |
|---|---|
| *size_* | dimensions of the matrix |
| *value* | value used to fill the elements of the matrix |

```
138                                     {}, ValueType value = zero<ValueType>())
139         :  size{size_}
140     {
141         if (is_zero(value)) {
142             return;
143         }
144         nonzeros.reserve(size[0] * size[1]);
145         for (size_type row = 0; row < size[0]; ++row) {
146             for (size_type col = 0; col < size[1]; ++col) {
147                 nonzeros.emplace_back(row, col, value);
148             }
149         }
150     }
```

### 48.158.2.2 matrix_data() [2/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename RandomDistribution , typename RandomEngine >
gko::matrix_data< ValueType, IndexType >::matrix_data (
             dim< 2 > size_,
             RandomDistribution && dist,
             RandomEngine && engine )  [inline]
```

Initializes a matrix with random values from the specified distribution.

#### Template Parameters

| | |
|---|---|
| *RandomDistribution* | random distribution type |
| *RandomEngine* | random engine type |

#### Parameters

| | |
|---|---|
| *size*↩ _ | dimensions of the matrix |
| *dist* | random distribution of the elements of the matrix |
| *engine* | random engine used to generate random values |

```
164         :  size{size_}
165     {
166         nonzeros.reserve(size[0] * size[1]);
167         for (size_type row = 0; row < size[0]; ++row) {
168             for (size_type col = 0; col < size[1]; ++col) {
169                 const auto value =
170                     detail::get_rand_value<ValueType>(dist, engine);
171                 if (is_nonzero(value)) {
172                     nonzeros.emplace_back(row, col, value);
173                 }
174             }
175         }
176     }
```

### 48.158.2.3 matrix_data() [3/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix_data< ValueType, IndexType >::matrix_data (
             std::initializer_list< std::initializer_list< ValueType >> values )  [inline]
```

List-initializes the structure from a matrix of values.

**Parameters**

| | |
|---|---|
| *values* | a 2D braced-init-list of matrix values. |

```
184         :  size{values.size(), 0}
185     {
186         for (size_type row = 0; row < values.size(); ++row) {
187             const auto row_data = begin(values)[row];
188             size[1] = std::max(size[1], row_data.size());
189             for (size_type col = 0; col < row_data.size(); ++col) {
190                 const auto& val = begin(row_data)[col];
191                 if (is_nonzero(val)) {
192                     nonzeros.emplace_back(row, col, val);
193                 }
194             }
195         }
196     }
```

### 48.158.2.4 matrix_data() [4/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix_data< ValueType, IndexType >::matrix_data (
             dim< 2 > size_,
             std::initializer_list< detail::input_triple< ValueType, IndexType >> nonzeros_ )
[inline]
```

Initializes the structure from a list of nonzeros.

**Parameters**

| | |
|---|---|
| *size_* | dimensions of the matrix |
| *nonzeros_* | list of nonzero elements |

```
208         :  size{size_}, nonzeros()
209     {
210         nonzeros.reserve(nonzeros_.size());
211         for (const auto& elem :  nonzeros_) {
212             nonzeros.emplace_back(elem.row, elem.col, elem.val);
213         }
214     }
```

### 48.158.2.5 matrix_data() [5/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix_data< ValueType, IndexType >::matrix_data (
             dim< 2 > size_,
             const matrix_data< ValueType, IndexType > & block )  [inline]
```

Initializes a matrix out of a matrix block via duplication.

**Parameters**

| | |
|---|---|
| *size_* | size of the block-matrix (in blocks) |
| *block* | matrix block used to fill the complete matrix |

```
223         :  size{size_ * block.size}
224     {
225         nonzeros.reserve(size_[0] * size_[1] * block.nonzeros.size());
226         for (size_type row = 0; row < size_[0]; ++row) {
227             for (size_type col = 0; col < size_[1]; ++col) {
228                 for (const auto& elem :  block.nonzeros) {
229                     nonzeros.emplace_back(row * block.size[0] + elem.row,
230                                           col * block.size[1] + elem.column,
231                                           elem.value);
232                 }
233             }
234         }
235         this->sort_row_major();
236     }
```

References gko::matrix_data< ValueType, IndexType >::size.

#### 48.158.2.6 matrix_data() [6/6]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename Accessor >
gko::matrix_data< ValueType, IndexType >::matrix_data (
             const range< Accessor > & data )  [inline]
```

Initializes a matrix from a range.

**Template Parameters**

| | |
|---|---|
| *Accessor* | accessor type of the input range |

**Parameters**

| | |
|---|---|
| *data* | range used to initialize the matrix |

```
247         :  size{data.length(0), data.length(1)}
248     {
249         for (gko::size_type row = 0; row < size[0]; ++row) {
250             for (gko::size_type col = 0; col < size[1]; ++col) {
251                 if (is_nonzero(data(row, col))) {
252                     nonzeros.emplace_back(row, col, data(row, col));
253                 }
254             }
255         }
256     }
```

References gko::range< Accessor >::length().

### 48.158.3 Member Function Documentation

#### 48.158.3.1 cond() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename RandomDistribution , typename RandomEngine >
static matrix_data gko::matrix_data< ValueType, IndexType >::cond (
```

```
            size_type size,
            remove_complex< ValueType > condition_number,
            RandomDistribution && dist,
            RandomEngine && engine )  [inline], [static]
```

Initializes a random dense matrix with a specific condition number.

The matrix is generated by applying a series of random Hausholder reflectors to a diagonal matrix with diagonal entries uniformly distributed between `sqrt(condition_number)` and `1/sqrt(condition_number)`.

This version of the function applies `size - 1` reflectors to each side of the diagonal matrix.

**Template Parameters**

| | |
|---|---|
| *RandomDistribution* | the type of the random distribution |
| *RandomEngine* | the type of the random engine |

**Parameters**

| | |
|---|---|
| *size* | number of rows and columns of the matrix |
| *condition_number* | condition number of the matrix |
| *dist* | random distribution used to generate reflectors |
| *engine* | random engine used to generate reflectors |

**Returns**

the dense matrix with the specified condition number

```
436     {
437         return cond(size, condition_number,
438                     std::forward<RandomDistribution>(dist),
439                     std::forward<RandomEngine>(engine), size - 1);
440     }
```

References gko::matrix_data< ValueType, IndexType >::cond(), and gko::matrix_data< ValueType, IndexType >::size.

### 48.158.3.2 cond() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename RandomDistribution , typename RandomEngine >
static matrix_data gko::matrix_data< ValueType, IndexType >::cond (
            size_type size,
            remove_complex< ValueType > condition_number,
            RandomDistribution && dist,
            RandomEngine && engine,
            size_type num_reflectors )  [inline], [static]
```

Initializes a random dense matrix with a specific condition number.

The matrix is generated by applying a series of random Hausholder reflectors to a diagonal matrix with diagonal entries uniformly distributed between `sqrt(condition_number)` and `1/sqrt(condition_number)`.

**Template Parameters**

| | |
|---|---|
| *RandomDistribution* | the type of the random distribution |
| *RandomEngine* | the type of the random engine |

**Parameters**

| | |
|---|---|
| *size* | number of rows and columns of the matrix |
| *condition_number* | condition number of the matrix |
| *dist* | random distribution used to generate reflectors |
| *engine* | random engine used to generate reflectors |
| *num_reflectors* | number of reflectors to apply from each side |

**Returns**

the dense matrix with the specified condition number

```
393     {
394         using range = range<accessor::row_major<ValueType, 2»;
395         std::vector<ValueType> mtx_data(size * size, zero<ValueType>());
396         std::vector<ValueType> ref_data(size);
397         std::vector<ValueType> work(size);
398         range matrix(mtx_data.data(), size, size, size);
399         range reflector(ref_data.data(), size, 1u, 1u);
400 
401         initialize_diag_with_cond(condition_number, matrix);
402         for (size_type i = 0; i < num_reflectors; ++i) {
403             generate_random_reflector(dist, engine, reflector);
404             reflect_domain(reflector, matrix, work.data());
405             generate_random_reflector(dist, engine, reflector);
406             reflect_range(reflector, matrix, work.data());
407         }
408         return matrix;
409     }
```

References gko::matrix_data< ValueType, IndexType >::size.

Referenced by gko::matrix_data< ValueType, IndexType >::cond().

### 48.158.3.3 diag() [1/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static matrix_data gko::matrix_data< ValueType, IndexType >::diag (
             dim< 2 > size_,
             const matrix_data< ValueType, IndexType > & block )  [inline], [static]
```

Initializes a block-diagonal matrix.

**Parameters**

| | |
|---|---|
| *size_* | the size of the matrix |
| *block* | matrix used to fill diagonal blocks |

**Returns**

the block-diagonal matrix

```
309     {
310         matrix_data res(size_ * block.size);
311         const auto num_blocks = std::min(size_[0], size_[1]);
312         res.nonzeros.reserve(num_blocks * block.nonzeros.size());
313         for (size_type b = 0; b < num_blocks; ++b) {
314             for (const auto& elem :  block.nonzeros) {
315                 res.nonzeros.emplace_back(b * block.size[0] + elem.row,
316                                           b * block.size[1] + elem.column,
317                                           elem.value);
318             }
319         }
320         return res;
321     }
```

References gko::matrix_data< ValueType, IndexType >::nonzeros, and gko::matrix_data< ValueType, IndexType >::size.

#### 48.158.3.4 diag() [2/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static matrix_data gko::matrix_data< ValueType, IndexType >::diag (
             dim< 2 > size_,
             std::initializer_list< ValueType > nonzeros_ )  [inline], [static]
```

Initializes a diagonal matrix using a list of diagonal elements.

**Parameters**

| | |
|---|---|
| *size_* | dimensions of the matrix |
| *nonzeros_* | list of diagonal elements |

**Returns**

the diagonal matrix

```
289     {
290         matrix_data res(size_);
291         res.nonzeros.reserve(nonzeros_.size());
292         int pos = 0;
293         for (auto value :  nonzeros_) {
294             res.nonzeros.emplace_back(pos, pos, value);
295             ++pos;
296         }
297         return res;
298     }
```

References gko::matrix_data< ValueType, IndexType >::nonzeros.

#### 48.158.3.5 diag() [3/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static matrix_data gko::matrix_data< ValueType, IndexType >::diag (
             dim< 2 > size_,
             ValueType value )  [inline], [static]
```

Initializes a diagonal matrix.

**Parameters**

| | |
|---|---|
| *size_* | dimensions of the matrix |
| *value* | value used to fill the elements of the matrix |

**Returns**

the diagonal matrix

```
267     {
268         matrix_data res(size_);
269         if (is_nonzero(value)) {
270             const auto num_nnz = std::min(size_[0], size_[1]);
271             res.nonzeros.reserve(num_nnz);
272             for (size_type i = 0; i < num_nnz; ++i) {
273                 res.nonzeros.emplace_back(i, i, value);
274             }
275         }
276         return res;
277     }
```

References gko::is_nonzero(), and gko::matrix_data< ValueType, IndexType >::nonzeros.

Referenced by gko::matrix_data< ValueType, IndexType >::diag().

### 48.158.3.6 diag() [4/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename ForwardIterator >
static matrix_data gko::matrix_data< ValueType, IndexType >::diag (
            ForwardIterator begin,
            ForwardIterator end )  [inline], [static]
```

Initializes a block-diagonal matrix from a list of diagonal blocks.

**Template Parameters**

| | |
|---|---|
| *ForwardIterator* | type of list iterator |

**Parameters**

| | |
|---|---|
| *begin* | the first iterator of the list |
| *end* | the last iterator of the list |

**Returns**

the block-diagonal matrix with diagonal blocks set to the blocks between begin (inclusive) and end (exclusive)

```
336     {
337         matrix_data res(std::accumulate(
338             begin, end, dim<2>{}, [](dim<2> s, const matrix_data& d) {
339                 return dim<2>{s[0] + d.size[0], s[1] + d.size[1]};
340             }));
341
342         size_type row_offset{};
343         size_type col_offset{};
```

```
344         for (auto it = begin; it != end; ++it) {
345             for (const auto& elem :  it->nonzeros) {
346                 res.nonzeros.emplace_back(row_offset + elem.row,
347                                           col_offset + elem.column, elem.value);
348             }
349             row_offset += it->size[0];
350             col_offset += it->size[1];
351         }
352 
353         return res;
354     }
```

References gko::matrix_data< ValueType, IndexType >::nonzeros.

#### 48.158.3.7 diag() [5/5]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static matrix_data gko::matrix_data< ValueType, IndexType >::diag (
             std::initializer_list< matrix_data< ValueType, IndexType > > blocks )  [inline],
[static]
```

Initializes a block-diagonal matrix from a list of diagonal blocks.

**Parameters**

| | |
|---|---|
| *blocks* | a list of blocks to initialize from |

**Returns**

the block-diagonal matrix with diagonal blocks set to the blocks passed in blocks

```
365     {
366         return diag(begin(blocks), end(blocks));
367     }
```

References gko::matrix_data< ValueType, IndexType >::diag().

#### 48.158.3.8 sum_duplicates()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix_data< ValueType, IndexType >::sum_duplicates ( )  [inline]
```

Sum up all values that refer to the same matrix entry.

The result is sorted in row-major order.

```
491     {
492         this->sort_row_major();
493         std::vector<nonzero_type> new_nonzeros;
494         if (!nonzeros.empty()) {
495             new_nonzeros.emplace_back(nonzeros.front().row,
496                                       nonzeros.front().column,
497                                       zero<ValueType>());
498             for (auto entry :  nonzeros) {
499                 if (entry.row != new_nonzeros.back().row ||
500                     entry.column != new_nonzeros.back().column) {
501                     new_nonzeros.emplace_back(entry.row, entry.column,
502                                               zero<ValueType>());
503                 }
504                 new_nonzeros.back().value += entry.value;
505             }
506             nonzeros = std::move(new_nonzeros);
507         }
508     }
```

References gko::matrix_data< ValueType, IndexType >::nonzeros, and gko::matrix_data< ValueType, IndexType >::sort_row_major().

### 48.158.4 Member Data Documentation

#### 48.158.4.1 nonzeros

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::vector<nonzero_type> gko::matrix_data< ValueType, IndexType >::nonzeros
```

A vector of tuples storing the non-zeros of the matrix.

The first two elements of the tuple are the row index and the column index of a matrix element, and its third element is the value at that position.

Referenced by gko::matrix_data< ValueType, IndexType >::diag(), gko::matrix_data< ValueType, IndexType >::remove_zeros(), gko::matrix_data< ValueType, IndexType >::sort_row_major(), and gko::matrix_data< ValueType, IndexType >::sum_duplicates().

The documentation for this struct was generated from the following file:

- ginkgo/core/base/matrix_data.hpp

## 48.159 gko::matrix_data_entry< ValueType, IndexType > Struct Template Reference

Type used to store nonzeros.

```
#include <ginkgo/core/base/matrix_data.hpp>
```

### 48.159.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**struct gko::matrix_data_entry< ValueType, IndexType >**

Type used to store nonzeros.

The documentation for this struct was generated from the following file:

- ginkgo/core/base/matrix_data.hpp

## 48.160 gko::experimental::reorder::Mc64< ValueType, IndexType > Class Template Reference

MC64 is an algorithm for permuting large entries to the diagonal of a sparse matrix.

```
#include <ginkgo/core/reorder/mc64.hpp>
```

## Public Member Functions

- const parameters_type & get_parameters () const

  *Returns the parameters used to construct the factory.*
- std::unique_ptr< result_type > generate (std::shared_ptr< const LinOp > system_matrix) const

## Static Public Member Functions

- static parameters_type build ()

  *Creates a new parameter_type to set up the factory.*

### 48.160.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::experimental::reorder::Mc64< ValueType, IndexType >**

MC64 is an algorithm for permuting large entries to the diagonal of a sparse matrix.

This approach can increase numerical stability of e.g. an LU factorization without pivoting. Under the assumption of working on a nonsingular square matrix, the algorithm computes a minimum weight perfect matching on a weighted edge bipartite graph of the matrix. It is described in detail in "On Algorithms for Permuting Large Entries to the Diagonal of a Sparse Matrix" (Duff, Koster, 2001, https://doi.org/10.1137/S0895479899358443). There are two strategies for choosing the weights supported:

- Maximizing the product of the absolute values on the diagonal. For this strategy, the weights are computed as $c(i,j) = \log_2(a_i) - \log_2(|a(i,j)|)$ if $a(i,j) \neq 0$ and $c(i,j) = \infty$ otherwise. Here, $a_i$ is the maximum absolute value in row $i$ of the matrix $A$. In this case, the implementation computes a row permutation $P$ and row and column scaling coefficients $L$ and $R$ such that the matrix $PLAR$ has values with unity absolute value on the diagonal and smaller or equal entries everywhere else.
- Maximizing the sum of the absolute values on the diagonal. For this strategy, the weights are computed as $c(i,j) = a_i - |a(i,j)|$ if $a(i,j) \neq 0$ and $c(i,j) = \infty$ otherwise. In this case, no scaling coefficients are computed.

This class creates a Combination of two ScaledPermutations representing the row and column permutation and scaling factors computed by this algorithm.

**Template Parameters**

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

### 48.160.2 Member Function Documentation

#### 48.160.2.1 generate()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<result_type> gko::experimental::reorder::Mc64< ValueType, IndexType >::generate
(
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix to compute the reordering for. |

**Note**

This function overrides the default LinOpFactory::generate to return a Permutation instead of a generic LinOp, which would need to be cast to ScaledPermutation again to access its indices. It is only necessary because smart pointers aren't covariant.

#### 48.160.2.2 get_parameters()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const parameters_type& gko::experimental::reorder::Mc64< ValueType, IndexType >::get_parameters
( ) const  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
114 { return parameters_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/reorder/mc64.hpp

## 48.161 gko::MetisError Class Reference

MetisError is thrown when METIS routine throws an error code.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- MetisError (const std::string &file, int line, const std::string &func, const std::string &error)

  *Initializes a METIS error.*

### 48.161.1 Detailed Description

MetisError is thrown when METIS routine throws an error code.

### 48.161.2 Constructor & Destructor Documentation

#### 48.161.2.1 MetisError()

```
gko::MetisError::MetisError (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & error )  [inline]
```

Initializes a METIS error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the METIS routine that failed |
| *error* | The resulting METIS error name |

```
407         :  Error(file, line, func + ":  " + error)
408     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.162 gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit Class Reference

minimal_storage_limit is a strategy_type which decides the number of stored elements per row of the ell part.

```
#include <ginkgo/core/matrix/hybrid.hpp>
```

### Public Member Functions

- minimal_storage_limit ()

  *Creates a minimal_storage_limit strategy.*
- size_type compute_ell_num_stored_elements_per_row (array< size_type > *row_nnz) const override

  *Computes the number of stored elements per row of the ell part.*
- auto get_percentage () const

  *Get the percent setting.*

## 48.162.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit

minimal_storage_limit is a strategy_type which decides the number of stored elements per row of the ell part.

It is determined by the size of ValueType and IndexType, the storage is the minimum among all partition.

## 48.162.2 Member Function Documentation

### 48.162.2.1 compute_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit::compute_ell_↩
num_stored_elements_per_row (
            array< size_type > * row_nnz ) const  [inline], [override], [virtual]
```

Computes the number of stored elements per row of the ell part.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |

**Returns**

the number of stored elements per row of the ell part

Implements gko::matrix::Hybrid< ValueType, IndexType >::strategy_type.

```
331         {
332             return strategy_.compute_ell_num_stored_elements_per_row(row_nnz);
333         }
```

References gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::compute_ell_num_stored_elements↩
_per_row().

### 48.162.2.2 get_percentage()

```
template<typename ValueType = default_precision, typename IndexType = int32>
auto gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit::get_percentage ( )
const  [inline]
```

Get the percent setting.

**Returns**

percent

```
340 { return strategy_.get_percentage(); }
```

References gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit::get_percentage().

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/hybrid.hpp

# 48.163 gko::solver::Minres< ValueType > Class Template Reference

Minres is an iterative type Krylov subspace method, which is suitable for indefinite and full-rank symmetric/hermitian operators.

`#include <ginkgo/core/solver/minres.hpp>`

## Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.163.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::solver::Minres< ValueType >**

Minres is an iterative type Krylov subspace method, which is suitable for indefinite and full-rank symmetric/hermitian operators.

It is a specialization of the Gmres method for symmetric/hermitian operators, and can be computed using short recurrences, similar to the CG method.

For symmetric $A$, the Arnoldi process collapses to a three-term Lanczos recurrence and the Hessenberg matrix $\bar{H}_m$ becomes tridiagonal $\bar{T}_m$. Minres exploits this structure to minimize the residual norm

$$y_m = \arg\min_{y \in \mathbb{R}^m} \|\beta e_1 - \bar{T}_m y\|_2, \qquad \beta = \|r_0\|_2,$$

The least-squares problem is not re-solved from scratch: a QR factorization of $\bar{T}_m$ is carried along and extended in every iteration. Since $\bar{T}_m$ is tridiagonal, a new column is brought to upper-triangular form by re-applying the two previous Givens rotations and then one freshly computed rotation, so only those rotations, three basis vectors and three update directions have to be kept. Each iteration therefore costs one matrix-vector product and a constant amount of memory, in contrast to GMRES's growing Krylov basis.

The implementation in Ginkgo makes use of the merged kernel to make the best use of data locality. The inner operations in one iteration of Minres are merged into 2 separate steps.

#### References


- Greenbaum, A. *Iterative Methods for Solving Linear Systems.* SIAM Frontiers in Applied Mathematics, 17, 1997. https://doi.org/10.1137/1.9781611970937
- Choi, S.-C. T. *Iterative Methods for Singular Linear Equations and Least-Squares Problems.* PhD thesis, Stanford University, 2006. https://web.stanford.edu/group/SOL/dissertations/sou-cheng-choi-thesis.pdf


#### Note

The Minres solver only reports an approximation of the residual norm directly to the stopping criteria. Neither the actual residual, nor the actual residual norm are reported. Thus, to get the minimal overhead, the gko::stop::ImplicitResidualNorm criteria should be used. The gko::stop::ResidualNorm criteria will require an additional matrix-vector product and global reduction.

#### Template Parameters

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.163.2 Member Function Documentation

### 48.163.2.1 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Minres< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

#### Returns

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.163.2.2 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::Minres< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.163.2.3 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::Minres< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/minres.hpp

## 48.164 gko::MpiError Class Reference

MpiError is thrown when a MPI routine throws a non-zero error code.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- MpiError (const std::string &file, int line, const std::string &func, int64 error_code)

  *Initializes a MPI error.*

### 48.164.1 Detailed Description

MpiError is thrown when a MPI routine throws a non-zero error code.

## 48.164.2 Constructor & Destructor Documentation

### 48.164.2.1 MpiError()

```
gko::MpiError::MpiError (
            const std::string & file,
            int line,
            const std::string & func,
            int64 error_code )  [inline]
```

Initializes a MPI error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the MPI routine that failed |
| *error_code* | The resulting MPI error code |

```
153         :  Error(file, line, func + ":  " + get_error(error_code))
154     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.165 gko::solver::Multigrid Class Reference

Multigrid solves a linear system by combining cheap iterative sweeps (smoothers) on a hierarchy of progressively coarser representations of the operator with corrections transferred between adjacent levels.

`#include <ginkgo/core/solver/multigrid.hpp>`

### Classes

- class reuse_data_type

  *Records the hierarchy of generate_reuse() and the reuse data of its levels, smoothers and coarsest solver.*

### Public Member Functions

- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess or false if multigrid always set the input as zero.*
- std::vector< std::shared_ptr< const gko::multigrid::MultigridLevel > > get_mg_level_list () const

  *Gets the list of MultigridLevel operators.*
- std::vector< std::shared_ptr< const LinOp > > get_pre_smoother_list () const

  *Gets the list of pre-smoother operators.*
- std::vector< std::shared_ptr< const LinOp > > get_mid_smoother_list () const

  *Gets the list of mid-smoother operators.*
- std::vector< std::shared_ptr< const LinOp > > get_post_smoother_list () const

  *Gets the list of post-smoother operators.*
- std::shared_ptr< const LinOp > get_coarsest_solver () const

  *Gets the operator at the coarsest level.*
- multigrid::cycle get_cycle () const

  *Get the cycle of multigrid.*
- void set_cycle (multigrid::cycle cycle)

  *Set the cycle of multigrid.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor<>())

  *Create the parameters from the property_tree.*

## 48.165.1 Detailed Description

Multigrid solves a linear system by combining cheap iterative sweeps (smoothers) on a hierarchy of progressively coarser representations of the operator with corrections transferred between adjacent levels.

A smoother on the fine level removes the high-frequency components of the error quickly; the remaining low-frequency error is well-resolved on a coarser space, where it is computed by a cheaper solve — or, recursively, by another multigrid call. The correction is prolongated back to the fine level and a second smoothing sweep removes any high-frequency error introduced by the transfer. Because each coarser level holds only a fraction of the unknowns of the next, the total work per cycle stays close to that of a single fine-level matrix-vector apply.

Each level $\ell$ holds a system matrix $A_\ell$, a smoother $S_\ell$, a restriction operator $R_\ell : \mathbb{R}^{n_\ell} \to \mathbb{R}^{n_{\ell+1}}$ and a prolongation operator $P_\ell : \mathbb{R}^{n_{\ell+1}} \to \mathbb{R}^{n_\ell}$. The Galerkin coarse operator is $A_{\ell+1} = R_\ell A_\ell P_\ell$. One V-cycle step at level $\ell$ on a fine-level system $A_\ell x = b$ is:

$$
\begin{aligned}
x &\leftarrow x + S_\ell^{\text{pre}}(b - A_\ell x) && \text{(pre-smooth)},\\
r_{\ell+1} &= R_\ell(b - A_\ell x) && \text{(restrict)},\\
e_{\ell+1} &\approx A_{\ell+1}^{-1} r_{\ell+1} && \text{(recursive coarse solve)},\\
x &\leftarrow x + P_\ell e_{\ell+1} && \text{(prolongate and correct)},\\
x &\leftarrow x + S_\ell^{\text{post}}(b - A_\ell x) && \text{(post-smooth)}.
\end{aligned}
$$

At the coarsest level the recursive call is replaced by a direct or iterative coarse solver. W- and F-cycles differ from V only in how many recursive coarse-solves are performed per level (see the `cycle` enum above).

Ginkgo uses the index from 0 for finest level (original problem size) ∼ N for the coarsest level (the coarsest solver), and its level counts is N (N multigrid level generation).

**References**

- Briggs, W. L., Henson, V. E., McCormick, S. F. *A Multigrid Tutorial.* 2nd ed. SIAM, 2000. https://doi.org/10.1137/1.9780898719505
- Trottenberg, U., Oosterlee, C. W., Schüller, A. *Multigrid.* 1st ed. Academic Press, 2001. ISBN 978-0-12-701070-0.

The factory supports LinOpFactory::generate_reuse(): the recorded hierarchy is reused as long as its levels reproduce their coarse sizes, and levels, smoothers and the coarsest solver are generated through their own generate_reuse(). Below the first level that comes out different, everything is generated anew. Levels such as Pgm keep choices made from the values of the first matrix, so it should be representative of the later ones.

## 48.165.2 Member Function Documentation

### 48.165.2.1 apply_uses_initial_guess()

```
bool gko::solver::Multigrid::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess or false if multigrid always set the input as zero.

**Returns**

bool it is related to parameters variable zero_guess

```
165     {
166         return this->get_default_initial_guess() ==
167                initial_guess_mode::provided;
168     }
```

References gko::solver::provided.

#### 48.165.2.2 get_coarsest_solver()

```
std::shared_ptr<const LinOp> gko::solver::Multigrid::get_coarsest_solver ( ) const  [inline]
```

Gets the operator at the coarsest level.

**Returns**

the coarsest operator

```
217     {
218         return coarsest_solver_;
219     }
```

#### 48.165.2.3 get_cycle()

```
multigrid::cycle gko::solver::Multigrid::get_cycle ( ) const  [inline]
```

Get the cycle of multigrid.

**Returns**

the multigrid::cycle

```
226 { return parameters_.cycle; }
```

#### 48.165.2.4 get_mg_level_list()

```
std::vector<std::shared_ptr<const gko::multigrid::MultigridLevel> > gko::solver::Multigrid↩
::get_mg_level_list ( ) const  [inline]
```

Gets the list of MultigridLevel operators.

**Returns**

the list of MultigridLevel operators

```
177     {
178         return mg_level_list_;
179     }
```

#### 48.165.2.5 get_mid_smoother_list()

```
std::vector<std::shared_ptr<const LinOp> > gko::solver::Multigrid::get_mid_smoother_list ( )
const  [inline]
```

Gets the list of mid-smoother operators.

**Returns**

the list of mid-smoother operators

```
197     {
198         return mid_smoother_list_;
199     }
```

### 48.165.2.6 get_post_smoother_list()

```
std::vector<std::shared_ptr<const LinOp> > gko::solver::Multigrid::get_post_smoother_list ( )
const  [inline]
```

Gets the list of post-smoother operators.

#### Returns

the list of post-smoother operators

```
207     {
208         return post_smoother_list_;
209     }
```

### 48.165.2.7 get_pre_smoother_list()

```
std::vector<std::shared_ptr<const LinOp> > gko::solver::Multigrid::get_pre_smoother_list ( )
const  [inline]
```

Gets the list of pre-smoother operators.

#### Returns

the list of pre-smoother operators

```
187     {
188         return pre_smoother_list_;
189     }
```

### 48.165.2.8 parse()

```
static parameters_type gko::solver::Multigrid::parse (
             const config::pnode & config,
             const config::registry & context,
             const config::type_descriptor & td_for_child = config::make_type_descriptor<>() )
[static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

#### Parameters

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs |

**Returns**

parameters

**Note**

Multigrid does not support the selector option in file config

#### 48.165.2.9 set_cycle()

```
void gko::solver::Multigrid::set_cycle (
            multigrid::cycle cycle )  [inline]
```

Set the cycle of multigrid.

**Parameters**

| | |
|---|---|
| *multigrid::cycle* | the new cycle |

```
233 { parameters_.cycle = cycle; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/solver/multigrid.hpp

## 48.166 gko::multigrid::MultigridLevel Class Reference

This class represents two levels in a multigrid hierarchy.

`#include <ginkgo/core/multigrid/multigrid_level.hpp>`

### Public Member Functions

- virtual std::shared_ptr< const LinOp > get_fine_op () const =0
  *Returns the operator on fine level.*
- virtual std::shared_ptr< const LinOp > get_restrict_op () const =0
  *Returns the restrict operator.*
- virtual std::shared_ptr< const LinOp > get_coarse_op () const =0
  *Returns the operator on coarse level.*
- virtual std::shared_ptr< const LinOp > get_prolong_op () const =0
  *Returns the prolong operator.*

### 48.166.1 Detailed Description

This class represents two levels in a multigrid hierarchy.

The MultigridLevel is an interface that allows to get the individual components of multigrid level. Each implementation of a multigrid level should inherit from this interface. Use EnableMultigridLevel<ValueType> to implement this interface with composition by default.

## 48.166.2 Member Function Documentation

### 48.166.2.1 get_coarse_op()

```
virtual std::shared_ptr<const LinOp> gko::multigrid::MultigridLevel::get_coarse_op ( ) const
[pure virtual]
```

Returns the operator on coarse level.

**Returns**

the operator on coarse level.

Implemented in gko::multigrid::EnableMultigridLevel< ValueType >.

### 48.166.2.2 get_fine_op()

```
virtual std::shared_ptr<const LinOp> gko::multigrid::MultigridLevel::get_fine_op ( ) const
[pure virtual]
```

Returns the operator on fine level.

**Returns**

the operator on fine level.

Implemented in gko::multigrid::EnableMultigridLevel< ValueType >.

### 48.166.2.3 get_prolong_op()

```
virtual std::shared_ptr<const LinOp> gko::multigrid::MultigridLevel::get_prolong_op ( ) const
[pure virtual]
```

Returns the prolong operator.

**Returns**

the prolong operator.

Implemented in gko::multigrid::EnableMultigridLevel< ValueType >.

#### 48.166.2.4 get_restrict_op()

```
virtual std::shared_ptr<const LinOp> gko::multigrid::MultigridLevel::get_restrict_op ( ) const
[pure virtual]
```

Returns the restrict operator.

**Returns**

the restrict operator.

Implemented in gko::multigrid::EnableMultigridLevel< ValueType >.

The documentation for this class was generated from the following file:

- ginkgo/core/multigrid/multigrid_level.hpp

## 48.167 gko::matrix::Csr< ValueType, IndexType >::multiply_add_reuse_info Class Reference

Class describing the internal lookup structures created by multiply_add_reuse to recompute a sparse matrix-matrix product with updated values.

```
#include <ginkgo/core/matrix/csr.hpp>
```

### Public Member Functions

- void update_values (ptr_param< const Csr > mtx, ptr_param< const Dense< value_type >> scale_mult, ptr_param< const Csr > mtx_mult, ptr_param< const Dense< value_type >> scale_add, ptr_param< const Csr > mtx_add, ptr_param< Csr > out) const

  *Recomputes the sparse matrix-matrix product* `out = scale_mult * mtx * mtx_mult + scale_add * mtx_add` *when only the values of mtx, scale_mult, mtx_mult, scale_add, mtx_add changed, but the sparsity patterns of mtx, mtx_mult, mtx_add and out are unchanged.*

### 48.167.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Csr< ValueType, IndexType >::multiply_add_reuse_info**

Class describing the internal lookup structures created by multiply_add_reuse to recompute a sparse matrix-matrix product with updated values.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/csr.hpp

# 48.168 gko::matrix::Csr< ValueType, IndexType >::multiply_reuse_info Class Reference

Class describing the internal lookup structures created by multiply_reuse(const Csr*) to recompute a sparse matrix-matrix product with updated values.

`#include <ginkgo/core/matrix/csr.hpp>`

## Public Member Functions

- void update_values (ptr_param< const Csr > mtx1, ptr_param< const Csr > mtx2, ptr_param< Csr > out) const

  *Recomputes the sparse matrix-matrix product `out = mtx1 * mtx2` when only the values of mtx1 and mtx2 changed, but the sparsity patterns of mtx1, mtx2 and out are unchanged.*

### 48.168.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Csr< ValueType, IndexType >::multiply_reuse_info**

Class describing the internal lookup structures created by multiply_reuse(const Csr*) to recompute a sparse matrix-matrix product with updated values.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/csr.hpp

# 48.169 gko::batch::MultiVector< ValueType > Class Template Reference

MultiVector stores multiple vectors in a batched fashion and is useful for batched operations.

`#include <ginkgo/core/base/batch_multi_vector.hpp>`

## Public Member Functions

- std::unique_ptr< unbatch_type > create_view_for_item (size_type item_id)

  *Creates a mutable view (of matrix::Dense type) of one item of the Batch MultiVector object.*
- std::unique_ptr< const unbatch_type > create_const_view_for_item (size_type item_id) const

  *Creates a mutable view (of matrix::Dense type) of one item of the Batch MultiVector object.*
- batch_dim< 2 > get_size () const

  *Returns the batch size.*
- size_type get_num_batch_items () const

  *Returns the number of batch items.*
- dim< 2 > get_common_size () const

  *Returns the common size of the batch items.*
- value_type * get_values () noexcept

  *Returns a pointer to the array of values of the multi-vector.*

- const value_type * get_const_values () const noexcept

  *Returns a pointer to the array of values of the multi-vector.*
- value_type * get_values_for_item (size_type batch_id) noexcept

  *Returns a pointer to the array of values of the multi-vector for a specific batch item.*
- const value_type * get_const_values_for_item (size_type batch_id) const noexcept

  *Returns a pointer to the array of values of the multi-vector for a specific batch item.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.*
- size_type get_cumulative_offset (size_type batch_id) const

  *Get the cumulative storage size offset.*
- value_type & at (size_type batch_id, size_type row, size_type col)

  *Returns a single element for a particular batch item.*
- value_type at (size_type batch_id, size_type row, size_type col) const

  *Returns a single element for a particular batch item.*
- ValueType & at (size_type batch_id, size_type idx) noexcept

  *Returns a single element for a particular batch item.*
- ValueType at (size_type batch_id, size_type idx) const noexcept

  *Returns a single element for a particular batch item.*
- void scale (ptr_param< const MultiVector< ValueType >> alpha)

  *Scales the vector with a scalar (aka: BLAS scal).*
- void add_scaled (ptr_param< const MultiVector< ValueType >> alpha, ptr_param< const MultiVector< ValueType >> b)

  *Adds `b` scaled by `alpha` to the vector (aka: BLAS axpy).*
- void compute_dot (ptr_param< const MultiVector< ValueType >> b, ptr_param< MultiVector< ValueType >> result) const

  *Computes the column-wise dot product of each multi-vector in this batch and its corresponding entry in `b`.*
- void compute_conj_dot (ptr_param< const MultiVector< ValueType >> b, ptr_param< MultiVector< ValueType >> result) const

  *Computes the column-wise conjugate dot product of each multi-vector in this batch and its corresponding entry in `b`.*
- void compute_norm2 (ptr_param< MultiVector< remove_complex< ValueType >>> result) const

  *Computes the Euclidean (L^2) norm of each multi-vector in this batch.*
- void fill (ValueType value)

  *Fills the input MultiVector with a given value.*

## Static Public Member Functions

- static std::unique_ptr< MultiVector > create_with_config_of (ptr_param< const MultiVector > other)

  *Creates a MultiVector with the configuration of another MultiVector.*
- static std::unique_ptr< MultiVector > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size=batch_dim< 2 >{})

  *Creates an uninitialized multi-vector of the specified size.*
- static std::unique_ptr< MultiVector > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, array< value_type > values)

  *Creates a MultiVector from an already allocated (and initialized) array.*
- template<typename InputValueType >
  static std::unique_ptr< MultiVector > create (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &size, std::initializer_list< InputValueType > values)

  *create(std::shared_ptr<const*
- static std::unique_ptr< const MultiVector > create_const (std::shared_ptr< const Executor > exec, const batch_dim< 2 > &sizes, gko::detail::const_array_view< ValueType > &&values)

  *Creates a constant (immutable) batch multi-vector from a constant array.*

## 48.169.1 Detailed Description

template<typename ValueType = default_precision>
class gko::batch::MultiVector< ValueType >

MultiVector stores multiple vectors in a batched fashion and is useful for batched operations.

For example, two batch items, each a 3 x 2 multi-vector, would be laid out as:

```
batch item 0      batch item 1
  1 2               3 4
  1 2               3 4
  1 2               3 4
```

In memory, both items are stored consecutively as a single row-major array:

```
[ 1 2  1 2  1 2 | 3 4  3 4  3 4 ]
  item 0          item 1
```

The accessor `at()` can be used to reach an individual entry of a specific batch item.

The values of the different batch items are stored consecutively and in each batch item, the multi-vectors are stored in a row-major fashion.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of multi-vector elements |

## 48.169.2 Member Function Documentation

### 48.169.2.1 add_scaled()

```
template<typename ValueType = default_precision>
void gko::batch::MultiVector< ValueType >::add_scaled (
             ptr_param< const MultiVector< ValueType >> alpha,
             ptr_param< const MultiVector< ValueType >> b )
```

Adds `b` scaled by `alpha` to the vector (aka: BLAS axpy).

**Parameters**

| | |
|---|---|
| *alpha* | the scalar |
| *b* | a multi-vector of the same dimension as this |

**Note**

If alpha is 1x1 MultiVector matrix, the entire multi-vector (all batches) is scaled by alpha. If it is a MultiVector row vector of values, then i-th column of the vector is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of columns of the multi-vector).

#### 48.169.2.2 at() [1/4]

```
template<typename ValueType = default_precision>
ValueType gko::batch::MultiVector< ValueType >::at (
              size_type batch_id,
              size_type idx ) const  [inline], [noexcept]
```

Returns a single element for a particular batch item.

Useful for iterating across all elements of the vector. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the batch item index to be queried |
| *idx* | a linear index of the requested element |

**Note**

the method has to be called on the same Executor the vector is stored at (e.g. trying to call this method on a GPU multi-vector from the OMP results in a runtime error)

```
285     {
286         return values_.get_const_data()[linearize_index(batch_id, idx)];
287     }
```

References gko::array< ValueType >::get_const_data().

#### 48.169.2.3 at() [2/4]

```
template<typename ValueType = default_precision>
ValueType& gko::batch::MultiVector< ValueType >::at (
              size_type batch_id,
              size_type idx )  [inline], [noexcept]
```

Returns a single element for a particular batch item.

Useful for iterating across all elements of the vector. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *batch↩ _id* | the batch item index to be queried |
| *idx* | a linear index of the requested element |

**Note**

the method has to be called on the same Executor the vector is stored at (e.g. trying to call this method on a GPU multi-vector from the OMP results in a runtime error)

```
277     {
278         return values_.get_data()[linearize_index(batch_id, idx)];
279     }
```

References gko::array< ValueType >::get_data().

#### 48.169.2.4 at() [3/4]

```
template<typename ValueType = default_precision>
value_type& gko::batch::MultiVector< ValueType >::at (
              size_type batch_id,
              size_type row,
              size_type col )  [inline]
```

Returns a single element for a particular batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the batch item index to be queried |
| *row* | the row of the requested element |
| *col* | the column of the requested element |

**Note**

the method has to be called on the same Executor the vector is stored at (e.g. trying to call this method on a GPU multi-vector from the OMP results in a runtime error)

```
248     {
249         GKO_ASSERT(batch_id < this->get_num_batch_items());
250         return values_.get_data()[linearize_index(batch_id, row, col)];
251     }
```

References gko::array< ValueType >::get_data(), and gko::batch::MultiVector< ValueType >::get_num_batch_items().

#### 48.169.2.5 at() [4/4]

```
template<typename ValueType = default_precision>
value_type gko::batch::MultiVector< ValueType >::at (
              size_type batch_id,
              size_type row,
              size_type col ) const  [inline]
```

Returns a single element for a particular batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the batch item index to be queried |
| *row* | the row of the requested element |
| *col* | the column of the requested element |

**Note**

the method has to be called on the same Executor the vector is stored at (e.g. trying to call this method on a GPU multi-vector from the OMP results in a runtime error)

```
257     {
258         GKO_ASSERT(batch_id < this->get_num_batch_items());
259         return values_.get_const_data()[linearize_index(batch_id, row, col)];
260     }
```

References gko::array< ValueType >::get_const_data(), and gko::batch::MultiVector< ValueType >::get_num_batch_items().

### 48.169.2.6 compute_conj_dot()

```
template<typename ValueType = default_precision>
void gko::batch::MultiVector< ValueType >::compute_conj_dot (
            ptr_param< const MultiVector< ValueType >> b,
            ptr_param< MultiVector< ValueType >> result ) const
```

Computes the column-wise conjugate dot product of each multi-vector in this batch and its corresponding entry in b.

If the vector has complex value_type, then the conjugate of this is taken.

**Parameters**

| | |
|---|---|
| *b* | a MultiVector of same dimension as this |
| *result* | a MultiVector row vector, used to store the dot product (the number of column in the vector must match the number of columns of this) |

### 48.169.2.7 compute_dot()

```
template<typename ValueType = default_precision>
void gko::batch::MultiVector< ValueType >::compute_dot (
            ptr_param< const MultiVector< ValueType >> b,
            ptr_param< MultiVector< ValueType >> result ) const
```

Computes the column-wise dot product of each multi-vector in this batch and its corresponding entry in b.

**Parameters**

| | |
|---|---|
| *b* | a MultiVector of same dimension as this |
| *result* | a MultiVector row vector, used to store the dot product |

### 48.169.2.8 compute_norm2()

```
template<typename ValueType = default_precision>
```

```
void gko::batch::MultiVector< ValueType >::compute_norm2 (
            ptr_param< MultiVector< remove_complex< ValueType >>> result ) const
```

Computes the Euclidean (L^2) norm of each multi-vector in this batch.

**Parameters**

| | |
|---|---|
| *result* | a MultiVector, used to store the norm (the number of columns in the vector must match the number of columns of this) |

### 48.169.2.9 create() [1/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<MultiVector> gko::batch::MultiVector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size,
            array< value_type > values )  [static]
```

Creates a MultiVector from an already allocated (and initialized) array.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the vector |
| *size* | sizes of the batch matrices in a batch_dim object |
| *values* | array of values |

**Note**

If `values` is not an rvalue, not an array of ValueType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the vector.

### 48.169.2.10 create() [2/3]

```
template<typename ValueType = default_precision>
template<typename InputValueType >
static std::unique_ptr<MultiVector> gko::batch::MultiVector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size,
            std::initializer_list< InputValueType > values )  [inline], [static]
```

create(std::shared_ptr<const

create(std::shared_ptr<const executor>="">, const batch_dim<2>&, array<value_type>)

```
394     {
395         return create(exec, size, array<index_type>{exec, std::move(values)});
396     }
```

References gko::batch::MultiVector< ValueType >::create().

#### 48.169.2.11 create() [3/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<MultiVector> gko::batch::MultiVector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & size = batch_dim< 2 >{} )  [static]
```

Creates an uninitialized multi-vector of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the vector |
| *size* | size of the batch multi vector |

**Returns**

A smart pointer to the newly created matrix.

Referenced by gko::batch::MultiVector< ValueType >::create().

#### 48.169.2.12 create_const()

```
template<typename ValueType = default_precision>
static std::unique_ptr<const MultiVector> gko::batch::MultiVector< ValueType >::create_const
(
            std::shared_ptr< const Executor > exec,
            const batch_dim< 2 > & sizes,
            gko::detail::const_array_view< ValueType > && values )  [static]
```

Creates a constant (immutable) batch multi-vector from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the vector on |
| *sizes* | the dimensions of the vector |
| *values* | the value array of the vector |
| *stride* | the row-stride of the vector |

**Returns**

A smart pointer to the constant multi-vector wrapping the input array (if it resides on the same executor as the vector) or a copy of the array on the correct executor.

#### 48.169.2.13 create_const_view_for_item()

```
template<typename ValueType = default_precision>
std::unique_ptr<const unbatch_type> gko::batch::MultiVector< ValueType >::create_const_view↩
```

```
_for_item (
            size_type item_id ) const
```

Creates a mutable view (of matrix::Dense type) of one item of the Batch MultiVector object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a matrix::Dense object with the data from the batch item at the given index.

### 48.169.2.14 create_view_for_item()

```
template<typename ValueType = default_precision>
std::unique_ptr<unbatch_type> gko::batch::MultiVector< ValueType >::create_view_for_item (
            size_type item_id )
```

Creates a mutable view (of matrix::Dense type) of one item of the Batch MultiVector object.

Does not perform any deep copies, but only returns a view of the data.

**Parameters**

| | |
|---|---|
| *item↩ _id* | The index of the batch item |

**Returns**

a matrix::Dense object with the data from the batch item at the given index.

### 48.169.2.15 create_with_config_of()

```
template<typename ValueType = default_precision>
static std::unique_ptr<MultiVector> gko::batch::MultiVector< ValueType >::create_with_↩
config_of (
            ptr_param< const MultiVector< ValueType > > other )  [static]
```

Creates a MultiVector with the configuration of another MultiVector.

**Parameters**

| | |
|---|---|
| *other* | The other multi-vector whose configuration needs to copied. |

#### 48.169.2.16 fill()

```
template<typename ValueType = default_precision>
void gko::batch::MultiVector< ValueType >::fill (
            ValueType value )
```

Fills the input MultiVector with a given value.

**Parameters**

| | |
|---|---|
| *value* | the value to be filled |

#### 48.169.2.17 get_common_size()

```
template<typename ValueType = default_precision>
dim<2> gko::batch::MultiVector< ValueType >::get_common_size ( ) const  [inline]
```

Returns the common size of the batch items.

**Returns**

the common size stored

```
162 { return batch_size_.get_common_size(); }
```

References gko::batch_dim< Dimensionality, DimensionType >::get_common_size().

Referenced by gko::batch::MultiVector< ValueType >::get_cumulative_offset().

#### 48.169.2.18 get_const_values()

```
template<typename ValueType = default_precision>
const value_type* gko::batch::MultiVector< ValueType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns a pointer to the array of values of the multi-vector.

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
179     {
180         return values_.get_const_data();
181     }
```

References gko::array< ValueType >::get_const_data().

### 48.169.2.19 get_const_values_for_item()

```
template<typename ValueType = default_precision>
const value_type* gko::batch::MultiVector< ValueType >::get_const_values_for_item (
            size_type batch_id ) const  [inline], [noexcept]
```

Returns a pointer to the array of values of the multi-vector for a specific batch item.

**Parameters**

| | |
|---|---|
| *batch_id* | the id of the batch item. |

**Returns**

the pointer to the array of values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
206     {
207         GKO_ASSERT(batch_id < this->get_num_batch_items());
208         return values_.get_const_data() + this->get_cumulative_offset(batch_id);
209     }
```

References gko::array< ValueType >::get_const_data(), gko::batch::MultiVector< ValueType >::get_cumulative_offset(), and gko::batch::MultiVector< ValueType >::get_num_batch_items().

### 48.169.2.20 get_cumulative_offset()

```
template<typename ValueType = default_precision>
size_type gko::batch::MultiVector< ValueType >::get_cumulative_offset (
            size_type batch_id ) const  [inline]
```

Get the cumulative storage size offset.

**Parameters**

| | |
|---|---|
| *batch_id* | the batch id |

**Returns**

the cumulative offset

```
231     {
232         return batch_id * this->get_common_size()[0] *
233                this->get_common_size()[1];
234     }
```

References gko::batch::MultiVector< ValueType >::get_common_size().

Referenced by gko::batch::MultiVector< ValueType >::get_const_values_for_item(), and gko::batch::MultiVector< ValueType >::get_values_for_item().

#### 48.169.2.21 get_num_batch_items()

```
template<typename ValueType = default_precision>
size_type gko::batch::MultiVector< ValueType >::get_num_batch_items ( ) const  [inline]
```

Returns the number of batch items.

**Returns**

the number of batch items

```
153     {
154         return batch_size_.get_num_batch_items();
155     }
```

References gko::batch_dim< Dimensionality, DimensionType >::get_num_batch_items().

Referenced by gko::batch::MultiVector< ValueType >::at(), gko::batch::MultiVector< ValueType >::get_const_↩ values_for_item(), and gko::batch::MultiVector< ValueType >::get_values_for_item().

#### 48.169.2.22 get_num_stored_elements()

```
template<typename ValueType = default_precision>
size_type gko::batch::MultiVector< ValueType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the batch matrix, cumulative across all the batch items.

**Returns**

the number of elements explicitly stored in the vector, cumulative across all the batch items

```
219     {
220         return values_.get_size();
221     }
```

References gko::array< ValueType >::get_size().

#### 48.169.2.23 get_size()

```
template<typename ValueType = default_precision>
batch_dim<2> gko::batch::MultiVector< ValueType >::get_size ( ) const  [inline]
```

Returns the batch size.

**Returns**

the batch size

```
145 { return batch_size_; }
```

### 48.169.2.24 get_values()

```
template<typename ValueType = default_precision>
value_type* gko::batch::MultiVector< ValueType >::get_values ( )  [inline], [noexcept]
```

Returns a pointer to the array of values of the multi-vector.

#### Returns

the pointer to the array of values

```
169 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

### 48.169.2.25 get_values_for_item()

```
template<typename ValueType = default_precision>
value_type* gko::batch::MultiVector< ValueType >::get_values_for_item (
            size_type batch_id )  [inline], [noexcept]
```

Returns a pointer to the array of values of the multi-vector for a specific batch item.

#### Parameters

| | |
|---|---|
| *batch_id* | the id of the batch item. |

#### Returns

the pointer to the array of values

```
192     {
193         GKO_ASSERT(batch_id < this->get_num_batch_items());
194         return values_.get_data() + this->get_cumulative_offset(batch_id);
195     }
```

References gko::batch::MultiVector< ValueType >::get_cumulative_offset(), gko::array< ValueType >::get_data(), and gko::batch::MultiVector< ValueType >::get_num_batch_items().

### 48.169.2.26 scale()

```
template<typename ValueType = default_precision>
void gko::batch::MultiVector< ValueType >::scale (
            ptr_param< const MultiVector< ValueType >> alpha )
```

Scales the vector with a scalar (aka: BLAS scal).

**Parameters**

| | |
|---|---|
| *alpha* | the scalar |

**Note**

If alpha is 1x1 MultiVector matrix, the entire multi-vector (all batches) is scaled by alpha. If it is a MultiVector row vector of values, then i-th column of the vector is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of columns of the multi-vector). If it is a MultiVector of the same size as this, then an element wise scaling is performed.

The documentation for this class was generated from the following file:

- ginkgo/core/base/batch_multi_vector.hpp

## 48.170 gko::experimental::mpi::NeighborhoodCommunicator Class Reference

A CollectiveCommunicator that uses a neighborhood topology.

`#include <ginkgo/core/distributed/neighborhood_communicator.hpp>`

### Public Member Functions

- NeighborhoodCommunicator (communicator base)

  *Default constructor with empty communication pattern.*
- template<typename LocalIndexType , typename GlobalIndexType >
  NeighborhoodCommunicator (communicator base, const distributed::index_map< LocalIndexType, GlobalIndexType > &imap)

  *Create a NeighborhoodCommunicator from an index map.*
- std::unique_ptr< CollectiveCommunicator > create_with_same_type (communicator base, index_map_ptr imap) const override

  *Creates a new CollectiveCommunicator with the same dynamic type.*
- std::unique_ptr< CollectiveCommunicator > create_inverse () const override

  *Creates the inverse NeighborhoodCommunicator by switching sources and destinations.*
- std::pair< std::unique_ptr< CollectiveCommunicator >, std::vector< comm_index_type > > resize (std::shared_ptr< const Executor > exec, const std::vector< comm_index_type > &send_factors) const override

  *Create new communicator that sends and receives a different number of elements to/from each rank in its current pattern.*
- comm_index_type get_recv_size () const override

  *Get the number of elements received by this process within this communication pattern.*
- comm_index_type get_send_size () const override

  *Get the number of elements sent by this process within this communication pattern.*
- template<typename SendType , typename RecvType >
  request i_all_to_all_v (std::shared_ptr< const Executor > exec, const SendType *send_buffer, RecvType *recv_buffer) const

  *Non-blocking all-to-all communication.*
- request i_all_to_all_v (std::shared_ptr< const Executor > exec, const void *send_buffer, MPI_Datatype send_type, void *recv_buffer, MPI_Datatype recv_type) const

## Friends

- bool operator== (const NeighborhoodCommunicator &a, const NeighborhoodCommunicator &b)

  *Compares two communicators for equality locally.*
- bool operator!= (const NeighborhoodCommunicator &a, const NeighborhoodCommunicator &b)

  *Compares two communicators for inequality.*

## Additional Inherited Members

### 48.170.1 Detailed Description

A CollectiveCommunicator that uses a neighborhood topology.

The neighborhood communicator is defined by a list of neighbors this rank sends data to and a list of neighbors this rank receives data from. No communication with any ranks that is not in one of those lists will take place.

### 48.170.2 Constructor & Destructor Documentation

#### 48.170.2.1 NeighborhoodCommunicator() [1/2]

```
gko::experimental::mpi::NeighborhoodCommunicator::NeighborhoodCommunicator (
            communicator base )  [explicit]
```

Default constructor with empty communication pattern.

**Parameters**

| | |
|---|---|
| *base* | the base communicator |

#### 48.170.2.2 NeighborhoodCommunicator() [2/2]

```
template<typename LocalIndexType , typename GlobalIndexType >
gko::experimental::mpi::NeighborhoodCommunicator::NeighborhoodCommunicator (
            communicator base,
            const distributed::index_map< LocalIndexType, GlobalIndexType > & imap )
```

Create a NeighborhoodCommunicator from an index map.

The receiving neighbors are defined by the remote indices and their owning ranks of the index map. The send neighbors are deduced from that through collective communication.

**Template Parameters**

| | |
|---|---|
| *LocalIndexType* | the local index type of the map |
| *GlobalIndexType* | the global index type of the map |

**Parameters**

| | |
|---|---|
| *base* | the base communicator |
| *imap* | the index map that defines the communication pattern |

## 48.170.3 Member Function Documentation

### 48.170.3.1 create_inverse()

```
std::unique_ptr<CollectiveCommunicator> gko::experimental::mpi::NeighborhoodCommunicator↩
::create_inverse ( ) const  [override], [virtual]
```

Creates the inverse NeighborhoodCommunicator by switching sources and destinations.

**Returns**

CollectiveCommunicator with the inverse communication pattern

Implements gko::experimental::mpi::CollectiveCommunicator.

### 48.170.3.2 create_with_same_type()

```
std::unique_ptr<CollectiveCommunicator> gko::experimental::mpi::NeighborhoodCommunicator↩
::create_with_same_type (
            communicator base,
            index_map_ptr imap ) const  [override], [virtual]
```

Creates a new CollectiveCommunicator with the same dynamic type.

**Parameters**

| | |
|---|---|
| *base* | The base communicator |
| *imap* | The index_map that defines the communication pattern |

**Returns**

a CollectiveCommunicator with the same dynamic type

Implements gko::experimental::mpi::CollectiveCommunicator.

#### 48.170.3.3 get_recv_size()

```
comm_index_type gko::experimental::mpi::NeighborhoodCommunicator::get_recv_size ( ) const
[override], [virtual]
```

Get the number of elements received by this process within this communication pattern.

**Returns**

number of received elements.

Implements gko::experimental::mpi::CollectiveCommunicator.

#### 48.170.3.4 get_send_size()

```
comm_index_type gko::experimental::mpi::NeighborhoodCommunicator::get_send_size ( ) const
[override], [virtual]
```

Get the number of elements sent by this process within this communication pattern.

**Returns**

number of sent elements.

Implements gko::experimental::mpi::CollectiveCommunicator.

#### 48.170.3.5 i_all_to_all_v() [1/2]

```
template<typename SendType , typename RecvType >
request gko::experimental::mpi::CollectiveCommunicator::i_all_to_all_v (
            typename SendType ,
            typename RecvType  )
```

Non-blocking all-to-all communication.

The send_buffer must have allocated at least get_send_size number of elements, and the recv_buffer must have allocated at least get_recv_size number of elements.

**Template Parameters**

| | |
|---|---|
| *SendType* | the type of the elements to send |
| *RecvType* | the type of the elements to receive |

**Parameters**

| | |
|---|---|
| *exec* | the executor for the communication |

**Parameters**

| | |
|---|---|
| *send_buffer* | the send buffer |
| *recv_buffer* | the receive buffer |

**Returns**

a request handle

```
160 {
161     return this->i_all_to_all_v(std::move(exec), send_buffer,
162                                 type_impl<SendType>::get_type(), recv_buffer,
163                                 type_impl<RecvType>::get_type());
164 }
```

#### 48.170.3.6 i_all_to_all_v() [2/2]

```
request gko::experimental::mpi::CollectiveCommunicator::i_all_to_all_v
```

#### 48.170.3.7 resize()

```
std::pair<std::unique_ptr<CollectiveCommunicator>, std::vector<comm_index_type> > gko↩
::experimental::mpi::NeighborhoodCommunicator::resize (
            std::shared_ptr< const Executor > exec,
            const std::vector< comm_index_type > & send_factors ) const  [override], [virtual]
```

Create new communicator that sends and receives a different number of elements to/from each rank in its current pattern.

The number of elements to send is deduces from the send_factors. If this communicator is used to send 2 elements to rank 0, and 3 elements to rank 1, then send_factors needs to have size 5, which is equal to get_send_size(). An send_factors = { 4, 2, 1, 3, 2} means that instead of the first element to send to rank 0, four elements are sent, instead of the second element to send to rank 0, two elements are sent, and so on. A visual representation of the transformation is:

```
[ 1 1 | 1 1 1 ] -> [ 4 2 | 1 3 2] -> [ 1 1 1 1 1 1 | 1 1 1 1 1 1 ]
```

The communication pattern will not change. Even if send_factors is zero for all elements of one rank, the rank will stay in the new pattern.

**Note**

This is a collective call.

**Parameters**

| | |
|---|---|
| *exec* | the executor to use for the new communicator |
| *send_factors* | the new number of elements to send. |

**Returns**

The new communicator and the recv_factors implied by the send_factors

Implements gko::experimental::mpi::CollectiveCommunicator.

### 48.170.4 Friends And Related Function Documentation

#### 48.170.4.1 operator"!=

```
bool operator!= (
            const NeighborhoodCommunicator & a,
            const NeighborhoodCommunicator & b )  [friend]
```

Compares two communicators for inequality.

**See also**

operator==

#### 48.170.4.2 operator==

```
bool operator== (
            const NeighborhoodCommunicator & a,
            const NeighborhoodCommunicator & b )  [friend]
```

Compares two communicators for equality locally.

Equality is defined as having identical or congruent communicators and their communication pattern is equal. No communication is done, i.e. there is no reduction over the local equality check results.

**Returns**

true if both communicators are equal.

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/neighborhood_communicator.hpp

## 48.171 gko::log::ProfilerHook::NestedSummaryWriter Class Reference

Receives the results from ProfilerHook::create_nested_summary().

```
#include <ginkgo/core/log/profiler_hook.hpp>
```

### Public Member Functions

- virtual void write_nested (const nested_summary_entry &root, std::chrono::nanoseconds overhead)=0

  *Callback to write out the summary results.*

### 48.171.1 Detailed Description

Receives the results from ProfilerHook::create_nested_summary().

### 48.171.2 Member Function Documentation

#### 48.171.2.1 write_nested()

```
virtual void gko::log::ProfilerHook::NestedSummaryWriter::write_nested (
            const nested_summary_entry & root,
            std::chrono::nanoseconds overhead )  [pure virtual]
```

Callback to write out the summary results.

**Parameters**

| | |
|---|---|
| *root* | the root range with runtime and count. |
| *overhead* | an estimate of the profiler overhead |

Implemented in gko::log::ProfilerHook::TableSummaryWriter.

The documentation for this class was generated from the following file:

- ginkgo/core/log/profiler_hook.hpp

## 48.172 gko::NotCompiled Class Reference

NotCompiled is thrown when attempting to call an operation which is a part of a module that was not compiled on the system.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- NotCompiled (const std::string &file, int line, const std::string &func, const std::string &module_name)

  *Initializes a NotCompiled error.*

### 48.172.1 Detailed Description

NotCompiled is thrown when attempting to call an operation which is a part of a module that was not compiled on the system.

### 48.172.2 Constructor & Destructor Documentation

#### 48.172.2.1 NotCompiled()

```
gko::NotCompiled::NotCompiled (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & module_name )  [inline]
```

Initializes a NotCompiled error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the function that has not been compiled |
| *module_name* | The name of the module which contains the function |

```
108         :  Error(file, line,
109                  "feature " + func + " is part of the " + module_name +
110                      " module, which is not compiled on this system")
111     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.173 gko::NotImplemented Class Reference

NotImplemented is thrown in case an operation has not yet been implemented (but will be implemented in the future).

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- NotImplemented (const std::string &file, int line, const std::string &func)

  *Initializes a NotImplemented error.*

### 48.173.1 Detailed Description

NotImplemented is thrown in case an operation has not yet been implemented (but will be implemented in the future).

### 48.173.2 Constructor & Destructor Documentation

#### 48.173.2.1 NotImplemented()

```
gko::NotImplemented::NotImplemented (
            const std::string & file,
            int line,
            const std::string & func )  [inline]
```

Initializes a NotImplemented error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the not-yet implemented function |

```
87         :  Error(file, line, func + " is not implemented")
88     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.174 gko::NotSupported Class Reference

NotSupported is thrown in case it is not possible to perform the requested operation on the given object type.

```
#include <ginkgo/core/base/exception.hpp>
```

### Public Member Functions

- NotSupported (const std::string &file, int line, const std::string &func, const std::string &obj_type)

  *Initializes a NotSupported error.*

### 48.174.1 Detailed Description

NotSupported is thrown in case it is not possible to perform the requested operation on the given object type.

### 48.174.2 Constructor & Destructor Documentation

#### 48.174.2.1 NotSupported()

```
gko::NotSupported::NotSupported (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & obj_type )  [inline]
```

Initializes a NotSupported error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the function where the error occurred |
| *obj_type* | The object type on which the requested operation cannot be performed. |

```
132         :  Error(file, line,
133                  "Operation " + func + " does not support parameters of type " +
134                      obj_type)
135     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.175 gko::null_deleter< T > Class Template Reference

This is a deleter that does not delete the object.

`#include <ginkgo/core/base/utils_helper.hpp>`

### Public Member Functions

- void operator() (pointer) const noexcept

  *Deletes the object.*

### 48.175.1 Detailed Description

**template<typename T>**
**class gko::null_deleter< T >**

This is a deleter that does not delete the object.

It is useful where the object has been allocated elsewhere and will be deleted manually.

### 48.175.2 Member Function Documentation

#### 48.175.2.1 operator()()

```
template<typename T >
void gko::null_deleter< T >::operator() (
            pointer  ) const  [inline], [noexcept]
```

Deletes the object.

**Parameters**

| | |
|---|---|
| *ptr* | pointer to the object being deleted |

```
506 {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/utils_helper.hpp

## 48.176 gko::solver::Nullspace< ValueType > Class Template Reference

Nullspace represents the (right or left) nullspace of an operator and *is* the orthogonal projector $P = I - VV^H$ onto its complement, where $V$ is an orthonormal basis of the nullspace.

```
#include <ginkgo/core/solver/nullspace.hpp>
```

### Public Member Functions

- bool contains_constant () const noexcept
- size_type get_dimension () const noexcept
- std::shared_ptr< const LinOp > get_basis () const noexcept
- void project (ptr_param< LinOp > v) const

  *Applies the projector in place: $v \leftarrow (I - VV^H)v$, for each column of $v$ independently.*

### Static Public Member Functions

- static std::unique_ptr< Nullspace > create (std::shared_ptr< const Executor > exec, std::vector< std::shared_ptr< const LinOp >> basis, bool contains_constant=false)

  *Creates a Nullspace from an explicit basis.*
- static std::unique_ptr< Nullspace > create_from_constant (std::shared_ptr< const Executor > exec, dim< 2 > size={})

  *Creates a Nullspace consisting only of the constant vector (e.g.*

## 48.176.1 Detailed Description

template<typename ValueType = default_precision>
class gko::solver::Nullspace< ValueType >

Nullspace represents the (right or left) nullspace of an operator and *is* the orthogonal projector $P = I - VV^H$ onto its complement, where $V$ is an orthonormal basis of the nullspace.

- The constant vector is handled implicitly by subtracting the mean, so a constant-only nullspace works with any vector layout (`matrix::Dense` or `experimental::distributed::Vector` with any partition).
- An explicit basis is copied and orthonormalized once at creation time (twice-iterated Gram-Schmidt, also against the constant if it is part of the nullspace). Columns that are numerically linearly dependent (relative norm below $\sqrt{\epsilon}$ after orthogonalization) are dropped; get_dimension() reports the remaining dimension.
- project() removes the nullspace components of all columns of its input $X$ at once: the coefficients $C = V^H X$ and the column means are computed in a single pass (and, for distributed vectors, a single all-reduce), followed by $X \leftarrow X - VC$.

The typical use is as the `nullspace` / `left_nullspace` parameter of an iterative solver, see enable_preconditioned_iterative_solver_factory_parameters.

The object is immutable after creation apart from internal scratch buffers. Solvers and preconditioner::NullspaceProjection work on their own copy, which shares the basis, so the same object can be passed to any number of factories. Like other Ginkgo LinOps with internal caches, a single instance must not be applied concurrently from multiple threads.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type of the vectors it projects. |

## 48.176.2 Member Function Documentation

### 48.176.2.1 contains_constant()

```
template<typename ValueType = default_precision>
bool gko::solver::Nullspace< ValueType >::contains_constant ( ) const  [noexcept]
```

**Returns**

whether the constant vector is part of the nullspace.

### 48.176.2.2 create()

```
template<typename ValueType = default_precision>
static std::unique_ptr<Nullspace> gko::solver::Nullspace< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            std::vector< std::shared_ptr< const LinOp >> basis,
            bool contains_constant = false )  [static]
```

Creates a Nullspace from an explicit basis.

**Parameters**

| | |
|---|---|
| *exec* | the executor the projector (and its basis) lives on |
| *basis* | the nullspace basis vectors. Each entry is an `n x k_i` matrix::Dense<ValueType> or experimental::distributed::Vector<ValueType> (all entries of the same kind and, if distributed, the same partition); their columns together span the nullspace. They do not need to be orthonormal: they are copied and orthonormalized, dropping numerically dependent columns. |
| *contains_constant* | whether the constant vector is also part of the nullspace. |

#### 48.176.2.3 create_from_constant()

```
template<typename ValueType = default_precision>
static std::unique_ptr<Nullspace> gko::solver::Nullspace< ValueType >::create_from_constant (
            std::shared_ptr< const Executor > exec,
            dim< 2 > size = {} )  [static]
```

Creates a Nullspace consisting only of the constant vector (e.g.

a pure-Neumann Poisson problem). The constant is never materialized, so the result can project vectors of any type and distribution.

Solvers adapt a constant-only nullspace to the size and value type of their system matrix, so the same object can be used in solver factories that are generated for matrices of different sizes (e.g. on the levels of a multigrid hierarchy) or precisions. Without a size, project() accepts vectors of any size; the size is only needed to use the Nullspace as a LinOp.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *size* | the (square, global) size `n x n` of the operator. |

#### 48.176.2.4 get_basis()

```
template<typename ValueType = default_precision>
std::shared_ptr<const LinOp> gko::solver::Nullspace< ValueType >::get_basis ( ) const  [noexcept]
```

**Returns**

the orthonormalized explicit basis as a single `n x k` vector of the same kind as the input basis (orthogonal to the constant if contains_constant()), or nullptr if there is none.

#### 48.176.2.5 get_dimension()

```
template<typename ValueType = default_precision>
size_type gko::solver::Nullspace< ValueType >::get_dimension ( ) const  [noexcept]
```

**Returns**

the dimension of the nullspace: the number of orthonormal explicit basis vectors, plus one if the constant is included.

#### 48.176.2.6 project()

```
template<typename ValueType = default_precision>
void gko::solver::Nullspace< ValueType >::project (
            ptr_param< LinOp > v ) const
```

Applies the projector in place: $v \leftarrow (I - VV^H)v$, for each column of $v$ independently.

**Parameters**

| | |
|---|---|
| *v* | a matrix::Dense or experimental::distributed::Vector with as many rows as this operator (any number of rows for a constant-only nullspace created without a size). Other precisions are converted like in apply(). If the Nullspace has an explicit basis, $v$ must be of the same kind (and distribution) as that basis. |

The documentation for this class was generated from the following file:

- ginkgo/core/solver/nullspace.hpp

## 48.177 gko::solver::nullspace_traits< Solver > Struct Template Reference

Traits describing how an iterative solver handles the nullspaces given by the `nullspace` and `left_nullspace` factory parameters.

```
#include <ginkgo/core/solver/solver_base.hpp>
```

### Static Public Attributes

- static constexpr bool is_supported = false

  *Whether the solver removes the nullspaces.*
- static constexpr bool requires_hermitian = false

  *Whether the solver requires a Hermitian system matrix.*
- static constexpr bool two_sided_projection = false

  *Whether the solver needs the two-sided projection $PMQ$ of its preconditioner $M$, which is symmetric for symmetric problems (e.g.*

### 48.177.1 Detailed Description

**template<typename Solver>**
**struct gko::solver::nullspace_traits< Solver >**

Traits describing how an iterative solver handles the nullspaces given by the `nullspace` and `left_nullspace` factory parameters.

Solvers that support them specialize this class.

**Template Parameters**

| | |
|---|---|
| *Solver* | the solver type |

### 48.177.2 Member Data Documentation

#### 48.177.2.1 is_supported

```
template<typename Solver >
constexpr bool gko::solver::nullspace_traits< Solver >::is_supported = false  [static], [constexpr]
```

Whether the solver removes the nullspaces.

Other solvers throw NotSupported when a nullspace is set instead of ignoring it.

#### 48.177.2.2 requires_hermitian

```
template<typename Solver >
constexpr bool gko::solver::nullspace_traits< Solver >::requires_hermitian = false  [static],
[constexpr]
```

Whether the solver requires a Hermitian system matrix.

Then the left and right nullspace coincide, so setting either sets both.

#### 48.177.2.3 two_sided_projection

```
template<typename Solver >
constexpr bool gko::solver::nullspace_traits< Solver >::two_sided_projection = false  [static],
[constexpr]
```

Whether the solver needs the two-sided projection $PMQ$ of its preconditioner $M$, which is symmetric for symmetric problems (e.g.

for Minres). Otherwise, only the output of the preconditioner is projected, $PM$.

The documentation for this struct was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

# 48.178 gko::preconditioner::NullspaceProjection< ValueType > Class Template Reference

NullspaceProjection wraps a preconditioner $M$ with projections onto the complement of a nullspace, $z = PMQr$.

`#include <ginkgo/core/preconditioner/nullspace_projection.hpp>`

## Public Member Functions

- std::shared_ptr< const LinOp > get_preconditioner () const
- std::shared_ptr< const solver::Nullspace< ValueType > > get_nullspace () const
- std::shared_ptr< const solver::Nullspace< ValueType > > get_left_nullspace () const
- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

## 48.178.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::preconditioner::NullspaceProjection< ValueType >**

NullspaceProjection wraps a preconditioner $M$ with projections onto the complement of a nullspace, $z = PMQr$.

$P$ removes the nullspace from the preconditioned vector, which keeps the search directions of a Krylov solver, and thus its solution, orthogonal to the nullspace. The optional $Q$ removes the left nullspace from the input. If both are the same nullspace, $PMP$ is symmetric for a symmetric $M$, which solvers like Minres need on ill-conditioned problems or when iterated past convergence.

Iterative solvers that support nullspaces (see solver::nullspace_traits) wrap their preconditioner in a NullspaceProjection when their `nullspace` parameter is set, unless it already is one removing the nullspace. It can also be used as the preconditioner of any solver directly; then the right-hand side and the initial guess need to be projected by the user, see solver::Nullspace::project.

A constant-only nullspace is adapted to the size and value type of the system matrix, so the factory can be used for matrices of any size.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type of the vectors |

## 48.178.2 Member Function Documentation

### 48.178.2.1 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::preconditioner::NullspaceProjection< ValueType >::conj_transpose
( ) const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.178.2.2 get_left_nullspace()

```
template<typename ValueType = default_precision>
std::shared_ptr<const solver::Nullspace<ValueType> > gko::preconditioner::NullspaceProjection<
ValueType >::get_left_nullspace ( ) const
```

**Returns**

the left nullspace removed before the preconditioner, or nullptr if there is none

### 48.178.2.3 get_nullspace()

```
template<typename ValueType = default_precision>
std::shared_ptr<const solver::Nullspace<ValueType> > gko::preconditioner::NullspaceProjection<
ValueType >::get_nullspace ( ) const
```

**Returns**

the nullspace removed after the preconditioner, or nullptr if there is none

#### 48.178.2.4 get_preconditioner()

```
template<typename ValueType = default_precision>
std::shared_ptr<const LinOp> gko::preconditioner::NullspaceProjection< ValueType >::get_↩
preconditioner ( ) const
```

**Returns**

the wrapped preconditioner

#### 48.178.2.5 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::preconditioner::NullspaceProjection< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value type in the property tree is ignored. The nullspaces and the generated preconditioner are read from the registry.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.178.2.6 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::preconditioner::NullspaceProjection< ValueType >::transpose ( )
const  [override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/nullspace_projection.hpp

## 48.179 gko::nvidia_device Class Reference

nvidia_device handles the number of executor on Nvidia devices and have the corresponding recursive_mutex.

`#include <ginkgo/core/base/device.hpp>`

### 48.179.1 Detailed Description

nvidia_device handles the number of executor on Nvidia devices and have the corresponding recursive_mutex.

The documentation for this class was generated from the following file:

- ginkgo/core/base/device.hpp

## 48.180 gko::OmpExecutor Class Reference

This is the Executor subclass which represents the OpenMP device (typically CPU).

`#include <ginkgo/core/base/executor.hpp>`

### Public Member Functions

- std::shared_ptr< Executor > get_master () noexcept override

  *Returns the master OmpExecutor of this Executor.*
- std::shared_ptr< const Executor > get_master () const noexcept override

  *Returns the master OmpExecutor of this Executor.*
- void synchronize () const override

  *Synchronize the operations launched on the executor with its master.*
- std::string get_description () const override
- virtual void run (const Operation &op) const=0

  *Runs the specified Operation using this Executor.*
- template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*
- template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩ Dpcpp >
  void run (std::string name, const ClosureReference &op_ref, const ClosureOmp &op_omp, const Closure↩ Cuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

### Static Public Member Functions

- static std::shared_ptr< OmpExecutor > create (std::shared_ptr< CpuAllocatorBase > alloc=std::make_↩ shared< CpuAllocator >())

  *Creates a new OmpExecutor.*

## 48.180.1 Detailed Description

This is the Executor subclass which represents the OpenMP device (typically CPU).

## 48.180.2 Member Function Documentation

### 48.180.2.1 get_description()

```
std::string gko::OmpExecutor::get_description ( ) const  [override], [virtual]
```

**Returns**

a textual representation of the executor and its device.

Implements gko::Executor.

Reimplemented in gko::ReferenceExecutor.

### 48.180.2.2 get_master() [1/2]

```
std::shared_ptr<const Executor> gko::OmpExecutor::get_master ( ) const  [override], [virtual],
[noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

### 48.180.2.3 get_master() [2/2]

```
std::shared_ptr<Executor> gko::OmpExecutor::get_master ( )  [override], [virtual], [noexcept]
```

Returns the master OmpExecutor of this Executor.

**Returns**

the master OmpExecutor of this Executor.

Implements gko::Executor.

### 48.180.2.4 run() [1/3]

```
template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
Dpcpp >
void gko::Executor::run (
            typename ClosureOmp ,
            typename ClosureCuda ,
            typename ClosureHip ,
            typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *op_omp* | functor to run in case of a OmpExecutor or ReferenceExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
660     {
661         LambdaOperation<ClosureOmp, ClosureOmp, ClosureCuda, ClosureHip,
662                         ClosureDpcpp>
663             op(op_omp, op_cuda, op_hip, op_dpcpp);
664         this->run(op);
665     }
```

### 48.180.2.5 run() [2/3]

```
virtual void gko::Executor::run
```

Runs the specified Operation using this Executor.

**Parameters**

| | |
|---|---|
| *op* | the operation to run |

### 48.180.2.6 run() [3/3]

```
template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename
ClosureHip , typename ClosureDpcpp >
void gko::Executor::run (
             typename ClosureReference ,
             typename ClosureOmp ,
             typename ClosureCuda ,
             typename ClosureHip ,
             typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureReference* | type of op_ref |
| *ClosureOmp* | type of op_omp |

**Template Parameters**

| | |
|---|---|
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *name* | the name of the operation |
| *op_ref* | functor to run in case of a ReferenceExecutor |
| *op_omp* | functor to run in case of a OmpExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
688     {
689         LambdaOperation<ClosureReference, ClosureOmp, ClosureCuda, ClosureHip,
690                         ClosureDpcpp>
691             op(std::move(name), op_ref, op_omp, op_cuda, op_hip, op_dpcpp);
692         this->run(op);
693     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

# 48.181 gko::Operation Class Reference

Operations can be used to define functionalities whose implementations differ among devices.

`#include <ginkgo/core/base/executor.hpp>`

## Public Member Functions

- virtual const char * get_name () const noexcept

  *Returns the operation's name.*

### 48.181.1 Detailed Description

Operations can be used to define functionalities whose implementations differ among devices.

This is done by extending the Operation class and implementing the overloads of the Operation::run() method for all Executor types. When invoking the Executor::run() method with the Operation as input, the library will select the Operation::run() overload corresponding to the dynamic type of the Executor instance.

Consider an overload of `operator<<` for Executors, which prints some basic device information (e.g. device type and id) of the Executor to a C++ stream:

```
std::ostream& operator«(std::ostream &os, const gko::Executor &exec);
```

One possible implementation would be to use RTTI to find the dynamic type of the Executor, However, using the Operation feature of Ginkgo, there is a more elegant approach which utilizes polymorphism. The first step is to define an Operation that will print the desired information for each Executor type.

```
class DeviceInfoPrinter :  public gko::Operation {
public:
    explicit DeviceInfoPrinter(std::ostream &os) :  os_(os) {}
    void run(const gko::OmpExecutor *)const override { os_ « "OMP"; }
    void run(const gko::CudaExecutor *exec)const override
{ os_ « "CUDA(" « exec->get_device_id() « ")"; }
    void run(const gko::HipExecutor *exec)const override
{ os_ « "HIP(" « exec->get_device_id() « ")"; }
    void run(const gko::DpcppExecutor *exec)const override
{ os_ « "DPC++(" « exec->get_device_id() « ")"; }
    // This is optional, if not overloaded, defaults to OmpExecutor overload
    void run(const gko::ReferenceExecutor *)const override
{ os_ « "Reference CPU"; }
private:
    std::ostream &os_;
};
```

Using DeviceInfoPrinter, the implementation of `operator<<` is as simple as calling the run() method of the executor.

```
std::ostream& operator«(std::ostream &os, const gko::Executor &exec)
{
    DeviceInfoPrinter printer(os);
    exec.run(printer);
    return os;
}
```

Now it is possible to write the following code:

```
auto omp = gko::OmpExecutor::create();
std::cout « *omp « std::endl
          « *gko::CudaExecutor::create(0, omp) « std::endl
          « *gko::HipExecutor::create(0, omp) « std::endl
          « *gko::DpcppExecutor::create(0, omp) « std::endl
          « *gko::ReferenceExecutor::create() « std::endl;
```

which produces the expected output:

```
OMP
CUDA(0)
HIP(0)
DPC++(0)
Reference CPU
```

One might feel that this code is too complicated for such a simple task. Luckily, there is an overload of the Executor::run() method, which is designed to facilitate writing simple operations like this one. The method takes four closures as input: one which is run for OMP, one for CUDA executors, one for HIP executors, and the last one for DPC++ executors. Using this method, there is no need to implement an Operation subclass:

```
std::ostream& operator«(std::ostream &os, const gko::Executor &exec)
{
    exec.run(
        [&]() { os « "OMP"; },  // OMP closure
        [&]() { os « "CUDA("    // CUDA closure
                  « static_cast<gko::CudaExecutor&>(exec)
                      .get_device_id()
                  « ")"; },
        [&]() { os « "HIP("     // HIP closure
                  « static_cast<gko::HipExecutor&>(exec)
                      .get_device_id()
                  « ")"; });
        [&]() { os « "DPC++("    // DPC++ closure
                  « static_cast<gko::DpcppExecutor&>(exec)
                      .get_device_id()
                  « ")"; });
    return os;
}
```

Using this approach, however, it is impossible to distinguish between a OmpExecutor and ReferenceExecutor, as both of them call the OMP closure.

### 48.181.2 Member Function Documentation

#### 48.181.2.1 get_name()

```
virtual const char* gko::Operation::get_name ( ) const  [virtual], [noexcept]
```

Returns the operation's name.

**Returns**

the operation's name

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

## 48.182 gko::log::operation_data Struct Reference

Struct representing Operator related data.

`#include <ginkgo/core/log/record.hpp>`

### 48.182.1 Detailed Description

Struct representing Operator related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.183 gko::OutOfBoundsError Class Reference

OutOfBoundsError is thrown if a memory access is detected to be out-of-bounds.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- OutOfBoundsError (const std::string &file, int line, size_type index, size_type bound)
  *Initializes an OutOfBoundsError.*

### 48.183.1 Detailed Description

OutOfBoundsError is thrown if a memory access is detected to be out-of-bounds.

### 48.183.2 Constructor & Destructor Documentation

#### 48.183.2.1 OutOfBoundsError()

```
gko::OutOfBoundsError::OutOfBoundsError (
            const std::string & file,
            int line,
            size_type index,
            size_type bound )  [inline]
```

Initializes an OutOfBoundsError.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *index* | The position that was accessed |
| *bound* | The first out-of-bound index |

```
562         :  Error(file, line,
563                  "trying to access index " + std::to_string(index) +
564                      " in a memory block of " + std::to_string(bound) +
565                      " elements")
566     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.184 gko::OverflowError Class Reference

OverflowError is thrown when an index calculation for storage requirements overflows.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- OverflowError (const std::string &file, const int line, const std::string &index_type)

### 48.184.1 Detailed Description

OverflowError is thrown when an index calculation for storage requirements overflows.

This most likely means that the index type is too small.

### 48.184.2 Constructor & Destructor Documentation

#### 48.184.2.1 OverflowError()

```
gko::OverflowError::OverflowError (
             const std::string & file,
             const int line,
             const std::string & index_type )  [inline]
```

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *index_type* | The integer type that overflowed |

```
583         :  Error(file, line, "Overflowing " + index_type)
584     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

# 48.185 gko::log::Papi< ValueType > Class Template Reference

Papi is a Logger which logs every event to the PAPI software.

`#include <ginkgo/core/log/papi.hpp>`

## Public Member Functions

- const std::string get_handle_name () const

  *Returns the unique name of this logger, which can be used in the PAPI_read() call.*
- const papi_handle_t get_handle () const

  *Returns the corresponding papi_handle_t for this logger.*

## Static Public Member Functions

- static std::shared_ptr< Papi > create (std::shared_ptr< const gko::Executor >, const Logger::mask_type &enabled_events=Logger::all_events_mask)

  *Creates a Papi Logger.*
- static std::shared_ptr< Papi > create (const Logger::mask_type &enabled_events=Logger::all_events_↩ mask)

  *Creates a Papi Logger.*

### 48.185.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::log::Papi< ValueType >**

Papi is a Logger which logs every event to the PAPI software.

Thanks to this logger, applications which interface with PAPI can access Ginkgo internal data through PAPI. For an example of usage, see examples/papi_logging/papi_logging.cpp

The logged values for each event are the following:

- all allocation events: number of bytes per executor
- all free events: number of calls per executor
- copy_started: number of bytes per executor from (to), in copy_started_from (respectively copy_started_to).
- copy_completed: number of bytes per executor from (to), in copy_completed_from (respectively copy_↩ completed_to).
- all polymorphic objects and operation events: number of calls per executor
- all apply events: number of calls per LinOp (argument "A").
- all factory events: number of calls per factory
- criterion_check_completed event: the residual norm is stored in a record (per criterion)
- iteration_complete event: the number of iteration is counted (per solver)

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type of values stored in the class (e.g. residuals) |

## 48.185.2 Member Function Documentation

### 48.185.2.1 create() [1/2]

```
template<typename ValueType = default_precision>
static std::shared_ptr<Papi> gko::log::Papi< ValueType >::create (
            const Logger::mask_type & enabled_events = Logger::all_events_mask )  [inline],
[static]
```

Creates a Papi Logger.

**Parameters**

| | |
|---|---|
| *enabled_events* | the events enabled for this Logger |

```
197     {
198         return std::shared_ptr<Papi>(new Papi(enabled_events), [](auto logger) {
199             auto handle = logger->get_handle();
200             delete logger;
201             papi_sde_shutdown(handle);
202         });
203     }
```

References gko::log::Papi< ValueType >::get_handle().

### 48.185.2.2 create() [2/2]

```
template<typename ValueType = default_precision>
static std::shared_ptr<Papi> gko::log::Papi< ValueType >::create (
            std::shared_ptr< const gko::Executor > ,
            const Logger::mask_type & enabled_events = Logger::all_events_mask )  [inline],
[static]
```

Creates a Papi Logger.

**Parameters**

| | |
|---|---|
| *enabled_events* | the events enabled for this Logger |

```
186     {
187         return Papi::create(enabled_events);
188     }
```

#### 48.185.2.3 get_handle()

```
template<typename ValueType = default_precision>
const papi_handle_t gko::log::Papi< ValueType >::get_handle ( ) const  [inline]
```

Returns the corresponding papi_handle_t for this logger.

**Returns**

the corresponding papi_handle_t for this logger

```
218 { return papi_handle; }
```

Referenced by gko::log::Papi< ValueType >::create().

#### 48.185.2.4 get_handle_name()

```
template<typename ValueType = default_precision>
const std::string gko::log::Papi< ValueType >::get_handle_name ( ) const  [inline]
```

Returns the unique name of this logger, which can be used in the PAPI_read() call.

**Returns**

the unique name of this logger

```
211 { return name; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/log/papi.hpp

## 48.186 gko::factorization::ParIc< ValueType, IndexType > Class Template Reference

ParIC is an incomplete Cholesky factorization which is computed in parallel.

```
#include <ginkgo/core/factorization/par_ic.hpp>
```

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## Additional Inherited Members

### 48.186.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::factorization::ParIc< ValueType, IndexType >**

ParIC is an incomplete Cholesky factorization which is computed in parallel.

$L$ is a lower triangular matrix, which approximates a given matrix $A$ with $A \approx LL^H$. Here, $L + L^H$ has the same sparsity pattern as $A$, which is also called IC(0).

The ParIC algorithm generates the incomplete factor iteratively, using a fixed-point iteration of the form

$$F(L)_{ij} = \begin{cases} \sqrt{a_{ii} - \sum_{k=1}^{i-1} |l_{ik}|^2}, & i = j, \\ a_{ij} - \sum_{k=1}^{i-1} l_{ik}\overline{l_{jk}}, & i < j. \end{cases}$$

In general, the entries of $L$ can be iterated in parallel and in asynchronous fashion; the algorithm asymptotically converges to incomplete factors $L$ and $L^H$ fulfilling $(R = A - LL^H)|_{\mathcal{S}} = 0|_{\mathcal{S}}$ where $\mathcal{S}$ is the pre-defined sparsity pattern (in case of IC(0), the sparsity pattern of the system matrix $A$). The number of ParIC sweeps needed for convergence depends on the parallelism level: For sequential execution, a single sweep is sufficient; for fine-grained parallelism, the number of sweeps necessary to get a good approximation of the incomplete factors depends heavily on the problem. On the OpenMP executor, 3 sweeps usually give a decent approximation in our experiments, while GPU executors can take 10 or more iterations.

#### References

- Chow, E., Patel, A. *Fine-Grained Parallel Incomplete LU Factorization.* SIAM Journal on Scientific Computing, 37 (2), C169–C193, 2015. https://doi.org/10.1137/140968896

#### Template Parameters

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

### 48.186.2 Member Function Documentation

#### 48.186.2.1 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::factorization::ParIc< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/par_ic.hpp

## 48.187 gko::factorization::ParIct< ValueType, IndexType > Class Template Reference

ParICT is an incomplete threshold-based Cholesky factorization which is computed in parallel.

`#include <ginkgo/core/factorization/par_ict.hpp>`

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

### Additional Inherited Members

### 48.187.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::factorization::ParIct< ValueType, IndexType >**

ParICT is an incomplete threshold-based Cholesky factorization which is computed in parallel.

$L$ is a lower triangular matrix which approximates a given symmetric positive definite matrix $A$ with $A \approx LL^T$. Here, $L$ has a sparsity pattern that is improved iteratively based on its element-wise magnitude. The initial sparsity pattern is chosen based on the lower triangle of $A$.

One iteration of the ParICT algorithm consists of the following steps:

1. Calculate the residual $R = A - LL^T$.
2. Add new non-zero locations from $R$ to $L$. The new non-zero locations are initialized from the corresponding residual entries.

3. Execute a fixed-point iteration on $L$ according to

$$F(L)_{ij} = \begin{cases} \frac{1}{l_{jj}} \left( a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right), & i \neq j, \\ \sqrt{a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}}, & i = j. \end{cases}$$

4. Remove the smallest entries (by magnitude) from $L$.
5. Execute a fixed-point iteration on the (now sparser) $L$.

This ParICT algorithm thus improves the sparsity pattern and the approximation of $L$ simultaneously.

#### References

- Anzt, H., Chow, E., Dongarra, J. *ParILUT — A Parallel Threshold ILU for GPUs.* 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 231–241. https://doi.org/10.1109/IPDPS.2019.00033

#### Template Parameters

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

## 48.187.2 Member Function Documentation

### 48.187.2.1 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::factorization::ParIct< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_descriptor is only used for children configs.

#### Parameters

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/par_ict.hpp

## 48.188 gko::factorization::ParIlu< ValueType, IndexType > Class Template Reference

ParILU is an incomplete LU factorization which is computed in parallel.

`#include <ginkgo/core/factorization/par_ilu.hpp>`

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

### Additional Inherited Members

### 48.188.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::factorization::ParIlu< ValueType, IndexType >**

ParILU is an incomplete LU factorization which is computed in parallel.

$L$ is a lower unitriangular, while $U$ is an upper triangular matrix, which approximate a given matrix $A$ with $A \approx LU$. Here, $L$ and $U$ have the same sparsity pattern as $A$, which is also called ILU(0).

The ParILU algorithm generates the incomplete factors iteratively, using a fixed-point iteration of the form

$$F(L,U)_{ij} = \begin{cases} \frac{1}{u_{jj}} \left( a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj} \right), & i > j, \\ a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj}, & i \leq j. \end{cases}$$

In general, the entries of $L$ and $U$ can be iterated in parallel and in asynchronous fashion; the algorithm asymptotically converges to incomplete factors $L$ and $U$ fulfilling $(R = A - LU)|_{\mathcal{S}} = 0|_{\mathcal{S}}$ where $\mathcal{S}$ is the pre-defined sparsity pattern (in case of ILU(0), the sparsity pattern of the system matrix $A$). The number of ParILU sweeps needed for convergence depends on the parallelism level: For sequential execution, a single sweep is sufficient; for fine-grained parallelism, the number of sweeps necessary to get a good approximation of the incomplete factors depends heavily on the problem. On the OpenMP executor, 3 sweeps usually give a decent approximation in our experiments, while GPU executors can take 10 or more iterations.

**References**

- Chow, E., Patel, A. *Fine-Grained Parallel Incomplete LU Factorization.* SIAM Journal on Scientific Computing, 37 (2), C169–C193, 2015. https://doi.org/10.1137/140968896

**Template Parameters**

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

## 48.188.2 Member Function Documentation

### 48.188.2.1 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::factorization::ParIlu< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/par_ilu.hpp

# 48.189 gko::factorization::ParIlut< ValueType, IndexType > Class Template Reference

ParILUT is an incomplete threshold-based LU factorization which is computed in parallel.

```
#include <ginkgo/core/factorization/par_ilut.hpp>
```

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## Additional Inherited Members

### 48.189.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::factorization::ParIlut< ValueType, IndexType >**

ParILUT is an incomplete threshold-based LU factorization which is computed in parallel.

$L$ is a lower unitriangular, while $U$ is an upper triangular matrix, which approximate a given matrix $A$ with $A \approx LU$. Here, $L$ and $U$ have a sparsity pattern that is improved iteratively based on their element-wise magnitude. The initial sparsity pattern is chosen based on the $ILU(0)$ factorization of $A$.

One iteration of the ParILUT algorithm consists of the following steps:

1. Calculate the residual $R = A - LU$.
2. Add new non-zero locations from $R$ to $L$ and $U$. The new non-zero locations are initialized from the corresponding residual entries.
3. Execute a fixed-point iteration on $L$ and $U$ according to

$$F(L,U)_{ij} = \begin{cases} \frac{1}{u_{jj}} \left( a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj} \right), & i > j, \\ a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj}, & i \leq j. \end{cases}$$

   For a more detailed description of the fixed-point iteration, see Parllu.
4. Remove the smallest entries (by magnitude) from $L$ and $U$.
5. Execute a fixed-point iteration on the (now sparser) $L$ and $U$.

This ParILUT algorithm thus improves the sparsity pattern and the approximation of $L$ and $U$ simultaneously.

#### References

- Anzt, H., Chow, E., Dongarra, J. *ParILUT — A Parallel Threshold ILU for GPUs.* 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 231–241. https://doi.org/10.1109/IPDPS.2019.00033

#### Template Parameters

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

## 48.189.2 Member Function Documentation

### 48.189.2.1 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::factorization::ParIlut< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/factorization/par_ilut.hpp

# 48.190 gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType > Class Template Reference

Represents a partition of a range of indices [0, size) into a disjoint set of parts.

```
#include <ginkgo/core/distributed/partition.hpp>
```

## Public Member Functions

- size_type get_size () const

  *Returns the total number of elements represented by this partition.*
- size_type get_num_ranges () const noexcept

  *Returns the number of ranges stored by this partition.*
- comm_index_type get_num_parts () const noexcept

  *Returns the number of parts represented in this partition.*

- bool equals (const Partition &other) const

  *Checks whether this partition describes the same distribution as another one, i.e.*
- comm_index_type get_num_empty_parts () const noexcept

  *Returns the number of empty parts within this partition.*
- const global_index_type * get_range_bounds () const noexcept

  *Returns the ranges boundary array stored by this partition.*
- const comm_index_type * get_part_ids () const noexcept

  *Returns the part IDs of the ranges in this partition.*
- const local_index_type * get_range_starting_indices () const noexcept

  *Returns the part-local starting index for each range in this partition.*
- const local_index_type * get_part_sizes () const noexcept

  *Returns the part size array.*
- local_index_type get_part_size (comm_index_type part) const

  *Returns the size of a part given by its part ID.*
- const segmented_array< size_type > & get_ranges_by_part () const

  *Returns the range IDs segmented by their part ID.*
- bool has_connected_parts () const

  *Checks if each part has no more than one contiguous range.*
- bool has_ordered_parts () const

  *Checks if the ranges are ordered by their part index.*

## Static Public Member Functions

- static std::unique_ptr< Partition > build_from_mapping (std::shared_ptr< const Executor > exec, const array< comm_index_type > &mapping, comm_index_type num_parts)

  *Builds a partition from a given mapping global_index -> part_id.*
- static std::unique_ptr< Partition > build_from_contiguous (std::shared_ptr< const Executor > exec, const array< global_index_type > &ranges, const array< comm_index_type > &part_ids={})

  *Builds a partition consisting of contiguous ranges, one for each part.*
- static std::unique_ptr< Partition > build_from_global_size_uniform (std::shared_ptr< const Executor > exec, comm_index_type num_parts, global_index_type global_size)

  *Builds a partition by evenly distributing the global range.*

### 48.190.1 Detailed Description

**template<typename LocalIndexType = int32, typename GlobalIndexType = int64>**
**class gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >**

Represents a partition of a range of indices [0, size) into a disjoint set of parts.

The partition is stored as a set of consecutive ranges [begin, end) with an associated part ID and local index (number of indices in this part before `begin`). Global indices are stored as 64 bit signed integers (int64), part-local indices use LocalIndexType, Part IDs use 32 bit signed integers (int).

For example, consider the interval [0, 13) that is partitioned into the following ranges:
`[0,3), [3, 6), [6, 8), [8, 10), [10, 13).`

These ranges are distributed on three part with:
```
p_0 = [0, 3) + [6, 8) + [10, 13),
p_1 = [3, 6),
p_2 = [8, 10).
```

The part ids can be queried from the get_part_ids array, and the ranges are represented as offsets, accessed by get_range_bounds, leading to the offset array:

```
r = [0, 3, 6, 8, 10, 13]
```

so that individual ranges are given by `[r[i], r[i + 1])`. Since each part may be associated with multiple ranges, it is possible to get the starting index for each range that is local to the owning part, see get_range_starting_indices. These indices can be used to easily iterate over part local data. For example, the above partition has the following starting indices

```
starting_index[0] = 0,
starting_index[1] = 0,
starting_index[2] = 3,  // second range of part 0
starting_index[3] = 0,
starting_index[4] = 5,  // third range of part 0
```

which you can use to iterate only over the the second range of part 0 (the third global range) with

```
for(int i = 0; i < r[3] - r[2]; ++i){
  data[starting_index[2] + i] = val;
}
```

**Template Parameters**

| | |
|---|---|
| *LocalIndexType* | The index type used for part-local indices. To prevent overflows, no single part's size may exceed this index type's maximum value. |
| *GlobalIndexType* | The index type used for the global indices. Needs to be at least as large a type as LocalIndexType. |

## 48.190.2 Member Function Documentation

### 48.190.2.1 build_from_contiguous()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
static std::unique_ptr<Partition> gko::experimental::distributed::Partition< LocalIndexType,
GlobalIndexType >::build_from_contiguous (
            std::shared_ptr< const Executor > exec,
            const array< global_index_type > & ranges,
            const array< comm_index_type > & part_ids = {} )  [static]
```

Builds a partition consisting of contiguous ranges, one for each part.

**Parameters**

| | |
|---|---|
| *exec* | the Executor on which the partition should be built |
| *ranges* | the boundaries of the ranges representing each part. Part part_id[i] contains the indices [ranges[i], ranges[i + 1]). Has to contain at least one element. The first element has to be 0. |
| *part_ids* | the part ids of the provided ranges. If empty, then it will assume range i belongs to part i. |

**Returns**

a Partition representing the given contiguous partitioning.

#### 48.190.2.2 build_from_global_size_uniform()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
static std::unique_ptr<Partition> gko::experimental::distributed::Partition< LocalIndexType,
GlobalIndexType >::build_from_global_size_uniform (
            std::shared_ptr< const Executor > exec,
            comm_index_type num_parts,
            global_index_type global_size ) [static]
```

Builds a partition by evenly distributing the global range.

**Parameters**

| | |
|---|---|
| *exec* | the Executor on which the partition should be built |
| *num_parts* | the number of parst used in this partition |
| *global_size* | the global size of this partition |

**Returns**

a Partition where each range has either `floor(global_size/num_parts)` or `floor(global_size/num_parts) + 1` indices.

#### 48.190.2.3 build_from_mapping()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
static std::unique_ptr<Partition> gko::experimental::distributed::Partition< LocalIndexType,
GlobalIndexType >::build_from_mapping (
            std::shared_ptr< const Executor > exec,
            const array< comm_index_type > & mapping,
            comm_index_type num_parts ) [static]
```

Builds a partition from a given mapping global_index -> part_id.

**Parameters**

| | |
|---|---|
| *exec* | the Executor on which the partition should be built |
| *mapping* | the mapping from global indices to part IDs. |
| *num_parts* | the number of parts used in the mapping. |

**Returns**

a Partition representing the given mapping as a set of ranges

#### 48.190.2.4 equals()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
bool gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::equals (
            const Partition< LocalIndexType, GlobalIndexType > & other ) const
```

Checks whether this partition describes the same distribution as another one, i.e.

both span the same global size and assign the same ranges to the same parts.

Comparing two distinct partitions that live on a device executor requires copying their range data to the host, so this is not a cheap operation. Comparing a partition with itself is free.

**Parameters**

| | |
|---|---|
| *other* | the partition to compare against. |

**Returns**

true if both partitions describe the same distribution.

### 48.190.2.5 get_num_empty_parts()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
comm_index_type gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >↩
::get_num_empty_parts ( ) const  [inline], [noexcept]
```

Returns the number of empty parts within this partition.

**Returns**

number of empty parts.

```
141     {
142         return num_empty_parts_;
143     }
```

### 48.190.2.6 get_num_parts()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
comm_index_type gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >↩
::get_num_parts ( ) const  [inline], [noexcept]
```

Returns the number of parts represented in this partition.

**Returns**

number of parts.

```
118 { return num_parts_; }
```

#### 48.190.2.7 get_num_ranges()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
size_type gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_↩
num_ranges ( ) const  [inline], [noexcept]
```

Returns the number of ranges stored by this partition.

This size refers to the data returned by get_range_bounds().

**Returns**

number of ranges.

```
109     {
110         return offsets_.get_size() - 1;
111     }
```

References gko::array< ValueType >::get_size().

#### 48.190.2.8 get_part_ids()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
const comm_index_type* gko::experimental::distributed::Partition< LocalIndexType, Global↩
IndexType >::get_part_ids ( ) const  [inline], [noexcept]
```

Returns the part IDs of the ranges in this partition.

For each range from get_range_bounds(), it stores the part ID in the interval [0, get_num_parts() - 1].

**Returns**

part ID array.

```
165     {
166         return part_ids_.get_const_data();
167     }
```

References gko::array< ValueType >::get_const_data().

#### 48.190.2.9 get_part_size()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
local_index_type gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType
>::get_part_size (
            comm_index_type part ) const
```

Returns the size of a part given by its part ID.

**Warning**

Triggers a copy from device to host.

**Parameters**

| | |
|---|---|
| *part* | the part ID. |

**Returns**

size of part.

### 48.190.2.10 get_part_sizes()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
const local_index_type* gko::experimental::distributed::Partition< LocalIndexType, Global↩
IndexType >::get_part_sizes ( ) const  [inline], [noexcept]
```

Returns the part size array.

part_sizes[p] stores the total number of indices in part `p`.

**Returns**

part size array.

```
194     {
195         return part_sizes_.get_const_data();
196     }
```

References gko::array< ValueType >::get_const_data().

### 48.190.2.11 get_range_bounds()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
const global_index_type* gko::experimental::distributed::Partition< LocalIndexType, Global↩
IndexType >::get_range_bounds ( ) const  [inline], [noexcept]
```

Returns the ranges boundary array stored by this partition.

`range_bounds[i]` is the beginning (inclusive) and `range_bounds[i + 1]` is the end (exclusive) of the ith range.

**Returns**

range boundaries array.

```
153     {
154         return offsets_.get_const_data();
155     }
```

References gko::array< ValueType >::get_const_data().

#### 48.190.2.12 get_range_starting_indices()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
const local_index_type* gko::experimental::distributed::Partition< LocalIndexType, Global↩
IndexType >::get_range_starting_indices ( ) const  [inline], [noexcept]
```

Returns the part-local starting index for each range in this partition.

Consider the partition on `[0, 10)` with
`p_1 = [0-4) + [7-10),`
`p_2 = [4-7).`

Then `range_starting_indices[0] = 0`, `range_starting_indices[1] = 0`, `range_↩starting_indices[2] = 4`.

**Returns**

part-local starting index array.

```
183     {
184         return starting_indices_.get_const_data();
185     }
```

References gko::array< ValueType >::get_const_data().

#### 48.190.2.13 get_ranges_by_part()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
const segmented_array<size_type>& gko::experimental::distributed::Partition< LocalIndexType,
GlobalIndexType >::get_ranges_by_part ( ) const  [inline]
```

Returns the range IDs segmented by their part ID.

**Returns**

range IDs segmented by part IDs

```
214     {
215         return ranges_by_part_;
216     }
```

#### 48.190.2.14 get_size()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
size_type gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::get_↩
size ( ) const  [inline]
```

Returns the total number of elements represented by this partition.

**Returns**

number elements.

```
100 { return size_; }
```

#### 48.190.2.15 has_connected_parts()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
bool gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::has_↩
connected_parts ( ) const
```

Checks if each part has no more than one contiguous range.

**Returns**

true if each part has no more than one contiguous range.

#### 48.190.2.16 has_ordered_parts()

```
template<typename LocalIndexType = int32, typename GlobalIndexType = int64>
bool gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >::has_↩
ordered_parts ( ) const
```

Checks if the ranges are ordered by their part index.

Implies that the partition is connected.

**Returns**

true if the ranges are ordered by their part index.

The documentation for this class was generated from the following files:

- ginkgo/core/distributed/assembly.hpp
- ginkgo/core/distributed/partition.hpp

## 48.191 gko::log::PerformanceHint Class Reference

PerformanceHint is a Logger which analyzes the performance of the application and outputs hints for unnecessary copies and allocations.

```
#include <ginkgo/core/log/performance_hint.hpp>
```

### Public Member Functions

- void print_status () const

  *Writes out the cross-executor writes and allocations that have been stored so far.*

### Static Public Member Functions

- static std::unique_ptr< PerformanceHint > create (std::ostream &os=std::cerr, size_type allocation_size_↩ limit=16, size_type copy_size_limit=16, size_type histogram_max_size=1024)

  *Creates a PerformanceHint logger.*

### 48.191.1 Detailed Description

PerformanceHint is a Logger which analyzes the performance of the application and outputs hints for unnecessary copies and allocations.

The specific patterns it checks for are:

- repeated cross-executor copies from or to the same pointer
- repeated allocation/free pairs of the same size

### 48.191.2 Member Function Documentation

#### 48.191.2.1 create()

```
static std::unique_ptr<PerformanceHint> gko::log::PerformanceHint::create (
            std::ostream & os = std::cerr,
            size_type allocation_size_limit = 16,
            size_type copy_size_limit = 16,
            size_type histogram_max_size = 1024 )  [inline], [static]
```

Creates a PerformanceHint logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *os* | the stream used for this logger |
| *allocation_size_limit* | ignore allocations below this limit (bytes) |
| *copy_size_limit* | ignore copies below this limit (bytes) |
| *histogram_max_size* | how many allocation sizes and/or pointers to keep track of at most? |

**Returns**

an std::unique_ptr to the the constructed object

```
64     {
65         return std::unique_ptr<PerformanceHint>(new PerformanceHint(
66             os, allocation_size_limit, copy_size_limit, histogram_max_size));
67     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/log/performance_hint.hpp

## 48.192 gko::Permutable< IndexType > Class Template Reference

Linear operators which support permutation should implement the Permutable interface.

```
#include <ginkgo/core/base/lin_op.hpp>
```

## Public Member Functions

- virtual std::unique_ptr< LinOp > permute (const array< IndexType > *permutation_indices) const

  *Returns a LinOp representing the symmetric row and column permutation of the Permutable object.*
- virtual std::unique_ptr< LinOp > inverse_permute (const array< IndexType > *permutation_indices) const

  *Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.*
- virtual std::unique_ptr< LinOp > row_permute (const array< IndexType > *permutation_indices) const =0

  *Returns a LinOp representing the row permutation of the Permutable object.*
- virtual std::unique_ptr< LinOp > column_permute (const array< IndexType > *permutation_indices) const =0

  *Returns a LinOp representing the column permutation of the Permutable object.*
- virtual std::unique_ptr< LinOp > inverse_row_permute (const array< IndexType > *permutation_indices) const =0

  *Returns a LinOp representing the row permutation of the inverse permuted object.*
- virtual std::unique_ptr< LinOp > inverse_column_permute (const array< IndexType > *permutation_↩ indices) const =0

  *Returns a LinOp representing the row permutation of the inverse permuted object.*

### 48.192.1 Detailed Description

**template<typename IndexType>**
**class gko::Permutable< IndexType >**

Linear operators which support permutation should implement the Permutable interface.

It provides functions to permute the rows and columns of a LinOp, independently or symmetrically, and with a regular or inverted permutation.

After a regular row permutation with permutation array `perm` the row `i` in the output LinOp contains the row `perm[i]` from the input LinOp. After an inverse row permutation, the row `perm[i]` in the output LinOp contains the row `i` from the input LinOp. Equivalently, after a column permutation, the output stores in column `i` the column `perm[i]` from the input, and an inverse column permutation stores in column `perm[i]` the column `i` from the input. A symmetric permutation is functionally equivalent to calling `as<Permutable>(A->row_↩ permute(perm))->column_permute(perm)`, but the implementation can provide better performance due to kernel fusion.

#### 48.192.1.1 Example: Permuting a Csr matrix:

```
 {c++}
//Permuting an object of LinOp type.
//The object you want to permute.
auto op = matrix::Csr::create(exec);
//Permute the object by first converting it to a Permutable type.
auto perm = op->row_permute(permutation_indices);
```

### 48.192.2 Member Function Documentation

#### 48.192.2.1 column_permute()

```
template<typename IndexType>
virtual std::unique_ptr<LinOp> gko::Permutable< IndexType >::column_permute (
            const array< IndexType > * permutation_indices ) const  [pure virtual]
```

Returns a LinOp representing the column permutation of the Permutable object.

In the resulting LinOp, the column `i` contains the input column `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new column permuted object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, and gko::matrix::Csr< ValueType, IndexType >.

#### 48.192.2.2 inverse_column_permute()

```
template<typename IndexType>
virtual std::unique_ptr<LinOp> gko::Permutable< IndexType >::inverse_column_permute (
            const array< IndexType > * permutation_indices ) const  [pure virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the column `perm[i]` contains the input column `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, and gko::matrix::Csr< ValueType, IndexType >.

#### 48.192.2.3 inverse_permute()

```
template<typename IndexType>
virtual std::unique_ptr<LinOp> gko::Permutable< IndexType >::inverse_permute (
            const array< IndexType > * permutation_indices ) const  [inline], [virtual]
```

Returns a LinOp representing the symmetric inverse row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(perm[i],perm[j])` contains the input value `(i,j)`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}AP^{-T}$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, and gko::matrix::Csr< ValueType, IndexType >.

```
584     {
585         return as<Permutable>(this->inverse_row_permute(permutation_indices))
586             ->inverse_column_permute(permutation_indices);
587     }
```

### 48.192.2.4 inverse_row_permute()

```
template<typename IndexType>
virtual std::unique_ptr<LinOp> gko::Permutable< IndexType >::inverse_row_permute (
            const array< IndexType > * permutation_indices ) const  [pure virtual]
```

Returns a LinOp representing the row permutation of the inverse permuted object.

In the resulting LinOp, the row `perm[i]` contains the input row `i`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $P^{-1}A$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order `perm`. |

**Returns**

a pointer to the new inverse permuted object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, and gko::matrix::Csr< ValueType, IndexType >.

### 48.192.2.5 permute()

```
template<typename IndexType>
virtual std::unique_ptr<LinOp> gko::Permutable< IndexType >::permute (
            const array< IndexType > * permutation_indices ) const  [inline], [virtual]
```

Returns a LinOp representing the symmetric row and column permutation of the Permutable object.

In the resulting LinOp, the entry at location `(i,j)` contains the input value `(perm[i],perm[j])`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PAP^T$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Reimplemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, and gko::matrix::Csr< ValueType, IndexType >.

```
563     {
564         return as<Permutable>(this->row_permute(permutation_indices))
565             ->column_permute(permutation_indices);
566     }
```

#### 48.192.2.6 row_permute()

```
template<typename IndexType>
virtual std::unique_ptr<LinOp> gko::Permutable< IndexType >::row_permute (
            const array< IndexType > * permutation_indices ) const  [pure virtual]
```

Returns a LinOp representing the row permutation of the Permutable object.

In the resulting LinOp, the row `i` contains the input row `perm[i]`.

From the linear algebra perspective, with $P_{ij} = \delta_{i\pi(i)}$, this represents the operation $PA$.

**Parameters**

| | |
|---|---|
| *permutation_indices* | the array of indices containing the permutation order. |

**Returns**

a pointer to the new permuted object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, and gko::matrix::Csr< ValueType, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.193 gko::matrix::Permutation< IndexType > Class Template Reference

Permutation is a matrix format that represents a permutation matrix, i.e.

```
#include <ginkgo/core/matrix/permutation.hpp>
```

## Public Member Functions

- index_type * get_permutation () noexcept

  *Returns a pointer to the array of permutation.*
- const index_type * get_const_permutation () const noexcept

  *Returns a pointer to the array of permutation.*
- size_type get_permutation_size () const noexcept

  *Returns the number of elements explicitly stored in the permutation array.*
- std::unique_ptr< Permutation > compute_inverse () const

  *Returns the inverse permutation.*
- std::unique_ptr< Permutation > compose (ptr_param< const Permutation > other) const

  *Composes this permutation with another permutation.*
- void write (gko::matrix_data< value_type, index_type > &data) const override

  *Writes a matrix to a matrix_data structure.*
- void validate_data () const override

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*

## Static Public Member Functions

- static std::unique_ptr< Permutation > create (std::shared_ptr< const Executor > exec, size_type size=0)

  *Creates an uninitialized Permutation arrays on the specified executor.*
- static std::unique_ptr< Permutation > create (std::shared_ptr< const Executor > exec, array< IndexType > permutation_indices)

  *Creates a Permutation matrix from an already allocated (and initialized) row and column permutation arrays.*
- static std::unique_ptr< const Permutation > create_const (std::shared_ptr< const Executor > exec, size_type size, gko::detail::const_array_view< IndexType > &&perm_idxs, mask_type enabled_↩ permute=row_permute)

  *Creates a constant (immutable) Permutation matrix from a constant array.*
- static std::unique_ptr< const Permutation > create_const (std::shared_ptr< const Executor > exec, gko↩ ::detail::const_array_view< IndexType > &&perm_idxs)

  *Creates a constant (immutable) Permutation matrix from a constant array.*

### 48.193.1 Detailed Description

**template<typename IndexType = int32>**
**class gko::matrix::Permutation< IndexType >**

Permutation is a matrix format that represents a permutation matrix, i.e.

a matrix where each row and column has exactly one entry. The matrix can only be applied to Dense inputs, where it represents a row permutation: $A' = PA$ means $A'(i,j) = A(p[i],j)$.

**Template Parameters**

| | |
|---|---|
| *IndexType* | precision of permutation array indices. |

## 48.193.2 Member Function Documentation

### 48.193.2.1 compose()

```
template<typename IndexType = int32>
std::unique_ptr<Permutation> gko::matrix::Permutation< IndexType >::compose (
            ptr_param< const Permutation< IndexType > > other ) const
```

Composes this permutation with another permutation.

The resulting permutation fulfills `result[i] = this[other[i]]` or `result = other * this` from the matrix perspective, which is equivalent to first permuting by `this` and then by `other`: Combining permutations $P_1$ and $P_2$ with `P = P_1.combine(P_2)` performs the operation permute(A, P) = permute(permute(A, P_1), P_2).

**Parameters**

| | |
|---|---|
| *other* | the other permutation |

**Returns**

the combined permutation

### 48.193.2.2 compute_inverse()

```
template<typename IndexType = int32>
std::unique_ptr<Permutation> gko::matrix::Permutation< IndexType >::compute_inverse ( ) const
```

Returns the inverse permutation.

**Returns**

a newly created Permutation object storing the inverse permutation of this Permutation.

### 48.193.2.3 create() [1/2]

```
template<typename IndexType = int32>
static std::unique_ptr<Permutation> gko::matrix::Permutation< IndexType >::create (
            std::shared_ptr< const Executor > exec,
            array< IndexType > permutation_indices )  [static]
```

Creates a Permutation matrix from an already allocated (and initialized) row and column permutation arrays.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the permutation array. |
| *permutation_indices* | array of permutation array |
| *enabled_permute* | mask for the type of permutation to apply. |

**Note**

If `permutation_indices` is not an rvalue, not an array of IndexType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

### 48.193.2.4 create() [2/2]

```
template<typename IndexType = int32>
static std::unique_ptr<Permutation> gko::matrix::Permutation< IndexType >::create (
            std::shared_ptr< const Executor > exec,
            size_type size = 0 )  [static]
```

Creates an uninitialized Permutation arrays on the specified executor.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the LinOp |
| *size* | the size of the permutation |

**Returns**

A smart pointer to the newly created matrix.

### 48.193.2.5 create_const() [1/2]

```
template<typename IndexType = int32>
static std::unique_ptr<const Permutation> gko::matrix::Permutation< IndexType >::create_const
(
            std::shared_ptr< const Executor > exec,
            gko::detail::const_array_view< IndexType > && perm_idxs )  [static]
```

Creates a constant (immutable) Permutation matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the size of the square matrix |
| *perm_idxs* | the permutation index array of the matrix |
| *enabled_permute* | the mask describing the type of permutation |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

#### 48.193.2.6 create_const() [2/2]

```
template<typename IndexType = int32>
static std::unique_ptr<const Permutation> gko::matrix::Permutation< IndexType >::create_const
(
            std::shared_ptr< const Executor > exec,
            size_type size,
            gko::detail::const_array_view< IndexType > && perm_idxs,
            mask_type enabled_permute = row_permute )  [static]
```

Creates a constant (immutable) Permutation matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the size of the square matrix |
| *perm_idxs* | the permutation index array of the matrix |
| *enabled_permute* | the mask describing the type of permutation |

**Returns**

A smart pointer to the constant matrix wrapping the input array (if it resides on the same executor as the matrix) or a copy of the array on the correct executor.

#### 48.193.2.7 get_const_permutation()

```
template<typename IndexType = int32>
const index_type* gko::matrix::Permutation< IndexType >::get_const_permutation ( ) const  [inline],
[noexcept]
```

Returns a pointer to the array of permutation.

**Returns**

the pointer to the row permutation array.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
135     {
136         return permutation_.get_const_data();
137     }
```

References gko::array< ValueType >::get_const_data().

### 48.193.2.8 get_permutation()

```
template<typename IndexType = int32>
index_type* gko::matrix::Permutation< IndexType >::get_permutation ( )  [inline], [noexcept]
```

Returns a pointer to the array of permutation.

**Returns**

the pointer to the row permutation array.

```
125 { return permutation_.get_data(); }
```

References gko::array< ValueType >::get_data().

### 48.193.2.9 get_permutation_size()

```
template<typename IndexType = int32>
size_type gko::matrix::Permutation< IndexType >::get_permutation_size ( ) const  [noexcept]
```

Returns the number of elements explicitly stored in the permutation array.

**Returns**

the number of elements explicitly stored in the permutation array.

### 48.193.2.10 write()

```
template<typename IndexType = int32>
void gko::matrix::Permutation< IndexType >::write (
            gko::matrix_data< value_type, index_type > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< default_precision, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/permutation.hpp

## 48.194 gko::matrix::Csr< ValueType, IndexType >::permuting_reuse_info Struct Reference

A struct describing a transformation of the matrix that reorders the values of the matrix into the transformed matrix.

`#include <ginkgo/core/matrix/csr.hpp>`

### Public Member Functions

- permuting_reuse_info ()
  *Creates an empty reuse info.*
- permuting_reuse_info (std::unique_ptr< Permutation< index_type >> value_permutation)
  *Creates a reuse info structure from its value permutation.*
- void update_values (ptr_param< const Csr > input, ptr_param< Csr > output) const
  *Propagates the values from an input matrix to the transformed matrix.*

### 48.194.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**struct gko::matrix::Csr< ValueType, IndexType >::permuting_reuse_info**

A struct describing a transformation of the matrix that reorders the values of the matrix into the transformed matrix.

### 48.194.2 Member Function Documentation

#### 48.194.2.1 update_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Csr< ValueType, IndexType >::permuting_reuse_info::update_values (
            ptr_param< const Csr > input,
            ptr_param< Csr > output ) const
```

Propagates the values from an input matrix to the transformed matrix.

The output matrix needs to have been computed using the transformation that was also used to generate this reuse data. Internally, this permutes the input value vector into the output value vector.

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/csr.hpp

# 48.195 gko::Perturbation< ValueType > Class Template Reference

The Perturbation class can be used to construct a LinOp to represent the operation `(identity + scalar * basis * projector)`.

`#include <ginkgo/core/base/perturbation.hpp>`

## Public Member Functions

- const std::shared_ptr< const LinOp > get_basis () const noexcept

  *Returns the basis of the perturbation.*
- const std::shared_ptr< const LinOp > get_projector () const noexcept

  *Returns the projector of the perturbation.*
- const std::shared_ptr< const LinOp > get_scalar () const noexcept

  *Returns the scalar of the perturbation.*

## Static Public Member Functions

- static std::unique_ptr< Perturbation > create (std::shared_ptr< const Executor > exec)

  *Creates an empty perturbation operator (0x0 operator).*
- static std::unique_ptr< Perturbation > create (std::shared_ptr< const LinOp > scalar, std::shared_ptr< const LinOp > basis)

  *Creates a perturbation with scalar and basis by setting projector to the conjugate transpose of basis.*
- static std::unique_ptr< Perturbation > create (std::shared_ptr< const LinOp > scalar, std::shared_ptr< const LinOp > basis, std::shared_ptr< const LinOp > projector)

  *Creates a perturbation of scalar, basis and projector.*

### 48.195.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::Perturbation< ValueType >**

The Perturbation class can be used to construct a LinOp to represent the operation `(identity + scalar * basis * projector)`.

This operator adds a movement along a direction constructed by `basis` and `projector` on the LinOp. `projector` gives the coefficient of `basis` to decide the direction.

For example, the Householder matrix can be represented with the Perturbation operator as follows. If u is the Householder factor then we can generate the `Householder transformation`, H = (I - 2 u u*). In this case, the parameters of Perturbation class are scalar = -2, basis = u, and projector = u*.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of input and result vectors |

**Note**

the apply operations of Perturbation class are not thread safe

## 48.195.2 Member Function Documentation

### 48.195.2.1 create() [1/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Perturbation> gko::Perturbation< ValueType >::create (
            std::shared_ptr< const Executor > exec )  [static]
```

Creates an empty perturbation operator (0x0 operator).

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the perturbation |

**Returns**

A smart pointer to the newly created perturbation.

### 48.195.2.2 create() [2/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Perturbation> gko::Perturbation< ValueType >::create (
            std::shared_ptr< const LinOp > scalar,
            std::shared_ptr< const LinOp > basis )  [static]
```

Creates a perturbation with scalar and basis by setting projector to the conjugate transpose of basis.

Basis must be transposable. Perturbation will throw gko::NotSupported if basis is not transposable.

**Parameters**

| | |
|---|---|
| *scalar* | scaling of the movement |
| *basis* | the direction basis |

**Returns**

A smart pointer to the newly created perturbation.

### 48.195.2.3 create() [3/3]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Perturbation> gko::Perturbation< ValueType >::create (
            std::shared_ptr< const LinOp > scalar,
            std::shared_ptr< const LinOp > basis,
            std::shared_ptr< const LinOp > projector )  [static]
```

Creates a perturbation of scalar, basis and projector.

**Parameters**

| | |
|---|---|
| *scalar* | scaling of the movement |
| *basis* | the direction basis |
| *projector* | decides the coefficient of basis |

**Returns**

A smart pointer to the newly created perturbation.

### 48.195.2.4 get_basis()

```
template<typename ValueType = default_precision>
const std::shared_ptr<const LinOp> gko::Perturbation< ValueType >::get_basis ( ) const  [inline],
[noexcept]
```

Returns the basis of the perturbation.

**Returns**

the basis of the perturbation

```
54     {
55         return basis_;
56     }
```

### 48.195.2.5 get_projector()

```
template<typename ValueType = default_precision>
const std::shared_ptr<const LinOp> gko::Perturbation< ValueType >::get_projector ( ) const
[inline], [noexcept]
```

Returns the projector of the perturbation.

**Returns**

the projector of the perturbation

```
64     {
65         return projector_;
66     }
```

#### 48.195.2.6 get_scalar()

```
template<typename ValueType = default_precision>
const std::shared_ptr<const LinOp> gko::Perturbation< ValueType >::get_scalar ( ) const  [inline],
[noexcept]
```

Returns the scalar of the perturbation.

**Returns**

the scalar of the perturbation

```
74     {
75         return scalar_;
76     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/perturbation.hpp

## 48.196 gko::multigrid::Pgm< ValueType, IndexType > Class Template Reference

Parallel graph match (Pgm) is the aggregate method introduced in the paper M.

```
#include <ginkgo/core/multigrid/pgm.hpp>
```

### Classes

- class reuse_data_type

  *Records the aggregates, the transfer operators and the coarse-to-fine mappings of the first generate_reuse() call.*

### Public Member Functions

- std::shared_ptr< const LinOp > get_system_matrix () const

  *Returns the system operator (matrix) of the linear system.*
- IndexType * get_agg () noexcept

  *Returns the aggregate group.*
- const IndexType * get_const_agg () const noexcept

  *Returns the aggregate group.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## 48.196.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::multigrid::Pgm< ValueType, IndexType >

Parallel graph match (Pgm) is the aggregate method introduced in the paper M.

Naumov et al., "AmgX: A Library for GPU Accelerated Algebraic Multigrid and Preconditioned Iterative Methods". Current implementation only contains size = 2 version.

Pgm creates the aggregate group according to the matrix value not the structure. Pgm gives two steps (one-phase handshaking) to group the elements. 1: get the strongest neighbor of each unaggregated element. 2: group the elements whose strongest neighbor is each other. repeating until reaching the given conditions. After that, the un-aggregated elements are assigned to an aggregated group or are left alone.

The factory supports LinOpFactory::generate_reuse(): later calls on a matrix with the same sparsity pattern keep the aggregates and only recompute the coarse matrix, so the result can differ from generate(). The aggregates come from the values of the first matrix, so it should be representative of the later ones.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.196.2 Member Function Documentation

### 48.196.2.1 get_agg()

```
template<typename ValueType = default_precision, typename IndexType = int32>
IndexType* gko::multigrid::Pgm< ValueType, IndexType >::get_agg ( )  [inline], [noexcept]
```

Returns the aggregate group.

Aggregate group whose size is same as the number of rows. Stores the mapping information from row index to coarse row index. i.e., agg[row_idx] = coarse_row_idx.

**Returns**

the aggregate group.

```
84 { return agg_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.196.2.2 get_const_agg()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const IndexType* gko::multigrid::Pgm< ValueType, IndexType >::get_const_agg ( ) const  [inline],
[noexcept]
```

Returns the aggregate group.

Aggregate group whose size is same as the number of rows. Stores the mapping information from row index to coarse row index. i.e., agg[row_idx] = coarse_row_idx.

**Returns**

the aggregate group.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
94     {
95         return agg_.get_const_data();
96     }
```

References gko::array< ValueType >::get_const_data().

#### 48.196.2.3 get_system_matrix()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::multigrid::Pgm< ValueType, IndexType >::get_system_matrix (
) const  [inline]
```

Returns the system operator (matrix) of the linear system.

**Returns**

the system operator (matrix)

```
71     {
72         return system_matrix_;
73     }
```

#### 48.196.2.4 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::multigrid::Pgm< ValueType, IndexType >::parse (
             const config::pnode & config,
             const config::registry & context,
             const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
) [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following files:

- ginkgo/core/distributed/matrix.hpp
- ginkgo/core/multigrid/pgm.hpp

## 48.197 gko::solver::PipeCg< ValueType > Class Template Reference

PIPE_CG or the pipelined conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.

```
#include <ginkgo/core/solver/pipe_cg.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- bool apply_uses_initial_guess () const override

  *Return true as iterative solvers use the data in x as an initial guess.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType >())

  *Create the parameters from the property_tree.*

### 48.197.1 Detailed Description

template<typename ValueType = default_precision>
class gko::solver::PipeCg< ValueType >

PIPE_CG or the pipelined conjugate gradient method is an iterative type Krylov subspace method which is suitable for symmetric positive definite methods.

It improves upon the CG method by allowing computation of inner products and norms to be overlapped with operator and preconditioner application. The pipelined method scales up to the 10^6 nodes of the assumed exascale machine, while its standard counterpart level off about one order of magnitude earlier, as suggested in the referenced paper (see below).

Mathematically the iterates $x_k$ are the same as those produced by CG: the algorithm minimizes the energy-norm error over the Krylov subspace $x_0 + \mathcal{K}_k(A, r_0)$ and uses the same Fletcher-Reeves $\beta_k$. CG, however, needs two global reductions per iteration that cannot be merged: $\rho_k = \langle r_k, z_k \rangle$ is needed to build $p_k$, and only afterwards can $q_k = Ap_k$ and the second reduction $\langle p_k, q_k \rangle$ be computed.

The pipelined variant removes that second synchronization point by carrying an extra vector $w_k = Az_k$ — the operator applied to the preconditioned residual $z_k = Mr_k$, which plain CG never forms. It gives access to $\delta_k = \langle w_k, z_k \rangle = \langle Az_k, z_k \rangle$, from which the denominator of $\alpha_k$ follows by a recurrence instead of a second reduction:

$$\langle p_k, Ap_k \rangle = \delta_k - \left| \frac{\rho_k}{\rho_{k-1}} \right|^2 \langle p_{k-1}, Ap_{k-1} \rangle .$$

Because $\rho_k$ and $\delta_k$ are available at the same point of the iteration, they are computed in a single global reduction that can be overlapped with the operator and preconditioner applies updating the auxiliary vectors $w_k$, $q_k = Ap_k$, $m_k = Mw_k$ and $n_k = Am_k$, all of which are advanced by short recurrences. In exact arithmetic the trajectory is identical to CG; in finite arithmetic the decoupling of dependencies amplifies round-off and the residuals can deviate from the classical iterates.

Possible issues:

1. Numerical instability: Due to the rearrangement of the operations, the method is known to be less stable than standard PCG.
2. The method is suitable for cases where a large number of iterations need to be performed and when the solver is distributed over a large number of distributed nodes. The advantage of lesser number of reductions that need to be performed comes at the cost of increased vector operations, and the cost of increased storage of vectors.
3. As the CG itself, this method performs very well for symmetric positive definite matrices but it is in general not suitable for general matrices.

#### References


- Sanan, P., Schnepp, S. M., May, D. A. *Pipelined, Flexible Krylov Subspace Methods.* SIAM Journal on Scientific Computing, 38 (5), 2016. https://doi.org/10.1137/15M1049130


#### Template Parameters

| | |
|---|---|
| *ValueType* | precision of matrix elements |

## 48.197.2 Member Function Documentation

### 48.197.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision>
bool gko::solver::PipeCg< ValueType >::apply_uses_initial_guess ( ) const  [inline], [override]
```

Return true as iterative solvers use the data in x as an initial guess.

**Returns**

true as iterative solvers use the data in x as an initial guess.

```
111 { return true; }
```

### 48.197.2.2 conj_transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::PipeCg< ValueType >::conj_transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

### 48.197.2.3 parse()

```
template<typename ValueType = default_precision>
static parameters_type gko::solver::PipeCg< ValueType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType >()
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value type of this class. |

**Returns**

parameters

#### 48.197.2.4 transpose()

```
template<typename ValueType = default_precision>
std::unique_ptr<LinOp> gko::solver::PipeCg< ValueType >::transpose ( ) const  [override],
[virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/pipe_cg.hpp

## 48.198 gko::multigrid::Pmis< ValueType, IndexType > Class Template Reference

Parallel modified independent set (Pmis) is the classical coarsening method introduced in the paper H.

```
#include <ginkgo/core/multigrid/pmis.hpp>
```

### Public Member Functions

- std::shared_ptr< const LinOp > get_system_matrix () const

  *Returns the system operator (matrix) of the linear system.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## 48.198.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::multigrid::Pmis< ValueType, IndexType >

Parallel modified independent set (Pmis) is the classical coarsening method introduced in the paper H.

De Sterck et al., "Reducing complexity in parallel algebraic multigrid preconditioners".

Pmis creates the coarse- and fine-points group according to the matrix value not the structure.

**Note**

The restriction is $P^T$ and the coarse operator the Galerkin product $P^T AP$, which assumes a symmetric positive definite system. Non-symmetric systems still coarsen, but without a convergence guarantee.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indices |

## 48.198.2 Member Function Documentation

### 48.198.2.1 get_system_matrix()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::multigrid::Pmis< ValueType, IndexType >::get_system_matrix
( ) const  [inline]
```

Returns the system operator (matrix) of the linear system.

**Returns**

the system operator (matrix)

```
57     {
58         return system_matrix_;
59     }
```

### 48.198.2.2 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::multigrid::Pmis< ValueType, IndexType >::parse (
             const config::pnode & config,
             const config::registry & context,
             const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following files:

- ginkgo/core/distributed/matrix.hpp
- ginkgo/core/multigrid/pmis.hpp

## 48.199 gko::config::pnode Class Reference

pnode describes a tree of properties.

`#include <ginkgo/core/config/property_tree.hpp>`

### Public Types

- enum tag_t

  *tag_t is the indicator for the current node storage.*

### Public Member Functions

- pnode ()

  *Default constructor: create an empty node.*
- pnode (bool boolean)

  *Constructor for bool.*
- template<typename T , std::enable_if_t< std::is_integral< T >::value > * = nullptr> pnode (T integer)

  *Constructor for integer with all integer type.*
- pnode (const std::string &str)

  *Constructor for string.*
- pnode (const char *str)

  *Constructor for char* (otherwise, it will use bool)*
- pnode (double real)

  *Constructor for double (and also float)*
- pnode (const array_type &array)

  *Constructor for array.*
- pnode (const map_type &map)

  *Constructor for map.*
- operator bool () const noexcept

  *bool conversion.*

- bool operator== (const pnode &rhs) const

  *Check whether the representing data of two pnodes are the same.*
- bool operator!= (const pnode &rhs) const

  *Check whether the representing data of two pnodes are different.*
- tag_t get_tag () const

  *Get the current node tag.*
- const array_type & get_array () const

  *Access the array stored in this property node.*
- const map_type & get_map () const

  *Access the map stored in this property node.*
- bool get_boolean () const

  *Access the boolean value stored in this property node.*
- std::int64_t get_integer () const
- double get_real () const

  *Access the real floating point value stored in this property node.*
- const std::string & get_string () const

  *Access the string stored in this property node.*
- const pnode & get (const std::string &key) const

  *This function is to access the data under the map.*
- const pnode & get (int index) const

  *This function is to access the data under the array.*

## 48.199.1 Detailed Description

pnode describes a tree of properties.

A pnode can either be empty, hold a value (a string, integer, real, or bool), contain an array of pnode., or contain a mapping between strings and pnodes.

## 48.199.2 Constructor & Destructor Documentation

### 48.199.2.1 pnode() [1/7]

```
gko::config::pnode::pnode (
            bool boolean )  [explicit]
```

Constructor for bool.

**Parameters**

| | |
|---|---|
| *boolean* | the bool type value |

#### 48.199.2.2 pnode() [2/7]

```
template<typename T , std::enable_if_t< std::is_integral< T >::value > * >
gko::config::pnode::pnode (
            T integer )  [explicit]
```

Constructor for integer with all integer type.

**Template Parameters**

| | |
|---|---|
| *T* | input type |

**Parameters**

| | |
|---|---|
| *integer* | the integer type value |

```
210                     :  tag_(tag_t::integer)
211 {
212     if (integer > std::numeric_limits<std::int64_t>::max() ||
213         (std::is_signed<T>::value &&
214          integer < std::numeric_limits<std::int64_t>::min())) {
215         throw std::runtime_error("The input is out of the range of int64_t.");
216     }
217     union_data_.integer_ = static_cast<std::int64_t>(integer);
218 }
```

#### 48.199.2.3 pnode() [3/7]

```
gko::config::pnode::pnode (
            const std::string & str )  [explicit]
```

Constructor for string.

**Parameters**

| | |
|---|---|
| *str* | string type value |

#### 48.199.2.4 pnode() [4/7]

```
gko::config::pnode::pnode (
            const char * str )  [explicit]
```

Constructor for char* (otherwise, it will use bool)

**Parameters**

| | |
|---|---|
| *str* | the string like "..." |

#### 48.199.2.5 pnode() [5/7]

```
gko::config::pnode::pnode (
            double real )  [explicit]
```

Constructor for double (and also float)

**Parameters**

| | |
|---|---|
| *real* | the floating point type value |

#### 48.199.2.6 pnode() [6/7]

```
gko::config::pnode::pnode (
            const array_type & array )  [explicit]
```

Constructor for array.

**Parameters**

| | |
|---|---|
| *array* | an pnode array |

#### 48.199.2.7 pnode() [7/7]

```
gko::config::pnode::pnode (
            const map_type & map )  [explicit]
```

Constructor for map.

**Parameters**

| | |
|---|---|
| *map* | a (string, pnode)-map |

### 48.199.3 Member Function Documentation

#### 48.199.3.1 get() [1/2]

```
const pnode& gko::config::pnode::get (
            const std::string & key ) const
```

This function is to access the data under the map.

It will throw error when it does not hold a map. When access non-existent key in the map, it will return an empty node.

**Parameters**

| | |
|---|---|
| *key* | the key for the node of the map |

**Returns**

node. If the map does not have the key, return an empty node.

#### 48.199.3.2 get() [2/2]

```
const pnode& gko::config::pnode::get (
            int index ) const
```

This function is to access the data under the array.

It will throw error when it does not hold an array or access out-of-bound index.

**Parameters**

| | |
|---|---|
| *index* | the node index in array |

**Returns**

node.

#### 48.199.3.3 get_array()

```
const array_type& gko::config::pnode::get_array ( ) const
```

Access the array stored in this property node.

Throws gko::InvalidStateError if the property node does not store an array.

**Returns**

the array

#### 48.199.3.4 get_boolean()

```
bool gko::config::pnode::get_boolean ( ) const
```

Access the boolean value stored in this property node.

Throws gko::InvalidStateError if the property node does not store a boolean value.

**Returns**

the boolean value

#### 48.199.3.5 get_integer()

```
std::int64_t gko::config::pnode::get_integer ( ) const
```

- Access the integer value stored in this property node. Throws gko::InvalidStateError if the property node does not store an integer value.

**Returns**

the integer value

#### 48.199.3.6 get_map()

```
const map_type& gko::config::pnode::get_map ( ) const
```

Access the map stored in this property node.

Throws gko::InvalidStateError if the property node does not store a map.

**Returns**

the map

#### 48.199.3.7 get_real()

```
double gko::config::pnode::get_real ( ) const
```

Access the real floating point value stored in this property node.

Throws gko::InvalidStateError if the property node does not store a real value

**Returns**

the real floating point value

#### 48.199.3.8 get_string()

```
const std::string& gko::config::pnode::get_string ( ) const
```

Access the string stored in this property node.

Throws gko::InvalidStateError if the property node does not store a string.

**Returns**

the string

#### 48.199.3.9 get_tag()

```
tag_t gko::config::pnode::get_tag ( ) const
```

Get the current node tag.

**Returns**

the tag

#### 48.199.3.10 operator bool()

```
gko::config::pnode::operator bool ( ) const  [explicit], [noexcept]
```

bool conversion.

It's true if and only if it is not empty.

The documentation for this class was generated from the following file:

- ginkgo/core/config/property_tree.hpp

## 48.200 gko::log::polymorphic_object_data Struct Reference

Struct representing PolymorphicObject related data.

```
#include <ginkgo/core/log/record.hpp>
```

### 48.200.1 Detailed Description

Struct representing PolymorphicObject related data.

The documentation for this struct was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.201 gko::PolymorphicObject Class Reference

A PolymorphicObject is the abstract base for all "heavy" objects in Ginkgo that behave polymorphically.

`#include <ginkgo/core/base/polymorphic_object.hpp>`

### Public Member Functions

- virtual void validate_data () const

  *Throws gko::InvalidData exception if we found the data inside the object does not fulfill certain property up to our knowledge.*
- std::shared_ptr< const Executor > get_executor () const noexcept

  *Returns the Executor of the object.*

### 48.201.1 Detailed Description

A PolymorphicObject is the abstract base for all "heavy" objects in Ginkgo that behave polymorphically.

It defines the basic utilities (copying moving, cloning, clearing the objects) for all such objects. It takes into account that these objects are dynamically allocated, managed by smart pointers, and used polymorphically. Additionally, it assumes their data can be allocated on different executors, and that they can be copied between those executors.

**Note**

Most of the public methods of this class should not be overridden directly, and are thus not virtual. Instead, there are equivalent protected methods (ending in <method_name>_impl) that should be overridden instead. This allows polymorphic objects to implement default behavior around virtual methods (parameter checking, type casting).

### 48.201.2 Member Function Documentation

#### 48.201.2.1 get_executor()

```
std::shared_ptr<const Executor> gko::PolymorphicObject::get_executor ( ) const  [inline],
[noexcept]
```

Returns the Executor of the object.

**Returns**

Executor of the object

```
70     {
71         return exec_;
72     }
```

Referenced by gko::matrix::Csr< ValueType, IndexType >::inv_scale(), and gko::matrix::Csr< ValueType, Index↩ Type >::scale().

The documentation for this class was generated from the following file:

- ginkgo/core/base/polymorphic_object.hpp

## 48.202 gko::precision_reduction Class Reference

This class is used to encode storage precisions of low precision algorithms.

```
#include <ginkgo/core/base/types.hpp>
```

### Public Types

- using storage_type = uint8

  *The underlying datatype used to store the encoding.*

### Public Member Functions

- constexpr precision_reduction () noexcept

  *Creates a default precision_reduction encoding.*
- constexpr precision_reduction (storage_type preserving, storage_type nonpreserving) noexcept

  *Creates a precision_reduction encoding with the specified number of conversions.*
- constexpr operator storage_type () const noexcept

  *Extracts the raw data of the encoding.*
- constexpr storage_type get_preserving () const noexcept

  *Returns the number of preserving conversions in the encoding.*
- constexpr storage_type get_nonpreserving () const noexcept

  *Returns the number of non-preserving conversions in the encoding.*

### Static Public Member Functions

- constexpr static precision_reduction autodetect () noexcept

  *Returns a special encoding which instructs the algorithm to automatically detect the best precision.*
- constexpr static precision_reduction common (precision_reduction x, precision_reduction y) noexcept

  *Returns the common encoding of input encodings.*

### 48.202.1 Detailed Description

This class is used to encode storage precisions of low precision algorithms.

Some algorithms in Ginkgo can improve their performance by storing parts of the data in lower precision, while doing computation in full precision. This class is used to encode the precisions used to store the data. From the user's perspective, some algorithms can provide a parameter for fine-tuning the storage precision. Commonly, the special value returned by precision_reduction::autodetect() should be used to allow the algorithm to automatically choose an appropriate value, though manually selected values can be used for fine-tuning.

In general, a lower precision floating point value can be obtained by either dropping some of the insignificant bits of the significand (keeping the same number of exponent bits, and thus preserving the range of representable values) or using one of the hardware or software supported conversions between IEEE formats, such as double to float or float to half (reducing both the number of exponent, as well as significand bits, and thus decreasing the range of representable values).

The precision_reduction class encodes the lower precision format relative to the base precision used and the algorithm in question. The encoding is done by specifying the amount of range non-preserving conversions and the amount of range preserving conversions that should be done on the base precision to obtain the lower precision format. For example, starting with a double precision value (11 exp, 52 sig. bits), the encoding specifying 1 non-preserving conversion and 1 preserving conversion would first use a hardware-supported non-preserving conversion to obtain a single precision value (8 exp, 23 sig. bits), followed by a preserving bit truncation to obtain a value with 8 exponent and 7 significand bits. Note that non-preserving conversion are always done first, as preserving conversions usually result in datatypes that are not supported by builtin conversions (thus, it is generally not possible to apply a non-preserving conversion to the result of a preserving conversion).

If the specified conversion is not supported by the algorithm, it will most likely fall back to using full precision for storing the data. Refer to the documentation of specific algorithms using this class for details about such special cases.

### 48.202.2 Constructor & Destructor Documentation

#### 48.202.2.1 precision_reduction() [1/2]

```
constexpr gko::precision_reduction::precision_reduction ( )  [inline], [constexpr], [noexcept]
```

Creates a default precision_reduction encoding.

This encoding represents the case where no conversions are performed.

```
272 :  data_{0x0} {}
```

Referenced by common().

#### 48.202.2.2 precision_reduction() [2/2]

```
constexpr gko::precision_reduction::precision_reduction (
            storage_type preserving,
            storage_type nonpreserving )  [inline], [constexpr], [noexcept]
```

Creates a precision_reduction encoding with the specified number of conversions.

**Parameters**

| | |
|---|---|
| *preserving* | the number of range preserving conversion |
| *nonpreserving* | the number of range non-preserving conversions |

```
283         :  data_((GKO_ASSERT(preserving < (0x1 « preserving_bits) - 1),
284                   GKO_ASSERT(nonpreserving < (0x1 « nonpreserving_bits) - 1),
285                   (preserving « nonpreserving_bits) | nonpreserving))
286     {}
```

## 48.202.3 Member Function Documentation

### 48.202.3.1 autodetect()

```
constexpr static precision_reduction gko::precision_reduction::autodetect ( )  [inline], [static],
[constexpr], [noexcept]
```

Returns a special encoding which instructs the algorithm to automatically detect the best precision.

**Returns**

a special encoding instructing the algorithm to automatically detect the best precision.

```
326     {
327         return precision_reduction{preserving_mask | nonpreserving_mask};
328     }
```

### 48.202.3.2 common()

```
constexpr static precision_reduction gko::precision_reduction::common (
              precision_reduction x,
              precision_reduction y )  [inline], [static], [constexpr], [noexcept]
```

Returns the common encoding of input encodings.

The common encoding is defined as the encoding that does not have more preserving, nor non-preserving conversions than the input encodings.

**Parameters**

| | |
|---|---|
| *x* | an encoding |
| *y* | an encoding |

**Returns**

the common encoding of `x` and `y`

```
343     {
344         return precision_reduction(
```

```
345             min(x.data_ & preserving_mask, y.data_ & preserving_mask) |
346             min(x.data_ & nonpreserving_mask, y.data_ & nonpreserving_mask));
347     }
```

References precision_reduction().

#### 48.202.3.3 get_nonpreserving()

```
constexpr storage_type gko::precision_reduction::get_nonpreserving ( ) const  [inline], [constexpr],
[noexcept]
```

Returns the number of non-preserving conversions in the encoding.

**Returns**

the number of non-preserving conversions in the encoding.

```
314     {
315         return data_ & nonpreserving_mask;
316     }
```

#### 48.202.3.4 get_preserving()

```
constexpr storage_type gko::precision_reduction::get_preserving ( ) const  [inline], [constexpr],
[noexcept]
```

Returns the number of preserving conversions in the encoding.

**Returns**

the number of preserving conversions in the encoding.

```
304     {
305         return (data_ & preserving_mask) » nonpreserving_bits;
306     }
```

#### 48.202.3.5 operator storage_type()

```
constexpr gko::precision_reduction::operator storage_type ( ) const  [inline], [constexpr],
[noexcept]
```

Extracts the raw data of the encoding.

**Returns**

the raw data of the encoding

```
294     {
295         return data_;
296     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/types.hpp

## 48.203 gko::Preconditionable Class Reference

A LinOp implementing this interface can be preconditioned.

`#include <ginkgo/core/base/lin_op.hpp>`

### Public Member Functions

- virtual std::shared_ptr< const LinOp > get_preconditioner () const

  *Returns the preconditioner operator used by the Preconditionable.*
- virtual void set_preconditioner (std::shared_ptr< const LinOp > new_precond)

  *Sets the preconditioner operator used by the Preconditionable.*

### 48.203.1 Detailed Description

A LinOp implementing this interface can be preconditioned.

### 48.203.2 Member Function Documentation

#### 48.203.2.1 get_preconditioner()

```
virtual std::shared_ptr<const LinOp> gko::Preconditionable::get_preconditioner ( ) const
[inline], [virtual]
```

Returns the preconditioner operator used by the Preconditionable.

**Returns**

the preconditioner operator used by the Preconditionable

```
768     {
769         return preconditioner_;
770     }
```

Referenced by gko::solver::EnablePreconditionedIterativeSolver< ValueType, Cg< ValueType > >::get_unprojected_preconditioner(), and gko::solver::EnablePreconditionable< Chebyshev< ValueType > >::operator=().

#### 48.203.2.2 set_preconditioner()

```
virtual void gko::Preconditionable::set_preconditioner (
            std::shared_ptr< const LinOp > new_precond )  [inline], [virtual]
```

Sets the preconditioner operator used by the Preconditionable.

**Parameters**

| | |
|---|---|
| *new_precond* | the new preconditioner operator used by the Preconditionable |

Reimplemented in gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, gko::solver::EnablePreconditionedI
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Idr< ValueType > >, gko::solver::EnablePreconditionedIterativeSolve
gko::solver::EnablePreconditionedIterativeSolver< ValueType, CbGmres< ValueType > >, gko::solver::EnablePreconditionedIterativ
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Bicg< ValueType > >, gko::solver::EnablePreconditionedIterativeSolv
gko::solver::EnablePreconditionedIterativeSolver< ValueType, PipeCg< ValueType > >, gko::solver::EnablePreconditionedIterativeS
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Bicgstab< ValueType > >, gko::solver::EnablePreconditionedIterative
gko::solver::EnablePreconditionedIterativeSolver< ValueType, Cg< ValueType > >, gko::solver::EnablePreconditionable< DerivedT
gko::solver::EnablePreconditionable< PipeCg< ValueType > >, gko::solver::EnablePreconditionable< Bicgstab< ValueType > >,
gko::solver::EnablePreconditionable< Gmres< ValueType > >, gko::solver::EnablePreconditionable< CbGmres< ValueType > >,
gko::solver::EnablePreconditionable< Fcg< ValueType > >, gko::solver::EnablePreconditionable< Minres< ValueType > >,
gko::solver::EnablePreconditionable< Idr< ValueType > >, gko::solver::EnablePreconditionable< Cgs< ValueType > >,
gko::solver::EnablePreconditionable< Gcr< ValueType > >, gko::solver::EnablePreconditionable< Cg< ValueType > >,
gko::solver::EnablePreconditionable< Bicg< ValueType > >, and gko::solver::EnablePreconditionable< Chebyshev< ValueType >

```
779     {
780         preconditioner_ = new_precond;
781     }
```

Referenced by gko::solver::EnablePreconditionable< Chebyshev< ValueType > >::set_preconditioner().

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.204 gko::log::ProfilerHook Class Reference

This Logger can be used to annotate the execution of Ginkgo functionality with profiler-specific ranges.

`#include <ginkgo/core/log/profiler_hook.hpp>`

### Classes

- class NestedSummaryWriter

  *Receives the results from ProfilerHook::create_nested_summary().*
- class SummaryWriter

  *Receives the results from ProfilerHook::create_summary().*
- class TableSummaryWriter

  *Writes the results from ProfilerHook::create_summary() and ProfilerHook::create_nested_summary() to a ASCII table in Markdown format.*

### Public Member Functions

- void set_object_name (ptr_param< const PolymorphicObject > obj, std::string name)

  *Sets the name for an object to be profiled.*
- void set_synchronization (bool synchronize)

  *Should the events call executor->synchronize on operations and copy/allocation? This leads to a certain overhead, but makes the execution timeline of kernels synchronous.*
- profiling_scope_guard user_range (const char *name) const

  *Creates a scope guard for a user-defined range to be included in the profile.*

## Static Public Member Functions

- static std::shared_ptr< ProfilerHook > create_tau (bool initialize=true)

  *Creates a logger annotating Ginkgo events with TAU ranges via PerfStubs.*
- static std::shared_ptr< ProfilerHook > create_vtune ()

  *Creates a logger annotating Ginkgo events with VTune ITT ranges.*
- static std::shared_ptr< ProfilerHook > create_nvtx (uint32 color_argb=color_yellow_argb)

  *Creates a logger annotating Ginkgo events with NVTX ranges for CUDA.*
- static std::shared_ptr< ProfilerHook > create_roctx ()

  *Creates a logger annotating Ginkgo events with ROCTX ranges for HIP.*
- static std::shared_ptr< ProfilerHook > create_for_executor (std::shared_ptr< const Executor > exec)

  *Creates a logger annotating Ginkgo events with the most suitable backend for the given executor: NVTX for NSight Systems in CUDA, ROCTX for rocprof in HIP, TAU for everything else.*
- static std::shared_ptr< ProfilerHook > create_summary (std::shared_ptr< Timer > timer=std::make_shared< CpuTimer >(), std::unique_ptr< SummaryWriter > writer=std::make_unique< TableSummaryWriter >(), bool debug_check_nesting=false)

  *Creates a logger measuring the runtime of Ginkgo events and printing a summary when it is destroyed.*
- static std::shared_ptr< ProfilerHook > create_nested_summary (std::shared_ptr< Timer > timer=std::make_shared< CpuTimer >(), std::unique_ptr< NestedSummaryWriter > writer=std::make_unique< TableSummaryWriter >(), bool debug_check_nesting=false)

  *Creates a logger measuring the runtime of Ginkgo events in a nested fashion and printing a summary when it is destroyed.*
- static std::shared_ptr< ProfilerHook > create_custom (hook_function begin, hook_function end)

  *Creates a logger annotating Ginkgo events with a custom set of functions for range begin and end.*

## Static Public Attributes

- constexpr static uint32 color_yellow_argb = 0xFFFFCB05U

  *The Ginkgo yellow background color as packed 32 bit ARGB value.*

### 48.204.1 Detailed Description

This Logger can be used to annotate the execution of Ginkgo functionality with profiler-specific ranges.

It currently supports TAU, VTune, NSightSystems (NVTX) and rocPROF(ROCTX) and custom profiler hooks.

The Logger should be attached to the Executor that is being used to run the application for a full, program-wide annotation, or to individual objects to only highlight events caused directly by them (not operations and memory allocations though)

### 48.204.2 Member Function Documentation

#### 48.204.2.1 create_nested_summary()

```
static std::shared_ptr<ProfilerHook> gko::log::ProfilerHook::create_nested_summary (
            std::shared_ptr< Timer > timer = std::make_shared< CpuTimer >(),
            std::unique_ptr< NestedSummaryWriter > writer = std::make_unique< TableSummaryWriter >(),
            bool debug_check_nesting = false )  [static]
```

Creates a logger measuring the runtime of Ginkgo events in a nested fashion and printing a summary when it is destroyed.

**Parameters**

| | |
|---|---|
| *timer* | The timer used to record time points. |
| *writer* | The NestedSummaryWriter to receive the performance results. |
| *debug_check_nesting* | Enable this flag if the output looks like it might contain incorrect nesting. This increases the overhead slightly, but recognizes mismatching push/pop pairs on the range stack. |

**Note**

For this logger to provide reliable GPU timings, either use Timer::create_for_executor or enable synchronization via `set_synchronization(true)`.

### 48.204.2.2 create_nvtx()

```
static std::shared_ptr<ProfilerHook> gko::log::ProfilerHook::create_nvtx (
            uint32 color_argb = color_yellow_argb )  [static]
```

Creates a logger annotating Ginkgo events with NVTX ranges for CUDA.

**Parameters**

| | |
|---|---|
| *color_argb* | The color of the NVTX ranges in the NSight Systems output. It has to be a 32 bit packed ARGB value. |

### 48.204.2.3 create_summary()

```
static std::shared_ptr<ProfilerHook> gko::log::ProfilerHook::create_summary (
            std::shared_ptr< Timer > timer = std::make_shared< CpuTimer >(),
            std::unique_ptr< SummaryWriter > writer = std::make_unique< TableSummaryWriter >(),
            bool debug_check_nesting = false )  [static]
```

Creates a logger measuring the runtime of Ginkgo events and printing a summary when it is destroyed.

**Parameters**

| | |
|---|---|
| *timer* | The timer used to record time points. |
| *writer* | The SummaryWriter to receive the performance results. |
| *debug_check_nesting* | Enable this flag if the output looks like it might contain incorrect nesting. This increases the overhead slightly, but recognizes mismatching push/pop pairs on the range stack. |

**Note**

For this logger to provide reliable GPU timings, either use Timer::create_for_executor or enable synchronization via `set_synchronization(true)`.

#### 48.204.2.4 create_tau()

```
static std::shared_ptr<ProfilerHook> gko::log::ProfilerHook::create_tau (
            bool initialize = true )  [static]
```

Creates a logger annotating Ginkgo events with TAU ranges via PerfStubs.

**Parameters**

| | |
|---|---|
| *initialize* | Should we call TAU's initialization and finalization functions, or does the application take care of it? The initialization will happen immediately, the finalization at program exit. |

#### 48.204.2.5 set_object_name()

```
void gko::log::ProfilerHook::set_object_name (
            ptr_param< const PolymorphicObject > obj,
            std::string name )
```

Sets the name for an object to be profiled.

Every instance of that object in the profile will be replaced by the name instead of its runtime type.

**Parameters**

| | |
|---|---|
| *obj* | the object |
| *name* | its name |

#### 48.204.2.6 user_range()

```
profiling_scope_guard gko::log::ProfilerHook::user_range (
            const char * name ) const
```

Creates a scope guard for a user-defined range to be included in the profile.

**Parameters**

| | |
|---|---|
| *name* | the name of the range |

**Returns**

the scope guard. It will begin a range immediately and end it at the end of its scope.

The documentation for this class was generated from the following file:

- ginkgo/core/log/profiler_hook.hpp

# 48.205 gko::log::profiling_scope_guard Class Reference

Scope guard that annotates its scope with the provided profiler hooks.

`#include <ginkgo/core/log/profiler_hook.hpp>`

## Public Member Functions

- profiling_scope_guard ()

  *Creates an empty (moved-from) scope guard.*
- profiling_scope_guard (const char *name, profile_event_category category, ProfilerHook::hook_function begin, ProfilerHook::hook_function end)

  *Creates the scope guard.*
- ~profiling_scope_guard ()

  *Calls the range end function if the scope guard was not moved from.*

## 48.205.1 Detailed Description

Scope guard that annotates its scope with the provided profiler hooks.

## 48.205.2 Constructor & Destructor Documentation

### 48.205.2.1 profiling_scope_guard()

```
gko::log::profiling_scope_guard::profiling_scope_guard (
            const char * name,
            profile_event_category category,
            ProfilerHook::hook_function begin,
            ProfilerHook::hook_function end )
```

Creates the scope guard.

**Parameters**

| | |
|---|---|
| *name* | the name of the profiler range |
| *category* | the category of the profiler range |
| *begin* | the hook function to begin a range |
| *end* | the hook function to end a range |

The documentation for this class was generated from the following file:

- ginkgo/core/log/profiler_hook.hpp

# 48.206 gko::ptr_param< T > Class Template Reference

This class is used for function parameters in the place of raw pointers.

`#include <ginkgo/core/base/utils_helper.hpp>`

## Public Member Functions

- ptr_param (T *ptr)

  *Initializes the ptr_param from a raw pointer.*
- template<typename U , std::enable_if_t< std::is_base_of< T, U >::value > * = nullptr>
  ptr_param (const std::shared_ptr< U > &ptr)

  *Initializes the ptr_param from a shared_ptr.*
- template<typename U , typename Deleter , std::enable_if_t< std::is_base_of< T, U >::value > * = nullptr>
  ptr_param (const std::unique_ptr< U, Deleter > &ptr)

  *Initializes the ptr_param from a unique_ptr.*
- template<typename U , std::enable_if_t< std::is_base_of< T, U >::value > * = nullptr>
  ptr_param (const ptr_param< U > &ptr)

  *Initializes the ptr_param from a ptr_param of a derived type.*
- T & operator* () const
- T * operator-> () const
- T * get () const
- operator bool () const

### 48.206.1 Detailed Description

**template<typename T>**
**class gko::ptr_param< T >**

This class is used for function parameters in the place of raw pointers.

Pointer parameters should be used for everything that does not involve transfer of ownership. It can be converted to from raw pointers, shared pointers and unique pointers of the specified type or any derived type. This allows functions to be called without having to use gko::lend or calling .get() for every pointer argument. It probably has no use outside of function parameters, as it is immutable.

**Template Parameters**

| | |
|---|---|
| *T* | the pointed-to type |

### 48.206.2 Member Function Documentation

#### 48.206.2.1 get()

```
template<typename T>
T* gko::ptr_param< T >::get ( ) const  [inline]
```

**Returns**

the underlying pointer.

```
77 { return ptr_; }
```

Referenced by gko::as(), gko::Executor::copy_from(), gko::matrix::Diagonal< ValueType >::inverse_apply(), gko::ptr_param< T >::ptr_param(), and gko::matrix::Diagonal< ValueType >::rapply().

#### 48.206.2.2 operator bool()

```
template<typename T>
gko::ptr_param< T >::operator bool ( ) const  [inline], [explicit]
```

**Returns**

true iff the underlying pointer is non-null.

```
80 { return ptr_; }
```

#### 48.206.2.3 operator*()

```
template<typename T>
T& gko::ptr_param< T >::operator* ( ) const  [inline]
```

**Returns**

a reference to the underlying pointee.

```
71 { return *ptr_; }
```

#### 48.206.2.4 operator->()

```
template<typename T>
T* gko::ptr_param< T >::operator-> ( ) const  [inline]
```

**Returns**

the underlying pointer.

```
74 { return ptr_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/utils_helper.hpp

## 48.207 gko::syn::range< Start, End, Step > Struct Template Reference

range records start, end, step in template

`#include <ginkgo/core/synthesizer/containers.hpp>`

### 48.207.1 Detailed Description

**template<int Start, int End, int Step = 1>**
**struct gko::syn::range< Start, End, Step >**

range records start, end, step in template

**Template Parameters**

| | |
|---|---|
| *Start* | start of range |
| *End* | end of range |
| *Step* | step of range. default is 1 |

The documentation for this struct was generated from the following file:

- ginkgo/core/synthesizer/containers.hpp

## 48.208 gko::range< Accessor > Class Template Reference

A range is a multidimensional view of the memory.

`#include <ginkgo/core/base/range.hpp>`

### Public Types

- using accessor = Accessor

  *The type of the underlying accessor.*

### Public Member Functions

- ~range ()=default

  *Use the default destructor.*

- template<typename... AccessorParams, typename = std::enable_if_t< sizeof...(AccessorParams) != 1 || !std::is_same< range, std::decay<detail::head_t<AccessorParams...>>>::value>>
  constexpr range (AccessorParams &&... params)

  *Creates a new range.*

- template<typename... DimensionTypes>
  constexpr auto operator() (DimensionTypes &&... dimensions) const -> decltype(std::declval< accessor >()(std::forward< DimensionTypes >(dimensions)...))

  *Returns a value (or a sub-range) with the specified indexes.*

- template<typename OtherAccessor >
  const range & operator= (const range< OtherAccessor > &other) const
- const range & operator= (const range &other) const

  *Assigns another range to this range.*

- constexpr size_type length (size_type dimension) const

  *Returns the length of the specified dimension of the range.*

- constexpr const accessor * operator-> () const noexcept

  *Returns a pointer to the accessor.*

- constexpr const accessor & get_accessor () const noexcept

  *Returns a reference to the accessor.*

### Static Public Attributes

- static constexpr size_type dimensionality = accessor::dimensionality

  *The number of dimensions of the range.*

## 48.208.1 Detailed Description

**template<typename Accessor>**
**class gko::range< Accessor >**

A range is a multidimensional view of the memory.

The range does not store any of its values by itself. Instead, it obtains the values through an accessor (e.g. accessor::row_major) which describes how the indexes of the range map to physical locations in memory.

There are several advantages of using ranges instead of plain memory pointers:

1. Code using ranges is easier to read and write, as there is no need for index linearizations.
2. Code using ranges is safer, as it is impossible to accidentally miscalculate an index or step out of bounds, since range accessors perform bounds checking in debug builds. For performance, this can be disabled in release builds by defining the `NDEBUG` flag.
3. Ranges enable generalized code, as algorithms can be written independent of the memory layout. This does not impede various optimizations based on memory layout, as it is always possible to specialize algorithms for ranges with specific memory layouts.
4. Ranges have various pointwise operations predefined, which reduces the amount of loops that need to be written.

### 48.208.1.1 Range operations

Ranges define a complete set of pointwise unary and binary operators which extend the basic arithmetic operators in C++, as well as a few pointwise operations and mathematical functions useful in ginkgo, and a couple of non-pointwise operations. Compound assignment (+=, *=, etc.) is not yet supported at this moment. Here is a complete list of operations:

- standard unary operations: +, -, !, ~
- standard binary operations: +, * (this is pointwise, not matrix multiplication), /, %, <, >, <=, >=, ==, !=, ||, &&, |, &, ^, <<, >>
- useful unary functions: `zero`, `one`, `abs`, `real`, `imag`, `conj`, `squared_norm`
- useful binary functions: `min`, `max`

All binary pointwise operations also work as expected if one of the operands is a scalar and the other is a range. The scalar operand will have the effect as if it was a range of the same size as the other operand, filled with the specified scalar.

Two "global" functions `transpose` and `mmul` are also supported. `transpose` transposes the first two dimensions of the range (i.e. `transpose(r)(i, j, ...) == r(j, i, ...)`). `mmul` performs a (batched) matrix multiply of the ranges - the first two dimensions represent the matrices, while the rest represent the batch. For example, given the ranges `r1` and `r2` of dimensions `(3, 2, 3)` and `(2, 4, 3)`, respectively, `mmul(r1, r2)` will return a range of dimensions `(3, 4, 3)`, obtained by multiplying the 3 frontal slices of the range, and stacking the result back vertically.

#### 48.208.1.2 Compound operations

Multiple range operations can be combined into a single expression. For example, an "axpy" operation can be obtained using `y = alpha * x + y`, where `x` an `y` are ranges, and `alpha` is a scalar. Range operations are optimized for memory access, and the above code does not allocate additional storage for intermediate ranges `alpha * x` or `alpha * x + y`. In fact, the entire computation is done during the assignment, and the results of operations + and ∗ only register the data, and the types of operations that will be computed once the results are needed.

It is possible to store and reuse these intermediate expressions. The following example will overwrite the range `x` with it's 4th power:

```
 {c++}
auto square = x * x;  // this is range constructor, not range assignment!
x = square;  // overwrites x with x*x (this is range assignment)
x = square;  // overwrites new x (x*x) with (x*x)*(x*x) (as is this)
```

#### 48.208.1.3 Caveats

__`mmul` is not a highly-optimized BLAS-3 version of the matrix multiplication.__ The current design of ranges and accessors prevents that, so if you need a high-performance matrix multiplication, you should use one of the libraries that provide that, or implement your own (you can use pointwise range operations to help simplify that). However, range design might get improved in the future to allow efficient implementations of BLAS-3 kernels.

Aliasing the result range in `mmul` and `transpose` is not allowed. Constructs like `A = transpose(A)`, `A = mmul(A, A)`, or `A = mmul(A, A) + C` lead to undefined behavior. However, aliasing input arguments is allowed: `C = mmul(A, A)`, and even `C = mmul(A, A) + C` is valid code (in the last example, only pointwise operations are aliased). `C = mmul(A, A + C)` is not valid though.

#### 48.208.1.4 Examples

The range unit tests in core/test/base/range.cpp contain lots of simple 1-line examples of range operations. The accessor unit tests in core/test/base/range.cpp show how to use ranges with concrete accessors, and how to use range slices using `span`s as arguments to range function call operator. Finally, examples/range contains a complete example where ranges are used to implement a simple version of the right-looking LU factorization.

**Template Parameters**

| | |
|---|---|
| *Accessor* | underlying accessor of the range |

### 48.208.2 Constructor & Destructor Documentation

#### 48.208.2.1 range()

```
template<typename Accessor>
template<typename...  AccessorParams, typename = std::enable_if_t< sizeof...(AccessorParams)
!= 1 || !std::is_same< range, std::decay<detail::head_t<AccessorParams...>>>::value>>
constexpr gko::range< Accessor >::range (
            AccessorParams &&...  params )  [inline], [explicit], [constexpr]
```

Creates a new range.

**Template Parameters**

| | |
|---|---|
| *AccessorParam* | types of parameters forwarded to the accessor constructor |

**Parameters**

| | |
|---|---|
| *params* | parameters forwarded to Accessor constructor. |

```
334         :  accessor_{std::forward<AccessorParams>(params)...}
335     {}
```

## 48.208.3 Member Function Documentation

### 48.208.3.1 get_accessor()

```
template<typename Accessor>
constexpr const accessor& gko::range< Accessor >::get_accessor ( ) const  [inline], [constexpr],
[noexcept]
```

Returns a reference to the accessor.

**Returns**

reference to the accessor

```
428     {
429         return accessor_;
430     }
```

Referenced by gko::range< Accessor >::operator=().

### 48.208.3.2 length()

```
template<typename Accessor>
constexpr size_type gko::range< Accessor >::length (
             size_type dimension ) const  [inline], [constexpr]
```

Returns the length of the specified dimension of the range.

**Parameters**

| | |
|---|---|
| *dimension* | the dimensions whose length is returned |

**Returns**

the length of the `dimension`-th dimension of the range

```
406     {
407         return accessor_.length(dimension);
408     }
```

Referenced by gko::matrix_data< ValueType, IndexType >::matrix_data().

#### 48.208.3.3 operator()()

```
template<typename Accessor>
template<typename...  DimensionTypes>
constexpr auto gko::range< Accessor >::operator() (
            DimensionTypes &&...  dimensions ) const -> decltype(std::declval<accessor>()(
std::forward<DimensionTypes>(dimensions)...))    [inline], [constexpr]
```

Returns a value (or a sub-range) with the specified indexes.

**Template Parameters**

| | |
|---|---|
| *DimensionTypes* | The types of indexes. Supported types depend on the underlying accessor, but are usually either integer types or spans. If at least one index is a span, the returned value will be a sub-range. |

**Parameters**

| | |
|---|---|
| *dimensions* | the indexes of the values. |

**Returns**

a value on position `(dimensions...)`.

```
353     {
354         static_assert(sizeof...(DimensionTypes) <= dimensionality,
355                       "Too many dimensions in range call");
356         return accessor_(std::forward<DimensionTypes>(dimensions)...);
357     }
```

References gko::range< Accessor >::dimensionality.

#### 48.208.3.4 operator->()

```
template<typename Accessor>
constexpr const accessor* gko::range< Accessor >::operator-> ( ) const  [inline], [constexpr],
[noexcept]
```

Returns a pointer to the accessor.

Can be used to access data and functions of a specific accessor.

**Returns**

pointer to the accessor

```
418     {
419         return &accessor_;
420     }
```

### 48.208.3.5 operator=() [1/2]

```
template<typename Accessor>
const range& gko::range< Accessor >::operator= (
            const range< Accessor > & other ) const  [inline]
```

Assigns another range to this range.

The order of assignment is defined by the accessor of this range, thus the memory access will be optimized for the resulting range, and not for the other range. If the sizes of two ranges do not match, the result is undefined. Sizes of the ranges are checked at runtime in debug builds.

#### Note

Temporary accessors are allowed to define the implementation of the assignment as deleted, so do not expect `r1 * r2 = r2` to work.

#### Parameters

| | |
|---|---|
| *other* | the range to copy the data from |

```
390     {
391         GKO_ASSERT(detail::equal_dimensions(*this, other));
392         accessor_.copy_from(other.get_accessor());
393         return *this;
394     }
```

References gko::range< Accessor >::get_accessor().

### 48.208.3.6 operator=() [2/2]

```
template<typename Accessor>
template<typename OtherAccessor >
const range& gko::range< Accessor >::operator= (
            const range< OtherAccessor > & other ) const  [inline]
```

This is a version of the function which allows to copy between ranges of different accessors.

#### Template Parameters

| | |
|---|---|
| *OtherAccessor* | accessor of the other range |

```
370     {
371         GKO_ASSERT(detail::equal_dimensions(*this, other));
372         accessor_.copy_from(other);
373         return *this;
374     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/range.hpp

# 48.209 gko::reorder::Rcm< ValueType, IndexType > Class Template Reference

Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.

`#include <ginkgo/core/reorder/rcm.hpp>`

## Public Member Functions

- std::shared_ptr< const PermutationMatrix > get_permutation () const

  *Gets the permutation (permutation matrix, output of the algorithm) of the linear operator.*
- std::shared_ptr< const PermutationMatrix > get_inverse_permutation () const

  *Gets the inverse permutation (permutation matrix, output of the algorithm) of the linear operator.*

## 48.209.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::reorder::Rcm< ValueType, IndexType >**

Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.

Such a reordering typically also significantly reduces fill-in, though usually not as effective as more complex algorithms, specifically AMD and nested dissection schemes. The advantage of this algorithm is its low runtime. It requires the input matrix to be structurally symmetric.

**Note**

This class is derived from polymorphic object but is not a LinOp as it does not make sense for this class to implement the apply methods. The objective of this class is to generate a reordering/permutation vector (in the form of the Permutation matrix), which can be used to apply to reorder a matrix as required.

There are two "starting strategies" currently available: minimum degree and pseudo-peripheral. These strategies control how a starting vertex for a connected component is chosen, which is then renumbered as first vertex in the component, starting the algorithm from there. In general, the bandwidths obtained by choosing a pseudo-peripheral vertex are slightly smaller than those obtained from choosing a vertex of minimum degree. On the other hand, this strategy is much more expensive, relatively. The algorithm for finding a pseudo-peripheral vertex as described in "Computer Solution of Sparse Linear Systems" (George, Liu, Ng, Oak Ridge National Laboratory, 1994) is implemented here.

**Template Parameters**

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

## 48.209.2 Member Function Documentation

#### 48.209.2.1 get_inverse_permutation()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const PermutationMatrix> gko::reorder::Rcm< ValueType, IndexType >::get_↩
inverse_permutation ( ) const  [inline]
```

Gets the inverse permutation (permutation matrix, output of the algorithm) of the linear operator.

**Returns**

the inverse permutation (permutation matrix)

```
98      {
99          return inv_permutation_;
100     }
```

#### 48.209.2.2 get_permutation()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const PermutationMatrix> gko::reorder::Rcm< ValueType, IndexType >::get_↩
permutation ( ) const  [inline]
```

Gets the permutation (permutation matrix, output of the algorithm) of the linear operator.

**Returns**

the permutation (permutation matrix)

```
87      {
88          return permutation_;
89      }
```

The documentation for this class was generated from the following file:

- ginkgo/core/reorder/rcm.hpp

## 48.210 gko::experimental::reorder::Rcm< IndexType > Class Template Reference

Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.

```
#include <ginkgo/core/reorder/rcm.hpp>
```

### Public Member Functions

- const parameters_type & get_parameters ()

  *Returns the parameters used to construct the factory.*
- std::unique_ptr< permutation_type > generate (std::shared_ptr< const LinOp > system_matrix) const

## Static Public Member Functions

- static parameters_type build ()

  *Creates a new parameter_type to set up the factory.*

### 48.210.1 Detailed Description

**template<typename IndexType = int32>**
**class gko::experimental::reorder::Rcm< IndexType >**

Rcm (Reverse Cuthill-McKee) is a reordering algorithm minimizing the bandwidth of a matrix.

Such a reordering typically also significantly reduces fill-in, though usually not as effective as more complex algorithms, specifically AMD and nested dissection schemes. The advantage of this algorithm is its low runtime.

The class is a LinOpFactory generating a Permutation matrix out of a Csr system matrix, to be used with `Csr::permute(...)`.

There are two "starting strategies" currently available: minimum degree and pseudo-peripheral. These strategies control how a starting vertex for a connected component is chosen, which is then renumbered as first vertex in the component, starting the algorithm from there. In general, the bandwidths obtained by choosing a pseudo-peripheral vertex are slightly smaller than those obtained from choosing a vertex of minimum degree. On the other hand, this strategy is much more expensive, relatively. The algorithm for finding a pseudo-peripheral vertex as described in "Computer Solution of Sparse Linear Systems" (George, Liu, Ng, Oak Ridge National Laboratory, 1994) is implemented here.

**Template Parameters**

| | |
|---|---|
| *IndexType* | Type of the indices of all matrices used in this class |

### 48.210.2 Member Function Documentation

#### 48.210.2.1 generate()

```
template<typename IndexType = int32>
std::unique_ptr<permutation_type> gko::experimental::reorder::Rcm< IndexType >::generate (
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix to compute the reordering for. |

**Note**

This function overrides the default LinOpFactory::generate to return a Permutation instead of a generic LinOp, which would need to be cast to Permutation again to access its indices. It is only necessary because smart pointers aren't covariant.

#### 48.210.2.2 get_parameters()

```
template<typename IndexType = int32>
const parameters_type& gko::experimental::reorder::Rcm< IndexType >::get_parameters ( )  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
198 { return parameters_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/reorder/rcm.hpp

## 48.211 gko::ReadableFromMatrixData< ValueType, IndexType > Class Template Reference

A LinOp implementing this interface can read its data from a matrix_data structure.

`#include <ginkgo/core/base/lin_op.hpp>`

### Public Member Functions

- virtual void read (const matrix_data< ValueType, IndexType > &data)=0
  *Reads a matrix from a matrix_data structure.*
- void read (const matrix_assembly_data< ValueType, IndexType > &data)
  *Reads a matrix from a matrix_assembly_data structure.*
- virtual void read (const device_matrix_data< ValueType, IndexType > &data)
  *Reads a matrix from a device_matrix_data structure.*
- virtual void read (device_matrix_data< ValueType, IndexType > &&data)
  *Reads a matrix from a device_matrix_data structure.*

### 48.211.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::ReadableFromMatrixData< ValueType, IndexType >**

A LinOp implementing this interface can read its data from a matrix_data structure.

### 48.211.2 Member Function Documentation

#### 48.211.2.1 read() [1/4]

```
template<typename ValueType, typename IndexType>
virtual void gko::ReadableFromMatrixData< ValueType, IndexType >::read (
            const device_matrix_data< ValueType, IndexType > & data )  [inline], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented in gko::matrix::Hybrid< ValueType, IndexType >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::matrix::Coo< ValueType, IndexType >, gko::matrix::Ell< ValueType, IndexType >, gko::matrix::Sellp< ValueType, IndexType >, and gko::matrix::SparsityCsr< ValueType, IndexType >.

```
694     {
695         this->read(data.copy_to_host());
696     }
```

#### 48.211.2.2 read() [2/4]

```
template<typename ValueType, typename IndexType>
void gko::ReadableFromMatrixData< ValueType, IndexType >::read (
            const matrix_assembly_data< ValueType, IndexType > & data )  [inline]
```

Reads a matrix from a matrix_assembly_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_assembly_data structure |

```
684     {
685         this->read(data.get_ordered_data());
686     }
```

#### 48.211.2.3 read() [3/4]

```
template<typename ValueType, typename IndexType>
virtual void gko::ReadableFromMatrixData< ValueType, IndexType >::read (
            const matrix_data< ValueType, IndexType > & data )  [pure virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implemented in gko::matrix::Hybrid< ValueType, IndexType >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::matrix::Coo< ValueType, IndexType >, gko::matrix::Ell< ValueType, IndexType >, gko::matrix::Sellp< ValueType, IndexType >, and gko::matrix::SparsityCsr< ValueType, IndexType >.

#### 48.211.2.4 read() [4/4]

```
template<typename ValueType, typename IndexType>
virtual void gko::ReadableFromMatrixData< ValueType, IndexType >::read (
            device_matrix_data< ValueType, IndexType > && data )  [inline], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented in gko::matrix::Hybrid< ValueType, IndexType >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::matrix::Coo< ValueType, IndexType >, gko::matrix::Ell< ValueType, IndexType >, gko::matrix::Sellp< ValueType, IndexType >, and gko::matrix::SparsityCsr< ValueType, IndexType >.

```
705     {
706         this->read(data.copy_to_host());
707         data.empty_out();
708     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.212 gko::log::Record Class Reference

Record is a Logger which logs every event to an object.

```
#include <ginkgo/core/log/record.hpp>
```

### Classes

- struct logged_data

  *Struct storing the actually logged data.*

### Public Member Functions

- const logged_data & get () const noexcept

  *Returns the logged data.*
- logged_data & get () noexcept

### Static Public Member Functions

- static std::unique_ptr< Record > create (std::shared_ptr< const Executor > exec, const mask_type &enabled_events=Logger::all_events_mask, size_type max_storage=1)

  *Creates a Record logger.*
- static std::unique_ptr< Record > create (const mask_type &enabled_events=Logger::all_events_mask, size_type max_storage=1)

  *Creates a Record logger.*

### 48.212.1 Detailed Description

Record is a Logger which logs every event to an object.

The object can then be accessed at any time by asking the logger to return it.

**Note**

Please note that this logger can have significant memory and performance overhead. In particular, when logging events such as the `check` events, all parameters are cloned. If it is sufficient to clone one parameter, consider implementing a specific logger for this. In addition, it is advised to tune the history size in order to control memory overhead.

### 48.212.2 Member Function Documentation

#### 48.212.2.1 create() [1/2]

```
static std::unique_ptr<Record> gko::log::Record::create (
            const mask_type & enabled_events = Logger::all_events_mask,
            size_type max_storage = 1 )  [inline], [static]
```

Creates a Record logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *enabled_events* | the events enabled for this logger. By default all events. |
| *max_storage* | the size of storage (i.e. history) wanted by the user. By default 0 is used, which means unlimited storage. It is advised to control this to reduce memory overhead of this logger. |

**Returns**

an std::unique_ptr to the the constructed object

```
447     {
448         return std::unique_ptr<Record>(new Record(enabled_events, max_storage));
449     }
```

#### 48.212.2.2 create() [2/2]

```
static std::unique_ptr<Record> gko::log::Record::create (
            std::shared_ptr< const Executor > exec,
            const mask_type & enabled_events = Logger::all_events_mask,
            size_type max_storage = 1 )  [inline], [static]
```

Creates a Record logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *enabled_events* | the events enabled for this logger. By default all events. |
| *max_storage* | the size of storage (i.e. history) wanted by the user. By default 0 is used, which means unlimited storage. It is advised to control this to reduce memory overhead of this logger. |

**Returns**

an std::unique_ptr to the the constructed object

```
426     {
427         return std::unique_ptr<Record>(new Record(enabled_events, max_storage));
428     }
```

#### 48.212.2.3 get() [1/2]

```
const logged_data& gko::log::Record::get ( ) const  [inline], [noexcept]
```

Returns the logged data.

**Returns**

the logged data

```
456 { return data_; }
```

#### 48.212.2.4 get() [2/2]

```
logged_data& gko::log::Record::get ( )  [inline], [noexcept]
461 { return data_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/log/record.hpp

## 48.213 gko::ReferenceExecutor Class Reference

This is a specialization of the OmpExecutor, which runs the reference implementations of the kernels used for debugging purposes.

```
#include <ginkgo/core/base/executor.hpp>
```

## Public Member Functions

- std::string get_description () const override
- void run (const Operation &op) const override

  *Runs the specified Operation using this Executor.*
- virtual void run (const Operation &op) const=0

  *Runs the specified Operation using this Executor.*
- template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename ClosureDpcpp >
  void run (const ClosureOmp &op_omp, const ClosureCuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*
- template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩ Dpcpp >
  void run (std::string name, const ClosureReference &op_ref, const ClosureOmp &op_omp, const Closure↩ Cuda &op_cuda, const ClosureHip &op_hip, const ClosureDpcpp &op_dpcpp) const

  *Runs one of the passed in functors, depending on the Executor type.*

## Additional Inherited Members

### 48.213.1 Detailed Description

This is a specialization of the OmpExecutor, which runs the reference implementations of the kernels used for debugging purposes.

### 48.213.2 Member Function Documentation

#### 48.213.2.1 get_description()

```
std::string gko::ReferenceExecutor::get_description ( ) const  [inline], [override], [virtual]
```

**Returns**

a textual representation of the executor and its device.

Reimplemented from gko::OmpExecutor.

```
1469 { return "ReferenceExecutor"; }
```

#### 48.213.2.2 run() [1/4]

```
template<typename ClosureOmp , typename ClosureCuda , typename ClosureHip , typename Closure↩
Dpcpp >
void gko::Executor::run (
            typename ClosureOmp ,
            typename ClosureCuda ,
            typename ClosureHip ,
            typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *op_omp* | functor to run in case of a OmpExecutor or ReferenceExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
660     {
661         LambdaOperation<ClosureOmp, ClosureOmp, ClosureCuda, ClosureHip,
662                         ClosureDpcpp>
663             op(op_omp, op_cuda, op_hip, op_dpcpp);
664         this->run(op);
665     }
```

### 48.213.2.3 run() [2/4]

```
void gko::ReferenceExecutor::run (
            const Operation & op ) const  [inline], [override], [virtual]
```

Runs the specified Operation using this Executor.

**Parameters**

| | |
|---|---|
| *op* | the operation to run |

Implements gko::Executor.

```
1472     {
1473         this->template log<log::Logger::operation_launched>(this, &op);
1474         op.run(std::static_pointer_cast<const ReferenceExecutor>(
1475             this->shared_from_this()));
1476         this->template log<log::Logger::operation_completed>(this, &op);
1477     }
```

### 48.213.2.4 run() [3/4]

```
virtual void gko::Executor::run
```

Runs the specified Operation using this Executor.

**Parameters**

| | |
|---|---|
| *op* | the operation to run |

#### 48.213.2.5 run() [4/4]

```
template<typename ClosureReference , typename ClosureOmp , typename ClosureCuda , typename
ClosureHip , typename ClosureDpcpp >
void gko::Executor::run (
             typename ClosureReference ,
             typename ClosureOmp ,
             typename ClosureCuda ,
             typename ClosureHip ,
             typename ClosureDpcpp  )  [inline]
```

Runs one of the passed in functors, depending on the Executor type.

**Template Parameters**

| | |
|---|---|
| *ClosureReference* | type of op_ref |
| *ClosureOmp* | type of op_omp |
| *ClosureCuda* | type of op_cuda |
| *ClosureHip* | type of op_hip |
| *ClosureDpcpp* | type of op_dpcpp |

**Parameters**

| | |
|---|---|
| *name* | the name of the operation |
| *op_ref* | functor to run in case of a ReferenceExecutor |
| *op_omp* | functor to run in case of a OmpExecutor |
| *op_cuda* | functor to run in case of a CudaExecutor |
| *op_hip* | functor to run in case of a HipExecutor |
| *op_dpcpp* | functor to run in case of a DpcppExecutor |

```
688     {
689         LambdaOperation<ClosureReference, ClosureOmp, ClosureCuda, ClosureHip,
690                         ClosureDpcpp>
691             op(std::move(name), op_ref, op_omp, op_cuda, op_hip, op_dpcpp);
692         this->run(op);
693     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/executor.hpp

## 48.214 gko::config::registry Class Reference

This class stores additional context for creating Ginkgo objects from configuration files.

```
#include <ginkgo/core/config/registry.hpp>
```

## Public Member Functions

- registry (const configuration_map &additional_map={})

  *registry constructor*
- registry (const std::unordered_map< std::string, detail::allowed_ptr > &stored_map, const configuration_↩ map &additional_map={})

  *registry constructor*
- template<typename T >
  bool emplace (std::string key, std::shared_ptr< T > data)

  *Store the data with the key.*

### 48.214.1 Detailed Description

This class stores additional context for creating Ginkgo objects from configuration files.

The context can contain user-provided objects that derive from the following base types:

- LinOp
- LinOpFactory
- CriterionFactory

Additionally, users can provide mappings from a configuration (provided as a pnode) to user-defined types that are derived from LinOpFactory

### 48.214.2 Constructor & Destructor Documentation

#### 48.214.2.1 registry() [1/2]

```
gko::config::registry::registry (
            const configuration_map & additional_map = {} )
```

registry constructor

**Parameters**

| | |
|---|---|
| *additional_map* | the additional map to dispatch the class base. Users can extend the map to fit their own LinOpFactory. We suggest using "usr::" as the prefix in the key to simply avoid conflict with ginkgo's map. |

#### 48.214.2.2 registry() [2/2]

```
gko::config::registry::registry (
```

```
            const std::unordered_map< std::string, detail::allowed_ptr > & stored_map,
            const configuration_map & additional_map = {} )
```

registry constructor

**Parameters**

| | |
|---|---|
| *stored_map* | the map stores the shared pointer of users' objects. It can hold any type whose base type is LinOp/LinOpFactory/CriterionFactory. |
| *additional_map* | the additional map to dispatch the class base. Users can extend the map to fit their own LinOpFactory. We suggest using "usr::" as the prefix in the key to simply avoid conflict with ginkgo's map. |

### 48.214.3 Member Function Documentation

#### 48.214.3.1 emplace()

```
template<typename T >
bool gko::config::registry::emplace (
            std::string key,
            std::shared_ptr< T > data )  [inline]
```

Store the data with the key.

**Template Parameters**

| | |
|---|---|
| *T* | the type |

**Parameters**

| | |
|---|---|
| *key* | the unique key string |
| *data* | the shared pointer of the object |

```
236 {
237     auto it = stored_map_.emplace(key, data);
238     return it.second;
239 }
```

The documentation for this class was generated from the following file:

- ginkgo/core/config/registry.hpp

## 48.215 gko::stop::RelativeResidualNorm< ValueType > Class Template Reference

The RelativeResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the right-hand side, i.e.

```
#include <ginkgo/core/stop/residual_norm.hpp>
```

### Additional Inherited Members

### 48.215.1 Detailed Description

template<typename ValueType = default_precision>
class gko::stop::RelativeResidualNorm< ValueType >

The RelativeResidualNorm class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the right-hand side, i.e.

when norm(residual) / norm(right_hand_side) < threshold. For better performance, the checks are run thanks to kernels on the executor where the algorithm is executed.

**Note**

To use this stopping criterion there are some dependencies. The constructor depends on `b` in order to compute the norm of the right-hand side. If this is not correctly provided, an exception ::gko::NotSupported() is thrown.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/residual_norm.hpp

## 48.216 gko::reorder::ReorderingBase< IndexType > Class Template Reference

The ReorderingBase class is a base class for all the reordering algorithms.

`#include <ginkgo/core/reorder/reordering_base.hpp>`

### Additional Inherited Members

### 48.216.1 Detailed Description

template<typename IndexType = int32>
class gko::reorder::ReorderingBase< IndexType >

The ReorderingBase class is a base class for all the reordering algorithms.

It contains a factory to instantiate the reorderings. It is up to each specific reordering to decide what to do with the data that is passed to it.

The documentation for this class was generated from the following file:

- ginkgo/core/reorder/reordering_base.hpp

## 48.217 gko::reorder::ReorderingBaseArgs Struct Reference

This struct is used to pass parameters to the EnableDefaultReorderingBaseFactory::generate() method.

`#include <ginkgo/core/reorder/reordering_base.hpp>`

### 48.217.1 Detailed Description

This struct is used to pass parameters to the EnableDefaultReorderingBaseFactory::generate() method.

It is the ComponentsType of ReorderingBaseFactory.

The documentation for this struct was generated from the following file:

- ginkgo/core/reorder/reordering_base.hpp

## 48.218 gko::experimental::mpi::request Class Reference

The request class is a light, move-only wrapper around the MPI_Request handle.

`#include <ginkgo/core/base/mpi.hpp>`

### Public Member Functions

- request ()
  *The default constructor.*
- MPI_Request * get ()
  *Get a pointer to the underlying MPI_Request handle.*
- status wait ()
  *Allows a rank to wait on a particular request handle.*

### 48.218.1 Detailed Description

The request class is a light, move-only wrapper around the MPI_Request handle.

### 48.218.2 Constructor & Destructor Documentation

#### 48.218.2.1 request()

`gko::experimental::mpi::request::request ( )  [inline]`

The default constructor.

It creates a null MPI_Request of MPI_REQUEST_NULL type.
`336 :  req_(MPI_REQUEST_NULL) {}`

### 48.218.3 Member Function Documentation

#### 48.218.3.1 get()

```
MPI_Request* gko::experimental::mpi::request::get ( )  [inline]
```

Get a pointer to the underlying MPI_Request handle.

**Returns**

a pointer to MPI_Request handle

```
367 { return &this->req_; }
```

Referenced by gko::experimental::mpi::communicator::i_all_gather(), gko::experimental::mpi::communicator::i_all_reduce(), gko::experimental::mpi::communicator::i_all_to_all(), gko::experimental::mpi::communicator::i_all_to_all_v(), gko::experimental::mpi::communicator::i_broadcast(), gko::experimental::mpi::communicator::i_gather(), gko::experimental::mpi::communicator::i_gather_v(), gko::experimental::mpi::communicator::i_recv(), gko::experimental::mpi::communicator::i_reduce(), gko::experimental::mpi::communicator::i_scan(), gko::experimental::mpi::communicator::i_scatter(), gko::experimental::mpi::communicator::i_scatter_v(), gko::experimental::mpi::communicator::i_send(), gko::experimental::mpi::window< ValueType >::r_accumulate(), gko::experimental::mpi::window< ValueType >::r_get(), gko::experimental::mpi::window< ValueType >::r_get_accumulate(), and gko::experimental::mpi::window< ValueType >::r_put().

#### 48.218.3.2 wait()

```
status gko::experimental::mpi::request::wait ( )  [inline]
```

Allows a rank to wait on a particular request handle.

**Parameters**

| | |
|---|---|
| *req* | The request to wait on. |
| *status* | The status variable that can be queried. |

```
376     {
377         status status;
378         GKO_ASSERT_NO_MPI_ERRORS(MPI_Wait(&req_, status.get()));
379         return status;
380     }
```

References gko::experimental::mpi::status::get().

The documentation for this class was generated from the following file:

- ginkgo/core/base/mpi.hpp

## 48.219 gko::stop::ResidualNorm< ValueType > Class Template Reference

Stopping criterion based on the explicit residual norm $\|r_k\| = \|b - Ax_k\|$.

`#include <ginkgo/core/stop/residual_norm.hpp>`

### Additional Inherited Members

### 48.219.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::stop::ResidualNorm< ValueType >**

Stopping criterion based on the explicit residual norm $\|r_k\| = \|b - Ax_k\|$.

The iteration halts once $\|r_k\| \leq \tau \cdot \beta$, where $\tau$ is the factory parameter `reduction_factor` and the baseline $\beta$ is selected by the `baseline` factory parameter:

- `mode::rhs_norm` (default) — relative to the right-hand side: $\beta = \|b\|$, so the condition is $\|r_k\| \leq \tau\|b\|$.
- `mode::initial_resnorm` — relative to the initial residual: $\beta = \|r_0\|$, so the condition is $\|r_k\| \leq \tau\|r_0\|$.
- `mode::absolute` — absolute threshold: $\beta = 1$, so the condition is $\|r_k\| \leq \tau$.

Per-iteration, the criterion prefers a pre-computed residual norm passed by the solver (via `Updater::residual_norm`); otherwise it falls back to computing the 2-norm of the residual vector itself (via `Updater::residual`), which costs one global reduction. When even cheaper checks are needed and the solver maintains an internal squared-norm estimate, use ImplicitResidualNorm instead.

**Note**

Baseline prerequisites at construction time:

- `mode::rhs_norm` requires the right-hand side $b$.
- `mode::initial_resnorm` requires either the initial residual $r_0$ explicitly, or the triple $(A, b, x_0)$ from which it is computed as $r_0 = b - Ax_0$.
- `mode::absolute` requires $b$ as well — to determine the number of right-hand sides; its baseline is then filled with ones. If the required arguments are missing, ::gko::NotSupported() is thrown.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/residual_norm.hpp

## 48.220 gko::stop::ResidualNormBase< ValueType > Class Template Reference

The ResidualNormBase class provides a framework for stopping criteria related to the residual norm.

`#include <ginkgo/core/stop/residual_norm.hpp>`

### Additional Inherited Members

### 48.220.1 Detailed Description

template<typename ValueType>
class gko::stop::ResidualNormBase< ValueType >

The ResidualNormBase class provides a framework for stopping criteria related to the residual norm.

These criteria differ in the way they initialize starting_tau_, so in the value they compare the residual norm against. The provided check_impl uses the actual residual to check for convergence.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/residual_norm.hpp

## 48.221 gko::stop::ResidualNormReduction< ValueType > Class Template Reference

The ResidualNormReduction class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the initial residual, i.e.

`#include <ginkgo/core/stop/residual_norm.hpp>`

### Additional Inherited Members

### 48.221.1 Detailed Description

template<typename ValueType = default_precision>
class gko::stop::ResidualNormReduction< ValueType >

The ResidualNormReduction class is a stopping criterion which stops the iteration process when the residual norm is below a certain threshold relative to the norm of the initial residual, i.e.

when norm(residual) / norm(initial_residual) < threshold. For better performance, the checks are run thanks to kernels on the executor where the algorithm is executed.

**Note**

To use this stopping criterion there are some dependencies. The constructor depends on `initial_residual` in order to compute the first relative residual norm. The check method depends on either the `residual_norm` or the `residual` being set. When any of those is not correctly provided, an exception ::gko::NotSupported() is thrown.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/residual_norm.hpp

## 48.222 gko::multigrid::Pgm< ValueType, IndexType >::reuse_data_type Class Reference

Records the aggregates, the transfer operators and the coarse-to-fine mappings of the first generate_reuse() call.

`#include <ginkgo/core/multigrid/pgm.hpp>`

### 48.222.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::multigrid::Pgm< ValueType, IndexType >::reuse_data_type**

Records the aggregates, the transfer operators and the coarse-to-fine mappings of the first generate_reuse() call.

The documentation for this class was generated from the following file:

- ginkgo/core/multigrid/pgm.hpp

## 48.223 gko::solver::Multigrid::reuse_data_type Class Reference

Records the hierarchy of generate_reuse() and the reuse data of its levels, smoothers and coarsest solver.

`#include <ginkgo/core/solver/multigrid.hpp>`

### 48.223.1 Detailed Description

Records the hierarchy of generate_reuse() and the reuse data of its levels, smoothers and coarsest solver.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/multigrid.hpp

## 48.224 gko::LinOpFactory::ReuseData Class Reference

Opaque base for the data a factory records in generate_reuse() to speed up later generations.

`#include <ginkgo/core/base/lin_op.hpp>`

### 48.224.1 Detailed Description

Opaque base for the data a factory records in generate_reuse() to speed up later generations.

Factories supporting reuse derive from it.

**Note**

This interface is experimental.

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.225 gko::accessor::row_major< ValueType, Dimensionality > Class Template Reference

A row_major accessor is a bridge between a range and the row-major memory layout.

`#include <ginkgo/core/base/range_accessors.hpp>`

### Public Types

- using value_type = ValueType
  *Type of values returned by the accessor.*
- using data_type = value_type *
  *Type of underlying data storage.*

### Public Member Functions

- constexpr value_type & operator() (size_type row, size_type col) const
  *Returns the data element at position (row, col)*
- constexpr range< row_major > operator() (const span &rows, const span &cols) const
  *Returns the sub-range spanning the range (rows, cols)*
- constexpr size_type length (size_type dimension) const
  *Returns the length in dimension `dimension`.*
- template<typename OtherAccessor >
  void copy_from (const OtherAccessor &other) const
  *Copies data from another accessor.*

### Public Attributes

- const data_type data
  *Reference to the underlying data.*
- const std::array< const size_type, dimensionality > lengths
  *An array of dimension sizes.*
- const size_type stride
  *Distance between consecutive rows.*

## Static Public Attributes

- static constexpr size_type dimensionality = 2

  *Number of dimensions of the accessor.*

## 48.225.1 Detailed Description

**template<typename ValueType, size_type Dimensionality>**
**class gko::accessor::row_major< ValueType, Dimensionality >**

A row_major accessor is a bridge between a range and the row-major memory layout.

You should never try to explicitly create an instance of this accessor. Instead, supply it as a template parameter to a range, and pass the constructor parameters for this class to the range (it will forward it to this class).

**Warning**

The current implementation is incomplete, and only allows for 2-dimensional ranges.

**Template Parameters**

| | |
|---|---|
| *ValueType* | type of values this accessor returns |
| *Dimensionality* | number of dimensions of this accessor (has to be 2) |

## 48.225.2 Member Function Documentation

### 48.225.2.1 copy_from()

```
template<typename ValueType , size_type Dimensionality>
template<typename OtherAccessor >
void gko::accessor::row_major< ValueType, Dimensionality >::copy_from (
            const OtherAccessor & other ) const  [inline]
```

Copies data from another accessor.

**Warning**

Do not use this function since it is not optimized for a specific executor. It will always be performed sequentially. Please write an optimized version (adjusted to the architecture) by iterating through the values yourself.

**Template Parameters**

| | |
|---|---|
| *OtherAccessor* | type of the other accessor |

**Parameters**

| | |
|---|---|
| *other* | other accessor |

```
140     {
141         for (size_type i = 0; i < lengths[0]; ++i) {
142             for (size_type j = 0; j < lengths[1]; ++j) {
143                 (*this)(i, j) = other(i, j);
144             }
145         }
146     }
```

References gko::accessor::row_major< ValueType, Dimensionality >::lengths.

### 48.225.2.2 length()

```
template<typename ValueType , size_type Dimensionality>
constexpr size_type gko::accessor::row_major< ValueType, Dimensionality >::length (
            size_type dimension ) const  [inline], [constexpr]
```

Returns the length in dimension `dimension`.

**Parameters**

| | |
|---|---|
| *dimension* | a dimension index |

**Returns**

length in dimension `dimension`

```
122     {
123         return dimension < 2 ?  lengths[dimension] :  1;
124     }
```

References gko::accessor::row_major< ValueType, Dimensionality >::lengths.

### 48.225.2.3 operator()() [1/2]

```
template<typename ValueType , size_type Dimensionality>
constexpr range<row_major> gko::accessor::row_major< ValueType, Dimensionality >::operator()
(
            const span & rows,
            const span & cols ) const  [inline], [constexpr]
```

Returns the sub-range spanning the range (rows, cols)

**Parameters**

| | |
|---|---|
| *rows* | row span |
| *cols* | column span |

**Returns**

sub-range spanning the range (rows, cols)

```
105     {
106         return GKO_ASSERT(rows.is_valid()), GKO_ASSERT(cols.is_valid()),
107                GKO_ASSERT(rows <= span{lengths[0]}),
108                GKO_ASSERT(cols <= span{lengths[1]}),
109                range<row_major>(data + rows.begin * stride + cols.begin,
110                                 rows.end - rows.begin, cols.end - cols.begin,
111                                 stride);
112     }
```

References gko::span::begin, gko::accessor::row_major< ValueType, Dimensionality >::data, gko::span::end, gko::span::is_valid(), gko::accessor::row_major< ValueType, Dimensionality >::lengths, and gko::accessor::row↩_major< ValueType, Dimensionality >::stride.

#### 48.225.2.4 operator()() [2/2]

```
template<typename ValueType , size_type Dimensionality>
constexpr value_type& gko::accessor::row_major< ValueType, Dimensionality >::operator() (
             size_type row,
             size_type col ) const  [inline], [constexpr]
```

Returns the data element at position (row, col)

**Parameters**

| | |
|---|---|
| *row* | row index |
| *col* | column index |

**Returns**

data element at (row, col)

```
90      {
91          return GKO_ASSERT(row < lengths[0]), GKO_ASSERT(col < lengths[1]),
92                 data[row * stride + col];
93      }
```

References gko::accessor::row_major< ValueType, Dimensionality >::data, gko::accessor::row_major< Value↩Type, Dimensionality >::lengths, and gko::accessor::row_major< ValueType, Dimensionality >::stride.

The documentation for this class was generated from the following file:

- ginkgo/core/base/range_accessors.hpp

## 48.226 gko::matrix::RowGatherer< IndexType > Class Template Reference

RowGatherer is a matrix "format" which stores the gather indices arrays which can be used to gather rows to another matrix.

```
#include <ginkgo/core/matrix/row_gatherer.hpp>
```

## Public Member Functions

- index_type * get_row_idxs () noexcept

  *Returns a pointer to the row index array for gathering.*
- const index_type * get_const_row_idxs () const noexcept

  *Returns a pointer to the row index array for gathering.*

## Static Public Member Functions

- static std::unique_ptr< RowGatherer > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size={})

  *Creates uninitialized RowGatherer arrays of the specified size.*
- static std::unique_ptr< RowGatherer > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< index_type > row_idxs)

  *Creates a RowGatherer matrix from an already allocated (and initialized) row gathering array.*
- static std::unique_ptr< const RowGatherer > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< IndexType > &&row_idxs)

  *Creates a constant (immutable) RowGatherer matrix from a constant array.*

### 48.226.1 Detailed Description

**template<typename IndexType = int32>**
**class gko::matrix::RowGatherer< IndexType >**

RowGatherer is a matrix "format" which stores the gather indices arrays which can be used to gather rows to another matrix.

**Template Parameters**

| | |
|---|---|
| *IndexType* | precision of rowgatherer array indices. |

**Note**

This format is used mainly to allow for an abstraction of the rowgatherer and provides the user with an apply method which calls the respective Dense rowgatherer operation. As such it only stores an array of the rowgatherer indices.

### 48.226.2 Member Function Documentation

#### 48.226.2.1 create() [1/2]

```
template<typename IndexType = int32>
static std::unique_ptr<RowGatherer> gko::matrix::RowGatherer< IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< index_type > row_idxs )  [static]
```

Creates a RowGatherer matrix from an already allocated (and initialized) row gathering array.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the rowgatherer array. |
| *row_idxs* | array of rowgatherer array |

**Note**

If `row_idxs` is not an rvalue, not an array of IndexType, or is on the wrong executor, an internal copy will be created, and the original array data will not be used in the matrix.

**Returns**

A smart pointer to the newly created matrix.

### 48.226.2.2 create() [2/2]

```
template<typename IndexType = int32>
static std::unique_ptr<RowGatherer> gko::matrix::RowGatherer< IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = {} )  [static]
```

Creates uninitialized RowGatherer arrays of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the RowGatherable matrix |

**Returns**

A smart pointer to the newly created matrix.

### 48.226.2.3 create_const()

```
template<typename IndexType = int32>
static std::unique_ptr<const RowGatherer> gko::matrix::RowGatherer< IndexType >::create_const
(
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            gko::detail::const_array_view< IndexType > && row_idxs )  [static]
```

Creates a constant (immutable) RowGatherer matrix from a constant array.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *row_idxs* | the gathered row indices of the matrix |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of the arrays on the correct executor.

### 48.226.2.4 get_const_row_idxs()

```
template<typename IndexType = int32>
const index_type* gko::matrix::RowGatherer< IndexType >::get_const_row_idxs ( ) const  [inline],
[noexcept]
```

Returns a pointer to the row index array for gathering.

**Returns**

the pointer to the row index array for gathering.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
63     {
64         return row_idxs_.get_const_data();
65     }
```

References gko::array< ValueType >::get_const_data().

### 48.226.2.5 get_row_idxs()

```
template<typename IndexType = int32>
index_type* gko::matrix::RowGatherer< IndexType >::get_row_idxs ( )  [inline], [noexcept]
```

Returns a pointer to the row index array for gathering.

**Returns**

the pointer to the row index array for gathering.

```
53 { return row_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/row_gatherer.hpp

# 48.227 gko::experimental::distributed::RowGatherer< LocalIndexType > Class Template Reference

The distributed::RowGatherer gathers the rows of distributed::Vector that are located on other processes.

`#include <ginkgo/core/distributed/row_gatherer.hpp>`

## Public Member Functions

- mpi::request apply_async (ptr_param< const LinOp > b, ptr_param< LinOp > x) const

  *Asynchronous version of LinOp::apply.*
- mpi::request apply_async (ptr_param< const LinOp > b, ptr_param< LinOp > x, gko::detail::GenericDenseCache &workspace) const

  *Asynchronous version of LinOp::apply.*
- dim< 2 > get_size () const

  *Returns the size of the row gatherer.*
- std::shared_ptr< const mpi::CollectiveCommunicator > get_collective_communicator () const

  *Get the used collective communicator.*
- const LocalIndexType * get_const_send_idxs () const

  *Read access to the (local) rows indices.*
- size_type get_num_send_idxs () const

  *Returns the number of (local) row indices.*

## Static Public Member Functions

- template<typename GlobalIndexType = int64, typename = std::enable_if_t<sizeof(GlobalIndexType) >= sizeof(LocalIndexType)> static std::unique_ptr< RowGatherer > create (std::shared_ptr< const Executor > exec, std::shared_ptr< const mpi::CollectiveCommunicator > coll_comm, const index_map< LocalIndexType, GlobalIndexType > &imap)

  *Creates a distributed::RowGatherer from a given collective communicator and index map.*

## 48.227.1 Detailed Description

**template<typename LocalIndexType = int32>**
**class gko::experimental::distributed::RowGatherer< LocalIndexType >**

The distributed::RowGatherer gathers the rows of distributed::Vector that are located on other processes.

Example usage:

```
 {c++}
auto coll_comm = std::make_shared<mpi::neighborhood_communicator>(comm,
                                                                  imap);
auto rg = distributed::RowGatherer<int32>::create(exec, coll_comm, imap);
auto b = distributed::Vector<double>::create(...);
auto x = matrix::Dense<double>::create(...);
auto req = rg->apply_async(b, x);
// users can do some computation that doesn't modify b, or access x
req.wait();
// x now contains the gathered rows of b
```

**Note**

The output vector for the apply_async functions *must* use an executor that is compatible with the MPI implementation. In particular, if the MPI implementation is not GPU aware, then the output vector *must* use a CPU executor. Otherwise, an exception will be thrown.

**Template Parameters**

| | |
|---|---|
| *LocalIndexType* | the index type for the stored indices |

## 48.227.2 Member Function Documentation

### 48.227.2.1 apply_async() [1/2]

```
template<typename LocalIndexType = int32>
mpi::request gko::experimental::distributed::RowGatherer< LocalIndexType >::apply_async (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x ) const
```

Asynchronous version of LinOp::apply.

**Warning**

Only one mpi::request can be active at any given time. Calling this function again without waiting on the previous mpi::request will lead to undefined behavior.

**Parameters**

| | |
|---|---|
| *b* | the input distributed::Vector. |
| *x* | the output matrix::Dense with the rows gathered from b. Its executor has to be compatible with the MPI implementation, see the class documentation. |

**Returns**

a mpi::request for this task. The task is guaranteed to be completed only after `.wait()` has been called on it.

### 48.227.2.2 apply_async() [2/2]

```
template<typename LocalIndexType = int32>
mpi::request gko::experimental::distributed::RowGatherer< LocalIndexType >::apply_async (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > x,
            gko::detail::GenericDenseCache & workspace ) const
```

Asynchronous version of LinOp::apply.

**Warning**

Calling this multiple times with the same workspace and without waiting on each previous request will lead to incorrect data transfers.

**Parameters**

| | |
|---|---|
| *b* | the input distributed::Vector. |
| *x* | the output matrix::Dense with the rows gathered from b. Its executor has to be compatible with the MPI implementation, see the class documentation. |
| *workspace* | a workspace to store temporary data for the operation. This might not be modified before the request is waited on. |

**Returns**

a mpi::request for this task. The task is guaranteed to be completed only after `.wait()` has been called on it.

### 48.227.2.3 create()

```
template<typename LocalIndexType = int32>
template<typename GlobalIndexType = int64, typename = std::enable_if_t<sizeof(GlobalIndexType)
>= sizeof(LocalIndexType)>>
static std::unique_ptr<RowGatherer> gko::experimental::distributed::RowGatherer< LocalIndex↩
Type >::create (
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< const mpi::CollectiveCommunicator > coll_comm,
            const index_map< LocalIndexType, GlobalIndexType > & imap )  [inline], [static]
```

Creates a distributed::RowGatherer from a given collective communicator and index map.

@TODO: using a segmented array instead of the imap would probably be more general

**Template Parameters**

| | |
|---|---|
| *GlobalIndexType* | the global index type of the index map |

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *coll_comm* | the collective communicator |
| *imap* | the index map defining which rows to gather |

**Note**

The coll_comm and imap have to be compatible. The coll_comm must send and recv exactly as many rows as the imap defines.

This is a collective operation, all participating processes have to execute this operation.

**Returns**

a shared_ptr to the created distributed::RowGatherer

```
201     {
202         return std::unique_ptr<RowGatherer>(
203             new RowGatherer(std::move(exec), std::move(coll_comm), imap));
204     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/row_gatherer.hpp

# 48.228 gko::multigrid::Rs< ValueType, IndexType > Class Template Reference

Rs implements the Ruge–Stueben (classical) Algebraic Multigrid (AMG) coarsening strategy for M-matrices.

`#include <ginkgo/core/multigrid/rs.hpp>`

## Public Member Functions

- std::shared_ptr< const LinOp > get_system_matrix () const

  *Returns the system operator (matrix) of the linear system.*

## 48.228.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::multigrid::Rs< ValueType, IndexType >**

Rs implements the Ruge–Stueben (classical) Algebraic Multigrid (AMG) coarsening strategy for M-matrices.

Given a sparse system $Ax = b$, it produces one level of an AMG hierarchy: a C/F splitting, a prolongation operator $P$, and a coarse-grid operator $A_c = RAP$.

Coarsening proceeds in three steps. First, neighbour $j$ is marked as *strongly influencing* row $i$ when $-a_{ij} \geq \theta \cdot \max_{k \neq i}(-a_{ik})$. Second, a greedy pass selects C-points by repeatedly picking the undecided node with the most strong neighbours, marking its undecided strong neighbours as F-points, and updating neighbour counts accordingly. Third, we create the coarse grid and compute the interpolation via the classical RS direct interpolation formula, accounting for both strong C- and F-neighbours.

Ruge, J. W., & Stueben, K. (1987). Algebraic multigrid, Multigrid Methods (Vol. 3, pp. 73–130). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9781611971057.ch4

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

## 48.228.2 Member Function Documentation

#### 48.228.2.1 get_system_matrix()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::multigrid::Rs< ValueType, IndexType >::get_system_matrix (
) const  [inline]
```

Returns the system operator (matrix) of the linear system.

**Returns**

the system operator (matrix)

```
65     {
66         return system_matrix_;
67     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/multigrid/rs.hpp

## 48.229 gko::matrix::Csr< ValueType, IndexType >::scale_add_reuse_info Class Reference

Class describing the internal lookup structures created by scale_add_reuse to recompute a sparse matrix-matrix sum with updated values.

```
#include <ginkgo/core/matrix/csr.hpp>
```

### Public Member Functions

- void update_values (ptr_param< const Dense< value_type >> scale1, ptr_param< const Csr > mtx1, ptr_param< const Dense< value_type >> scale2, ptr_param< const Csr > mtx2, ptr_param< Csr > out) const

  *Recomputes the sparse matrix-matrix sum* `out = scale1 * mtx1 + scale2 * mtx2` *when only the values of mtx1, scale1, mtx2, scale2 changed, but the sparsity patterns of mtx1, mtx2 and out are unchanged.*

### 48.229.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Csr< ValueType, IndexType >::scale_add_reuse_info**

Class describing the internal lookup structures created by scale_add_reuse to recompute a sparse matrix-matrix sum with updated values.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/csr.hpp

# 48.230 gko::ScaledIdentityAddable Class Reference

Adds the operation M <- a I + b M for matrix M, identity operator I and scalars a and b, where M is the calling object.

`#include <ginkgo/core/base/lin_op.hpp>`

## Public Member Functions

- void add_scaled_identity (ptr_param< const LinOp > const a, ptr_param< const LinOp > const b)

  *Scales this and adds another scalar times the identity to it.*

## 48.230.1 Detailed Description

Adds the operation M <- a I + b M for matrix M, identity operator I and scalars a and b, where M is the calling object.

## 48.230.2 Member Function Documentation

### 48.230.2.1 add_scaled_identity()

```
void gko::ScaledIdentityAddable::add_scaled_identity (
            ptr_param< const LinOp > const a,
            ptr_param< const LinOp > const b )  [inline]
```

Scales this and adds another scalar times the identity to it.

**Parameters**

| | |
|---|---|
| *a* | Scalar to multiply the identity operator before adding. |
| *b* | Scalar to multiply this before adding the scaled identity to it. |

```
908     {
909         GKO_ASSERT_IS_SCALAR(a);
910         GKO_ASSERT_IS_SCALAR(b);
911         auto ae = make_temporary_clone(as<LinOp>(this)->get_executor(), a);
912         auto be = make_temporary_clone(as<LinOp>(this)->get_executor(), b);
913         add_scaled_identity_impl(ae.get(), be.get());
914     }
```

References gko::make_temporary_clone().

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

# 48.231 gko::matrix::ScaledPermutation< ValueType, IndexType > Class Template Reference

ScaledPermutation is a matrix combining a permutation with scaling factors.

`#include <ginkgo/core/matrix/scaled_permutation.hpp>`

## Public Member Functions

- value_type * get_scaling_factors () noexcept

  *Returns a pointer to the scaling factors.*
- const value_type * get_const_scaling_factors () const noexcept

  *Returns a pointer to the scaling factors.*
- index_type * get_permutation () noexcept

  *Returns a pointer to the permutation indices.*
- const index_type * get_const_permutation () const noexcept

  *Returns a pointer to the permutation indices.*
- std::unique_ptr< ScaledPermutation > compute_inverse () const

  *Returns the inverse of this operator as a scaled permutation.*
- std::unique_ptr< ScaledPermutation > compose (ptr_param< const ScaledPermutation > other) const

  *Composes this scaled permutation with another scaled permutation.*
- void write (gko::matrix_data< value_type, index_type > &data) const override

  *Writes a matrix to a matrix_data structure.*

## Static Public Member Functions

- static std::unique_ptr< ScaledPermutation > create (std::shared_ptr< const Executor > exec, size_type size=0)

  *Creates an uninitialized ScaledPermutation matrix.*
- static std::unique_ptr< ScaledPermutation > create (ptr_param< const Permutation< IndexType >> permutation)

  *Create a ScaledPermutation from a Permutation.*
- static std::unique_ptr< ScaledPermutation > create (std::shared_ptr< const Executor > exec, array< value_type > scaling_factors, array< index_type > permutation_indices)

  *Creates a ScaledPermutation matrix from already allocated arrays.*
- static std::unique_ptr< const ScaledPermutation > create_const (std::shared_ptr< const Executor > exec, gko::detail::const_array_view< value_type > &&scale, gko::detail::const_array_view< index_type > &&perm_idxs)

  *Creates a constant (immutable) ScaledPermutation matrix from constant arrays.*

### 48.231.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::ScaledPermutation< ValueType, IndexType >**

ScaledPermutation is a matrix combining a permutation with scaling factors.

It is a combination of Diagonal and Permutation, and can be read as $SP = P \cdot S$, i.e. the scaling gets applied before the permutation.

**Template Parameters**

| | |
|---|---|
| *IndexType* | index type of permutation indices |
| *ValueType* | value type of the scaling factors |

## 48.231.2 Member Function Documentation

### 48.231.2.1 compose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<ScaledPermutation> gko::matrix::ScaledPermutation< ValueType, IndexType >↩
::compose (
            ptr_param< const ScaledPermutation< ValueType, IndexType > > other ) const
```

Composes this scaled permutation with another scaled permutation.

This means `result = other * this` from the matrix perspective, which is equivalent to first scaling and permuting by `this` and then by `other`.

**Parameters**

| | |
|---|---|
| *other* | the other permutation |

**Returns**

the combined permutation

### 48.231.2.2 compute_inverse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<ScaledPermutation> gko::matrix::ScaledPermutation< ValueType, IndexType >↩
::compute_inverse ( ) const
```

Returns the inverse of this operator as a scaled permutation.

It is computed via $(PS)^-1 = P^{-1}(PSP^{-1})$.

**Returns**

a newly created ScaledPermutation object storing the inverse of the permutation and scaling factors of this ScaledPermutation.

### 48.231.2.3 create() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<ScaledPermutation> gko::matrix::ScaledPermutation< ValueType, Index↩
Type >::create (
            ptr_param< const Permutation< IndexType >> permutation )  [static]
```

Create a ScaledPermutation from a Permutation.

The permutation will be copied, the scaling factors are all set to 1.0.

**Parameters**

| | |
|---|---|
| *permutation* | the permutation |

**Returns**

A smart pointer to the newly created matrix.

### 48.231.2.4 create() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<ScaledPermutation> gko::matrix::ScaledPermutation< ValueType, Index↩
Type >::create (
            std::shared_ptr< const Executor > exec,
            array< value_type > scaling_factors,
            array< index_type > permutation_indices )  [static]
```

Creates a ScaledPermutation matrix from already allocated arrays.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *permutation_indices* | array of permutation indices |
| *scaling_factors* | array of scaling factors |

**Returns**

A smart pointer to the newly created matrix.

### 48.231.2.5 create() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<ScaledPermutation> gko::matrix::ScaledPermutation< ValueType, Index↩
Type >::create (
            std::shared_ptr< const Executor > exec,
            size_type size = 0 )  [static]
```

Creates an uninitialized ScaledPermutation matrix.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | dimensions of the (square) scaled permutation matrix |

**Returns**

A smart pointer to the newly created matrix.

### 48.231.2.6 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const ScaledPermutation> gko::matrix::ScaledPermutation< ValueType,
IndexType >::create_const (
            std::shared_ptr< const Executor > exec,
            gko::detail::const_array_view< value_type > && scale,
            gko::detail::const_array_view< index_type > && perm_idxs )  [static]
```

Creates a constant (immutable) ScaledPermutation matrix from constant arrays.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the object on |
| *perm_idxs* | the permutation index array of the matrix |
| *scale* | the scaling factor array |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if it resides on the same executor as the matrix) or a copy of the arrays on the correct executor.

### 48.231.2.7 get_const_permutation()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::ScaledPermutation< ValueType, IndexType >::get_const_permutation
( ) const  [inline], [noexcept]
```

Returns a pointer to the permutation indices.

**Returns**

the pointer to the permutation indices.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
80     {
81         return permutation_.get_const_data();
82     }
```

References gko::array< ValueType >::get_const_data().

#### 48.231.2.8 get_const_scaling_factors()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::ScaledPermutation< ValueType, IndexType >::get_const_scaling_↩
factors ( ) const  [inline], [noexcept]
```

Returns a pointer to the scaling factors.

**Returns**

the pointer to the scaling factors.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
61     {
62         return scale_.get_const_data();
63     }
```

References gko::array< ValueType >::get_const_data().

#### 48.231.2.9 get_permutation()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::ScaledPermutation< ValueType, IndexType >::get_permutation ( )  [inline],
[noexcept]
```

Returns a pointer to the permutation indices.

**Returns**

the pointer to the permutation indices.

```
70 { return permutation_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.231.2.10 get_scaling_factors()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::ScaledPermutation< ValueType, IndexType >::get_scaling_factors ( )
[inline], [noexcept]
```

Returns a pointer to the scaling factors.

**Returns**

the pointer to the scaling factors.

```
51 { return scale_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.231.2.11 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::ScaledPermutation< ValueType, IndexType >::write (
            gko::matrix_data< value_type, index_type > & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/scaled_permutation.hpp

# 48.232 gko::experimental::reorder::ScaledReordered< ValueType, IndexType > Class Template Reference

Provides an interface to wrap reorderings like Rcm and diagonal scaling like equilibration around a LinOp like e.g.

`#include <ginkgo/core/reorder/scaled_reordered.hpp>`

## 48.232.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::experimental::reorder::ScaledReordered< ValueType, IndexType >**

Provides an interface to wrap reorderings like Rcm and diagonal scaling like equilibration around a LinOp like e.g.

a sparse direct solver.

Reorderings can be useful for reducing fill-in in the numerical factorization phase of direct solvers, diagonal scaling can help improve the numerical stability by reducing the condition number of the system matrix.

With a permutation matrix P, a row scaling R and a column scaling C, the inner operator is applied to the system matrix P∗R∗A∗C∗P^T instead of A. Instead of A∗x = b, the inner operator attempts to solve the equivalent linear system P∗R∗A∗C∗P^T∗y = P∗R∗b and retrieves the solution x = C∗P^T∗y. Note: The inner system matrix is computed from a clone of A, so the original system matrix is not changed.

**Template Parameters**

| | |
|---|---|
| *ValueType* | Type of the values of all matrices used in this class |
| *IndexType* | Type of the indices of all matrices used in this class |

The documentation for this class was generated from the following file:

- ginkgo/core/reorder/scaled_reordered.hpp

# 48.233 gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType > Class Template Reference

A Schwarz preconditioner is a simple domain decomposition preconditioner that generalizes the Block Jacobi preconditioner, incorporating options for different local subdomain solvers and overlaps between the subdomains.

`#include <ginkgo/core/distributed/preconditioner/schwarz.hpp>`

## Public Member Functions

- bool apply_uses_initial_guess () const override

  *Return whether the local solvers use the data in x as an initial guess.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, LocalIndexType, GlobalIndexType >())

  *Create the parameters from the property_tree.*

## 48.233.1 Detailed Description

**template<typename ValueType = default_precision, typename LocalIndexType = int32, typename GlobalIndexType = int64>**
**class gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType >**

A Schwarz preconditioner is a simple domain decomposition preconditioner that generalizes the Block Jacobi preconditioner, incorporating options for different local subdomain solvers and overlaps between the subdomains.

A L1 smoother variant is also available, which updates the diagonal block with the sums of the off-diagonal block row sums.

See Iterative Methods for Sparse Linear Systems (Y. Saad) for a general treatment and variations of the method.

A Two-level variant is also available. To enable two-level preconditioning, you need to specify a LinOpFactory that can generate a multigrid::MultigridLevel and a solver for the coarse level solution. Currently, only additive coarse correction is supported with an optional weighting between the local and the coarse solutions, for cases when the coarse solutions might tend to overcorrect.

- See Smith, Bjorstad, Gropp, Domain Decomposition, 1996, Cambridge University Press.

**Note**

Currently overlap is not supported (TODO).

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix element |
| *LocalIndexType* | local integer type of the matrix |
| *GlobalIndexType* | global integer type of the matrix |

## 48.233.2 Member Function Documentation

### 48.233.2.1 apply_uses_initial_guess()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
bool gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, Global↩
IndexType >::apply_uses_initial_guess ( ) const  [override]
```

Return whether the local solvers use the data in x as an initial guess.

**Returns**

true when the local solvers use the data in x as an initial guess. otherwise, false.

**Note**

TODO: after adding refining step, need to revisit this.

### 48.233.2.2 parse()

```
template<typename ValueType = default_precision, typename LocalIndexType = int32, typename
GlobalIndexType = int64>
static parameters_type gko::experimental::distributed::preconditioner::Schwarz< ValueType,
LocalIndexType, GlobalIndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, LocalInde
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children objects.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children objects. The default uses the value/local/global index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following file:

- ginkgo/core/distributed/preconditioner/schwarz.hpp

## 48.234 gko::scoped_device_id_guard Class Reference

This move-only class uses RAII to set the device id within a scoped block, if necessary.

`#include <ginkgo/core/base/scoped_device_id_guard.hpp>`

### Public Member Functions

- scoped_device_id_guard (const ReferenceExecutor *exec, int device_id)
  *Create a scoped device id from an Reference.*
- scoped_device_id_guard (const OmpExecutor *exec, int device_id)
  *Create a scoped device id from an OmpExecutor.*
- scoped_device_id_guard (const CudaExecutor *exec, int device_id)
  *Create a scoped device id from an CudaExecutor.*
- scoped_device_id_guard (const HipExecutor *exec, int device_id)
  *Create a scoped device id from an HipExecutor.*
- scoped_device_id_guard (const DpcppExecutor *exec, int device_id)
  *Create a scoped device id from an DpcppExecutor.*

### 48.234.1 Detailed Description

This move-only class uses RAII to set the device id within a scoped block, if necessary.

The class behaves similar to std::scoped_lock. The scoped guard will make sure that the device code is run on the correct device within one scoped block, when run with multiple devices. Depending on the executor it will record the current device id and set the device id to the one being passed in. After the scope has been exited, the destructor sets the device_id back to the one before entering the scope. The OmpExecutor and DpcppExecutor don't require setting an device id, so in those cases, the class is a no-op.

The device id scope has to be constructed from a executor with concrete type (not plain Executor) and a device id. Only the type of the executor object is relevant, so the pointer will not be accessed, and may even be a nullptr. From the executor type the correct derived class of detail::generic_scoped_device_id_guard is picked. The following illustrates the usage of this class:

```
{
  scoped_device_id_guard g{static_cast<CudaExecutor>(nullptr), 1};
  // now the device id is set to 1
}
// now the device id is reverted again
```

### 48.234.2 Constructor & Destructor Documentation

#### 48.234.2.1 scoped_device_id_guard() [1/5]

```
gko::scoped_device_id_guard::scoped_device_id_guard (
            const ReferenceExecutor * exec,
            int device_id )
```

Create a scoped device id from an Reference.

The resulting object will be a noop.

**Parameters**

| | |
|---|---|
| *exec* | Not used. |
| *device_id* | Not used. |

#### 48.234.2.2 scoped_device_id_guard() [2/5]

```
gko::scoped_device_id_guard::scoped_device_id_guard (
            const OmpExecutor * exec,
            int device_id )
```

Create a scoped device id from an OmpExecutor.

The resulting object will be a noop.

**Parameters**

| | |
|---|---|
| *exec* | Not used. |
| *device_id* | Not used. |

#### 48.234.2.3 scoped_device_id_guard() [3/5]

```
gko::scoped_device_id_guard::scoped_device_id_guard (
            const CudaExecutor * exec,
            int device_id )
```

Create a scoped device id from an CudaExecutor.

The resulting object will set the cuda device id accordingly.

**Parameters**

| | |
|---|---|
| *exec* | Not used. |
| *device_id* | The device id to use within the scope. |

#### 48.234.2.4 scoped_device_id_guard() [4/5]

```
gko::scoped_device_id_guard::scoped_device_id_guard (
            const HipExecutor * exec,
            int device_id )
```

Create a scoped device id from an HipExecutor.

The resulting object will set the hip device id accordingly.

**Parameters**

| | |
|---|---|
| *exec* | Not used. |
| *device_id* | The device id to use within the scope. |

#### 48.234.2.5 scoped_device_id_guard() [5/5]

```
gko::scoped_device_id_guard::scoped_device_id_guard (
            const DpcppExecutor * exec,
            int device_id )
```

Create a scoped device id from an DpcppExecutor.

The resulting object will be a noop.

**Parameters**

| | |
|---|---|
| *exec* | Not used. |
| *device_id* | Not used. |

The documentation for this class was generated from the following file:

- ginkgo/core/base/scoped_device_id_guard.hpp

## 48.235 gko::segmented_array< T > Struct Template Reference

A minimal interface for a segmented array.

```
#include <ginkgo/core/base/segmented_array.hpp>
```

### Public Member Functions

- segmented_array (std::shared_ptr< const Executor > exec)
  *Create an empty segmented array.*
- segmented_array (std::shared_ptr< const Executor > exec, const segmented_array &other)
  *Copies a segmented array to a different executor.*
- segmented_array (std::shared_ptr< const Executor > exec, segmented_array &&other)
  *Moves a segmented array to a different executor.*
- size_type get_size () const
  *Get the total size of the stored buffer.*

- size_type get_segment_count () const

  *Get the number of segments.*
- T * get_flat_data ()

  *Access to the flat buffer.*
- const T * get_const_flat_data () const

  *Const-access to the flat buffer.*
- const gko::array< int64 > & get_offsets () const

  *Access to the segment offsets.*
- std::shared_ptr< const Executor > get_executor () const

  *Access the executor.*

## Static Public Member Functions

- static segmented_array create_from_sizes (const gko::array< int64 > &sizes)

  *Creates an uninitialized segmented array with predefined segment sizes.*
- static segmented_array create_from_sizes (gko::array< T > buffer, const gko::array< int64 > &sizes)

  *Creates a segmented array from a flat buffer and segment sizes.*
- static segmented_array create_from_offsets (gko::array< int64 > offsets)

  *Creates an uninitialized segmented array from offsets.*
- static segmented_array create_from_offsets (gko::array< T > buffer, gko::array< int64 > offsets)

  *Creates a segmented array from a flat buffer and offsets.*

### 48.235.1 Detailed Description

**template<typename T>**
**struct gko::segmented_array< T >**

A minimal interface for a segmented array.

The segmented array is stored as a flat buffer with an offsets array. The segment `i` contains the index range `[offset[i], offset[i + 1])` of the flat buffer.

**Template Parameters**

| | |
|---|---|
| *T* | value type stored in the arrays |

### 48.235.2 Constructor & Destructor Documentation

#### 48.235.2.1 segmented_array() [1/3]

```
template<typename T>
gko::segmented_array< T >::segmented_array (
            std::shared_ptr< const Executor > exec )  [explicit]
```

Create an empty segmented array.

**Parameters**

| | |
|---|---|
| *exec* | executor for storage arrays |

#### 48.235.2.2 segmented_array() [2/3]

```
template<typename T>
gko::segmented_array< T >::segmented_array (
            std::shared_ptr< const Executor > exec,
            const segmented_array< T > & other )
```

Copies a segmented array to a different executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor to copy to |
| *other* | the segmented array to copy from |

#### 48.235.2.3 segmented_array() [3/3]

```
template<typename T>
gko::segmented_array< T >::segmented_array (
            std::shared_ptr< const Executor > exec,
            segmented_array< T > && other )
```

Moves a segmented array to a different executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor to move to |
| *other* | the segmented array to move from |

### 48.235.3 Member Function Documentation

#### 48.235.3.1 create_from_offsets() [1/2]

```
template<typename T>
static segmented_array gko::segmented_array< T >::create_from_offsets (
            gko::array< int64 > offsets )  [static]
```

Creates an uninitialized segmented array from offsets.

**Parameters**

| | |
|---|---|
| *offsets* | the index offsets for each segment, and the total size of the buffer as last element |

### 48.235.3.2 create_from_offsets() [2/2]

```
template<typename T>
static segmented_array gko::segmented_array< T >::create_from_offsets (
            gko::array< T > buffer,
            gko::array< int64 > offsets )  [static]
```

Creates a segmented array from a flat buffer and offsets.

**Parameters**

| | |
|---|---|
| *buffer* | the flat buffer whose size has to match the last element of offsets |
| *offsets* | the index offsets for each segment, and the total size of the buffer as last element |

### 48.235.3.3 create_from_sizes() [1/2]

```
template<typename T>
static segmented_array gko::segmented_array< T >::create_from_sizes (
            const gko::array< int64 > & sizes )  [static]
```

Creates an uninitialized segmented array with predefined segment sizes.

**Parameters**

| | |
|---|---|
| *exec* | executor for storage arrays |
| *sizes* | the sizes of each segment |

### 48.235.3.4 create_from_sizes() [2/2]

```
template<typename T>
static segmented_array gko::segmented_array< T >::create_from_sizes (
            gko::array< T > buffer,
            const gko::array< int64 > & sizes )  [static]
```

Creates a segmented array from a flat buffer and segment sizes.

**Parameters**

| | |
|---|---|
| *buffer* | the flat buffer whose size has to match the sum of sizes |
| *sizes* | the sizes of each segment |

#### 48.235.3.5 get_const_flat_data()

```
template<typename T>
const T* gko::segmented_array< T >::get_const_flat_data ( ) const
```

Const-access to the flat buffer.

**Returns**

the flat buffer

#### 48.235.3.6 get_executor()

```
template<typename T>
std::shared_ptr<const Executor> gko::segmented_array< T >::get_executor ( ) const
```

Access the executor.

**Returns**

the executor

#### 48.235.3.7 get_flat_data()

```
template<typename T>
T* gko::segmented_array< T >::get_flat_data ( )
```

Access to the flat buffer.

**Returns**

the flat buffer

#### 48.235.3.8 get_offsets()

```
template<typename T>
const gko::array<int64>& gko::segmented_array< T >::get_offsets ( ) const
```

Access to the segment offsets.

**Returns**

the segment offsets

#### 48.235.3.9 get_segment_count()

```
template<typename T>
size_type gko::segmented_array< T >::get_segment_count ( ) const
```

Get the number of segments.

**Returns**

the number of segments

#### 48.235.3.10 get_size()

```
template<typename T>
size_type gko::segmented_array< T >::get_size ( ) const
```

Get the total size of the stored buffer.

**Returns**

the total size of the stored buffer.

The documentation for this struct was generated from the following file:

- ginkgo/core/base/segmented_array.hpp

## 48.236 gko::matrix::view::sellp< ValueType, IndexType > Struct Template Reference

Non-owning view of a matrix::Sellp to be used inside device kernels.

```
#include <ginkgo/core/matrix/device_views.hpp>
```

### Public Member Functions

- constexpr sellp (dim< 2 > size, size_type slice_size, size_type stride_factor, size_type total_cols, ValueType *values, IndexType *col_idxs, adapt_size_type *slice_lengths, adapt_size_type *slice_sets)

  *Constructs a sellp view.*
- constexpr sellp< const ValueType, const IndexType > as_const () const

  *Returns a const view of the same values.*
- constexpr ValueType & val_at (size_type row, size_type slice_set, size_type idx) const

  *accessing the value of the idx-th element within the given row*
- constexpr IndexType & col_at (size_type row, size_type slice_set, size_type idx) const

  *accessing the column index of the idx-th element within the given row*

#### 48.236.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**struct gko::matrix::view::sellp< ValueType, IndexType >**

Non-owning view of a matrix::Sellp to be used inside device kernels.

This type is used to provide a simple and stable ABI for passing data between libraries.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the value type used to store matrix values. |
| *IndexType* | the index type used to store matrix columns. |

The documentation for this struct was generated from the following file:

- ginkgo/core/matrix/device_views.hpp

## 48.237 gko::matrix::Sellp< ValueType, IndexType > Class Template Reference

SELL-P is a matrix format similar to ELL format.

`#include <ginkgo/core/matrix/sellp.hpp>`

### Public Member Functions

- void read (const mat_data &data) override

  *Reads a matrix from a matrix_data structure.*
- void read (const device_mat_data &data) override

  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override

  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override

*Writes a matrix to a matrix_data structure.*

- std::unique_ptr< Diagonal< ValueType > > extract_diagonal () const override

  *Extracts the diagonal entries of the matrix into a vector.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- value_type * get_values () noexcept

  *Returns the values of the matrix.*
- const value_type * get_const_values () const noexcept

  *Returns the values of the matrix.*
- index_type * get_col_idxs () noexcept

  *Returns the column indexes of the matrix.*
- const index_type * get_const_col_idxs () const noexcept

  *Returns the column indexes of the matrix.*
- size_type * get_slice_lengths () noexcept

  *Returns the lengths(columns) of slices.*
- const size_type * get_const_slice_lengths () const noexcept

  *Returns the lengths(columns) of slices.*
- size_type * get_slice_sets () noexcept

  *Returns the offsets of slices.*
- const size_type * get_const_slice_sets () const noexcept

  *Returns the offsets of slices.*
- size_type get_slice_size () const noexcept

  *Returns the size of a slice.*
- size_type get_stride_factor () const noexcept

  *Returns the stride factor(t) of SELL-P.*
- size_type get_total_cols () const noexcept

  *Returns the total column number.*
- size_type get_num_stored_elements () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*
- value_type & val_at (size_type row, size_type slice_set, size_type idx) noexcept

  *Returns the `idx`-th non-zero element of the `row`-th row with `slice_set` slice set.*
- value_type val_at (size_type row, size_type slice_set, size_type idx) const noexcept

  *Returns the `idx`-th non-zero element of the `row`-th row with `slice_set` slice set.*
- index_type & col_at (size_type row, size_type slice_set, size_type idx) noexcept

  *Returns the `idx`-th column index of the `row`-th row with `slice_set` slice set.*
- index_type col_at (size_type row, size_type slice_set, size_type idx) const noexcept

  *Returns the `idx`-th column index of the `row`-th row with `slice_set` slice set.*
- device_view get_device_view ()

  *get the non-owning device view*
- const_device_view get_const_device_view () const

  *get the const non-owning device view*
- Sellp & operator= (const Sellp &)

  *Copy-assigns a Sellp matrix.*
- Sellp & operator= (Sellp &&)

  *Move-assigns a Sellp matrix.*
- Sellp (const Sellp &)

  *Copy-assigns a Sellp matrix.*
- Sellp (Sellp &&)

  *Move-assigns a Sellp matrix.*

## Static Public Member Functions

- static std::unique_ptr< Sellp > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size={}, size_type total_cols=0)

  *Creates an uninitialized Sellp matrix of the specified size.*
- static std::unique_ptr< Sellp > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, size_type slice_size, size_type stride_factor, size_type total_cols)

  *Creates an uninitialized Sellp matrix of the specified size.*

### 48.237.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Sellp< ValueType, IndexType >**

SELL-P is a matrix format similar to ELL format.

The difference is that SELL-P format divides rows into smaller slices and store each slice with ELL format.

This implementation uses the column index value invalid_index<IndexType>() to mark padding entries that are not part of the sparsity pattern.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

### 48.237.2 Constructor & Destructor Documentation

#### 48.237.2.1 Sellp() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Sellp< ValueType, IndexType >::Sellp (
            const Sellp< ValueType, IndexType > &  )
```

Copy-assigns a Sellp matrix.

Inherits the executor, copies the data and parameters.

#### 48.237.2.2 Sellp() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::Sellp< ValueType, IndexType >::Sellp (
            Sellp< ValueType, IndexType > &&  )
```

Move-assigns a Sellp matrix.

Inherits the executor, moves the data and parameters. The moved-from object is empty (0x0 with valid slice_sets and unchanged parameters).

## 48.237.3 Member Function Documentation

### 48.237.3.1 col_at() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type gko::matrix::Sellp< ValueType, IndexType >::col_at (
              size_type row,
              size_type slice_set,
              size_type idx ) const  [inline], [noexcept]
```

Returns the `idx`-th column index of the `row`-th row with `slice_set` slice set.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element in the slice |
| *slice_set* | the slice set of the slice |
| *idx* | the idx-th stored element of the row in the slice |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the CPU results in a runtime error)

```
310     {
311         return this
312             ->get_const_col_idxs()[this->linearize_index(row, slice_set, idx)];
313     }
```

### 48.237.3.2 col_at() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type& gko::matrix::Sellp< ValueType, IndexType >::col_at (
              size_type row,
              size_type slice_set,
              size_type idx )  [inline], [noexcept]
```

Returns the `idx`-th column index of the `row`-th row with `slice_set` slice set.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element in the slice |
| *slice_set* | the slice set of the slice |
| *idx* | the idx-th stored element of the row in the slice |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the CPU results in a runtime error)

```
301     {
302         return this->get_col_idxs()[this->linearize_index(row, slice_set, idx)];
303     }
```

References gko::matrix::Sellp< ValueType, IndexType >::get_col_idxs().

#### 48.237.3.3 compute_absolute()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<absolute_type> gko::matrix::Sellp< ValueType, IndexType >::compute_absolute (
) const  [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Sellp< ValueType, IndexType > > >.

#### 48.237.3.4 create() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Sellp> gko::matrix::Sellp< ValueType, IndexType >::create (
              std::shared_ptr< const Executor > exec,
              const dim< 2 > & size,
              size_type slice_size,
              size_type stride_factor,
              size_type total_cols )  [static]
```

Creates an uninitialized Sellp matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *slice_size* | number of rows in each slice |
| *stride_factor* | factor for the stride in each slice (strides should be multiples of the stride_factor) |
| *total_cols* | number of the sum of all cols in every slice. |

**Returns**

A smart pointer to the newly created matrix.

#### 48.237.3.5 create() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<Sellp> gko::matrix::Sellp< ValueType, IndexType >::create (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = {},
            size_type total_cols = 0 )  [static]
```

Creates an uninitialized Sellp matrix of the specified size.

(The slice_size and stride_factor are set to the default values.)

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *total_cols* | number of the sum of all cols in every slice. |

**Returns**

A smart pointer to the newly created matrix.

#### 48.237.3.6 extract_diagonal()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<Diagonal<ValueType> > gko::matrix::Sellp< ValueType, IndexType >::extract_↩
diagonal ( ) const  [override], [virtual]
```

Extracts the diagonal entries of the matrix into a vector.

**Parameters**

| | |
|---|---|
| *diag* | the vector into which the diagonal will be written |

Implements gko::DiagonalExtractable< ValueType >.

#### 48.237.3.7 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::Sellp< ValueType, IndexType >::get_col_idxs ( )  [inline], [noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

```
170 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

Referenced by gko::matrix::Sellp< ValueType, IndexType >::col_at().

#### 48.237.3.8 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::Sellp< ValueType, IndexType >::get_const_col_idxs ( ) const
[inline], [noexcept]
```

Returns the column indexes of the matrix.

**Returns**

the column indexes of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
180     {
181         return col_idxs_.get_const_data();
182     }
```

References gko::array< ValueType >::get_const_data().

#### 48.237.3.9 get_const_slice_lengths()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const size_type* gko::matrix::Sellp< ValueType, IndexType >::get_const_slice_lengths ( ) const
[inline], [noexcept]
```

Returns the lengths(columns) of slices.

**Returns**

the lengths(columns) of slices.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
202     {
203         return slice_lengths_.get_const_data();
204     }
```

References gko::array< ValueType >::get_const_data().

### 48.237.3.10 get_const_slice_sets()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const size_type* gko::matrix::Sellp< ValueType, IndexType >::get_const_slice_sets ( ) const
[inline], [noexcept]
```

Returns the offsets of slices.

#### Returns

the offsets of slices.

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
221     {
222         return slice_sets_.get_const_data();
223     }
```

References gko::array< ValueType >::get_const_data().

### 48.237.3.11 get_const_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::Sellp< ValueType, IndexType >::get_const_values ( ) const  [inline],
[noexcept]
```

Returns the values of the matrix.

#### Returns

the values of the matrix.

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
161     {
162         return values_.get_const_data();
163     }
```

References gko::array< ValueType >::get_const_data().

### 48.237.3.12 get_num_stored_elements()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Sellp< ValueType, IndexType >::get_num_stored_elements ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the matrix.

#### Returns

the number of elements explicitly stored in the matrix

```
255     {
256         return values_.get_size();
257     }
```

References gko::array< ValueType >::get_size().

### 48.237.3.13 get_slice_lengths()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type* gko::matrix::Sellp< ValueType, IndexType >::get_slice_lengths ( )  [inline], [noexcept]
```

Returns the lengths(columns) of slices.

#### Returns

the lengths(columns) of slices.

```
190     {
191         return slice_lengths_.get_data();
192     }
```

References gko::array< ValueType >::get_data().

### 48.237.3.14 get_slice_sets()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type* gko::matrix::Sellp< ValueType, IndexType >::get_slice_sets ( )  [inline], [noexcept]
```

Returns the offsets of slices.

#### Returns

the offsets of slices.

```
211 { return slice_sets_.get_data(); }
```

References gko::array< ValueType >::get_data().

### 48.237.3.15 get_slice_size()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Sellp< ValueType, IndexType >::get_slice_size ( ) const  [inline],
[noexcept]
```

Returns the size of a slice.

#### Returns

the size of a slice.

```
230 { return slice_size_; }
```

### 48.237.3.16 get_stride_factor()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Sellp< ValueType, IndexType >::get_stride_factor ( ) const  [inline],
[noexcept]
```

Returns the stride factor(t) of SELL-P.

#### Returns

the stride factor(t) of SELL-P.

```
237 { return stride_factor_; }
```

### 48.237.3.17 get_total_cols()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Sellp< ValueType, IndexType >::get_total_cols ( ) const  [inline],
[noexcept]
```

Returns the total column number.

#### Returns

the total column number.

```
245     {
246         return values_.get_size() / slice_size_;
247     }
```

References gko::array< ValueType >::get_size().

#### 48.237.3.18 get_values()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::Sellp< ValueType, IndexType >::get_values ( )  [inline], [noexcept]
```

Returns the values of the matrix.

**Returns**

the values of the matrix.

```
151 { return values_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.237.3.19 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Sellp& gko::matrix::Sellp< ValueType, IndexType >::operator= (
            const Sellp< ValueType, IndexType > &  )
```

Copy-assigns a Sellp matrix.

Preserves the executor, copies the data and parameters.

#### 48.237.3.20 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
Sellp& gko::matrix::Sellp< ValueType, IndexType >::operator= (
            Sellp< ValueType, IndexType > &&  )
```

Move-assigns a Sellp matrix.

Preserves the executor, moves the data and parameters. The moved-from object is empty (0x0 with valid slice_sets and unchanged parameters).

#### 48.237.3.21 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Sellp< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.237.3.22 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Sellp< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.237.3.23 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Sellp< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.237.3.24 val_at() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type gko::matrix::Sellp< ValueType, IndexType >::val_at (
            size_type row,
            size_type slice_set,
            size_type idx ) const  [inline], [noexcept]
```

Returns the `idx`-th non-zero element of the `row`-th row with `slice_set` slice set.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element in the slice |
| *slice_set* | the slice set of the slice |
| *idx* | the idx-th stored element of the row in the slice |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the CPU results in a runtime error)

```
282     {
283         return values_
284             .get_const_data()[this->linearize_index(row, slice_set, idx)];
285     }
```

References gko::array< ValueType >::get_const_data().

### 48.237.3.25 val_at() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type& gko::matrix::Sellp< ValueType, IndexType >::val_at (
              size_type row,
              size_type slice_set,
              size_type idx )  [inline], [noexcept]
```

Returns the `idx`-th non-zero element of the `row`-th row with `slice_set` slice set.

**Parameters**

| | |
|---|---|
| *row* | the row of the requested element in the slice |
| *slice_set* | the slice set of the slice |
| *idx* | the idx-th stored element of the row in the slice |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the CPU results in a runtime error)

```
273     {
274         return values_.get_data()[this->linearize_index(row, slice_set, idx)];
275     }
```

References gko::array< ValueType >::get_data().

### 48.237.3.26 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Sellp< ValueType, IndexType >::write (
              mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following files:

- ginkgo/core/matrix/csr.hpp
- ginkgo/core/matrix/sellp.hpp

## 48.238 gko::log::SolverProgress Class Reference

This Logger outputs the value of all scalar values (and potentially vectors) stored internally by the solver after each iteration.

`#include <ginkgo/core/log/solver_progress.hpp>`

### Static Public Member Functions

- static std::shared_ptr< SolverProgress > create_scalar_table_writer (std::ostream &output, int precision=6, int column_width=12)

  *Creates a logger printing the value for all scalar values in the solver after each iteration in an ASCII table.*
- static std::shared_ptr< SolverProgress > create_scalar_csv_writer (std::ostream &output, int precision=6, char separator=',')

  *Creates a logger printing the value for all scalar values in the solver after each iteration in a CSV table.*
- static std::shared_ptr< SolverProgress > create_vector_storage (std::string output_file_prefix="solver_↩ ", bool binary=false)

  *Creates a logger storing all vectors and scalar values in the solver after each iteration on disk.*

### 48.238.1 Detailed Description

This Logger outputs the value of all scalar values (and potentially vectors) stored internally by the solver after each iteration.

It needs to be attached to the solver being inspected.

### 48.238.2 Member Function Documentation

#### 48.238.2.1 create_scalar_csv_writer()

```
static std::shared_ptr<SolverProgress> gko::log::SolverProgress::create_scalar_csv_writer (
            std::ostream & output,
            int precision = 6,
            char separator = ',' )  [static]
```

Creates a logger printing the value for all scalar values in the solver after each iteration in a CSV table.

If the solver is applied to multiple right-hand sides, only the first right-hand side gets printed.

**Parameters**

| | |
|---|---|
| *output* | the stream to write the output to. |
| *precision* | the number of digits of precision to print |
| *separator* | the character separating columns from each other |

#### 48.238.2.2 create_scalar_table_writer()

```
static std::shared_ptr<SolverProgress> gko::log::SolverProgress::create_scalar_table_writer (
            std::ostream & output,
            int precision = 6,
            int column_width = 12 )  [static]
```

Creates a logger printing the value for all scalar values in the solver after each iteration in an ASCII table.

If the solver is applied to multiple right-hand sides, only the first right-hand side gets printed.

**Parameters**

| | |
|---|---|
| *output* | the stream to write the output to. |
| *precision* | the number of digits of precision to print |
| *column_width* | the number of characters an output column is wide |

#### 48.238.2.3 create_vector_storage()

```
static std::shared_ptr<SolverProgress> gko::log::SolverProgress::create_vector_storage (
            std::string output_file_prefix = "solver_",
            bool binary = false )  [static]
```

Creates a logger storing all vectors and scalar values in the solver after each iteration on disk.

This logger can handle multiple right-hand sides, in contrast to create_scalar_table_writer or create_scalar_csv_↩ writer.

**Parameters**

| | |
|---|---|
| *output_file_prefix* | the path and file name prefix used to generate the output file names. |
| *precision* | the number of digits of precision to print when outputting matrices in text format |
| *binary* | if true, write data in Ginkgo's own binary format (lossless), if false write data in the MatrixMarket format (potentially lossy) |

The documentation for this class was generated from the following file:

- ginkgo/core/log/solver_progress.hpp

# 48.239 gko::preconditioner::Sor< ValueType, IndexType > Class Template Reference

This class generates the (S)SOR preconditioner.

`#include <ginkgo/core/preconditioner/sor.hpp>`

## Public Member Functions

- const parameters_type & get_parameters ()

  *Returns the parameters used to construct the factory.*
- const parameters_type & get_parameters () const

  *Returns the parameters used to construct the factory.*
- std::unique_ptr< composition_type > generate (std::shared_ptr< const LinOp > system_matrix) const

## Static Public Member Functions

- static parameters_type build ()

  *Creates a new parameter_type to set up the factory.*

### 48.239.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::preconditioner::Sor< ValueType, IndexType >**

This class generates the (S)SOR preconditioner.

The SOR preconditioner starts from a splitting of the matrix $A$ into $A = D + L + U$, where $L$ contains all entries below the diagonal, and $U$ contains all entries above the diagonal. The application of the preconditioner is then defined as solving $Mx = y$ with

$$M = \frac{1}{\omega}(D + \omega L), \quad 0 < \omega < 2.$$

$\omega$ is known as the relaxation factor. The preconditioner can be made symmetric, leading to the SSOR preconditioner. Here, $M$ is defined as

$$M = \frac{1}{\omega(2-\omega)}(D + \omega L)D^{-1}(D + \omega U), \quad 0 < \omega < 2.$$

This class is a factory, which will only generate the preconditioner. The resulting LinOp will represent the application of $M^{-1}$.

**References**

- Saad, Y. *Iterative Methods for Sparse Linear Systems.* 2nd ed. SIAM, 2003, ch. 4.1. https://doi.org/10.1137/1.9780898718003

**Template Parameters**

| | |
|---|---|
| *ValueType* | The value type of the internally used CSR matrix |
| *IndexType* | The index type of the internally used CSR matrix |

## 48.239.2 Member Function Documentation

### 48.239.2.1 generate()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<composition_type> gko::preconditioner::Sor< ValueType, IndexType >::generate
(
            std::shared_ptr< const LinOp > system_matrix ) const
```

**Parameters**

| | |
|---|---|
| *system_matrix* | the source matrix used to generate the preconditioner. |

**Note**

This function overrides the default LinOpFactory::generate to return a Factorization instead of a generic LinOp, which would need to be cast to Factorization again to access its factors. It is only necessary because smart pointers aren't covariant.

### 48.239.2.2 get_parameters() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
const parameters_type& gko::preconditioner::Sor< ValueType, IndexType >::get_parameters ( )
[inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
91 { return parameters_; }
```

### 48.239.2.3 get_parameters() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
const parameters_type& gko::preconditioner::Sor< ValueType, IndexType >::get_parameters ( )
const  [inline]
```

Returns the parameters used to construct the factory.

**Returns**

the parameters used to construct the factory.

```
96 { return parameters_; }
```

The documentation for this class was generated from the following file:

- ginkgo/core/preconditioner/sor.hpp

# 48.240 gko::span Struct Reference

A span is a lightweight structure used to create sub-ranges from other ranges.

`#include <ginkgo/core/base/range.hpp>`

## Public Member Functions

- constexpr span (size_type point) noexcept

  *Creates a span representing a point `point`.*
- constexpr span (size_type begin, size_type end) noexcept

  *Creates a span.*
- constexpr bool is_valid () const

  *Checks if a span is valid.*
- constexpr size_type length () const

  *Returns the length of a span.*

## Public Attributes

- const size_type begin

  *Beginning of the span.*
- const size_type end

  *End of the span.*

### 48.240.1 Detailed Description

A span is a lightweight structure used to create sub-ranges from other ranges.

A `span s` represents a contiguous set of indexes in one dimension of the range, starting on index `s.begin` (inclusive) and ending at index `s.end` (exclusive). A span is only valid if its starting index is smaller than its ending index.

Spans can be compared using the `==` and `!=` operators. Two spans are identical if both their `begin` and `end` values are identical.

Spans also have two distinct partial orders defined on them:

1. `x < y` (`y > x`) if and only if `x.end < y.begin`
2. `x <= y` (`y >= x`) if and only if `x.end <= y.begin`

Note that the orders are in fact partial - there are spans `x` and `y` for which none of the following inequalities holds: `x < y`, `x > y`, `x == y`, `x <= y`, `x >= y`. An example are spans `span{0, 2}` and `span{1, 3}`.

In addition, `<=` is a distinct order from `<`, and not just an extension of the strict order to its weak equivalent. Thus, `x <= y` is not equivalent to `x < y || x == y`.

### 48.240.2 Constructor & Destructor Documentation

#### 48.240.2.1 span() [1/2]

```
constexpr gko::span::span (
            size_type point )  [inline], [constexpr], [noexcept]
```

Creates a span representing a point `point`.

The `begin` of this span is set to `point`, and the `end` to `point + 1`.

**Parameters**

| | |
|---|---|
| *point* | the point which the span represents |

```
54         :  span{point, point + 1}
55     {}
```

#### 48.240.2.2 span() [2/2]

```
constexpr gko::span::span (
             size_type begin,
             size_type end )  [inline], [constexpr], [noexcept]
```

Creates a span.

**Parameters**

| | |
|---|---|
| *begin* | the beginning of the span |
| *end* | the end of the span |

```
64         :  begin{begin}, end{end}
65     {}
```

References begin.

### 48.240.3 Member Function Documentation

#### 48.240.3.1 is_valid()

```
constexpr bool gko::span::is_valid ( ) const  [inline], [constexpr]
```

Checks if a span is valid.

**Returns**

true if and only if `this->begin <= this->end`

```
72 { return begin <= end; }
```

References begin, and end.

Referenced by gko::accessor::row_major< ValueType, Dimensionality >::operator()().

#### 48.240.3.2 length()

```
constexpr size_type gko::span::length ( ) const  [inline], [constexpr]
```

Returns the length of a span.

**Returns**

```
this->end - this->begin
```

```
79 { return end - begin; }
```

References begin, and end.

The documentation for this struct was generated from the following file:

- ginkgo/core/base/range.hpp

## 48.241 gko::matrix::SparsityCsr< ValueType, IndexType > Class Template Reference

SparsityCsr is a matrix format which stores only the sparsity pattern of a sparse matrix by compressing each row of the matrix (compressed sparse row format).

```
#include <ginkgo/core/matrix/sparsity_csr.hpp>
```

### Public Member Functions

- void read (const mat_data &data) override
  *Reads a matrix from a matrix_data structure.*
- void read (const device_mat_data &data) override
  *Reads a matrix from a device_matrix_data structure.*
- void read (device_mat_data &&data) override
  *Reads a matrix from a device_matrix_data structure.*
- void write (mat_data &data) const override
  *Writes a matrix to a matrix_data structure.*
- std::unique_ptr< LinOp > transpose () const override
  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override
  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- std::unique_ptr< SparsityCsr > to_adjacency_matrix () const
  *Transforms the sparsity matrix to an adjacency matrix.*
- void sort_by_column_index ()
  *Sorts each row by column index.*
- index_type * get_col_idxs () noexcept
  *Returns the column indices of the matrix.*
- const index_type * get_const_col_idxs () const noexcept
  *Returns the column indices of the matrix.*
- index_type * get_row_ptrs () noexcept
  *Returns the row pointers of the matrix.*

- const index_type * get_const_row_ptrs () const noexcept

  *Returns the row pointers of the matrix.*
- value_type * get_value () noexcept

  *Returns the value stored in the matrix.*
- const value_type * get_const_value () const noexcept

  *Returns the value stored in the matrix.*
- size_type get_num_nonzeros () const noexcept

  *Returns the number of elements explicitly stored in the matrix.*
- SparsityCsr & operator= (const SparsityCsr &)

  *Copy-assigns a SparsityCsr matrix.*
- SparsityCsr & operator= (SparsityCsr &&)

  *Move-assigns a SparsityCsr matrix.*
- SparsityCsr (const SparsityCsr &)

  *Copy-constructs a SparsityCsr matrix.*
- SparsityCsr (SparsityCsr &&)

  *Move-constructs a SparsityCsr matrix.*

## Static Public Member Functions

- static std::unique_ptr< SparsityCsr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size=dim< 2 >{}, size_type num_nonzeros={})

  *Creates an uninitialized SparsityCsr matrix of the specified size.*
- static std::unique_ptr< SparsityCsr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, array< index_type > col_idxs, array< index_type > row_ptrs, value_type value=one< ValueType >())

  *Creates a SparsityCsr matrix from already allocated (and initialized) row pointer and column index arrays.*
- template<typename ColIndexType , typename RowPtrType >
  static std::unique_ptr< SparsityCsr > create (std::shared_ptr< const Executor > exec, const dim< 2 > &size, std::initializer_list< ColIndexType > col_idxs, std::initializer_list< RowPtrType > row_ptrs, value_type value=one< ValueType >())

  *create(std::shared_ptr<const*
- static std::unique_ptr< SparsityCsr > create (std::shared_ptr< const Executor > exec, std::shared_ptr< const LinOp > matrix)

  *Creates a Sparsity matrix from an existing matrix.*
- static std::unique_ptr< const SparsityCsr > create_const (std::shared_ptr< const Executor > exec, const dim< 2 > &size, gko::detail::const_array_view< IndexType > &&col_idxs, gko::detail::const_array_view< IndexType > &&row_ptrs, ValueType value=one< ValueType >())

  *Creates a constant (immutable) SparsityCsr matrix from constant arrays.*

### 48.241.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::SparsityCsr< ValueType, IndexType >**

SparsityCsr is a matrix format which stores only the sparsity pattern of a sparse matrix by compressing each row of the matrix (compressed sparse row format).

The values of the nonzero elements are stored as a value array of length 1. All the values in the matrix are equal to this value. By default, this value is set to 1.0. A row pointer array also stores the linearized starting index of each row. An additional column index array is used to identify the column where a nonzero is present.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of vectors in apply |
| *IndexType* | precision of matrix indexes |

## 48.241.2 Constructor & Destructor Documentation

### 48.241.2.1 SparsityCsr() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::SparsityCsr< ValueType, IndexType >::SparsityCsr (
            const SparsityCsr< ValueType, IndexType > &  )
```

Copy-constructs a SparsityCsr matrix.

Inherits executor, strategy and data.

### 48.241.2.2 SparsityCsr() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::matrix::SparsityCsr< ValueType, IndexType >::SparsityCsr (
            SparsityCsr< ValueType, IndexType > &&  )
```

Move-constructs a SparsityCsr matrix.

Inherits executor, moves the data and leaves the moved-from object in an empty state (0x0 LinOp with unchanged executor, no nonzeros and valid row pointers).

## 48.241.3 Member Function Documentation

### 48.241.3.1 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::SparsityCsr< ValueType, IndexType >::conj_transpose ( )
const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.241.3.2 create() [1/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<SparsityCsr> gko::matrix::SparsityCsr< ValueType, IndexType >::create
(
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            array< index_type > col_idxs,
            array< index_type > row_ptrs,
            value_type value = one< ValueType >() )  [static]
```

Creates a SparsityCsr matrix from already allocated (and initialized) row pointer and column index arrays.

**Template Parameters**

| | |
|---|---|
| *ColIdxsArray* | type of `col_idxs` array |
| *RowPtrsArray* | type of `row_ptrs` array |

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *col_idxs* | array of column indexes |
| *row_ptrs* | array of row pointers |
| *value* | value stored. (same value for all matrix elements) |

**Note**

If one of `row_ptrs` or `col_idxs` is not an rvalue, not an array of IndexType and IndexType respectively, or is on the wrong executor, an internal copy of that array will be created, and the original array data will not be used in the matrix.

#### 48.241.3.3 create() [2/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
template<typename ColIndexType , typename RowPtrType >
static std::unique_ptr<SparsityCsr> gko::matrix::SparsityCsr< ValueType, IndexType >::create
(
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            std::initializer_list< ColIndexType > col_idxs,
            std::initializer_list< RowPtrType > row_ptrs,
            value_type value = one<ValueType>() )  [inline], [static]
```

create(std::shared_ptr<const

create(std::shared_ptr<const executor>=""">, const dim<2>&, array<index_type>, array<index_type>, value↩
_type)

```
236     {
237         return create(exec, size, array<index_type>{exec, std::move(col_idxs)},
238                       array<index_type>{exec, std::move(row_ptrs)}, value);
239     }
```

References gko::matrix::SparsityCsr< ValueType, IndexType >::create().

#### 48.241.3.4 create() [3/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<SparsityCsr> gko::matrix::SparsityCsr< ValueType, IndexType >::create
(
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size = dim< 2 >{},
            size_type num_nonzeros = {} )  [static]
```

Creates an uninitialized SparsityCsr matrix of the specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *size* | size of the matrix |
| *num_nonzeros* | number of nonzeros |

Referenced by gko::matrix::SparsityCsr< ValueType, IndexType >::create().

#### 48.241.3.5 create() [4/4]

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<SparsityCsr> gko::matrix::SparsityCsr< ValueType, IndexType >::create
(
            std::shared_ptr< const Executor > exec,
            std::shared_ptr< const LinOp > matrix )  [static]
```

Creates a Sparsity matrix from an existing matrix.

Uses the `copy_and_convert_to` functionality.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated to the matrix |
| *matrix* | The input matrix |

#### 48.241.3.6 create_const()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static std::unique_ptr<const SparsityCsr> gko::matrix::SparsityCsr< ValueType, IndexType >↩
::create_const (
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & size,
            gko::detail::const_array_view< IndexType > && col_idxs,
            gko::detail::const_array_view< IndexType > && row_ptrs,
            ValueType value = one<ValueType>() )  [inline], [static]
```

Creates a constant (immutable) SparsityCsr matrix from constant arrays.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create the matrix on |
| *size* | the dimensions of the matrix |
| *value* | the value stored for all the matrix elements |
| *col_idxs* | the column index array of the matrix |
| *row_ptrs* | the row pointer array of the matrix |
| *strategy* | the strategy the matrix uses for SpMV operations |

**Returns**

A smart pointer to the constant matrix wrapping the input arrays (if they reside on the same executor as the matrix) or a copy of these arrays on the correct executor.

```
270     {
271         // cast const-ness away, but return a const object afterwards,
272         // so we can ensure that no modifications take place.
273         return std::unique_ptr<const SparsityCsr>(new SparsityCsr{
274             exec, size, gko::detail::array_const_cast(std::move(col_idxs)),
275             gko::detail::array_const_cast(std::move(row_ptrs)), value});
276     }
```

### 48.241.3.7 get_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::SparsityCsr< ValueType, IndexType >::get_col_idxs ( )  [inline],
[noexcept]
```

Returns the column indices of the matrix.

**Returns**

the column indices of the matrix.

```
128 { return col_idxs_.get_data(); }
```

References gko::array< ValueType >::get_data().

### 48.241.3.8 get_const_col_idxs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::SparsityCsr< ValueType, IndexType >::get_const_col_idxs ( )
const  [inline], [noexcept]
```

Returns the column indices of the matrix.

**Returns**

the column indices of the matrix.

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
138     {
139         return col_idxs_.get_const_data();
140     }
```

References gko::array< ValueType >::get_const_data().

### 48.241.3.9 get_const_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const index_type* gko::matrix::SparsityCsr< ValueType, IndexType >::get_const_row_ptrs ( )
const  [inline], [noexcept]
```

Returns the row pointers of the matrix.

#### Returns

the row pointers of the matrix.

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
157     {
158         return row_ptrs_.get_const_data();
159     }
```

References gko::array< ValueType >::get_const_data().

### 48.241.3.10 get_const_value()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const value_type* gko::matrix::SparsityCsr< ValueType, IndexType >::get_const_value ( ) const
[inline], [noexcept]
```

Returns the value stored in the matrix.

#### Returns

the value of the matrix.

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
176     {
177         return value_.get_const_data();
178     }
```

References gko::array< ValueType >::get_const_data().

#### 48.241.3.11 get_num_nonzeros()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::SparsityCsr< ValueType, IndexType >::get_num_nonzeros ( ) const  [inline],
[noexcept]
```

Returns the number of elements explicitly stored in the matrix.

**Returns**

the number of elements explicitly stored in the matrix

```
185 { return col_idxs_.get_size(); }
```

References gko::array< ValueType >::get_size().

#### 48.241.3.12 get_row_ptrs()

```
template<typename ValueType = default_precision, typename IndexType = int32>
index_type* gko::matrix::SparsityCsr< ValueType, IndexType >::get_row_ptrs ( )  [inline],
[noexcept]
```

Returns the row pointers of the matrix.

**Returns**

the row pointers of the matrix.

```
147 { return row_ptrs_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.241.3.13 get_value()

```
template<typename ValueType = default_precision, typename IndexType = int32>
value_type* gko::matrix::SparsityCsr< ValueType, IndexType >::get_value ( )  [inline], [noexcept]
```

Returns the value stored in the matrix.

**Returns**

the value of the matrix.

```
166 { return value_.get_data(); }
```

References gko::array< ValueType >::get_data().

#### 48.241.3.14 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
SparsityCsr& gko::matrix::SparsityCsr< ValueType, IndexType >::operator= (
            const SparsityCsr< ValueType, IndexType > &  )
```

Copy-assigns a SparsityCsr matrix.

Preserves executor, copies everything else.

#### 48.241.3.15 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
SparsityCsr& gko::matrix::SparsityCsr< ValueType, IndexType >::operator= (
            SparsityCsr< ValueType, IndexType > &&  )
```

Move-assigns a SparsityCsr matrix.

Preserves executor, moves the data and leaves the moved-from object in an empty state (0x0 LinOp with unchanged executor, no nonzeros and valid row pointers).

#### 48.241.3.16 read() [1/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::SparsityCsr< ValueType, IndexType >::read (
            const device_mat_data & data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.241.3.17 read() [2/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::SparsityCsr< ValueType, IndexType >::read (
            const mat_data & data )  [override], [virtual]
```

Reads a matrix from a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.241.3.18 read() [3/3]

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::SparsityCsr< ValueType, IndexType >::read (
            device_mat_data && data )  [override], [virtual]
```

Reads a matrix from a device_matrix_data structure.

The structure may be emptied by this function.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure. |

Reimplemented from gko::ReadableFromMatrixData< ValueType, IndexType >.

#### 48.241.3.19 to_adjacency_matrix()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<SparsityCsr> gko::matrix::SparsityCsr< ValueType, IndexType >::to_adjacency↩
_matrix ( ) const
```

Transforms the sparsity matrix to an adjacency matrix.

As the adjacency matrix has to be square, the input SparsityCsr matrix for this function to work has to be square.

**Note**

The adjacency matrix in this case is the sparsity pattern but with the diagonal ones removed. This is mainly used for the reordering/partitioning as taken in by graph libraries such as METIS.

#### 48.241.3.20 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::matrix::SparsityCsr< ValueType, IndexType >::transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

#### 48.241.3.21 write()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::SparsityCsr< ValueType, IndexType >::write (
            mat_data & data ) const  [override], [virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implements gko::WritableToMatrixData< ValueType, IndexType >.

The documentation for this class was generated from the following files:

- ginkgo/core/matrix/csr.hpp
- ginkgo/core/matrix/sparsity_csr.hpp

## 48.242 gko::experimental::mpi::status Struct Reference

The status struct is a light wrapper around the MPI_Status struct.

```
#include <ginkgo/core/base/mpi.hpp>
```

### Public Member Functions

- status ()

  *The default constructor.*
- MPI_Status * get ()

  *Get a pointer to the underlying MPI_Status object.*
- template<typename T >
  int get_count (const T *data) const

  *Get the count of the number of elements received by the communication call.*

### 48.242.1 Detailed Description

The status struct is a light wrapper around the MPI_Status struct.

### 48.242.2 Constructor & Destructor Documentation

#### 48.242.2.1 status()

```
gko::experimental::mpi::status::status ( )  [inline]
```

The default constructor.

It creates an empty MPI_Status
```
294 :  status_(MPI_Status{}) {}
```

### 48.242.3 Member Function Documentation

#### 48.242.3.1 get()

```
MPI_Status* gko::experimental::mpi::status::get ( )  [inline]
```

Get a pointer to the underlying MPI_Status object.

**Returns**

a pointer to MPI_Status object

```
301 { return &this->status_; }
```

Referenced by gko::experimental::mpi::communicator::recv(), and gko::experimental::mpi::request::wait().

#### 48.242.3.2 get_count()

```
template<typename T >
int gko::experimental::mpi::status::get_count (
             const T * data ) const  [inline]
```

Get the count of the number of elements received by the communication call.

**Template Parameters**

| | |
|---|---|
| *T* | The datatype of the object that was received. |

**Parameters**

| | |
|---|---|
| *data* | The data object of type T that was received. |

**Returns**

the count

```
315     {
```

```
316         int count;
317         MPI_Get_count(&status_, type_impl<T>::get_type(), &count);
318         return count;
319     }
```

The documentation for this struct was generated from the following file:

- ginkgo/core/base/mpi.hpp

# 48.243 gko::stopping_status Class Reference

This class is used to keep track of the stopping status of one vector.

`#include <ginkgo/core/stop/stopping_status.hpp>`

## Public Member Functions

- bool has_stopped () const noexcept
  *Check if any stopping criteria was fulfilled.*
- bool has_converged () const noexcept
  *Check if convergence was reached.*
- bool is_finalized () const noexcept
  *Check if the corresponding vector stores the finalized result.*
- uint8 get_id () const noexcept
  *Get the id of the stopping criterion which caused the stop.*
- void reset () noexcept
  *Clear all flags.*
- void stop (uint8 id, bool set_finalized=true) noexcept
  *Call if a stop occurred due to a hard limit (and convergence was not reached).*
- void converge (uint8 id, bool set_finalized=true) noexcept
  *Call if convergence occurred.*
- void finalize () noexcept
  *Set the result to be finalized (it needs to be stopped or converged first).*

## Friends

- bool operator== (const stopping_status &x, const stopping_status &y) noexcept
  *Checks if two stopping statuses are equivalent.*
- bool operator!= (const stopping_status &x, const stopping_status &y) noexcept
  *Checks if two stopping statuses are different.*

## 48.243.1 Detailed Description

This class is used to keep track of the stopping status of one vector.

## 48.243.2 Member Function Documentation

### 48.243.2.1 converge()

```
void gko::stopping_status::converge (
            uint8 id,
            bool set_finalized = true )  [inline], [noexcept]
```

Call if convergence occurred.

**Parameters**

| | |
|---|---|
| *id* | id of the stopping criteria. |
| *set_finalized* | Controls if the current version should count as finalized (set to true) or not (set to false). |

```
96      {
97          if (!this->has_stopped()) {
98              data_ |= converged_mask_ | (id & id_mask_);
99              if (set_finalized) {
100                 data_ |= finalized_mask_;
101             }
102         }
103     }
```

References has_stopped().

#### 48.243.2.2 get_id()

```
uint8 gko::stopping_status::get_id ( ) const  [inline], [noexcept]
```

Get the id of the stopping criterion which caused the stop.

**Returns**

Returns the id of the stopping criterion which caused the stop.

```
61      {
62          return data_ & id_mask_;
63      }
```

Referenced by has_stopped().

#### 48.243.2.3 has_converged()

```
bool gko::stopping_status::has_converged ( ) const  [inline], [noexcept]
```

Check if convergence was reached.

**Returns**

Returns true if convergence was reached.

```
42      {
43          return data_ & converged_mask_;
44      }
```

#### 48.243.2.4 has_stopped()

```
bool gko::stopping_status::has_stopped ( ) const  [inline], [noexcept]
```

Check if any stopping criteria was fulfilled.

**Returns**

Returns true if any stopping criteria was fulfilled.

```
33     {
34         return get_id();
35     }
```

References get_id().

Referenced by converge(), finalize(), and stop().

#### 48.243.2.5 is_finalized()

```
bool gko::stopping_status::is_finalized ( ) const  [inline], [noexcept]
```

Check if the corresponding vector stores the finalized result.

**Returns**

Returns true if the corresponding vector stores the finalized result.

```
52     {
53         return data_ & finalized_mask_;
54     }
```

#### 48.243.2.6 stop()

```
void gko::stopping_status::stop (
            uint8 id,
            bool set_finalized = true )  [inline], [noexcept]
```

Call if a stop occurred due to a hard limit (and convergence was not reached).

**Parameters**

| | |
|---|---|
| *id* | id of the stopping criteria. |
| *set_finalized* | Controls if the current version should count as finalized (set to true) or not (set to false). |

```
79     {
80         if (!this->has_stopped()) {
81             data_ |= (id & id_mask_);
82             if (set_finalized) {
83                 data_ |= finalized_mask_;
84             }
85         }
86     }
```

References has_stopped().

### 48.243.3 Friends And Related Function Documentation

#### 48.243.3.1 operator"!=

```
bool operator!= (
            const stopping_status & x,
            const stopping_status & y )  [friend]
```

Checks if two stopping statuses are different.

**Parameters**

| | |
|---|---|
| *x* | a stopping status |
| *y* | a stopping status |

**Returns**

true if and only if `!(x == y)`

```
151 {
152     return x.data_ != y.data_;
153 }
```

#### 48.243.3.2 operator==

```
bool operator== (
            const stopping_status & x,
            const stopping_status & y )  [friend]
```

Checks if two stopping statuses are equivalent.

**Parameters**

| | |
|---|---|
| *x* | a stopping status |
| *y* | a stopping status |

**Returns**

true if and only if both `x` and `y` have the same mask and converged and finalized state

```
136 {
137     return x.data_ == y.data_;
138 }
```

The documentation for this class was generated from the following file:

- ginkgo/core/stop/stopping_status.hpp

# 48.244 gko::matrix::Hybrid< ValueType, IndexType >::strategy_type Class Reference

strategy_type is to decide how to set the hybrid config.

`#include <ginkgo/core/matrix/hybrid.hpp>`

## Public Member Functions

- strategy_type ()

  *Creates a strategy_type.*
- void compute_hybrid_config (const array< size_type > &row_nnz, size_type *ell_num_stored_elements_per_row, size_type *coo_nnz)

  *Computes the config of the Hybrid matrix (ell_num_stored_elements_per_row and coo_nnz).*
- size_type get_ell_num_stored_elements_per_row () const noexcept

  *Returns the number of stored elements per row of the ell part.*
- size_type get_coo_nnz () const noexcept

  *Returns the number of nonzeros of the coo part.*
- virtual size_type compute_ell_num_stored_elements_per_row (array< size_type > *row_nnz) const =0

  *Computes the number of stored elements per row of the ell part.*

### 48.244.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::matrix::Hybrid< ValueType, IndexType >::strategy_type**

strategy_type is to decide how to set the hybrid config.

It computes the number of stored elements per row of the ell part and then set the number of residual nonzeros as the number of nonzeros of the coo part.

The practical strategy method should inherit strategy_type and implement its `compute_ell_num_stored_elements_per_row` function.

### 48.244.2 Member Function Documentation

#### 48.244.2.1 compute_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
virtual size_type gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::compute_ell_num_stored_elements_per_row (
            array< size_type > * row_nnz ) const  [pure virtual]
```

Computes the number of stored elements per row of the ell part.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |

**Returns**

the number of stored elements per row of the ell part

Implemented in gko::matrix::Hybrid< ValueType, IndexType >::automatic, gko::matrix::Hybrid< ValueType, IndexType >::minimal_sto
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit, gko::matrix::Hybrid< ValueType, IndexType >::imbalance_
and gko::matrix::Hybrid< ValueType, IndexType >::column_limit.

Referenced by gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::compute_hybrid_config().

### 48.244.2.2 compute_hybrid_config()

```
template<typename ValueType = default_precision, typename IndexType = int32>
void gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::compute_hybrid_config (
            const array< size_type > & row_nnz,
            size_type * ell_num_stored_elements_per_row,
            size_type * coo_nnz )  [inline]
```

Computes the config of the Hybrid matrix (ell_num_stored_elements_per_row and coo_nnz).

For now, it copies row_nnz to the reference executor and performs all operations on the reference executor.

**Parameters**

| | |
|---|---|
| *row_nnz* | the number of nonzeros of each row |
| *ell_num_stored_elements_per_row* | the output number of stored elements per row of the ell part |
| *coo_nnz* | the output number of nonzeros of the coo part |

```
123         {
124             array<size_type> ref_row_nnz(row_nnz.get_executor()->get_master(),
125                                          row_nnz.get_size());
126             ref_row_nnz = row_nnz;
127             ell_num_stored_elements_per_row_ =
128                 this->compute_ell_num_stored_elements_per_row(&ref_row_nnz);
129             coo_nnz_ = this->compute_coo_nnz(ref_row_nnz);
130             *ell_num_stored_elements_per_row = ell_num_stored_elements_per_row_;
131             *coo_nnz = coo_nnz_;
132         }
```

References gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::compute_ell_num_stored_elements_↩
per_row(), gko::array< ValueType >::get_executor(), and gko::array< ValueType >::get_size().

### 48.244.2.3 get_coo_nnz()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::get_coo_nnz ( ) const
[inline], [noexcept]
```

Returns the number of nonzeros of the coo part.

**Returns**

the number of nonzeros of the coo part

```
149 { return coo_nnz_; }
```

#### 48.244.2.4 get_ell_num_stored_elements_per_row()

```
template<typename ValueType = default_precision, typename IndexType = int32>
size_type gko::matrix::Hybrid< ValueType, IndexType >::strategy_type::get_ell_num_stored_↩
elements_per_row ( ) const  [inline], [noexcept]
```

Returns the number of stored elements per row of the ell part.

**Returns**

the number of stored elements per row of the ell part

```
140         {
141             return ell_num_stored_elements_per_row_;
142         }
```

The documentation for this class was generated from the following file:

- ginkgo/core/matrix/hybrid.hpp

## 48.245 gko::log::Stream< ValueType > Class Template Reference

Stream is a Logger which logs every event to a stream.

```
#include <ginkgo/core/log/stream.hpp>
```

### Static Public Member Functions

- static std::unique_ptr< Stream > create (std::shared_ptr< const Executor > exec, const Logger::mask_type &enabled_events=Logger::all_events_mask, std::ostream &os=std::cout, bool verbose=false)

  *Creates a Stream logger.*
- static std::unique_ptr< Stream > create (const Logger::mask_type &enabled_events=Logger::all_events_↩ mask, std::ostream &os=std::cerr, bool verbose=false)

  *Creates a Stream logger.*

### 48.245.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::log::Stream< ValueType >**

Stream is a Logger which logs every event to a stream.

This can typically be used to log to a file or to the console.

**Template Parameters**

| | |
|---|---|
| *ValueType* | the type of values stored in the class (i.e. ValueType template parameter of the concrete Loggable this class will log) |

## 48.245.2 Member Function Documentation

### 48.245.2.1 create() [1/2]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Stream> gko::log::Stream< ValueType >::create (
            const Logger::mask_type & enabled_events = Logger::all_events_mask,
            std::ostream & os = std::cerr,
            bool verbose = false )  [inline], [static]
```

Creates a Stream logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *enabled_events* | the events enabled for this logger. By default all events. |
| *os* | the stream used for this logger |
| *verbose* | whether we want detailed information or not. This includes always printing residuals and other information which can give a large output. |

**Returns**

an std::unique_ptr to the the constructed object

```
197     {
198         return std::unique_ptr<Stream>(new Stream(enabled_events, os, verbose));
199     }
```

### 48.245.2.2 create() [2/2]

```
template<typename ValueType = default_precision>
static std::unique_ptr<Stream> gko::log::Stream< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            const Logger::mask_type & enabled_events = Logger::all_events_mask,
            std::ostream & os = std::cout,
            bool verbose = false )  [inline], [static]
```

Creates a Stream logger.

This dynamically allocates the memory, constructs the object and returns an std::unique_ptr to this object.

**Parameters**

| | |
|---|---|
| *exec* | the executor |
| *enabled_events* | the events enabled for this logger. By default all events. |
| *os* | the stream used for this logger |
| *verbose* | whether we want detailed information or not. This includes always printing residuals and other information which can give a large output. |

**Returns**

an std::unique_ptr to the the constructed object

```
176     {
177         return std::unique_ptr<Stream>(new Stream(enabled_events, os, verbose));
178     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/log/stream.hpp

# 48.246 gko::StreamError Class Reference

StreamError is thrown if accessing a stream failed.

`#include <ginkgo/core/base/exception.hpp>`

## Public Member Functions

- StreamError (const std::string &file, int line, const std::string &func, const std::string &message)

  *Initializes a file access error.*

## 48.246.1 Detailed Description

StreamError is thrown if accessing a stream failed.

## 48.246.2 Constructor & Destructor Documentation

### 48.246.2.1 StreamError()

```
gko::StreamError::StreamError (
            const std::string & file,
            int line,
            const std::string & func,
            const std::string & message )  [inline]
```

Initializes a file access error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The name of the function that tried to access the file |
| *message* | The error message |

```
603         :  Error(file, line, func + ":  " + message)
604     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.247 gko::log::ProfilerHook::SummaryWriter Class Reference

Receives the results from ProfilerHook::create_summary().

`#include <ginkgo/core/log/profiler_hook.hpp>`

### Public Member Functions

- virtual void write (const std::vector< summary_entry > &entries, std::chrono::nanoseconds overhead)=0

  *Callback to write out the summary results.*

### 48.247.1 Detailed Description

Receives the results from ProfilerHook::create_summary().

### 48.247.2 Member Function Documentation

#### 48.247.2.1 write()

```
virtual void gko::log::ProfilerHook::SummaryWriter::write (
            const std::vector< summary_entry > & entries,
            std::chrono::nanoseconds overhead )  [pure virtual]
```

Callback to write out the summary results.

**Parameters**

| | |
|---|---|
| *entries* | the vector of ranges with runtime and count. |
| *overhead* | an estimate of the profiler overhead |

Implemented in gko::log::ProfilerHook::TableSummaryWriter.

The documentation for this class was generated from the following file:

- ginkgo/core/log/profiler_hook.hpp

## 48.248 gko::log::ProfilerHook::TableSummaryWriter Class Reference

Writes the results from ProfilerHook::create_summary() and ProfilerHook::create_nested_summary() to a ASCII table in Markdown format.

`#include <ginkgo/core/log/profiler_hook.hpp>`

### Public Member Functions

- TableSummaryWriter (std::ostream &output=std::cerr, std::string header="Runtime summary")

  *Constructs a writer on an output stream.*
- void write (const std::vector< summary_entry > &entries, std::chrono::nanoseconds overhead) override

  *Callback to write out the summary results.*
- void write_nested (const nested_summary_entry &root, std::chrono::nanoseconds overhead) override

  *Callback to write out the summary results.*

### 48.248.1 Detailed Description

Writes the results from ProfilerHook::create_summary() and ProfilerHook::create_nested_summary() to a ASCII table in Markdown format.

### 48.248.2 Constructor & Destructor Documentation

#### 48.248.2.1 TableSummaryWriter()

```
gko::log::ProfilerHook::TableSummaryWriter::TableSummaryWriter (
            std::ostream & output = std::cerr,
            std::string header = "Runtime summary" )
```

Constructs a writer on an output stream.

**Parameters**

| | |
|---|---|
| *output* | the output stream to write the table to. |
| *header* | the header to write above the table. |

### 48.248.3 Member Function Documentation

#### 48.248.3.1 write()

```
void gko::log::ProfilerHook::TableSummaryWriter::write (
            const std::vector< summary_entry > & entries,
            std::chrono::nanoseconds overhead )  [override], [virtual]
```

Callback to write out the summary results.

**Parameters**

| | |
|---|---|
| *entries* | the vector of ranges with runtime and count. |
| *overhead* | an estimate of the profiler overhead |

Implements gko::log::ProfilerHook::SummaryWriter.

#### 48.248.3.2 write_nested()

```
void gko::log::ProfilerHook::TableSummaryWriter::write_nested (
            const nested_summary_entry & root,
            std::chrono::nanoseconds overhead )  [override], [virtual]
```

Callback to write out the summary results.

**Parameters**

| | |
|---|---|
| *root* | the root range with runtime and count. |
| *overhead* | an estimate of the profiler overhead |

Implements gko::log::ProfilerHook::NestedSummaryWriter.

The documentation for this class was generated from the following file:

- ginkgo/core/log/profiler_hook.hpp

## 48.249 gko::stop::Time Class Reference

The Time class is a stopping criterion which stops the iteration process after a certain amount of time has passed.

```
#include <ginkgo/core/stop/time.hpp>
```

### Additional Inherited Members

### 48.249.1 Detailed Description

The Time class is a stopping criterion which stops the iteration process after a certain amount of time has passed.

The documentation for this class was generated from the following file:

- ginkgo/core/stop/time.hpp

## 48.250 gko::time_point Class Reference

An opaque wrapper for a time point generated by a timer.

`#include <ginkgo/core/base/timer.hpp>`

### 48.250.1 Detailed Description

An opaque wrapper for a time point generated by a timer.

The documentation for this class was generated from the following file:

- ginkgo/core/base/timer.hpp

## 48.251 gko::Timer Class Reference

Represents a generic timer that can be used to record time points and measure time differences on host or device streams.

`#include <ginkgo/core/base/timer.hpp>`

### Public Member Functions

- time_point create_time_point ()
  *Returns a newly created time point.*
- virtual void record (time_point &time)=0
  *Records a time point at the current time.*
- virtual void wait (time_point &time)=0
  *Waits until all kernels in-process when recording the time point are finished.*
- std::chrono::nanoseconds difference (time_point &start, time_point &stop)
  *Computes the difference between the two time points in nanoseconds.*
- virtual std::chrono::nanoseconds difference_async (const time_point &start, const time_point &stop)=0
  *Computes the difference between the two time points in nanoseconds.*

## Static Public Member Functions

- static std::unique_ptr< Timer > create_for_executor (std::shared_ptr< const Executor > exec)

  *Creates the timer type most suitable for recording accurate timings of kernels on the given executor.*

## 48.251.1 Detailed Description

Represents a generic timer that can be used to record time points and measure time differences on host or device streams.

To keep the runtime overhead of timing minimal, time points need to be allocated beforehand using Timer::create_time_point:

```
auto begin = timer->create_time_point();
auto end = timer->create_time_point();
// ...
timer->record(begin);
run_expensive_operation();
timer->record(end);
auto elapsed = timer->difference(begin, end);
```

## 48.251.2 Member Function Documentation

### 48.251.2.1 create_for_executor()

```
static std::unique_ptr<Timer> gko::Timer::create_for_executor (
            std::shared_ptr< const Executor > exec )  [static]
```

Creates the timer type most suitable for recording accurate timings of kernels on the given executor.

**Parameters**

| | |
|---|---|
| *exec* | the executor to create a Timer for |

**Returns**

CpuTimer for ReferenceExecutor and OmpExecutor, CudaTimer for CudaExecutor, HipTimer for HipExecutor or DpcppTimer for DpcppExecutor.

### 48.251.2.2 create_time_point()

```
time_point gko::Timer::create_time_point ( )
```

Returns a newly created time point.

Time points may only be used with the timer they were created with.

#### 48.251.2.3 difference()

```
std::chrono::nanoseconds gko::Timer::difference (
            time_point & start,
            time_point & stop )
```

Computes the difference between the two time points in nanoseconds.

The function synchronizes with `stop` before computing the difference.

**Parameters**

| | |
|---|---|
| *start* | the first time point (earlier) |
| *stop* | the second time point (later) |

**Returns**

the difference between the time points in nanoseconds.

#### 48.251.2.4 difference_async()

```
virtual std::chrono::nanoseconds gko::Timer::difference_async (
            const time_point & start,
            const time_point & stop )  [pure virtual]
```

Computes the difference between the two time points in nanoseconds.

This asynchronous version does not synchronize itself, so the time points need to have been synchronized with, i.e. `timer->wait(stop)` needs to have been called. The version is intended for more advanced users who want to measure the overhead of timing functionality separately.

**Parameters**

| | |
|---|---|
| *start* | the first time point (earlier) |
| *stop* | the second time point (later) |

**Returns**

the difference between the time points in nanoseconds.

Implemented in gko::DpcppTimer, gko::HipTimer, gko::CudaTimer, and gko::CpuTimer.

The documentation for this class was generated from the following file:

- ginkgo/core/base/timer.hpp

# 48.252 gko::Transposable Class Reference

Linear operators which support transposition should implement the Transposable interface.

`#include <ginkgo/core/base/lin_op.hpp>`

## Public Member Functions

- virtual std::unique_ptr< LinOp > transpose () const =0

  *Returns a LinOp representing the transpose of the Transposable object.*
- virtual std::unique_ptr< LinOp > conj_transpose () const =0

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*

## 48.252.1 Detailed Description

Linear operators which support transposition should implement the Transposable interface.

It provides two functionalities, the normal transpose and the conjugate transpose.

The normal transpose returns the transpose of the linear operator without changing any of its elements representing the operation, $B = A^T$.

The conjugate transpose returns the conjugate of each of the elements and additionally transposes the linear operator representing the operation, $B = A^H$.

### 48.252.1.1 Example: Transposing a Csr matrix:

```
 {c++}
//Transposing an object of LinOp type.
//The object you want to transpose.
auto op = matrix::Csr::create(exec);
//Transpose the object by first converting it to a transposable type.
auto trans = op->transpose();
```

## 48.252.2 Member Function Documentation

### 48.252.2.1 conj_transpose()

`virtual std::unique_ptr<LinOp> gko::Transposable::conj_transpose ( ) const  [pure virtual]`

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Fft3, gko::preconditioner::Jacobi< ValueType, IndexType >, gko::solver::UpperTrs< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, gko::preconditioner::Ic< ValueType, IndexType >, gko::matrix::Fft2, gko::matrix::Coo< ValueType, IndexType >, gko::solver::Ir< ValueType >, gko::solver::PipeCg< ValueType >, gko::solver::Gmres< ValueType >, gko::matrix::SparsityCsr< ValueType, IndexType >, gko::solver::Cgs< ValueType >, gko::solver::Idr< ValueType >, gko::solver::Minres< ValueType >, gko::solver::Chebyshev< ValueType >, gko::matrix::Diagonal< ValueType >, gko::solver::Fcg< ValueType >, gko::preconditioner::NullspaceProjection< ValueType >, gko::solver::Cg< ValueType >, gko::solver::Bicg< ValueType >, gko::solver::Gcr< ValueType >, gko::solver::LowerTrs< ValueType, gko::solver::Bicgstab< ValueType >, gko::Combination< ValueType >, gko::matrix::Fft, gko::Composition< ValueType >, gko::matrix::Identity< ValueType >, and gko::experimental::solver::Direct< ValueType, IndexType >.

#### 48.252.2.2 transpose()

```
virtual std::unique_ptr<LinOp> gko::Transposable::transpose ( ) const  [pure virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implemented in gko::matrix::Dense< ValueType >, gko::matrix::Dense< value_type >, gko::matrix::Csr< ValueType, IndexType >, gko::matrix::Fft3, gko::preconditioner::Jacobi< ValueType, IndexType >, gko::solver::UpperTrs< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, IndexType >, gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, gko::preconditioner::Ic< ValueType, IndexType >, gko::matrix::Fft2, gko::matrix::Coo< ValueType, IndexType >, gko::solver::Ir< ValueType >, gko::solver::PipeCg< ValueType >, gko::solver::Gmres< ValueType >, gko::matrix::SparsityCsr< ValueType, IndexType >, gko::solver::Cgs< ValueType >, gko::solver::Idr< ValueType >, gko::solver::Minres< ValueType >, gko::solver::Chebyshev< ValueType >, gko::matrix::Diagonal< ValueType >, gko::solver::Fcg< ValueType >, gko::preconditioner::NullspaceProjection< ValueType >, gko::solver::Cg< ValueType >, gko::solver::Bicg< ValueType >, gko::solver::Gcr< ValueType >, gko::solver::LowerTrs< ValueType, gko::solver::Bicgstab< ValueType >, gko::Combination< ValueType >, gko::matrix::Fft, gko::Composition< ValueType >, gko::matrix::Identity< ValueType >, and gko::experimental::solver::Direct< ValueType, IndexType >.

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

## 48.253 gko::config::type_descriptor Class Reference

This class describes the value and index types to be used when building a Ginkgo type from a configuration file.

```
#include <ginkgo/core/config/type_descriptor.hpp>
```

### Public Member Functions

- type_descriptor (std::string value_typestr="float64", std::string index_typestr="int32", std::string global_↩ index_typestr="int64")

  *type_descriptor constructor.*
- const std::string & get_value_typestr () const

  *Get the value type string.*
- const std::string & get_index_typestr () const

  *Get the index type string.*
- const std::string & get_local_index_typestr () const

  *Get the local index type string, which gives the same result as get_index_typestr()*
- const std::string & get_global_index_typestr () const

  *Get the global index type string.*

### 48.253.1 Detailed Description

This class describes the value and index types to be used when building a Ginkgo type from a configuration file.

A type_descriptor is passed in order to define the parse function defines which template parameters, in terms of value_type and/or index_type, the created object will have. For example, a CG solver created like this:

```
auto cg = parse(config, context, type_descriptor("float64", "int32"));
```

will have the value type `float64` and the index type `int32`. Any Ginkgo object that does not require one of these types will just ignore it. In value_type, one additional value `void` can be used to specify that no default type is provided. In this case, the configuration has to provide the necessary template types.

If the configuration specifies one field (only allow value_type now):

```
value_type:  "some_value_type"
```

this type will take precedence over the type_descriptor.

### 48.253.2 Constructor & Destructor Documentation

#### 48.253.2.1 type_descriptor()

```
gko::config::type_descriptor::type_descriptor (
            std::string value_typestr = "float64",
            std::string index_typestr = "int32",
            std::string global_index_typestr = "int64" )  [explicit]
```

type_descriptor constructor.

There is free function `make_type_descriptor` to create the object by template.

**Parameters**

| | |
|---|---|
| *value_typestr* | the value type string. "void" means no default. |
| *index_typestr* | the (local) index type string. |
| *global_index_typestr* | the global index type string. |

**Note**

there is no way to call the constructor with explicit template, so we create another free function to handle it.

The documentation for this class was generated from the following file:

- ginkgo/core/config/type_descriptor.hpp

## 48.254 gko::experimental::mpi::type_impl< T > Struct Template Reference

A struct that is used to determine the MPI_Datatype of a specified type.

`#include <ginkgo/core/base/mpi.hpp>`

### 48.254.1 Detailed Description

**template<typename T>**
**struct gko::experimental::mpi::type_impl< T >**

A struct that is used to determine the MPI_Datatype of a specified type.

**Template Parameters**

| | |
|---|---|
| *T* | type of which the MPI_Datatype should be inferred. |

**Note**

any specialization of this type hast to provide a static function `get_type()` that returns an MPI_Datatype

The documentation for this struct was generated from the following file:

- ginkgo/core/base/mpi.hpp

## 48.255 gko::syn::type_list< Types > Struct Template Reference

type_list records several types in template

`#include <ginkgo/core/synthesizer/containers.hpp>`

### 48.255.1 Detailed Description

**template<typename... Types>**
**struct gko::syn::type_list< Types >**

type_list records several types in template

**Template Parameters**

| | |
|---|---|
| *Types* | the types in the list |

The documentation for this struct was generated from the following file:

- ginkgo/core/synthesizer/containers.hpp

## 48.256 gko::multigrid::UniformCoarsening< ValueType, IndexType > Class Template Reference

UniformCoarsening is a simple coarse grid generation algorithm.

`#include <ginkgo/core/multigrid/uniform_coarsening.hpp>`

## Public Member Functions

- std::shared_ptr< const LinOp > get_system_matrix () const

  *Returns the system operator (matrix) of the linear system.*
- IndexType * get_coarse_rows () noexcept

  *Returns the selected coarse rows.*
- const IndexType * get_const_coarse_rows () const noexcept

  *Returns the selected coarse rows.*

## Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

### 48.256.1 Detailed Description

**template<typename ValueType = default_precision, typename IndexType = int32>**
**class gko::multigrid::UniformCoarsening< ValueType, IndexType >**

UniformCoarsening is a simple coarse grid generation algorithm.

It selects the coarse rows by a constant stride `coarse_skip` over the fine-row index space and builds the coarse system from them. The choice is purely index-based — neither matrix values nor mesh geometry are consulted. Fine row `i` either contributes only when it is itself a selected coarse row (injection-style), or is mapped to its coarse row `floor(i / coarse_skip)` (aggregation-style), depending on the `aggregation` parameter.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indexes |

### 48.256.2 Member Function Documentation

#### 48.256.2.1 get_coarse_rows()

```
template<typename ValueType = default_precision, typename IndexType = int32>
IndexType* gko::multigrid::UniformCoarsening< ValueType, IndexType >::get_coarse_rows ( )
[inline], [noexcept]
```

Returns the selected coarse rows.

**Returns**

the selected coarse rows.

```
81 { return coarse_rows_.get_data(); }
```

References gko::array< ValueType >::get_data().

### 48.256.2.2 get_const_coarse_rows()

```
template<typename ValueType = default_precision, typename IndexType = int32>
const IndexType* gko::multigrid::UniformCoarsening< ValueType, IndexType >::get_const_coarse↩
_rows ( ) const  [inline], [noexcept]
```

Returns the selected coarse rows.

#### Returns

the selected coarse rows.

#### Note

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

```
91     {
92         return coarse_rows_.get_const_data();
93     }
```

References gko::array< ValueType >::get_const_data().

### 48.256.2.3 get_system_matrix()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::shared_ptr<const LinOp> gko::multigrid::UniformCoarsening< ValueType, IndexType >::get↩
_system_matrix ( ) const  [inline]
```

Returns the system operator (matrix) of the linear system.

#### Returns

the system operator (matrix)

```
72     {
73         return system_matrix_;
74     }
```

### 48.256.2.4 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::multigrid::UniformCoarsening< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

The documentation for this class was generated from the following files:

- ginkgo/core/distributed/matrix.hpp
- ginkgo/core/multigrid/uniform_coarsening.hpp

## 48.257 gko::UnsupportedMatrixProperty Class Reference

Exception throws if a matrix does not have a property required by a numerical method.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- UnsupportedMatrixProperty (const std::string &file, const int line, const std::string &msg)
  *Initializes the UnsupportedMatrixProperty error.*

### 48.257.1 Detailed Description

Exception throws if a matrix does not have a property required by a numerical method.

Currently, a message is specified at the call-site manually.

### 48.257.2 Constructor & Destructor Documentation

#### 48.257.2.1 UnsupportedMatrixProperty()

```
gko::UnsupportedMatrixProperty::UnsupportedMatrixProperty (
            const std::string & file,
            const int line,
            const std::string & msg )  [inline]
```

Initializes the UnsupportedMatrixProperty error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *msg* | A message describing the property required. |

```
644         :  Error(file, line, msg)
645     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

# 48.258 gko::stop::Criterion::Updater Class Reference

The Updater class serves for convenient argument passing to the Criterion's check function.

`#include <ginkgo/core/stop/criterion.hpp>`

## Public Member Functions

- Updater (const Updater &)=delete

  *Prevent copying and moving the object This is to enforce the use of argument passing and calling check at the same time.*
- bool check (uint8 stopping_id, bool set_finalized, array< stopping_status > *stop_status, bool *one_changed) const

  *Calls the parent Criterion object's check method.*

## 48.258.1 Detailed Description

The Updater class serves for convenient argument passing to the Criterion's check function.

The pattern used is a Builder, except Updater builds a function's arguments before calling the function itself, and does not build an object. This allows calling a Criterion's check in the form

```
stop_criterion->update()
    .num_iterations(num_iterations)
    .ignore_residual_check(ignore_residual_check)
    .residual_norm(residual_norm)
    .implicit_sq_residual_norm(implicit_sq_residual_norm)
    .residual(residual)
    .solution(solution)
    .check(converged);
```

If there is a need for a new form of data to pass to the Criterion, it should be added here.

## 48.258.2 Member Function Documentation

#### 48.258.2.1 check()

```
bool gko::stop::Criterion::Updater::check (
            uint8 stopping_id,
            bool set_finalized,
            array< stopping_status > * stop_status,
            bool * one_changed ) const  [inline]
```

Calls the parent Criterion object's check method.

```
79         {
80             auto converged = parent_->check(stopping_id, set_finalized,
81                                             stop_status, one_changed, *this);
82             return converged;
83         }
```

References gko::stop::Criterion::check().

The documentation for this class was generated from the following file:

- ginkgo/core/stop/criterion.hpp

## 48.259 gko::solver::UpperTrs< ValueType, IndexType > Class Template Reference

UpperTrs is the triangular solver which solves the system U x = b, when U is an upper triangular matrix.

```
#include <ginkgo/core/solver/triangular.hpp>
```

### Public Member Functions

- std::unique_ptr< LinOp > transpose () const override

  *Returns a LinOp representing the transpose of the Transposable object.*
- std::unique_ptr< LinOp > conj_transpose () const override

  *Returns a LinOp representing the conjugate transpose of the Transposable object.*
- UpperTrs (const UpperTrs &)

  *Copy-assigns a triangular solver.*
- UpperTrs (UpperTrs &&)

  *Move-assigns a triangular solver.*
- UpperTrs & operator= (const UpperTrs &)

  *Copy-constructs a triangular solver.*
- UpperTrs & operator= (UpperTrs &&)

  *Move-constructs a triangular solver.*

### Static Public Member Functions

- static parameters_type parse (const config::pnode &config, const config::registry &context, const config::type_descriptor &td_for_child=config::make_type_descriptor< ValueType, IndexType >())

  *Create the parameters from the property_tree.*

## 48.259.1 Detailed Description

template<typename ValueType = default_precision, typename IndexType = int32>
class gko::solver::UpperTrs< ValueType, IndexType >

UpperTrs is the triangular solver which solves the system U x = b, when U is an upper triangular matrix.

It works best when passing in a matrix in CSR format. If the matrix is not in CSR, then the generate step converts it into a CSR matrix. The generation fails if the matrix is not convertible to CSR.

**Note**

As the constructor uses the copy and convert functionality, it is not possible to create a empty solver or a solver with a matrix in any other format other than CSR, if none of the executor modules are being compiled with.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of matrix elements |
| *IndexType* | precision of matrix indices |

## 48.259.2 Constructor & Destructor Documentation

### 48.259.2.1 UpperTrs() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::solver::UpperTrs< ValueType, IndexType >::UpperTrs (
            const UpperTrs< ValueType, IndexType > &  )
```

Copy-assigns a triangular solver.

Preserves the executor, shallow-copies the system matrix. If the executors mismatch, clones system matrix onto this executor. Solver analysis information will be regenerated.

### 48.259.2.2 UpperTrs() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
gko::solver::UpperTrs< ValueType, IndexType >::UpperTrs (
            UpperTrs< ValueType, IndexType > &&  )
```

Move-assigns a triangular solver.

Preserves the executor, moves the system matrix. If the executors mismatch, clones system matrix onto this executor and regenerates solver analysis information. Moved-from object is empty (0x0 and nullptr system matrix)

## 48.259.3 Member Function Documentation

#### 48.259.3.1 conj_transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::solver::UpperTrs< ValueType, IndexType >::conj_transpose ( )
const  [override], [virtual]
```

Returns a LinOp representing the conjugate transpose of the Transposable object.

**Returns**

a pointer to the new conjugate transposed object

Implements gko::Transposable.

#### 48.259.3.2 operator=() [1/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
UpperTrs& gko::solver::UpperTrs< ValueType, IndexType >::operator= (
            const UpperTrs< ValueType, IndexType > &  )
```

Copy-constructs a triangular solver.

Preserves the executor, shallow-copies the system matrix. Solver analysis information will be regenerated.

#### 48.259.3.3 operator=() [2/2]

```
template<typename ValueType = default_precision, typename IndexType = int32>
UpperTrs& gko::solver::UpperTrs< ValueType, IndexType >::operator= (
            UpperTrs< ValueType, IndexType > &&  )
```

Move-constructs a triangular solver.

Preserves the executor, moves the system matrix and solver analysis information. Moved-from object is empty (0x0 and nullptr system matrix)

#### 48.259.3.4 parse()

```
template<typename ValueType = default_precision, typename IndexType = int32>
static parameters_type gko::solver::UpperTrs< ValueType, IndexType >::parse (
            const config::pnode & config,
            const config::registry & context,
            const config::type_descriptor & td_for_child = config::make_type_descriptor< ValueType, IndexType
)  [static]
```

Create the parameters from the property_tree.

Because this is directly tied to the specific type, the value/index type settings within config are ignored and type_↩ descriptor is only used for children configs.

**Parameters**

| | |
|---|---|
| *config* | the property tree for setting |
| *context* | the registry |
| *td_for_child* | the type descriptor for children configs. The default uses the value/index type of this class. |

**Returns**

parameters

#### 48.259.3.5 transpose()

```
template<typename ValueType = default_precision, typename IndexType = int32>
std::unique_ptr<LinOp> gko::solver::UpperTrs< ValueType, IndexType >::transpose ( ) const
[override], [virtual]
```

Returns a LinOp representing the transpose of the Transposable object.

**Returns**

a pointer to the new transposed object

Implements gko::Transposable.

The documentation for this class was generated from the following file:

- ginkgo/core/solver/triangular.hpp

## 48.260 gko::UseComposition< ValueType > Class Template Reference

The UseComposition class can be used to store the composition information in LinOp.

```
#include <ginkgo/core/base/composition.hpp>
```

### Public Member Functions

- std::shared_ptr< Composition< ValueType > > get_composition () const
  *Returns the composition operators.*
- std::shared_ptr< const LinOp > get_operator_at (size_type index) const
  *Returns the operator at index-th position of composition.*

### 48.260.1 Detailed Description

**template<typename ValueType = default_precision>**
**class gko::UseComposition< ValueType >**

The UseComposition class can be used to store the composition information in LinOp.

**Template Parameters**

| | |
|---|---|
| *ValueType* | precision of input and result vectors |

## 48.260.2 Member Function Documentation

### 48.260.2.1 get_composition()

```
template<typename ValueType = default_precision>
std::shared_ptr<Composition<ValueType> > gko::UseComposition< ValueType >::get_composition (
) const  [inline]
```

Returns the composition operators.

**Returns**

composition

```
187     {
188         return composition_;
189     }
```

### 48.260.2.2 get_operator_at()

```
template<typename ValueType = default_precision>
std::shared_ptr<const LinOp> gko::UseComposition< ValueType >::get_operator_at (
             size_type index ) const  [inline]
```

Returns the operator at index-th position of composition.

**Returns**

index-th operator

**Note**

when this composition is not set, this function always returns nullptr. However, when this composition is set, it will throw exception when exceeding index.

**Exceptions**

| | |
|---|---|
| *std::out_of_range* | if index is out of bound when composition is existed. |

```
204     {
205         if (composition_ == nullptr) {
206             return nullptr;
```

```
207         } else {
208             return composition_->get_operators().at(index);
209         }
210     }
```

Referenced by gko::multigrid::EnableMultigridLevel< ValueType >::get_coarse_op(), gko::multigrid::EnableMultigridLevel< ValueType >::get_prolong_op(), and gko::multigrid::EnableMultigridLevel< ValueType >::get_restrict_op().

The documentation for this class was generated from the following file:

- ginkgo/core/base/composition.hpp

## 48.261 gko::syn::value_list< T, Values > Struct Template Reference

value_list records several values with the same type in template.

`#include <ginkgo/core/synthesizer/containers.hpp>`

### 48.261.1 Detailed Description

**template<typename T, T... Values>**
**struct gko::syn::value_list< T, Values >**

value_list records several values with the same type in template.

**Template Parameters**

| | |
|---|---|
| *T* | the value type of the list |
| *Values* | the values in the list |

The documentation for this struct was generated from the following file:

- ginkgo/core/synthesizer/containers.hpp

## 48.262 gko::ValueMismatch Class Reference

ValueMismatch is thrown if two values are not equal.

`#include <ginkgo/core/base/exception.hpp>`

### Public Member Functions

- ValueMismatch (const std::string &file, int line, const std::string &func, size_type val1, size_type val2, const std::string &clarification)

  *Initializes a value mismatch error.*

### 48.262.1 Detailed Description

ValueMismatch is thrown if two values are not equal.

### 48.262.2 Constructor & Destructor Documentation

#### 48.262.2.1 ValueMismatch()

```
gko::ValueMismatch::ValueMismatch (
            const std::string & file,
            int line,
            const std::string & func,
            size_type val1,
            size_type val2,
            const std::string & clarification )  [inline]
```

Initializes a value mismatch error.

**Parameters**

| | |
|---|---|
| *file* | The name of the offending source file |
| *line* | The source code line number where the error occurred |
| *func* | The function name where the error occurred |
| *val1* | The first value to be compared. |
| *val2* | The second value to be compared. |
| *clarification* | An additional message further describing the error |

```
517         :  Error(file, line,
518                  func + ":  Value mismatch :  " + std::to_string(val1) + " and " +
519                      std::to_string(val2) + " :  " + clarification)
520     {}
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/exception.hpp

## 48.263 gko::experimental::distributed::Vector< ValueType > Class Template Reference

Vector is a format which explicitly stores (multiple) distributed column vectors in a dense storage format.

```
#include <ginkgo/core/distributed/vector.hpp>
```

## Public Member Functions

- void read_distributed (const device_matrix_data< ValueType, int64 > &data, ptr_param< const Partition< int64, int64 >> partition)

  *Reads a vector from the device_matrix_data structure and a global row partition.*
- void read_distributed (const matrix_data< ValueType, int64 > &data, ptr_param< const Partition< int64, int64 >> partition)

  *Reads a vector from the matrix_data structure and a global row partition.*
- std::unique_ptr< absolute_type > compute_absolute () const override

  *Gets the AbsoluteLinOp.*
- void compute_absolute_inplace () override

  *Compute absolute inplace on each element.*
- std::unique_ptr< complex_type > make_complex () const

  *Creates a complex copy of the original vectors.*
- void make_complex (ptr_param< complex_type > result) const

  *Writes a complex copy of the original vectors to given complex vectors.*
- std::unique_ptr< real_type > get_real () const

  *Creates new real vectors and extracts the real part of the original vectors into that.*
- void get_real (ptr_param< real_type > result) const

  *Extracts the real part of the original vectors into given real vectors.*
- std::unique_ptr< real_type > get_imag () const

  *Creates new real vectors and extracts the imaginary part of the original vectors into that.*
- void get_imag (ptr_param< real_type > result) const

  *Extracts the imaginary part of the original vectors into given real vectors.*
- void fill (ValueType value)

  *Fill the distributed vectors with a given value.*
- void scale (ptr_param< const LinOp > alpha)

  *Scales the vectors with a scalar (aka: BLAS scal).*
- void inv_scale (ptr_param< const LinOp > alpha)

  *Scales the vectors with the inverse of a scalar.*
- void add_scaled (ptr_param< const LinOp > alpha, ptr_param< const LinOp > b)

  *Adds `b` scaled by `alpha` to the vectors (aka: BLAS axpy).*
- void sub_scaled (ptr_param< const LinOp > alpha, ptr_param< const LinOp > b)

  *Subtracts `b` scaled by `alpha` from the vectors (aka: BLAS axpy).*
- void compute_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result) const

  *Computes the column-wise dot product of this (multi-)vector and `b` using a global reduction.*
- void compute_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise dot product of this (multi-)vector and `b` using a global reduction.*
- void compute_conj_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result) const

  *Computes the column-wise dot product of this (multi-)vector and `conj(b)` using a global reduction.*
- void compute_conj_dot (ptr_param< const LinOp > b, ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise dot product of this (multi-)vector and `conj(b)` using a global reduction.*
- void compute_squared_norm2 (ptr_param< LinOp > result) const

  *Computes the square of the column-wise Euclidean ( $L^2$) norm of this (multi-)vector using a global reduction.*
- void compute_squared_norm2 (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the square of the column-wise Euclidean ( $L^2$) norm of this (multi-)vector using a global reduction.*
- void compute_norm2 (ptr_param< LinOp > result) const

  *Computes the Euclidean (L^2) norm of this (multi-)vector using a global reduction.*
- void compute_norm2 (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the Euclidean (L^2) norm of this (multi-)vector using a global reduction.*

- void compute_norm1 (ptr_param< LinOp > result) const

  *Computes the column-wise (L^1) norm of this (multi-)vector.*
- void compute_norm1 (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise (L^1) norm of this (multi-)vector using a global reduction.*
- void compute_mean (ptr_param< LinOp > result) const

  *Computes the column-wise mean of this (multi-)vector using a global reduction.*
- void compute_mean (ptr_param< LinOp > result, array< char > &tmp) const

  *Computes the column-wise arithmetic mean of this (multi-)vector using a global reduction.*
- value_type & at_local (size_type row, size_type col) noexcept

  *Returns a single element of the multi-vector.*
- value_type at_local (size_type row, size_type col) const noexcept
- ValueType & at_local (size_type idx) noexcept

  *Returns a single element of the multi-vector.*
- ValueType at_local (size_type idx) const noexcept
- value_type * get_local_values ()

  *Returns a pointer to the array of local values of the multi-vector.*
- const value_type * get_const_local_values () const

  *Returns a pointer to the array of local values of the multi-vector.*
- const local_vector_type * get_local_vector () const

  *Direct (read) access to the underlying local local_vector_type vectors.*
- std::unique_ptr< const real_type > create_real_view () const

  *Create a real view of the (potentially) complex original multi-vector.*
- std::unique_ptr< real_type > create_real_view ()

  *Create a real view of the (potentially) complex original multi-vector.*
- std::unique_ptr< Vector > create_submatrix (local_span rows, local_span columns, dim< 2 > global_size)

  *Creates a view of a submatrix of this vector.*

## Static Public Member Functions

- static std::unique_ptr< Vector > create_with_config_of (ptr_param< const Vector > other)

  *Creates a distributed Vector with the same size and stride as another Vector.*
- static std::unique_ptr< Vector > create_with_type_of (ptr_param< const Vector > other, std::shared_ptr< const Executor > exec)

  *Creates an empty Vector with the same type as another Vector, but on a different executor.*
- static std::unique_ptr< Vector > create_with_type_of (ptr_param< const Vector > other, std::shared_ptr< const Executor > exec, const dim< 2 > &global_size, const dim< 2 > &local_size, size_type stride)

  *Creates an Vector with the same type as another Vector, but on a different executor and with a different size.*
- static std::unique_ptr< Vector > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, dim< 2 > global_size, dim< 2 > local_size, size_type stride)

  *Creates an empty distributed vector with a specified size.*
- static std::unique_ptr< Vector > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, dim< 2 > global_size={}, dim< 2 > local_size={})

  *Creates an empty distributed vector with a specified size.*
- static std::unique_ptr< Vector > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, dim< 2 > global_size, std::unique_ptr< local_vector_type > local_vector)

  *Creates a distributed vector from local vectors with a specified size.*
- static std::unique_ptr< Vector > create (std::shared_ptr< const Executor > exec, mpi::communicator comm, std::unique_ptr< local_vector_type > local_vector)

  *Creates a distributed vector from local vectors.*

- static std::unique_ptr< const Vector > create_const (std::shared_ptr< const Executor > exec, mpi::communicator comm, dim< 2 > global_size, std::unique_ptr< const local_vector_type > local_↩ vector)

  *Creates a constant (immutable) distributed Vector from a constant local vector.*
- static std::unique_ptr< const Vector > create_const (std::shared_ptr< const Executor > exec, mpi::communicator comm, std::unique_ptr< const local_vector_type > local_vector)

  *Creates a constant (immutable) distributed Vector from a constant local vector.*

## 48.263.1 Detailed Description

**template<typename ValueType = double>**
**class gko::experimental::distributed::Vector< ValueType >**

Vector is a format which explicitly stores (multiple) distributed column vectors in a dense storage format.

The (multi-)vector is distributed by row, which is described by a Partition. The local vectors are stored using the matrix::Dense format. The vector should be filled using the read_distributed method, e.g.

```
auto part = Partition<...>::build_from_mapping(...);
auto vector = Vector<...>::create(exec, comm);
vector->read_distributed(matrix_data, part);
```

Using this approach the size of the global vectors, as well as the size of the local vectors, will be automatically inferred. It is possible to create a vector with specified global and local sizes and fill the local vectors using the accessor get_local_vector.

**Note**

Operations between two vectors (axpy, dot product, etc.) are only valid if both vectors where created using the same partition.

**Template Parameters**

| | |
|---|---|
| *ValueType* | The precision of vector elements. |

## 48.263.2 Member Function Documentation

### 48.263.2.1 add_scaled()

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::add_scaled (
             ptr_param< const LinOp > alpha,
             ptr_param< const LinOp > b )
```

Adds `b` scaled by `alpha` to the vectors (aka: BLAS axpy).

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, the all vectors of b are scaled by alpha. If it is a Dense row vector of values, then i-th column vector of b is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of vectors). |
| *b* | a (multi-)vector of the same dimension as this |

#### 48.263.2.2 at_local() [1/4]

```
template<typename ValueType = double>
ValueType gko::experimental::distributed::Vector< ValueType >::at_local (
            size_type idx ) const  [noexcept]
```

#### 48.263.2.3 at_local() [2/4]

```
template<typename ValueType = double>
ValueType& gko::experimental::distributed::Vector< ValueType >::at_local (
            size_type idx )  [noexcept]
```

Returns a single element of the multi-vector.

Useful for iterating across all elements of the multi-vector. However, it is less efficient than the two-parameter variant of this method.

**Parameters**

| | |
|---|---|
| *idx* | a linear index of the requested element (ignoring the stride) |

**Note**

the method has to be called on the same Executor the matrix is stored at (e.g. trying to call this method on a GPU matrix from the OMP results in a runtime error)

#### 48.263.2.4 at_local() [3/4]

```
template<typename ValueType = double>
value_type gko::experimental::distributed::Vector< ValueType >::at_local (
            size_type row,
            size_type col ) const  [noexcept]
```

### 48.263.2.5 at_local() [4/4]

```
template<typename ValueType = double>
value_type& gko::experimental::distributed::Vector< ValueType >::at_local (
            size_type row,
            size_type col ) [noexcept]
```

Returns a single element of the multi-vector.

**Parameters**

| | |
|---|---|
| *row* | the local row of the requested element |
| *col* | the local column of the requested element |

**Note**

the method has to be called on the same Executor the multi-vector is stored at (e.g. trying to call this method on a GPU multi-vector from the OMP results in a runtime error)

### 48.263.2.6 compute_absolute()

```
template<typename ValueType = double>
std::unique_ptr<absolute_type> gko::experimental::distributed::Vector< ValueType >::compute↩
_absolute ( ) const [override], [virtual]
```

Gets the AbsoluteLinOp.

**Returns**

a pointer to the new absolute object

Implements gko::EnableAbsoluteComputation< remove_complex< Vector< ValueType > > >.

### 48.263.2.7 compute_conj_dot() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_conj_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result ) const
```

Computes the column-wise dot product of this (multi-)vector and `conj(b)` using a global reduction.

**Parameters**

| | |
|---|---|
| *b* | a (multi-)vector of same dimension as this |
| *result* | a Dense row matrix, used to store the dot product (the number of column in result must match the number of columns of this) |

#### 48.263.2.8 compute_conj_dot() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_conj_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise dot product of this (multi-)vector and `conj(b)` using a global reduction.

**Parameters**

| | |
|---|---|
| *b* | a (multi-)vector of same dimension as this |
| *result* | a Dense row matrix, used to store the dot product (the number of column in result must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.263.2.9 compute_dot() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result ) const
```

Computes the column-wise dot product of this (multi-)vector and `b` using a global reduction.

**Parameters**

| | |
|---|---|
| *b* | a (multi-)vector of same dimension as this |
| *result* | a Dense row matrix, used to store the dot product (the number of column in result must match the number of columns of this) |

#### 48.263.2.10 compute_dot() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_dot (
            ptr_param< const LinOp > b,
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise dot product of this (multi-)vector and `b` using a global reduction.

**Parameters**

| | |
|---|---|
| *b* | a (multi-)vector of same dimension as this |
| *result* | a Dense row matrix, used to store the dot product (the number of column in result must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.263.2.11 compute_mean() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_mean (
            ptr_param< LinOp > result ) const
```

Computes the column-wise mean of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row matrix, used to store the mean (the number of columns in result must match the number of columns of this) |

#### 48.263.2.12 compute_mean() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_mean (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise arithmetic mean of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row matrix, used to store the mean (the number of columns in result must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.263.2.13 compute_norm1() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_norm1 (
            ptr_param< LinOp > result ) const
```

Computes the column-wise (L^1) norm of this (multi-)vector.

**Parameters**

| | |
|---|---|
| *result* | a Dense row matrix, used to store the norm (the number of columns in result must match the number of columns of this) |

#### 48.263.2.14 compute_norm1() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_norm1 (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the column-wise (L^1) norm of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row matrix, used to store the norm (the number of columns in result must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.263.2.15 compute_norm2() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_norm2 (
            ptr_param< LinOp > result ) const
```

Computes the Euclidean (L^2) norm of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row matrix, used to store the norm (the number of columns in result must match the number of columns of this) |

#### 48.263.2.16 compute_norm2() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_norm2 (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the Euclidean (L^2) norm of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row matrix, used to store the norm (the number of columns in result must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.263.2.17 compute_squared_norm2() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_squared_norm2 (
            ptr_param< LinOp > result ) const
```

Computes the square of the column-wise Euclidean ( $L^2$) norm of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |

#### 48.263.2.18 compute_squared_norm2() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::compute_squared_norm2 (
            ptr_param< LinOp > result,
            array< char > & tmp ) const
```

Computes the square of the column-wise Euclidean ( $L^2$) norm of this (multi-)vector using a global reduction.

**Parameters**

| | |
|---|---|
| *result* | a Dense row vector, used to store the norm (the number of columns in the vector must match the number of columns of this) |
| *tmp* | the temporary storage to use for partial sums during the reduction computation. It may be resized and/or reset to the correct executor. |

#### 48.263.2.19 create() [1/4]

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
```

```
            dim< 2 > global_size,
            dim< 2 > local_size,
            size_type stride )  [static]
```

Creates an empty distributed vector with a specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with vector |
| *comm* | Communicator associated with vector |
| *global_size* | Global size of the vector |
| *local_size* | Processor-local size of the vector |
| *stride* | Stride of the local vector. |

**Returns**

A smart pointer to the newly created vector.

### 48.263.2.20 create() [2/4]

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            dim< 2 > global_size,
            std::unique_ptr< local_vector_type > local_vector )  [static]
```

Creates a distributed vector from local vectors with a specified size.

**Note**

The data form the local_vector will be moved into the new distributed vector. You could either move in a std↩ ::unique_ptr directly, copy a local vector with gko::clone, or create a unique non-owning view of a given local vector with gko::make_dense_view.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this vector |
| *comm* | Communicator associated with this vector |
| *global_size* | The global size of the vector |
| *local_vector* | The underlying local vector, the data will be moved into this |

**Returns**

A smart pointer to the newly created vector.

### 48.263.2.21 create() [3/4]

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            dim< 2 > global_size = {},
            dim< 2 > local_size = {} )  [static]
```

Creates an empty distributed vector with a specified size.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with vector |
| *comm* | Communicator associated with vector |
| *global_size* | Global size of the vector |
| *local_size* | Processor-local size of the vector, uses local_size[1] as the stride |

**Returns**

A smart pointer to the newly created vector.

### 48.263.2.22 create() [4/4]

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            std::unique_ptr< local_vector_type > local_vector )  [static]
```

Creates a distributed vector from local vectors.

The global size will be deduced from the local sizes, which will incur a collective communication.

**Note**

The data form the local_vector will be moved into the new distributed vector. You could either move in a std↩ ::unique_ptr directly, copy a local vector with gko::clone, or create a unique non-owning view of a given local vector with gko::make_dense_view.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this vector |
| *comm* | Communicator associated with this vector |
| *local_vector* | The underlying local vector, the data will be moved into this. |

**Returns**

A smart pointer to the newly created vector.

### 48.263.2.23 create_const() [1/2]

```
template<typename ValueType = double>
static std::unique_ptr<const Vector> gko::experimental::distributed::Vector< ValueType >↩
::create_const (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            dim< 2 > global_size,
            std::unique_ptr< const local_vector_type > local_vector )  [static]
```

Creates a constant (immutable) distributed Vector from a constant local vector.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this vector |
| *comm* | Communicator associated with this vector |
| *global_size* | The global size of the vector |
| *local_vector* | The underlying local vector, of which a view is created |

**Returns**

A smart pointer to the newly created vector.

### 48.263.2.24 create_const() [2/2]

```
template<typename ValueType = double>
static std::unique_ptr<const Vector> gko::experimental::distributed::Vector< ValueType >↩
::create_const (
            std::shared_ptr< const Executor > exec,
            mpi::communicator comm,
            std::unique_ptr< const local_vector_type > local_vector )  [static]
```

Creates a constant (immutable) distributed Vector from a constant local vector.

The global size will be deduced from the local sizes, which will incur a collective communication.

**Parameters**

| | |
|---|---|
| *exec* | Executor associated with this vector |
| *comm* | Communicator associated with this vector |
| *local_vector* | The underlying local vector, of which a view is created |

**Returns**

A smart pointer to the newly created vector.

### 48.263.2.25 create_real_view() [1/2]

```
template<typename ValueType = double>
std::unique_ptr<real_type> gko::experimental::distributed::Vector< ValueType >::create_real↩
_view ( )
```

Create a real view of the (potentially) complex original multi-vector.

If the original vector is real, nothing changes. If the original vector is complex, the result is created by viewing the complex vector with as real with a reinterpret_cast with twice the number of columns and double the stride.

### 48.263.2.26 create_real_view() [2/2]

```
template<typename ValueType = double>
std::unique_ptr<const real_type> gko::experimental::distributed::Vector< ValueType >::create↩
_real_view ( ) const
```

Create a real view of the (potentially) complex original multi-vector.

If the original vector is real, nothing changes. If the original vector is complex, the result is created by viewing the complex vector with as real with a reinterpret_cast with twice the number of columns and double the stride.

### 48.263.2.27 create_submatrix()

```
template<typename ValueType = double>
std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create_submatrix
(
            local_span rows,
            local_span columns,
            dim< 2 > global_size )
```

Creates a view of a submatrix of this vector.

**Parameters**

| | |
|---|---|
| *rows* | The local rows of the submatrix |
| *columns* | The local columns of the submatrix |
| *global_size* | The global size of the submatrix |

**Returns**

A view of a submatrix.

#### 48.263.2.28 create_with_config_of()

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create_↩
with_config_of (
            ptr_param< const Vector< ValueType > > other )  [static]
```

Creates a distributed Vector with the same size and stride as another Vector.

**Parameters**

| | |
|---|---|
| *other* | The other vector whose configuration needs to copied. |

#### 48.263.2.29 create_with_type_of() [1/2]

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create_↩
with_type_of (
            ptr_param< const Vector< ValueType > > other,
            std::shared_ptr< const Executor > exec )  [static]
```

Creates an empty Vector with the same type as another Vector, but on a different executor.

**Parameters**

| | |
|---|---|
| *other* | The other multi-vector whose type we target. |
| *exec* | The executor of the new multi-vector. |

**Note**

The new multi-vector uses the same communicator as other.

**Returns**

an empty Vector with the type of other.

#### 48.263.2.30 create_with_type_of() [2/2]

```
template<typename ValueType = double>
static std::unique_ptr<Vector> gko::experimental::distributed::Vector< ValueType >::create_↩
with_type_of (
            ptr_param< const Vector< ValueType > > other,
            std::shared_ptr< const Executor > exec,
            const dim< 2 > & global_size,
            const dim< 2 > & local_size,
            size_type stride )  [static]
```

Creates an Vector with the same type as another Vector, but on a different executor and with a different size.

**Parameters**

| | |
|---|---|
| *other* | The other multi-vector whose type we target. |
| *exec* | The executor of the new multi-vector. |
| *global_size* | The global size of the multi-vector. |
| *local_size* | The local size of the multi-vector. |
| *stride* | The stride of the new multi-vector. |

**Returns**

a Vector of specified size with the type of other.

### 48.263.2.31 fill()

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::fill (
            ValueType value )
```

Fill the distributed vectors with a given value.

**Parameters**

| | |
|---|---|
| *value* | the value to be filled |

### 48.263.2.32 get_const_local_values()

```
template<typename ValueType = double>
const value_type* gko::experimental::distributed::Vector< ValueType >::get_const_local_values
( ) const
```

Returns a pointer to the array of local values of the multi-vector.

**Returns**

the pointer to the array of local values

**Note**

This is the constant version of the function, which can be significantly more memory efficient than the non-constant version, so always prefer this version.

#### 48.263.2.33 get_local_values()

```
template<typename ValueType = double>
value_type* gko::experimental::distributed::Vector< ValueType >::get_local_values ( )
```

Returns a pointer to the array of local values of the multi-vector.

**Returns**

the pointer to the array of local values

#### 48.263.2.34 get_local_vector()

```
template<typename ValueType = double>
const local_vector_type* gko::experimental::distributed::Vector< ValueType >::get_local_vector
( ) const
```

Direct (read) access to the underlying local local_vector_type vectors.

**Returns**

a constant pointer to the underlying local_vector_type vectors

#### 48.263.2.35 inv_scale()

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::inv_scale (
            ptr_param< const LinOp > alpha )
```

Scales the vectors with the inverse of a scalar.

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, the all vectors are scaled by 1 / alpha. If it is a Dense row vector of values, then i-th column vector is scaled with the inverse of the i-th element of alpha (the number of columns of alpha has to match the number of vectors). |

#### 48.263.2.36 make_complex() [1/2]

```
template<typename ValueType = double>
std::unique_ptr<complex_type> gko::experimental::distributed::Vector< ValueType >::make_↩
complex ( ) const
```

Creates a complex copy of the original vectors.

If the original vectors were real, the imaginary part of the result will be zero.

#### 48.263.2.37 make_complex() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::make_complex (
            ptr_param< complex_type > result ) const
```

Writes a complex copy of the original vectors to given complex vectors.

If the original vectors were real, the imaginary part of the result will be zero.

#### 48.263.2.38 read_distributed() [1/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::read_distributed (
            const device_matrix_data< ValueType, int64 > & data,
            ptr_param< const Partition< int64, int64 >> partition )
```

Reads a vector from the device_matrix_data structure and a global row partition.

The number of rows of the matrix data is ignored, only its number of columns is relevant. Both the number of local and global rows are inferred from the row partition.

**Note**

The matrix data can contain entries for rows other than those owned by the process. Entries for those rows are discarded.

**Parameters**

| | |
|---|---|
| *data* | The device_matrix_data structure |
| *partition* | The global row partition |

#### 48.263.2.39 read_distributed() [2/2]

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::read_distributed (
            const matrix_data< ValueType, int64 > & data,
            ptr_param< const Partition< int64, int64 >> partition )
```

Reads a vector from the matrix_data structure and a global row partition.

See read_distributed().

**Note**

For efficiency it is advised to use the device_matrix_data overload.

#### 48.263.2.40 scale()

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::scale (
            ptr_param< const LinOp > alpha )
```

Scales the vectors with a scalar (aka: BLAS scal).

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrx, the all vectors are scaled by alpha. If it is a Dense row vector of values, then i-th column vector is scaled with the i-th element of alpha (the number of columns of alpha has to match the number of vectors). |

#### 48.263.2.41 sub_scaled()

```
template<typename ValueType = double>
void gko::experimental::distributed::Vector< ValueType >::sub_scaled (
            ptr_param< const LinOp > alpha,
            ptr_param< const LinOp > b )
```

Subtracts `b` scaled by `alpha` from the vectors (aka: BLAS axpy).

**Parameters**

| | |
|---|---|
| *alpha* | If alpha is 1x1 Dense matrix, the all vectors of b are scaled by alpha. If it is a Dense row vector of values, then i-th column vector of b is scaled with the i-th element of alpha (the number of c |
| *b* | a (multi-)vector of the same dimension as this |

The documentation for this class was generated from the following files:

- ginkgo/core/distributed/matrix.hpp
- ginkgo/core/distributed/vector.hpp

## 48.264 gko::version Struct Reference

This structure is used to represent versions of various Ginkgo modules.

`#include <ginkgo/core/base/version.hpp>`

### Public Attributes

- const uint64 major
  *The major version number.*
- const uint64 minor
  *The minor version number.*
- const uint64 patch
  *The patch version number.*
- const char *const tag
  *Addition tag string that describes the version in more detail.*

### 48.264.1 Detailed Description

This structure is used to represent versions of various Ginkgo modules.

Version structures can be compared using the usual relational operators.

### 48.264.2 Member Data Documentation

#### 48.264.2.1 tag

```
const char* const gko::version::tag
```

Addition tag string that describes the version in more detail.

It does not participate in comparisons.

Referenced by gko::operator<<().

The documentation for this struct was generated from the following file:

- ginkgo/core/base/version.hpp

## 48.265 gko::version_info Class Reference

Ginkgo uses version numbers to label new features and to communicate backward compatibility guarantees:

```
#include <ginkgo/core/base/version.hpp>
```

### Static Public Member Functions

- static const version_info & get ()
  *Returns an instance of version_info.*

### Public Attributes

- version header_version
  *Contains version information of the header files.*
- version core_version
  *Contains version information of the core library.*
- version reference_version
  *Contains version information of the reference module.*
- version omp_version
  *Contains version information of the OMP module.*
- version cuda_version
  *Contains version information of the CUDA module.*
- version hip_version
  *Contains version information of the HIP module.*
- version dpcpp_version
  *Contains version information of the DPC++ module.*

### 48.265.1 Detailed Description

Ginkgo uses version numbers to label new features and to communicate backward compatibility guarantees:

1. Versions with different major version number have incompatible interfaces (parts of the earlier interface may not be present anymore, and new interfaces can appear).
2. Versions with the same major number X, but different minor numbers Y1 and Y2 numbers keep the same interface as version X.0.0, but additions to the interface in X.0.0 present in X.Y1.0 may not be present in X.Y2.0 and vice versa.
3. Versions with the same major an minor version numbers, but different patch numbers have exactly the same interface, but the functionality may be different (something that is not implemented or has a bug in an earlier version may have this implemented or fixed in a later version).

This structure provides versions of different parts of Ginkgo: the headers, the core and the kernel modules (reference, OpenMP, CUDA, HIP, DPCPP). To obtain an instance of version_info filled with information about the current version of Ginkgo, call the version_info::get() static method.

### 48.265.2 Member Function Documentation

#### 48.265.2.1 get()

```
static const version_info& gko::version_info::get ( )  [inline], [static]
```

Returns an instance of version_info.

**Returns**

an instance of version info

```
139     {
140         static version_info info{};
141         return info;
142     }
```

### 48.265.3 Member Data Documentation

#### 48.265.3.1 core_version

```
version gko::version_info::core_version
```

Contains version information of the core library.

This is the version of the static/shared library called "ginkgo".

#### 48.265.3.2 cuda_version

```
version gko::version_info::cuda_version
```

Contains version information of the CUDA module.

This is the version of the static/shared library called "ginkgo_cuda".

#### 48.265.3.3 dpcpp_version

```
version gko::version_info::dpcpp_version
```

Contains version information of the DPC++ module.

This is the version of the static/shared library called "ginkgo_dpcpp".

#### 48.265.3.4 hip_version

```
version gko::version_info::hip_version
```

Contains version information of the HIP module.

This is the version of the static/shared library called "ginkgo_hip".

#### 48.265.3.5 omp_version

```
version gko::version_info::omp_version
```

Contains version information of the OMP module.

This is the version of the static/shared library called "ginkgo_omp".

#### 48.265.3.6 reference_version

```
version gko::version_info::reference_version
```

Contains version information of the reference module.

This is the version of the static/shared library called "ginkgo_reference".

The documentation for this class was generated from the following file:

- ginkgo/core/base/version.hpp

## 48.266 gko::experimental::mpi::window< ValueType > Class Template Reference

This class wraps the MPI_Window class with RAII functionality.

```
#include <ginkgo/core/base/mpi.hpp>
```

## Public Types

- enum create_type

  *The create type for the window object.*
- enum lock_type

  *The lock type for passive target synchronization of the windows.*

## Public Member Functions

- window ()

  *The default constructor.*
- window (window &&other) noexcept

  *The move constructor.*
- window & operator= (window &&other) noexcept

  *The move assignment operator.*
- window (std::shared_ptr< const Executor > exec, ValueType *base, int num_elems, const communicator &comm, const int disp_unit=sizeof(ValueType), MPI_Info input_info=MPI_INFO_NULL, create_type c_↩ type=create_type::create)

  *Create a window object with a given data pointer and type.*
- MPI_Win get_window () const

  *Get the underlying window object of MPI_Win type.*
- void fence (int assert=0) const

  *The active target synchronization using MPI_Win_fence for the window object.*
- void lock (int rank, lock_type lock_t=lock_type::shared, int assert=0) const

  *Create an epoch using MPI_Win_lock for the window object.*
- void unlock (int rank) const

  *Close the epoch using MPI_Win_unlock for the window object.*
- void lock_all (int assert=0) const

  *Create the epoch on all ranks using MPI_Win_lock_all for the window object.*
- void unlock_all () const

  *Close the epoch on all ranks using MPI_Win_unlock_all for the window object.*
- void flush (int rank) const

  *Flush the existing RDMA operations on the target rank for the calling process for the window object.*
- void flush_local (int rank) const

  *Flush the existing RDMA operations on the calling rank from the target rank for the window object.*
- void flush_all () const

  *Flush all the existing RDMA operations for the calling process for the window object.*
- void flush_all_local () const

  *Flush all the local existing RDMA operations on the calling rank for the window object.*
- void sync () const

  *Synchronize the public and private buffers for the window object.*
- ~window ()

  *The deleter which calls MPI_Win_free when the window leaves its scope.*
- template<typename PutType >
  void put (std::shared_ptr< const Executor > exec, const PutType *origin_buffer, const int origin_count, const int target_rank, const unsigned int target_disp, const int target_count) const

  *Put data into the target window.*
- template<typename PutType >
  request r_put (std::shared_ptr< const Executor > exec, const PutType *origin_buffer, const int origin_count, const int target_rank, const unsigned int target_disp, const int target_count) const

  *Put data into the target window.*

- template<typename PutType >
  void accumulate (std::shared_ptr< const Executor > exec, const PutType *origin_buffer, const int origin_count, const int target_rank, const unsigned int target_disp, const int target_count, MPI_Op operation) const

  *Accumulate data into the target window.*
- template<typename PutType >
  request r_accumulate (std::shared_ptr< const Executor > exec, const PutType *origin_buffer, const int origin_count, const int target_rank, const unsigned int target_disp, const int target_count, MPI_Op operation) const

  *(Non-blocking) Accumulate data into the target window.*
- template<typename GetType >
  void get (std::shared_ptr< const Executor > exec, GetType *origin_buffer, const int origin_count, const int target_rank, const unsigned int target_disp, const int target_count) const

  *Get data from the target window.*
- template<typename GetType >
  request r_get (std::shared_ptr< const Executor > exec, GetType *origin_buffer, const int origin_count, const int target_rank, const unsigned int target_disp, const int target_count) const

  *Get data (with handle) from the target window.*
- template<typename GetType >
  void get_accumulate (std::shared_ptr< const Executor > exec, GetType *origin_buffer, const int origin_count, GetType *result_buffer, const int result_count, const int target_rank, const unsigned int target_disp, const int target_count, MPI_Op operation) const

  *Get Accumulate data from the target window.*
- template<typename GetType >
  request r_get_accumulate (std::shared_ptr< const Executor > exec, GetType *origin_buffer, const int origin_count, GetType *result_buffer, const int result_count, const int target_rank, const unsigned int target_disp, const int target_count, MPI_Op operation) const

  *(Non-blocking) Get Accumulate data (with handle) from the target window.*
- template<typename GetType >
  void fetch_and_op (std::shared_ptr< const Executor > exec, GetType *origin_buffer, GetType *result_buffer, const int target_rank, const unsigned int target_disp, MPI_Op operation) const

  *Fetch and operate on data from the target window (An optimized version of Get_accumulate).*

## 48.266.1 Detailed Description

**template<typename ValueType>**
**class gko::experimental::mpi::window< ValueType >**

This class wraps the MPI_Window class with RAII functionality.

Different create and lock type methods are setup with enums.

MPI_Window is primarily used for one sided communication and this class provides functionalities to fence, lock, unlock and flush the communication buffers.

## 48.266.2 Constructor & Destructor Documentation

#### 48.266.2.1 window() [1/3]

```
template<typename ValueType >
gko::experimental::mpi::window< ValueType >::window ( )  [inline]
```

The default constructor.

It creates a null window of MPI_WIN_NULL type.

```
1618 :  window_(MPI_WIN_NULL) {}
```

#### 48.266.2.2 window() [2/3]

```
template<typename ValueType >
gko::experimental::mpi::window< ValueType >::window (
            window< ValueType > && other )  [inline], [noexcept]
```

The move constructor.

Move the other object and replace it with MPI_WIN_NULL

**Parameters**

| | |
|---|---|
| *other* | the window object to be moved. |

```
1631         :  window_{std::exchange(other.window_, MPI_WIN_NULL)}
1632     {}
```

#### 48.266.2.3 window() [3/3]

```
template<typename ValueType >
gko::experimental::mpi::window< ValueType >::window (
            std::shared_ptr< const Executor > exec,
            ValueType * base,
            int num_elems,
            const communicator & comm,
            const int disp_unit = sizeof(ValueType),
            MPI_Info input_info = MPI_INFO_NULL,
            create_type c_type = create_type::create )  [inline]
```

Create a window object with a given data pointer and type.

A collective operation.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the base pointer is located. |
| *base* | the base pointer for the window object. |
| *num_elems* | the num_elems of type ValueType the window points to. |
| *comm* | the communicator whose ranks will have windows created. |
| *disp_unit* | the displacement from base for the window object. |
| *input_info* | the MPI_Info object used to set certain properties. |
| *c_type* | the type of creation method to use to create the window. |

```
1667     {
1668         auto guard = exec->get_scoped_device_id_guard();
1669         unsigned size = num_elems * sizeof(ValueType);
1670         if (c_type == create_type::create) {
1671             GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_create(
1672                 base, size, disp_unit, input_info, comm.get(), &this->window_));
1673         } else if (c_type == create_type::dynamic_create) {
1674             GKO_ASSERT_NO_MPI_ERRORS(
1675                 MPI_Win_create_dynamic(input_info, comm.get(), &this->window_));
1676         } else if (c_type == create_type::allocate) {
1677             GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_allocate(
1678                 size, disp_unit, input_info, comm.get(), base, &this->window_));
1679         } else {
1680             GKO_NOT_IMPLEMENTED;
1681         }
1682     }
```

References gko::experimental::mpi::communicator::get().

## 48.266.3 Member Function Documentation

### 48.266.3.1 accumulate()

```
template<typename ValueType >
template<typename PutType >
void gko::experimental::mpi::window< ValueType >::accumulate (
             std::shared_ptr< const Executor > exec,
             const PutType * origin_buffer,
             const int origin_count,
             const int target_rank,
             const unsigned int target_disp,
             const int target_count,
             MPI_Op operation ) const  [inline]
```

Accumulate data into the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to put |
| *target_rank* | the rank to put the data to |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |
| *operation* | the reduce operation. See @MPI_Op |

```
1875     {
1876         auto guard = exec->get_scoped_device_id_guard();
1877         GKO_ASSERT_NO_MPI_ERRORS(MPI_Accumulate(
1878             origin_buffer, origin_count, type_impl<PutType>::get_type(),
1879             target_rank, target_disp, target_count,
1880             type_impl<PutType>::get_type(), operation, this->get_window()));
1881     }
```

References gko::experimental::mpi::window< ValueType >::get_window().

### 48.266.3.2 fence()

```
template<typename ValueType >
void gko::experimental::mpi::window< ValueType >::fence (
            int assert = 0 ) const  [inline]
```

The active target synchronization using MPI_Win_fence for the window object.

This is called on all associated ranks.

**Parameters**

| | |
|---|---|
| *assert* | the optimization level. 0 is always valid. |

```
1698     {
1699         GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_fence(assert, this->window_));
1700     }
```

### 48.266.3.3 fetch_and_op()

```
template<typename ValueType >
template<typename GetType >
void gko::experimental::mpi::window< ValueType >::fetch_and_op (
             std::shared_ptr< const Executor > exec,
             GetType * origin_buffer,
             GetType * result_buffer,
             const int target_rank,
             const unsigned int target_disp,
             MPI_Op operation ) const  [inline]
```

Fetch and operate on data from the target window (An optimized version of Get_accumulate).

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *result_buffer* | the buffer to store the previous target value in |
| *target_rank* | the rank to get the data from |
| *target_disp* | the displacement at the target window |
| *operation* | the reduce operation. See @MPI_Op |

```
2038     {
2039         auto guard = exec->get_scoped_device_id_guard();
2040         GKO_ASSERT_NO_MPI_ERRORS(MPI_Fetch_and_op(
2041             origin_buffer, result_buffer, type_impl<GetType>::get_type(),
2042             target_rank, target_disp, operation, this->get_window()));
2043     }
```

References gko::experimental::mpi::window< ValueType >::get_window().

### 48.266.3.4 flush()

```
template<typename ValueType >
```

```
void gko::experimental::mpi::window< ValueType >::flush (
            int rank ) const  [inline]
```

Flush the existing RDMA operations on the target rank for the calling process for the window object.

**Parameters**

| | |
|---|---|
| *rank* | the target rank. |

```
1762     {
1763         GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_flush(rank, this->window_));
1764     }
```

### 48.266.3.5 flush_local()

```
template<typename ValueType >
void gko::experimental::mpi::window< ValueType >::flush_local (
            int rank ) const  [inline]
```

Flush the existing RDMA operations on the calling rank from the target rank for the window object.

**Parameters**

| | |
|---|---|
| *rank* | the target rank. |

```
1773     {
1774         GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_flush_local(rank, this->window_));
1775     }
```

### 48.266.3.6 get()

```
template<typename ValueType >
template<typename GetType >
void gko::experimental::mpi::window< ValueType >::get (
            std::shared_ptr< const Executor > exec,
            GetType * origin_buffer,
            const int origin_count,
            const int target_rank,
            const unsigned int target_disp,
            const int target_count ) const  [inline]
```

Get data from the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to get |
| *target_rank* | the rank to get the data from |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |

```
1926     {
1927         auto guard = exec->get_scoped_device_id_guard();
1928         GKO_ASSERT_NO_MPI_ERRORS(
1929             MPI_Get(origin_buffer, origin_count, type_impl<GetType>::get_type(),
1930                     target_rank, target_disp, target_count,
1931                     type_impl<GetType>::get_type(), this->get_window()));
1932     }
```

References gko::experimental::mpi::window< ValueType >::get_window().

#### 48.266.3.7 get_accumulate()

```
template<typename ValueType >
template<typename GetType >
void gko::experimental::mpi::window< ValueType >::get_accumulate (
             std::shared_ptr< const Executor > exec,
             GetType * origin_buffer,
             const int origin_count,
             GetType * result_buffer,
             const int result_count,
             const int target_rank,
             const unsigned int target_disp,
             const int target_count,
             MPI_Op operation ) const  [inline]
```

Get Accumulate data from the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to get |
| *result_buffer* | the buffer to receive the target data |
| *result_count* | the number of elements to get |
| *target_rank* | the rank to get the data from |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |
| *operation* | the reduce operation. See @MPI_Op |

```
1979     {
1980         auto guard = exec->get_scoped_device_id_guard();
1981         GKO_ASSERT_NO_MPI_ERRORS(MPI_Get_accumulate(
1982             origin_buffer, origin_count, type_impl<GetType>::get_type(),
1983             result_buffer, result_count, type_impl<GetType>::get_type(),
1984             target_rank, target_disp, target_count,
1985             type_impl<GetType>::get_type(), operation, this->get_window()));
1986     }
```

References gko::experimental::mpi::window< ValueType >::get_window().

#### 48.266.3.8 get_window()

```
template<typename ValueType >
MPI_Win gko::experimental::mpi::window< ValueType >::get_window ( ) const  [inline]
```

Get the underlying window object of MPI_Win type.

**Returns**

the underlying window object.

```
1689 { return this->window_; }
```

Referenced by gko::experimental::mpi::window< ValueType >::accumulate(), gko::experimental::mpi::window< ValueType >::fetch_and_op(), gko::experimental::mpi::window< ValueType >::get(), gko::experimental::mpi::window< ValueType >::get_accumulate(), gko::experimental::mpi::window< ValueType >::put(), gko::experimental::mpi::window< ValueType >::r_accumulate(), gko::experimental::mpi::window< ValueType >::r_get(), gko::experimental::mpi::window< ValueType >::r_get_accumulate(), and gko::experimental::mpi::window< ValueType >::r_put().

### 48.266.3.9 lock()

```
template<typename ValueType >
void gko::experimental::mpi::window< ValueType >::lock (
            int rank,
            lock_type lock_t = lock_type::shared,
            int assert = 0 ) const  [inline]
```

Create an epoch using MPI_Win_lock for the window object.

**Parameters**

| | |
|---|---|
| *rank* | the target rank. |
| *lock_t* | the type of the lock: shared or exclusive |
| *assert* | the optimization level. 0 is always valid. |

```
1712     {
1713         if (lock_t == lock_type::shared) {
1714             GKO_ASSERT_NO_MPI_ERRORS(
1715                 MPI_Win_lock(MPI_LOCK_SHARED, rank, assert, this->window_));
1716         } else if (lock_t == lock_type::exclusive) {
1717             GKO_ASSERT_NO_MPI_ERRORS(
1718                 MPI_Win_lock(MPI_LOCK_EXCLUSIVE, rank, assert, this->window_));
1719         } else {
1720             GKO_NOT_IMPLEMENTED;
1721         }
1722     }
```

### 48.266.3.10 lock_all()

```
template<typename ValueType >
void gko::experimental::mpi::window< ValueType >::lock_all (
            int assert = 0 ) const  [inline]
```

Create the epoch on all ranks using MPI_Win_lock_all for the window object.

**Parameters**

| | |
|---|---|
| *assert* | the optimization level. 0 is always valid. |

```
1742     {
1743         GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_lock_all(assert, this->window_));
1744     }
```

#### 48.266.3.11 operator=()

```
template<typename ValueType >
window& gko::experimental::mpi::window< ValueType >::operator= (
             window< ValueType > && other )  [inline], [noexcept]
```

The move assignment operator.

Frees the current window, then moves the other object and replaces it with MPI_WIN_NULL

**Parameters**

| | |
|---|---|
| *other* | the window object to be moved. |

```
1641     {
1642         if (this != &other) {
1643             if (this->window_ && this->window_ != MPI_WIN_NULL) {
1644                 MPI_Win_free(&this->window_);
1645             }
1646             window_ = std::exchange(other.window_, MPI_WIN_NULL);
1647         }
1648         return *this;
1649     }
```

#### 48.266.3.12 put()

```
template<typename ValueType >
template<typename PutType >
void gko::experimental::mpi::window< ValueType >::put (
              std::shared_ptr< const Executor > exec,
              const PutType * origin_buffer,
              const int origin_count,
              const int target_rank,
              const unsigned int target_disp,
              const int target_count ) const  [inline]
```

Put data into the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to put |
| *target_rank* | the rank to put the data to |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |

```
1824     {
1825         auto guard = exec->get_scoped_device_id_guard();
```

```
1826         GKO_ASSERT_NO_MPI_ERRORS(
1827             MPI_Put(origin_buffer, origin_count, type_impl<PutType>::get_type(),
1828                     target_rank, target_disp, target_count,
1829                     type_impl<PutType>::get_type(), this->get_window()));
1830     }
```

References gko::experimental::mpi::window< ValueType >::get_window().

#### 48.266.3.13 r_accumulate()

```
template<typename ValueType >
template<typename PutType >
request gko::experimental::mpi::window< ValueType >::r_accumulate (
             std::shared_ptr< const Executor > exec,
             const PutType * origin_buffer,
             const int origin_count,
             const int target_rank,
             const unsigned int target_disp,
             const int target_count,
             MPI_Op operation ) const  [inline]
```

(Non-blocking) Accumulate data into the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to put |
| *target_rank* | the rank to put the data to |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |
| *operation* | the reduce operation. See @MPI_Op |

**Returns**

the request handle for the send call

```
1901     {
1902         auto guard = exec->get_scoped_device_id_guard();
1903         request req;
1904         GKO_ASSERT_NO_MPI_ERRORS(MPI_Raccumulate(
1905             origin_buffer, origin_count, type_impl<PutType>::get_type(),
1906             target_rank, target_disp, target_count,
1907             type_impl<PutType>::get_type(), operation, this->get_window(),
1908             req.get()));
1909         return req;
1910     }
```

References gko::experimental::mpi::request::get(), and gko::experimental::mpi::window< ValueType >::get_window().

#### 48.266.3.14 r_get()

```
template<typename ValueType >
template<typename GetType >
```

```
request gko::experimental::mpi::window< ValueType >::r_get (
            std::shared_ptr< const Executor > exec,
            GetType * origin_buffer,
            const int origin_count,
            const int target_rank,
            const unsigned int target_disp,
            const int target_count ) const  [inline]
```

Get data (with handle) from the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to get |
| *target_rank* | the rank to get the data from |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |

**Returns**

the request handle for the send call

```
1950     {
1951         auto guard = exec->get_scoped_device_id_guard();
1952         request req;
1953         GKO_ASSERT_NO_MPI_ERRORS(MPI_Rget(
1954             origin_buffer, origin_count, type_impl<GetType>::get_type(),
1955             target_rank, target_disp, target_count,
1956             type_impl<GetType>::get_type(), this->get_window(), req.get()));
1957         return req;
1958     }
```

References gko::experimental::mpi::request::get(), and gko::experimental::mpi::window< ValueType >::get_window().

### 48.266.3.15 r_get_accumulate()

```
template<typename ValueType >
template<typename GetType >
request gko::experimental::mpi::window< ValueType >::r_get_accumulate (
            std::shared_ptr< const Executor > exec,
            GetType * origin_buffer,
            const int origin_count,
            GetType * result_buffer,
            const int result_count,
            const int target_rank,
            const unsigned int target_disp,
            const int target_count,
            MPI_Op operation ) const  [inline]
```

(Non-blocking) Get Accumulate data (with handle) from the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffers are located. |

**Parameters**

| | |
|---|---|
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to get |
| *result_buffer* | the buffer to receive the target data |
| *result_count* | the number of elements to get |
| *target_rank* | the rank to get the data from |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |
| *operation* | the reduce operation. See @MPI_Op |

**Returns**

the request handle for the send call

```
2010     {
2011         auto guard = exec->get_scoped_device_id_guard();
2012         request req;
2013         GKO_ASSERT_NO_MPI_ERRORS(MPI_Rget_accumulate(
2014             origin_buffer, origin_count, type_impl<GetType>::get_type(),
2015             result_buffer, result_count, type_impl<GetType>::get_type(),
2016             target_rank, target_disp, target_count,
2017             type_impl<GetType>::get_type(), operation, this->get_window(),
2018             req.get()));
2019         return req;
2020     }
```

References gko::experimental::mpi::request::get(), and gko::experimental::mpi::window< ValueType >::get_↩ window().

### 48.266.3.16 r_put()

```
template<typename ValueType >
template<typename PutType >
request gko::experimental::mpi::window< ValueType >::r_put (
             std::shared_ptr< const Executor > exec,
             const PutType * origin_buffer,
             const int origin_count,
             const int target_rank,
             const unsigned int target_disp,
             const int target_count ) const  [inline]
```

Put data into the target window.

**Parameters**

| | |
|---|---|
| *exec* | The executor, on which the message buffer is located. |
| *origin_buffer* | the buffer to send |
| *origin_count* | the number of elements to put |
| *target_rank* | the rank to put the data to |
| *target_disp* | the displacement at the target window |
| *target_count* | the request handle for the send call |

**Returns**

the request handle for the send call

```
1849     {
1850         auto guard = exec->get_scoped_device_id_guard();
1851         request req;
1852         GKO_ASSERT_NO_MPI_ERRORS(MPI_Rput(
1853             origin_buffer, origin_count, type_impl<PutType>::get_type(),
1854             target_rank, target_disp, target_count,
1855             type_impl<PutType>::get_type(), this->get_window(), req.get()));
1856         return req;
1857     }
```

References gko::experimental::mpi::request::get(), and gko::experimental::mpi::window< ValueType >::get_window().

#### 48.266.3.17 unlock()

```
template<typename ValueType >
void gko::experimental::mpi::window< ValueType >::unlock (
             int rank ) const  [inline]
```

Close the epoch using MPI_Win_unlock for the window object.

**Parameters**

| | |
|---|---|
| *rank* | the target rank. |

```
1731     {
1732         GKO_ASSERT_NO_MPI_ERRORS(MPI_Win_unlock(rank, this->window_));
1733     }
```

The documentation for this class was generated from the following file:

- ginkgo/core/base/mpi.hpp

## 48.267 gko::solver::workspace_traits< Solver > Struct Template Reference

Traits class providing information on the type and location of workspace vectors inside a solver.

```
#include <ginkgo/core/solver/solver_base.hpp>
```

### 48.267.1 Detailed Description

**template<typename Solver>**
**struct gko::solver::workspace_traits< Solver >**

Traits class providing information on the type and location of workspace vectors inside a solver.

The documentation for this struct was generated from the following file:

- ginkgo/core/solver/solver_base.hpp

# 48.268 gko::WritableToMatrixData< ValueType, IndexType > Class Template Reference

A LinOp implementing this interface can write its data to a matrix_data structure.

`#include <ginkgo/core/base/lin_op.hpp>`

## Public Member Functions

- virtual void write (matrix_data< ValueType, IndexType > &data) const =0
  *Writes a matrix to a matrix_data structure.*
- virtual void write (device_matrix_data< ValueType, IndexType > &data) const
  *Writes a matrix to a device_matrix_data structure.*

## 48.268.1 Detailed Description

**template<typename ValueType, typename IndexType>**
**class gko::WritableToMatrixData< ValueType, IndexType >**

A LinOp implementing this interface can write its data to a matrix_data structure.

## 48.268.2 Member Function Documentation

### 48.268.2.1 write() [1/2]

```
template<typename ValueType, typename IndexType>
virtual void gko::WritableToMatrixData< ValueType, IndexType >::write (
            device_matrix_data< ValueType, IndexType > & data ) const  [inline], [virtual]
```

Writes a matrix to a device_matrix_data structure.

The entries are written on the executor of the output data. The default implementation goes through the host; formats that can produce the entries on the executor should override this.

**Parameters**

| | |
|---|---|
| *data* | the device_matrix_data structure |

Reimplemented in gko::matrix::Csr< ValueType, IndexType >.

```
743     {
744         matrix_data<ValueType, IndexType> host_data;
745         this->write(host_data);
746         data = device_matrix_data<ValueType, IndexType>::create_from_host(
747             data.get_executor(), host_data);
748     }
```

#### 48.268.2.2 write() [2/2]

```
template<typename ValueType, typename IndexType>
virtual void gko::WritableToMatrixData< ValueType, IndexType >::write (
            matrix_data< ValueType, IndexType > & data ) const  [pure virtual]
```

Writes a matrix to a matrix_data structure.

**Parameters**

| | |
|---|---|
| *data* | the matrix_data structure |

Implemented in gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, gko::matrix::Fft3, gko::matrix::Fft2, gko::matrix::Fft, gko::matrix::Fft3, gko::matrix::Fft2, gko::matrix::Fft, gko::matrix::Fft3, gko::matrix::Fft2, gko::matrix::Fft, gko::matrix::Fft3, gko::matrix::Fft2, gko::matrix::Fft, gko::matrix::Hybrid< ValueType, IndexType >, gko::matrix::Csr< ValueType, IndexType >, gko::preconditioner::Jacobi< ValueType, IndexType >, gko::matrix::Fbcsr< ValueType, I gko::matrix::Coo< ValueType, IndexType >, gko::matrix::Ell< ValueType, IndexType >, gko::matrix::Sellp< ValueType, IndexType >, gko::matrix::SparsityCsr< ValueType, IndexType >, gko::matrix::Permutation< IndexType >, and gko::matrix::ScaledPermutation< V

The documentation for this class was generated from the following file:

- ginkgo/core/base/lin_op.hpp

# Index


abs
    gko, 374
absolute_implicit_residual_norm
    gko::stop, 450
absolute_residual_norm
    gko::stop, 450
accumulate
    gko::experimental::mpi::window< ValueType >, 1247
add_logger
    gko::Executor, 800
    gko::log::Loggable, 955
add_scale_reuse
    gko::matrix::Csr< ValueType, IndexType >, 599
add_scaled
    gko::batch::MultiVector< ValueType >, 1011
    gko::experimental::distributed::Vector< ValueType >, 1225
    gko::matrix::Dense< ValueType >, 674
add_scaled_identity
    gko::batch::matrix::Csr< ValueType, IndexType >, 625
    gko::batch::matrix::Dense< ValueType >, 657
    gko::batch::matrix::Ell< ValueType, IndexType >, 753
    gko::ScaledIdentityAddable, 1145
add_value
    gko::matrix_assembly_data< ValueType, IndexType >, 979
all_gather
    gko::experimental::mpi::communicator, 539
all_reduce
    gko::experimental::mpi::communicator, 540, 541
all_to_all
    gko::experimental::mpi::communicator, 541, 542
all_to_all_v
    gko::experimental::mpi::communicator, 543
alloc
    gko::Executor, 800
allocation_mode
    gko, 373
AllocationError
    gko::AllocationError, 462
apply
    gko::batch::matrix::Csr< ValueType, IndexType >, 625, 626
    gko::batch::matrix::Dense< ValueType >, 658, 659
    gko::batch::matrix::Ell< ValueType, IndexType >, 753–755
    gko::batch::matrix::Identity< ValueType >, 887, 888
apply2
    gko::matrix::Coo< ValueType, IndexType >, 580, 581
apply_async
    gko::experimental::distributed::RowGatherer< LocalIndexType >, 1141
apply_uses_initial_guess
    gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType >, 1153
    gko::solver::Bicg< ValueType >, 497
    gko::solver::Bicgstab< ValueType >, 500
    gko::solver::Cg< ValueType >, 515
    gko::solver::Cgs< ValueType >, 517
    gko::solver::Chebyshev< ValueType >, 521
    gko::solver::Fcg< ValueType >, 825
    gko::solver::Gcr< ValueType >, 842
    gko::solver::Gmres< ValueType >, 846
    gko::solver::Idr< ValueType >, 894
    gko::solver::Ir< ValueType >, 929
    gko::solver::Multigrid, 1003
    gko::solver::PipeCg< ValueType >, 1085
array
    gko, 374
    gko::array< ValueType >, 468, 470–473
as
    gko, 375–377, 379
as_array
    gko::syn, 456
as_const_view
    gko::array< ValueType >, 473
as_list
    gko::syn, 456
as_view
    gko::array< ValueType >, 474
assemble_rows_from_neighbors
    gko::experimental::distributed, 424
assembly_mode
    gko::experimental::distributed, 423
at
    gko::batch::matrix::Dense< ValueType >, 659–661
    gko::batch::MultiVector< ValueType >, 1011–1013
    gko::matrix::Dense< ValueType >, 675, 676
at_local
    gko::experimental::distributed::Vector< ValueType >, 1226
autodetect

gko::precision_reduction, 1098
automatic
gko, 374

BadDimension
gko::BadDimension, 486
batch_dim
gko::batch_dim< Dimensionality, DimensionType >, 488
Batched Linear Operators, 315
EnableDefaultBatchLinOpFactory, 318
GKO_ENABLE_BATCH_LIN_OP_FACTORY, 316
block_at
gko::BlockOperator, 507
BlockOperator
gko::BlockOperator, 507
BlockSizeError
gko::BlockSizeError< IndexType >, 510
broadcast
gko::experimental::mpi::communicator, 544
build_from_contiguous
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1059
build_from_global_size_uniform
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1059
build_from_mapping
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1060
build_partition_from_local_range
gko::experimental::distributed, 424
build_partition_from_local_size
gko::experimental::distributed, 425
build_smoother
gko::solver, 445, 446

ceildiv
gko, 380
Chebyshev
gko::solver::Chebyshev< ValueType >, 520
check
gko::stop::Criterion, 592
gko::stop::Criterion::Updater, 1215
check_reuse_consistent
gko::LinOpFactory, 951
clear
gko::array< ValueType >, 474
gko::index_set< IndexType >, 918
clone
gko, 381
gko::EnableCloneable< ConcreteType >, 779, 780
col_at
gko::matrix::Ell< ValueType, IndexType >, 765, 766
gko::matrix::Sellp< ValueType, IndexType >, 1165
col_scale
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 966
column_limit
gko::matrix::Hybrid< ValueType, IndexType >::column_limit, 529
column_permute
gko::matrix::Csr< ValueType, IndexType >, 600
gko::matrix::Dense< ValueType >, 676–678
gko::Permutable< IndexType >, 1067
columns
gko::matrix, 438
Combination
gko::Combination< ValueType >, 531, 532
combine
Stopping criteria, 356
combined
gko::experimental::distributed, 423
comm_index_type
gko::experimental::mpi, 431
common
gko::precision_reduction, 1098
communicator
gko::experimental::mpi::communicator, 538, 539
compose
gko::matrix::Permutation< IndexType >, 1072
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1147
Composition
gko::Composition< ValueType >, 568
compute_absolute
gko::EnableAbsoluteComputation< AbsoluteLinOp >, 777
gko::experimental::distributed::Vector< ValueType >, 1227
gko::matrix::Coo< ValueType, IndexType >, 581
gko::matrix::Csr< ValueType, IndexType >, 600
gko::matrix::Dense< ValueType >, 678
gko::matrix::Diagonal< ValueType >, 726
gko::matrix::Ell< ValueType, IndexType >, 766
gko::matrix::Fbcsr< ValueType, IndexType >, 814
gko::matrix::Hybrid< ValueType, IndexType >, 866
gko::matrix::Sellp< ValueType, IndexType >, 1166
compute_absolute_linop
gko::AbsoluteComputable, 459
gko::EnableAbsoluteComputation< AbsoluteLinOp >, 777
compute_conj_dot
gko::batch::MultiVector< ValueType >, 1014
gko::experimental::distributed::Vector< ValueType >, 1227, 1228
gko::matrix::Dense< ValueType >, 679
compute_dot
gko::batch::MultiVector< ValueType >, 1014
gko::experimental::distributed::Vector< ValueType >, 1228
gko::matrix::Dense< ValueType >, 679, 680
compute_ell_num_stored_elements_per_row
gko::matrix::Hybrid< ValueType, IndexType >::automatic, 485
gko::matrix::Hybrid< ValueType, IndexType >::column_limit, 530

gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit, 905
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit, 907
gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit, 997
gko::matrix::Hybrid< ValueType, IndexType >::strategy_type, 1197
compute_hybrid_config
gko::matrix::Hybrid< ValueType, IndexType >::strategy_type, 1198
compute_inverse
gko::matrix::Permutation< IndexType >, 1072
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1147
compute_mean
gko::experimental::distributed::Vector< ValueType >, 1229
gko::matrix::Dense< ValueType >, 680, 681
compute_norm1
gko::experimental::distributed::Vector< ValueType >, 1229, 1230
gko::matrix::Dense< ValueType >, 681
compute_norm2
gko::batch::MultiVector< ValueType >, 1014
gko::experimental::distributed::Vector< ValueType >, 1230
gko::matrix::Dense< ValueType >, 682
compute_squared_norm2
gko::experimental::distributed::Vector< ValueType >, 1231
gko::matrix::Dense< ValueType >, 682
compute_storage_space
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 503
concatenate
gko::syn, 456
cond
gko::matrix_data< ValueType, IndexType >, 987, 988
conj
gko, 382
conj_transpose
gko::Combination< ValueType >, 532
gko::Composition< ValueType >, 569
gko::experimental::solver::Direct< ValueType, IndexType >, 740
gko::matrix::Coo< ValueType, IndexType >, 581
gko::matrix::Csr< ValueType, IndexType >, 600
gko::matrix::Dense< ValueType >, 683
gko::matrix::Diagonal< ValueType >, 726
gko::matrix::Fbcsr< ValueType, IndexType >, 814
gko::matrix::Fft, 828
gko::matrix::Fft2, 832
gko::matrix::Fft3, 836
gko::matrix::Identity< ValueType >, 890
gko::matrix::SparsityCsr< ValueType, IndexType >, 1183
gko::preconditioner::Ic< ValueType, IndexType >, 884
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 902
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 934
gko::preconditioner::Jacobi< ValueType, IndexType >, 944
gko::preconditioner::NullspaceProjection< ValueType >, 1038
gko::solver::Bicg< ValueType >, 497
gko::solver::Bicgstab< ValueType >, 501
gko::solver::Cg< ValueType >, 515
gko::solver::Cgs< ValueType >, 518
gko::solver::Chebyshev< ValueType >, 521
gko::solver::Fcg< ValueType >, 826
gko::solver::Gcr< ValueType >, 843
gko::solver::Gmres< ValueType >, 846
gko::solver::Idr< ValueType >, 894
gko::solver::Ir< ValueType >, 929
gko::solver::LowerTrs< ValueType, IndexType >, 957
gko::solver::Minres< ValueType >, 999
gko::solver::PipeCg< ValueType >, 1085
gko::solver::UpperTrs< ValueType, IndexType >, 1217
gko::Transposable, 1208
const_view
gko::array< ValueType >, 474
contains
gko::index_set< IndexType >, 918, 919
gko::matrix_assembly_data< ValueType, IndexType >, 980
contains_constant
gko::solver::Nullspace< ValueType >, 1033
contiguous_type
gko::experimental::mpi::contiguous_type, 571
converge
gko::stopping_status, 1193
convert_to
gko::ConvertibleTo< ResultType >, 577
gko::EnableCloneable< ConcreteType >, 780
gko::matrix::Fbcsr< ValueType, IndexType >, 815
gko::preconditioner::Jacobi< ValueType, IndexType >, 944
coordinate
gko, 374
copy
gko::Executor, 801
copy_and_convert_to
gko, 382–384
copy_from
gko::accessor::row_major< ValueType, Dimensionality >, 1134
gko::Executor, 801
copy_to_host
gko::array< ValueType >, 475
gko::device_matrix_data< ValueType, IndexType

>, 719
copy_val_to_host
gko::Executor, 802
core_version
gko::version_info, 1242
create
gko::batch::log::BatchConvergence< ValueType >, 491
gko::batch::matrix::Csr< ValueType, IndexType >, 626, 627
gko::batch::matrix::Dense< ValueType >, 661, 662
gko::batch::matrix::Ell< ValueType, IndexType >, 755, 756
gko::batch::matrix::Identity< ValueType >, 888
gko::batch::MultiVector< ValueType >, 1015
gko::BlockOperator, 508
gko::CudaExecutor, 640
gko::DpcppExecutor, 744
gko::EnableDefaultFactory< ConcreteFactory, ProductType, ParametersType, PolymorphicBase >, 783
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 966–971
gko::experimental::distributed::RowGatherer< LocalIndexType >, 1142
gko::experimental::distributed::Vector< ValueType >, 1231–1233
gko::HipExecutor, 854
gko::log::Convergence< ValueType >, 573, 574
gko::log::Papi< ValueType >, 1048
gko::log::PerformanceHint, 1066
gko::log::Record, 1120
gko::log::Stream< ValueType >, 1200
gko::matrix::Coo< ValueType, IndexType >, 581, 582
gko::matrix::Csr< ValueType, IndexType >, 601–603
gko::matrix::Dense< ValueType >, 683, 684
gko::matrix::Diagonal< ValueType >, 726, 727
gko::matrix::Ell< ValueType, IndexType >, 766, 767
gko::matrix::Fbcsr< ValueType, IndexType >, 815–817
gko::matrix::Fft, 828, 829
gko::matrix::Fft2, 832, 833
gko::matrix::Fft3, 836, 837
gko::matrix::Hybrid< ValueType, IndexType >, 866–868
gko::matrix::Identity< ValueType >, 890, 891
gko::matrix::IdentityFactory< ValueType >, 892
gko::matrix::Permutation< IndexType >, 1072, 1073
gko::matrix::RowGatherer< IndexType >, 1137, 1138
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1147, 1148
gko::matrix::Sellp< ValueType, IndexType >, 1166
gko::matrix::SparsityCsr< ValueType, IndexType >, 1183–1185
gko::Perturbation< ValueType >, 1078
gko::solver::Nullspace< ValueType >, 1033
create_const
gko::batch::matrix::Csr< ValueType, IndexType >, 628
gko::batch::matrix::Dense< ValueType >, 663
gko::batch::matrix::Ell< ValueType, IndexType >, 756
gko::batch::MultiVector< ValueType >, 1016
gko::experimental::distributed::Vector< ValueType >, 1234
gko::matrix::Coo< ValueType, IndexType >, 583
gko::matrix::Csr< ValueType, IndexType >, 603, 604
gko::matrix::Dense< ValueType >, 685
gko::matrix::Diagonal< ValueType >, 728
gko::matrix::Ell< ValueType, IndexType >, 768
gko::matrix::Fbcsr< ValueType, IndexType >, 817
gko::matrix::Permutation< IndexType >, 1073, 1074
gko::matrix::RowGatherer< IndexType >, 1138
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1149
gko::matrix::SparsityCsr< ValueType, IndexType >, 1185
create_const_view_for_item
gko::batch::matrix::Csr< ValueType, IndexType >, 628
gko::batch::matrix::Dense< ValueType >, 663
gko::batch::matrix::Ell< ValueType, IndexType >, 757
gko::batch::MultiVector< ValueType >, 1016
create_const_view_of
gko::matrix::Dense< ValueType >, 685
create_empty_reuse_data
gko::LinOpFactory, 952
create_for_executor
gko::Timer, 1206
create_from_combined_lu
gko::experimental::factorization::Factorization< ValueType, IndexType >, 810
create_from_composition
gko::experimental::factorization::Factorization< ValueType, IndexType >, 810
create_from_constant
gko::solver::Nullspace< ValueType >, 1034
create_from_host
gko::device_matrix_data< ValueType, IndexType >, 720
create_from_offsets
gko::segmented_array< T >, 1158, 1159
create_from_sizes
gko::segmented_array< T >, 1159
create_from_symm_composition
gko::experimental::factorization::Factorization< ValueType, IndexType >, 811
create_inverse

gko::experimental::mpi::CollectiveCommunicator, 526
gko::experimental::mpi::DenseCommunicator, 712
gko::experimental::mpi::NeighborhoodCommunicator, 1024
create_nested_summary
gko::log::ProfilerHook, 1102
create_nvtx
gko::log::ProfilerHook, 1103
create_owning
gko::experimental::mpi::communicator, 545
create_real_view
gko::experimental::distributed::Vector< ValueType >, 1235
gko::matrix::Dense< ValueType >, 686
create_scalar_csv_writer
gko::log::SolverProgress, 1175
create_scalar_table_writer
gko::log::SolverProgress, 1176
create_submatrix
gko::experimental::distributed::Vector< ValueType >, 1235
gko::matrix::Csr< ValueType, IndexType >, 604, 605
gko::matrix::Dense< ValueType >, 686, 687
create_summary
gko::log::ProfilerHook, 1103
create_tau
gko::log::ProfilerHook, 1103
create_time_point
gko::Timer, 1206
create_vector_storage
gko::log::SolverProgress, 1176
create_view_for_item
gko::batch::matrix::Csr< ValueType, IndexType >, 629
gko::batch::matrix::Dense< ValueType >, 664
gko::batch::matrix::Ell< ValueType, IndexType >, 757
gko::batch::MultiVector< ValueType >, 1017
create_view_of
gko::matrix::Dense< ValueType >, 687
create_with_config_of
gko::batch::MultiVector< ValueType >, 1017
gko::experimental::distributed::Vector< ValueType >, 1235
gko::matrix::Dense< ValueType >, 688
create_with_same_type
gko::experimental::mpi::CollectiveCommunicator, 526
gko::experimental::mpi::DenseCommunicator, 712
gko::experimental::mpi::NeighborhoodCommunicator, 1024
create_with_type_of
gko::experimental::distributed::Vector< ValueType >, 1236
gko::matrix::Dense< ValueType >, 688–690
criterion
gko::log, 436
Csr
gko::matrix::Csr< ValueType, IndexType >, 599
CublasError
gko::CublasError, 635
CUDA Executor, 319
cuda_stream
gko::cuda_stream, 636
cuda_version
gko::version_info, 1242
CudaError
gko::CudaError, 638
CufftError
gko::CufftError, 650
CurandError
gko::CurandError, 650
CusparseError
gko::CusparseError, 651
cycle
gko::solver::multigrid, 447

deferred_factory_parameter
gko::deferred_factory_parameter< FactoryType >, 655
Dense
gko::matrix::Dense< ValueType >, 674
DenseCommunicator
gko::experimental::mpi::DenseCommunicator, 711
device_matrix_data
gko::device_matrix_data< ValueType, IndexType >, 718, 719
diag
gko::matrix_data< ValueType, IndexType >, 989–992
difference
gko::Timer, 1206
difference_async
gko::CpuTimer, 590
gko::CudaTimer, 647
gko::DpcppTimer, 751
gko::HipTimer, 861
gko::Timer, 1207
dim
gko::dim< Dimensionality, DimensionType >, 733, 735
DimensionMismatch
gko::DimensionMismatch, 739
Distributed, 320
DPC++ Executor, 321
dpcpp_version
gko::version_info, 1243

Ell
gko::matrix::Ell< ValueType, IndexType >, 765
ell_col_at
gko::matrix::Hybrid< ValueType, IndexType >, 869
ell_val_at
gko::matrix::Hybrid< ValueType, IndexType >, 869, 870

emplace
gko::config::registry, 1126
empty_out
gko::device_matrix_data< ValueType, IndexType >, 720
EnableDefaultBatchLinOpFactory
Batched Linear Operators, 318
EnableDefaultCriterionFactory
gko::stop, 449
EnableDefaultLinOpFactory
Linear Operators, 338
EnableDefaultReorderingBaseFactory
gko::reorder, 442
EnableIterativeBase
gko::solver::EnableIterativeBase< DerivedType >, 785
EnablePreconditionable
gko::solver::EnablePreconditionable< DerivedType >, 790
EnableSolverBase
gko::solver::EnableSolverBase< DerivedType, MatrixType >, 794
environment
gko::experimental::mpi::environment, 795
equals
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1060
Error
gko::Error, 797
executor_deleter
gko::executor_deleter< T >, 808
Executors, 322
GKO_REGISTER_HOST_OPERATION, 323
GKO_REGISTER_OPERATION, 324
extract_diagonal
gko::DiagonalExtractable< ValueType >, 731
gko::matrix::Coo< ValueType, IndexType >, 583
gko::matrix::Csr< ValueType, IndexType >, 605
gko::matrix::Dense< ValueType >, 690
gko::matrix::Ell< ValueType, IndexType >, 769
gko::matrix::Fbcsr< ValueType, IndexType >, 818
gko::matrix::Hybrid< ValueType, IndexType >, 870
gko::matrix::Sellp< ValueType, IndexType >, 1167
extract_diagonal_linop
gko::DiagonalExtractable< ValueType >, 731
gko::DiagonalLinOpExtractable, 732

Factorizations, 326
factory
gko::log, 436
Fbcsr
gko::matrix::Fbcsr< ValueType, IndexType >, 814
fence
gko::experimental::mpi::window< ValueType >, 1247
fetch_and_op
gko::experimental::mpi::window< ValueType >, 1248
fill
gko::array< ValueType >, 475
gko::batch::MultiVector< ValueType >, 1018
gko::experimental::distributed::Vector< ValueType >, 1237
gko::matrix::Dense< ValueType >, 691
flush
gko::experimental::mpi::window< ValueType >, 1248
flush_local
gko::experimental::mpi::window< ValueType >, 1249
free
gko::Executor, 803

gather
gko::experimental::mpi::communicator, 545
gather_v
gko::experimental::mpi::communicator, 546
generate
gko::AbstractFactory< AbstractProductType, ComponentsType >, 461
gko::experimental::factorization::Cholesky< ValueType, IndexType >, 523
gko::experimental::factorization::Lu< ValueType, IndexType >, 960
gko::experimental::reorder::Amd< IndexType >, 464
gko::experimental::reorder::Mc64< ValueType, IndexType >, 994
gko::experimental::reorder::Rcm< IndexType >, 1116
gko::preconditioner::GaussSeidel< ValueType, IndexType >, 840
gko::preconditioner::Sor< ValueType, IndexType >, 1178
generate_reuse
gko::LinOpFactory, 952
get
gko::config::pnode, 1091, 1092
gko::cuda_stream, 636
gko::experimental::mpi::communicator, 546
gko::experimental::mpi::contiguous_type, 571
gko::experimental::mpi::request, 1129
gko::experimental::mpi::status, 1192
gko::experimental::mpi::window< ValueType >, 1249
gko::hip_stream, 850
gko::log::Record, 1121
gko::ptr_param< T >, 1106
gko::version_info, 1242
get_accessor
gko::range< Accessor >, 1111
get_accumulate
gko::experimental::mpi::window< ValueType >, 1250
get_agg
gko::multigrid::Pgm< ValueType, IndexType >, 1081
get_approximate_inverse

gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 934
get_array
gko::config::pnode, 1092
get_basis
gko::Perturbation< ValueType >, 1079
gko::solver::Nullspace< ValueType >, 1034
get_blas_handle
gko::CudaExecutor, 642
gko::HipExecutor, 854
get_block_offset
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 504
get_block_size
gko::BlockOperator, 508
gko::matrix::Fbcsr< ValueType, IndexType >, 818
get_blocks
gko::preconditioner::Jacobi< ValueType, IndexType >, 945
get_boolean
gko::config::pnode, 1092
get_coarse_op
gko::multigrid::EnableMultigridLevel< ValueType >, 788
gko::multigrid::MultigridLevel, 1007
get_coarse_rows
gko::multigrid::UniformCoarsening< ValueType, IndexType >, 1212
get_coarsest_solver
gko::solver::Multigrid, 1003
get_coefficients
gko::Combination< ValueType >, 532
get_col_idxs
gko::batch::matrix::Csr< ValueType, IndexType >, 629
gko::batch::matrix::Ell< ValueType, IndexType >, 758
gko::device_matrix_data< ValueType, IndexType >, 720
gko::matrix::Coo< ValueType, IndexType >, 584
gko::matrix::Csr< ValueType, IndexType >, 606
gko::matrix::Ell< ValueType, IndexType >, 769
gko::matrix::Fbcsr< ValueType, IndexType >, 818
gko::matrix::Sellp< ValueType, IndexType >, 1167
gko::matrix::SparsityCsr< ValueType, IndexType >, 1186
get_col_idxs_for_item
gko::batch::matrix::Ell< ValueType, IndexType >, 758
get_common_size
gko::batch::MultiVector< ValueType >, 1018
gko::batch_dim< Dimensionality, DimensionType >, 488
get_communicator
gko::experimental::distributed::DistributedBase, 742
get_complex_subspace
gko::solver::Idr< ValueType >, 895
get_composition
gko::UseComposition< ValueType >, 1220
get_conditioning
gko::preconditioner::Jacobi< ValueType, IndexType >, 945
get_config_map
gko::ext::cuda::solver::Cudss< ValueType, IndexType >, 648
get_const_agg
gko::multigrid::Pgm< ValueType, IndexType >, 1081
get_const_block_pointers
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 939
get_const_blocks
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 939
get_const_blocks_cumulative_offsets
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 940
get_const_coarse_rows
gko::multigrid::UniformCoarsening< ValueType, IndexType >, 1212
get_const_col_idxs
gko::batch::matrix::Csr< ValueType, IndexType >, 630
gko::batch::matrix::Ell< ValueType, IndexType >, 758
gko::device_matrix_data< ValueType, IndexType >, 720
gko::matrix::Coo< ValueType, IndexType >, 584
gko::matrix::Csr< ValueType, IndexType >, 606
gko::matrix::Ell< ValueType, IndexType >, 769
gko::matrix::Fbcsr< ValueType, IndexType >, 819
gko::matrix::Sellp< ValueType, IndexType >, 1168
gko::matrix::SparsityCsr< ValueType, IndexType >, 1186
get_const_col_idxs_for_item
gko::batch::matrix::Ell< ValueType, IndexType >, 759
get_const_coo_col_idxs
gko::matrix::Hybrid< ValueType, IndexType >, 872
get_const_coo_row_idxs
gko::matrix::Hybrid< ValueType, IndexType >, 872
get_const_coo_values
gko::matrix::Hybrid< ValueType, IndexType >, 872
get_const_data
gko::array< ValueType >, 475
get_const_device_view
gko::matrix::Coo< ValueType, IndexType >, 584
gko::matrix::Csr< ValueType, IndexType >, 606
gko::matrix::Hybrid< ValueType, IndexType >, 873
get_const_ell_col_idxs
gko::matrix::Hybrid< ValueType, IndexType >, 873
get_const_ell_values
gko::matrix::Hybrid< ValueType, IndexType >, 873
get_const_flat_data
gko::segmented_array< T >, 1160

get_const_local_values
gko::experimental::distributed::Vector< ValueType >, 1237
get_const_map_block_to_row
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 940
get_const_permutation
gko::matrix::Permutation< IndexType >, 1074
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1149
get_const_row_idxs
gko::device_matrix_data< ValueType, IndexType >, 721
gko::matrix::Coo< ValueType, IndexType >, 585
gko::matrix::RowGatherer< IndexType >, 1139
get_const_row_ptrs
gko::batch::matrix::Csr< ValueType, IndexType >, 630
gko::matrix::Csr< ValueType, IndexType >, 606
gko::matrix::Fbcsr< ValueType, IndexType >, 819
gko::matrix::SparsityCsr< ValueType, IndexType >, 1186
get_const_scaling_factors
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1149
get_const_slice_lengths
gko::matrix::Sellp< ValueType, IndexType >, 1168
get_const_slice_sets
gko::matrix::Sellp< ValueType, IndexType >, 1168
get_const_srow
gko::matrix::Csr< ValueType, IndexType >, 607
get_const_value
gko::matrix::SparsityCsr< ValueType, IndexType >, 1187
get_const_values
gko::batch::matrix::Csr< ValueType, IndexType >, 630
gko::batch::matrix::Dense< ValueType >, 664
gko::batch::matrix::Ell< ValueType, IndexType >, 759
gko::batch::MultiVector< ValueType >, 1018
gko::device_matrix_data< ValueType, IndexType >, 721
gko::matrix::Coo< ValueType, IndexType >, 585
gko::matrix::Csr< ValueType, IndexType >, 607
gko::matrix::Dense< ValueType >, 691
gko::matrix::Diagonal< ValueType >, 728
gko::matrix::Ell< ValueType, IndexType >, 770
gko::matrix::Fbcsr< ValueType, IndexType >, 819
gko::matrix::Sellp< ValueType, IndexType >, 1169
get_const_values_for_item
gko::batch::matrix::Csr< ValueType, IndexType >, 631
gko::batch::matrix::Dense< ValueType >, 664
gko::batch::matrix::Ell< ValueType, IndexType >, 760
gko::batch::MultiVector< ValueType >, 1018
get_coo
gko::matrix::Hybrid< ValueType, IndexType >, 874
get_coo_col_idxs
gko::matrix::Hybrid< ValueType, IndexType >, 874
get_coo_nnz
gko::matrix::Hybrid< ValueType, IndexType >::strategy_type, 1198
get_coo_num_stored_elements
gko::matrix::Hybrid< ValueType, IndexType >, 874
get_coo_row_idxs
gko::matrix::Hybrid< ValueType, IndexType >, 875
get_coo_values
gko::matrix::Hybrid< ValueType, IndexType >, 875
get_count
gko::experimental::mpi::status, 1192
get_cublas_handle
gko::CudaExecutor, 642
get_cumulative_offset
gko::batch::matrix::Dense< ValueType >, 665
gko::batch::MultiVector< ValueType >, 1019
get_cusparse_handle
gko::CudaExecutor, 642
get_cycle
gko::solver::Multigrid, 1004
get_data
gko::array< ValueType >, 476
get_description
gko::CudaExecutor, 642
gko::DpcppExecutor, 744
gko::Executor, 803
gko::HipExecutor, 854
gko::OmpExecutor, 1041
gko::ReferenceExecutor, 1122
get_deterministic
gko::solver::Idr< ValueType >, 895
get_device_id
gko::DpcppExecutor, 745
get_device_type
gko::DpcppExecutor, 745
get_device_view
gko::matrix::Coo< ValueType, IndexType >, 585
gko::matrix::Csr< ValueType, IndexType >, 608
gko::matrix::Hybrid< ValueType, IndexType >, 875
get_diag_matrix
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 972
get_dimension
gko::solver::Nullspace< ValueType >, 1034
get_dynamic_type
gko::name_demangling, 439
get_ell
gko::matrix::Hybrid< ValueType, IndexType >, 875
get_ell_col_idxs
gko::matrix::Hybrid< ValueType, IndexType >, 876
get_ell_num_stored_elements
gko::matrix::Hybrid< ValueType, IndexType >, 876
get_ell_num_stored_elements_per_row
gko::matrix::Hybrid< ValueType, IndexType >, 876

gko::matrix::Hybrid< ValueType, IndexType >::strategy_type, 1199
get_ell_stride
gko::matrix::Hybrid< ValueType, IndexType >, 877
get_ell_values
gko::matrix::Hybrid< ValueType, IndexType >, 877
get_executor
gko::array< ValueType >, 477
gko::device_matrix_data< ValueType, IndexType >, 721
gko::index_set< IndexType >, 919
gko::PolymorphicObject, 1095
gko::segmented_array< T >, 1160
get_fine_op
gko::multigrid::EnableMultigridLevel< ValueType >, 788
gko::multigrid::MultigridLevel, 1007
get_flat_data
gko::segmented_array< T >, 1160
get_global_block_offset
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 504
get_global_index
gko::index_set< IndexType >, 919
get_group_offset
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 504
get_group_size
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 505
get_handle
gko::log::Papi< ValueType >, 1048
get_handle_name
gko::log::Papi< ValueType >, 1049
get_hipblas_handle
gko::HipExecutor, 855
get_hipsparse_handle
gko::HipExecutor, 855
get_id
gko::stopping_status, 1194
get_implicit_sq_resnorm
gko::log::Convergence< ValueType >, 574
get_integer
gko::config::pnode, 1093
get_inverse_permutation
gko::reorder::Rcm< ValueType, IndexType >, 1114
get_kappa
gko::solver::Idr< ValueType >, 895
get_krylov_dim
gko::solver::CbGmres< ValueType >, 511
gko::solver::Gcr< ValueType >, 843
gko::solver::Gmres< ValueType >, 846
get_l_solver
gko::preconditioner::Ic< ValueType, IndexType >, 884
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 902
get_left_nullspace
gko::preconditioner::NullspaceProjection< ValueType >, 1038
gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, 792
get_lh_solver
gko::preconditioner::Ic< ValueType, IndexType >, 884
get_local_index
gko::index_set< IndexType >, 920
get_local_matrix
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 972
get_local_values
gko::experimental::distributed::Vector< ValueType >, 1237
get_local_vector
gko::experimental::distributed::Vector< ValueType >, 1238
get_loggers
gko::log::Loggable, 955
get_map
gko::config::pnode, 1093
get_master
gko::CudaExecutor, 643
gko::DpcppExecutor, 745
gko::Executor, 803
gko::HipExecutor, 855
gko::OmpExecutor, 1041
get_max_block_size
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 941
get_max_iterations
gko::batch::solver::BatchSolver, 493
get_max_subgroup_size
gko::DpcppExecutor, 746
get_max_workgroup_size
gko::DpcppExecutor, 746
get_max_workitem_sizes
gko::DpcppExecutor, 746
get_mg_level_list
gko::solver::Multigrid, 1004
get_mid_smoother_list
gko::solver::Multigrid, 1004
get_name
gko::Operation, 1044
get_non_local_matrix
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 972
get_nonpreserving
gko::precision_reduction, 1099
get_nullspace
gko::preconditioner::NullspaceProjection< ValueType >, 1038
gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, 792
get_num_batch_items
gko::batch::MultiVector< ValueType >, 1020
gko::batch_dim< Dimensionality, DimensionType

>, 488
get_num_block_cols
gko::matrix::Fbcsr< ValueType, IndexType >, 820
get_num_block_rows
gko::matrix::Fbcsr< ValueType, IndexType >, 820
get_num_blocks
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 941
gko::preconditioner::Jacobi< ValueType, IndexType >, 945
get_num_columns
gko::matrix::Hybrid< ValueType, IndexType >::column_limit, 530
get_num_computing_units
gko::DpcppExecutor, 746
get_num_devices
gko::DpcppExecutor, 747
get_num_elements_per_item
gko::batch::matrix::Csr< ValueType, IndexType >, 631
gko::batch::matrix::Dense< ValueType >, 665
gko::batch::matrix::Ell< ValueType, IndexType >, 760
get_num_elems
gko::array< ValueType >, 477
gko::device_matrix_data< ValueType, IndexType >, 722
gko::index_set< IndexType >, 921
get_num_empty_parts
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1061
get_num_iterations
gko::batch::log::BatchConvergence< ValueType >, 491
gko::log::Convergence< ValueType >, 574
get_num_nonzeros
gko::matrix::SparsityCsr< ValueType, IndexType >, 1187
get_num_parts
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1061
get_num_ranges
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1061
get_num_srow_elements
gko::matrix::Csr< ValueType, IndexType >, 608
get_num_stored_blocks
gko::matrix::Fbcsr< ValueType, IndexType >, 820
get_num_stored_elements
gko::batch::matrix::Csr< ValueType, IndexType >, 632
gko::batch::matrix::Dense< ValueType >, 666
gko::batch::matrix::Ell< ValueType, IndexType >, 761
gko::batch::MultiVector< ValueType >, 1020
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 941
gko::device_matrix_data< ValueType, IndexType >, 722
gko::matrix::Coo< ValueType, IndexType >, 586
gko::matrix::Csr< ValueType, IndexType >, 608
gko::matrix::Dense< ValueType >, 691
gko::matrix::Ell< ValueType, IndexType >, 770
gko::matrix::Fbcsr< ValueType, IndexType >, 821
gko::matrix::Hybrid< ValueType, IndexType >, 877
gko::matrix::Sellp< ValueType, IndexType >, 1169
gko::matrix_assembly_data< ValueType, IndexType >, 980
gko::preconditioner::Jacobi< ValueType, IndexType >, 946
get_num_stored_elements_per_row
gko::batch::matrix::Ell< ValueType, IndexType >, 761
gko::matrix::Ell< ValueType, IndexType >, 770
get_num_subsets
gko::index_set< IndexType >, 921
get_off_diag_matrix
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 973
get_offsets
gko::segmented_array< T >, 1160
get_operator_at
gko::UseComposition< ValueType >, 1220
get_operators
gko::Combination< ValueType >, 532
gko::Composition< ValueType >, 569
get_ordered_data
gko::matrix_assembly_data< ValueType, IndexType >, 981
get_parameters
gko::EnableDefaultFactory< ConcreteFactory, ProductType, ParametersType, PolymorphicBase >, 783
gko::experimental::factorization::Cholesky< ValueType, IndexType >, 524
gko::experimental::factorization::Lu< ValueType, IndexType >, 960, 961
gko::experimental::reorder::Amd< IndexType >, 464
gko::experimental::reorder::Mc64< ValueType, IndexType >, 995
gko::experimental::reorder::Rcm< IndexType >, 1117
gko::preconditioner::GaussSeidel< ValueType, IndexType >, 841
gko::preconditioner::Sor< ValueType, IndexType >, 1178
get_part_ids
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1062
get_part_size
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1062
get_part_sizes
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1063

get_percentage
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit, 906
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit, 908
gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit, 997
get_permutation
gko::matrix::Permutation< IndexType >, 1075
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1150
gko::reorder::Rcm< ValueType, IndexType >, 1115
get_permutation_size
gko::matrix::Permutation< IndexType >, 1075
get_post_smoother_list
gko::solver::Multigrid, 1004
get_pre_smoother_list
gko::solver::Multigrid, 1005
get_preconditioner
gko::batch::solver::BatchSolver, 493
gko::Preconditionable, 1100
gko::preconditioner::NullspaceProjection< ValueType >, 1038
get_preserving
gko::precision_reduction, 1099
get_projector
gko::Perturbation< ValueType >, 1079
get_prolong_op
gko::multigrid::EnableMultigridLevel< ValueType >, 788
gko::multigrid::MultigridLevel, 1007
get_provided_thread_support
gko::experimental::mpi::environment, 796
get_range_bounds
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1063
get_range_starting_indices
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1063
get_ranges_by_part
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1064
get_ratio
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit, 906
get_real
gko::config::pnode, 1093
get_recv_size
gko::experimental::mpi::CollectiveCommunicator, 527
gko::experimental::mpi::DenseCommunicator, 712
gko::experimental::mpi::NeighborhoodCommunicator, 1024
get_remote_global_idxs
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 911
get_remote_local_idxs
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 912
get_remote_target_ids
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 912
get_residual
gko::log::Convergence< ValueType >, 575
get_residual_norm
gko::batch::log::BatchConvergence< ValueType >, 491
gko::log::Convergence< ValueType >, 575
get_restart_ratio
gko::solver::Gmres< ValueType >, 846
get_restrict_op
gko::multigrid::EnableMultigridLevel< ValueType >, 788
gko::multigrid::MultigridLevel, 1007
get_row_gatherer
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 973
get_row_idxs
gko::device_matrix_data< ValueType, IndexType >, 722
gko::matrix::Coo< ValueType, IndexType >, 586
gko::matrix::RowGatherer< IndexType >, 1139
get_row_partition
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 973
get_row_ptrs
gko::batch::matrix::Csr< ValueType, IndexType >, 632
gko::matrix::Csr< ValueType, IndexType >, 609
gko::matrix::Fbcsr< ValueType, IndexType >, 821
gko::matrix::SparsityCsr< ValueType, IndexType >, 1188
get_scalar
gko::Perturbation< ValueType >, 1079
get_scaling_factors
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1150
get_segment_count
gko::segmented_array< T >, 1161
get_send_size
gko::experimental::mpi::CollectiveCommunicator, 527
gko::experimental::mpi::DenseCommunicator, 712
gko::experimental::mpi::NeighborhoodCommunicator, 1025
get_significant_bit
gko, 385
get_size
gko::array< ValueType >, 478
gko::batch::MultiVector< ValueType >, 1020
gko::device_matrix_data< ValueType, IndexType >, 723
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1064
gko::index_set< IndexType >, 921

gko::matrix_assembly_data< ValueType, IndexType >, 981
gko::segmented_array< T >, 1161
get_slice_lengths
gko::matrix::Sellp< ValueType, IndexType >, 1170
get_slice_sets
gko::matrix::Sellp< ValueType, IndexType >, 1170
get_slice_size
gko::matrix::Sellp< ValueType, IndexType >, 1170
get_solver
gko::solver::Ir< ValueType >, 930
get_sparselib_handle
gko::CudaExecutor, 643
gko::HipExecutor, 856
get_srow
gko::matrix::Csr< ValueType, IndexType >, 609
get_static_type
gko::name_demangling, 440
get_stop_criterion_factory
gko::solver::IterativeBase, 937
get_storage_precision
gko::solver::CbGmres< ValueType >, 511
get_storage_scheme
gko::preconditioner::Jacobi< ValueType, IndexType >, 946
get_strategy
gko::matrix::Csr< ValueType, IndexType >, 609
gko::matrix::Hybrid< ValueType, IndexType >, 878
get_stream
gko::CudaExecutor, 643
get_stride
gko::matrix::Dense< ValueType >, 692
gko::matrix::Ell< ValueType, IndexType >, 771
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 505
get_stride_factor
gko::matrix::Sellp< ValueType, IndexType >, 1171
get_string
gko::config::pnode, 1093
get_subgroup_sizes
gko::DpcppExecutor, 747
get_subsets_begin
gko::index_set< IndexType >, 921
get_subsets_end
gko::index_set< IndexType >, 922
get_subspace_dim
gko::solver::Idr< ValueType >, 895
get_superior_power
gko, 385
get_superset_indices
gko::index_set< IndexType >, 922
get_system_matrix
gko::batch::solver::BatchSolver, 494
gko::multigrid::FixedCoarsening< ValueType, IndexType >, 839
gko::multigrid::Pgm< ValueType, IndexType >, 1082
gko::multigrid::Pmis< ValueType, IndexType >, 1087
gko::multigrid::Rs< ValueType, IndexType >, 1143
gko::multigrid::UniformCoarsening< ValueType, IndexType >, 1213
get_tag
gko::config::pnode, 1094
get_tolerance
gko::batch::solver::BatchSolver, 494
get_tolerance_type
gko::batch::solver::BatchSolver, 494
get_total_cols
gko::matrix::Sellp< ValueType, IndexType >, 1171
get_u_solver
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 902
get_unprojected_preconditioner
gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, 792
get_value
gko::matrix::SparsityCsr< ValueType, IndexType >, 1188
gko::matrix_assembly_data< ValueType, IndexType >, 981
get_values
gko::batch::matrix::Csr< ValueType, IndexType >, 632
gko::batch::matrix::Dense< ValueType >, 666
gko::batch::matrix::Ell< ValueType, IndexType >, 761
gko::batch::MultiVector< ValueType >, 1020
gko::device_matrix_data< ValueType, IndexType >, 723
gko::matrix::Coo< ValueType, IndexType >, 586
gko::matrix::Csr< ValueType, IndexType >, 610
gko::matrix::Dense< ValueType >, 692
gko::matrix::Diagonal< ValueType >, 728
gko::matrix::Ell< ValueType, IndexType >, 771
gko::matrix::Fbcsr< ValueType, IndexType >, 821
gko::matrix::Sellp< ValueType, IndexType >, 1171
get_values_for_item
gko::batch::matrix::Csr< ValueType, IndexType >, 633
gko::batch::matrix::Dense< ValueType >, 666
gko::batch::matrix::Ell< ValueType, IndexType >, 762
gko::batch::MultiVector< ValueType >, 1021
get_walltime
gko::experimental::mpi, 431
get_window
gko::experimental::mpi::window< ValueType >, 1250
give
gko, 386
gko, 359
abs, 374
allocation_mode, 373
array, 374

as, 375–377, 379
automatic, 374
ceildiv, 380
clone, 381
conj, 382
coordinate, 374
copy_and_convert_to, 382–384
get_significant_bit, 385
get_superior_power, 385
give, 386
highest_precision, 371
imag, 386
is_complex, 387
is_complex_or_scalar, 387
is_complex_or_scalar_s, 371
is_complex_s, 372
is_finite, 388
is_nan, 389
is_nonzero, 390
is_zero, 390
layout_type, 374
lend, 391
log_propagation_mode, 374
make_array_view, 392
make_const_array_view, 392
make_const_dense_view, 393
make_dense_view, 394
make_temporary_clone, 394
make_temporary_conversion, 395
make_temporary_output_clone, 396
max, 396
min, 397
mixed_precision_dispatch, 397
mixed_precision_dispatch_real_complex, 398
nan, 399
never, 374
one, 399, 400
operator!=, 400, 401
operator<<, 401, 402
operator==, 402
pi, 403
precision_dispatch, 403
precision_dispatch_real_complex, 404, 405
previous_precision_base, 372
read, 406
read_binary, 406
read_binary_raw, 407
read_generic, 408
read_generic_raw, 409
read_raw, 409
real, 410
reduce_add, 410, 411
remove_complex, 372
round_down, 411
round_up, 412
safe_divide, 412
share, 413
squared_norm, 413
to_complex, 373
to_real, 373
transpose, 414
unit_root, 415
with_matrix_type, 415
write, 416, 417
write_binary, 417
write_binary_raw, 418
write_raw, 418
zero, 420
gko::AbsoluteComputable, 459
compute_absolute_linop, 459
gko::AbstractFactory< AbstractProductType, ComponentsType >, 460
generate, 461
gko::accessor, 421
gko::accessor::row_major< ValueType, Dimensionality >, 1133
copy_from, 1134
length, 1135
operator(), 1135, 1136
gko::AllocationError, 462
AllocationError, 462
gko::Allocator, 463
gko::amd_device, 464
gko::are_all_integral< Args >, 465
gko::array< ValueType >, 466
array, 468, 470–473
as_const_view, 473
as_view, 474
clear, 474
const_view, 474
copy_to_host, 475
fill, 475
get_const_data, 475
get_data, 476
get_executor, 477
get_num_elems, 477
get_size, 478
is_owning, 478
operator=, 479–482
resize_and_reset, 482
set_executor, 483
view, 484
gko::BadDimension, 486
BadDimension, 486
gko::batch::BatchLinOpFactory, 492
gko::batch::log, 421
gko::batch::log::BatchConvergence< ValueType >, 490
create, 491
get_num_iterations, 491
get_residual_norm, 491
gko::batch::matrix::Csr< ValueType, IndexType >, 623
add_scaled_identity, 625
apply, 625, 626
create, 626, 627
create_const, 628
create_const_view_for_item, 628

create_view_for_item, 629
get_col_idxs, 629
get_const_col_idxs, 630
get_const_row_ptrs, 630
get_const_values, 630
get_const_values_for_item, 631
get_num_elements_per_item, 631
get_num_stored_elements, 632
get_row_ptrs, 632
get_values, 632
get_values_for_item, 633
scale, 633
gko::batch::matrix::Dense< ValueType >, 656
add_scaled_identity, 657
apply, 658, 659
at, 659–661
create, 661, 662
create_const, 663
create_const_view_for_item, 663
create_view_for_item, 664
get_const_values, 664
get_const_values_for_item, 664
get_cumulative_offset, 665
get_num_elements_per_item, 665
get_num_stored_elements, 666
get_values, 666
get_values_for_item, 666
scale, 667
scale_add, 667
gko::batch::matrix::Ell< ValueType, IndexType >, 751
add_scaled_identity, 753
apply, 753–755
create, 755, 756
create_const, 756
create_const_view_for_item, 757
create_view_for_item, 757
get_col_idxs, 758
get_col_idxs_for_item, 758
get_const_col_idxs, 758
get_const_col_idxs_for_item, 759
get_const_values, 759
get_const_values_for_item, 760
get_num_elements_per_item, 760
get_num_stored_elements, 761
get_num_stored_elements_per_row, 761
get_values, 761
get_values_for_item, 762
scale, 762
gko::batch::matrix::Identity< ValueType >, 886
apply, 887, 888
create, 888
gko::batch::MultiVector< ValueType >, 1009
add_scaled, 1011
at, 1011–1013
compute_conj_dot, 1014
compute_dot, 1014
compute_norm2, 1014
create, 1015
create_const, 1016
create_const_view_for_item, 1016
create_view_for_item, 1017
create_with_config_of, 1017
fill, 1018
get_common_size, 1018
get_const_values, 1018
get_const_values_for_item, 1018
get_cumulative_offset, 1019
get_num_batch_items, 1020
get_num_stored_elements, 1020
get_size, 1020
get_values, 1020
get_values_for_item, 1021
scale, 1021
gko::batch::preconditioner::Jacobi< ValueType, IndexType >, 938
get_const_block_pointers, 939
get_const_blocks, 939
get_const_blocks_cumulative_offsets, 940
get_const_map_block_to_row, 940
get_max_block_size, 941
get_num_blocks, 941
get_num_stored_elements, 941
gko::batch::solver::BatchSolver, 493
get_max_iterations, 493
get_preconditioner, 493
get_system_matrix, 494
get_tolerance, 494
get_tolerance_type, 494
reset_max_iterations, 494
reset_tolerance, 495
reset_tolerance_type, 495
gko::batch::solver::Bicgstab< ValueType >, 499
gko::batch::solver::Cg< ValueType >, 513
gko::batch::solver::EnableBatchSolver< ConcreteSolver, ValueType, PolymorphicBase >, 778
gko::batch_dim< Dimensionality, DimensionType >, 487
batch_dim, 488
get_common_size, 488
get_num_batch_items, 488
operator!=, 489
operator==, 489
gko::bfloat16, 496
gko::BlockOperator, 506
block_at, 507
BlockOperator, 507
create, 508
get_block_size, 508
operator=, 509
gko::BlockSizeError< IndexType >, 509
BlockSizeError, 510
gko::Cloneable, 525
gko::Combination< ValueType >, 531
Combination, 531, 532
conj_transpose, 532
get_coefficients, 532

get_operators, 532
operator=, 533
transpose, 533
gko::Composition< ValueType >, 567
Composition, 568
conj_transpose, 569
get_operators, 569
operator=, 569
transpose, 570
gko::config::pnode, 1088
get, 1091, 1092
get_array, 1092
get_boolean, 1092
get_integer, 1093
get_map, 1093
get_real, 1093
get_string, 1093
get_tag, 1094
operator bool, 1094
pnode, 1089–1091
gko::config::registry, 1124
emplace, 1126
registry, 1125
gko::config::type_descriptor, 1209
type_descriptor, 1210
gko::ConvertibleTo< ResultType >, 576
convert_to, 577
move_to, 577
gko::CpuAllocator, 589
gko::CpuAllocatorBase, 590
gko::CpuTimer, 590
difference_async, 590
gko::cpx_real_type< T >, 591
type, 591
gko::CublasError, 634
CublasError, 635
gko::cuda_stream, 635
cuda_stream, 636
get, 636
gko::CudaAllocator, 637
gko::CudaAllocatorBase, 637
gko::CudaError, 637
CudaError, 638
gko::CudaExecutor, 638
create, 640
get_blas_handle, 642
get_cublas_handle, 642
get_cusparse_handle, 642
get_description, 642
get_master, 643
get_sparselib_handle, 643
get_stream, 643
run, 644, 646
gko::CudaTimer, 646
difference_async, 647
gko::CufftError, 649
CufftError, 650
gko::CurandError, 650
CurandError, 650
gko::CusparseError, 651
CusparseError, 651
gko::default_converter< S, R >, 652
operator(), 652
gko::deferred_factory_parameter< FactoryType >, 654
deferred_factory_parameter, 655
on, 655
gko::device_matrix_data< ValueType, IndexType >, 715
copy_to_host, 719
create_from_host, 720
device_matrix_data, 718, 719
empty_out, 720
get_col_idxs, 720
get_const_col_idxs, 720
get_const_row_idxs, 721
get_const_values, 721
get_executor, 721
get_num_elems, 722
get_num_stored_elements, 722
get_row_idxs, 722
get_size, 723
get_values, 723
remove_zeros, 723
resize_and_reset, 723, 724
sum_duplicates, 724
gko::device_matrix_data< ValueType, IndexType >::arrays, 484
gko::DiagonalExtractable< ValueType >, 730
extract_diagonal, 731
extract_diagonal_linop, 731
gko::DiagonalLinOpExtractable, 732
extract_diagonal_linop, 732
gko::dim< Dimensionality, DimensionType >, 732
dim, 733, 735
operator bool, 735
operator!=, 736
operator<<, 737
operator*, 737
operator==, 738
operator[], 735, 736
gko::DimensionMismatch, 738
DimensionMismatch, 739
gko::DpcppExecutor, 743
create, 744
get_description, 744
get_device_id, 745
get_device_type, 745
get_master, 745
get_max_subgroup_size, 746
get_max_workgroup_size, 746
get_max_workitem_sizes, 746
get_num_computing_units, 746
get_num_devices, 747
get_subgroup_sizes, 747
run, 747, 749
gko::DpcppTimer, 750
difference_async, 751

gko::enable_parameters_type< ConcreteParameterType, Factory >, 775
on, 776
gko::EnableAbsoluteComputation< AbsoluteLinOp >, 776
compute_absolute, 777
compute_absolute_linop, 777
gko::EnableCloneable< ConcreteType >, 779
clone, 779, 780
convert_to, 780
move_to, 781
gko::EnableCreateMethod< ConcreteType >, 781
gko::EnableDefaultFactory< ConcreteFactory, ProductType, ParametersType, PolymorphicBase >, 782
create, 783
get_parameters, 783
gko::EnableDefaultReusableLinOpFactory< ConcreteFactory, ConcreteLinOp, ParametersType, ReuseDataType, PolymorphicBase >, 783
gko::Error, 796
Error, 797
gko::Executor, 797
add_logger, 800
alloc, 800
copy, 801
copy_from, 801
copy_val_to_host, 802
free, 803
get_description, 803
get_master, 803
memory_accessible, 804
remove_logger, 804
run, 804, 805
set_log_propagation_mode, 806
should_propagate_log, 806
gko::executor_deleter< T >, 807
executor_deleter, 808
operator(), 808
gko::experimental::distributed, 421
assemble_rows_from_neighbors, 424
assembly_mode, 423
build_partition_from_local_range, 424
build_partition_from_local_size, 425
combined, 423
index_space, 423
local, 423
make_temporary_conversion, 425, 426
non_local, 423
precision_dispatch, 427
precision_dispatch_real_complex, 428, 429
gko::experimental::distributed::DistributedBase, 741
get_communicator, 742
operator=, 742
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 909
get_remote_global_idxs, 911
get_remote_local_idxs, 912
get_remote_target_ids, 912
index_map, 911
map_to_global, 912
map_to_local, 912
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 962
col_scale, 966
create, 966–971
get_diag_matrix, 972
get_local_matrix, 972
get_non_local_matrix, 972
get_off_diag_matrix, 973
get_row_gatherer, 973
get_row_partition, 973
Matrix, 965
multiply, 974
operator=, 974, 975
read_distributed, 975–977
row_scale, 977
transpose, 978
write, 978
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1057
build_from_contiguous, 1059
build_from_global_size_uniform, 1059
build_from_mapping, 1060
equals, 1060
get_num_empty_parts, 1061
get_num_parts, 1061
get_num_ranges, 1061
get_part_ids, 1062
get_part_size, 1062
get_part_sizes, 1063
get_range_bounds, 1063
get_range_starting_indices, 1063
get_ranges_by_part, 1064
get_size, 1064
has_connected_parts, 1064
has_ordered_parts, 1065
gko::experimental::distributed::preconditioner, 430
gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType >, 1152
apply_uses_initial_guess, 1153
parse, 1153
gko::experimental::distributed::RowGatherer< LocalIndexType >, 1140
apply_async, 1141
create, 1142
gko::experimental::distributed::Vector< ValueType >, 1222
add_scaled, 1225
at_local, 1226
compute_absolute, 1227
compute_conj_dot, 1227, 1228
compute_dot, 1228
compute_mean, 1229
compute_norm1, 1229, 1230

compute_norm2, 1230
compute_squared_norm2, 1231
create, 1231–1233
create_const, 1234
create_real_view, 1235
create_submatrix, 1235
create_with_config_of, 1235
create_with_type_of, 1236
fill, 1237
get_const_local_values, 1237
get_local_values, 1237
get_local_vector, 1238
inv_scale, 1238
make_complex, 1238
read_distributed, 1239
scale, 1239
sub_scaled, 1240
gko::experimental::factorization::Cholesky< ValueType, IndexType >, 523
generate, 523
get_parameters, 524
parse, 524
gko::experimental::factorization::Factorization< ValueType, IndexType >, 809
create_from_combined_lu, 810
create_from_composition, 810
create_from_symm_composition, 811
unpack, 811
gko::experimental::factorization::Lu< ValueType, IndexType >, 959
generate, 960
get_parameters, 960, 961
parse, 961
gko::experimental::mpi, 430
comm_index_type, 431
get_walltime, 431
map_rank_to_device_id, 432
wait_all, 432
gko::experimental::mpi::CollectiveCommunicator, 525
create_inverse, 526
create_with_same_type, 526
get_recv_size, 527
get_send_size, 527
i_all_to_all_v, 527, 528
resize, 528
gko::experimental::mpi::communicator, 534
all_gather, 539
all_reduce, 540, 541
all_to_all, 541, 542
all_to_all_v, 543
broadcast, 544
communicator, 538, 539
create_owning, 545
gather, 545
gather_v, 546
get, 546
i_all_gather, 547
i_all_reduce, 548
i_all_to_all, 549, 550
i_all_to_all_v, 551, 552
i_broadcast, 552
i_gather, 553
i_gather_v, 554
i_recv, 555
i_reduce, 556
i_scan, 556
i_scatter, 557
i_scatter_v, 558
i_send, 559
is_congruent, 560
is_identical, 560
node_local_rank, 560
operator!=, 561
operator=, 561
operator==, 561
rank, 562
recv, 562
reduce, 563
scan, 564
scatter, 564
scatter_v, 565
send, 566
size, 566
synchronize, 567
gko::experimental::mpi::contiguous_type, 570
contiguous_type, 571
get, 571
operator=, 572
gko::experimental::mpi::DenseCommunicator, 710
create_inverse, 712
create_with_same_type, 712
DenseCommunicator, 711
get_recv_size, 712
get_send_size, 712
i_all_to_all_v, 713
operator!=, 714
operator==, 714
resize, 713
gko::experimental::mpi::environment, 795
environment, 795
get_provided_thread_support, 796
gko::experimental::mpi::NeighborhoodCommunicator, 1022
create_inverse, 1024
create_with_same_type, 1024
get_recv_size, 1024
get_send_size, 1025
i_all_to_all_v, 1025, 1026
NeighborhoodCommunicator, 1023
operator!=, 1027
operator==, 1027
resize, 1026
gko::experimental::mpi::request, 1128
get, 1129
request, 1128
wait, 1129

gko::experimental::mpi::status, 1191
get, 1192
get_count, 1192
status, 1191
gko::experimental::mpi::type_impl< T >, 1210
gko::experimental::mpi::window< ValueType >, 1243
accumulate, 1247
fence, 1247
fetch_and_op, 1248
flush, 1248
flush_local, 1249
get, 1249
get_accumulate, 1250
get_window, 1250
lock, 1251
lock_all, 1251
operator=, 1252
put, 1252
r_accumulate, 1253
r_get, 1253
r_get_accumulate, 1254
r_put, 1255
unlock, 1256
window, 1245, 1246
gko::experimental::reorder, 432
mc64_strategy, 433
gko::experimental::reorder::Amd< IndexType >, 463
generate, 464
get_parameters, 464
gko::experimental::reorder::Mc64< ValueType, IndexType >, 993
generate, 994
get_parameters, 995
gko::experimental::reorder::Rcm< IndexType >, 1115
generate, 1116
get_parameters, 1117
gko::experimental::reorder::ScaledReordered< ValueType, IndexType >, 1151
gko::experimental::solver::Direct< ValueType, IndexType >, 739
conj_transpose, 740
parse, 740
transpose, 741
gko::ext::cuda::solver::Cudss< ValueType, IndexType >, 648
get_config_map, 648
refactorize, 649
gko::factorization, 433
incomplete_algorithm, 434
gko::factorization::Ic< ValueType, IndexType >, 880
parse, 881
gko::factorization::Ilu< ValueType, IndexType >, 898
parse, 899
gko::factorization::ParIc< ValueType, IndexType >, 1049
parse, 1050
gko::factorization::ParIct< ValueType, IndexType >, 1052
parse, 1053
gko::factorization::ParIlu< ValueType, IndexType >, 1054
parse, 1055
gko::factorization::ParIlut< ValueType, IndexType >, 1055
parse, 1057
gko::half, 848
gko::hip_stream, 849
get, 850
hip_stream, 850
gko::HipAllocatorBase, 851
gko::HipblasError, 851
HipblasError, 851
gko::HipError, 852
HipError, 852
gko::HipExecutor, 853
create, 854
get_blas_handle, 854
get_description, 854
get_hipblas_handle, 855
get_hipsparse_handle, 855
get_master, 855
get_sparselib_handle, 856
run, 856, 857
gko::HipfftError, 858
HipfftError, 858
gko::HiprandError, 859
HiprandError, 859
gko::HipsparseError, 860
HipsparseError, 860
gko::HipTimer, 861
difference_async, 861
gko::index_set< IndexType >, 913
clear, 918
contains, 918, 919
get_executor, 919
get_global_index, 919
get_local_index, 920
get_num_elems, 921
get_num_subsets, 921
get_size, 921
get_subsets_begin, 921
get_subsets_end, 922
get_superset_indices, 922
index_set, 915–918
is_contiguous, 922
map_global_to_local, 923
map_local_to_global, 923
operator=, 924
to_global_indices, 925
gko::InvalidData, 925
InvalidData, 926
gko::InvalidStateError, 926
InvalidStateError, 927
gko::KernelNotFound, 949
KernelNotFound, 949
gko::LinOpFactory, 950

check_reuse_consistent, 951
create_empty_reuse_data, 952
generate_reuse, 952
gko::LinOpFactory::ReuseData, 1132
gko::local_span, 953
span, 953, 954
gko::log, 434
criterion, 436
factory, 436
internal, 436
linop, 436
memory, 436
object, 436
operation, 436
profile_event_category, 435
solver, 436
user, 436
gko::log::Convergence< ValueType >, 572
create, 573, 574
get_implicit_sq_resnorm, 574
get_num_iterations, 574
get_residual, 575
get_residual_norm, 575
has_converged, 575
gko::log::criterion_data, 593
gko::log::EnableLogging< ConcreteLoggable, PolymorphicBase >, 786
gko::log::executor_data, 807
gko::log::iteration_complete_data, 936
gko::log::linop_data, 949
gko::log::linop_factory_data, 950
gko::log::Loggable, 954
add_logger, 955
get_loggers, 955
remove_logger, 955
gko::log::operation_data, 1045
gko::log::Papi< ValueType >, 1047
create, 1048
get_handle, 1048
get_handle_name, 1049
gko::log::PerformanceHint, 1065
create, 1066
gko::log::polymorphic_object_data, 1094
gko::log::ProfilerHook, 1101
create_nested_summary, 1102
create_nvtx, 1103
create_summary, 1103
create_tau, 1103
set_object_name, 1104
user_range, 1104
gko::log::ProfilerHook::NestedSummaryWriter, 1027
write_nested, 1028
gko::log::ProfilerHook::SummaryWriter, 1202
write, 1202
gko::log::ProfilerHook::TableSummaryWriter, 1203
TableSummaryWriter, 1203
write, 1204
write_nested, 1204
gko::log::profiling_scope_guard, 1105
profiling_scope_guard, 1105
gko::log::Record, 1119
create, 1120
get, 1121
gko::log::Record::logged_data, 956
gko::log::SolverProgress, 1175
create_scalar_csv_writer, 1175
create_scalar_table_writer, 1176
create_vector_storage, 1176
gko::log::Stream< ValueType >, 1199
create, 1200
gko::matrix, 436
columns, 438
inverse, 438
inverse_columns, 438
inverse_rows, 438
inverse_symmetric, 438
none, 438
permute_mode, 437
rows, 438
symmetric, 438
gko::matrix::Coo< ValueType, IndexType >, 578
apply2, 580, 581
compute_absolute, 581
conj_transpose, 581
create, 581, 582
create_const, 583
extract_diagonal, 583
get_col_idxs, 584
get_const_col_idxs, 584
get_const_device_view, 584
get_const_row_idxs, 585
get_const_values, 585
get_device_view, 585
get_num_stored_elements, 586
get_row_idxs, 586
get_values, 586
read, 587
transpose, 588
write, 588
gko::matrix::Csr< ValueType, IndexType >, 594
add_scale_reuse, 599
column_permute, 600
compute_absolute, 600
conj_transpose, 600
create, 601–603
create_const, 603, 604
create_submatrix, 604, 605
Csr, 599
extract_diagonal, 605
get_col_idxs, 606
get_const_col_idxs, 606
get_const_device_view, 606
get_const_row_ptrs, 606
get_const_srow, 607
get_const_values, 607
get_device_view, 608

get_num_srow_elements, 608
get_num_stored_elements, 608
get_row_ptrs, 609
get_srow, 609
get_strategy, 609
get_values, 610
inv_scale, 610
inverse_column_permute, 610
inverse_permute, 611
inverse_row_permute, 611
multiply, 612
multiply_add, 612
multiply_add_reuse, 613
multiply_reuse, 613
operator=, 614
permute, 614, 615
permute_reuse, 616
read, 618
row_permute, 619
scale, 619
scale_add, 620
scale_permute, 620, 621
set_strategy, 621
transpose, 621
transpose_reuse, 622
write, 622
gko::matrix::Csr< ValueType, IndexType >::multiply_add_reuse_info, 1008
gko::matrix::Csr< ValueType, IndexType >::multiply_reuse_info, 1009
gko::matrix::Csr< ValueType, IndexType >::permuting_reuse_info, 1076
update_values, 1076
gko::matrix::Csr< ValueType, IndexType >::scale_add_reuse_info, 1144
gko::matrix::Dense< ValueType >, 668
add_scaled, 674
at, 675, 676
column_permute, 676–678
compute_absolute, 678
compute_conj_dot, 679
compute_dot, 679, 680
compute_mean, 680, 681
compute_norm1, 681
compute_norm2, 682
compute_squared_norm2, 682
conj_transpose, 683
create, 683, 684
create_const, 685
create_const_view_of, 685
create_real_view, 686
create_submatrix, 686, 687
create_view_of, 687
create_with_config_of, 688
create_with_type_of, 688–690
Dense, 674
extract_diagonal, 690
fill, 691
get_const_values, 691
get_num_stored_elements, 691
get_stride, 692
get_values, 692
inv_scale, 692
inverse_column_permute, 693, 694
inverse_permute, 694–696
inverse_row_permute, 696, 697
make_complex, 697
operator=, 698
permute, 698–702
row_gather, 702–704
row_permute, 704–706
scale, 706
scale_permute, 706–708
sub_scaled, 709
transpose, 709
gko::matrix::Diagonal< ValueType >, 724
compute_absolute, 726
conj_transpose, 726
create, 726, 727
create_const, 728
get_const_values, 728
get_values, 728
inverse_apply, 729
rapply, 729
transpose, 730
gko::matrix::Ell< ValueType, IndexType >, 763
col_at, 765, 766
compute_absolute, 766
create, 766, 767
create_const, 768
Ell, 765
extract_diagonal, 769
get_col_idxs, 769
get_const_col_idxs, 769
get_const_values, 770
get_num_stored_elements, 770
get_num_stored_elements_per_row, 770
get_stride, 771
get_values, 771
operator=, 771
read, 772
val_at, 773
write, 774
gko::matrix::Fbcsr< ValueType, IndexType >, 811
compute_absolute, 814
conj_transpose, 814
convert_to, 815
create, 815–817
create_const, 817
extract_diagonal, 818
Fbcsr, 814
get_block_size, 818
get_col_idxs, 818
get_const_col_idxs, 819
get_const_row_ptrs, 819
get_const_values, 819

get_num_block_cols, 820
get_num_block_rows, 820
get_num_stored_blocks, 820
get_num_stored_elements, 821
get_row_ptrs, 821
get_values, 821
is_sorted_by_column_index, 822
operator=, 822
read, 822, 823
transpose, 823
write, 824
gko::matrix::Fft, 827
conj_transpose, 828
create, 828, 829
transpose, 829
write, 829, 830
gko::matrix::Fft2, 831
conj_transpose, 832
create, 832, 833
transpose, 833
write, 833, 834
gko::matrix::Fft3, 835
conj_transpose, 836
create, 836, 837
transpose, 837
write, 837, 838
gko::matrix::Hybrid< ValueType, IndexType >, 863
compute_absolute, 866
create, 866–868
ell_col_at, 869
ell_val_at, 869, 870
extract_diagonal, 870
get_const_coo_col_idxs, 872
get_const_coo_row_idxs, 872
get_const_coo_values, 872
get_const_device_view, 873
get_const_ell_col_idxs, 873
get_const_ell_values, 873
get_coo, 874
get_coo_col_idxs, 874
get_coo_num_stored_elements, 874
get_coo_row_idxs, 875
get_coo_values, 875
get_device_view, 875
get_ell, 875
get_ell_col_idxs, 876
get_ell_num_stored_elements, 876
get_ell_num_stored_elements_per_row, 876
get_ell_stride, 877
get_ell_values, 877
get_num_stored_elements, 877
get_strategy, 878
Hybrid, 865
operator=, 878, 879
read, 879, 880
write, 880
gko::matrix::Hybrid< ValueType, IndexType >::automatic, 485
compute_ell_num_stored_elements_per_row, 485
gko::matrix::Hybrid< ValueType, IndexType >::column_limit, 529
column_limit, 529
compute_ell_num_stored_elements_per_row, 530
get_num_columns, 530
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_bounded_limit, 904
compute_ell_num_stored_elements_per_row, 905
get_percentage, 906
get_ratio, 906
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit, 906
compute_ell_num_stored_elements_per_row, 907
get_percentage, 908
imbalance_limit, 907
gko::matrix::Hybrid< ValueType, IndexType >::minimal_storage_limit, 996
compute_ell_num_stored_elements_per_row, 997
get_percentage, 997
gko::matrix::Hybrid< ValueType, IndexType >::strategy_type, 1197
compute_ell_num_stored_elements_per_row, 1197
compute_hybrid_config, 1198
get_coo_nnz, 1198
get_ell_num_stored_elements_per_row, 1199
gko::matrix::Identity< ValueType >, 889
conj_transpose, 890
create, 890, 891
transpose, 891
gko::matrix::IdentityFactory< ValueType >, 891
create, 892
gko::matrix::Permutation< IndexType >, 1070
compose, 1072
compute_inverse, 1072
create, 1072, 1073
create_const, 1073, 1074
get_const_permutation, 1074
get_permutation, 1075
get_permutation_size, 1075
write, 1075
gko::matrix::RowGatherer< IndexType >, 1136
create, 1137, 1138
create_const, 1138
get_const_row_idxs, 1139
get_row_idxs, 1139
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1145
compose, 1147
compute_inverse, 1147
create, 1147, 1148
create_const, 1149
get_const_permutation, 1149
get_const_scaling_factors, 1149
get_permutation, 1150
get_scaling_factors, 1150
write, 1150

gko::matrix::Sellp< ValueType, IndexType >, 1162
col_at, 1165
compute_absolute, 1166
create, 1166
extract_diagonal, 1167
get_col_idxs, 1167
get_const_col_idxs, 1168
get_const_slice_lengths, 1168
get_const_slice_sets, 1168
get_const_values, 1169
get_num_stored_elements, 1169
get_slice_lengths, 1170
get_slice_sets, 1170
get_slice_size, 1170
get_stride_factor, 1171
get_total_cols, 1171
get_values, 1171
operator=, 1172
read, 1172, 1173
Sellp, 1164
val_at, 1173, 1174
write, 1174
gko::matrix::SparsityCsr< ValueType, IndexType >, 1181
conj_transpose, 1183
create, 1183–1185
create_const, 1185
get_col_idxs, 1186
get_const_col_idxs, 1186
get_const_row_ptrs, 1186
get_const_value, 1187
get_num_nonzeros, 1187
get_row_ptrs, 1188
get_value, 1188
operator=, 1188, 1189
read, 1189, 1190
SparsityCsr, 1183
to_adjacency_matrix, 1190
transpose, 1190
write, 1190
gko::matrix::view::coo< ValueType, IndexType >, 589
gko::matrix::view::csr< ValueType, IndexType >, 634
gko::matrix::view::dense< ValueType >, 668
gko::matrix::view::ell< ValueType, IndexType >, 774
gko::matrix::view::hybrid< ValueType, IndexType >, 862
gko::matrix::view::sellp< ValueType, IndexType >, 1161
gko::matrix_assembly_data< ValueType, IndexType >, 979
add_value, 979
contains, 980
get_num_stored_elements, 980
get_ordered_data, 981
get_size, 981
get_value, 981
set_value, 982
gko::matrix_data< ValueType, IndexType >, 982
cond, 987, 988
diag, 989–992
matrix_data, 984–987
nonzeros, 993
sum_duplicates, 992
gko::matrix_data_entry< ValueType, IndexType >, 993
gko::MetisError, 995
MetisError, 996
gko::MpiError, 1000
MpiError, 1001
gko::multigrid, 438
gko::multigrid::EnableMultigridLevel< ValueType >, 787
get_coarse_op, 788
get_fine_op, 788
get_prolong_op, 788
get_restrict_op, 788
gko::multigrid::FixedCoarsening< ValueType, IndexType >, 839
get_system_matrix, 839
gko::multigrid::MultigridLevel, 1006
get_coarse_op, 1007
get_fine_op, 1007
get_prolong_op, 1007
get_restrict_op, 1007
gko::multigrid::Pgm< ValueType, IndexType >, 1080
get_agg, 1081
get_const_agg, 1081
get_system_matrix, 1082
parse, 1082
gko::multigrid::Pgm< ValueType, IndexType >::reuse_data_type, 1132
gko::multigrid::Pmis< ValueType, IndexType >, 1086
get_system_matrix, 1087
parse, 1087
gko::multigrid::Rs< ValueType, IndexType >, 1143
get_system_matrix, 1143
gko::multigrid::UniformCoarsening< ValueType, IndexType >, 1211
get_coarse_rows, 1212
get_const_coarse_rows, 1212
get_system_matrix, 1213
parse, 1213
gko::name_demangling, 439
get_dynamic_type, 439
get_static_type, 440
gko::NotCompiled, 1028
NotCompiled, 1029
gko::NotImplemented, 1029
NotImplemented, 1030
gko::NotSupported, 1030
NotSupported, 1031
gko::null_deleter< T >, 1031
operator(), 1032
gko::nvidia_device, 1040
gko::OmpExecutor, 1040
get_description, 1041
get_master, 1041
run, 1041, 1042
gko::Operation, 1043
get_name, 1044

gko::OutOfBoundsError, 1045
OutOfBoundsError, 1045
gko::OverflowError, 1046
OverflowError, 1046
gko::Permutable< IndexType >, 1066
column_permute, 1067
inverse_column_permute, 1068
inverse_permute, 1068
inverse_row_permute, 1069
permute, 1069
row_permute, 1070
gko::Perturbation< ValueType >, 1077
create, 1078
get_basis, 1079
get_projector, 1079
get_scalar, 1079
gko::PolymorphicObject, 1095
get_executor, 1095
gko::precision_reduction, 1096
autodetect, 1098
common, 1098
get_nonpreserving, 1099
get_preserving, 1099
operator storage_type, 1099
precision_reduction, 1097
gko::Preconditionable, 1100
get_preconditioner, 1100
set_preconditioner, 1100
gko::preconditioner, 440
isai_type, 441
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 502
compute_storage_space, 503
get_block_offset, 504
get_global_block_offset, 504
get_group_offset, 504
get_group_size, 505
get_stride, 505
group_power, 506
gko::preconditioner::GaussSeidel< ValueType, IndexType >, 840
generate, 840
get_parameters, 841
gko::preconditioner::Ic< ValueType, IndexType >, 882
conj_transpose, 884
get_l_solver, 884
get_lh_solver, 884
Ic, 883
operator=, 885
parse, 885
transpose, 886
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 900
conj_transpose, 902
get_l_solver, 902
get_u_solver, 902
Ilu, 901
operator=, 903
parse, 903
transpose, 904
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 932
conj_transpose, 934
get_approximate_inverse, 934
Isai, 933, 934
operator=, 934, 935
parse, 935
transpose, 935
gko::preconditioner::Jacobi< ValueType, IndexType >, 942
conj_transpose, 944
convert_to, 944
get_blocks, 945
get_conditioning, 945
get_num_blocks, 945
get_num_stored_elements, 946
get_storage_scheme, 946
Jacobi, 944
move_to, 946
operator=, 947
parse, 947
transpose, 948
write, 948
gko::preconditioner::NullspaceProjection< ValueType >, 1037
conj_transpose, 1038
get_left_nullspace, 1038
get_nullspace, 1038
get_preconditioner, 1038
parse, 1039
transpose, 1039
gko::preconditioner::Sor< ValueType, IndexType >, 1177
generate, 1178
get_parameters, 1178
gko::ptr_param< T >, 1106
get, 1106
operator bool, 1107
operator*, 1107
operator->, 1107
gko::range< Accessor >, 1108
get_accessor, 1111
length, 1111
operator(), 1112
operator->, 1112
operator=, 1112, 1113
range, 1110
gko::ReadableFromMatrixData< ValueType, IndexType >, 1117
read, 1117, 1118
gko::ReferenceExecutor, 1121
get_description, 1122
run, 1122–1124
gko::reorder, 441
EnableDefaultReorderingBaseFactory, 442
gko::reorder::Rcm< ValueType, IndexType >, 1114

get_inverse_permutation, 1114
get_permutation, 1115
gko::reorder::ReorderingBase< IndexType >, 1127
gko::reorder::ReorderingBaseArgs, 1128
gko::ScaledIdentityAddable, 1145
add_scaled_identity, 1145
gko::scoped_device_id_guard, 1154
scoped_device_id_guard, 1154–1156
gko::segmented_array< T >, 1156
create_from_offsets, 1158, 1159
create_from_sizes, 1159
get_const_flat_data, 1160
get_executor, 1160
get_flat_data, 1160
get_offsets, 1160
get_segment_count, 1161
get_size, 1161
segmented_array, 1157, 1158
gko::solver, 442
build_smoother, 445, 446
initial_guess_mode, 445
provided, 445
rhs, 445
trisolve_algorithm, 445
zero, 445
gko::solver::ApplyWithInitialGuess, 465
gko::solver::Bicg< ValueType >, 496
apply_uses_initial_guess, 497
conj_transpose, 497
parse, 498
transpose, 498
gko::solver::Bicgstab< ValueType >, 500
apply_uses_initial_guess, 500
conj_transpose, 501
parse, 501
transpose, 502
gko::solver::CbGmres< ValueType >, 510
get_krylov_dim, 511
get_storage_precision, 511
parse, 512
set_krylov_dim, 512
gko::solver::Cg< ValueType >, 514
apply_uses_initial_guess, 515
conj_transpose, 515
parse, 515
transpose, 516
gko::solver::Cgs< ValueType >, 516
apply_uses_initial_guess, 517
conj_transpose, 518
parse, 518
transpose, 518
gko::solver::Chebyshev< ValueType >, 519
apply_uses_initial_guess, 521
Chebyshev, 520
conj_transpose, 521
operator=, 521
parse, 522
transpose, 522
gko::solver::EnableApplyWithInitialGuess< DerivedType >, 778
gko::solver::EnableIterativeBase< DerivedType >, 784
EnableIterativeBase, 785
operator=, 785
set_stop_criterion_factory, 786
gko::solver::EnablePreconditionable< DerivedType >, 789
EnablePreconditionable, 790
operator=, 790
set_preconditioner, 790
gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, 791
get_left_nullspace, 792
get_nullspace, 792
get_unprojected_preconditioner, 792
operator=, 792
set_preconditioner, 793
gko::solver::EnableSolverBase< DerivedType, MatrixType >, 793
EnableSolverBase, 794
operator=, 794
gko::solver::Fcg< ValueType >, 824
apply_uses_initial_guess, 825
conj_transpose, 826
parse, 826
transpose, 826
gko::solver::Gcr< ValueType >, 841
apply_uses_initial_guess, 842
conj_transpose, 843
get_krylov_dim, 843
parse, 843
set_krylov_dim, 844
transpose, 844
gko::solver::Gmres< ValueType >, 844
apply_uses_initial_guess, 846
conj_transpose, 846
get_krylov_dim, 846
get_restart_ratio, 846
parse, 847
set_krylov_dim, 847
set_restart_ratio, 848
transpose, 848
gko::solver::has_with_criteria< SolverType, typename >, 849
gko::solver::Idr< ValueType >, 893
apply_uses_initial_guess, 894
conj_transpose, 894
get_complex_subspace, 895
get_deterministic, 895
get_kappa, 895
get_subspace_dim, 895
parse, 896
set_complex_subpsace, 896
set_complex_subspace, 897
set_deterministic, 897
set_kappa, 897
set_subspace_dim, 898

transpose, 898
gko::solver::Ir< ValueType >, 927
apply_uses_initial_guess, 929
conj_transpose, 929
get_solver, 930
Ir, 929
operator=, 930
parse, 930
set_solver, 931
transpose, 931
gko::solver::IterativeBase, 937
get_stop_criterion_factory, 937
set_stop_criterion_factory, 937
gko::solver::LowerTrs< ValueType, IndexType >, 956
conj_transpose, 957
LowerTrs, 957
operator=, 958
parse, 958
transpose, 959
gko::solver::Minres< ValueType >, 998
conj_transpose, 999
parse, 999
transpose, 1000
gko::solver::Multigrid, 1002
apply_uses_initial_guess, 1003
get_coarsest_solver, 1003
get_cycle, 1004
get_mg_level_list, 1004
get_mid_smoother_list, 1004
get_post_smoother_list, 1004
get_pre_smoother_list, 1005
parse, 1005
set_cycle, 1006
gko::solver::multigrid, 446
cycle, 447
mid_smooth_type, 447
gko::solver::Multigrid::reuse_data_type, 1132
gko::solver::Nullspace< ValueType >, 1032
contains_constant, 1033
create, 1033
create_from_constant, 1034
get_basis, 1034
get_dimension, 1034
project, 1035
gko::solver::nullspace_traits< Solver >, 1035
is_supported, 1036
requires_hermitian, 1036
two_sided_projection, 1036
gko::solver::PipeCg< ValueType >, 1083
apply_uses_initial_guess, 1085
conj_transpose, 1085
parse, 1085
transpose, 1086
gko::solver::UpperTrs< ValueType, IndexType >, 1216
conj_transpose, 1217
operator=, 1218
parse, 1218
transpose, 1219
UpperTrs, 1217
gko::solver::workspace_traits< Solver >, 1256
gko::span, 1179
is_valid, 1180
length, 1180
span, 1179, 1180
gko::stop, 448
absolute_implicit_residual_norm, 450
absolute_residual_norm, 450
EnableDefaultCriterionFactory, 449
initial_implicit_residual_norm, 451
initial_residual_norm, 452
max_iters, 452
min_iters, 453
relative_implicit_residual_norm, 453
relative_residual_norm, 454
time_limit, 454
gko::stop::AbsoluteResidualNorm< ValueType >, 460
gko::stop::Combined, 534
gko::stop::Criterion, 592
check, 592
update, 593
gko::stop::Criterion::Updater, 1215
check, 1215
gko::stop::CriterionArgs, 594
gko::stop::ImplicitResidualNorm< ValueType >, 909
gko::stop::Iteration, 936
gko::stop::RelativeResidualNorm< ValueType >, 1126
gko::stop::ResidualNorm< ValueType >, 1129
gko::stop::ResidualNormBase< ValueType >, 1130
gko::stop::ResidualNormReduction< ValueType >, 1131
gko::stop::Time, 1204
gko::stopping_status, 1193
converge, 1193
get_id, 1194
has_converged, 1194
has_stopped, 1194
is_finalized, 1195
operator!=, 1196
operator==, 1196
stop, 1195
gko::StreamError, 1201
StreamError, 1201
gko::syn, 455
as_array, 456
as_list, 456
concatenate, 456
gko::syn::range< Start, End, Step >, 1107
gko::syn::type_list< Types >, 1211
gko::syn::value_list< T, Values >, 1221
gko::time_point, 1205
gko::Timer, 1205
create_for_executor, 1206
create_time_point, 1206
difference, 1206
difference_async, 1207
gko::Transposable, 1208

conj_transpose, 1208
transpose, 1208
gko::UnsupportedMatrixProperty, 1214
UnsupportedMatrixProperty, 1214
gko::UseComposition< ValueType >, 1219
get_composition, 1220
get_operator_at, 1220
gko::ValueMismatch, 1221
ValueMismatch, 1222
gko::version, 1240
tag, 1241
gko::version_info, 1241
core_version, 1242
cuda_version, 1242
dpcpp_version, 1243
get, 1242
hip_version, 1243
omp_version, 1243
reference_version, 1243
gko::WritableToMatrixData< ValueType, IndexType >, 1257
write, 1257
gko::xstd, 457
GKO_CREATE_FACTORY_PARAMETERS
Linear Operators, 334
GKO_ENABLE_BATCH_LIN_OP_FACTORY
Batched Linear Operators, 316
GKO_ENABLE_BUILD_METHOD
Linear Operators, 335
GKO_ENABLE_LIN_OP_FACTORY
Linear Operators, 335
GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE
Linear Operators, 336
GKO_ENABLE_REORDERING_BASE_FACTORY
Reordering Algorithms, 351
GKO_FACTORY_PARAMETER
Linear Operators, 337
GKO_FACTORY_PARAMETER_SCALAR
Linear Operators, 338
GKO_FACTORY_PARAMETER_VECTOR
Linear Operators, 338
GKO_REGISTER_HOST_OPERATION
Executors, 323
GKO_REGISTER_OPERATION
Executors, 324
group_power
gko::preconditioner::block_interleaved_storage_scheme< IndexType >, 506

has_connected_parts
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1064
has_converged
gko::log::Convergence< ValueType >, 575
gko::stopping_status, 1194
has_ordered_parts
gko::experimental::distributed::Partition< LocalIndexType, GlobalIndexType >, 1065
has_stopped
gko::stopping_status, 1194
highest_precision
gko, 371
HIP Executor, 327
hip_stream
gko::hip_stream, 850
hip_version
gko::version_info, 1243
HipblasError
gko::HipblasError, 851
HipError
gko::HipError, 852
HipfftError
gko::HipfftError, 858
HiprandError
gko::HiprandError, 859
HipsparseError
gko::HipsparseError, 860
Hybrid
gko::matrix::Hybrid< ValueType, IndexType >, 865

i_all_gather
gko::experimental::mpi::communicator, 547
i_all_reduce
gko::experimental::mpi::communicator, 548
i_all_to_all
gko::experimental::mpi::communicator, 549, 550
i_all_to_all_v
gko::experimental::mpi::CollectiveCommunicator, 527, 528
gko::experimental::mpi::communicator, 551, 552
gko::experimental::mpi::DenseCommunicator, 713
gko::experimental::mpi::NeighborhoodCommunicator, 1025, 1026
i_broadcast
gko::experimental::mpi::communicator, 552
i_gather
gko::experimental::mpi::communicator, 553
i_gather_v
gko::experimental::mpi::communicator, 554
i_recv
gko::experimental::mpi::communicator, 555
i_reduce
gko::experimental::mpi::communicator, 556
i_scan
gko::experimental::mpi::communicator, 556
i_scatter
gko::experimental::mpi::communicator, 557
i_scatter_v
gko::experimental::mpi::communicator, 558
i_send
gko::experimental::mpi::communicator, 559
Ic
gko::preconditioner::Ic< ValueType, IndexType >, 883
Ilu
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 901
imag

gko, 386
imbalance_limit
gko::matrix::Hybrid< ValueType, IndexType >::imbalance_limit, 907
incomplete_algorithm
gko::factorization, 434
index_map
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 911
index_set
gko::index_set< IndexType >, 915–918
index_space
gko::experimental::distributed, 423
initial_guess_mode
gko::solver, 445
initial_implicit_residual_norm
gko::stop, 451
initial_residual_norm
gko::stop, 452
initialize
Linear Operators, 339–341
internal
gko::log, 436
inv_scale
gko::experimental::distributed::Vector< ValueType >, 1238
gko::matrix::Csr< ValueType, IndexType >, 610
gko::matrix::Dense< ValueType >, 692
InvalidData
gko::InvalidData, 926
InvalidStateError
gko::InvalidStateError, 927
inverse
gko::matrix, 438
inverse_apply
gko::matrix::Diagonal< ValueType >, 729
inverse_column_permute
gko::matrix::Csr< ValueType, IndexType >, 610
gko::matrix::Dense< ValueType >, 693, 694
gko::Permutable< IndexType >, 1068
inverse_columns
gko::matrix, 438
inverse_permute
gko::matrix::Csr< ValueType, IndexType >, 611
gko::matrix::Dense< ValueType >, 694–696
gko::Permutable< IndexType >, 1068
inverse_row_permute
gko::matrix::Csr< ValueType, IndexType >, 611
gko::matrix::Dense< ValueType >, 696, 697
gko::Permutable< IndexType >, 1069
inverse_rows
gko::matrix, 438
inverse_symmetric
gko::matrix, 438
Ir
gko::solver::Ir< ValueType >, 929
is_complex
gko, 387
is_complex_or_scalar
gko, 387
is_complex_or_scalar_s
gko, 371
is_complex_s
gko, 372
is_congruent
gko::experimental::mpi::communicator, 560
is_contiguous
gko::index_set< IndexType >, 922
is_finalized
gko::stopping_status, 1195
is_finite
gko, 388
is_identical
gko::experimental::mpi::communicator, 560
is_nan
gko, 389
is_nonzero
gko, 390
is_owning
gko::array< ValueType >, 478
is_sorted_by_column_index
gko::matrix::Fbcsr< ValueType, IndexType >, 822
is_supported
gko::solver::nullspace_traits< Solver >, 1036
is_valid
gko::span, 1180
is_zero
gko, 390
Isai
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 933, 934
isai_type
gko::preconditioner, 441

Jacobi
gko::preconditioner::Jacobi< ValueType, IndexType >, 944
Jacobi Preconditioner, 328

KernelNotFound
gko::KernelNotFound, 949

layout_type
gko, 374
lend
gko, 391
length
gko::accessor::row_major< ValueType, Dimensionality >, 1135
gko::range< Accessor >, 1111
gko::span, 1180
Linear Operators, 329
EnableDefaultLinOpFactory, 338
GKO_CREATE_FACTORY_PARAMETERS, 334
GKO_ENABLE_BUILD_METHOD, 335
GKO_ENABLE_LIN_OP_FACTORY, 335

GKO_ENABLE_LIN_OP_FACTORY_WITH_REUSE, 336
GKO_FACTORY_PARAMETER, 337
GKO_FACTORY_PARAMETER_SCALAR, 338
GKO_FACTORY_PARAMETER_VECTOR, 338
initialize, 339–341
linop
gko::log, 436
local
gko::experimental::distributed, 423
lock
gko::experimental::mpi::window< ValueType >, 1251
lock_all
gko::experimental::mpi::window< ValueType >, 1251
log_propagation_mode
gko, 374
Logging, 343
LowerTrs
gko::solver::LowerTrs< ValueType, IndexType >, 957

make_array_view
gko, 392
make_complex
gko::experimental::distributed::Vector< ValueType >, 1238
gko::matrix::Dense< ValueType >, 697
make_const_array_view
gko, 392
make_const_dense_view
gko, 393
make_dense_view
gko, 394
make_temporary_clone
gko, 394
make_temporary_conversion
gko, 395
gko::experimental::distributed, 425, 426
make_temporary_output_clone
gko, 396
map_global_to_local
gko::index_set< IndexType >, 923
map_local_to_global
gko::index_set< IndexType >, 923
map_rank_to_device_id
gko::experimental::mpi, 432
map_to_global
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 912
map_to_local
gko::experimental::distributed::index_map< LocalIndexType, GlobalIndexType >, 912
Matrix
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 965
matrix_data
gko::matrix_data< ValueType, IndexType >, 984–987
max
gko, 396
max_iters
gko::stop, 452
mc64_strategy
gko::experimental::reorder, 433
memory
gko::log, 436
memory_accessible
gko::Executor, 804
MetisError
gko::MetisError, 996
mid_smooth_type
gko::solver::multigrid, 447
min
gko, 397
min_iters
gko::stop, 453
mixed_precision_dispatch
gko, 397
mixed_precision_dispatch_real_complex
gko, 398
mode
Stopping criteria, 356
move_to
gko::ConvertibleTo< ResultType >, 577
gko::EnableCloneable< ConcreteType >, 781
gko::preconditioner::Jacobi< ValueType, IndexType >, 946
MpiError
gko::MpiError, 1001
Multigrid, 346
Multigrid Level, 347
multiply
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 974
gko::matrix::Csr< ValueType, IndexType >, 612
multiply_add
gko::matrix::Csr< ValueType, IndexType >, 612
multiply_add_reuse
gko::matrix::Csr< ValueType, IndexType >, 613
multiply_reuse
gko::matrix::Csr< ValueType, IndexType >, 613

nan
gko, 399
NeighborhoodCommunicator
gko::experimental::mpi::NeighborhoodCommunicator, 1023
never
gko, 374
node_local_rank
gko::experimental::mpi::communicator, 560
non_local
gko::experimental::distributed, 423
none
gko::matrix, 438

nonzeros
gko::matrix_data< ValueType, IndexType >, 993
NotCompiled
gko::NotCompiled, 1029
NotImplemented
gko::NotImplemented, 1030
NotSupported
gko::NotSupported, 1031

object
gko::log, 436
omp_version
gko::version_info, 1243
on
gko::deferred_factory_parameter< FactoryType >, 655
gko::enable_parameters_type< ConcreteParametersType, Factory >, 776
one
gko, 399, 400
OpenMP Executor, 348
operation
gko::log, 436
operator bool
gko::config::pnode, 1094
gko::dim< Dimensionality, DimensionType >, 735
gko::ptr_param< T >, 1107
operator storage_type
gko::precision_reduction, 1099
operator!=
gko, 400, 401
gko::batch_dim< Dimensionality, DimensionType >, 489
gko::dim< Dimensionality, DimensionType >, 736
gko::experimental::mpi::communicator, 561
gko::experimental::mpi::DenseCommunicator, 714
gko::experimental::mpi::NeighborhoodCommunicator, 1027
gko::stopping_status, 1196
operator<<
gko, 401, 402
gko::dim< Dimensionality, DimensionType >, 737
operator*
gko::dim< Dimensionality, DimensionType >, 737
gko::ptr_param< T >, 1107
operator()
gko::accessor::row_major< ValueType, Dimensionality >, 1135, 1136
gko::default_converter< S, R >, 652
gko::executor_deleter< T >, 808
gko::null_deleter< T >, 1032
gko::range< Accessor >, 1112
operator->
gko::ptr_param< T >, 1107
gko::range< Accessor >, 1112
operator=
gko::array< ValueType >, 479–482
gko::BlockOperator, 509
gko::Combination< ValueType >, 533
gko::Composition< ValueType >, 569
gko::experimental::distributed::DistributedBase, 742
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 974, 975
gko::experimental::mpi::communicator, 561
gko::experimental::mpi::contiguous_type, 572
gko::experimental::mpi::window< ValueType >, 1252
gko::index_set< IndexType >, 924
gko::matrix::Csr< ValueType, IndexType >, 614
gko::matrix::Dense< ValueType >, 698
gko::matrix::Ell< ValueType, IndexType >, 771
gko::matrix::Fbcsr< ValueType, IndexType >, 822
gko::matrix::Hybrid< ValueType, IndexType >, 878, 879
gko::matrix::Sellp< ValueType, IndexType >, 1172
gko::matrix::SparsityCsr< ValueType, IndexType >, 1188, 1189
gko::preconditioner::Ic< ValueType, IndexType >, 885
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 903
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 934, 935
gko::preconditioner::Jacobi< ValueType, IndexType >, 947
gko::range< Accessor >, 1112, 1113
gko::solver::Chebyshev< ValueType >, 521
gko::solver::EnableIterativeBase< DerivedType >, 785
gko::solver::EnablePreconditionable< DerivedType >, 790
gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, 792
gko::solver::EnableSolverBase< DerivedType, MatrixType >, 794
gko::solver::Ir< ValueType >, 930
gko::solver::LowerTrs< ValueType, IndexType >, 958
gko::solver::UpperTrs< ValueType, IndexType >, 1218
operator==
gko, 402
gko::batch_dim< Dimensionality, DimensionType >, 489
gko::dim< Dimensionality, DimensionType >, 738
gko::experimental::mpi::communicator, 561
gko::experimental::mpi::DenseCommunicator, 714
gko::experimental::mpi::NeighborhoodCommunicator, 1027
gko::stopping_status, 1196
operator[]
gko::dim< Dimensionality, DimensionType >, 735, 736
OutOfBoundsError
gko::OutOfBoundsError, 1045
OverflowError

gko::OverflowError, 1046

parse
gko::experimental::distributed::preconditioner::Schwarz< ValueType, LocalIndexType, GlobalIndexType >, 1153
gko::experimental::factorization::Cholesky< ValueType, IndexType >, 524
gko::experimental::factorization::Lu< ValueType, IndexType >, 961
gko::experimental::solver::Direct< ValueType, IndexType >, 740
gko::factorization::Ic< ValueType, IndexType >, 881
gko::factorization::Ilu< ValueType, IndexType >, 899
gko::factorization::ParIc< ValueType, IndexType >, 1050
gko::factorization::ParIct< ValueType, IndexType >, 1053
gko::factorization::ParIlu< ValueType, IndexType >, 1055
gko::factorization::ParIlut< ValueType, IndexType >, 1057
gko::multigrid::Pgm< ValueType, IndexType >, 1082
gko::multigrid::Pmis< ValueType, IndexType >, 1087
gko::multigrid::UniformCoarsening< ValueType, IndexType >, 1213
gko::preconditioner::Ic< ValueType, IndexType >, 885
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 903
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 935
gko::preconditioner::Jacobi< ValueType, IndexType >, 947
gko::preconditioner::NullspaceProjection< ValueType >, 1039
gko::solver::Bicg< ValueType >, 498
gko::solver::Bicgstab< ValueType >, 501
gko::solver::CbGmres< ValueType >, 512
gko::solver::Cg< ValueType >, 515
gko::solver::Cgs< ValueType >, 518
gko::solver::Chebyshev< ValueType >, 522
gko::solver::Fcg< ValueType >, 826
gko::solver::Gcr< ValueType >, 843
gko::solver::Gmres< ValueType >, 847
gko::solver::Idr< ValueType >, 896
gko::solver::Ir< ValueType >, 930
gko::solver::LowerTrs< ValueType, IndexType >, 958
gko::solver::Minres< ValueType >, 999
gko::solver::Multigrid, 1005
gko::solver::PipeCg< ValueType >, 1085
gko::solver::UpperTrs< ValueType, IndexType >, 1218

permute
gko::matrix::Csr< ValueType, IndexType >, 614, 615
gko::matrix::Dense< ValueType >, 698–702
gko::Permutable< IndexType >, 1069

permute_mode
gko::matrix, 437

permute_reuse
gko::matrix::Csr< ValueType, IndexType >, 616

pi
gko, 403

pnode
gko::config::pnode, 1089–1091

precision_dispatch
gko, 403
gko::experimental::distributed, 427

precision_dispatch_real_complex
gko, 404, 405
gko::experimental::distributed, 428, 429

precision_reduction
gko::precision_reduction, 1097

Preconditioners, 349

previous_precision_base
gko, 372

profile_event_category
gko::log, 435

profiling_scope_guard
gko::log::profiling_scope_guard, 1105

project
gko::solver::Nullspace< ValueType >, 1035

provided
gko::solver, 445

put
gko::experimental::mpi::window< ValueType >, 1252

r_accumulate
gko::experimental::mpi::window< ValueType >, 1253

r_get
gko::experimental::mpi::window< ValueType >, 1253

r_get_accumulate
gko::experimental::mpi::window< ValueType >, 1254

r_put
gko::experimental::mpi::window< ValueType >, 1255

range
gko::range< Accessor >, 1110

rank
gko::experimental::mpi::communicator, 562

rapply
gko::matrix::Diagonal< ValueType >, 729

read
gko, 406
gko::matrix::Coo< ValueType, IndexType >, 587
gko::matrix::Csr< ValueType, IndexType >, 618
gko::matrix::Ell< ValueType, IndexType >, 772

gko::matrix::Fbcsr< ValueType, IndexType >, 822, 823
gko::matrix::Hybrid< ValueType, IndexType >, 879, 880
gko::matrix::Sellp< ValueType, IndexType >, 1172, 1173
gko::matrix::SparsityCsr< ValueType, IndexType >, 1189, 1190
gko::ReadableFromMatrixData< ValueType, IndexType >, 1117, 1118
read_binary
gko, 406
read_binary_raw
gko, 407
read_distributed
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 975–977
gko::experimental::distributed::Vector< ValueType >, 1239
read_generic
gko, 408
read_generic_raw
gko, 409
read_raw
gko, 409
real
gko, 410
recv
gko::experimental::mpi::communicator, 562
reduce
gko::experimental::mpi::communicator, 563
reduce_add
gko, 410, 411
refactorize
gko::ext::cuda::solver::Cudss< ValueType, IndexType >, 649
Reference Executor, 350
reference_version
gko::version_info, 1243
registry
gko::config::registry, 1125
relative_implicit_residual_norm
gko::stop, 453
relative_residual_norm
gko::stop, 454
remove_complex
gko, 372
remove_logger
gko::Executor, 804
gko::log::Loggable, 955
remove_zeros
gko::device_matrix_data< ValueType, IndexType >, 723
Reordering Algorithms, 351
GKO_ENABLE_REORDERING_BASE_FACTORY, 351
request
gko::experimental::mpi::request, 1128
requires_hermitian
gko::solver::nullspace_traits< Solver >, 1036
reset_max_iterations
gko::batch::solver::BatchSolver, 494
reset_tolerance
gko::batch::solver::BatchSolver, 495
reset_tolerance_type
gko::batch::solver::BatchSolver, 495
resize
gko::experimental::mpi::CollectiveCommunicator, 528
gko::experimental::mpi::DenseCommunicator, 713
gko::experimental::mpi::NeighborhoodCommunicator, 1026
resize_and_reset
gko::array< ValueType >, 482
gko::device_matrix_data< ValueType, IndexType >, 723, 724
rhs
gko::solver, 445
round_down
gko, 411
round_up
gko, 412
row_gather
gko::matrix::Dense< ValueType >, 702–704
row_permute
gko::matrix::Csr< ValueType, IndexType >, 619
gko::matrix::Dense< ValueType >, 704–706
gko::Permutable< IndexType >, 1070
row_scale
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 977
rows
gko::matrix, 438
run
gko::CudaExecutor, 644, 646
gko::DpcppExecutor, 747, 749
gko::Executor, 804, 805
gko::HipExecutor, 856, 857
gko::OmpExecutor, 1041, 1042
gko::ReferenceExecutor, 1122–1124

safe_divide
gko, 412
scale
gko::batch::matrix::Csr< ValueType, IndexType >, 633
gko::batch::matrix::Dense< ValueType >, 667
gko::batch::matrix::Ell< ValueType, IndexType >, 762
gko::batch::MultiVector< ValueType >, 1021
gko::experimental::distributed::Vector< ValueType >, 1239
gko::matrix::Csr< ValueType, IndexType >, 619
gko::matrix::Dense< ValueType >, 706
scale_add
gko::batch::matrix::Dense< ValueType >, 667
gko::matrix::Csr< ValueType, IndexType >, 620

scale_permute
gko::matrix::Csr< ValueType, IndexType >, 620, 621
gko::matrix::Dense< ValueType >, 706–708
scan
gko::experimental::mpi::communicator, 564
scatter
gko::experimental::mpi::communicator, 564
scatter_v
gko::experimental::mpi::communicator, 565
scoped_device_id_guard
gko::scoped_device_id_guard, 1154–1156
segmented_array
gko::segmented_array< T >, 1157, 1158
Sellp
gko::matrix::Sellp< ValueType, IndexType >, 1164
send
gko::experimental::mpi::communicator, 566
set_complex_subpsace
gko::solver::Idr< ValueType >, 896
set_complex_subspace
gko::solver::Idr< ValueType >, 897
set_cycle
gko::solver::Multigrid, 1006
set_deterministic
gko::solver::Idr< ValueType >, 897
set_executor
gko::array< ValueType >, 483
set_kappa
gko::solver::Idr< ValueType >, 897
set_krylov_dim
gko::solver::CbGmres< ValueType >, 512
gko::solver::Gcr< ValueType >, 844
gko::solver::Gmres< ValueType >, 847
set_log_propagation_mode
gko::Executor, 806
set_object_name
gko::log::ProfilerHook, 1104
set_preconditioner
gko::Preconditionable, 1100
gko::solver::EnablePreconditionable< DerivedType >, 790
gko::solver::EnablePreconditionedIterativeSolver< ValueType, DerivedType >, 793
set_restart_ratio
gko::solver::Gmres< ValueType >, 848
set_solver
gko::solver::Ir< ValueType >, 931
set_stop_criterion_factory
gko::solver::EnableIterativeBase< DerivedType >, 786
gko::solver::IterativeBase, 937
set_strategy
gko::matrix::Csr< ValueType, IndexType >, 621
set_subspace_dim
gko::solver::Idr< ValueType >, 898
set_value
gko::matrix_assembly_data< ValueType, IndexType >, 982
share
gko, 413
should_propagate_log
gko::Executor, 806
size
gko::experimental::mpi::communicator, 566
solver
gko::log, 436
Solvers, 353
span
gko::local_span, 953, 954
gko::span, 1179, 1180
SparsityCsr
gko::matrix::SparsityCsr< ValueType, IndexType >, 1183
SpMV employing different Matrix formats, 344
squared_norm
gko, 413
status
gko::experimental::mpi::status, 1191
stop
gko::stopping_status, 1195
Stopping criteria, 355
combine, 356
mode, 356
StreamError
gko::StreamError, 1201
sub_scaled
gko::experimental::distributed::Vector< ValueType >, 1240
gko::matrix::Dense< ValueType >, 709
sum_duplicates
gko::device_matrix_data< ValueType, IndexType >, 724
gko::matrix_data< ValueType, IndexType >, 992
symmetric
gko::matrix, 438
synchronize
gko::experimental::mpi::communicator, 567

TableSummaryWriter
gko::log::ProfilerHook::TableSummaryWriter, 1203
tag
gko::version, 1241
time_limit
gko::stop, 454
to_adjacency_matrix
gko::matrix::SparsityCsr< ValueType, IndexType >, 1190
to_complex
gko, 373
to_global_indices
gko::index_set< IndexType >, 925
to_real
gko, 373
transpose
gko, 414

gko::Combination< ValueType >, 533
gko::Composition< ValueType >, 570
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 978
gko::experimental::solver::Direct< ValueType, IndexType >, 741
gko::matrix::Coo< ValueType, IndexType >, 588
gko::matrix::Csr< ValueType, IndexType >, 621
gko::matrix::Dense< ValueType >, 709
gko::matrix::Diagonal< ValueType >, 730
gko::matrix::Fbcsr< ValueType, IndexType >, 823
gko::matrix::Fft, 829
gko::matrix::Fft2, 833
gko::matrix::Fft3, 837
gko::matrix::Identity< ValueType >, 891
gko::matrix::SparsityCsr< ValueType, IndexType >, 1190
gko::preconditioner::Ic< ValueType, IndexType >, 886
gko::preconditioner::Ilu< ValueType, ReverseApply, IndexType >, 904
gko::preconditioner::Isai< IsaiType, ValueType, IndexType >, 935
gko::preconditioner::Jacobi< ValueType, IndexType >, 948
gko::preconditioner::NullspaceProjection< ValueType >, 1039
gko::solver::Bicg< ValueType >, 498
gko::solver::Bicgstab< ValueType >, 502
gko::solver::Cg< ValueType >, 516
gko::solver::Cgs< ValueType >, 518
gko::solver::Chebyshev< ValueType >, 522
gko::solver::Fcg< ValueType >, 826
gko::solver::Gcr< ValueType >, 844
gko::solver::Gmres< ValueType >, 848
gko::solver::Idr< ValueType >, 898
gko::solver::Ir< ValueType >, 931
gko::solver::LowerTrs< ValueType, IndexType >, 959
gko::solver::Minres< ValueType >, 1000
gko::solver::PipeCg< ValueType >, 1086
gko::solver::UpperTrs< ValueType, IndexType >, 1219
gko::Transposable, 1208
transpose_reuse
gko::matrix::Csr< ValueType, IndexType >, 622
trisolve_algorithm
gko::solver, 445
two_sided_projection
gko::solver::nullspace_traits< Solver >, 1036
type
gko::cpx_real_type< T >, 591
type_descriptor
gko::config::type_descriptor, 1210

unit_root
gko, 415
unlock
gko::experimental::mpi::window< ValueType >, 1256
unpack
gko::experimental::factorization::Factorization< ValueType, IndexType >, 811
UnsupportedMatrixProperty
gko::UnsupportedMatrixProperty, 1214
update
gko::stop::Criterion, 593
update_values
gko::matrix::Csr< ValueType, IndexType >::permuting_reuse_info, 1076
UpperTrs
gko::solver::UpperTrs< ValueType, IndexType >, 1217
user
gko::log, 436
user_range
gko::log::ProfilerHook, 1104

val_at
gko::matrix::Ell< ValueType, IndexType >, 773
gko::matrix::Sellp< ValueType, IndexType >, 1173, 1174
ValueMismatch
gko::ValueMismatch, 1222
view
gko::array< ValueType >, 484

wait
gko::experimental::mpi::request, 1129
wait_all
gko::experimental::mpi, 432
window
gko::experimental::mpi::window< ValueType >, 1245, 1246
with_matrix_type
gko, 415
write
gko, 416, 417
gko::experimental::distributed::Matrix< ValueType, LocalIndexType, GlobalIndexType >, 978
gko::log::ProfilerHook::SummaryWriter, 1202
gko::log::ProfilerHook::TableSummaryWriter, 1204
gko::matrix::Coo< ValueType, IndexType >, 588
gko::matrix::Csr< ValueType, IndexType >, 622
gko::matrix::Ell< ValueType, IndexType >, 774
gko::matrix::Fbcsr< ValueType, IndexType >, 824
gko::matrix::Fft, 829, 830
gko::matrix::Fft2, 833, 834
gko::matrix::Fft3, 837, 838
gko::matrix::Hybrid< ValueType, IndexType >, 880
gko::matrix::Permutation< IndexType >, 1075
gko::matrix::ScaledPermutation< ValueType, IndexType >, 1150
gko::matrix::Sellp< ValueType, IndexType >, 1174
gko::matrix::SparsityCsr< ValueType, IndexType >, 1190

gko::preconditioner::Jacobi< ValueType, IndexType >, 948
gko::WritableToMatrixData< ValueType, IndexType >, 1257
write_binary
gko, 417
write_binary_raw
gko, 418
write_nested
gko::log::ProfilerHook::NestedSummaryWriter, 1028
gko::log::ProfilerHook::TableSummaryWriter, 1204
write_raw
gko, 418

zero
gko, 420
gko::solver, 445